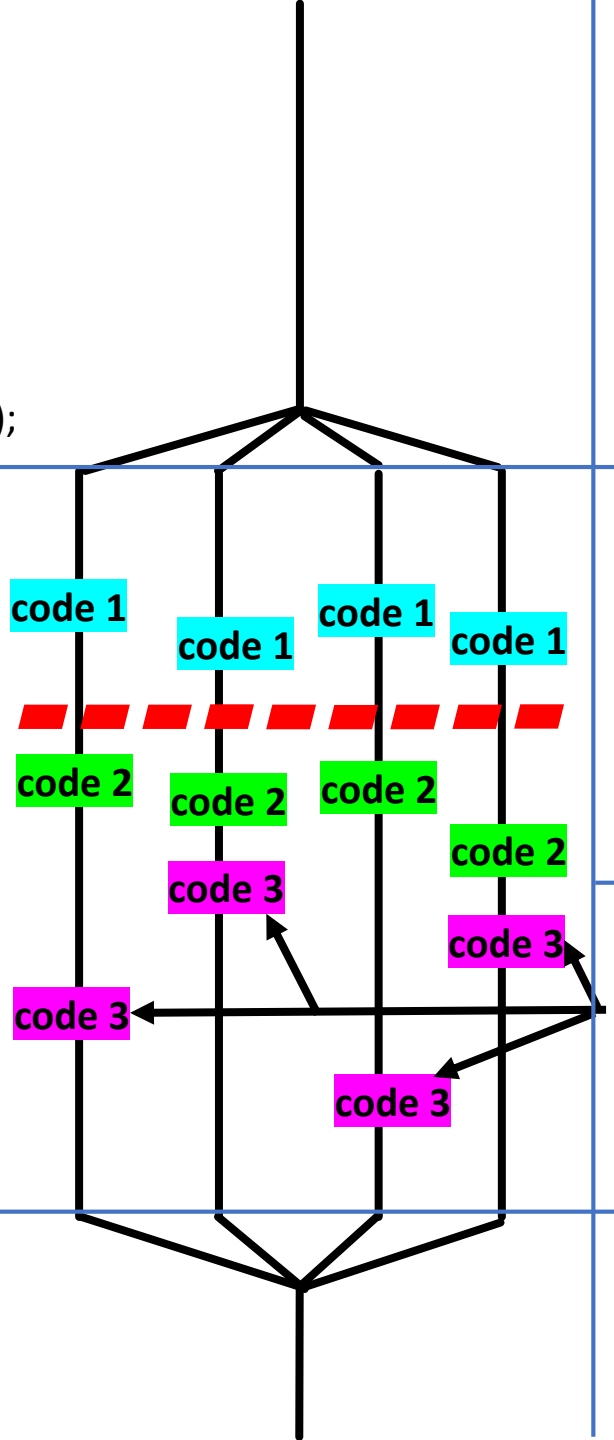
```
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);

pthread_barrier_t barrier;
pthread_barrier_init(&barrier, NULL, num_threads);

pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);
```

```
void * threadFunction(void* arg)
{
    code 1
    pthread_barrier_wait(&barrier);
    code 2

    pthread_mutex_lock(&mutex);
    code 3
    pthread_mutex_unlock(&mutex);
}
```

```
pthread_join(thread, NULL);
pthread_barrier_destroy(&barrier);
pthread_mutex_destroy(&mutex);
```



#include <pthread.h>

Any **code 2** must execute **after all code 1**

When **code 3** starts execution it must end before it can start on a different thread

Compiling:
gcc main.c –fpthread
Number of cores:
cat /proc/cpuinfo

PThread Cheat Sheet

Made by Cristian Chilipirea

MPI Cheat Sheet

Arguments from the main function
Called at the start of any MPI program

int MPI_Init(↕ int *, ↕ char *)**

&argc	&argv
NULL	NULL

Called at the end of any MPI program

int MPI_Finalize()

Gives the number of tasks

int MPI_Comm_size (↓ MPI_Comm, ↑ int *)

MPI_COMM_WORLD

&num_tasks

Gives the id (rank) of the current (calling) task

int MPI_Comm_rank (↓ MPI_Comm, ↑ int *)

MPI_COMM_WORLD

&rank

Synchronizez all tasks at the call of the barrier

int MPI_Barrier (↓ MPI_Comm comm)

MPI_COMM_WORLD

Send from buffer **b**, **c** elements of data type d to rank r. The communication is marked with tag **t**.
The function is blocking, b can safely be used after it but data may not have yet been delivered.

int MPI_Send(↓ void *b, ↓ int c, ↓ MPI_Datatype d, ↓ int reiceiver, ↓ int t, ↓ MPI_Comm)

v	num_el(v)	MPI_INT	[0, **num_tasks**)	[0, ..)	MPI_COMM_WORLD
&v[3]	[0,..)	MPI_CHAR			
&a		MPI_FLOAT			
v+5		MPI_LONG			

Receive in buffer **b**, **c** elements of data type d from rank r. The communication is marked with tag **t**.
The function is bloking, b can be safely used and the data was delivered.

int MPI_Recv(↑ void *b, ↓ int c, ↓ MPI_Datatype d, ↓ int sender, ↓ int t, ↓ MPI_Comm, ↑ MPI_Status *)

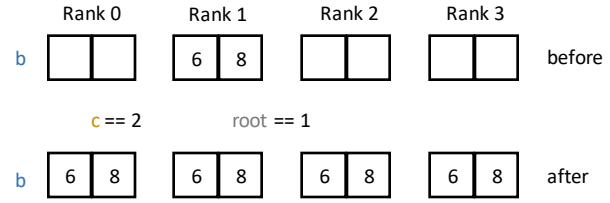
v	num_el(v)	MPI_INT	[0, **num_tasks**)	MPI_COMM_WORLD	
&v[3]	[0,..)	MPI_CHAR	MPI_ANY_SOURCE		
&a		MPI_FLOAT		&Stat	
v+5		MPI_LONG	[0, ..)	MPI_STATUS_IGNORE	
			MPI_ANY_TAG	***Stat.MPI_SOURCE, Stat.MPI_TAG***	

Made by Cristian Chilipirea

Sends (Broadcasts) **c** elements of data type d from buffer **b** from rank r to all other tasks in buffer **b**.
All tasks have to call this function with the same value for root.

int MPI_Bcast (↕ void *b, ↓ int c, ↓ MPI_Datatype d, ↓ int root, ↓ MPI_Comm)

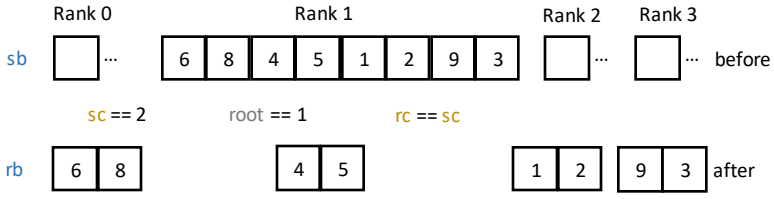
v	num_el(v)	MPI_INT	MPI_COMM_WORLD
&v[3]	[0,..)	MPI_CHAR	[0, **num_tasks**)
&a		MPI_FLOAT	
v+5		MPI_LONG	



Splits the elements from **sb** of datatype sd on rank root in num_tasks chunks of size **sc**.
Every task receives its appropriate chunk in **rb**. For simplicity **sc** == **rc**, sd == rd.
All tasks have to call this function with the same value for root.

int MPI_Scatter (↓ void *sb, ↓ int sc, ↓ MPI_Datatype sd, ↑ void *rb, ↓ int rc, ↓ MPI_Datatype rd, ↓ int root, ↓ MPI_Comm)

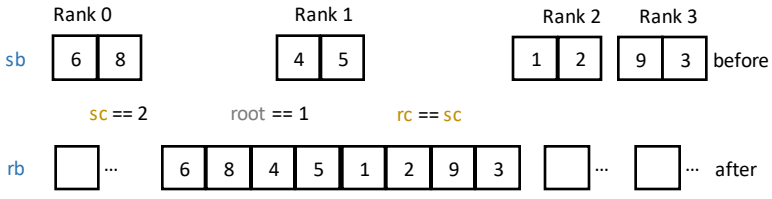
v	num_el(v)/**num_tasks**	MPI_INT	v	num_el(v)/**num_tasks**	MPI_INT	MPI_COMM_WORLD
&v[3]	[0,..)	MPI_CHAR	&v[3]	[0,..)	MPI_CHAR	[0, **num_tasks**)
&a		MPI_FLOAT	&a		MPI_FLOAT	
v+5		MPI_LONG	v+5		MPI_LONG	



Gathers **sc** elements from all **sb** of datatype sd on all tasks and places the **num_tasks** chunks of size **rc** in **rb** on task of rank root.
Every task sends its appropriate chunk in **rb**. For simplicity **sc** == **rc**, sd == rd.
All tasks have to call this function with the same value for root.

int MPI_Gather (↓ void *sb, ↓ int sc, ↓ MPI_Datatype sd, ↑ void *rb, ↓ int rc, ↓ MPI_Datatype rd, ↓ int root, ↓ MPI_Comm)

v	num_el(v)/**num_tasks**	MPI_INT	v	num_el(v)/**num_tasks**	MPI_INT	MPI_COMM_WORLD
&v[3]	[0,..)	MPI_CHAR	&v[3]	[0,..)	MPI_CHAR	[0, **num_tasks**)
&a		MPI_FLOAT	&a		MPI_FLOAT	
v+5		MPI_LONG	v+5		MPI_LONG	



Varianta A

1. **Ce garantii ofera o bariera?**

   Tot codul dinainte de bariera este executat inainte de codul de dupa bariera.

2. **Cum poate scala o solutie paralelizabila cu 50% folosind legea lui Amdahl?**

   S = 1/(f + (1-f)/P) => S = 1/(1/2 + 1/2P) = 1/((1+P)/2P) = 2P/(1+P)
   P = 1 => S = 1
   P = 2 => S = 1.33
   P = 3 => S = 1.5
   P = 4 => S = 1.6
   P = 8 => S = 1.77

3. **Presudocod pentru Producator in cazul algoritmului Producator-Consumator.**

```
int N = MAX_BUF_SIZE;
int prim = 0;
int ultim = 0;
type buf[N];
sem gol = N;
sem plin = 0;
sem mutexP = 1;

Producator [i = 1…P] {
        while(true) {
                type elem = produce();
                P(gol);
                P(mutexP);
                buf[ultim] = elem;
                ultim = (ultim + 1) % N;
                V(mutexP);
                V(plin);
        }
}
```

4. **Implementare paralela pentru un joc de Sudoku.**

   ExecutorService. Se porneste in main cu configuratia initiala a tabloului de Sudoku si se trimite in ExecutorService. Functia din ExecutorService pune in coada de task-uri toate variantele posibile si corecte pornind de la configuratia primita. Intr-un final, cand se gaseste

solutie pentru tabloul Sudoku, se apeleaza functiile shutdown(), awaitTermination(time, measurementUnit), shutdownNow().

5. **Diferenta dintre ceas vectorial si ceas logic.**

   Ceasul logic este o singura valoare ce reprezinta cate evenimente (fie interne procesului, fie send sau receive) stie procesul curent ca s-au intamplat. In schimb, ceasul vectorial contine mai multe valori (egal cu numarul de procese) unde valoarea de pe pozitia i reprezinta prin cate evenimente stie procesul current ca a trecut procesul i.

6. **De ce algoritmul arbore este un algoritm unda?**

   Deoarece el indeplineste cele 3 conditii ale unui algoritm unda:
   a) Terminare – algoritmul se termina dupa un numar finit de pasi
   b) Decizie – algoritmul contine cel putin un eveniment decide
   c) Dependenta – fiecare decide este precedat cauzal de un eveniment in fiecare proces

7. **Rolul SHA-1 in algoritmul Chord.**

   SHA-1 asigura o distrubutie uniforma atat a identificatorilor cheilor, cat si a identificatorilor nodurilor.

8. **Ce garantie de consistenta (daca exista) e implementata in Blockchain?**

   Strong consistency. Trebuie ca toate nodurile sa stie noul lant de blocuri (cel mai lung).

Varianta B

1. **Descrieti taxonomia Flynn.**

   Sistemele pot fi grupate in 4 categorii:
   a) SISD – single instruction stream, single data stream (calculatorul classic single core)
   b) SIMD – single instruction stream, multiple data stream (placi video)
   c) MISD – multiple instruction stream, single data stream (sisteme specializate)
   d) MIMD – multiple instruction stream, multiple data stream (procesoare cu mai multe nuclee)

2. **Calculati speed-up dupa formula lui Amdahl cu un cod pentru care s-a reusit paralelizarea in proportie de 90%.**

   S = 1/(f + (1-f)/P) => S = 1/(1/10 + 9/10P) = 1/((9+P)/10P) = 10P/(9+P)
   P = 1 => S = 1
   P = 2 => S = 1.81
   P = 3 => S = 2.5
   P = 4 => S = 3.07
   P = 8 => S = 4.7

3. **Ce se intampla la rularea codului pentru algoritmul Producer-Consumer cu mai multi producatori si consumatori?**

   Cand un producator doreste sa adauge date in buffer, el scade valoarea semaforului “gol” (cel care permite sau nu scrierea in buffer), adauga datele in buffer si creste valoarea semaforului “plin” (cel care permite sau nu citirea din buffer). In mod analog, un consumator scade valoarea semaforului “plin” (pentru ca daca vrea sa citeasca, trebuie sa fie cel putin un element in buffer), citeste din buffer si creste valoarea semaforului “gol” (adica s-a eliberat o pozitie in buffer).

4. **Problema colorarii hartilor in paralel in 4 culori cu tari vecine care sa nu aiba aceeasi culoare.**

   ExecutorService. Se porneste de la a da unei tari o culoare anume si apoi functia run din ExecutorService va primi o configuratie si va adauga in coada de taskuri doar configuratii in care adauga o culoare astfel incat sa fie colorari partiale bune. Cand s-a gasit o solutie (s-au colorat toate tarile si regulile sunt respectate) se apeleaza metodele shutdown(), awaitTermination(time, measurementUnit), shutdownNow().

5. **Pd primeste un mesaj si vtm il pune in coada de intarziere si il livreaza doar daca …..**

   - vtm[s] = Vd[s] + 1 (este urmatorul timestamp pe care d il asteapta de la s)
   - vtm[k] <= Vd[k] pentru k != s (d a vazut toate mesajele ce au fost vazute de s la momentul cand a trimis mesajul

6. **Algoritmul ecou.**

   Procesul initiator trimite mesaje spre vecinii sai. Acestia trimit mesaje mai departe spre ceilalti vecini. In momentul in care un nod a primit mesaje de la toti vecinii sai, trimite un mesaj inapoi parintelui si poate decide. Intr-un final, initiatorul primeste mesaje de la toti vecinii si poate crea un eveniment decide.

7. Mesaje de confirmare versus mesaje cu marcaje.

La mesajele de confirmare, se realizeaza urmatoarele actiuni (pentru un arbore):

- Cand un process frunza termina, el isi anunta parintele
- Cand un nod oarecare termina, asteapta terminarea tuturor fiilor si apoi isi anunta parintele
- Notificarile de terminare se propaga pana la procesul initiator

In plus, daca este un graf dirijat aciclic:

- Deficitul unei legaturi = (nr mesaje de date transmise) – (nr semnale de confirmare primite)
- La terminare, un nod asteapta primirea semnalelor pe legaturile de iesire a datelor ca sa duca deficitul la 0 pe acele legaturi
- Tot la terminare, un nod trimite pe fiecare legatura de intrare a datelor un numar de semnale egal cu deficitul acelei legaturi

Daca graful are cicluri:

- Se genereaza un arbore de acoperire (fiecare nod isi salveaza ca “prim” primul nod de unde a primit mesaj (el fiind calea spre initiator)
- Pentru terminare, se trimit semnale pe toate legaturile de intrare (mai putin prim), apoi se receptioneaza semnale pentru toate legaturile de iesire, apoi se trimite semnal spre prim

La mesajele folosind marcaje:

- Procesele transmit mesaje de marcaj la sfarsitul comunicatiei
- Procesele primesc semnale de confirmare doar pentru marcaje
- Pentru terminare:
    - Se asteapta marcaje pe legaturile de intrare
    - Se transmit marcaje pe legaturile de iesire
    - Se transmit semnale pe legaturile de intrare (mai putin prim)
    - Se asteapta confirmare pe legaturile de iesire
    - Se transmite semnal pentru prim

Fiecare arc continue doua attribute: active si marcaj

Pentru fiecare arc de intrare(iesire) se inregistreaza daca:

- S-a primit(trimis) un mesaj (se pune activ pe true)
- S-a primit(trimis) un marcaj (se pune marcaj pe true)
- S-a primit(trimis) un semnal de confirmare (se pun activ si marcaj pe false)

8. Descrieti mecanismul de Blockchain forks.

Toate nodurile vor alege lantul de lungime mai mare. Daca un nod onest foloseste un blockchain de lungime N si este notificat de un blockchain de lungime M (M > N), nodul onest va comuta imediat pe folosirea acestuia. Tranzactiile din blocurile ramase “orfane” sun anulate.

Ca avantaje:

- toate nodurile bitcoin converg rapid catre un blockchain comun
- cu cat sunt mai multe blocuri adaugate pe chain, cu atat e mai improbabil ca tranzactiile noastre sa fie retrase
- blocurile extra servesc unor confirmari superioare (ele confirma validitatea tranzactiei)

Varianta C

1. **Ce garanteaza o zona critica? (o singura propozitie)**

   O zona critica garanteaza ca un singur fir de executie va executa codul din acea zona la un moment dat.

2. **Dati un exemplu de race condition, altul decat cel prezentat in curs. Explicati de ce este race condition.**

```
int x = 5;
int y;

T1
if (x == 5) {
        y = x * 2;
}

T2
x = 8;
```

   Posibile raspunsuri: y = 10 sau y = 16

3. **Ce se intampla cand se executa codul de mai jos?**

```
typeT buf[1:k];
int prim = 1, ultim = 1;
sem gol = k, plin = 0, mutex = 1;

process Producator[i = 1 to M] {
        typeT v;
        while(true) {
                v = produce();
                P(gol);
                   P(mutex);
```

```
                buf[ultim] = v;
                ultim = ultim mod k + 1;
            V(mutex);
        V(plin);
    }
}

process Consumator[i = 1 to N] {
    typeT w;
    while (true) {
        P(plin);
            P(mutex);
                w = buf[prim];
                prim = prim mod k + 1;
            V(mutex);
        V(gol);
        consuma(w);
    }
}
```

Raspuns: implementarea este foarte asemanatoare cu a unui producer-consumer cu mai multi producatori si mai multi consumatori si, desigur, un buffer cu mai mult de 1 element. Diferenta este ca in codul de mai sus bufferul poate fi accesat doar de un proces (fie consummator, fie producator) la un moment dat, deci un consummator nu poate scoate date in buffer in timp ce un producator adauga date in buffer si nici invers.

4. **Se da o lista de elemente (numere naturale). Scrieti pseudocod pentru o solutie paralela, eficienta, de a determina cea mai mica distanta intre oricare doua elemente.**

5. **Considerati o implementare de semafor distribuit cu doua tipuri de procese: procesele ce apeleaza operatiile semaforului (P si V) – Utiliz(i) si cele care implementeaza logica semaforului distribuit – Ajutor(i). Completati pseudocodul pentru procesele Ajutor in cazul receptionarii unui mesaj:**

```
while(true) {
    receive opsem[i](transm, k ,ts);
        // transm = ID sender; k = felul mesajului: P, V sau ack;
        // ts = timestamp-ul de trimitere
    TODO
    // sem = semafor
    // qm = coada de mesaje ordonata dupa ts
    cl = max(ts, cl) + 1;
    if (k == P or k == V) {
        insereaza (transm, k, ts) in locul corespunzator in qm;
```

```
                cl = cl + 1;
                broadcast opsem(I, ack, cl);
        } else if (k == ack) {
                inregistreaza transmiterea unui ack;
                for [mesajele V complet confirmate] {
                        scoate mesaj din qm;
                        sem = sem + 1;
                }
                for [mesajele P complet confirmate st sem > 0] {
                        scoate mesaj (transm, k, ts) din qm; //atentie la coliziuni
                        sem = sem – 1;
                        if (transm == i) {
                                cl = cl + 1;
                                send start[i](cl);
                        }
                }
        }
}
```

6. **Descrieti in pseudocod faza de wakeup pentru Algoritmul Arbore. Ce variabile foloseste fiecare process pentru aceasta faza?**

```
If (p este initiator) {
        ws = true;
        for [q = 1 to r st Vecini[q]] {
                wakeup[q]();
        }
}

while (wr < numar_vecini_p) {
        receive wakeup[p]();
        wr = wr + 1;
        if (NOT ws) {
                ws = true;
                for [q = 1 to n st Vecini[q]] {
                        send wakeup[q]();
                }
        }
}
```

Procesele folosesc variabilele:

- ws = un Boolean care asigura ca procesele trimit o singura data mesaje de wakeup
- wr = un intreg ce reprezinta cate procese vecine au trimit mesaj de wakeup procesului current

7. **Explicati ce reprezinta Finger Table in algoritmul Chord. Unde sta acest Finger Table?**

Un mechanism ce permite operatii de lookup in timp logN. Finger I redirecteaza catre succesorul lui n + 2^i. Exista un finger table in fiecare nod din reteaua inel peer-to-peer.

8. **Descrieti functionarea algoritmului Huang. Care sunt operatiile facute la trimiterea/receptionarea unui mesaj.**

   Se porneste cu un agent de control ce are initial ponderea 1. Toate celelalte procese au ponderea 0. Agentul de control trimite un mesaj de date unui process. Astfel, acel process devine activ si poate trimite mai departe mesaje de date.
   Operatii:
   - Send B(DW) – se imparte ponderea curenta W in doua ponderi W1 > 0 si W2 > 0 astfel incat W1 + W2 = W. Procesul current preia ponderea W1 si transmite ponderea W2
   - Receive B(DW) – se aduna ponderea primita la ponderea curenta: W_nou = W_vechi + DW. Daca nodul era liber, trece in stare active
   - Send C(DW) – se trimite intreaga pondere agentului de control si procesul devine liber
   - Receive C(DW) – doar agentul de control executa aceasta operatie. Aduna ponderea primita la ponderea curenta, iar daca este 1, declara “terminarea” algoritmului

Varianta D

1. **Definiti si dati formula pentru legea lui Amdahl.**

   Legea lui Amdahl spune ca speed-up-ul unui program paralel depinde de procentul de operatii din algoritmul secvential ce nu se pot executa in paralel.
   Sau
   Timpul de executie al unui program parallel depinde de procentul de operatii din algoritmul secvential ce nu se pot executa in parallel.

   Formula:
   $S = 1 / (f + (1 – f) / P)$
   Sau
   $T = fG + (1 – f) * (G / P)$

2. **Specificati macar un motiv pentru care ati regasi o situatie cand S > P.**

   Cand paralelizarea codului duce la impartirea memoriei folosite astfel incat aceasta sa se afle mai des in cache, lucru ce mareste considerabil executia programului. Programul secvential trebuia sa tot preia date din memoria RAM deoarece nu se putea salva pe toate in cache, in timp ce programul parallel are de cele mai multe ori (daca nu chiar de fiecare data) datele in

cache, facand de mult mai putine ori accese la memoria RAM decat algoritmul secvential. De aceea, intr-o situatie de acest fel se poate ca S > P.
O alta posibilitate este in rezolvarea problemelor prin algoritmi backtracking unde paralelizarea problemei poate dupa la gasirea mult mai rapid a unei solutii corecte.

3. **Dati un exemplu de situatie in Java cand folositi synchronized, si altul cand folositit wait/notify. Justificati utilitate folosirii unei constructii in detrimentul celeilalte.**

   Synchronized ca sa izolezi o zona critica. El deblocheaza singur accesul la zona critica ("da un unlock la final"). Wait si notify se folosesc intr-un bloc synchronized si ajuta la comunicarea intre threaduri deoarece daca un thread a ajuns la wait, el va fi pornit doar daca altul apeleaza notify.

4. **Parallel scan presudocod de corectat.**

   Lipseste bariera deoarece for-ul executa operatii pe elemente ce isi schimba valoarea de la un pas la altul si asta ar duce la rezultate aleatoare. Bariera trebuie pusa in for (fie prima linie din for, fie ultima).

5. **Lamport (relatia de "petrecut inainte")**

   Fie doua evenimente a si b:
   - Daca a si b sunt evenimente interne ale unui process si a il precede in timp pe b, atunci a->b
   - Daca a reprezinta un eveniment de transmitere a unui mesaj, iar b evenimentul de receptionare a aceluiasi mesaj, atunci a->b
   - Daca a->b si b->c, atunci a->c

6. **Caz favorabil pentru algoritmul LeLann de alegere a liderului.**

   Algoritmul LeLann classic nu are un caz favorabil anume (sau cel putin nu e prin curs), deci este vorba despre algoritmul LeLann-Chang-Roberts, unde inelul este "parcurs" (adica mesajele se transmit) in sensul acelor de ceasornic. Asta face ca aranjarea nodurilor in ordine crescatoare in sensul acelor de ceasornic sa dupa la cazul favorabil al algoritmului.

7. **Pseudocod algoritmul fazelor.**

   Curs 9, slide 34

8. **Mesaje de confirmare vs mesaje cu marcaje.**

   Varianta B, ex 7

Laboratorul 1 - Introducere în programarea paralelă cu Pthreads

Responsabili: Radu Ciobanu, Florin Mihalache, Andrei Damian, Delia Stuparu, Dragoș Ușurelu

Programarea paralelă

În mod tradițional, programele pe care le-ați implementat până acum au fost scrise pentru **calcul serial**. Astfel, o problemă care trebuia rezolvată era împărțită într-un set discret de instrucțiuni, care erau executate pe un singur procesor în mod secvențial, una după cealaltă. La un moment dat de timp, o singură instrucțiune putea fi executată.

De cealaltă parte, **programarea paralelă** reprezintă utilizarea simultană a mai multor resurse de calcul pentru a rezolva o problemă. Din punct de vedere al pașilor de rezolvare, problema trebuie întâi împărțită în componente discrete care pot fi rezolvate concurent. Apoi, fiecare astfel de componentă trebuie mai departe împărțită în câte un set de instrucțiuni. În final, pentru a obține paralelism, instrucțiuni ale componentelor diferite ale problemei se pot executa simultan pe procesoare diferite. Pentru a se putea realiza acest lucru, este necesar un mecanism de coordonare a execuției diferitelor componente ale problemei.

Pentru a putea paraleliza în mod eficient o problemă, ea trebuie să poată fi împărțită logic în componente separate care pot fi executate simultan, iar durata execuției paralele a acestor componente trebuie să fie mai mică atunci când avem mai multe resurse de calcul decât atunci când avem o singură astfel de resursă. Pentru a putea executa un program paralel, este necesar să avem fie o mașină de calcul cu mai multe procesoare/core-uri, fie un număr arbitrar de astfel de mașini de calcul conectate printr-o rețea (dar aici extindem conceptul de programare paralelă către **programarea distribuită**).

Atunci când implementăm un program paralel, trebuie să ținem cont de mai multe considerente de design, cum ar fi:

- cum partiționăm problema?
- cum balansăm încărcarea?
- cum realizăm comunicația între componentele care rulează în paralel?
- ce dependențe de date avem?
- cum sincronizăm componentele paralele ale programului nostru?
- cât de mare este efortul de a paraleliza o problemă?

De-a lungul acestui semestru, vom încerca să adresăm cât mai multe din aceste probleme.

Fire de execuție

Un fir de execuție (sau **thread** în engleză) este definit ca un flux independent de instrucțiuni care pot fi planificate de către sistemul de operare. Din punct de vedere al unui programator, un fir de execuție poate fi descris cel mai bine ca o funcție care rulează independent de programul principal, iar un program paralel (cu mai multe fire de execuție, sau **multi-threaded**) poate fi privit ca toată mulțimea de astfel de funcții care pot fi planificate să ruleze simultan și/sau independent de către sistemul de operare.

Atenție! Există o distincție foarte importantă între conceptul de **proces** și cel de **thread**! Veți intra în detaliu la alte materii, dar, în acest moment, este foarte important de reținut faptul că un proces este o instanță de rulare a unui program (și deci **două procese distincte nu partajează spațiul de adrese**, care include stiva de program, variabile, date, etc.), pe când un thread este o unitate de lucru a unui proces (deci **mai multe thread-uri pot avea acces partajat la variabile și alte date**).

Datorită faptului ca firele de execuție ale aceluiași proces partajează resurse, modificările făcute de către un thread asupra acelor resurse (cum ar fi, de exemplu, închiderea unui fișier) vor fi observate de toate thread-urile acelui proces. Mai mult, doi pointeri cu aceeași valoare referă aceleași date, iar scrierea și citirea în/din aceeași zonă de memorie este posibilă, dar necesită sincronizare explicită din partea programatorului (mai multe detalii despre ce înseamnă sincronizarea și cum se realizează veți afla în laboratorul 2).

În general, programele care pot beneficia de implementare multi-threaded au câteva trăsături comune:

- conțin componente computaționale care se pot executa în paralel
- au date pe care se poate opera în paralel
- se blochează ocazional așteptând după I/O
- trebuie să răspundă la evenimente asincrone
- anumite componente de execuție au o prioritate mai mare decât altele.

Pthreads (thread-uri POSIX)

La începutul programării paralele, producătorii de hardware își implementau propriile versiuni de thread-uri, care difereau considerabil între ele, ceea ce ducea la dificultăți în dezvoltarea de aplicații multi-threaded portabile. Din acest motiv, o interfață standardizată de programare multi-thread a fost necesară, iar acest lucru a fost specificat, pentru sisteme UNIX, de către standardul IEEE POSIX 1003.1c în anul 1995. Astfel, implementările de thread-uri care aderă la acest standard sunt denumite thread-uri POSIX, sau Pthreads, iar astăzi majoritatea furnizorilor de hardware oferă și Pthreads.

Din punct de vedere al unui programator, Pthreads sunt definite ca o mulțime de tipuri și funcții pentru limbajul C, implementate într-un header numit **pthread.h** și o bibliotecă numită **pthread** (deși, în unele implementări de Pthreads, această bibliotecă poate fi inclusă într-o altă bibliotecă).

Implementarea unui program paralel folosind Pthreads

Compilare și rulare

Pentru a compila un program care folosește Pthreads, va trebui să link-ăm biblioteca de Pthreads. Rularea programului se realizează ca pentru orice alt program C. De exemplu, dacă folosim compilatorul GCC, putem compila și rula un program Pthreads astfel:

```
gcc -o program program.c -lpthread
./program
```

Atenție! Ca să implementăm un program care folosește Pthreads, trebuie să includem header-ul **pthread.h**.

Crearea și terminarea thread-urilor

Într-un program C cu Pthreads, inițial există un singur fir de execuție, numit thread-ul principal (**main thread**). Oricare alt fir de execuție trebuie creat și pornit în mod explicit de către programator, acest lucru făcându-se prin intermediul funcției **pthread_create**, care poate fi apelată de oricâte ori și de oriunde din cod. Funcția are următoarea semnătură:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg);
```

Parametrul *thread* (de tip *pthread_t*) reprezintă un identificator pentru noul thread returnat de această funcție, ce poate fi apoi folosit pentru a opri sau a referi thread-ul. Parametrul *attr* este utilizat pentru a seta diferite atribute pentru firul de execuție care se creează, și poate fi pus pe NULL dacă se dorește păstrarea valorilor implicite. *start_routine* este un pointer la funcția care va fi executată pe firul de execuție nou-creat la pornirea sa. În final, prin intermediul parametrului *arg*, putem trimite un singur parametru către funcția de thread, care trebuie pasat prin referință ca un pointer de tipul *void* (dacă nu se dorește trimiterea unui parametru, putem lăsa NULL). Funcția returnează 0 dacă thread-ul nou s-a creat și s-a pornit cu succes, sau un cod de eroare în caz contrar.

Un thread creat astfel poate la rândul lui crea alte thread-uri, neexistând o ierarhie sau dependență între ele. Numărul maxim de thread-uri care poate fi creat de un proces depinde de implementarea Pthreads, dar în general nu se recomandă să avem un număr de fire de execuție mai mare decât numărul de core-uri de pe mașina pe care rulăm, din motive de overhead (aceasta este o discuție mai lungă pe care vă recomandăm să o purtați cu asistentul vostru).

Funcția pe care un thread o execută atunci când apelăm **pthread_create** trebuie să aibă următoarea semnătură:

```
void *f(void *arg) {
    [...]

    pthread_exit(NULL);
}
```

Parametrul *arg* al funcției de mai sus este primit de funcția **f** la execuția ei pe un thread nou atunci când se apelează **pthread_create**, fiind echivalentului parametrului *arg* de la **pthread_create**. Limitarea dată de faptul că putem trimite un singur parametru funcției de thread poate fi rezolvată prin crearea unei structuri cu oricâți membri, pe care o putem apoi da ca parametru prin referință. Funcția **pthread_exit** trebuie apelată pentru a termina thread-ul care o apelează, și primește ca parametru fie NULL, fie o valoare de retur care va fi pasată mai departe.

În momentul în care apelăm **pthread_create**, firul de execuție nou-creat va rula în paralel cu thread-ul principal. Acest lucru înseamnă că tot codul de după apelul funcției **pthread_create** se va executa în paralel cu codul noului thread. Dacă dorim ca un fir de execuție să aștepte terminarea unui alt thread, putem realiza acest lucru prin funcția **pthread_join**. Prin apelul acestei funcții, ne putem asigura că thread-ul apelant se blochează până când celălalt fir de execuție își termină procesarea (adică termină de executat funcția de thread). Funcția **pthread_join** are următoarea semnătură:

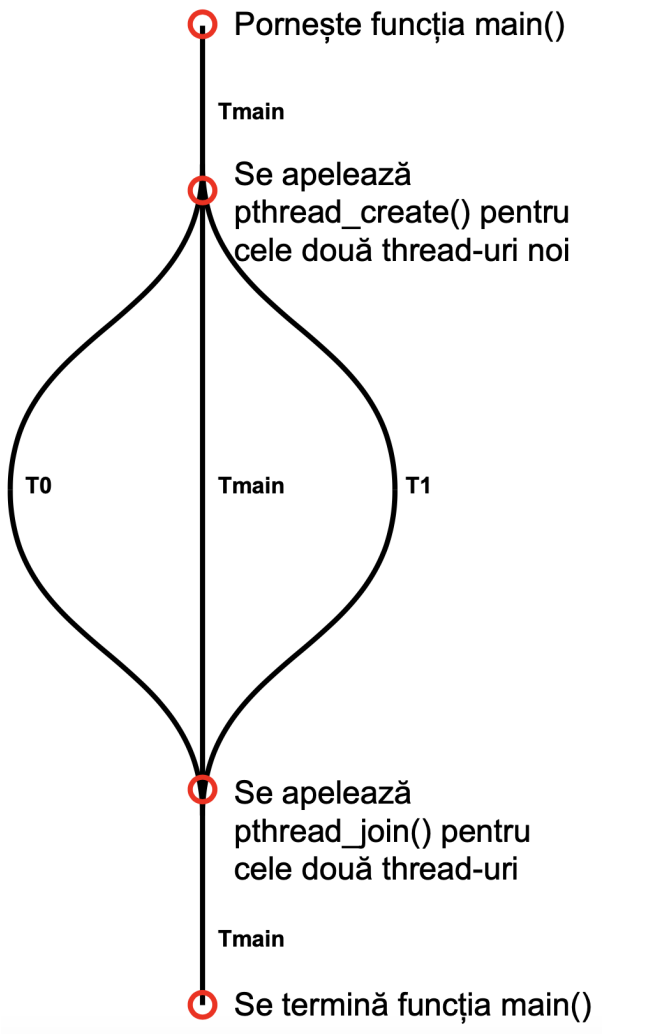
```
int pthread_join(pthread_t thread, void **retval);
```

Parametrul *thread* reprezintă identificatorul thread-ului pe care îl așteptăm, iar *retval* reprezintă valoarea de retur a funcției thread-ului așteptat (și poate fi pus pe NULL dacă nu avem nevoie de această informație). Funcția returnează 0 în caz de succes sau un cod de eroare în caz contrar.

Un exemplu complet de program C care folosește Pthreads se poate observa mai jos. În acest exemplu, se creează două fire de execuție care vor rula aceeași funcție **f**, dar fiecare din ele va primi ca parametru ID-ul curent (0 sau 1). După ce pornește cele două fire de execuție noi, thread-ul principal le așteaptă pe ambele să termine, după care iese și el.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUM_THREADS 2

void *f(void *arg)
{
    long id = *(long*) arg;
    printf("Hello World din thread-ul %ld!\n", id);
    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    pthread_t threads[NUM_THREADS];
    int r;
    long id;
    void *status;
    long arguments[NUM_THREADS];

    for (id = 0; id < NUM_THREADS; id++) {
        arguments[id] = id;
        r = pthread_create(&threads[id], NULL, f, (void *) &arguments[id]);

        if (r) {
            printf("Eroare la crearea thread-ului %ld\n", id);
            exit(-1);
        }
    }

    for (id = 0; id < NUM_THREADS; id++) {
        r = pthread_join(threads[id], &status);

        if (r) {
            printf("Eroare la asteptarea thread-ului %ld\n", id);
            exit(-1);
        }
    }

    pthread_exit(NULL);
}
```

O reprezentare vizuală a execuției programului de mai sus poate fi observată în figura de mai jos.



În figura de deasupra, se poate observa că inițial există un singur fir de execuție, cel principal (marcat cu **Tmain**). În momentul în care acesta apelează **pthread_create** de două ori, el creează și pornește alte două fire de execuție noi (**T0** și **T1**) care vor rula funcția **f** cu parametri diferiți (0, respectiv 1). În acest moment, avem trei thread-uri care rulează în paralel. Când thread-ul principal apelează **pthread_join**, acesta se blochează până când cele două fire de execuție secundare se termină (adică până când termină de executat funcția **f**). Dacă unul din thread-urile secundare și-a terminat execuția înainte de apelul **pthread_join**, funcția va returna instant. La finalul programului, thread-ul principal se va opri și el.

Atenție! Găsiți aici un „cheat sheet” cu cele mai importante elemente Pthreads.

Exerciții

1. Compilați și executați codul din fișierul **example.c** din arhiva de laborator. Schimbați numărul de thread-uri și observați cum se schimbă comportamentul programului. **(2p)**
2. Schimbați numărul de thread-uri din cod astfel încât să fie egal cu numărul de core-uri de pe mașina pe care rulați, astfel încât, rulând codul pe un alt calculator, numărul de thread-uri să se schimbe automat. Verificați **Hint 1** de mai jos pentru informații suplimentare. **(2p)**
3. Modificați funcția **f** astfel încât mesajul „Hello World” să fie afișat iterativ de 100 de ori de fiecare thread, împreună cu indicele iterației. Întrebare: codul afișează mesajele în ordinea așteptată de voi? **(1p)**
4. Modificați programul astfel încât să creeze două thread-uri, fiecare thread rulând propria sa funcție. **(2p)**
5. Pornind de la codul din fișierul **add_serial.c** din arhiva de laborator, paralelizați incrementarea elementelor unui vector cu 100. Acest lucru va presupune împărțirea iterației de adunare la toate

thread-urile într-un mod cât mai echitabil. Verificați **Hint 2** de mai jos pentru informații suplimentare. **(2p)**

6. Demonstrați că programul vostru scalează (adică durează mai puțin când rulați cu mai multe thread-uri). Verificați **Hint 3** de mai jos pentru informații suplimentare. **(1p)**

Hint 1

Pentru a putea obține numărul de core-uri de pe un calculator, se poate folosi funcția **sysconf** astfel:

```
#include <unistd.h>

long cores = sysconf(_SC_NPROCESSORS_CONF);
```

Hint 2

Pentru exercițiul 5, avem un vector de **N** elemente pe care vrem să-l împărțim în mod (aproximativ) egal la **P** thread-uri, unde fiecare thread are un **ID** de la 0 la P-1. Astfel, fiecare thread va itera pe câte o secțiune din vectorul inițial, fără a afecta operațiile celorlalte thread-uri. Este necesar deci să calculăm indexul de **start** și indexul de final (**end**) pentru fiecare thread. Un mod de a calcula aceste două valori poate fi următorul:

```
int start = ID * (double)N / P;
int end = min((ID + 1) * (double)N / P, N);
```

Hint 3

Pentru a putea observa mai bine scalabilitatea unui program, este necesar ca acesta să dureze măcar câteva secunde, deoarece, în caz contrar, timpul de inițializare, alte programe care rulează pe calculator și overhead-ul cauzat de planificarea firelor de execuție ar putea afecta timpii de execuție suficient cât să nu vedem scalabilitatea doar prin măsurarea timpului total de execuție.

Mai mult, inițializarea serială a vectorului (în main) are o durată comparabilă cu execuția pe un thread a operației de paralelizat. De aceea, **pentru exercițiul 5, se recomandă să creșteți durata de execuție a unui thread prin repetarea iterativă a operațiilor realizate în funcția de thread**.

Ca să verificați dacă un program scalează, trebuie să îi măsurați durata de execuție atât la o rulare secvențială (cu un singur thread), cât și la o rulare cu mai multe fire de execuție. Pentru acest lucru, puteți folosi comanda **time** în linia de comandă, astfel:

```
$ time ./program
real    0m6.958s
user    0m6.745s
sys     0m0.010s
```

Timpul care ne interesează este cel real (adică timpul „pe ceas”).

Hint 4

O metodă bună de a face debugging la un program C multi-threaded este utiliarul **gdb**. Pe lângă comenzile **gdb** pe care le știți deja, ar mai fi de interes comenzile **info threads** (care afișează informații despre thread-urile existente la momentul curent de timp) și **thread <N>** (care mută contextul de execuție pe thread-ul N).

apd/laboratoare/01.txt · Last modified: 2021/01/11 21:09 by florin.mihalache

Laboratorul 2 - Elemente de sincronizare în Pthreads

Responsabili: Radu Ciobanu, Andrei Damian, Delia Stuparu, Dragoș Cocîrlea

Introducere

Când vorbim de programarea paralelă, putem avea situații în care mai multe fire de execuție care rulează în paralel vor să acceseze simultan aceleași resurse. De exemplu, putem avea situația următoare. Avem două thread-uri (**T0** și **T1**) care au acces partajat la o variabilă întreagă *a* inițializată cu 0. În funcția de thread, atât **T0** cât și **T1** incrementează *a* cu 2. În mod normal, ne-am aștepta ca valoarea variabilei *a* după execuția programului nostru să fie 4, pentru că avem două incrementări ale variabilei în cele două thread-uri.

În realitate, situația nu este întotdeauna așa. Dacă am traduce incrementarea unei variabile întregi în cod de asamblare, această operație ar putea arăta în felul următor (în exemplul de mai jos, *eax0* reprezintă registrul *eax* al thread-ului **T0**, iar *eax1* desemnează registrul *eax* al thread-ului **T1**):

T0	T1
load(a, eax0)	load(a, eax1)
eax0 = eax0 + 2	eax1 = eax1 + 2
write(a, eax0)	write(a, eax1)

Putem avea următorul scenariu:

- **T0** citește valoarea lui *a* (0) în propriul registru *eax0*
- în același timp, **T1** citește valoarea lui *a* (tot 0) în propriul registru *eax1*
- **T0** incrementează valoarea lui *eax0*, care devine 2
- **T1** face același lucru, iar *eax1* devine tot 2
- **T0** scrie valoarea din *eax0* în a, care devine 2
- **T1** scrie valoarea din *eax1* în a, care rămâne tot 2.

Se poate deci observa că, în funcție de modul în care thread-urile **T0** și **T1** sunt planificate, este posibil ca rezultatul secvenței de mai sus să fie 2 sau 4. Acest lucru se numește **race condition** și este cauzat de faptul că rezultatul calculului este condiționat de modul de planificare a unor evenimente necontrolabile. Operația de incrementare a lui *a* cu 2 nu este **atomică**, fiind compusă din mai multe operații care se pot intercala atunci când rulăm pe mai multe fire de execuție.

Mutex

Un **mutex** (abreviere de la „mutual exclusion”) este o primitivă de sincronizare prin care putem proteja accesul la date atunci când avem scrieri (potențial) concurente. El funcționează ca un „zăvor” ce protejează accesarea unor resurse partajate.

Un mutex se folosește pentru a delimita o **regiune critică**, adică o zonă a programului în care se poate afla cel mult un thread la un moment dat de timp. Dacă un thread **T1** încearcă să intre într-o regiune critică atunci când alt thread **T0** este deja acolo, **T1** se va bloca până când **T0** va ieși din regiunea critică.

În cadrul exemplului de mai sus, am putea folosi un mutex pentru a defini o regiune critică în jurul operației de incrementare a lui *a*, lucru care ar face imposibilă intercalarea operațiilor celor două thread-uri. Primul thread care intră în regiunea critică va incrementa *a* în mod exclusiv la 2, iar cel de-al doilea thread nu va putea incrementa *a* decât atunci când el este deja 2.

Un mutex are două operații principale: închidere (**lock**) și deschidere (**unlock**). Prin închidere, un thread marchează intrarea în zona critică, adică specifică faptul că orice alt thread care va încerca să facă o operație de închidere va trebui să aștepte. Prin deschidere, se marchează ieșirea din zona critică și deci permisiunea ca un alt thread să intre în zona critică.

În Pthreads, o secvență tipică de folosire a unui mutex arată în felul următor:

1. se creează și se inițializează o variabilă de tip mutex
2. mai multe thread-uri încearcă să închidă mutexul (adică să intre în zona critică)
3. unul singur dintre ele reușește acest lucru și ajunge să „dețină” mutexul (adică se află în regiunea critică)
4. thread-ul aflat în zona critică realizează diverse operații pe datele protejate
5. thread-ul care deține mutexul iese din zona critică (deschide mutexul)
6. alt thread intră în zona critică și repetă procesul
7. la final, variabila de tip mutex este distrusă.

În Pthreads, un mutex se reprezintă printr-o variabilă de tip *pthread_mutex_t*, și se inițializează folosind următoarea funcție:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

Primul parametru reprezintă o referință la variabila mutex, iar al doilea parametru specifică atributele mutexului nou-creat (dacă se dorește un comportament implicit, parametrul *attr* se poate lăsa NULL).

Pentru a dezaloca un mutex, se folosește următoarea funcție, care primește ca parametru un pointer la mutexul care urmează a fi dezalocat:

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Pentru a se face lock pe un mutex, se folosește următoarea funcție, care primește ca parametrul mutexul:

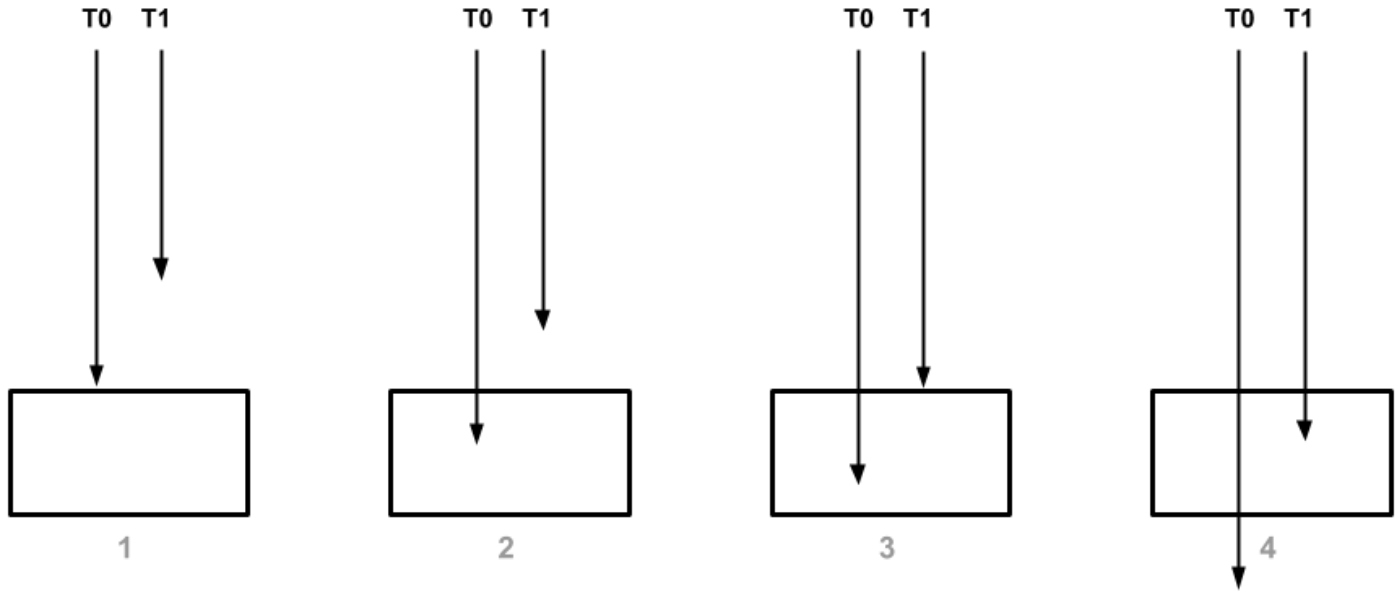
```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

Operația inversă, prin care se specifică ieșirea dintr-o zonă critică (adică deschiderea mutexului), este executată prin intermediul următoarei funcții:

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Toate patru funcțiile de mutex returnează 0 dacă s-au executat cu succes, sau un cod de eroare în caz contrar.

O reprezentare grafică a funcționării unui mutex se poate vedea în figura de mai jos, într-un scenariu în care avem două thread-uri (**T0** și **T1**) și o regiune critică controlată de mutex (chenarul negru din imagine). La momentul de timp **1** (în partea stângă a figurii), **T0** încearcă să intre în regiunea critică. Pentru că, la momentul respectiv de timp, niciun alt thread nu deține mutexul (adică nu se află în regiunea critică), **T0** intră în regiunea critică (momentul de timp **2**). Mai departe, atunci când **T1** ajunge la intrarea în regiunea critică (încearcă să facă lock pe mutex) la momentul de timp **3**, se blochează pentru că mutexul este curent deținut de **T0** (acesta se află în regiunea critică). Abia în momentul în care **T0** a ieșit din zona critică (la momentul de timp **4**), **T1** se va putea debloca și își va continua execuția.



Atenție! Dacă vrem să protejăm o secțiune din programul nostru folosind un mutex, atunci fiecare thread care accesează acea secțiune trebuie să facă lock și unlock pe aceeași variabilă mutex. De asemenea, dacă un thread vrea să facă unlock pe un mutex pe care nu îl deține (nu a făcut lock pe el în prealabil), va rezulta un comportament nedefinit.

Barieră

O altă primitivă de sincronizare folosită în calculul paralel este **bariera**. Ea are rolul de a se asigura că niciun thread nu poate trece mai departe de punctul în care este plasată decât atunci când toate thread-urile gestionate de barieră ajung în acel punct. Un exemplu de utilizare este atunci când împărțim un calcul pe mai multe thread-uri și vrem să nu mergem mai departe cu execuția programului decât în momentul în care fiecare thread și-a terminat propriile calcule.

În Pthreads, o barieră este reprezentată prin tipul *pthread_barrier_t* și inițializată prin următoarea funcție:

```
int pthread_barrier_init(pthread_barrier_t *barrier, const pthread_barrierattr_t *attr, unsigned count);
```

Primul parametru reprezintă o referință la barieră, al doilea parametru poate fi folosit pentru setarea unor atribute ale barierei (la fel ca la mutex), iar ultimul parametru denotă numărul de thread-uri care trebuie să ajungă la barieră pentru ca aceasta să se deblocheze. Acest lucru înseamnă că bariera are un contor intern care numără thread-urile care așteaptă deblocarea ei. Atunci când contorul ajunge la numărul setat la inițializarea barierei, thread-urile își pot continua execuția paralelă.

Pentru a dezaloca o barieră, se folosește următoarea funcție:

```
int pthread_barrier_destroy(pthread_barrier_t *barrier);
```

Ambele funcții returnează 0 dacă s-au executat cu succes sau un cod de eroare în caz contrar.

Pentru a face un thread să aștepte la o barieră (pentru a „pune o barieră” în cod), se folosește următoarea funcție:

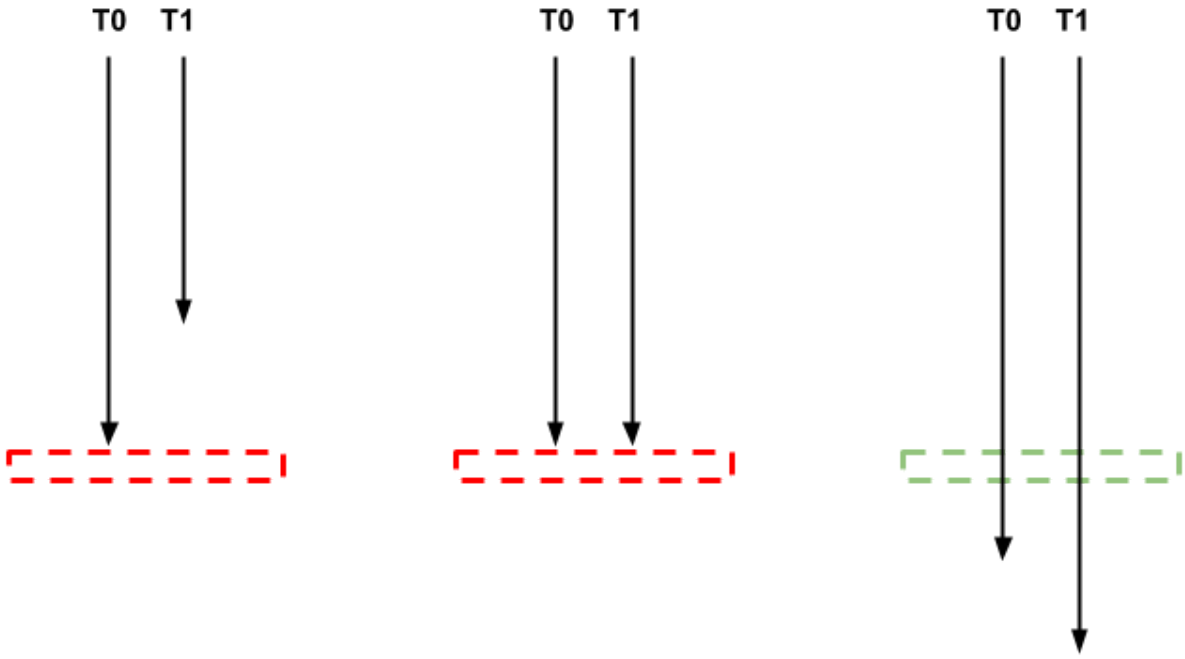
```
int pthread_barrier_wait(pthread_barrier_t *barrier);
```

Funcția de mai sus va returna PTHREAD_BARRIER_SERIAL_THREAD pentru un singur thread arbitrar de la barieră și 0 pentru toate celelalte. Dacă funcția a avut vreo eroare, un cod eroare va fi returnat.

Atenție! Fiecare thread care trebuie să aștepte la barieră va apela funcția de mai sus pe aceeași variabilă de tip *pthread_barrier_t*. Dacă numărul de thread-uri care apelează **pthread_barrier_wait** este mai mic decât parametrul cu care a fost inițializată bariera, aceasta nu se va debloca niciodată.

O reprezentare grafică a modului de funcționare a unei bariere se poate vedea în figura de mai jos, unde avem o barieră inițializată cu 2. Atunci când thread-ul **T0** ajunge la barieră, acesta se blochează

în așteptare, așa cum se observă în partea stângă a imaginii. La un moment dat de timp, **T1** va ajunge și el la barieră, așa cum se vede în centrul imaginii. Abia în acel moment, cele două thread-uri își vor putea continua execuția individual, cum se observă în partea dreapta a imaginii.



Mai multe informații detaliate despre API-ul Pthreads găsiți în tutorialul LLNL [https://computing.llnl.gov/tutorials/pthreads/] și în man [https://man7.org/linux/man-pages/man7/pthreads.7.html].

Exerciții

1. Compilați fișierul **mutex.c** din arhiva de laborator și rulați de mai multe ori programul obținut. Veți observa că rezultatul nu este întotdeauna cel așteptat. Rezolvați problema de sincronizare folosind un mutex. **(2p)**
2. Pornind de la fișierul **barrier.c** din arhiva de laborator, folosiți o barieră pentru a vă asigura că output-ul va fi întotdeauna „1\n2”. Verificați **Hint 1** de mai jos pentru informații suplimentare. **(2p)**
3. Pornind de la fișierul **multiply.c** din arhiva de laborator, paralelizați programul prin împărțirea iterației exterioare la mai multe thread-uri. Verificați corectitudinea și scalabilitatea programului rezultat. Verificați **Hint 2** de mai jos pentru informații suplimentare. **(2p)**
4. Pornind de același cod, paralelizați doar cea de-a doua buclă de iterație. Verificați corectitudinea și scalabilitatea programului rezultat. **(2p)**
5. Pornind de la același cod, paralelizați doar bucla interioară. Verificați corectitudinea și scalabilitatea programului rezultat. Verificați **Hint 3** de mai jos pentru informații suplimentare. **(2p)**
6. Pornind de la fișierul **strassen.c** din arhiva de laborator, paralelizați înmulțirea unor matrice cu algoritmul Strassen folosind 7 thread-uri. Verificați **Hint 4** de mai jos pentru informații suplimentare. **(1p bonus)**

Hint 1

Pe sisteme MacOS, biblioteca de Pthreads nu conține implementarea pentru barieră. Pentru a putea face totuși acest exercițiu, există în arhiva de laborator un fișier numit *pthread_barrier_mac.h* pe care trebuie să-l includeți în fișierul vostru sursă.

Hint 2

Pentru a putea testa corectitudinea paralelizării de la exercițiile 3, 4 și 5, găsiți în arhiva de laborator un script numit *test.sh*. Acesta face următorii pași:

1. verifică să existe un binar numit *multiply_seq* pentru implementarea secvențială a înmulțirii de matrice, pentru care aveți fișierul sursă *multiply_seq.c* în arhiva de laborator (acesta va servi drept etalon pentru corectitudinea implementării paralele)
2. verifică să existe un binar numit *multiply* pentru implementarea voastră paralelă
3. rulează programul secvențial
4. rulează programul paralel
5. compară rezultatele celor două rulări cu *diff*; dacă nu există diferențe, scriptul nu afișează nimic; dacă implementarea paralelă este incorectă, se vor afișa diferențele dintre rularea implementării secvențiale și a celei paralele.

În mod implicit, scriptul rulează pe matrice de 1000×1000 de elemente, cu două thread-uri pentru implementarea paralelă. Dacă doriți să modificați aceste valori (și vă recomandăm să o faceți, pentru o testare cât mai completă), puteți modifica valorile variabilelor **N** și **P** din script.

Hint 3

Veți observa la acest exercițiu că, dacă paralelizați bucla interioară așa cum ați făcut la precedentele două exerciții, rezultatele nu vor fi întotdeauna corecte. De ce? Ce trebuie să faceți pentru o implementare corectă?

Hint 4

Algoritmul lui Strassen [http://stanford.edu/~rezab/classes/cme323/S16/notes/Lecture03/cme323_lec3.pdf] este un algoritm pentru înmulțire de matrice mai rapid decât metoda standard, având o complexitate de $O(N^{2.8074})$. În acest algoritm, la primul pas se definesc 7 matrice adiționale obținute prin înmulțiri ale matricelor bloc obținute din matricele inițiale. La al doilea pas, aceste 7 matrice noi sunt folosite pentru a calcula (prin operații de adunare și scădere) componentele bloc ale matricei finale.

Pentru exercițiul 6, aveți deja implementat calculul matricelor adiționale și calculul final, așa că voi trebuie doar să paralelizați aceste operații.

apd/laboratoare/02.txt · Last modified: 2021/01/11 21:09 by florin.mihalache

Laboratorul 3 - Algoritmi paraleli de sortare

Responsabili: Radu Ciobanu, Andrei Damian, Delia Stuparu, Dragoș Cocîrlea

În acest laborator, vom discuta despre câteva exemple de algoritmi paraleli de sortare.

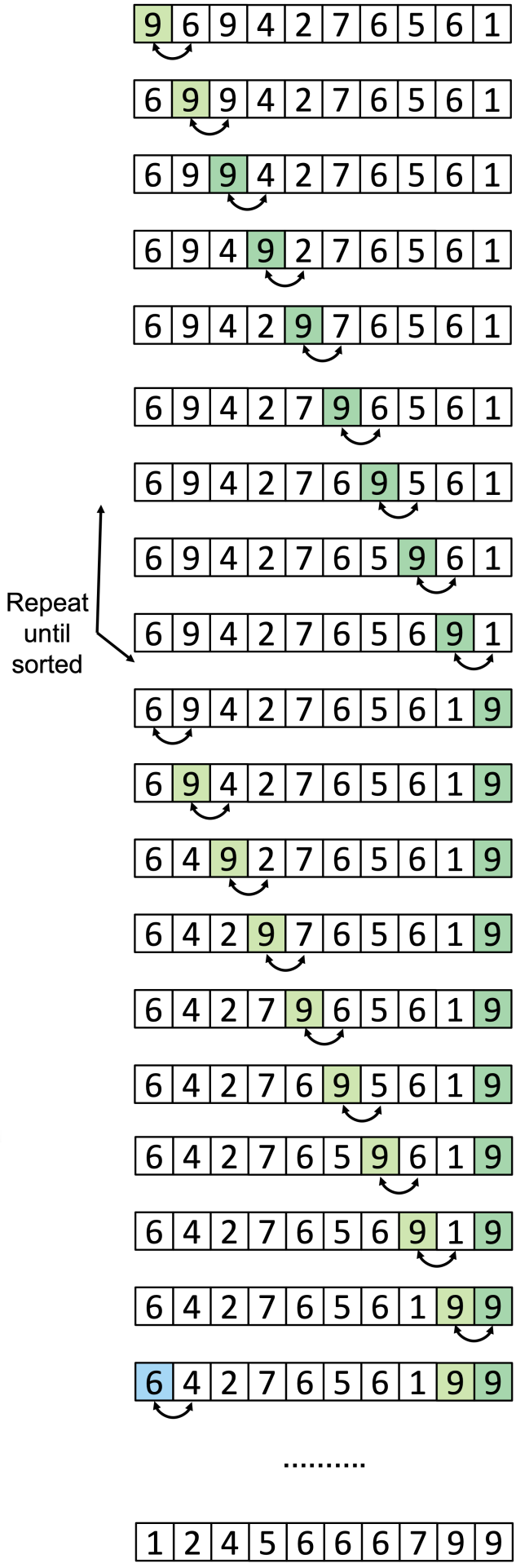
Odd-even transposition sort (OETS)

Unul din cei mai cunoscuți (chiar dacă nu neapărat eficienți) algoritmi de sortare este **bubble sort**. Acesta funcționează pe baza următorului pseudocod:

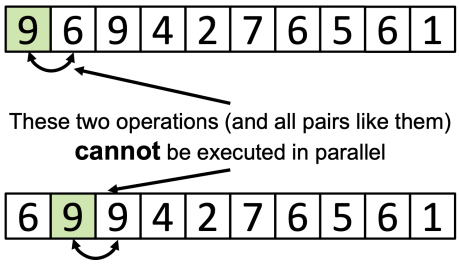
```
function bubbleSort(list) {
  sorted = false;
  while (!sorted) {
    sorted = true;
    for (var i = 0; i < list.length - 1; i++) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
        sorted = false;
      }
    }
  }
}
```

Așa cum se poate observa mai sus, bubble sort parcurge șirul de sortat element cu element, comparând elementul curent cu vecinul din dreapta. Dacă numărul din dreapta este mai mic, se realizează o interschimbare între elementul curent și cel din dreapta sa. Complexitatea acestui algoritm este $\mathbf{O(N^2)}$, pentru că se termină în cel mult N parcurgeri ale șirului (unde N este numărul de elemente din șirul de sortat).

Un exemplu de comportament al bubble sort se poate observa în imaginea de mai jos.



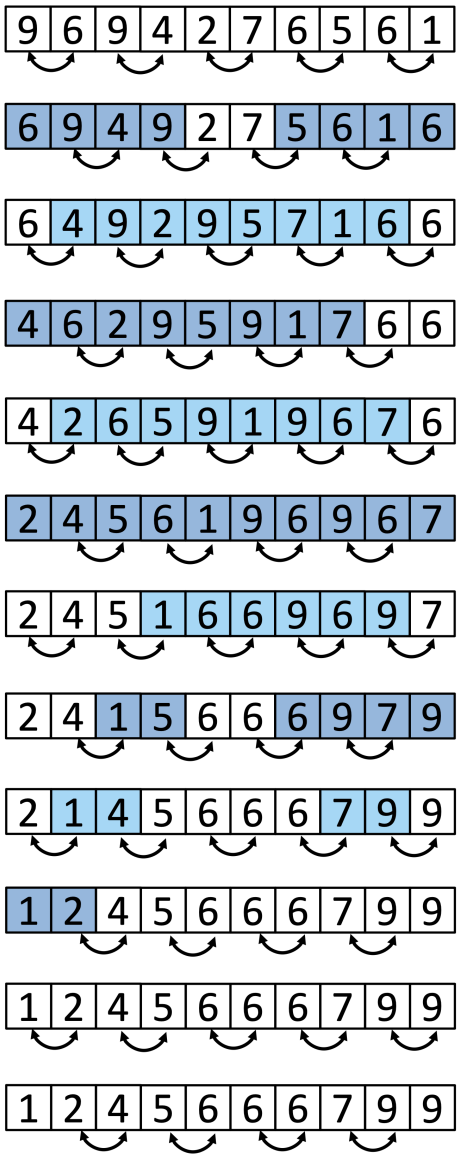
Dacă am dori să paralelizăm acest algoritm, un potențial mod de abordare ar fi să realizăm în paralel comparația și potențiala interschimbare de elemente vecine, dar acest lucru ar putea duce la problema prezentată în imaginea de mai jos.



Mai precis, operații pe elemente adiacente nu se pot realiza simultan, pentru că se poate ajunge la un race condition. Din acest motiv, un mod de a paraleliza bubble sort este dat de algoritmul **odd-even transposition sort**, care funcționează după următorul pseudocod:

```
function oddEvenSort(list) {
  for (var k = 0; k < list.length; k++) {
    for (i = 0; i < list.length - 1; i += 2) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
      }
    }
    for (i = 1; i < list.length - 1; i += 2) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
      }
    }
  }
}
```

Așa cum se poate observa mai sus, odd-even transposition sort are două faze. În **faza pară**, elementele de pe poziții pare din șirul de sortat sunt comparate (și eventual interschimbate) cu vecinii din dreapta. După ce se termină faza pară (adică după ce toate elementele pare au fost procesate), urmează **faza impară**, în care elementele impare sunt analizate și comparate cu vecinii din dreapta. La fel ca la bubble sort, numărul maxim de iterații necesare pentru a sorta un șir va fi N (numărul de elemente din șir). Dacă avem P fire de execuție, complexitatea acestui algoritm va fi **O(N/P*N)**, sau **O(N)** pentru P=N. În imaginea de mai jos, se poate observa o reprezentare grafică a modului de funcționare al odd-even transposition sort.



Algoritmul odd-even transposition sort a fost gândit inițial pentru a fi rulat pe șiruri de procesoare (processor arrays [https://en.wikipedia.org/wiki/Massively_parallel_processor_array]), unde un procesor conține o singură valoare din șirul de sortat și poate comunica doar cu procesorul din stânga și cu cel din dreapta.

Shear sort

Un alt exemplu de algoritm de sortare care a fost conceput pentru sisteme multi-procesor unde un procesor este conectat doar la o parte din celelalte procesoare este **shear sort** (cunoscut de asemenea ca **row-column sort** sau **snake-order sort**), care presupune că lucrăm pe procesoare conectate într-o formă de matrice. Astfel, un procesor poate să comunice cu vecinii din stânga, din dreapta, de sus și de jos. Dacă ne imaginăm deci că procesoarele sunt așezate într-o matrice, cele două faze ale algoritmului shear sort sunt următoarele:

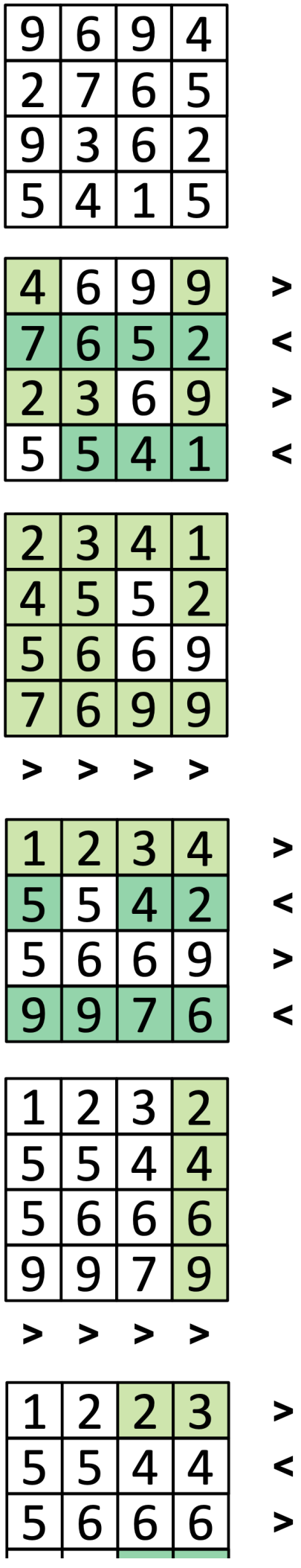
- se sortează liniile matricei astfel încât randurile pare au valorile ordonate crescător, iar rândurile impare au valorile ordonate descrescător
- se sortează coloanele crescător.

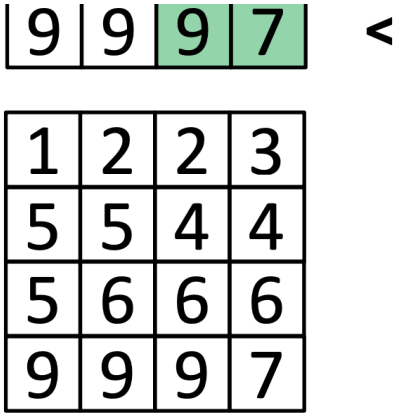
Se garantează că algoritmul va sorta numerele după cel mult **sup($\log_2$N) + 1** faze, unde N este numărul de elemente ce trebuie sortate. Din acest motiv, algoritmul are complexitatea **O(N$\log_2$N)**. Pseudocodul algoritmului este prezentat mai jos.

```
function shearSort(matrix) {
  for (k = 0; k < ceil(log2(matrix.lines * matrix.columns)) + 1; k += 2) {
    for (i = 0; i < matrix.lines; i += 2) {
      sortAscendingLine(i);
    }
```

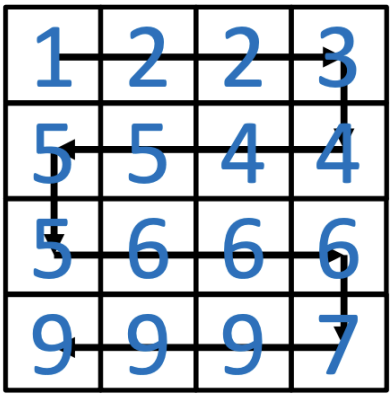
```
    for (i = 1; i < matrix.lines; i += 2) {
      sortDescendingLine(i);
    }
    for (var i = 0; i < matrix.columns; i++) {
      sortAscendingColumn(i);
    }
  }
}
```

O reprezentare grafică a funcționării shear sort este prezentată în figura de mai jos.


9 6 9 4
2 7 6 5
9 3 6 2
5 4 1 5
4 6 9 9 >
7 6 5 2 <
2 3 6 9 >
5 5 4 1 <
2 3 4 1
4 5 5 2
5 6 6 9
7 6 9 9
> > > >
1 2 3 4 >
5 5 4 2 <
5 6 6 9 >
9 9 7 6 <
1 2 3 2
5 5 4 4
5 6 6 6
9 9 7 9
> > > >
1 2 2 3 >
5 5 4 4 <
5 6 6 6 >



La finalul rulării algoritmului, lista de numere va fi sortată într-un mod „șerpuit”, de unde și numele algoritmului. Acest lucru se poate observa în imaginea de mai jos.



Merge sort paralel

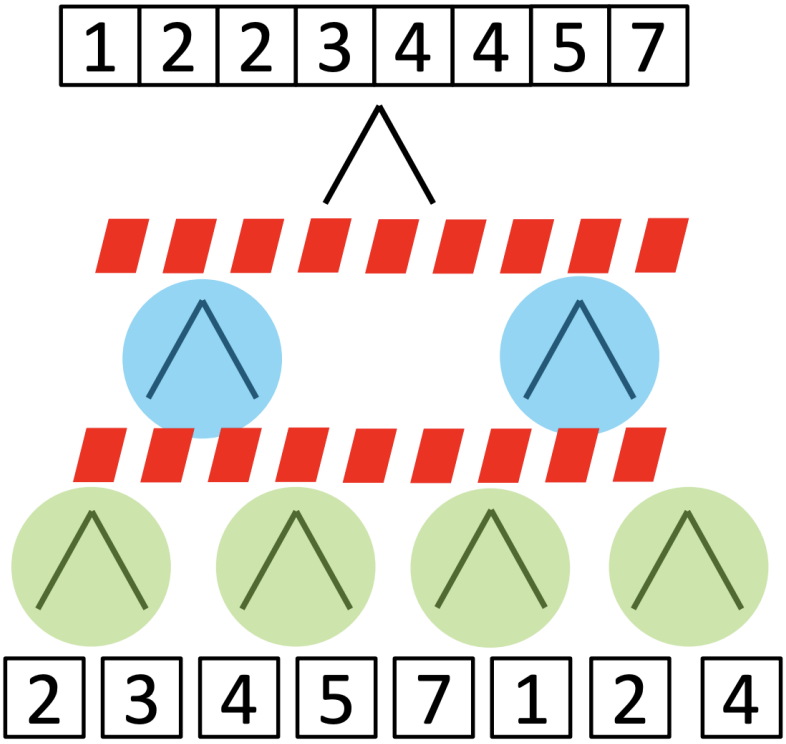
Merge sort (sau sortarea prin interclasare) este un algoritm de sortare de tip ***divide et impera*** care presupune următorii pași generali:

1. se împarte șirul de N elemente de sortat în N șiruri de lungime 1
2. se aplica operația de interclasare („merge”) între câte două astfel de șiruri de lungime 1, rezultând N/2 șiruri sortate de lungime 2
3. se repetă pașii de mai sus realizând interclasări între șiruri din ce în ce mai mari, până se ajunge la un șir sortat de N elemente.

Numărul de pași de interclasare necesari este $\log_2 N$, iar operațiile de interclasare de la un pas se realizează în O(N), deci complexitatea algoritmului merge sort este **$O(N\log_2 N)$**.

Pentru a paraleliza acest algoritm, putem observa că operațiile de interclasare de la fiecare pas se pot realiza în paralel. Totuși, operațiile de „merge” de la fiecare pas trebuie terminate în totalitate înainte de a trece la următorul pas, deci avem nevoie de o barieră (sau un mecanism similar) după fiecare pas de interclasare. Se poate observa că gradul de paralelism de la un pas de interclasări este din ce în ce mai mic pe măsură ce avansăm în algoritm, pentru că numărul de operații de „merge” de la fiecare pas scade. Complexitatea algoritmului paralel este **O(N)** pentru P=N.

O reprezentare grafică a algoritmului de merge sort paralel se poate observa în imaginea de mai jos, unde operațiile cu aceeași culoare pot fi realizate în paralel, iar simbolurile cu roșu reprezintă bariere.



Exerciții

1. Pornind de la implementarea secvențială a algoritmului bubble sort aflată în fișierul **oets.c** din arhiva de laborator, implementați și paralelizați algoritmul odd-event transposition sort. **(3p)**
2. Analizați scalabilitatea implementării de la punctul precedent. Verificați **Hint 1** de mai jos pentru informații suplimentare. **(1p)**
3. Pornind de la implementarea secvențială a algoritmului shear sort aflată în fișierul **shear.c** din arhiva de laborator, paralelizați algoritmul. **(3p)**
4. Analizați scalabilitatea implementării de la punctul precedent. Verificați **Hint 1** de mai jos pentru informații suplimentare. **(1p)**
5. Pornind de la implementarea secvențială a algoritmului merge sort aflată în fișierul **merge.c** din arhiva de laborator, paralelizați algoritmul. Verificați **Hint 2** de mai jos pentru informații suplimentare. **(2p)**

Hint 1

În toate fișierele sursă din arhiva de laborator, verificarea corectitudinii implementării paralele se realizează prin sortarea șirului folosind quick sort într-un array folosit ca etalon la final (se realizează o comparație element cu element). Atunci când doriți să verificați scalabilitatea programului vostru, comentați sau eliminați sortarea folosind quick sort și comparația aferentă, pentru a avea niște timpi cât mai corecți. De asemenea, încercați să eliminați toate afișările în terminal.

Hint 2

Implementarea merge sort din arhiva de laborator funcționează pentru un N putere a lui 2. Testați-vă implementarea paralelă doar pe acest caz.

apd/laboratoare/03.txt · Last modified: 2021/01/11 21:09 by florin.mihalache

Laboratorul 4 - Introducere în Java Multithreading

Responsabili: Alexandru-Ionuț Mustață, Gabriel Guțu-Robu

Suportul de multithreading oferit de Java

Spre deosebire de limbajul C unde implementarea unui thread și a mecanismelor de sincronizare depinde de o bibliotecă ce este specifică unui anumit tip de sistem de operare (Linux, Windows, MacOSX), Java oferă suport pentru lucrul cu thread-urile direct din cadrul SDK său.

Implementarea unui nou thread

În Java există două modalități prin care putem să implementăm un nou thread.

Prima variantă de implementare are la bază crearea unei clase ce implementează interfața **Runnable** ce conține metoda **void run()**. Codul ce va reprezenta logica noului thread va fi plasat în interiorul acestei metode.

```
public class Task implements Runnable {
    public void run() {
        System.out.println("Hello from my new thread!");
    }
}
```

O altă metodă de implementare a unui thread constă din crearea unei clase ce extinde clasa **Thread** și suprascrie (overrides) metoda **void run()** din cadrul acesteia. La fel ca în primul caz, logica noului thread va fi implementată în cadrul acestei metode.

```
public class MyThread extends Thread {
    public void run() {
        System.out.println("Hello from my new thread!");
    }
}
```

Rularea în paralel a noului thread

În cazul în care s-a folosit mecanismul de implementare a interfeței **Runnable**, atunci pentru a putea crea un thread nou ce va conține logica definită în clasa **Task** se va crea o instanță a clasei **Task** ce va fi furnizată ca parametru constructorului clasei **Thread**. Pentru a rula în paralel noul thread creat cu ajutorul constructorului se va apela metoda **public void start()** a acestuia.

```
public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(new Task());
        t.start();
    }
}
```

În cazul în care s-a extins clasa **Thread** pentru implementarea noului tip de thread se va putea crea un thread nou prin instanțierea directă a clasei **MyThread** și pentru a se porni excecuția în paralel a acestuia se va apela metoda **public void start()** moștenită din cadrul clasei **Thread**.

```
public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Atenție! Există o distincție foarte importantă între metodele de **start()** și **run()** ale clasei **Thread**! Atunci când este apelată metoda **run()** codul prezent în aceasta se va executa secvențial în cadrul thread-ului care a apelat-o. Atunci când este apelată metoda **start()** JVM (Java Virtual Machine) va crea un nou thread ce va executa instrucțiunile prezente în cadrul metodei **run()** în paralel cu thread-ul care a apelat metoda **start()**.

Așteptarea terminării execuției unui thread

Pentru a aștepta terminarea execuției unui thread, Java ne pune la dispoziție metoda **public final void join()** a clasei **Thread**. Trebuie ținut cont de faptul că această metodă poate arunca excepții de tipul **InterruptedException**.

```
public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();

        try {
            t.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Trimiterea de parametrii unui thread și obținerea de rezultate de la acesta

Pentru a trimite parametrii unui thread se va folosi constructorul clasei care inglobeaza logica thread-ului pe care dorim sa il implementam (indiferent de metoda de implementare a acestuia - prin moștenire sau implementeare de interfață). Pentru a obține un rezultat de la un thread care și-a terminat execuția (apelul metodei **join()** a întors) putem folosi fie metode de tip getter care vor întoarce rezultatul fie putem accesa în mod direct câmpul rezultat în cazul în care acest este definit ca public.

```
public class Task extends Thread {

    private int id;
    private int result;

    public Task(int id) {
        this.id = id;
    }

    public void run() {
        result = id * id;
    }

    public int getResult() {
        return result;
    }

}
```

```
public class Main {

    public static void main(String[] args) {
        int NUMBER_OF_THREADS = 4;
        Thread[] t = new Thread[NUMBER_OF_THREADS];

        for (int i = 0; i < NUMBER_OF_THREADS; ++i) {
            t[i] = new Task(id);
            t[i].start();
        }

        for (int i = 0; i < NUMBER_OF_THREADS; ++i) {
            try {
                t[i].join();
                System.out.println("Thread " + i + " computed result " + ((Task)t[i]).getResult() + ".");
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Synchronized

Cuvântul rezervat **synchronized** are rolul de a defini blocuri de cod și metode ce reprezintă secțiuni/regiuni critice.

```
public class MyConcurrentArray<T> {

    private static int numberOfInstances = 0;
    private T[] content;
```

```
    public MyConcurrentArray(int size) {
        if (size > 0) {
            content = new T[size];
        } else {
          throw new RuntimeException("Negative size provided for MyConcurrentArray instantiation.");
        }

        synchronized(MyConcurrentArray.class) {
            ++numberOfInstances;
        }
    }

    //Metodă sincronizată.
    public synchronized T get(int index) {
        if (index < content.length) {
            return content[index];
        }
        throw new IndexOutOfBoundsException(index + " is out of bounds for MyConcurrentArray of size " + content.length);
    }

    public void set(int index, T newT) {
        //Bloc de cod sincronizat ce folosește instanța curentă (this) pe post de zăvor.
        synchronized(this) {
            if (index < content.length) {
                content[index] = newT;
            }
            throw new IndexOutOfBoundsException(index + " is out of bounds for MyConcurrentArray of size " + content.length);
        }
    }

    //Metodă sincronizată statică.
    public static synchronized int getNumberOfInstances(){
        return numberOfInstances;
    }

    public void size() {
        return content.length;
    }
}
```

Pentru a putea discuta comportamentul blocurilor de cod sincronizate vom folosi exemplul de mai sus în care este definită o clasă ce implementează sumar conceptul de vector de dimensiune fixă ce poate fi folosit într-un program multithreading (structură de date thread-safe).

Se observă faptul ca metoda **get** este definită ca fiind **synchronized**. În momentul când un thread va apela aceasta metodă pe o instanță a clasei **MyConcurrentArray** va trebui mai întâi să obțină monitorul [https://en.wikipedia.org/wiki/Monitor_%28synchronization%29] asociat acesteia (obiectului) pentru a putea executa corpul metodei. Dacă monitorul nu este deținut de nici un alt thread, atunci thread-ul apelant va putea să execute corpul de instrucțiuni al metodei, altfel se va bloca (va aștepta) până când acesta devine disponibil. La încheierea execuției corpului metodei acesta restituie accesul la monitor.

În cazul metodei **set** este prezent un bloc de instrucțiuni sincronizat. Acesta va folosi monitorul obiectului desemnat între paranteze pentru a oferi thread-ului curent acces excluziv în cadrul regiunii critice. În cazul nostru se folosește instanța curentă a obiectului (**this**). Mecanismul de intrare și ieșire în/din secțiunea critică este același ca cel prezentat mai sus pentru cazul metodelor sincronizate.

În cazul apelării metodelor sincronizate statice se va încerca obținerea monitorului asociat clasei pentru a executa codul acestora. Acest lucru se întâmplă deoarce o metodă statică aparține clasei, ea nu aparține nici unei instanțe a clasei. De aceea când este necesar accesul exclusiv la un câmp static dintr-o clasă se va folosi clasa în antetul blocului synchronized (MyClass.class, după cum este prezentat și în constructorul din exemplu).

Atenție! Metodele și blocurile de cod **synchronized** din Java sunt **reentrante**. Dacă un thread a obținut monitorul unui obiect, atunci el va putea intra în orice alt bloc și metodă sincronizate ce sunt asociate cu acel obiect (implicit cu acel monitor). Acest comportament nu este activat în mod implicit pentru **pthread_mutex_t** definit în C (se poate obține prin specificarea de atribute la creare, **Hint**: PTHREAD_MUTEX_RECURSIVE).

CyclicBarrier

Bariera ciclică reprezintă un mecanism de (re)sincronizare al mai multor thread-uri care are scopul de a bloca un număr specificat de thread-uri și de a le lăsa sa își continua execuția într-un mod sincron după ce toate au apelat metoda acesteia de resincronizare. În Java aceast mecanism este reprezentat de clasa

CyclicBarrier. La instanțierea acesteia se va specifica numărul de thread-uri pe care aceasta le va resincroniza. Acest mecanism de sincronizare al thread-urilor este util în cazul algoritmilor iterativi ce sunt rulați în paralel și au nevoie de o etapă de resincronizare a firelor de execuția înainte de a trece la noua iterație de calcul. Trebuie ținut cont de faptul că apelul metodei **await()** pe bariera ciclică poate arunca o excepție de tipul **BrokenBarrierException** sau **InterruptedException**.

```
public class Task extends Thread {

    private int id;

    public Task(int id) {
        this.id = id;
    }

    public void run() {
        iterationCounter = 0;
        while (!solved) {
            executeAlgorithmStep();

            try {
                //Resincronizarea thread-urilor pentru urmatorul pas al algoritmului.
                Main.barrier.await();
                ++iterationCounter;
            } catch (BrokenBarrierException | InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

}
```

```
public class Main {

    public static CyclicBarrier barrier;

    public static void main(String[] args) {
        int NUMBER_OF_THREADS = 4;
        barrier = new CyclicBarrier(NUMBER_OF_THREADS);
        Task[] t = new Task[NUMBER_OF_THREADS];

        for (int i = 0; i < NUMBER_OF_THREADS; ++i) {
            t[i] = new Task(id);
            t[i].start();
        }

        for (int i = 0; i < NUMBER_OF_THREADS; ++i) {
            try {
                t[i].join();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

CheatSheet CyclicBarrier

Volatile

Cuvântul rezervat **volatile** asociat unei variabile specifică faptul că acea variabilă nu va intra în procesul de cacheing (nici la nivel de regiștrii, nici la nivel de cache L1, L2, L3) și fiecare scriere și citire asociată acesteia se va realiza direct lucrând cu memoria RAM. Acest lucru este util în prevenția citirii unei date vechi din cache-ul sau registrul asociat core-ului pe care rulează un thread în momentul în care variabila a căpătat o valoare nouă pe alt thread ce lucrează pe alt core.

```
public static volatile int counter = 0;
```

Este important de menționat faptul că variabilele volatile cresc timpul de execuție al programului, deoarece fiecare modificare adusă lor trebuie propagată și celorlalte thread-uri, în timp ce variabilele normale pot fi cache-uite la un moment dat chiar până la nivelul regiștrilor procesorului (în cazul unui contor de buclă).

Variabile atomice

În cadrul laboratorului 2 am observat cum operația de incrementare (exemplul cu instrucțiunea a+=2) nu este o operație atomică (operație ce nu poate fi divizată atunci când este executată de către un thread). Java oferă suport pentru o serie de tipuri de date (tipuri atomice) ce au asociate operații atomice [1] [https://docs.oracle.com/javase/8/docs/api/?java/util/concurrent/atomic/package-summary.html], [2] [https://docs.oracle.com/javase/tutorial/essential/concurrency/atomicvars.html]. Acestea sunt utile pentru a fi folosite pe post de contori sau acumulatori fără a mai folosi mecanisme de sincronizate.

Exerciții

1. Creați un program care să lanseze un număr de thread-uri egal cu numărul de core-uri de care dispune calculatorul vostru. Fiecare thread trebuie să afișeze la consolă un text de tipul ”Hello from thread #id”. **[2P]**

2. Rezolvați bug-urile prezente în arhiva de laborator. Folosiți-vă de **hint-uri** din surse. **[3P]**

3. Paralelizați dublarea elementelor unui vector. **[2P]**

4. Paralelizați algoritmul Floyd-Warshall. **[3P]**

apd/laboratoare/04.txt · Last modified: 2021/01/11 21:10 by florin.mihalache

Laboratorul 5 - Probleme de sincronizare

wait, notify & notifyAll

Sa presupunem ca avem 2 thread-uri: thread-ul 0 care calculeaza o valoare si o plaseaza in variabila `a` si thread-ul 1 care atunci cand variabila `a` se actualizeaza, o afiseaza in interfata grafica a unui program. O posibila implementare pentru functia thread-ului 0, in pseudocod, ar fi:

```
function_thread_0:
  noua_valoare = calculeaza_valoare_a()
  mutex_a.lock()
      a = noua_valoare
  mutex_a.unlock()
```

O posibila implementare pentru functia thread-ului 1, in pseudocod, ar fi:

```
function_thread_1:
  while(a nu s-a actualizat):
      mutex_a.lock()
      if (a s-a actualizat):
          afiseaza(a)
          a = -1 //  Resetam valoarea lui a.
      mutex_a.unlock()
```

Observam o problema cu aceasta abordare: thread-ul 1 va cicla in bucla while de mai multe ori, chiar daca thread-ul 0 nu-l actualizeaza pe `a`, ocupand inutil timp pe procesor. Aceasta abordare poarta numele de **busy waiting**. Desi **busy waiting** nu este intotdeauna o idee rea, in cazul de fata, prespunuem ca pentru a calcula noua valoarea a lui a, thread-ului 0 ii ia un timp indelungat. Astfel, exista o solutie mai buna oferita de primitevele Java **wait()**, **notify()** și **notifyAll()**.

wait() - forțează thread-ul curent să intre în așteptare până când alt thread apelează **notify()** sau **notifyAll()** pe același obiect. Pentru ca aceasta să se întâmple, thread-ul curent trebuie să dețină monitorul obiectului respectiv. Deținerea monitorului se poate îmtâmpla în următoarele situații:

- s-a executat o metodă synchronized pe obiectul respectiv
- s-a executat un bloc synchronized pe obiectul respectiv
- s-a executat o metodă synchronized statică pe clasa din care face parte obiectul respectiv

Evitați folosirea **wait()** asupra obiectelor cu vizibilitate globală (statice) sau asupra String-urilor cu valoare constanta (de ex: `String myMonitorObject = "";`), deoarece JVM optimizează intern accesul la astfel de variabile, având doar o singură instanță în cadrul programului.

notify() - alege aleatoriu un thread care așteaptă (a apelat **wait()**) monitorul obiectului respectiv și trece-l din starea de *waiting* în starea de *running*

notifyAll() - trece toate thread-urile care așteaptă (au apelat **wait()**) monitorul obiectului respectiv și trece-le din starea de *waiting* în starea de *running*

CheatSheet Wait-Notify

Semafoare

Un semafor poate fi văzut ca un lock ce permite mai multor thread-uri să coexiste într-o anumită regiunie critică la un moment dat. Semaforul folosește un contor care determină câte thread-uri mai pot intra. Odată ajuns la semafor, un thread este lăsat să intre doar dacă numărul de thread-uri aflate în zona critică este mai mic decât numărul maxim de thread-uri setat la crearea semaforului.

Pe un semafor se pot realiza următoarele operații:

- **acquire** - se încearcă trecerea de semafor; dacă numărul de thread-uri aflate în zona critică este mai mic decât numărul maxim de thread-uri acceptate, thread-ul poate intra

- **release** - un thread apelează metoda pentru a anunța faptul că și-a terminat treaba în zona critică, permițând astfel incrementarea numărului de thread-uri car pot intra în zona critică

```
class Something {

    static int resource = 0;
}

public class MySemaphore extends Thread {

    private int id;
    private Semaphore sem;

    public MySemaphore(int id, Semaphore sem) {
        this.id = id;
        this.sem = sem;
    }

    public static void main(String args[]) {
        Semaphore sem = new Semaphore(1);
        MySemaphore mt1 = new MySemaphore(0, sem);
        MySemaphore mt2 = new MySemaphore(1, sem);

        mt1.start();
        mt2.start();

        try {
            mt1.join();
            mt2.join();
        } catch (InterruptedException ex) {
            Logger.getLogger(MySemaphore.class.getName()).log(Level.SEVERE, null, ex);
        }

        System.out.println("count: " + Something.resource);
    }

    @Override
    public void run() {
        switch (this.id) {
            case 0:
                System.out.println("Starting thread " + id);
                try {
                    System.out.println("Thread " + id + " is waiting for a permit.");
                    sem.acquire();
                    System.out.println("Thread " + id + " gets a permit.");

                    for (int i = 0; i < 5; i++) {
                        Something.resource++;
                        System.out.println("Thread " + id + ": " + Something.resource);
                        Thread.sleep(10);
                    }
                } catch (InterruptedException exc) {
                    System.out.println(exc);
                }

                System.out.println("Thread " + id + " releases the permit.");
                sem.release();
                break;
            case 1:
                System.out.println("Starting thread " + id);
                try {
                    System.out.println("Thread " + id + " is waiting for a permit.");
                    sem.acquire();
                    System.out.println("Thread " + id + " gets a permit.");

                    for (int i = 0; i < 5; i++) {
                        Something.resource--;
                        System.out.println("Thread " + id + ": " + Something.resource);
                        Thread.sleep(10);
                    }
                } catch (InterruptedException exc) {
                    System.out.println(exc);
                }

                // Release the permit.
                System.out.println("Thread " + id + " releases the permit.");
                sem.release();
                break;
```

```
        }
    }

    public int getThreadId() {
        return this.id;
    }

}
```

În codul de mai sus se creează un semafor ce acceptă un singur thread în zona critică (observați forma constructorului Semaphore). Practic, acest semafor este similar cu un lock. Semaforul se creează in main si se trimite ca parametru în constructorul thread-urilor. În cadrul metodei run, thread-urile încearcă să apeleze **acquire**, însă doar unul dintre ele va reuși să intre în zona de după semafor. Ulterior, după ce se realizează operațiile din zona critică, thread-ul respectiv anunță faptul că a ieșit prin intermediul unui apel de **release**. Observați faptul că, în funcție de thread-ul care reușește să intre în zona respectivă, contorul poate scădea până la -5, urmând să revină la 0 apoi, sau să fie crescut până la 5, urmând să ajungă la 0 apoi.

CheatSheet Semafoare

Probleme clasice de sincronizare

Producător - Consumator

Problema se referă la două thread-uri: producător și consumator. Producătorul inserează date într-un buffer, iar consumatorul extrage date din acel buffer. Buffer-ul are o dimensiune prestabilită, astfel că:

- producătorul nu poate insera date dacă buffer-ul este plin
- consumatorul nu poate extrage date dacă buffer-ul este gol
- producătorul și consumatorul nu pot acționa simultan asupra buffer-ului

O implementare corectă a problemei presupune asigurarea faptului că nu vor exista situații de deadlock, adică situații în care cele două thread-uri așteaptă unul după celălalt, neexitând posibilitatea de a se debloca.

CheatSheet Producator-Consumator

Problema filozofilor

Problema se referă la mai mulți filozofi (thread-uri) așezați la o masă circulară. Pe masă se află 5 farfurii și 5 tacâmuri, astfel încât fiecare filozof are un tacâm în stânga și unul în dreapta lui. În timp ce stau la masă, filozofii pot face două acțiuni: mănâncă sau se gândesc. Pentru a mânca, un filozof are nevoie de două tacâmuri (pe care le poate folosi doar dacă nu sunt luate de către vecinii săi). Rezolvarea trebuie să aibă în vedere dezvoltarea unui algoritm prin care să nu se ajungă la un deadlock (situația în care fiecare filozof ține câte un tacâm în mână și așteaptă ca vecinul să elibereze celălalt tacâm de care are nevoie).

CheatSheet Problema filozofilor

Căutarea binară paralelă

Algoritmul de căutare binară paralelă se bazează pe împărţirea secvenţei în care se face căutarea în N+1 subsecvenţe de aceeaşi lungime. La fiecare pas, fiecare din cele N procesoare inspectează o valoare s aflată în poziţia dintre două secvenţe adiacente. Dacă $x < s$, atunci toate elementele aflate la dreapta lui s sunt ignorate în pasul următor. Similar, dacă $x > s$, sunt ignorate elementele mai mici ca s. Următorul subinterval de căutare se obţine făcand intersecţia subintervalelor păstrate de diferitele procesoare. Deoarece fiecare etapă se aplică unei secvenţe de dimensiune $1/(N+1)$ din secvenţa precedentă, sunt necesare $O(\log_{N+1}(n+1))$ etape. Pentru a determina subintervalul care rămane în fiecare etapă, fiecare proces *Pi*, $1 \Leftarrow i \Leftarrow N$ utilizează o variabilă c[i] care ia una din valorile stg sau dr, după cum trebuie reţinută partea din stanga, respectiv cea din dreapta. În plus, se folosesc $c[0] =$

`dr` şi `c[N+1] = stg`. Iniţial, limitele subsecvenţei de căutare sunt `q = 1` şi `r = n`. De asemenea, `g` are iniţial valoarea numărului maxim de etape: `g = sup(log(n+1)/log(N+1))` şi scade cu o unitate la fiecare iteraţie. Algoritmul calculează valorile c[i] şi găseşte i pentru care c[i-1] <> c[i]. Pe baza lui se determină următorul subinterval [q, r] de căutare:

```
q = q + (i-1) * (N+1) * g - 1 + 1
r = q + i * (N+1) * g - 1 - 1
```

Un singur procesor calculează noile valori pentru q şi r, celelalte având acces la aceste valori în timp constant.

CheatSheet Cautare binara paralela

Exerciții

Schelet laborator 5

1. Pornind de la scheletul **oneProducerOneConsumer** furnizat, implementați algoritmul Producer-Consumer pentru un buffer de dimensiune 1. **[2P]**
2. Modificați algoritmul Producer-Consumer astfel încât să accepte mai mulți producători și mai mulți consumatori. De asemenea, modificați buffer-ul astfel încât să fie de dimensiune > 1. Porniți de la scheletul **multipleProducersMultipleConsumersNBuffer**. **[3P]**
3. Rezolvați problema din algoritmul filozofilor (pachetul **philosophersProblem**) și explicați-o. **[2P]**
4. Implementați căutarea binară paralelă. **[3P]**

Exercițiile din cadrul acestui laborator trebuie rezolvate folosind mecanisme de sincronizare precum primitivele wait/notify/notifyAll sau semafoare. Nu se accepta soluții ce folosesc obiecte concurente.

apd/laboratoare/05.txt · Last modified: 2021/01/11 21:10 by florin.mihalache

Laboratorul 6 - Structuri și operații atomice în Java

Responsabili: Radu Ciobanu, Carina Deaconu, Florin Mihalache

Semaphore (continuare)

Laboratorul trecut, am văzut cum semaforul reprezintă o generalizare a unui mutex. Un semafor iniţializat cu 1 poate fi folosit drept lock, pentru că doar un thread are acces la zona critică la un moment de timp. Totuși, un semafor poate avea întrebuinţări mult mai complexe, deoarece poate lua diverse valori, atât negative, cât şi pozitive.

Când un semafor este iniţializat cu valoarea pozitivă *x*, vă puteţi gândi că *x* thread-uri au voie să intre în secţiunea critică. Pe măsură ce un thread face **acquire()**, *x* este decrementat. Când se ajunge la 0, alte posibile thread-uri care vor să acceseze regiunea critică vor trebui să aştepte până când valoarea semaforului creşte la o valoare pozitivă (adică până când iese câte un thread din cele aflate în regiunea critică).

Complementar, când un semafor este iniţializat cu o valoare negativă cum ar fi -1, se aşteaptă ca cel puţin două thread-uri să facă întâi **release()** (pentru ca valoarea semaforului să crească de la -1 la 1), înainte ca o regiune critică să poată fi accesată (adică un alt thread să poată face **acquire()**). Vă puteţi imagina că un al treilea thread aşteaptă „la semafor” ca alte două thread-uri să îi dea un semnal prin apelul **release()** când şi-au „terminat treaba”. Puteți observa un exemplu în pseudocodul de mai jos.

```
Semaphore sem = new Semaphore(-1);
```

T0	T1	T2

`// așteaptă la semafor`
`// după celelalte 2 thread-uri`
`sem.acquire();`
 	
`// sem = 1, deci poate trece`
`System.out.println(`
`"Am trecut de semafor!");`
	`do_work1();`
	`sem.release();`
	`// sem = -1 + 1 = 0`
		`do_work2();`
		`sem.release();`
		`// sem = 0 + 1 = 1`

AtomicInteger

Clasa AtomicInteger [https://docs.oracle.com/en/java/javase/15/docs/api/java.base/java/util/concurrent/atomic/AtomicInteger.html] poate fi gândită ca o implementare atomică a unui Integer, pentru că reține o valoare de tip întreg și oferă operații atomice pe ea. Poate fi iniţializat cu o valoare sau nu, valoarea default fiind 0.

Metodele cele mai folosite sunt **get()** (întoarce valoarea de tip întreg), **set(x)** (îi setează valoarea la *x*), **compareAndSet(x, y)** (compară întregul cu *x*, iar, dacă cele două valori sunt egale, setează valoarea la *y*), **addAndGet(delta)** / **getAndAdd(delta)** (incrementează valoarea curentă cu *delta* și returnează valoarea nouă, respectiv pe cea veche). Un exemplu de utilizare a acestor metode poate fi observat mai jos.

```
AtomicInteger nr = new AtomicInteger(4);
System.out.println(nr.get()); // 4

if (nr.compareAndSet(4, 7))
    System.out.println(nr.get()); // nr devine 7

System.out.println(nr.addAndGet(2)); // se aduna 2, deci devine 9

nr.set(1);
System.out.println(nr.get()); // 1
```

Folosirea AtomicInteger este mai avantajoasă ca performanță (mai precis din punctul de vedere al timpilor de execuție) decât folosirea de locks (blocuri synchronized), deoarece la locks avem overhead creat de acquire() (lock()) și de release() (unlock()), așa cum poate fi observat mai jos.

```
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static final int size = 1000000;
    public static final int noThreads = 8;
    public static int[] arr = new int[size];
    public static final Object lock = new Object();

    public static void main(String[] args) {
        for (int i = 0; i < size; i++) {
            arr[i] = i;
        }

        Thread[] properThreads = new Thread[noThreads];
        Thread[] atomicThreads = new Thread[noThreads];

        long startTime = System.nanoTime();
        for (int i = 0; i < properThreads.length; i++) {
            properThreads[i] = new ProperThread(i);
            properThreads[i].start();
        }

        for (Thread properThread : properThreads) {
            try {
                properThread.join();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        long stopTime = System.nanoTime();
        System.out.println("With locks = " + (stopTime - startTime));

        startTime = System.nanoTime();
        for (int i = 0; i < atomicThreads.length; i++) {
            atomicThreads[i] = new AtomicThread(i);
            atomicThreads[i].start();
        }

        for (Thread atomicThread : atomicThreads) {
            try {
                atomicThread.join();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        stopTime = System.nanoTime();
        System.out.println("With AtomicInteger = " + (stopTime - startTime));

        System.out.println("Atomic sum = " + AtomicThread.sum.get());
        System.out.println("Locking sum = " + ProperThread.sum);
    }
}

class AtomicThread extends Thread {
    public static AtomicInteger sum = new AtomicInteger(0);
    private final int id;

    public AtomicThread(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        int start = id * (int) Math.ceil((double) Main.size / Main.noThreads);
        int end = Math.min(Main.size, (id + 1) * (int) Math.ceil((double) Main.size / Main.noThreads));
        for (int i = start; i < end; i++) {
            sum.getAndAdd(Main.arr[i]);
        }
    }
}

class ProperThread extends Thread {
    public static int sum = 0;
    private final int id;
```

```
    public ProperThread(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        int start = id * (int) Math.ceil((double) Main.size / Main.noThreads);
        int end = Math.min(Main.size, (id + 1) * (int) Math.ceil((double) Main.size / Main.noThreads));
        for (int i = start; i < end; i++) {
            synchronized (Main.lock) {
                sum += Main.arr[i];
            }
        }
    }
}
```

Java are astfel de implementări atomice precum toate tipurile primitive și nu numai (AtomicBoolean, AtomicLong, AtomicIntegerArray, etc.). Mai multe informații puteți găsi în documentația Java [https://docs.oracle.com/en/java/javase/15/docs/api/java.base/java/util/concurrent/atomic/package-summary.html].

ConcurrentHashMap

ConcurrentHashMap [https://docs.oracle.com/en/java/javase/15/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html] este o clasă care are aceleaşi funcţionalităţi ca un HashTable (nu HashMap, pentru că nu permite folosi *null* drept cheie sau valoare).

Dacă am folosi un HashMap clasic şi am adăuga un lock pentru accesul la structură, nu ar fi foarte eficient, pentru că doar un singur thread ar putea să o acceseze la un moment de timp, lucru care nu este necesar dacă, de exemplu, ele vor să modifice elemente diferite. Dacă, în schimb, am folosi câte un lock pentru fiecare element, ne-ar trebui destul de multe şi ne-ar îngreuna lucrul.

Într-un ConcurrentHashMap, operaţiile de tip **get** şi **put** blochează accesul la un anumit element (au mutexuri în implementarea lor), dar **NU** blochează accesul la întreaga structură de date. Cu alte cuvinte, două sau mai multe thread-uri o pot accesa în acelaşi timp, şi operaţiile de tip *get* se pot suprapune cu cele de tip update (**put**), dar implementarea se asigură că **get**-urile se execută la final, după ce s-au executat operațiile de update.

Metodele cele mai folosite sunt:

- **get(key)** - returnează valoarea asociată cu o cheie
- **contains(val)** - verifică dacă o valoare există în map
- **containsKey(key)** - verifică dacă o cheie există în map
- **put(key, val)** - adaugă o valoare la cheia specificată
- **putIfAbsent(key, val)** - adaugă o asociere doar dacă nu există deja o valoare pentru cheia respectivă; este întoarsă valoarea veche, dacă există, altfel e întors *null*
- **remove(key)** - şterge o asociere cheie-valoare
- **replace(key, val)** - modifică valoarea asociată cu o cheie.

Puteți observa mai jos un exemplu de utilizare.

```
ConcurrentHashMap<Integer, String> map = new ConcurrentHashMap<Integer, String>();

map.put(0, "Zero");
map.put(1, "One");

System.out.println(map.get(1)); // One

String val = map.putIfAbsent(1, "Unu"); // nu se modifică nimic, val va fi "One"

val = map.putIfAbsent(2, "Two"); // se adaugă maparea 2-"Two", val va fi null
```

BlockingQueue

Interfaţa BlockingQueue [https://docs.oracle.com/en/java/javase/15/docs/api/java.base/java/util/concurrent/BlockingQueue.html] extinde interfaţa Queue, clasele care o implementează fiind potrivite pentru programe paralele, în care pot exista probleme de concurenţă. Dintre acestea, cele mai relevante sunt ArrayBlockingQueue, LinkedBlockingQueue şi PriorityBlockingQueue. Toate simulează o structură de tip FIFO (coadă), cu operaţii thread-safe.

Diferenţa este la implementarea din spate: ArrayBlockingQueue e bazată pe un array, LinkedBlockingQueue pe o listă înlănţuită, iar pentru PriorityBlockingQueue poate fi specificat un comparator la iniţializare, păstrând o ordine în care să fie elementele. Niciuna dintre clase nu acceptă elemente de tip *null* şi li se poate specifica o capacitate maximă la iniţializare (pentru ArrayBlockingQueue este obligatoriu acest lucru).

Operaţiile sunt pe principiul FIFO (se inserează la final şi se extrage de la începutul cozii), așa cum se poate observa în tabelul de mai jos.

Operație	**Aruncă excepție**	**Valoare specială**	**Se blochează**	**Dă timeout**
Inserare	add(e)	offer(e)	put(e)	offer(e, time, unit)
Ștergere	remove()	poll()	take()	poll(time, unit)
Examinare	element()	peek()	-	-

Deşi sunt mai multe metode pentru aceeaşi operaţie asemănătoare, comportamentul lor este diferit. Cea mai importantă diferenţă o constituie proprietatea de a fi sau nu blocantă. De exemplu, **add(obj)** adaugă un element doar dacă este loc în coadă (adică dacă nu s-a atins capacitatea maximă) şi returnează *true* dacă operaţia a reuşit, sau IllegalStateException în caz contrar. În schimb, **put(obj)** se blochează dacă nu este spaţiu suficient în coadă şi aşteaptă până poate pune elementul cu succes.

Collections.synchronized

Dacă dorim să lucrăm cu colecții clasice, precum ArrayList, și să avem operațiile sincronizate, putem să le instanțiem folosind:

- Collections.synchronizedList (pentru liste):

```
List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());
```

- Collections.synchronizedMap (pentru dicționare):

```
Map<Integer, String> syncMap = Collections.synchronizedMap(new HashMap<>());
```

Arbori concurenți

Atunci când lucrăm cu structuri mai complexe, trebuie să avem grijă la sincronizarea accesului concurent la ele, dar și la eficiența finală a implementării. Ca să vedem un exemplu concret, vom presupune că avem un arbore binar. Când se dorește inserarea unui element într-un astfel de arbore, se verifică mai întâi dacă nodul unde inserăm are copil în partea stângă. Dacă are, atunci nodul copil va fi inserat în stânga, altfel va fi inserat în partea dreaptă:

```
if (node.left == null)
    node.left = child;
else
    node.right = child;
```

Când mai multe thread-uri acționează asupra unui arbore binar, pot apărea probleme de concurență la inserare. Un astfel de scenariu ar putea avea loc atunci când două thread-uri încearcă să insereze simultan câte o valoare diferită în același nod din arbore:

T0	**T1**
Verifică dacă există copil în stânga	Verifică dacă există copil în stânga
Condiția este adevărată, trece în prima ramură a if-ului	Condiția este adevărată, trece în prima ramură a if-ului
Inserează nodul în stânga	

	Inserează nodul în stânga → se suprascrie valoarea scrisă de **T0**

În acest caz, corect ar fi ca unul din thread-uri să insereze în stânga și celălalt să insereze în dreapta. O soluție este să folosim un lock pe operația de inserare unui nod. Dacă folosim un lock global (pentru tot arborele), două thread-uri nu pot insera noduri copii în același timp pentru două noduri părinte diferite. Din acest motiv, mai eficient ar fi să existe câte un lock pentru fiecare nod.

Exerciții

1. Pornind de la arhiva de laborator, rezolvați problema de sincronizare din pachetul *multipleProducersMultipleConsumers*, folosind ArrayBlockingQueue. **(2p)**
2. Rezolvați problema de sincronizare din pachetul *synchronizationProblem*, folosind un AtomicInteger. **(2p)**
3. Rezolvați problema de sincronizare din pachetul *bugConcurrentHashMap*, folosind metode din clasa ConcurrentHashMap. **(2p)**
4. Paralelizați programul din pachetul *synchronizedSortedList*, unde:
    a. trei thread-uri citesc numere din trei fișiere (câte un thread citește câte un fișier) și adaugă numerele din fișiere într-o listă partajată
    b. al patrulea thread sortează lista cu numerele
    c. lista poate fi sortată **doar** după ce citirea a fost efectuată de către toate cele trei thread-uri care citesc din fișier
    d. sincronizarea între thread-uri trebuie realizată folosind un semafor. **(2p)**
5. Paralelizați programul din pachetul *parallelTree*, unde:
    a. două thread-uri construiesc un arbore binar în paralel, citind ID-urile și părintele nodurilor noi din câte un fișier, și apoi executând operația de inserare
    b. al treilea verifică dacă arborele a fost construit corect
    c. verificarea arborelui nu se poate face decât după finalizarea alcătuirii sale
    d. sincronizarea între thread-uri trebuie realizată folosind o barieră
    e. la inserția în arbore, sincronizarea nu trebuie făcută global (pe tot arborele), ci pentru fiecare nod din arbore. **(2p)**

apd/laboratoare/06.txt · Last modified: 2021/01/11 21:09 by florin.mihalache

Laboratorul 7 - Modelul Replicated Workers

Responsabili: Radu Ciobanu, Carina Deaconu, Florin Mihalache

În programarea paralelă, modelul **Replicated Workers** (sau **Thread Pool**) este un design pattern folosit pentru obținerea de concurență în execuția unui program. În acest model, există două componente principale: un **pool** de sarcini de executat (task-uri), reprezentat în general ca o coadă, și un grup de **workeri** (care sunt în general thread-uri). Fiecare worker din modelul replicated workers are următorul comportament:

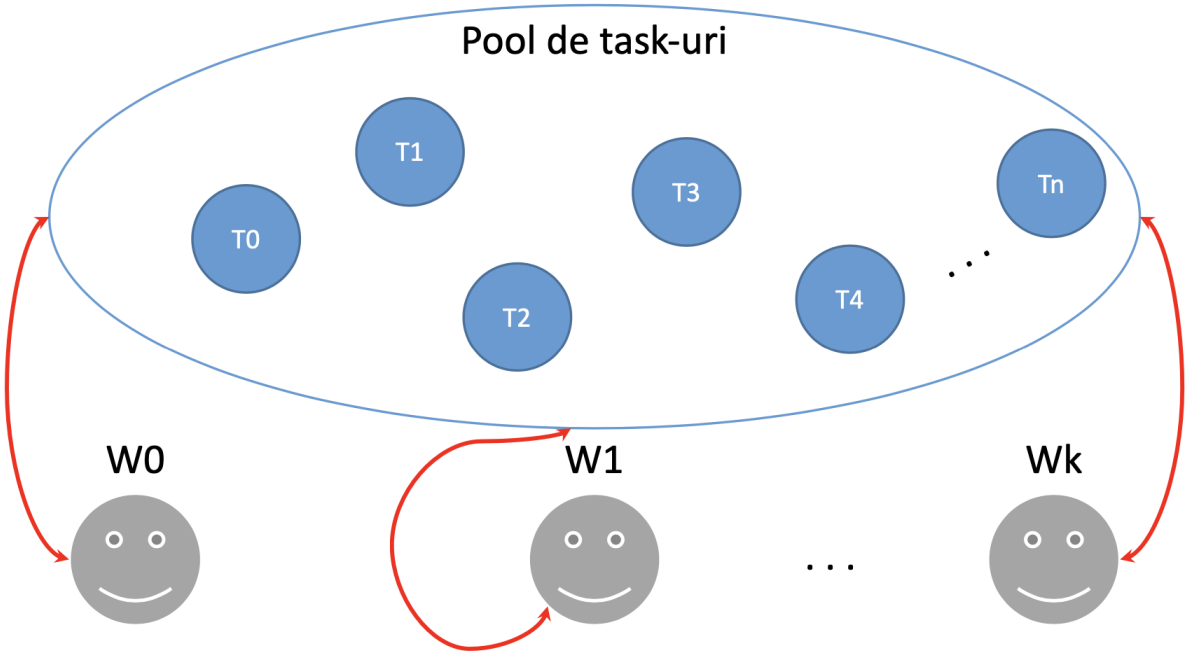
```
while (1)
    ia un task din pool
    execută task-ul
    adaugă zero sau mai multe task-uri rezultate în pool
```

Așa cum se poate observa în pseudocodul de mai sus, un worker ia un task din pool și îl execută. Ca urmare a execuției unui task, este posibil să se genereze task-uri adiționale, pe care worker-ul le adaugă în pool, după care repetă pașii precedenți.

Modelul Replicated Workers este util atunci când:

- nu se cunoaște în avans dimensiunea problemei, adică numărul de pași care se vor executa
- există multe task-uri de dimensiuni mici care trebuie executate și este mai eficient să se refolosească aceleași thread-uri decât să se creeze unele noi la fiecare pas
- există posibilitatea ca thread-urile să fie dezechilibrate (de exemplu, unele core-uri sunt mai ocupate, sau chiar diferite între ele, și o împărțire egală a sarcinilor de calcul ar duce la întârzieri în program).

O reprezentare grafică a modelului replicated workers poate fi observată în figura de mai jos.



ExecutorService

ExecutorService [https://docs.oracle.com/javase/7/docs/api/java/util/concurrent/ExecutorService.html] este o interfață în Java ce permite executarea de task-uri asincrone, în background, în mod concurent, pe baza modelului Replicated Workers. De exemplu, putem avea nevoie să trimitem un număr de cereri, dar ar fi ineficient să le trimitem pe rând, secvențial, așteptând după fiecare să se termine. Soluția ar fi să lucrăm asincron, adică să trimitem o cerere, să nu așteptăm „după ea” (deci să se trimită în background) și să folosim thread-uri pentru a împărți numărul de cereri și a trimite mai multe deodată (concurent). Acesta ar fi cazul când nu știm în avans dimensiunea problemei pe care o rezolvăm, și fie

avem nevoie de toate soluțiile problemei (pentru că trebuie să trimitem toate cererile), fie nu vrem să găsim toate soluțiile, ci minim una.

În continuare, se prezintă un exemplu de program care folosește ExecutorService pentru a afișa toate fișierele dintr-un director (inclusiv din fiecare subdirector în parte). Pentru că nu știm în avans câte fișiere și directoare va trebui să analizăm, putem spune că nu cunoaștem dimensiunea problemei.

```
import java.io.File;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Example1 {
    public static void main(String[] args) {
        AtomicInteger inQueue = new AtomicInteger(0);
        ExecutorService tpe = Executors.newFixedThreadPool(4);

        inQueue.incrementAndGet();
        tpe.submit(new MyRunnable("files", tpe, inQueue));
    }
}

class MyRunnable implements Runnable {
    String path;
    ExecutorService tpe;
    AtomicInteger inQueue;

    public MyRunnable(String path, ExecutorService tpe, AtomicInteger inQueue) {
        this.path = path;
        this.tpe = tpe;
        this.inQueue = inQueue;
    }

    @Override
    public void run() {
        File file = new File(path);
        if (file.isFile()) {
            System.out.println(file.getPath());
        } else if (file.isDirectory()) {
            File[] files = file.listFiles();

            if (files != null) {
                for (File f : files) {
                    inQueue.incrementAndGet();
                    tpe.submit(new MyRunnable(f.getPath(), tpe, inQueue));
                }
            }
        }

        int left = inQueue.decrementAndGet();
        if (left == 0) {
            tpe.shutdown();
        }
    }
}
```

Așa cum se poate observa în codul de mai sus, instanțierea unui ExecutorService cu 4 workeri se realizează în thread-ul principal. Prin metoda *submit()*, se introduce un task nou în pool, corespunzător directorului părinte. Metoda *submit()* poate primi un obiect ce implementează interfața Runnable și conține metoda *run()* sau unul care implementează interfața Callable și conține metoda *call()*. Diferența dintre ele este că *run()* returnează void, iar *call()* returnează un obiect și poate arunca și o excepție. Cu alte cuvinte, Callable este potrivit când ne interesează rezultatul final și eventualele erori la executarea task-ului. În cadrul laboratorului, vom lucra cu interfața Runnable.

În exemplul de mai sus, clasa noastră ce reprezintă un task se numește MyRunnable. În metoda run, se verifică dacă avem de-a face cu un fișier sau cu un director. Dacă lucrăm pe un fișier, doar se printează numele acestuia. Dacă lucrăm cu un director, înseamnă că trebuie create task-uri noi, pentru fiecare fișier sau subdirector în parte conținut de directorul părinte. Astfel, se creează câte o instanță de MyRunnable pentru fiecare din ele și se adaugă în pool-ul de ExecutorService.

În cazul exemplului prezentat în acest laborator, ne trebuie toate soluțille problemei (toate fișierele și subdirectoarele), dar nu știm dinainte dimensiunea problemei, așa că o metodă prin care ne putem opri este să ținem evidența task-urilor din pool și să finalizăm execuția atunci când acesta se golește. Altfel, programul nostru va rula la infinit. Modul prin care se finalizează execuția unui ExecutorService poate fi

observat în codul de mai sus. Mai precis, se folosește metoda *shutdown()* pentru a opri ExecutorService-ul din a primi task-uri noi. Dacă ne-ar fi interesat o singură soluție a problemei noastre (și nu toate, ca în exemplul de mai sus), am fi putut opri execuția în momentul în care ajungeam la acea soluție.

ForkJoinPool

O altă metodă de a implementa modelul Replicated Workers în Java este framework-ul Fork/Join, care folosește o abordare *divide et impera*, ceea ce înseamnă că întâi are locul procesul de împărțire (Fork) recursivă a task-ului inițial în subtask-uri independente mai mici, până când acestea sunt suficient de mici cât să poată fi executate asincron. După aceea, urmează partea de colectare recursivă a rezultatelor (Join) într-un singur rezultat (în cazul unui task care nu returnează un rezultat propriu-zis, pur și simplu se așteaptă ca toate subtask-urile să se termine de executat).

Pentru o execuție paralelă eficientă, framework-ul Fork/Join folosește un pool de thread-uri numit ForkJoinPool [https://docs.oracle.com/javase/7/docs/api/java/util/concurrent/ForkJoinPool.html], care gestionează thread-uri worker de tipul ForkJoinWorkerThread. ForkJoinPool este o implementare de ExecutorService care gestionează thread-uri de tip worker, care pot executa câte un singur task la un moment dat de timp. În implementarea sa, fiecare thread are propria sa coadă de task-uri, dar, atunci când aceasta se golește, worker-ul poate să „fure” task-uri din coada altui worker sau din pool-ul global de task-uri.

Un task care va fi executat folosind framework-ul Fork/Join poate fi definit în două moduri, în funcție de valoarea de retur. Dacă task-ul nu trebuie să returneze nimic, un task se definește prin moștenirea clasei RecursiveAction. Dacă task-ul returnează o valoare de tip *V*, atunci trebuie să se moștenească clasa RecursiveTask<V> pentru definirea unui task. Ambele clase părinte au o metodă numită *compute()* în care se definește logica unui task (echivalentul metodei *run()* de la ExecutorService).

Pentru a adăuga task-uri care trebuie executate de workeri, se poate folosi metoda *invoke()*, care creează un task și îi așteaptă rezultatul, sau combinația de metode *fork()* și *join()*. Pentru al doilea caz, metoda *fork()* trimite un task în pool, dar nu îl marchează spre execuție, acest lucru făcându-se explicit prin intermediul metodei *join()*. Atât *invoke()*, cât și *join()*, returnează o valoare de tipul *V* pentru un task de tip RecursiveTask.

Pentru că, în cazul unui task care nu returnează nimic, se poate folosi RecursiveTask<V>, prezentăm mai jos un exemplu complet de implementare care folosește RecursiveTask, pentru aceeași problemă ca în exemplul de ExecutorService.

```
import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;

public class Example2 {
    public static void main(String[] args) {
        ForkJoinPool fjp = new ForkJoinPool(4);
        fjp.invoke(new MyTask("files"));
        fjp.shutdown();
    }
}

class MyTask extends RecursiveTask<Void> {
    private final String path;

    public MyTask(String path) {
        this.path = path;
    }

    @Override
    protected Void compute() {
        File file = new File(path);
        if (file.isFile()) {
            System.out.println(file.getPath());
            return null;
        } else if (file.isDirectory()) {
            File[] files = file.listFiles();
            List<MyTask> tasks = new ArrayList<>();
            if (files != null) {
                for (File f : files) {
```

```
                    MyTask t = new MyTask(f.getPath());
                    tasks.add(t);
                    t.fork();
                }
            }
            for (MyTask task : tasks) {
                task.join();
            }
        }

        return null;
    }
}
```

În exemplul de mai sus, se pot observa ambele moduri de a crea și submite spre execuție un task nou. În *main()*, se folosește *invoke()*, care se va bloca până când toate task-urile s-au terminat, deci până este sigur să apelăm *shutdown()*. În schimb, în interiorul unui task, se adaugă toate task-urile folosind *fork()*, și apoi se așteaptă finalizarea lor folosind *join()*.

Exerciții

Hint

În acest laborator, veți lucra cu această arhivă, pe care o veți utiliza să paralelizați trei probleme pe baza modelului Replicated Workers, folosind ExecutorService și ForkJoinPool:

- găsirea căilor dintre nodurile unui graf
- colorarea unui graf → găsiți detalii adiționale aici
- problema damelor → găsiți detalii adiționale aici.

1. Paralelizați găsirea căilor între două noduri pe baza scheletului oferit (în pachetul *task1*) folosind ExecutorService. **(1p)**
2. Paralelizați problema colorării unui graf pe baza scheletului oferit (în pachetul *task2*) folosind ExecutorService. **(2p)**
3. Paralelizați problema damelor pe baza scheletului oferit (în pachetul *task3*, cu soluțiile *[(2, 1), (4, 2), (1, 3), (3, 4)]* și *[(3, 1), (1, 2), (4, 3), (2, 4)]*) folosind ExecutorService. **(2p)**
4. Paralelizați găsirea căilor între două noduri pe baza scheletului oferit (în pachetul *task4*) folosind ForkJoinPool. **(1p)**
5. Paralelizați problema colorării unui graf pe baza scheletului oferit (în pachetul *task5*) folosind ForkJoinPool. **(2p)**
6. Paralelizați problema damelor pe baza scheletului oferit (în pachetul *task6*) folosind ForkJoinPool. **(2p)**

apd/laboratoare/07.txt · Last modified: 2021/01/11 21:08 by florin.mihalache

Laboratorul 8 - Introducere în programarea distribuită cu MPI

Despre MPI și programarea distribuită

Până acum, ați lucrat cu programarea paralelă, care presupune existența a mai multor fire de execuție (threads), care execută instrucțiuni în mod paralel și concurent, ele având acces la aceeași zonă de memorie, în cadrul unei singure mașini de calcul, care are mai multe procesoare.

Putem extinde conceptul de programare paralelă prin folosirea a mai multor mașini de calcul, care sunt conectate în cadrul unei rețele într-un concept numit **programarea distribuită**.

Spre deosebire de programarea paralelă, în programarea distribuită nu există conceptul de memorie partajată (shared memory). Întrebarea este următoarea: cum poate o mașină să știe ce date are o altă mașină din cadrul rețelei? Soluția este **comunicarea prin mesaje**, realizată de către mașinile (de tip nod) dintr-o rețea. Comunicarea prin mesaje are două funcționalități:

- comunicarea: un nod (transmițător / sender) trimite date prin intermediul unui canal de comunicație către un alt nod (receptor / receiver)
- sincronizarea: un mesaj nu poate fi recepționat înainte ca acesta să fie transmis

MPI (Message Passing Interface) reprezintă un standard pentru comunicarea prin mesaje, elaborat de MPI Forum, și are la bază modelul proceselor comunicante prin mesaje.

Un proces reprezintă un program aflat în execuție și se poate defini ca o unitate de bază care poate executa una sau mai multe sarcini în cadrul unui sistem de operare. Spre deosebire de thread-uri, un proces are propriul său spațiu de adrese (propria zonă de memorie) și acesta poate avea, în cadrul său, mai multe thread-uri în execuție, care partajează resursele procesului.

În cadrul lucrului în C/C++, MPI reprezintă o bibliotecă, care are funcționalitățile implementate într-un header numit **mpi.h**. Pentru compilare, la MPI există un compilator specific:

- mpicc, pentru lucrul în C
- mpic++, pentru lucrul în C++

În ambele limbaje, pentru rularea unui program MPI folosim comanda **mpirun**, împreună cu parametrul **-np**, unde precizăm numărul de procese care rulează în cadrul programului distribuit.

Exemplu:

- compilare:
  - C: `mpicc hello.c -o hello`
  - C++: `mpic++ hello.cpp -o hello`
- rulare: `mpirun -np 4 hello` - rulare cu 4 procese

Dacă încercați să rulați comanda **mpirun** cu un număr de procese mai mare decât numărul de core-uri fizice disponibile pe procesorul vostru, este posibil să primiți o eroare cum ca nu aveți destule sloturi libere. Puteți elimina acea eroare adăugând parametrul **–oversubscribe** atunci când rulați mpirun.

Instalare MPI: Pentru a lucra cu MPI, trebuie să instalați biblioteca pentru MPI pe Linux, folosind următoarea comandă: `sudo apt install openmpi-bin openmpi-common openmpi-doc libopenmpi-dev`

În caz că lucrați cu MPI pe WSL, o să vă apară un warning, care poate fi fixat cu ajutorul hint-urilor de aici [https://github.com/microsoft/WSL/issues/3397]

Implementarea unui program distribuit în MPI

Exemplu de program MPI - Hello World:

```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>

#define  MASTER 0

int main (int argc, char *argv[]) {
    int numtasks, rank, len;
    char hostname[MPI_MAX_PROCESSOR_NAME];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Get_processor_name(hostname, &len);
    if (rank == MASTER)
        printf("MASTER: Number of MPI tasks is: %d\n",numtasks);
    else
        printf("WORKER: Rank: %d\n",rank);

    MPI_Finalize();
}
```

Un comunicator (`MPI_Comm`) reprezintă un grup de procese care comunică între ele. `MPI_COMM_WORLD` reprezintă comunicatorul default, din care fac parte toate procesele.

Funcții:

- `MPI_Init` - se inițializează programul MPI, mai precis se creează contextul în cadrul căruia rulează procesele. Argumentele din linie de comandă sunt pasate către contextul de rulare a proceselor.
- `MPI_Comm_size` - funcție care determină numărul de procese (numtasks) care rulează în cadrul comunicatorului (de regulă MPI_COMM_WORLD)
- `MPI_Comm_rank` - funcție care determină identificatorul (rangul) procesului curent în cadrul comunicatorului.
- `MPI_Get_processor_name` - determină numele procesorului
- `MPI_Finalize` - declanșează terminarea programului MPI

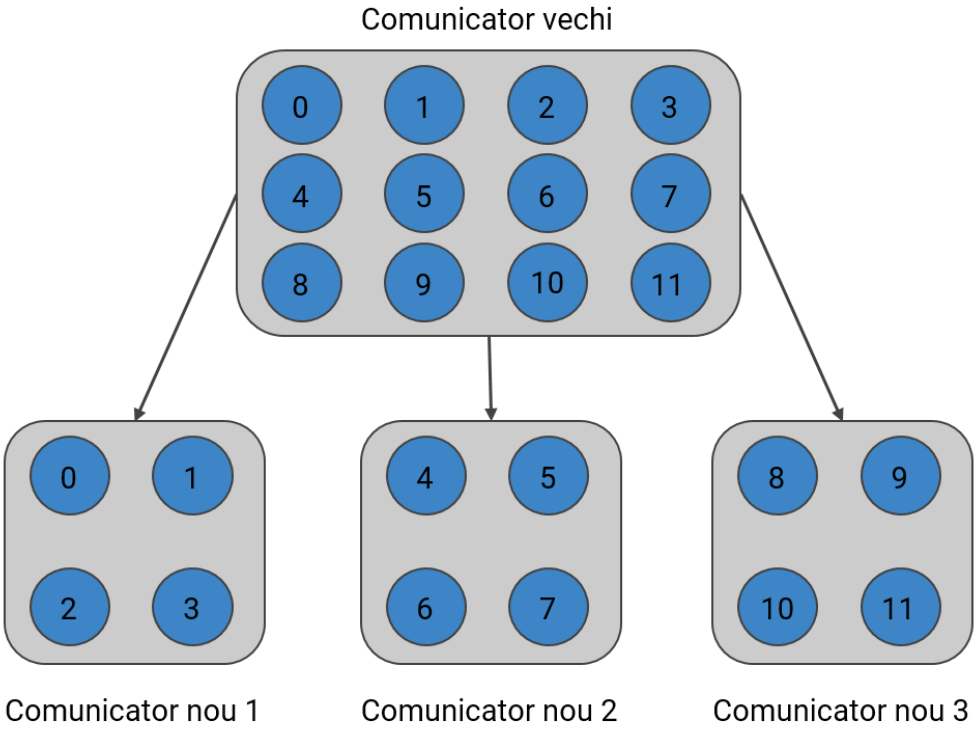
În cadrul schimbului de date între procese, este necesar mereu să precizăm tipul acestora. În MPI, se folosește enum-ul `MPI_Datatype`, care se mapează cu tipurile de date din C/C++, după cum puteți vedea în tabelul de mai jos:

MPI_Datatype	Echivalentul din C/C++
MPI_INT	int
MPI_LONG	long
MPI_CHAR	char
MPI_FLOAT	float
MPI_DOUBLE	double

Putem crea comunicatoare dintr-un alt comunicator folosind funcția MPI_Comm_split, care împarte un comunicator în mai multe comunicatoare mai mici. Semnătura funcției este următoarea: `int MPI_Comm_split(MPI_Comm comm, int color, int key, MPI_Comm * newcomm)`, unde:

- `comm` - comunicatorul este împărțit
- `color` - identificator al noului comunicator, din care face parte un proces (de regulă rang_vechi_proces / dimensiune_comunicator_nou)
- `key` - noul rang al procesului în cadrul noului comunicator (de regulă rang_vechi_proces % dimensiune_comunicator_nou)
- `newcomm` - noul comunicator format

Aveți mai jos o imagine care ilustrează cum funcționează `MPI_Comm_split`:



Funcții de transmisie a datelor în MPI

Convenție: vom marca cu ↓ parametrii de tip input ai unei funcții și cu ↑ parametrii de tip output în cadrul prezentării funcțiilor de mai jos.

MPI_Send

MPI_Send reprezintă funcția prin care un proces trimite date către un alt proces. Semnătura funcției este următoarea:

`int MPI_Send(void* data, int count, MPI_Datatype datatype, int destination, int tag, MPI_Comm communicator)`, unde:

- `data` (↓) - reprezintă datele trimise de la procesul sursă către procesul destinație
- `count` (↓) - dimensiunea datelor transmise
- `datatype` (↓) - tipul datelor transmise
- `destination` (↓) - rangul / identificatorului procesului destinație, către care se trimit datele
- `tag` (↓) - identificator al mesajului
- `communicator` (↓) - comunicatorul în cadrul căruia se face trimiterea datelor între cele două procese

MPI_Send este o funcție blocantă. Mai precis, programul se blochează până când bufferul dat ca prim parametru poate fi refolosit, chiar dacă nu se execută acțiunea de primire a mesajului transmis de procesul curent (MPI_Recv). Dacă apare cazul în care procesul P1 trimite date (MPI_Send) la procesul P2, iar P2 nu are suficient loc în buffer-ul de recepție (buffer-ul nu are suficient loc liber sau este plin) atunci P1 se va bloca.

MPI_Recv

MPI_Recv reprezintă funcția prin care un proces primește date de la un alt proces. Semnătura funcției este următoarea:

`int MPI_Recv(void* data, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm communicator, MPI_Status* status)`, unde:

- `data` (↑) - reprezintă datele primite de la procesul sursă de către procesul destinație
- `count` (↓) - dimensiunea datelor primite

- `datatype` (↓) - tipul datelor primite
- `source` (↓) - rangul / identificatorului procesului sursă, care trimite datele
- `tag` (↓) - identificator al mesajului
- `communicator` (↓) - comunicatorul în cadrul căruia se face trimiterea datelor între cele două procese
- `status` - conține date despre mesajul primit, `MPI_Status` fiind o structură ce conține informații despre mesajul primit (sursa, tag-ul mesajului, dimensiunea mesajului). Dacă nu dorim să ne folosim de datele despre mesajul primit, punem `MPI_STATUS_IGNORE`, prin care se ignoră status-ul mesajului.

În situația în care procesul P apelează funcția de MPI_Recv(), el se va bloca până va primi toate datele asteptate, astfel că dacă nu va primi nimic sau ceea ce primește este insuficient, P va rămâne blocat. Adică MPI_Recv() se termină doar în momentul în care buffer-ul a fost umplut cu datele așteptate.

Structura `MPI_Status` include următoarele câmpuri:

- `int count` - dimensiunea datelor primite
- `int MPI_SOURCE` - identificatorul procesului sursă, care a trimis datele
- `int MPI_TAG` - tag-ul mesajului primit

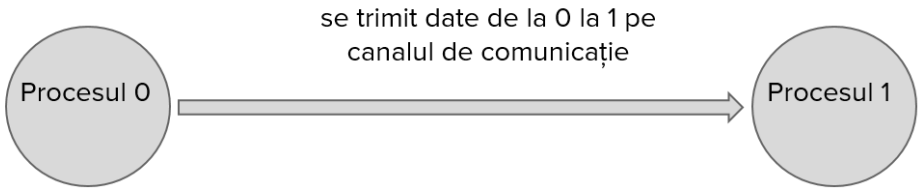
MPI_Recv este o funcție blocantă, mai precis programul se poate bloca până când se execută acțiunea de trimitere a mesajului către procesul sursă.

Un exemplu de program în care un proces trimite un mesaj către un alt proces:

```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[])
{
    int  numtasks, rank, len;
    char hostname[MPI_MAX_PROCESSOR_NAME];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks); // Total number of processes.
    MPI_Comm_rank(MPI_COMM_WORLD,&rank); // The current process ID / Rank.
    MPI_Get_processor_name(hostname, &len);

    srand(42);
    int random_num = rand();
    printf("Before send: process with rank %d has the number %d.\n", rank,
            random_num);

    if (rank == 0) {
        MPI_Send(&random_num, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else {
        MPI_Status status;
        MPI_Recv(&random_num, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        printf("Process with rank %d, received %d with tag %d.\n",
                rank, random_num, status.MPI_TAG);
    }

    printf("After send: process with rank %d has the number %d.\n", rank,
            random_num);

    MPI_Finalize();

}
```

Când un proces X trimite un mesaj către un proces Y, tag-ul T al mesajului din MPI_Send, executat de procesul X, trebuie să fie același cu tag-ul mesajului din MPI_Recv, executat de procesul Y, deoarece procesul Y așteaptă un mesaj care are tag-ul T, altfel, dacă sunt tag-uri diferite, programul se va bloca.

O ilustrație a modului cum funcționează împreună funcțiile MPI_Send și MPI_Recv:



Mai jos aveți un exemplu în care un proces trimite un întreg array de 100 de elemente către un alt proces:

```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[])
{
    int numtasks, rank, len;
    int size = 100;
    char hostname[MPI_MAX_PROCESSOR_NAME];
    int arr[size];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Get_processor_name(hostname, &len);

    srand(42);
    if (rank == 0) {
        for (int i = 0; i < size; i++) {
            arr[i] = i;
        }

        printf("Process with rank [%d] has the following array:\n", rank);
        for (int i = 0; i < size; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");

        MPI_Send(&arr, size, MPI_INT, 1, 1, MPI_COMM_WORLD);
        printf("Process with rank [%d] sent the array.\n", rank);
    } else {
        MPI_Status status;
        MPI_Recv(&arr, size, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
        printf("Process with rank [%d], received array with tag %d.\n",
                rank, status.MPI_TAG);

        printf("Process with rank [%d] has the following array:\n", rank);
        for (int i = 0; i < size; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }

    MPI_Finalize();

}
```

MPI_Bcast

MPI_Bcast reprezintă o funcție prin care un proces trimite un mesaj către toate procesele din comunicator (message broadcast), inclusiv lui însuși.

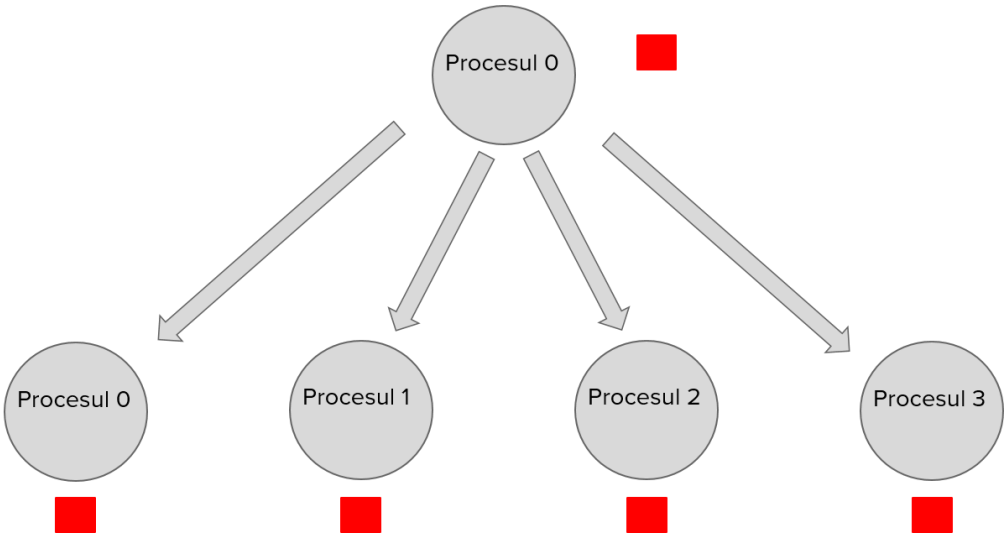
În cadrul implementării MPI_Bcast sunt executate acțiunile de trimitere și de recepționare de mesaje, așadar nu trebuie să apelați MPI_Recv.

Semnătura funcției este următoarea:

`int MPI_Bcast(void* data, int count, MPI_Datatype datatype, int root, MPI_Comm communicator)`, unde:

- `data` (↓ + ↑) - reprezintă datele care sunt transmise către toate procesele. Acest parametru este de tip input pentru procesul cu identificatorul `root` și este de tip output pentru restul proceselor.
- `count` (↓) - dimensiunea datelor trimise
- `datatype` (↓)- tipul datelor trimise
- `root` (↓) - rangul / identificatorului procesului sursă, care trimite datele către toate procesele din comunicator, inclusiv lui însuși
- `tag` (↓) - identificator al mesajului
- `communicator` (↓) - comunicatorul în cadrul căruia se face trimiterea datelor către toate procesele din cadrul acestuia

O ilustrație care arată cum funcționează MPI_Bcast aveți mai jos:



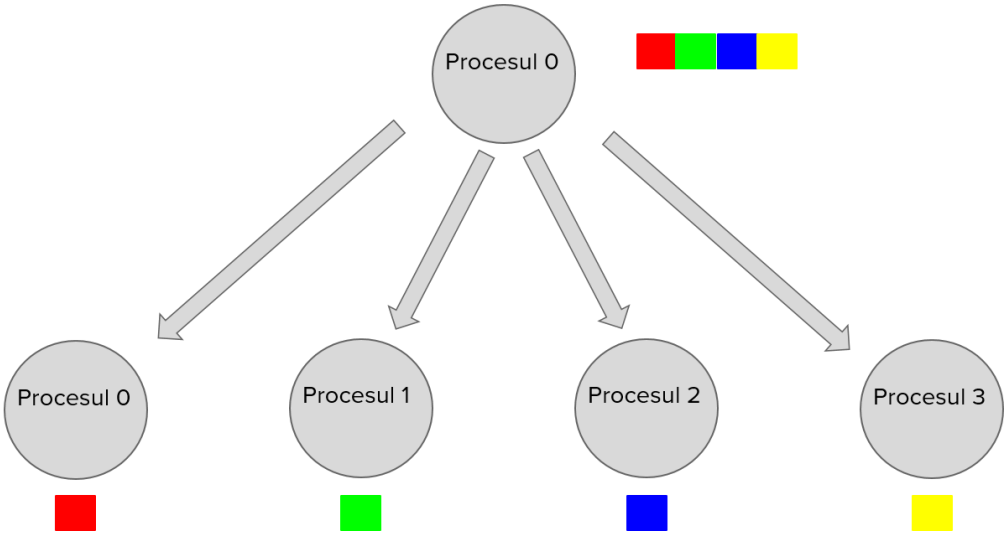
MPI_Scatter

MPI_Scatter este o funcție prin care un proces împarte un array pe bucăți egale ca dimensiuni, unde fiecare bucată revine, în ordine, fiecărui proces, și le trimite tuturor proceselor din comunicator, inclusiv lui însuși.

Semnătura funcției este următoarea: `int MPI_Scatter(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data, int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)`, unde:

- `send_data` (↓) - reprezintă datele care sunt împărțite și trimise către procesele din comunicator
- `send_count` (↓) - reprezintă dimensiunea bucății care revine fiecărui proces (de regulă se pune ca fiind dimensiunea_totală / număr_de_procese).
- `send_datatype` (↓) - tipul datelor trimise către procese
- `recv_data` (↑) - reprezintă datele care sunt primite și stocate de către procese
- `recv_count` (↓) - dimensiunea datelor primite (de regulă dimensiunea_totală / număr_de_procese)
- `recv_datatype` (↓) - tipul datelor primite de către procese (de regulă este același cu send_datatype)
- `root` (↓) - identificatorul procesului care împarte datele și care le trimite către procesele din comunicator, inclusiv lui însuși
- `communicator` (↓) - comunicatorul din care fac parte procesele (de regulă `MPI_COMM_WORLD`)

O ilustrație a modului cum funcționează MPI_Scatter:



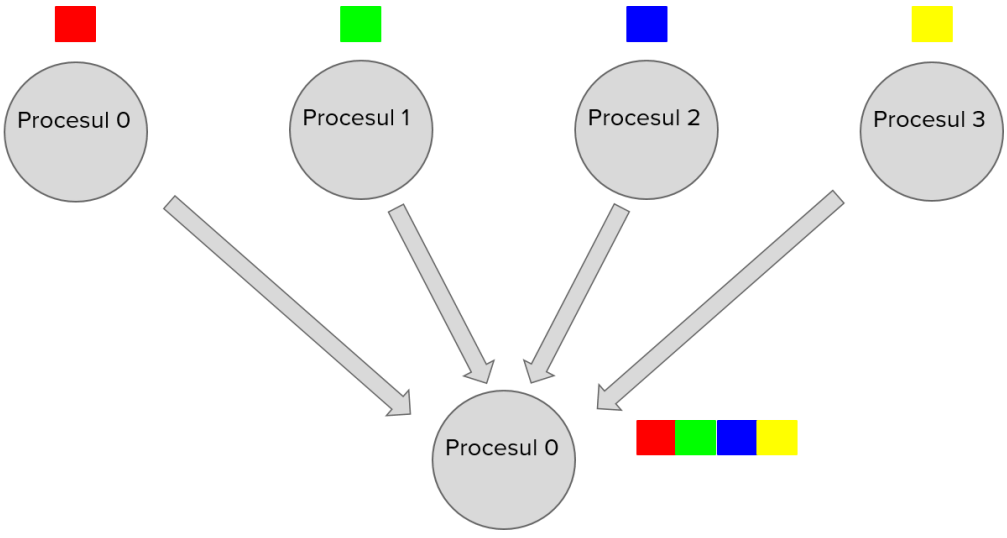
MPI_Gather

MPI_Gather este o funcție care reprezintă inversul lui MPI_Scatter, în sensul că un proces primește elemente de la fiecare proces din comunicator, inclusiv de la el însuși, și le unifică într-o singură colecție.

Semnătura funcției este următoarea: `int MPI_Gather(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data, int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)`, unde:

- `send_data` (↓) - reprezintă datele care trimise de fiecare proces către procesul cu id-ul root
- `send_count` (↓) - reprezintă dimensiunea bucății trimisă de fiecare proces (de regulă se pune ca fiind dimensiunea_totală / număr_de_procese).
- `send_datatype` (↓) - tipul datelor trimise de către procese
- `recv_data` (↑) - reprezintă datele care sunt primite și stocate de către procesul root
- `recv_count` (↓) - dimensiunea datelor primite (de regulă dimensiunea_totală / număr_de_procese)
- `recv_datatype` (↓) - tipul datelor primite de către procesul root (de regulă este același cu send_datatype)
- `root` (↓) - identificatorul procesului care primește datele (inclusiv de la el însuși)
- `communicator` (↓) - comunicatorul din care fac parte procesele (de regulă `MPI_COMM_WORLD`)

O ilustrare a modului cum funcționează MPI_Gather:



Mai jos aveți un exemplu de MPI_Scatter folosit împreună cu MPI_Gather:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define ROOT 0
#define CHUNK_SIZE 5 // numarul de elemente per proces

int main (int argc, char **argv) {
    int rank, proc, a;

    int* arr;
    int* process_arr;
    int* result_arr;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &proc);

    if (rank == ROOT) {
        arr = malloc (CHUNK_SIZE * proc * sizeof(int));
        for (int i = 0; i < proc * CHUNK_SIZE; ++i) {
            arr[i] = 0;
        }
    }

    process_arr = malloc (CHUNK_SIZE * sizeof(int));
    MPI_Scatter(arr, CHUNK_SIZE, MPI_INT, process_arr, CHUNK_SIZE, MPI_INT, ROOT, MPI_COMM_WORLD);

    for (int i = 0; i < CHUNK_SIZE; i++) {
        printf("Before: rank [%d] - value = %d\n", rank, process_arr[i]);
        process_arr[i] = i;
        printf("After: rank [%d] - value = %d\n", rank, process_arr[i]);
    }

    if (rank == ROOT) {
        result_arr = malloc (CHUNK_SIZE * proc * sizeof(int));
    }

    MPI_Gather(process_arr, CHUNK_SIZE, MPI_INT, result_arr, CHUNK_SIZE, MPI_INT, ROOT, MPI_COMM_WORLD);

    if (rank == ROOT) {
        for (int i = 0; i < CHUNK_SIZE * proc; i++) {
            printf("%d ", result_arr[i]);
        }
        printf("\n");
    }

    if (rank == ROOT) {
        free(arr);
        free(result_arr);
    }

    free(process_arr);

    MPI_Finalize();
    return 0;
}
```

Algoritmul inel

Algoritmul inel este un algoritm de tip undă, care funcționează pe topologii de sub formă de inel, unde fiecare nod proces are un vecin dedicat (Urm / next). În cadrul topologiilor inel, transmisia datelor se face folosind adresarea prin direcție, unde mai precis un nod proces trimite datele către vecinul său (Urm), astfel în cadrul topologiei formându-se un ciclu Hamiltonian.

În cadrul algoritmului, avem un nod proces inițiator, care trimite un jeton către vecinul său, care este pasat de fiecare proces către vecinul său de-a lungul ciclului, până când jetonul ajunge înapoi până la nodul proces inițiator.

Pseudocod:

```
chan token[1..n] (tok_type tok) //  canalul de comunicatie
```

```
// initiator
process P[i] {
    tok_type tok;
    send token[next](tok); // trimite token-ul la urmatorul nod
    receive token[i](tok); // primeste token-ul de la nodul precedent
    decizie;
}

// neinitiator
process P[k, k = 1..n, k != n] {
    tok_type tok;
    receive token[k](tok); // primeste token-ul de la nodul precedent
    send token[next](tok); // trimite token-ul la urmatorul nod
}
```

Pentru a vedea execuția pe pași al acestui algoritm, aveți această prezentare la dispoziție.

Exerciții

- **Exercițiul 1 (2p):** Implementați algoritmul inel folosind MPI, unde procesul cu id-ul 0 trimite un număr aleatoriu către procesul 1 (vecinul său), iar apoi celelalte noduri vor primi numărul de la procesul precedent, îl incrementează cu 2 și îl trimit la următorul proces (de exemplu procesul 2 primește valoarea de la procesul 1, o incrementează și o trimite mai departe procesului 3), totul terminându-se când valoarea ajunge la procesul 0. Pentru fiecare proces trebuie să afișați rangul acestuia și valoarea primită.
- **Exercițiul 2 (2p):** Implementați un program în MPI în care procesul 0 trimite o valoare generată aleatoriu către celelalte procese, folosind `MPI_Bcast`. **Atenție:** Nu folosiți `MPI_Recv`.
- **Exercițiul 3 (2p):** Implementați un program în MPI cu N procese, în care procesul 0 inițializează un array de dimensiune 5 * N cu 0, îl împarte (dimensiunea chunk-ului fiind 5, definită în schelet) și îl trimite tuturor proceselor cu `MPI_Scatter`. Procesele vor aduna fiecare element primit cu rangul fiecăruia, apoi ele vor trimite array-urile pe care au efectuat operația de adunare cu rangul către procesul 0 folosind `MPI_Gather`. Procesul 0 va afișa ulterior array-ul rezultat din `MPI_Gather`.
- **Exercițiul 4 (2p):** Implementați un program MPI cu 4 procese, unde 3 procese trimit o valoare către al patrulea proces, care va primi valoarea folosind `MPI_ANY_SOURCE` și va afișa valoarea primită și rangul procesului sursă folosind `MPI_SOURCE` din `MPI_Status` (status din `MPI_Recv`).

`MPI_ANY_SOURCE` este folosit în loc de rangul procesului sursă la `MPI_Recv`, indicând astfel că procesul receptor poate primi un mesaj de la oricare proces.

- **Exercițiul 5 (2p):** Implementați un program MPI cu 2 procese, unde procesul 0 trimite 10 valori cu tag-uri diferite către procesul 1, iar procesul 1 primește valorile folosind parametrul `MPI_ANY_TAG` în `MPI_Recv`. Afișați valoarea primită împreună cu tag-ul acesteia folosind `MPI_Status` (status din `MPI_Recv`).

`MPI_ANY_TAG` este folosit în loc de tag-ul mesajului la `MPI_Recv`, indicând faptul că poate primi orice mesaj de la procesul sursă, neținând cont de tag-ul pe care procesul sursă îl pune mesajului la `MPI_Send`.

- **Exercițiul 6 - Bonus (2p):** Se pornește un număr de procese N divizibil cu 4. Se împart procesele în grupe de câte 4 și își trimit în cerc (inel) rangul. Se afișează vechiul rang împreună cu dimensiunea vechiului grup. Se afișează noul rang împreună cu dimensiunea noului grup. Fiecare proces are voie sa comunice doar în grupul lui. Se va folosi `MPI_Comm_split`. Fiecare proces va afișa rangul său din noul grup, numărul grupului și ce număr a primit.

Resurse

- Schelet laborator

- MPI Cheatsheet

apd/laboratoare/08.txt · Last modified: 2021/01/30 22:22 by florin.mihalache

Laboratorul 9 - Operații și prelucrări distribuite în MPI

Reduce

Operația de reduce (cunoscută în limbajele funcționale - Haskell, Racket - drept fold) reprezintă un tip de operație prin care elementele unei colecții sunt restrânse / acumulate într-un singur rezultat, printr-o singură operație aplicată între elementele unei colecții (+, *, min, max etc.)

Exemplu:

```
l = [1, 2, 3, 4, 5, 6]
op = +
rezultat = 1 + 2 + 3 + 4 + 5 + 6 = 21
```

În MPI, operația reduce este implementată în funcția MPI_Reduce, însă în cadrul laboratorului vom discuta cum poate fi implementată, de mână, această operație în MPI.

Aici aveți atașate slide-uri, care descriu în detaliu pașii de implementare ai operației reduce: slides

În concluzie, primul proces va avea valoarea finală.

Pseudocod:

```
for (pas = 2; pas <= nr_procese; pas *= 2)
        if (rank % pas == 0)
                primește la procesul cu rank-ul [rank + (pas / 2)]
                adună
        else if (rank % (pas / 2) == 0)
                trimite la procesul cu rank-ul [rank - (pas / 2)]
```

Scan

Operația de scan este similară operației de reduce (acumularea elementelor unei colecții într-un singur rezultat). Diferența față de reduce este că ultimul proces din comunicator deține rezultatul final, practic scan reprezintă inversul lui reduce. În plus, fiecare proces are un rezultat parțial acumulat, în sensul că procesul 0 are valorea P0, procesul 1 are valoarea P0 + P1, procesul 2 are valoarea P0 + P1 + P2, etc.

Exemplu:

```
l = [1, 2, 3, 4, 5, 6]
op = +
rezultat = 1 + 2 + 3 + 4 + 5 + 6 = 21

Pași:
[1, 2, 3, 4, 5, 6]
[1, 3, 3, 4, 5, 6]
[1, 3, 6, 4, 5, 6]
[1, 3, 6, 10, 5, 6]
[1, 3, 6, 10, 15, 6]
[1, 3, 6, 10, 15, 21] -> rezultatul este 21
```

Aici aveți atașate slide-uri, care descriu în detaliu pașii de implementare ai operației scan: slides

Pseudocod:

```
for (pas = 1; pas < nr_procese; pas *= 2)
        if (rank + pas < nr_procese)
                trimite la procesul cu rank-ul [rank + pas]
        if (rank - pas >= 0)
                primește de la procesul cu rank-ul [rank - pas]
                adună
```

Broadcast

Operația de broadcast este o operație prin care un proces trimite o valoare tuturor proceselor din cadrul comunicatorului. Această operație este reprezentată, în MPI, de MPI_Bcast (remember laboratorul 8).

Aici aveți atașate slide-uri, care descriu în detaliu pașii de implementare ai operației broadcast: slides

Pseudocod:

```
for (pas = 1; pas < nr_procese; pas *= 2)
        if (rank < pas and rank + pas < nr_procese)
                trimite la procesul cu rank-ul [rank + pas]
        else if (rank >= pas and rank < pas * 2)
                primește de la procesul cu rank-ul [rank - pas]
```

Pipeline

În domeniul de computer science, un pipeline reprezintă un concept folosit în proiectarea procesoarelor și al echipamentelor hardware prin care se crește numărul de instrucțiuni ce sunt executate într-o unitate de timp.

Acest concept poate fi extins în MPI, unde un proces produce rezultate intermediare, care sunt transmise către procesul următor, care procesează și prelucrează rezultatele primite, pe care le trimite mai departe, către procesul următor, totul repetându-se până la ultimul proces.

Slides cu explicații și exemple: slides

Calculul unui polinom folosind pipeline

Un polinom, în matematică, reprezintă o ecuație de tipul: f(x) = x^2 + 2 * x + 1, unde x reprezintă variabila din ecuație.

În MPI, putem calcula un polinom folosind tehnica pipeline în felul următor: numărul de coeficienți va fi egal cu numărul de procese (care este egal cu numărul de pași din procesul de pipeline). În cazul exemplului de mai sus, o să avem 3 procese, fiecare proces corespunzând cu câte un coeficient din ecuație.

Aici aveți atașate slide-uri, care ilustrează pas cu pas: slides

Sortarea folosind pipeline

Dându-se un vector cu n elemente în ordine aleatoare, dorim să-l sortăm prin metoda pipeline, folosind MPI. Pentru aceasta, vom folosi o rețea cu n + 1 noduri (1 master și n workeri).

Punctul final este momentul în care fiecare proces worker deține stocat unul din numerele din vectorul inițial, iar `rank[i]→saved_value < rank[i + 1]→saved_value` pentru orice i din [1, n).

Fiecare worker este inițializat la început cu o valoare (-1 sau depinde de restricții), va primi (n - rank) numere de la rank-ul anterior și va trimite mai departe rank-ului următor (n - rank - 1) numere.

Atunci când primește un număr, worker-ul îl va compara cu valoarea stocată:

- dacă nr < saved_value, atunci salvăm noua valoare și o trimitem pe cea veche procesului următor
- dacă nr >= saved_value, atunci o trimitem direct procesului următor

La final, fiecare worker va trimite către master valoarea lui, iar master-ul va construi vectorul rezultat. Urmăriți slide-urile pas cu pas: slides

Rank Sort

Rank Sort este un algoritm de sortare a unui vector folosindu-se de rangul fiecărui element din vector. Rangul unui element presupune câte numere sunt mai mici decât el în vector.

Pași:

1. Se calculează rangul fiecărui element: pentru fiecare element din vector v[i], unde 0 ⇐ i < N (N - dimensiunea vectorului), se numără câte elemente din vector v[j], j != i sunt mai mici: v[j] < v[i], 0 ⇐ i, j < N, i != j.
2. Elementul de pe poziția "i" va fi pe poziția "rang" în noul vector sortat: q[rank] = v[i].

Algoritmul "Parallel rank sort" are ca scop paralelizarea pasului 1. Rangul fiecărui element poate fi căutat în paralel, adică fiecare proces sau fir de execuție va realiza căutarea rangurilor pentru elementele din intervalul său: [start, end).

Complexitatea temporală a algoritmului este O(n^2), ceea ce face complexitatea în cazul paralelizării pasului 1 a lui să devină O(n^2 / p), p - numărul de procesoare / fire de execuție.

Aici aveți atașate slide-uri, care ilustrează pas cu pas: slides

Exerciții

Schelet laborator

1. **(2p)** Implementați operația de reduction, unde fiecare proces are o valoare proprie, prin care avem o colecție de valori (a nu se folosi MPI_Reduction)
2. **(2p)** Implementați operația de scan, unde fiecare proces are o valoare proprie, prin care avem o colecție de valori (a nu se folosi MPI_Scan)
3. **(2p)** Implementați operația de broadcast, prin care procesul 0 trimite o valoare tuturor proceselor (a nu se folosi MPI_Bcast)
4. **(2p)** Implementați operația de calcul polinomial folosind pipeline, pentru f(5), unde f(x) reprezintă ecuația polinomială (construită prin citirea din fișier existentă în scheletul de cod, unde aveți hint-uri legate de implementare). Pentru testare, există două fișiere de test pentru input (a1.txt și a2.txt) și puteți să vă folosiți de oricare dintre ele, cu precizia că numărul de procese trebuie să fie același cu numărul de coeficienți. Pentru ușurință în testare, puteți folosi regulile run1 și run2 din Makefile.
5. **(2p)** Implementați sortarea prin pipeline, folosind scheletul de laborator și explicațiile din laborator și slide-uri
6. **(2p)** Să se sorteze un vector folosindu-se algoritmul Rank Sort. Procesul master va genera vectorul și îl va popula cu elemente. Va trimite tot vectorul celorlalte procese. Fiecare proces este responsabil pentru calcularea pozițiilor unei anumite părți din vector. La final toate procesele trimit procesului master noile poziții calculate. Procesul master va aranja elementele conform noilor poziții primite.

Resurse suplimentare

Aici aveți atașate resurse suplimentare (slide-uri) legate de conceptele din cadrul acestui laborator, ce pot fi utile pentru testele practice.

apd/laboratoare/09.txt · Last modified: 2021/01/12 12:24 by florin.mihalache

Laboratorul 10 - Stabilirea Topologiei. Alegerea Liderului.

In acest laborator vom explora utilizarea MPI in cadrul unei topologii de noduri. Vom invata sa stabilim o topologie, sa alegem un lider (folosind heart-beat) si determinăm numărul de noduri din topologie folosind un algoritm epidemic.

Responsabili: Alexandru Hogea, Cristian Condruz. Multumiri lui Dorinel Filip si Florin Mihalache pentru ajutor.

Organizarea unui cluster

In cazul unui sistem distribuit, elementul cheie este **comunicarea** dintre procese. Aceasta dicteaza performanta sistemului, toleranta la defecte si logica fluxului de transfer de informatii.

Din punct de vedere al arhitecturii, nodurile pot fi organizate in mai multe moduri intr-un cluster:

- **grid** → nodurile sunt organizate pe nivele. Fiecare nod poate comunica cu vecinii pe orizontala si verticala. Fluxul informatiei este vertical si orizontal. Maxim 4 vecini.
- **inel** → nodurile sunt organizate in cerc. Fiecare nod vorbeste cu vecinii din stanga si dreapta. Fluxul informatiei este orizontal. Maxim 2 vecini.
- **arbore** → nodurile sunt organizate ierarhic, sub forma de arbore. Fluxul informatiei este vertical. Numar nelimitat de vecini.
- **graf** → nodurile sunt organizate sub forma unui graf. Fiecare nod poate comunica cu unul sau mai multi vecini. Fluxul informatiei este abstract. Numar nelimitat de vecini.

In cadrul acestui laborator vom explora organizarea unui sistem sub forma de graf generic.

Stabilirea unei topologii

Exista trei moduri prin care o topologie poate fi stabilita in cadrul unui cluster. Nodurile pot cunoaste starea intregului cluster de la bun inceput, pot cunoaste doar vecinii sau nu pot sti nimic despre starea sistemului.

Pentru a stabili topologia, clusterul poate opera in doua moduri:

- nodurile pot transmite starea lor celorlalte moduri de mai multe ori pana cand toate nodurile au topologia stabilita. Numarul de dati necesar asigurarii topologiei intre noduri se numeste **convergenta**. Acest mod este simplu de realizat, dar ineficient
- nodurile aleg un **lider** care sa calculeze topologia sistemului

Problema care se ridica in cazul utilizarii unui lider care sa creeze matricea de topologie, este ca nodurile *nu stiu* calea catre lider, ci doar cine este acest lider. **Arborele de acoperire** rezolva aceasta problema. Pe baza arborelui de acoperire, nodurile trimit informatia pe care o detin catre lider, trecand prin nodurile intermediare care sunt reprezentate de parintii lor in arbore. Nodurile intermediare adauga informatia primita la informatia proprie si o trimit mai departe. In final, liderul are toata informatia sistemului si poate crea matricea de topologie. Matricea este trimisa, apoi, fiecarui nod in parte

In cadrul acestui laborator vom explora stabilirea topologiei prin alegerea liderului.

In cazul retelelor de calculatoare, ruterele si switchurile sunt organizate sub forma unui cluster unde nodurile nu stiu nimic despre starea sistemului la inceput

In cazul alegerii liderului, nu este nevoie sa se cunoasca topologia la inceput, ci doar vecinii cu care nodurile pot interactiona

Alegerea liderului

Clusterele de noduri pot sa fie ierarhizate sau nu. In cadrul clusterelor bazate pe noduri autonome, nu exista un lider bine stabilit, ci contracte clare de comunicare intre noduri. Pe de alta parte, in clusterele

care au lider, unul sau mai multe noduri indeplinesc functii elevate. Printre functiile elevate se numara si *stabilirea si distribuirea topologiei* catre celelalte noduri.

Liderul reprezinta un nod sau un grup de noduri ales pe baza unei euristici. Euristica poate tine cont de resursele nodului respectiv, de numarul sau natura proceselor care ruleaza pe el sau pur si simplu de lucruri simple, precum *eticheta sa*.

In cadrul acestui laborator vom explora alegerea liderului bazat pe rank-ul cel mai mare al unui nod. Alegerea liderului se va face pe baza unui algoritm de tipul **heartbeat**.

Alegerea liderului. Heartbeat

Functionarea unui algoritm **heartbeat** se aseamana cu bataile inimii:

1. **sistola** → partea de "expandare". Informatia este transmisa catre nodurile vecine.
2. **diastola** → partea de "contractie". Informatia este receptata inapoi de la nodurile vecine.

In cadrul alegerii liderului, algoritmul heartbeat va fi aplicat in felul urmator:

1. nodurile vor trimite rankul lor catre vecini
2. fiecare nod va calcula *liderul local* in functie de maximul dintre rankul propriu si informaia primita de la vecin
3. vecinii vor trimite inapoi liderul lor local catre nodurile de la care au primit mesaj
4. nodurile care au trimis mesajul initial isi vor actualiza valoarea liderului local

Algoritmul se repeta de un numar suficient de ori ca sa se asigure convergenta informatiei.

Un exemplu de rulare a acestui algoritm poate fi observat aici heartbeat_choosing_leader.pdf

Construirea arborelui de acoperire

Dupa ce liderul a fost ales, acesta va **initia construirea arborelui de acoperire**.

Scopul arborelui de acoperire este ca fiecare nod sa stie, in orice moment de timp, cum sa ajunga catre lider.

Arborele de acoperire se reprezinta, cel mai usor, printr-un *vector de tati*. Constuirea acestui arbore se realizeaza printr-un algoritm de tip **unda - ecou**.

Construirea arborelui de acoperire. Unda - ecou

Algoritmul unda are doua parti:

1. **Unda** → toate nodurile, cu exceptia liderului asteapta sa primeasca un mesaj. Liderul initiaza unda. La primirea **primei sonde**, fiecare nod isi noteaza parintele si trimite un mesaj tuturor vecinilor, mai putin celui de la care a primit (părintele său)
2. **Ecou** → se asteapta primirea unui mesaj de la toti vecinii, *mai puțin de la părinte*. Informatiile primite de la vecini sunt agregate intr-un vector de tati si acesta este trimis către părinte.

La final, liderul va detine vectorul de tati final, care reprezinta **arborele de acoperire**. Arborele de acoperire este apoi distribuit intregului sistem

Verificarea topologiei

Dupa ce liderul a fost ales si topologia a fost distribuita catre toate nodurile, este util ca informatia sa fie validata. Validarea poate implica procese foarte complexe, insa pentru acest laborator vom folosi o validare bazata pe numarul de noduri prezente in cluster.

Calcularea numarului de noduri se poate realiza folosind un algoritm de tipul **epidemic**.

Calcularea numarului de noduri. Epidemic

Functionalitatea unui algoritm de tipul **epidemic** se aseamana cu raspandirea unei boli in care exista un pacient 0:

1. "pacientul 0" transmite informatia la vecinii sai
2. vecinii pacientului 0 devin "infectati" si transmit si ei, mai departe, informatia, catre vecinii lor

Pentru calcularea numarului de noduri, pacientul 0 va fi nodul lider. Acesta va detine o valoare de inceput, **1**. Toate celelalte noduri detin valoarea **0**. Nodul lider va realiza media aritmetica dintre valoarea sa si valoarea vecinilor sai. Informatia va fi sincronizata in ambele parti. Celalte noduri vor executa aceeasi operatie cu vecinii lor.

Algoritmul se repeta pana cand se atinge convergenta solutiei. La final, toate nodurile vor detine o aceeasi valoare subunitara. Impartirea lui 1 la aceasta valoare va rezulta in numarul de noduri al clusterului.

Un exemplu de rulare a acestui algoritm poate fi observat aici epidemic_counting_nodes.pdf

Exercitii

Pornind de la scheletul de laborator, va trebui sa realizati urmatoarele sarcini:

1. Aflati nodul lider al clusterului folosind un algoritm *heartbeat* (**3p**)
2. Realizati arborele de acoperire, plecand din lider, folosind un algoritm *unda-ecou* (**3p**)
3. Asigurati-va ca numarul de elemente din arborele de acoperire a fost stabilit bine prin calcularea numarului de noduri folosind un algoritm *epidemic* (**3p**)
4. Folosind arborele de acoperire, trimiteti catre lider configuratia fiecarui nod si realizati in lider matricea de topologie a clusterului. Distribuiti topologia catre toate nodurile (**1p**)

apd/laboratoare/10.txt · Last modified: 2021/01/19 16:27 by gabriel.tuculina

Laboratorul 1
Introducere în pthreads

Utilitate APD

- multithreading:
  - mobile (Android)
  - backend - sunt multe întrebări de interviu legate de multithreading
- programare distribuită - servere, clustere (Kafka, Cassandra)

De ce multithreading?

- performanță
- scalabilitate
- eficiență
- reducerea costurilor

Thread vs. proces

Thread:

- fiecare thread are stiva sa proprie și propriul său instruction pointer

Proces:

- fiecare proces are propriul său spațiu virtual de memorie (un proces nu poate accesa spațiul de memorie al altui proces)
- în cadrul unui proces, pot rula mai multe thread-uri, care partajează același spațiu de memorie al procesului în cadrul căruia rulează

Multithreading la APD

La APD vom lucra cu multithreading în:

- C - pthreads - POSIX (3 laboratoare + o temă)
- Java (4 laboratoare + o temă)

La ASC veți mai lucra cu thread-uri în Python și în CUDA și la APP cu OpenMP în C.

pthreads

- reprezintă o libr… o bibliotecă în C/C++
- includerea acesteia:

```
#include <pthread.h>
```

- compilarea:

```
gcc -o program program.c -lpthread

./program
```

Crearea unui thread

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void
*(*start_routine) (void *), void *arg);
```

- thread - thread-ul pe care vrem să-l pornim
- attr - atributele unui thread (NULL - atribute default)
- start_routine - funcția pe care să o execute thread-ul
- arg - parametrul trimis la funcția executată de thread (dacă vrem să trimitem mai mulți parametri, îi împachetăm într-un struct)

Notă: un thread poate crea alte thread-uri, nu există ierarhii

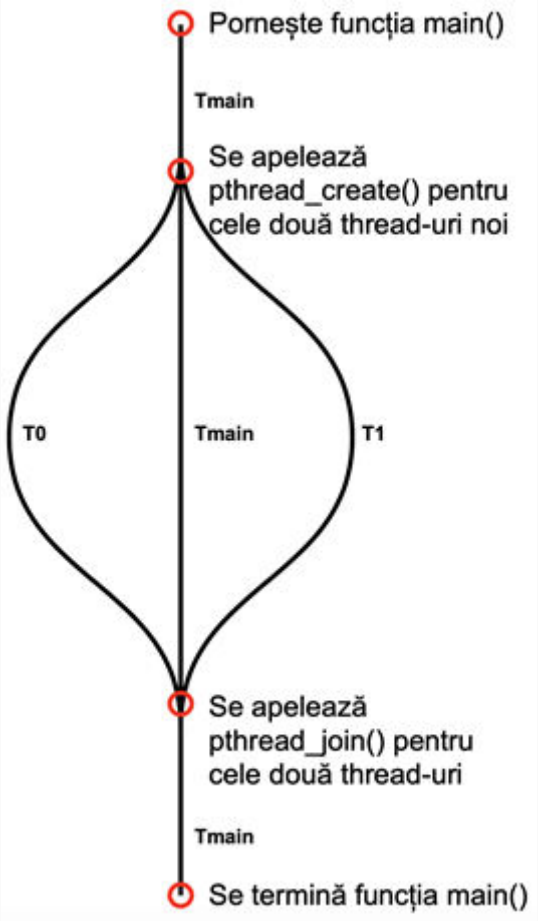
Terminarea unui thread

```
void *f(void *arg) {

// do stuff

    pthread_exit(NULL); // termină un thread - mereu apelat la finalul unei funcții executate de thread, dacă nu întoarcem un rezultat în funcție

}
```

Terminarea unui thread

```
int pthread_join(pthread_t thread, void **retval);
```

- se **așteaptă** terminarea unui thread - pthread_join **nu termină** execuția unui thread
- thread - thread-ul pe care îl așteptăm să termine
- retval - valoarea de retur a funcției executate de thread (poate fi NULL)



Crearea, rularea și terminarea thread-urilor

Cum trimitem mai mulți parametri la funcția thread-ului?

- În caz că dorim să trimitem mai mulți parametri funcției executate de threads, facem un `struct`, în care incapsulăm datele, și îl trimitem ca parametru al funcției executate de threads.
- Exemplu: https://gist.github.com/florinrm/361d0ff0b150fa77ece86cbbba47894d

Împărțirea unui array pe thread-uri

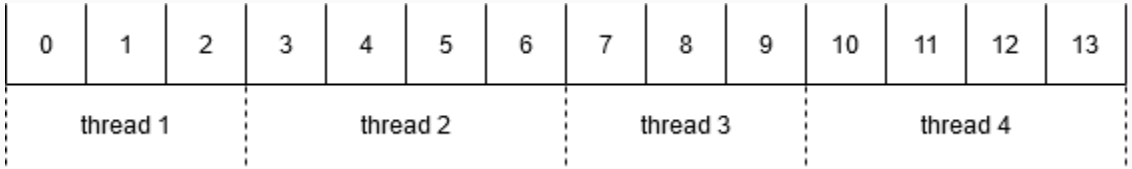
Când dorim să paralelizăm operații efectuate pe arrays, fiecărui thread îi va reveni o bucată din array, pe acea bucată executând funcția atribuită lui (thread-ului).

Formula de împărțire:

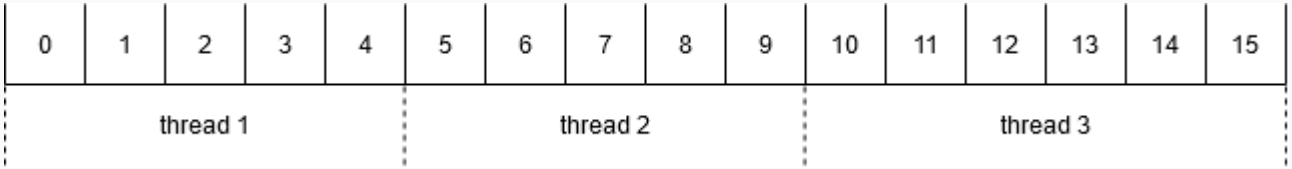
- `start_index = id * (double) n / p`
- `end_index = min(n, (id + 1) * (double) n / p))`, unde `id` = id-ul thread-ului, `n` = dimensiunea array-ului, `p` = numărul de threads

Împărțirea unui array pe thread-uri - exemplu

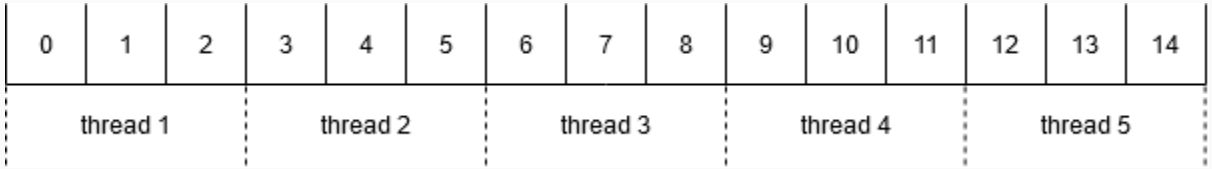
- n = 14, p = 4 => 3, 4, 3, 4 (th1: [0, 3), th2: [3, 7), th3: [7, 10), th4: [10, 14))



- n = 16, p = 3 => 5, 5, 6 (th1: [0, 5), th2: [5 -> 10), th3: [10 -> 16))



- n = 15, p = 5 => 3, 3, 3, 3, 3 ([0,3), [3, 6), [6, 9), [9, 12), [12 -> 15))



Întrebări (potențiale de interviu)

- Ce se va întâmpla dacă porniți un program cu mai multe threads decât cores?
- Care este diferența între un thread și un proces?
- Care este diferența între un thread și un procesor?
- Cine decide pe ce core va rula un anumit thread?
- Ce înseamna scalabilitate bună în programarea paralelă?
- Ce probleme pot apărea în cadrul unui program care funcționează pe mai multe threads?
- Dacă se creează două thread-uri folosind `pthread_create`, câte thread-uri vor exista în program (înainte de `pthread_join`)?

Feedback și trimiterea laboratorului

- Dați feedback legat de cum a fost la laborator aici.
- Deadline trimitere laborator: sâmbătă seara, pe moodle (este un assignment la fiecare laborator)

Resurse

https://man7.org/linux/man-pages/man3/pthread_create.3.html

https://man7.org/linux/man-pages/man3/pthread_exit.3.html

https://man7.org/linux/man-pages/man3/pthread_join.3.html

https://computing.llnl.gov/tutorials/pthreads/

Laboratorul 2
Elemente de sincronizare

Sincronizare - ce este și de ce

- în programarea paralelă, există situații când thread-urile accesează niște resurse comune (partajate) simultan (de regulă prin operații de read-write)
- situație exemplu: avem o variabilă globală `int a = 0` și două threads, care execută `a += 2`; este de așteptat ca rezultatul final să fie `a = 4`, dar nu mereu este așa

Incrementarea a += 2, cu doi threads

a = 0 (inițial)

T0	T1
`load(a, eax0) -> 0`	`load(a, eax1) -> 0`
`eax0 = eax0 + 2 -> 2`	`eax1 = eax1 + 2 -> 2`
`write(a, eax0) -> 2`	`write(a, eax1) -> 2`

Remember lab 1:
un thread are propria sa stivă, deci propriile registre

a = 4 sau a = 2

race condition

Aici nu avem operații atomice (vom discuta despre ele la laboratorul 5)

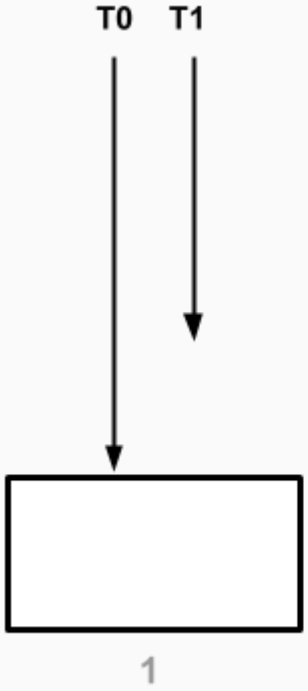
Primitive de sincronizare

- mutex
- barieră
- semafor (o să vorbim la laboratorul 4, există implementare de semafor în pthreads)
- variabile condiție (detalii la ASC și la SO)

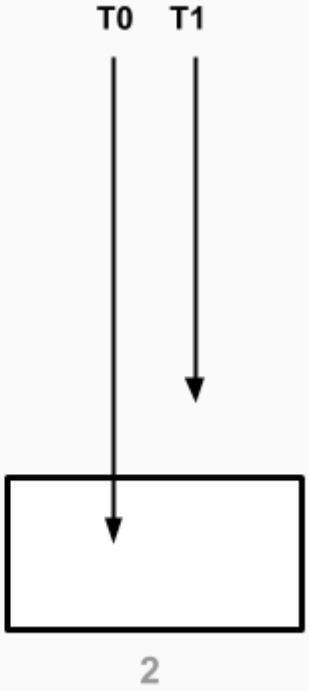
Lectură suplimentară: https://ocw.cs.pub.ro/courses/so/laboratoare/laborator-08

Mutex

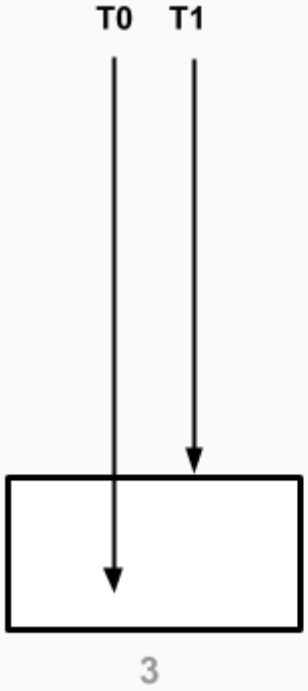
- un mutex (mutual exclusion) este folosit pentru a delimita și pentru a proteja o zonă critică, unde au loc, de regulă, operații de citire și de scriere
- un singur thread intră în zona critică (o rezervă pentru el - lock), unde se execută instrucțiuni, iar celelalte thread-uri așteaptă ca thread-ul curent să termine de executat instrucțiunile din zona critică
- după ce thread-ul curent termină de executat instrucțiuni în zona critică, aceasta o eliberează (unlock) și următorul thread urmează aceiași pași



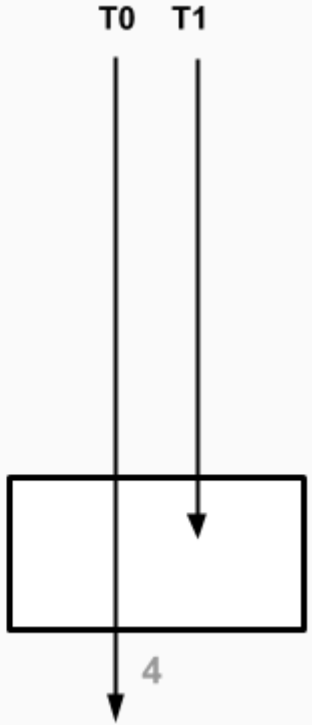
T0 face lock



T0 e în zona critică



T0 e în zona critică, T1 așteaptă



T0 iese din zona critică și face unlock, T1 intră în zona critică și face lock

Funcții

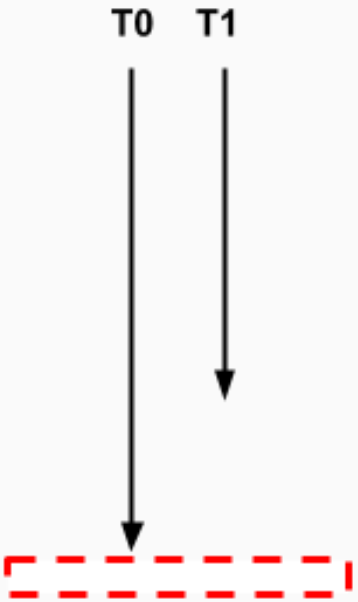
- crearea unui mutex: `int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);`
- lock pe mutex: `int pthread_mutex_lock(pthread_mutex_t *mutex);`
- unlock pe mutex: `int pthread_mutex_unlock(pthread_mutex_t *mutex);`
- distrugerea unui mutex: `int pthread_mutex_destroy(pthread_mutex_t *mutex);`

Folosire mutex

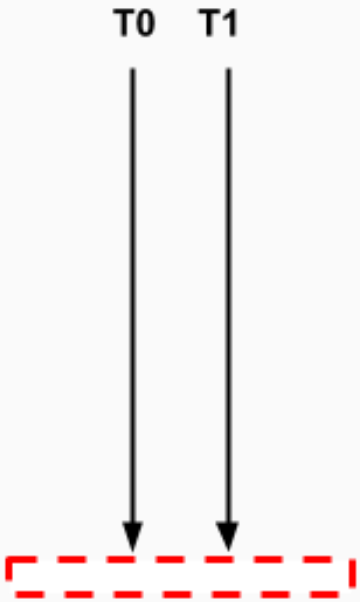
```
pthread_mutex_t mutex; // variabila globala
pthread_mutex_init(&mutex, NULL); // in main, inainte de pthread_create

void* f(void* arg) {
    pthread_mutex_lock(&mutex);
    [zona critica]
    pthread_mutex_unlock(&mutex);
}

pthread_mutex_destroy(&mutex); // in main, dupa pthread_join
```

Barieră

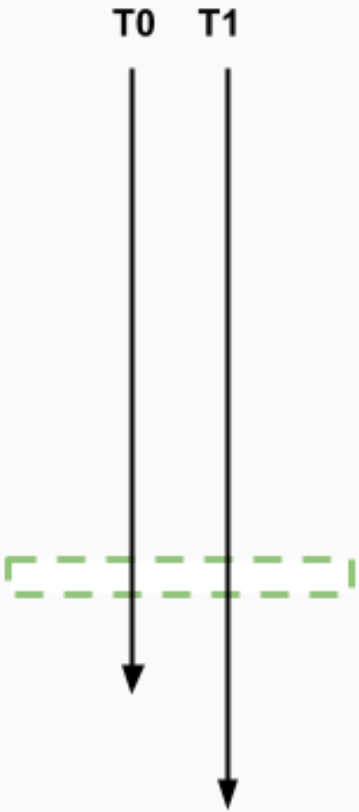
- bariera este folosită atunci când dorim să sincronizăm thread-urile încât să ajungă (să se sincronizeze) în același punct
- mai concret, ea asigură faptul că niciun thread, gestionat de barieră, nu trece mai departe de zona în care aceasta este amplasată decât atunci când toate thread-urile gestionate de barieră ajung în aceeași zonă



Bariera blocată, T0 e la barieră, se așteaptă T1



Bariera blocată, T1 ajunge la barieră



Bariera deblocată, thread-urile își continuă execuția

Funcții

- crearea unei bariere: `int pthread_barrier_init(pthread_barrier_t *barrier, const pthread_barrierattr_t *attr, unsigned count);`
- așteptarea thread-urilor la barieră: `int pthread_barrier_wait(pthread_barrier_t *barrier);`
- distrugerea unei bariere: `int pthread_barrier_destroy(pthread_barrier_t *barrier);`

Folosire barieră

```
pthread_barrier_t barrier; // variabila globala
pthread_barrier_init(&barrier, NULL, 2); // in main, inainte de pthread_create

void* f(void* arg) {
    [...]
    pthread_barrier_wait(&barrier);
    [...]
}

pthread_barrier_destroy(&barrier); // in main, dupa pthread_join
```

Înmulțirea matricelor - versiunea naivă

```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        for (k = 0; k < N; k++) {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

Înmulțirea matricelor - Strassen

- de paralelizat folosind 7 threads, câte un thread pentru fiecare matrice M

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$
$$M_2 = (A_{21} + A_{22})B_{11}$$
$$M_3 = A_{11}(B_{12} - B_{22})$$
$$M_4 = A_{22}(B_{21} - B_{11})$$
$$M_5 = (A_{11} + A_{12})B_{22}$$
$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$
$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = M_1 + M_4 - M_5 + M_7$$
$$C_{12} = M_3 + M_5$$
$$C_{21} = M_2 + M_4$$
$$C_{22} = M_1 - M_2 + M3 + M_6$$

Resurse utile

- Scripturi pentru testat cod paralelizat la laborator: https://gist.github.com/florinrm/c5edbff84b8a7f94423e2065b870235e

Întrebări (potențiale de interviu)

- Cum se poate delimita o zonă critică?
- Ce face o barieră?
- Care este complexitatea înmulțirii a două matrici în paralel?
- Ce poate cauza un comportament nedeterminist a unui program paralel?
- Dați un exemplu de caz de race condition și cum se ajunge la race condition.

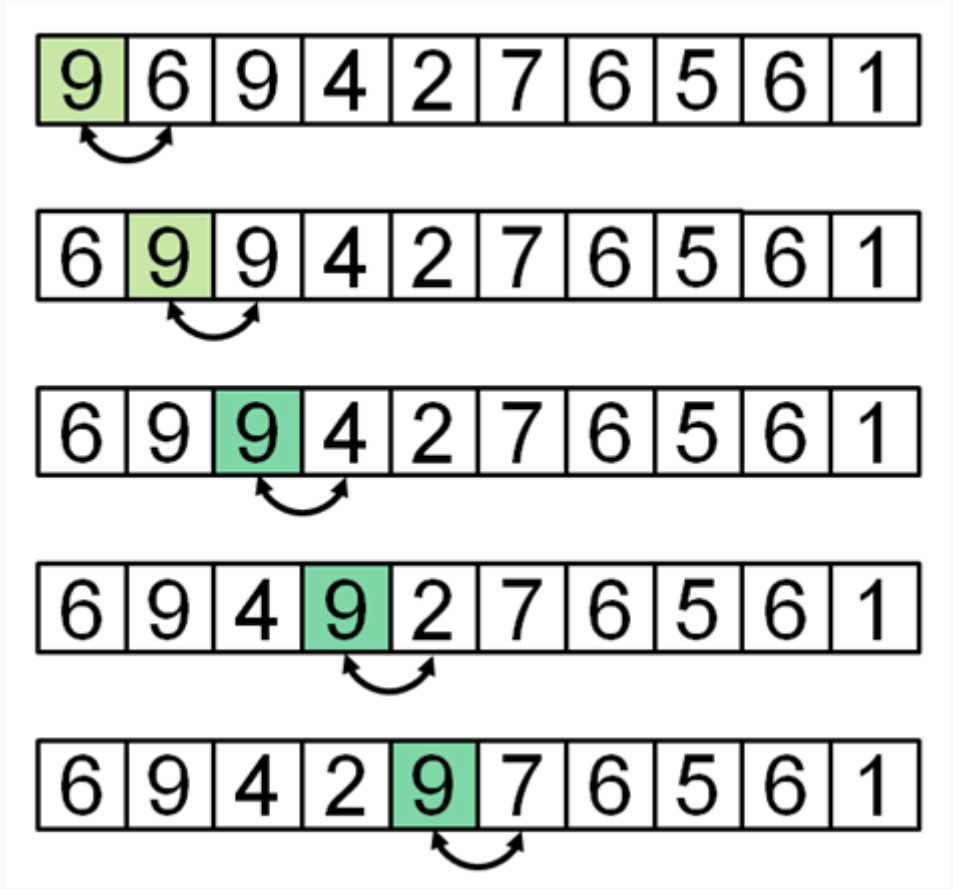
Laboratorul 3
Algoritmi paraleli - sortări

Bubble Sort

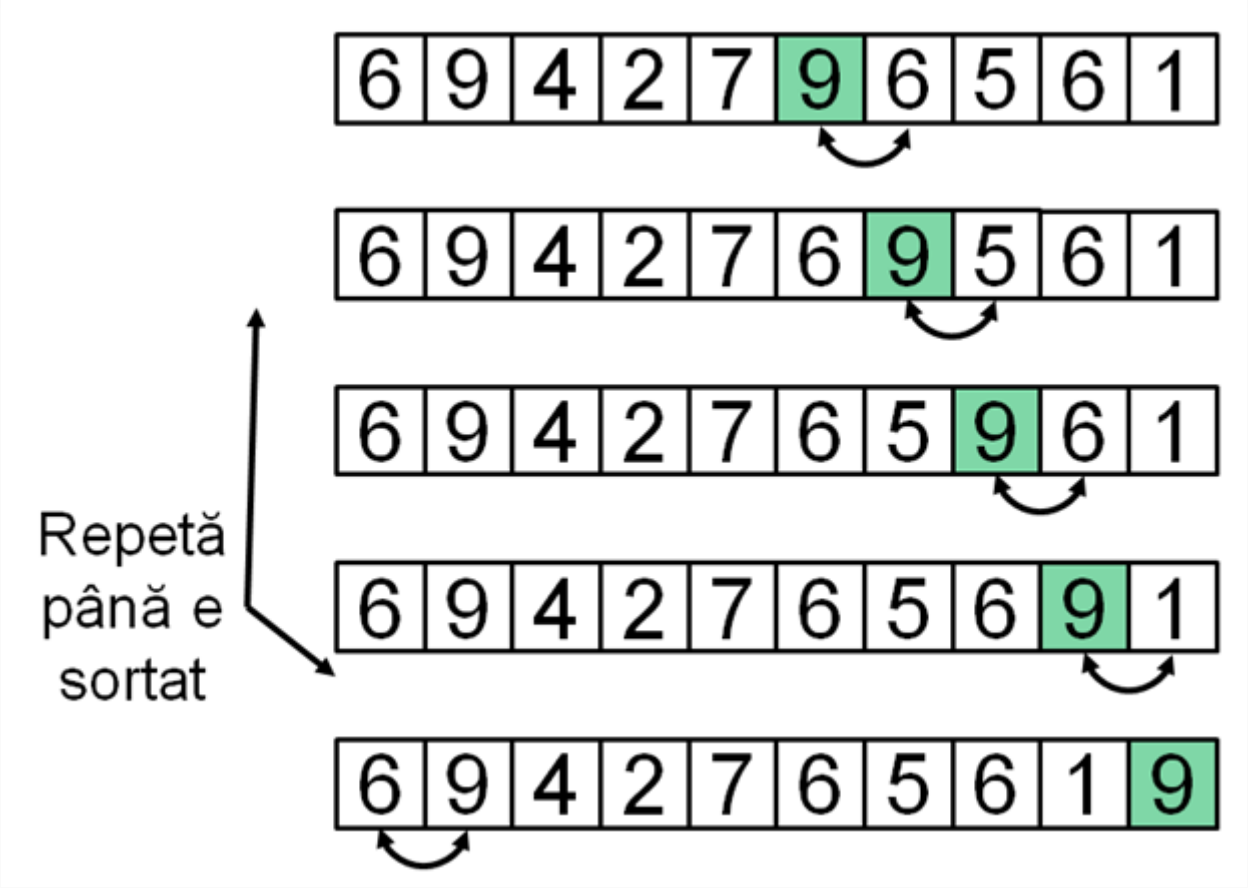
```
function bubbleSort(list) {
  sorted = false;
  while (!sorted) {
    sorted = true;
    for (var i = 0; i < list.length - 1; i++) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
        sorted = false;
      }
    }
  }
}
```

Complexitate temporală: $O(N^2)$, având maxim N parcurgeri ale șirului, cu dimensiune N

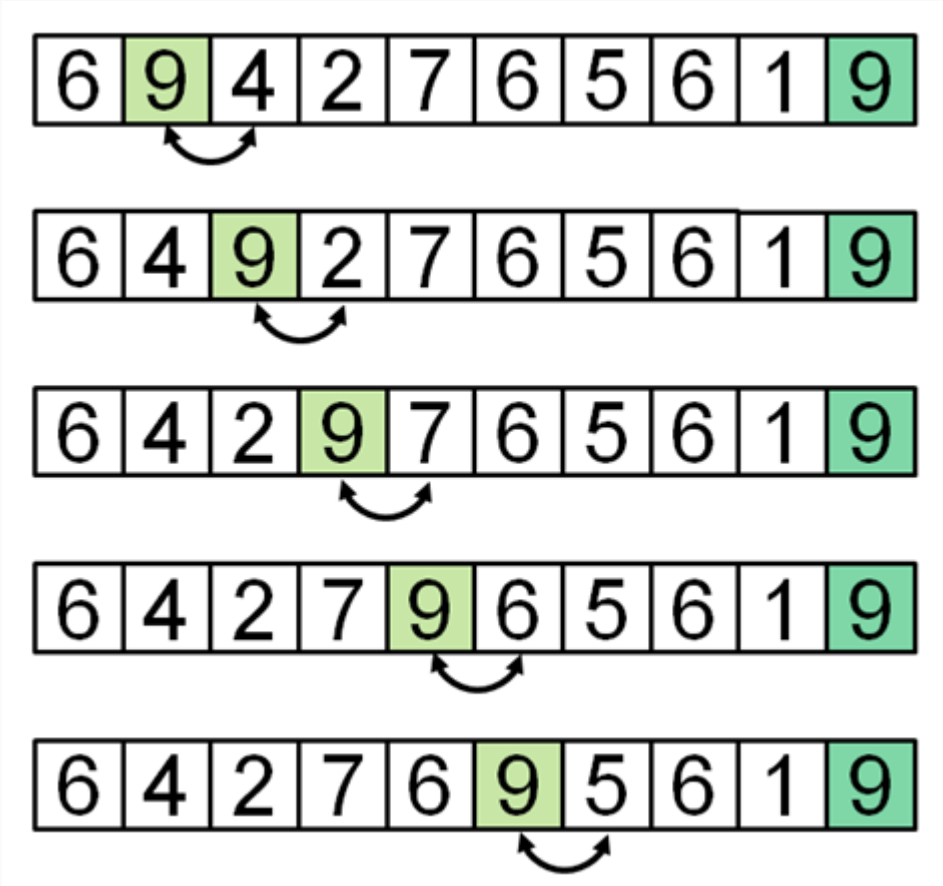
Bubble Sort - rulare



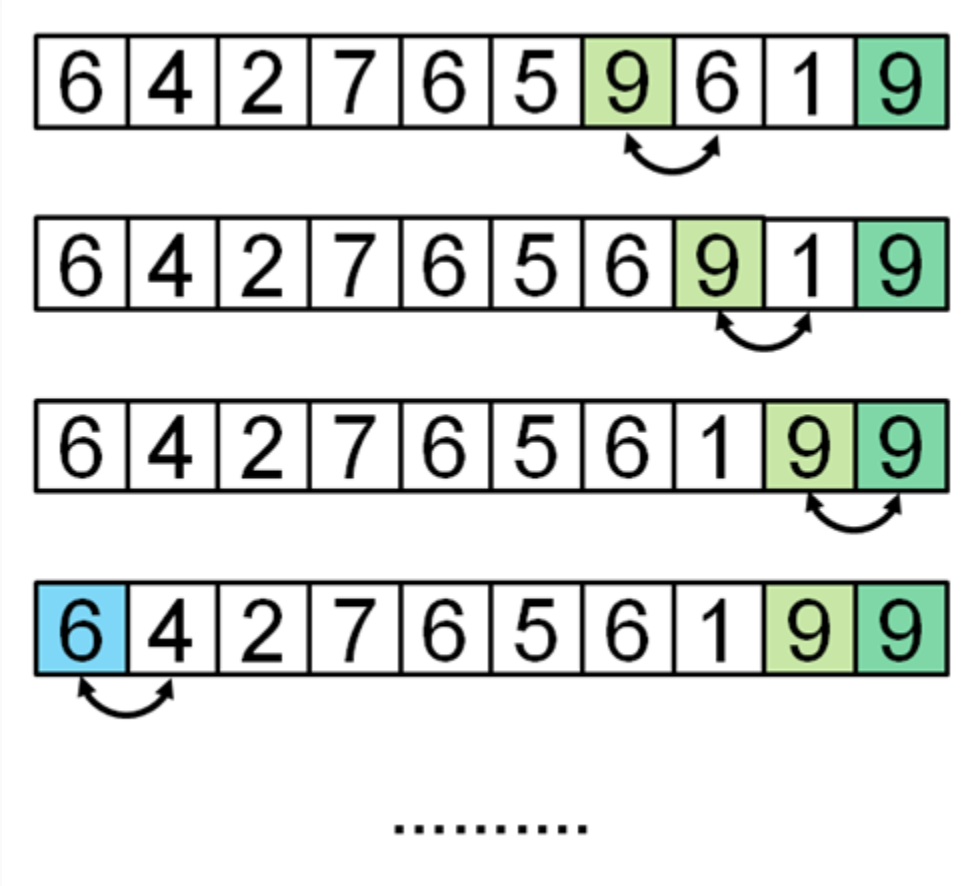
Bubble Sort - rulare



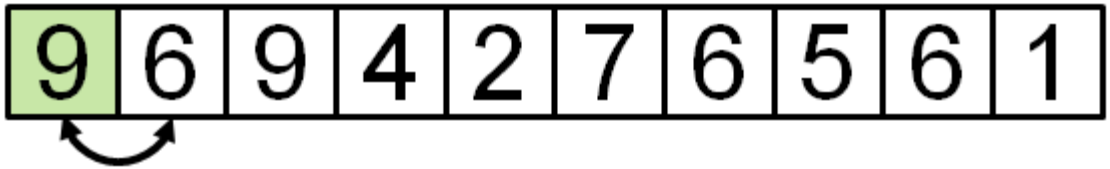
Bubble Sort - rulare



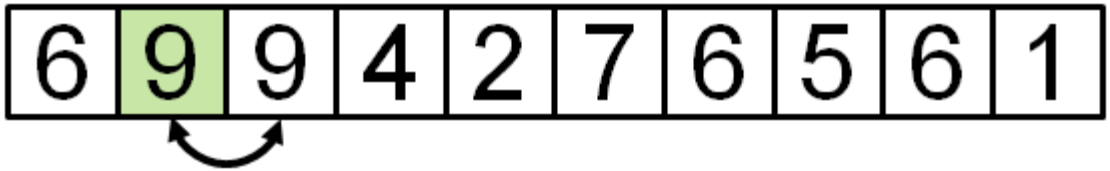
Bubble Sort - rulare



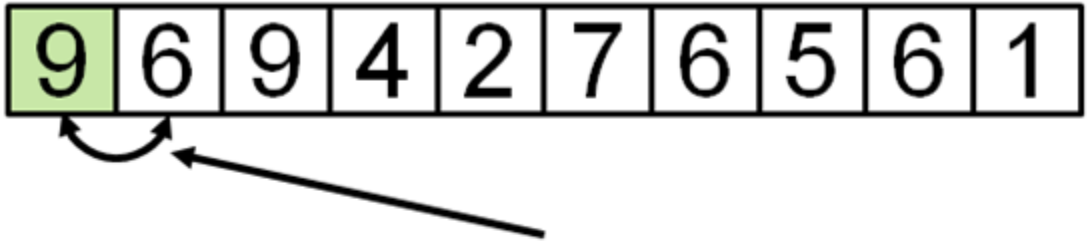
Bubble Sort - tentativă de paralelizare



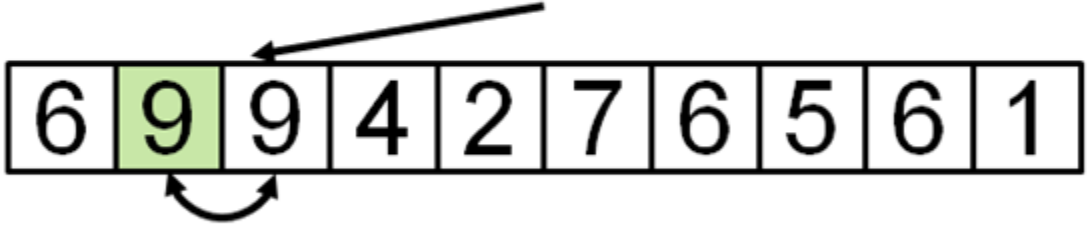
Cum paralelizăm?



Bubble Sort - tentativă de paralelizare



Aceste două operații (și toate perechile similare)
NU pot fi executate în același timp



Solution hint: Nu e necesară execuția operațiilor într-o anumită ordine

Odd-Even Transposition Sort - Bubble Sort paralelizat

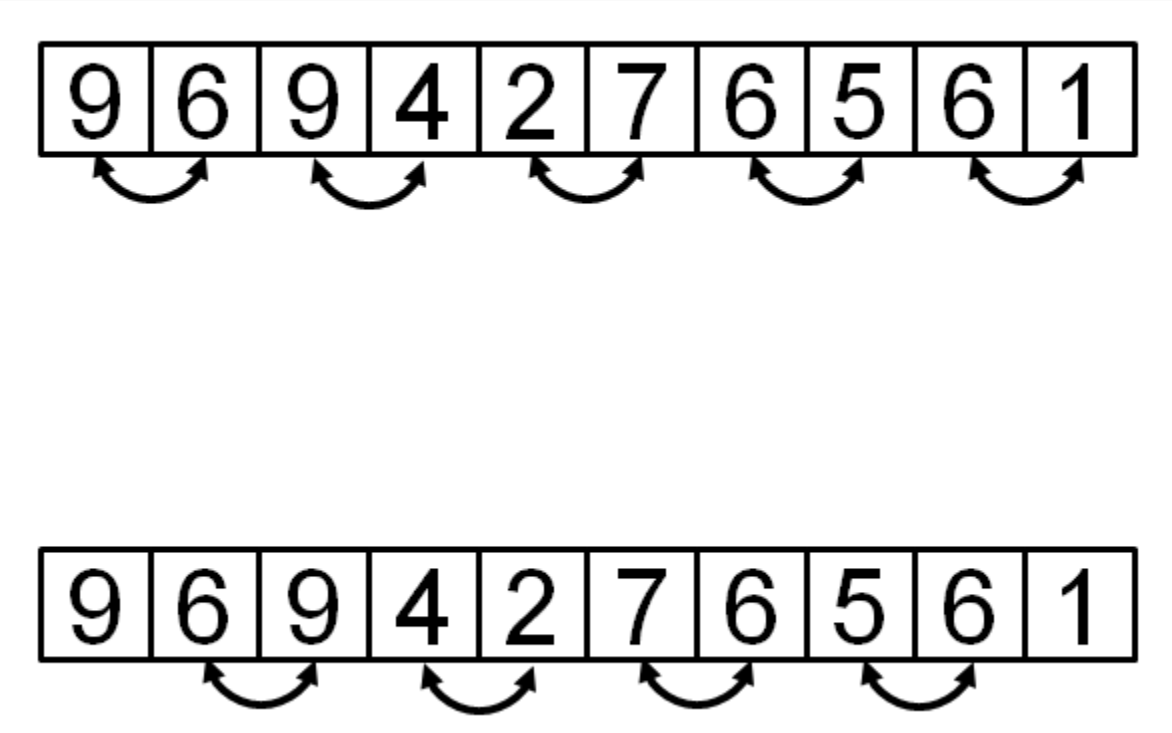
```
function oddEvenSort(list) {
  for (var k = 0; k < list.length; k++) {
    for (i = 0; i < list.length - 1; i += 2) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
      }
    }
    for (i = 1; i < list.length - 1; i += 2) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
        sorted = false;
      }
    }
  }
}
```

Această versiune paralelizată de Bubble Sort funcționează în două faze:

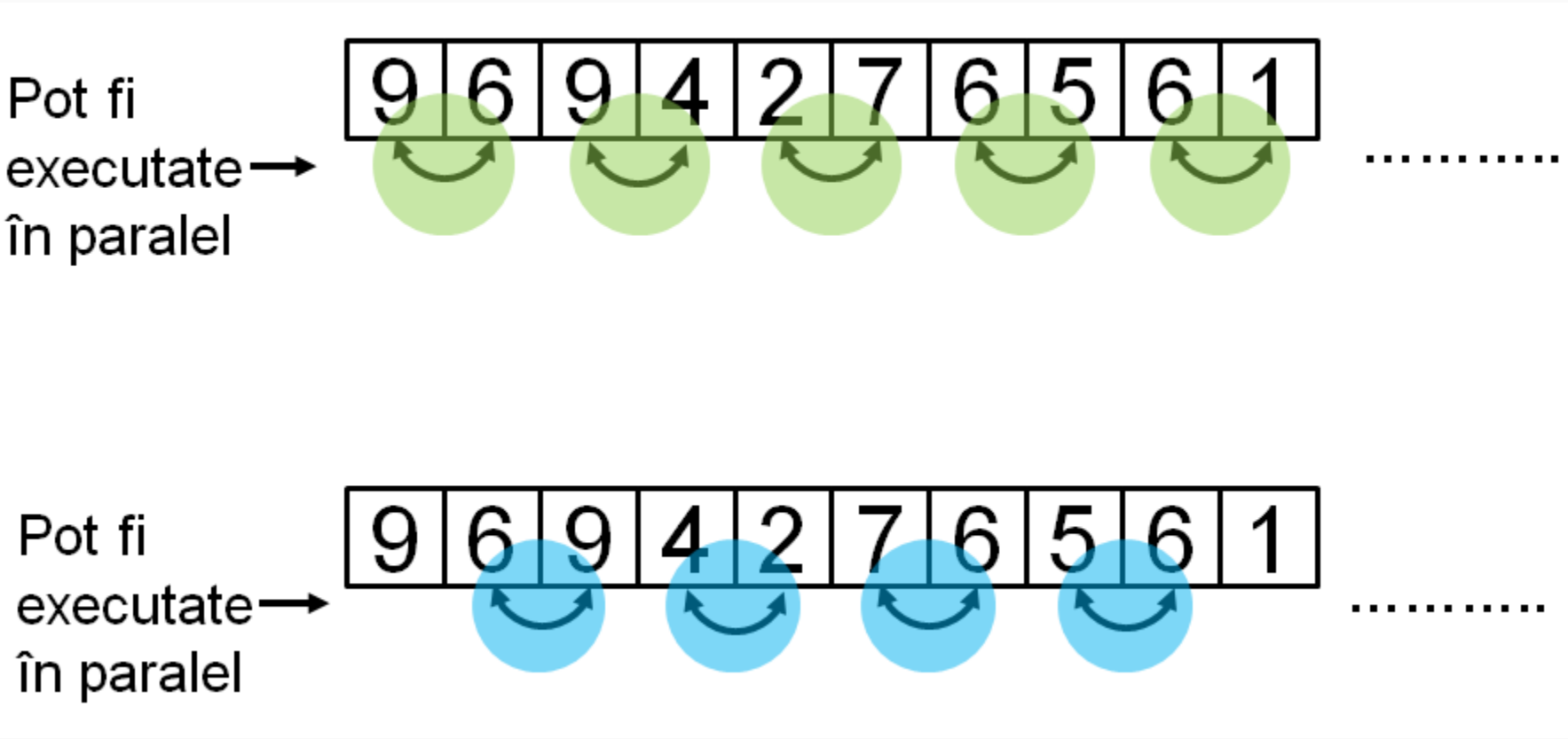
- faza pară - elementele de pe pozițiile **pare** sunt comparate cu vecinii de pe poziția următoare (i cu i + 1) și eventual interschimbate
- faza impară - ea începe după terminarea fazei impare. Elementele de pe pozițiile **impare** sunt comparate cu vecinii de pe poziția următoare (i cu i + 1) și eventual interschimbate

Complexitate temporală: $O(N^2 / P)$, P = numărul de thread-uri

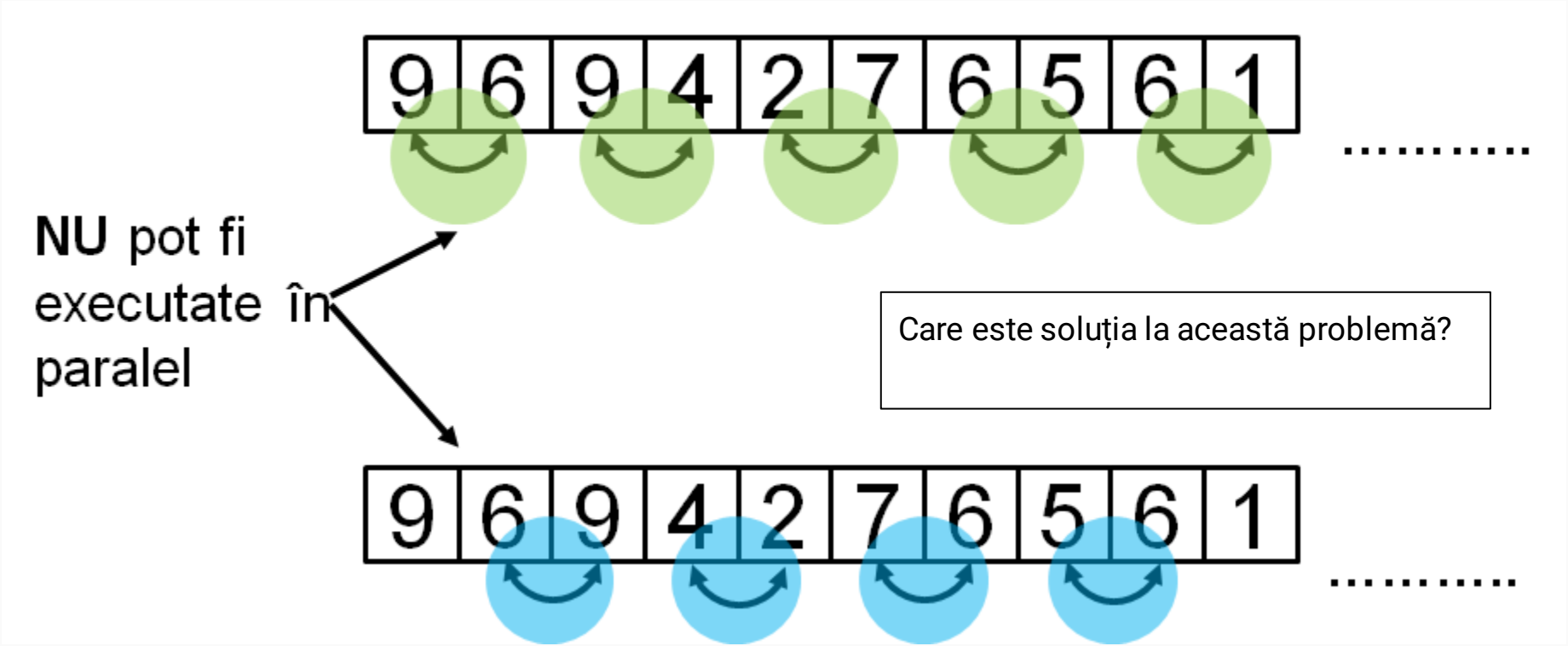
Odd-Even Transposition Sort - mod de funcționare



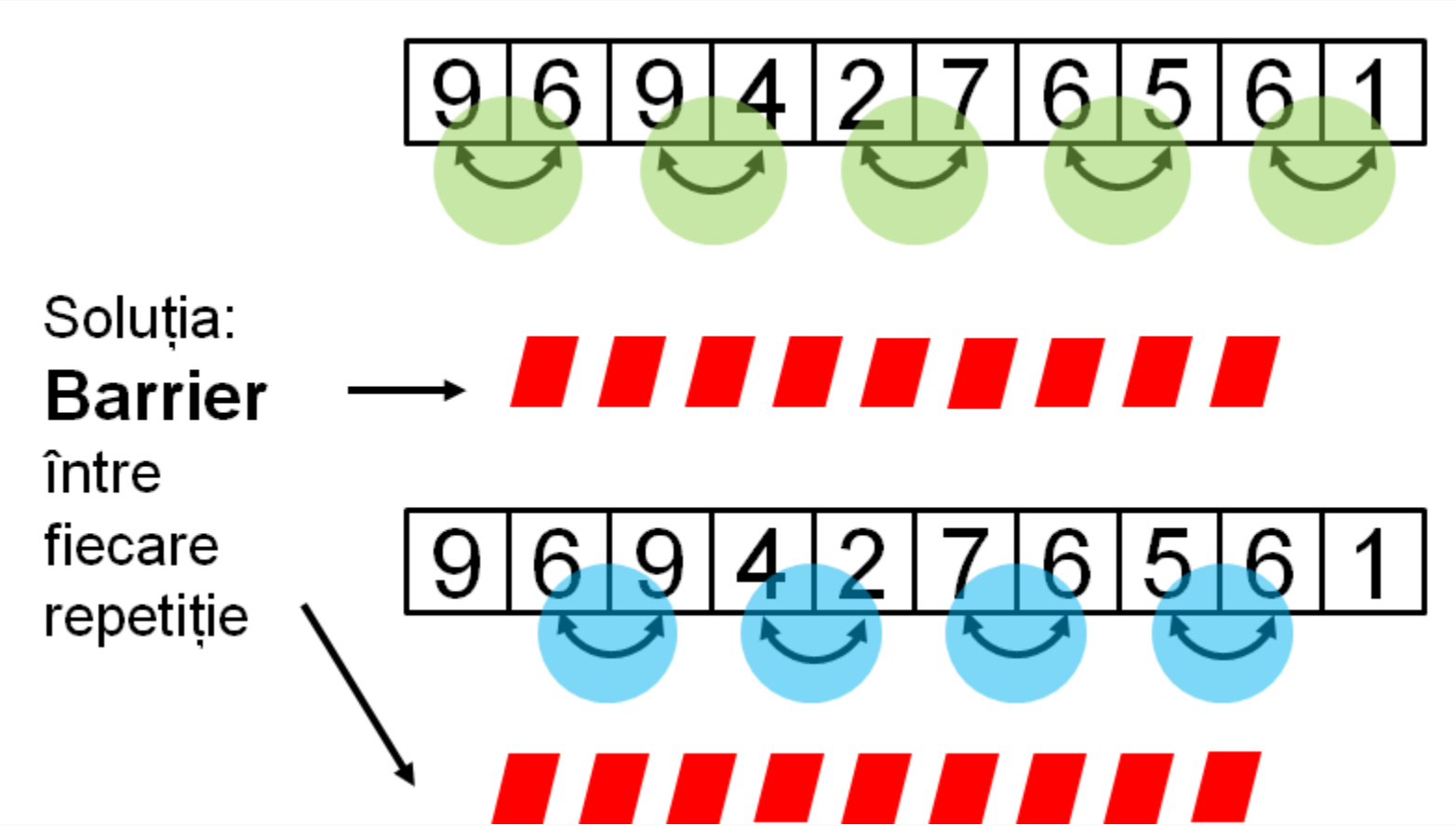
Odd-Even Transposition Sort - mod de funcționare paralelă



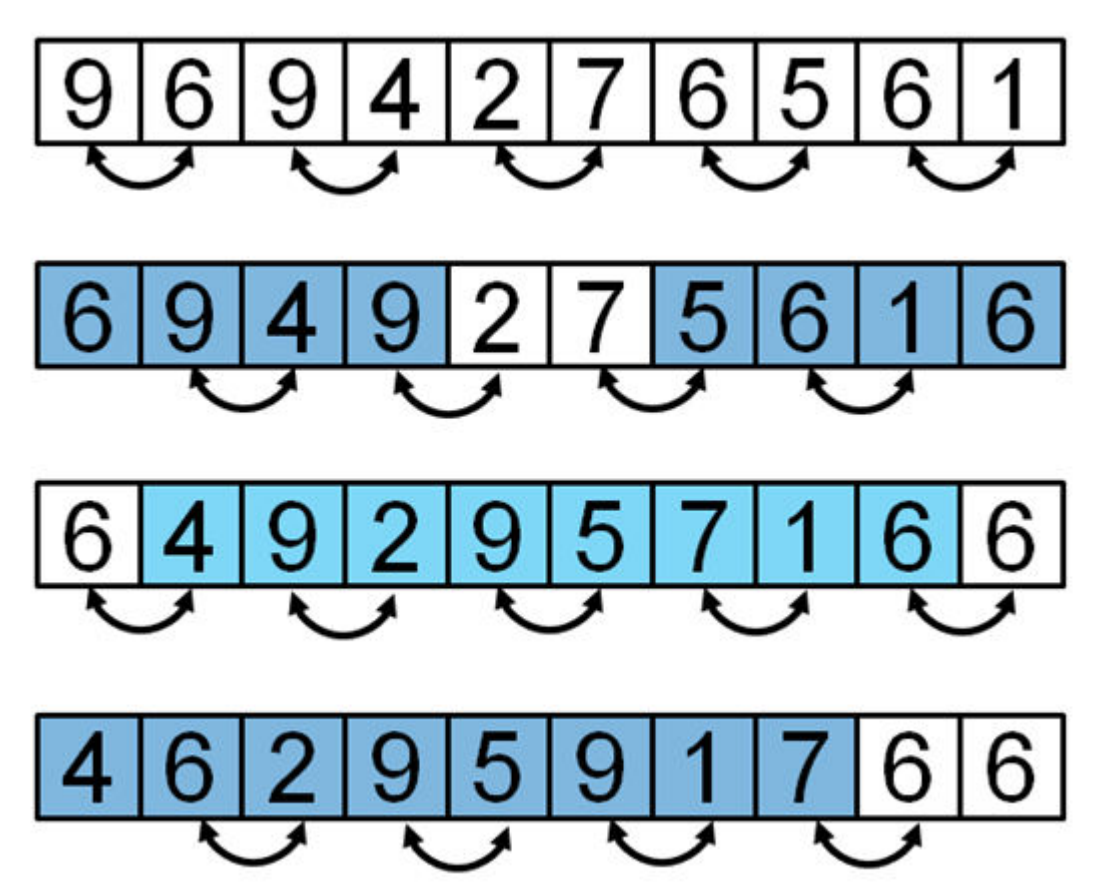
Odd-Even Transposition Sort - probleme de sincronizare



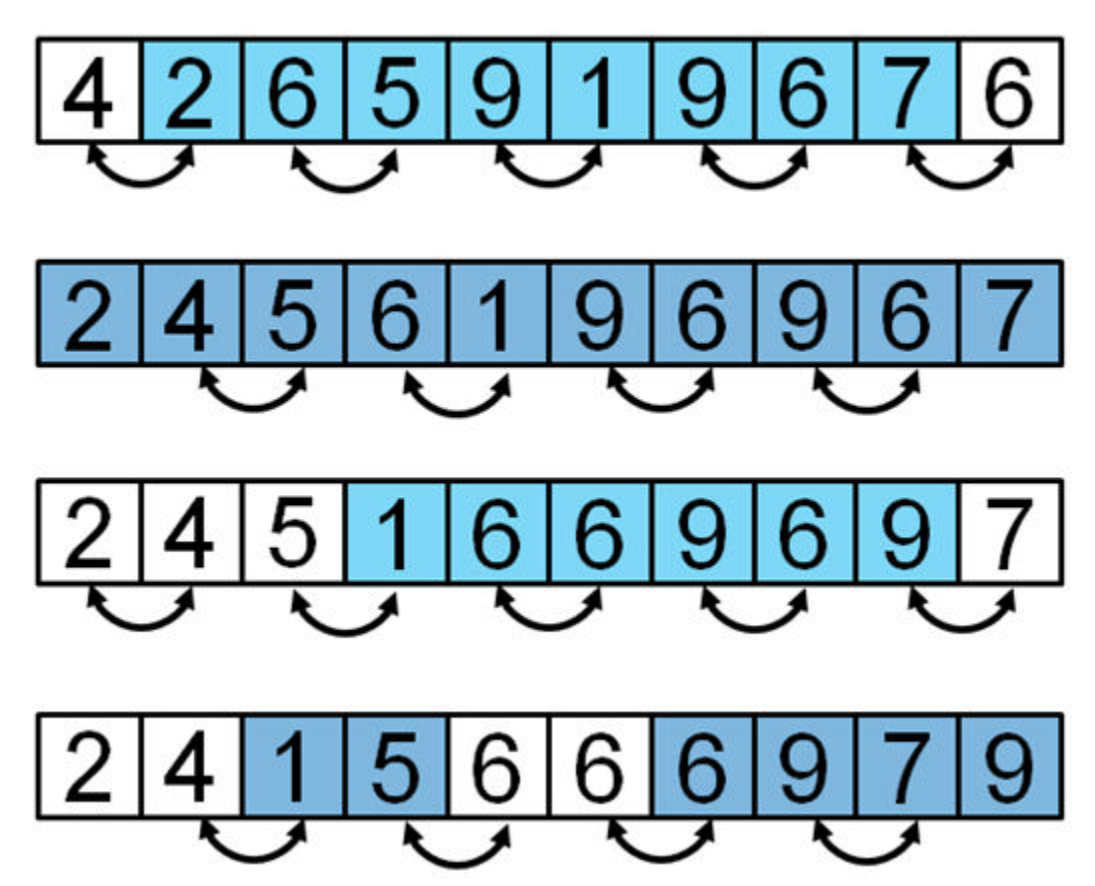
Odd-Even Transposition Sort - probleme de sincronizare - soluție



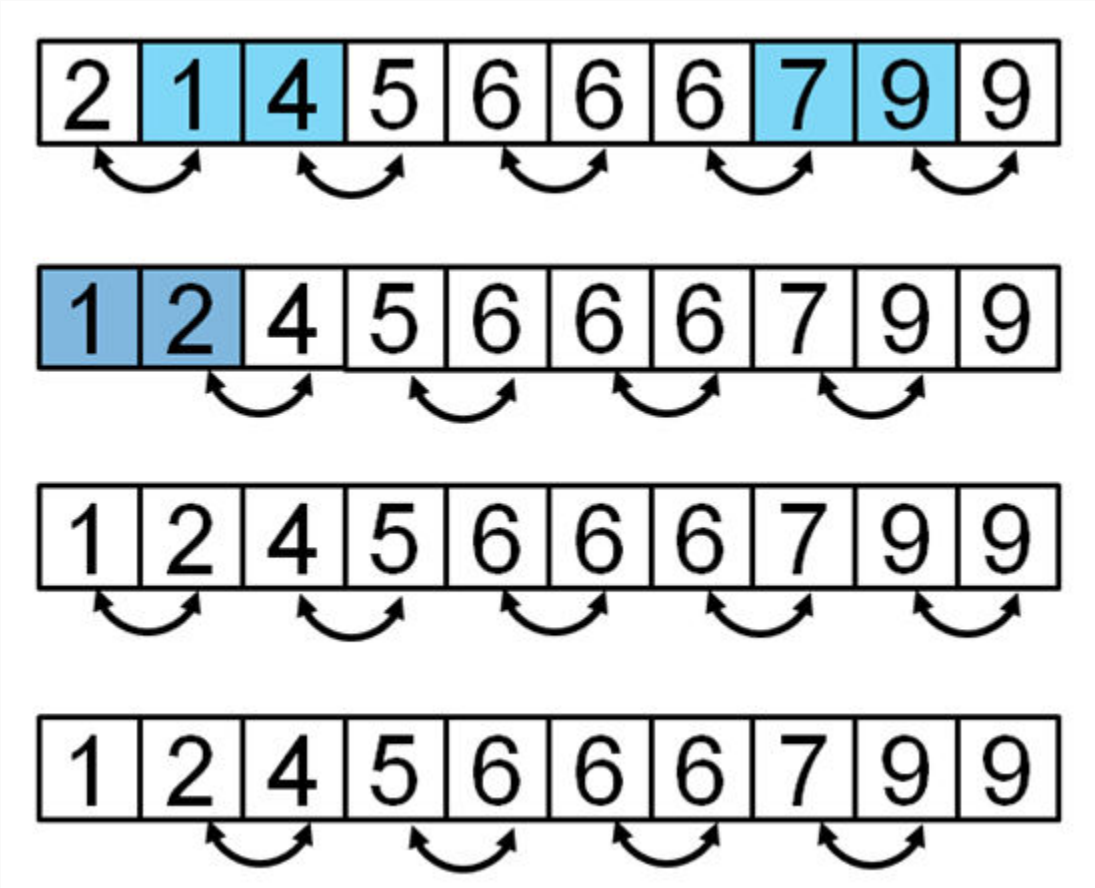
Rulare Odd-Even Transposition Sort



Rulare Odd-Even Transposition Sort



Rulare Odd-Even Transposition Sort



Paralelizare Odd-Even Transposition Sort

- **Atenție**: fiți atenți la indicele de start, mai exact cum îl folosiți în ambele faze ale algoritmului paralelizat, și la end (legat de faptul că ambele faze au for-ul `< N - 1`, N fiind dimensiunea array-ului => `i < min(end, N - 1)`)
- start impar => faza pară începe de la start + 1, faza impară de la start
- start par => faza pară începe de la start, faza impară de la start + 1

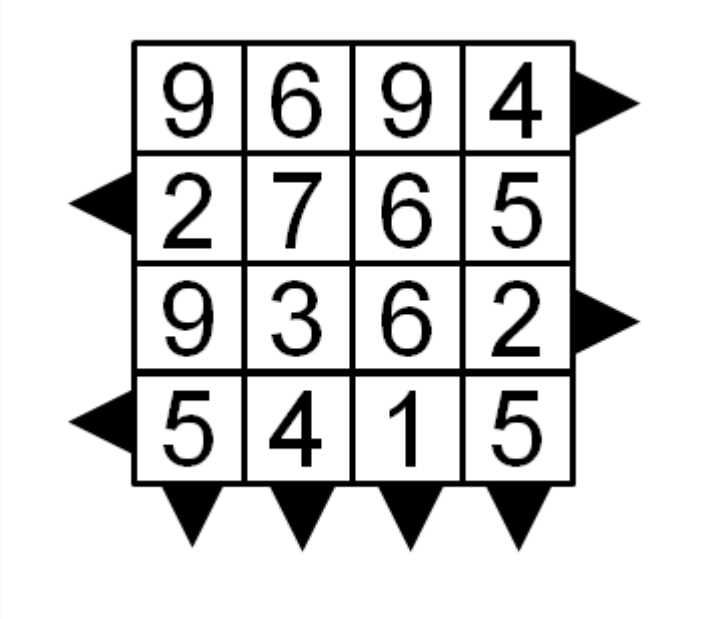
Row Column Sort (Snake Sort / Shear Sort)

- Shear Sort este un algoritm de sortare, conceput special pentru sisteme multi-procesor.
- Acest algoritm funcționează în două faze:
  - se sortează liniile matricei astfel încât randurile pare au valorile ordonate crescător, iar rândurile impare au valorile ordonate descrescător, ca să putem compara cel mai mare element de pe linia i cu cel mai mic de pe linia i + 1
  - se sortează coloanele crescător
- Elementele sunt sortate în maxim $\sup(\log_2 N) + 1$ faze, unde N = numărul de elemente
- Complexitatea: $O(N * \log_2 N)$

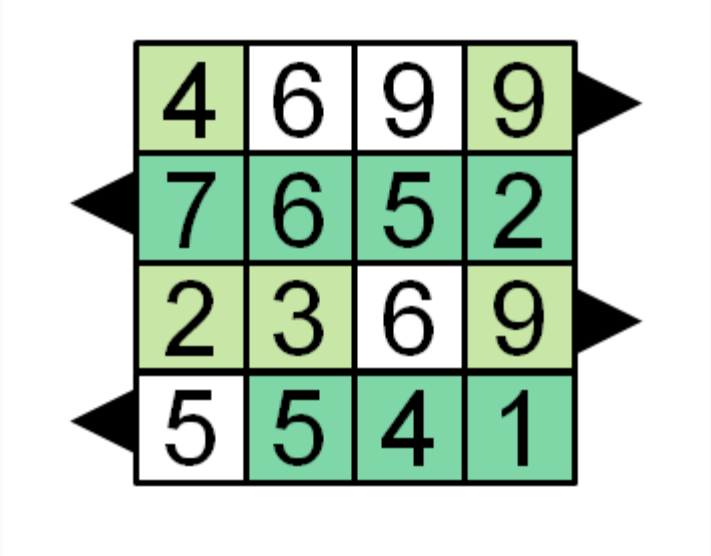
Shear Sort - pseudocod

```
function shearSort(matrix) {
  for (k = 0; k < ceil(log2(matrix.lines * matrix.columns)) + 1; k += 2) {
    for (i = 0; i < matrix.lines; i += 2) {
      sortAscendingLine(i);
    }
    for (i = 1; i < matrix.lines; i += 2) {
      sortDescendingLine(i);
    }
    for (var i = 0; i < matrix.columns; i++) {
      sortAscendingColumn(i);
    }
  }
}
```

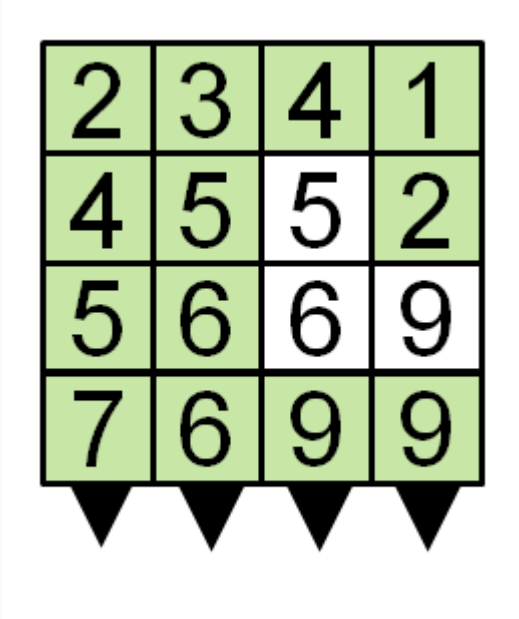
Rulare Shear Short - configurație inițială - sortăm liniile



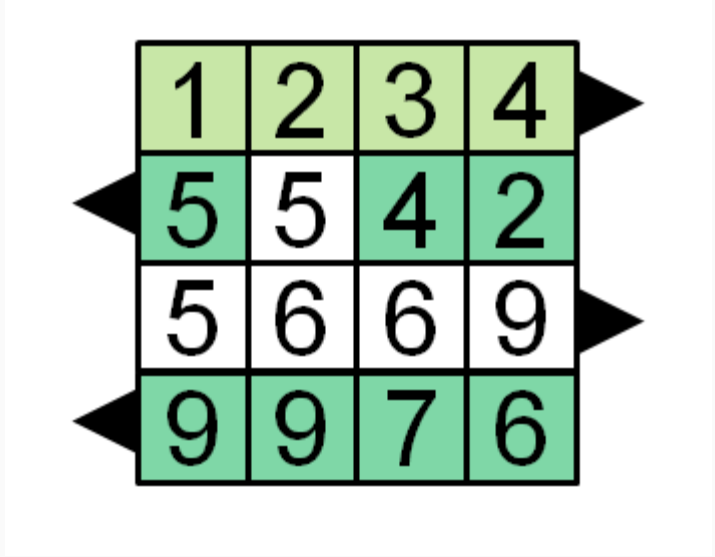
Rulare Shear Short - sortăm coloanele



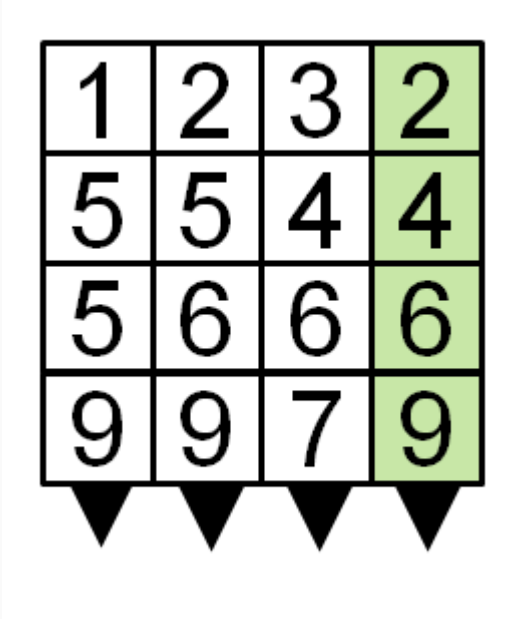
Rulare Shear Short - sortăm liniile



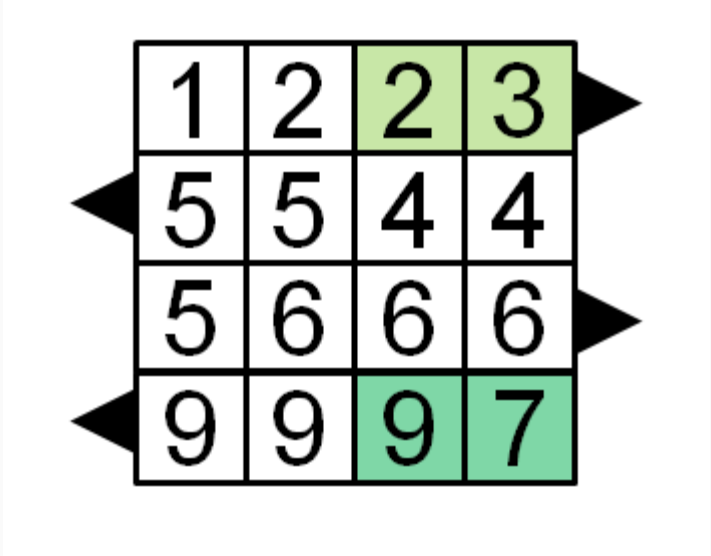
Rulare Shear Short - sortăm coloanele



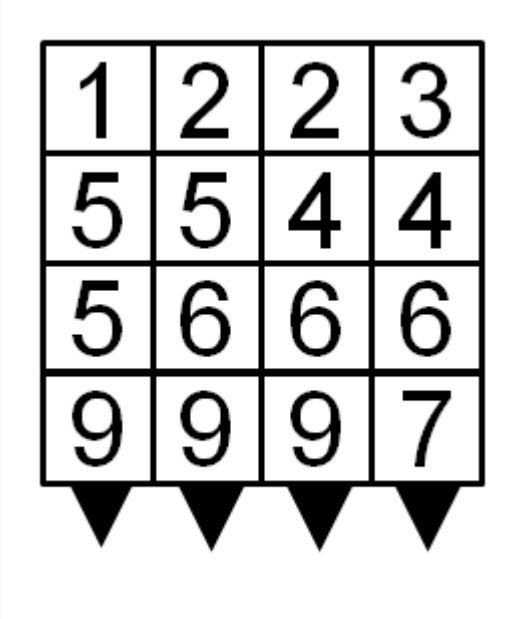
Rulare Shear Short - sortăm liniile



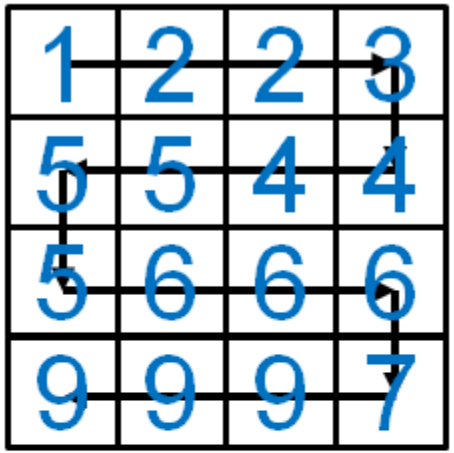
Rulare Shear Short - sortăm coloanele



Rulare Shear Short - gata - linii sortate, coloane sortate



Rulare Shear Short - rezultatul final

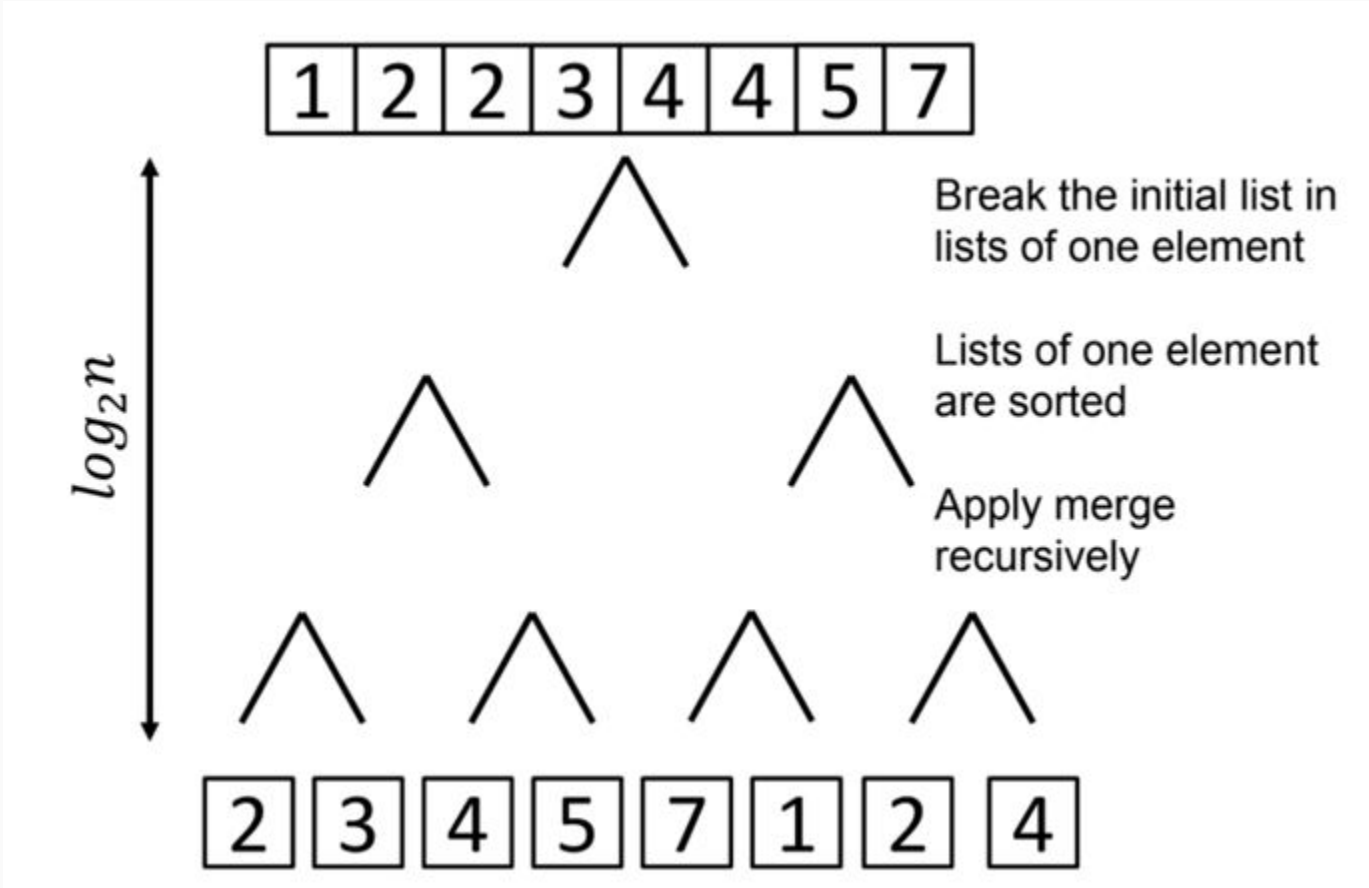


1	2	2	3	4	4	5	5	5	6	6	6	7	9	9	9

Merge Sort

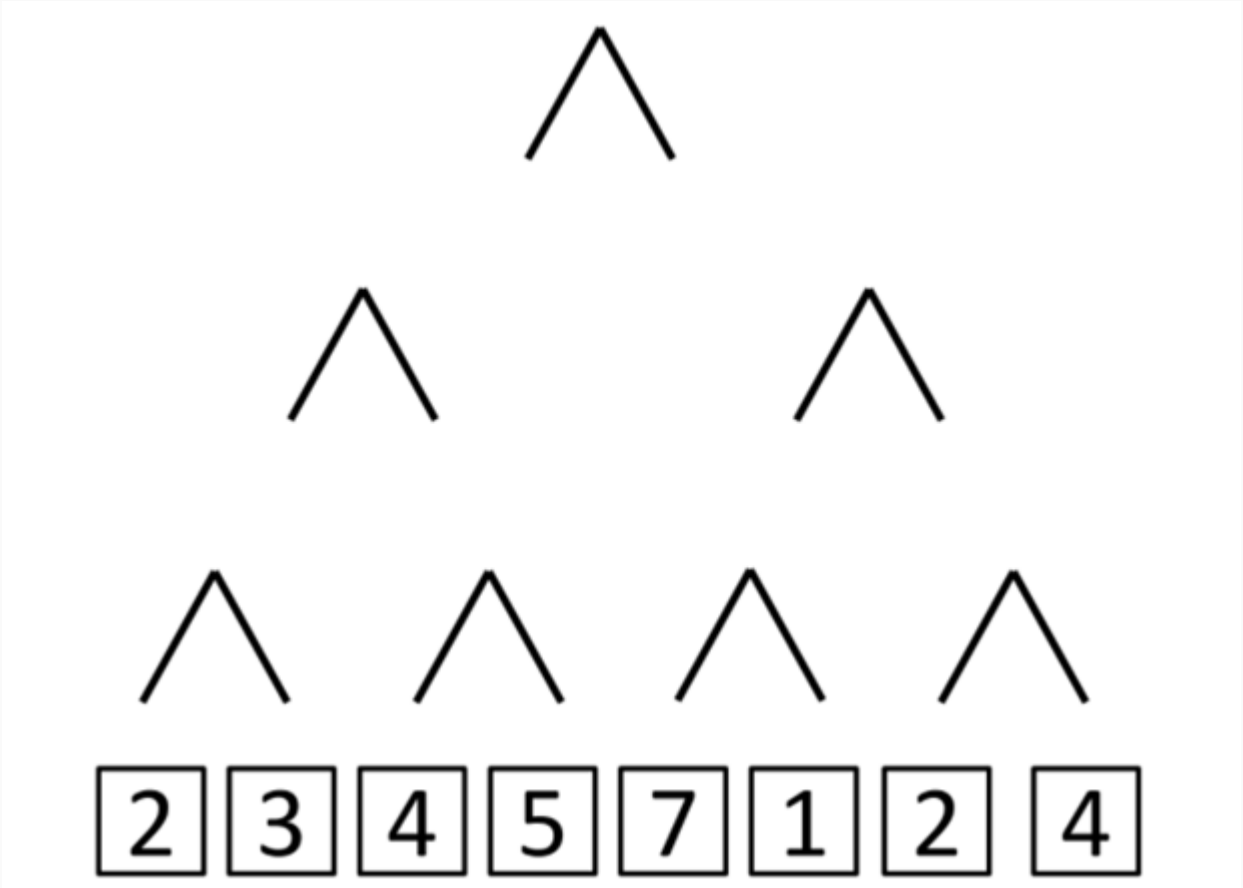
- Merge Sort este un algoritm de sortare, bazat pe tehnica divide et impera, care este format din următorii pași:
  - se împarte șirul de N elemente de sortat în N șiruri de lungime 1
  - se aplica operația de interclasare („merge”) între câte două astfel de șiruri de lungime 1, rezultând N/2 șiruri sortate de lungime 2
  - se repetă pașii de mai sus realizând interclasări între șiruri din ce în ce mai mari, până se ajunge la un șir sortat de N elemente.

Parallel Merge Sort

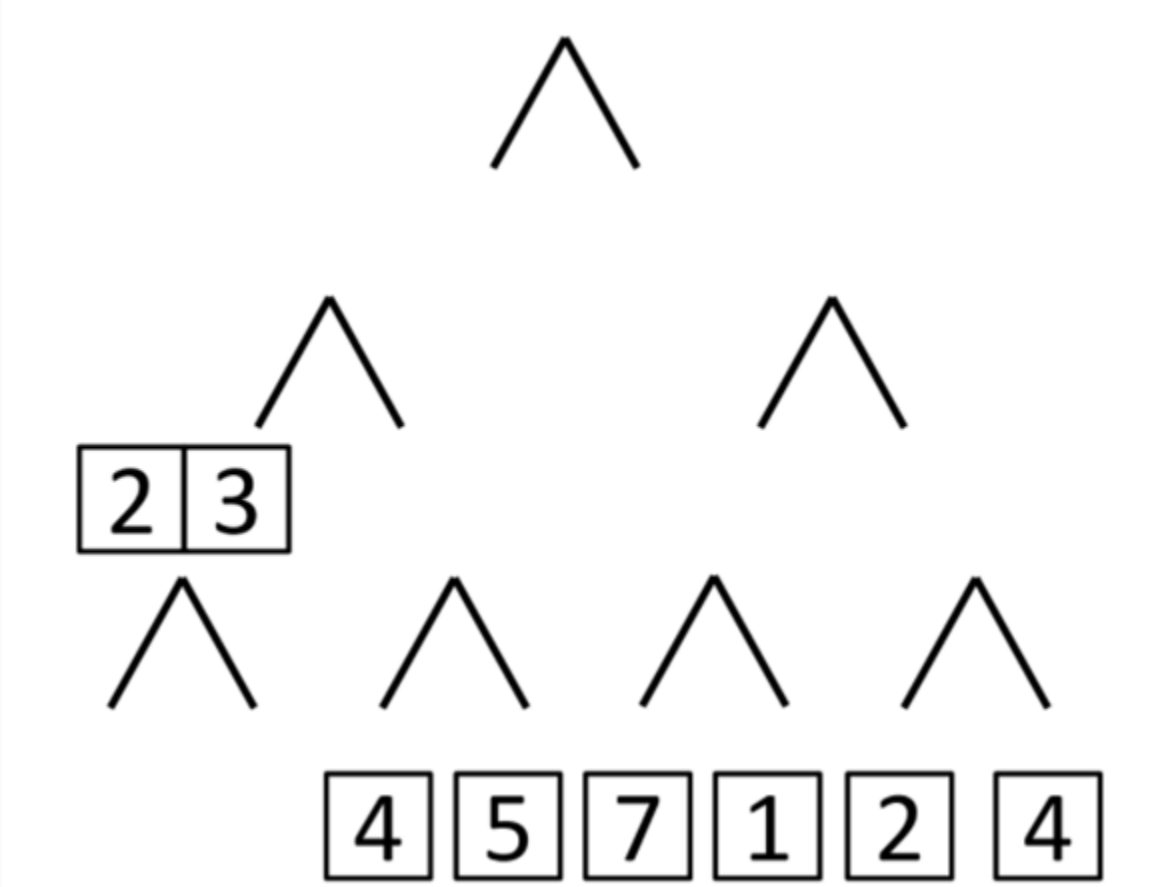


Complexitate temporală (N = P): O(N)

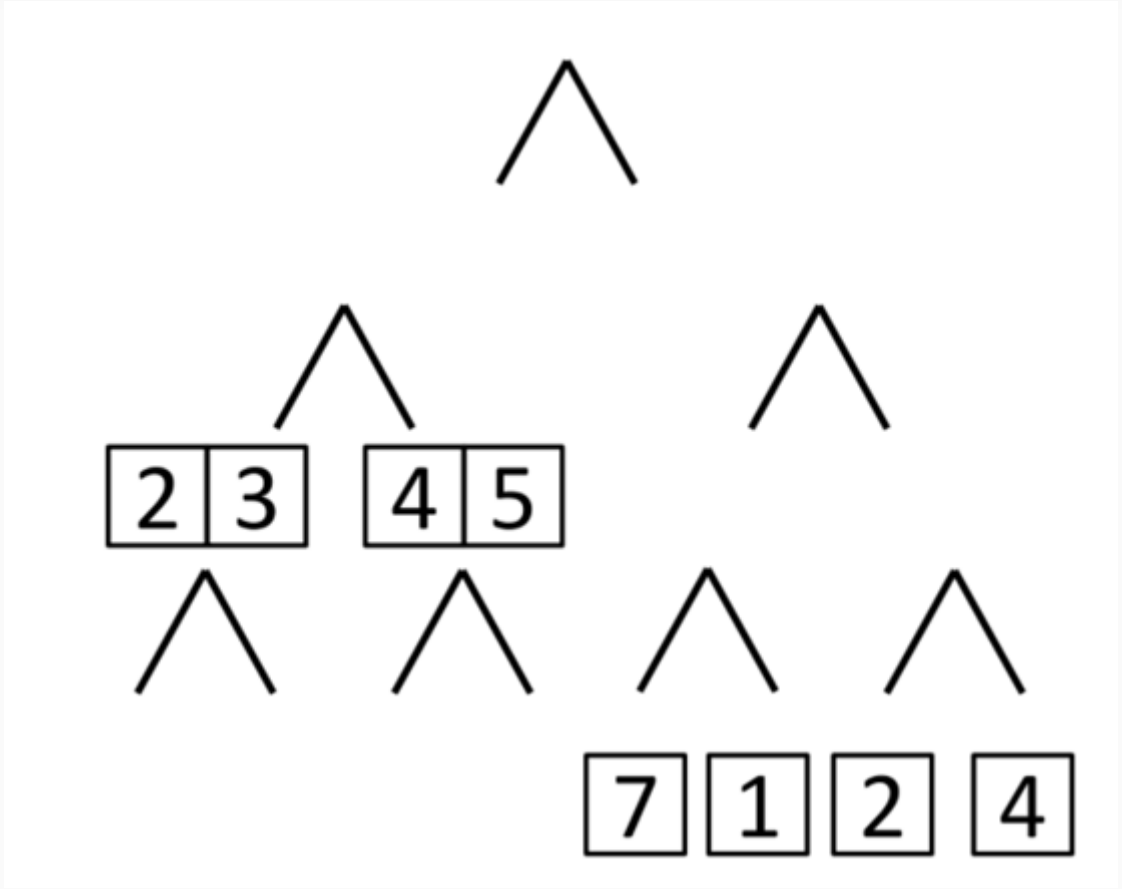
Parallel Merge Sort



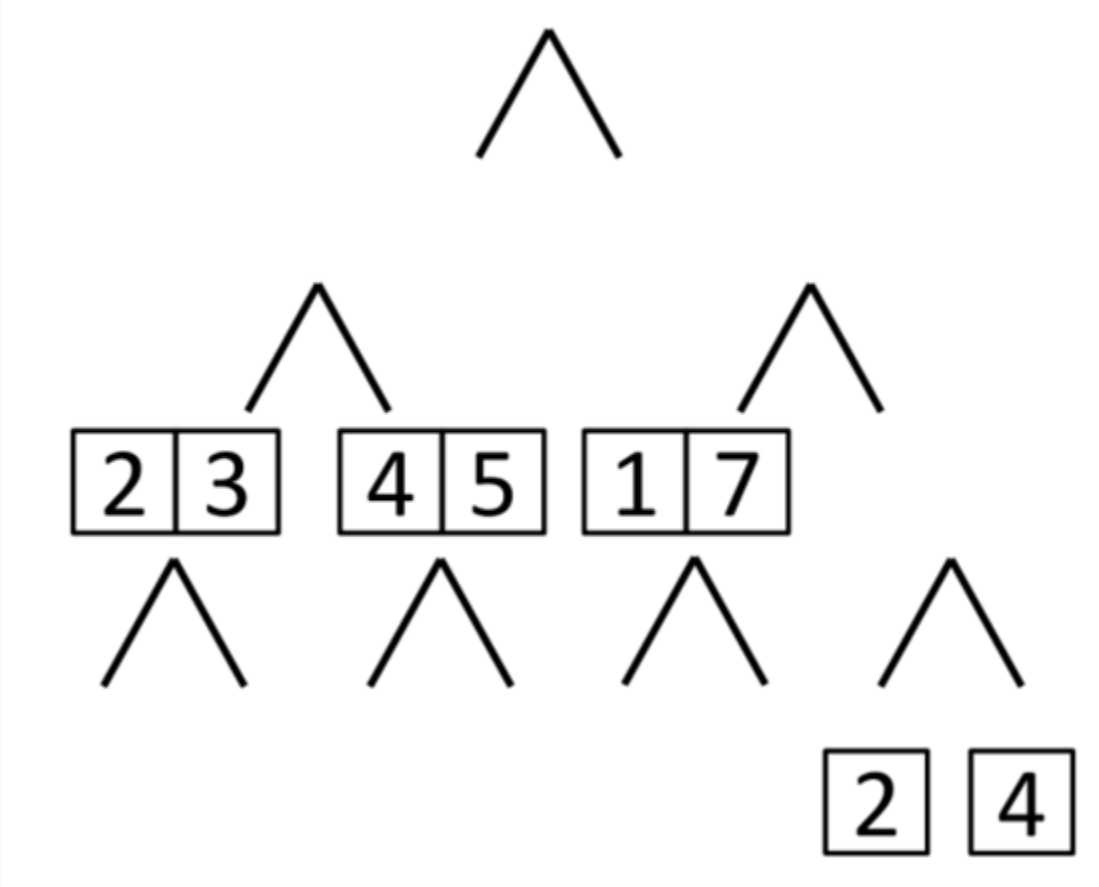
Parallel Merge Sort



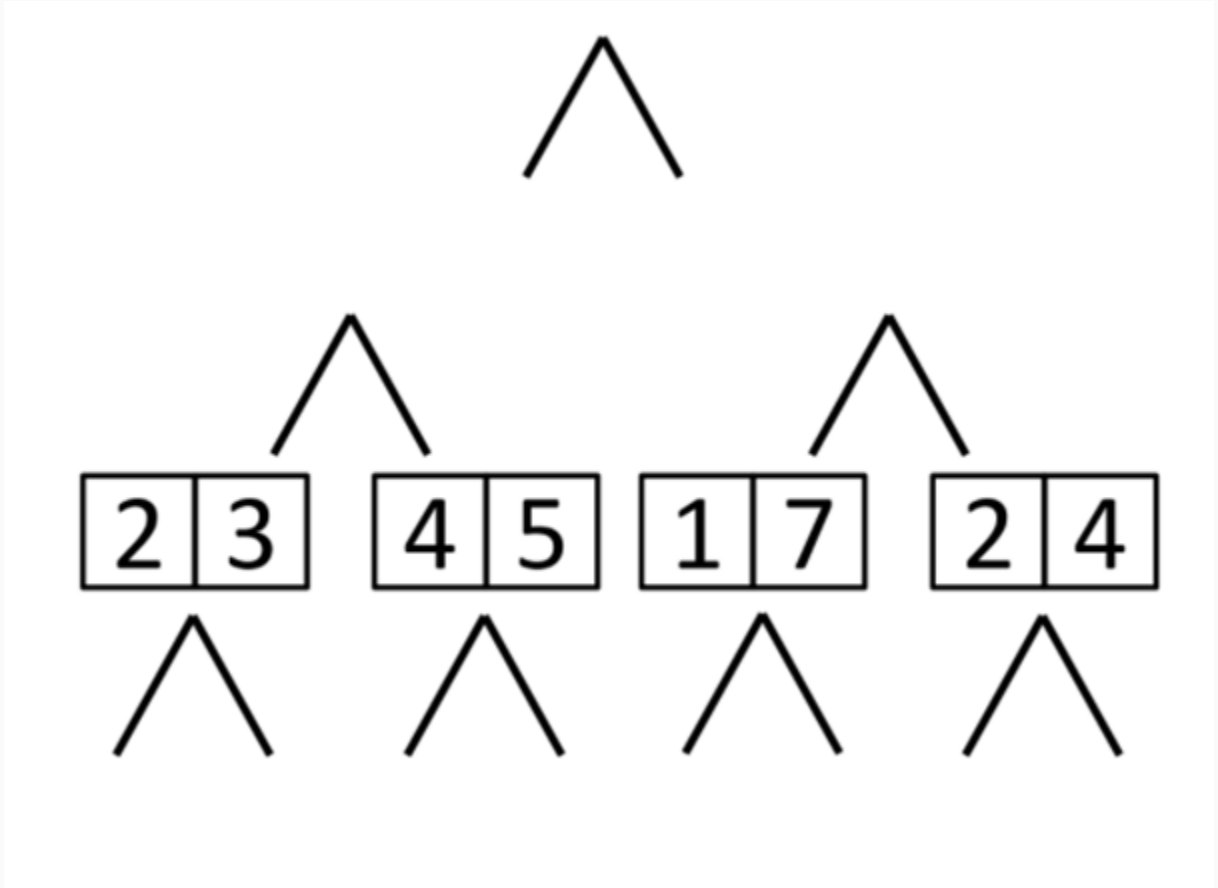
Parallel Merge Sort



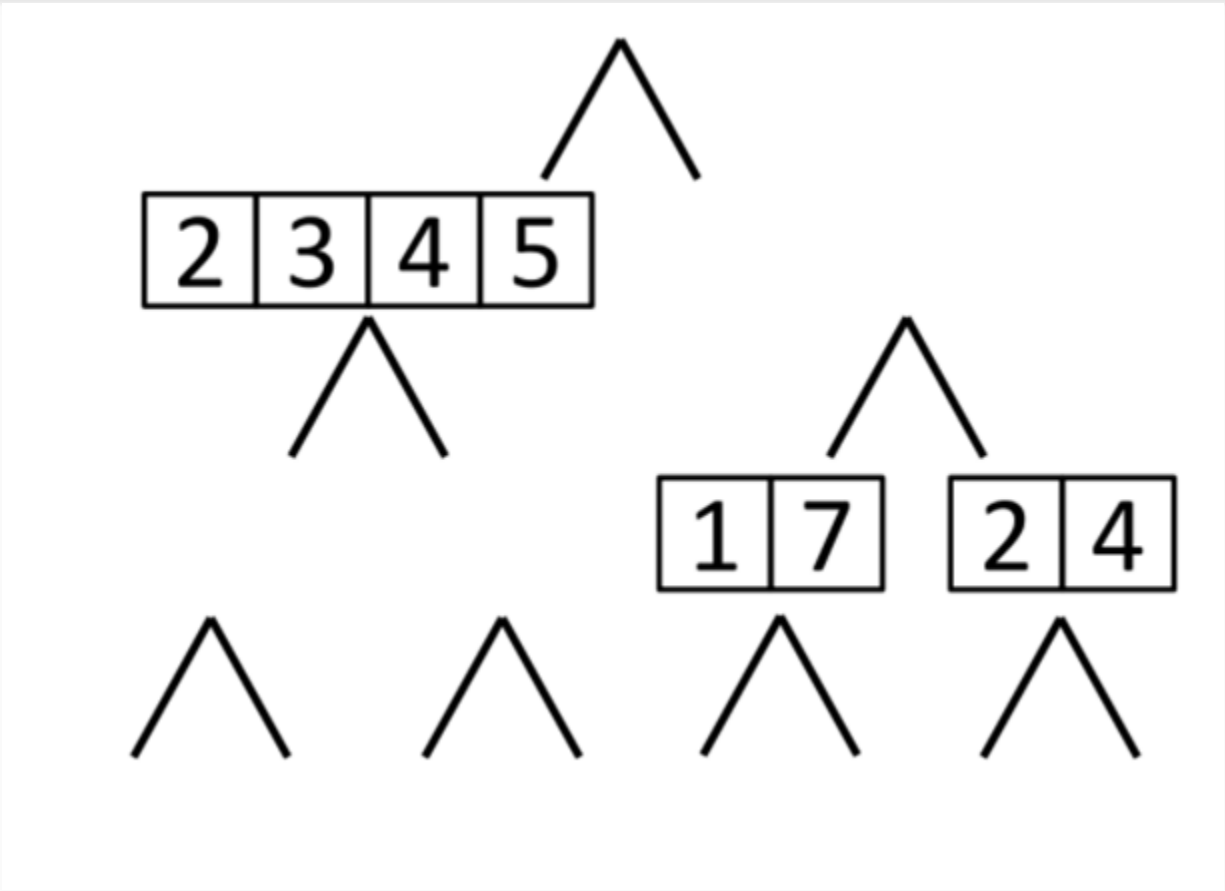
Parallel Merge Sort



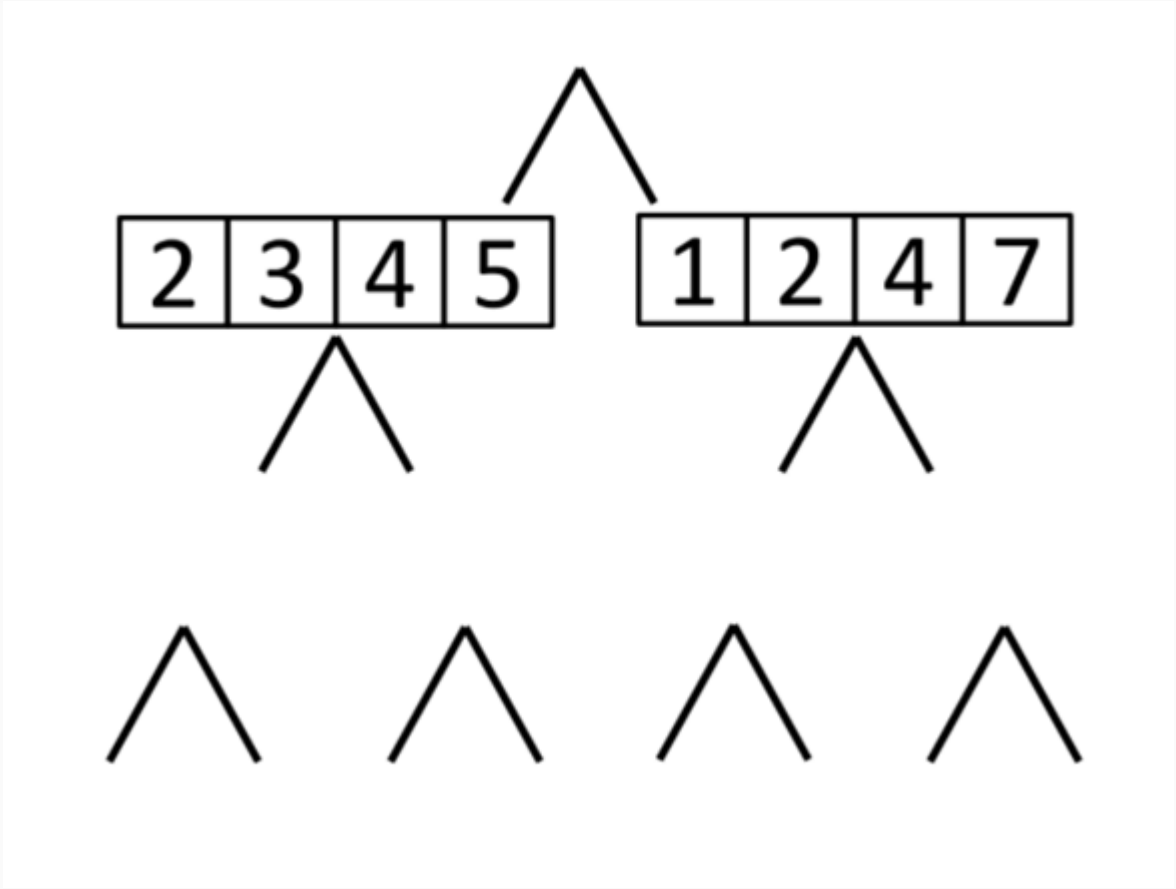
Parallel Merge Sort



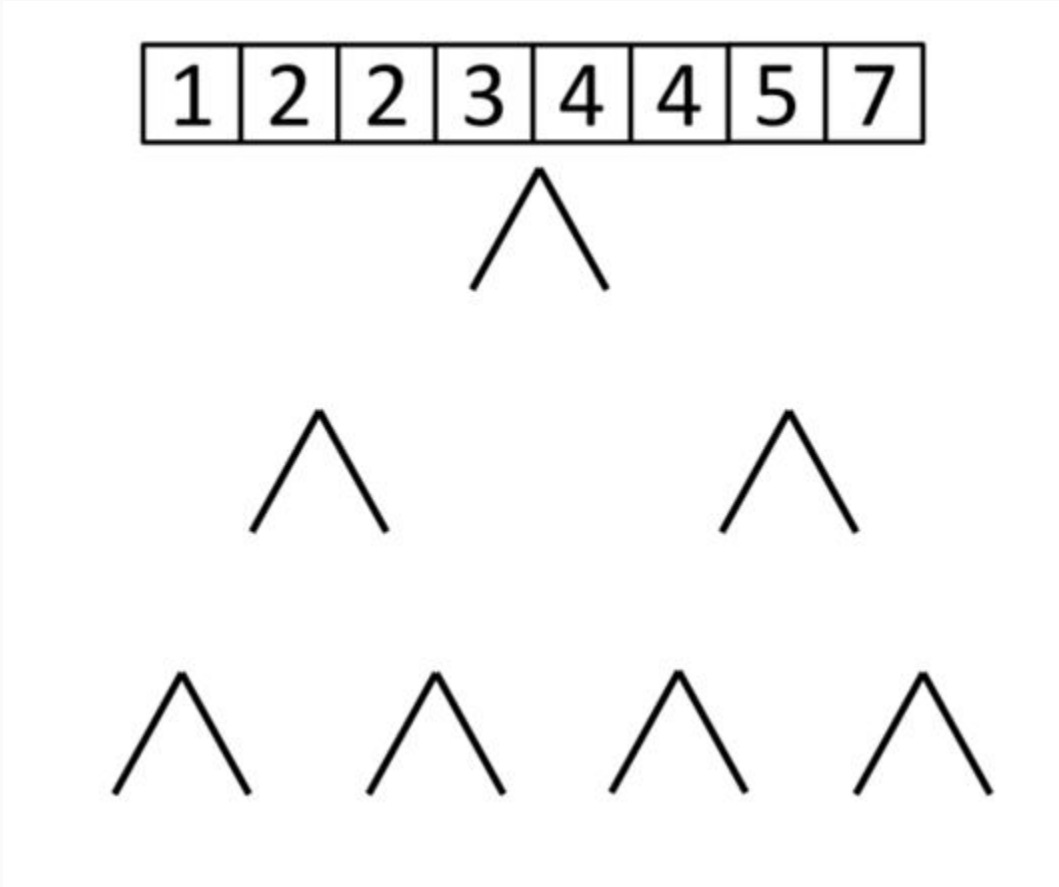
Parallel Merge Sort



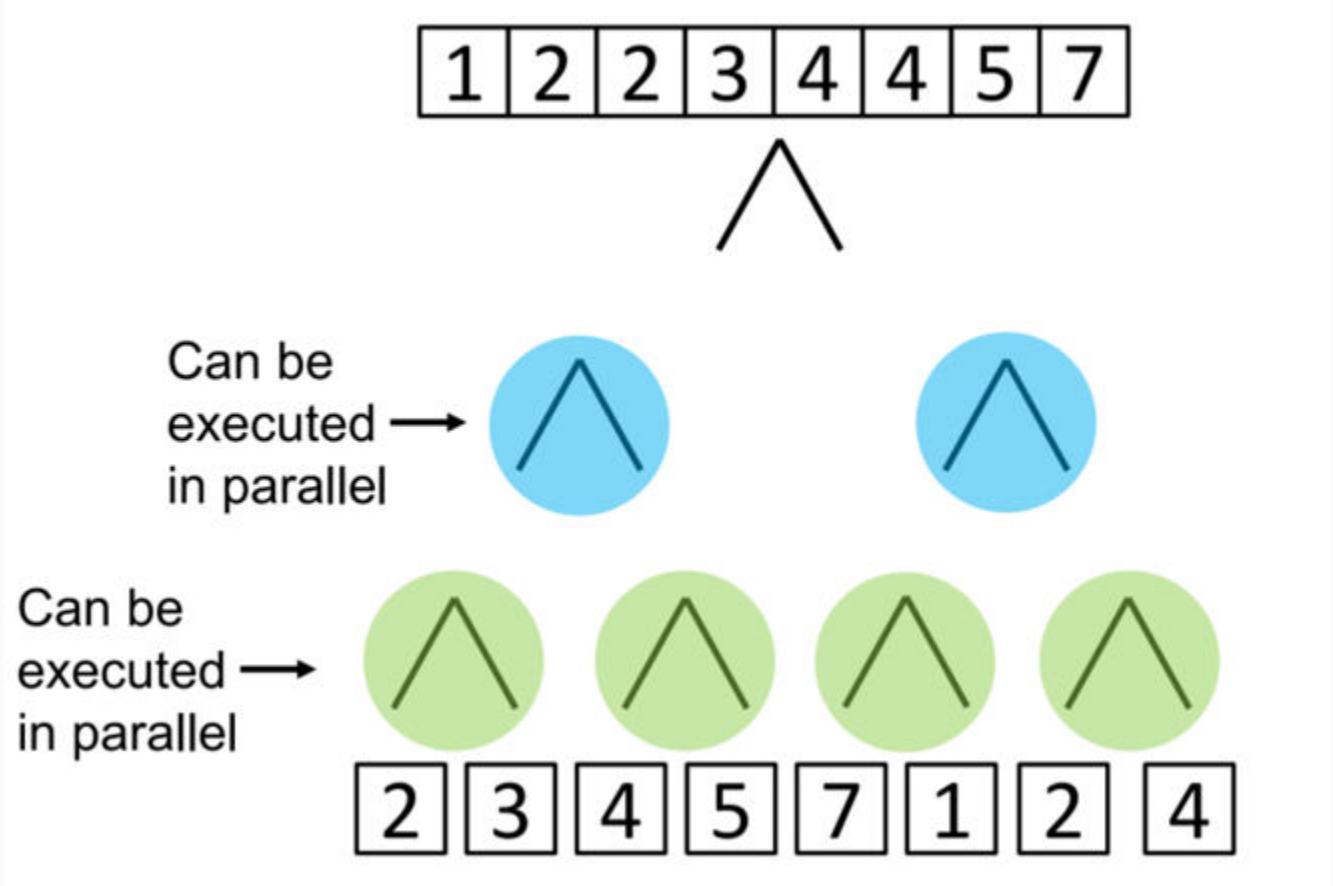
Parallel Merge Sort



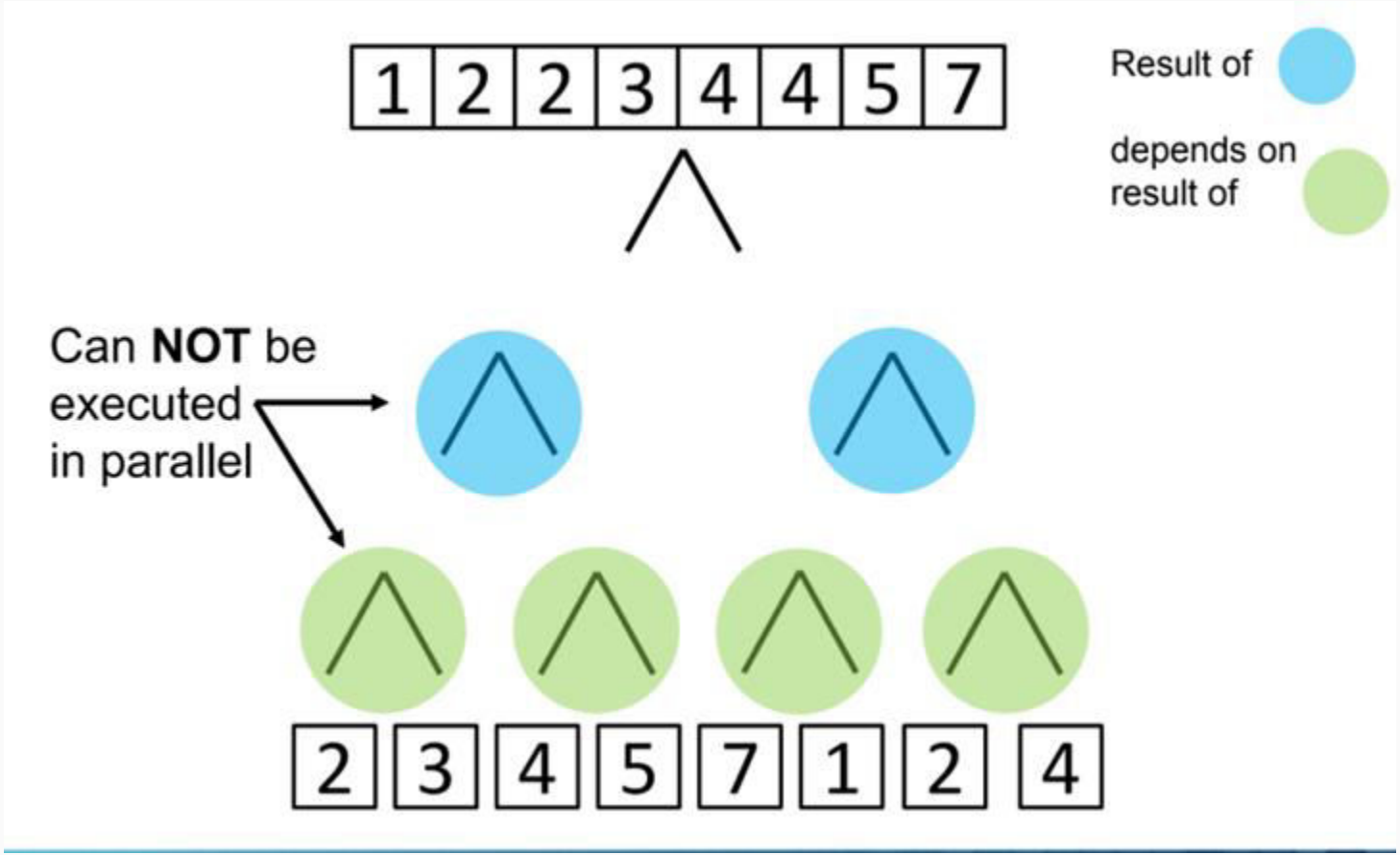
Parallel Merge Sort



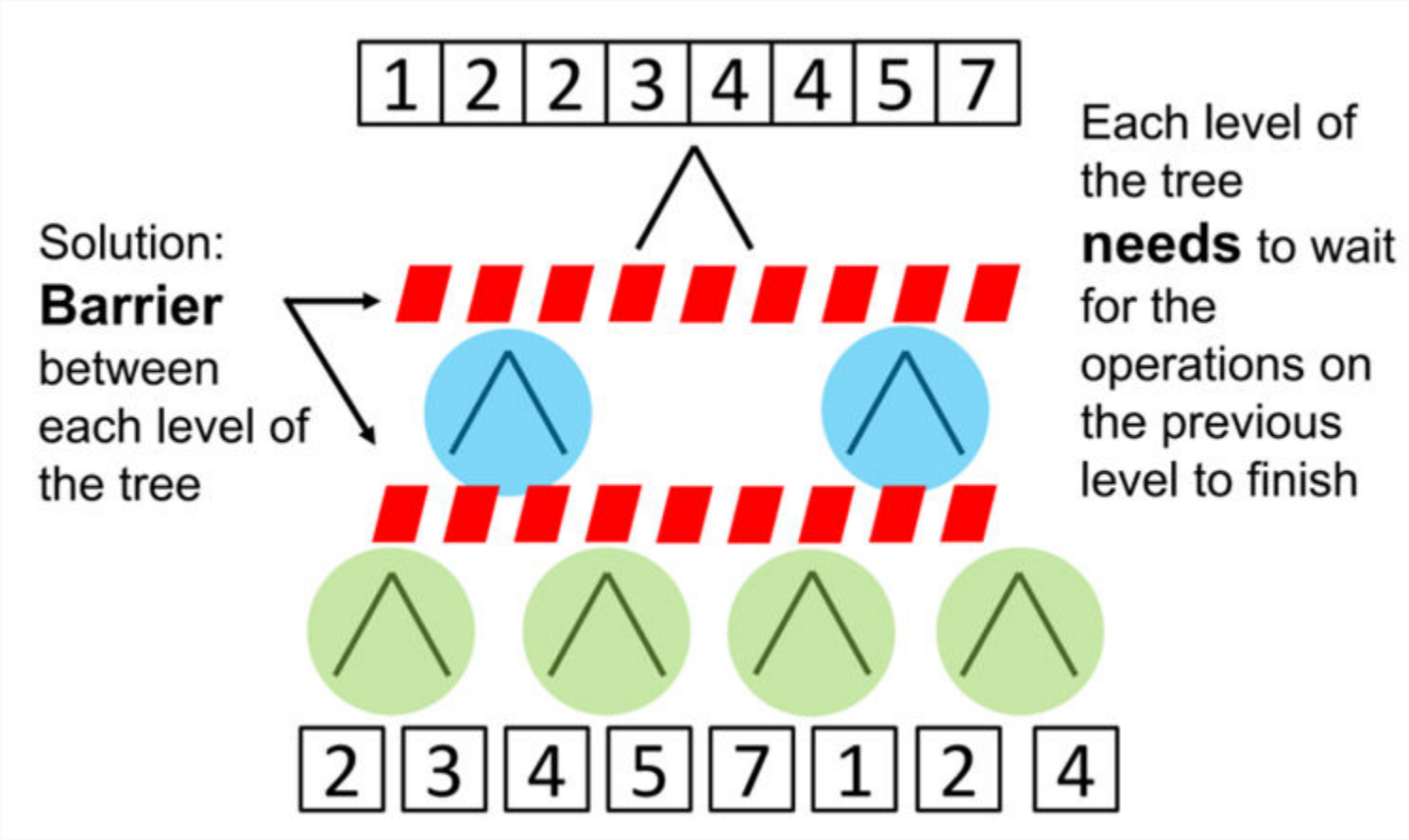
Parallel Merge Sort



Parallel Merge Sort



Parallel Merge Sort



Parallel Merge Sort - probleme

- După cum știți de la primele două laboratoare, folosim formule pentru împărțirea unui array pe thread-uri (**start** și **end**).
- Refreshment:
  - `start_index = id * (double) n / p`
  - `end_index = min(n, (id + 1) * (double) n / p))`, unde `id = id-ul thread-ului`, `n = dimensiunea array-ului`, `p = numărul de threads`

Parallel Merge Sort - probleme

Exemplu de indici pentru merge (N = 8, P = 4, dacă am aplica formulele obișnuite de start și de end):

- nivelul 1 - lungimea 1: (0, 2), (2, 4), (4, 6), (6, 8)
- nivelul 2 - lungimea 2: (0, 2), (2, 4), (4, 6), (6, 8)
- nivelul 3 - lungimea 4: (0, 2), (2, 4), (4, 6), (6, 8)

GREȘIT

Parallel Merge Sort - probleme

Teoretic, corect ar fi așa:

- nivelul 1 - lungimea 1: (0, 2), (2, 4), (4, 6), (6, 8)
- nivelul 2 - lungimea 2: (0, 4), (4, 8)
- nivelul 3 - lungimea 4: (0, 8)

Parallel Merge Sort - probleme

Ar trebui să avem o astfel de transformare (când avem width = 2):

- (0, 2) -> (0, 0)
- (2, 4) -> (0, 4)
- (4, 6) -> (4, 4)
- (6, 8) -> (4, 8)

Parallel Merge Sort - probleme

Ar trebui să avem o astfel de transformare (când avem width = 2):

- (0, 2) -> (0, 0): (0 / (2 * 2)) * (2 * 2), (2 / (2 * 2)) * (2 * 2)
- (2, 4) -> (0, 4): (2 / (2 * 2)) * (2 * 2), (4 / (2 * 2)) * (2 * 2)
- (4, 6) -> (4, 4): (4 / (2 * 2)) * (2 * 2), (6 / (2 * 2)) * (2 * 2)
- (6, 8) -> (4, 8): (6 / (2 * 2)) * (2 * 2), (8 / (2 * 2)) * (2 * 2)

Parallel Merge Sort - probleme

Ar trebui să avem o astfel de transformare (când avem width = 2):

- (0, 2) -> (0, 0): (0 / 4) * 4, (2 / 4) * 4
- (2, 4) -> (0, 4): (2 / 4) * 4, (4 / 4) * 4
- (4, 6) -> (4, 4): (4 / 4) * 4, (6 / 4) * 4
- (6, 8) -> (4, 8): (6 / 4) * 4, (8 / 4) * 4

Parallel Merge Sort - probleme

Ar trebui să avem o astfel de transformare (când avem width = 2):

- (0, 2) -> (0, 0): 0 * 4, 0 * 4 = (0, 0)
- (2, 4) -> (0, 4): 0 * 4, 1 * 4 = (0, 4)
- (4, 6) -> (4, 4): 1 * 4, 1 * 4 = (4, 4)
- (6, 8) -> (4, 8): 1 * 4, 2 * 4 = (4, 8)

Formula: (x / (width * 2)) * (width * 2)

Parallel Merge Sort - probleme

Exemplu de indici pentru merge (N = 8, P = 4, dacă am aplicăm formula de pe slide-ul anterior):

- nivelul 1 - lungimea 1: (0, 2), (2, 4), (4, 6), (6, 8)
- nivelul 2 - lungimea 2: (0, 0), (0, 4), (4, 4), (4, 8)
- nivelul 3 - lungimea 4: (0, 0), (0, 0), (0, 0), (0, 8)

CORECT

Vom avea thread-uri care nu vor face nimic!

Parallel Merge Sort - împărțirea corectă

Formula corectă de calcul (folosind trunchieri - împărțiri de integers):

- `start_index = id * (double) n / p`
- `end_index = (id + 1) * (double) n / p)`, unde `id = id-ul thread-ului, n = dimensiunea array-ului, p = numărul de threads`
- `start_local = (start_index / (2 * width)) * (2 * width)`
- `end_local = min(n, (end_index / (2 * width)) * (2 * width))`, unde `width e lungimea subarray-ului din nivelul curent de recursivitate la merge (2 * width = lungimea subaray-ului următor rezultat din merge)`

Întrebări

- Ce complexitate are Odd Even Transposition Sort pentru n=p (unde n este numărul de elemente de sortat și p este numărul de thread-uri)?

Laboratorul 4
Multithreading în Java

Multithreading în Java

- Mecanismul de multithreading este deja implementat în Java, astfel nu este nevoie să importăm o libr... bibliotecă, ca în C/C++ (`pthreads.h`)
- În Java, putem implementa funcționalitățile thread-urilor (adică să implementăm funcția executată de threads) în două moduri:
  - implementarea interfeței Runnable, cu implementarea metodei run()
  - extinderea clasei Thread și suprascrierea metodei run() (notă: clasa Thread implementează interfața Runnable)

Clasa Thread

```
public class MyThread extends Thread {

        @Override

    public void run() {

         System.out.println("Hello from my new thread!");

    }

}
```

Interfața Runnable

```
public class Task implements Runnable {

        @Override

    public void run() {

         System.out.println("Hello from my new thread!");

     }

}
```

Crearea, rularea și terminarea thread-urilor

- Folosind clasa Thread: https://gist.github.com/florinrm/eda49c9732701410e69abc3dda1b6d44
- Folosind interfața Runnable: https://gist.github.com/florinrm/23a4c2eb7de7ae1412beee9090f3d683

Rulare în linie de comandă

Dacă rulați în linie de comandă:

- compilare (în folderul cu sursele): javac *.java
- rulare (în folderul rădăcină al pachetului): java numePachet.Main (clasa care are metoda main)

run()

- În ambele cazuri (Thread și Runnable), `run()` reprezintă metoda executată de către thread, după ce acesta este lansat.
- Acesta reprezintă echivalentul lui `threadFunction`, trimis ca parametru la funcția `pthread_create` din pthreads.

start()

- Prin metoda `start()`, un thread este creat și lansat în execuție, fiind astfel executată metoda `run()`.
- **Notă**: a NU se confunda `start()` cu `run()`. Dacă am apela `run()` în locul lui `start()`, nu se va crea un nou thread și `run()` va fi executată de către thread-ul curent

join()

- Prin metoda `join()` se așteaptă terminarea execuției thread-urilor.
- Notă: metoda `join()` poate arunca excepții de tipul `InterruptedException`

synchronized

- Keyword-ul synchronized marchează o zonă critică în Java (acesta reprezintă un mutex), mai concret metode și blocuri de cod.
- Când să execute cod synchronized, thread-ul va folosi un monitor (un lock), pentru a rezerva zona critică în care execută instrucțiuni
- Putem avea:
  - metode sincronizate
  - blocuri de cod sincronizate
- **Important**: Metodele și blocurile de cod synchronized din Java sunt reentrante. Dacă un thread a obținut monitorul unui obiect, atunci el va putea intra în orice alt bloc și metodă sincronizate ce sunt asociate cu acel obiect (implicit cu acel monitor).

Metode sincronizate

```
public synchronized T get(int index) {

        if (index < content.length) {

            return content[index];

        }

        throw new IndexOutOfBoundsException(index + " is out of bounds for
MyConcurrentArray of size " + content.length);

}
```

Metode sincronizate

- Când un thread va apela metoda sincronizată get(), acesta va trebui să obțină monitorul (lock-ul) pentru a putea executa metoda respectivă.
- Dacă monitorul este liber, thread-ul va executa metoda.
- Altfel, dacă monitorul e folosit de alt thread, thread-ul curent va aștepta până când monitorul (lock-ul) va fi eliberat.

Blocuri sincronizate

```
public void set(int index, T newT) {

        //Bloc de cod sincronizat ce folosește instanța curentă (this) pe post de zăvor.

        synchronized(this) {

            if (index < content.length) {

                content[index] = newT;

            }

            throw new IndexOutOfBoundsException(index + " is out of bounds for MyConcurrentArray of size " + content.length);

        }

    }
```

Blocuri sincronizate

În cadrul metodei set(), avem o zonă de cod sincronizată.

În acest caz, folosim instanța curentă a clasei (this) drept zăvor / lock pentru accesul în zona critică

Sincronizare cod static

```
private static int numberOfInstances = 0;

public static synchronized int getNumberOfInstances(){

        return numberOfInstances;

}
```

Sincronizare cod static

```
public MyConcurrentArray() {
        synchronized(MyConcurrentArray.class) {
            ++numberOfInstances;
        }
    }
```

Sincronizare cod static

Când sincronizăm cod static (metode statice, câmpuri statice), monitorul / lock-ul folosit este unul asociat clasei (MyConcurrentArray.class), nu asociat instanței

Concluzie - cum folosim synchronized

- synchronized pe un obiect → se folosește monitor lock-ul acelui obiect
- synchronized pe o functie ne-statică → se folosește monitor lock-ul obiectului de care aparține funcția
- synchronized pe o clasă → se folosește monitor lock-ul clasei (se face locking pe orice instanță a clasei respective)
- synchronized pe o funcție statică → se folosește monitor lock-ul clasei de care aparține funcția

CyclicBarrier

- Bariera ciclică este un tip de barieră folosit în structuri iterative (mai precis în structuri repetitive de tip for și while), prin care thread-urile de resincronizează la fiecare iterație, astfel o barieră ciclică este una reutilizabilă (precum cea din pthreads).
- În Java, mecanismul de barieră reutilizabilă este reprezentat de CyclicBarrier.

CyclicBarrier

```
CyclicBarrier barrier = new CyclicBarrier(NUMBER_OF_THREADS); // in main

// in run()

try {

      barrier.await();

      // cod

} catch (BrokenBarrierException | InterruptedException e) {

      e.printStackTrace();

}
```

volatile

- `volatile` este un keyword asociat unei variabile, care specifică faptul că respectiva variabilă nu este cached (în CPU memory) și că citirea și scrierea ei se fac direct din memoria RAM, astfel se garantează faptul că o schimbare efectuată de un thread asupra variabilei respective este imediat vizibilă celorlaltor thread-uri
- Exemplu: `public static volatile int counter = 0;`

Operații atomice

```
AtomicInteger a = new AtomicInteger(0);

int x = a.get();

a.set(5);

x = a.getAndSet(6); // intoarce 5, seteaza 6

boolean b = a.compareAndSet(6, 5); // compara valoarea curenta cu 6 si,
                                   // daca este egala, modifica la 5
```

Întrebări de interviu

- https://dzone.com/articles/top-15-java-multithreading-concurrency-interview-q
- https://www.indeed.com/career-advice/interviewing/multithreading-interview-questions
- https://javarevisited.blogspot.com/2014/07/top-50-java-multithreading-interview-questions-answers.html

Temă de gândire (problemă de interviu)

Aveți următorul cod serial: link

Întrebări:

1) Ce reprezintă acest cod?
2) Ce probleme poate aduce dacă îl folosim într-o soluție paralelă și cum pot fi rezolvate?

Laboratorul 5
Probleme de sincronizare

Problemă

```
function_thread_0:
  noua_valoare =
calculeaza_valoare_a()
  mutex_a.lock()
      a = noua_valoare
  mutex_a.unlock()
```

```
function_thread_1:
  while(a nu s-a actualizat): // busy waiting
      mutex_a.lock()
      if (a s-a actualizat):
          afiseaza(a)
          a = -1 //  Resetam valoarea lui a.
      mutex_a.unlock()
```

- thread-ul 0 calculează o valoarea și o scrie în variabila ”a”
- thread-ul 1 verifică dacă valoarea lui ”a” s-a actualizat și o afișează

Problemă

- thread-ul 1 va cicla de mai multe ori, chiar dacă thread-ul 0 nu actualizează variabila ”a”, astfel se pierde timp inutil
- această abordare se numește **busy waiting**, care aici nu este o idee rea, însă avem dezavantajul că se irosesc resurse

wait()

- forțează thread-ul curent să între în starea de waiting (să aștepte), până când alt thread apelează `notify()` sau `notifyAll()` pe același obiect, pe care thread-ul curent trebuie să-l dețină, situație când:
  - se execută o metodă sau un bloc synchronized pe obiectul respectiv
  - se execută o metodă statică synchronized pe clasa din care face parte obiectul respectiv
- atenție: nu este recomandat să folosiți wait() pe obiecte statice sau pe string-uri constante (final), căci JVM optimizează accesul la aceste variabile

notify() + notifyAll()

- `notify()` - este ”trezit” din waiting un thread aleatoriu și pus în running
- `notifyAll()` - sunt ”trezite” toate thread-urile din waiting și puse în running

T0	T1

`synchronized(object) { // lock`
`  while (!condition) {`
`  object.wait();`
`  }`
`}`
	`[...]`
	`synchronized(object) { lock`
	`  condition = true;`
	`  object.notify();`
	`  // object.notifyAll();`
	`}`
	`[...]`

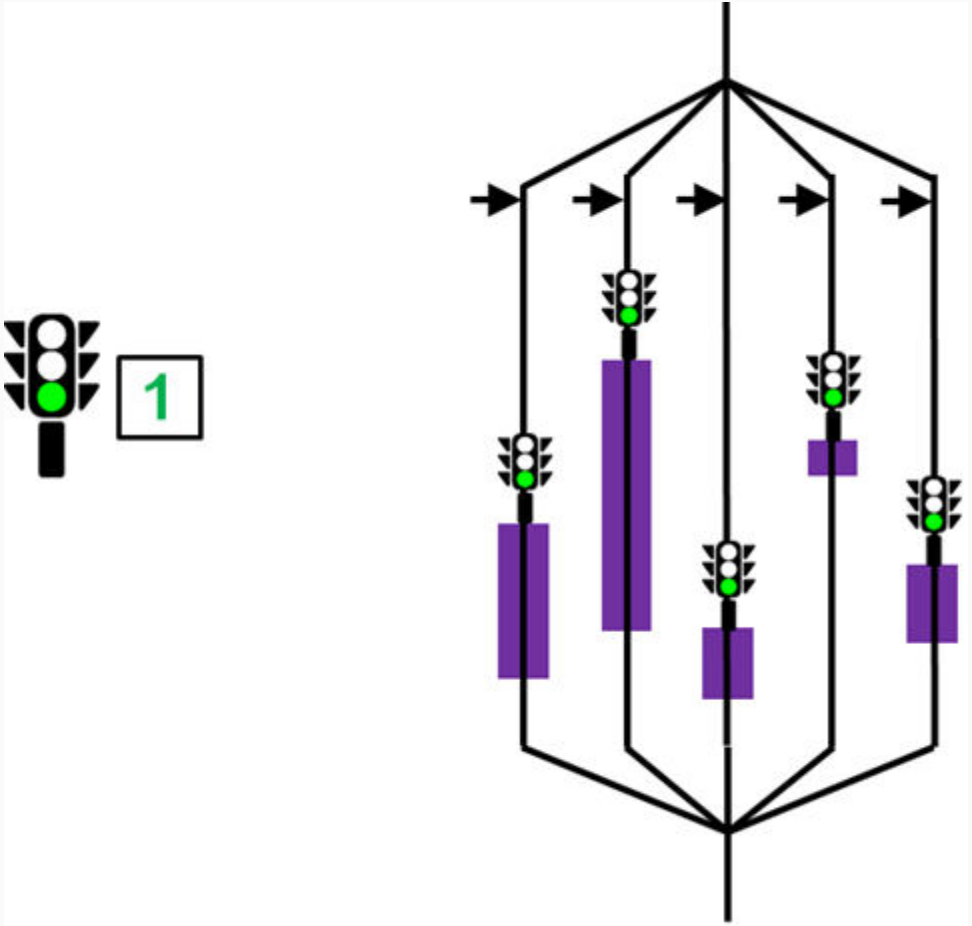
Important! Cand apelam wait() si notify(), trebuie sa detinem monitor lock-ul obiectului pe care apelam functiile.

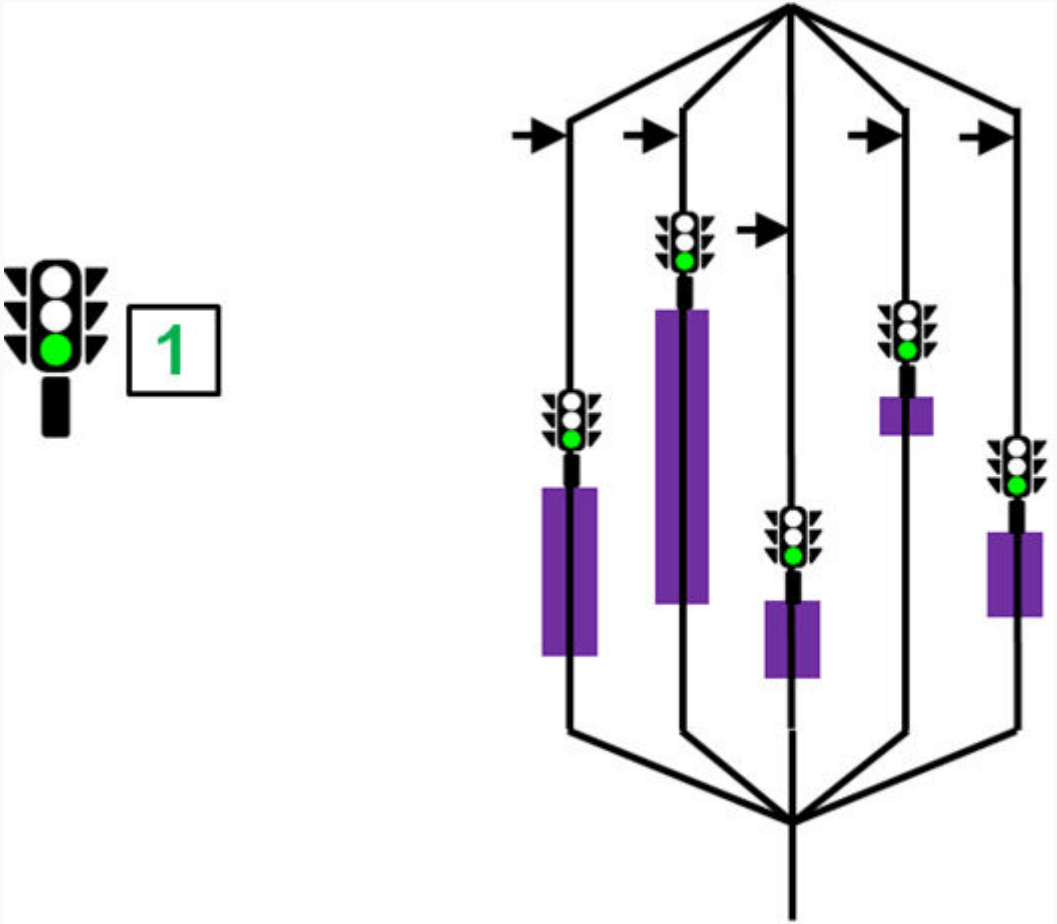
Semafor

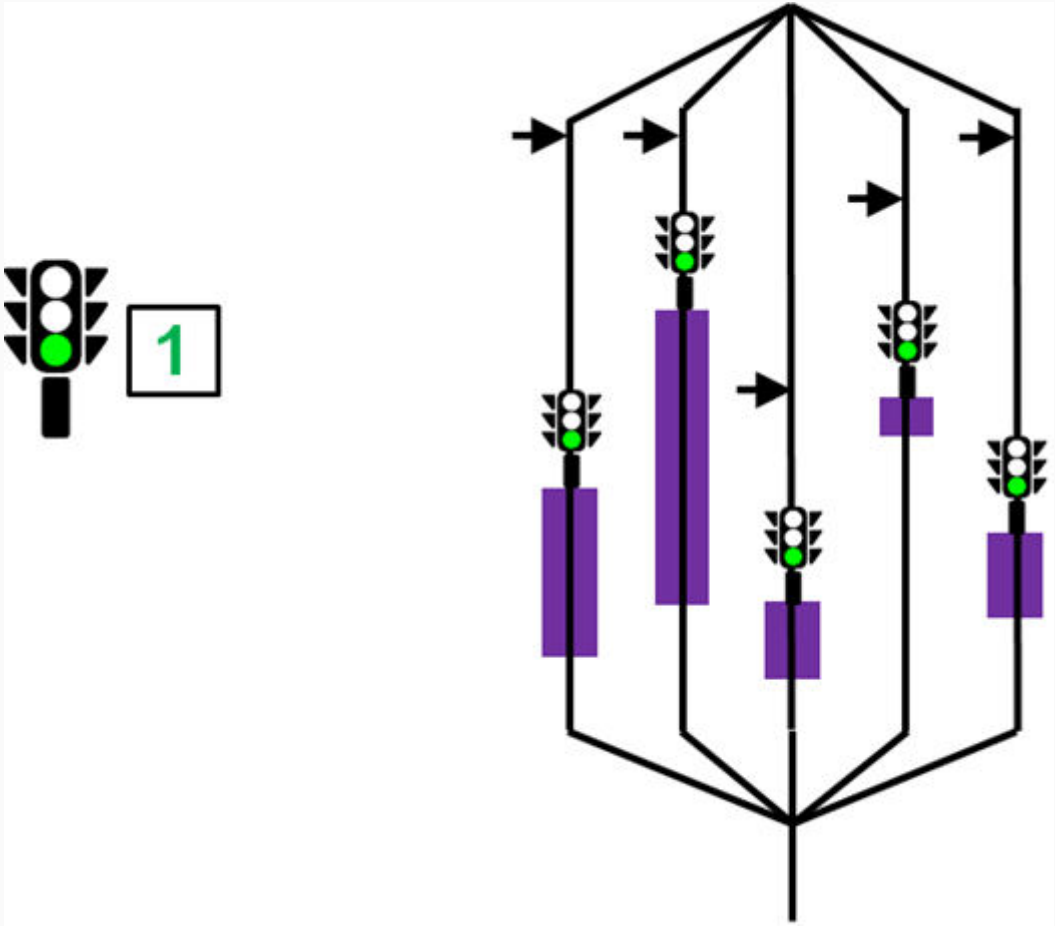
- semaforul este o generalizare a unui mutex, unde mai multe thread-uri pot intra într-o zona critică la un moment dat
- acesta are un contor care determină câte thread-uri mai pot intra în zona critică
- un thread este lăsat să intre în zona critică doar dacă contorul curent e mai mic decât numărul maxim de thread-uri permise în zona critică

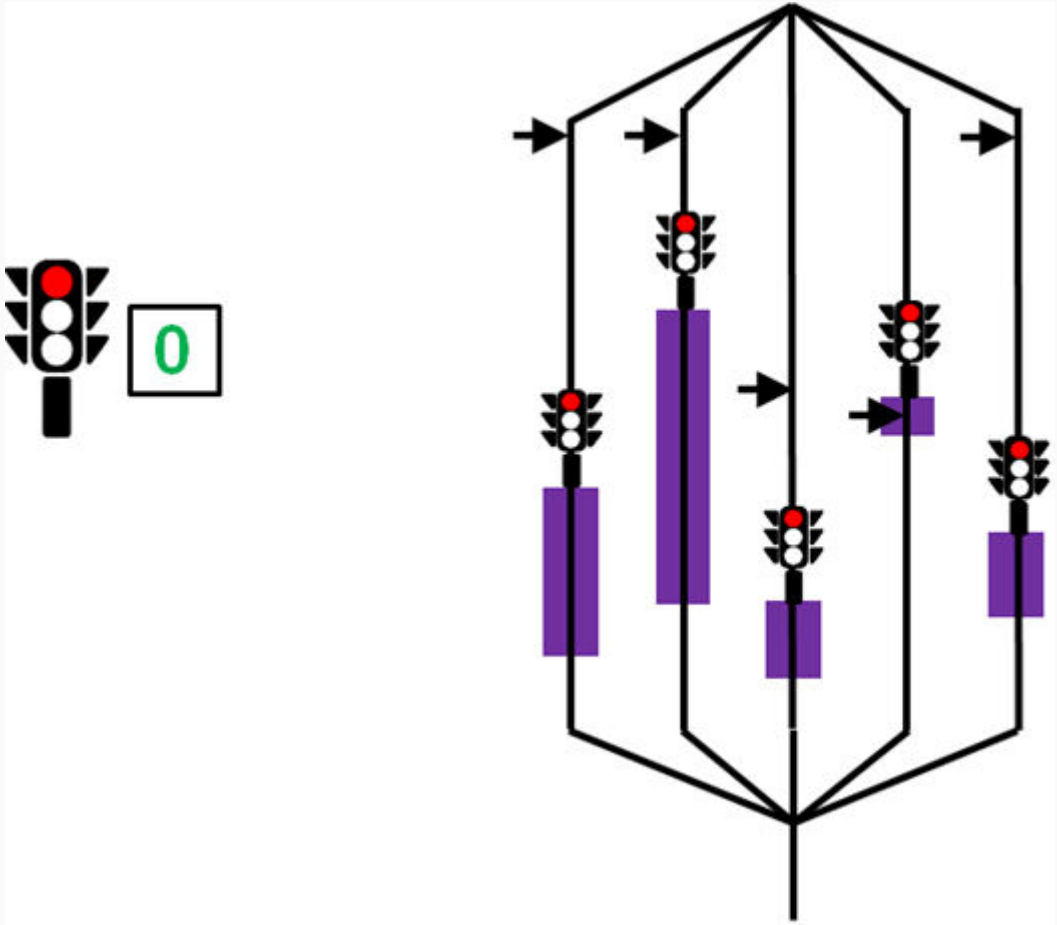
Semafor - funcții

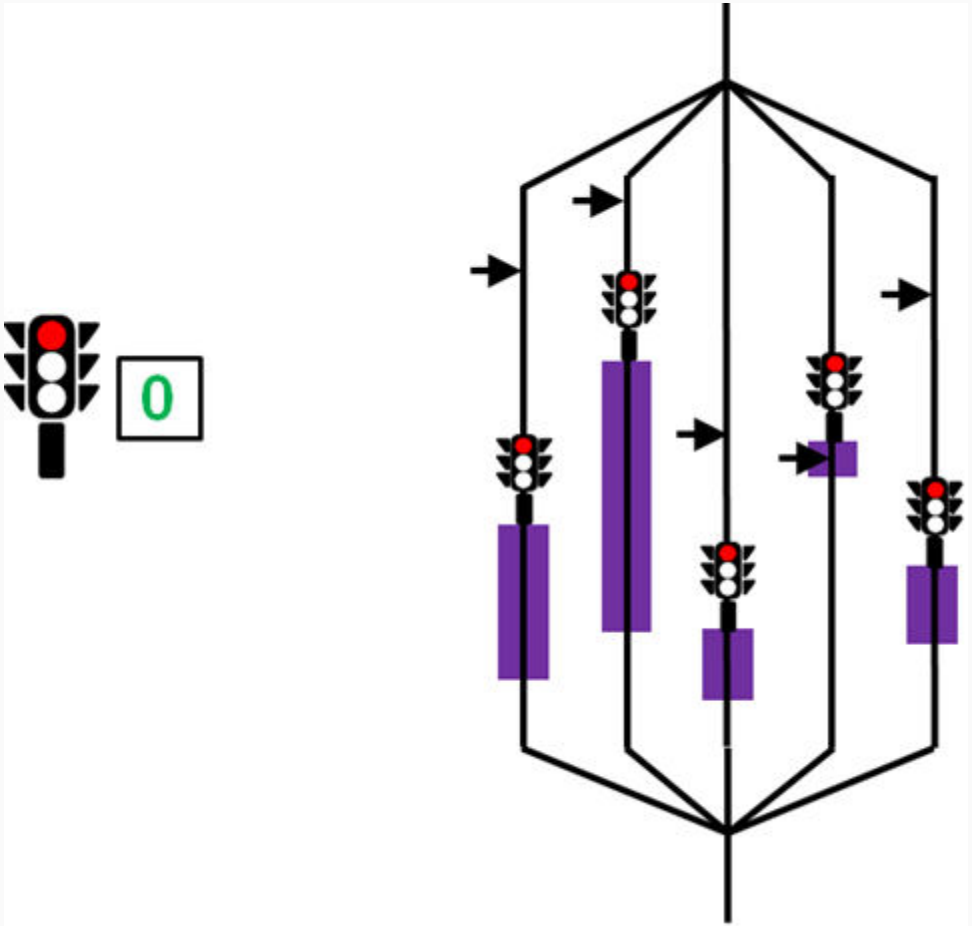
- acquire - se încearcă trecerea de semafor și intrarea în zona critică. Dacă contorul curent e mai mic decât numărul maxim de thread-uri permise, thread-ul intră și contorul este decrementat
- release - un thread iese din zona critică, contorul este incrementat

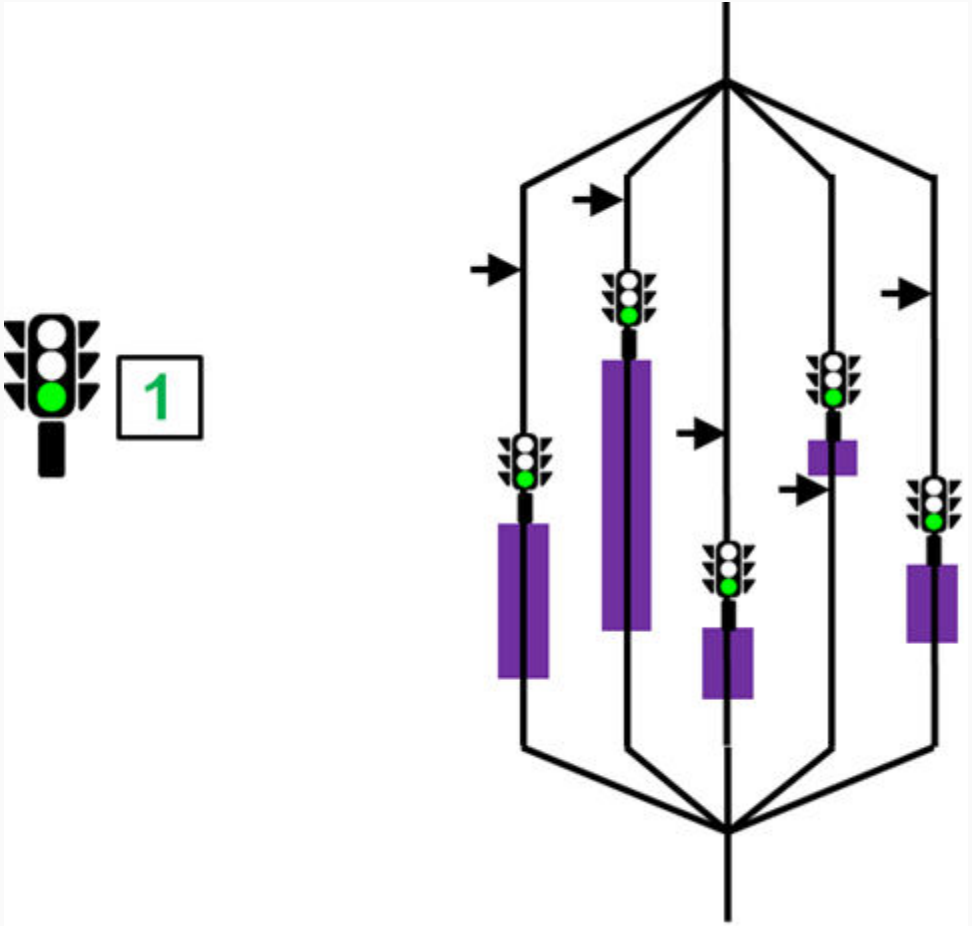

1

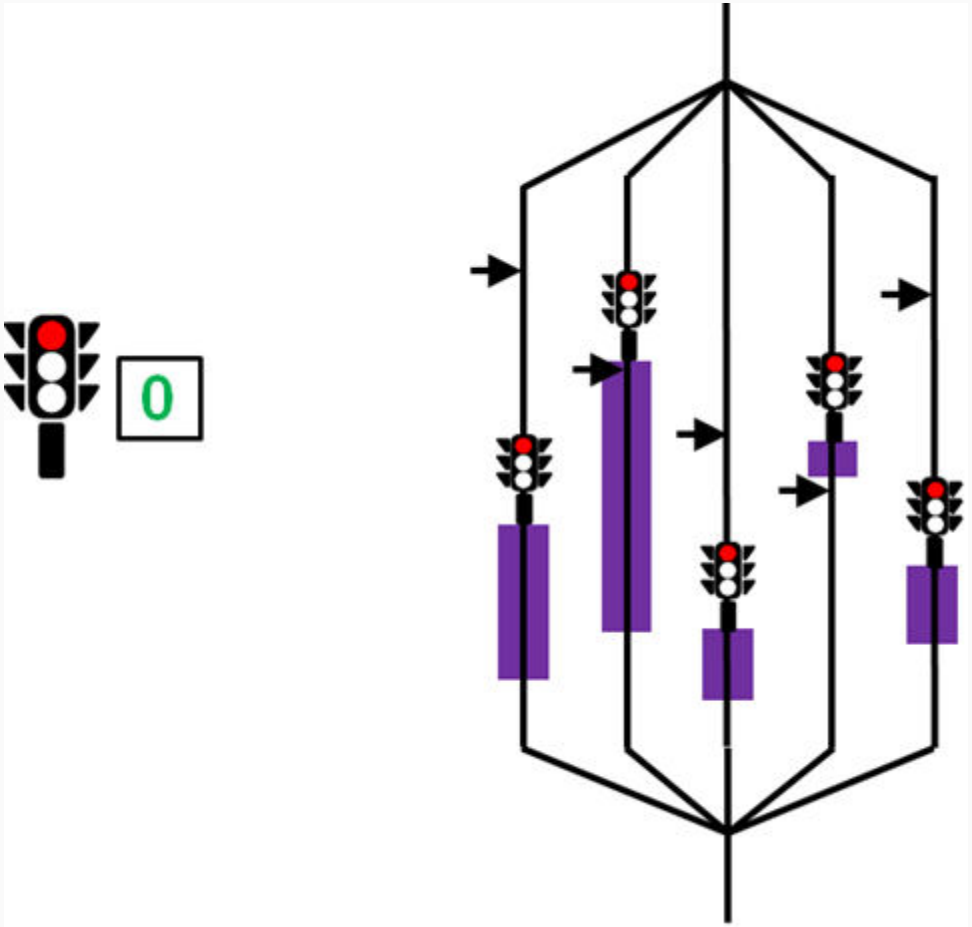

1

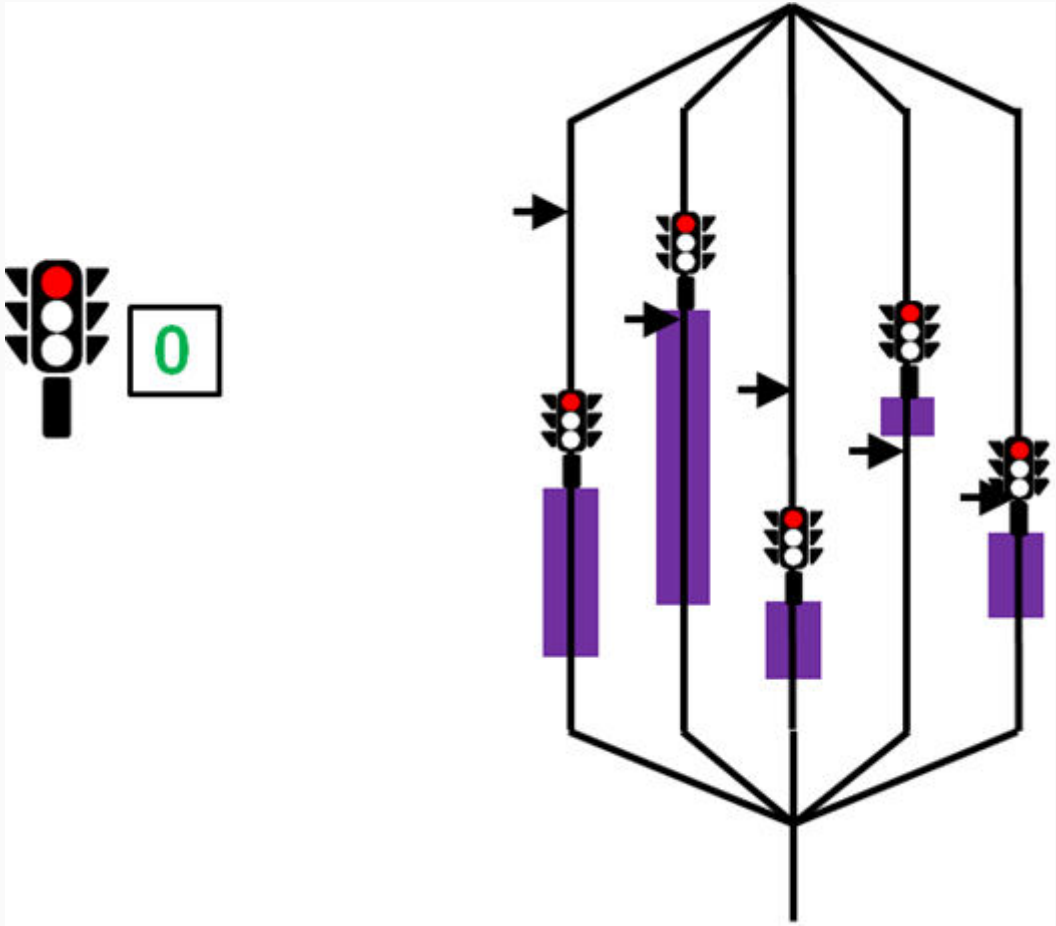

1

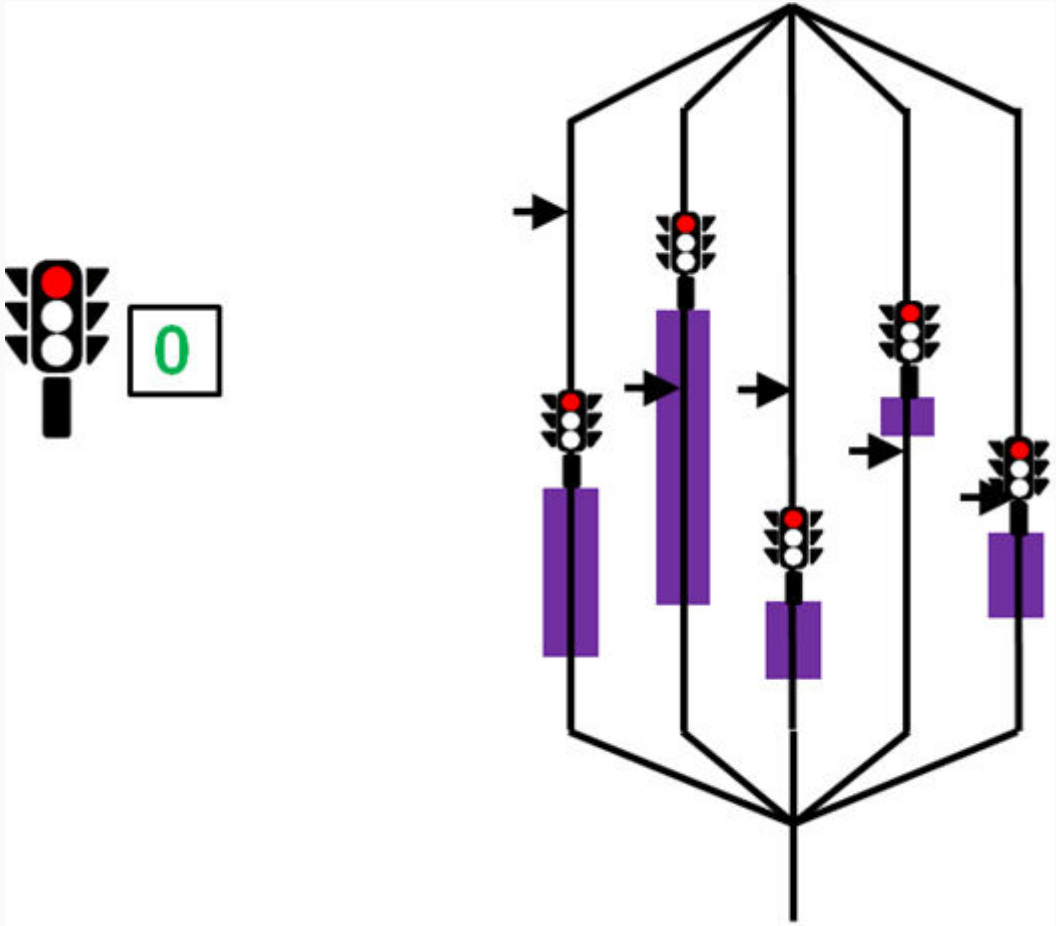

0

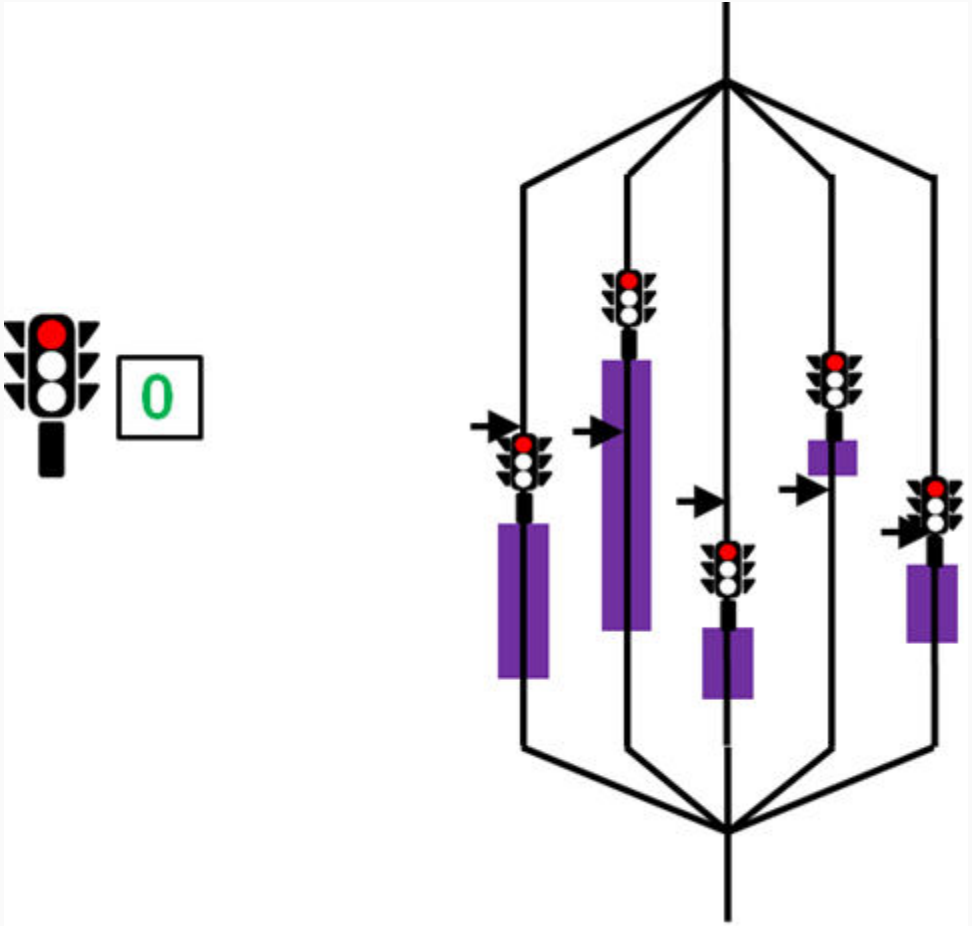

0

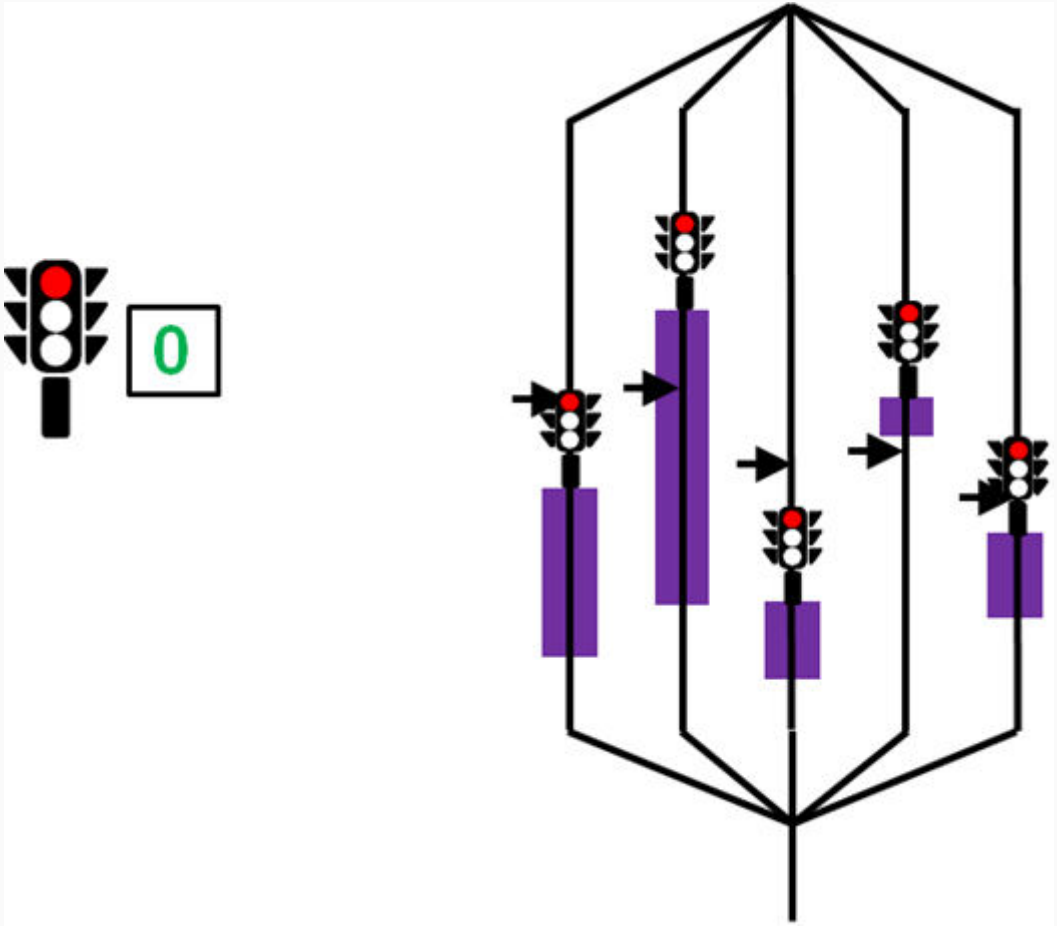

1

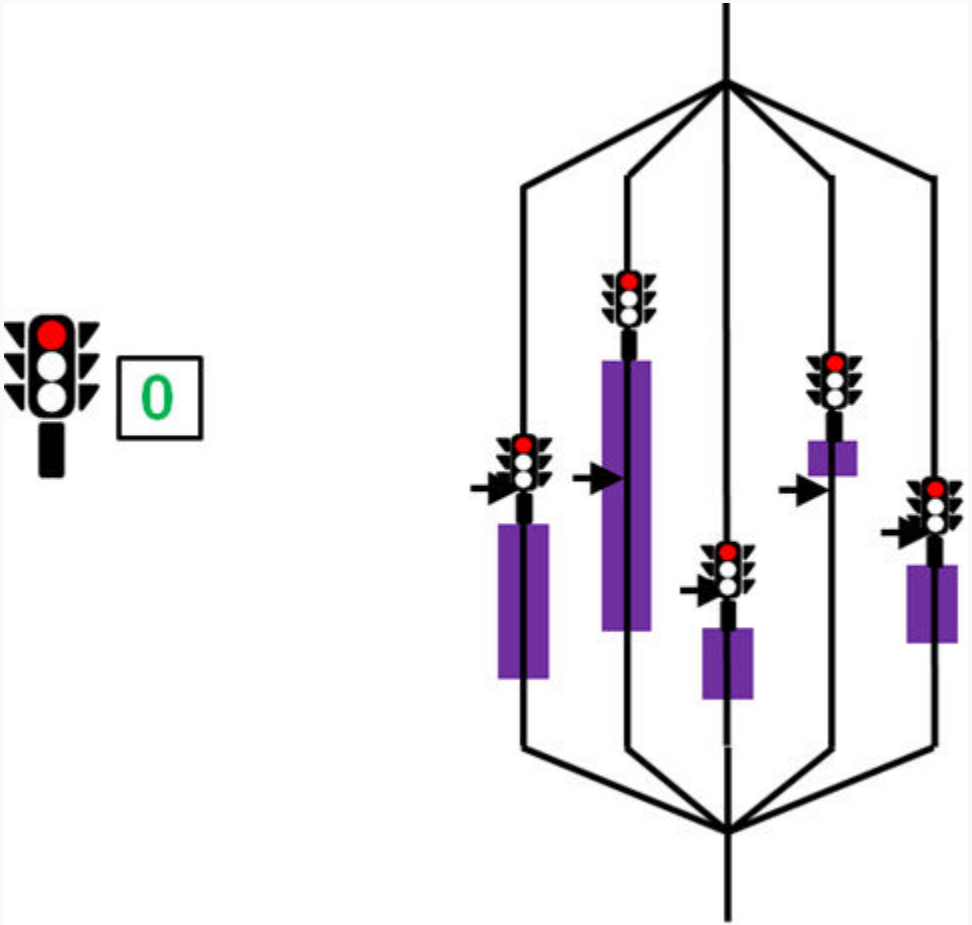

0

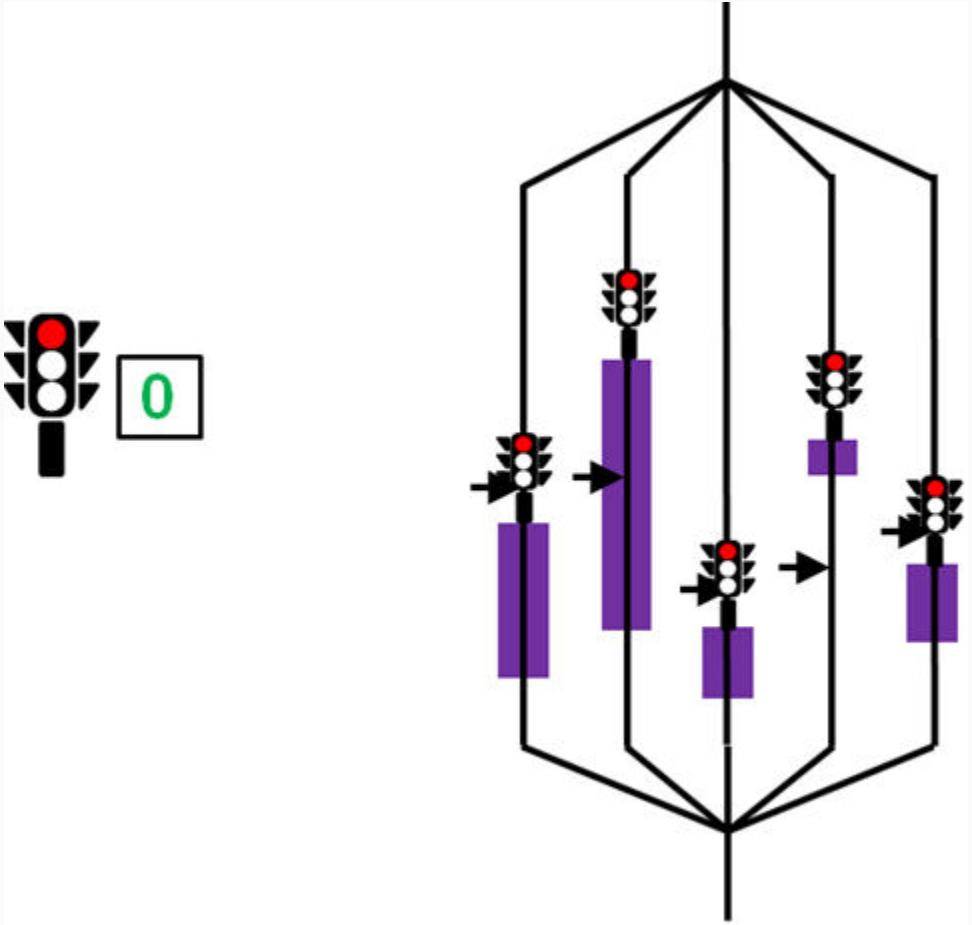

0

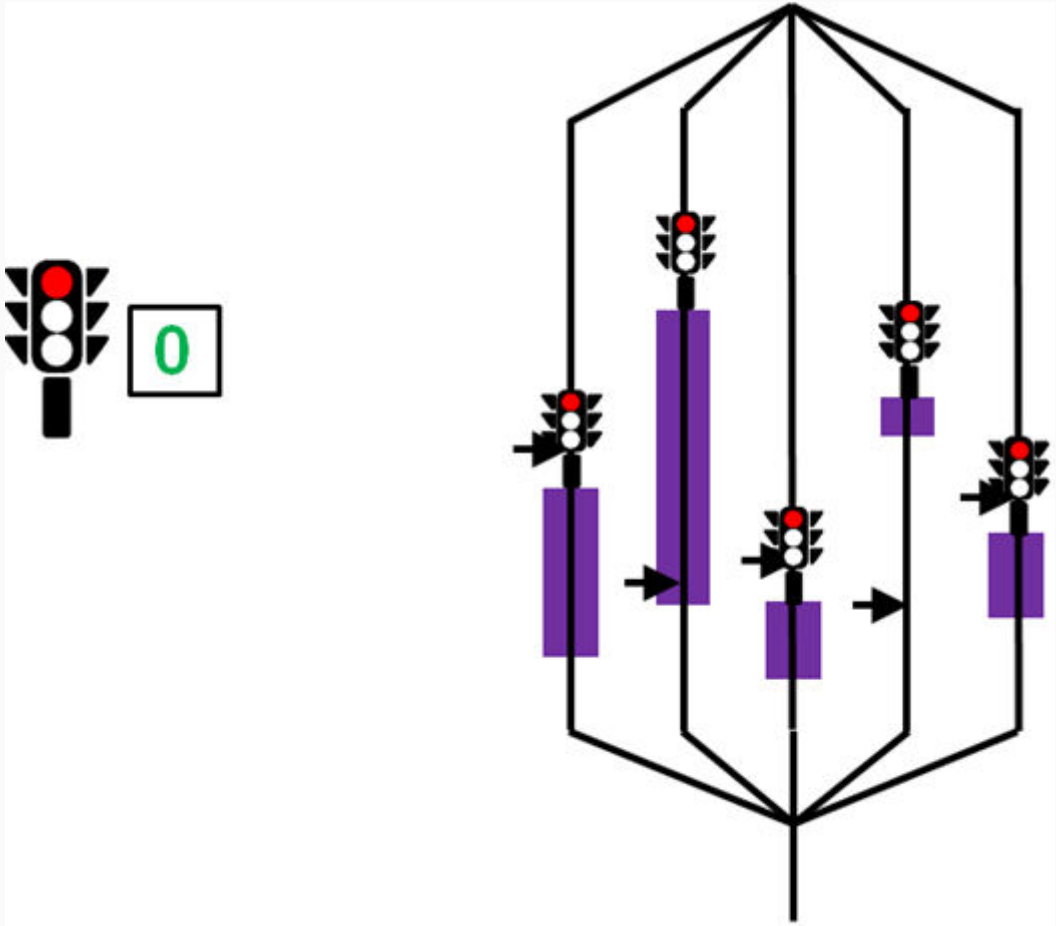

0

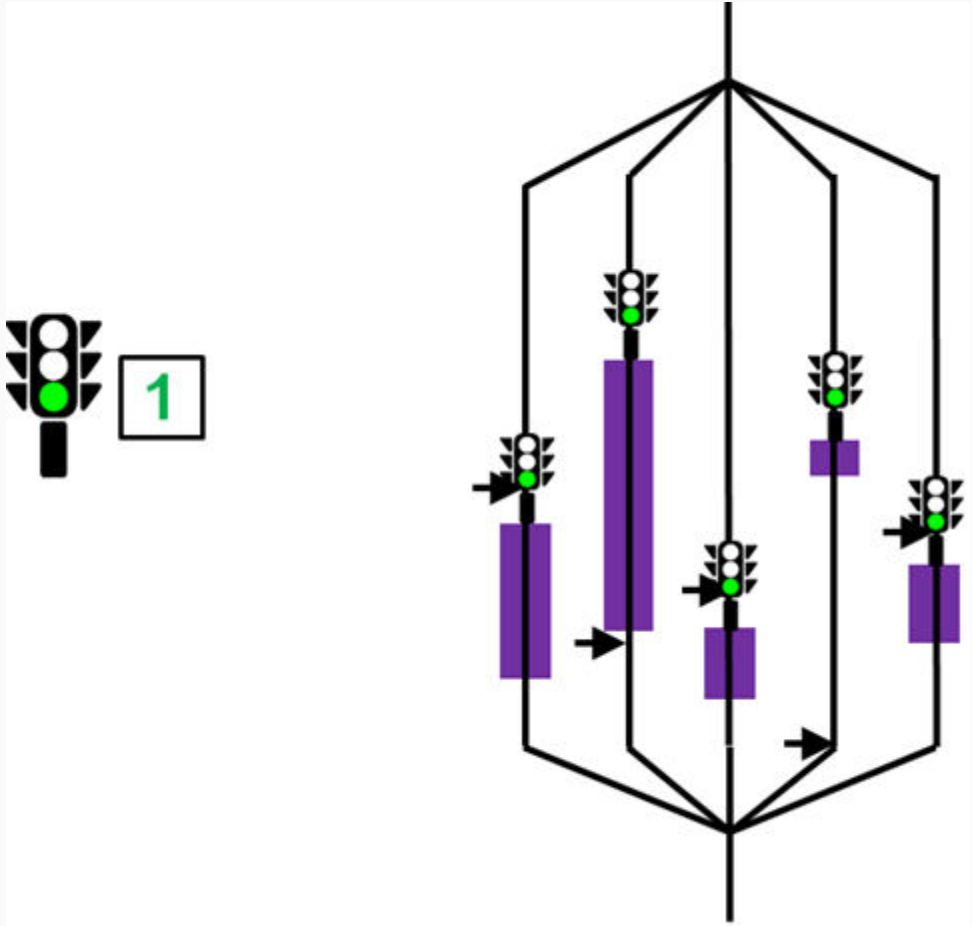

0

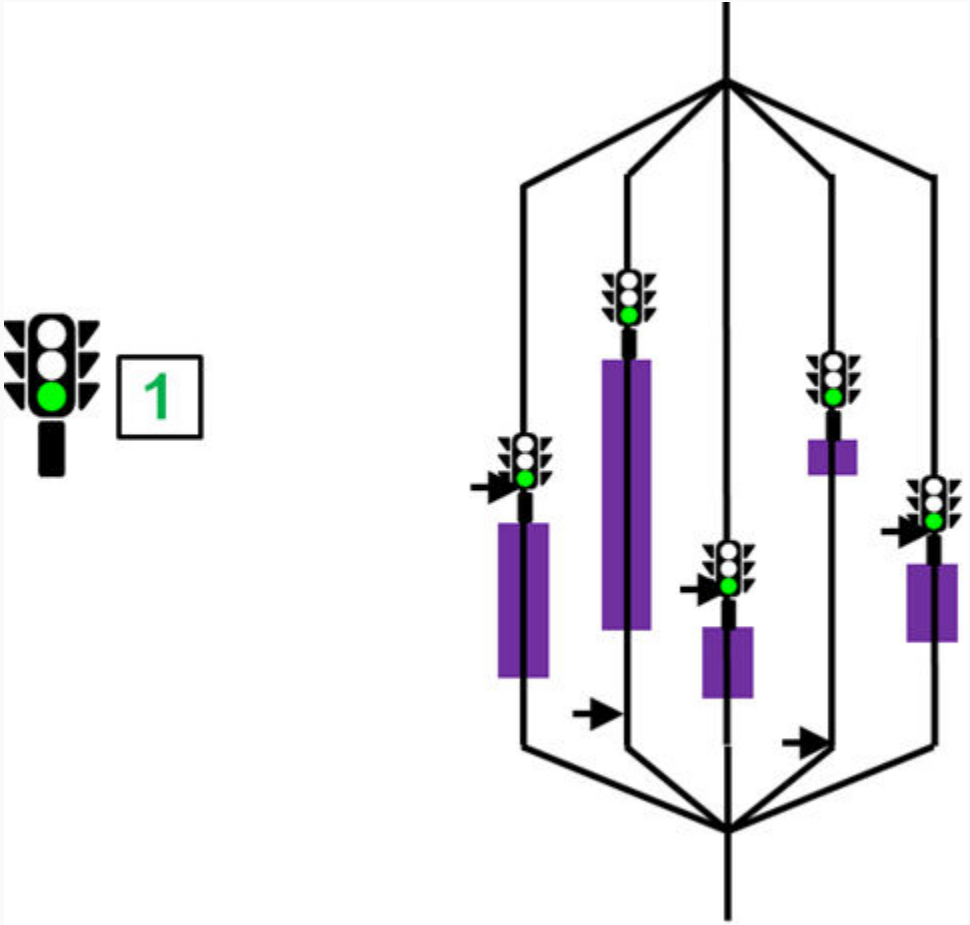

0

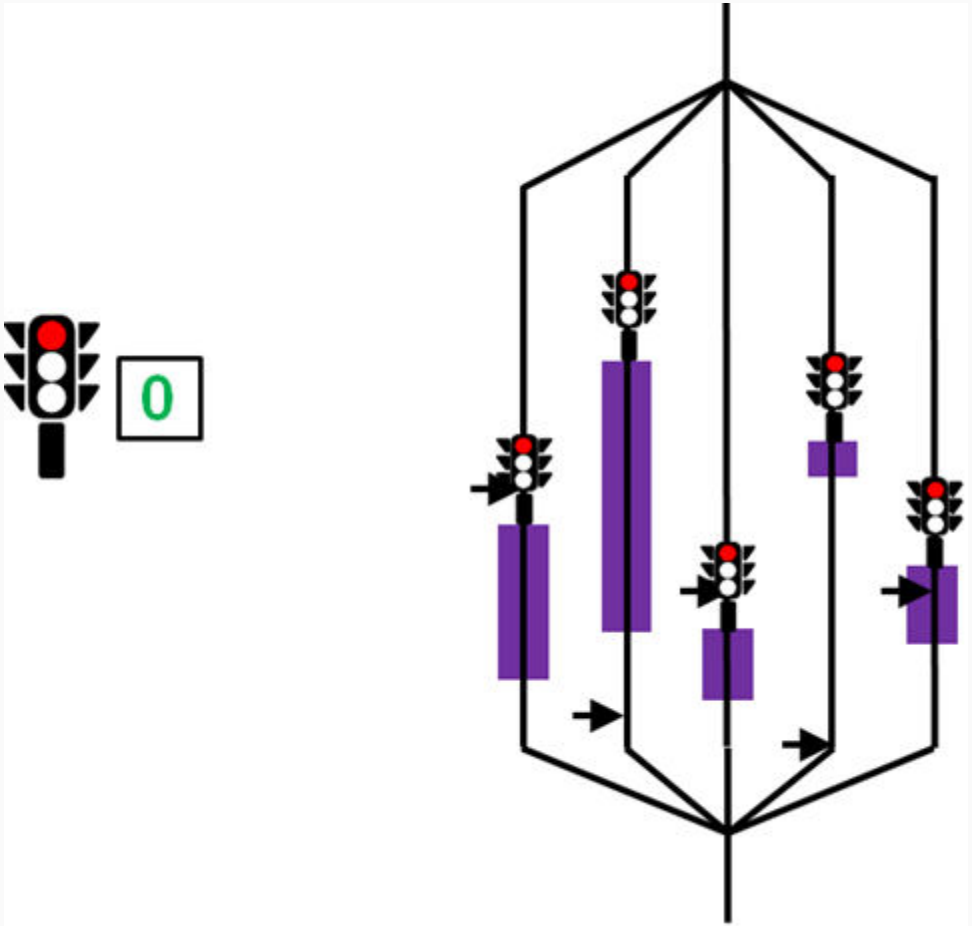

0


0


0


1


1


0

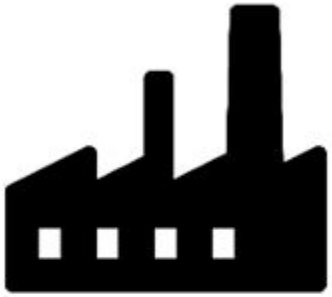
Producer consumer

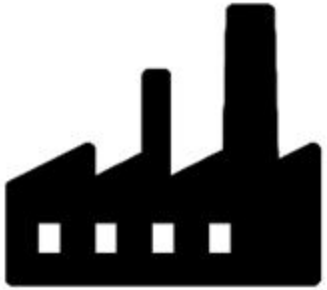
- avem două tipuri de thread-uri:
  - producător - inserează date într-un buffer
  - consumator - extrage date din acel buffer, care are o dimensiune stabilită
- condiționări:
  - producătorul nu poate pune date într-un buffer plin
  - consumatorul nu poate extrage date dintr-un buffer gol
  - producătorul și consumatorul nu pot acționa simultan asupra bufferului -> mutex / synchronized

Producer

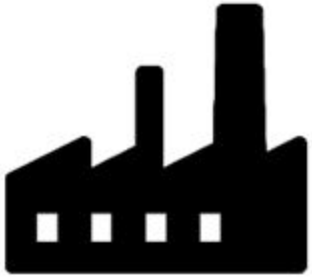
Buffer

Consumer



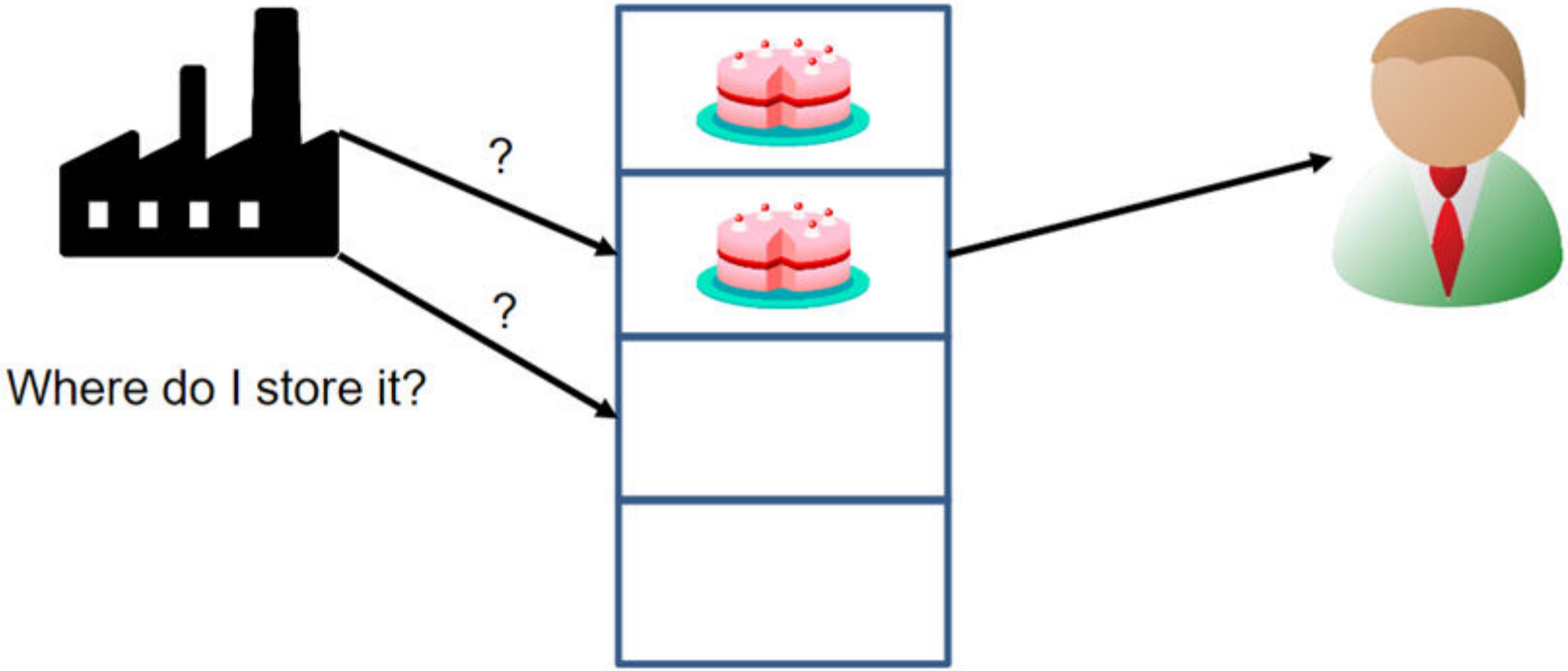


Can**not** produce
while buffer **full**
(must wait)

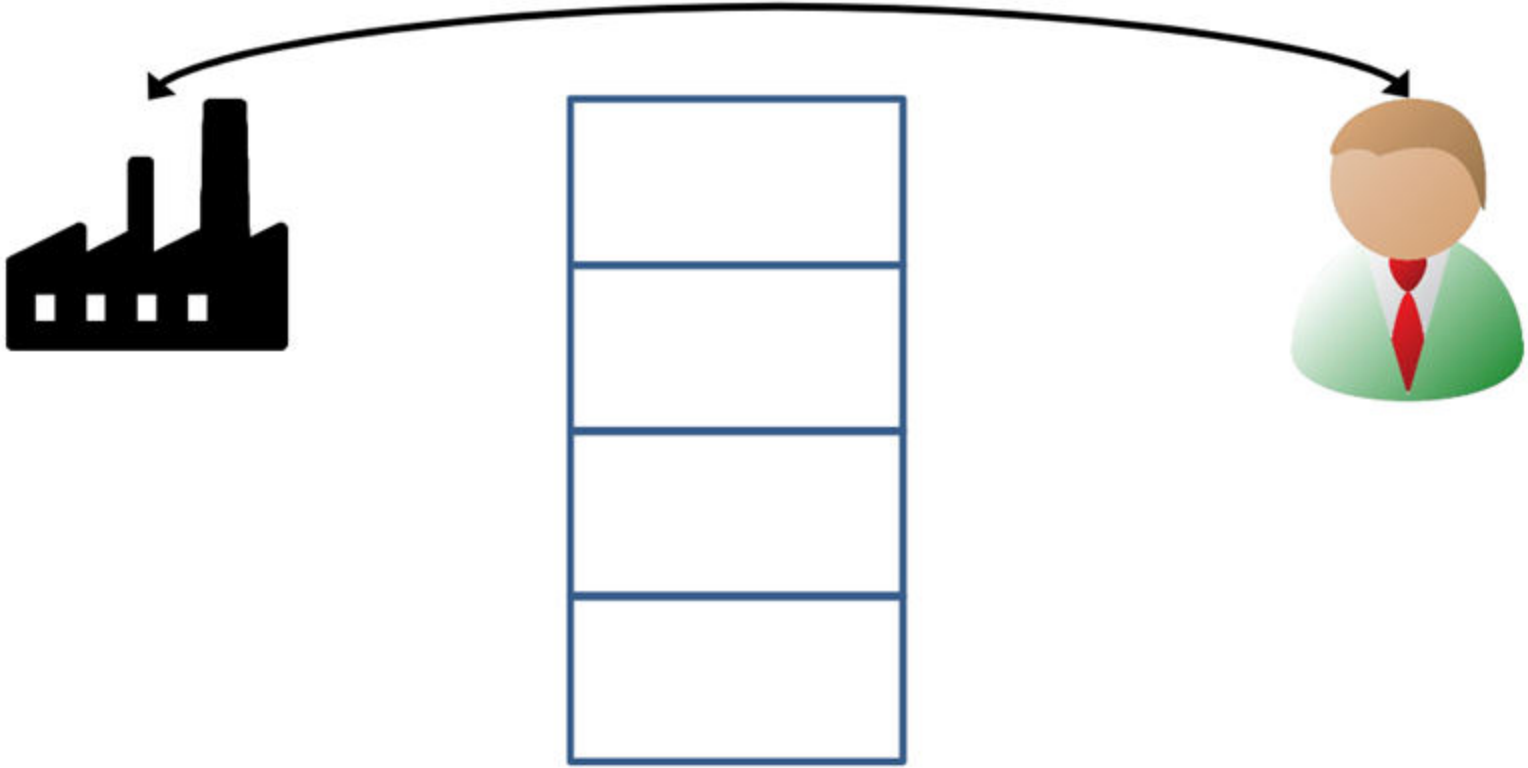


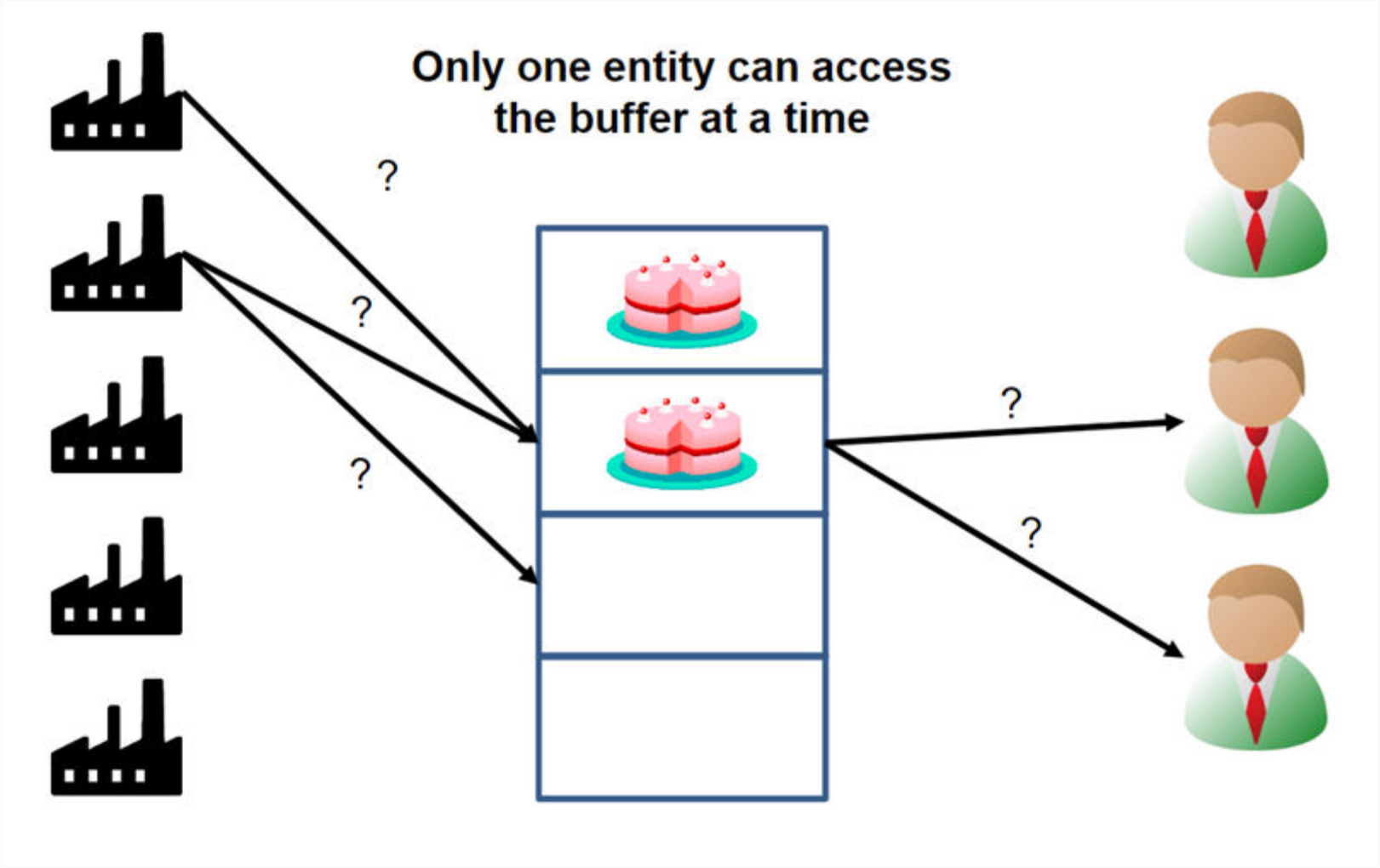
Can**not** consume while buffer **empty** (must wait)

Cannot produce while consuming



Does not known when the other works
(asynchronous)




Only one entity can access
the buffer at a time
?
?
?
?
?
?

If using a signaling system make sure you signal the right entity

Sleeping

Sleeping

Sleeping

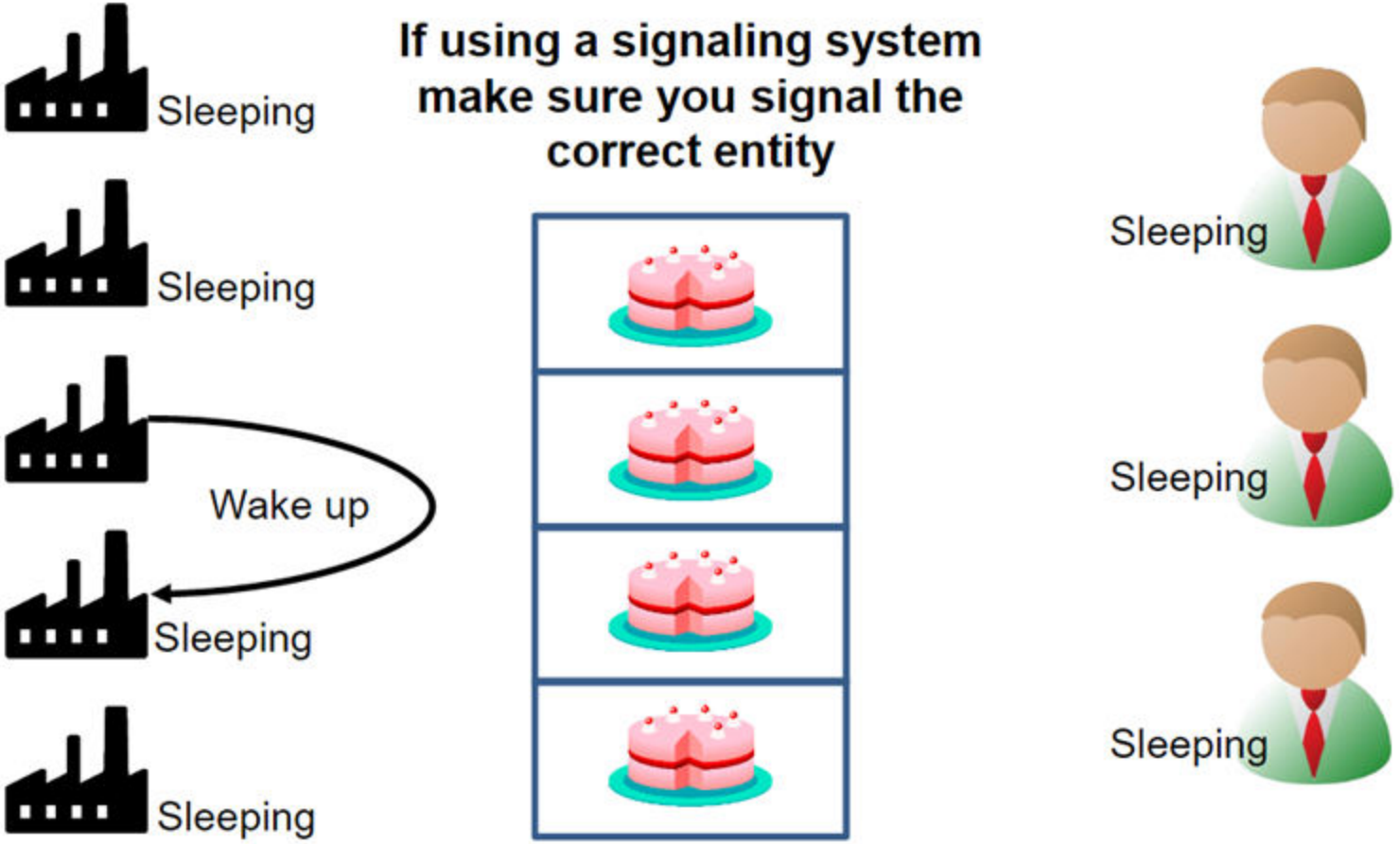
Sleeping

If using a signaling system make sure you signal the correct entity

Sleeping

Sleeping

Sleeping


If using a signaling system make sure you signal the correct entity
Sleeping
Sleeping
Wake up
Sleeping
Sleeping
Sleeping
Sleeping
Sleeping

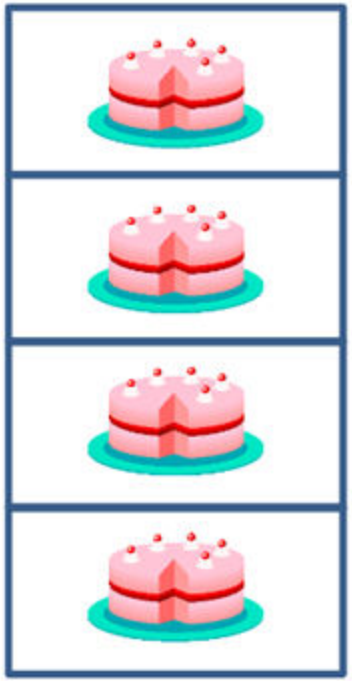
Sleeping

Sleeping

Sleeping

Sleeping

If using a signaling system make sure you signal the correct entity



Sleeping

Sleeping

Sleeping

Sleeping

Sleeping

Sleeping

Sleeping

Sleeping

If using a signaling system make sure you signal the correct entity

Sleeping

Sleeping

Sleeping

P	C

while (B.isFull()) {
    wait();
}
produce();
notify();
	while (B.isEmpty()) {
	    wait();
	}
	consume();
	notify();

Implementare cu wait-notify - one producer one consumer - se aplică și în cazul general, cu notifyAll() în loc de notify()

```
typeT buf[1:k];
int prim = 1, ultim = 1;
sem gol = k, plin = 0;
sem mutexP = 1, mutexC = 1;

process Producător[i=1 to M]{
    typeT v;
    while (true) {
      v = produce();
      P(gol);
        P(mutexP);
        buf[ultim] = v;
        ultim = ultim mod k + 1;
        V(mutexP);
      V(plin);
    }
}

process Consumator[i=1 to N]{
    typeT w;
    while (true) {
P(plin);
      P(mutexC);
      w = buf[prim];
      prim = prim mod k + 1;
     V(mutexC);
    V(gol);
    consumă(w);
    }
}
```

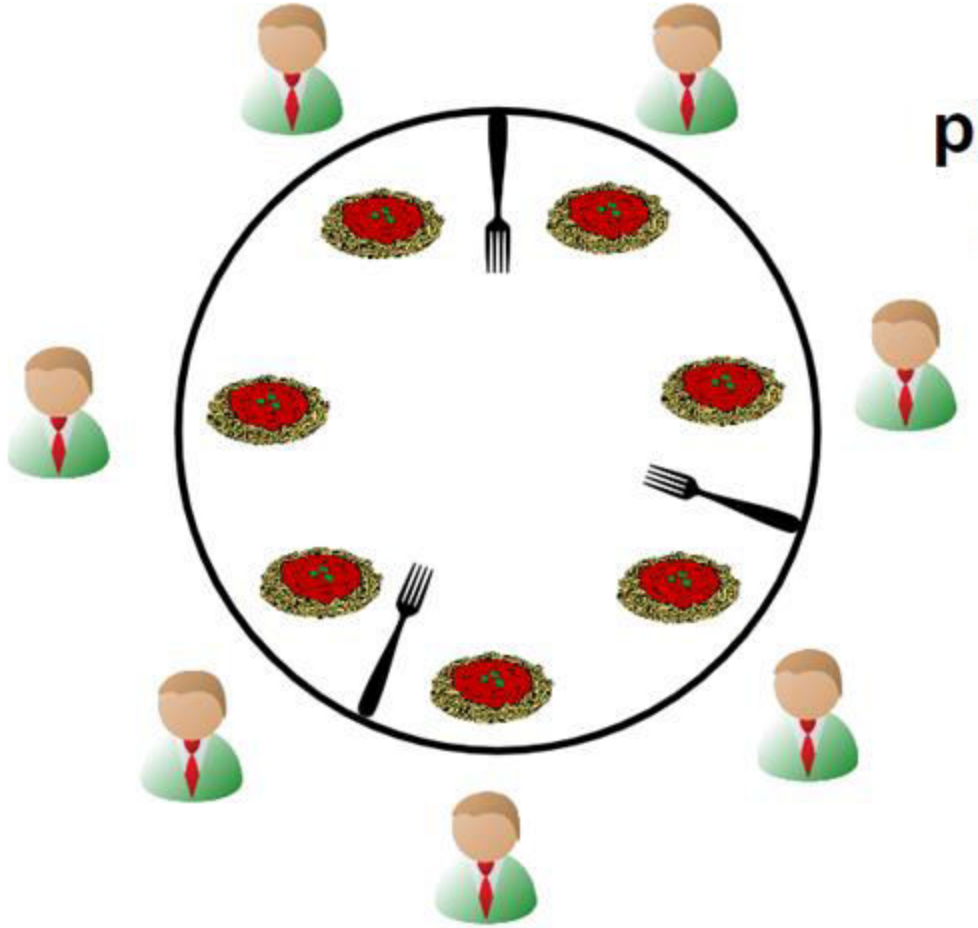
sectiune critica

semnalizare

Problema filosofilor

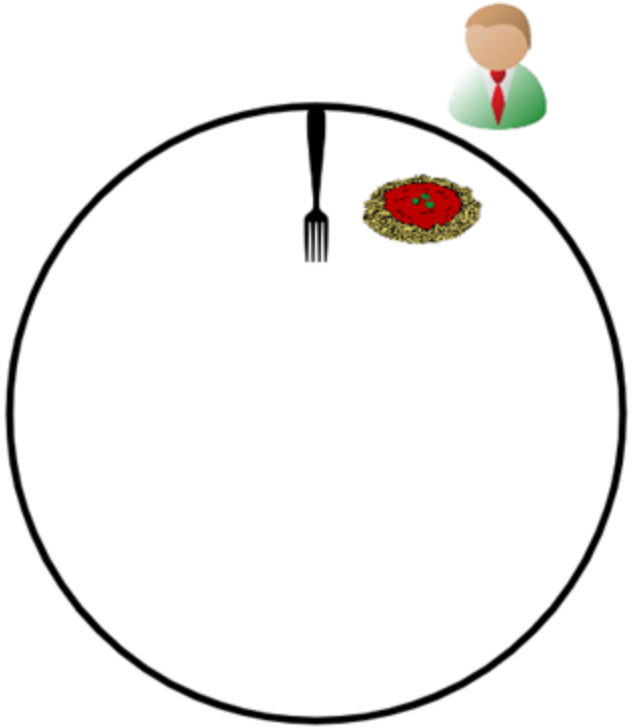
- avem n filosofi (threads) puși la o masă circulară, unde sunt puse n farfurii și n furculițe (locks), astfel încât fiecare filosof are două tacâmuri (unul în stânga, altul în dreapta) pentru a mânca
- soluție: n - 1 filosofi iau mai întâi furculița dreaptă, apoi pe cea stânga, iar al n-lea filosof va lua invers (mai întâi stânga, apoi dreapta)

The forks are locks



The philosophers are threads

The forks are locks

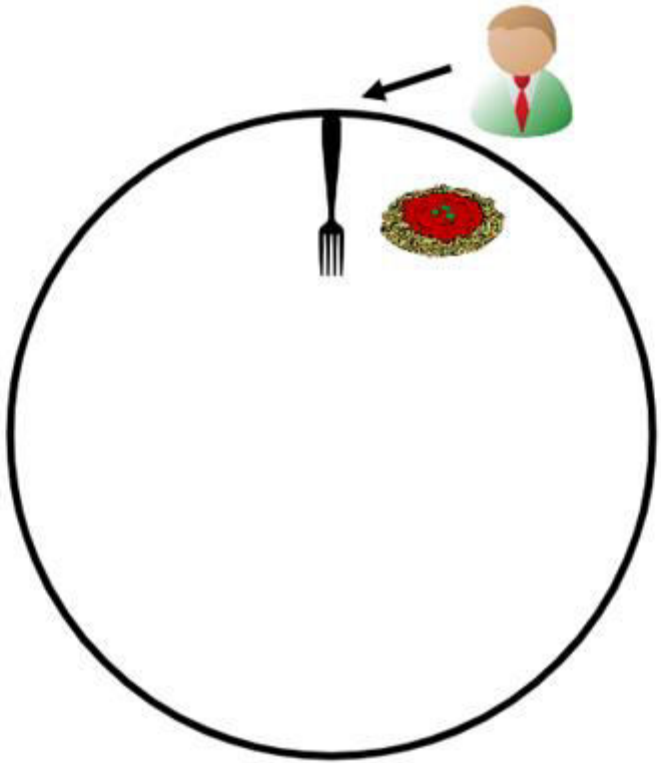


A philosopher wants to eat.

Eat is a task that required both forks (**locks**).

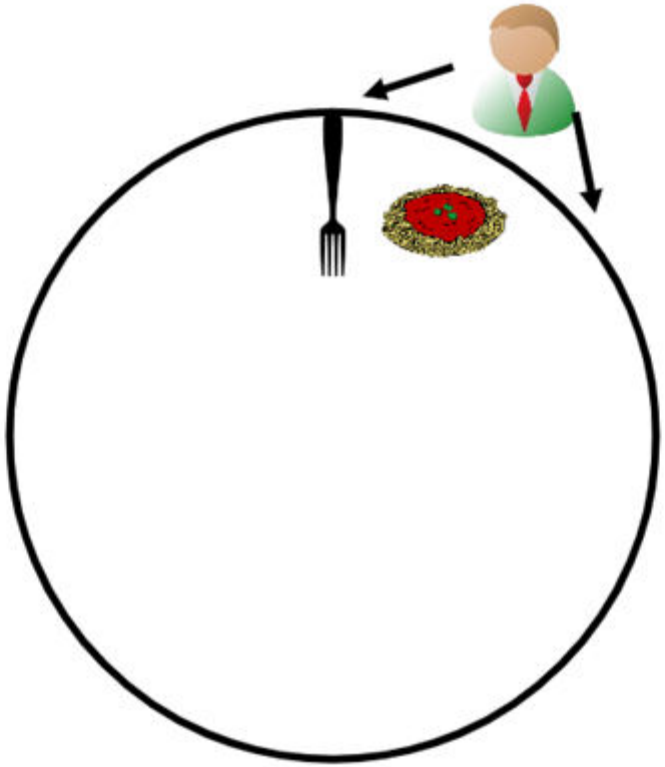
Eat could simply mean displaying a message.

The forks are locks



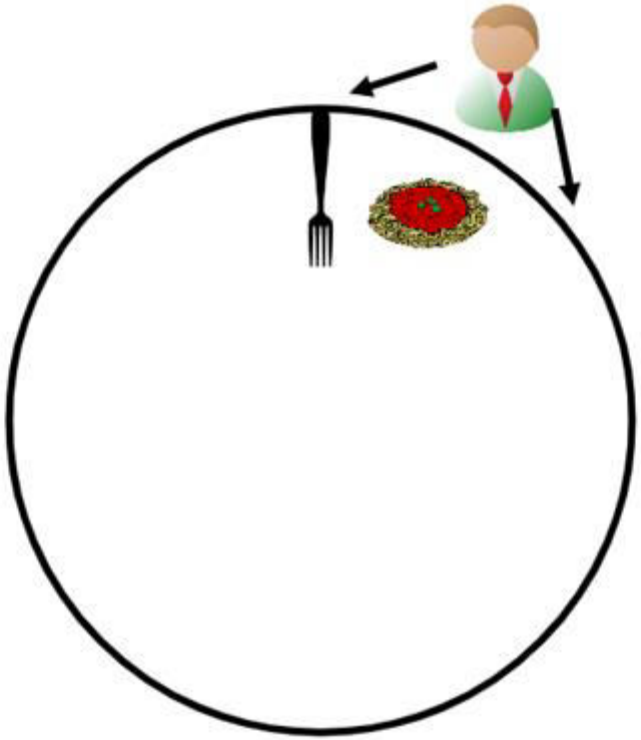
Take right fork (lock)

The forks are locks



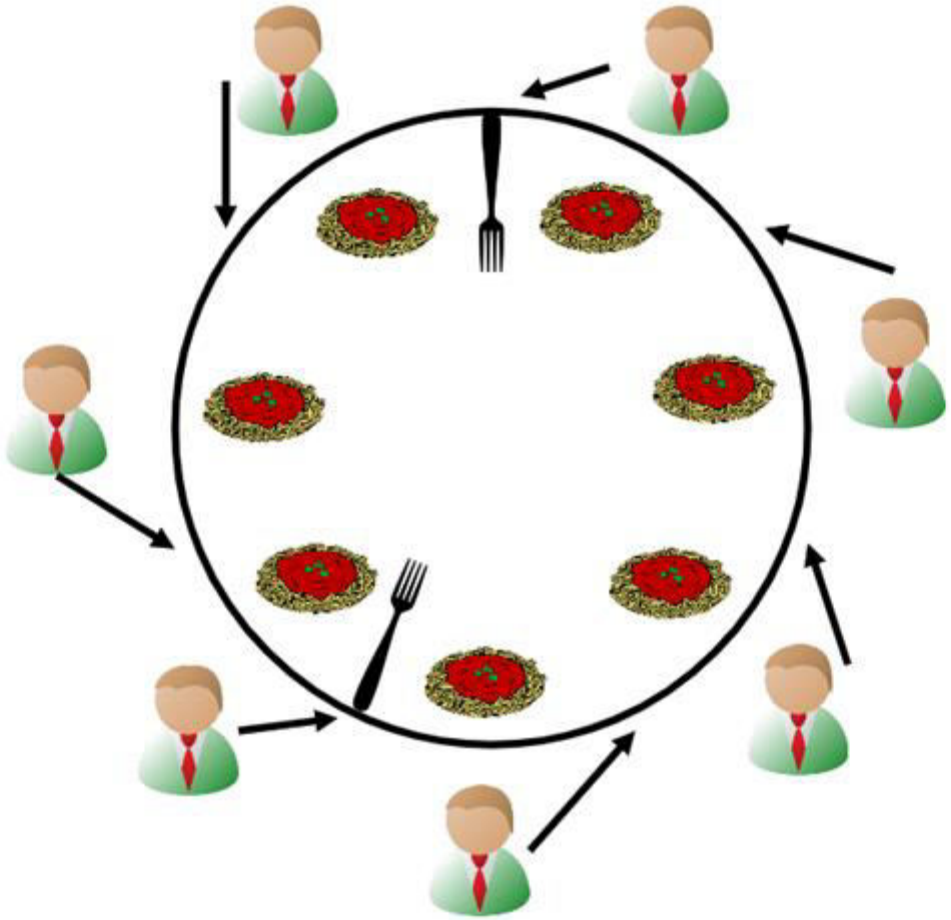
While holding right fork (lock) Take the left fork (lock)

The forks are locks

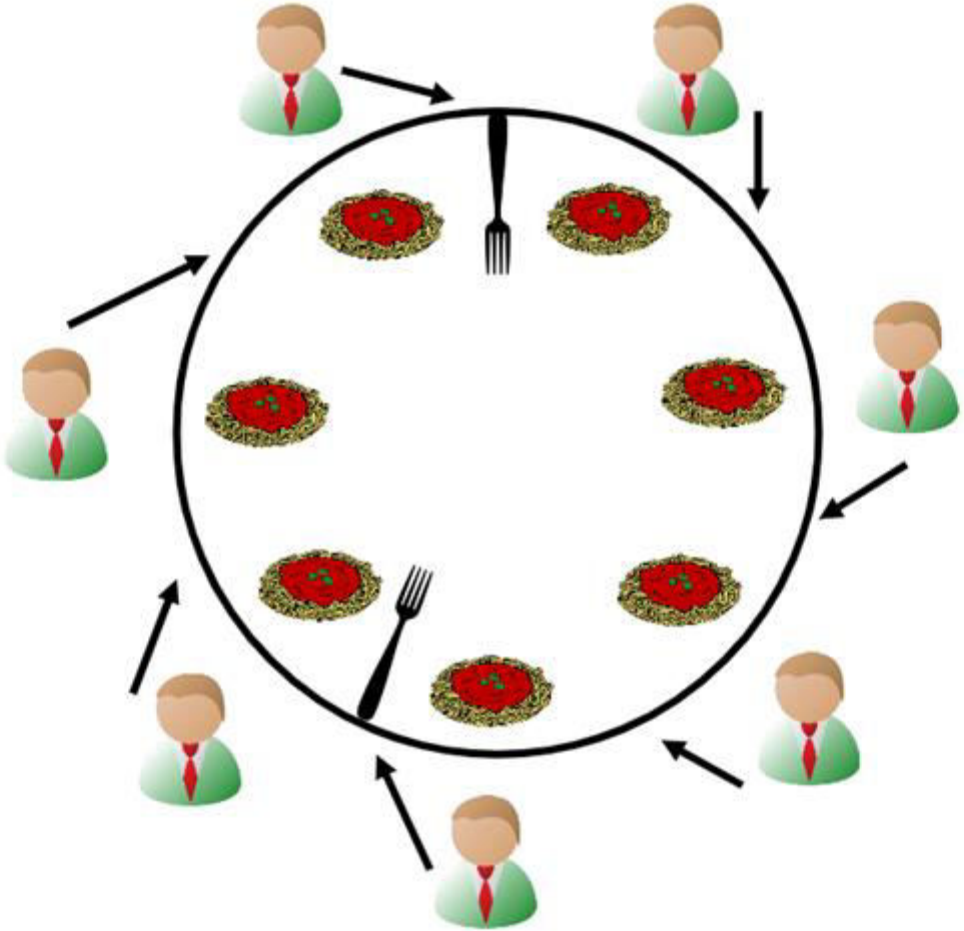


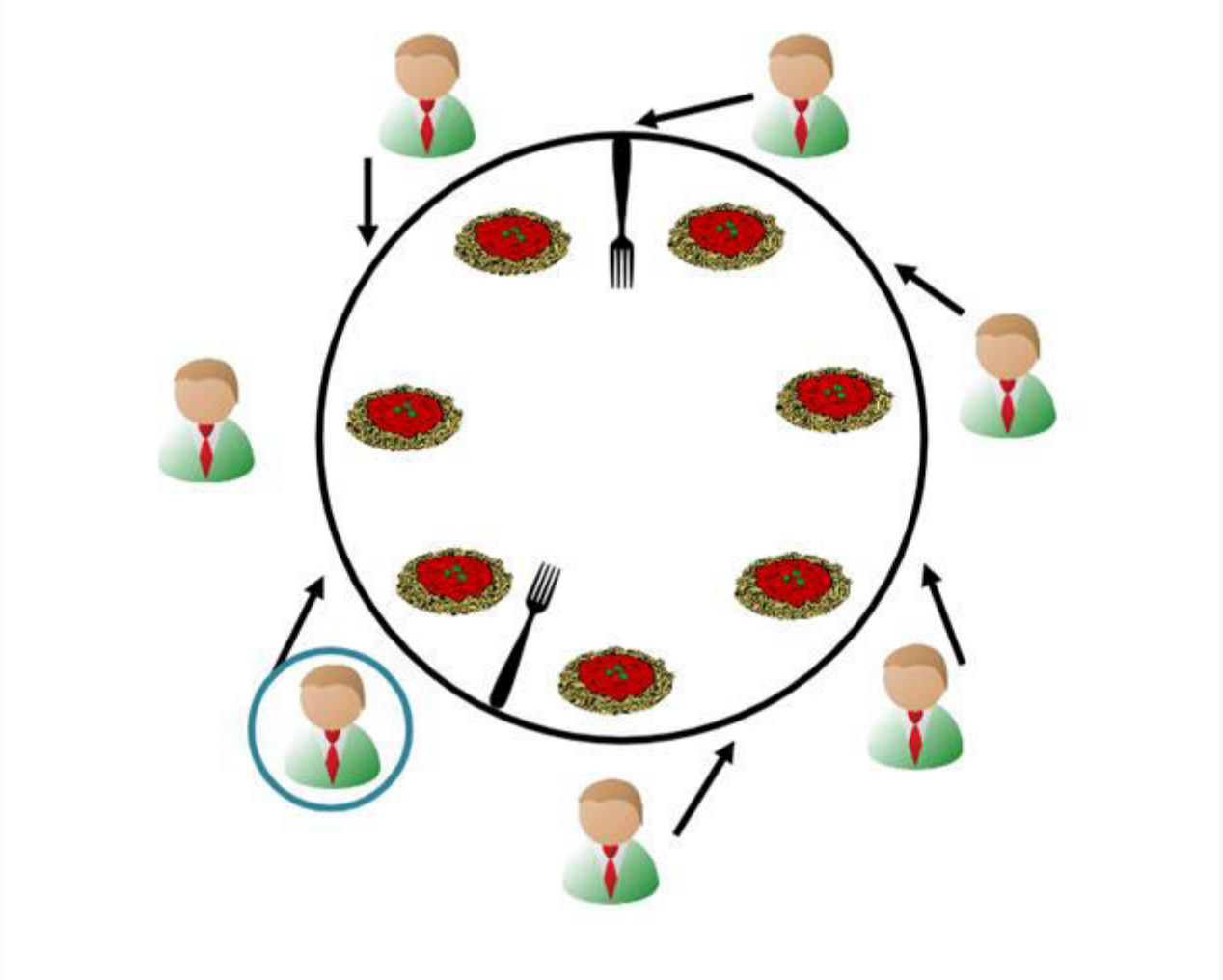
Now he can eat. Then he will let go of the forks (locks). This means someone else can take them.

If they all get the right fork (lock) first, they can all get stuck (we have a dead-lock)

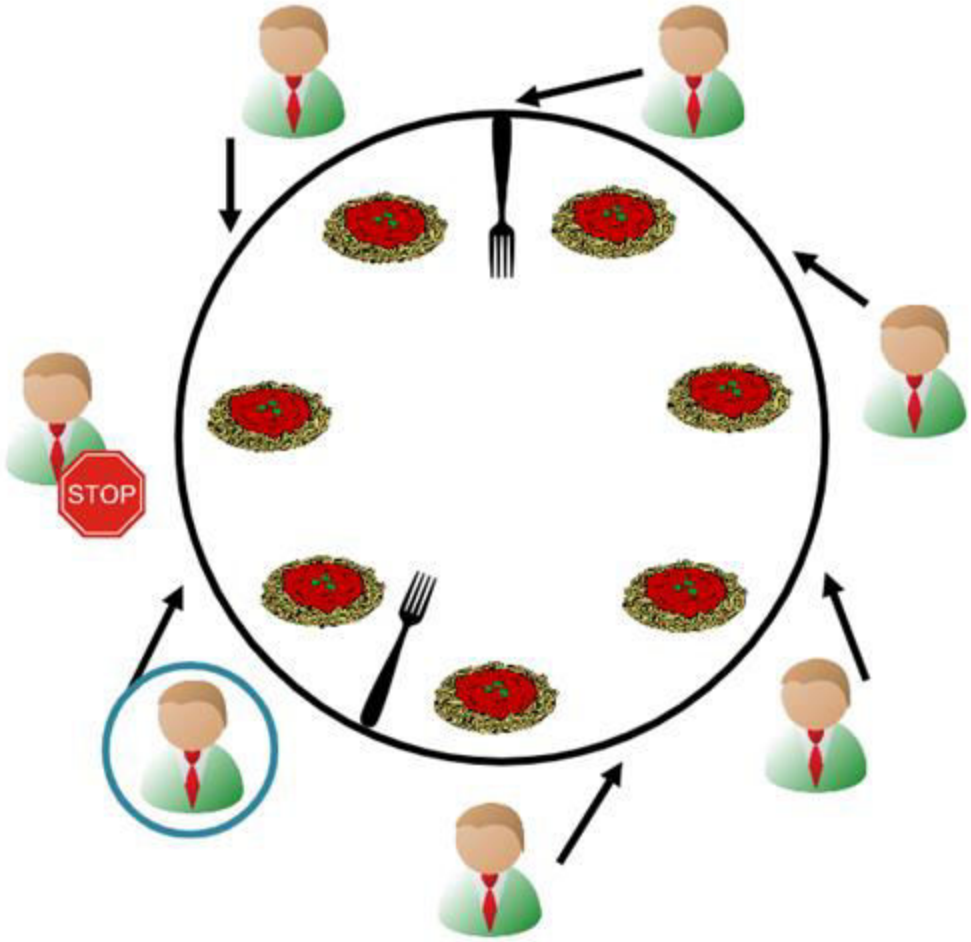


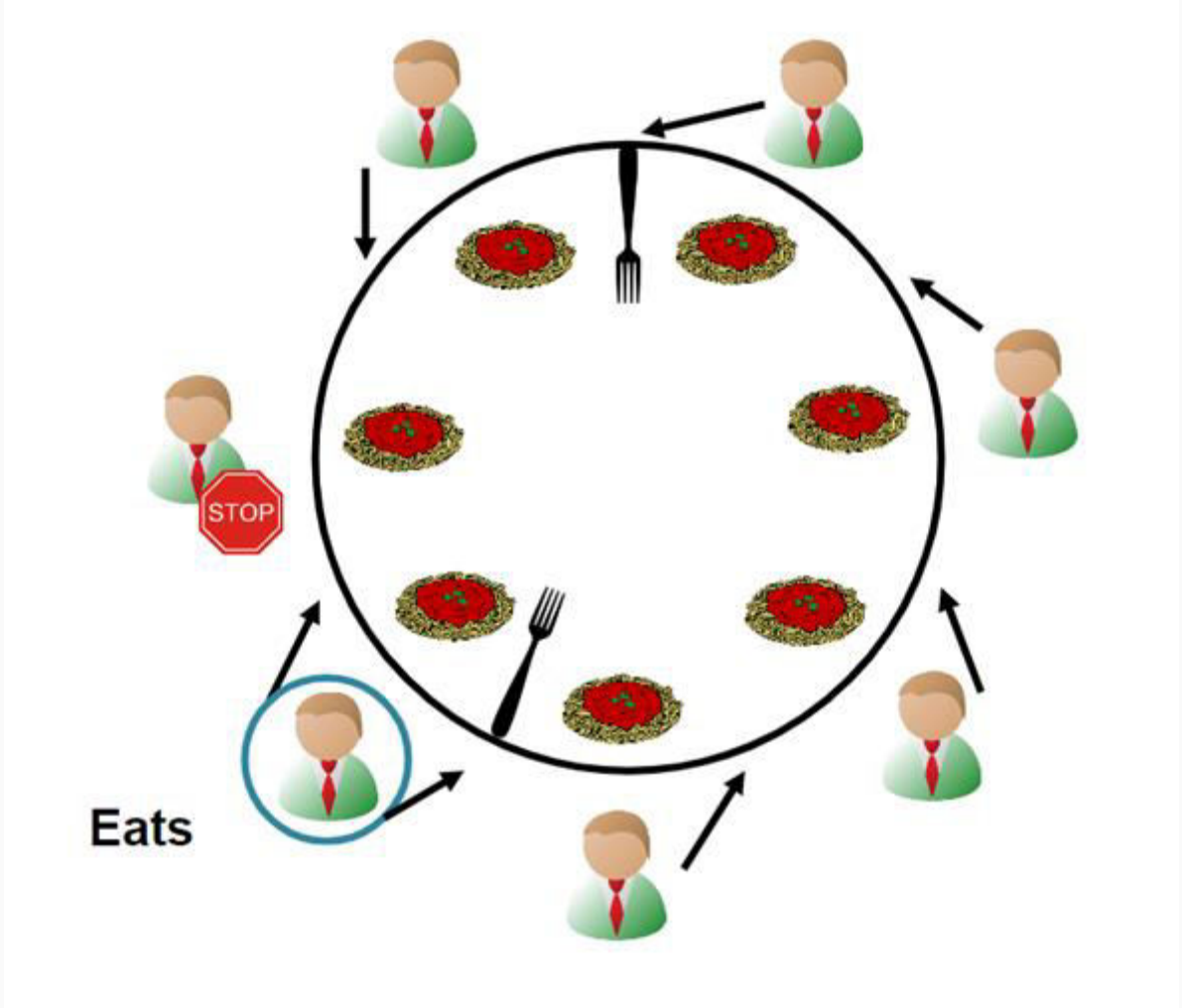
If they all get the left fork (lock) first, they can all get stuck (we have a dead-lock)

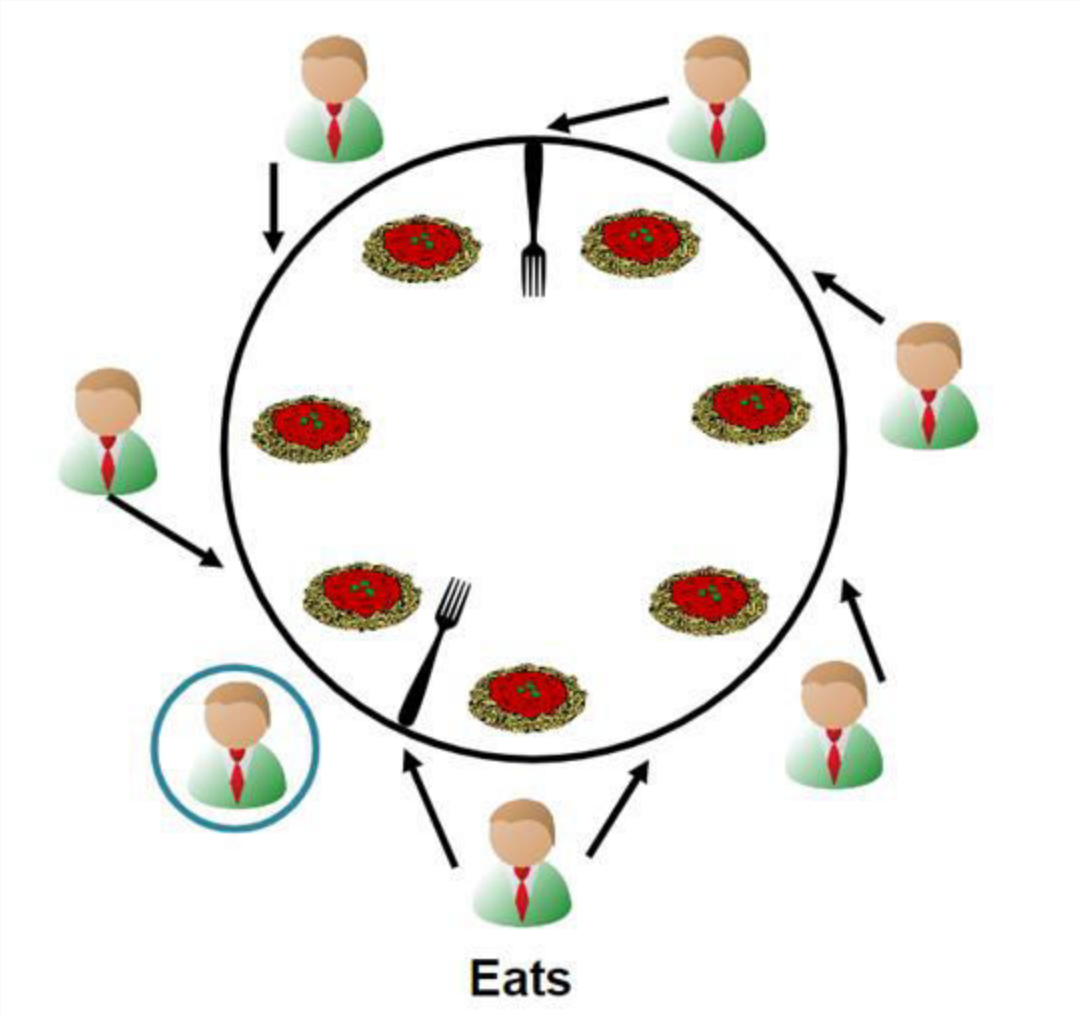


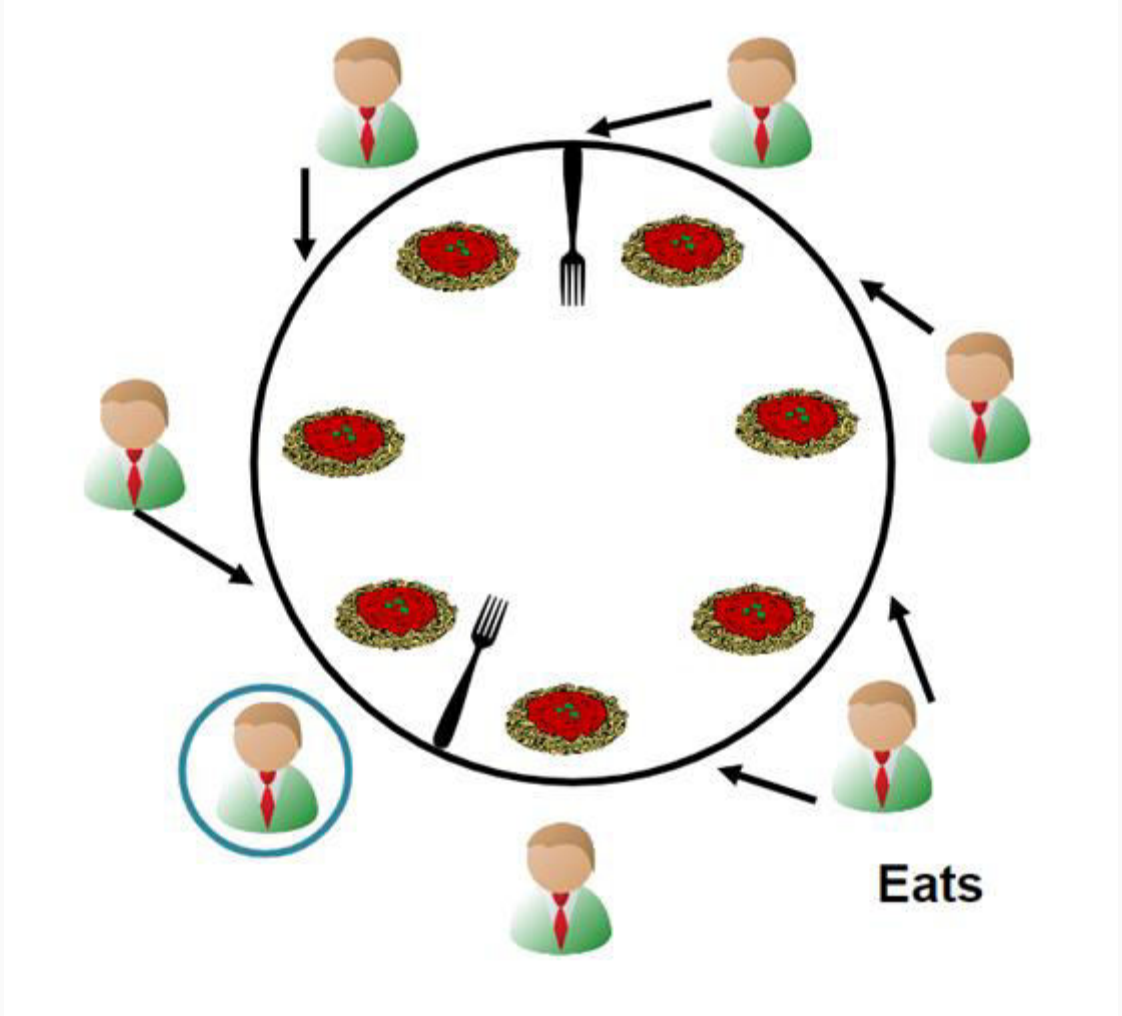


Can not take
any fork (lock)




STOP
Eats

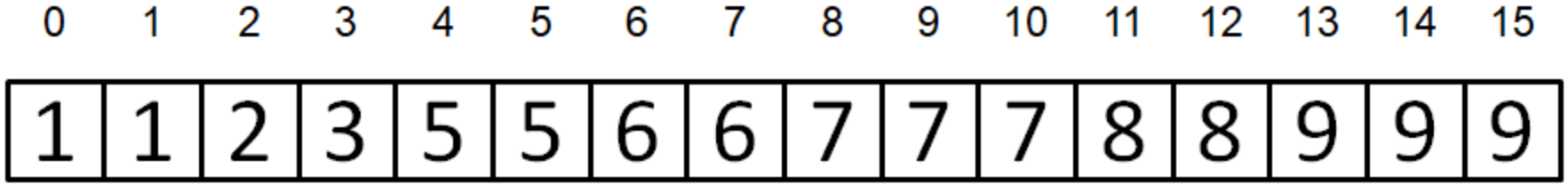

Eats


Eats

Căutare binară paralelă

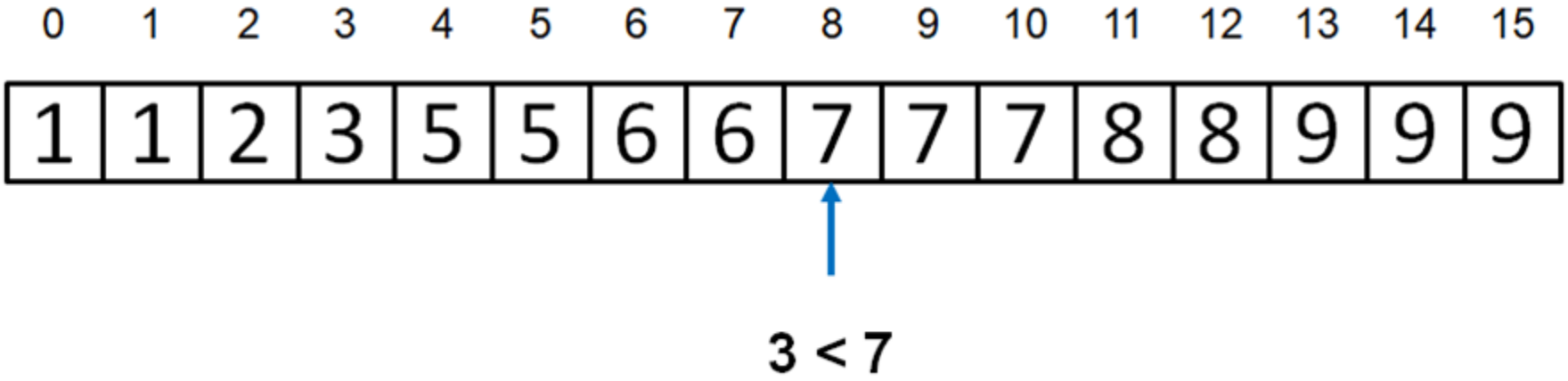
- șirul în care se caută dacă un element există trebuie să fie sortat
- se împarte șirul în P bucăți egale (aproximativ)
- fiecare thread verifică dacă elementul căutat se găsește între capetele bucății sale
- la pasul următor, se merge doar pe subșirul unde se găsește (eventual) elementul căutat
- pseudocod

Searching for 3



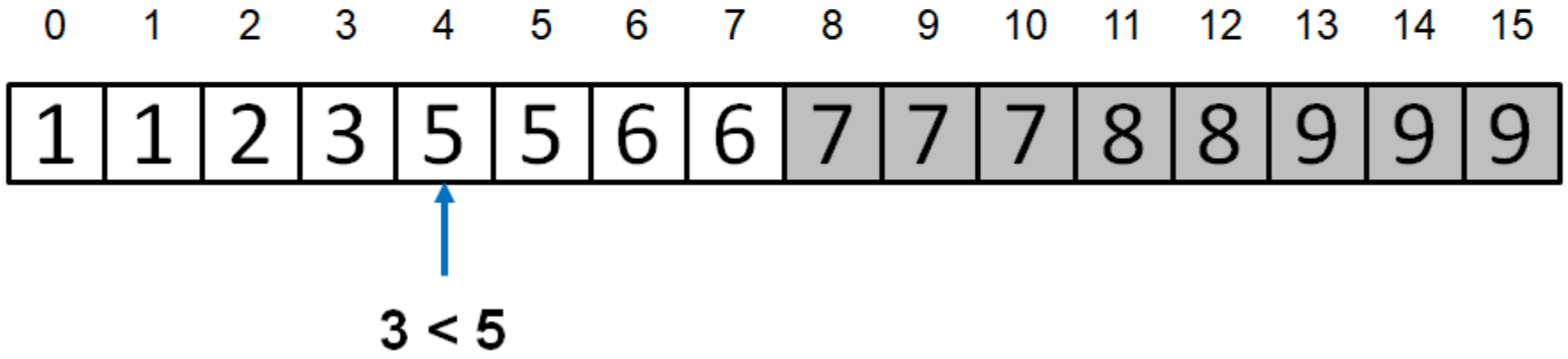
Searching for 3

Interest area 0 15



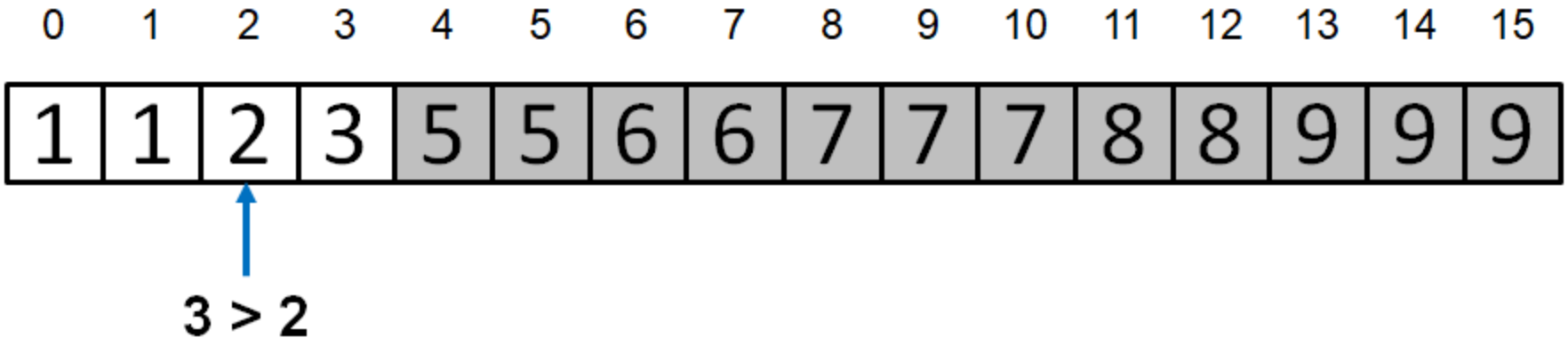
Searching for | 3 |

Interest area | 0 | 7 |



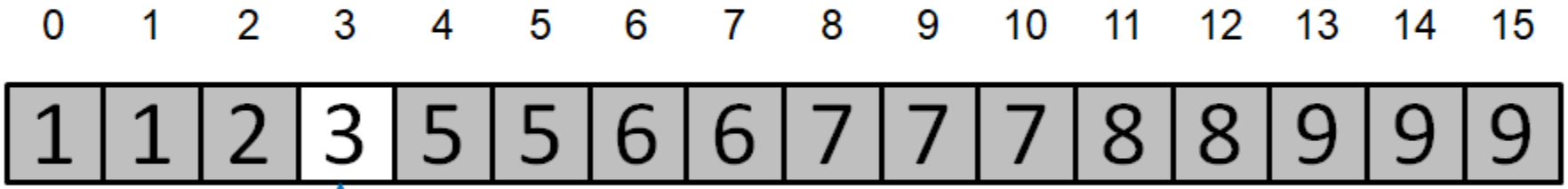
Searching for | 3 |

Interest area | 0 | 3 |



Searching for 3

Interest area 3 3

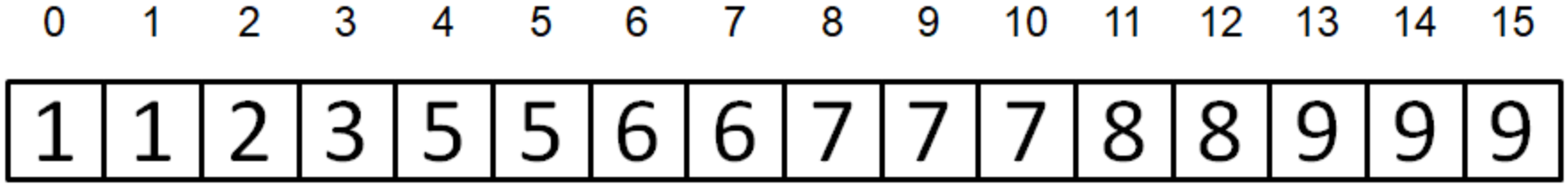


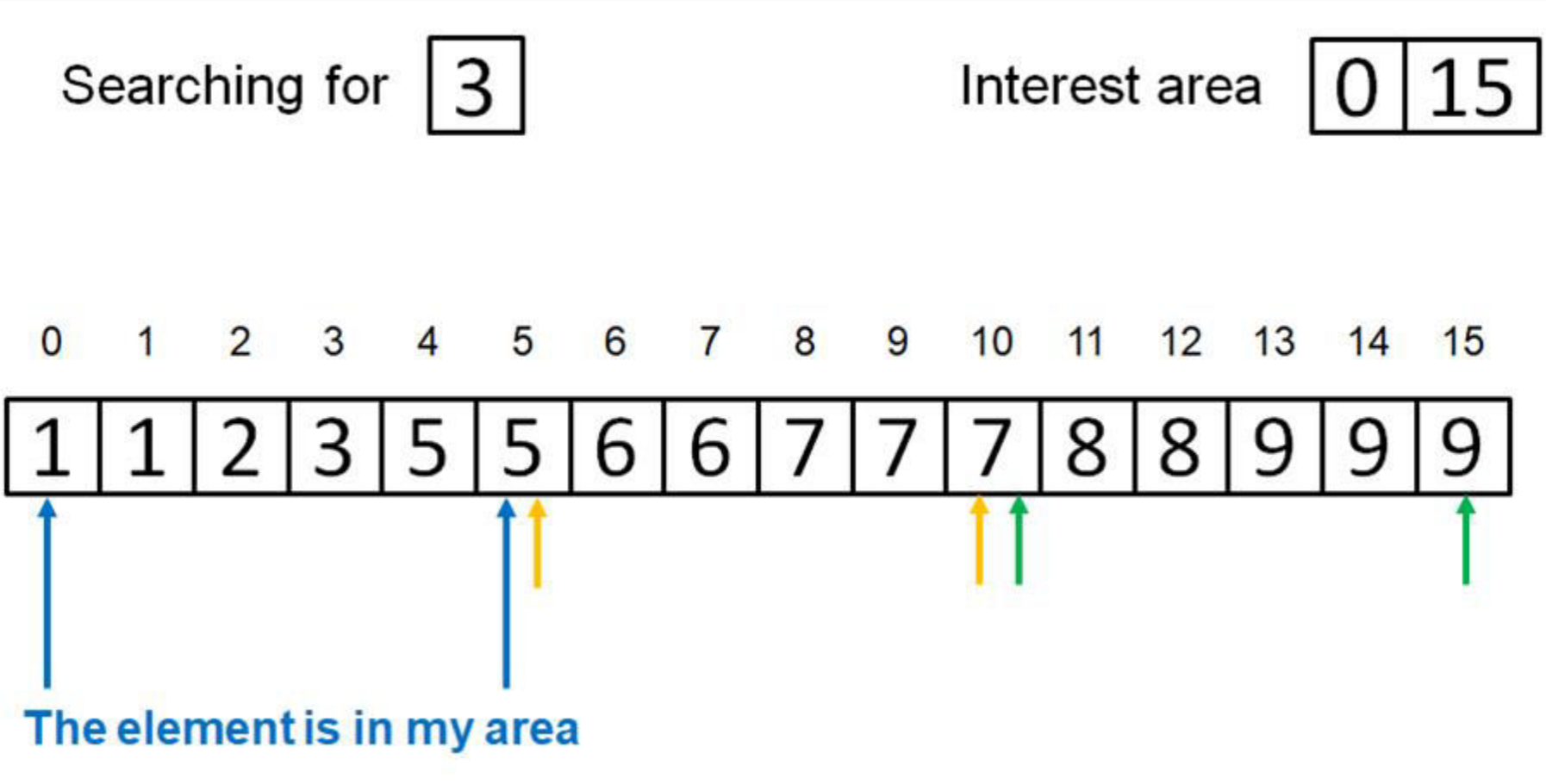
3 = 3

end

O($log_2(n)$) time

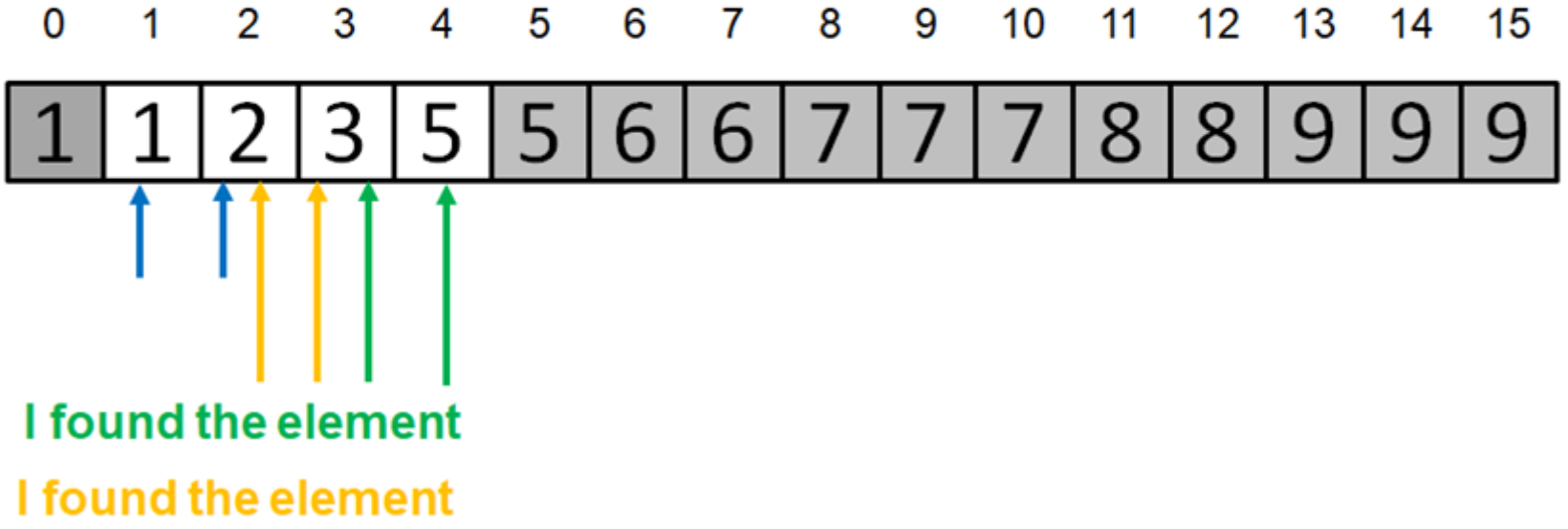
Searching for 3

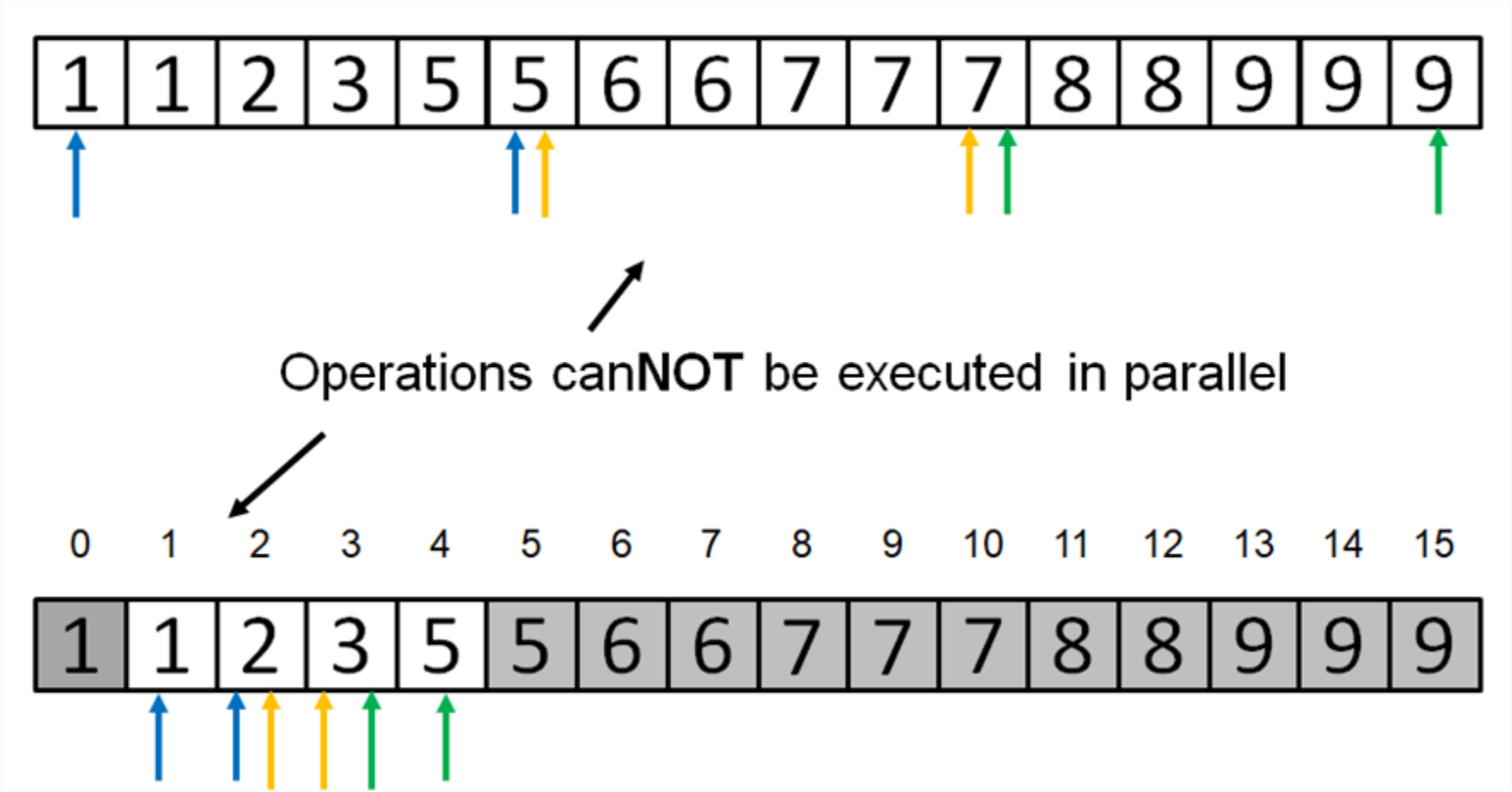


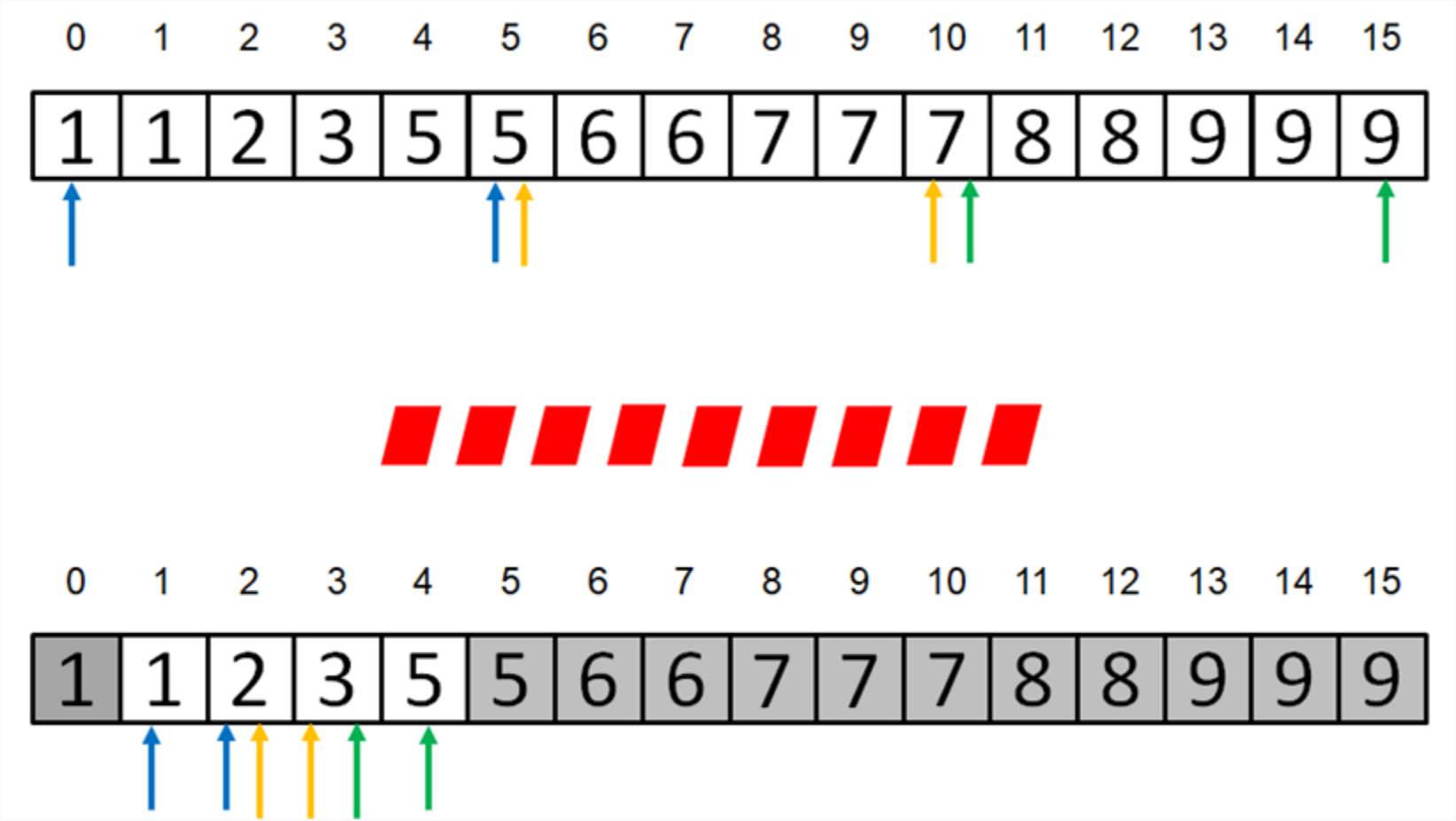

Searching for 3
Interest area 0 15
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
The element is in my area

Searching for 3

Interest area 1 4

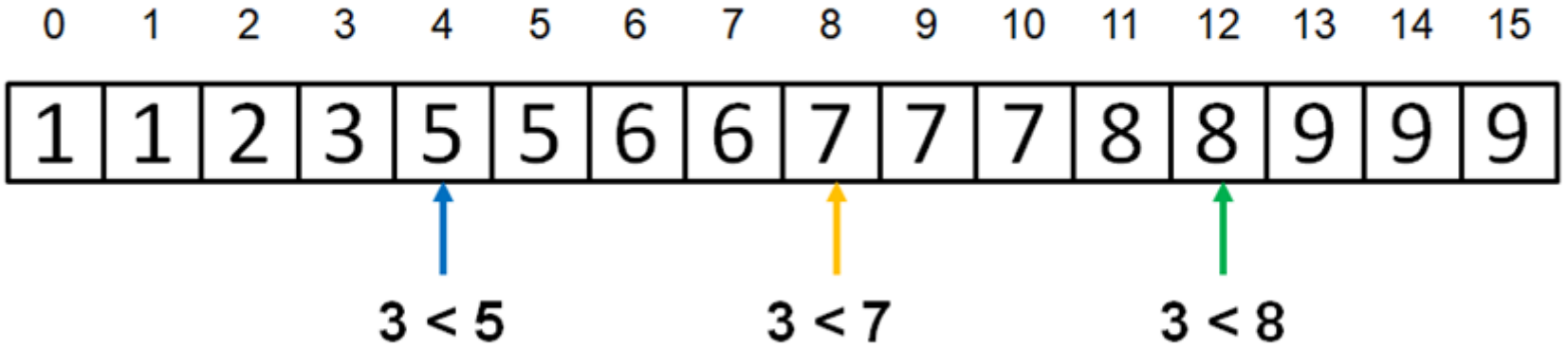



1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
Operations canNOT be executed in parallel
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9

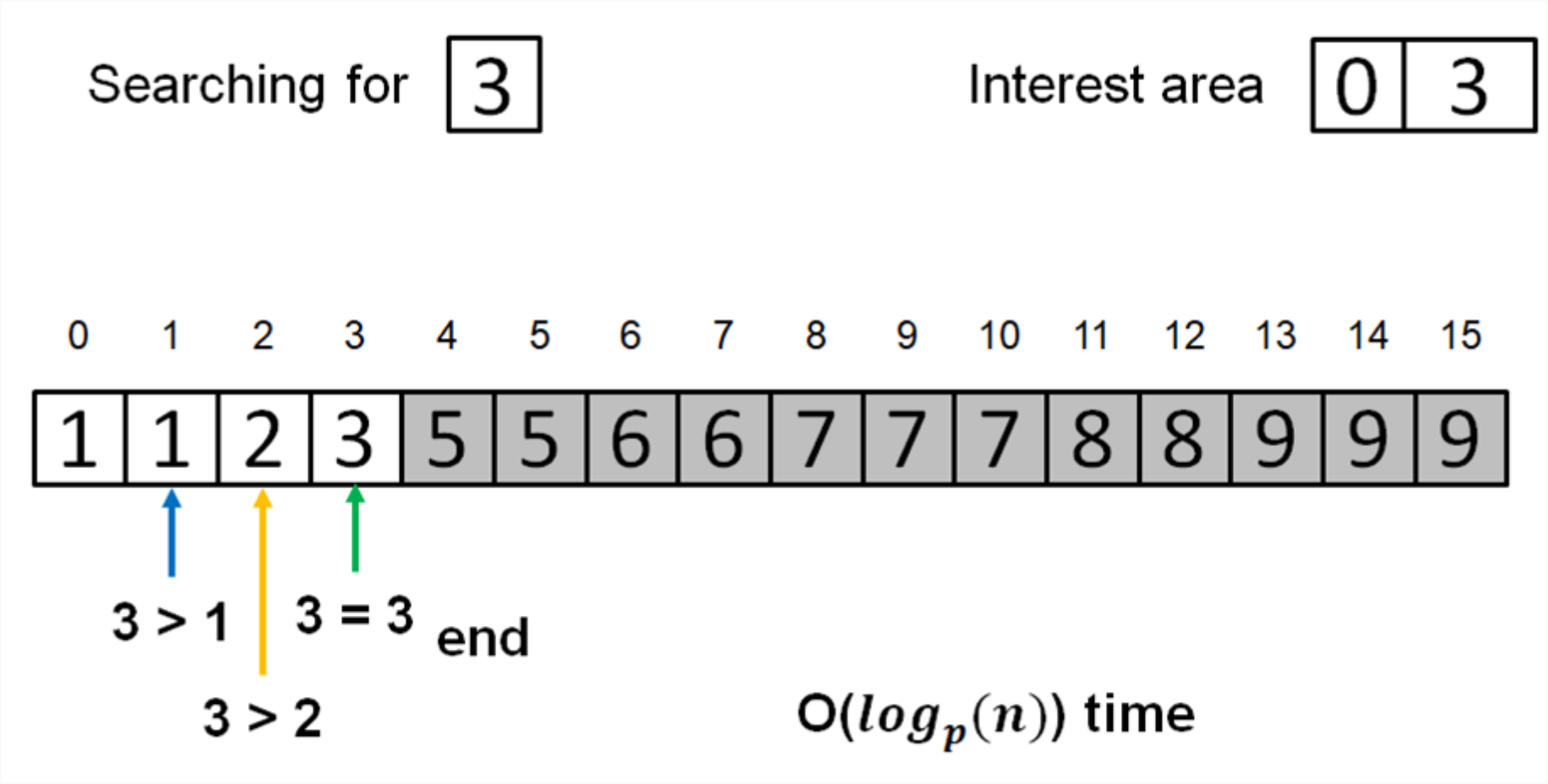

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9

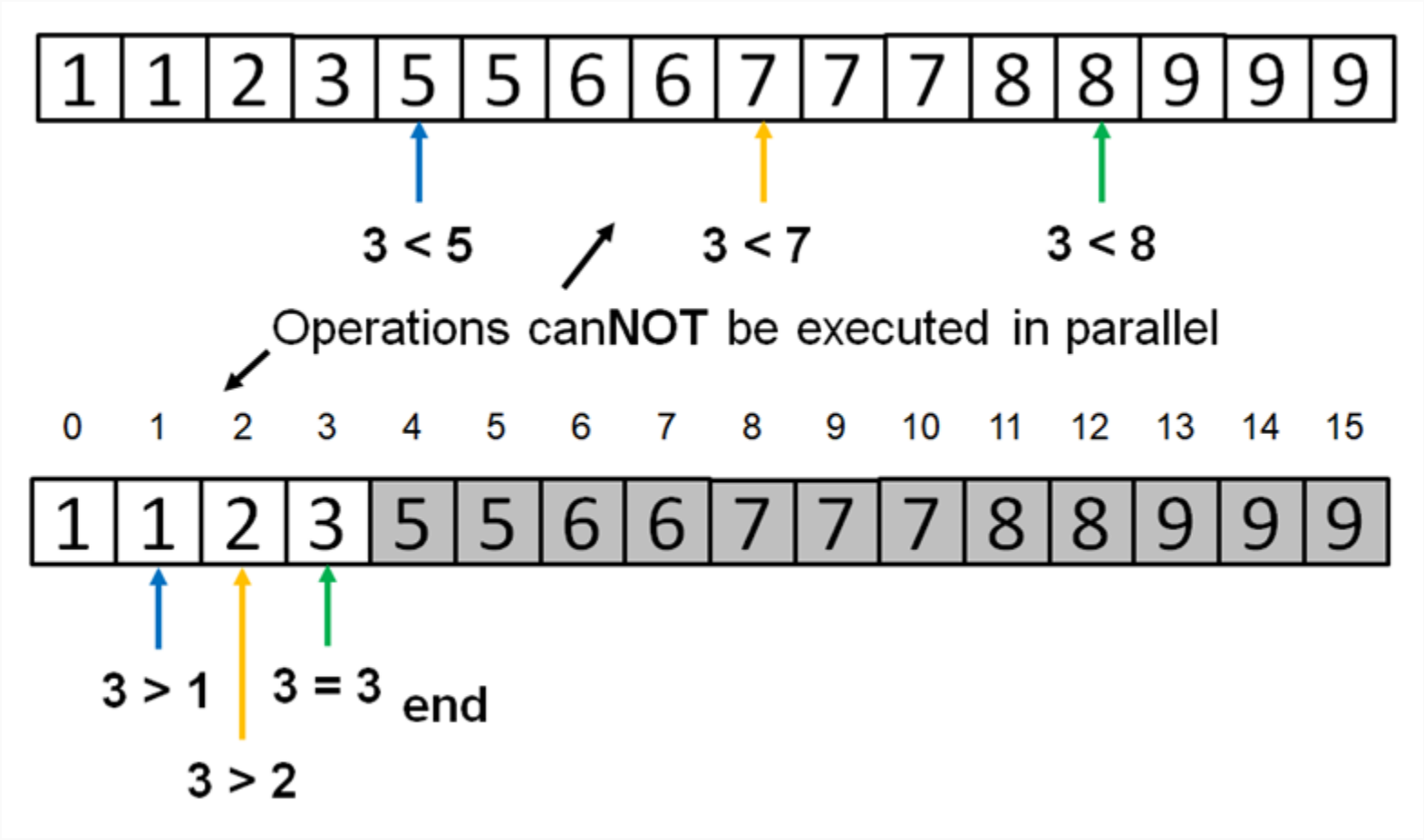
Searching for 3

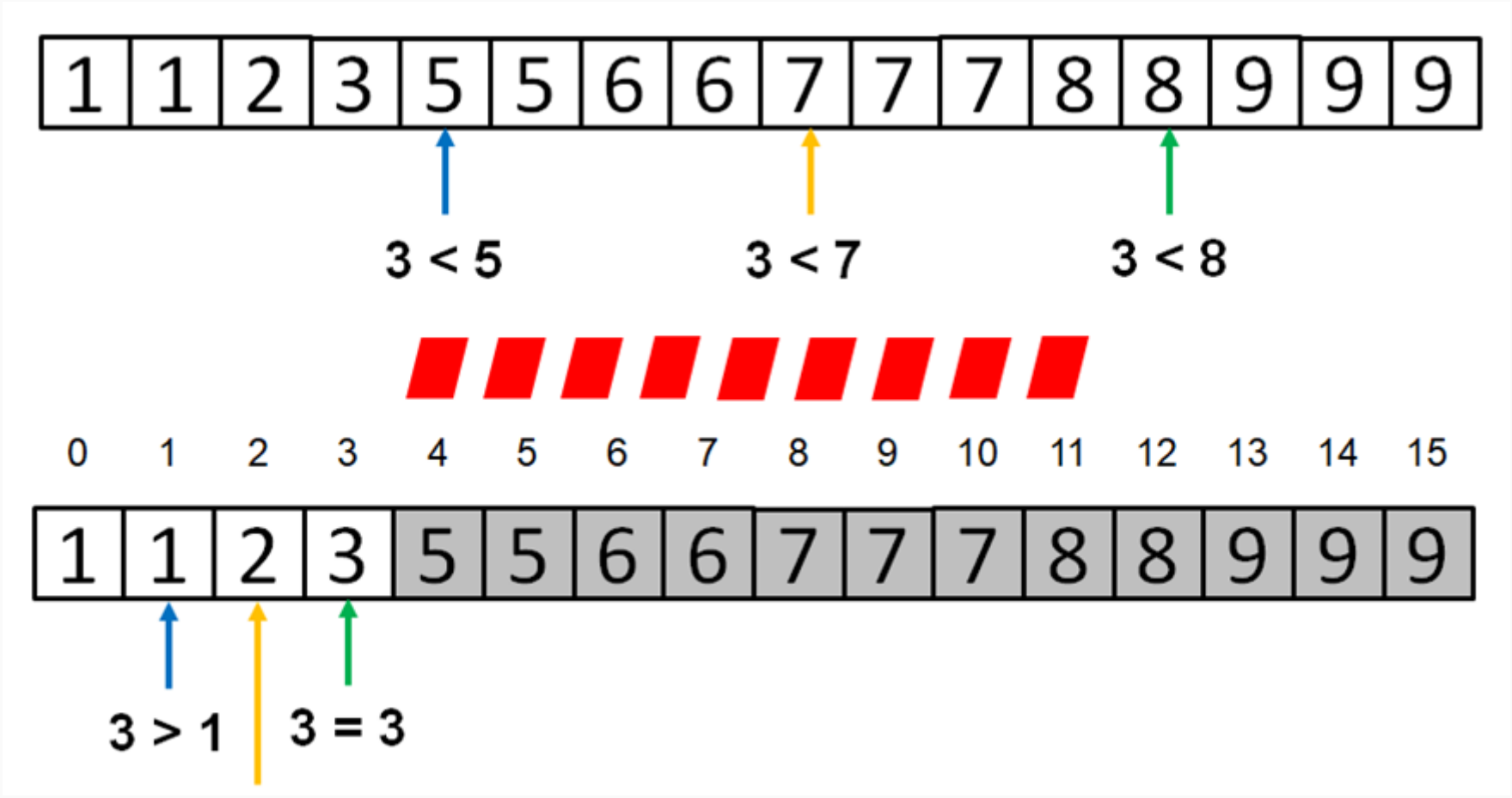
Interest area 0 15



Operations can be executed in parallel


Searching for 3
Interest area 0 3
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
3 > 1
3 = 3
end
3 > 2
O($log_p(n)$) time


1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
3 < 5
3 < 7
3 < 8
Operations canNOT be executed in parallel
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
3 > 1
3 = 3
end
3 > 2


1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
3 < 5
3 < 7
3 < 8
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1 1 2 3 5 5 6 6 7 7 7 8 8 9 9 9
3 > 1
3 = 3

Resurse

- schelet laborator updated

Întrebări

- ce reprezintă un deadlock? Dați un exemplu de caz / situație de deadlock
- modelați conceptul de producer consumer în viața reală

Laboratorul 6
Operații și structuri atomice

Semafor - remember lab 5

- `Semaphore sem = new Semaphore(x)`
- `acquire()` -> se decrementează x
- `release()` -> se incrementează x
- dacă x == 0, thread-urile care vor să acceseze zona critică așteaptă până când x > 0

Semafor - continuare

- la inițializarea semaforului, putem pune un număr negativ: `Semaphore sem = new Semaphore(-1)`
- dacă avem -1 la semafor și un thread vrea să între (acquire()), alte două thread-uri trebuie să facă release(), încât să avem valoare pozitivă, care face posibil un acquire()

Semafor - continuare

```
Semaphore sem = new Semaphore(-1);
```

T0	T1	T2

`// așteaptă la semafor`
`// după celelalte 2 thread-uri`
`sem.acquire();`
 	
`// sem = 1, deci poate trece`
`System.out.println(`
`"Am trecut de semafor!");`
	`do_work1();`
	`sem.release();`
	`// sem = -1 + 1 = 0`
		`do_work2();`
		`sem.release();`
		`// sem = 0 + 1 = 1`

AtomicInteger

- clasa AtomicInteger reprezintă o implementare atomică a clasei Integer (de care ne scapă de folosirea de locks, care este mai ineficientă ca timp de execuție - overhead de la acquire() și release() - demo)
- metode utile:
  - get() - întoarce valoarea de tip întreg
  - set(x) - îi setează valoarea la x
  - compareAndSet(y, x) - compară întregul cu x, iar, dacă cele două valori sunt egale, setează valoarea la y
  - addAndGet(delta) / getAndAdd(delta) - incrementează valoarea curentă cu delta și returnează valoarea nouă, respectiv pe cea veche

AtomicInteger - folosire

```
AtomicInteger nr = new AtomicInteger(4);

System.out.println(nr.get()); // 4

if (nr.compareAndSet(4, 7))

    System.out.println(nr.get()); // nr devine 7

System.out.println(nr.addAndGet(2)); // se aduna 2, deci devine 9

nr.set(1);

System.out.println(nr.get()); // 1
```

ConcurrentHashMap

- dacă am folosi un HashMap cu un singur lock pe întreaga structură, nu ar fi eficient deoarece doar un singur thread poate să acceseze HashMap-ul la un moment dat, chiar dacă thread-urile ar accesa elemente diferite
- dacă am folosi câte un lock pentru fiecare element, modul de lucru ar fi îngreunat
- soluție: ConcurrentHashMap - **nu admite chei null**

ConcurrentHashMap

- în ConcurrentHashMap, operațiile de get() și de put() blochează accesul la un anumit element printr-un lock (câte un lock pe fiecare element, deci nu e blocat accesul la întreaga structură) => două sau mai multe thread-uri pot face și get() și put(), dar avem garantat faptul că operațiile get() sunt executate după cele de put()

ConcurrentHashMap - metode

- `get(key)` - returnează valoarea asociată cu o cheie
- `contains(val)` - verifică dacă o valoare există în map
- `containsKey(key)` - verifică dacă o cheie există în map
- `put(key, val)` - adaugă o valoare la cheia specificată
- `putIfAbsent(key, val)` - adaugă o asociere doar dacă nu există deja o valoare pentru cheia respectivă; este întoarsă valoarea veche, dacă există, altfel e întors `null`
- `remove(key)` - şterge o asociere cheie-valoare
- `replace(key, val)` - modifică valoarea asociată cu o cheie.

ConcurrentHashMap - utilizare

```
ConcurrentHashMap<Integer, String> map = new ConcurrentHashMap<Integer,
String>();

map.put(0, "Zero");

map.put(1, "One");

System.out.println(map.get(1)); // One

String val = map.putIfAbsent(1, "Unu"); // nu se modifică nimic, val va fi "Unu"

val = map.putIfAbsent(2, "Two"); // se adaugă maparea 2-"Two", val va fi null
```

BlockingQueue

- `BlockingQueue` este o interfață ce extinde `Queue`, iar clasele care implementează `BlockingQueue` sunt colecții concurente:
  - `ArrayBlockingQueue` - bazat pe array, se specifică capacitatea maximă la inițializare
  - `LinkedBlockingQueue` - bazat pe LinkedList
  - `PriorityBlockingQueue` - coadă cu priorități, se poate specifica un Comparator la inițializare

Operație	Aruncă excepție	Valoare specială	Se blochează	Dă timeout
Inserare	add(e)	offer(e)	put(e)	offer(e, time, unit)
Ștergere	remove()	poll()	take()	poll(time, unit)
Examinare	element()	peek()	-	-

Collections.synchronized

- dacă dorim să lucrăm cu colecții clasice, precum ArrayList, și să avem operațiile sincronizate, putem să le instanțiem folosind:
  - Collections.synchronizedList (pentru liste): `List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());`
  - Collections.synchronizedMap (pentru dicționare): `Map<Integer, String> syncMap = Collections.synchronizedMap(new HashMap<>());`

Arbori concurenți

- inserția unui copil a unui nod într-un arbore binar (normal, nu de căutare) se face prin verificarea dacă are un nod-copil în stânga:

```
if (node.left == null)
    node.left = child;
else
    node.right = child;
```

Arbori concurenți - probleme de concurență

T0	T1
Verifică dacă există copil în stânga	Verifică dacă există copil în stânga
Condiția este adevărată, trece în prima ramură a if-ului	Condiția este adevărată, trece în prima ramură a if-ului
Inserează nodul în stânga	
	Inserează nodul în stânga → se suprascrie valoarea scrisă de T0

Corect ar fi ca T0 să insereze în stânga și T1 în dreapta

Arbori concurenți - soluție

- dacă am avea un lock pe tot arborele, doar un singur thread poate accesa arborele la un moment dat. Acest lucru nu este eficient, deoarece putem avea două thread-uri care pot insera noduri-copil în două noduri-părinte diferite.
- soluție: lock pe fiecare nod

Resurse

https://crunchify.com/hashmap-vs-concurrenthashmap-vs-synchronizedmap-how-a-hashmap-can-be-synchronized-in-java/

Laboratorul 7
Modelul Replicated Workers

Modelul Replicated Workers

- **Replicated Workers (Thread Pool)** este un pattern folosit în programarea concurentă
- avem două componente principale:
  - **pool** - coadă de task-uri
  - **workers** - thread-uri care execută task-urile

Cum funcționează Replicated Workers?

```
while (true)
    ia un task din pool
    execută task-ul, care poate genera task-uri suplimentare (forked)
    adaugă zero sau mai multe task-uri rezultate anterior în pool
```

Utilitate Replicated Workers

Este util când:

- nu putem să știm dimensiunea problemei în avans
- putem avea task-uri mai mici ca dimensiune, în acest sens este mai eficient să refolosim thread-urile decât să se creeze altele la un pas nou
- putem avea task-uri care nu sunt echilibrate ca workload, în acest fel thread-urile nu vor avea un workload echilibrat, fapt ce duce la întârzieri în cadrul programului

ExecutorService

- este o interfață în Java, prin care putem lucra cu task-uri asincrone, executate concurent în background
- putem să trimite un număr de cereri, însă ar fi ineficient dacă le-am trimite secvențial, așteptând după fiecare să termine
- **soluție**: lucrul asincron - trimiterea unei cereri, fără să așteptăm terminarea ei (să se execute în background), și apoi să trimitem alte cereri concurent
- este util când dorim toate soluțiile problemei (trimitem toate cererile) sau cel puțin o soluție

ExecutorService - metode

```
ExecutorService tpe = Executors.newFixedThreadPool(4); // instanțierea unui ExecutorService cu 4 workers (threads)

tpe.submit(new MyRunnable(...)); // punerea în pool a unui task ce trebuie să fie executat și lansarea acestuia în execuție

tpe.shutdown(); // oprește ExecutorService din a mai primi task-uri noi
```

ForkJoin

- framework-ul Fork / Join este o implementare a modelului Replicated Workers, cu o abordare *divide et impera*:
  - se împart recursiv task-urile în subtask-uri (**fork**) mai mici și independente, încât să poate fi executate asincron
  - se colectează recursiv rezultatele subtask-urilor într-un singur rezultat (**join**)
- **ForkJoinPool** este o implementare de ExecutorService, cu threads de tip **ForkJoinWorkerThread**, unde fiecare thread are propria sa coadă de task-uri
- fiecare thread, când are coada sa goală, poate să preia task-uri din coada altui thread sau din pool-ul global de task-uri

ForkJoinPool - tipuri de task-uri

Avem două tipuri de task-uri folosite la ForkJoinPool:

- `RecursiveAction` - task care nu returnează nimic
- `RecursiveTask<V>` - task care returnează un rezultat (dacă nu este întors nimic, putem folosi `RecursiveTask<Void>`, cu `return null`)

ForkJoinPool - metode

```
ForkJoinPool fjp = new ForkJoinPool(4); // creare de thread pool

fjp.invoke(new MyTask("files")); // adăugare de task ce va fi executat
de workers și se așteaptă rezultatul acestuia

fjp.shutdown(); // închiderea thread pool
```

RecursiveTask - metode

- compute() - calculul unui task (execuția acestuia)
- fork() - crearea unui nou task (dar nu și lansarea acestuia în execuție)
- join() - execuția unui task și așteptarea terminării acestuia

Laboratorul 8
Introducere în MPI

Despre programarea distribuită

- Până acum, ați lucrat cu fire de execuție (threads), care executau funcții și care aveau memorie partajată, având acces la variabile globale.
- În programarea distribuite, lucrăm pe procese de tip nod, care simulează o mașină de calcul. Spre deosebire de thread-uri, procesele au propria lor memorie (astfel ele nu partajează nimic), astfel ele au propriile lor variabile.
- Pentru ca un proces să informeze restul proceselor legate valorile unor variabile, acesta va trebui să trimită un mesaj care conține valorile respective către celelalte procese.

Ce este MPI?

- MPI (Message Passing Interface) reprezintă un framework (o bibliotecă) pentru programarea distribuită (mai concret, pe procese de tip nod)
- MPI este bazat pe modelul transmiterii de mesaje între procese prin intermediul unor canale de comunicație
- În MPI, nu avem memorie partajată, astfel toate variabilele sunt locale fiecărui proces.
- Pentru a transmite date de la un proces la altul, se folosesc funcții precum Send, Recv, Scather, Gather

Compilare și rulare

- Includere bibliotecă: `#include <mpi.h>`
- Compilare: `mpicc send_recv.c -o send_recv`
- Rulare: `mpirun -np 3 send_recv` (rulare cu 3 procese)
- Rulare cu un singur proces: `./send_recv`

Funcții legate de mediul MPI

- `MPI_Init(int *argc, char **argv)` - construiește environment-ul în care procesele se vor rula. De regulă se pasează ca argumente cele din linie din comandă (adică cele din `int main (int argc, char **argv)`)
- `MPI_Finalize()` - se termină procesele.
- `MPI_Comm_size(MPI_Comm comm, int *size)` - se determină numărul de procese (size) asociate cu un comunicator (de regulă `MPI_COMM_WORLD`)
- `MPI_Comm_rank(MPI_Comm comm, int *rank)` - determină rangul / id-ul unui proces

Tipuri de date în MPI

Tipul de date în MPI	Tipul de date în C
MPI_SHORT	short
MPI_INT	int
MPI_LONG	long
MPI_LONG_LONG	long long
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short
MPI_UNSIGNED_LONG	unsigned long

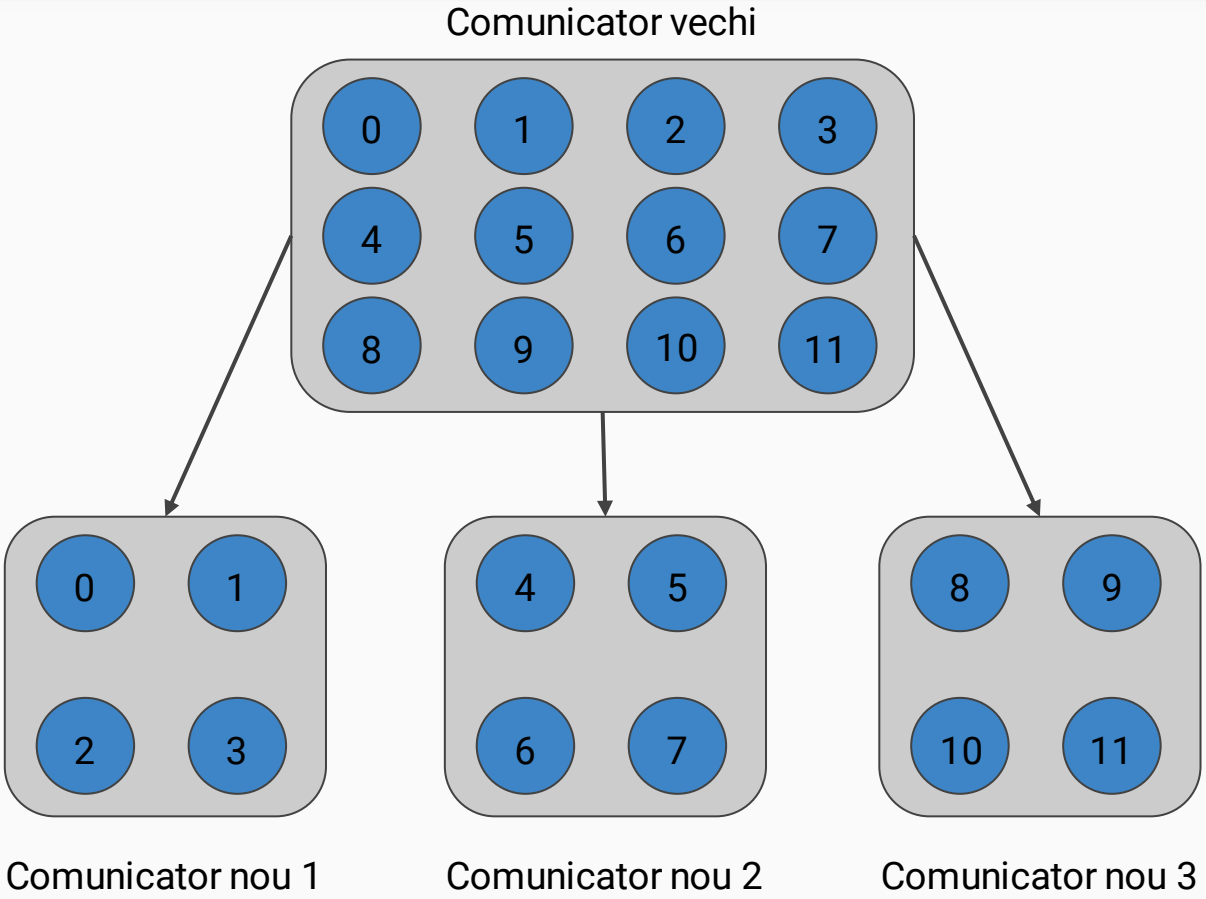
MPI_Comm_split

Putem crea comunicatoare dintr-un alt comunicator folosind funcția MPI_Comm_split, care împarte un comunicator în mai multe comunicatoare mai mici. Semnătura funcției este următoarea:

`int MPI_Comm_split(MPI_Comm comm, int color, int key, MPI_Comm * newcomm)`, unde:

- comm - comunicatorul este împărțit
- color - identificator al noului comunicator, din care face parte un proces (de regulă rang_vechi_proces / dimensiune_comunicator_nou)
- key - noul rang al procesului în cadrul noului comunicator (de regulă rang_vechi_proces % dimensiune_comunicator_nou)
- newcomm - noul comunicator format

MPI_Comm_split



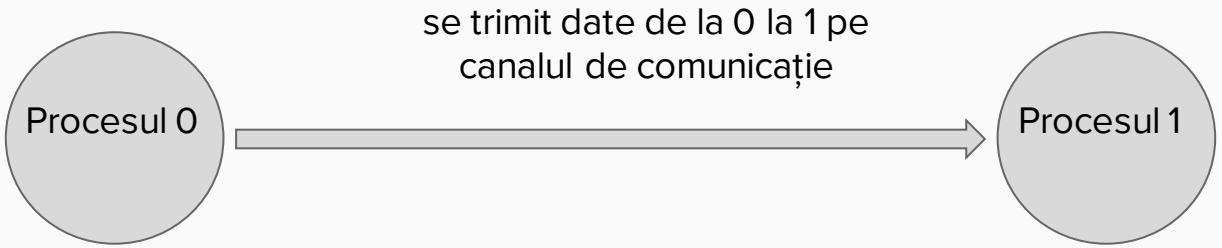
MPI_Send

- `MPI_Send(void* data, int count, MPI_Datatype datatype, int destination, int tag, MPI_Comm communicator)`
- un proces-sursă trimite date unui proces-destinație
- `data` = datele trimise
- `count` = numărul de date trimise
- `datatype` = tipul datelor trimise
- `destination` = id-ul procesului destinație
- `tag` = tag-ul mesajului trimis (adică al datelor trimise)
- `communicator` = comunicatorul (de regulă `MPI_COMM_WORLD`)

MPI_Recv

- `MPI_Recv(void* data, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm communicator, MPI_Status* status)`
- proces-destinație primește date de la un proces-sursă
- `data` = datele primite
- `count` = numărul de date primite
- `datatype` = tipul datelor primite
- `destination` = id-ul procesului sursă
- `tag` = tag-ul mesajului primite (adică al datelor primite)
- `communicator` = comunicatorul (de regulă `MPI_COMM_WORLD`)

Exemplificare send - receive (între două procese)



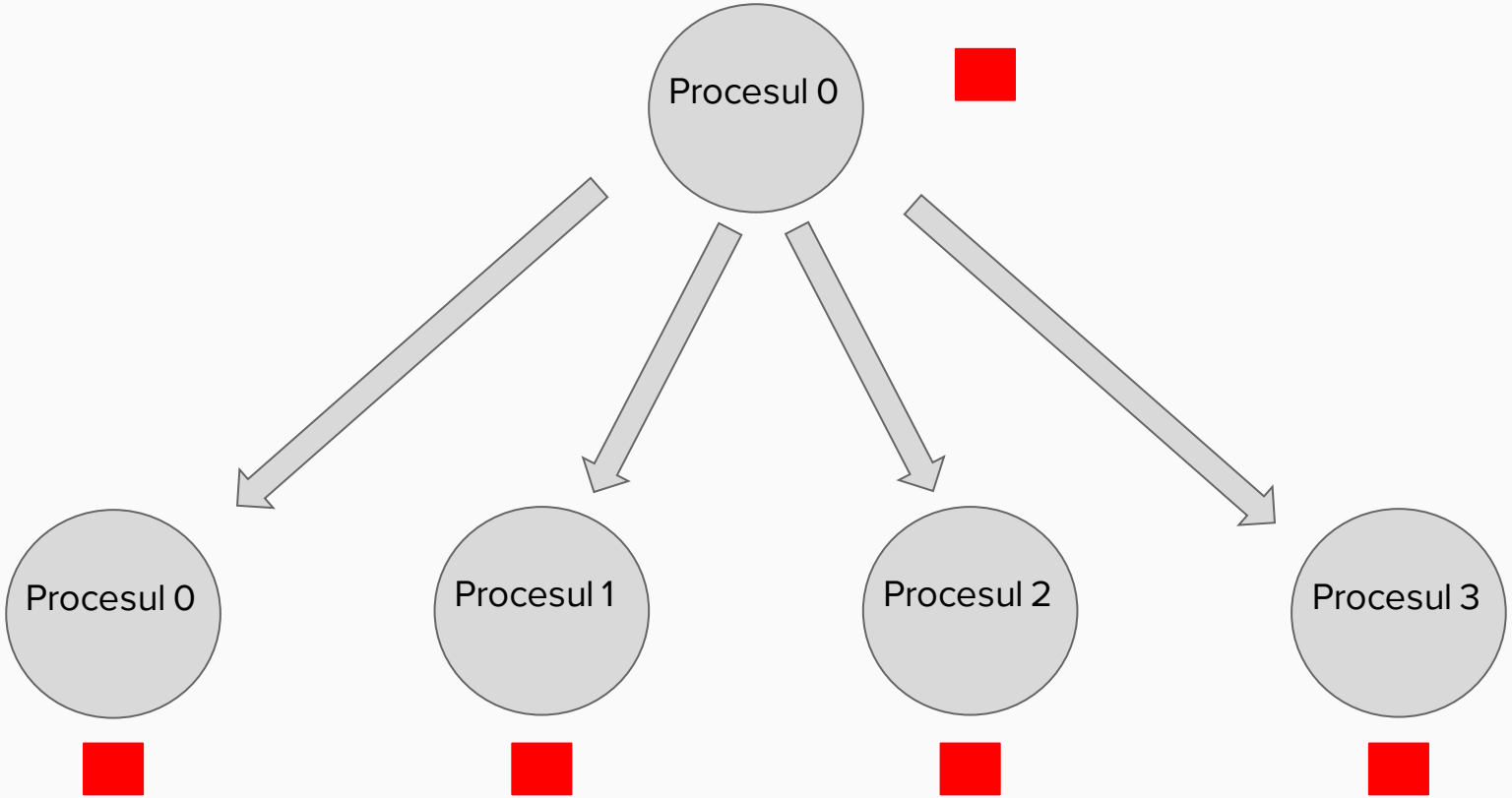
Execută MPI_Send (trimite date la procesul 1)

Execută MPI_Recv (primește date de la procesul 0)

MPI_Bcast

- `MPI_Bcast(void* data, int count, MPI_Datatype datatype, int root, MPI_Comm communicator)` - un proces trimite aceleași date tuturor proceselor din topologie (inclusiv lui însuși).
- `data` = datele trimise
- `count` = numărul de date trimise
- `datatype` = tipul datelor trimise
- `root` = id-ul procesului sursă
- `communicator` = comunicatorul (de regulă `MPI_COMM_WORLD`)

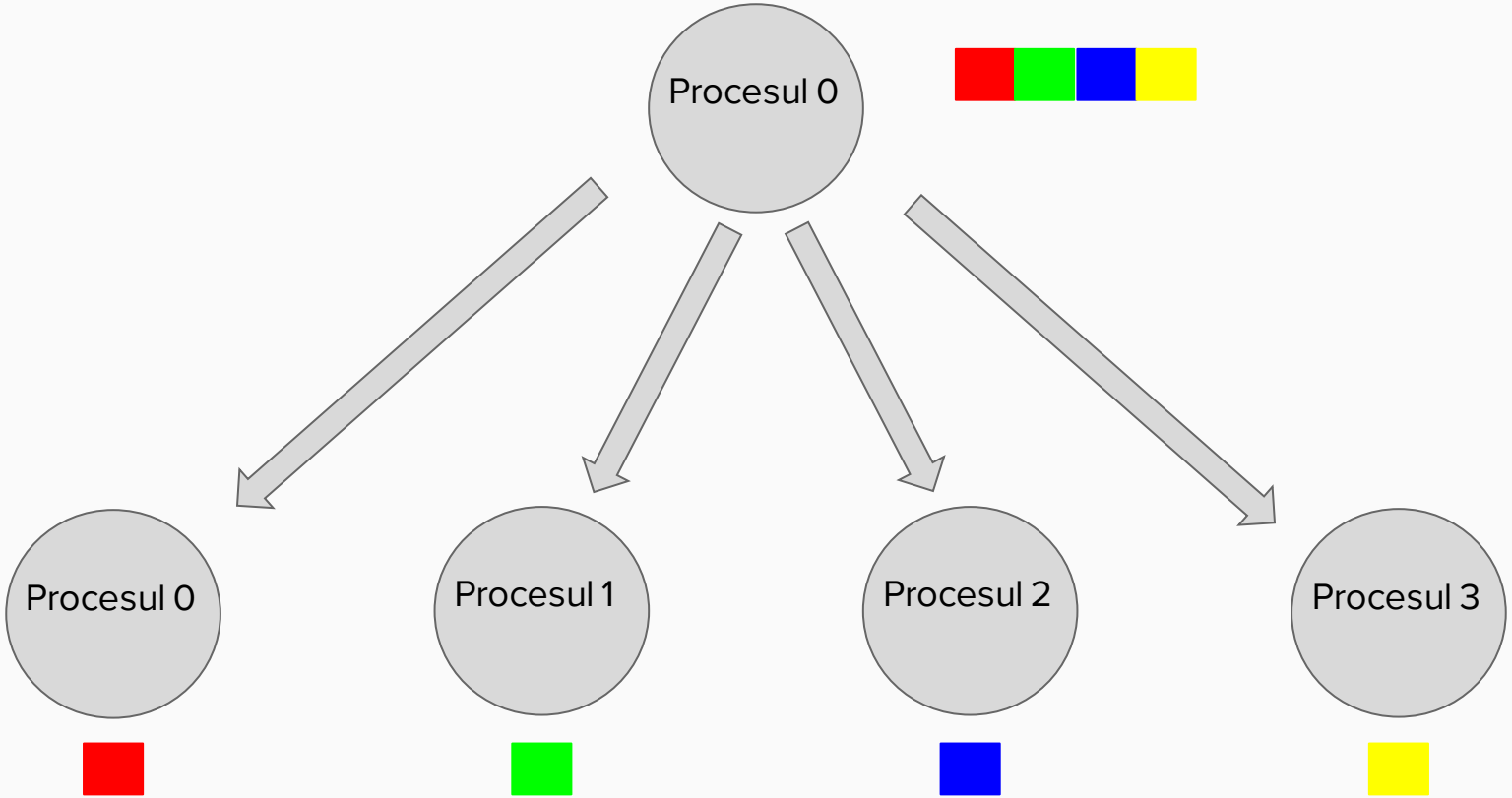
MPI_Bcast



MPI_Scatter

- `MPI_Scatter(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data, int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)` - un proces trimite câte o bucată dintr-un array / colecție tuturor proceselor în parte, inclusiv lui însuși (fiecare proces cu bucata sa din array)
- `send_data` = datele trimise
- `send_datatype` = tipul datelor trimise (de regulă recv_datatype == send_datatype)
- `send_count` = numărul de date trimise fiecărui proces în parte (de regulă recv_count == send_count)
- `recv_data` = datele primite
- `recv_count` = numărul de date primite de fiecare proces în parte (de regulă recv_count == send_count)
- `recv_datatype` = tipul datelor recepționate (de regulă recv_datatype == send_datatype)
- `root` = id-ul procesului sursă
- `communicator` = comunicatorul (de regulă `MPI_COMM_WORLD`)

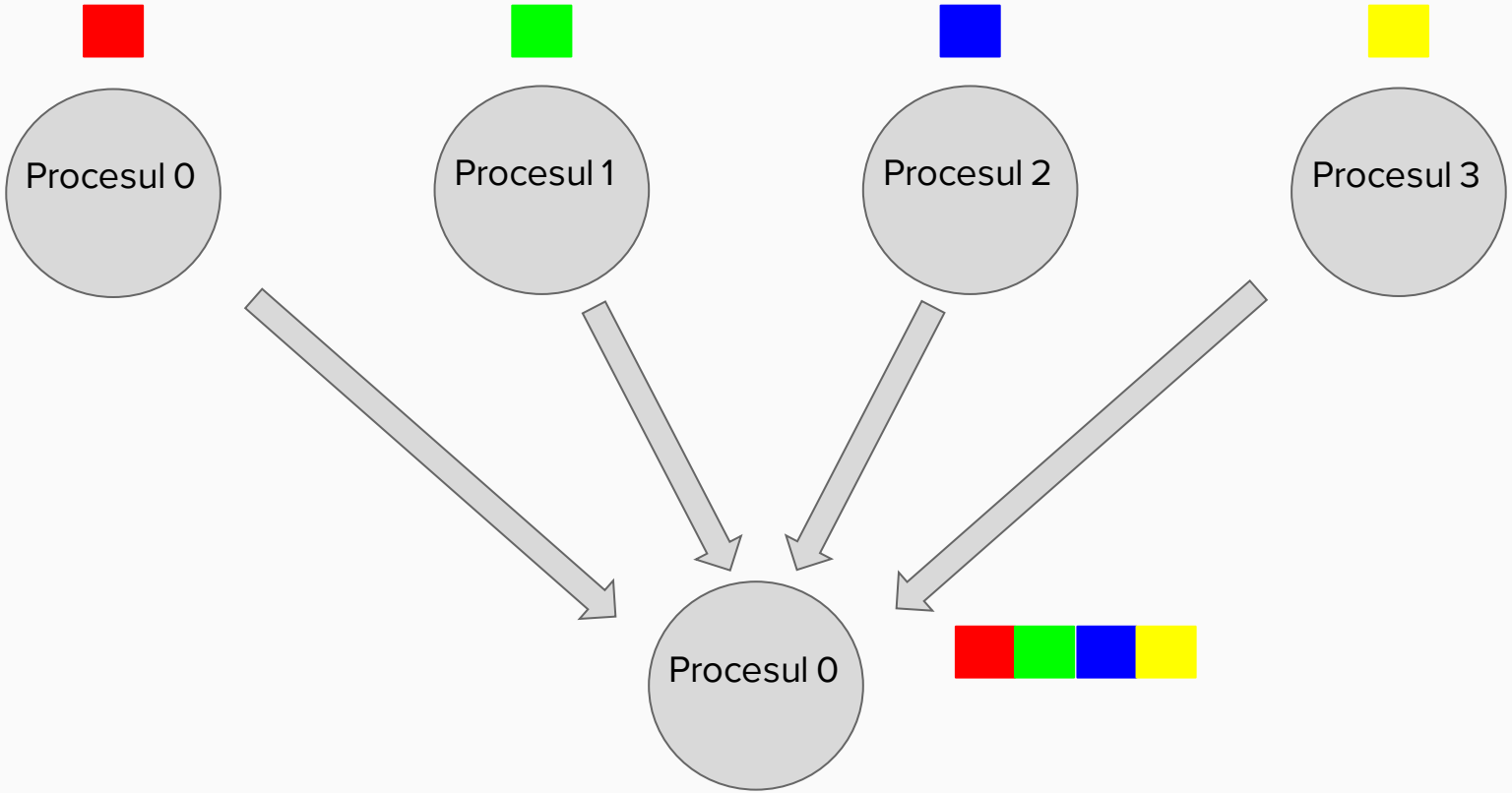
MPI_Scatter



MPI_Gather

- `MPI_Gather(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data, int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)` - un proces primește câte o bucată dintr-un array / colecție de la toate procesele, inclusiv de la însuși (fiecare proces cu bucata sa din array) - reprezintă inversul lui Scatter.
- `send_data` = datele trimise
- `send_datatype` = tipul datelor trimise (de regulă recv_datatype == send_datatype)
- `send_count` = numărul de date trimise fiecărui proces în parte (de regulă recv_count == send_count)
- `recv_data` = datele primite
- `recv_count` = numărul de date primite de fiecare proces în parte (de regulă recv_count == send_count)
- `recv_datatype` = tipul datelor recepționate (de regulă recv_datatype == send_datatype)
- `root` = id-ul procesului destinație
- `communicator` = comunicatorul (de regulă `MPI_COMM_WORLD`)

MPI_Gather



Algoritmul inel

- Algoritmul inel este un algoritm undă care funcționează pe rețele inelare de procese (cicluri Hamiltoniene), într-un sens unic.
- Fiecare proces are un vecin dedicat (next).
- Avem un proces inițiator, care inițiază transmiterea mesajului în cadrul rețelei.
- Inițiatorul trimite un mesaj (token) care este pasat de fiecare proces de-a lungul ciclului până când mesajul ajunge înapoi la inițiator, care apoi ia o decizie.

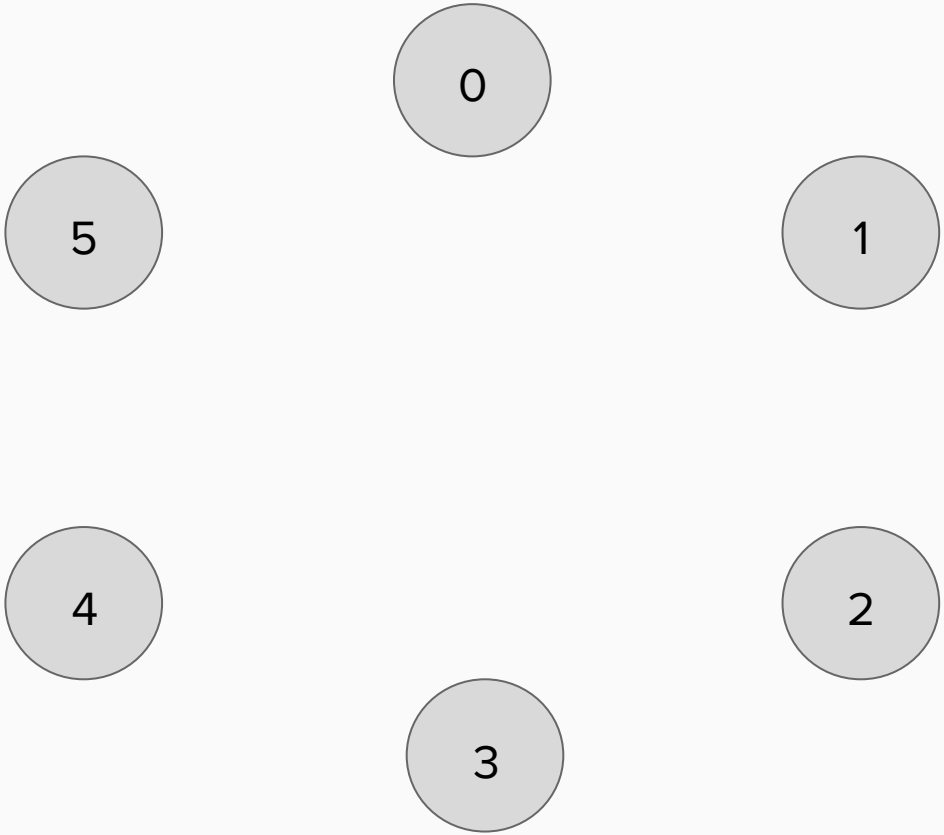
Algoritmul inel - pseudocod

```
chan token[1..n] (tok_type tok)

// initiator

process P[i] {

    tok_type tok;

    send token[next](tok); // trimite token-ul la urmatorul nod

    receive token[i](tok); // primeste token-ul de la nodul precedent

    decizie;

}
```

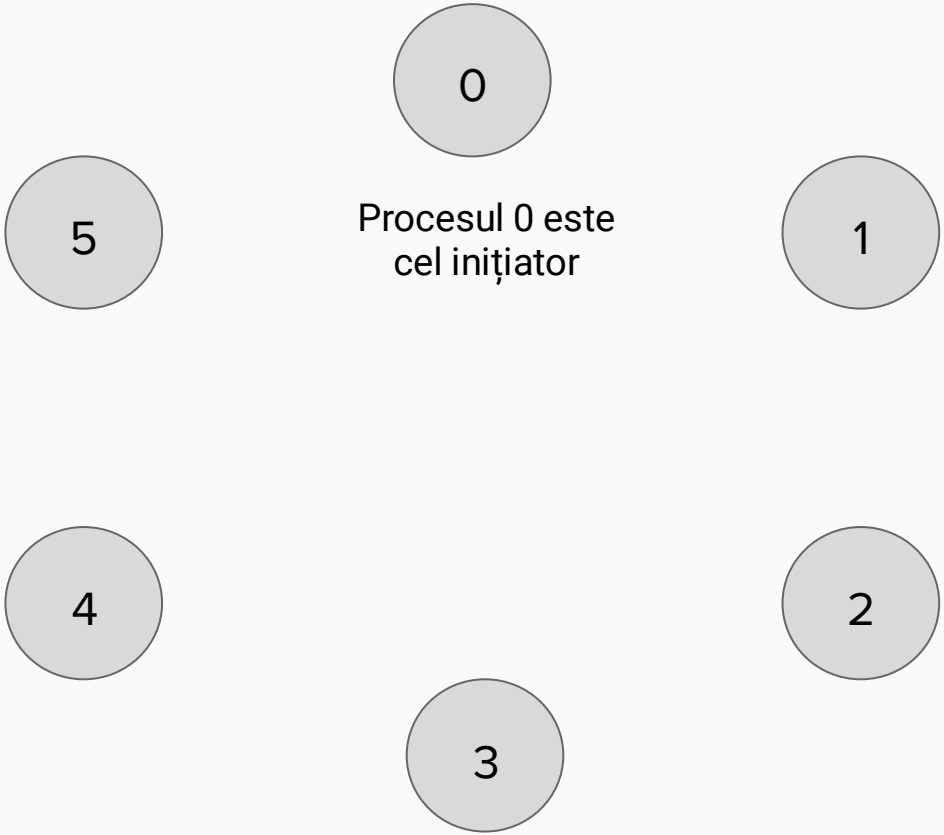
Algoritmul inel - pseudocod

```
// neinitiator

process P[k, k = 1..n, k != n] {

    tok_type tok;

    receive token[k](tok); // primeste token-ul de la nodul precedent

    send token[next](tok); // trimite token-ul la urmatorul nod

}
```

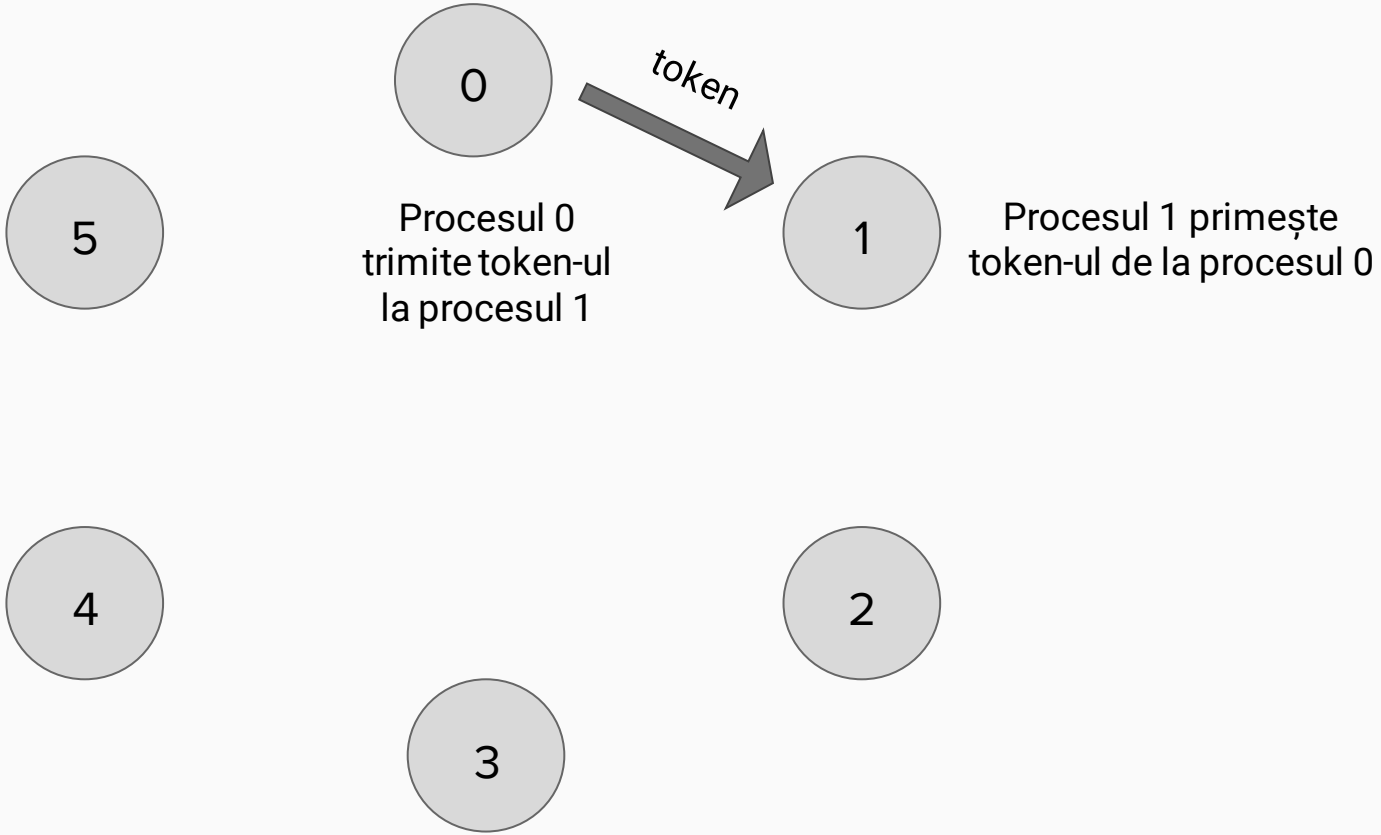
Algoritmul inel



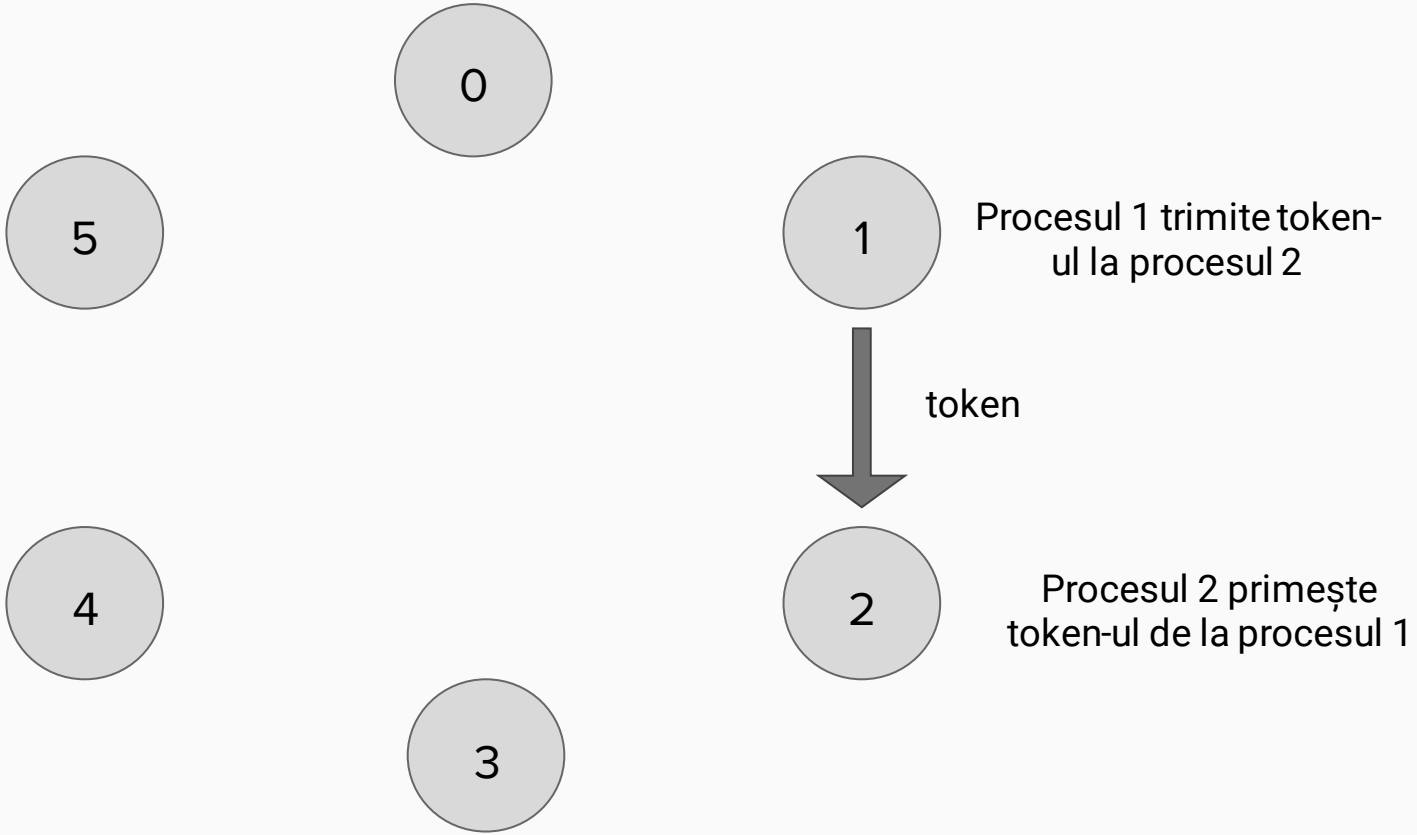
Algoritmul inel



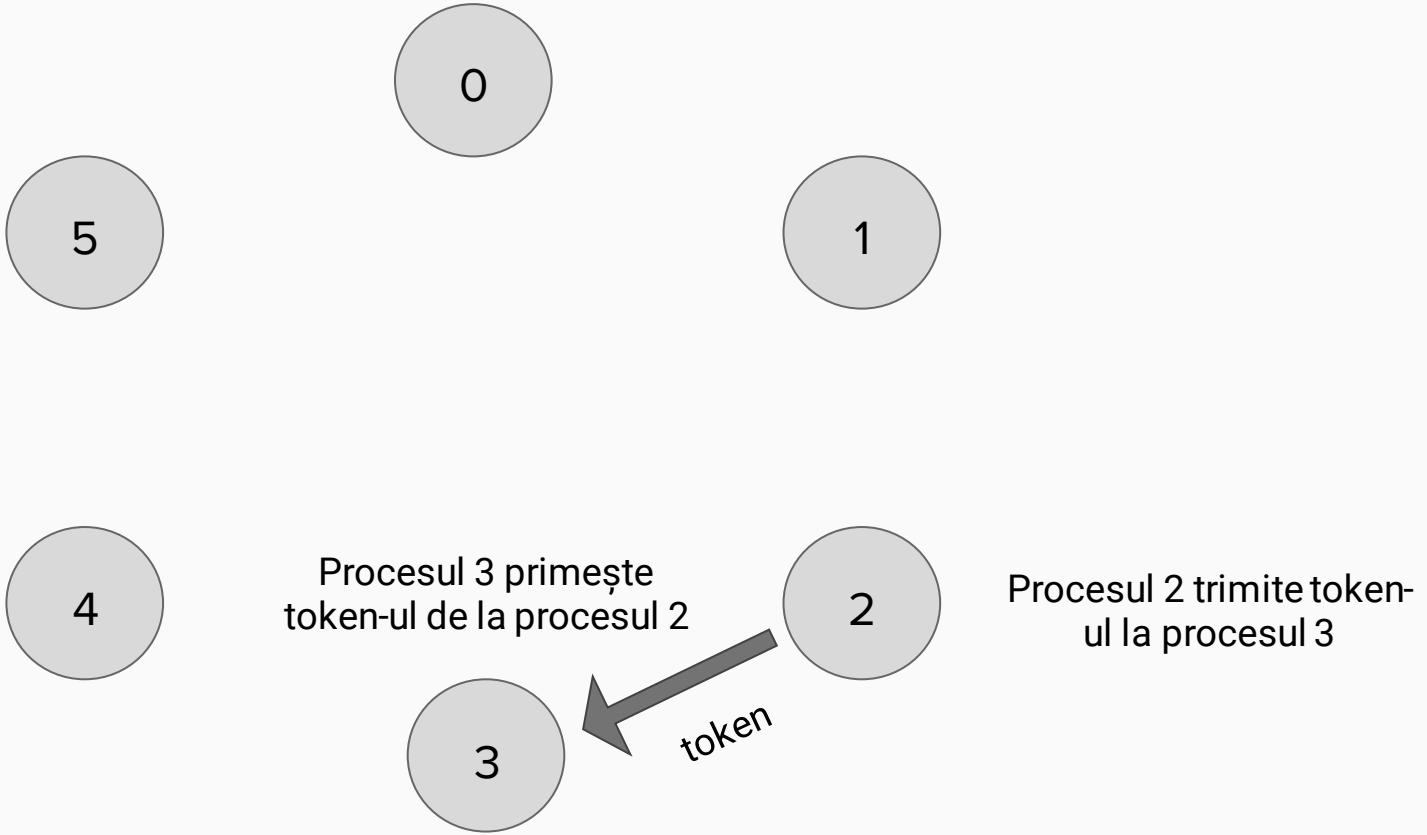
Algoritmul inel



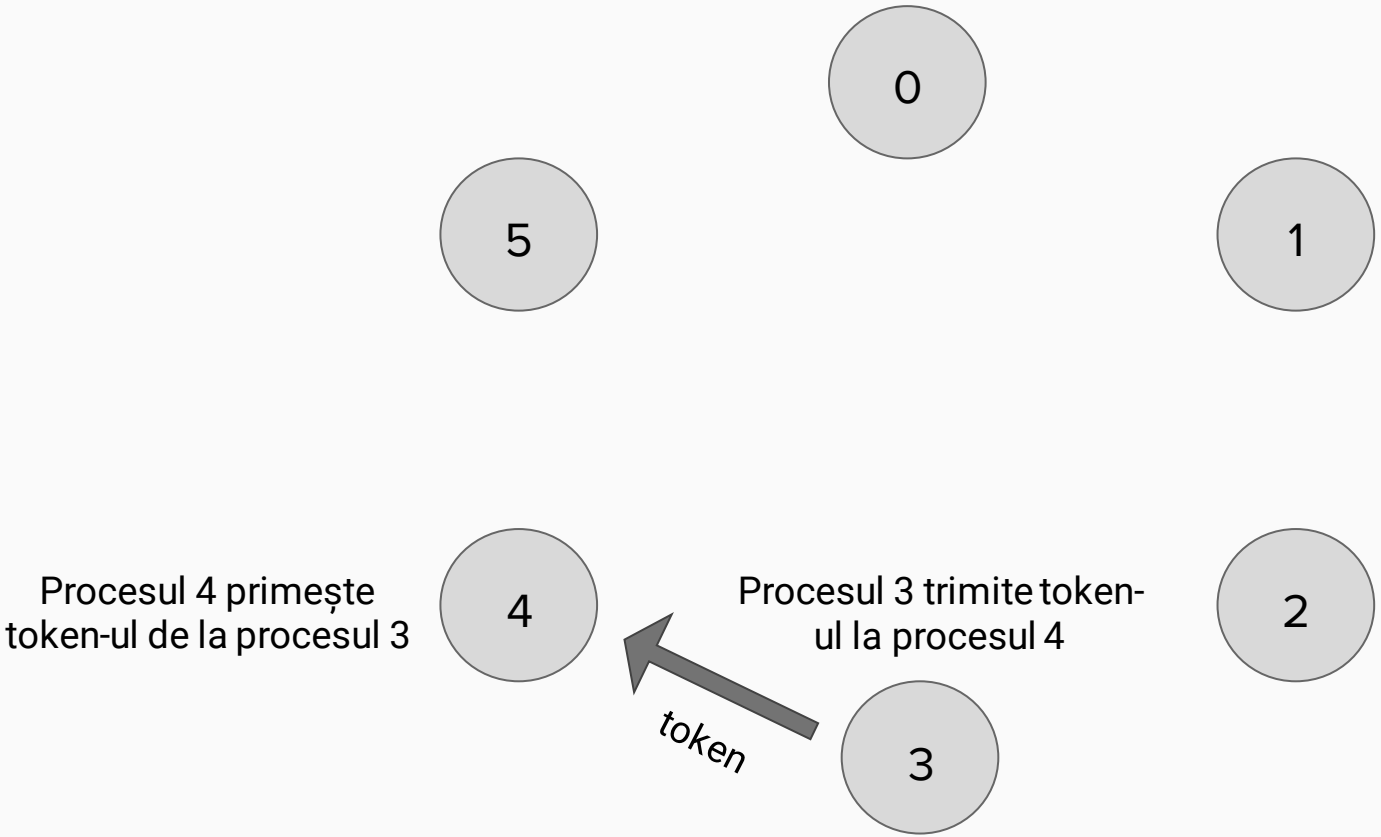
Algoritmul inel



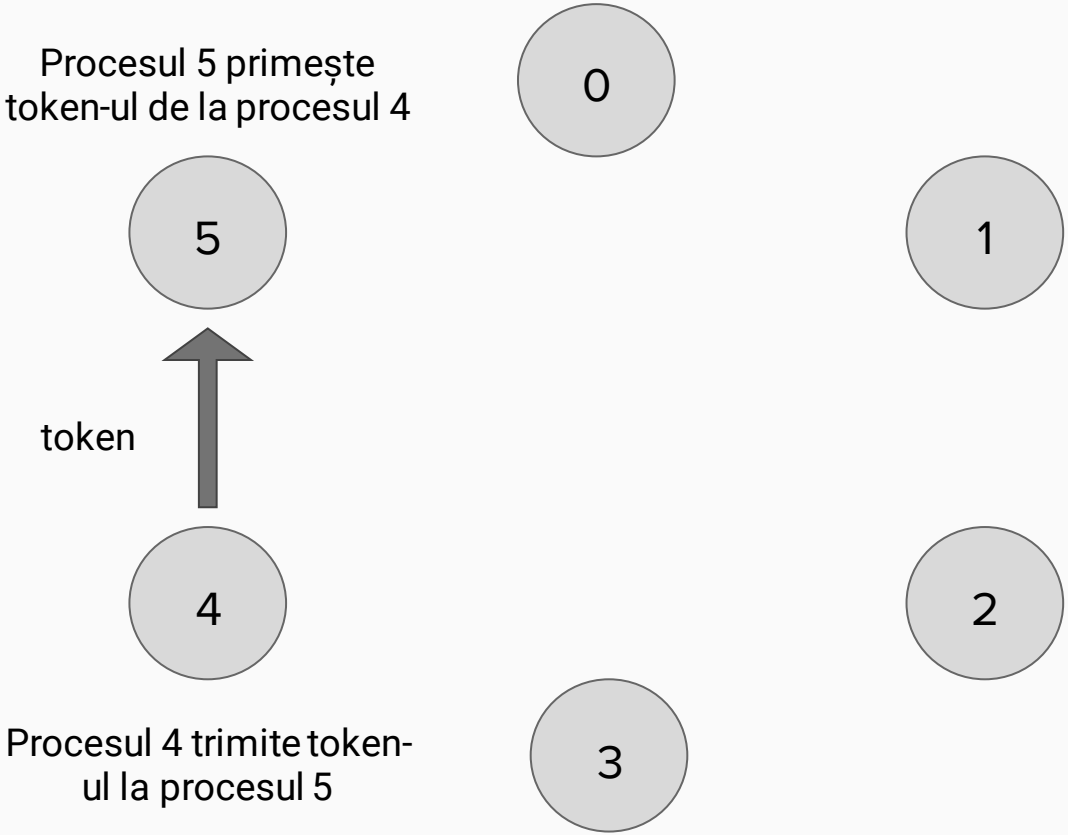
Algoritmul inel



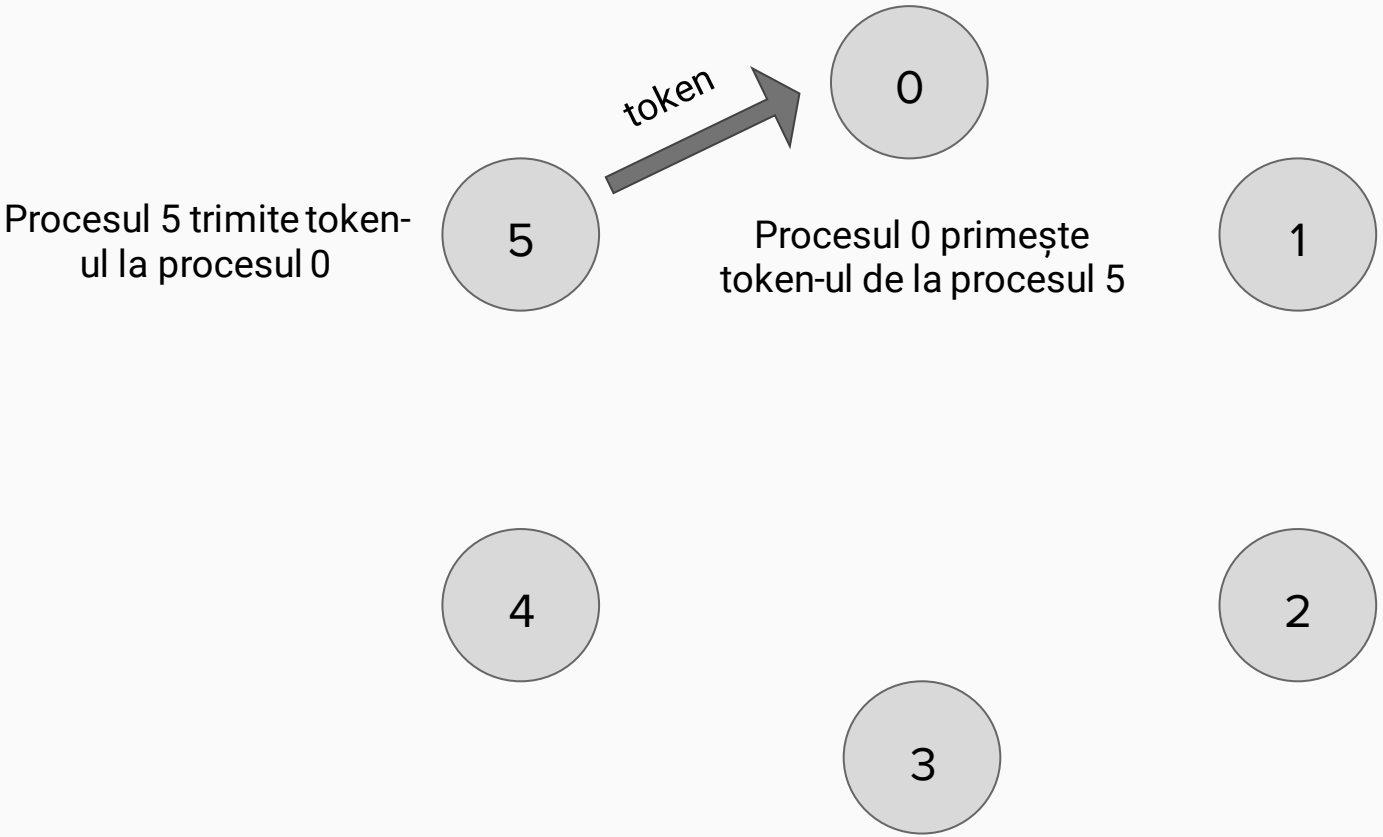
Algoritmul inel



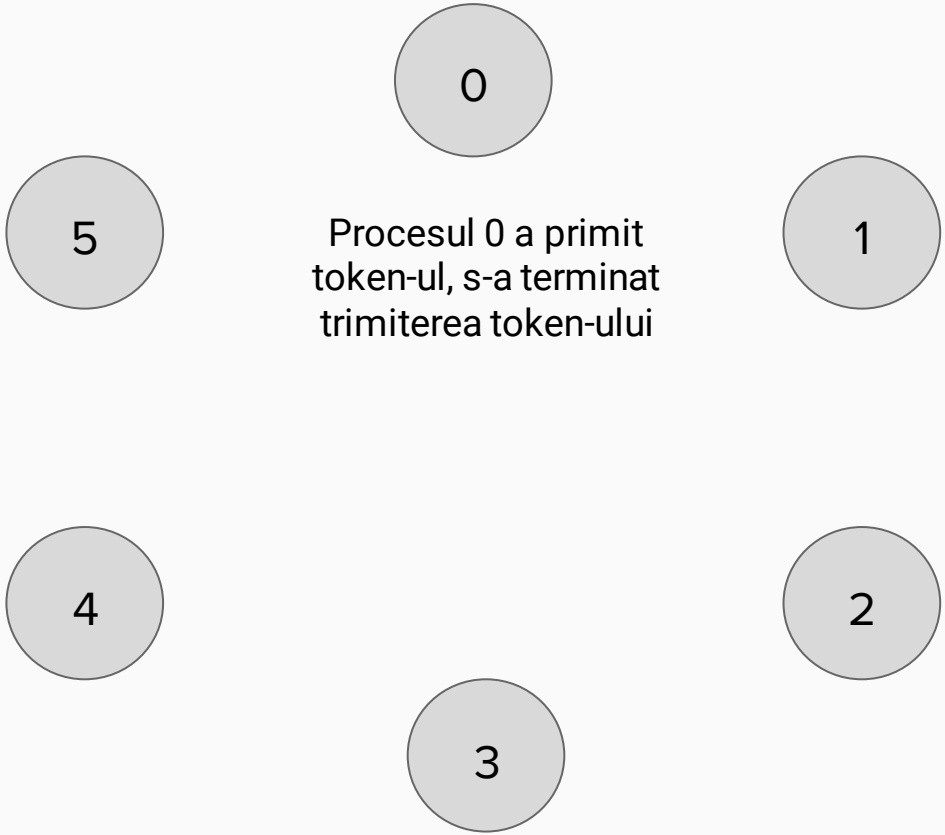
Algoritmul inel



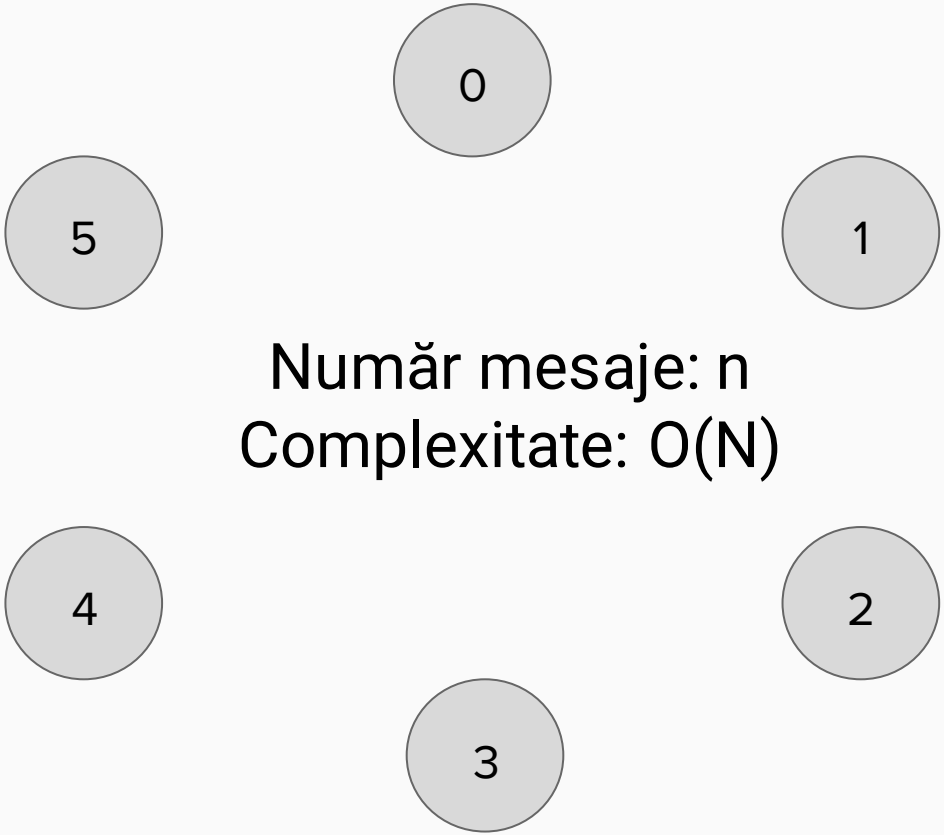
Algoritmul inel



Algoritmul inel



Algoritmul inel



Întrebări

Resurse

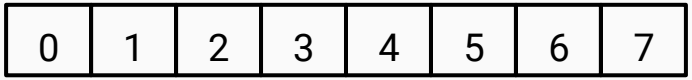
- https://mpitutorial.com/tutorials/
- https://computing.llnl.gov/tutorials/mpi/

Laboratorul 9
Operații distribuite în MPI

Reduce

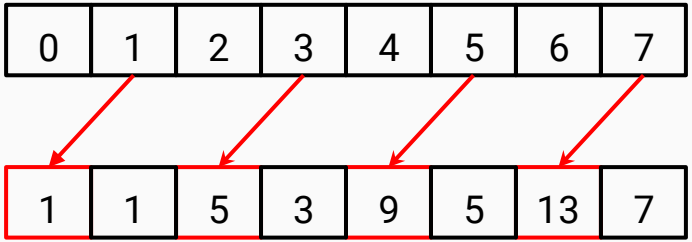
Operația de reduce (cunoscută în limbajele funcționale - Haskell, Racket - drept fold) reprezintă un tip de operație prin care elementele unei colecții sunt restrânse / acumulate într-un singur rezultat, printr-o singură operație aplicată între elementele unei colecții (+, *, min, max etc.)

Reduce



În acest exemplu, fiecare proces are câte un element (8 elemente - 8 procese)

Reduce

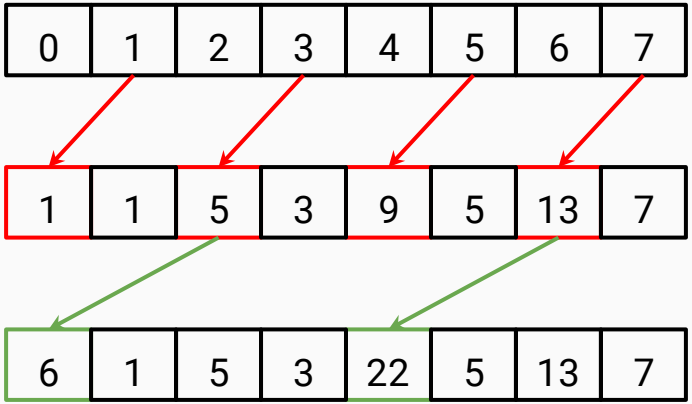


- procesul 1 trimite procesului 0, care adună valoarea primită (1 + 0 = 1)
- procesul 3 trimite procesului 2, care adună valoarea primită (3 + 2 = 5)
- procesul 5 trimite procesului 4, care adună valoarea primită (5 + 4 = 9)
- procesul 7 trimite procesului 6, care adună valoarea primită (7 + 6 = 13)

Deci:

- procesul 0 are valoarea 1
- procesul 2 are valoarea 5
- procesul 4 are valoarea 9
- procesul 6 are valoarea 13

Reduce

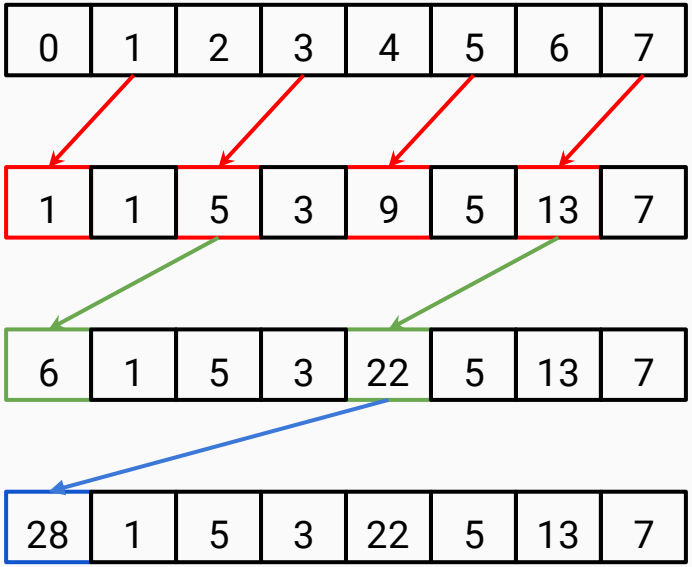


- procesul 2 trimite procesului 0, care adună valoarea primită (5 + 1 = 6)
- procesul 6 trimite procesului 4, care adună valoarea primită (13 + 9 = 22)

Deci:

- procesul 0 are valoarea 6
- procesul 4 are valoarea 22

Reduce



- procesul 4 trimite procesului 0, care adună valoarea primită (22 + 6 = 28)

Deci:

- procesul 0 are valoarea 28

Reduce - pseudocod

```
for (pas = 2; pas <= nr_procese; pas *= 2)
      if (rank % pas == 0 && (rank + (pas / 2)) < nr_procese)
            primește la procesul cu rank-ul [rank + (pas /
2)]
            adună
   else if (rank % (pas / 2) == 0 && (rank - (pas / 2)) >= 0)
      trimite la procesul cu rank-ul [rank - (pas / 2)]
```

Reduce - pseudocod - rulare

pas = 2:

- 1 (1 % (2 / 2) == 0) trimite lui 0 (1 - (2 / 2))
- 0 (0 % 2 == 0) primește de la 1 (0 + (2 / 2))

- 3 (3 % (2 / 2) == 0) trimite lui 2 (3 - (2 / 2))
- 2 (2 % 2 == 0) primește de la 3 (2 + (2 / 2))

- 5 (5 % (2 / 2) == 0) trimite lui 4 (5 - (2 / 2))
- 4 (4 % 2 == 0) primește de la 5 (4 + (2 / 2))

- 7 (7 % (2 / 2) == 0) trimite lui 6 (7 - (2 / 2))
- 6 (6 % 2 == 0) primește de la 7 (6 + (2 / 2))

Reduce - pseudocod - rulare

pas = 4:

- 2 (2 % (4 / 2) == 0) trimite lui 0 (2 - (4 / 2))
- 0 (0 % 4 == 0) primește de la 2 (0 + (4 / 2))

- 6 (6 % (4 / 2) == 0) trimite lui 4 (6 - (4 / 2))
- 4 (0 % 4 == 0) primește de la 6 (4 + (4 / 2))

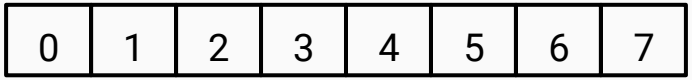
pas = 8:

- 4 (4 % (8 / 2) == 0) trimite lui 0 (4 - (8 / 2))
- 0 (0 % 8 == 0) primește de la 4 (0 + (8 / 2))

Scan

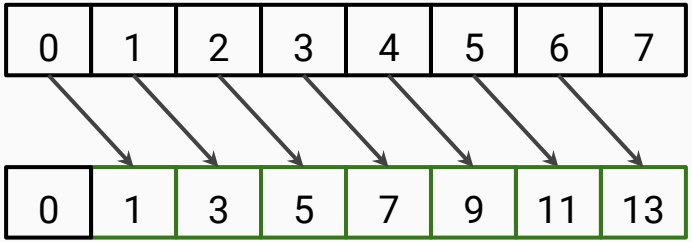
Operația de scan este similară operației de reduce (acumularea elementelor unei colecții într-un singur rezultat). Diferența față de reduce este că ultimul proces din comunicator deține rezultatul final, practic scan reprezintă inversul lui reduce.

Scan



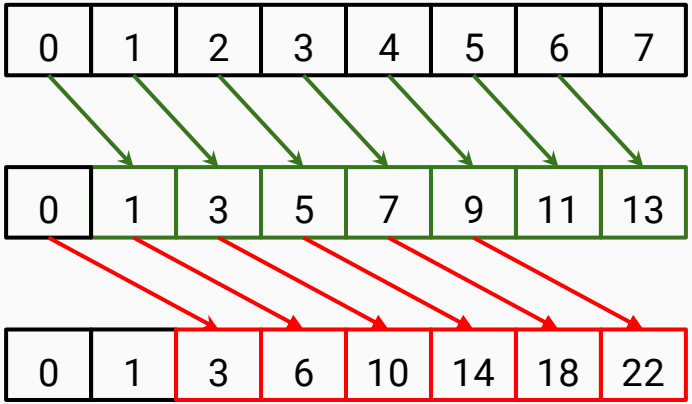
În acest exemplu, fiecare proces are câte un element (8 elemente - 8 procese)

Scan



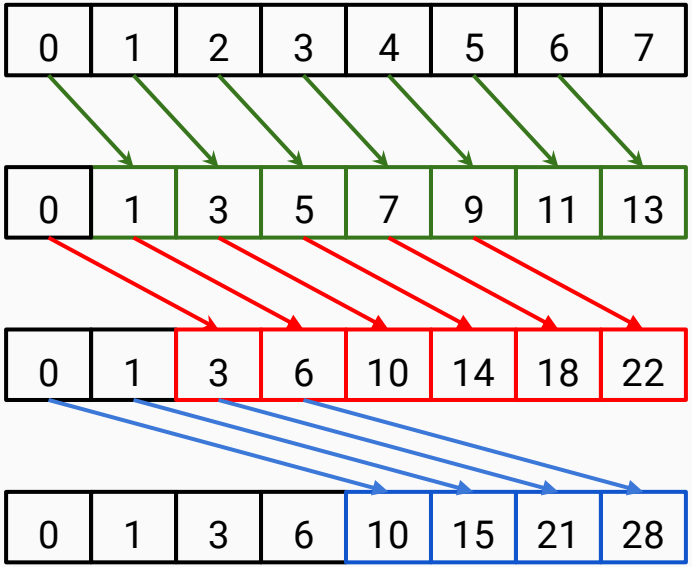
- procesul 0 trimite procesului 1, care adună valoarea primită (0 + 1 = 1)
- procesul 1 trimite procesului 2, care adună valoarea primită (1 + 2 = 3)
- procesul 2 trimite procesului 3, care adună valoarea primită (2 + 3 = 5)
- procesul 3 trimite procesului 3, care adună valoarea primită (3 + 4 = 7)
- procesul 4 trimite procesului 5, care adună valoarea primită (4 + 5 = 9)
- procesul 5 trimite procesului 6, care adună valoarea primită (5 + 6 = 11)
- procesul 6 trimite procesului 7, care adună valoarea primită (6 + 7 = 13)

Scan



- procesul 0 trimite procesului 2, care adună valoarea primită (0 + 3 = 3)
- procesul 1 trimite procesului 3, care adună valoarea primită (1 + 5 = 6)
- procesul 2 trimite procesului 4, care adună valoarea primită (3 + 7 = 10)
- procesul 3 trimite procesului 5, care adună valoarea primită (5 + 9 = 14)
- procesul 4 trimite procesului 6, care adună valoarea primită (7 + 11 = 18)
- procesul 5 trimite procesului 7, care adună valoarea primită (9 + 13 = 22)

Scan



- procesul 0 trimite procesului 4, care adună valoarea primită (0 + 10 = 10)
- procesul 1 trimite procesului 5, care adună valoarea primită (1 + 14 = 15)
- procesul 2 trimite procesului 6, care adună valoarea primită (3 + 18 = 21)
- procesul 3 trimite procesului 7, care adună valoarea primită (6 + 22 = 28)

Scan - pseudocod

```
for (pas = 1; pas < nr_procese; pas *= 2)
       if (rank + pas < nr_procese)
              trimite la procesul cu rank-ul [rank + pas]
    if (rank - pas >= 0)
       primește de la procesul cu rank-ul [rank - pas]
       adună
```

Scan - pseudocod - rulare

pas = 1:

- 0 trimite lui 1 (0 + 1)
- 1 primește de la 0 (1 - 1)

- 1 trimite lui 2 (1 + 1)
- 2 primește de la 1 (2 - 1)

- 2 trimite lui 3 (2 + 1)
- 3 primește de la 2 (3 - 1)

- 3 trimite lui 4 (3 + 1)
- 4 primește de la 3 (4 - 1)

pas = 1:

- 4 trimite lui 5 (4 + 1)
- 5 primește de la 4 (5 - 1)

- 5 trimite lui 6 (5 + 1)
- 6 primește de la 5 (6 - 1)

- 6 trimite lui 7 (6 + 1)
- 7 primește de la 6 (7 - 1)

Scan - pseudocod - rulare

pas = 2:

- 0 trimite lui 2 (0 + 2)
- 2 primește de la 0 (2 - 2)

- 1 trimite lui 3 (1 + 2)
- 3 primește de la 1 (3 - 2)

- 2 trimite lui 4 (2 + 2)
- 4 primește de la 2 (4 - 2)

- 3 trimite lui 5 (3 + 2)
- 5 primește de la 3 (5 - 2)

- 4 trimite lui 6 (4 + 2)
- 6 primește de la 4 (6 - 2)

- 5 trimite lui 7 (5 + 2)
- 7 primește de la 5 (7 - 2)

pas = 4:

- 0 trimite lui 4 (4 + 4)
- 4 primește de la 0 (4 - 4)

- 1 trimite lui 5 (1 + 4)
- 5 primește de la 1 (5 - 1)

- 2 trimite lui 6 (2 + 4)
- 6 primește de la 2 (6 - 4)

- 3 trimite lui 7 (3 + 4)
- 7 primește de la 3 (7 - 4)

Broadcast

Operația de broadcast este o operație prin care un proces trimite o valoare tuturor proceselor din cadrul comunicatorului. Această operație este reprezentată, în MPI, de MPI_Bcast (remember laboratorul 8).

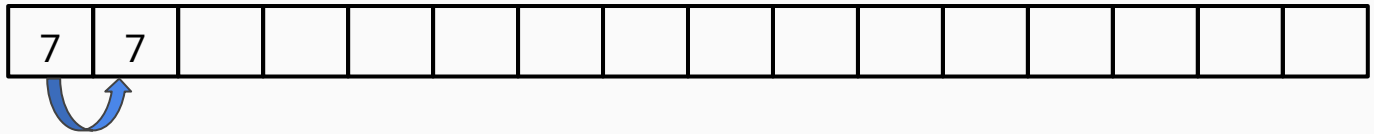
Broadcast

În acest exemplu, rulăm cu 16 procese (fiecare proces, mai puțin procesul 0, nu are nicio valoare)

```
for (pas = 1; pas < nr_procese; pas *= 2)
        if (rank < pas and rank + pas < nr_procese)
                trimite la procesul cu rank-ul [rank +
pas]
    else if (rank >= pas and rank < pas * 2)
        primește de la procesul cu rank-ul [rank - pas]
```

Broadcast

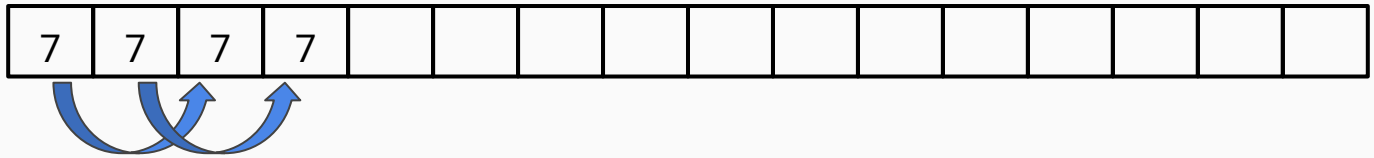
pas = 1



- procesul 0 (0 < 1, 0 + 1 < 16) trimite procesului 1 (0 + 1, 1 >= 1, 1 < 2 * 1)

Broadcast

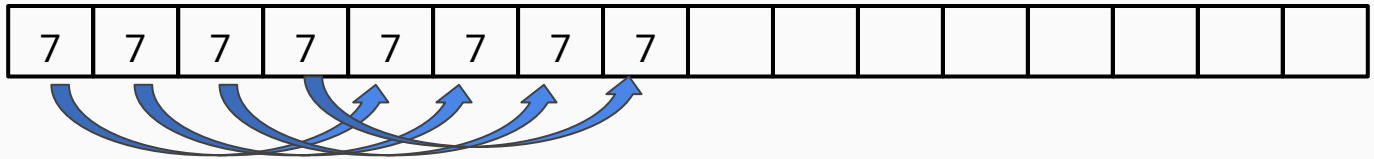
pas = 2



- procesul 0 (0 < 2, 0 + 2 < 16) trimite procesului 2 (0 + 2, 2 >= 2, 2 < 2 * 2)
- procesul 1 (1 < 2, 1 + 2 < 16) trimite procesului 3 (1 + 2, 3 >= 2, 3 < 2 * 2)

Broadcast

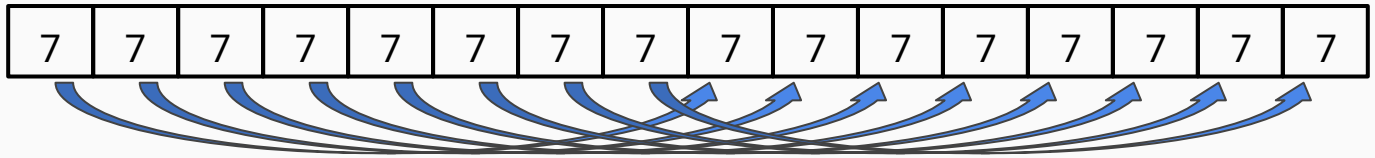
pas = 4



- procesul 0 (0 < 4, 0 + 4 < 16) trimite procesului 4 (0 + 4, 4 >= 4, 4 < 2 * 4)
- procesul 1 (1 < 4, 1 + 4 < 16) trimite procesului 5 (1 + 4, 5 >= 4, 5 < 2 * 4)
- procesul 2 (2 < 4, 2 + 4 < 16) trimite procesului 6 (2 + 4, 6 >= 4, 6 < 2 * 4)
- procesul 3 (3 < 4, 3 + 4 < 16) trimite procesului 7 (3 + 4, 7 >= 4, 7 < 2 * 4)

Broadcast

pas = 8



- procesul 0 (0 < 8, 0 + 8 < 16) trimite procesului 8 (0 + 8, 8 >= 8, 8 < 2 * 8)
- procesul 1 (1 < 8, 1 + 8 < 16) trimite procesului 9 (1 + 8, 9 >= 8, 9 < 2 * 8)
- procesul 2 (2 < 8, 2 + 8 < 16) trimite procesului 10 (2 + 8, 10 >= 8, 10 < 2 * 8)
- procesul 3 (3 < 8, 3 + 8 < 16) trimite procesului 11 (3 + 8, 11 >= 8, 11 < 2 * 8)
- procesul 4 (4 < 8, 4 + 8 < 16) trimite procesului 12 (4 + 8, 12 >= 8, 12 < 2 * 8)
- procesul 5 (5 < 8, 5 + 8 < 16) trimite procesului 13 (5 + 8, 13 >= 8, 13 < 2 * 8)
- procesul 6 (6 < 8, 6 + 8 < 16) trimite procesului 14 (6 + 8, 14 >= 8, 14 < 2 * 8)
- procesul 7 (7 < 8, 7 + 8 < 16) trimite procesului 15 (7 + 8, 15 >= 8, 15 < 2 * 8)

Pipeline

- În domeniul de computer science, un pipeline reprezintă un concept folosit în proiectarea procesoarelor și al echipamentelor hardware prin care se crește numărul de instrucțiuni ce sunt executate într-o unitate de timp.
- Acest concept poate fi extins în MPI, unde un proces produce rezultate intermerdiare, care sunt transmise către procesul următor, care procesează și prelucrează rezultatele primite, pe care le trimite mai departe, către procesul următor, totul repetându-se până la ultimul proces.

Pipeline

În acest exemplu, avem 5 pași, deci 5 procese, unde fiecare prelucrează date și le trimite mai departe, către următorul proces

Pipeline

Procesul 0 creează rezultate, pe care le trimite procesului 1

Pipeline

Procesul 1 primește date de la procesul 0, le procesează, aplică operații asupra acestora și trimite rezultatele către procesul 2

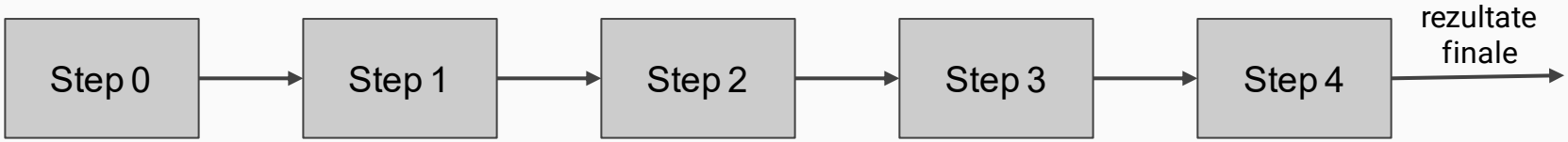
Pipeline

Procesul 2 primește date de la procesul 1, le procesează, aplică operații asupra acestora și trimite rezultatele către procesul 3

Pipeline

Procesul 3 primește date de la procesul 2, le procesează, aplică operații asupra acestora și trimite rezultatele către procesul 4

Pipeline



Procesul 4 primește date de la procesul 3, le procesează, aplică operații asupra acestora și se obțin rezultatele finale din cadrul procesului de pipeline

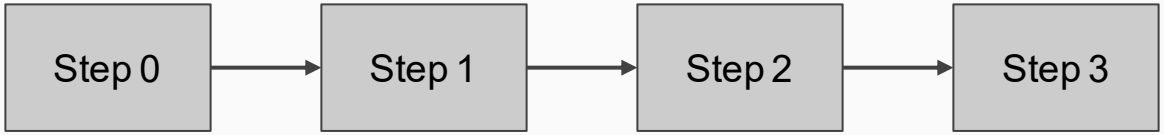
Calculul unui polinom cu pipeline

- Un polinom, în matematică, reprezintă o ecuație de tipul: f(x) = x^2+ 2 * x + 1, unde x reprezintă variabila din ecuație.
- În MPI, putem calcula un polinom folosind tehnica pipeline în felul următor: numărul de coeficienți va fi egal cu numărul de procese (care este egal cu numărul de pași din procesul de pipeline). În cazul exemplului de mai sus, o să avem 3 procese, fiecare proces corespunzând cu câte un coeficient din ecuație

Calcularea unui polinom

- Să luăm un exemplu de polinom: f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1
- Vrem să calculăm f(3) folosind polinom cu pipeline.
- În acest caz, avem un polinom cu 4 coeficienți (de grad 3), astfel vom lucra cu 4 procese (pipeline cu 4 pași de execuție)

Calcularea unui polinom



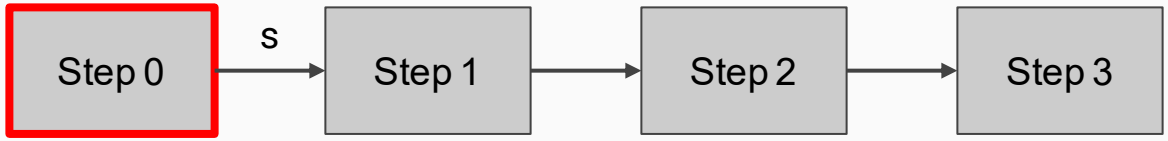
f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1

Calculăm f(3)

s = 0 (s va reprezenta în final rezultatul pipeline-ului, adică f(3)

Calcularea unui polinom

f(3), f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1



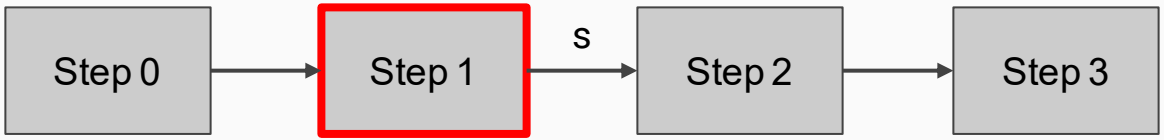
Procesul 0 se ocupă de x^0

```
s += 1 * (3 ^ 0) => s = 1
```

Procesul 0 trimite s către procesul 1

Calcularea unui polinom

$$f(3),\ f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1$$



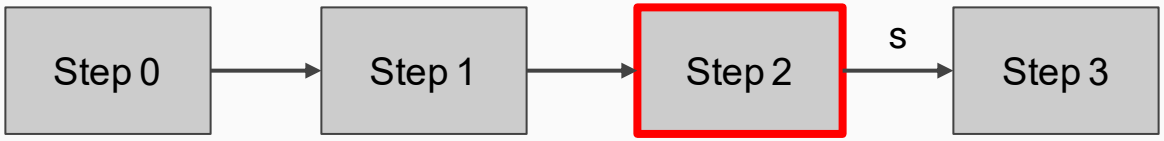
Procesul 1 primește rezultate
de la procesul 0
Procesul 1 se ocupă de x^1

```
s += 2 * (3 ^ 1) => s = 7
```

Procesul 1 trimite s către
procesul 2

Calcularea unui polinom

f(3), f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1



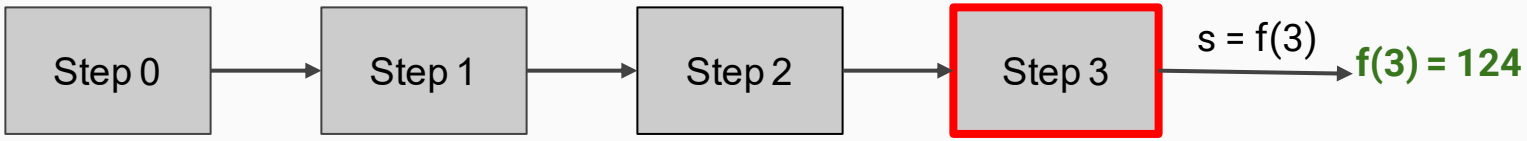
Procesul 2 primește rezultate
de la procesul 1
Procesul 2 se ocupă de x^2

```
s += 4 * (3 ^ 2) => s =
43
```

Procesul 2 trimite s către
procesul 3

Calcularea unui polinom

f(3), f(x) = 3 * x^3 + 4 * x^2 + 2 * x + 1



Procesul 3 primește rezultate de la procesul 2
Procesul 3 se ocupă de x^3

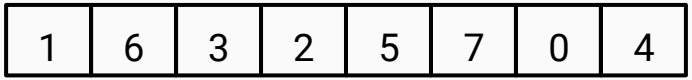
```
s += 3 * (3 ^ 3) => s = 124
```

Procesul 3 este ultimul și întoarce rezultatul final

Sortarea folosind pipeline

- Dându-se un vector cu n elemente în ordine aleatoare, dorim să-l sortăm prin metoda pipeline, folosind MPI. Pentru aceasta, vom folosi o rețea cu n + 1 noduri (1 master și n workeri).
- Punctul final este momentul în care fiecare proces worker deține stocat unul din numerele din vectorul inițial, iar rank[i]→saved_value < rank[i + 1]→saved_value pentru orice i din [1, n).
- Fiecare worker este inițializat la început cu o valoare (-1 sau depinde de restricții), va primi (n - rank) numere de la rank-ul anterior și va trimite mai departe rank-ului următor (n - rank - 1) numere.
- Atunci când primește un număr, worker-ul îl va compara cu valoarea stocată:
  - dacă recv_value < actual_value, atunci salvăm noua valoare și o trimitem pe cea veche procesului următor
  - dacă recv_value >= actual_value, atunci o trimitem direct procesului următor

Sortarea cu pipeline

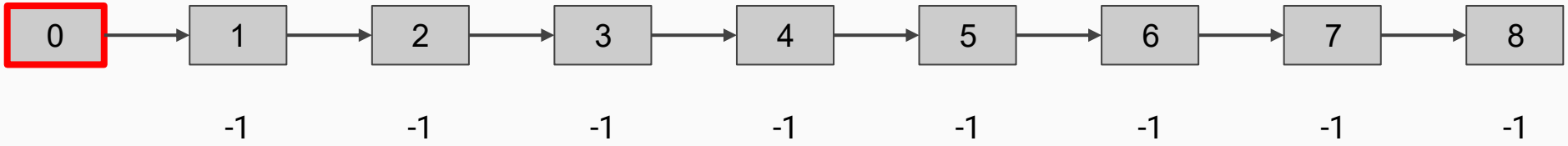


Avem un array nesortat de 8 elemente, așadar vom avea 9 procese:

- un proces master, care începe procesul de pipeline
- 8 procese worker, care se ocupă efectiv de sortarea array-ului

Sortarea cu pipeline

Sortarea cu pipeline



Procesul 0 (master):

- asignează o valoare inițială fiecărui proces worker (să spunem -1) - trimite acea valoare proceselor worker
- trimite întreg array-ul către procesul 1 (dim_array - rank, în acest caz dim_array - 0, rank = 0)

Sortarea cu pipeline

actual_value = -1
primește de la procesul 0: [1, 6, 3, 2, 5, 7, 0, 4]

```
if (actual_value == -1):
        actual_value = recv_value
else if (actual_value <= recv_value)
        trimite recv_value către următorul proces
else
        trimite actual_value către următorul proces
        actual_value = recv_value
```

Sortarea cu pipeline

actual_value = -1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 1
actual_value == -1 => actual_value = recv_value = 1

actual_value = 1

Sortarea cu pipeline

actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 6
actual_value <= recv_value => trimite recv_value (6) către procesul 2

actual_value = 1

Sortarea cu pipeline

actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 3
actual_value <= recv_value => trimite recv_value (3) către procesul 2

actual_value = 1

Sortarea cu pipeline

actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 2
actual_value <= recv_value => trimite recv_value (2) către procesul 2

actual_value = 1

Sortarea cu pipeline

actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 5
actual_value <= recv_value => trimite recv_value (5) către procesul 2

actual_value = 1

Sortarea cu pipeline

actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 7
actual_value <= recv_value => trimite recv_value (7) către procesul 2

actual_value = 1

Sortarea cu pipeline

actual_value = 1
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 0
actual_value > recv_value => trimite actual_value (1) către procesul 2, actual_value = recv_value = 0

actual_value = 0

Sortarea cu pipeline

actual_value = 0
[1, 6, 3, 2, 5, 7, 0, 4]

recv_value = 4
actual_value <= recv_value => trimite recv_value (4) către procesul 2

actual_value = 0

Stop: procesul 1 va avea valoarea 0, se trece la procesul 2, iar procesul 1 trimite valoarea procesului master

Sortarea cu pipeline

actual_value = -1
primește de la procesul 1: [6, 3, 2, 5, 7, 1, 4]

Sortarea cu pipeline

actual_value = -1
[6, 3, 2, 5, 7, 1, 4]

recv_value = 6
actual_value == -1 => actual_value = recv_value = 6

actual_value = 6

Sortarea cu pipeline

actual_value = 6
[6, 3, 2, 5, 7, 1, 4]

recv_value = 3
actual_value > recv_value => trimite actual_value (6) către procesul 3, actual_value = recv_value = 3

actual_value = 3

Sortarea cu pipeline

actual_value = 3
[6, 3, 2, 5, 7, 1, 4]

recv_value = 2
actual_value > recv_value => trimite actual_value (3) către procesul 3, actual_value = recv_value = 2

actual_value = 2

Sortarea cu pipeline

actual_value = 3
[6, 3, 2, 5, 7, 1, 4]

recv_value = 5
actual_value <= recv_value => trimite recv_value (5) către procesul 3

actual_value = 3

Sortarea cu pipeline

actual_value = 3
[6, 3, 2, 5, 7, 1, 4]

recv_value = 7
actual_value <= recv_value => trimite recv_value (7) către procesul 3

actual_value = 3

Sortarea cu pipeline

actual_value = 2
[6, 3, 2, 5, 7, 1, 4]

recv_value = 1
actual_value > recv_value => trimite actual_value (2) către procesul 3, actual_value = recv_value = 1

actual_value = 1

Sortarea cu pipeline

actual_value = 1
[6, 3, 2, 5, 7, 1, 4]

recv_value = 4
actual_value <= recv_value => trimite recv_value (4) către procesul 3

actual_value = 1

Stop: procesul 2 va avea valoarea 1, se trece la procesul 3, iar procesul 2 trimite valoarea procesului master

Sortarea cu pipeline

actual_value = -1
primește de la procesul 2: [6, 3, 5, 7, 2, 4]

Sortarea cu pipeline

actual_value = -1
[6, 3, 5, 7, 2, 4]

recv_value = 6
actual_value == -1 => actual_value = recv_value = 6

actual_value = 6

Sortarea cu pipeline

actual_value = 6
[6, 3, 5, 7, 2, 4]

recv_value = 3
actual_value > recv_value => trimite actual_value (6) către procesul 4, actual_value = recv_value = 3

actual_value = 3

Sortarea cu pipeline

actual_value = 3
[6, 3, 5, 7, 2, 4]

recv_value = 5
actual_value <= recv_value => trimite recv_value (5) către procesul 4

actual_value = 3

Sortarea cu pipeline

actual_value = 3
[6, 3, 5, 7, 2, 4]

recv_value = 7
actual_value <= recv_value => trimite recv_value (7) către procesul 4

actual_value = 3

Sortarea cu pipeline

actual_value = 3
[6, 3, 5, 7, 2, 4]

recv_value = 2
actual_value > recv_value => trimite actual_value (3) către procesul 4, actual_value = recv_value = 2

actual_value = 2

Sortarea cu pipeline

actual_value = 2
[6, 3, 5, 7, 2, 4]

recv_value = 4
actual_value <= recv_value => trimite recv_value (4) către procesul 4

actual_value = 2

Stop: procesul 3 va avea valoarea 2, se trece la procesul 4, iar procesul 3 trimite valoarea procesului master

Sortarea cu pipeline

actual_value = -1
primește de la procesul 3: [6, 5, 7, 3, 4]

Sortarea cu pipeline

actual_value = -1
[6, 5, 7, 3, 4]

recv_value = 6
actual_value == -1 => actual_value = recv_value = 6

actual_value = 6

Sortarea cu pipeline

actual_value = 6
[6, 5, 7, 3, 4]

recv_value = 5
actual_value > recv_value => trimite actual_value (6) către procesul 5, actual_value = recv_value = 5

actual_value = 5

Sortarea cu pipeline

actual_value = 5
[6, 5, 7, 3, 4]

recv_value = 7
actual_value <= recv_value => trimite recv_value (7) către procesul 5

actual_value = 5

Sortarea cu pipeline

actual_value = 5
[6, 5, 7, 3, 4]

recv_value = 3
actual_value > recv_value => trimite actual_value (5) către procesul 5, actual_value = recv_value = 3

actual_value = 3

Sortarea cu pipeline

actual_value = 3
[6, 5, 7, 3, 4]

recv_value = 4
actual_value <= recv_value => trimite recv_value (4) către procesul 5

actual_value = 3

Stop: procesul 4 va avea valoarea 3, se trece la procesul 5, iar procesul 4 trimite valoarea procesului master

Sortarea cu pipeline

actual_value = -1
primește de la procesul 4: [6, 7, 5, 4]

Sortarea cu pipeline

actual_value = -1
[6, 7, 5, 4]

recv_value = 6
actual_value == -1 => actual_value = recv_value = 6

actual_value = 6

Sortarea cu pipeline

actual_value = 6
[6, 7, 5, 4]

recv_value = 7
actual_value <= recv_value => trimite recv_value (7) către procesul 6

actual_value = 6

Sortarea cu pipeline

actual_value = 6
[6, 7, 5, 4]

recv_value = 5
actual_value > recv_value => trimite actual_value (6) către procesul 6, actual_value = recv_value = 5

actual_value = 5

Sortarea cu pipeline

actual_value = 5
[6, 7, 5, 4]

recv_value = 4
actual_value > recv_value => trimite actual_value (5) către procesul 6, actual_value = recv_value = 4

actual_value = 4

Stop: procesul 5 va avea valoarea 4, se trece la procesul 6, iar procesul 5 trimite valoarea procesului master

Sortarea cu pipeline

actual_value = -1
primește de la procesul 5: [7, 6, 5]

Sortarea cu pipeline

actual_value = -1
[7, 6, 5]

recv_value = 7
actual_value == -1 => actual_value = recv_value = 7

actual_value = 7

Sortarea cu pipeline

actual_value = 7
[7, 6, 5]

recv_value = 6
actual_value > recv_value => trimite actual_value (7) către procesul 7, actual_value = recv_value = 6

actual_value = 6

Sortarea cu pipeline

actual_value = 6
[7, 6, 5]

recv_value = 5
actual_value > recv_value => trimite actual_value (6) către procesul 7, actual_value = recv_value = 5

actual_value = 5

Stop: procesul 6 va avea valoarea 5, se trece la procesul 7, iar procesul 6 trimite valoarea procesului master

Sortarea cu pipeline

actual_value = -1
primește de la procesul 6: [7, 6]

Sortarea cu pipeline

actual_value = -1
[7, 6]

recv_value = 7
actual_value == -1 => actual_value = recv_value = 7

actual_value = 7

Sortarea cu pipeline

actual_value = 6
[7, 6]

recv_value = 6
actual_value > recv_value => trimite actual_value (7) către procesul 8, actual_value = recv_value = 6

actual_value = 6

Stop: procesul 7 va avea valoarea 6, se trece la procesul 8, iar procesul 7 trimite valoarea procesului master

Sortarea cu pipeline

actual_value = -1
primește de la procesul 7: [7]

Sortarea cu pipeline

actual_value = -1
[7]

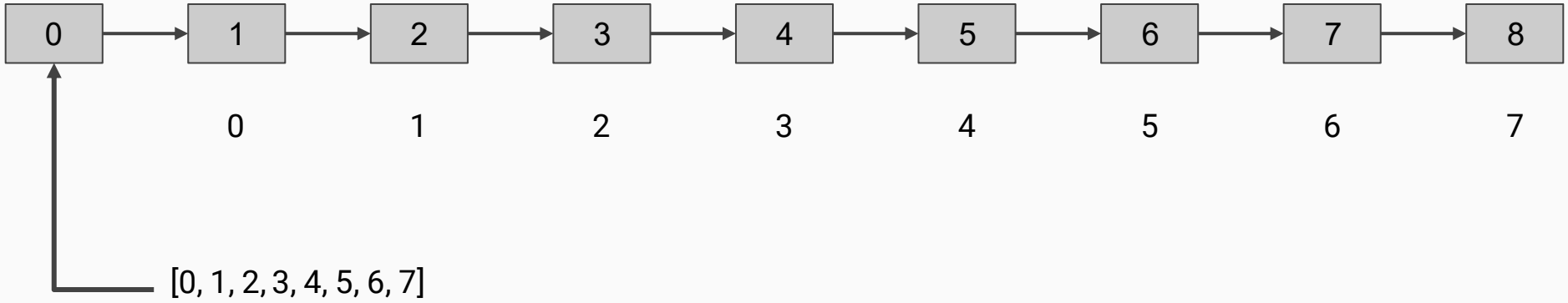
recv_value = 7
actual_value == -1 => actual_value = recv_value = 7

actual_value = 7

Stop: procesul 8 va avea valoarea 7, fiind ultimul proces pipeline-ul se oprește, iar procesul 7 trimite valoarea procesului master

Procesul master are acum toate valorile primite de la procesele worker, în ordine sortată

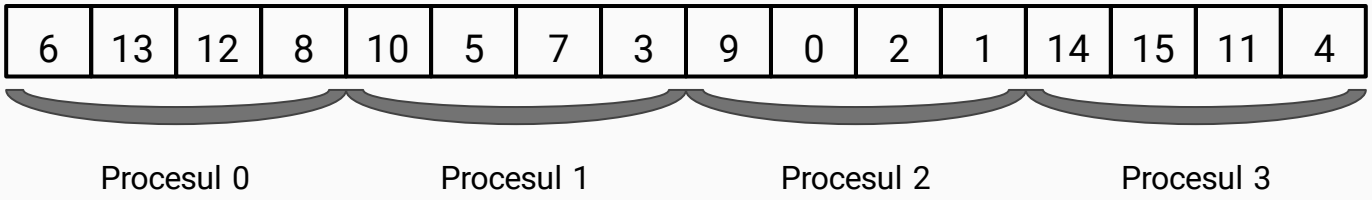
Sortarea cu pipeline



Rank Sort

- Rank Sort este un algoritm de sortare a unui vector folosindu-se de rangul fiecărui element din vector. Rangul unui element presupune câte numere sunt mai mici decât el în vector.
- Pași:
  - Se calculează rangul fiecărui element: pentru fiecare element din vector v[i], unde 0 <= i < N (N - dimensiunea vectorului), se numără câte elemente din vector v[j], j != i sunt mai mici: v[j] < v[i], 0 <= i, j < N, i != j.
  - Elementul de pe poziția “i” va fi pe poziția “rang” în noul vector sortat: q[rank] = v[i].
- Algoritmul “Parallel rank sort” are ca scop paralelizarea pasului 1. Rangul fiecărui element poate fi căutat în paralel, adică fiecare proces sau fir de execuție va realiza căutarea rangurilor pentru elementele din intervalul său: [start, end)

Rank Sort



- În acest exemplu, avem 4 procese.
- Trebuie să vedem, pentru fiecare element din array, câte elemente sunt mai mici decat elementul respectiv.
- Fiecare proces va număra câte elemente sunt mai mici decât un anumit element în bucata sa din array (în acest caz de dimensiune 4 = dim_array / nr_procese).

Rank Sort

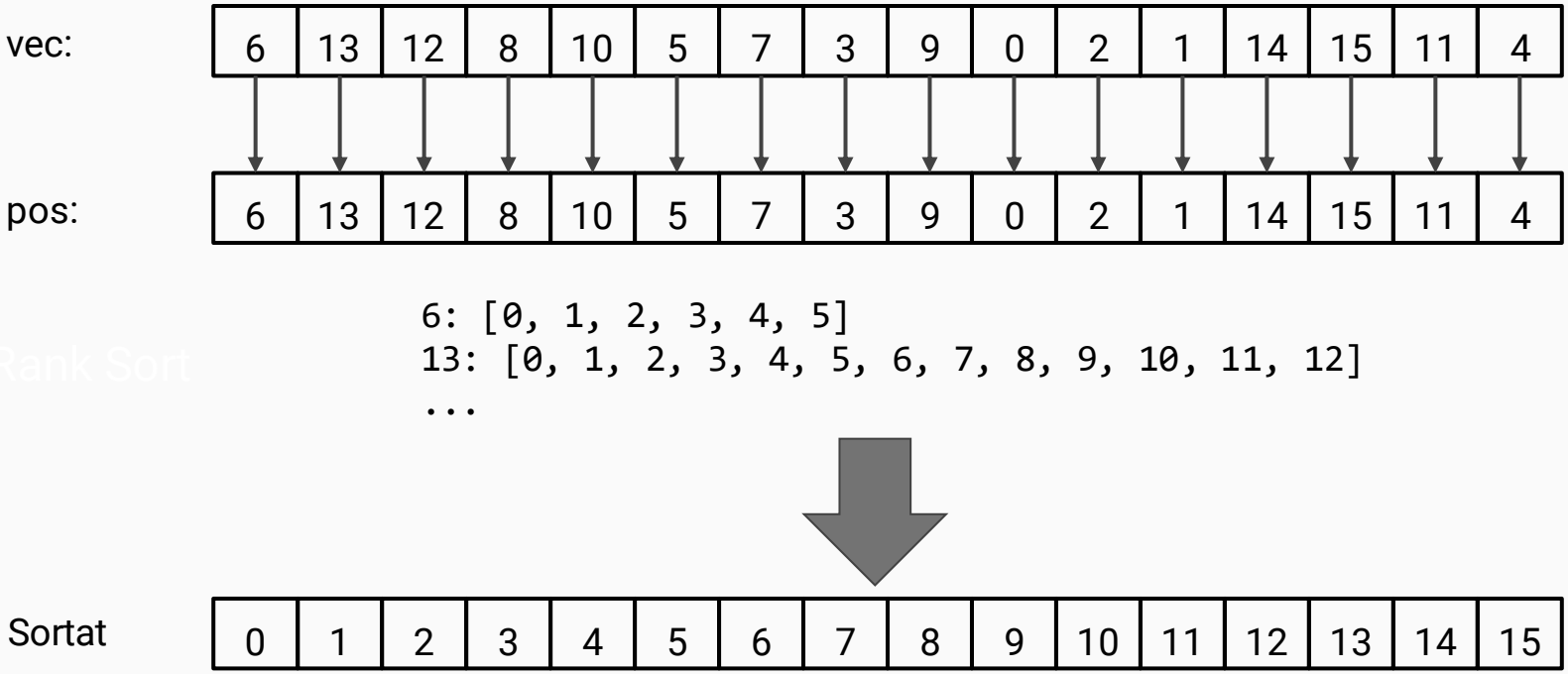
6	13	12	8	10	5	7	3	9	0	2	1	14	15	11	4

- Fiecare proces va avea întreg array-ul și va face numărarea elementelor mai mici în bucata sa de dimensiune dim_array / nr_procese, după cum este în pseudocodul de mai jos:

```
dim_proc = dim_array / nr_procese
for i = 0 to dim_proc
     for j = 0 to dim_array
               if v[rank * dim_proc + i] > v[j]
                         pos[rank * dim_proc + i]++
```

- Procesele 1, 2, 3 etc. vor trimite bucata lor din array-ul pos către procesul 0

Rank Sort



Rank Sort

Laboratorul 10
Stabilirea topologiei și alegerea unui lider

Ce este un algoritm undă?

Un algoritm undă este un tip de algoritm distribuit, folosit pentru propagarea informațiilor în cadrul unei rețele distribuite de procese.

Un algoritm undă este caracterizat de următoarele trăsături:

- există un proces inițiator, care declanșează trimiterea de mesaje în cadrul rețelei
- fiecare proces trimite informații către vecinii sai, cu excepția părintelui, de la care a primit informația pe care o pasează mai departe
- fiecare proces primește informații de la vecini (cu excepția părintelui) și o trimite părintelui
- la final se ia o decizie în legătură cu informațiile primite

Tipuri de arhitecturi de rețele distribuite

Avem 4 tipuri de arhitecturi de rețele de noduri (procese) distribuite din cadrul unui cluster:

- grid - fiecare nod poate comunica pe orizontală și pe verticală
- inel - forma de cerc, un nod comunica cu vecinul din stânga și din dreapta
- arbore - informația se propagă de la rădăcina către nodurile-copil
- graf - un nod poate comunica cu cel puțin un vecin de-al său

Stabilirea topologiei

La început, fiecare nod își cunoaște topologia locală (știe doar cine sunt vecinii sai), nu știe despre topologia globală a rețelei de noduri.

Putem sa stabilim topologia globală a rețelei de noduri în două feluri:

- nodurile transmit vecinilor topologia locală de mai multe ori până când toate nodurile au topologia stabilită - numărul de “valuri” de trimitere reprezintă convergența - simplu de realizat, dar ineficient
- se alege un lider (de regulă nodul cu rangul maxim), care calculează la final topologia rețelei

Stabilirea topologiei

- Nodurile își aleg un lider, însă acestea nu știu calea către lider, ci doar cine este acela.
- Pentru a rezolva aceasta problema, se construiește un arbore de acoperire, prin care se afla căile dintre noduri în așa fel încât orice nod poate comunica cu nodul lider, prin intermediul unor noduri intermediare care pasează informația de la nodul sursă la nodul lider: un nod trimite către părintele său, părintele către părintele său, totul se repetă până când se ajunge la nodul rădăcină din arbore, care e lider

Alegerea unui lider

- Rețelele de noduri pot fi ierarhizate (sau nu).
- Un lider poate avea mai multe roluri esentiale în cadrul unei rețele (în cazul de fata: stabilirea topologiei și distribuirea ei către restul nodurilor din rețea)
- Liderul unei rețele poate să fie un singur nod sau un grup de noduri, iar acesta este ales de către rețea pe baza unei euristici / unui algoritm.
- Pentru stabilirea unui lider, folosim algoritmul Heartbeat, prin care alegem ca lider nodul cu rangul cel mai mare.

Algoritmul Heartbeat

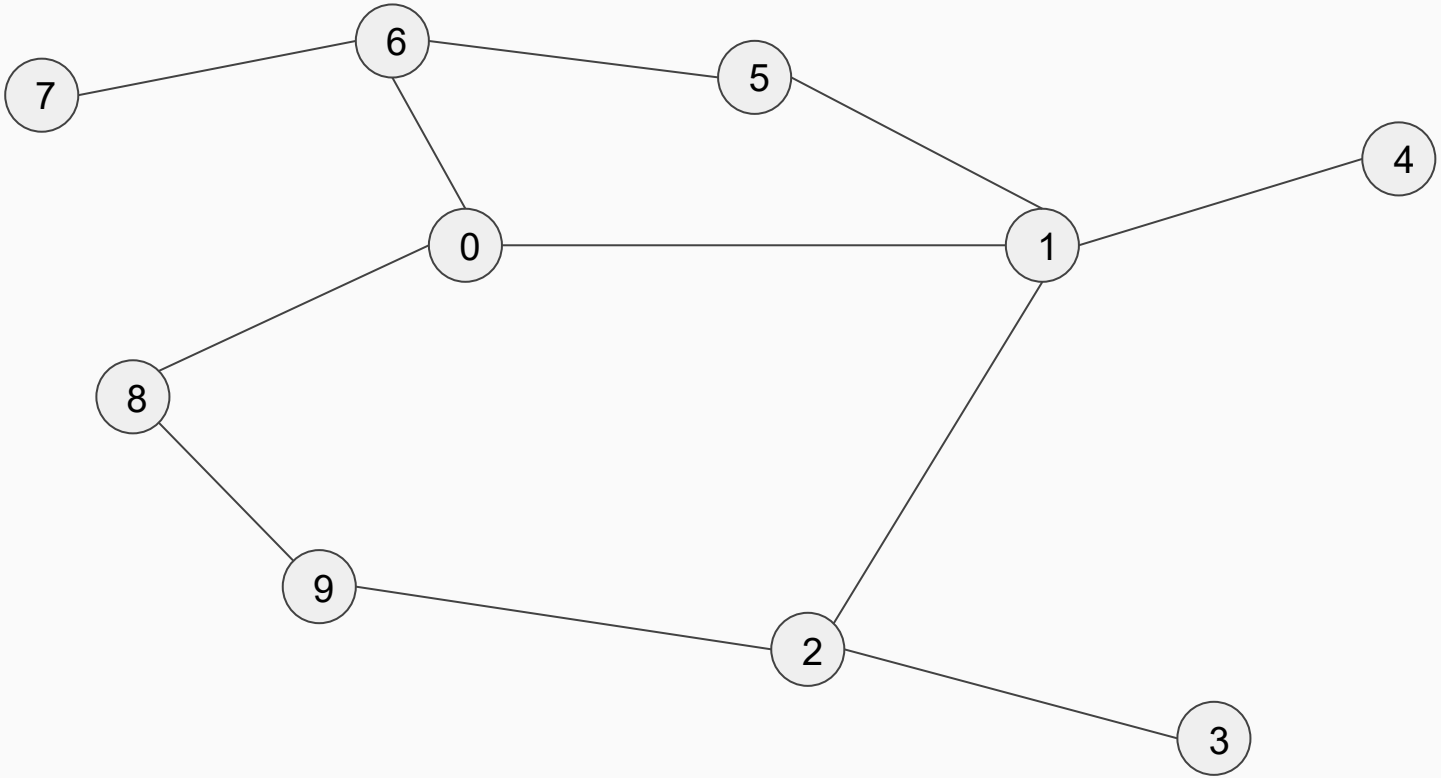
- Algoritmul Heartbeat funcționează în două părți:
  - sistolă / expandare - un nod trimite informația mai departe către vecini
  - diastolă / contractie - un nod primește informație de la vecini
- Algoritmul are următorii pași de execuție:
  - nodurile își trimit rangurile către vecini
  - fiecare nod va face maxim dintre valoarea curentă (lider local) și cea primită
  - vecinii își trimit liderul local înapoi către nodurile părinte
  - nodurile ce primesc liderul local al vecinilor își vor actualiza valoarea de lider local

Algoritmul Heartbeat - pseudocod

```
leader = rank; // la inceput, fiecare nod se considera lider

for (pas = 0; pas < convergenta; pas++) {
          for (i = 0 to nr_vecini) {
          int recv_val;
          trimite leader catre procesul vecin[i]
          primeste recv_val de la oricare proces // aici nu are relevanta, ca se
primeste de la orice vecin
          leader = max(recv_val, leader);
     }
}
```

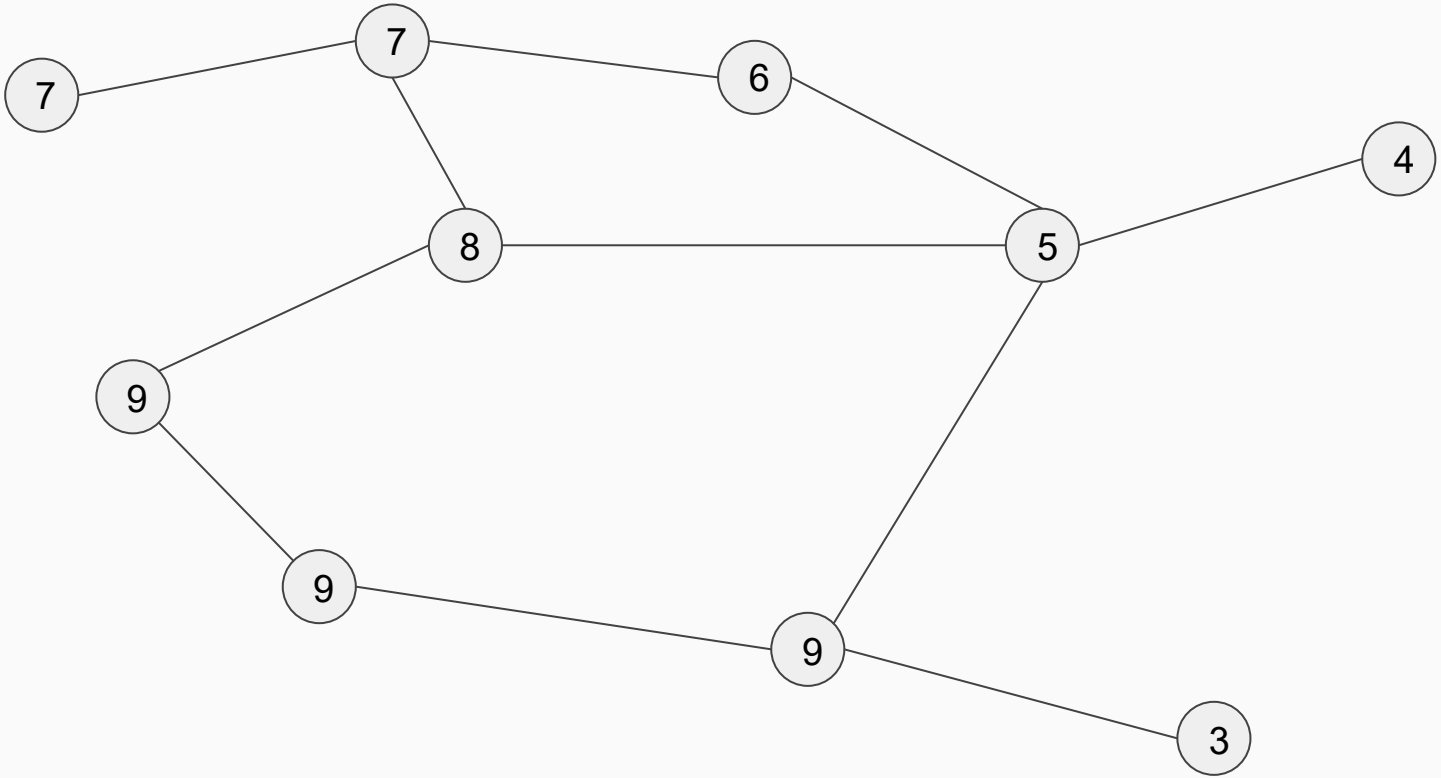
Algoritmul Heartbeat - desfășurare

initial



Algoritmul Heartbeat - desfășurare

pas = 1

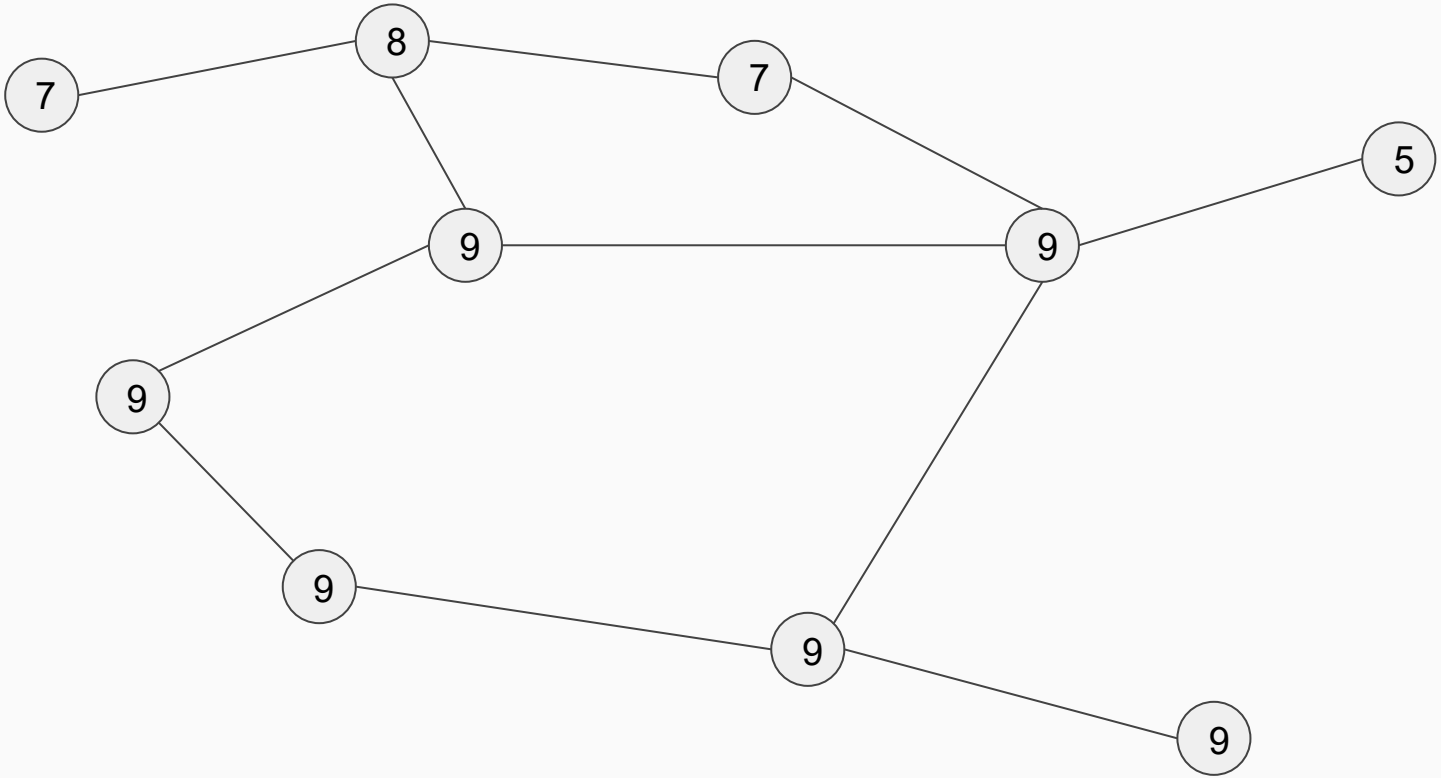


Algoritmul Heartbeat - desfășurare: pas = 1

- 0: max(0, 1, 6, 8) = 8
- 1: max(1, 0, 2, 4, 5) = 5
- 2: max(2, 1, 3, 9) = 9
- 3: max(3, 2) = 3
- 4: max(4, 1) = 4
- 5: max(5, 1, 6) = 6
- 6: max(6, 0, 5, 7) = 7
- 7: max(7, 6) = 7
- 8: max(8, 0, 9) = 9
- 9: max(9, 2, 8) = 9

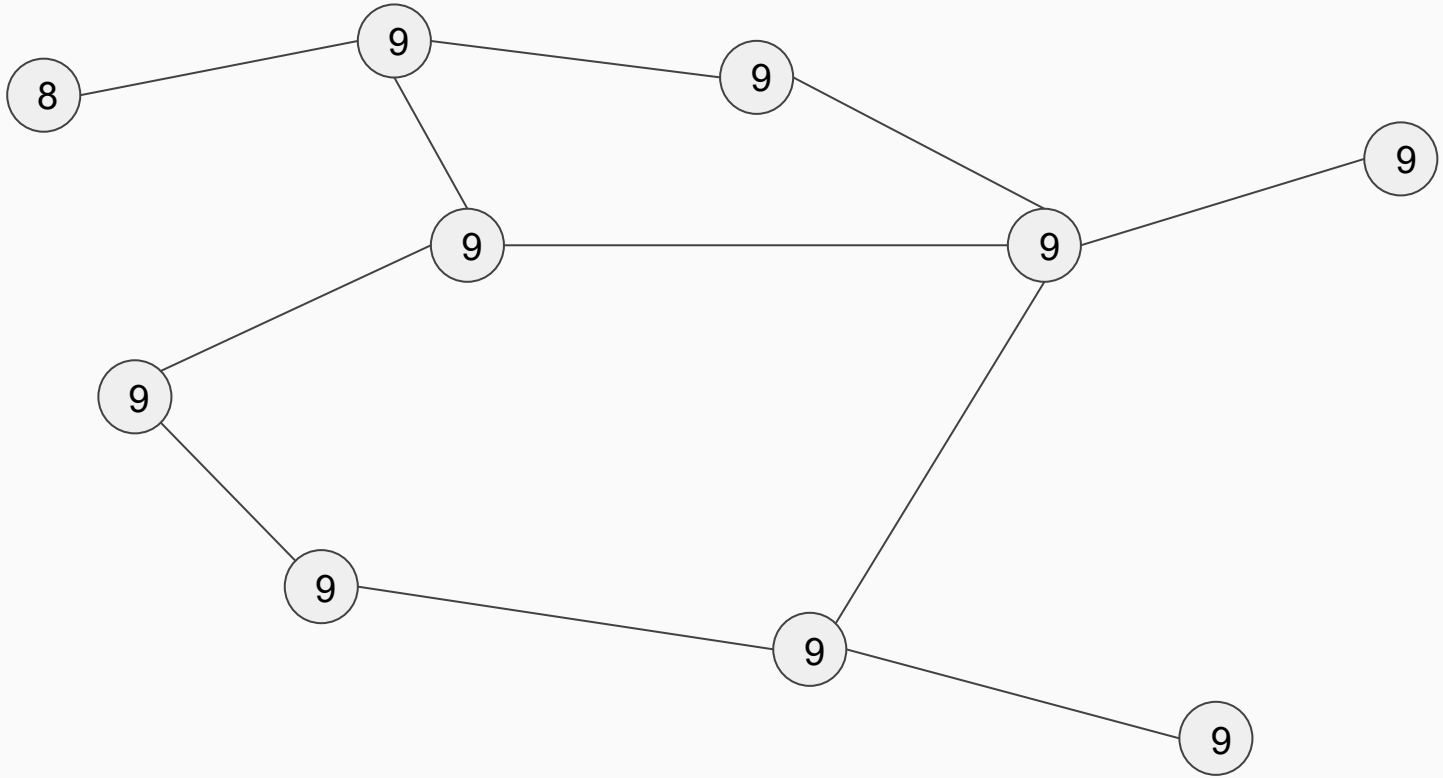
Algoritmul Heartbeat - desfășurare

pas = 2



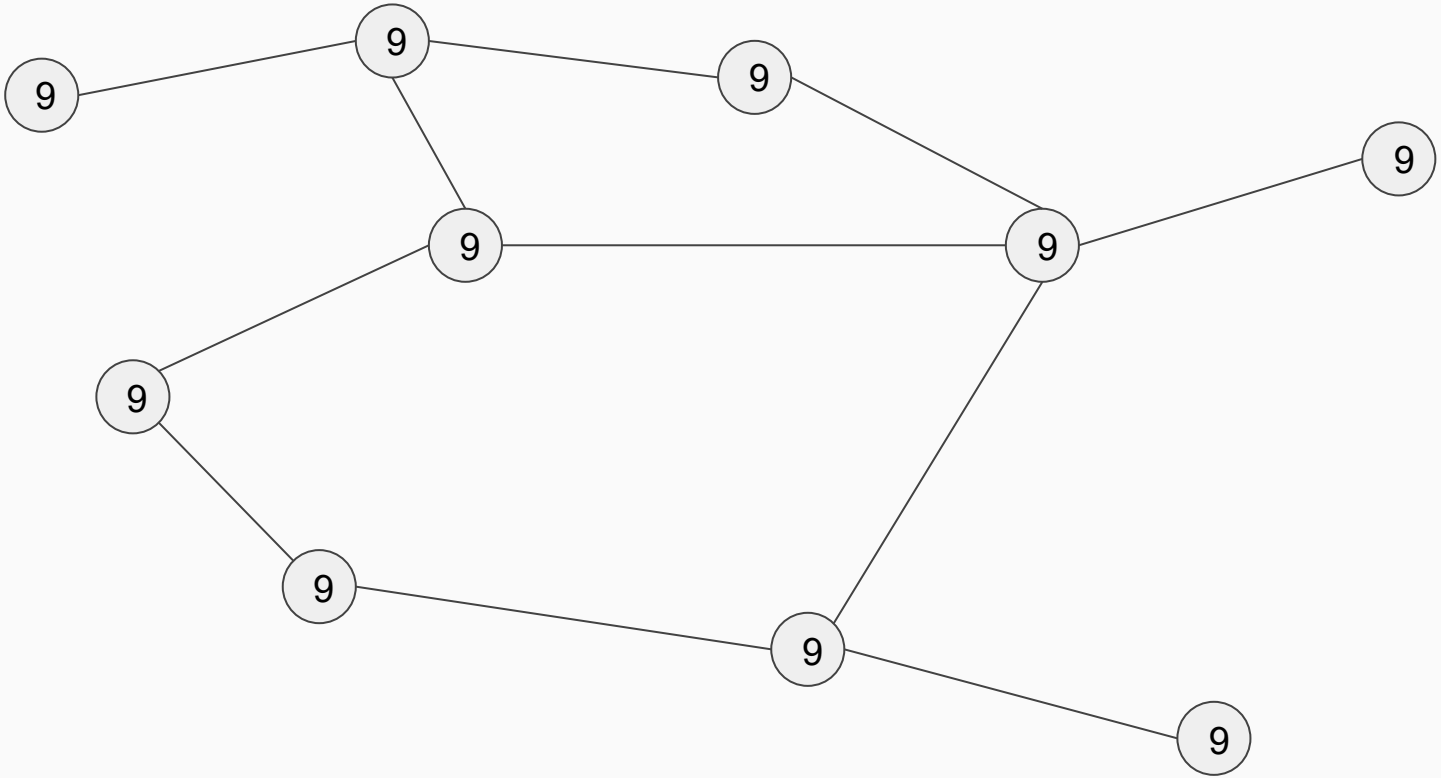
Algoritmul Heartbeat - desfășurare

pas = 3

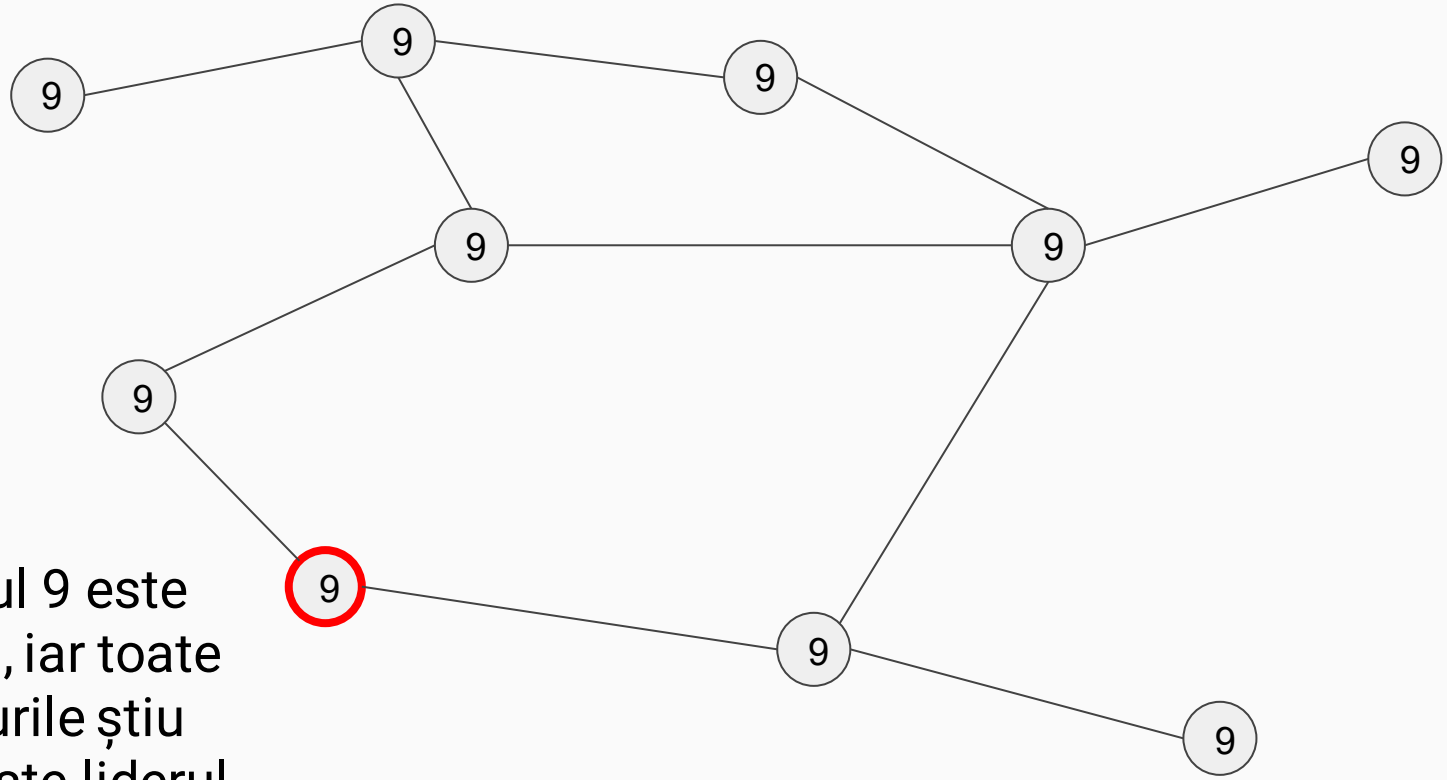


Algoritmul Heartbeat - desfășurare

pas = 4



Algoritmul Heartbeat - desfășurare



Nodul 9 este liderul, iar toate nodurile știu care este liderul

Construirea arborelui de acoperire

- După ce liderul a fost ales, trebuie construit un arbore de acoperire, pentru stabilirea topologiei, prin care fiecare nod să știe cum să ajungă la lider.
- Un arbore de acoperire îl reprezentăm folosind un vector de părinți.
- Pentru construirea unui arbore de acoperire, folosim algoritmul sondă-ecou (undă-ecou).

Algoritmul undă - ecou

- Algoritmul undă-ecou (sondă-ecou) este un algoritm undă care două părți:
  - unda - toate nodurile își trimit informațiile către vecini (cu excepția nodului părinte - liderul nu are părinte, el inițiază trimiterea de mesaje)
  - ecou - nodurile primesc mesaje de la vecini (cu excepția părintelui) și trimit mai departe către părinte. Informațiile primite de la vecini sunt adăugate într-un array de părinți, care e pasat părintelui.
- La final, liderul va obține array-ul de părinți, care va reprezenta arborele de acoperire, care este ulterior distribuit în rețea

Algoritmul undă ecou (sondă ecou) - pseudocod

```
for (i = 0; i < nr_vecini; i++) {
          // vedem daca vecinul curent nu este parintele nodului, caci primim date de la parinte pe care
le pasam celorlalti vecini
          if (vecin[i] != parinte[rank]) {
                    trimite array-ul de parinti (parinte[]) catre vecin[i] // sonda sau unda
     }
}

for (i = 0; i < nr_vecini; i++) {
          // vedem daca vecinul curent nu este parintele nodului
          if (vecin[i] != parinte[rank]) {
                    primeste array-ul de parinti (parinteRecv[]) de la vecin[i] // ecou
                    for (j = 0; j < nr_procese; j++) {
                              // daca nodul / procesul j are parinte (parinte[j] == -1 => nodul j nu
are parinte)
                              // se completeaza array-ul de parinti (bottom - top, de la radacina pana
in varf)
                              if (parinteRecv[j] != -1) {
                                        parinte[j] = parinteRecv[j];
               }
          }
     }
```

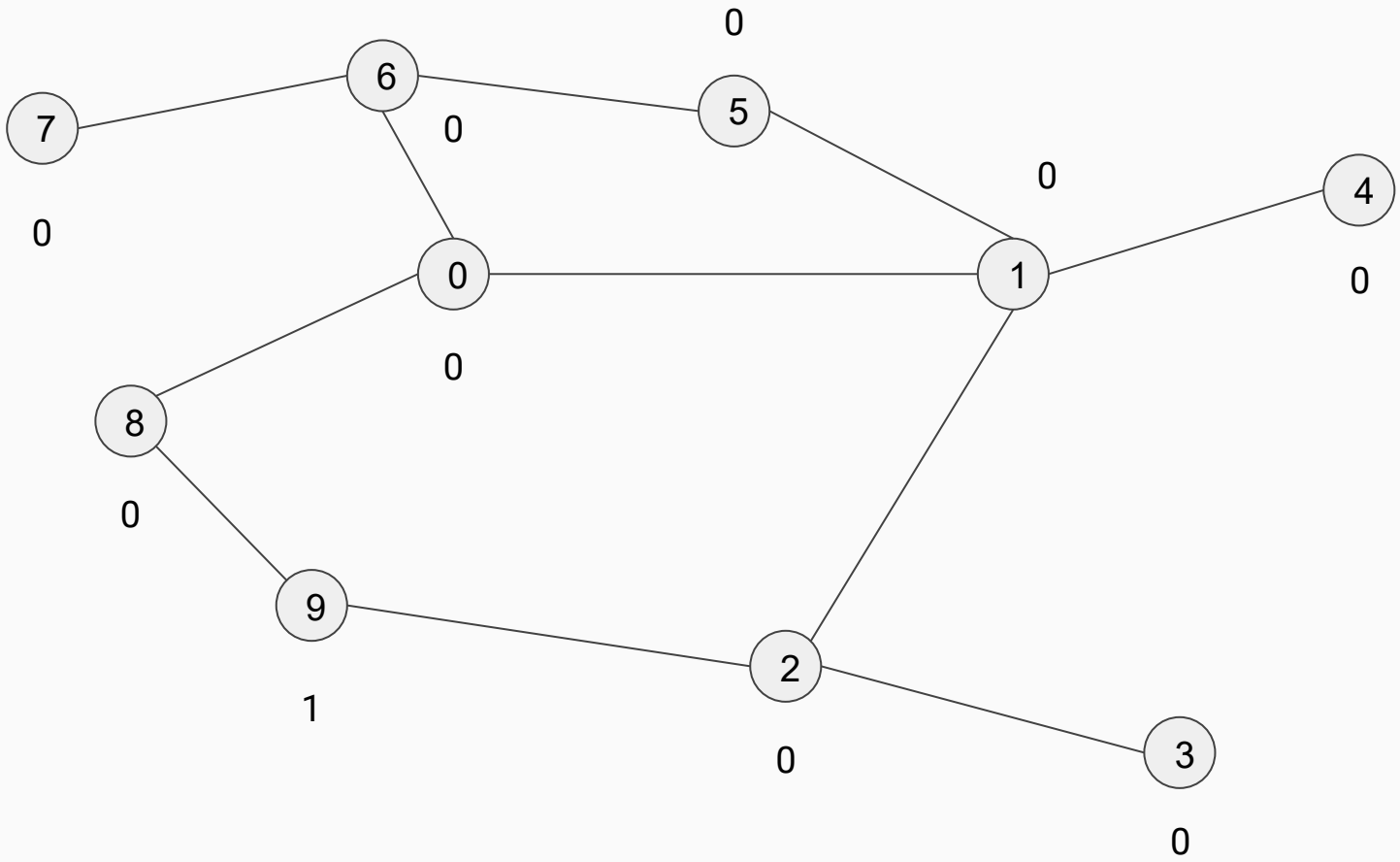
Algoritmul undă ecou (sondă ecou) - pseudocod - continuare

```
// topologia e stabilita (adica nodul curent are topologia formata de nodurile copii pana la nivelul
curent)
if (rank != lider) {
          // fiecare proces care nu e lider trimite topologia curenta catre parintele sau, care o va
completa
          trimite array de parinti (parinti[]) catre parinte[rank] // unda / sonda
          // parintele, dupa ce a completat topologia, o trimite inapoi catre nodul copil
          primeste array de parinit (parinti[]) de la parinte[rank] // ecou
}

// acum topologia este completa
for (i = 0; i < nr_proc; i++) {
          // procesul curent va trimite copiilor sai topologia completata
          if (parinte[i] == rank) {
                    trimite array de parinti (parinti[]) catre procesul / nodul i
     }
}
```

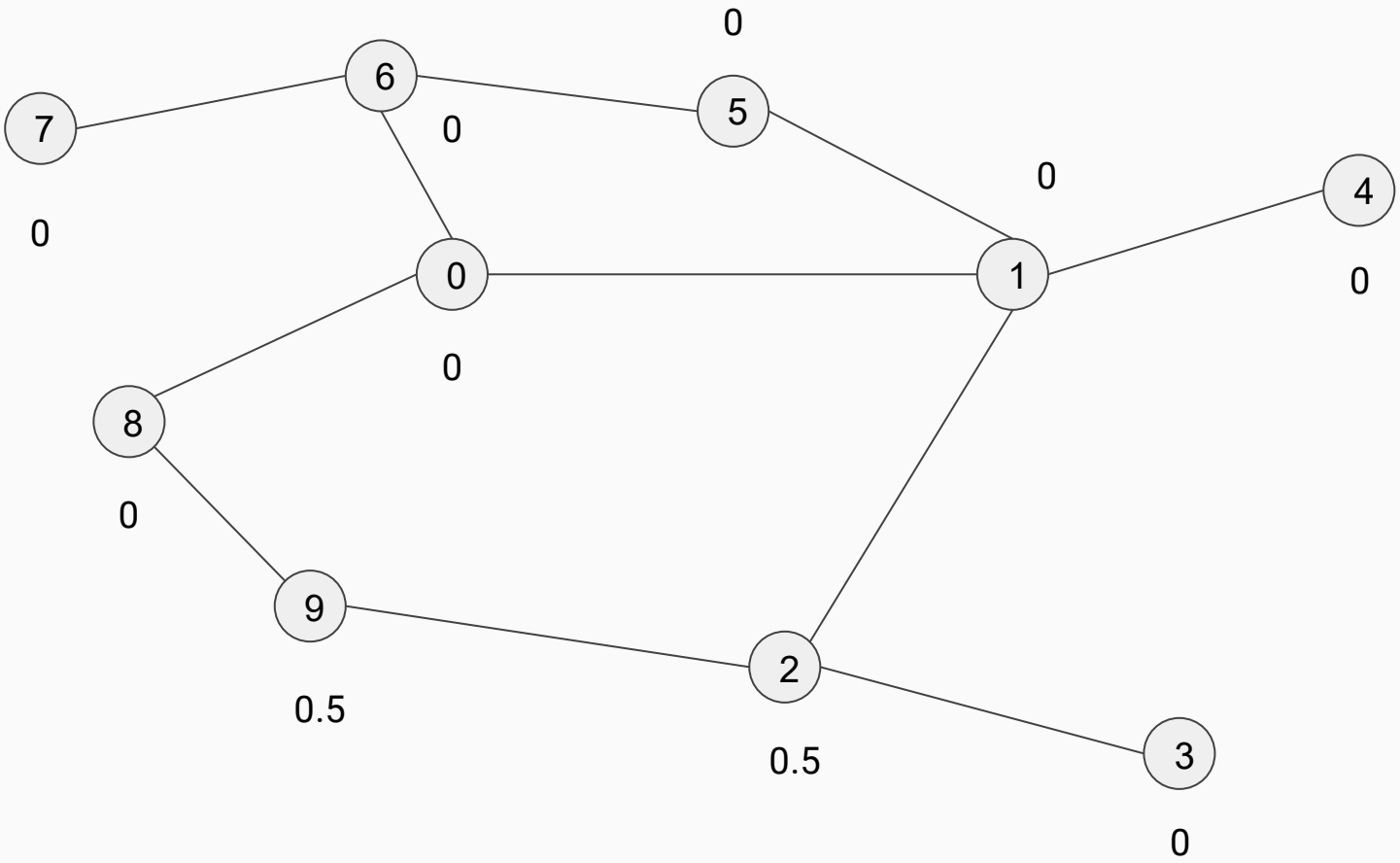
Numărarea nodurilor din topologie

- După ce am ales liderul și am distribuit topologia rețelei către toate nodurile, este nevoie să validăm această informație. În acest caz, validarea se face prin numărarea de noduri prezente în rețea, folosind un algoritm de tip epidemic.
- Algoritmul epidemic funcționează în felul următor:
  - nodul lider (pacient zero) își trimite informația către vecini
  - vecinii nodului lider primesc informația (se “infectează”) și trimit informația mai departe către vecini
  - un nod care o valoare val, iar cand primește o valoare recv_val de la un vecin, nodul curent își actualizează valoarea val în felul următor: val = (val + recv_val) / 2
  - numărul de noduri = 1 / val

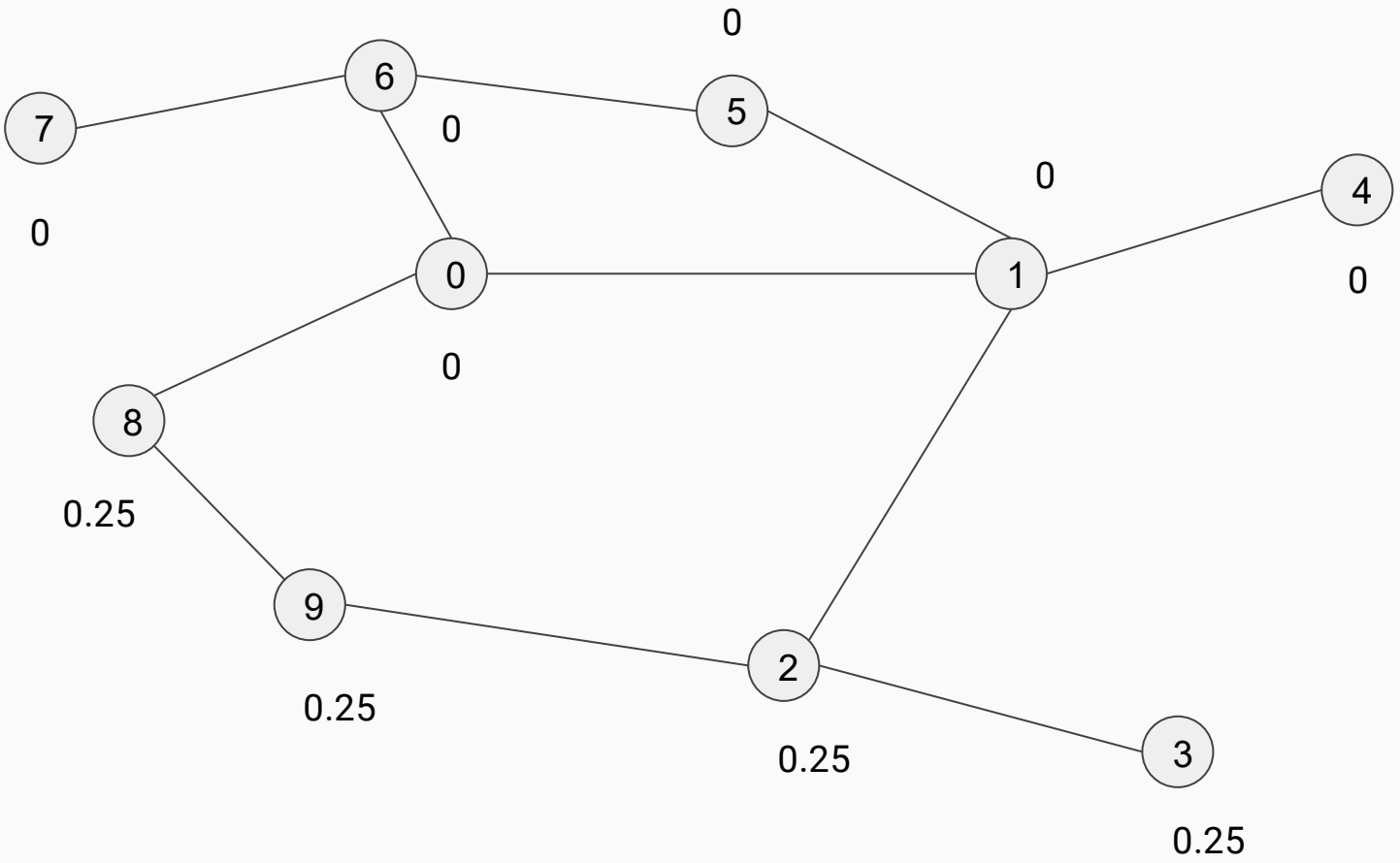
Numărarea nodurilor din topologie



Numărarea nodurilor din topologie



Numărarea nodurilor din topologie



Numărarea nodurilor din topologie - pseudocod

```
double val = 0

if (rank == leader)
          val = 1
else
          val = 0

for (pas = 0; pas < convergenta; pas++) {
          for (i = 0 to nr_vecini) {
          double recv_val;
          trimite val catre procesul vecin[i]
          primeste recv_val de la vecin[i]
          val = (val + recv_val) / 2;
     }
}

return (round) (1 / val); // numarul de noduri din rețea
```

Matricea de topologie a rețelei de noduri

- Folosindu-ne de array-ul de părinți, putem construi o matrice N x N (N fiind numărul de procese) care să arate, în ansamblu, topologia întreagă a rețelei de noduri, folosindu-ne de algoritmul sondă-ecou.
- Pașii sunt următorii:
  - fiecare nod primește topologia copiilor săi (mai puțin de la părinte), rând cu rând, și își actualizează matricea de topologie
  - fiecare nod își trimite matricea de topologie către părinte
  - fiecare nod (cu excepția liderului) primește înapoi matricea completată de părinte
  - fiecare nod trimite topologia completată copiilor săi

Matricea de topologie a rețelei de noduri - pseudocod

```
// se primesc informatii de la toti copiii procesului / nodului curent si se actualizeaza
matricea de topologie
for (i = 0 to nr_proc) {
        // pentru copiii procesului / nodului curent
        if (rank == parents[i]) {
                for (j = 0 to nr_proc) {
                        // se primeste, linie cu linie, topologia locala procesului /
nodului i
                        se primeste recvTopology[j] de la nodul i
                        for (k = 0 to nr_proc) {
                if (top[j][k] == 0) {
                        // actualizam matricea de topologie
                        top[j][k] = recvTopology[j][k];
                }
            }
        }
    }
}
```

Matricea de topologie a rețelei de noduri - pseudocod

```
// nodul curent trimite matricea de topologie proprie catre parinte, rand cu rand
for (i = 0 to nr_proc) {
            // daca procesul curent are parinte
            if (parents[rank] != -1) {
                        se trimite top[i] catre parents[rank]
      }
}

// daca nodul curent nu e lider, primeste topologia, rand cu rand, de la parinte
if (rank != leader) {
            // nodul lider nu are parinte
      for (i = 0 to nr_proc) {
            se primeste top[i] de la parents[rank]
      }
}

// se trimite topologia completa catre nodurile-copii, linie cu linie
for (i = 0 to nr_vecini) {
            for (j = 0 to nr_proc) {
                        trimite top[j] catre vecin[i]
      }
}
```

Alegerea liderului în topologii inel

Algoritmul LeLann

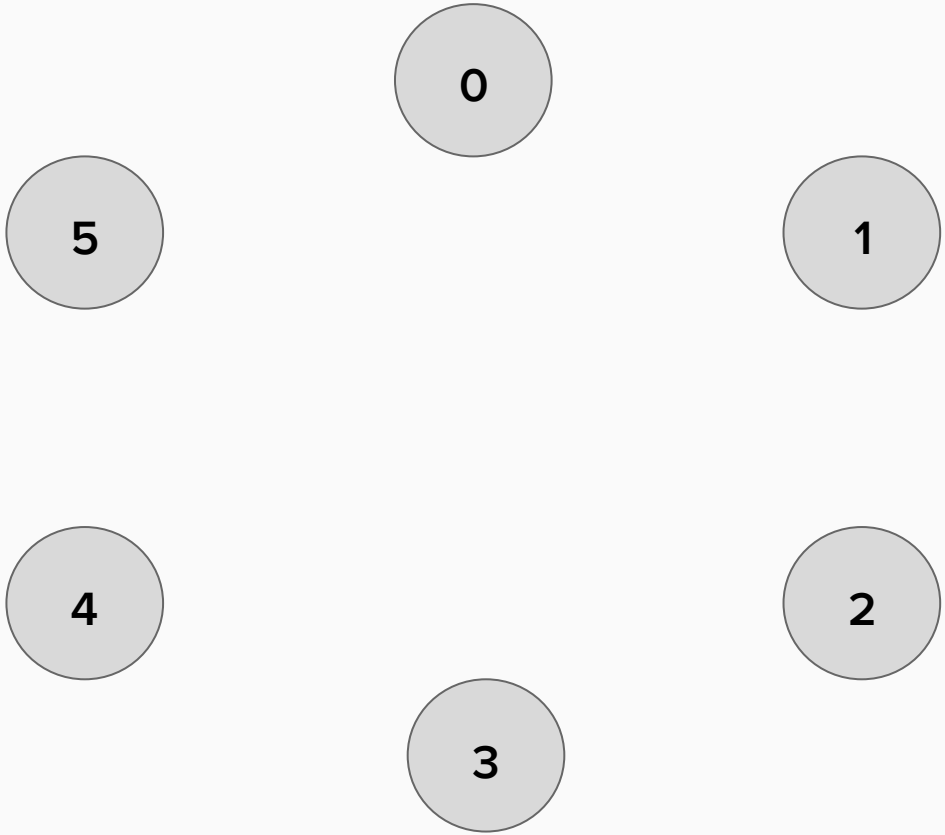
- Algoritmul LeLann este un algoritm undă pentru alegerea unui lider în cadrul unei rețele de tip inel de noduri.
- Acest algoritm se desfășoară în felul următor:
  - fiecare nod are o listă cu identificatorii pe care i-a primit
  - fiecare nod își trimite rangul către celelalte procese și colectează rangurile celorlaltor procese în lista de identificatori
  - fiecare nod primește valoarea de la nodul precedent și o pasează nodului următor
  - când nodul își primește rangul de la nodul precedent, se consideră că nodul curent are toate rangurile nodurilor din rețea în lista de identificatori
  - la final va face maximul din lista respectivă: dacă rangul său este egal cu maximul, atunci este nod lider

Algoritmul LeLann - pseudocod

```
enum STATE {CANDIDATE, LEADER, LOST}

process P[rank] {
            list = []
            list.append(rank)
            state = STATE.CANDIDATE

            int recv_val, val = rank
            trimite val către procesul următor
            primește recv_val de la procesul precedent
            val = recv_val

            while (recv_val != rank) {
                        list.append(recv_val)
                        trimite val către procesul următor
                        primește recv_val de la procesul precedent
                        recv_val = val
      }

      if (rank == max(list))
            state = STATE.LEADER
      else
            state = STATE.LOST

}
```

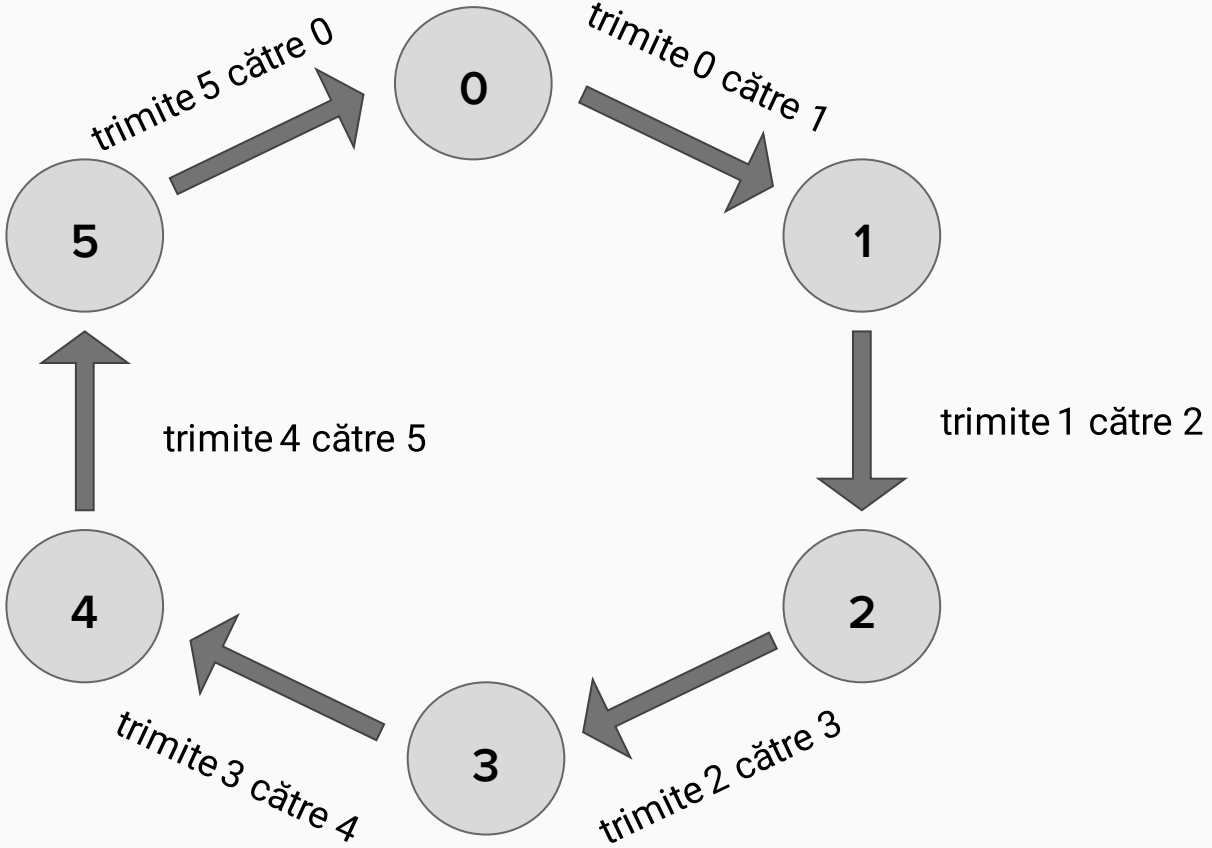
Algoritmul LeLann

0: [0]
1: [1]
2: [2]
3: [3]
4: [4]
5: [5]



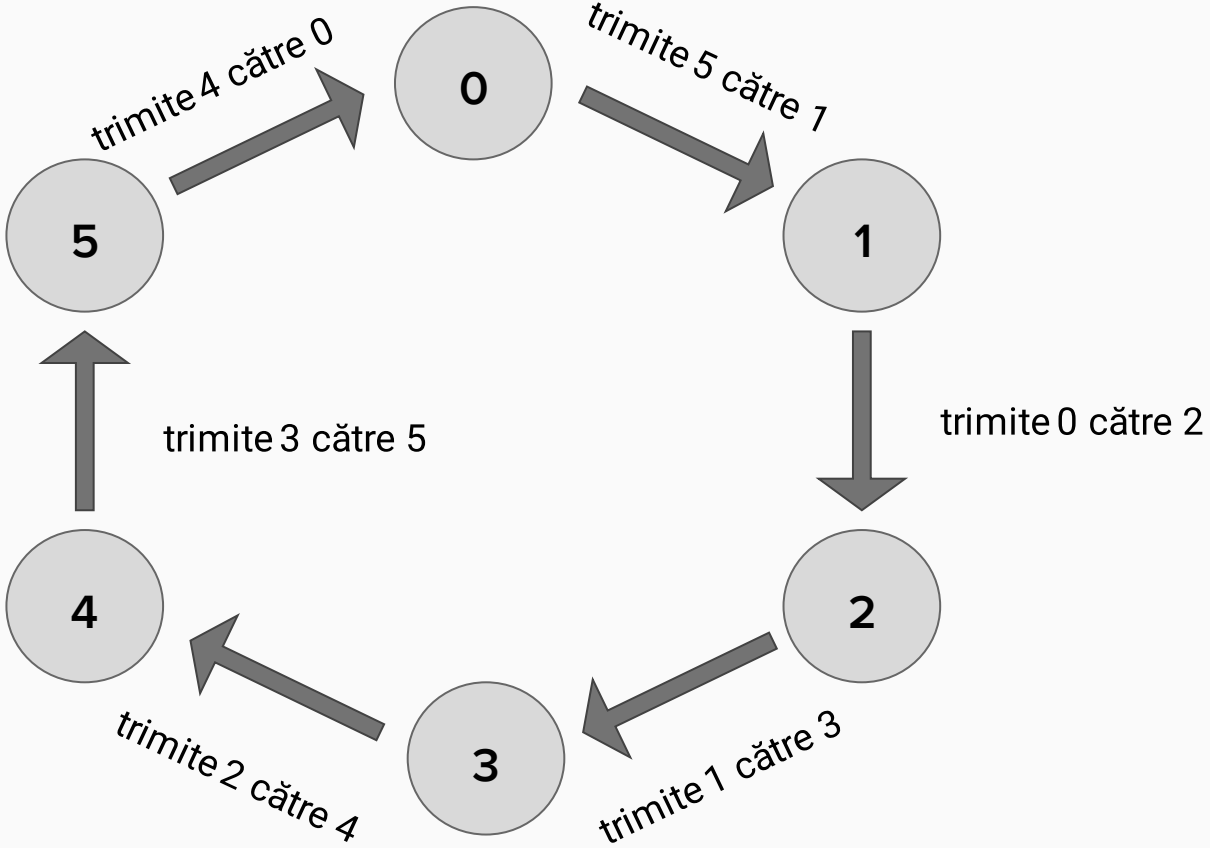
Algoritmul LeLann

0: [0, 5]
1: [1, 0]
2: [2, 1]
3: [3, 2]
4: [4, 3]
5: [5, 4]



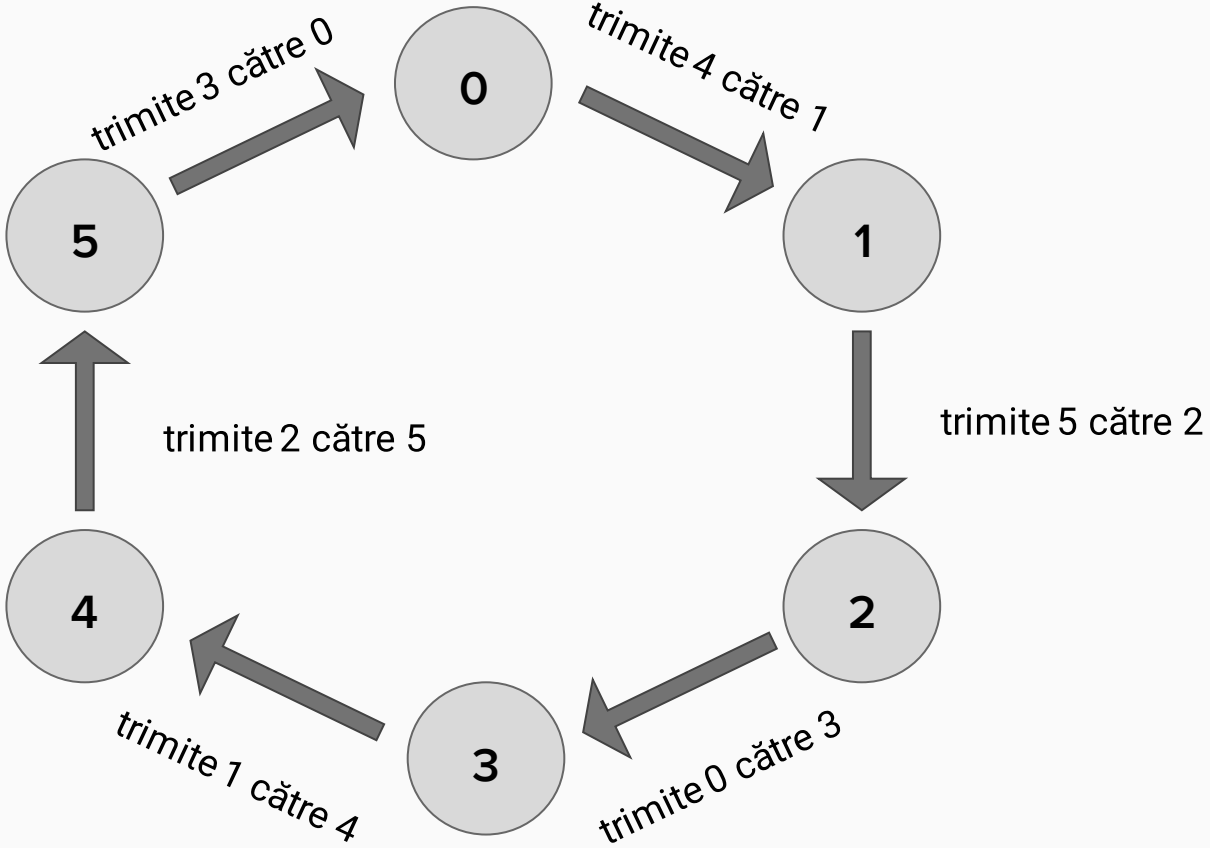
Algoritmul LeLann

0: [0, 5, 4]
1: [1, 0, 5]
2: [2, 1, 0]
3: [3, 2, 1]
4: [4, 3, 2]
5: [5, 4, 3]



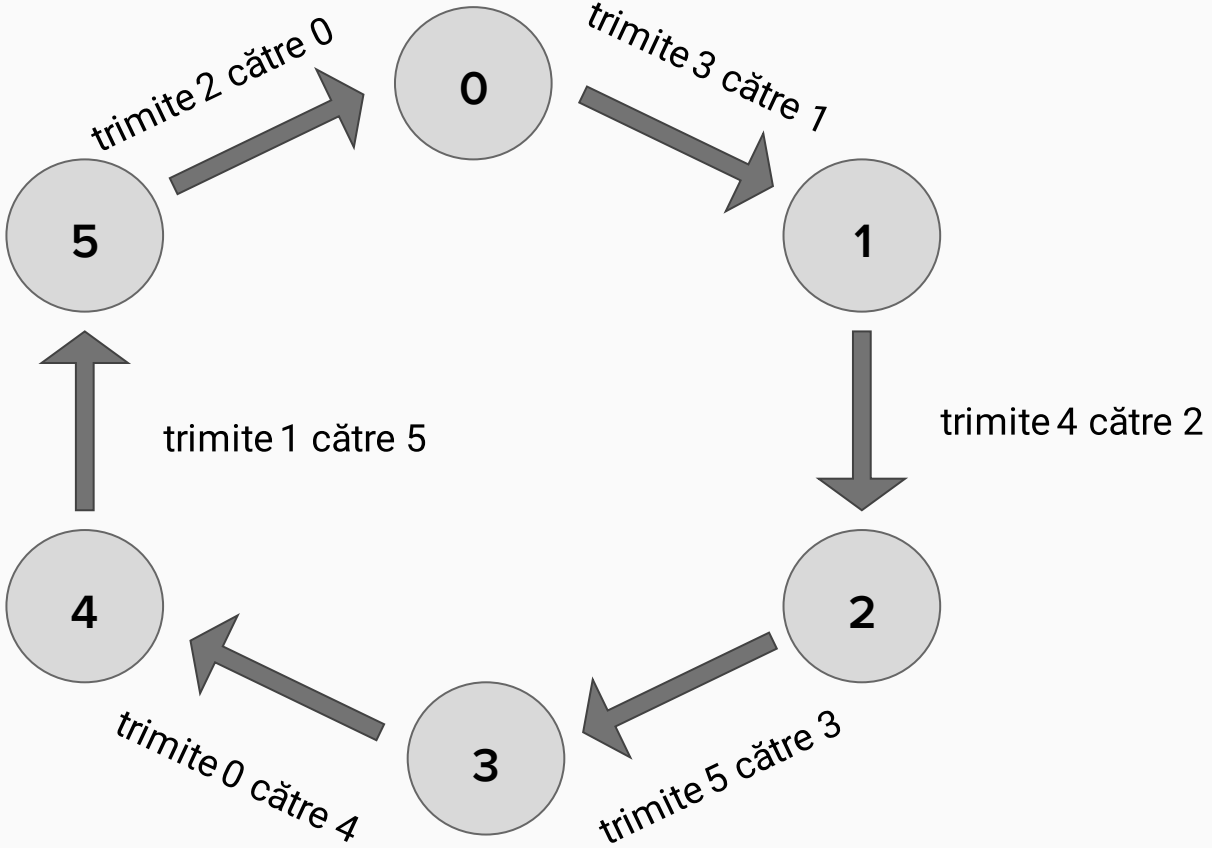
Algoritmul LeLann

0: [0, 5, 4, 3]
1: [1, 0, 5, 4]
2: [2, 1, 0, 5]
3: [3, 2, 1, 0]
4: [4, 3, 2, 1]
5: [5, 4, 3, 2]



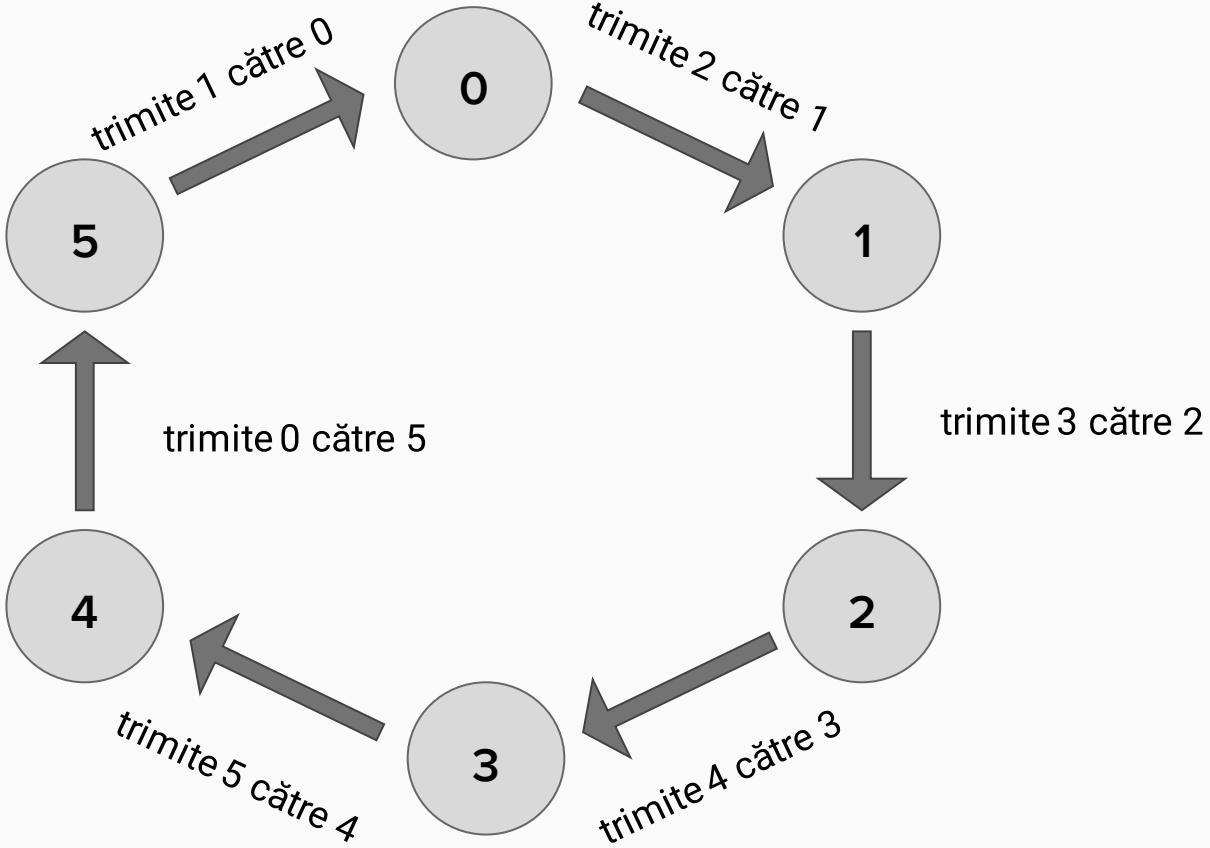
Algoritmul LeLann

0: [0, 5, 4, 3, 2]
1: [1, 0, 5, 4, 3]
2: [2, 1, 0, 5, 4]
3: [3, 2, 1, 0, 5]
4: [4, 3, 2, 1, 0]
5: [5, 4, 3, 2, 1]



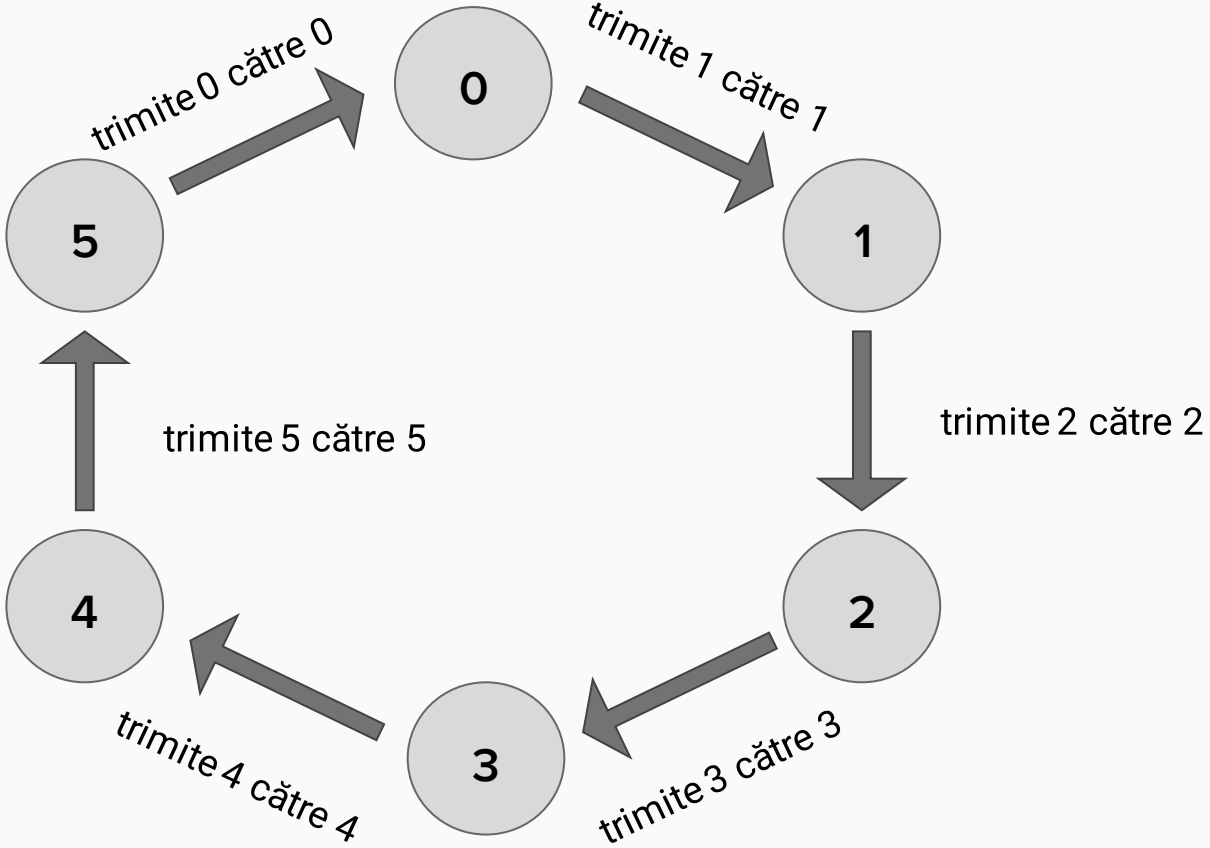
Algoritmul LeLann

0: [0, 5, 4, 3, 2, 1]
1: [1, 0, 5, 4, 3, 2]
2: [2, 1, 0, 5, 4, 3]
3: [3, 2, 1, 0, 5, 4]
4: [4, 3, 2, 1, 0, 5]
5: [5, 4, 3, 2, 1, 0]



Algoritmul LeLann

0: [0, 5, 4, 3, 2, 1]
1: [1, 0, 5, 4, 3, 2]
2: [2, 1, 0, 5, 4, 3]
3: [3, 2, 1, 0, 5, 4]
4: [4, 3, 2, 1, 0, 5]
5: [5, 4, 3, 2, 1, 0]

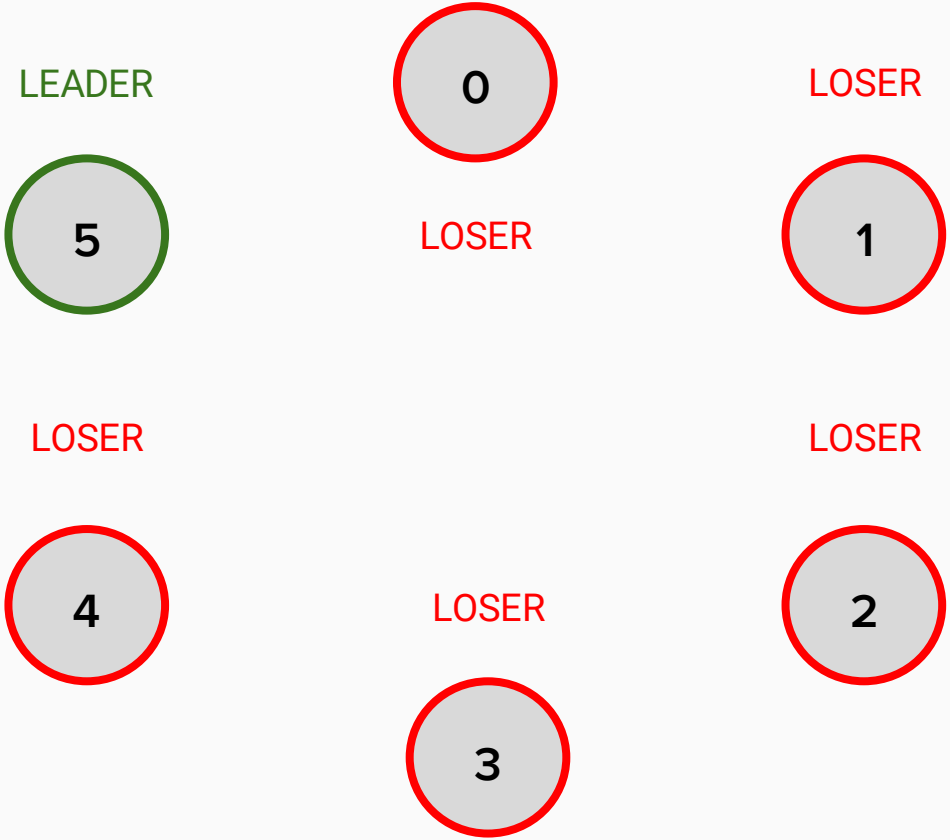


În acest moment, fiecare proces își primește propriul său rang -> **STOP**

Nodul 5 devine lider

Algoritmul LeLann

0: [0, 5, 4, 3, 2, 1]
1: [1, 0, 5, 4, 3, 2]
2: [2, 1, 0, 5, 4, 3]
3: [3, 2, 1, 0, 5, 4]
4: [4, 3, 2, 1, 0, 5]
5: [5, 4, 3, 2, 1, 0]



Algoritmul LeLann-Chang-Roberts

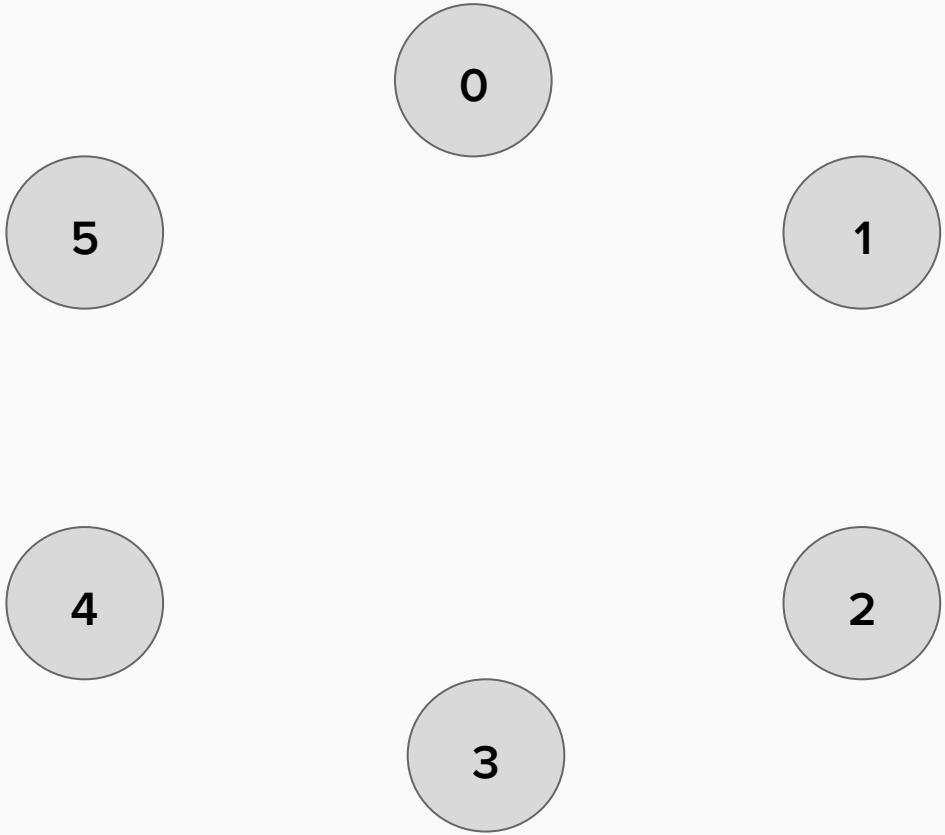
- Algoritmul LeLann-Chang-Roberts este un algoritm undă pentru alegerea unui lider în cadrul unei rețele de tip inel de noduri.
- Fiecare proces își trimite rangul mai departe, către celelalte procese și primește rangurile celorlaltor procese, pe care le pasează către vecini, ca la LeLann.
- Diferența este următoarea, la primirea de mesaje:
  - dacă un proces își primește rangul, acesta devine lider
  - dacă un proces primește un id mai mare decât rangul său, acesta este un nod loser, care pasează către următorul proces ce primește de la procesul precedent
  - dacă un proces primește un id mai mic decât rangul său, ignoră mesajul primit

Algoritmul LeLann-Chang-Roberts - pseudocod

```
enum STATE {CANDIDATE, LEADER, LOST}

process P[rank] {
            state = STATE.CANDIDATE

            int recv_val, val = rank
            trimite val către procesul următor

            while (state != STATE.LEADER) {
                        primește recv_val de la procesul precedent
                        if (recv_val == rank) {
                        state = STATE.LEADER;
            } else if (recv_val > rank) {
                        if (state == CANDIDATE) {
                                    state = STATE.LOST;
                                    val = recv_val;
                                    trimite val către procesul următor
                        }
            }
      }
}
```

Algoritmul LeLann-Chang-Roberts

0: [0]
1: [1]
2: [2]
3: [3]
4: [4]
5: [5]



Remove Watermark Now

Catalog

C01_introducere 1
C02_Complexitate+emabrassingly parallel 108
C03_Lucru cu semafoare 232
C04_solutii paralele 287
C05_solutii paralele logN 369
C06_algoritmi_genetici 422
C07_ComunicareaPrinMesaje 463
C08_CeaasuriLogice-OrdonareEvenimente 501
C09_AlgoritmiUnda 530
C10_AlegereLider 581
C11_StabilireaTopologiei 632
C12_TerminareaProgramelorDistribuite 657
C13_TolerantaDefecte 701
P01_pthread+Primitive sincronizare 778
P02_Java 912
P03_Sincronizare 974
P04_MPI 1009
P05_FT 1058
P06_CAP 1105
P07_P2P 1150

pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Introducere

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Despre curs

- Interactivitate – dialog
- Open-office: ????, PRECIS 605
- Regulament afișat pe pagina cursului
  - Forma de evaluare a examenului final poate fi decisă/schimbată de către titularul de curs, în funcție de un context specific. Forma de evaluare poate fi: scris sau oral.
- Punctajele obținute pe parcurs sau examen se pot păstra pentru un an universitar (nu acumulare)
- Temele se pot trimite doar pe parcursul primului semestru universitar

Vineri, 9-11 AM

pdfelement

Remove Watermark Now

Despre curs

- Nota:

  10 = **6** (Parcurs) + **4** (Examen)

Laborator+Curs **>= 3** Examen **>= 2**

Parcurs: 3 teme(3p) + laborator (2p) +
Evaluare (1p)

pdfelement

Întrebări?

Remove Watermark Now

Despre mine

pdfelement

http://cipsm.hpc.pub.ro

ciprian.dobre@cs.pub.ro

http://www.facebook.com/ciprian.dobre

Remove Watermark Now

Bibliografia

G.R.Andrews
Concurrent Programming. Principles and Practice

C.A.R. Hoare
Communicating Sequential Processes

S.G. Akl
The Design and Analysis of Parallel Algorithms

G.R. Andrews, R.A. Olsson
The SR Programming Language. Concurrency in Practice

Ian Foster
Designing and Building Parallel Programs

A.S. Tanenbaum
Structured Computer Organization (Fourth Edition)

Allen B. Downey
The Little Book of Semaphores

Wan Fokkink
Distributed Algorithms: An Intuitive Approach (2nd edition)

Nancy A. Lynch
Distributed Algorithms

pdfelement

Remove Watermark Now

De ce calcul paralel și distribuit?

- Timp de execuție mai scurt
- Permite abordarea problemelor de dimensiuni mari
- Accesul resurselor aflate la distanță
- Reducerea costurilor
- Toleranță la defecte
- Ascunderea timpilor de așteptare
- Redundanța
- Scalabilitatea
- Scăderea timpului de răspuns
- Securitate

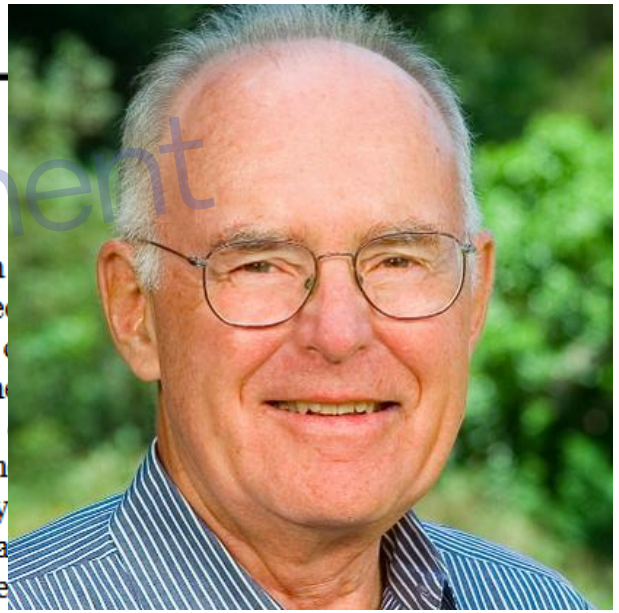
pdfelement

Remove Watermark Now

Limitele programării secvențiale?

Cramming More Components onto Integrated Circuits

GORDON E. MOORE, LIFE FELLOW, IEEE



With unit cost falling as the number of components per circuit rises, by 1975 economics may dictate squeezing as many as 65 000 components on a single silicon chip.

The future of integrated electronics is the future of electronics itself. The advantages of integration will bring about a proliferation of electronics, pushing this science into many new areas.

Integrated circuits will lead to such wonders as home computers—or at least terminals connected to a central computer—automatic controls for automobiles, and personal portable communications equipment. The electronic wristwatch needs only a display to be feasible today.

But the biggest potential lies in the production of large systems. In telephone communications, integrated circuits

Each approach
each borrowed te
believe the way
various approach

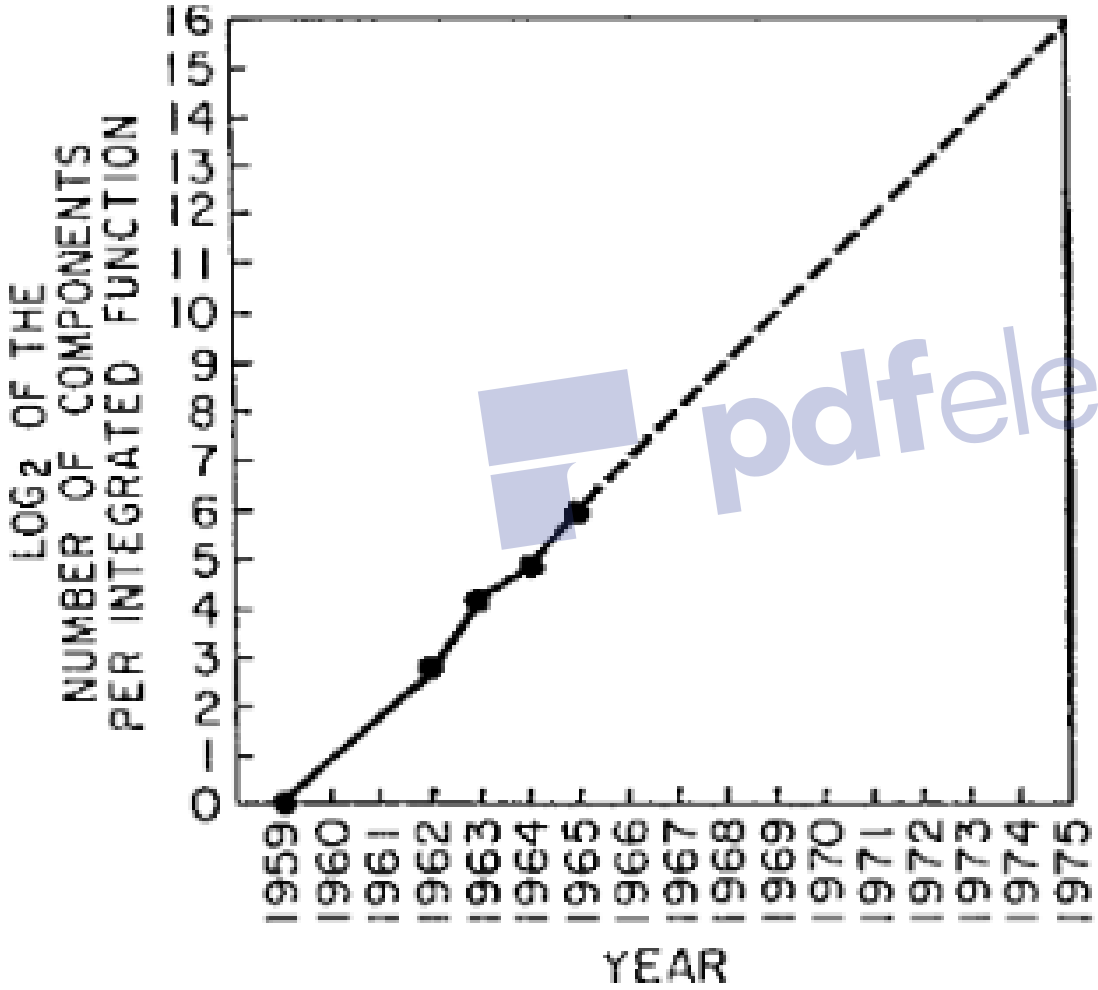
The advocates
already using th
resistors by apply
conductor substra
upon films are de
attachment of active semiconductor devices to the passive film arrays.

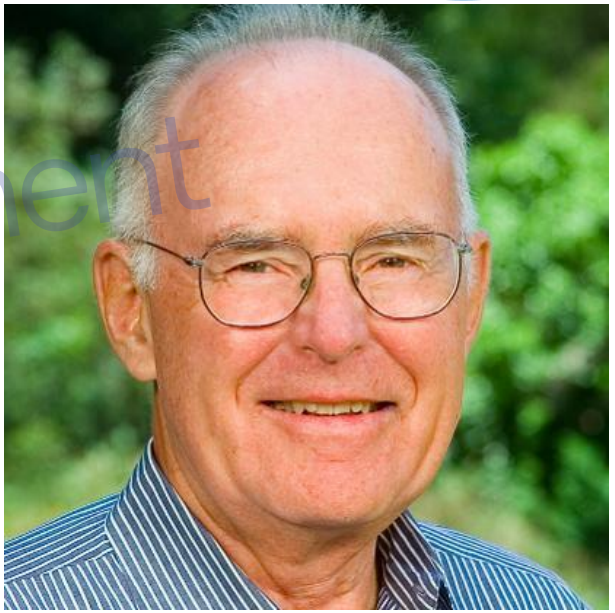
Both approaches have worked well and are being used in equipment today.

pdfelement

Remove Watermark Now

Limitele programării secvențiale?





cofounded Intel

pdfelement

Remove Watermark Now

Limitele programării secvențiale

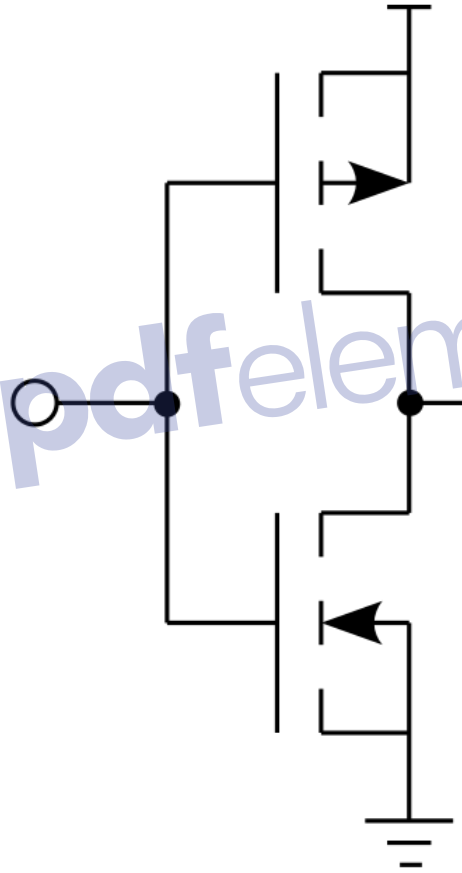
- Viteza de transmisie
  - Maxim c – viteza luminii
- Miniaturizare
  - Tranzistor de mărimea unui atom
- Economic
  - Costuri enorme pentru cercetare și proiectarea unui nou timp de procesor

pdfelement

Remove Watermark Now

Limitele programării secvențiale

- CMOS



pdfelement

Remove Watermark Now

Importanța cursului

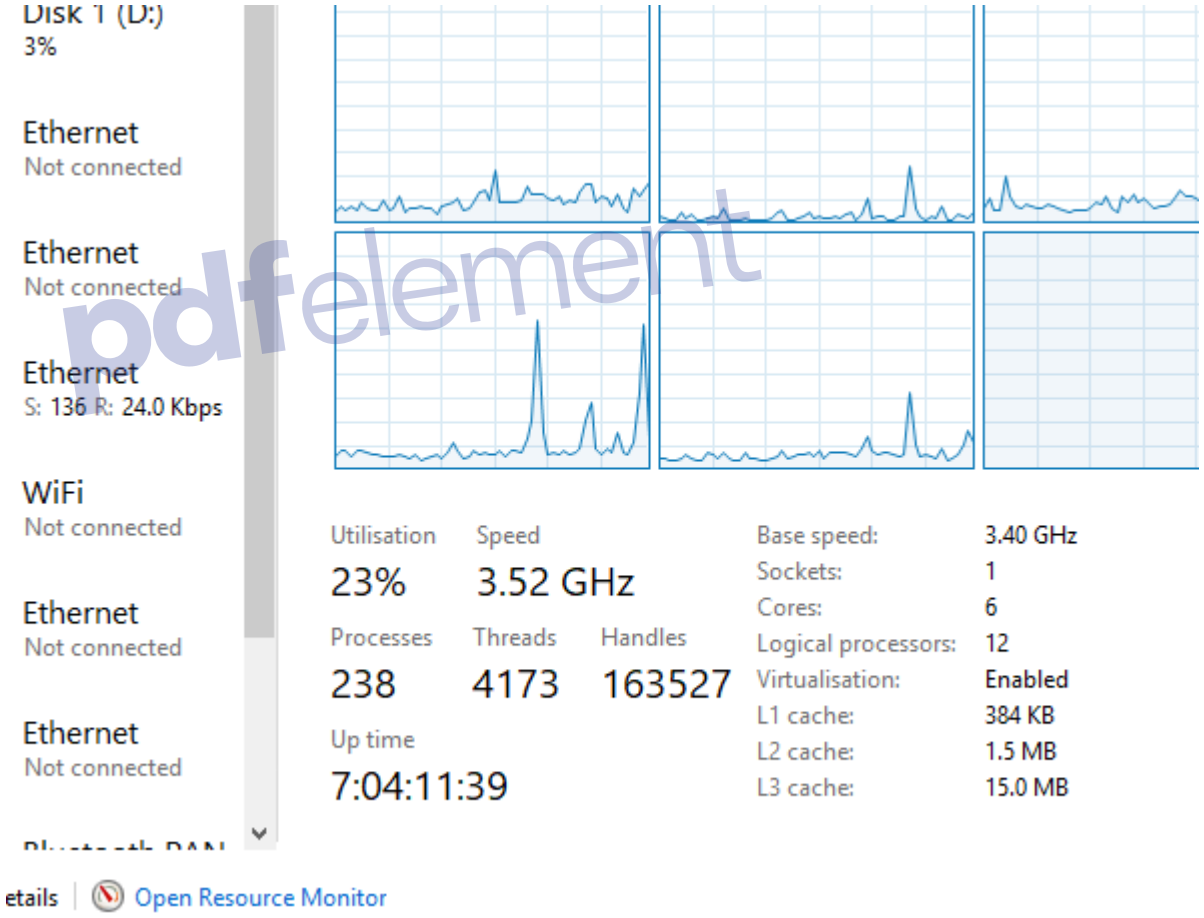
- Apar tot mai multe tehnologii distribuite
  - Blockchain; Peer-to-Peer
- Chiar și un procesor de ceas are mai multe core-uri
  - LG Watch Sport - MSM8909w Processor
    - Quad-Core
- Suport în noile IDE-uri
  - Eclipse; Visual Studio
- Număr mare de aplicații distribuite
  - Dropbox; Spark; Boinc

pdfelement

Remove Watermark Now

Importanța cursului

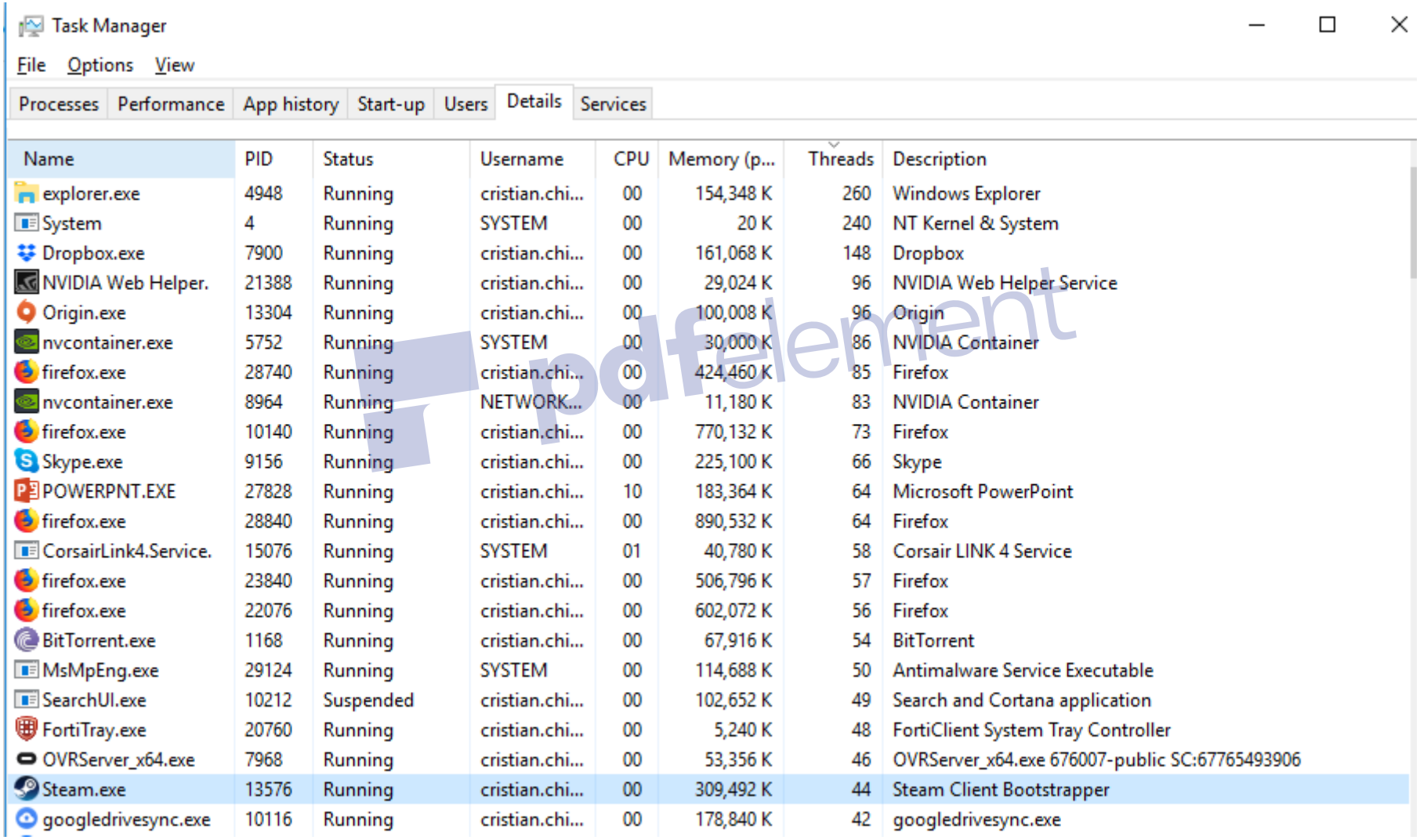
- Aproape toate aplicațiile folosesc mai multe thread-uri



pdfelement

Remove Watermark Now

Importanța cursului



Name	PID	Status	Username	CPU	Memory (p...	Threads	Description
explorer.exe	4948	Running	cristian.chi...	00	154,348 K	260	Windows Explorer
System	4	Running	SYSTEM	00	20 K	240	NT Kernel & System
Dropbox.exe	7900	Running	cristian.chi...	00	161,068 K	148	Dropbox
NVIDIA Web Helper.	21388	Running	cristian.chi...	00	29,024 K	96	NVIDIA Web Helper Service
Origin.exe	13304	Running	cristian.chi...	00	100,008 K	96	Origin
nvcontainer.exe	5752	Running	SYSTEM	00	30,000 K	86	NVIDIA Container
firefox.exe	28740	Running	cristian.chi...	00	424,460 K	85	Firefox
nvcontainer.exe	8964	Running	NETWORK...	00	11,180 K	83	NVIDIA Container
firefox.exe	10140	Running	cristian.chi...	00	770,132 K	73	Firefox
Skype.exe	9156	Running	cristian.chi...	00	225,100 K	66	Skype
POWERPNT.EXE	27828	Running	cristian.chi...	10	183,364 K	64	Microsoft PowerPoint
firefox.exe	28840	Running	cristian.chi...	00	890,532 K	64	Firefox
CorsairLink4.Service.	15076	Running	SYSTEM	01	40,780 K	58	Corsair LINK 4 Service
firefox.exe	23840	Running	cristian.chi...	00	506,796 K	57	Firefox
firefox.exe	22076	Running	cristian.chi...	00	602,072 K	56	Firefox
BitTorrent.exe	1168	Running	cristian.chi...	00	67,916 K	54	BitTorrent
MsMpEng.exe	29124	Running	SYSTEM	00	114,688 K	50	Antimalware Service Executable
SearchUI.exe	10212	Suspended	cristian.chi...	00	102,652 K	49	Search and Cortana application
FortiTray.exe	20760	Running	cristian.chi...	00	5,240 K	48	FortiClient System Tray Controller
OVRServer_x64.exe	7968	Running	cristian.chi...	00	53,356 K	46	OVRServer_x64.exe 676007-public SC:67765493906
Steam.exe	13576	Running	cristian.chi...	00	309,492 K	44	Steam Client Bootstrapper
googledrivesync.exe	10116	Running	cristian.chi...	00	178,840 K	42	googledrivesync.exe

pdfelement

Remove Watermark Now

Supercomputers

Rank	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	**Summit** - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband , IBM DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148,600.0	200,794.9	10,096
2	**Sierra** - IBM Power System S922LC, IBM POWER9 22C 3.1GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband , IBM / NVIDIA / Mellanox DOE/NNSA/LLNL United States	1,572,480	94,640.0	125,712.0	7,438
3	**Sunway TaihuLight** - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway , NRCPC National Supercomputing Center in Wuxi China	10,649,600	93,014.6	125,435.9	15,371
4	**Tianhe-2A** - TH-IVB-FEP Cluster, Intel Xeon E5-2692v2 12C 2.2GHz, TH Express-2, Matrix-2000 , NUDT National Super Computer Center in Guangzhou China	4,981,760	61,444.5	100,678.7	18,482
5	**Frontera** - Dell C6420, Xeon Platinum 8280 28C 2.7GHz, Mellanox InfiniBand HDR , Dell EMC Texas Advanced Computing Center/Univ. of Texas United States	448,448	23,516.4	38,745.9	

pdfelement

Remove Watermark Now

Summit

Remove Watermark Now

Sierra

Remove Watermark Now

Sunway TaihuLight

Remove Watermark Now

Computing power 2018

24-hour average: 28.019 PetaFLOPS.
Active: 156,141 volunteers, 648,324 computers.

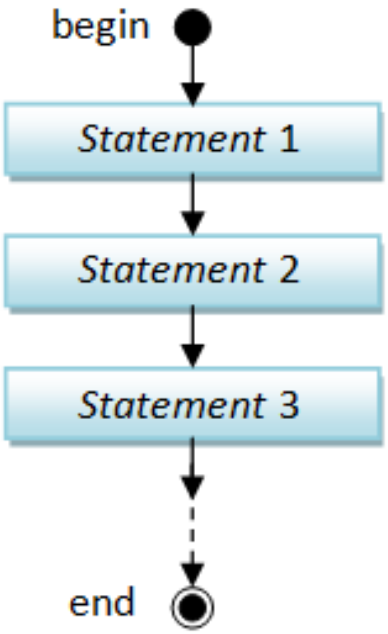
pdfelement

Computing power 2019

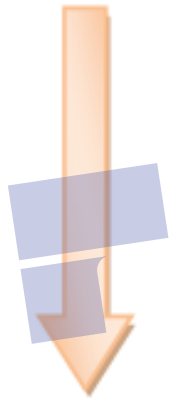
24-hour average: 23.702 PetaFLOPS.
Active: 135,048 volunteers, 473,747 computers.

Remove Watermark Now

Algoritmi Paraleli/Distribuiți vs Secvențiali

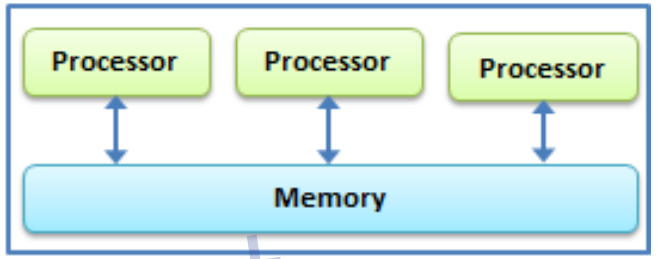


Sequential

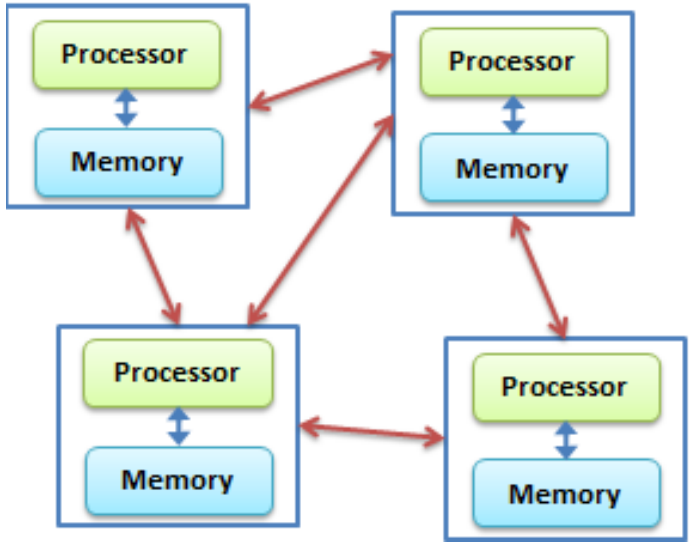


pdfelement

Parallel Computing



Distributed Computing

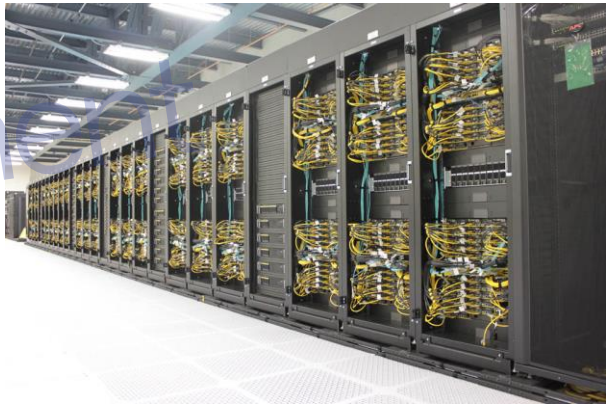


Remove Watermark Now

Resurse fizice

- Procesor – multi-core – 28 core-uri

- Cluster



pdfelement

- Grid/Cloud

Remove Watermark Now

Taxonomia Flynn

Some Computer Organizations and Their Effectiveness

MICHAEL J. FLYNN, MEMBER, IEEE

Abstract—A hierarchical model of computer organizations is developed, based on a tree model using request/service type resources as nodes. Two aspects of the model are distinguished: logical and physical.

General parallel- or multiple-stream organizations are examined as to type and effectiveness—especially regarding intrinsic logical difficulties.

The overlapped simplex processor (SISD) is limited by data dependencies. Branching has a particularly degenerative effect.

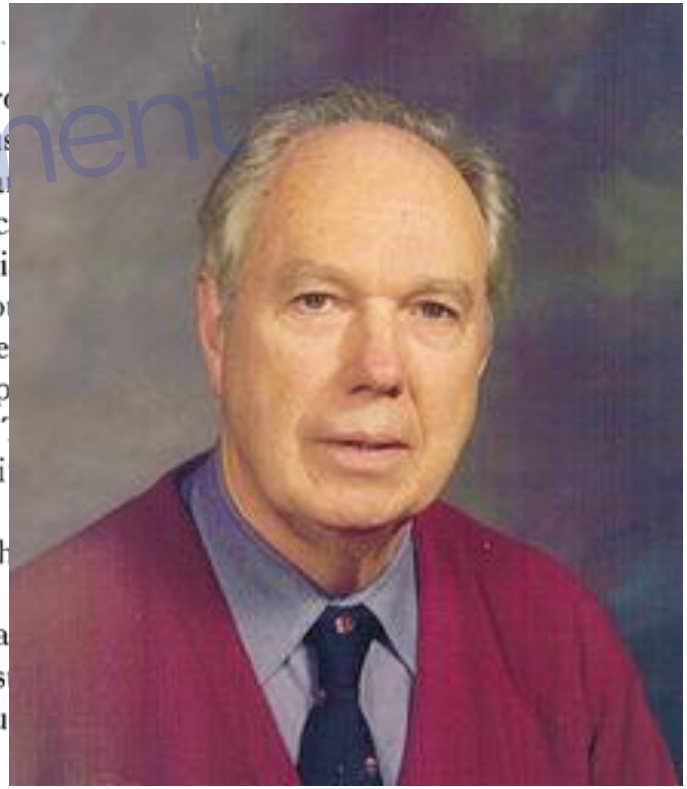
The parallel processors [single-instruction stream–multiple-data stream (SIMD)] are analyzed. In particular, a nesting type explanation is offered for Minsky's conjecture—the performance of a parallel processor increases as log M instead of M (the number of data stream processors).

Multiprocessors (MIMD) are subjected to a saturation syndrome based on general communications lockout. Simplified queuing models indicate that saturation develops when the fraction of task time spent locked out (L/E) approaches $1/n$, where n is the number of processors. Resources sharing in multiprocessors can be used to avoid

more "macr
particular us
must be sha
more signific

1) There i
limiting reso
est will eithe
tions will ap
figurations.
computer wi
cerned with
potential, wh
siderations.

2) We ma
sets. It is ass
set of instru



pdfelement

Remove Watermark Now

Taxonomia Flynn

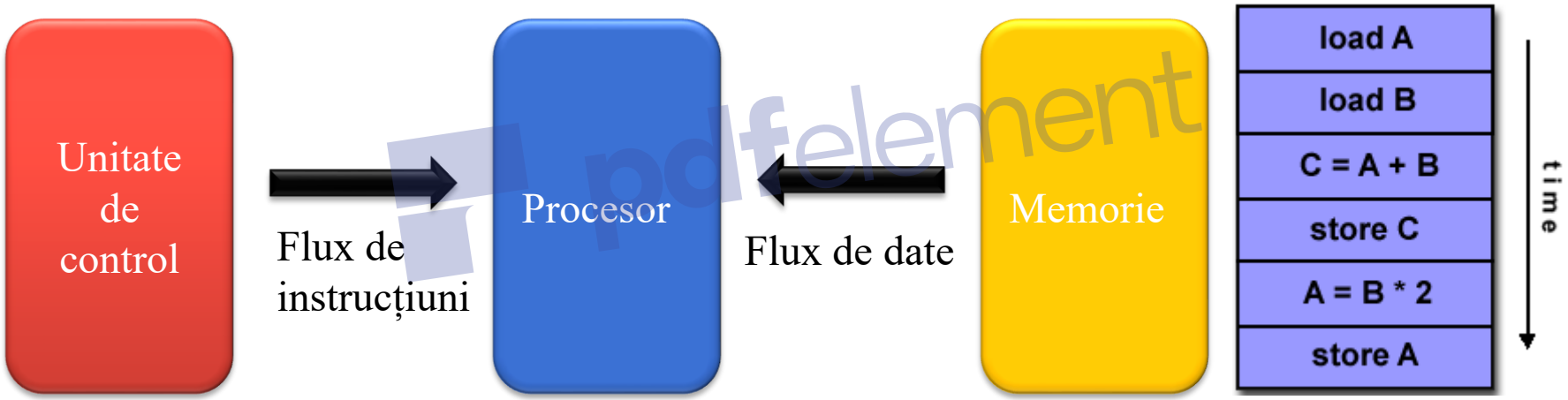
- SISD
  - Single Instruction Stream, Single Data Stream
  - Calculatorul Clasic one-core
- SIMD
  - Single Instruction Stream, Multiple Data Streams
  - Suportul SSE; procesoare GPU
- MISD
  - Multiple Instruction Streams, Single Data Stream
  - Sisteme specializate
- MIMD
  - Multiple Instruction Streams, Multiple Data Streams
  - Procesoare actuale (ce aveți acasă și în buzunar)

pdfelement

Remove Watermark Now

SISD

- Model clasic Arhitectura *von Neumann*



pdfelement

Remove Watermark Now

SISD

- Model clasic Arhitectura *von Neumann*

First Draft of a Report
on the EDVAC

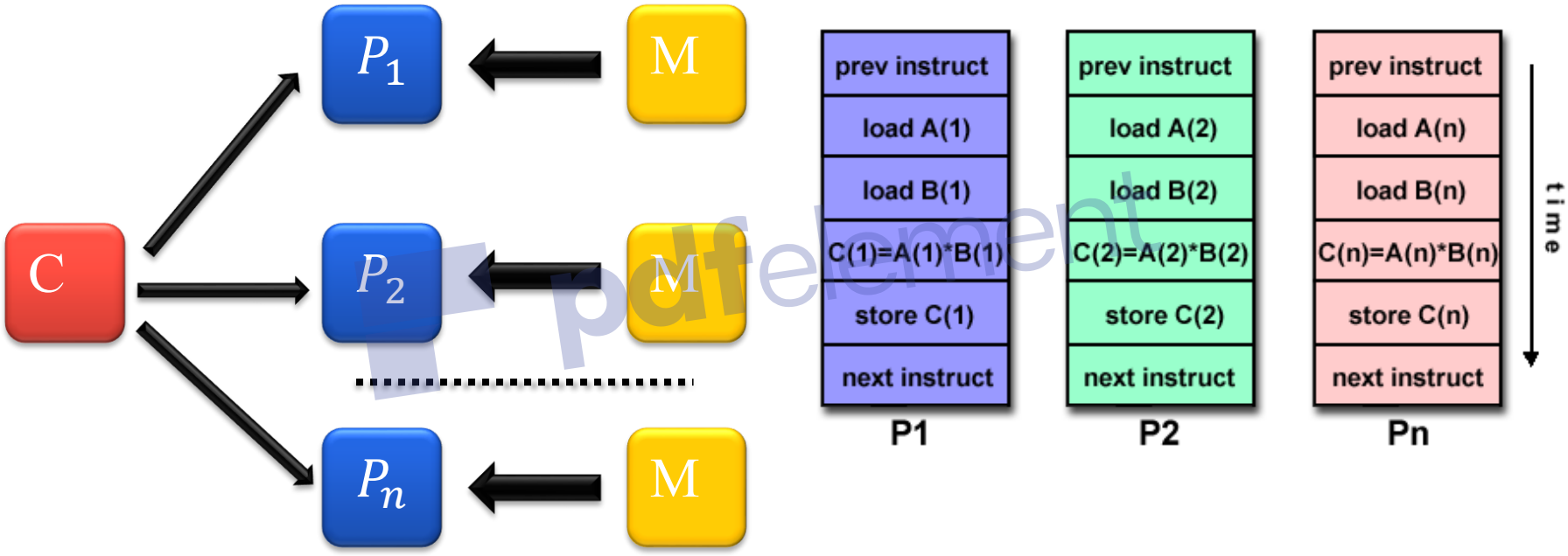
by

John von Neumann

pdfelement

Remove Watermark Now

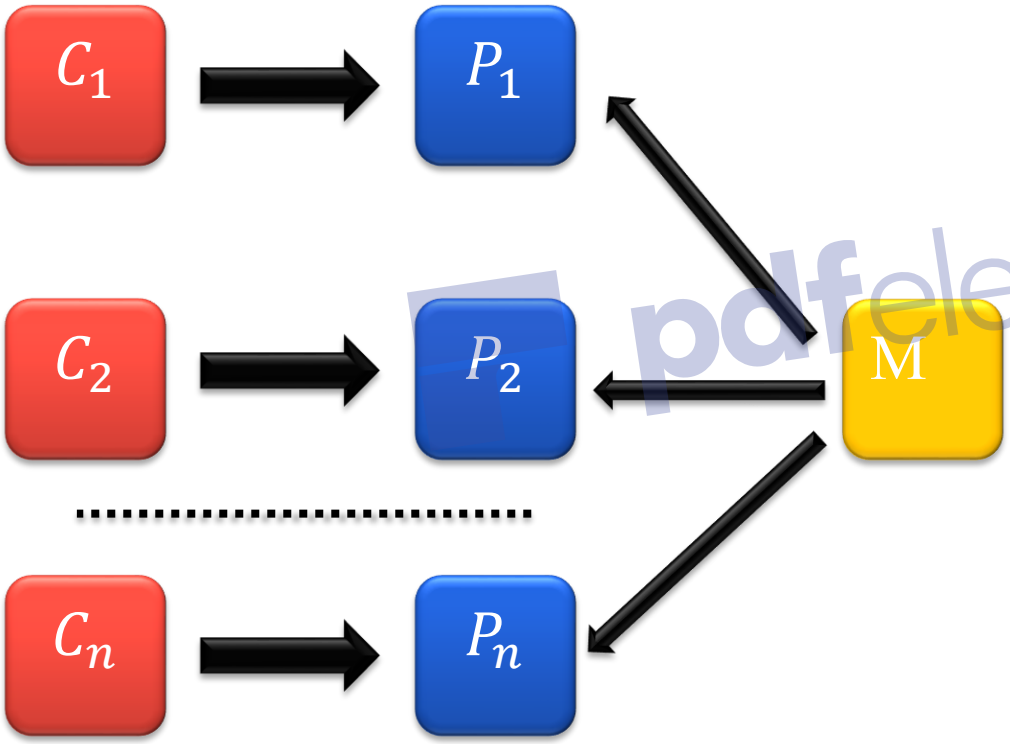
SIMD

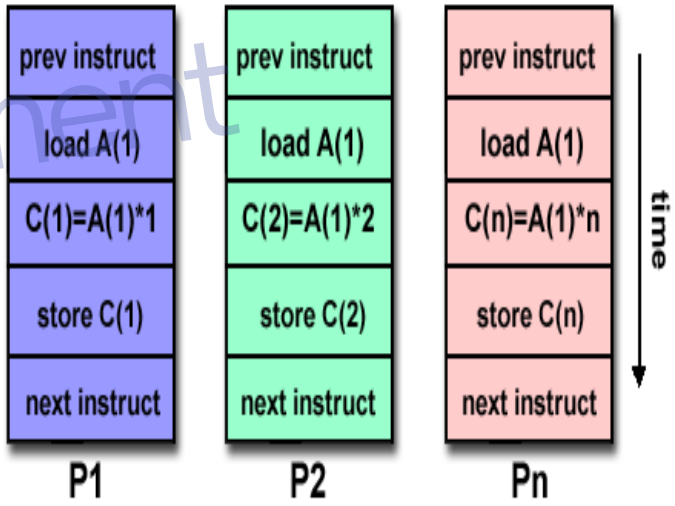


pdfelement

Remove Watermark Now

MISD

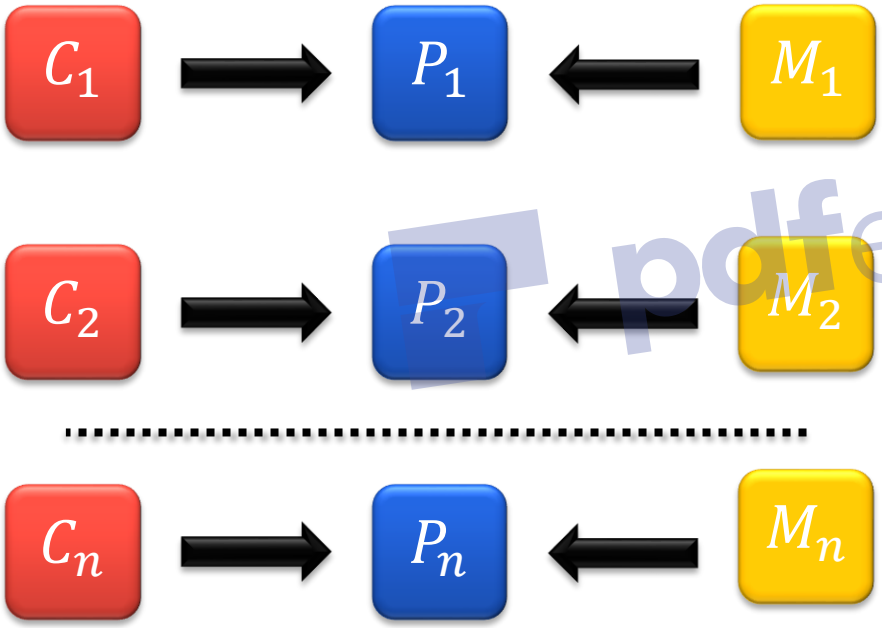


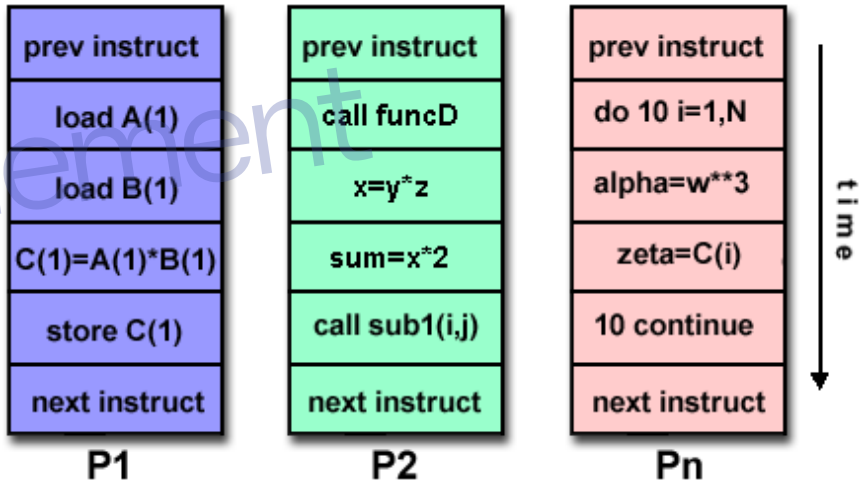


pdfelement

Remove Watermark Now

MIMD





pdfelement

Remove Watermark Now

Memorie Partajată

- Shared memory
  - Parallel Random Access Memory - **PRAM**
  - **EREW** – Exclusive Read Exclusive Write
  - **CREW** – **Concurrent Read Exclusive Write --** cel mai des întâlnit
  - **ERCW** – Exclusive Read Concurrent Write
  - **CRCW** – Concurrent Read Concurrent Write
  - O variabilă poate fi citită într-un pas în model **CR** dar în log(N) în model **ER**.

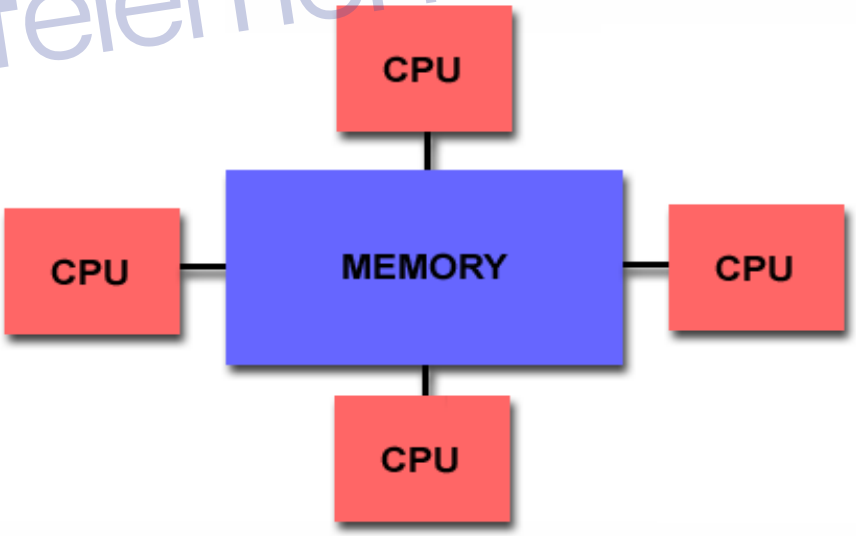
pdfelement

Remove Watermark Now

Memorie Partajată

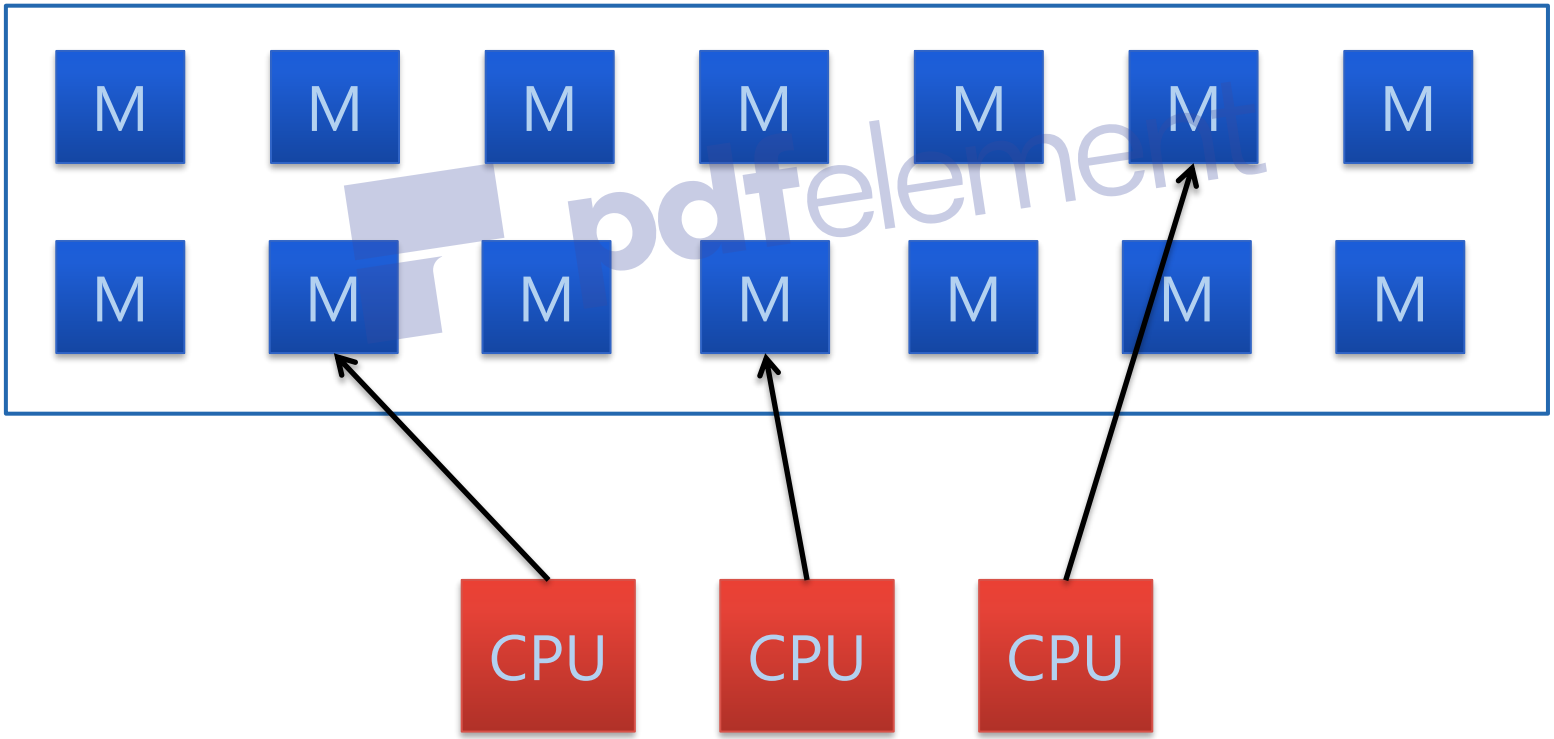
- Uniform Memory Access
  - **UMA**

pdfelement



Remove Watermark Now

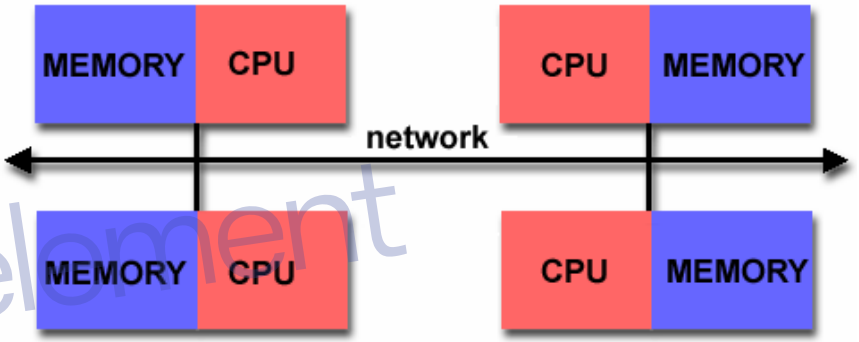
Memorie Partajată - Acces



Remove Watermark Now

Memorie Distribuită

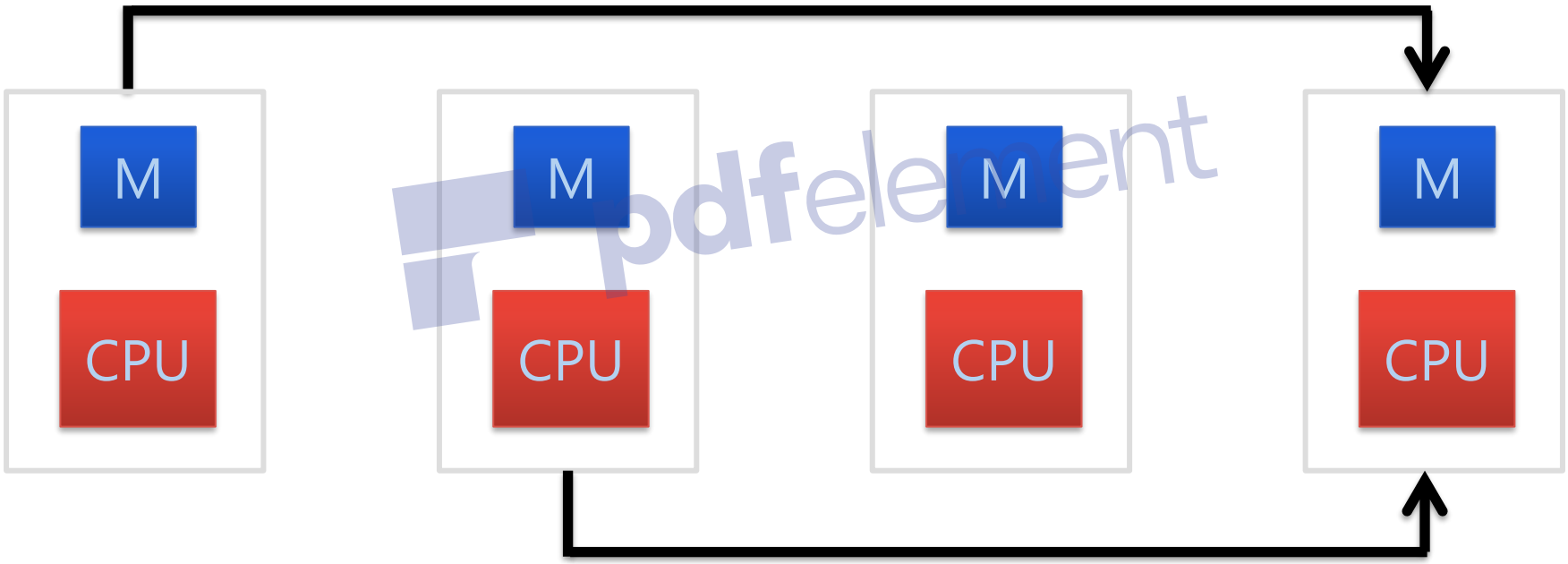
- Massively Parallel Processors
- Network of workstations
- Non-Uniform Memory Access
  - **NUMA**



pdfelement

Remove Watermark Now

Memorie Distribuită - Acces

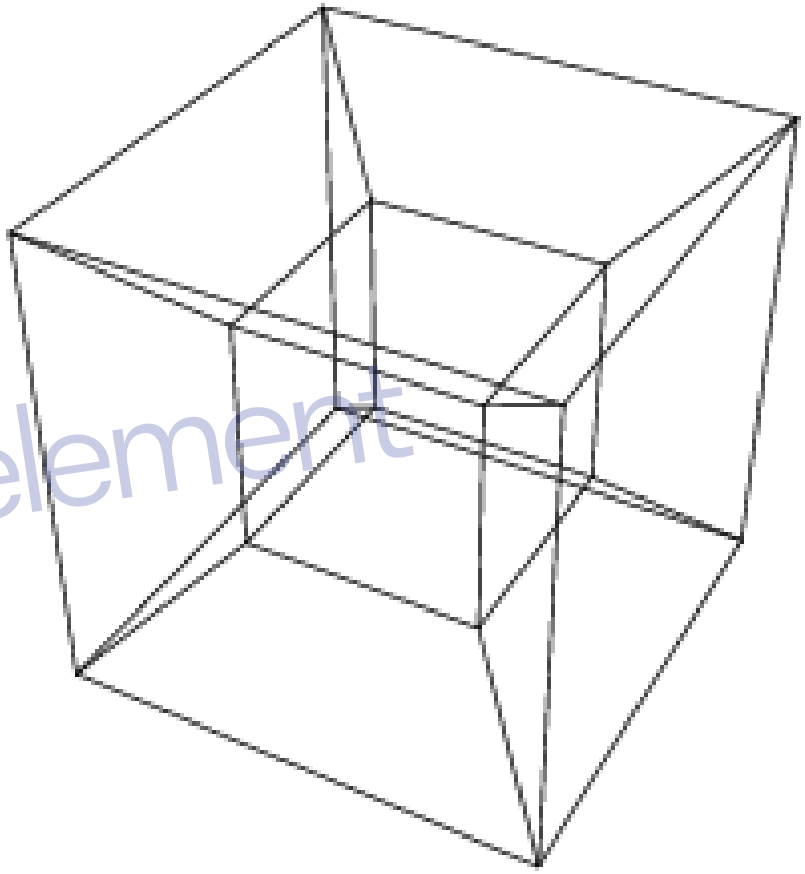


pdfelement

Remove Watermark Now

Rețele de configurare

- Topologii
  - Tablou
  - Arbore
  - Cub
  - Hipercub



pdfelement

- Depinde de
  - Aplicație
  - Performanțe dorite
  - Număr procesoare disponibile
- Exemple: IBM 9000, Cray C90, Fujitsu VP

Remove Watermark Now

Tipuri de paralelism

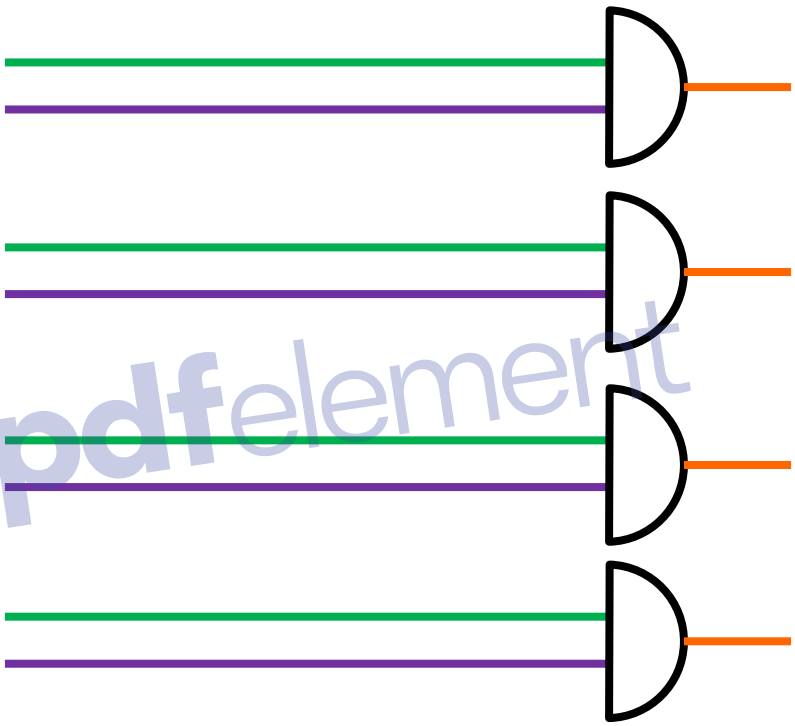
- La nivel de bit
- La nivel de instrucțiune
- La nivel de task

pdfelement

Remove Watermark Now

Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task

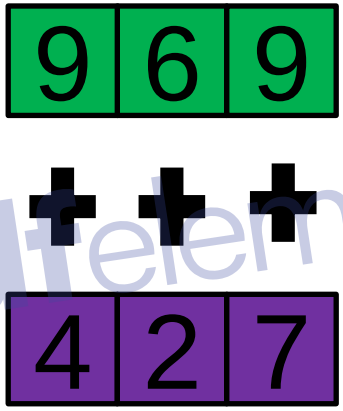


pdfelement

Remove Watermark Now

Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task



C = A + B

Adunarea a doi vectori

pdfelement

Remove Watermark Now

Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task
  - **Multi-Tasking (pot comunica și procesele)**
  - **Multi-Threading**

pdfelement

Remove Watermark Now

Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task
  - **Multi-Tasking (pot comunica și procesele)**
  - **Multi-Threading**

Cum pot comunica două procese?

pdfelement

Remove Watermark Now

Notații pseudocod

```
co S1 || S2 || ... || Sn oc
```

Ex.1:

```
x=0; y=0;

co x=x+1 || y=y+1 oc

z=x+y;
```

pdfelement

Remove Watermark Now

Notații pseudocod

```
co [cuantificator]{Sj}
```

Ex. 2:

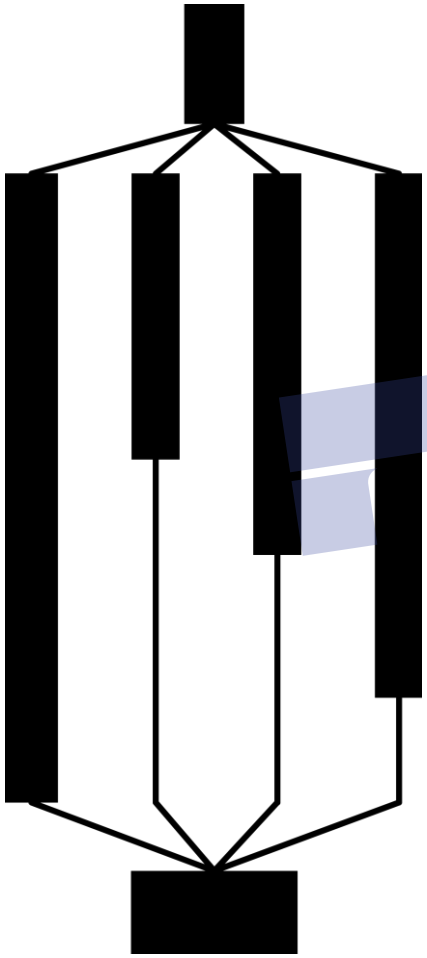
```
co [j=1 to n] {a[j]=0; b[j]=0;}
```

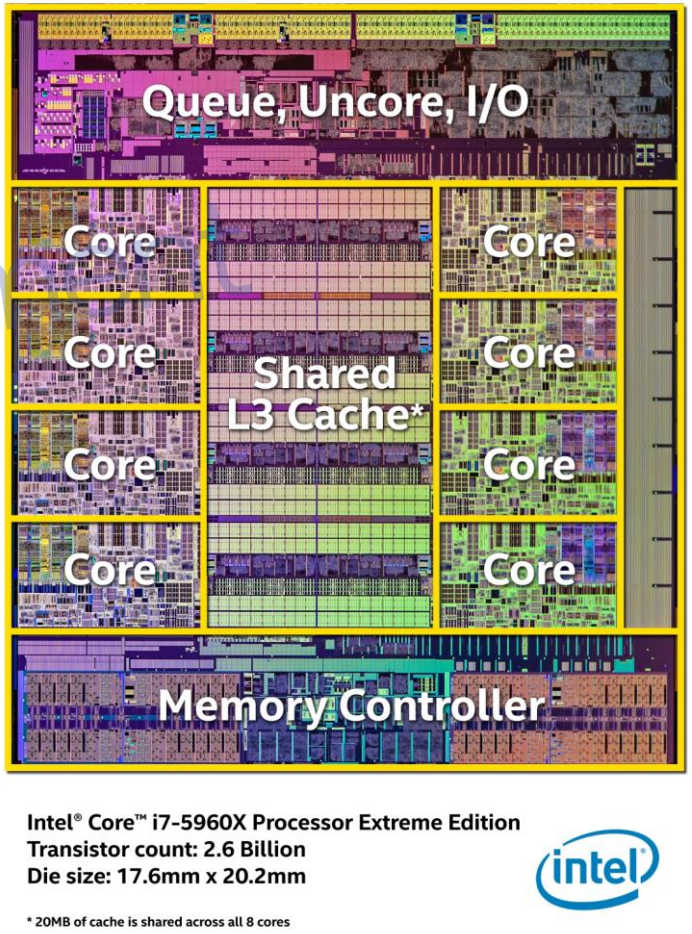
pdfelement

```
process Name[cuatificatori] { Sj }
```

Remove Watermark Now

Threads vs cores



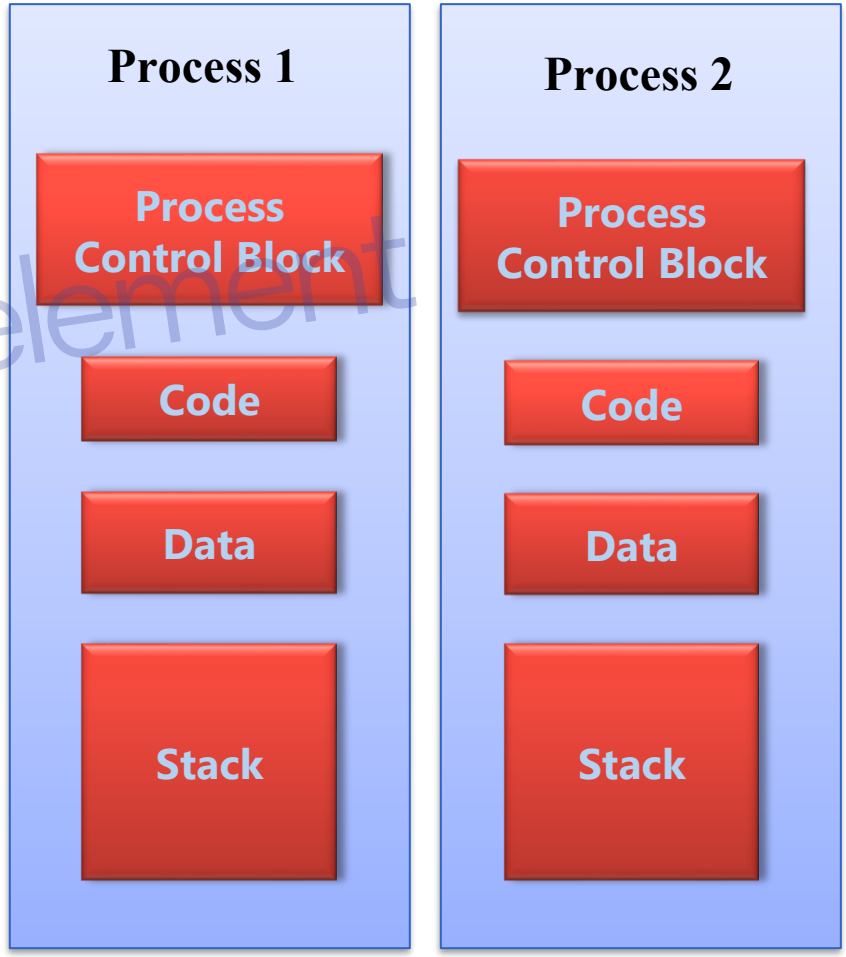


pdfelement

Remove Watermark Now

Procese și thread-uri

- Proces
  - Instanță a unui program în execuție
  - Pentru un proces, SO alocă:
    - Un spațiu în memorie (codul programului, zona de date, stiva)
    - Controlul anumitor resurse (fișiere, dispozitive I/O, …)

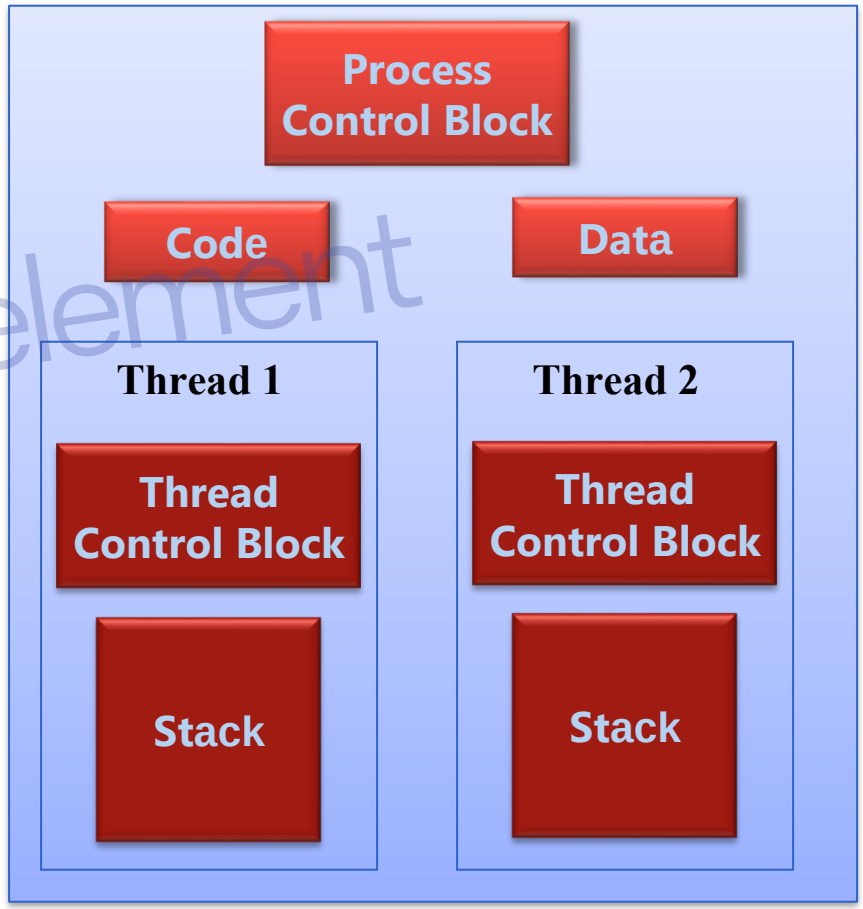


pdfelement

Remove Watermark Now

Thread (fir de execuție)

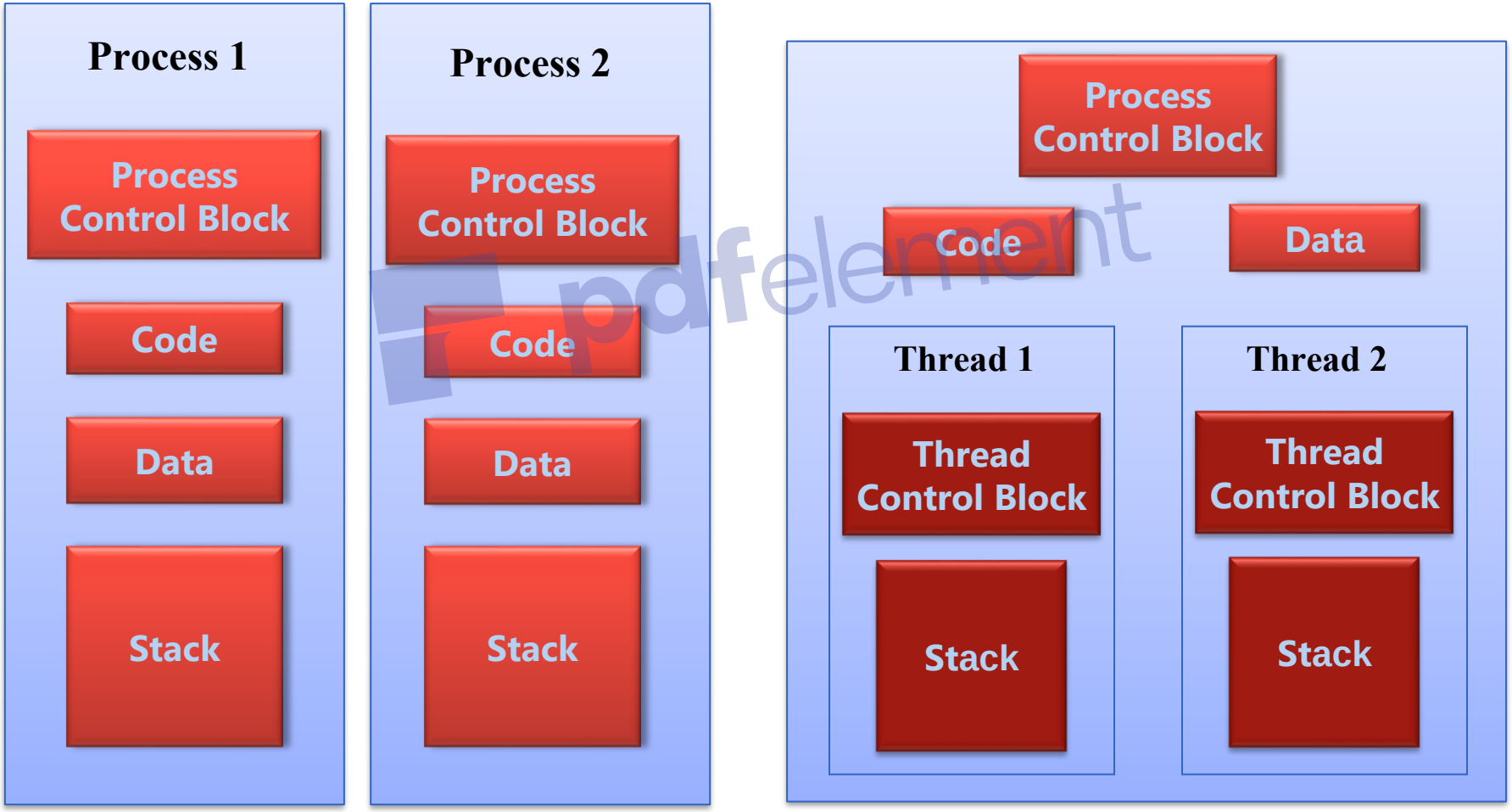
- Un proces poate avea mai multe fire de execuție
- Fir de execuție (thread):
  - Flux de control secvențial în cadrul unui proces
- Firele de execuție
  - împart același spațiu de adrese;
  - au în comun: zonele de cod și date din memorie, resursele procesului
  - zona de stivă – reprezintă starea curentă a thread-ului și este proprie thread-ului



pdfelement

Remove Watermark Now

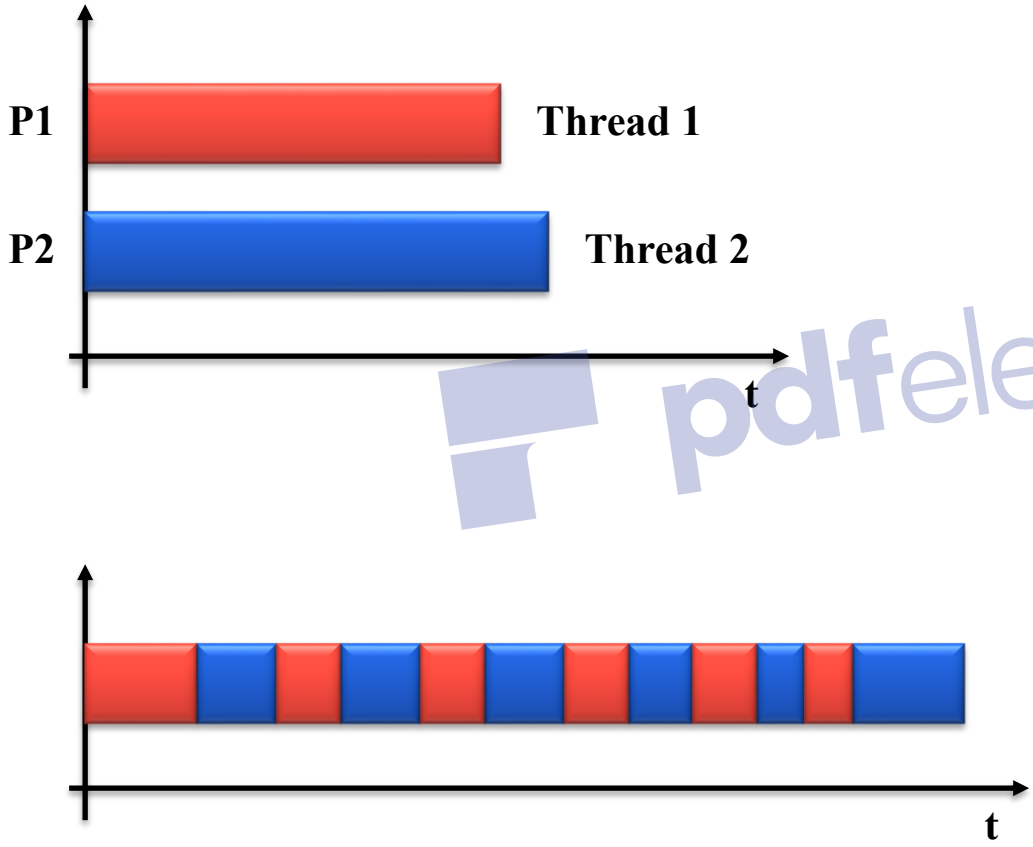
Multi-proces vs. multi-thread



pdfelement

Remove Watermark Now

Execuția thread-urilor



→ Putem considera execuție în paralel

Defapt:

→ planificarea este realizată de către sistemul de operare

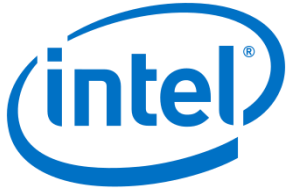
→ politici: divizarea timpului (*time sharing*), priorități

pdfelement

Remove Watermark Now

Hyperthreading – the confusion

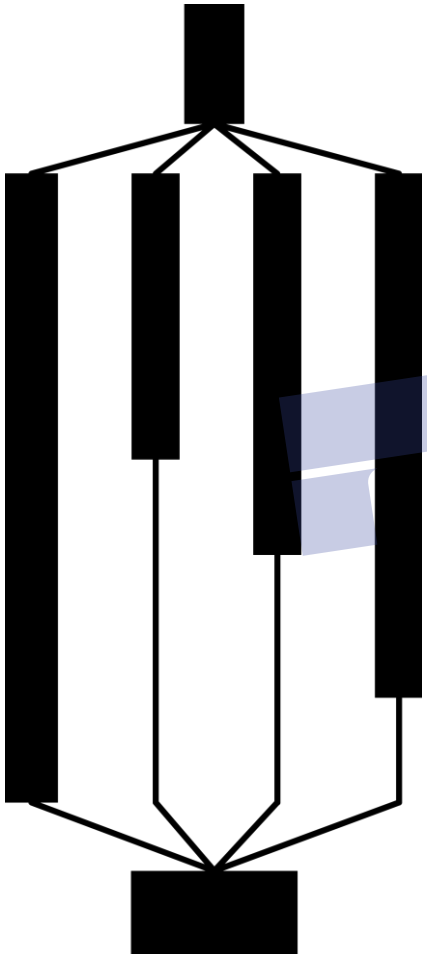
Thank you

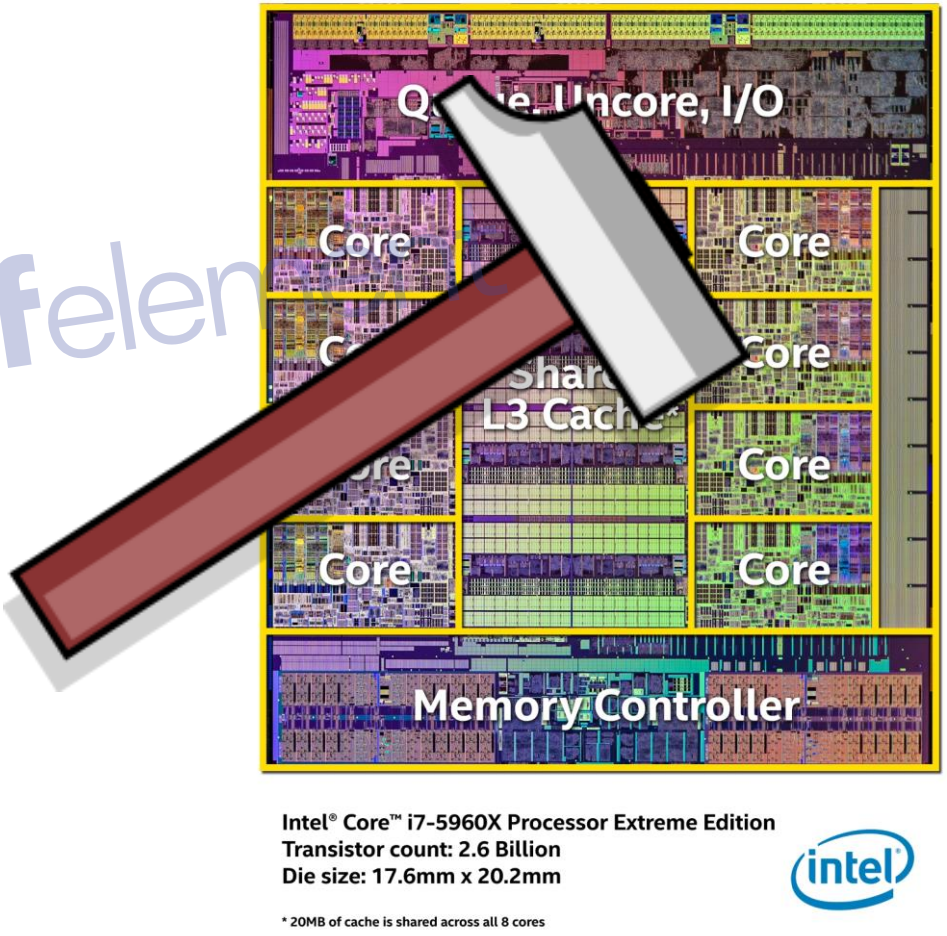


pdfelement

Remove Watermark Now

Hyperthreading – the confusion





Remove Watermark Now

Multi-tasking vs Multi-threading

pdfelement

Remove Watermark Now

Multi-tasking

Task 1

Task 2

pdfelement

Remove Watermark Now

Multi-tasking

Task 2

pdfelement

Task 1

Remove Watermark Now

Multi-tasking

Task 1

pdfelement

Task 2

Remove Watermark Now

Multi-tasking

Task 2

pdfelement

Task 1

Remove Watermark Now

Multi-threading

Thread 2

Thread 3

Thread 4

pdfelement

Thread 1

Remove Watermark Now

Multi-threading

Thread 1

Thread 2

Thread 4

Thread 3

pdfelement

Remove Watermark Now

Multi-threading

Thread 1

Thread 2

pdfelement

Thread 4 **Thread 3**

Remove Watermark Now

Multi-threading

Thread 2

Thread 3

Thread 4

pdfelement

Thread 1

Remove Watermark Now

Multi-threading

Thread 1

Thread 4

Thread 2 Thread 3

pdfelement

Remove Watermark Now

Multi-tasking vs Multi-threading

Care sunt diferențele?

- Un proces are mai multe thread-uri
- Thread-urile unui proces împart memoria

pdfelement

Remove Watermark Now

Multi-tasking vs Multi-threading

Poți avea multi-tasking pe mai multe core-uri?

Poți avea mai multe thread-uri pe un singur core?

pdfelement

Remove Watermark Now

Multi-tasking vs Multi-threading

Poți avea multi-tasking pe mai multe core-uri? **DA**

Poți avea mai multe thread-uri pe un singur core? **DA**

pdfelement

Remove Watermark Now

Notiuni cu care vom lucra

pdfelement

Remove Watermark Now

Race condition

Thread 1	Thread 2
a = a + 2	**a = a + 2**

pdfelement

Care este valoare lui a în acest moment?

Remove Watermark Now

Race condition

a = 0

Thread 1	Thread 2
a = a + 2	**a = a + 2**

pdfelement

Care este valoare lui a în acest moment?

Remove Watermark Now

Race condition

a = 0

Thread 1	Thread 2
a = a + 2	**a = a + 2**

pdfelement

What is the value of a?

4

Remove Watermark Now

Race condition

a = 0

Thread 1	Thread 2
a = a + 2	**a = a + 2**

pdfelement

What is the value of a?

4 **AND** 2

Remove Watermark Now

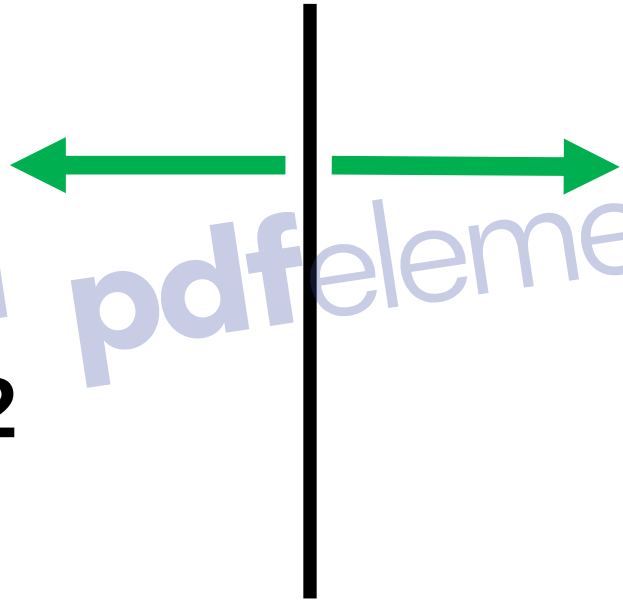
Un alt exemplu

pdfelement

Remove Watermark Now

Race condition

a = 0



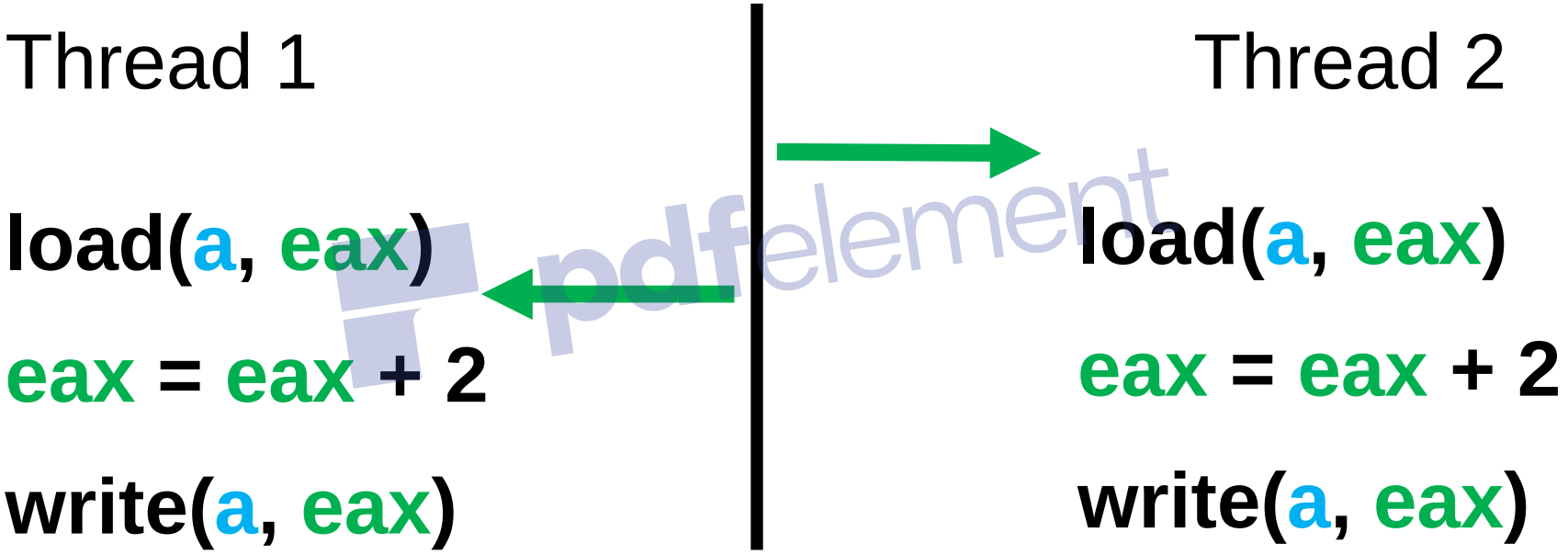
Thread 1	Thread 2
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
eax =	eax =

pdfelement

Remove Watermark Now

Race condition

a = 0



pdfelement

eax = 0

eax =

Remove Watermark Now

Race condition

a = 0

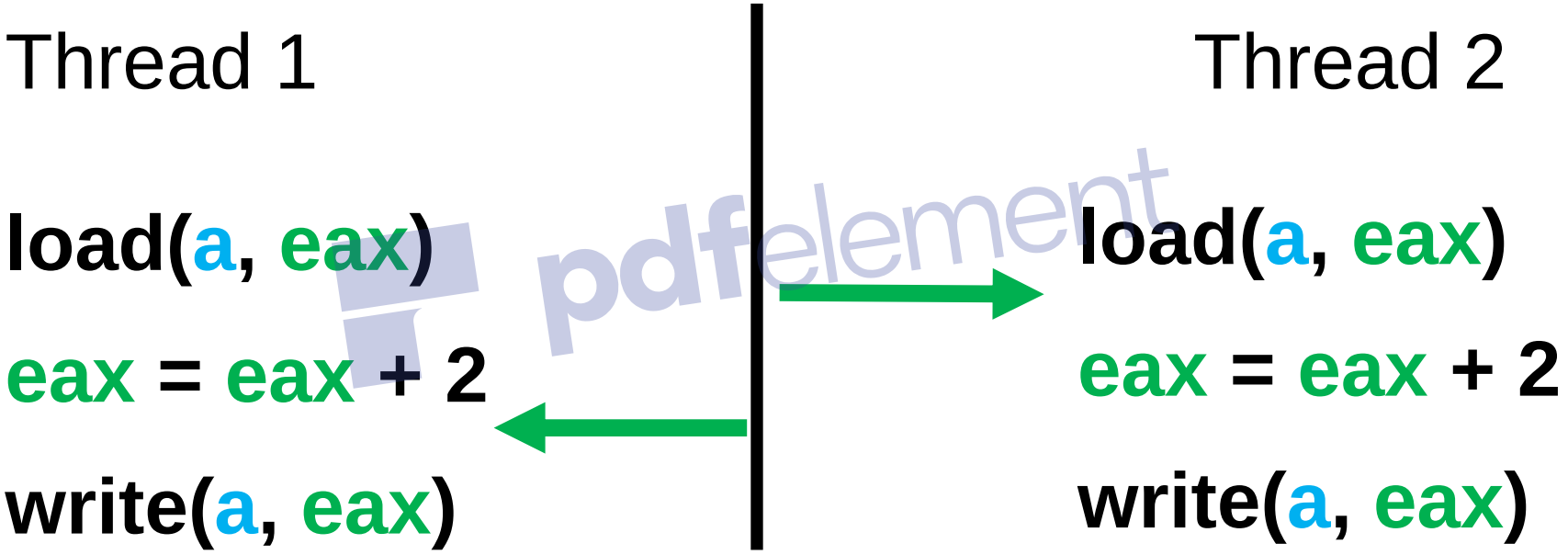
Thread 1	Thread 2
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
eax = 0	eax = 0

pdfelement

Remove Watermark Now

Race condition

a = 0



pdfelement

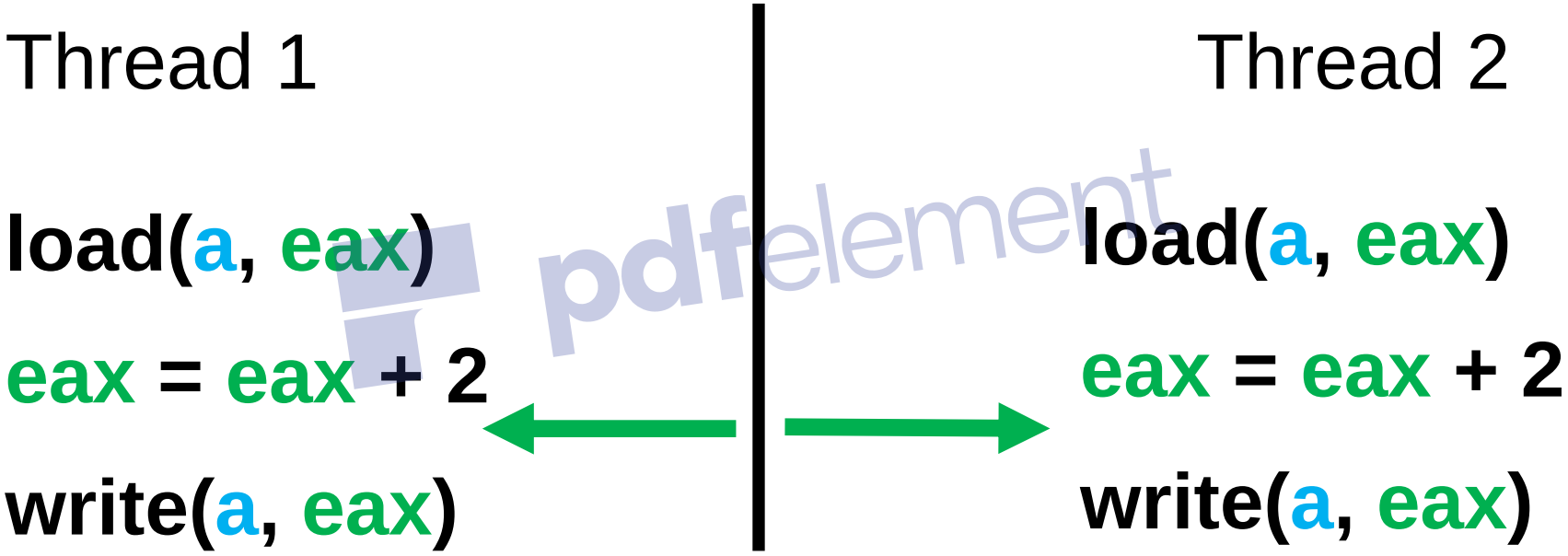
eax = 2

eax = 0

Remove Watermark Now

Race condition

a = 0



pdfelement

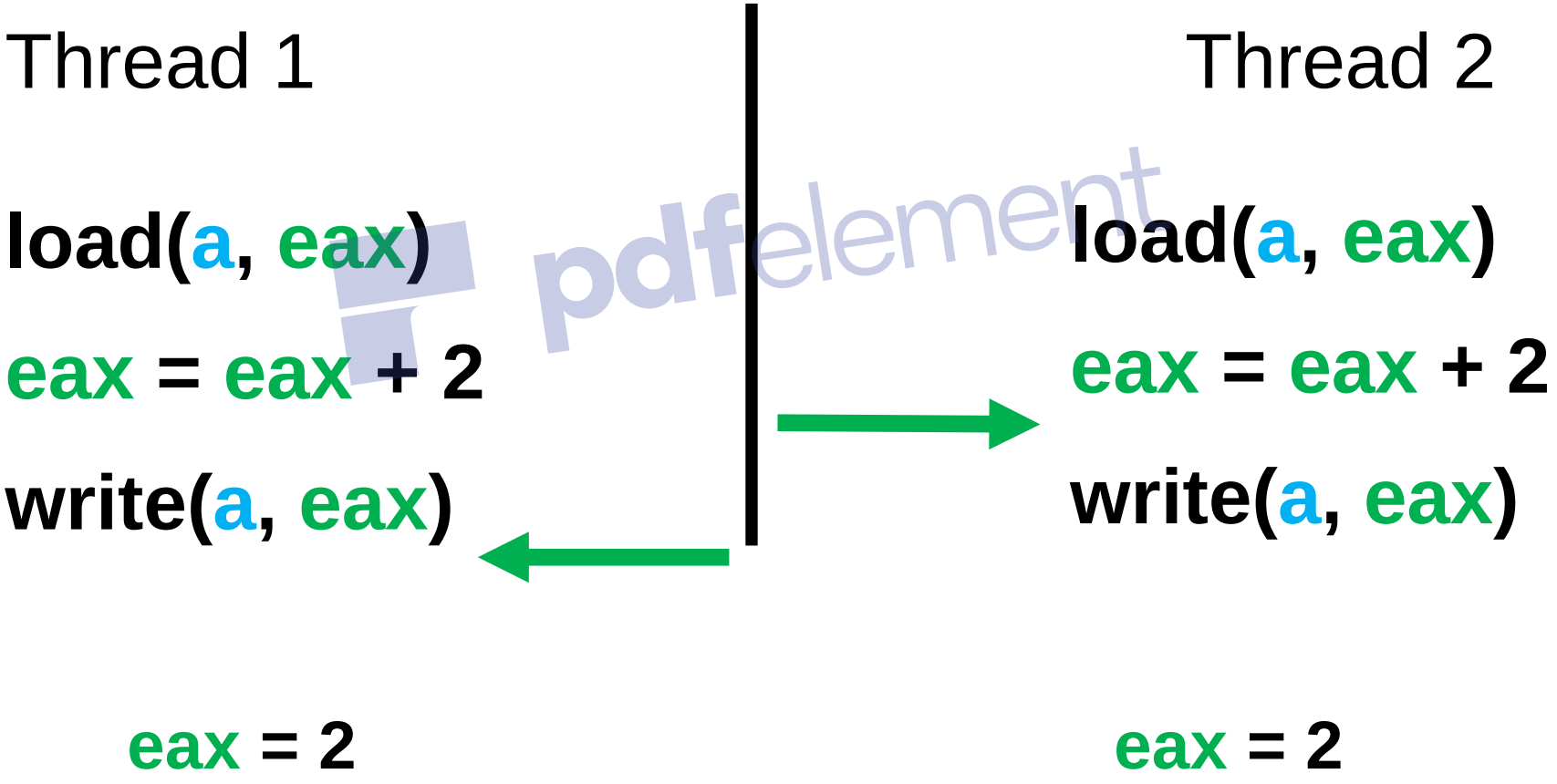
eax = 2

eax = 2

Remove Watermark Now

Race condition

a = 2



pdfelement

Remove Watermark Now

Race condition

a = 2

Thread 1	Thread 2
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
eax = 2	eax = 2

pdfelement

Remove Watermark Now

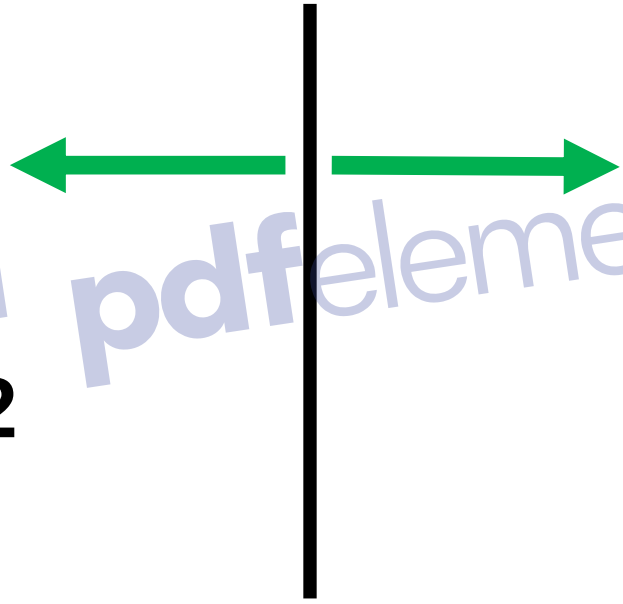
Un pas si mai departe...

pdfelement

Remove Watermark Now

Race condition

a = 0



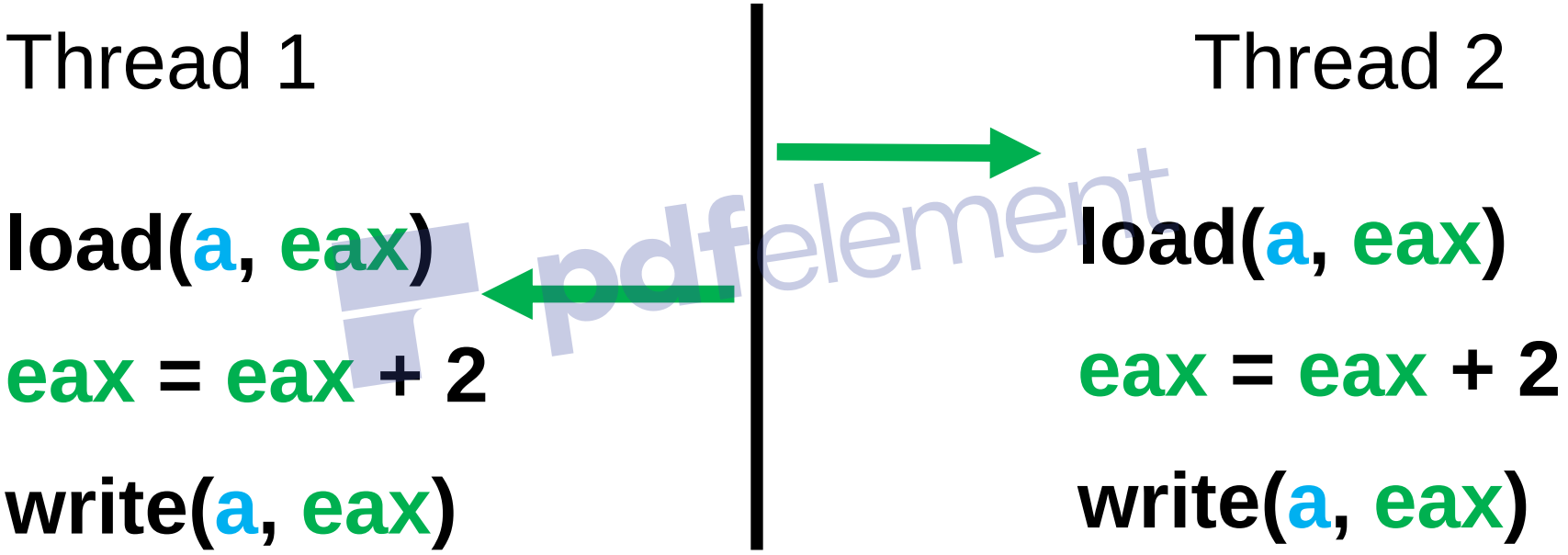
Thread 1	Thread 2
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
eax =	eax =

pdfelement

Remove Watermark Now

Race condition

a = 0



pdfelement

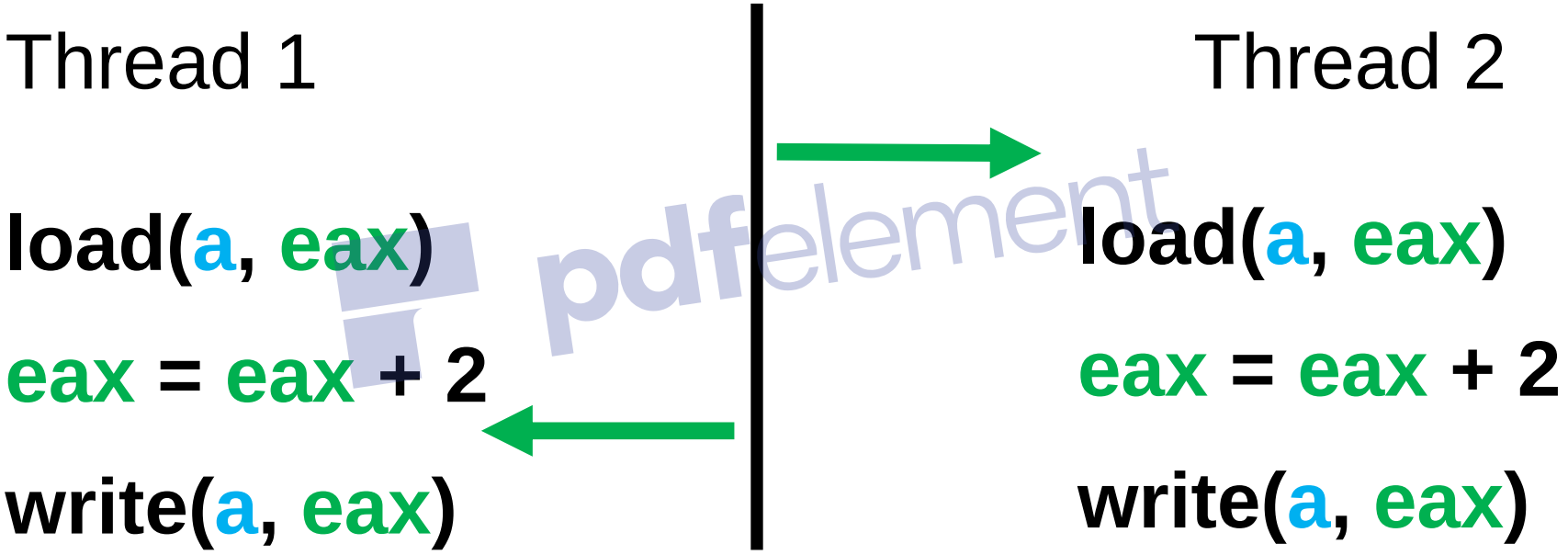
eax = 0

eax =

Remove Watermark Now

Race condition

a = 0



pdfelement

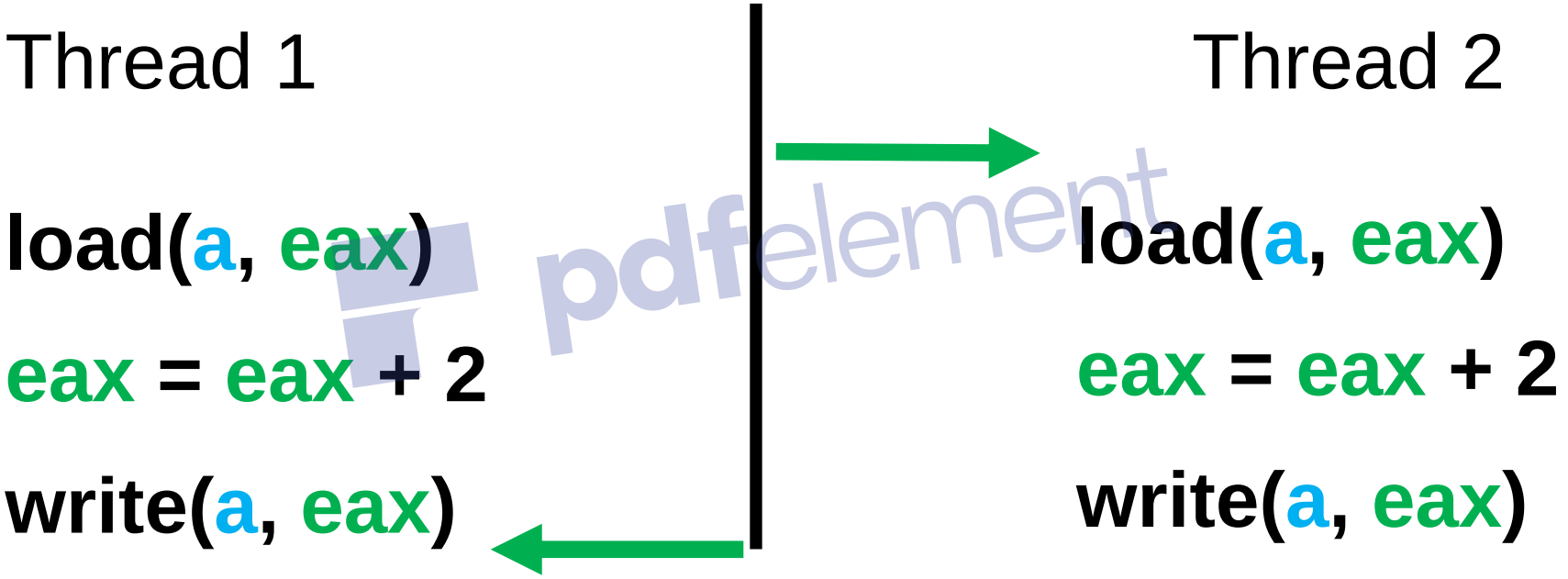
eax = 2

eax =

Remove Watermark Now

Race condition

a = 2



pdfelement

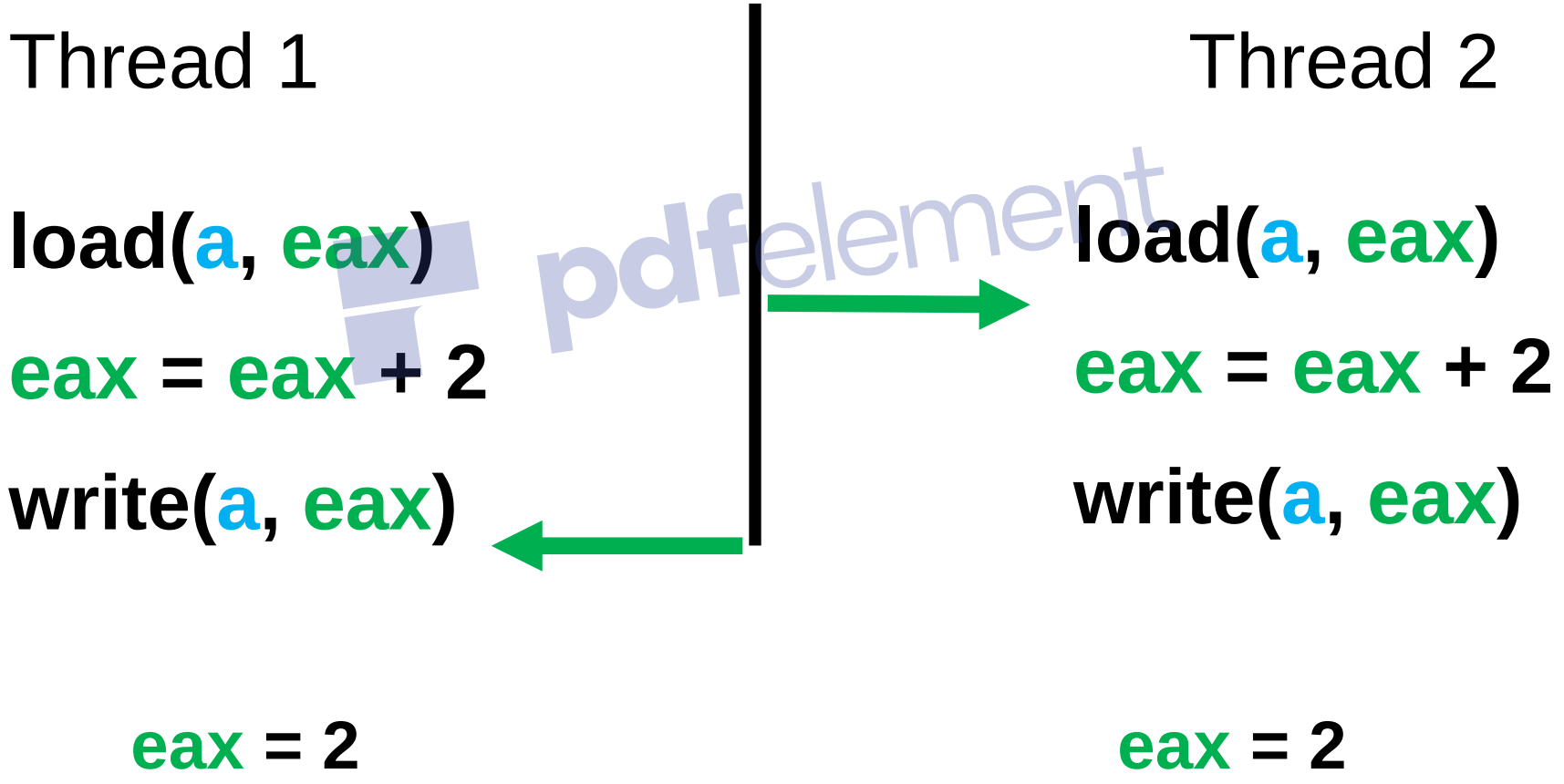
eax = 2

eax =

Remove Watermark Now

Race condition

a = 2

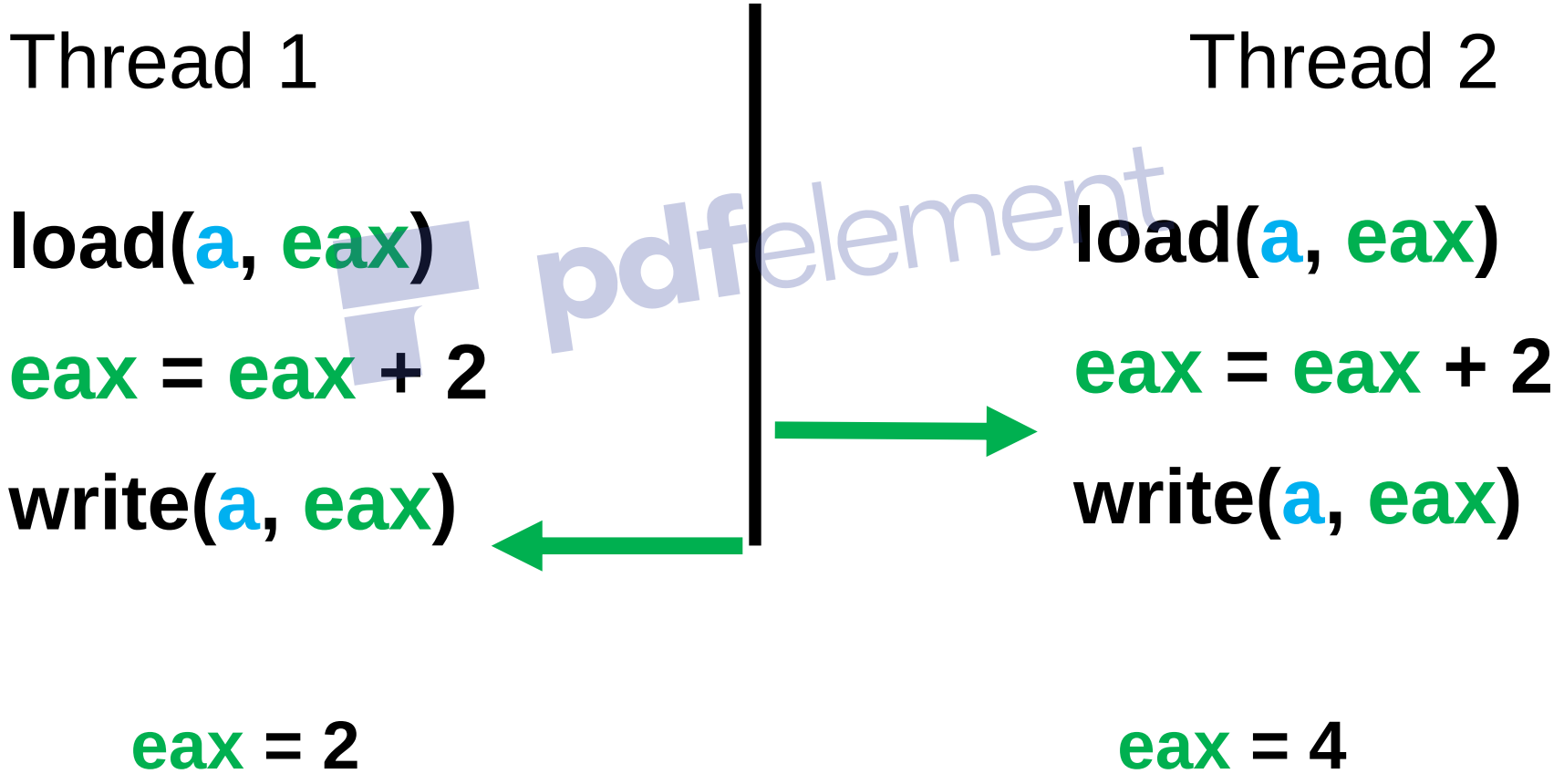


pdfelement

Remove Watermark Now

Race condition

a = 2

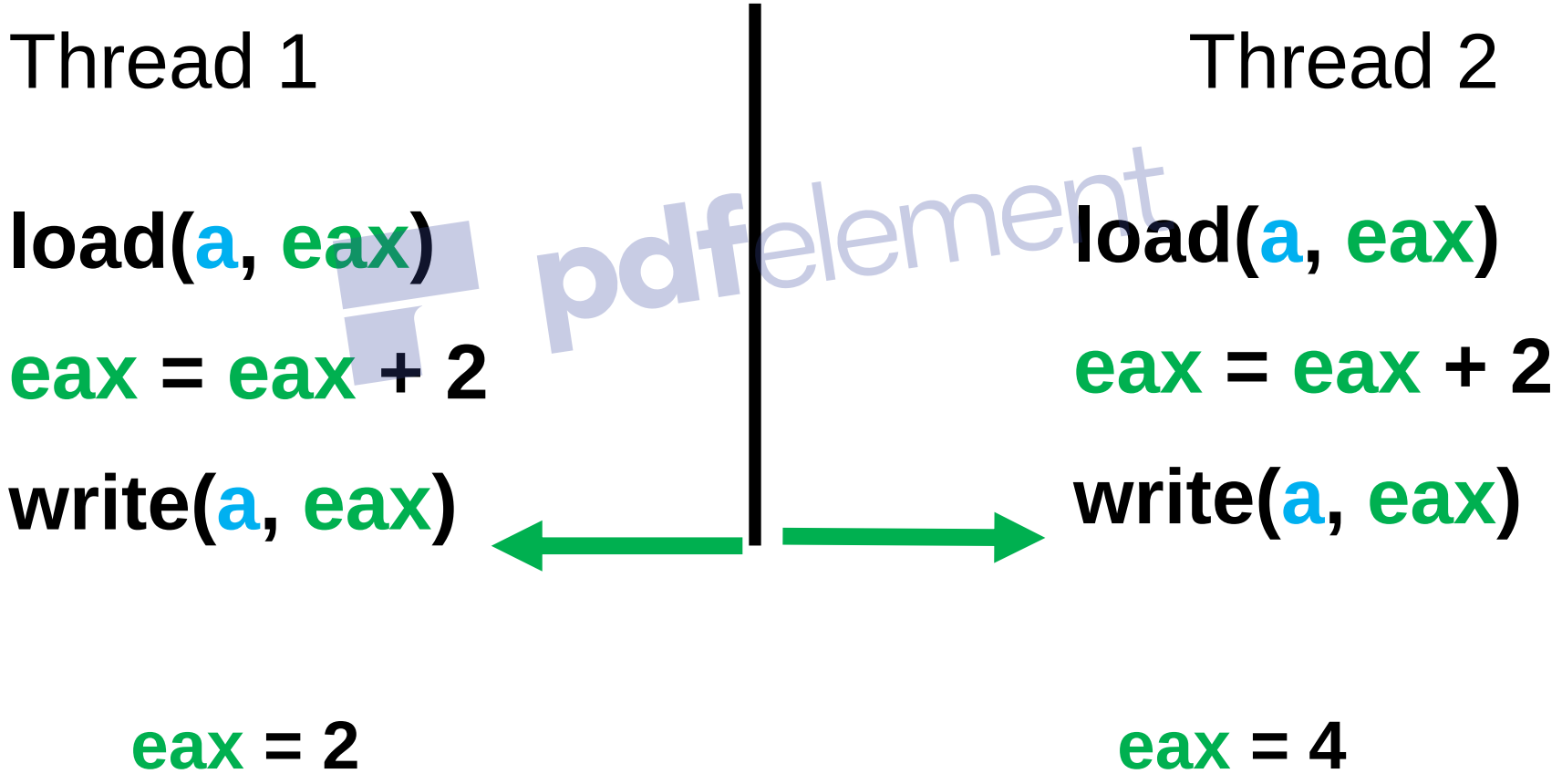


pdfelement

Remove Watermark Now

Race condition

a = 4



pdfelement

Remove Watermark Now

Primitive de sincronizare

pdfelement

Remove Watermark Now

Synchronization primitives

- Atomics
- Semaphore
  - Binary semaphore (Mutex)
  - Critical section
- Barrier

pdfelement

Remove Watermark Now

Atomics

- Fie operația de 64 biți pe un procesor de 64 biți **C = A + B**

```
load(A, eax)
load(B, ebx)
eax = eax + ebx
write(C, eax)
```

pdfelement

Remove Watermark Now

Atomics

- Fie operația de 64 biți pe un procesor de 32 biți **C = A + B**

```
load(A[0], eax)
load(B[0], ebx)
eax = eax + ebx
write(C[0], eax)
load(A[1], eax)
load(B[1], ebx)
eax = eax + ebx
write(C[1], eax)
```

pdfelement

Remove Watermark Now

Atomics

- Fie operația de 64 biți pe un procesor 32 biți **C = A + B**

```
load(A[0], eax)
load(B[0], ebx)
eax = eax + ebx
write(C[0], eax)
load(A[1], eax)
load(B[1], ebx)
eax = eax + ebx
write(C[1], eax)
```

Putem avea doar jumătate de **C** modificat

pdfelement

Remove Watermark Now

Atomics

- Fie operația de 64 biți pe un procesor 32 biți **C = A + B**

```
load(A[0], eax)
load(B[0], ebx)
eax = eax + ebx
write(C[0], eax)
load(A[1], eax)
load(B[1], ebx)
eax = eax + ebx
write(C[1], eax)
```

Atomicitatea asigură că **C** va fi vizibil doar complet modificat, sau complet nemodificat.

pdfelement

Remove Watermark Now

Excludere mutuală - mutex

pdfelement

Remove Watermark Now

Mutual exclusion – soluția lui Dekker

EWD35 – 1

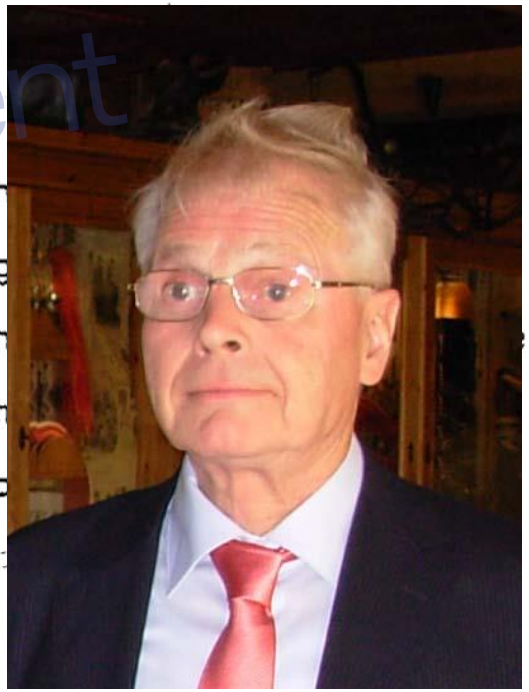
EWD35.htm

About the sequentiality of process descriptions.

Over de sequentialiteit van procesbeschrijvingen.

Het is niet ongebruikelijk, wanneer een spreker zijn
een inleiding. Omdat ik mij hier mogelijk richt tot een g
minder vertrouwd is met de problematiek, die ik wil aansn
die ik zal moeten gebruiken, wilde ik in dit geval ter in
inleidingen houden, nl. een om de achtergrond van de prob
een tweede, om U een gevoel te geven voor de aard van de
wij tegen het lijf zullen lopen.

pdfelement



Theodorus (Dirk) J. Dekker

Remove Watermark Now

Mutual exclusion – soluția lui Dijsktra

Solution of a Problem in Concurrent Programming Control

E. W. Dijkstra
Technological University, Eindhoven, The Netherlands

A number of mainly independent sequential-cyclic processes with restricted means of communication with each other can be made in such a way that at any moment one and only one of them is engaged in the "critical section" of its cycle.

Introduction

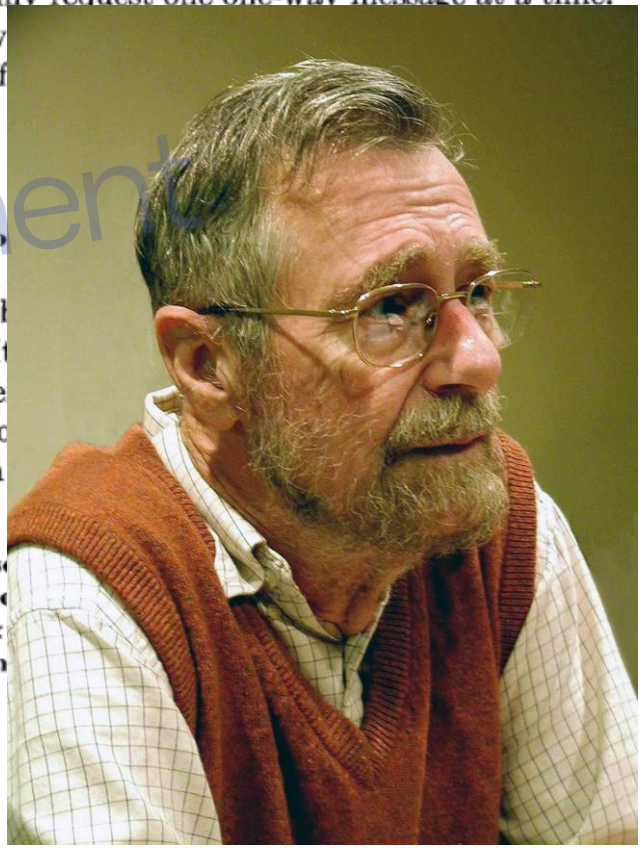
Given in this paper is a solution to a problem for which, to the knowledge of the author, has been an open question since at least 1962, irrespective of the solvability. The paper consists of three parts: the problem, the solution, and the proof. Although the setting of the problem might seem somewhat academic at first, the author trusts that anyone familiar with the logical problems that arise in

computer can only request one one-way message at a time. And only this w
this problem is f

The Solution

The common
"Boo
The integer
will only be set
by the others. It
well outside the
mentioned set to
The program

```
"integer j;
Li0:  b[i] := fals
Li1:  if k ≠ i the
Li2:  begin c[i] :
Li3:  if b[k] then
      go to Li1
      end
        else
```



pdfelement

Remove Watermark Now

Soluția lui Dekker

```
wants_to_enter[0] = true
while (wants_to_enter[1]) {
        if (turn != 0) {
                wants_to_enter[0] = false
                while (turn != 0) { // busy wait }
                wants_to_enter[0] = true
        }
}
// critical section ...
turn = 1
wants_to_enter[0] = false
```

pdfelement

Remove Watermark Now

Soluția lui Dijsktra

```
b[i] = fals
while(sw[i]) {
        sw[i] = F
        if (k!=i) {
                c[i] = true
                if(b[k])
                        k = i
                sw[i] = T
        } else {
        c[i] = false
        for(j=0;j<N;j++)
                if(j!=i && !c[j])
                        sw[i] = T
        }
 }
//critical
b[i] = true
c[i] = true
```

lock() - P

unlock() - V

pdfelement

flag[0] =
flag[0] =

Remove Watermark Now

Soluția lui Peterson

```
flag[0] = true;
P0_gate: turn = 1;
while (flag[1] && turn == 1) { // busy wait }
// critical section ...
flag[0] = false;
```

pdfelement

Remove Watermark Now

Soluție cu asistare hardware

```
while (test_and_set(lock));
// critical section
lock = 0
```

pdfelement

Remove Watermark Now

Race condition – soluție

a = 0

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 0	eax = 0

pdfelement

Remove Watermark Now

Race condition – soluție

?

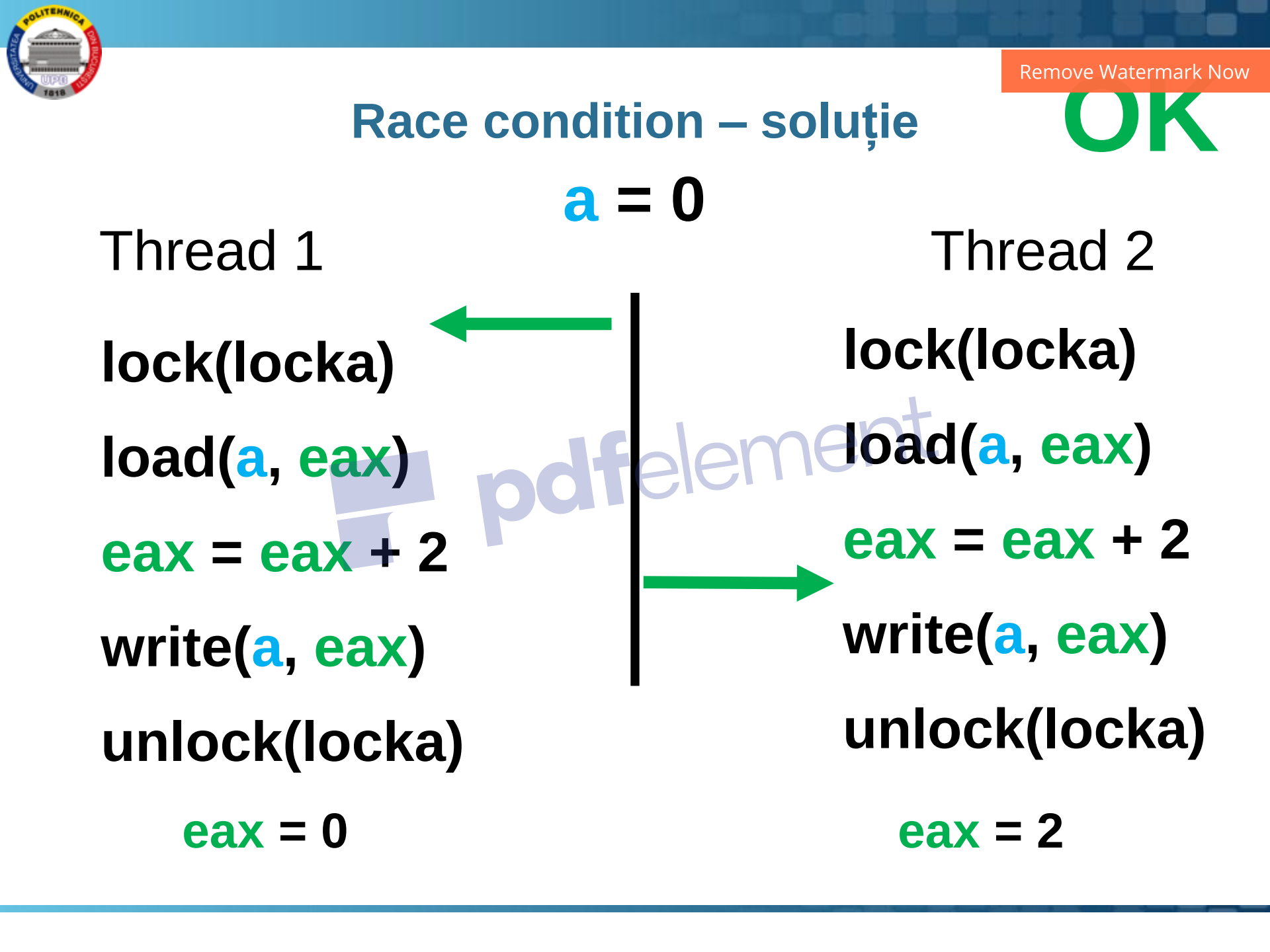
a = 0

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 0	eax = 2

pdfelement

Remove Watermark Now

Race condition – soluție



pdfelement

Remove Watermark Now

Race condition – soluție

?

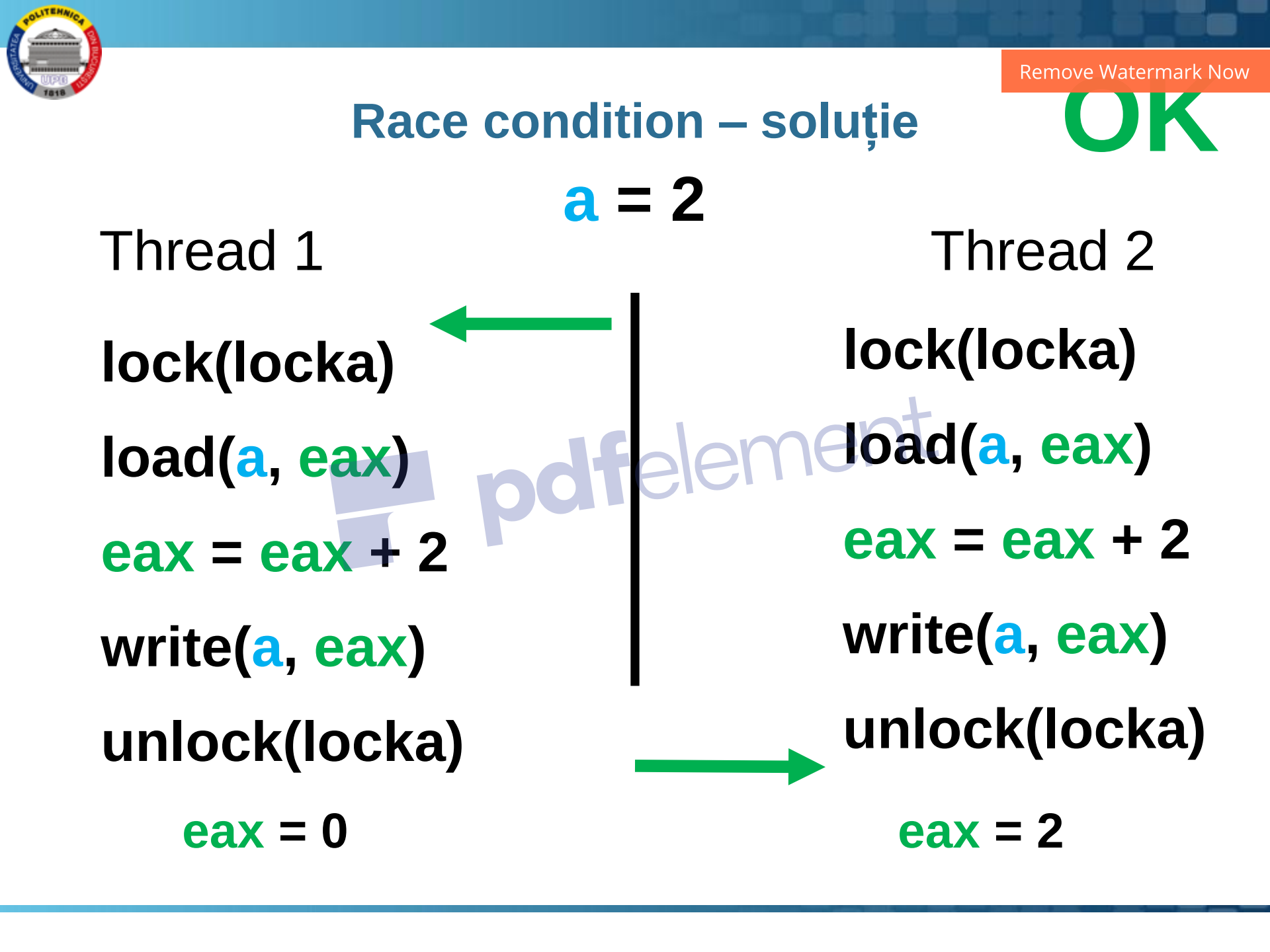
a = 2

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 0	eax = 2

pdfelement

Remove Watermark Now

Race condition – soluție



pdfelement

Remove Watermark Now

Race condition – soluție

?

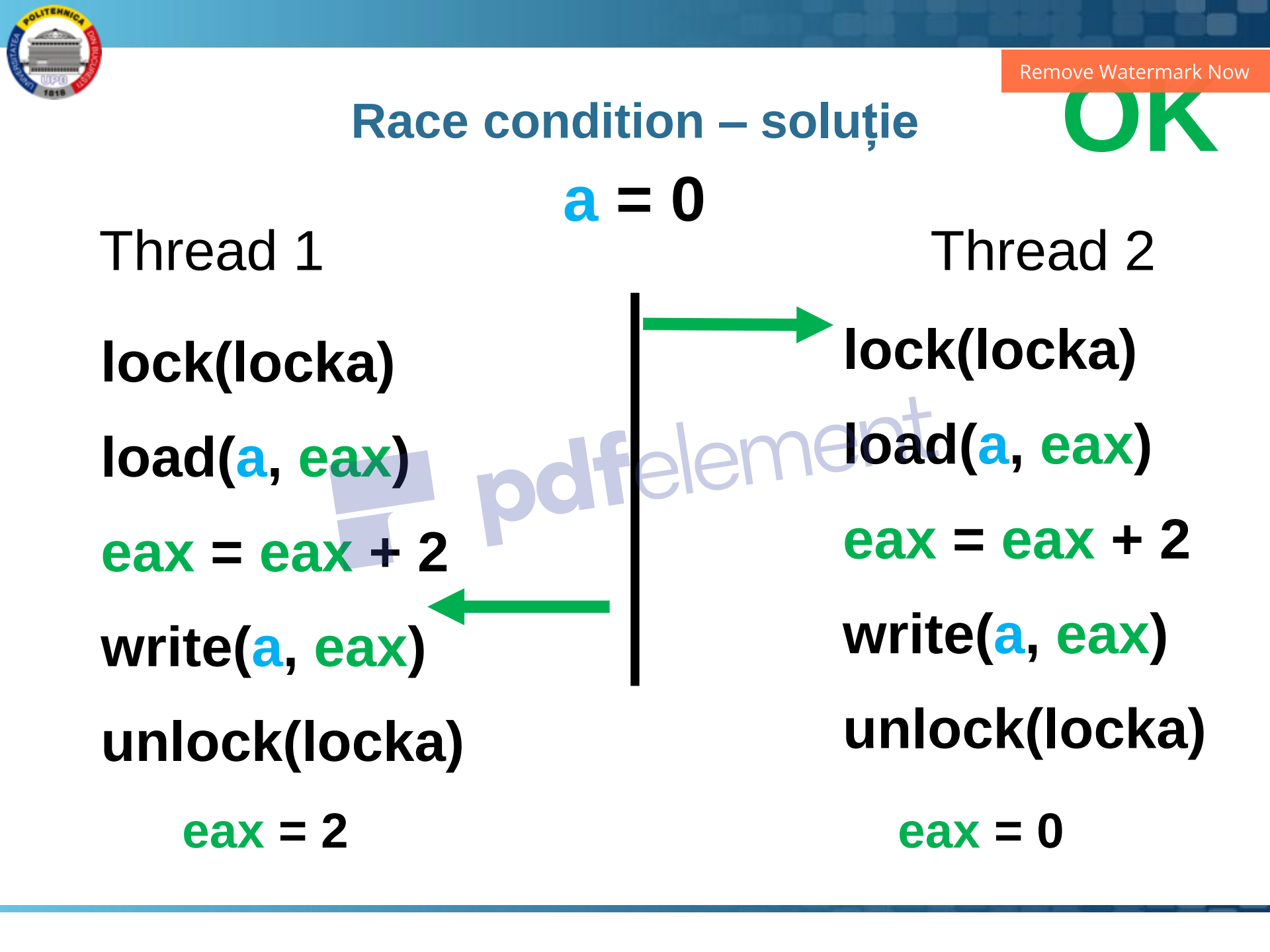
a = 0

Thread 1	Thread 2
lock(locka)	**lock(locka)**
load(a, eax)	**load(a, eax)**
eax = eax + 2	**eax = eax + 2**
write(a, eax)	**write(a, eax)**
unlock(locka)	**unlock(locka)**
eax = 2	**eax = 0**

pdfelement

Remove Watermark Now

Race condition – soluție



pdfelement

Remove Watermark Now

Race condition – soluție

?

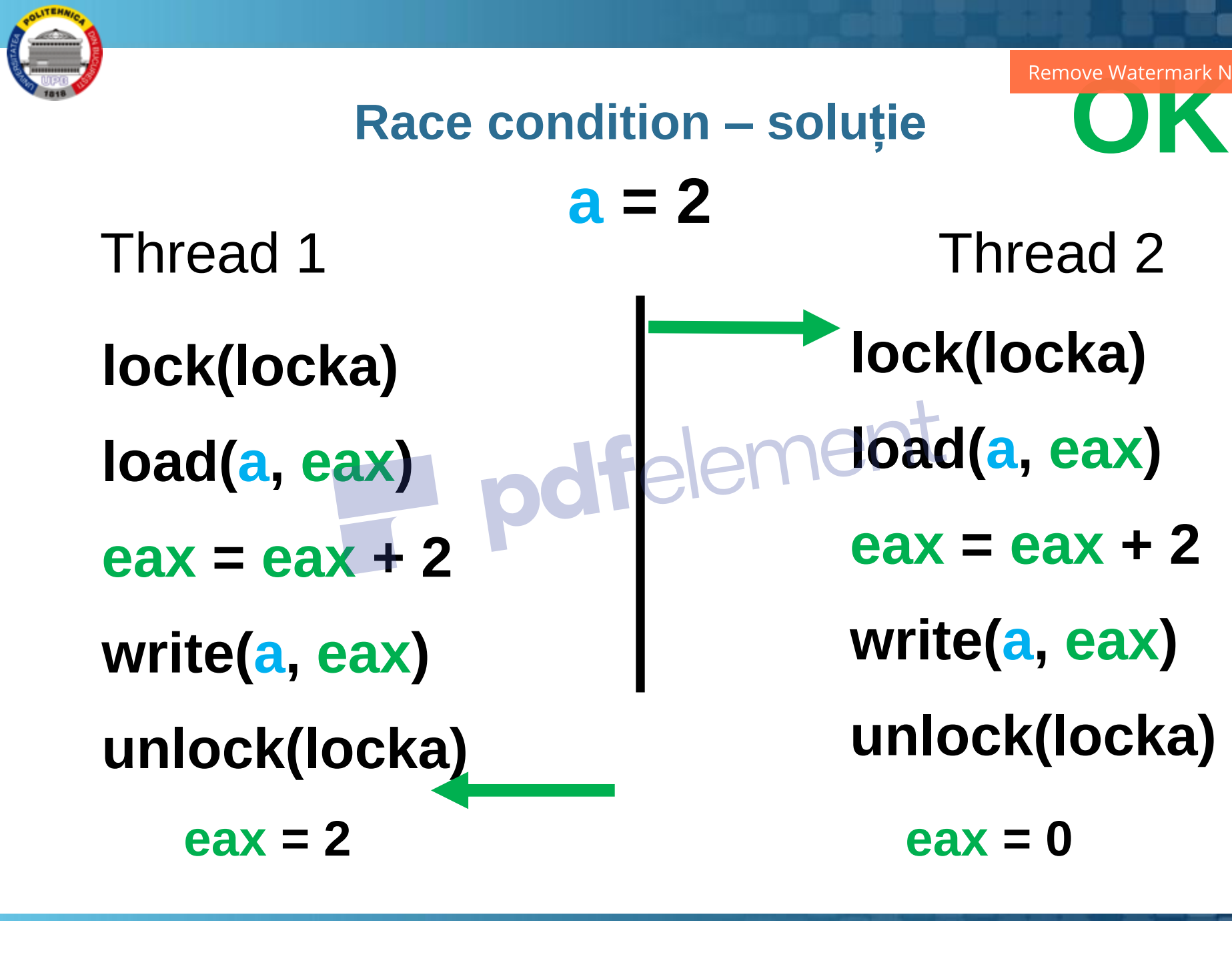
a = 2

Thread 1	Thread 2
lock(locka)	**lock(locka)**
load(a, eax)	**load(a, eax)**
eax = eax + 2	**eax = eax + 2**
write(a, eax)	**write(a, eax)**
unlock(locka)	**unlock(locka)**
eax = 2	**eax = 0**

pdfelement

Remove Watermark Now

Race condition – soluție



pdfelement

Remove Watermark Now

Race condition – soluție

?

a = 4

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 2	eax = 2

pdfelement

Remove Watermark Now

Race condition – soluție

a = 4

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 2	eax = 2

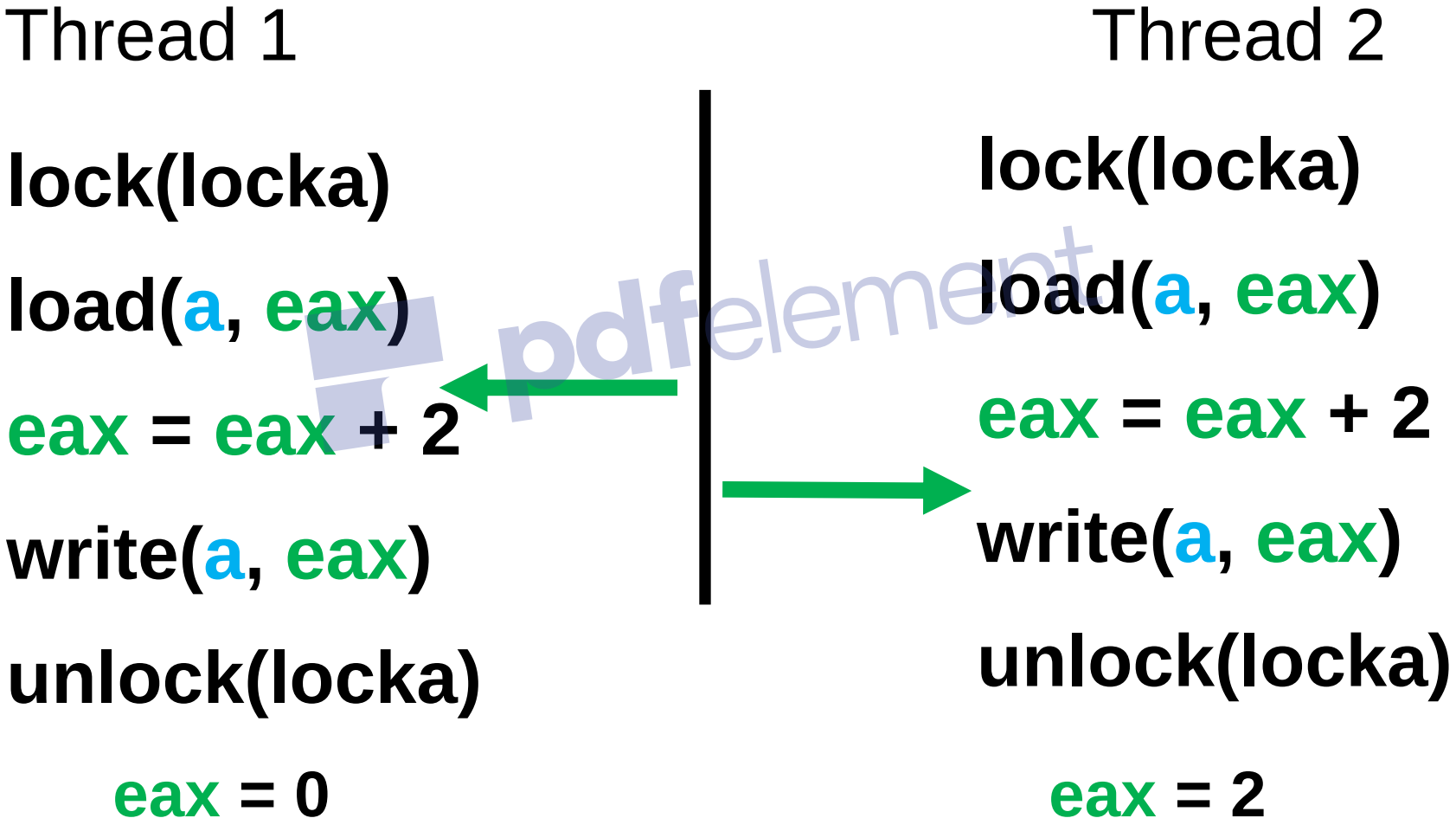
pdfelement

Remove Watermark Now

Race condition – soluție

?

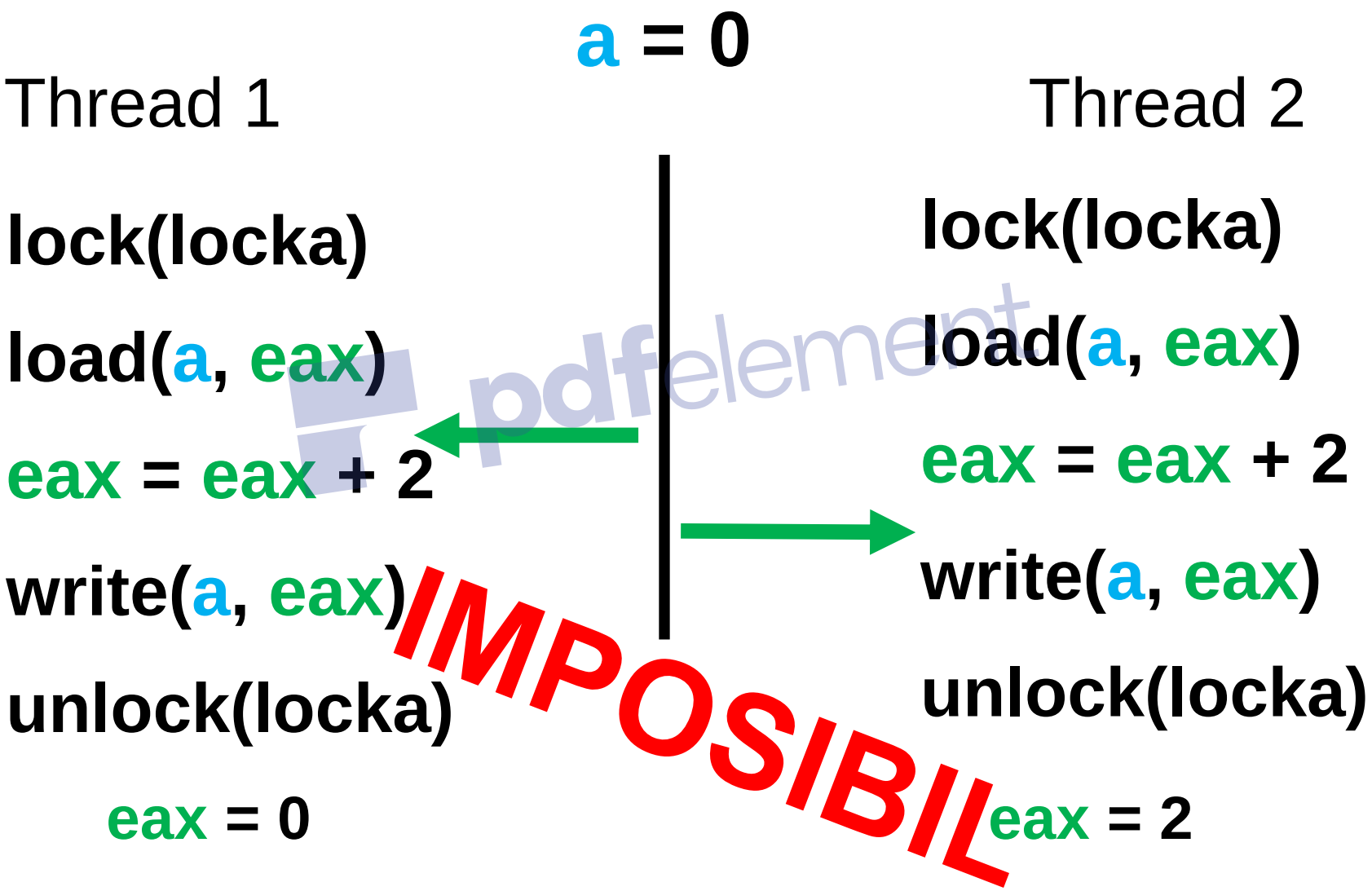
a = 0



pdfelement

Remove Watermark Now

Race condition – soluție



pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Complexitate algoritmi paraleli

Prof. Ciprian DOBRE
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Performanța

- Timp de execuție
- Memorie ocupată
- Număr de procese (thread-uri)
- Scalabilitate
- Toleranță la defecte
- Cost
- Eficiență
- Fiabilitate
- Portabilitate

pdfelement

Remove Watermark Now

Măsuri

- T - Timpul total necesar execuției programului paralel
- P - Numărul de procesoare utilizate
- G – Timp execuție **cel mai rapid algoritm secvențial**
- S – Speedup

$$- S = \frac{G}{T}$$

pdfelement

Remove Watermark Now

- Costul $C = T * P$

- Eficienţa $E = \frac{G}{C} = \frac{G}{TP} = \frac{S}{P}$

pdfelement

Remove Watermark Now

Complexitate

*3

pdfelement

Complexitate secvențială?

Remove Watermark Now

Complexitate

*3

pdfelement

27

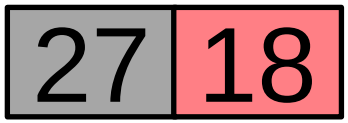
Complexitate secvențială?

Remove Watermark Now

Complexitate

*3

pdfelement



Complexitate secvențială?

Remove Watermark Now

Complexitate

*3

pdfelement

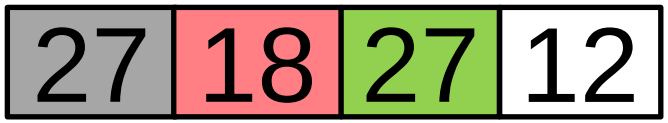
Complexitate secvențială?

Remove Watermark Now

Complexitate

*3

pdfelement



Complexitate secvențială?

Remove Watermark Now

Complexitate

*3

pdfelement

Complexitate secvențială? O(**N**)

Remove Watermark Now

Complexitate

*3

pdfelement

Complexitate paralelă?

Remove Watermark Now

Complexitate

*3

pdfelement

Complexitate paralelă? **O(1)** ?

Remove Watermark Now

Complexitate

*3

pdfelement

Complexitate paralelă? **O(1)** ? **P** = **N**

Remove Watermark Now

Complexitate

*3

pdfelement

Complexitate paralelă? $\mathbf{O}(\frac{N}{P})$

Remove Watermark Now

Complexitate

pdfelement

Speedup?

Remove Watermark Now

Complexitate

pdfelement

Speedup?

$T = O(\frac{N}{P})$

Remove Watermark Now

Complexitate

pdfelement

Speedup?

T = O($\frac{N}{P}$)

G = O(N)

Remove Watermark Now

Complexitate

pdfelement

Speedup?

T = O($\frac{N}{P}$)

G = O(N)

$$S = \frac{O(N)}{O(\frac{N}{P})}$$

Remove Watermark Now

Complexitate

pdfelement

Speedup?

T = O($\frac{N}{P}$)

G = O(N)

$$S = \frac{O(N)}{O(\frac{N}{P})} = O(P)$$

Remove Watermark Now

Complexitate

$$S = \frac{O(N)}{O(\frac{N}{P})} = O(P)$$

pdfelement

Eficiența?

T = O($\frac{N}{P}$)

G = O(N)

Remove Watermark Now

Complexitate

$$S = \frac{O(N)}{O(\frac{N}{P})} = O(P)$$

Eficiența?

$T = O(\frac{N}{P})$

$G = O(N)$

$$E = \frac{S}{P} = \frac{O(P)}{P} = 1$$

Remove Watermark Now

Legea lui Amdhal

pdfelement

Remove Watermark Now

Amdahl's law

Validity of the single processor approach to achieving large scale computing capabilities[1]

Gene M. Amdahl

IBM Sunnyvale, California

pdfelement

1 INTRODUCTION

For over a decade prophets have voiced the contention that the o
has reached its limits and that truly significant advences can be m
multiplicity of computers in such a manner as to permit cooperati
direction has been pointed out as general purpose computers wi
of memories, or as specialized computers with geometrically relat
controlled by one or more instruction streams.

Remove Watermark Now

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

pdfelement

$$S = \frac{G}{T}$$

Remove Watermark Now

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

pdfelement

$$S = \frac{G}{T} = \frac{G}{fG + \frac{(1-f)G}{P}} = \frac{1}{f + \frac{(1-f)}{P}}$$

Remove Watermark Now

Legea lui Amdahl

$$S = \frac{1}{f + \frac{(1-f)}{P}}$$

pdfelement

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

Remove Watermark Now

Legea lui Amdahl

$$S = \frac{1}{f + \frac{(1-f)}{P}}$$

pdfelement

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

$$\boldsymbol{S \leq \frac{1}{f}}$$

Remove Watermark Now

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

pdfelement

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

Remove Watermark Now

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

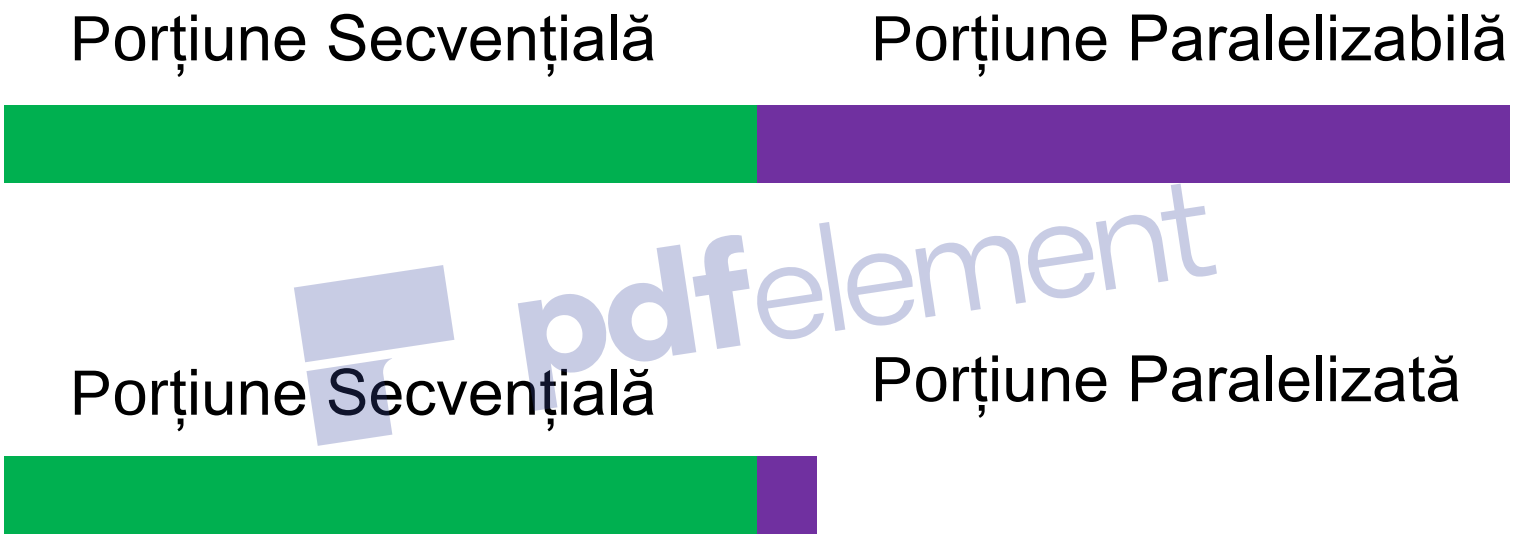
pdfelement

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

$$\boldsymbol{T \geq fG}$$

Remove Watermark Now

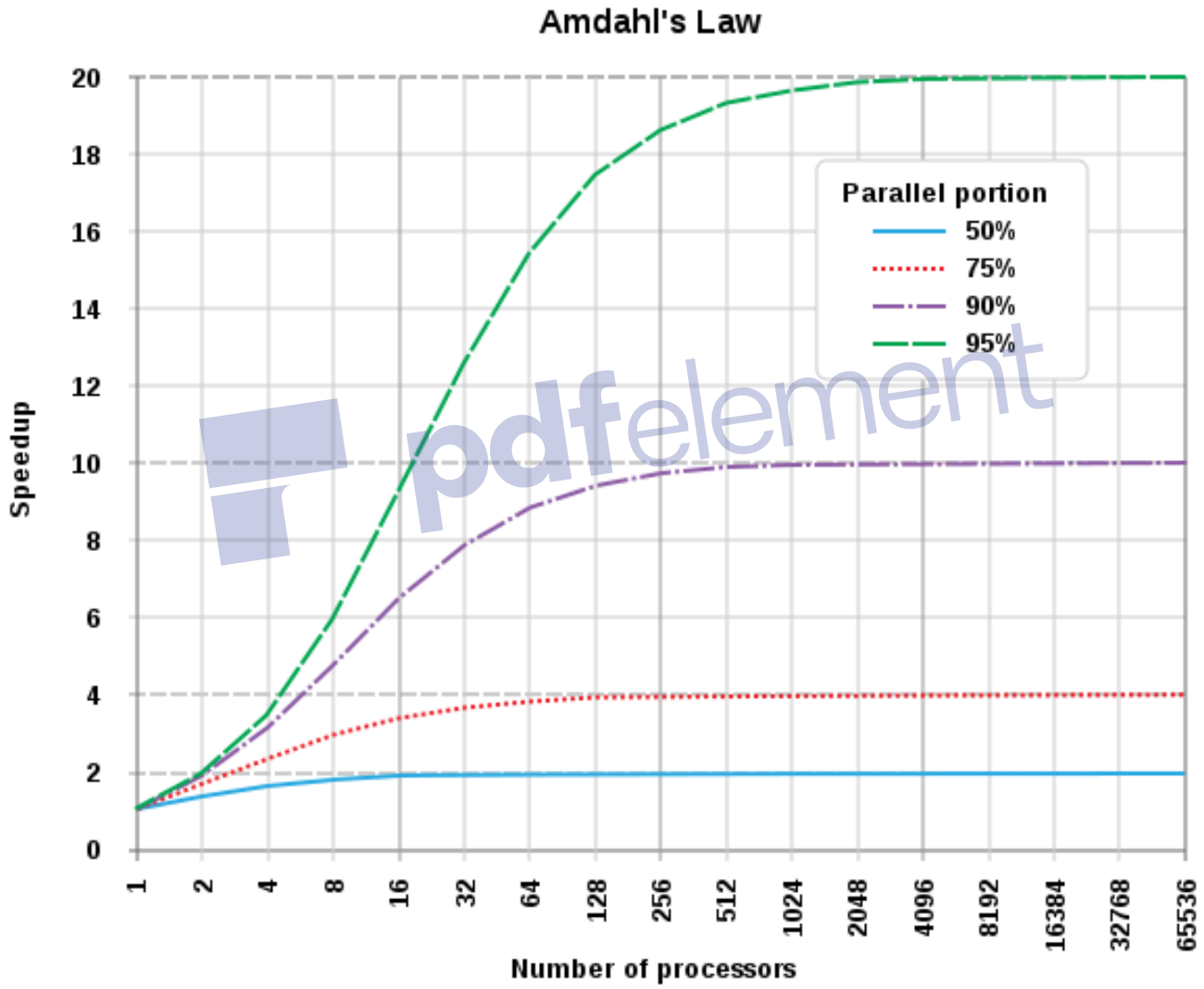
Legea lui Amdahl



pdfelement

Remove Watermark Now

Legea lui Amdahl



pdfelement

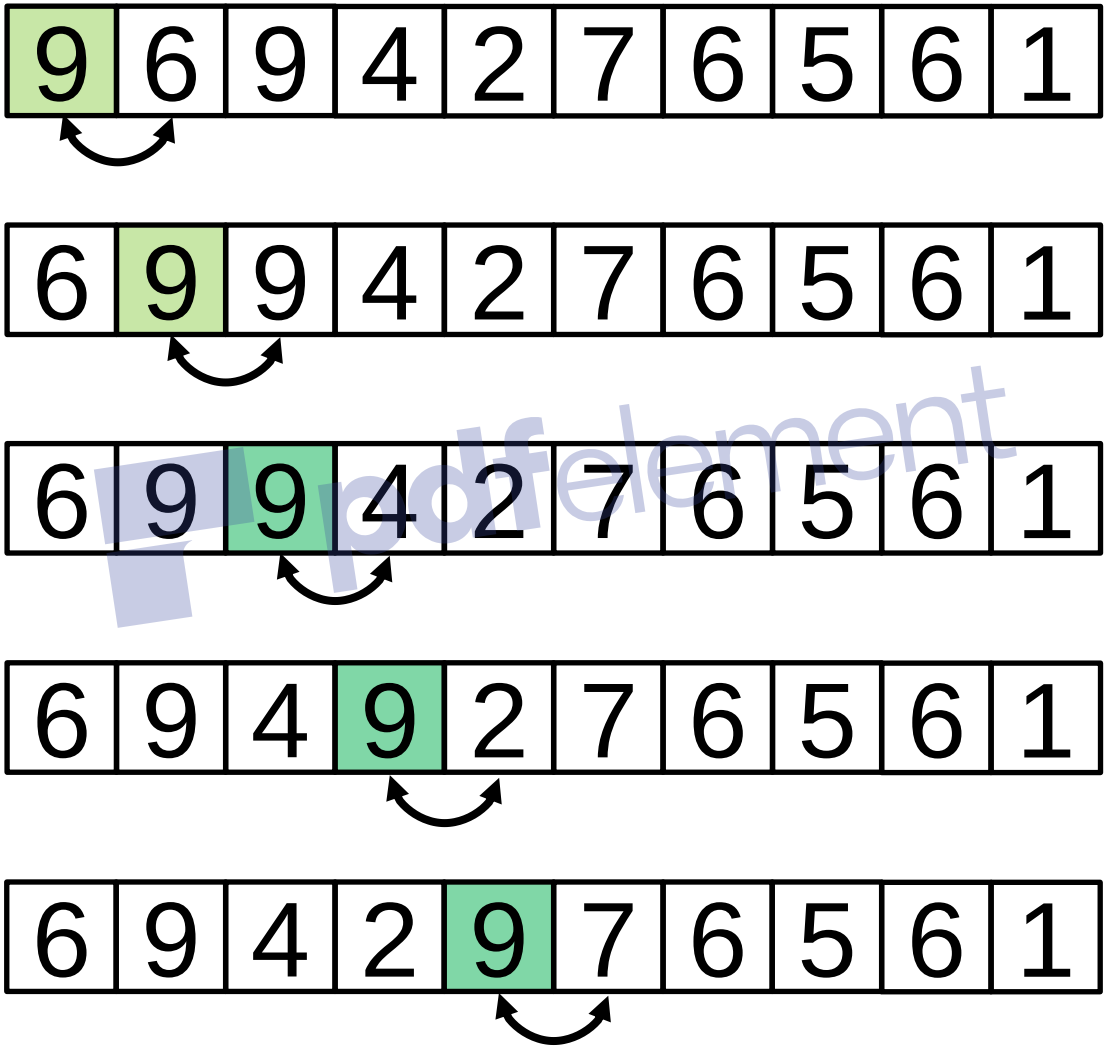
Remove Watermark Now

Exemplu: sortarea unui vector

pdfelement

Remove Watermark Now

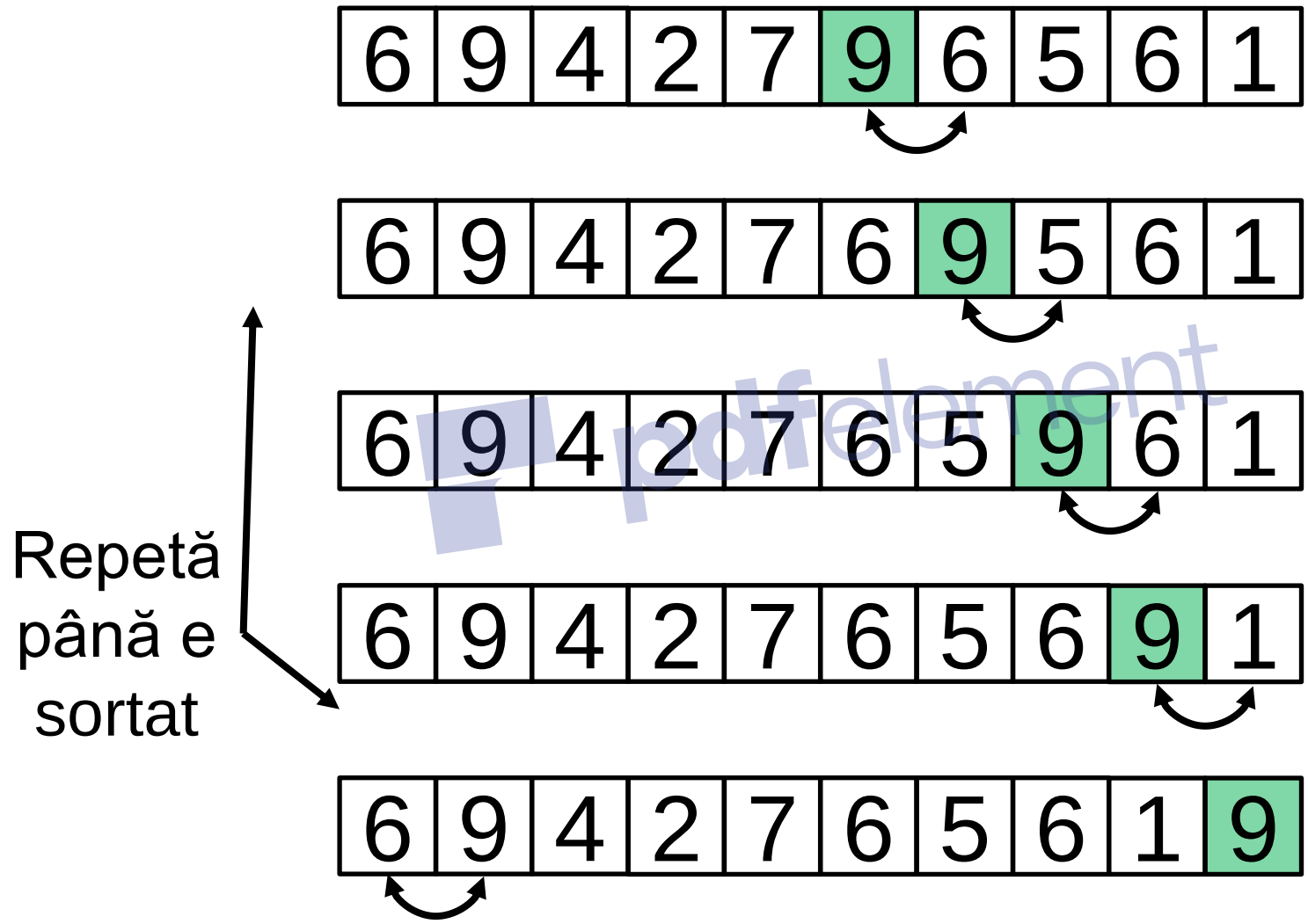
Bubble sort



pdfelement

Remove Watermark Now

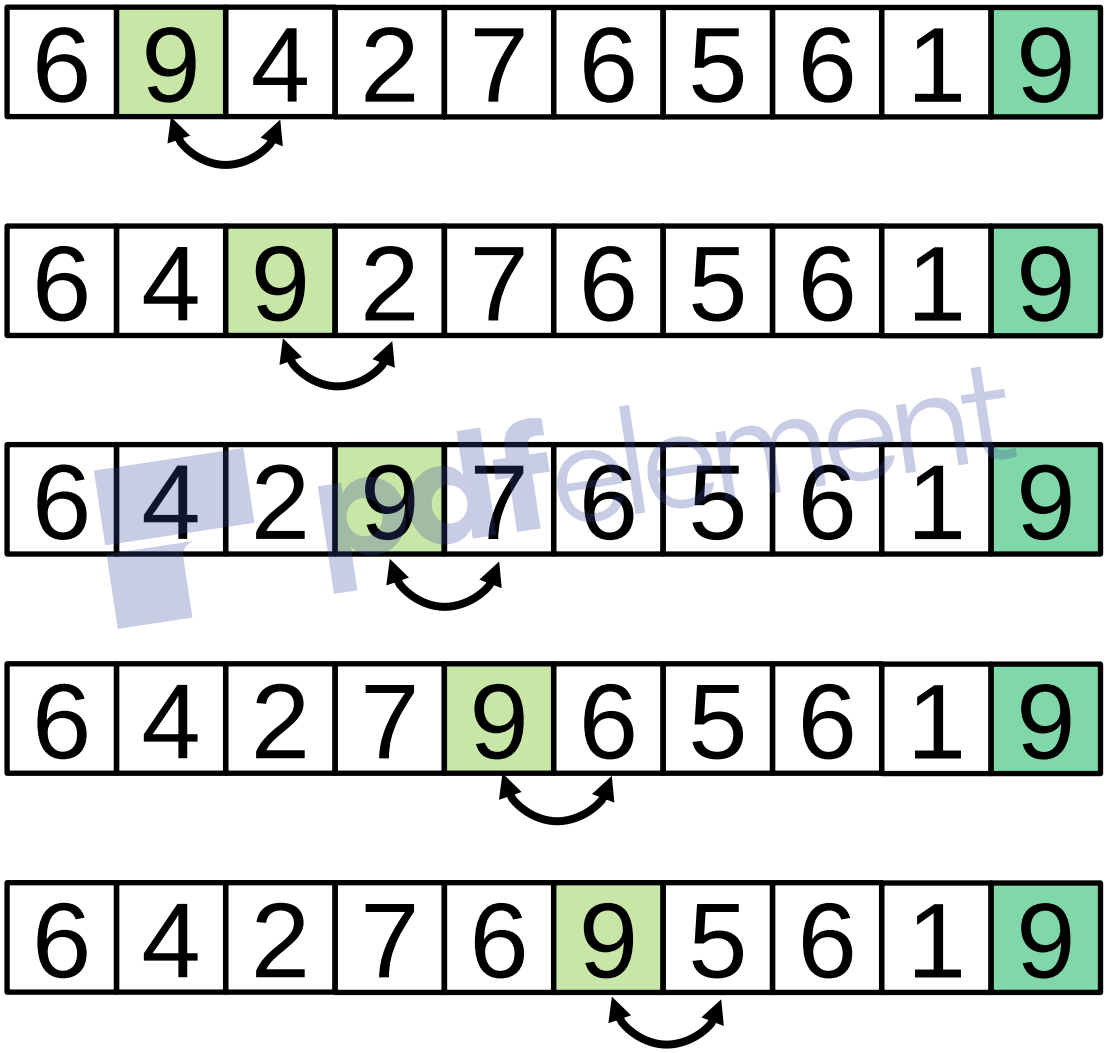
Bubble sort



pdfelement

Remove Watermark Now

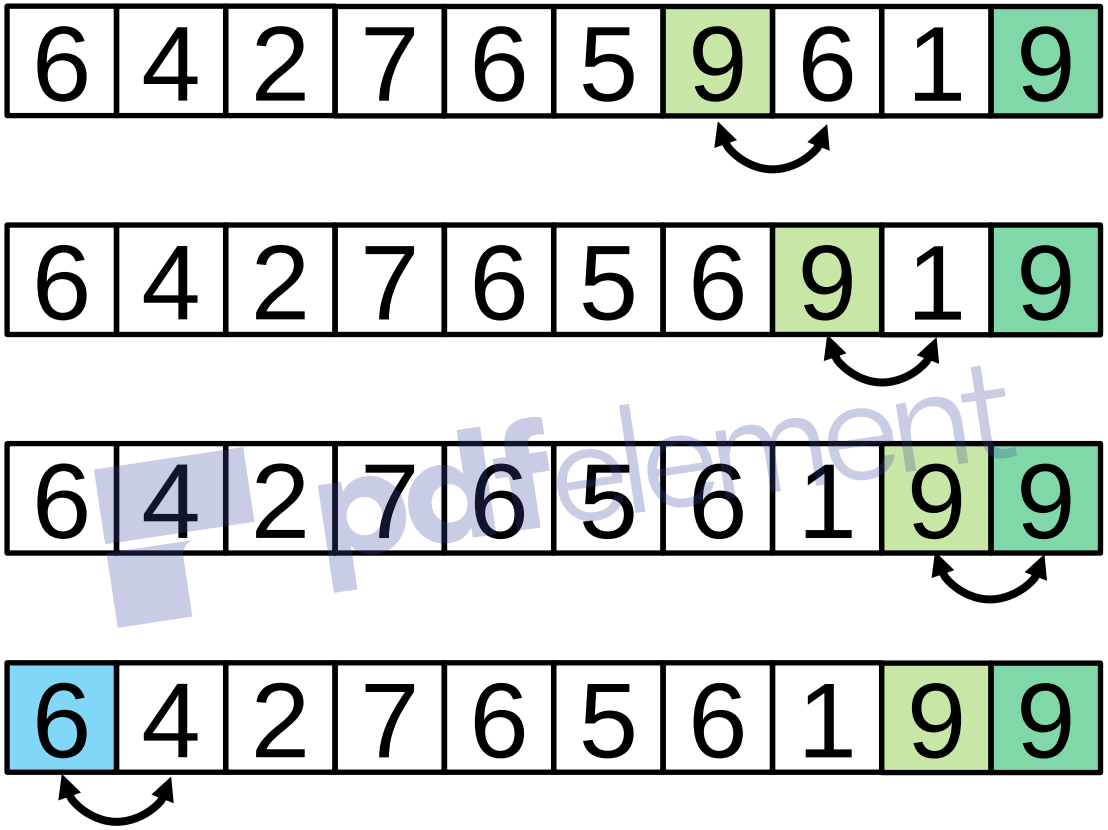
Bubble sort



pdfelement

Remove Watermark Now

Bubble sort

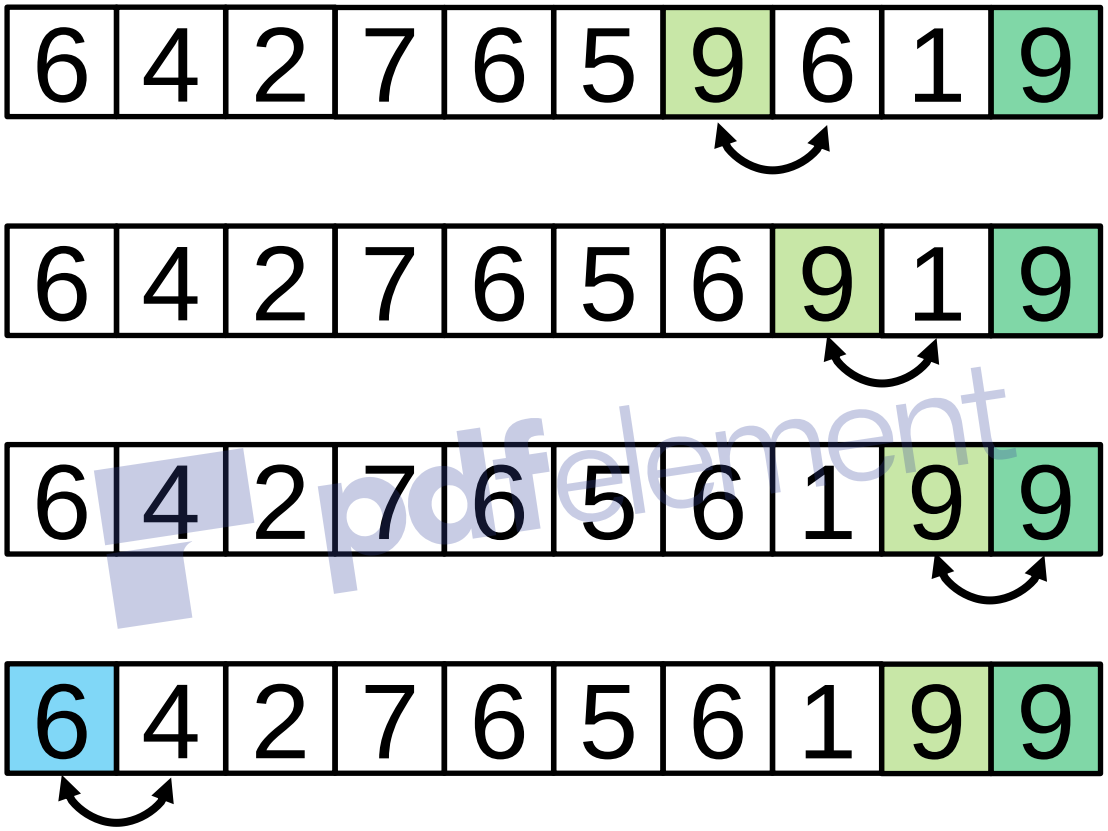


.....

pdfelement

Remove Watermark Now

Bubble sort



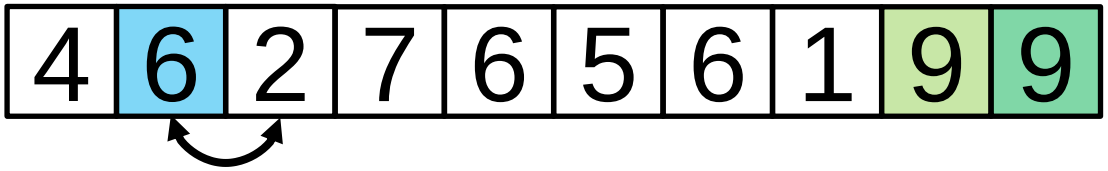
pdfelement

.....

Complexitate?

Remove Watermark Now

Bubble sort



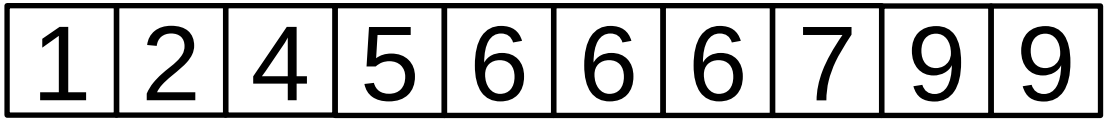
.....

pdfelement

Complexitate: **O(n^2)**

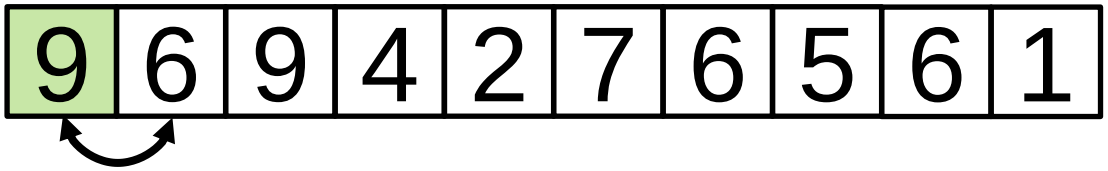
Un pas trece prin toate elementele

Garantat să termine după n repetiții

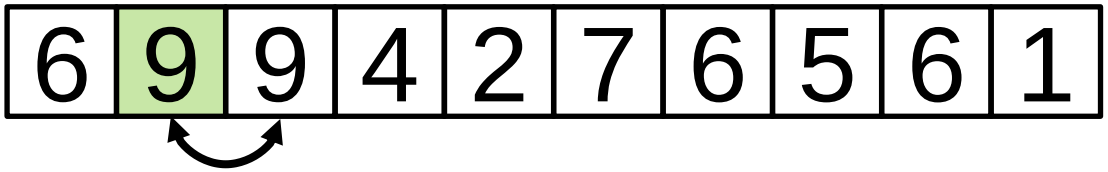


Remove Watermark Now

Parallel bubble sort



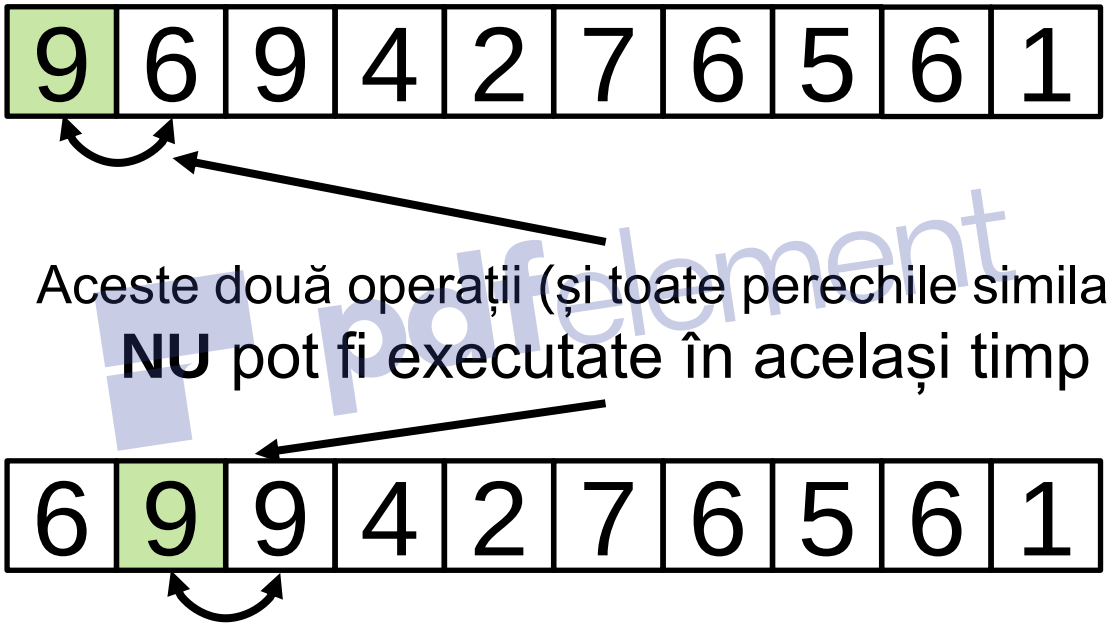
Cum paralelizăm?



pdfelement

Remove Watermark Now

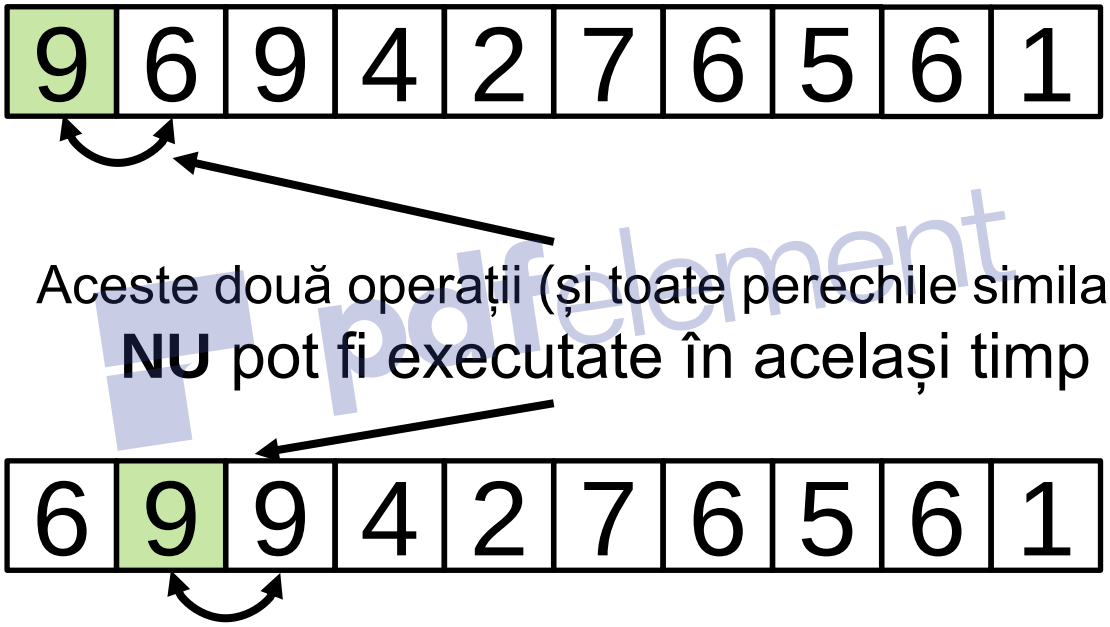
Parallel bubble sort



pdfelement

Remove Watermark Now

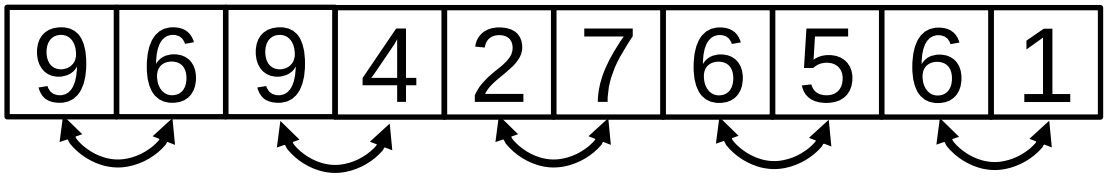
Parallel bubble sort



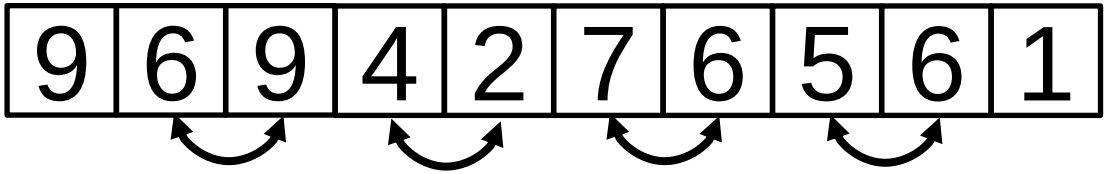
Solution hint: Nu e necesară execuția operațiilor într-o anumită ordine

Remove Watermark Now

OETS: Odd-Even Transposition Sort



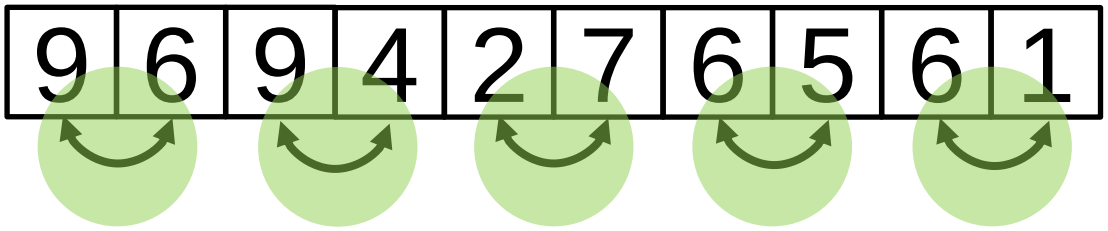
pdfelement



Remove Watermark Now

OETS: Odd-Even Transposition Sort

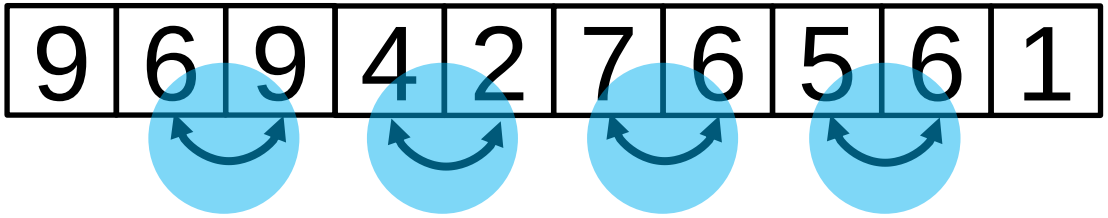
Pot fi executate în paralel →



.....

pdfelement

Pot fi executate în paralel →

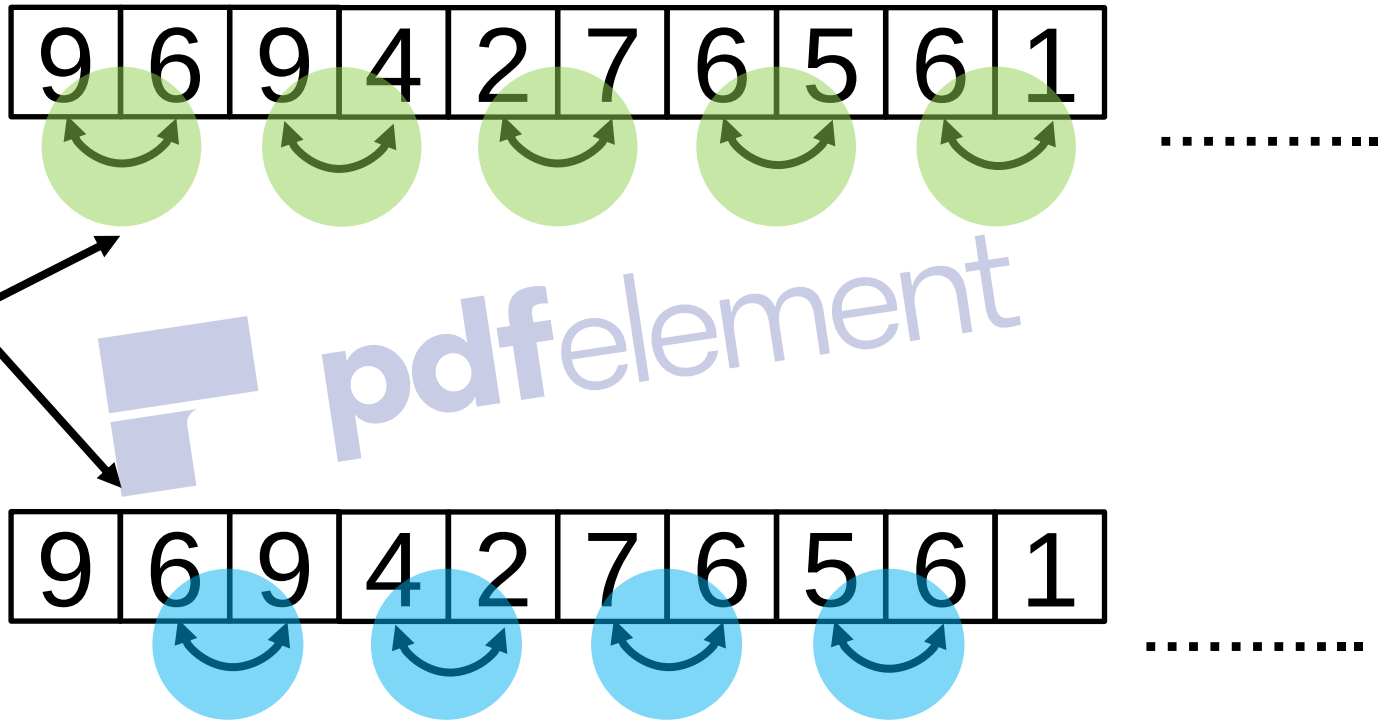


.....

.....

Remove Watermark Now

OETS: Odd-Even Transposition Sort

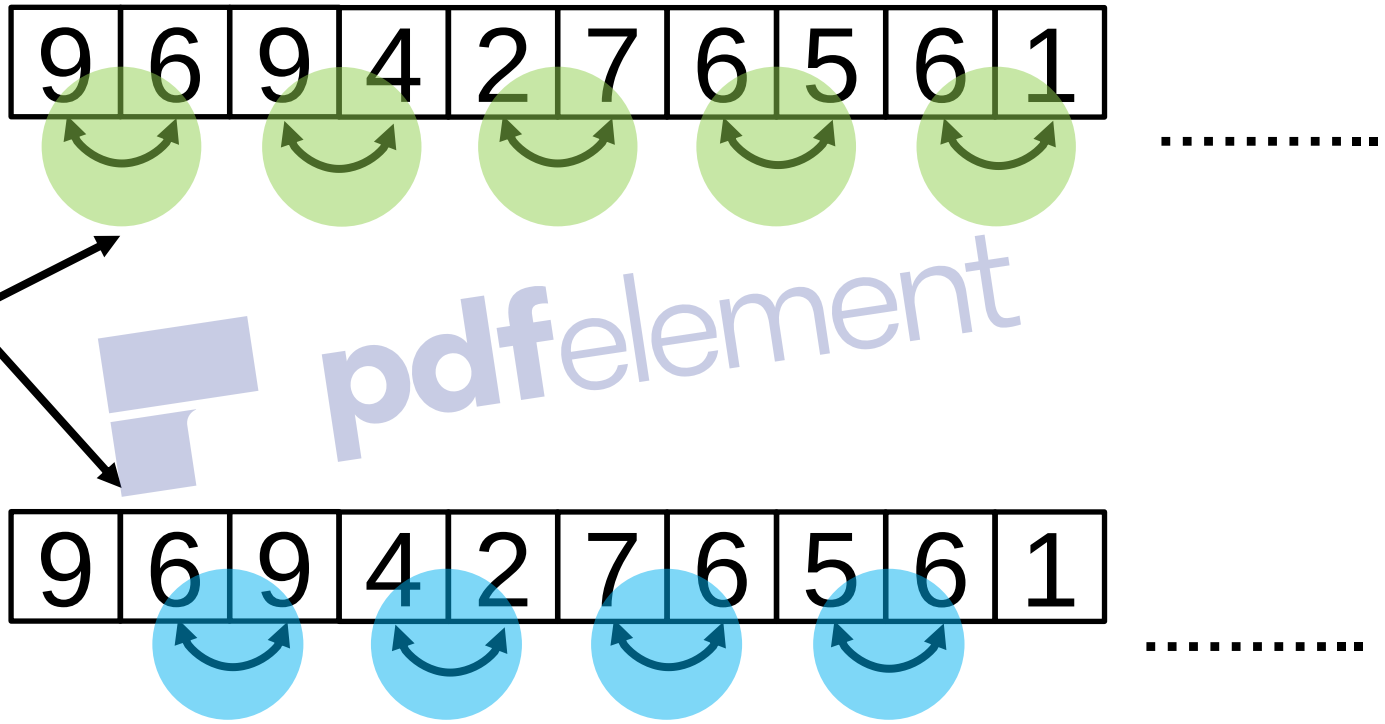


pdfelement

.....

Remove Watermark Now

OETS: Odd-Even Transposition Sort



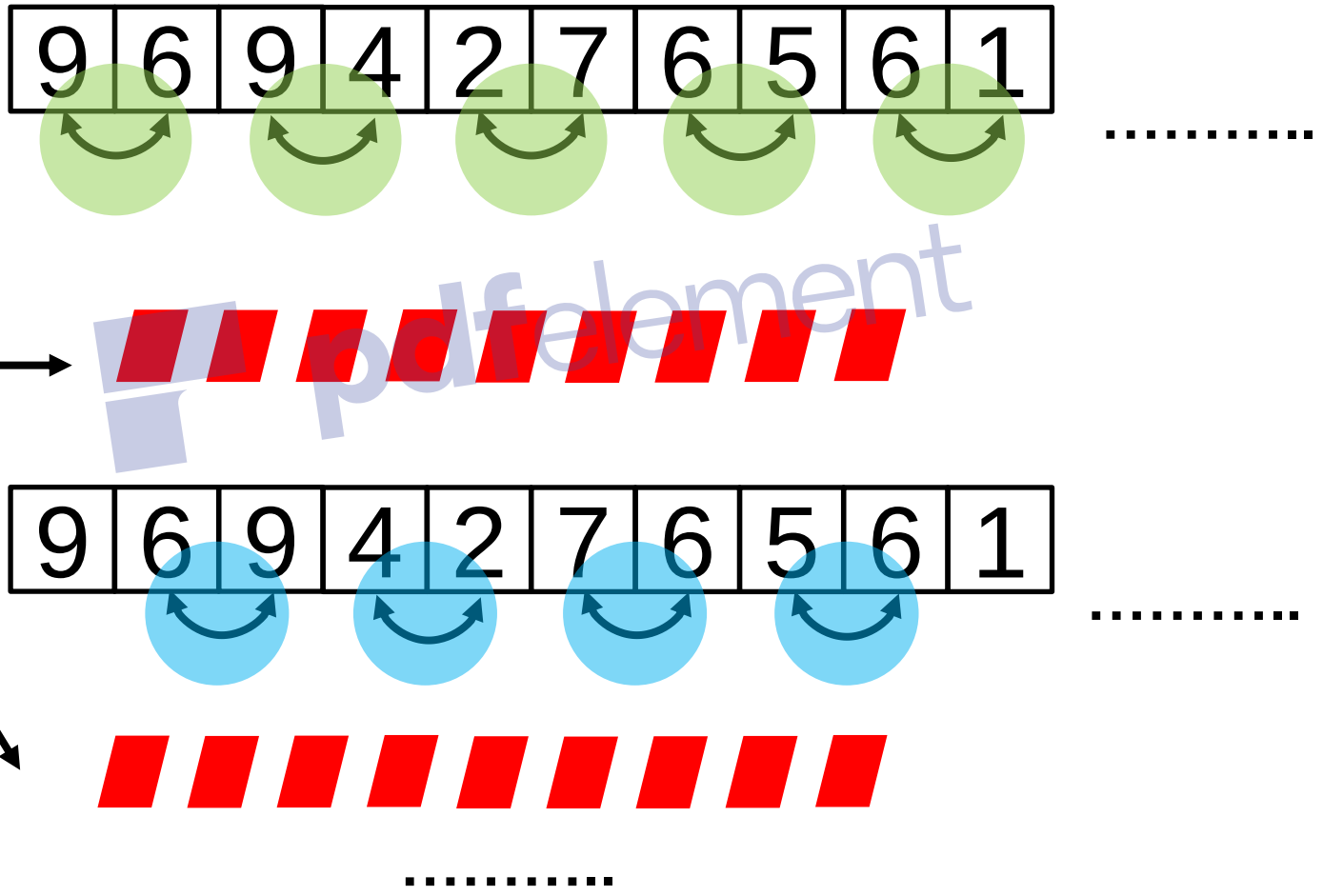
pdfelement

Ce facem? Ce folosim?

.....

Remove Watermark Now

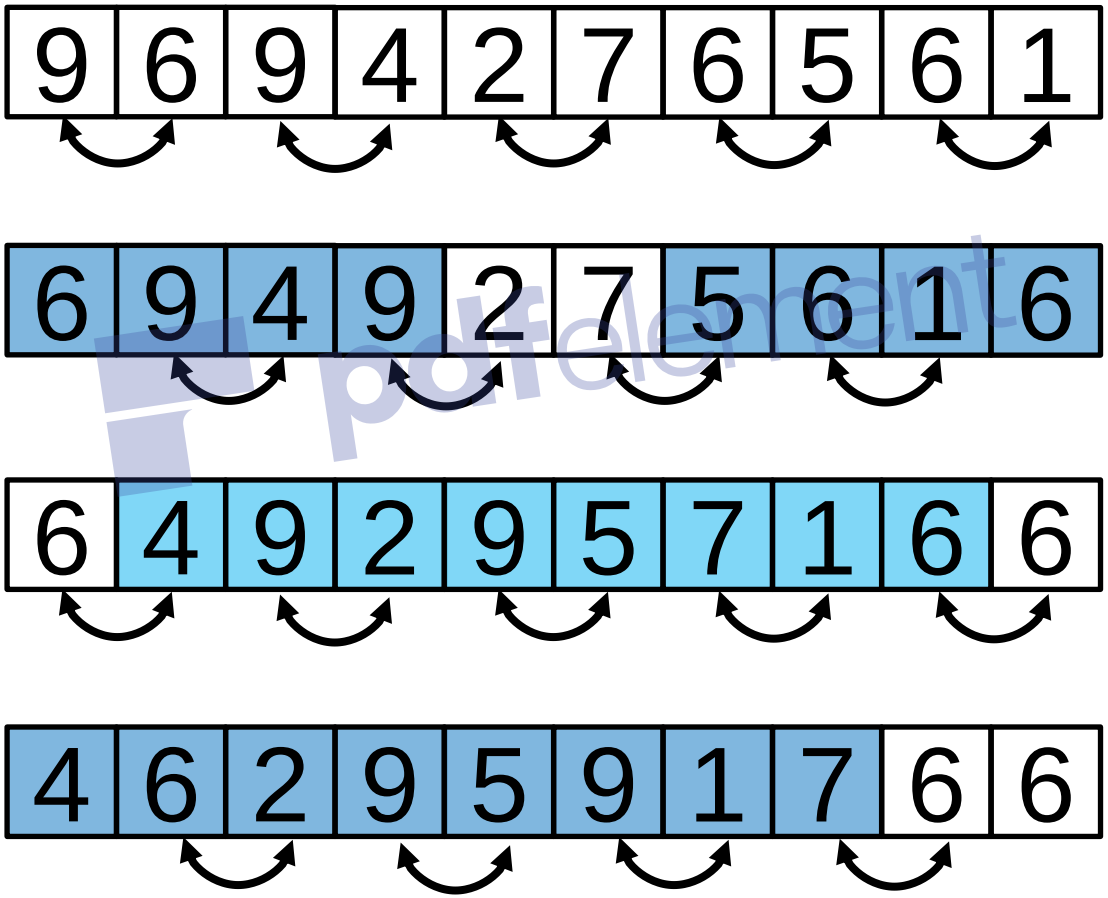
OETS: Odd-Even Transposition Sort



pdfelement

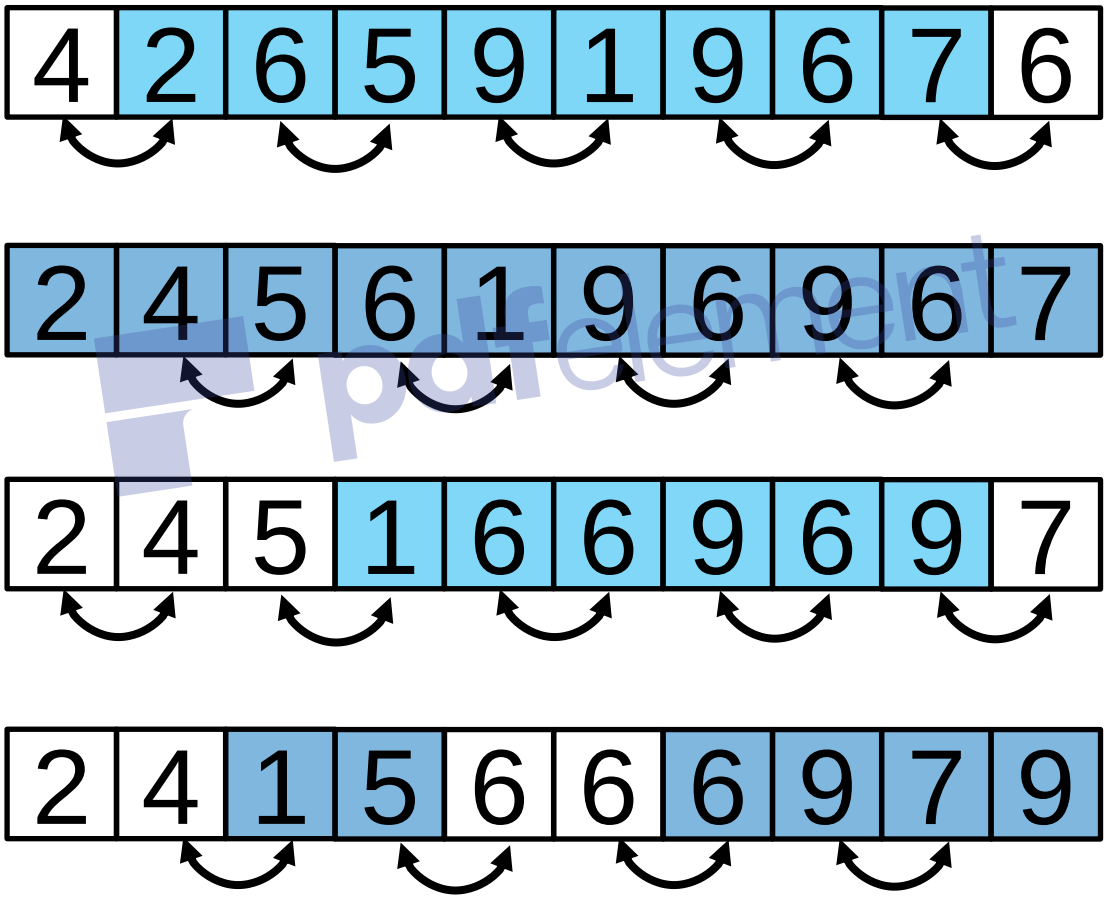
Remove Watermark Now

OETS: Odd-Even Transposition Sort



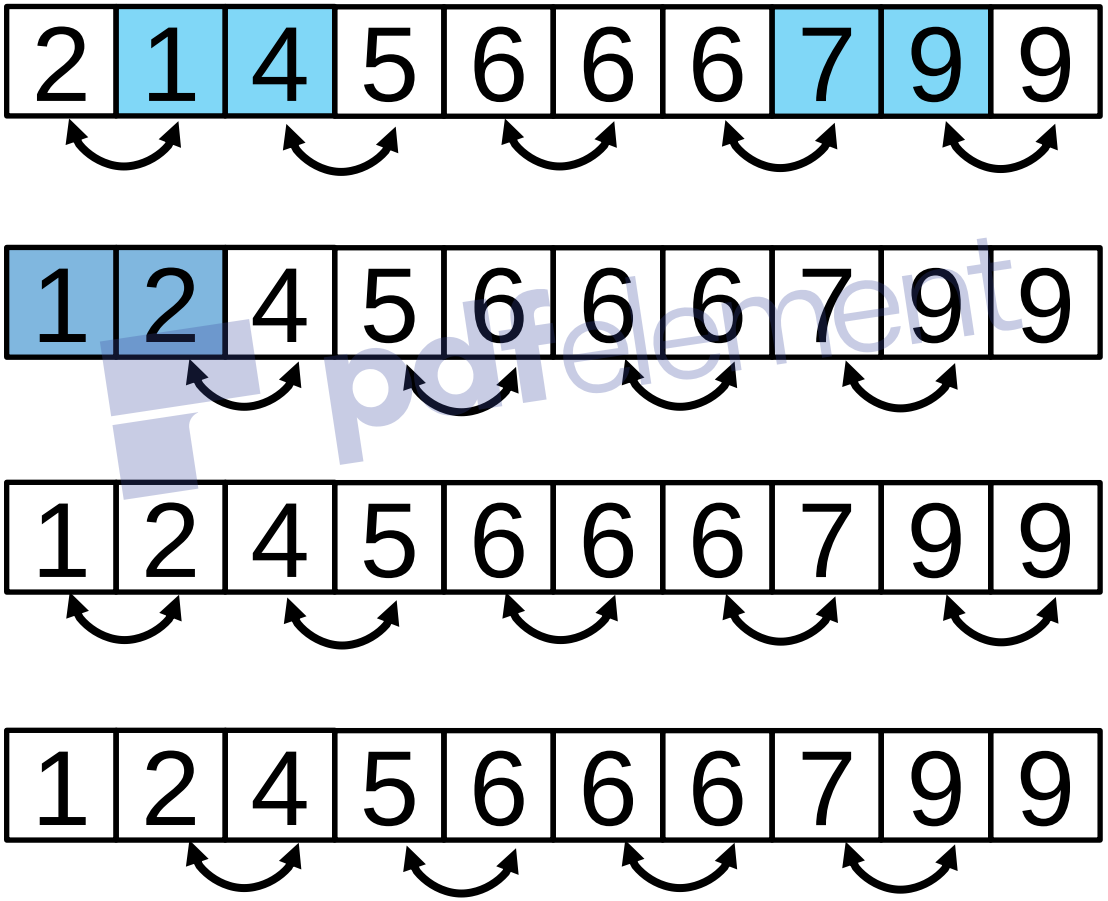
Remove Watermark Now

OETS: Odd-Even Transposition Sort



Remove Watermark Now

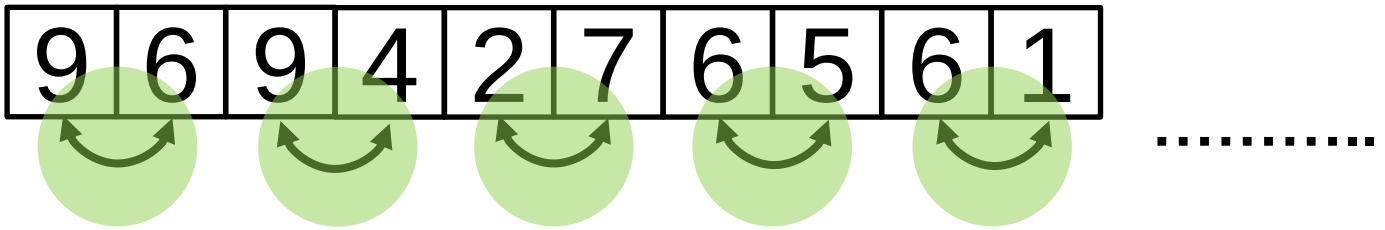
OETS: Odd-Even Transposition Sort



pdfelement

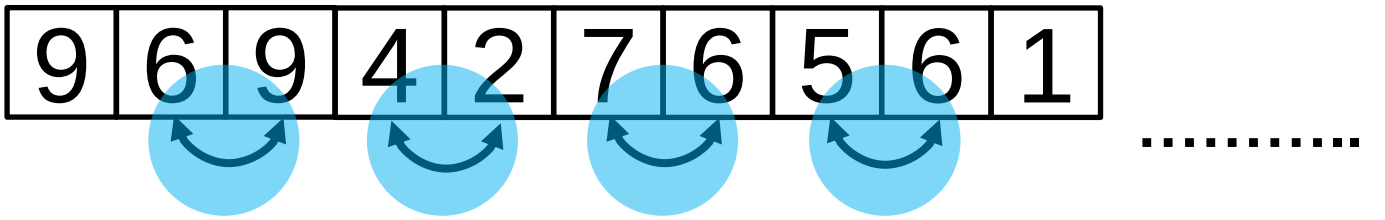
Remove Watermark Now

OETS: Odd-Even Transposition Sort



Complexitate a soluției paralele?

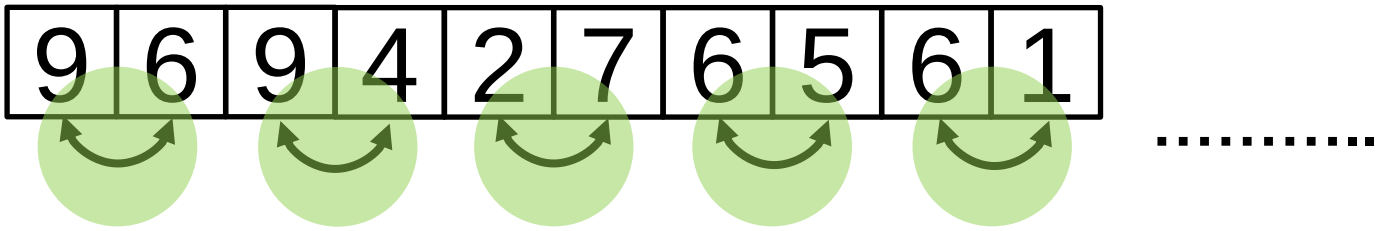
pdfelement



.....

Remove Watermark Now

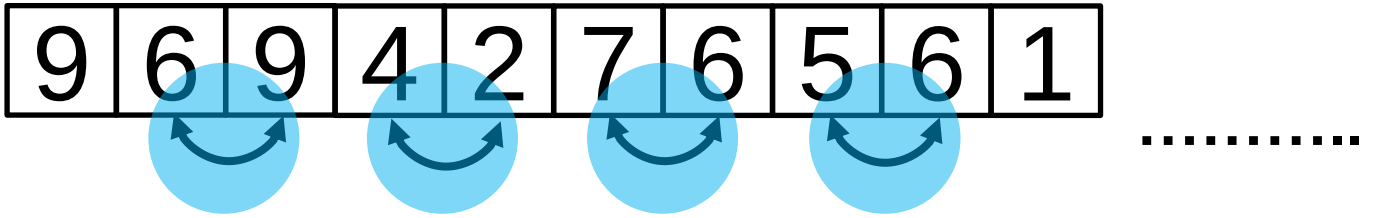
OETS: Odd-Even Transposition Sort



.....

Complexitate a soluției paralele?

pdfelement



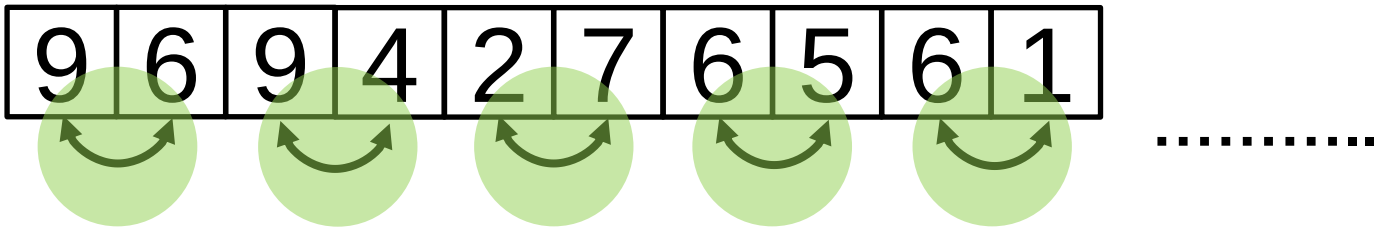
.....

$$T = O\left(\frac{N}{P} * N\right) (= O(N) \text{ pentru } P = N)$$

.....

Remove Watermark Now

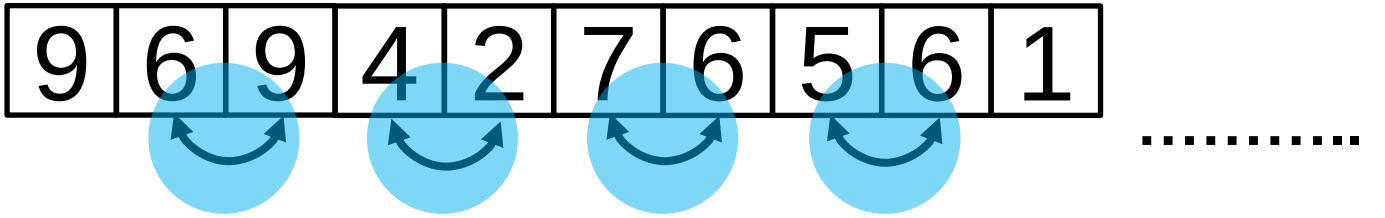
OETS: Odd-Even Transposition Sort



.....

Speedup?

pdfelement



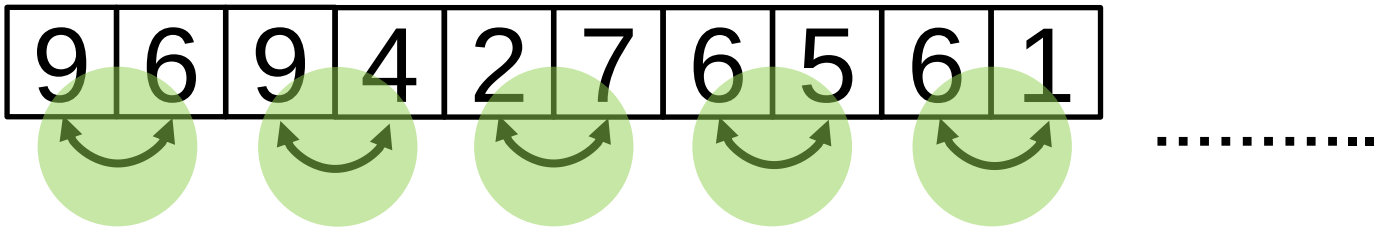
.....

$$T = O\left(\frac{N}{P} * N\right) (= O(N) \text{ pentru } P = N)$$

.....

Remove Watermark Now

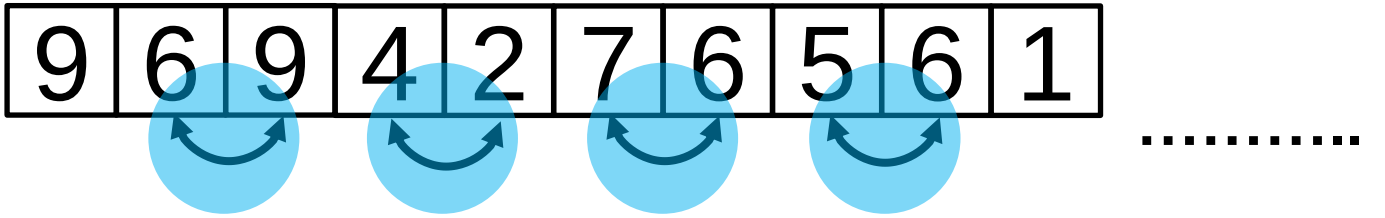
OETS: Odd-Even Transposition Sort



.....

Speedup?

pdfelement



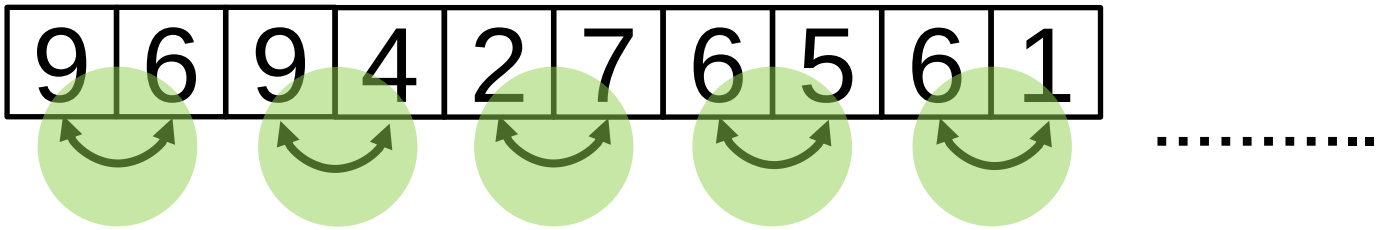
.....

$$S = \frac{N^2}{\frac{N^2}{P}} = P$$

.....

Remove Watermark Now

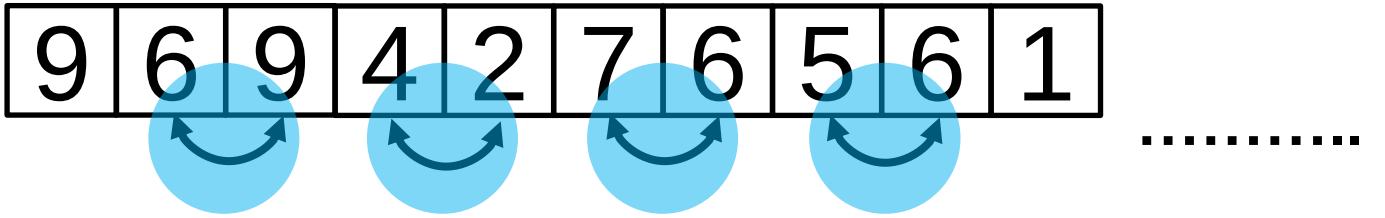
OETS: Odd-Even Transposition Sort



.....

Speedup? **Sigur**?

pdfelement



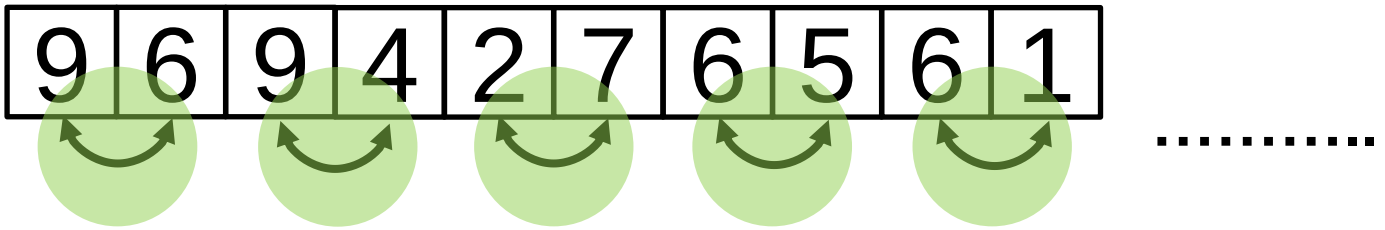
.....

$$S = \frac{N^2}{\frac{N^2}{P}} = P$$

.....

Remove Watermark Now

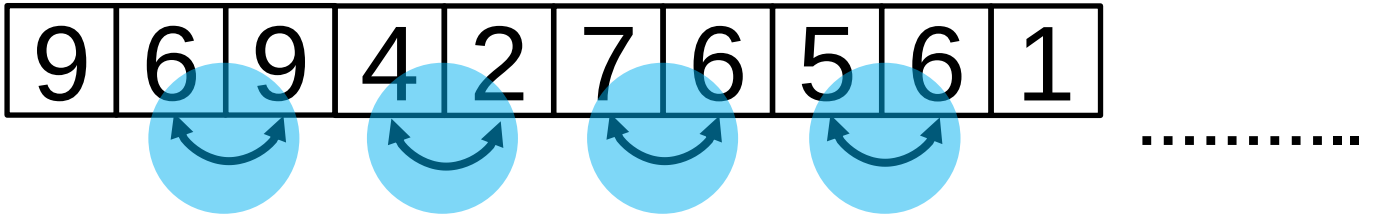
OETS: Odd-Even Transposition Sort



.....

Speedup?

pdfelement



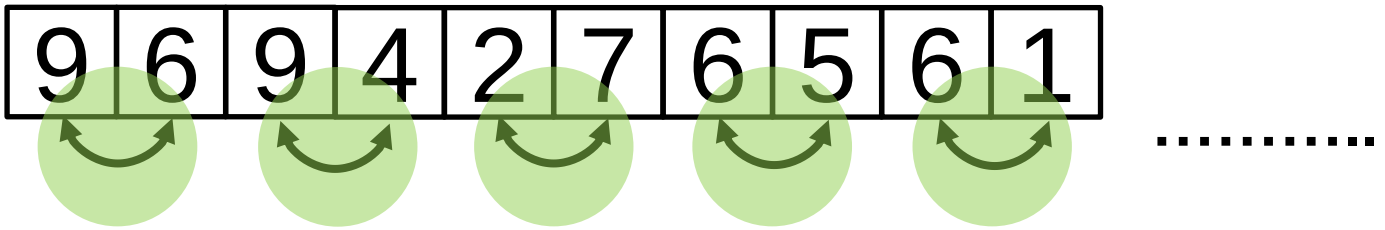
.....

$$S = \frac{N log_2 N}{\frac{N^2}{P}}$$

.....

Remove Watermark Now

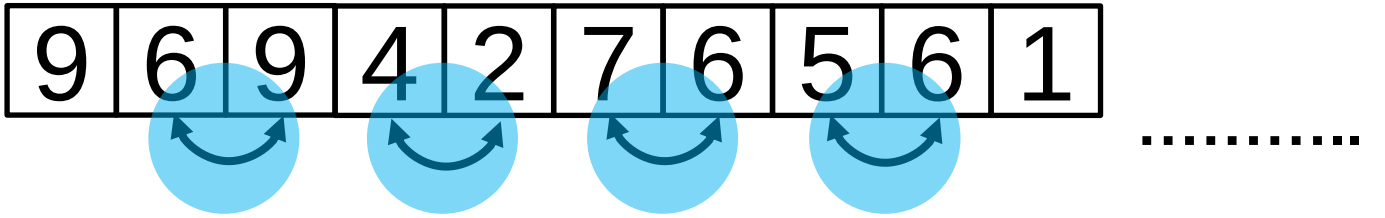
OETS: Odd-Even Transposition Sort



.....

Speedup?

pdfelement



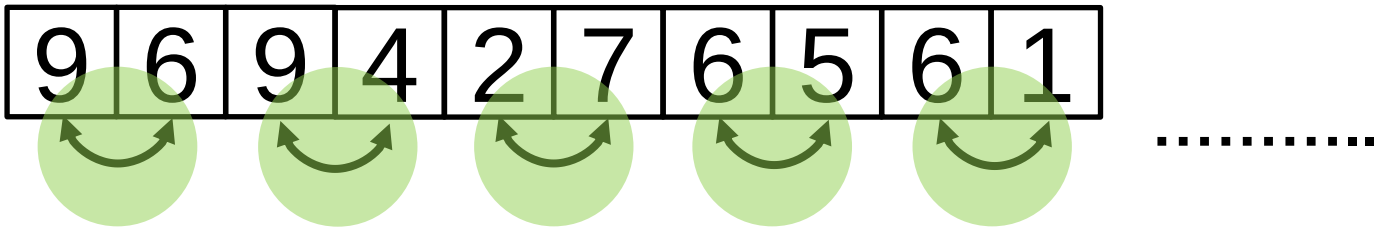
.....

$$S = \frac{Plog_2 N}{N}$$

.....

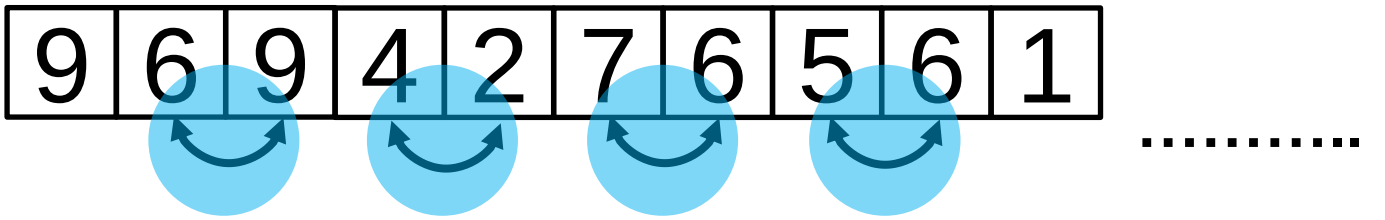
Remove Watermark Now

OETS: Odd-Even Transposition Sort



.....

Nu uitați: Așteptatul la barieră poate introduce timpi mari!



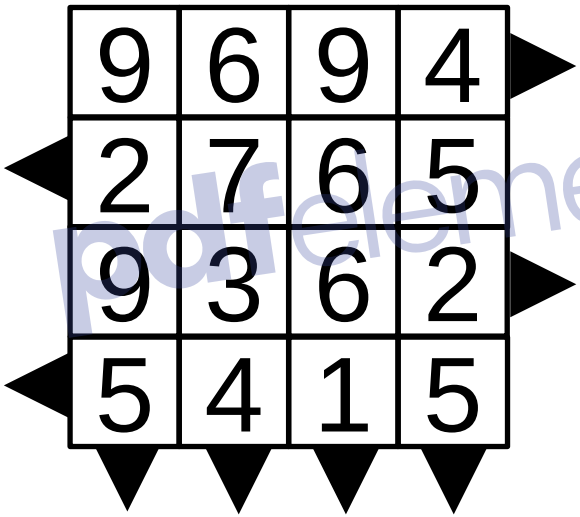
.....

$$S = \frac{Plog_2N}{N}$$

.....

Remove Watermark Now

Shear sort
(Row-column sort)
(Snake sort)



pdfelement

Sortează fiecare linie **pară** în mod **ascendent**
Sortează fiecare linie **impară** în mod **descendent**

Remove Watermark Now

Shear sort

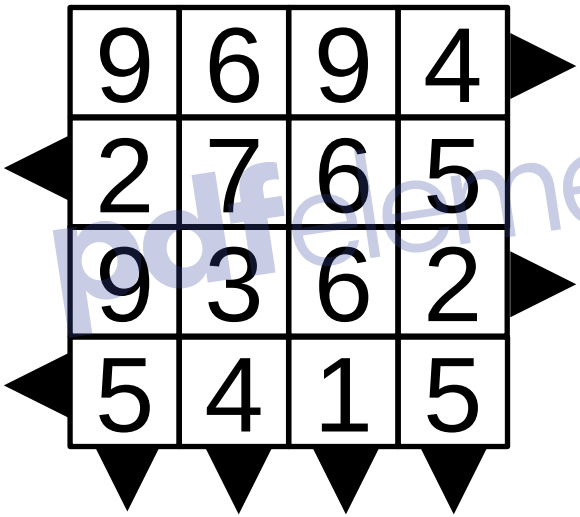
9	6	9	4
2	7	6	5
9	3	6	2
5	4	1	5

pdfelement

Sortează crescător coloanele

Remove Watermark Now

Shear sort

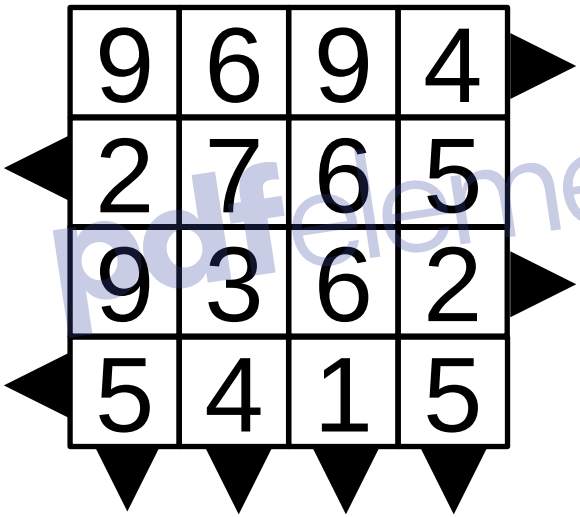


pdfelement

Repetă tot de $log_2 n$ ori

Remove Watermark Now

Shear sort

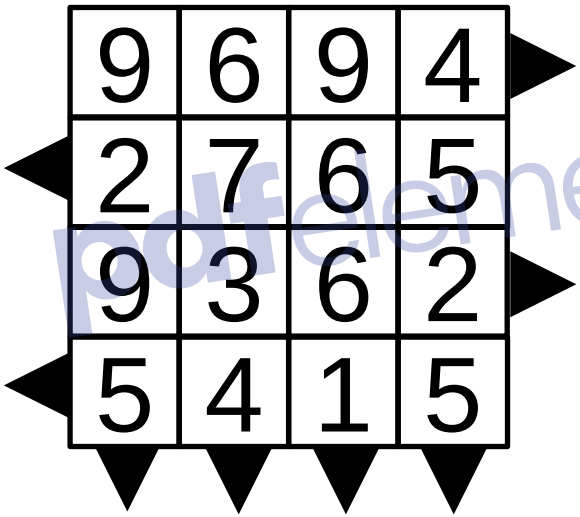


pdfelement

De ce sortăm liniile pare crescător și cele impare descrescător?

Remove Watermark Now

Shear sort

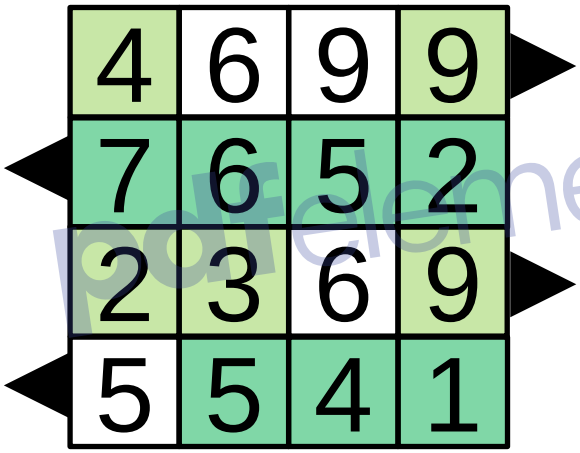


pdfelement

De ce liniile pare crescător și celelalte descrescător?
Dorim compararea celui mai mare element de pe linia i cu cel mai mic de pe linia i+1

Remove Watermark Now

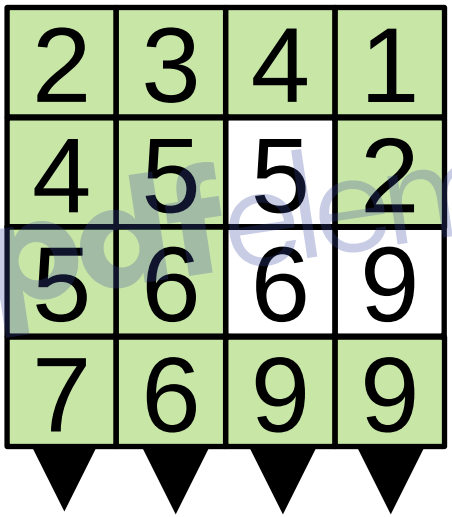
Shear sort



pdfelement

Remove Watermark Now

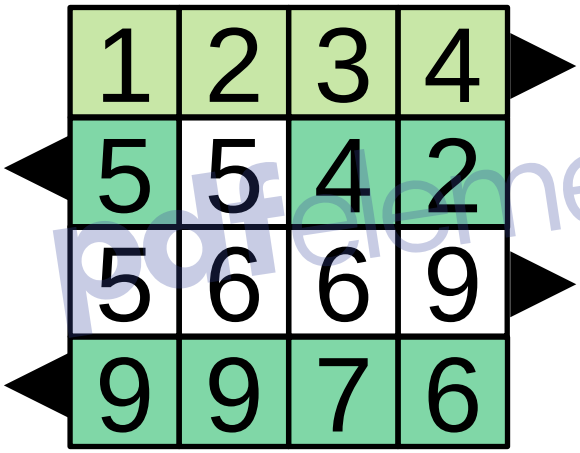
Shear sort



pdfelement

Remove Watermark Now

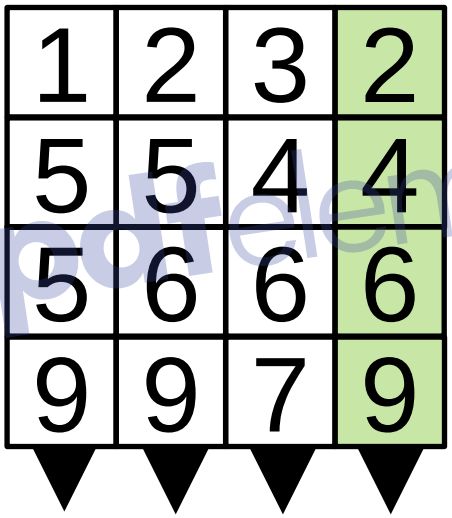
Shear sort



pdfelement

Remove Watermark Now

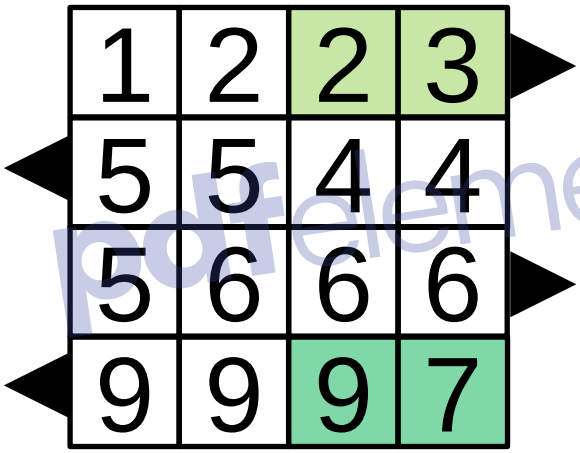
Shear sort



pdfelement

Remove Watermark Now

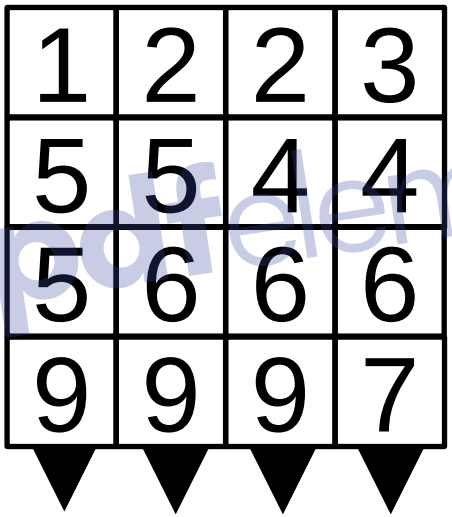
Shear sort



pdfelement

Remove Watermark Now

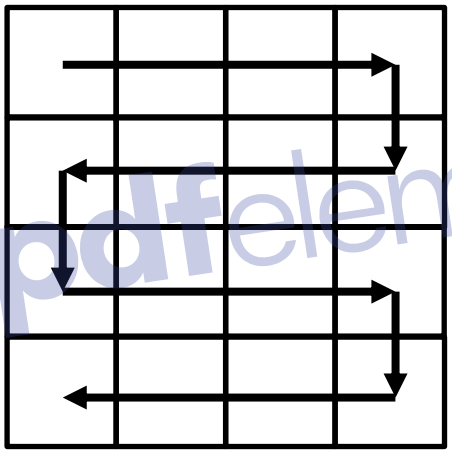
Shear sort



pdfelement

Remove Watermark Now

Shear sort

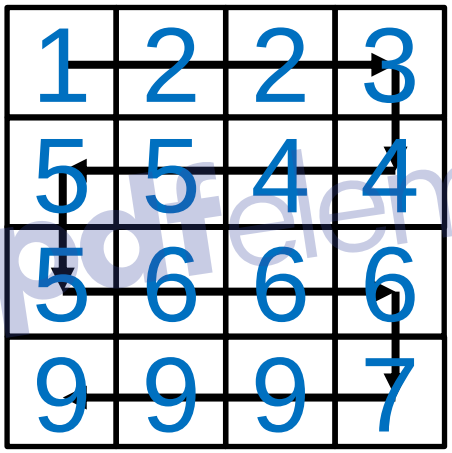


pdfelement

Lista finală se obține citind în formă de șerpuită (snake sort)

Remove Watermark Now

Shear sort

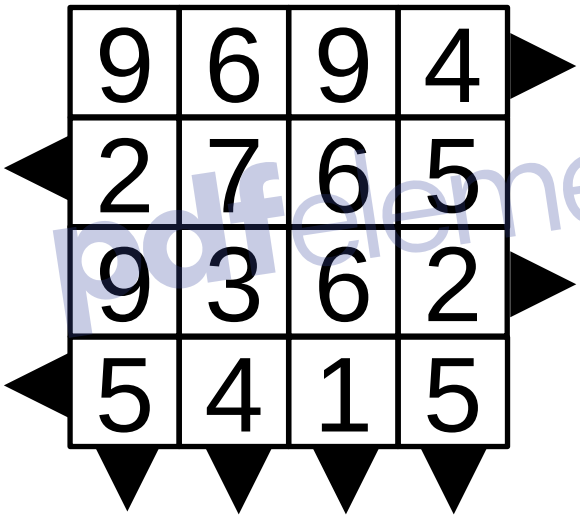


pdfelement

1	2	2	3	4	4	5	5	5	6	6	6	7	9	9	9

Remove Watermark Now

Shear sort

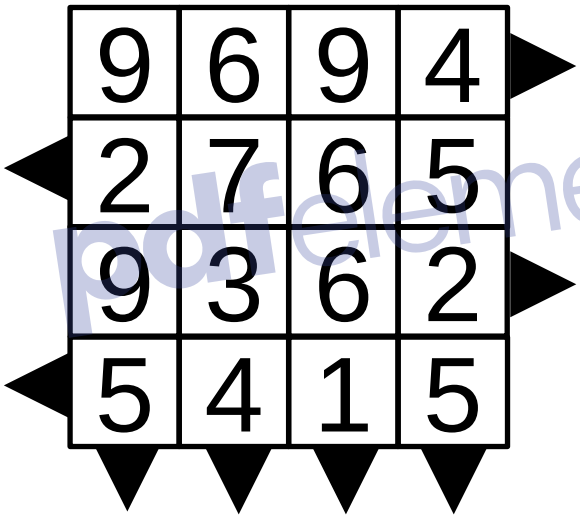


pdfelement

Complexitate? $G =$

Remove Watermark Now

Shear sort

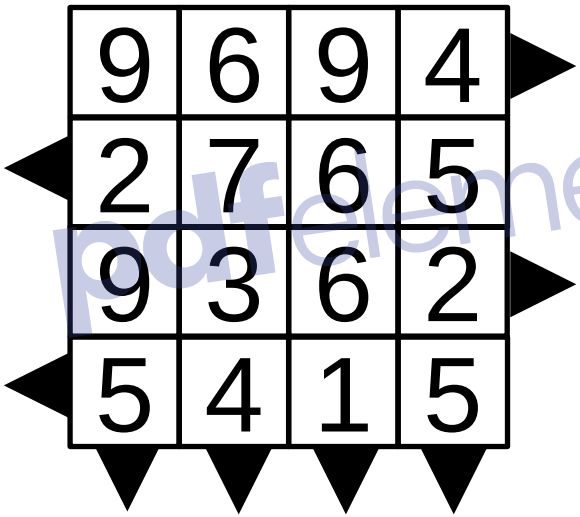


pdfelement

Complexitate? $G = log_2 N * log_2 N$ repetiții

Remove Watermark Now

Shear sort



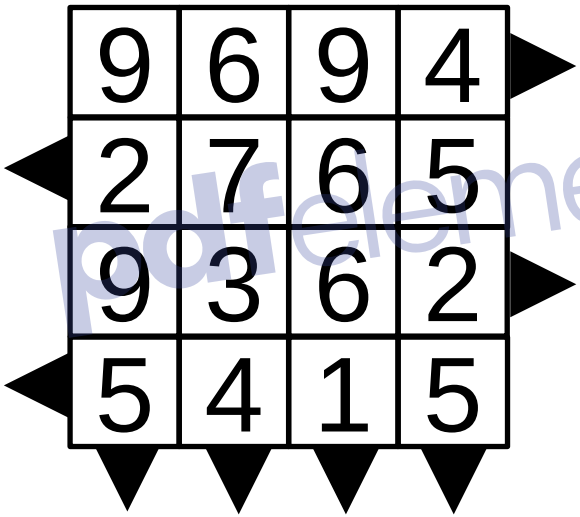
pdfelement

Complexitate? $G = log_2 N * 2 * \sqrt{N}$

$\sqrt{N}$ linii/coloane

Remove Watermark Now

Shear sort



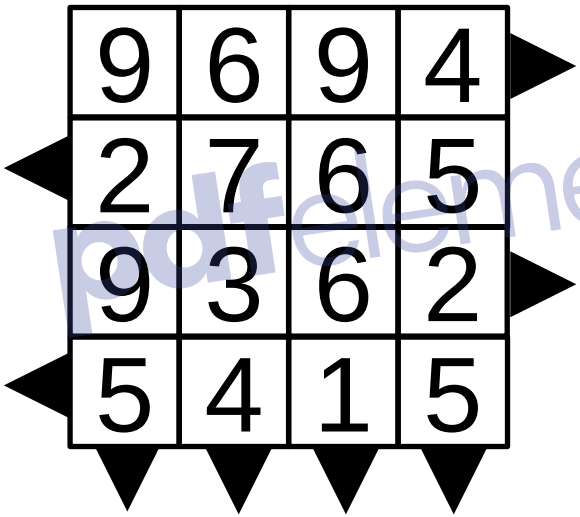
pdfelement

Complexitate? $G = log_2 N * 2 * \sqrt{N} * \sqrt{N} * log_2 \sqrt{N}$

$\sqrt{N} * log_2 \sqrt{N}$ cel mai bun algoritm secvențial de sortare

Remove Watermark Now

Shear sort

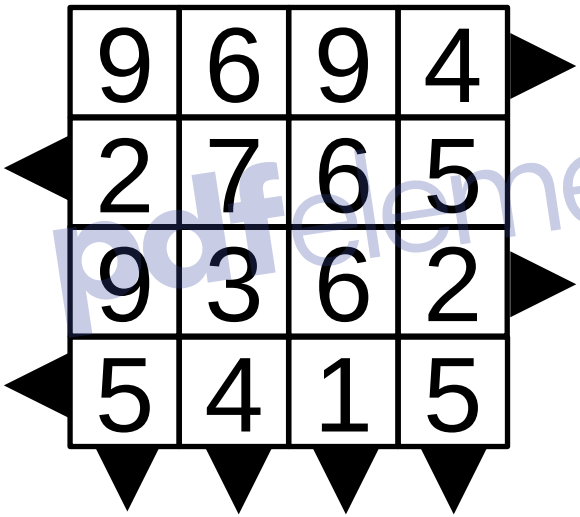


pdfelement

Complexitate? $G = log_2 N * N * log_2 \sqrt{N}$

Remove Watermark Now

Shear sort



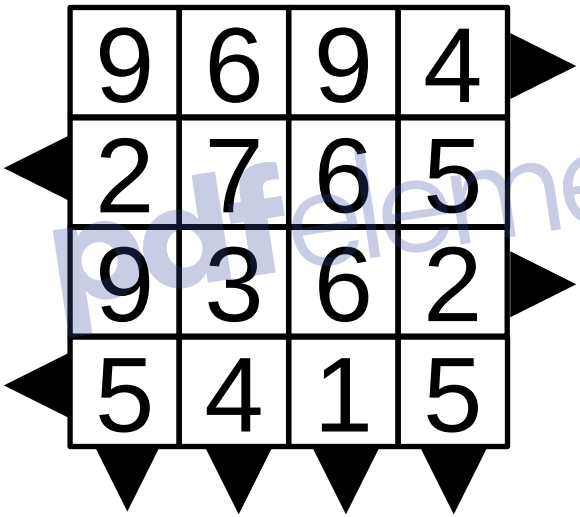
pdfelement

Complexitate? $G = Nlog_2N * log_2\sqrt{N}$

Cel mai bun algoritm secvențial rămâne $Nlog_2N$

Remove Watermark Now

Shear sort

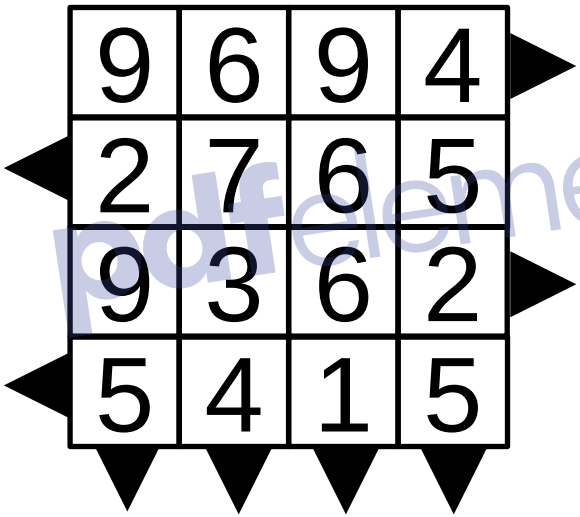


pdfelement

Cum paralelizăm?

Remove Watermark Now

Shear sort

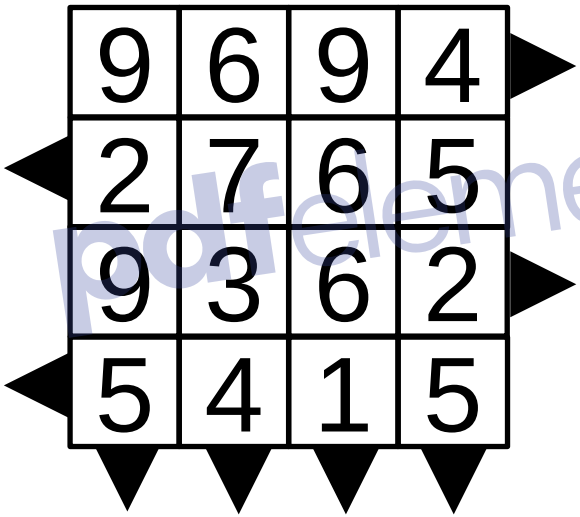


pdfelement

Cum paralelizăm?
Toate liniile în paralel, barieră, apoi toate coloanele, apoi barieră, și repetăm.

Remove Watermark Now

Shear sort

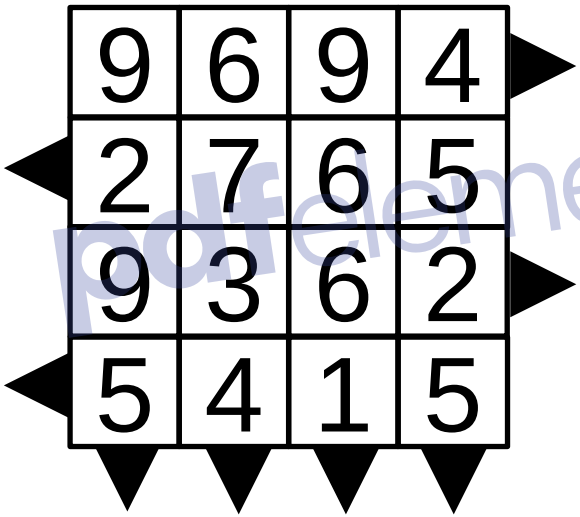


pdfelement

Cum paralelizăm?
Toate liniile în paralel, barieră, apoi toate coloanele, apoi barieră, și repetăm.

Remove Watermark Now

Shear sort



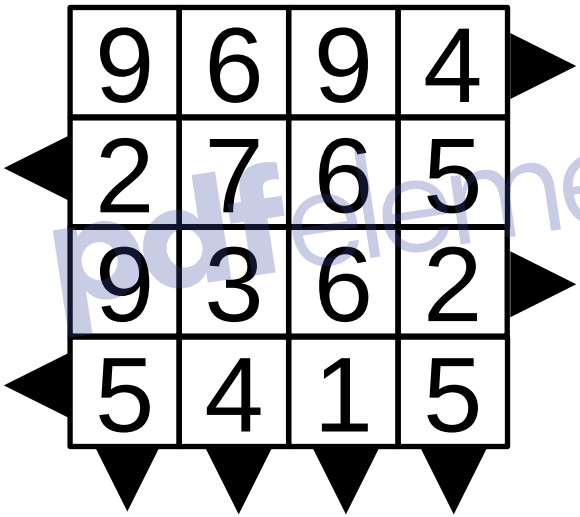
pdfelement

Complexitate versiune paralelă?

$$G = log_2 N * 2 * \sqrt{N} * \sqrt{N} * log_2 \sqrt{N}$$

Remove Watermark Now

Shear sort



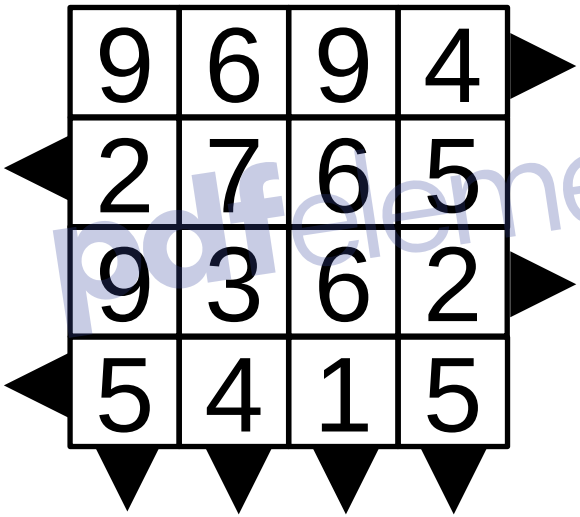
pdfelement

Complexitate versiune paralelă? N = P

$$T = log_2 N * 2 * \sqrt{N} * log_2 \sqrt{N}$$

Remove Watermark Now

Shear sort



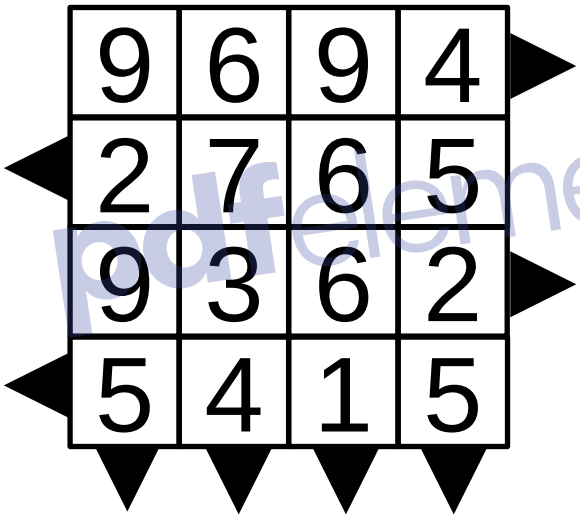
pdfelement

Complexitate versiune paralelă? N = P

$$T = \sqrt{N}\, log_2 \sqrt{N} * log_2 N$$

Remove Watermark Now

Shear sort



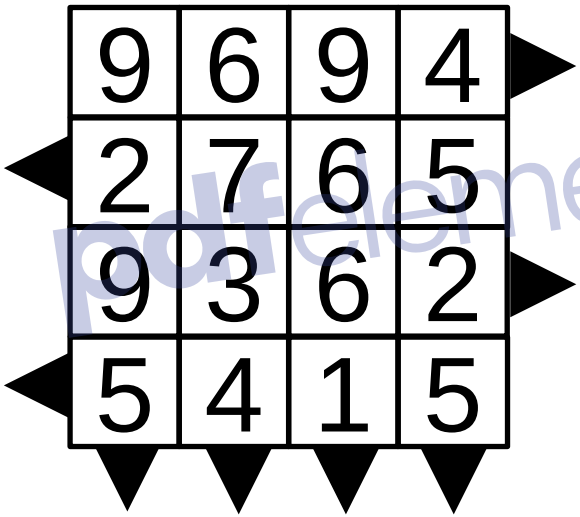
pdfelement

Complexitate versiune paralelă?

$$T = log_2 N * 2 * \frac{\sqrt{N}}{P} * \sqrt{N} * log_2 \sqrt{N}$$

Remove Watermark Now

Shear sort



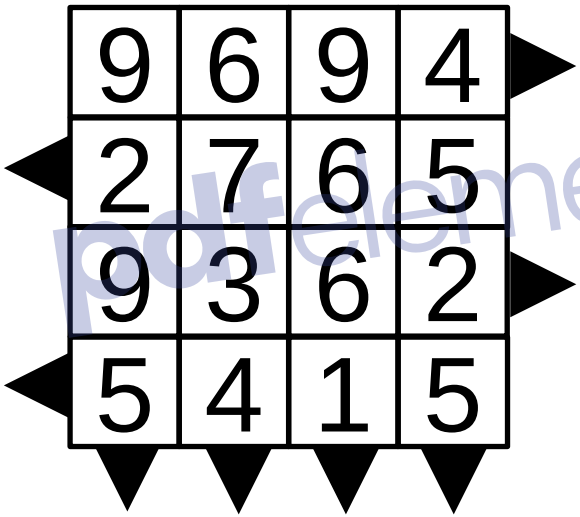
pdfelement

Complexitate versiune paralelă?

$$T = \frac{N}{P} log_2 N * log_2 \sqrt{N}$$

Remove Watermark Now

Shear sort



pdfelement

Speedup?

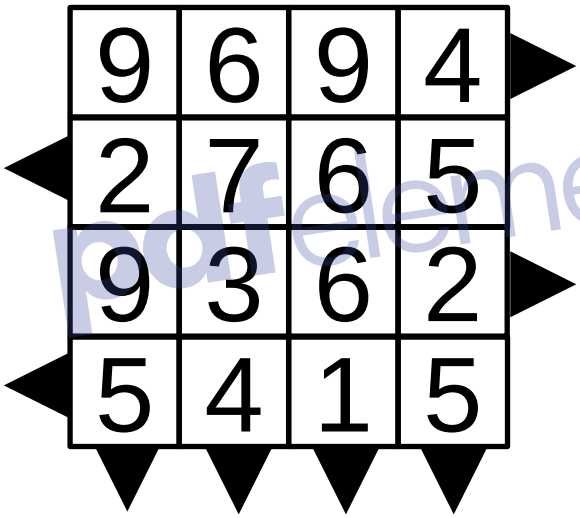
$$T = \frac{N}{P} log_2 N \ * \ log_2 \sqrt{N}$$

$$G = \ N log_2 N$$

Remove Watermark Now

Shear sort

$$S = \frac{Nlog_2N}{\frac{N}{P}log_2N \ * \ log_2\sqrt{N}}$$



pdfelement

Speedup?

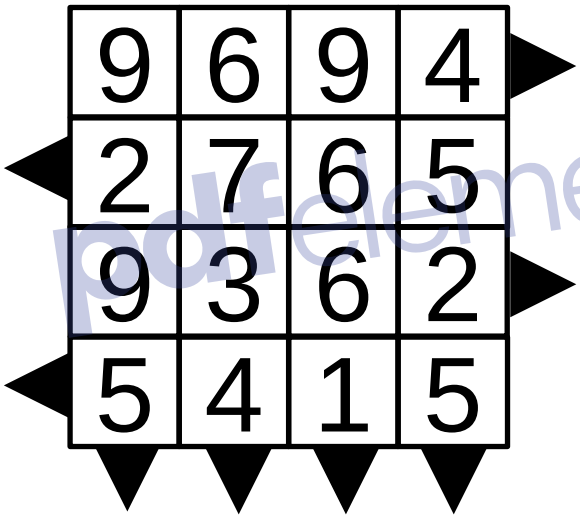
$$T = \frac{N}{P}log_2N \ * \ log_2\sqrt{N}$$

$$G = Nlog_2N$$

Remove Watermark Now

Shear sort

$$S = \frac{P}{log_2\sqrt{N}}$$



pdfelement

Speedup?

$$T = \frac{N}{P} log_2 N \ * \ log_2\sqrt{N}$$

$$G = \ Nlog_2 N$$

Remove Watermark Now

Rank Sort

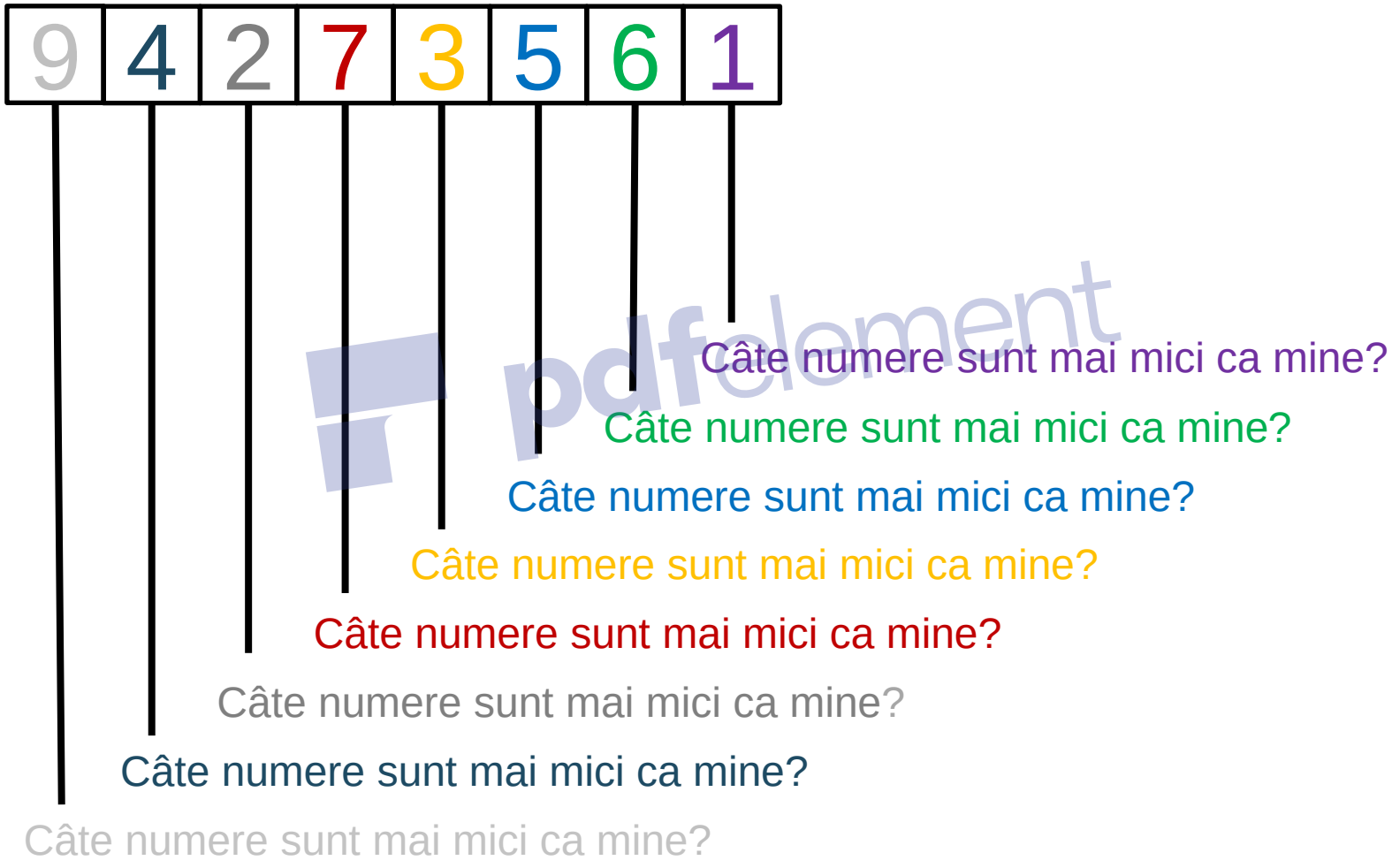
9	4	2	7	6	5	6	1

pdfelement

1	2	4	5	6	6	7	9

Remove Watermark Now

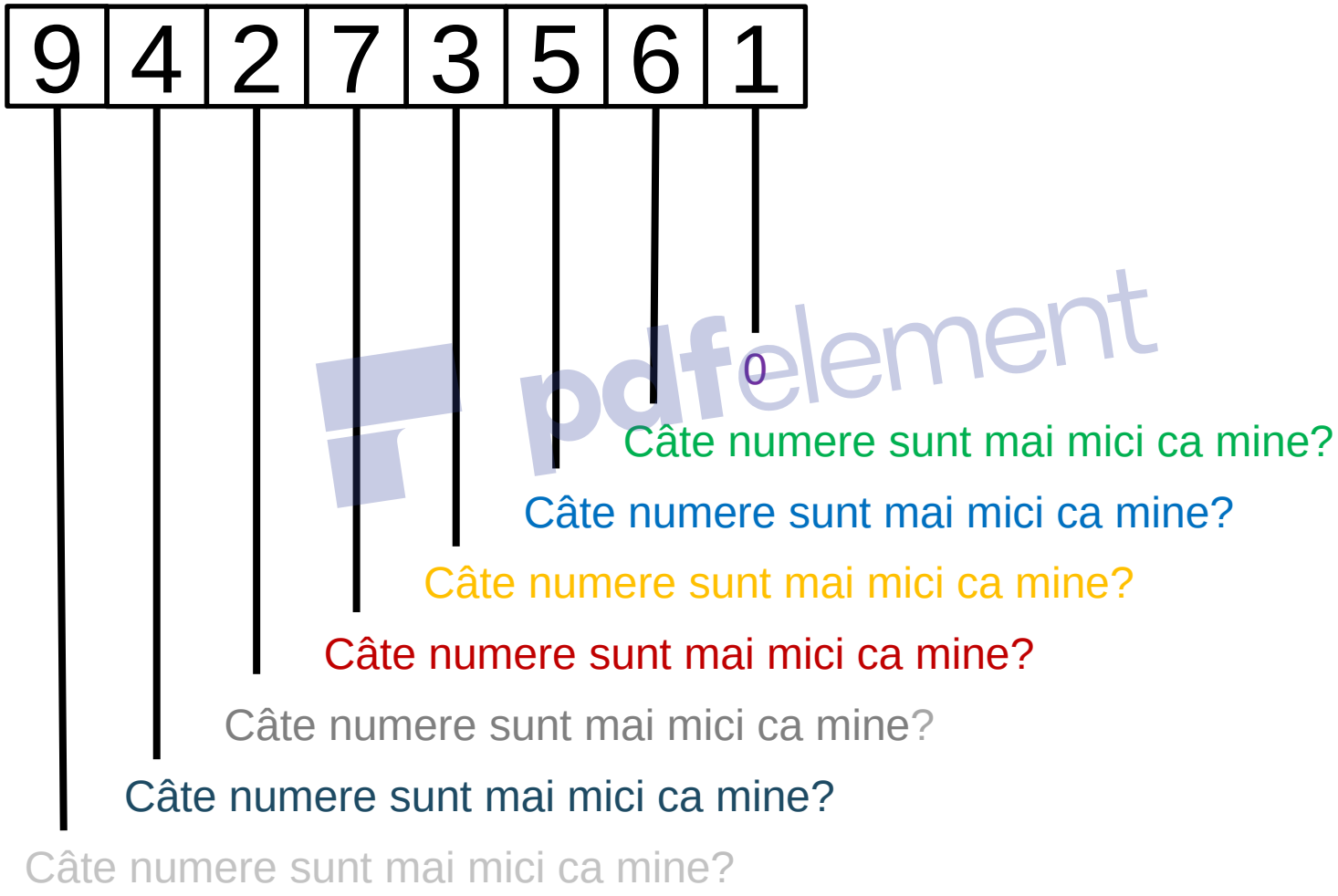
Rank Sort



pdfelement

Remove Watermark Now

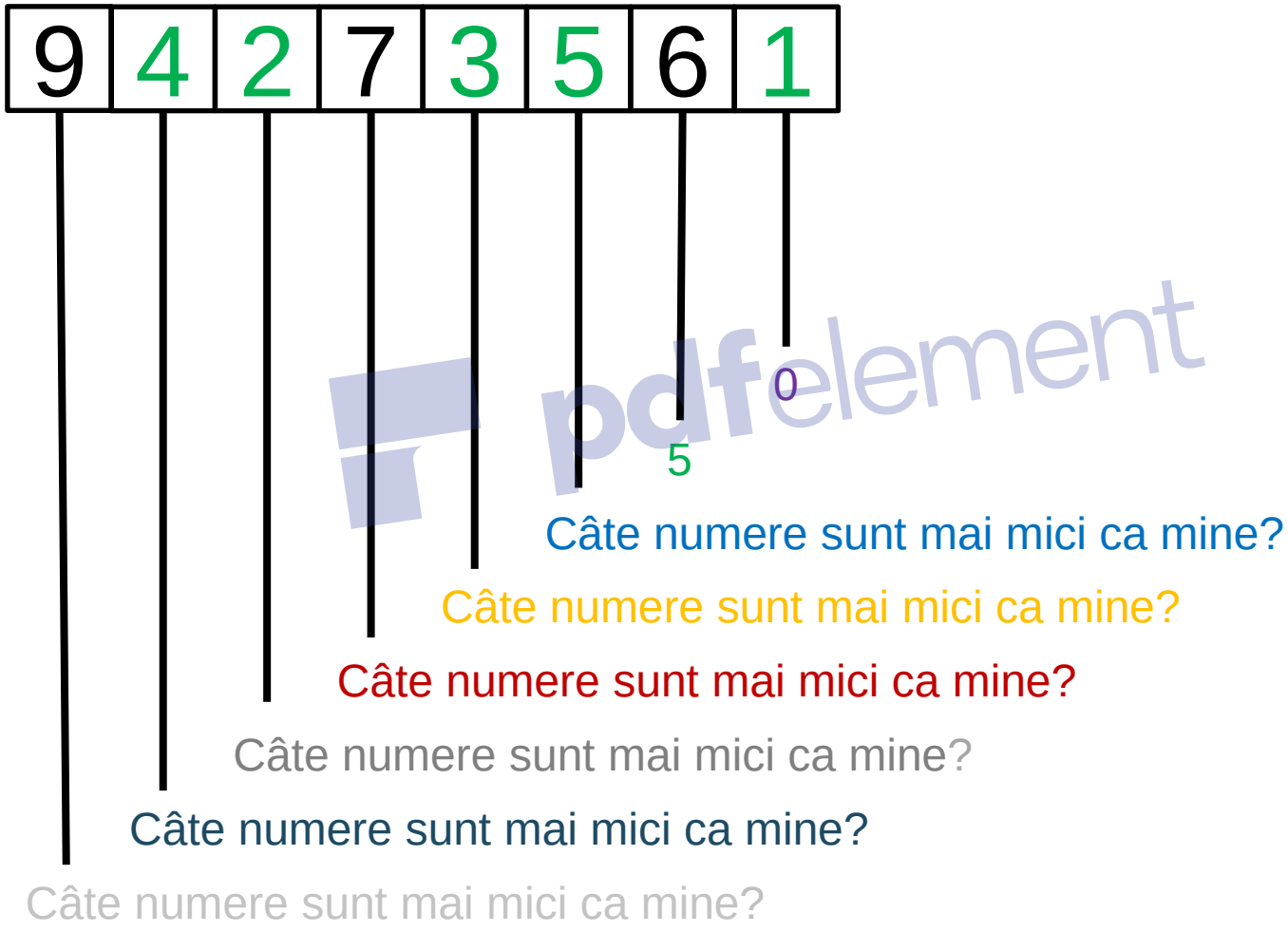
Rank Sort



pdfelement

Remove Watermark Now

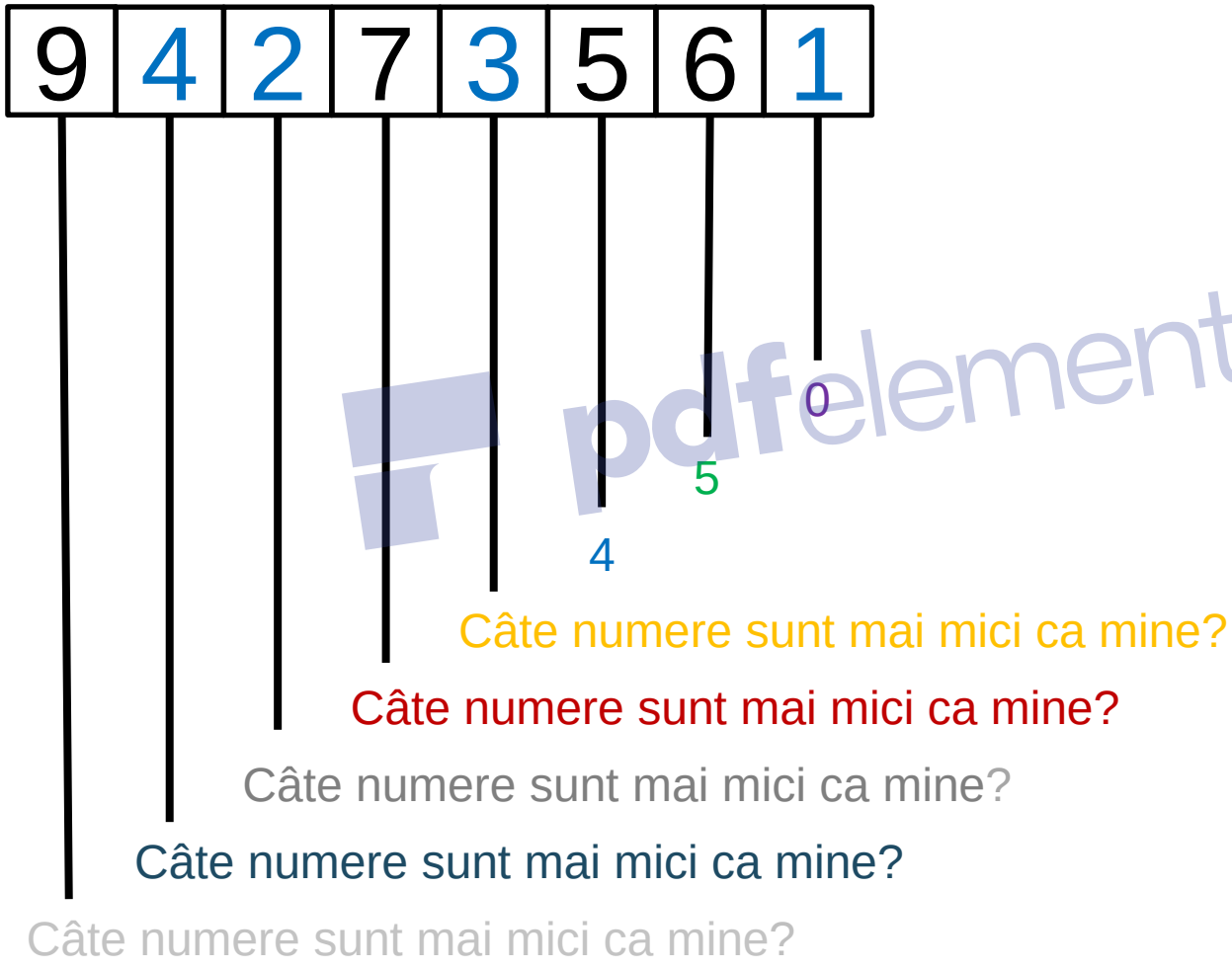
Rank Sort



pdfelement

Remove Watermark Now

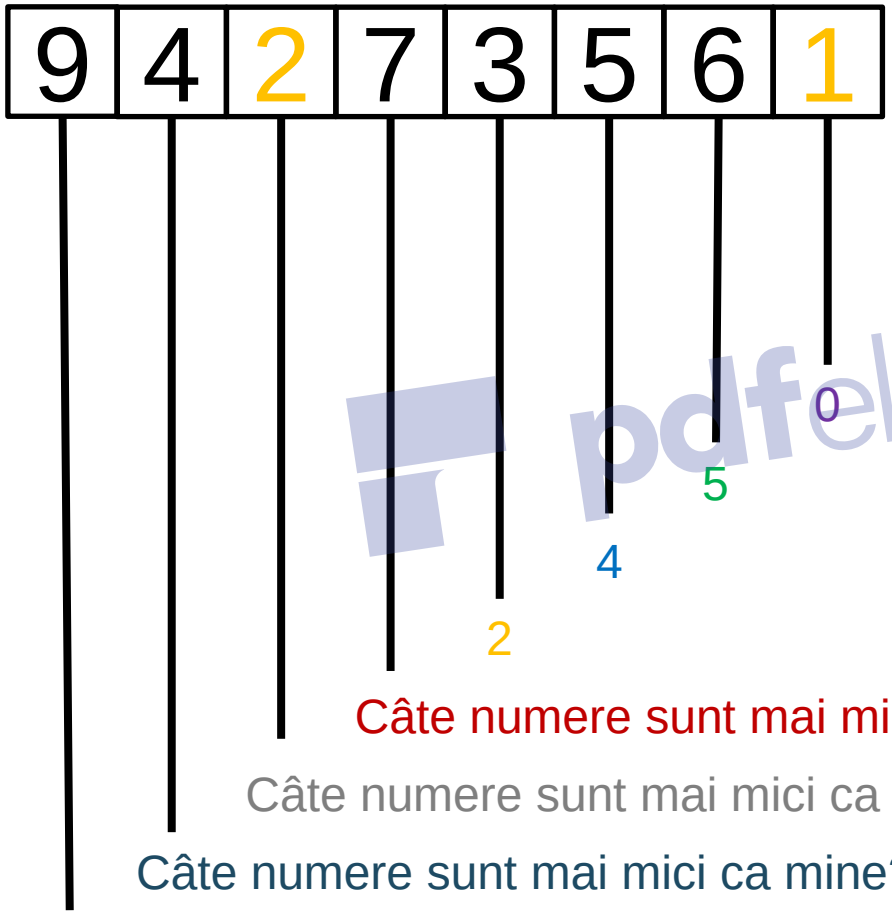
Rank Sort



pdfelement

Remove Watermark Now

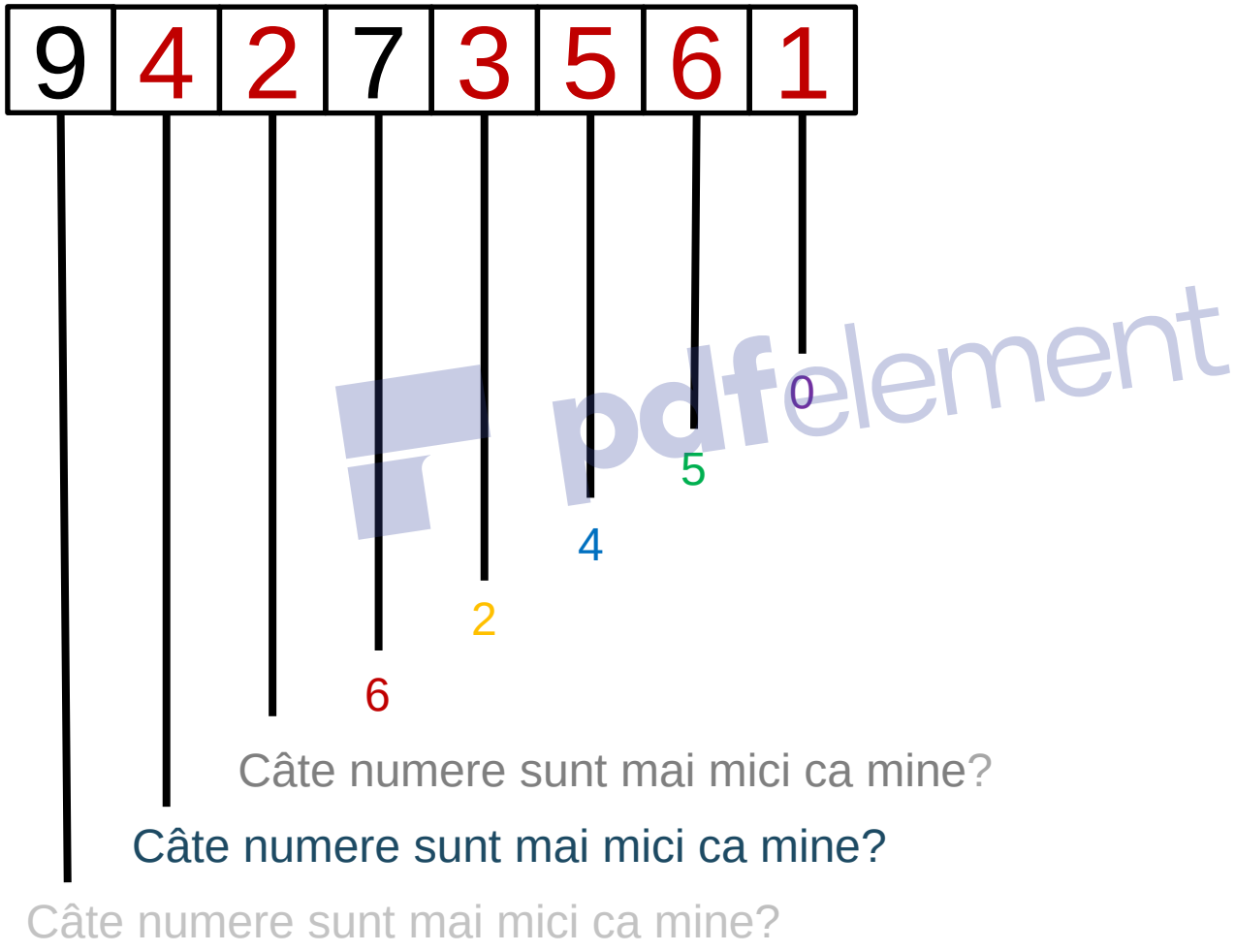
Rank Sort



pdfelement

Remove Watermark Now

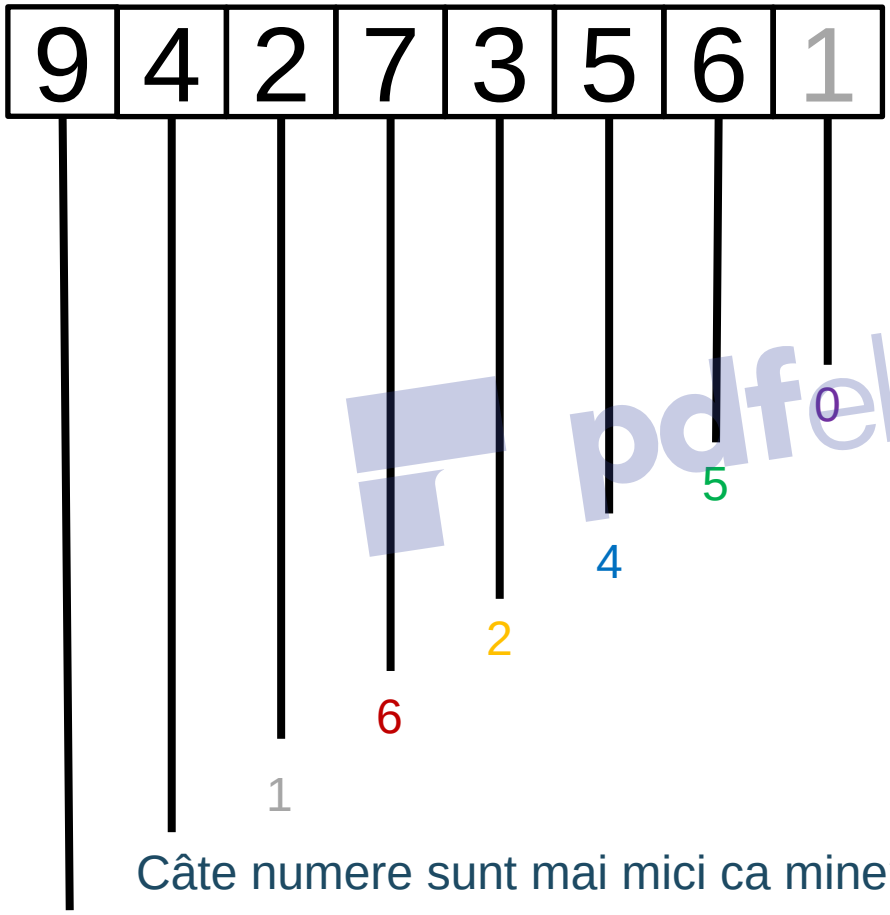
Rank Sort



pdfelement

Remove Watermark Now

Rank Sort

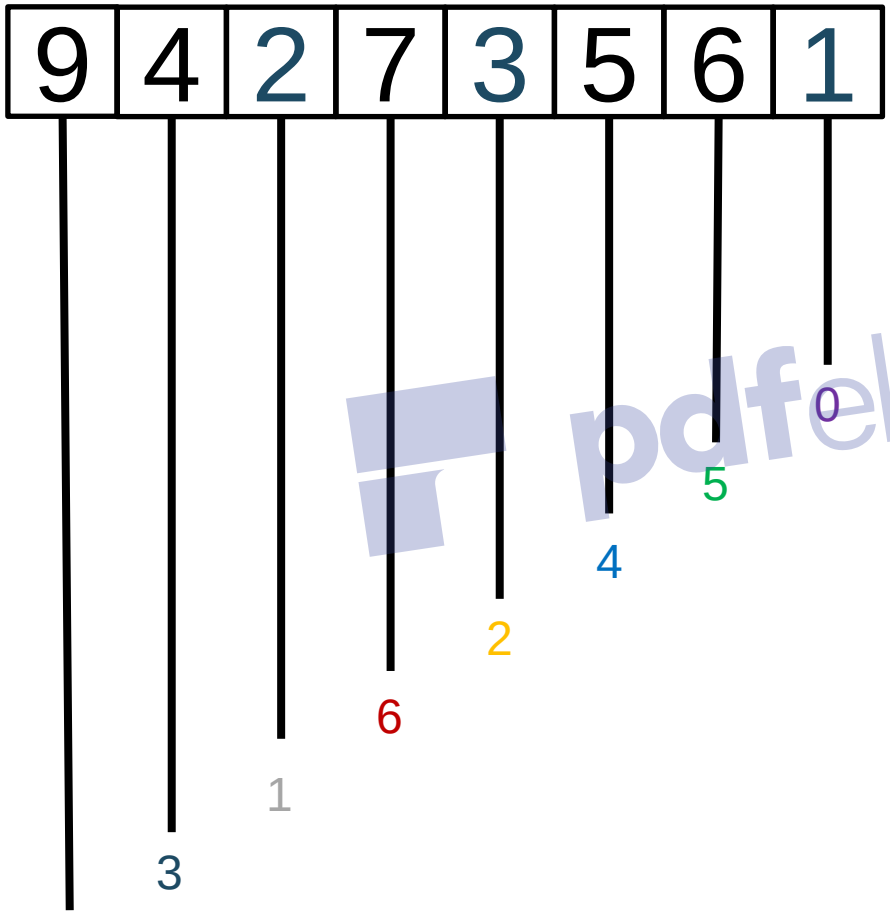


Câte numere sunt mai mici ca mine?

pdfelement

Remove Watermark Now

Rank Sort

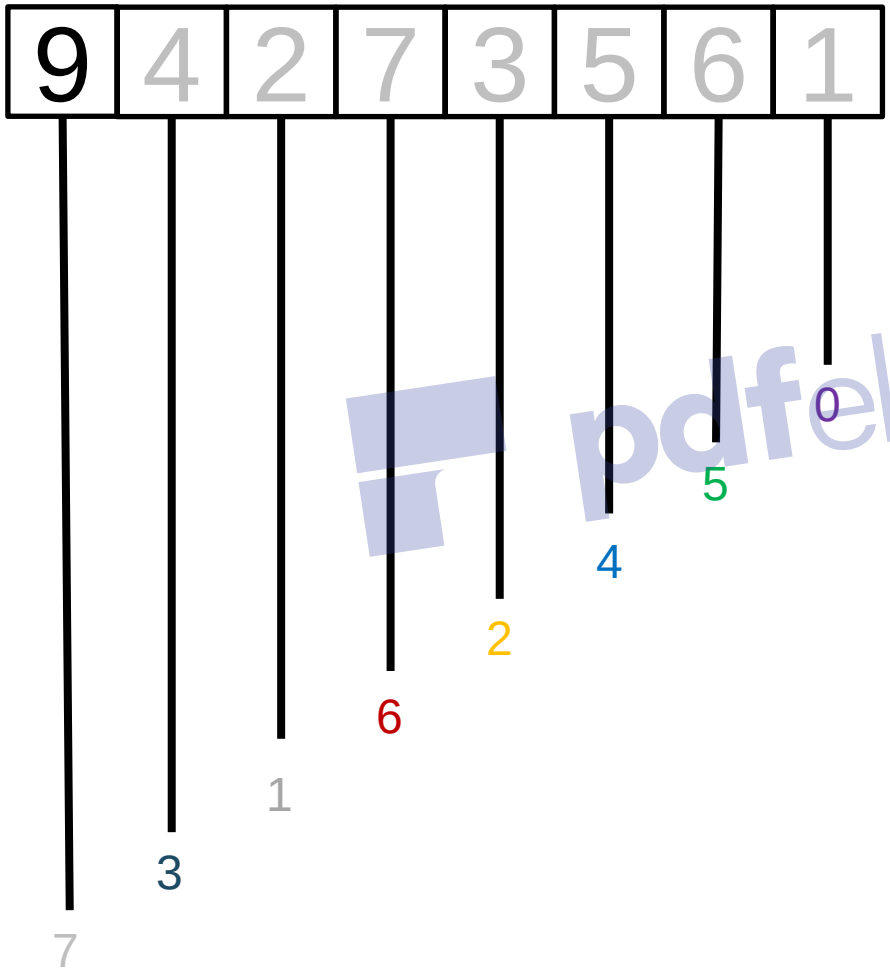


pdfelement

Câte numere sunt mai mici ca mine?

Remove Watermark Now

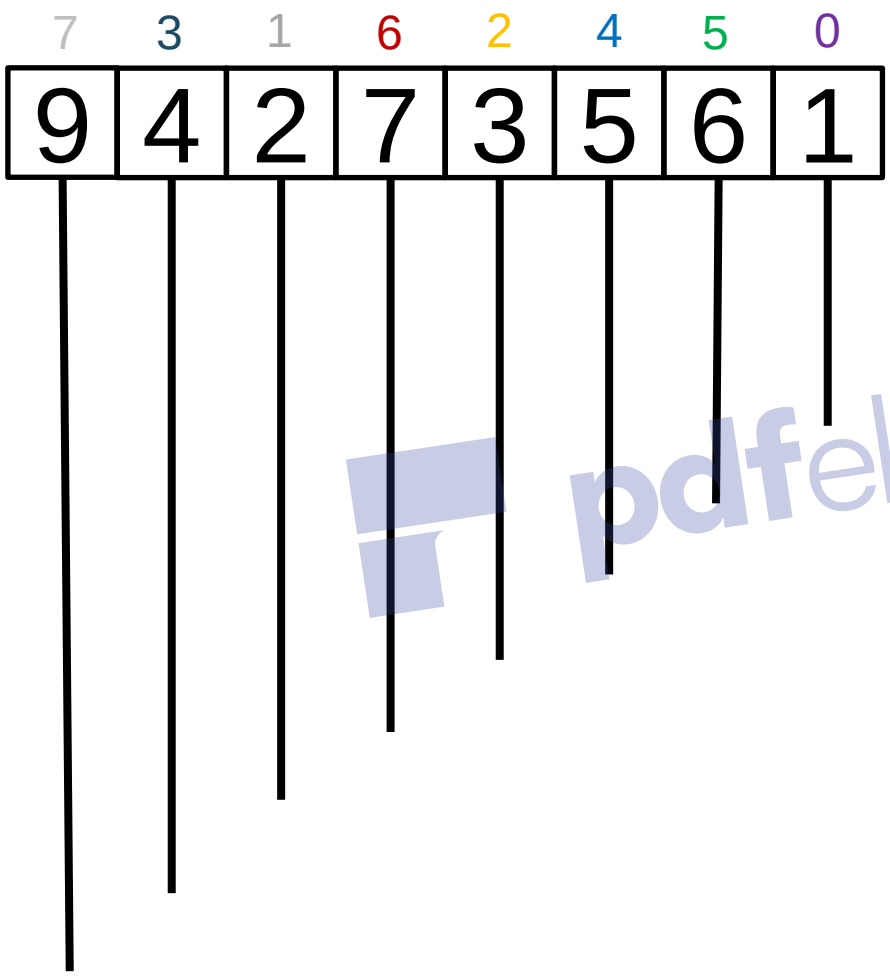
Rank Sort



pdfelement

Remove Watermark Now

Rank Sort

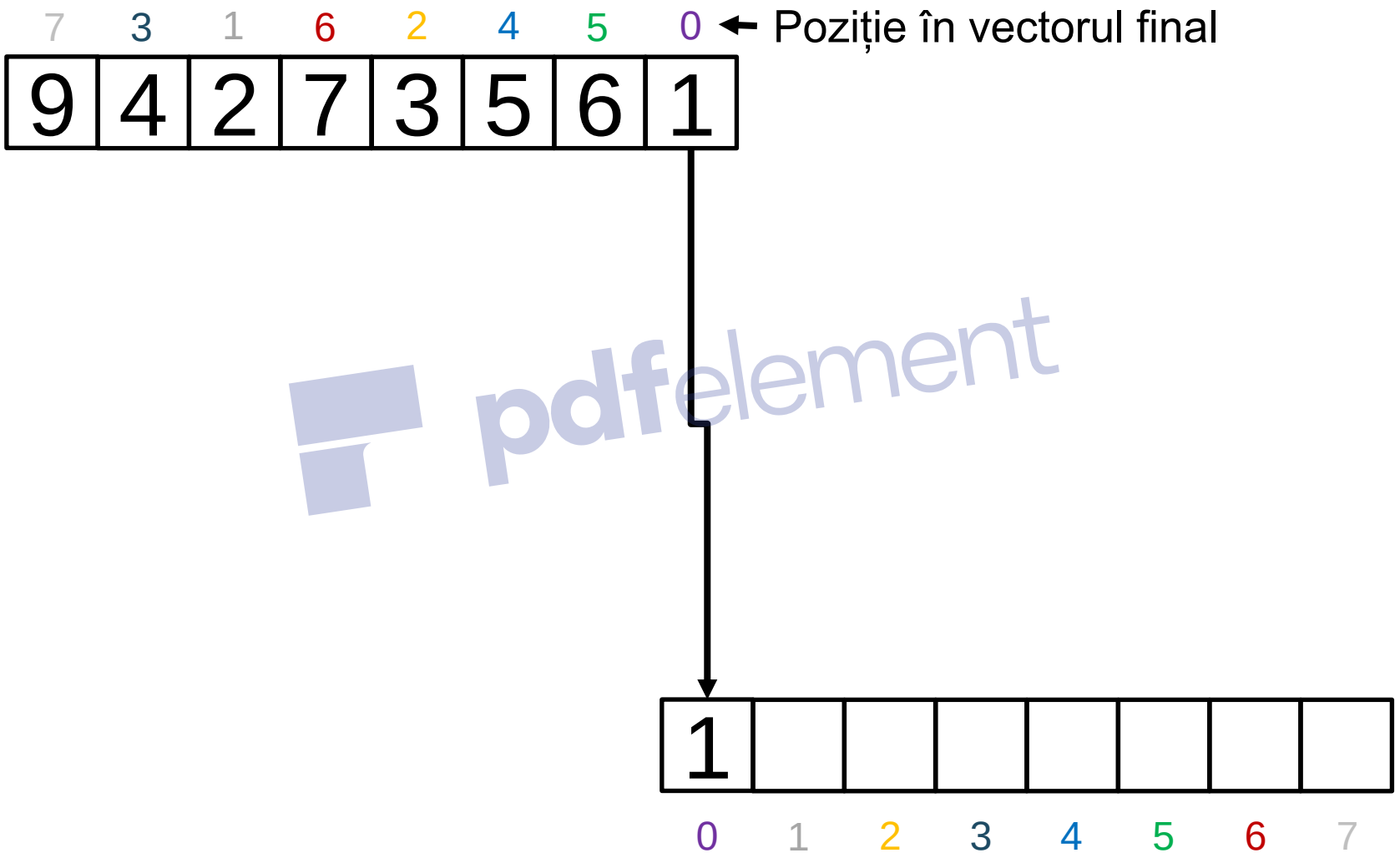


Poziții în vectorul final

pdfelement

Remove Watermark Now

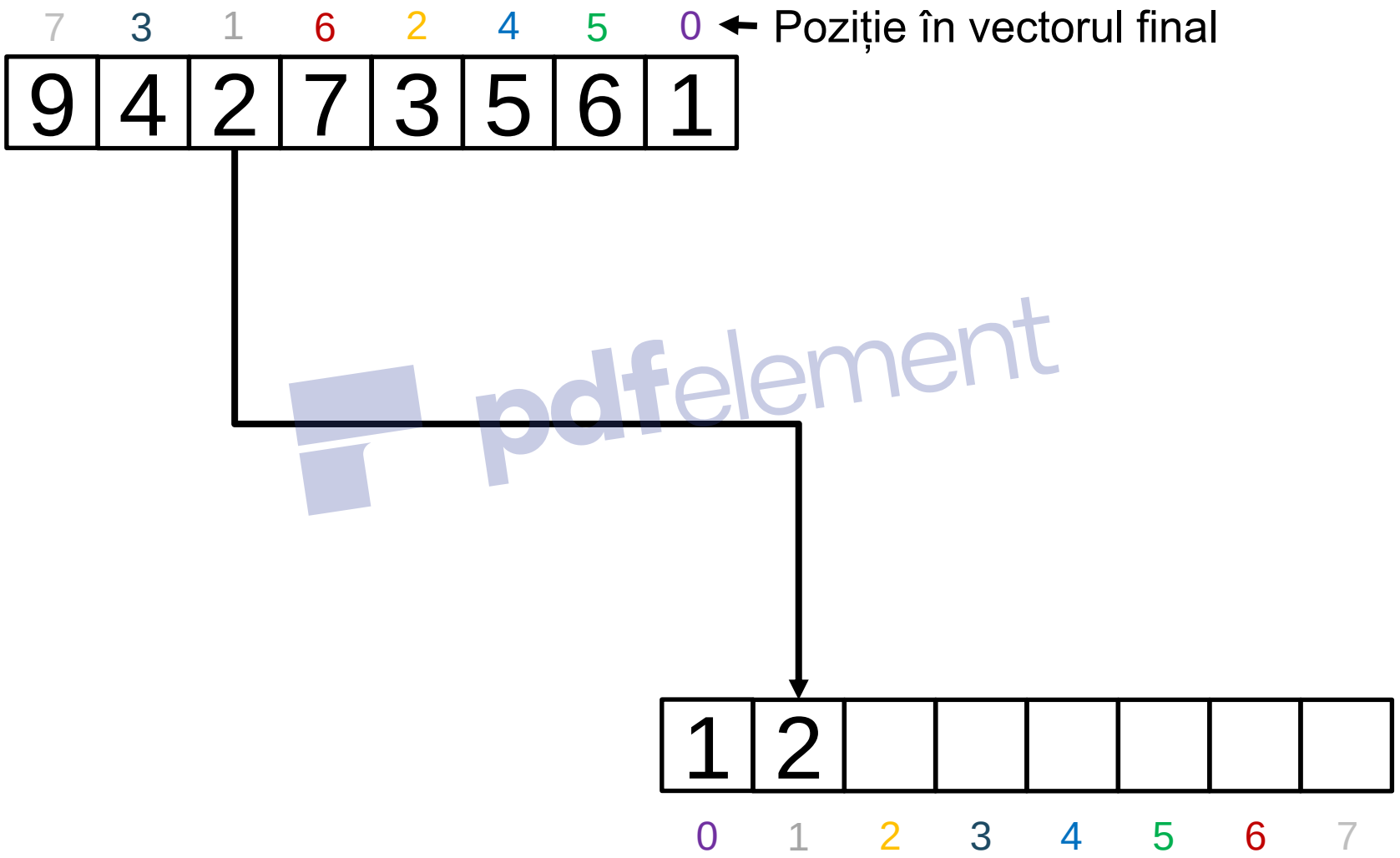
Rank Sort



pdfelement

Remove Watermark Now

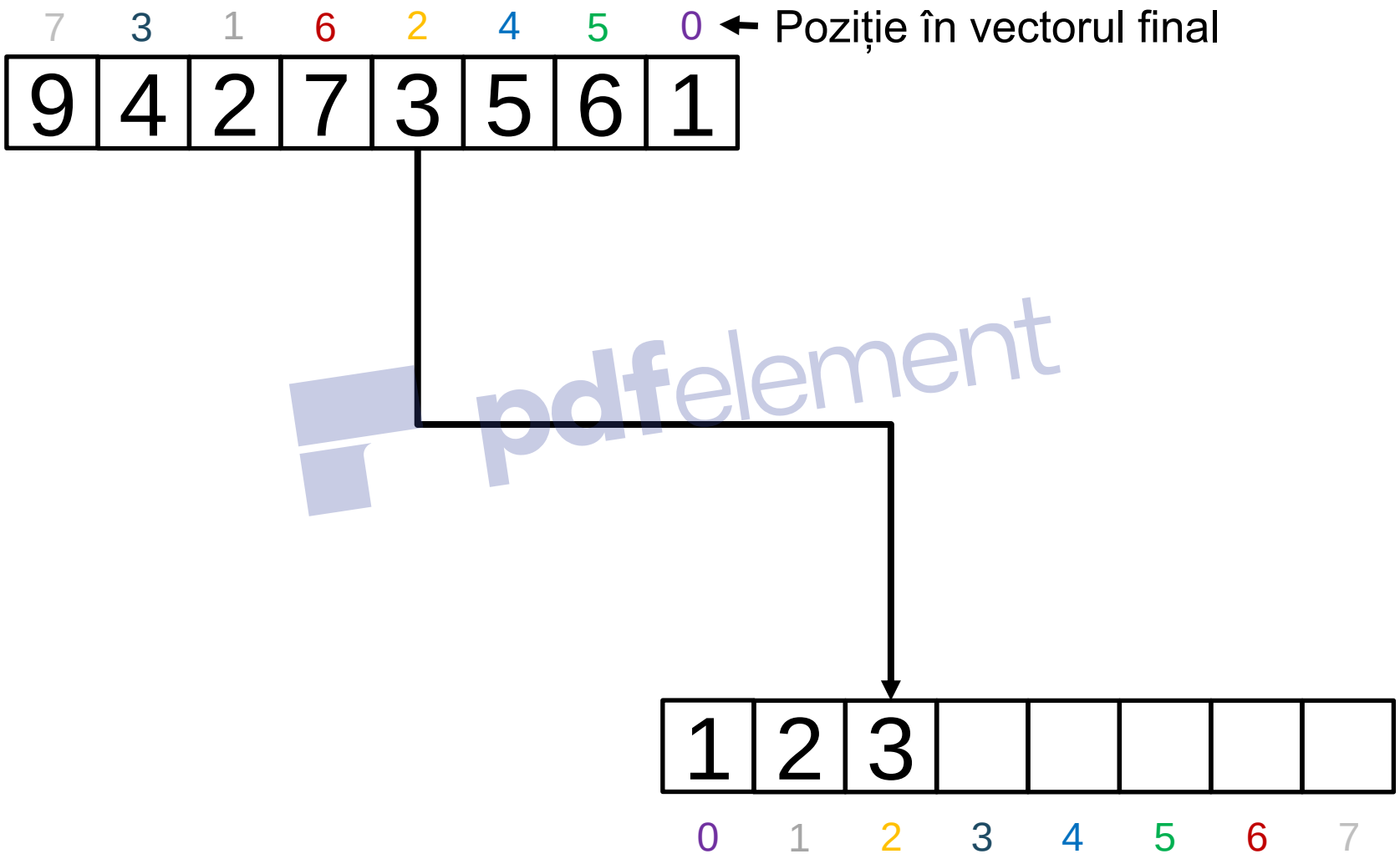
Rank Sort



pdfelement

Remove Watermark Now

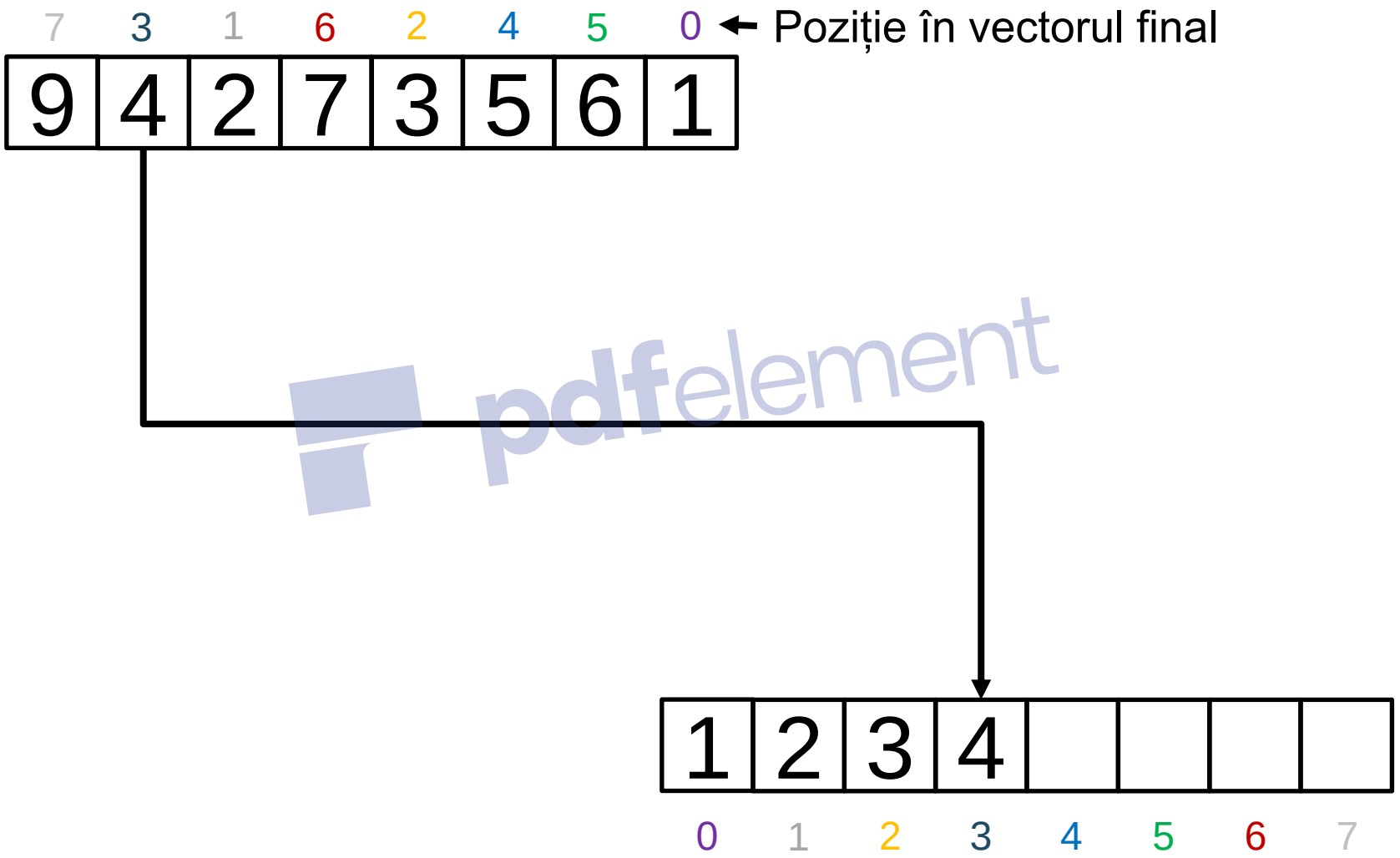
Rank Sort



pdfelement

Remove Watermark Now

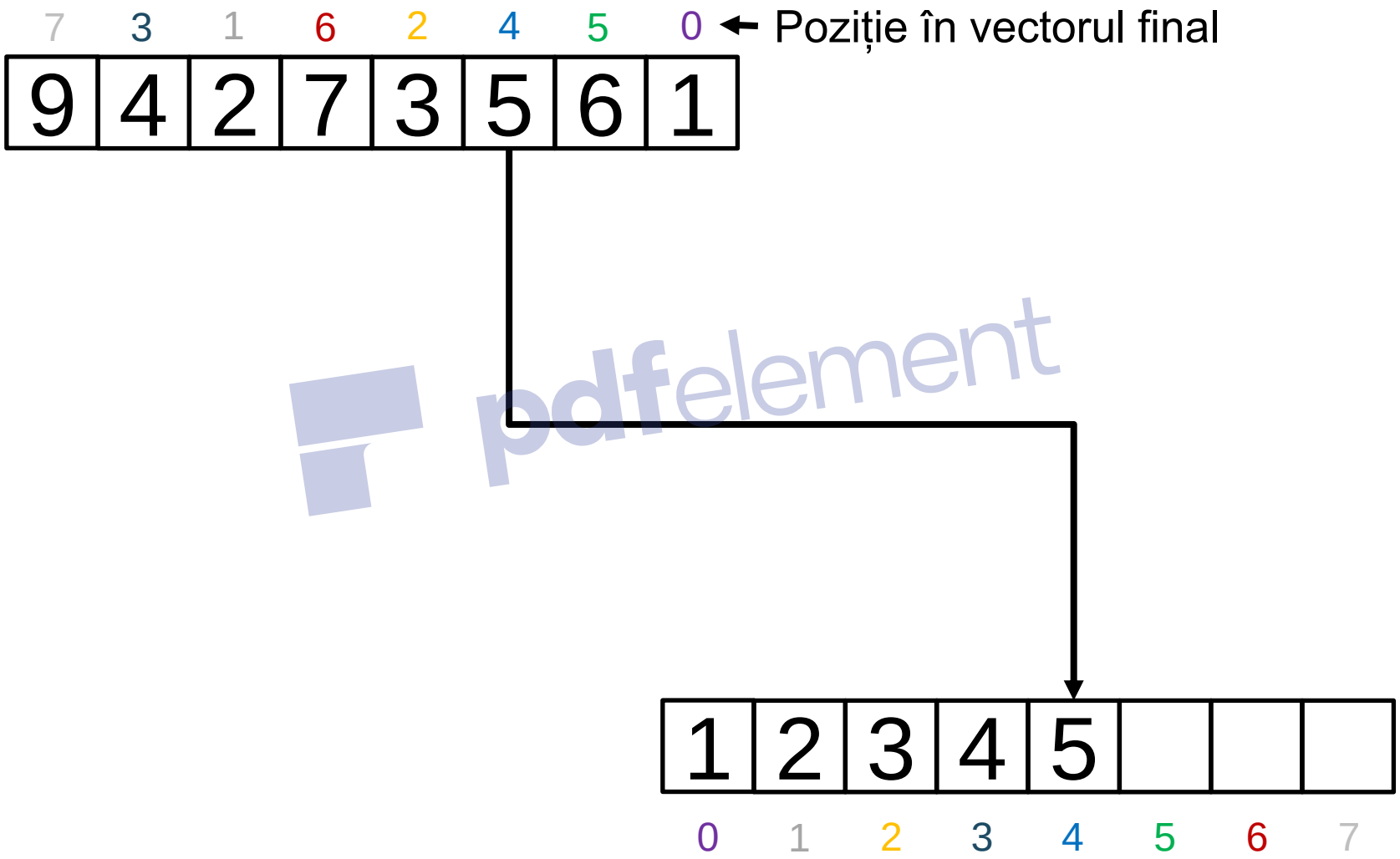
Rank Sort



pdfelement

Remove Watermark Now

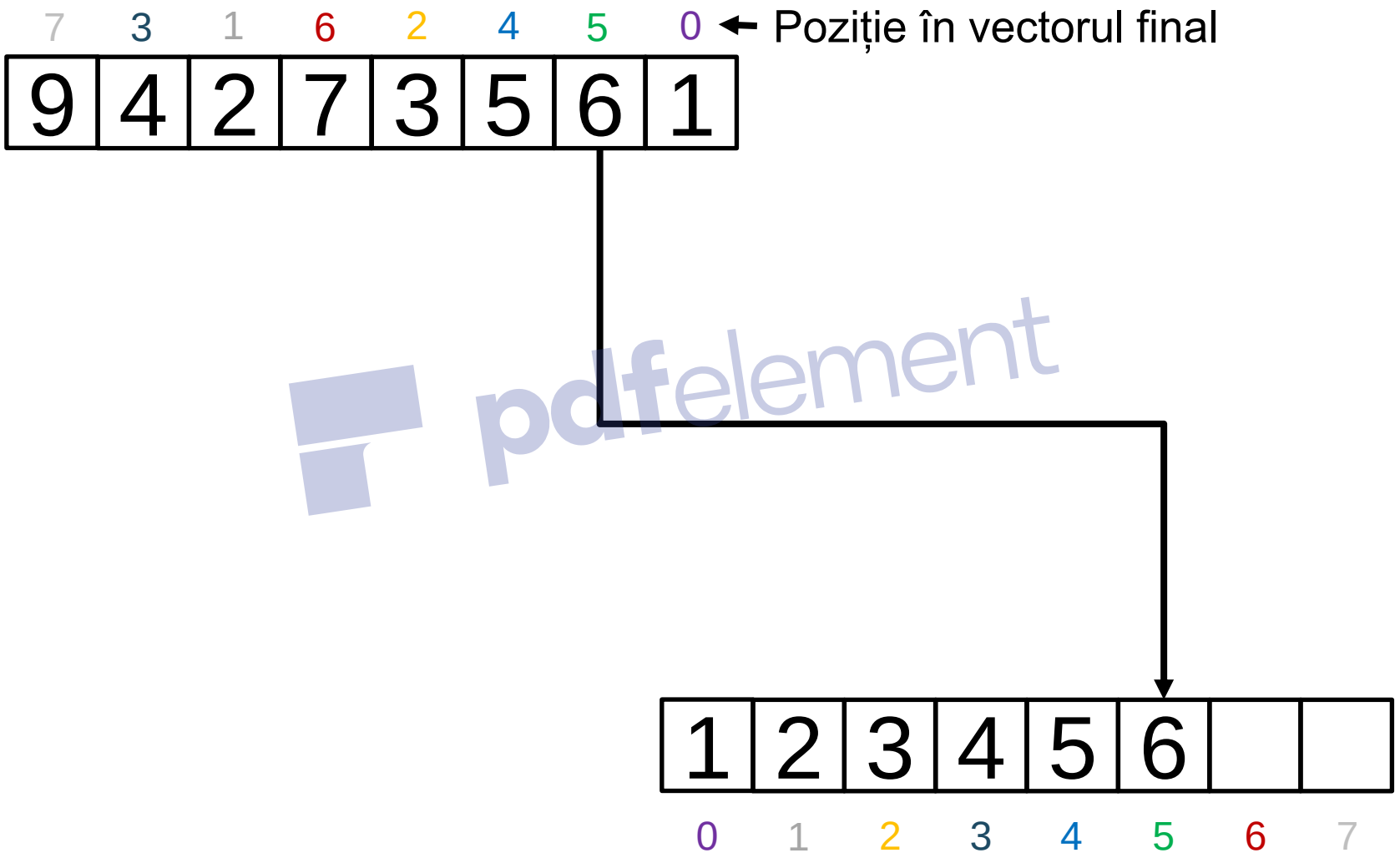
Rank Sort



pdfelement

Remove Watermark Now

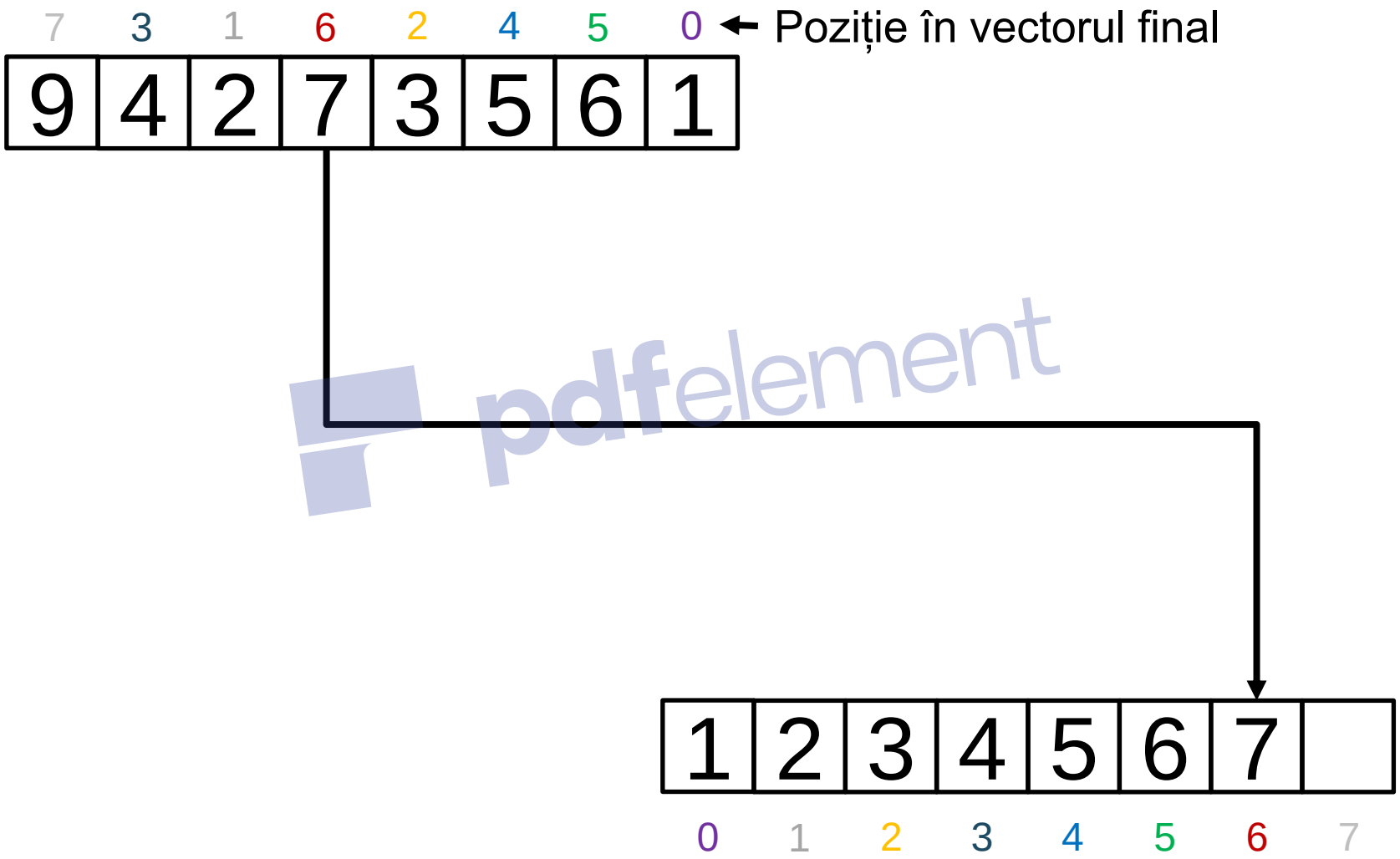
Rank Sort



pdfelement

Remove Watermark Now

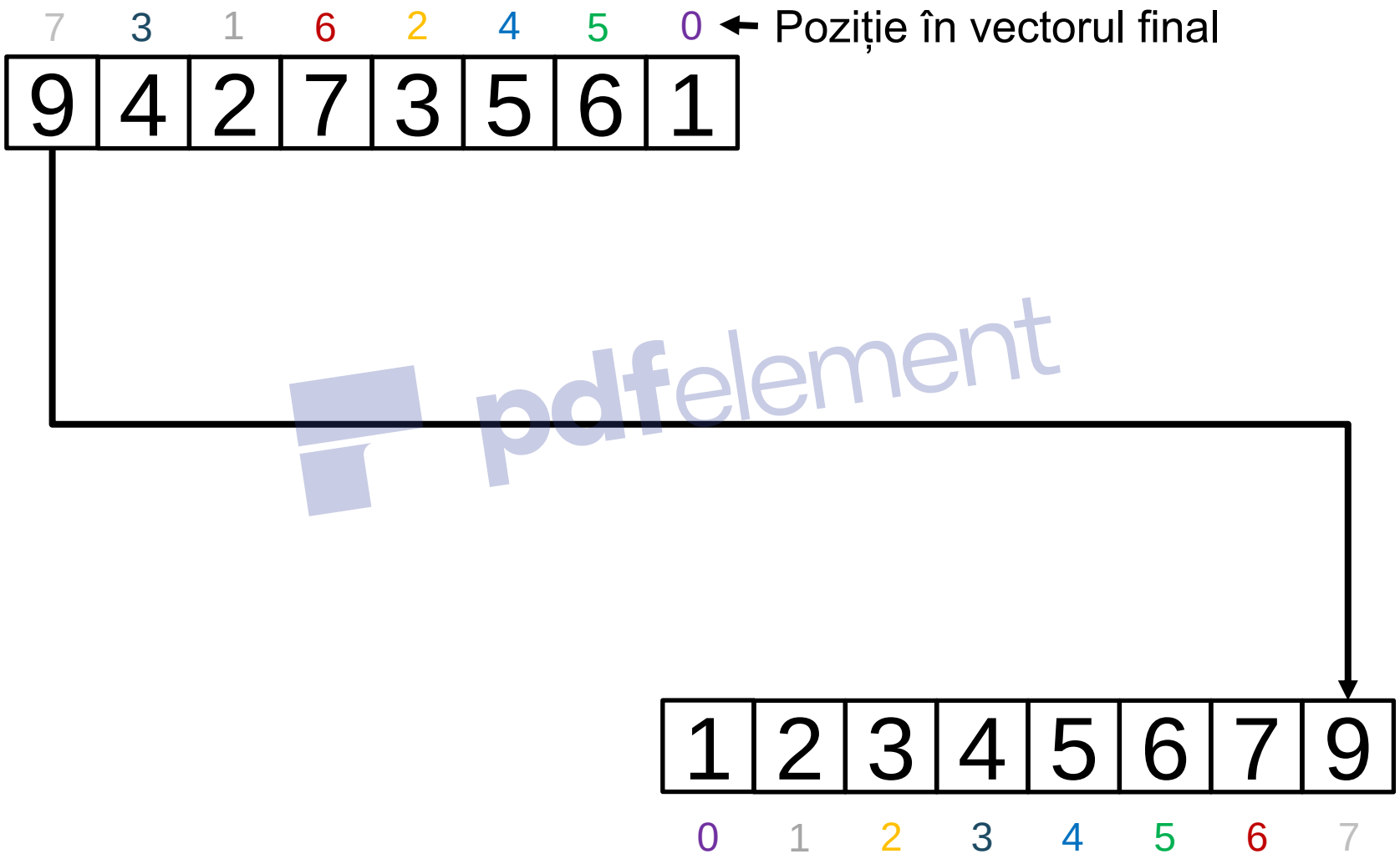
Rank Sort



pdfelement

Remove Watermark Now

Rank Sort



pdfelement

Remove Watermark Now

Parallel Rank Sort

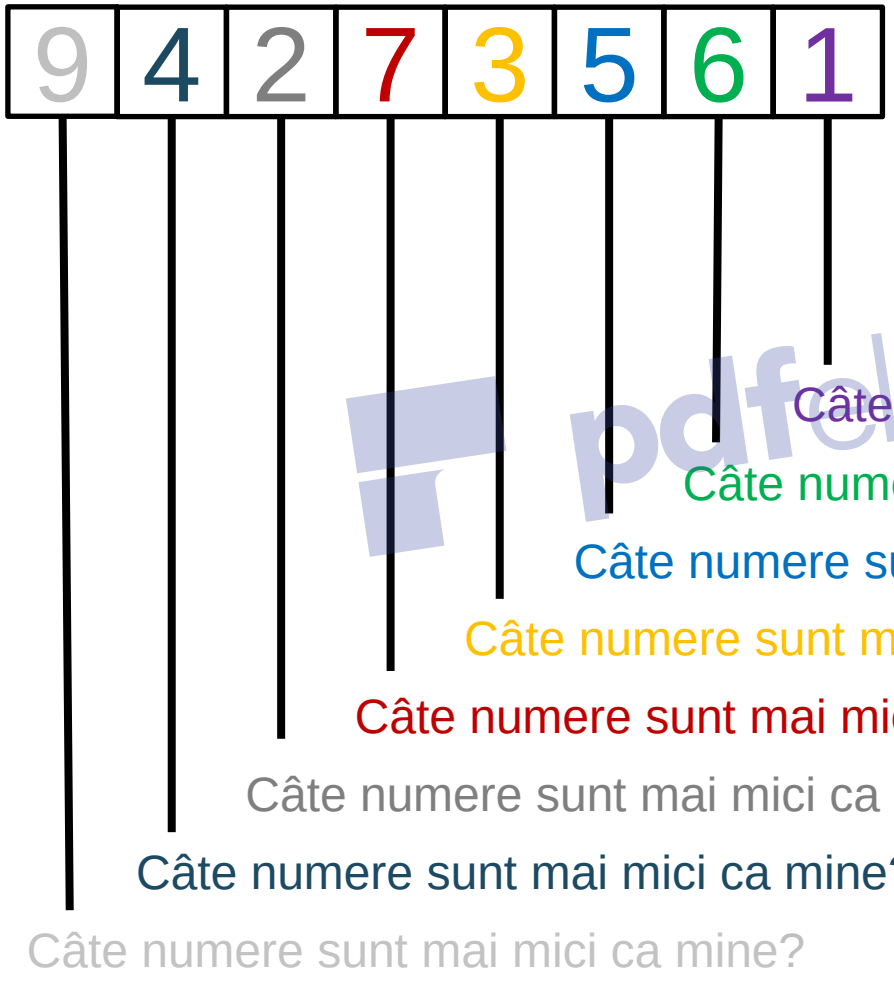
9	4	2	7	6	5	6	1

pdfelement

1	2	4	5	6	6	7	9

Remove Watermark Now

Parallel Rank Sort

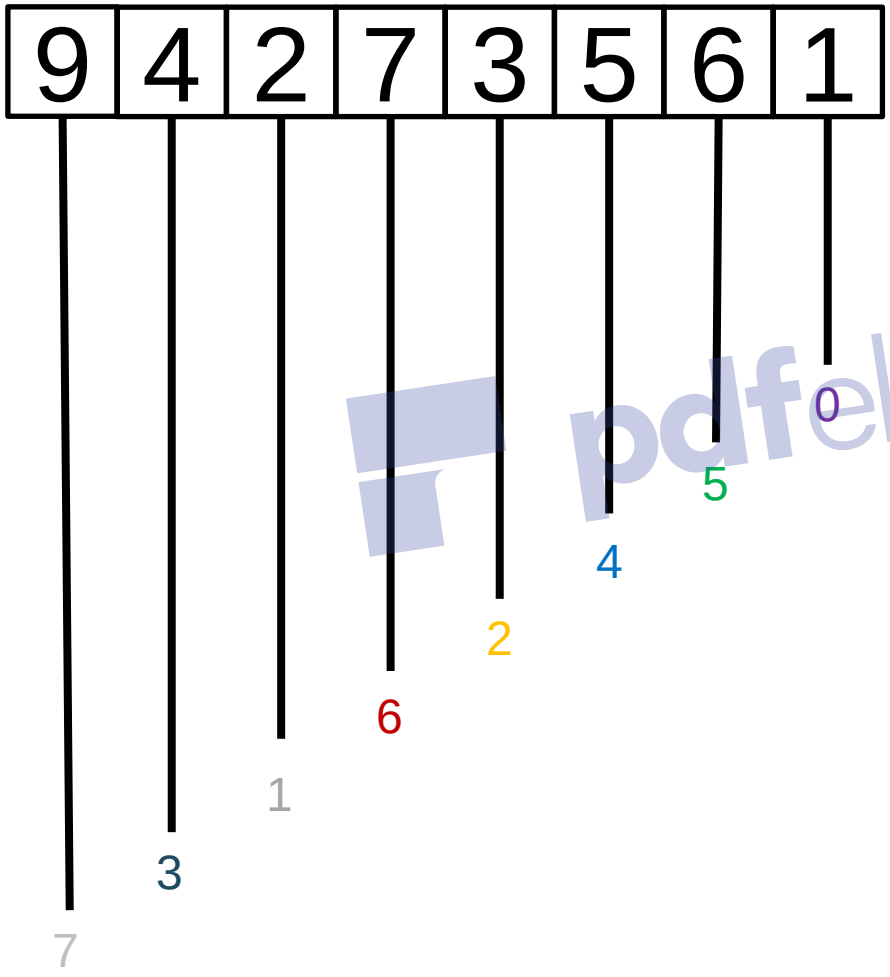


Toate întrebările pot fi răspunse în paralel.

pdfelement

Remove Watermark Now

Parallel Rank Sort



Toate întrebările pot fi răspunse în paralel.

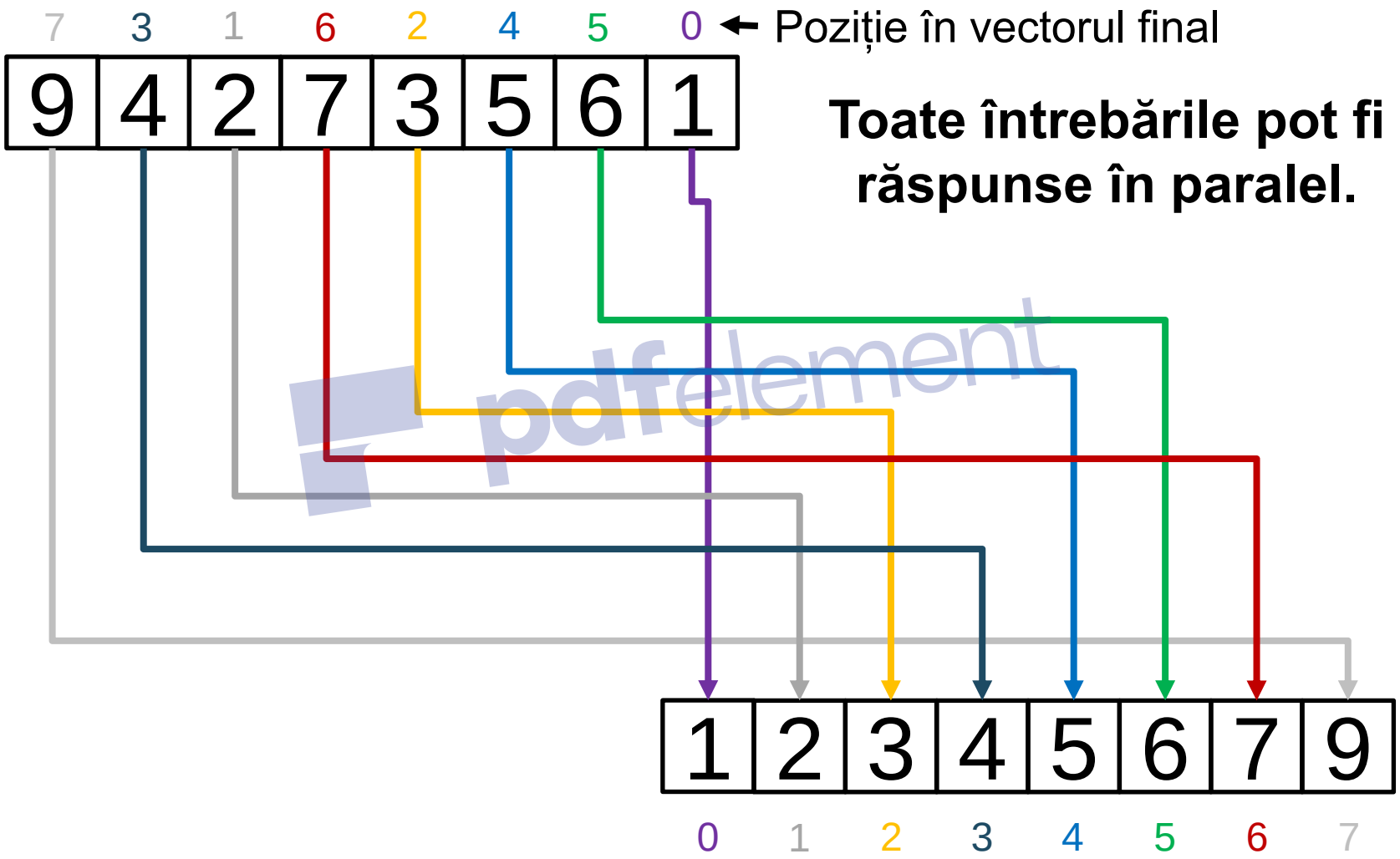
pdfelement

Remove Watermark Now

Parallel Rank Sort

Remove Watermark Now

Parallel Rank Sort



pdfelement

Remove Watermark Now

Inmultiri de matrici

pdfelement

Remove Watermark Now

Matrix multiply

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

pdfelement

Remove Watermark Now

Matrix multiply

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

pdfelement

Complexitate?

Remove Watermark Now

Matrix multiply

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

pdfelement

Complexitate?

$$G = N^3$$

Remove Watermark Now

Matrix multiply

Atenție avem N^2 elemente

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

pdfelement

Complexitate?

$$G = N^3$$

Remove Watermark Now

Matrix multiply

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

pdfelement

Remove Watermark Now

Matrix multiply

Complexitate?

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

pdfelement

Remove Watermark Now

Matrix multiply

Complexitate?

$$T = \frac{N^3}{P}$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

pdfelement

Remove Watermark Now

Matrix multiply

Speedup?

$$T = \frac{N^3}{P}$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

pdfelement

Remove Watermark Now

Matrix multiply

Speedup?

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

$$S = \frac{N^3}{\frac{N^3}{P}}$$

pdfelement

Remove Watermark Now

Matrix multiply

Speedup?

$$S = P$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

pdfelement

Remove Watermark Now

Super-linear speedup?

pdfelement

S > P?

Remove Watermark Now

- *"... the effort expended on achieving high parallel processing rates is wasted unless it is accompanied by achievements in sequential processing rates of very nearly the same magnitude."*

Amdhal, 1967

pdfelement

- *"... speedup should be measured by scaling the problem to the number of processors, not by fixing the problem size."*

Gustafson, 1988

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Lucru cu semafoare

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Sincronizare: Notiuni introductive

pdfelement

Remove Watermark Now

Dezvoltarea algoritmilor folosind variabile partajate (MIMD)

- Sincronizarea: excluderea mutuală şi sincronizarea condiţionată.
- Câteva rezultate notabile:
  - mutex
  - semafoare
    - regiuni critice
  - bariere
- Câteva dintre problemele *clasice*:
  - producători-consumatori
  - problema filozofilor
  - cititori-scriitori
  - problema bărbierului

pdfelement

Remove Watermark Now

Sectiuni critice

- Problema: fiecare proces P(i) al unei colectii de procese P(i:1..n) executa ciclic o sectiune critica, in care are acces exclusiv la anumite resurse partajate, urmata de o sectiune necritica in care foloseste doar resurse locale.

pdfelement

- Semafor = tip special de variabile partajate manipulate prin 2 operații atomice: P și V *(Dijkstra, 1965)*
  - Asigura excluderea mutuala intre procesele care acceseaza sectiunea critica.

Remove Watermark Now

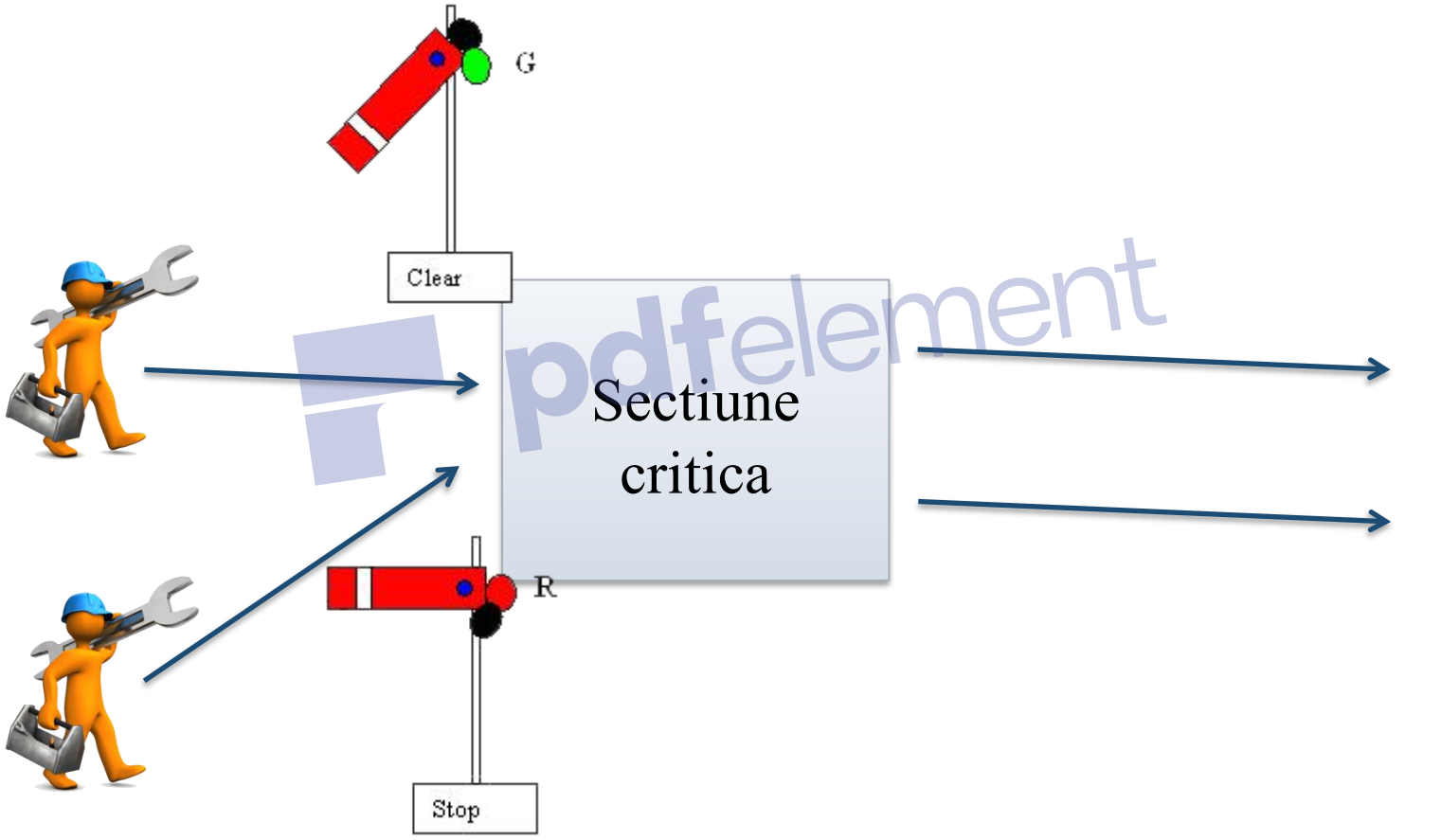
Secțiuni critice

```
sem mutex= 1;
process P[i=1 to n] {
    while (true) {
      P(mutex); /* acaparare secțiune critică */
      Secțiune critică;
      V(mutex); /* eliberare secțiune critică */
      Secțiune necritică
    }
}
```

pdfelement

Remove Watermark Now

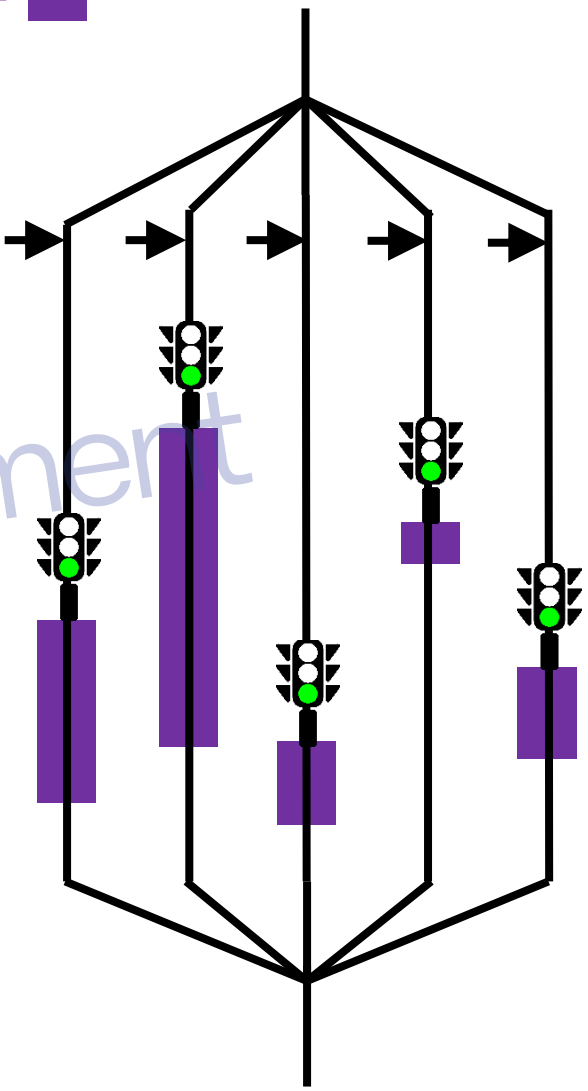
Exemplu executie…

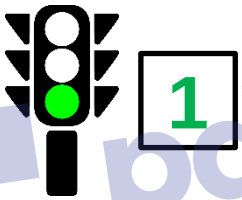


pdfelement

Remove Watermark Now

Zonă Critică





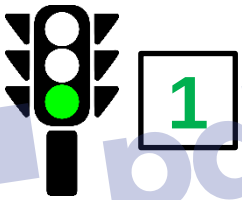
P() sau Proberen
V() sau Verhogen
- Dijsktra

Doar un thread poate fi în zona critică la un moment dat

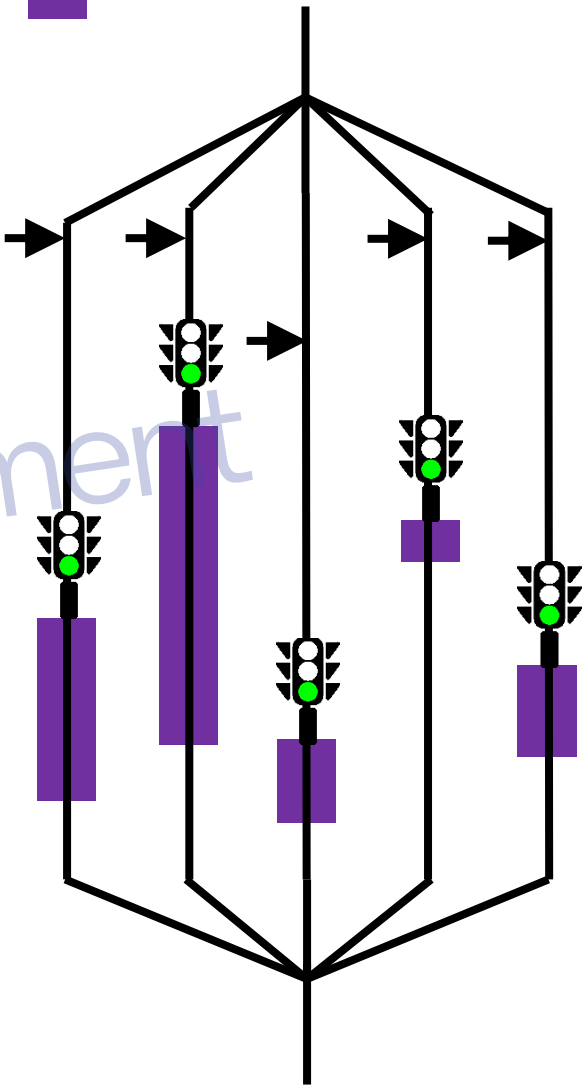
pdfelement

Remove Watermark Now

Zonă Critică



P() sau Proberen
V() sau Verhogen
- Dijsktra



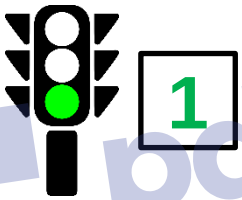
Doar un thread poate fi în zona critică la un moment dat

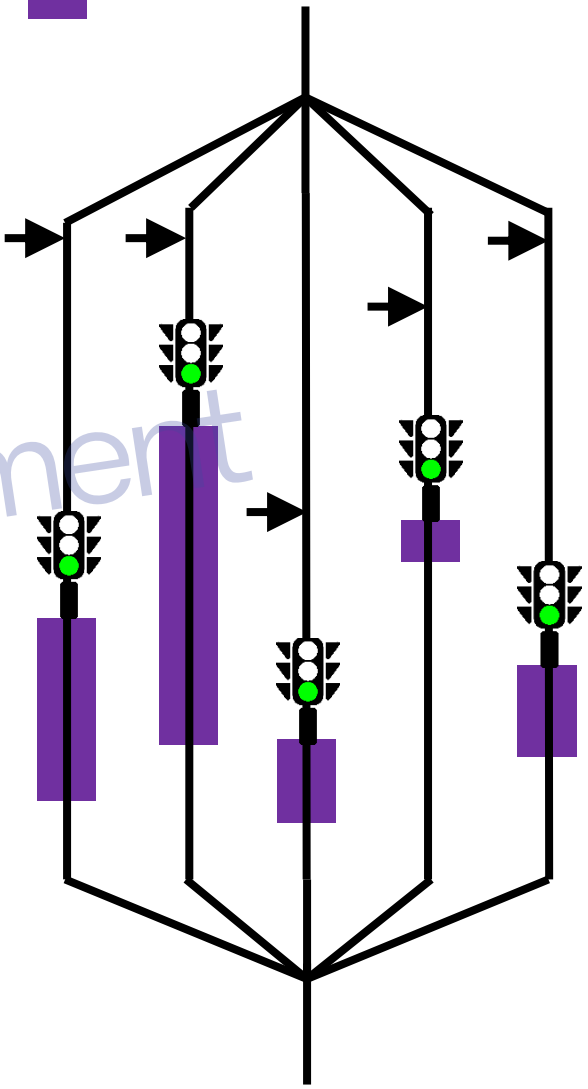
pdfelement

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





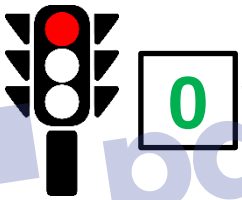
pdfelement

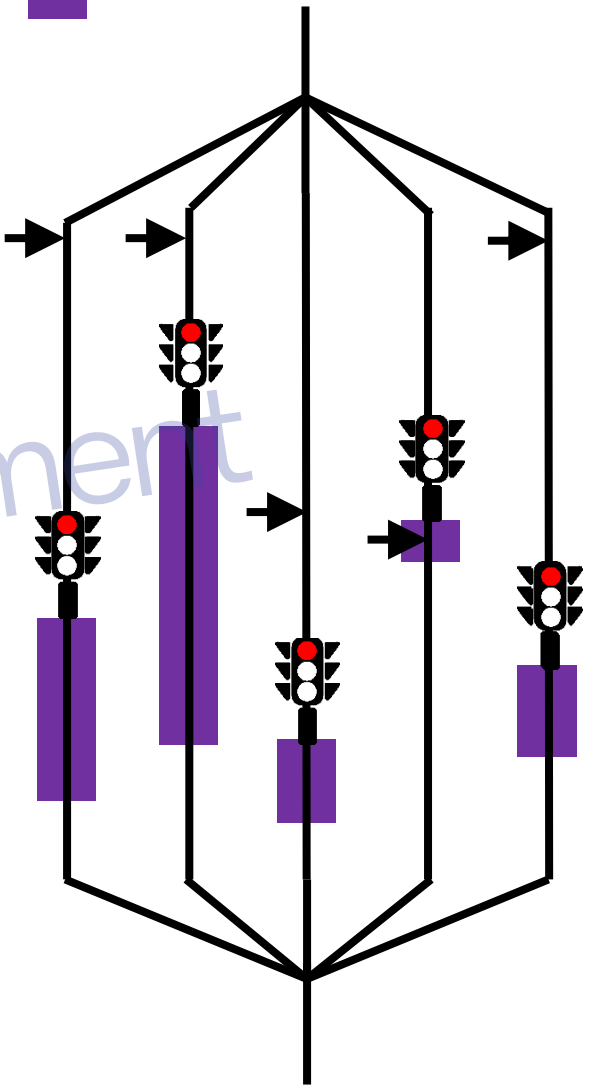
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





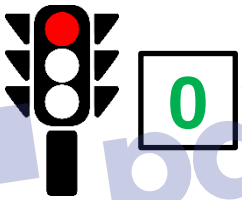
pdfelement

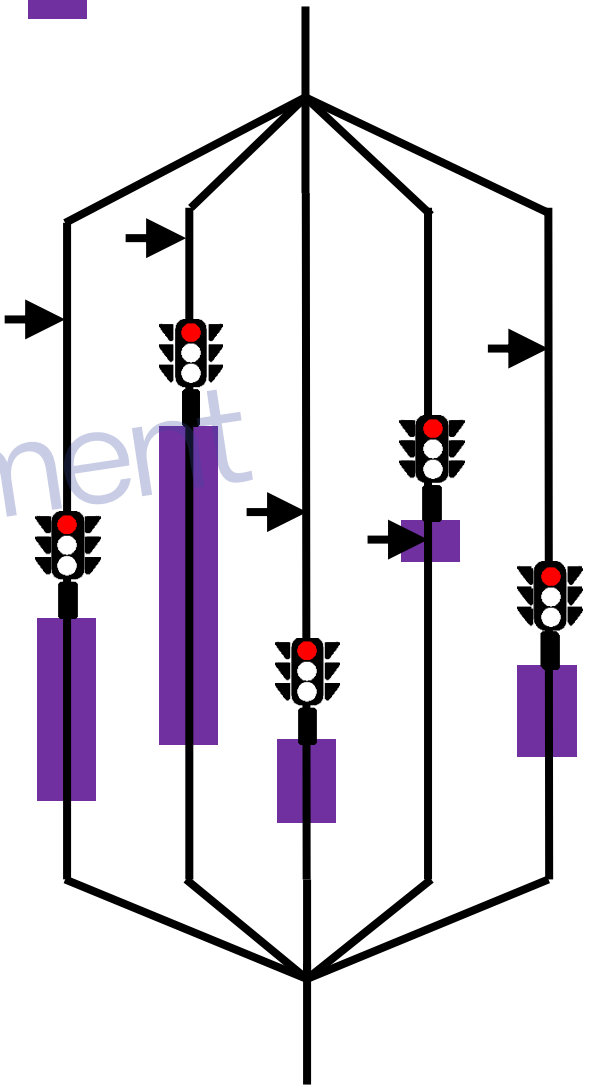
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra



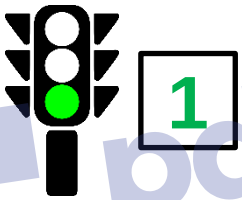


pdfelement

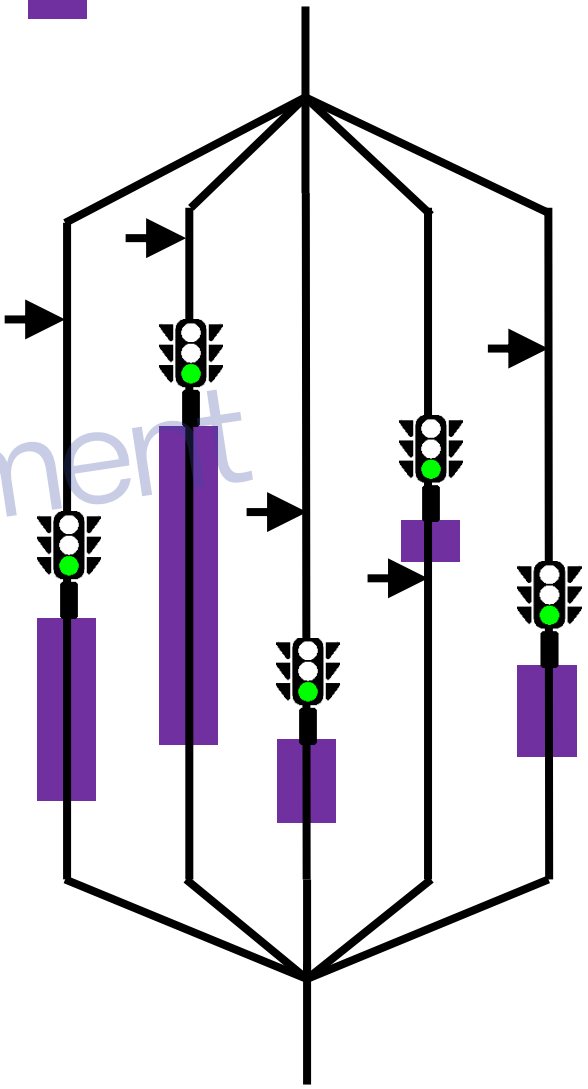
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică



P() sau Proberen
V() sau Verhogen
- Dijsktra



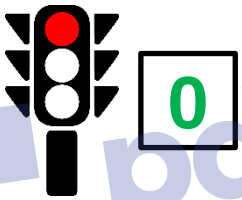
Doar un thread poate fi în zona critică la un moment dat

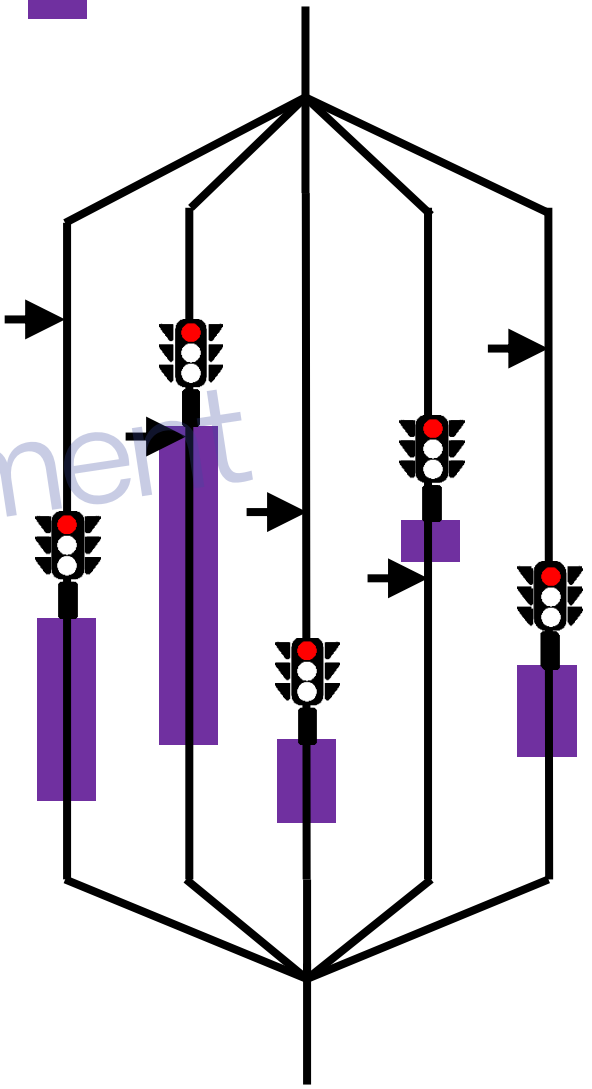
pdfelement

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





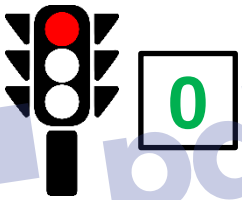
pdfelement

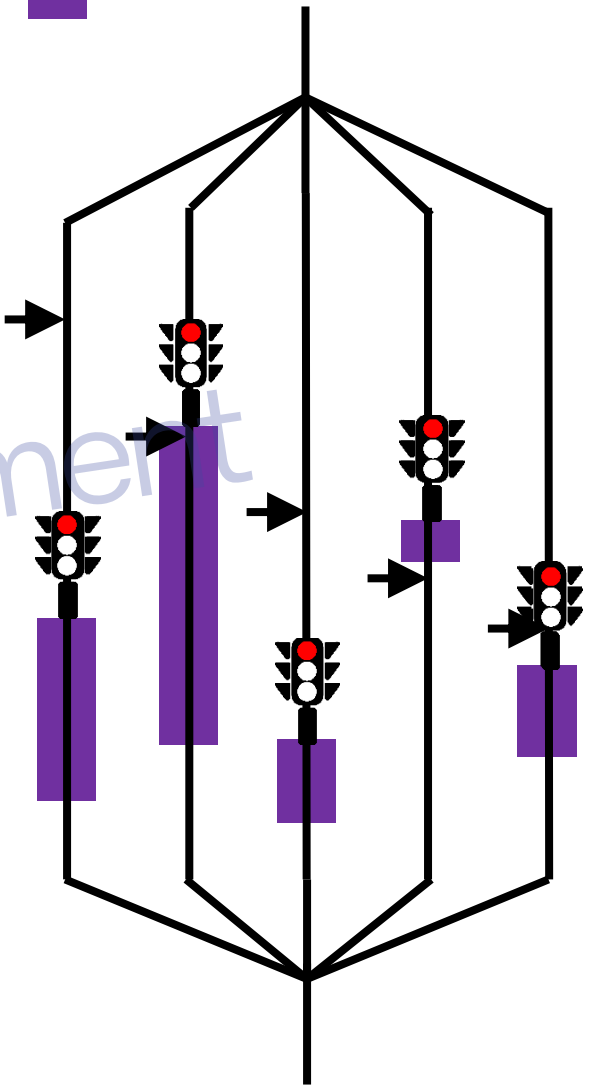
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





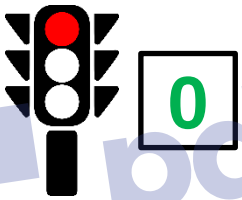
pdfelement

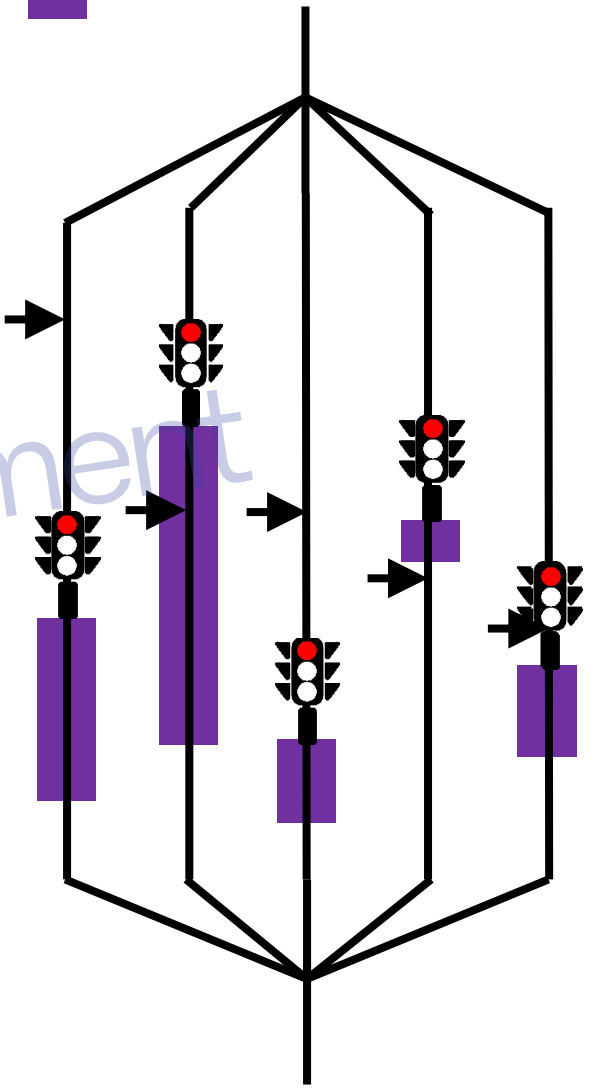
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





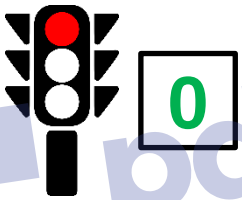
pdfelement

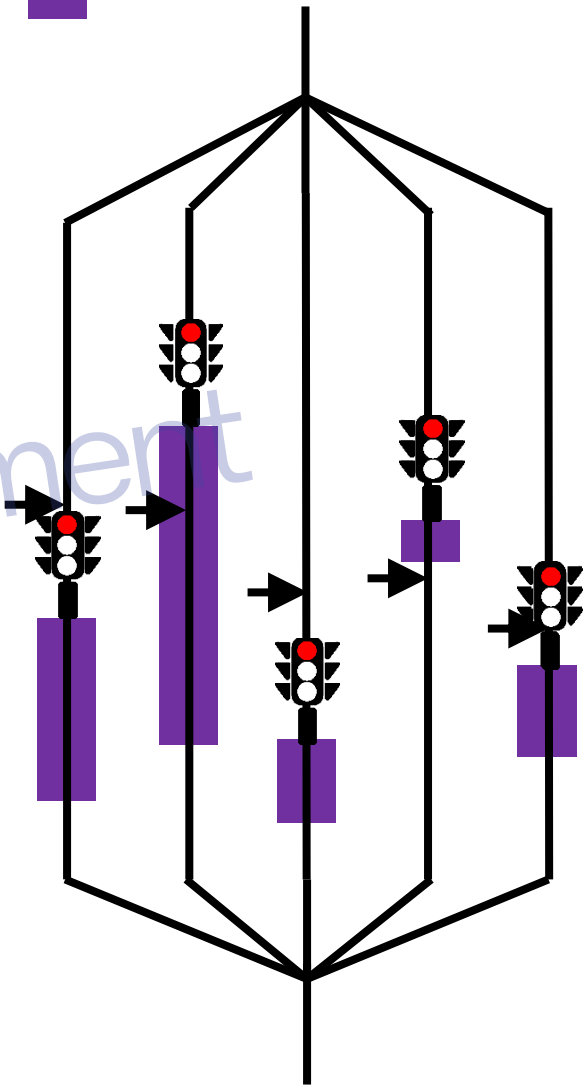
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





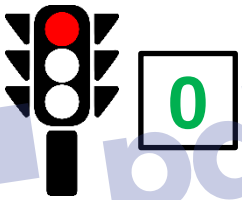
pdfelement

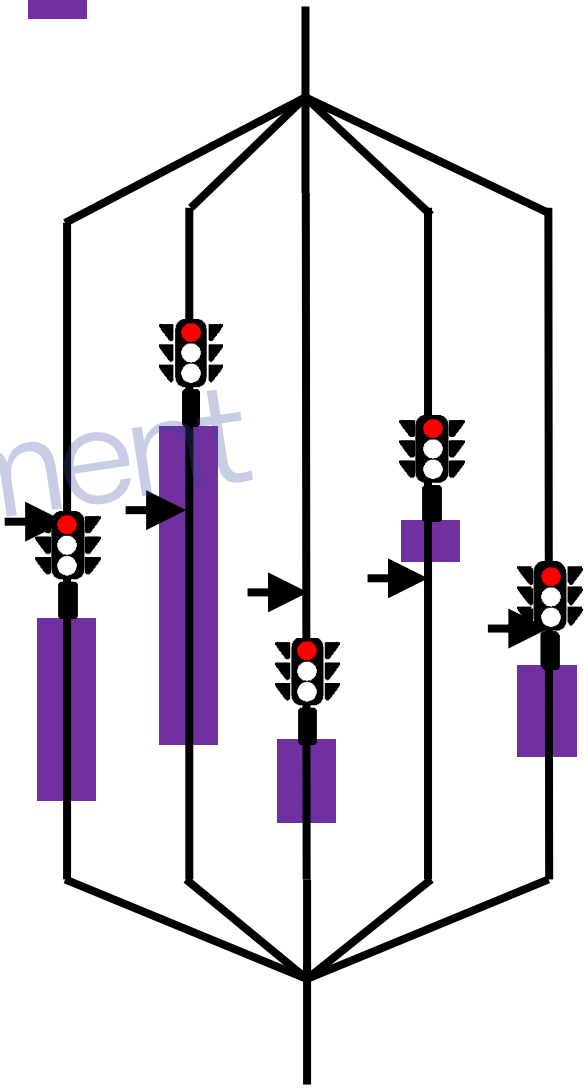
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





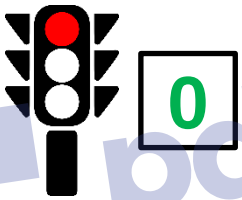
pdfelement

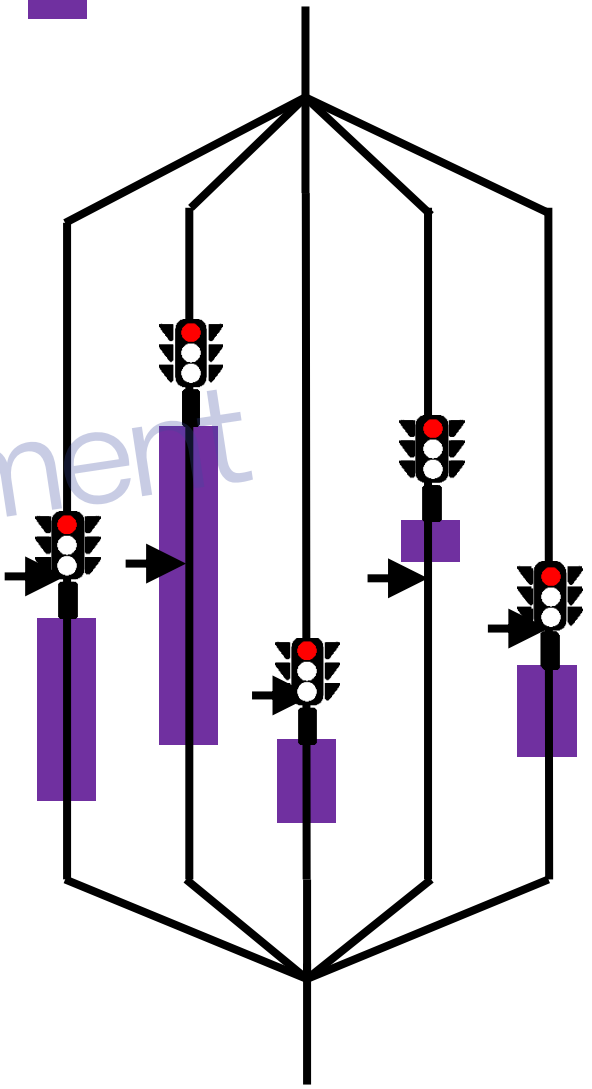
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





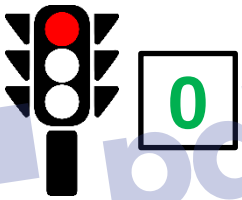
pdfelement

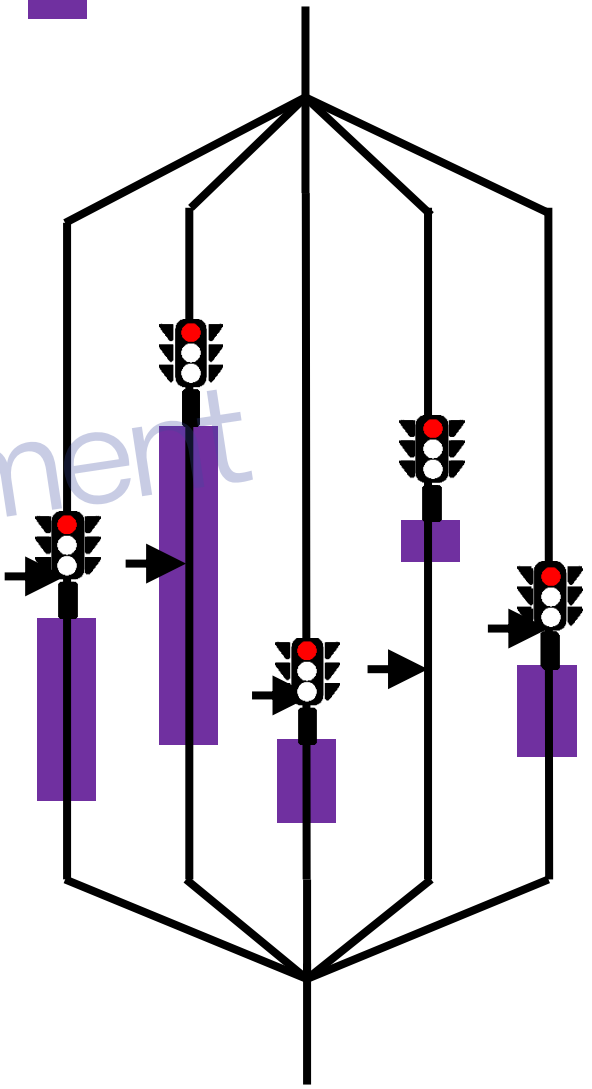
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





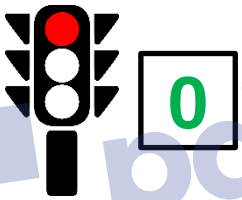
Doar un thread poate fi în zona critică la un moment dat

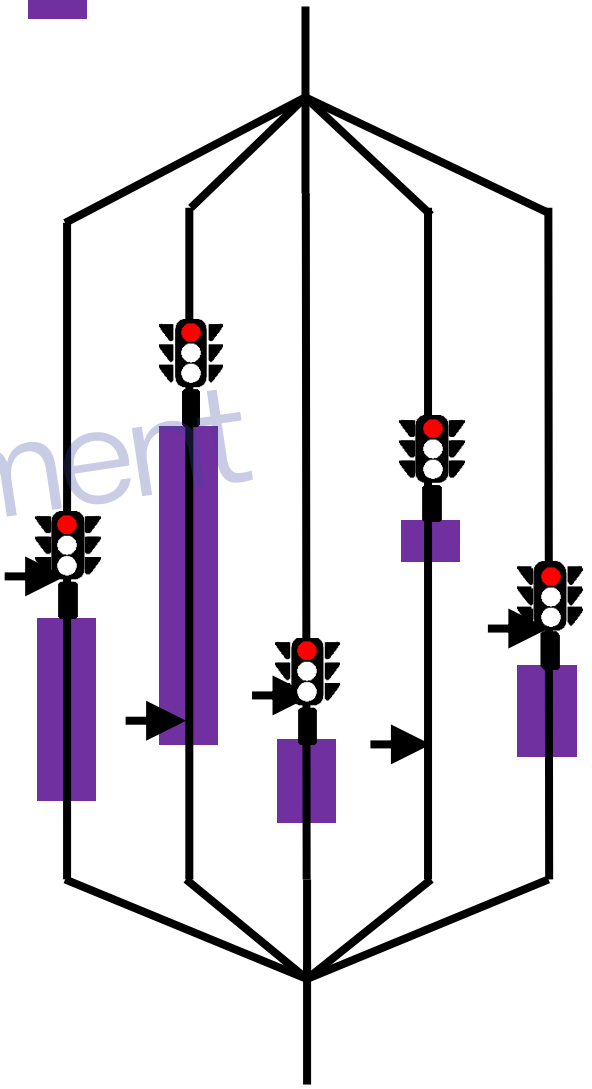
pdfelement

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





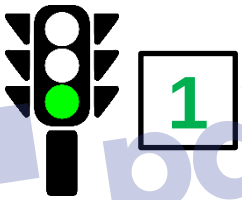
pdfelement

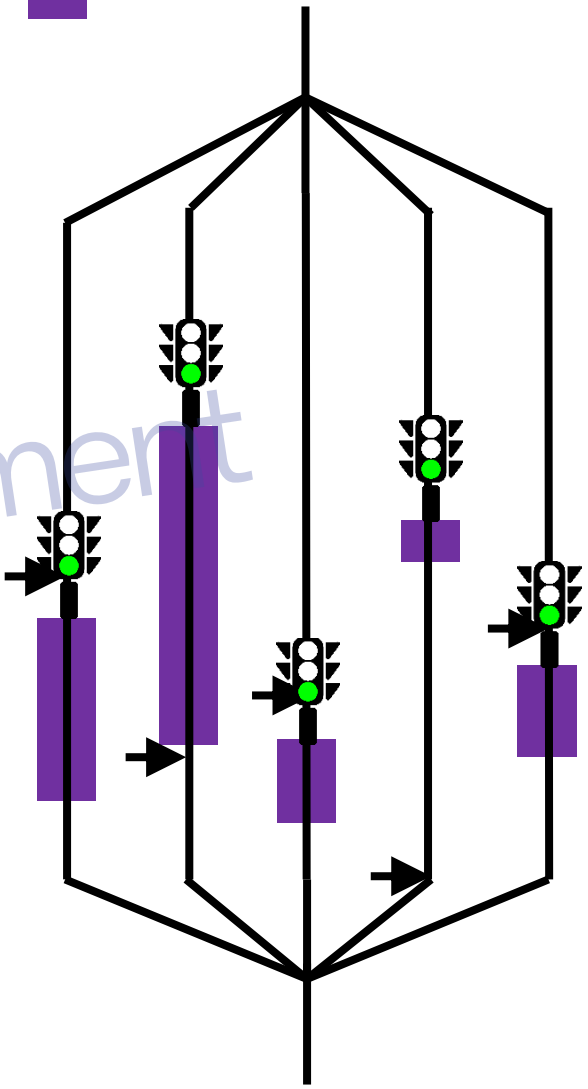
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





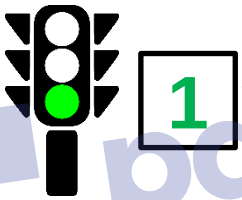
pdfelement

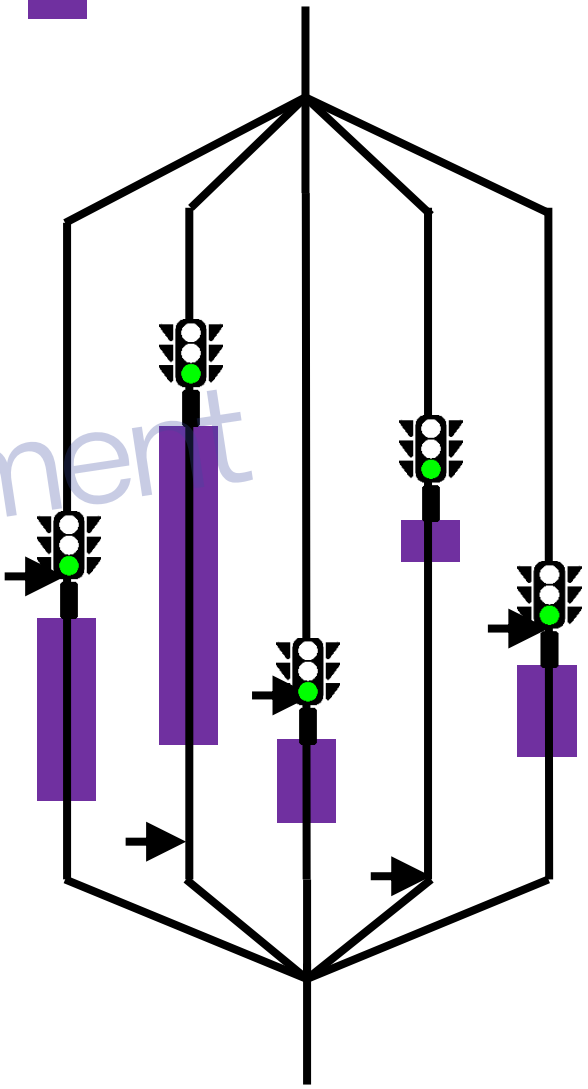
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





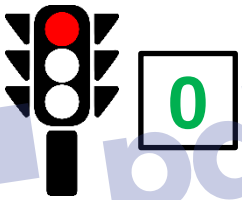
pdfelement

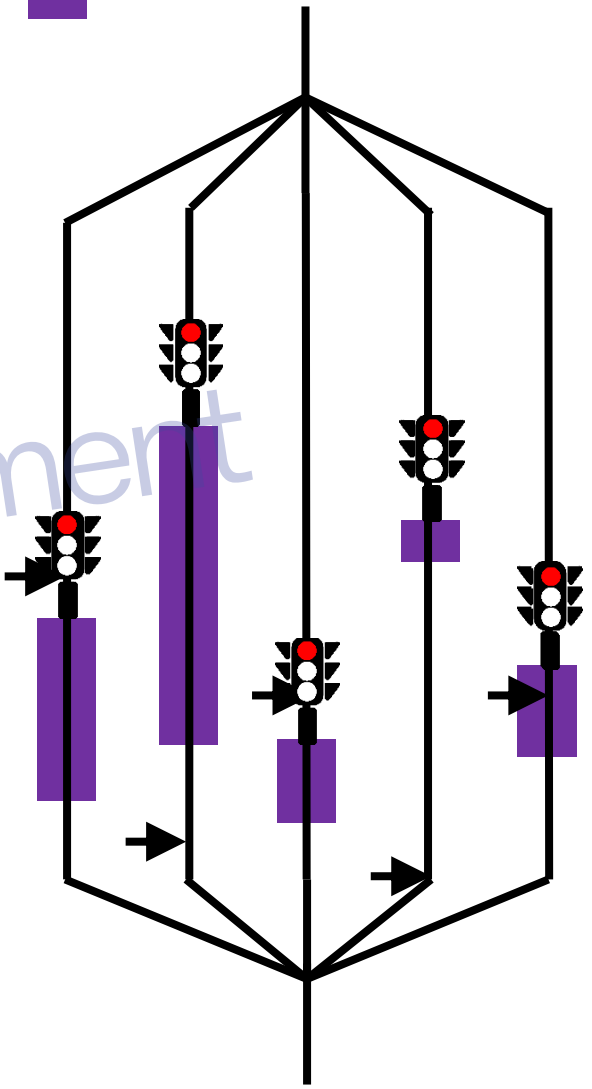
Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra





pdfelement

Doar un thread poate fi în zona critică la un moment dat

Remove Watermark Now

Producator-consumator

pdfelement

Remove Watermark Now

Producer - Consumer

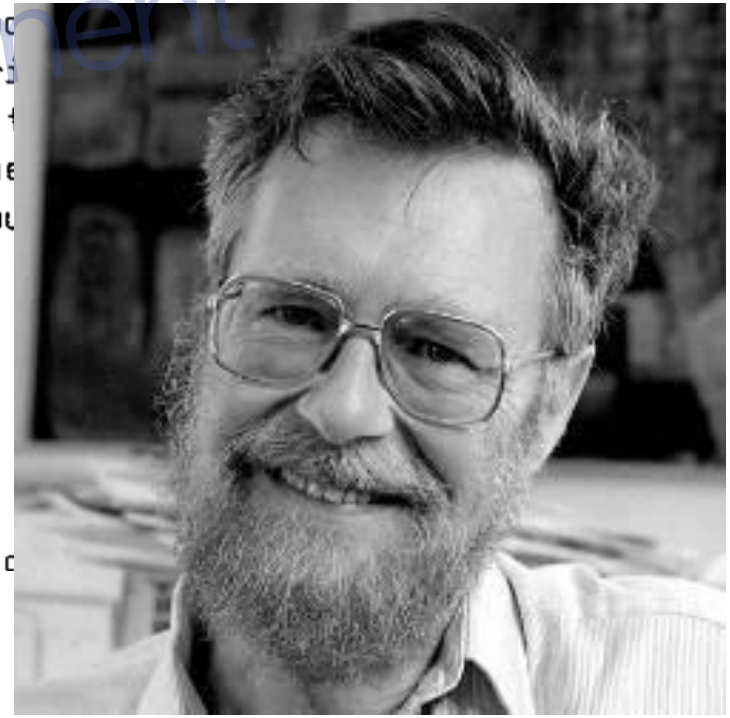
EWD209 - 0

EWD209.html

A Constructive Approach to the Problem of Program Correctness.

Summary. As an alternative to methods by which the correctness of given programs can be established a posteriori, this paper propo
of program generation such as to produce a priori
is treated to show the form that such a control
comes from the field of parallel programming; the
is representative for the way in which a whole mu
actually been constructed.

pdfelement



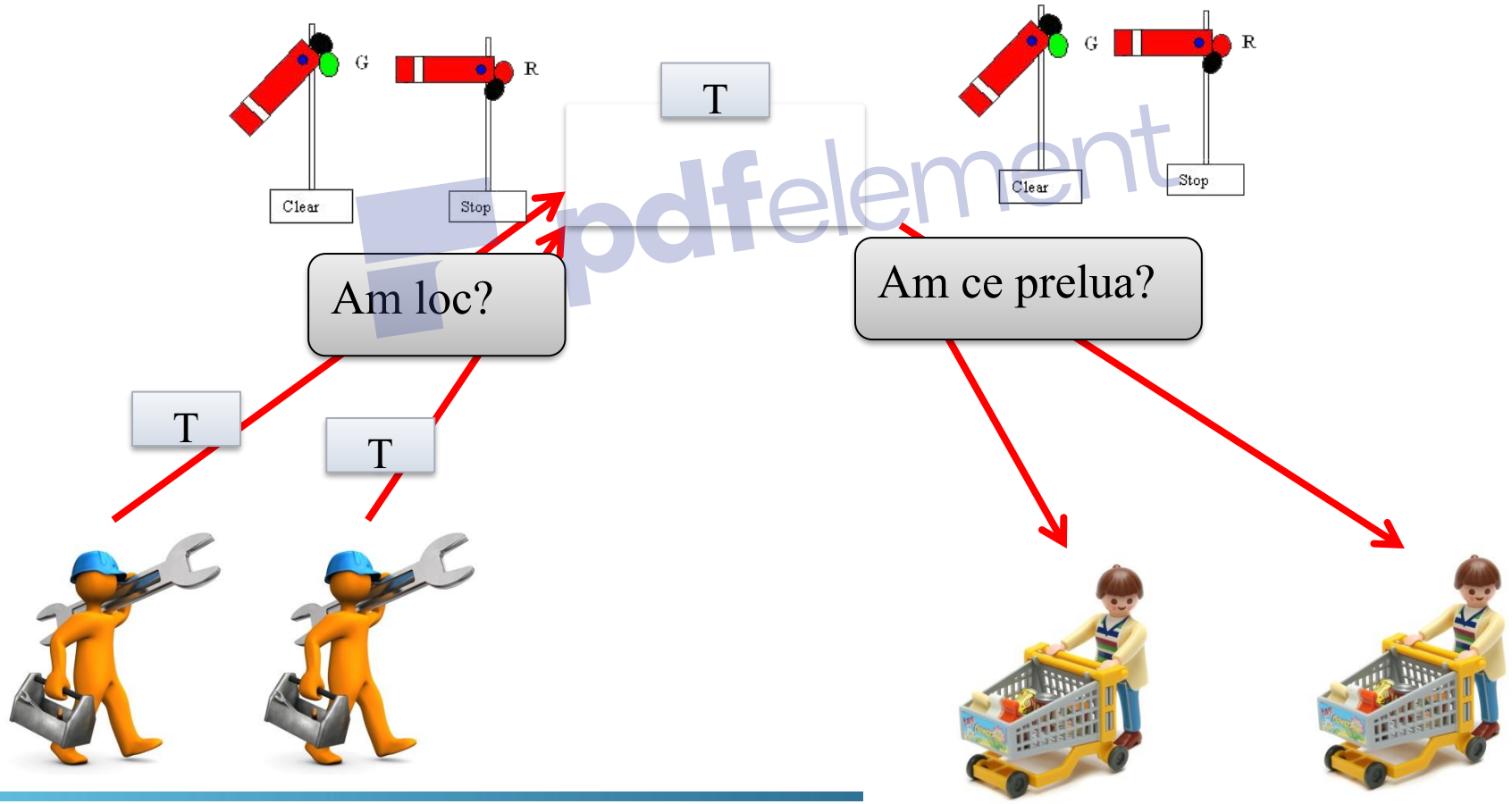
Introduction.

The more ambitious we become in our machine
becomes the problem of program correctness. The

Remove Watermark Now

Producători și consumatori

- Problema
  - Se consideră M producători și N consumatori care comunică printr-un singur buffer partajat.



pdfelement

Remove Watermark Now

Producători și consumatori

```
typeT buf; /* T - tipul datelor */
sem gol= 1, plin= 0; /* sem binare */
```

```
process Producător[i=1 to M]{
    typeT v;
    while(true) {
      v = produce();
      P(gol);
      buf = v;
      V(plin);
    }
}
```

```
process Consumator[i=1 to N]{
    typeT w;
    while(true) {
      P(plin);
      w = buf;
      V(gol);
      consumă(w);
    }
}
```

semnalizare

pdfelement

Remove Watermark Now

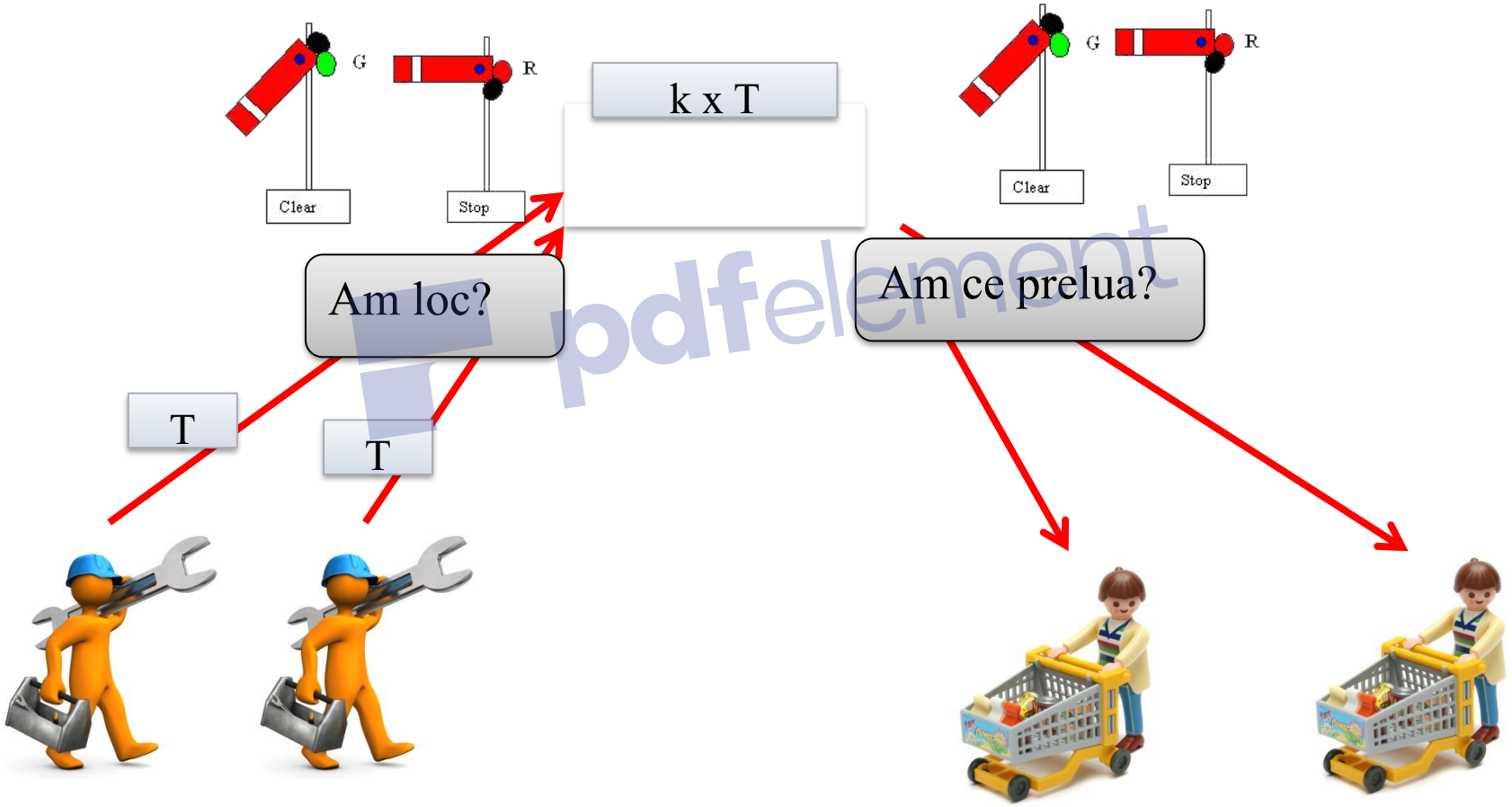
Comunicarea producător și consumator prin tampon limitat

```
typeT buf[1:k];
sem gol = k, plin = 0;  /* semafoare generale */
process Producător {
      typeT v;
      int ultim= 1;
      while (true) {
        v = produce();
        P(gol);                          /* există locuri goale? */
        buf[ultim] = v;
        ultim = ultim mod k + 1; /* circular */
        V(plin);
      } }
process Consumator {
      typeT w;
      int prim= 1;
      while (true) {
        P(plin);                        /* există valori în buffer? */
        w = buf[prim]; prim = prim mod k + 1;
        V(gol);
        consumă(w);
      }
}
```

pdfelement

Remove Watermark Now

Comunicarea producător și consumator prin tampon limitat



pdfelement

Remove Watermark Now

Mai mulți producători și mai mulți consumatori

```
typeT buf[1:k];
int prim = 1, ultim = 1;
sem gol = k, plin = 0;
sem mutexP = 1, mutexC = 1;
```

```
process Producător[i=1 to M]{
    typeT v;
    while (true) {
      v = produce();
      P(gol);
        P(mutexP);
        buf[ultim] = v;
        ultim = ultim mod k + 1;
        V(mutexP);
      V(plin);
    }
}
```

```
process Consumator[i=1 to N]{
    typeT w;
    while (true) {
P(plin);
      P(mutexC);
      w = buf[prim];
      prim = prim mod k + 1;
     V(mutexC);
V(gol);
consumă(w);
    }
}
```

secțiune critică

semnalizare

pdfelement

Remove Watermark Now

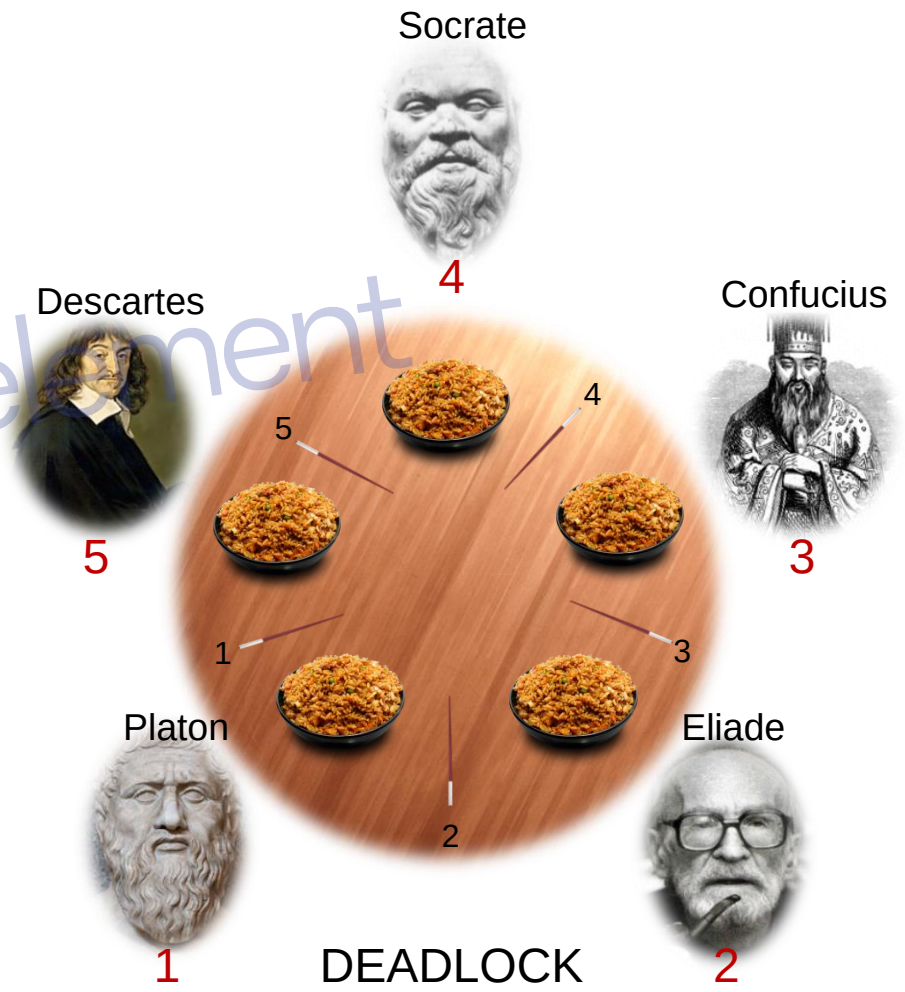
Problema filozofilor

pdfelement

Remove Watermark Now

Problema filozofilor

```
process Filozof[i=1 to 5]{
    while (true) {
      ia bețișoare;
      mănâncă;
      eliberează bețișoare;
      gândește;
    }
}
```

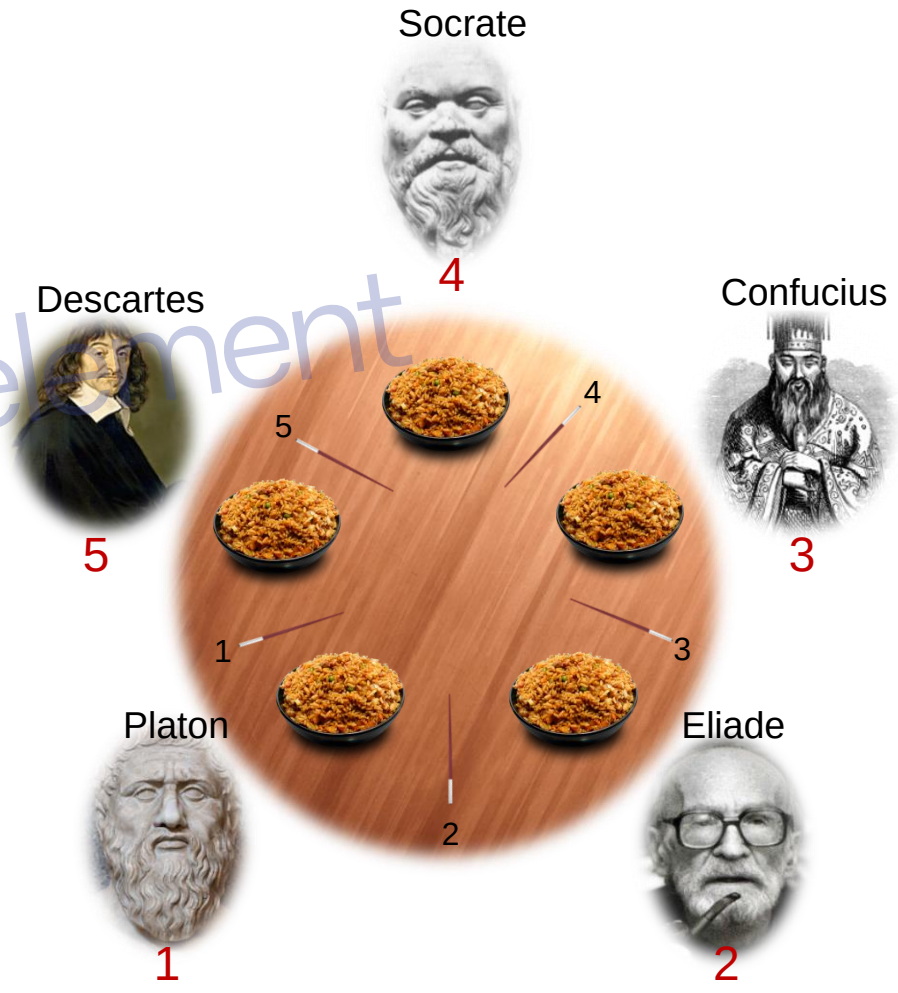


pdfelement

Remove Watermark Now

Problema filozofilor (2)

```
sem f[1:5] = ([5]1);
process Filozof[i=1 to 4]{
      while (true) {
        P(f[i]); P(f[i+1]);
        mănâncă;
        V(f[i]); V(f[i+1]);
        gândește;
} }
process Filozof[i=5]{
      while (true) {
        P(f[1]); P(f[5]);
        mănâncă;
        V(f[1]); V(f[5]);
        gândește;
} }
```



pdfelement

Remove Watermark Now

Problema cititori-scriitori

pdfelement

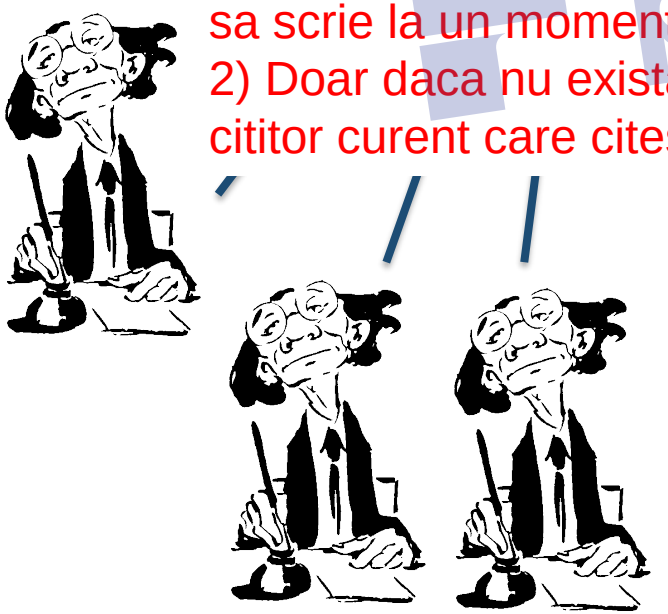
Remove Watermark Now

Problema cititorilor și scriitorilor
Excludere mutuală

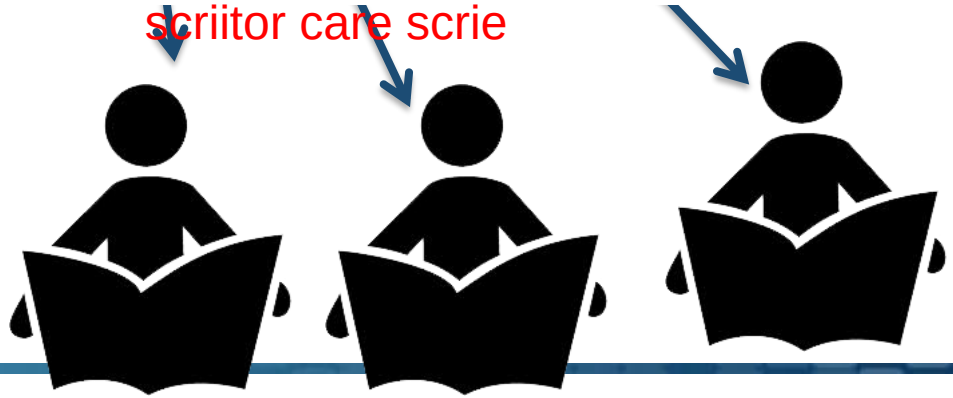
zona de memorie

(resursa critica)

1) Un singur scriitor are dreptul sa scrie la un moment dat,
2) Doar daca nu exista un cititor curent care citeste



1) Un cititor poate citi din memorie, indiferent daca deja exista alti cititori ce deja citesc
2) Doar daca nu exista un scriitor care scrie



pdfelement

Remove Watermark Now

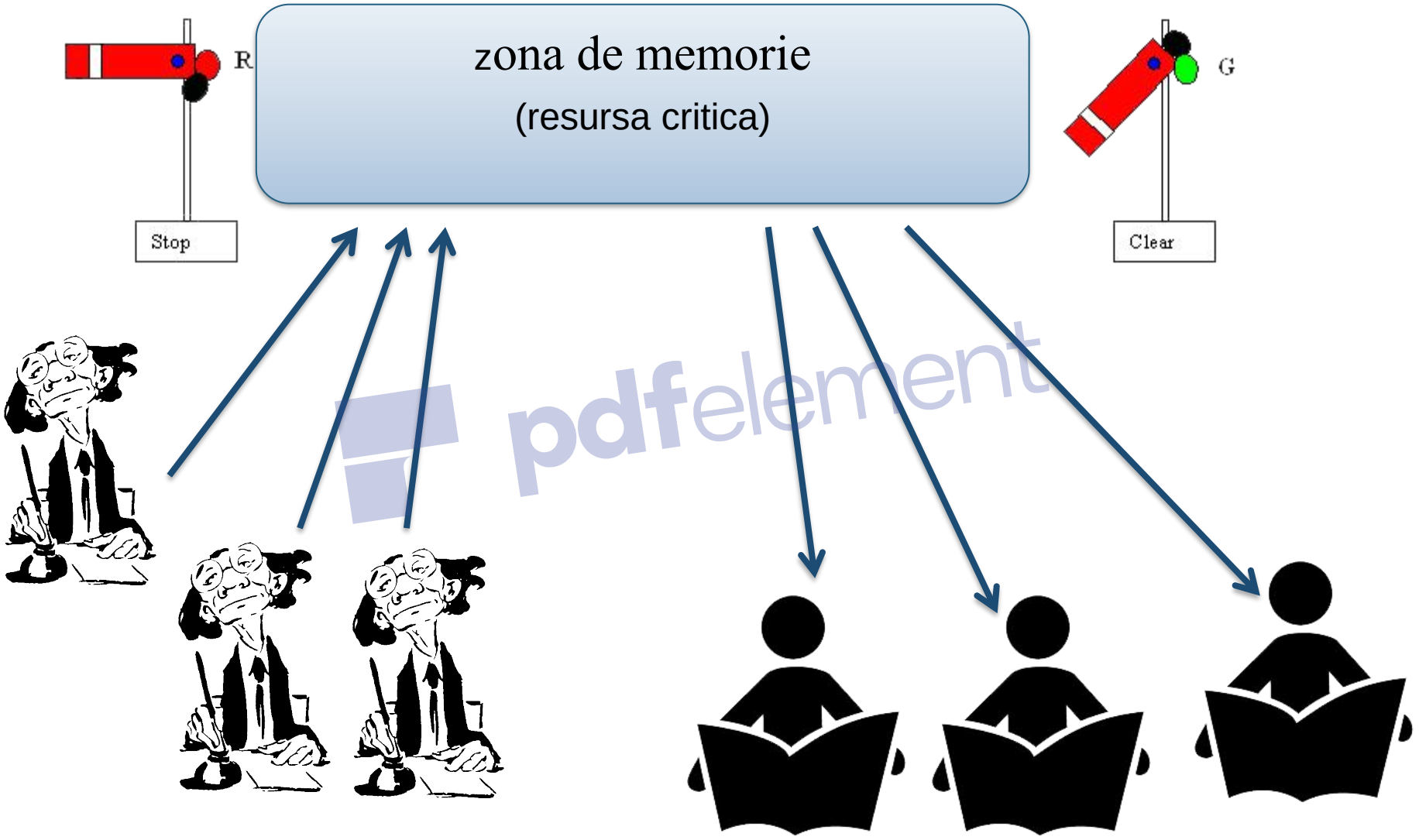
Problema cititorilor și scriitorilor. Excludere mutuală

```
sem rw = 1;
process Cititor[i=1 to m]{
      while (true){
         P(rw);
         citește din resursa comună;
         V(rw);
}    }
process Scriitor[j=1 to n]{
      while (true){
         P(rw);
         scrie în resursa comună;
         V(rw);
}    }
```

pdfelement

Excludere mutuala strica !
Un singur proces are voie sa execute opeartii pe resursa critica…

Remove Watermark Now



pdfelement

Remove Watermark Now

Problema cititorilor si scriitorilor. Excludere mutuală (2)

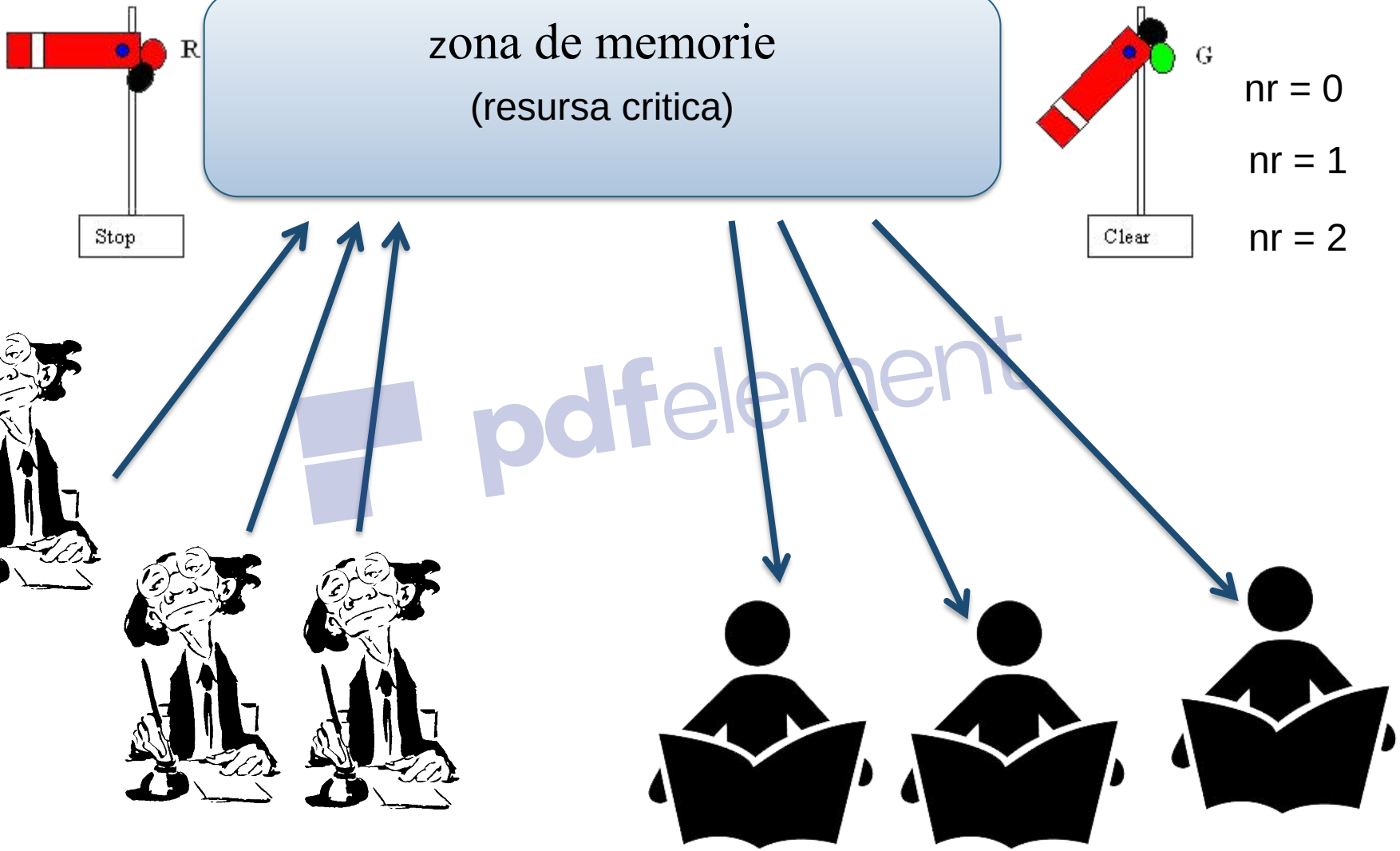
```
int nr = 0; sem mutexR = 1, rw = 1;
process Cititor[i=1 to m]{
      while (true){
        P(mutexR);
          nr = nr + 1;
          if (nr == 1) P(rw);    /* dacă primul cititor */
        V(mutexR);
        citește din resursa comună;
        P(mutexR);
          nr = nr - 1;
          if (nr == 0) V(rw);    /* dacă ultimul cititor */
        V(mutexR);
}     }
process Scriitor[j=1 to n]{
      while (true){
        P(rw);
        scrie în resursa comună;
        V(rw);
}     }
```

instructiune cu garda

secțiune critică

pdfelement

Remove Watermark Now



pdfelement

Remove Watermark Now

Problema cititorilor și scriitorilor
Sincronizare condiționată

- Invariant global:

```
RW: ( nr == 0 || nw == 0 ) && nw <= 1
```

- Proces Reader:
  - Incrementarea `nr` e condiționată de `(nw == 0)`

- Proces Writer:
  - Incrementarea `nw` e condiționată de `(nr == 0 && nw == 0)`

- Decrementarea nu trebuie condiționată

pdfelement

Problema cititorilor și scriitorilor. Sincronizare condiționată (2)

Remove Watermark Now

```
int nr = 0, nw = 0;
process Cititor[i=1 to m]{
      while (true){
        <await (nw == 0) nr = nr + 1>
        citește din resursa comună;
        <nr = nr - 1>
}     }
process Scriitor[j=1 to n]{
      while (true){
        <await (nr == 0 && nw == 0) nw = nw + 1>
        scrie în resursa comună;
        <nw = nw - 1>
}     }
```

pdfelement

Remove Watermark Now

Problema cititorilor și scriitorilor. Sincronizare condiționată (3)

Politici:

- noile cereri de la cititori sunt întârziate dacă un scriitor așteaptă
- un cititor întârziat este trezit doar dacă nu există un scriitor în așteptare

```
int nr = 0;   /* nr. cititori care folosesc resursa */
int nw = 0;   /* nr. scriitori care folosesc resursa */
sem e = 1;    /* intrare secțiune atomică */
sem r = 0;    /* intarzie cititori daca nw>0 sau dw>0 */
sem w = 0;    /* intarzie scriitori daca nw>0 sau nr>0 */
int dr = 0;   /* nr. cititori întârziați */
int dw = 0;   /* nr. scriitori întârziați */
```

pdfelement

Remove Watermark Now

Problema cititorilor și scriitorilor Sincronizare condiționată (4)

Split binary semaphore

- Folosit pentru a implementa atât excluderea mutuală cât și sincronizarea condiționată.
- Semafoarele e, r și w formează împreună un semafor ***splitat*** *(split binary semaphore)*:
  - cel mult un semafor este 1 la un moment dat $0 \leq e + r + w \leq 1$
  - fiecare cale de execuție începe cu un P și se termină cu un singur V
  - instrucțiunile între P și V se execută în excludere mutuală.
- Tehnica se numește ***pasarea ștafetei***:
  - Inițial un semafor este 1 și un proces poate prelua ștafeta printr-o operație P asupra semaforului
  - când un proces deține ștafeta (se execută într-o secțiune critică și toate semafoarele sunt 0), el poate pasa ștafeta altui proces printr-o operație V asupra unuia din cele trei semafoare.

pdfelement

Remove Watermark Now

Problema cititorilor și scriitorilor. Sincronizare condiționată (5)

```
process Cititor[i=1 to m]{
    while (true){
      P(e);
      if (nw > 0 or dw > 0){
        dr = dr + 1; V(e); P(r);
      }
      nr = nr + 1;
      if (dr > 0) { dr = dr - 1; V(r); }
      else if (dr == 0) V(e);
      citește din resursa comună;
      P(e);
      nr = nr - 1;
      if (nr == 0 and dw > 0) { dw = dw - 1; V(w); }
      else if (nr > 0 or dw == 0) V(e);
    }
}
```

pdfelement

Problema cititorilor și scriitorilo
Sincronizare condiționată (6)

Remove Watermark Now

```
process Scriitor[j=1 to n]{
  while (true){
    P(e);
    if (nr > 0 or nw > 0) {
      dw = dw + 1; V(e); P(w)
    }
    nw = nw + 1;
    V(e);
    scrie în resursa comună;
    P(e);
    nw = nw - 1;
    if (dr > 0 and dw == 0) { dr = dr - 1; V(r); }
    else if (dw > 0) { dw = dw-1; V(w); }
    else if (dr == 0 and dw == 0) V(e);
  }
}
```

pdfelement

Remove Watermark Now

Problema barbierului

pdfelement

Remove Watermark Now

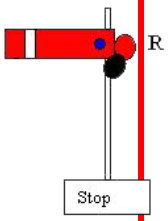
Problema bărbierului

- Problema:
  - O frizerie cu un bărbier, un scaun de bărbier, n scaune de așteptare.
  - Când nu sunt clienți, bărbierul doarme.
  - Când sosește un client fie trezește bărbierul, fie așteaptă dacă acesta e ocupat.
  - Dacă toate scaunele sunt ocupate, clientul pleacă.

pdfelement

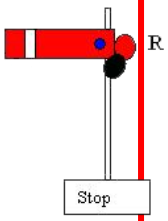
Pas 1

Remove Watermark Now



clienti

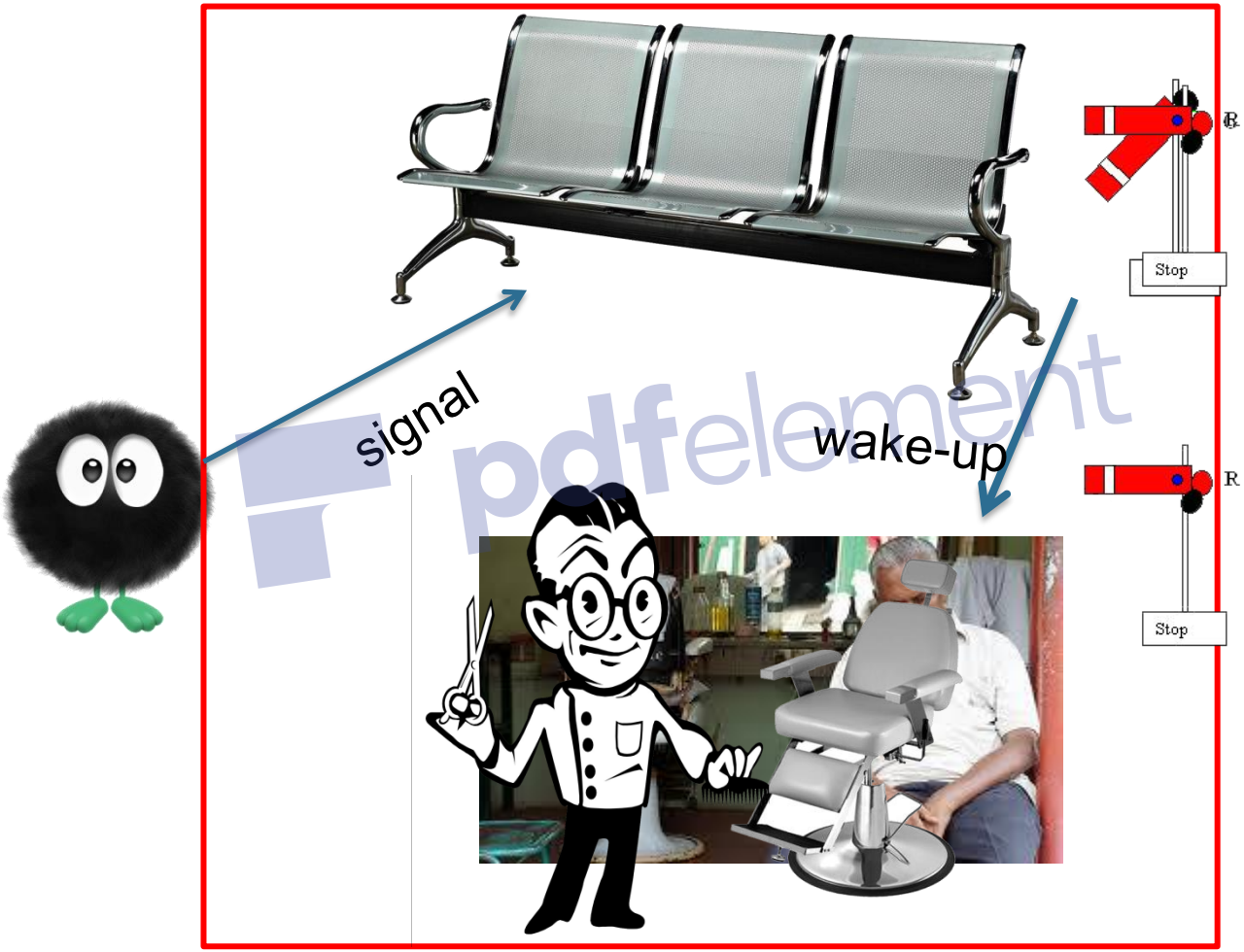
pdfelement



BarbierGata

Pas 2

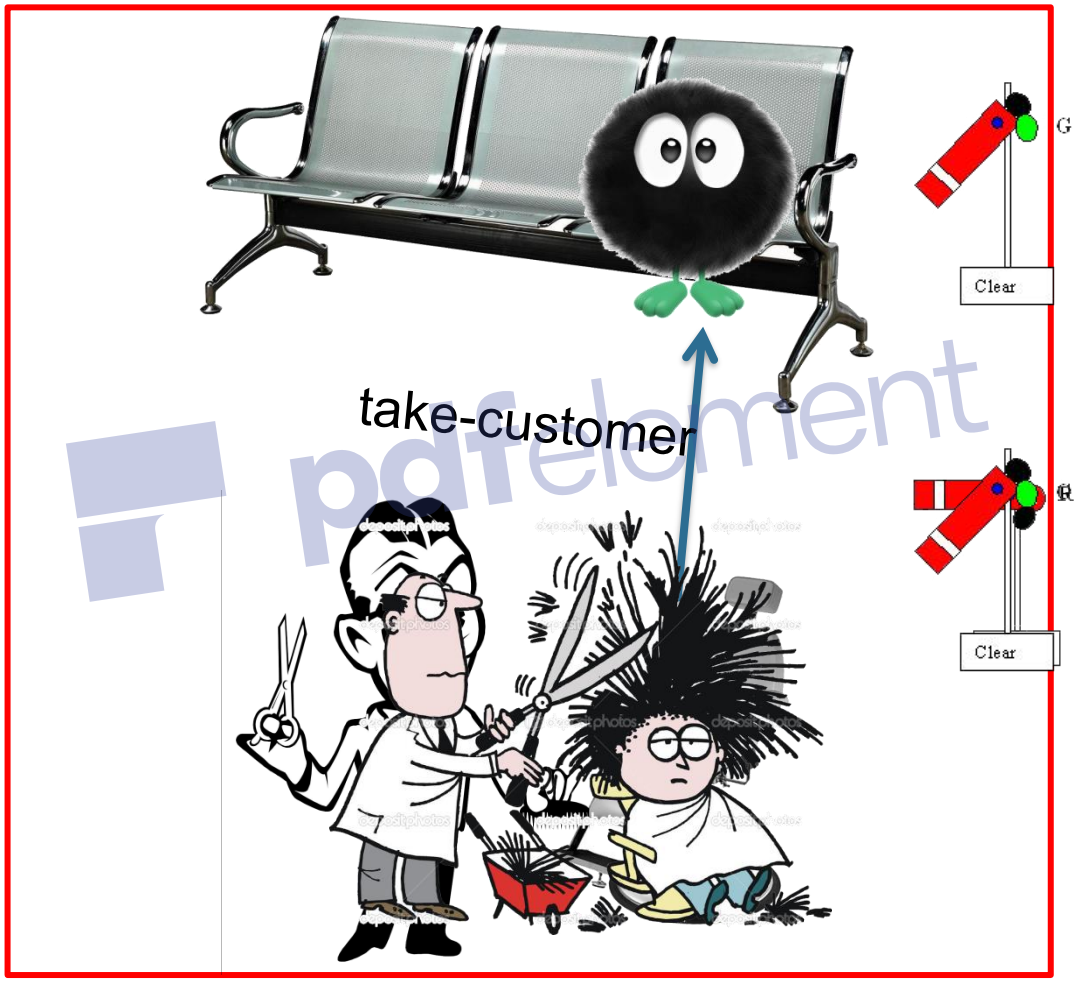
Remove Watermark Now



pdfelement

Pas 3

Remove Watermark Now

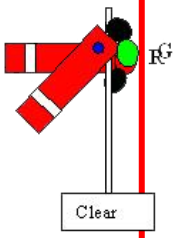


clienti

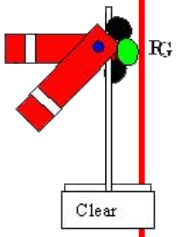
BarbierGata

Pas 4

Remove Watermark Now



clienti



BarbierGata

pdfelement

Problema bărbierului (1)

Remove Watermark Now

```
int NumărScauneLibere = n;
sem Clienți = 0;
sem BărbierGata = 0;
sem Scaune = 1;

process Bărbier{
   while (true){
     P(Clienți);                  /* se caută un client;
                                     dacă există, este chemat */
     P(Scaune);                   /* are client, va modifica
                                     NumărScauneLibere */
     NumărScauneLibere++;         /* se eliberează un scaun */
     V(BărbierGata);              /* bărbierul e gata să tundă */
     V(Scaune);                   /* a terminat de modificat
                                     NumărScauneLibere */
     /* Bărbierul tunde... */
   }
}
```

sincronizare cu Client

atomicitate

pdfelement

Problema bărbierului (2)

Remove Watermark Now

```
process Client[i=1 to m]{
    while (true) {
      P(Scaune);                          /* clientul încearcă să obțină
                                             un scaun liber */
      if (NumărScauneLibere > 0){         /* există un scaun disponibil? */
        NumărScauneLibere--;              /* clientul se așază */
       V(Clienți);                        /* se anunță bărbierul că s-a
                                             ocupat un scaun */
       V(Scaune);                         /* se renunță la accesul asupra
                                             NumărScauneLibere */
       P(BărbierGata);                    /* clientul așteaptă pentru a
                                             fi tuns */
        /* Clientul e tuns... */
      } else {                            /* nu sunt scaune libere */
        V(Scaune);                          /* eliberează mutexul */
        /* Clientul pleacă netuns... */
      }
    }
}
```

sincronizare cu Barbier

atomicitate

pdfelement

Remove Watermark Now

Problema bărbierului (var. mutex)

Bărbier

emptyChairs = N
Clients = 0
BarberReady = 0
Chairs = 1

Client

```
while(true) {
    Clients.lock();
    Chairs.lock();
    emptyChairs++;
    BarberReady.unlock();
    Chairs.unlock();
    cutHair();
}
```

```
Chairs.lock();
if(emptyChairs>0) {
    emptyChairs--;
    Clients.unlock();
    Chairs.unlock();
    BarberReady.lock();
    getHairCut();
} else {
    Chairs.unlock();
}
```

pdfelement

Remove Watermark Now

Sumar

- Dezvoltarea algoritmilor folosind variabile partajate (MIMD)
- Semafoare
- Secțiuni critice
- Probleme:
  - Producători și consumatori
  - Problema filozofilor
  - Problema cititorilor și scriitorilor
  - Problema bărbierului

pdfelement

Remove Watermark Now

Arhitecturi Paralele
Abordări probleme paralele

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

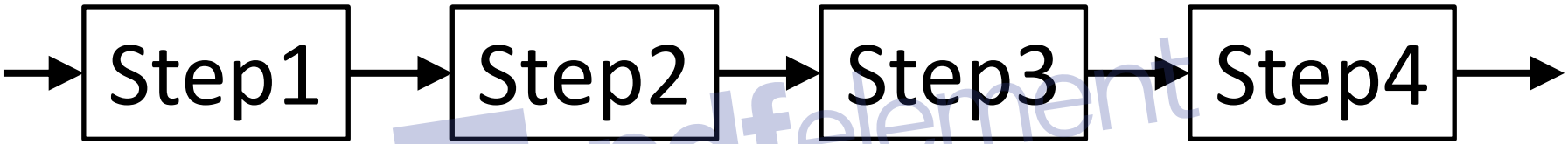
Paralelism: modele Pipeline

pdfelement

Remove Watermark Now

Pipeline

- CPU Instruction pipeline
- Graphics pipeline
- Various algorithms



pdfelement

Un pas poate fi executat de:

- thread
- process
- element hardware

Remove Watermark Now

Without Pipeline

pdfelement

Remove Watermark Now

Without Pipeline

pdfelement

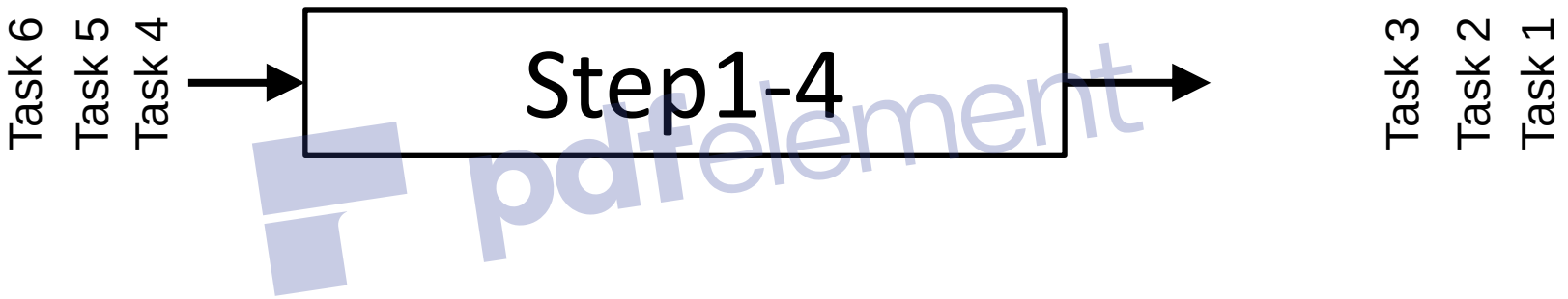
Remove Watermark Now

Without Pipeline

pdfelement

Remove Watermark Now

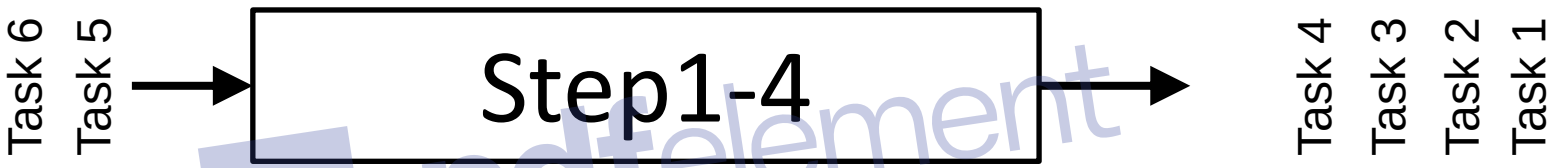
Without Pipeline



pdfelement

Remove Watermark Now

Without Pipeline



pdfelement

Remove Watermark Now

Without Pipeline

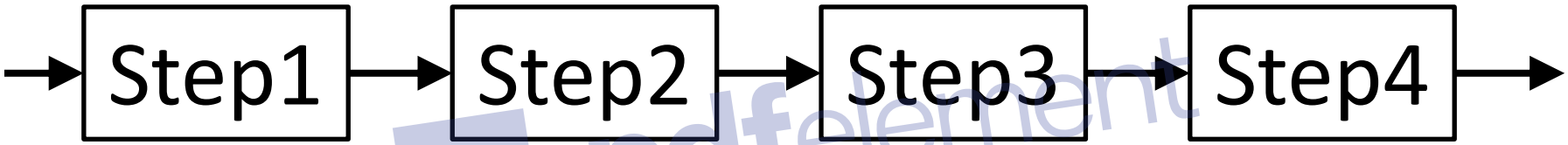
pdfelement

$$total_execution_time = task_execution_time * number_of_tasks$$

Remove Watermark Now

Pipeline

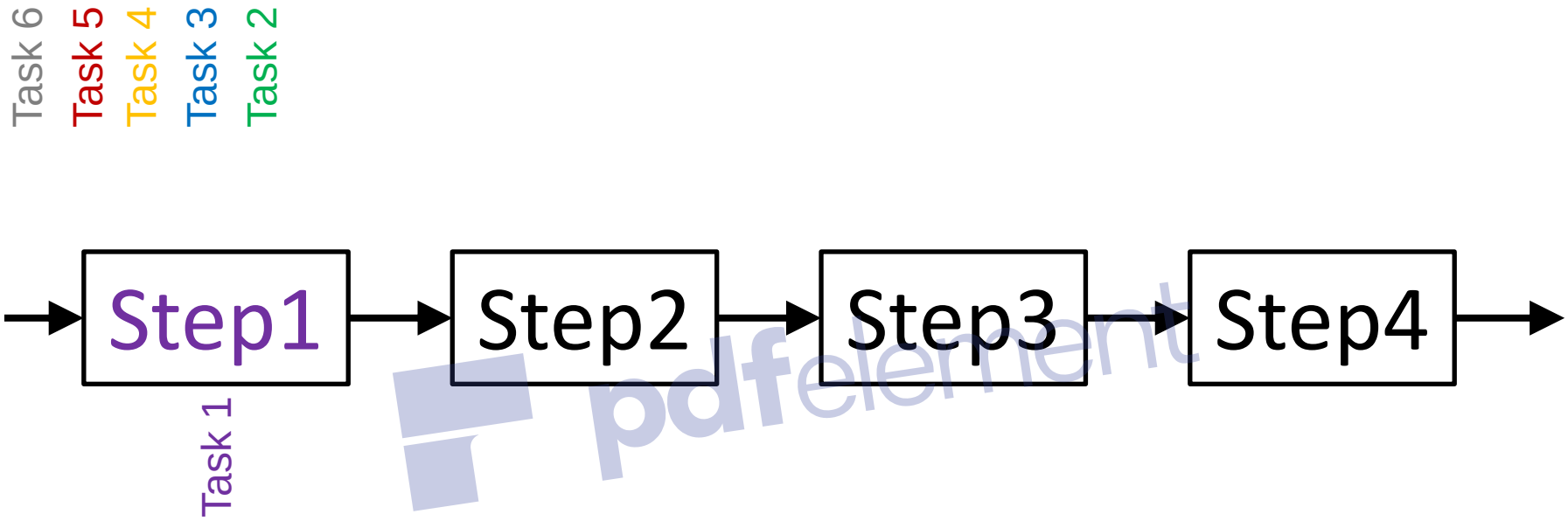
Task 6
Task 5
Task 4
Task 3
Task 2
Task 1



pdfelement

Remove Watermark Now

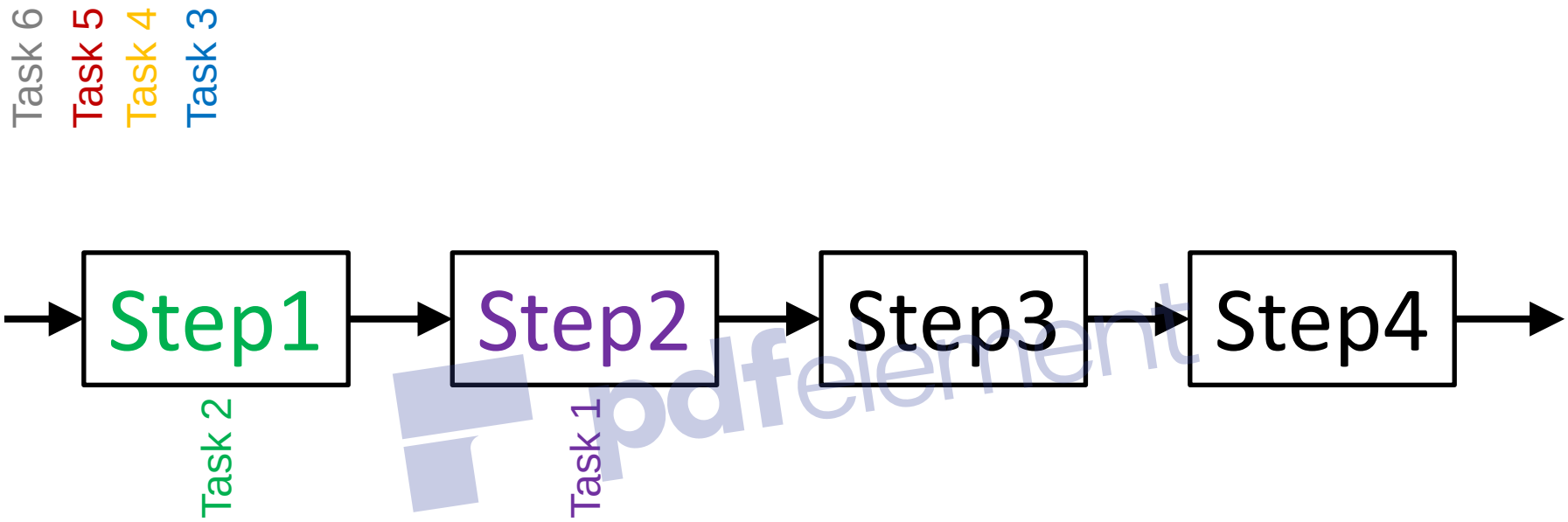
Pipeline



pdfelement

Remove Watermark Now

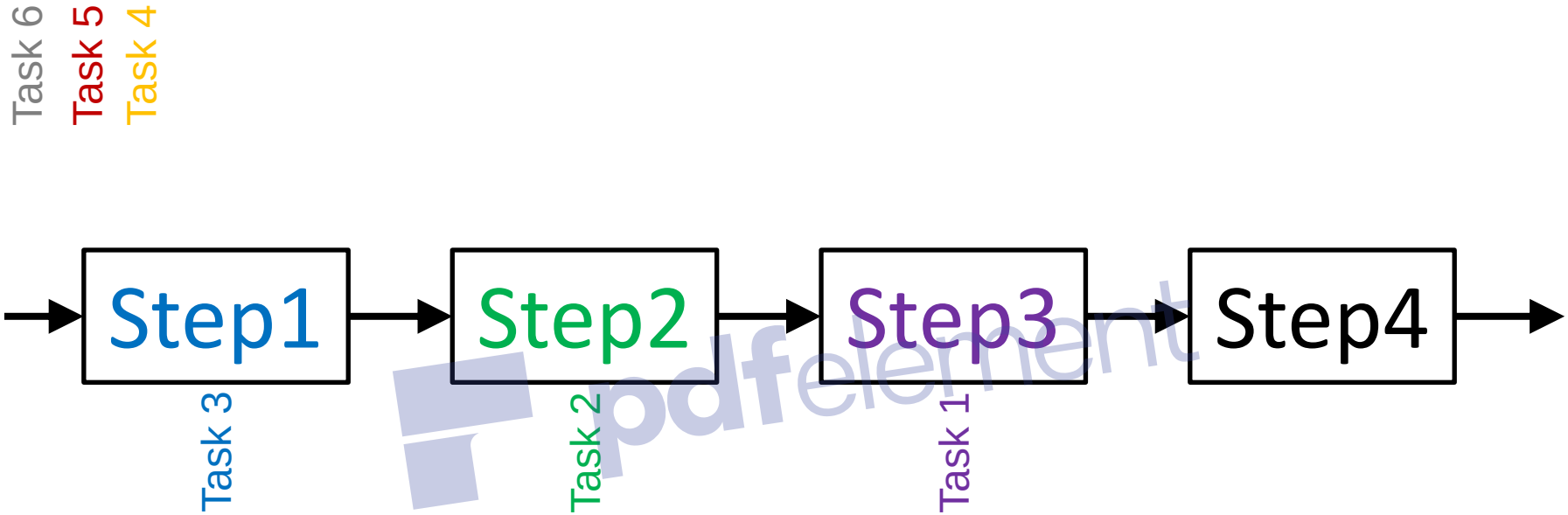
Pipeline



pdfelement

Remove Watermark Now

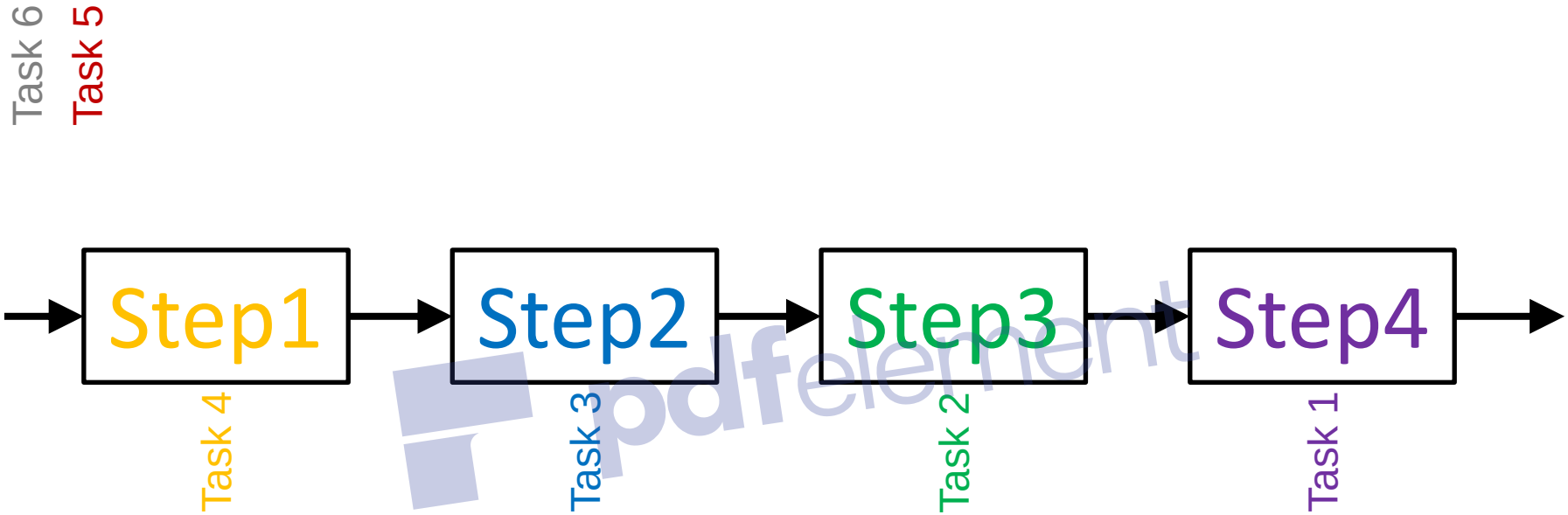
Pipeline



pdfelement

Remove Watermark Now

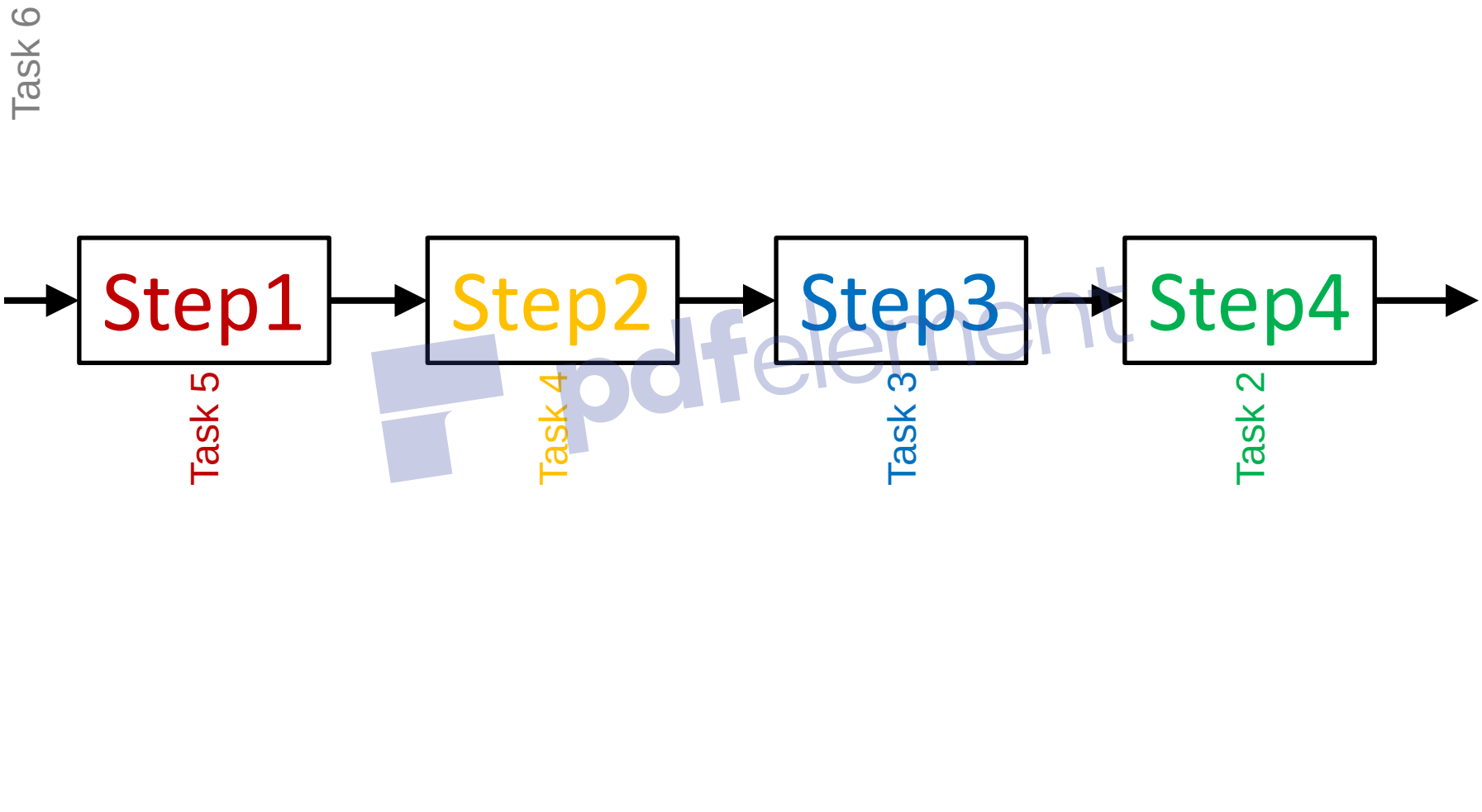
Pipeline



pdfelement

Remove Watermark Now

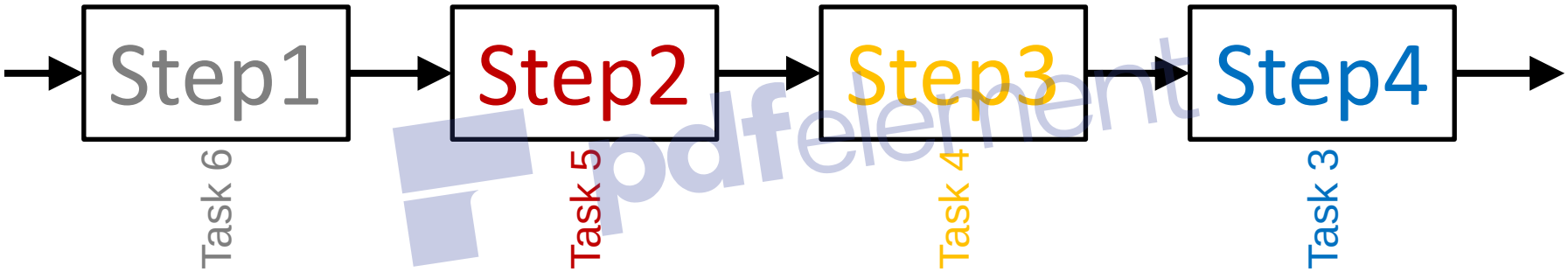
Pipeline



pdfelement

Remove Watermark Now

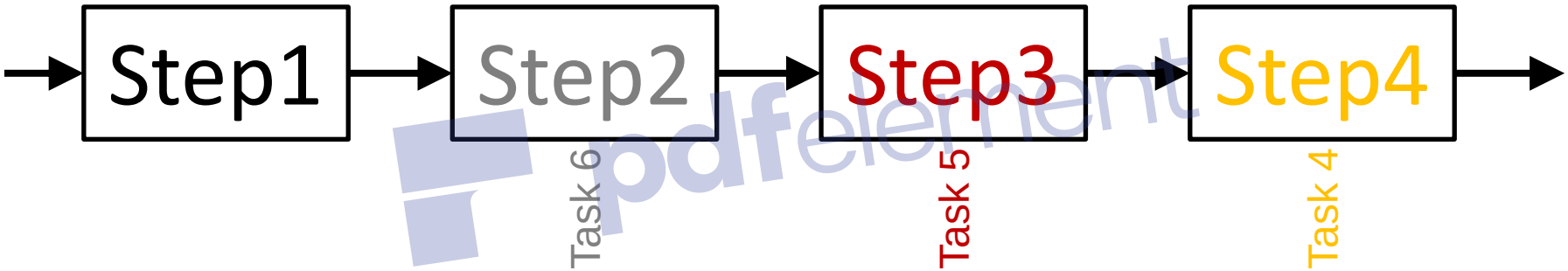
Pipeline



pdfelement

Remove Watermark Now

Pipeline

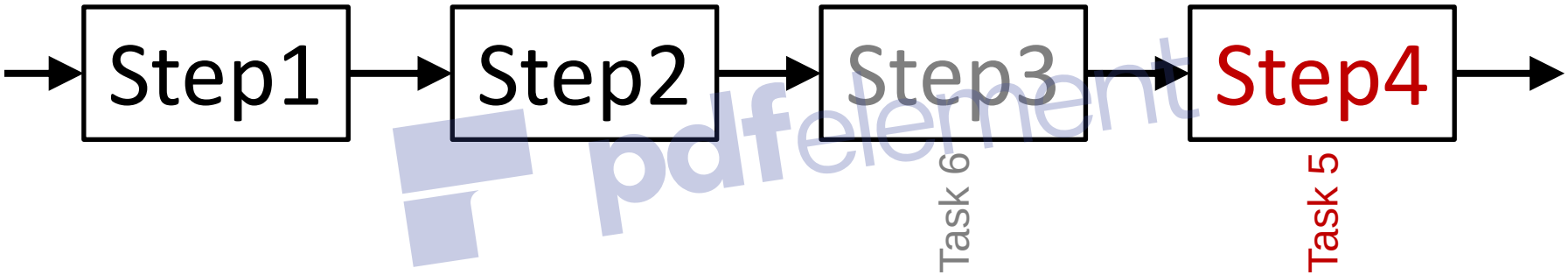


pdfelement

Task 3
Task 2
Task 1

Remove Watermark Now

Pipeline



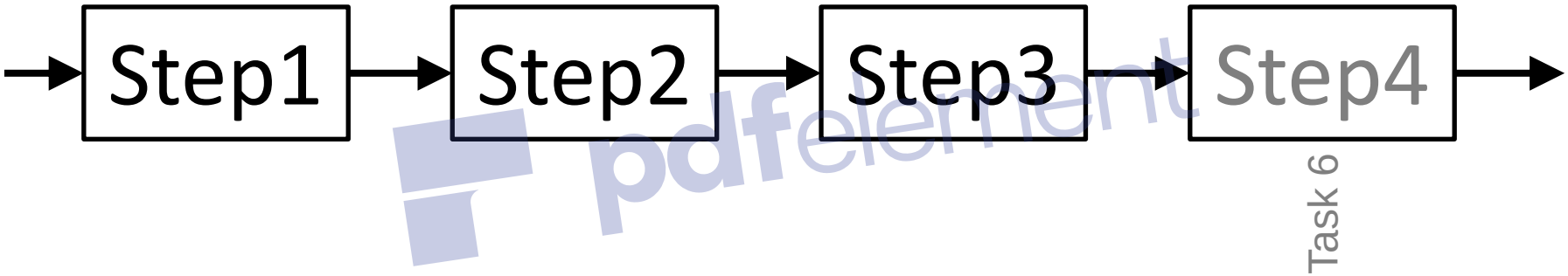
pdfelement

Task 4
Task 3
Task 2
Task 1

Remove Watermark Now

Pipeline

Ideal: $step_execution_time = \frac{task_execution_time}{number_of_steps}$



pdfelement

După $number_of_steps$ tascuri:

$total_execution_time = number_of_tasks * step_execution_time$

Un task se termină la fiecare “step tick”

Task 5 Task 4 Task 3 Task 2 Task 1

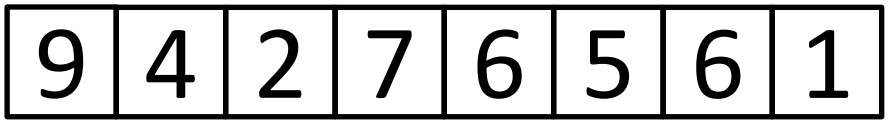
Remove Watermark Now

Sortare...

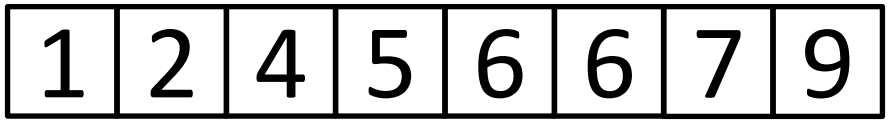
pdfelement

Remove Watermark Now

Sorting

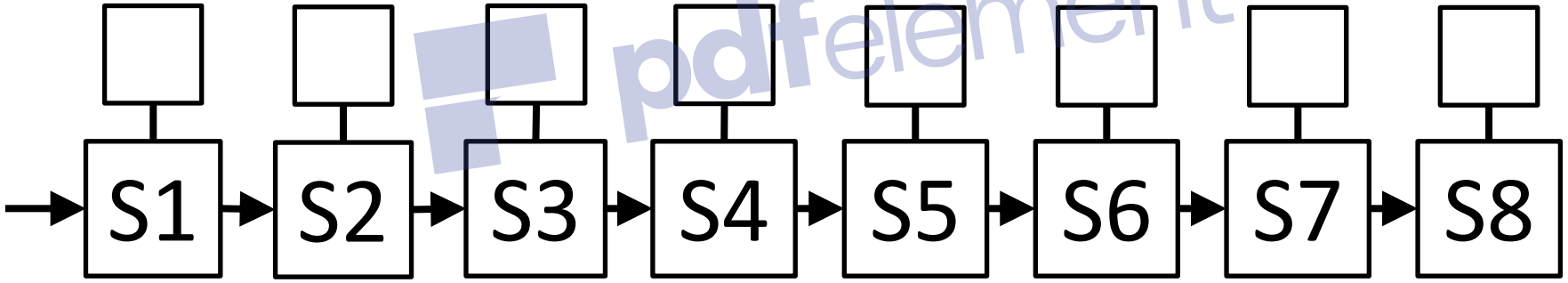


pdfelement



Remove Watermark Now

Sorting with pipeline

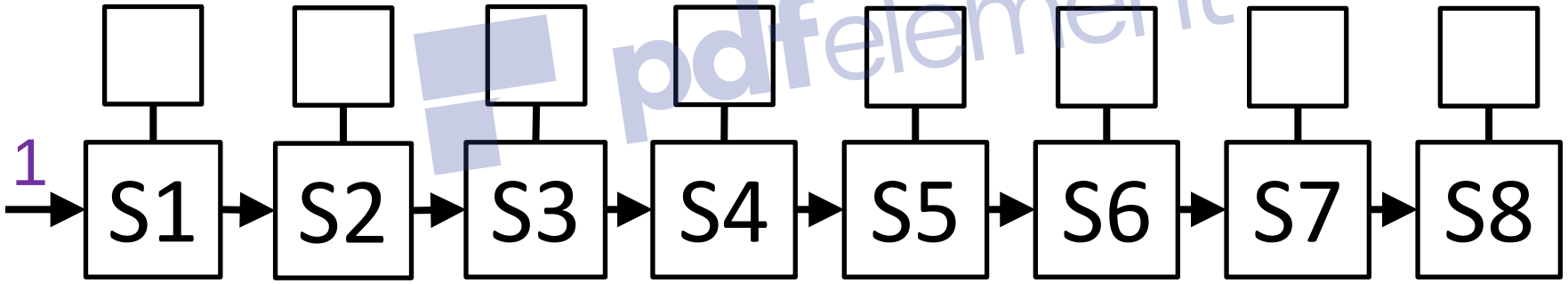


pdfelement

Remove Watermark Now

Sorting with pipeline

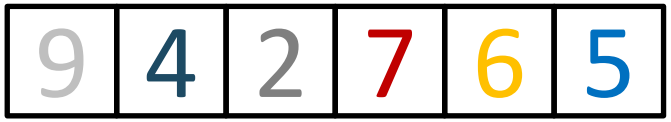
9	4	2	7	6	5	6

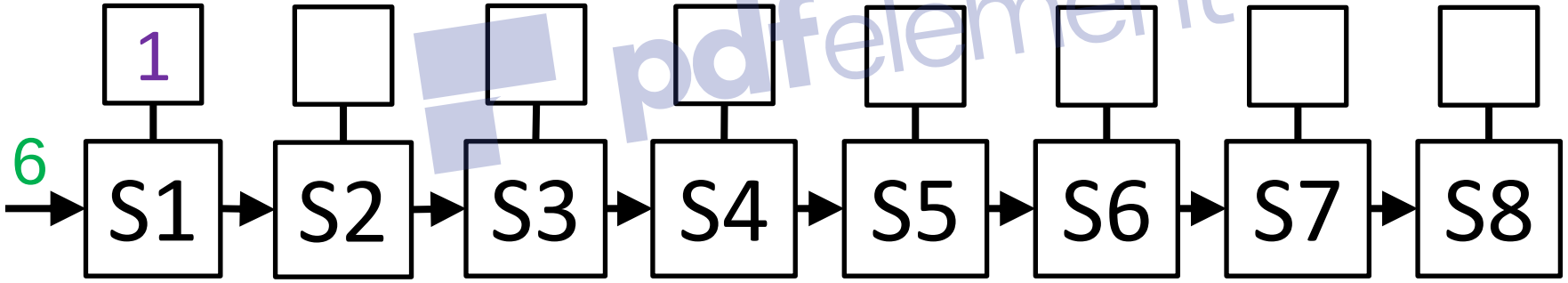


pdfelement

Remove Watermark Now

Sorting with pipeline



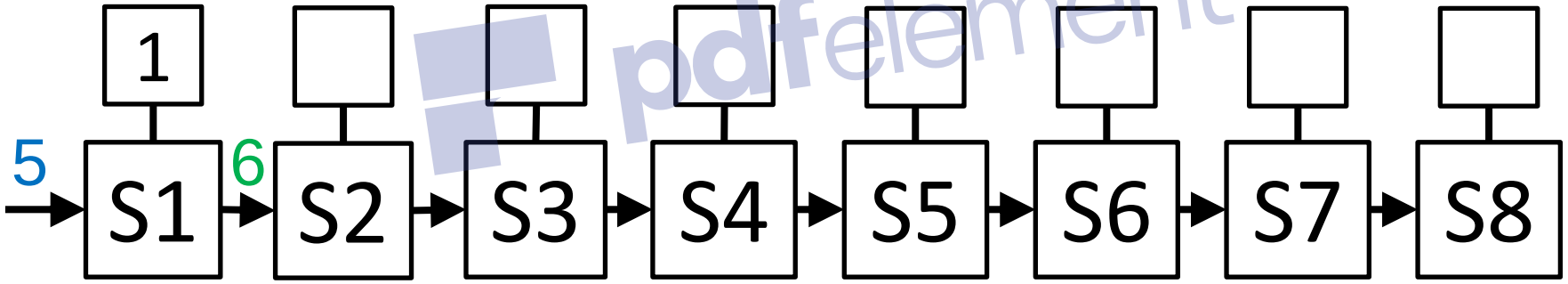


pdfelement

Remove Watermark Now

Sorting with pipeline

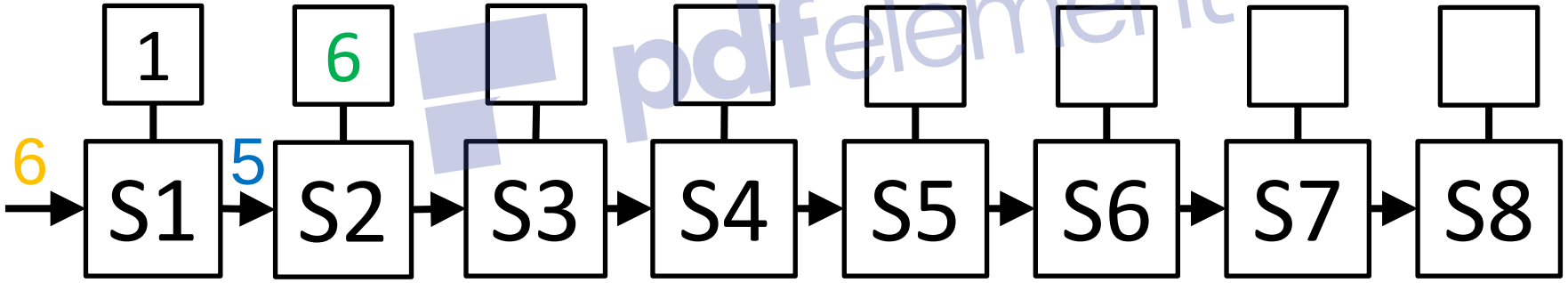
9	4	2	7	6



pdfelement

Remove Watermark Now

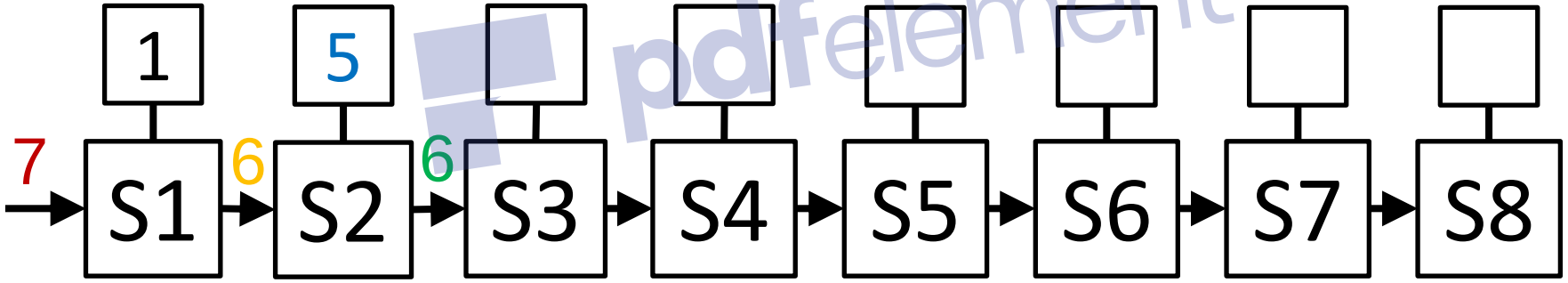
Sorting with pipeline



pdfelement

Remove Watermark Now

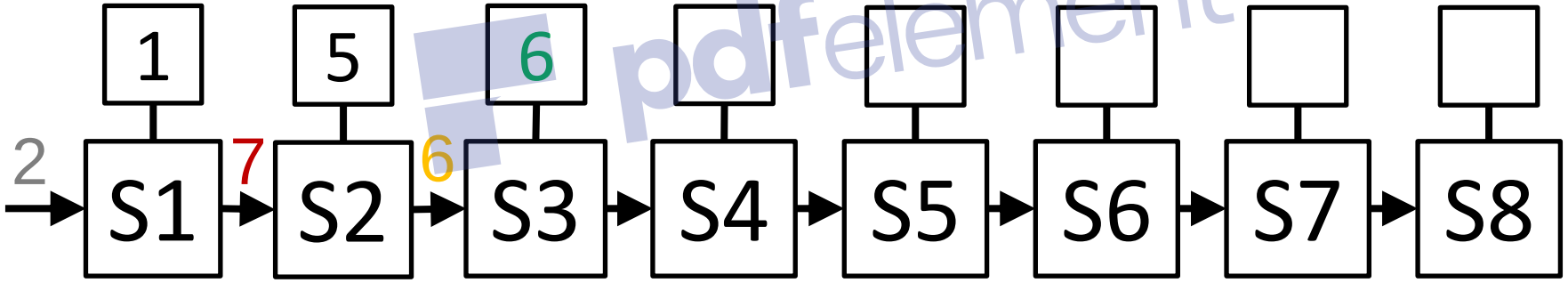
Sorting with pipeline



pdfelement

Remove Watermark Now

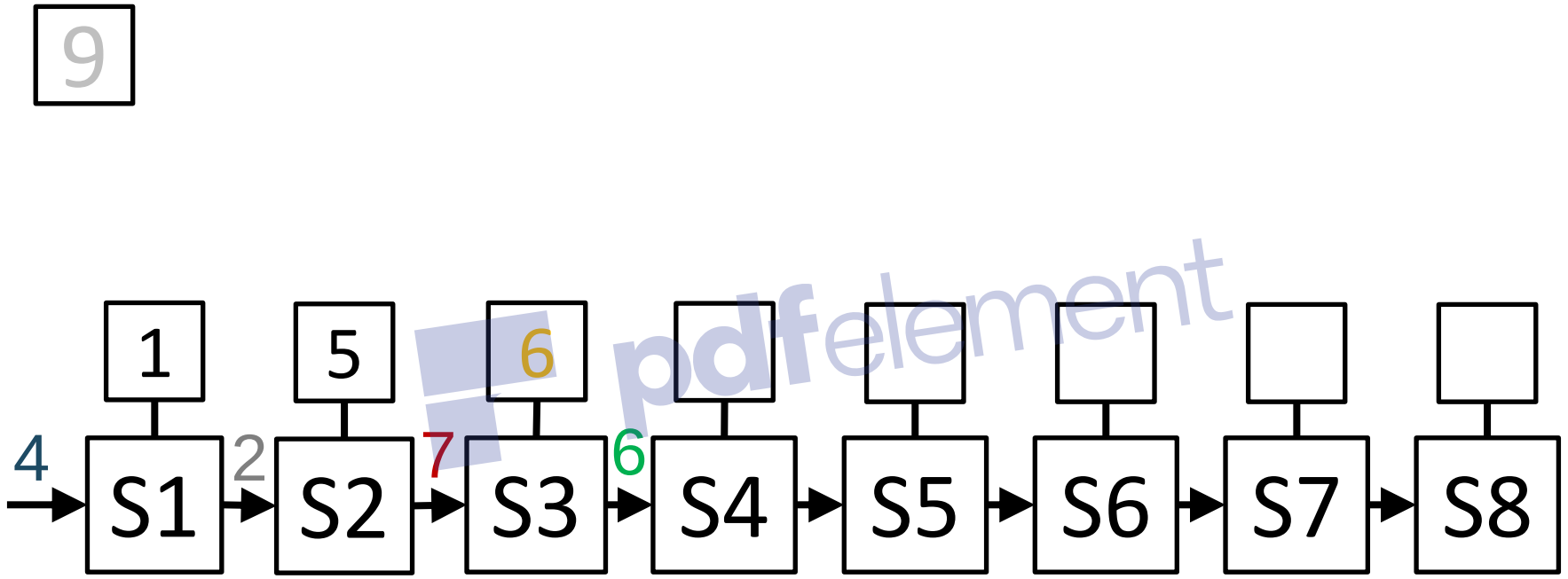
Sorting with pipeline



pdfelement

Remove Watermark Now

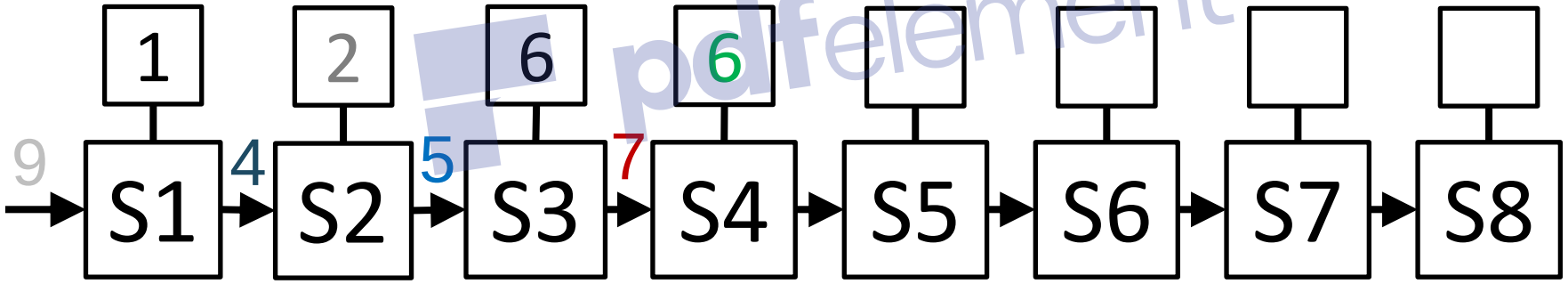
Sorting with pipeline



pdfelement

Remove Watermark Now

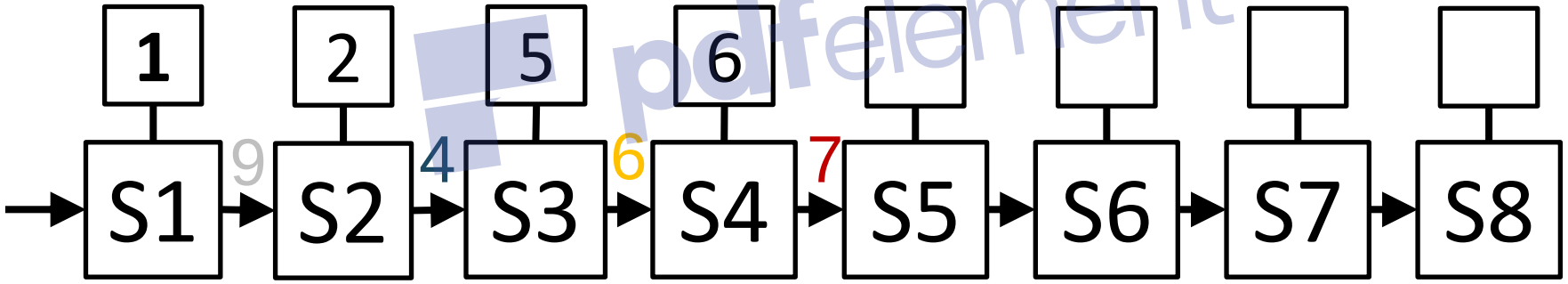
Sorting with pipeline



pdfelement

Remove Watermark Now

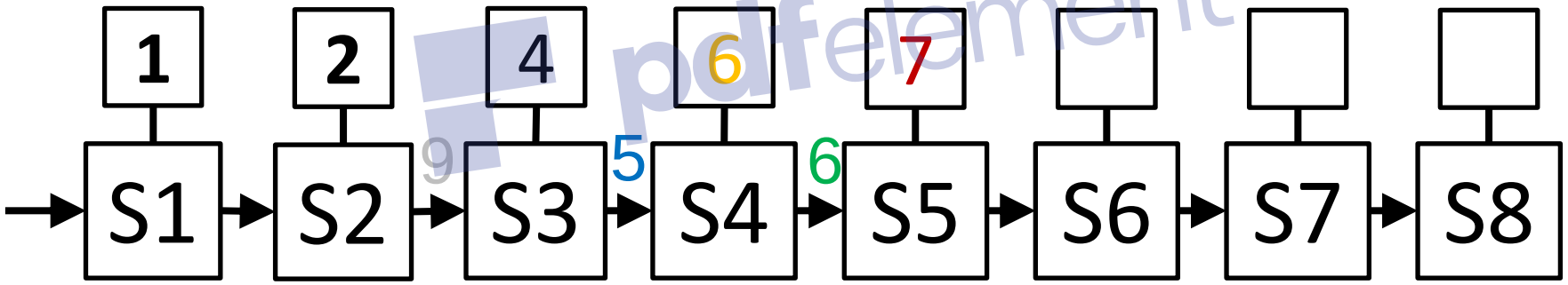
Sorting with pipeline



pdfelement

Remove Watermark Now

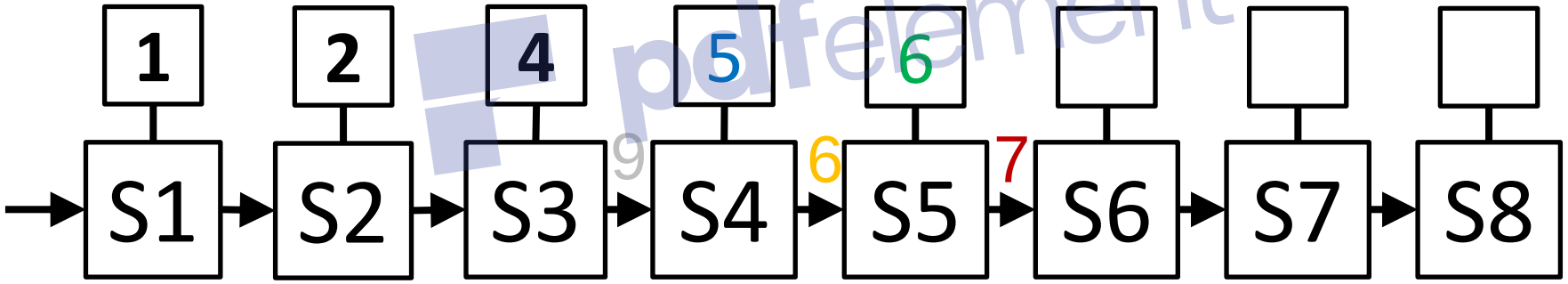
Sorting with pipeline



pdfelement

Remove Watermark Now

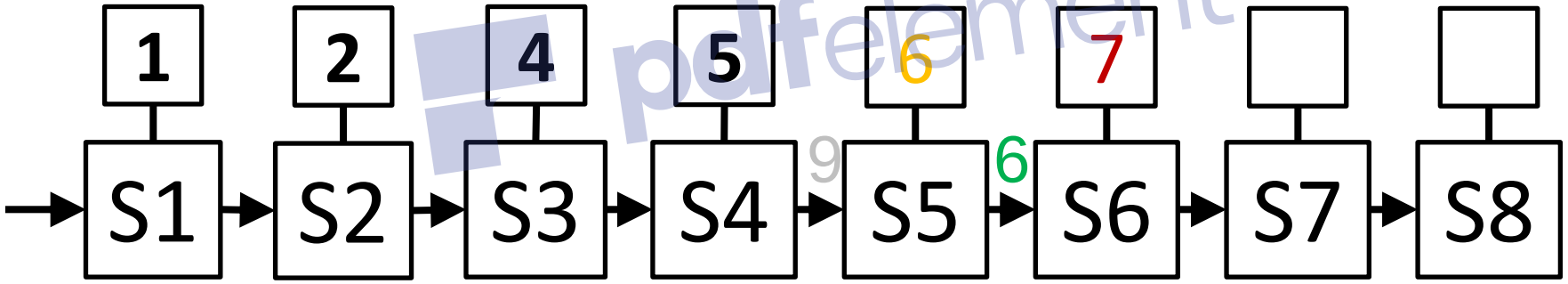
Sorting with pipeline



pdfelement

Remove Watermark Now

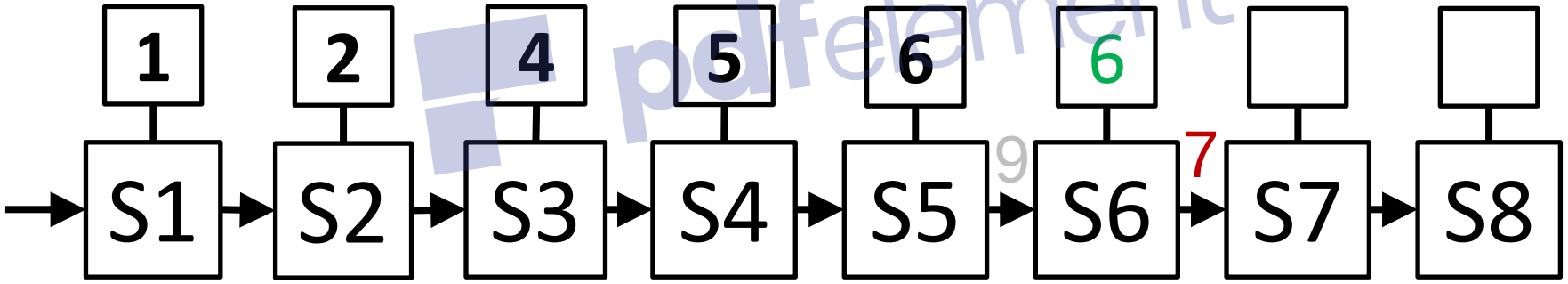
Sorting with pipeline



pdfelement

Remove Watermark Now

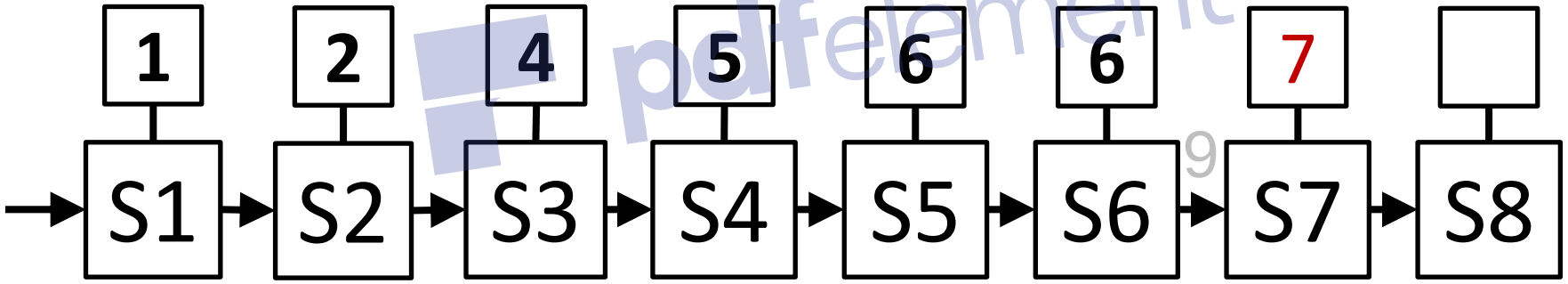
Sorting with pipeline



pdfelement

Remove Watermark Now

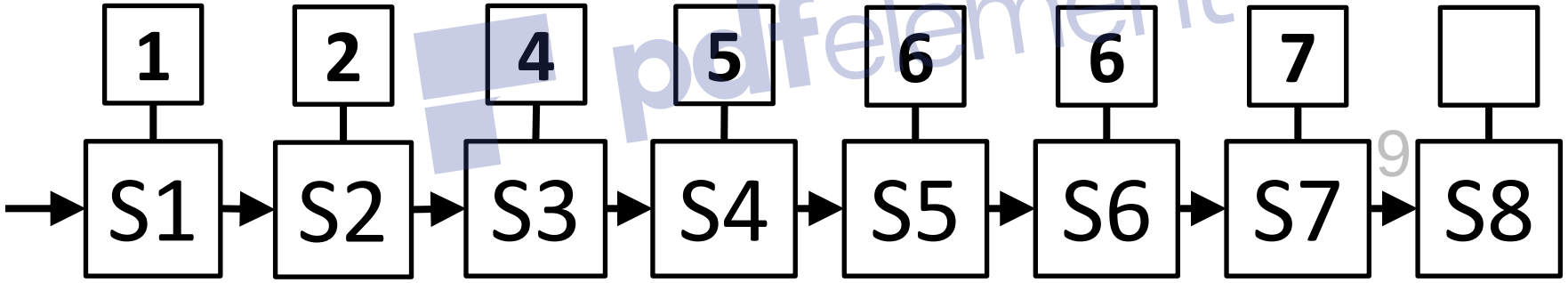
Sorting with pipeline



pdfelement

Remove Watermark Now

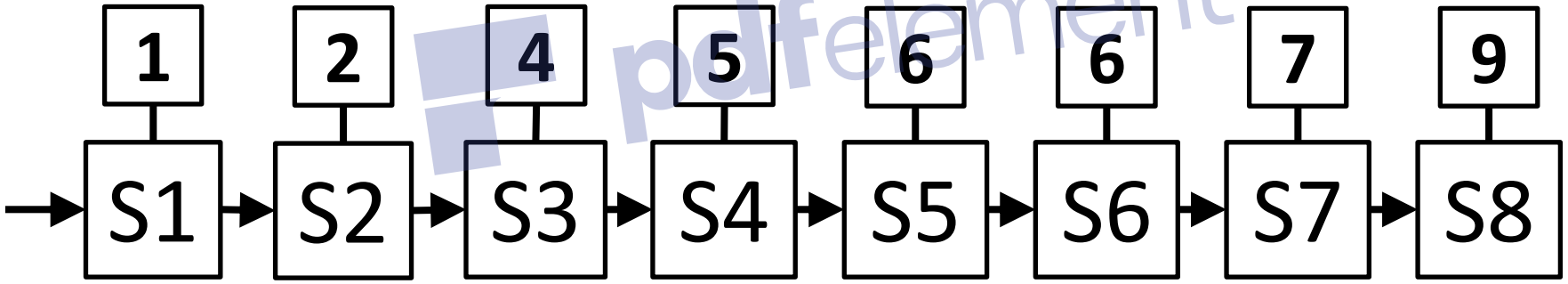
Sorting with pipeline



pdfelement

Remove Watermark Now

Sorting with pipeline



pdfelement

Remove Watermark Now

Calcule polinomiale...

pdfelement

Remove Watermark Now

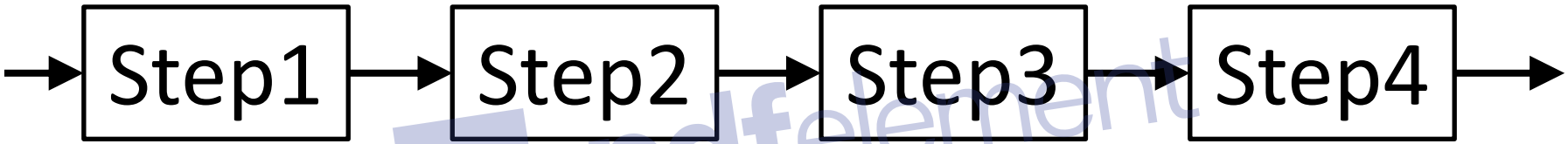
Polynomial

$$P(x) = \sum_{i=0}^{n} a_i x^i = a_0 x^0 + a_1 x^1 + a_2 x^2 + \cdots + a_{n-1} x^{n-1} + a_n x^n, n \geq 0$$

pdfelement

Remove Watermark Now

Pipeline



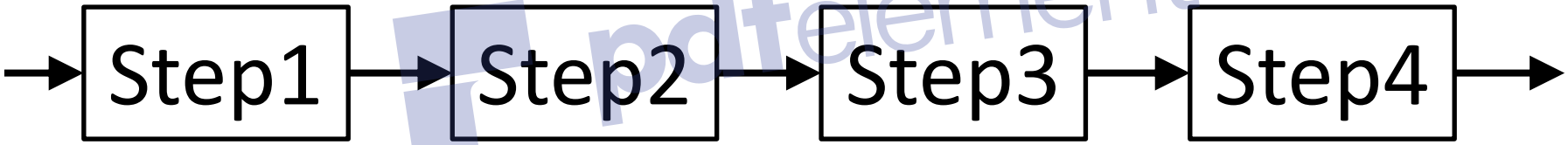
pdfelement

Remove Watermark Now

Polynomial calculation + Pipeline

$$P(x) = 1 + 8x + (-4)x^3 + x^4$$

	0	1	2	3	4
a	1	8	0	-4	1

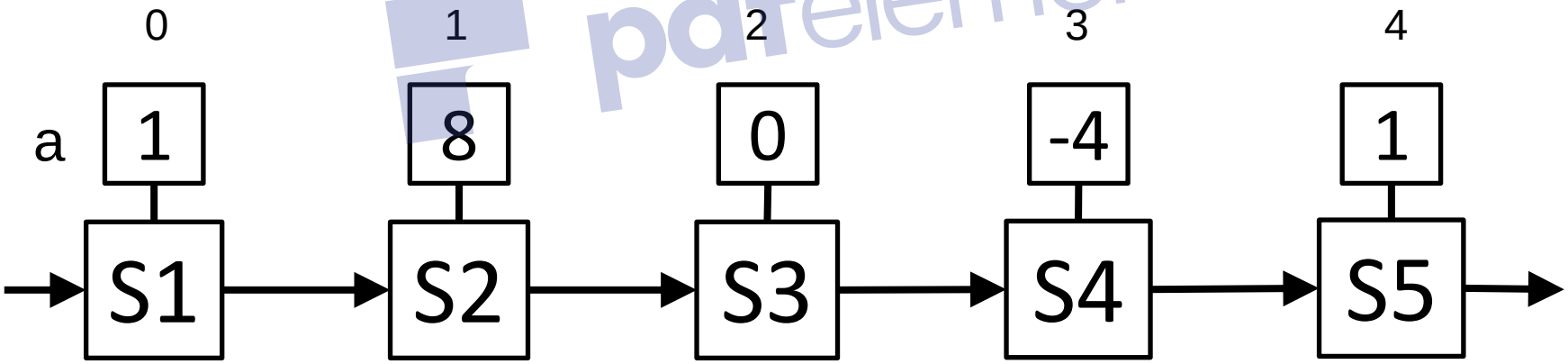


pdfelement

Remove Watermark Now

Polynomial calculation + Pipeline

$$P(x) = 1 + 8x + (-4)x^3 + x^4$$

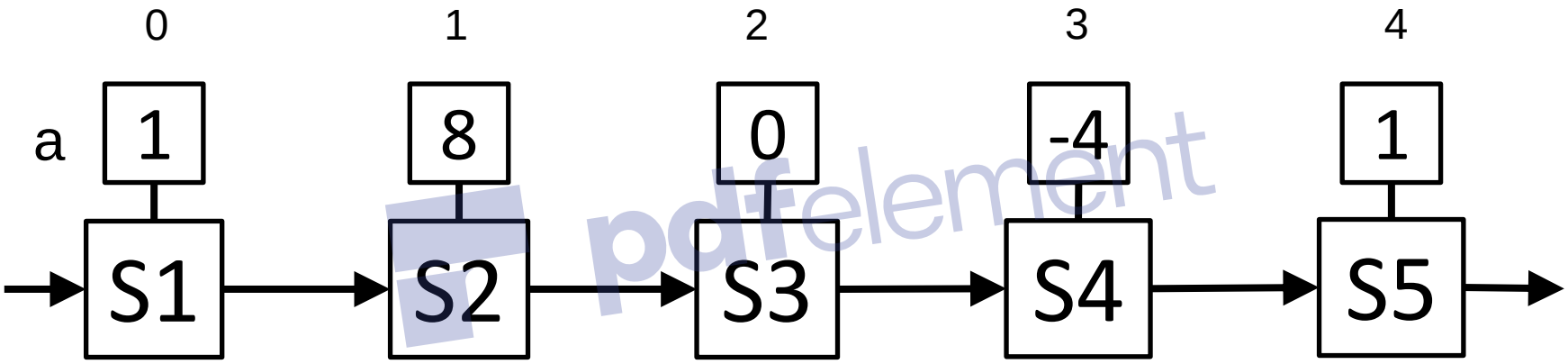


pdfelement

Remove Watermark Now

Polynomial calculation + Pipeline

x_1 x_2 x_3 x_4 x_5

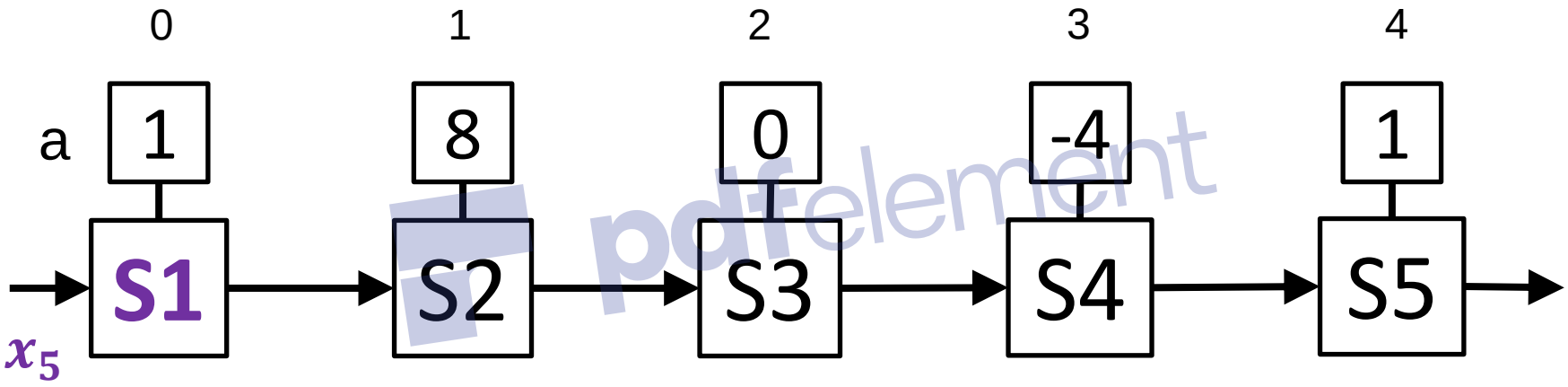


pdfelement

Remove Watermark Now

Polynomial calculation + Pipeline

x_1 x_2 x_3 x_4

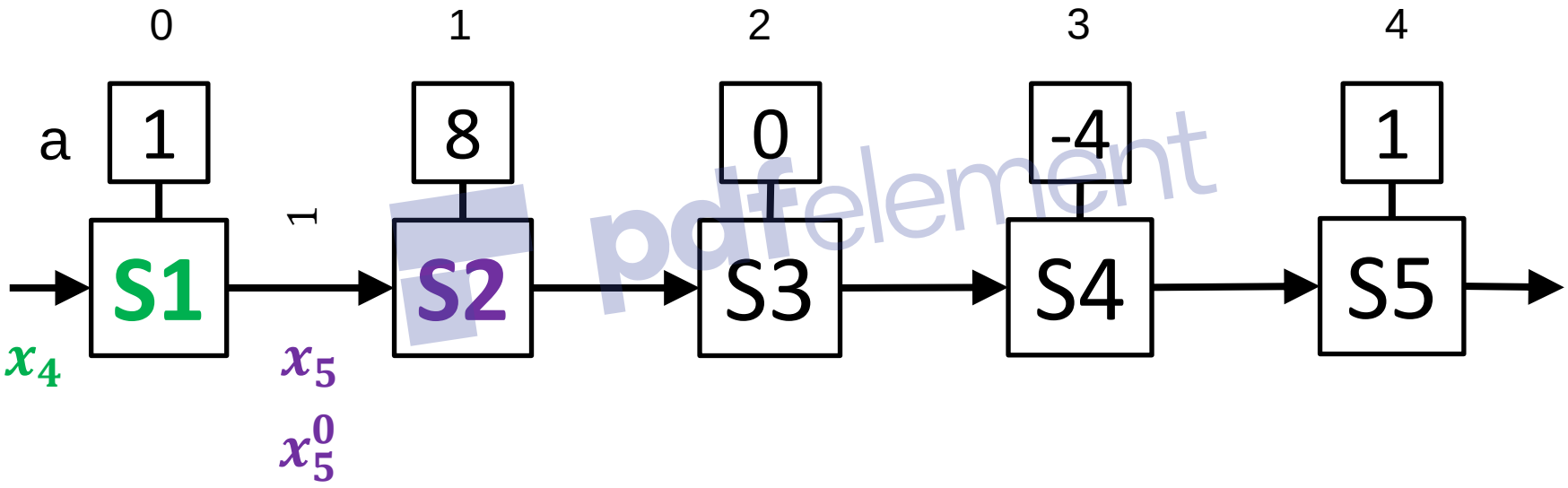


pdfelement

Remove Watermark Now

Polynomial calculation + Pipeline

x_1 x_2 x_3

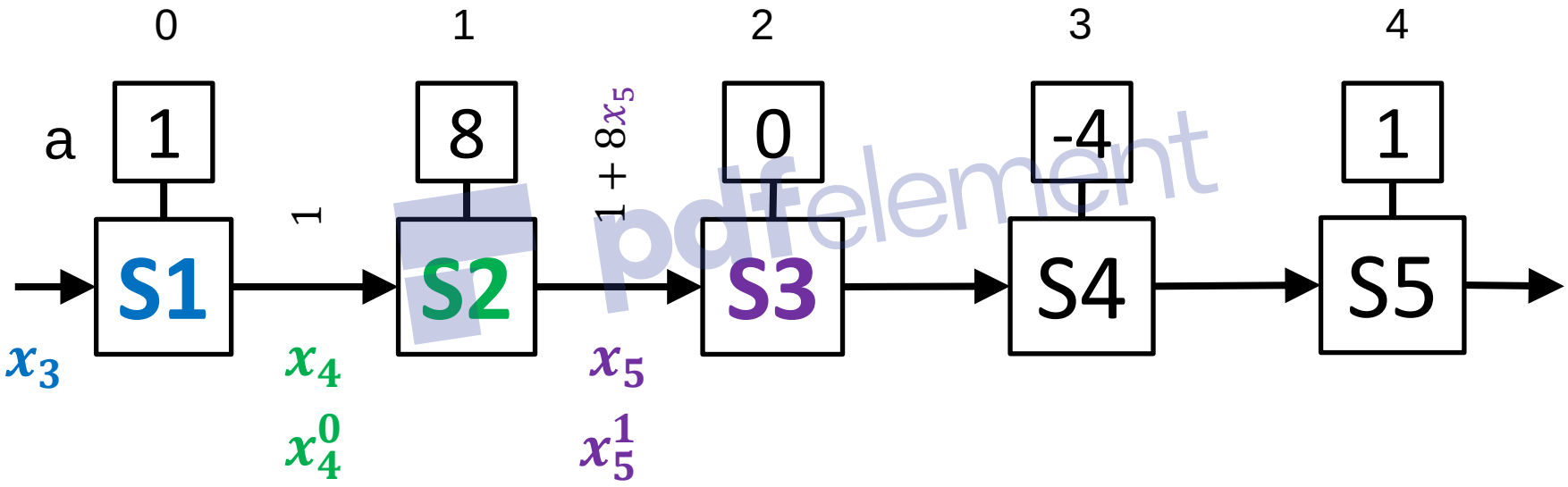


pdfelement

Remove Watermark Now

Polynomial calculation + Pipeline

x_1 x_2

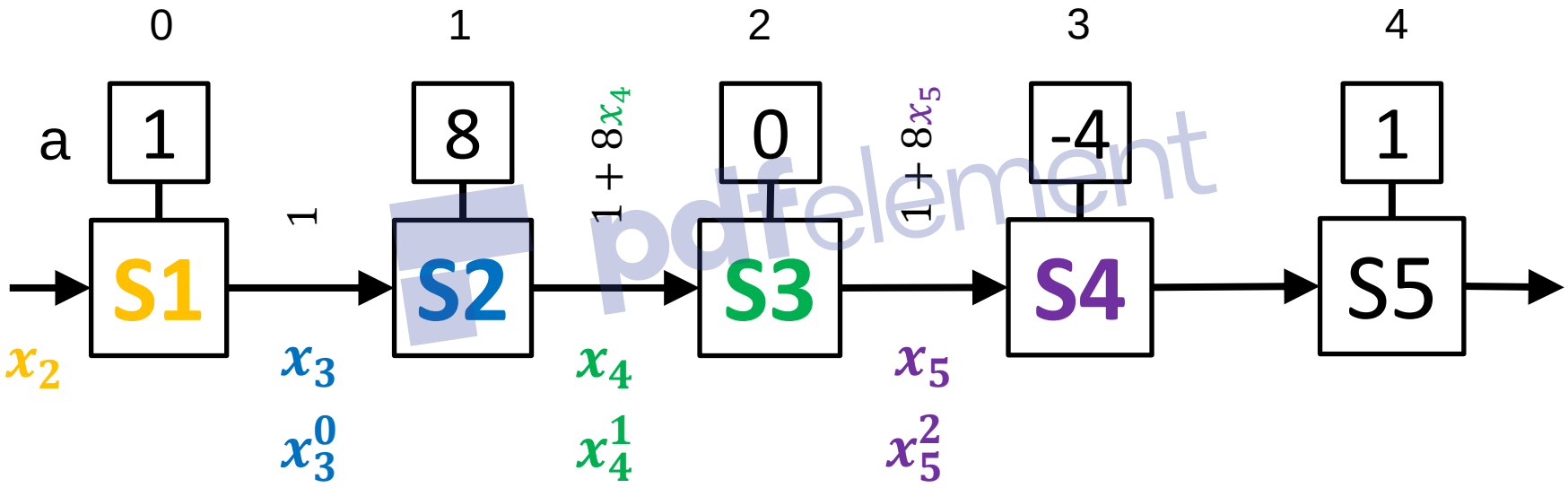


pdfelement

Remove Watermark Now

Polynomial calculation + Pipeline

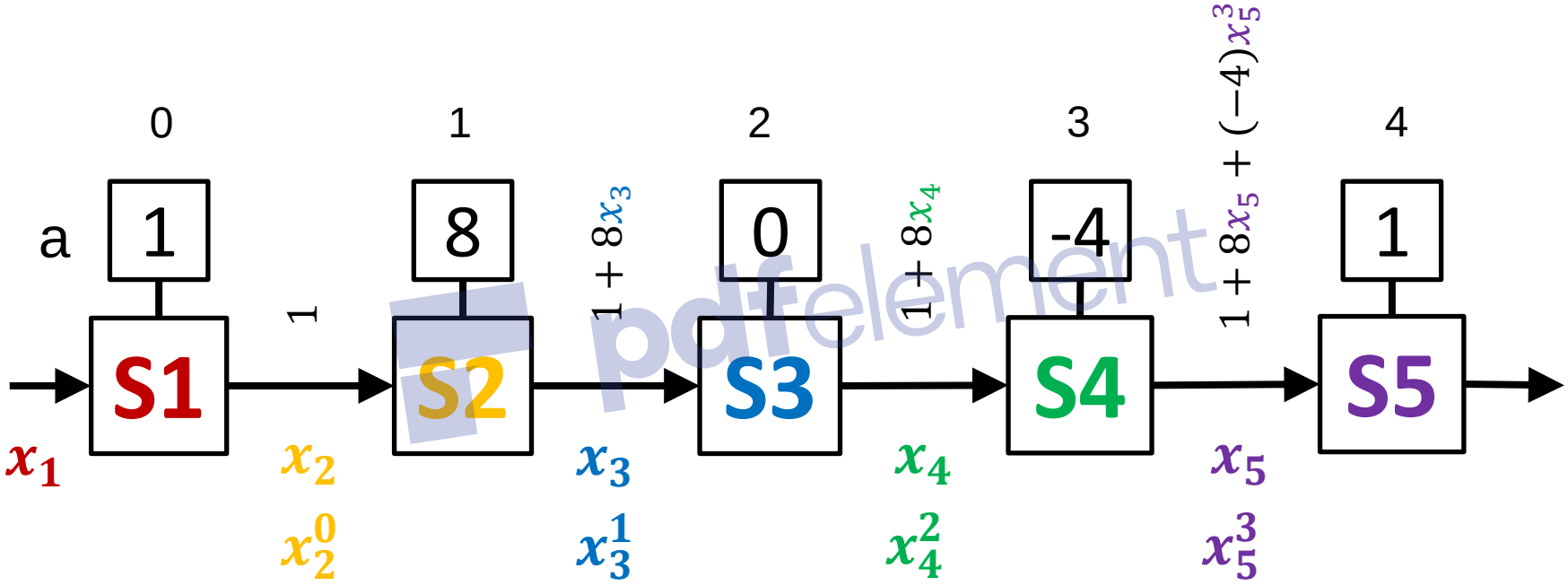
x_1



pdfelement

Remove Watermark Now

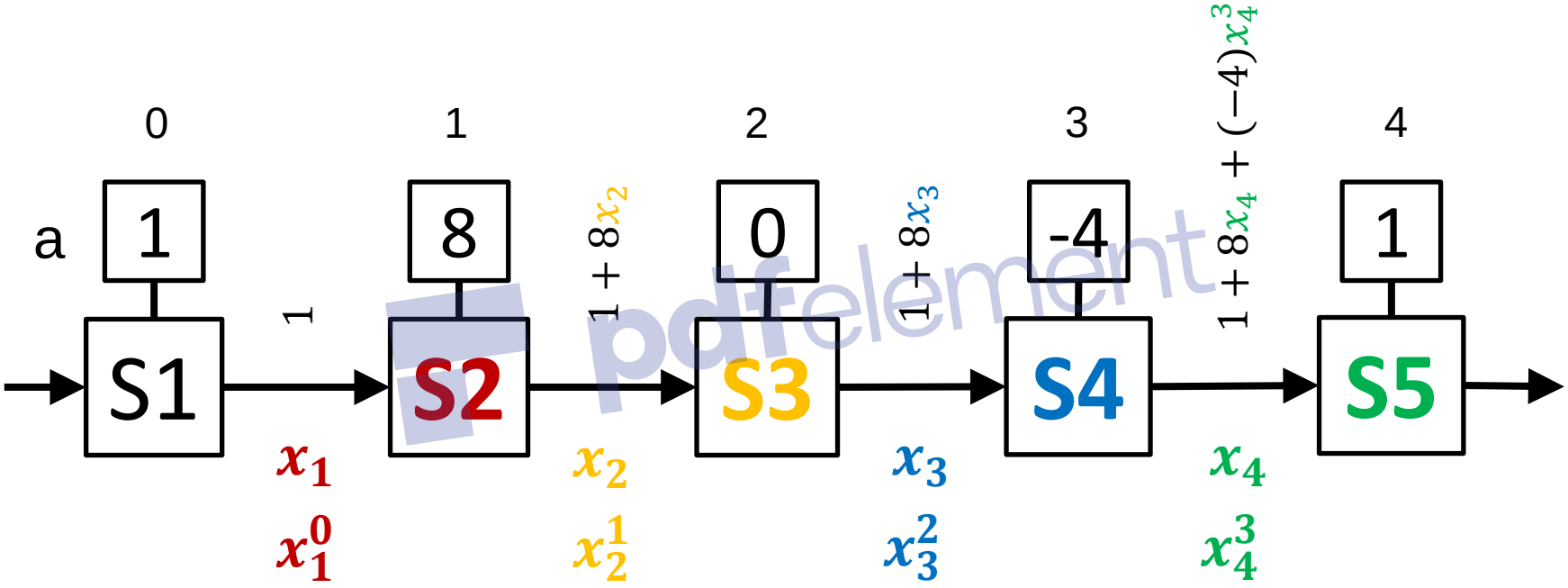
Polynomial calculation + Pipeline



pdfelement

Remove Watermark Now

Polynomial calculation + Pipeline

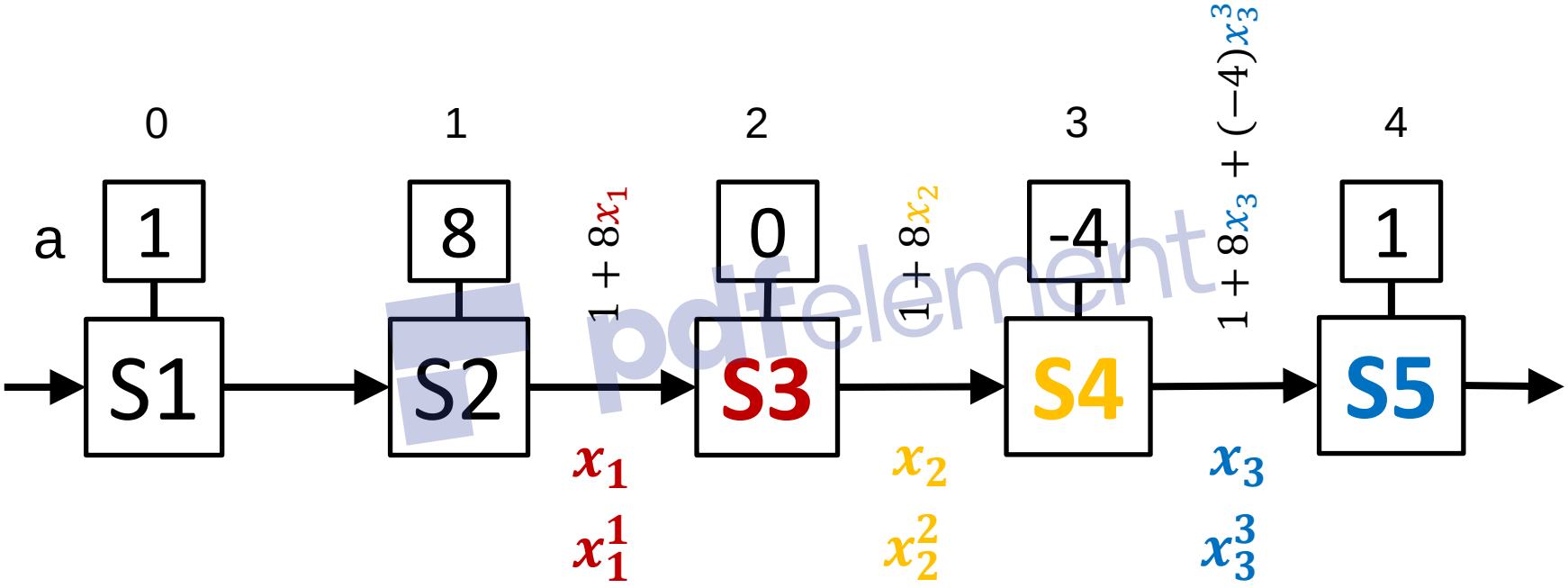


pdfelement

$$\mathbf{1}+\mathbf{8}x_5+(\mathbf{-4})x_5^3+x_5^4$$

Remove Watermark Now

Polynomial calculation + Pipeline



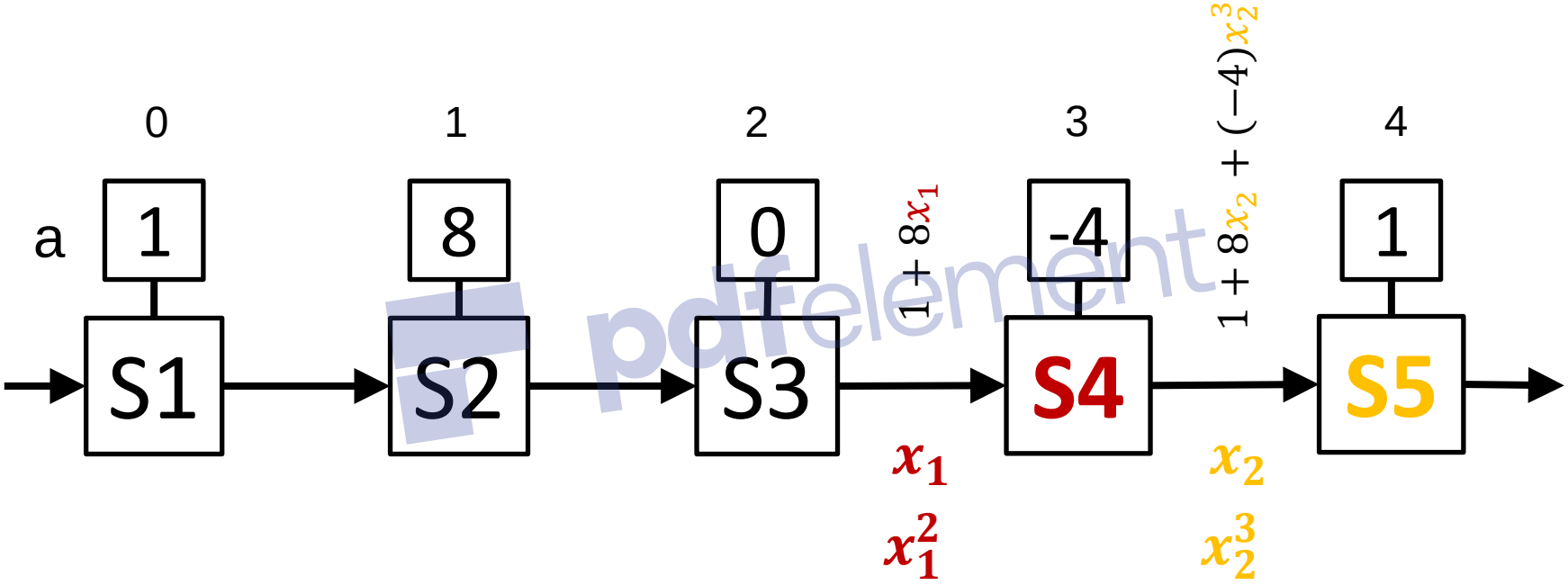
pdfelement

$$1+8x_4+(-4)x_4^3+x_4^4$$

$$1+8x_5+(-4)x_5^3+x_5^4$$

Remove Watermark Now

Polynomial calculation + Pipeline



pdfelement

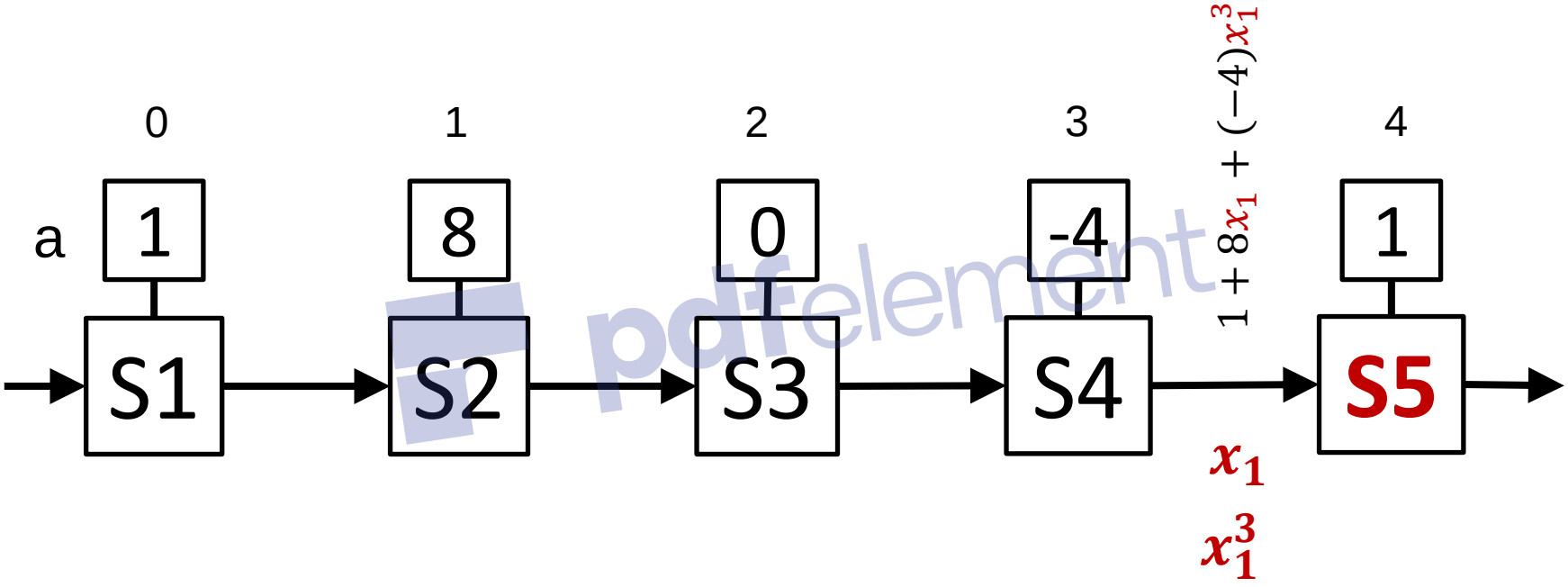
$$1+8x_3+(-4)x_3^3+x_3^4$$

$$1+8x_4+(-4)x_4^3+x_4^4$$

$$1+8x_5+(-4)x_5^3+x_5^4$$

Remove Watermark Now

Polynomial calculation + Pipeline



pdfelement

$1 + 8x_3 + (-4)x_3^3 + x_3^4$

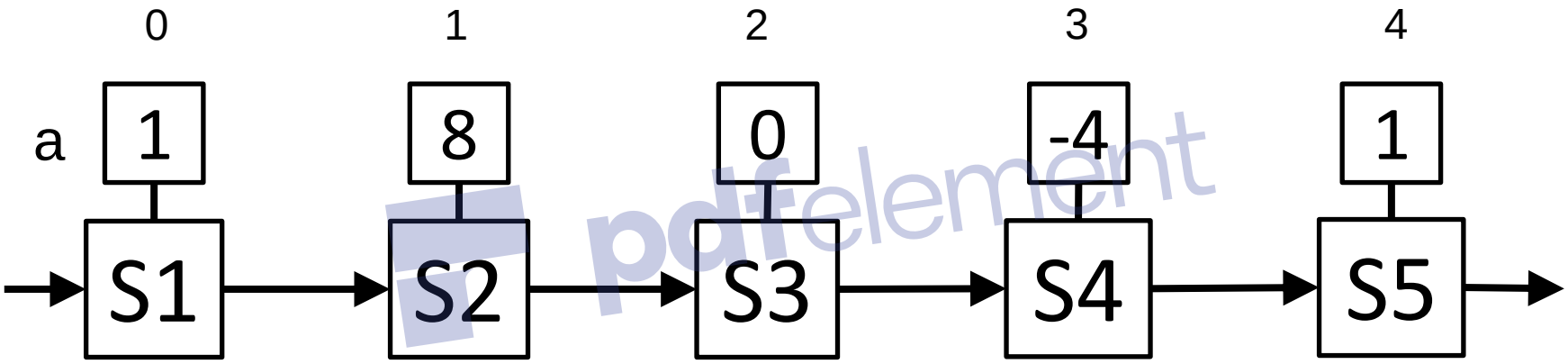
$1 + 8x_4 + (-4)x_4^3 + x_4^4$

$1 + 8x_2 + (-4)x_2^3 + x_2^4$

$1 + 8x_5 + (-4)x_5^3 + x_5^4$

Remove Watermark Now

Polynomial calculation + Pipeline



pdfelement

$$1 + 8x_3 + (-4)x_3^3 + x_3^4$$

$$1 + 8x_1 + (-4)x_1^3 + x_1^4 \qquad 1 + 8x_4 + (-4)x_4^3 + x_4^4$$

$$1 + 8x_2 + (-4)x_2^3 + x_2^4 \qquad 1 + 8x_5 + (-4)x_5^3 + x_5^4$$

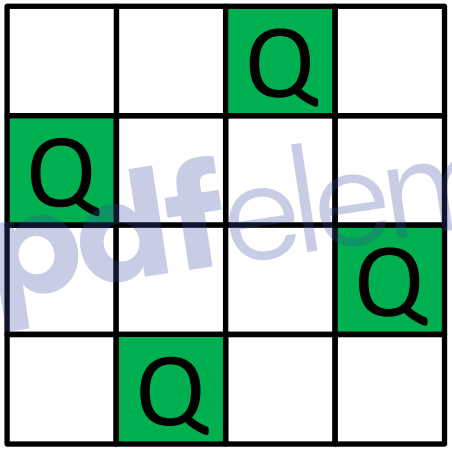
Remove Watermark Now

Aplicație: problema Damelor

pdfelement

Remove Watermark Now

N Queens Problem

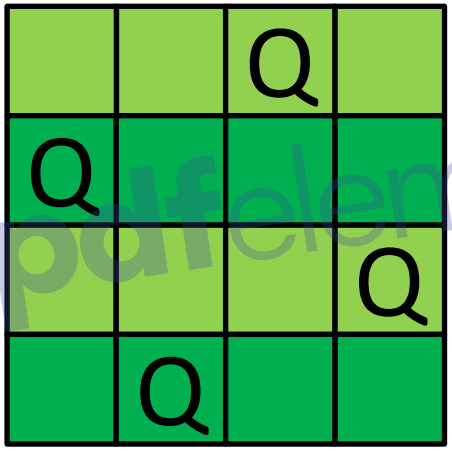


pdfelement

Remove Watermark Now

N Queens Problem

No more then one queen per line

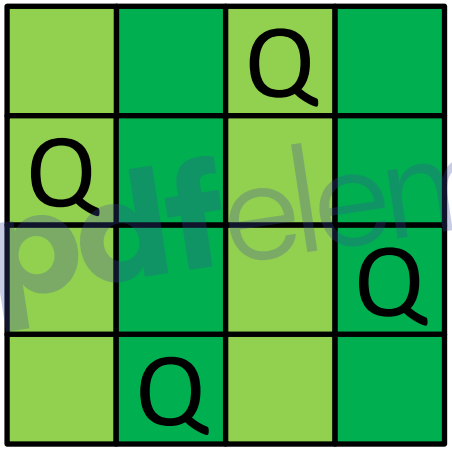


pdfelement

Remove Watermark Now

N Queens Problem

No more then one queen per column

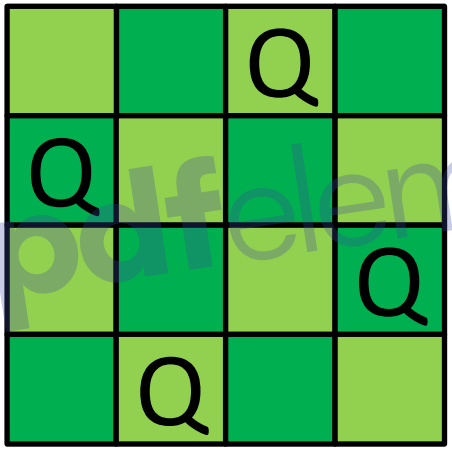


pdfelement

Remove Watermark Now

N Queens Problem

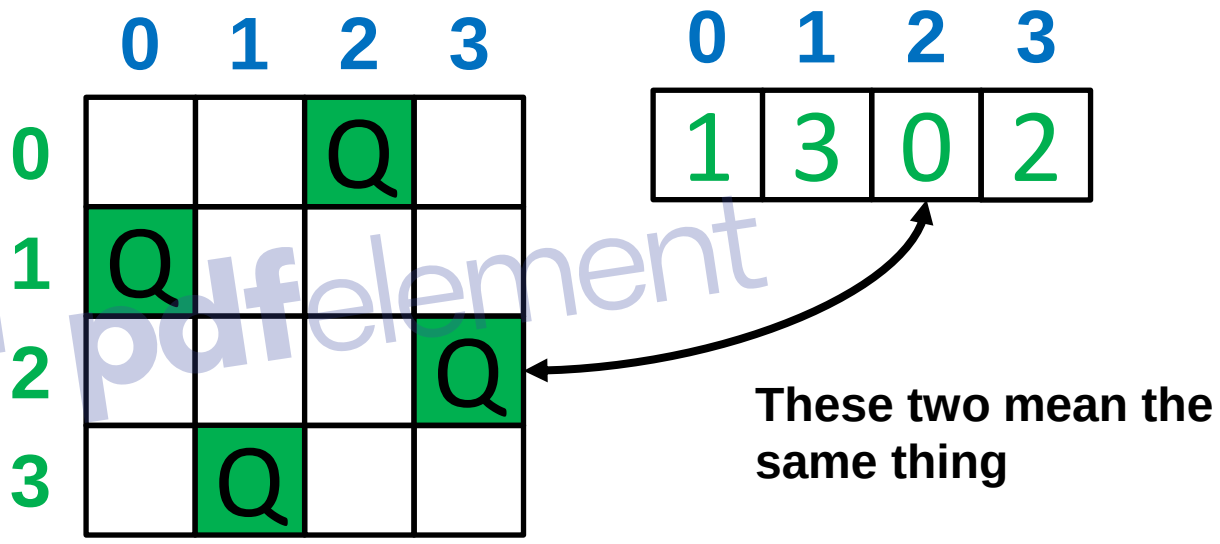
No more then one queen per diagonals



pdfelement

Remove Watermark Now

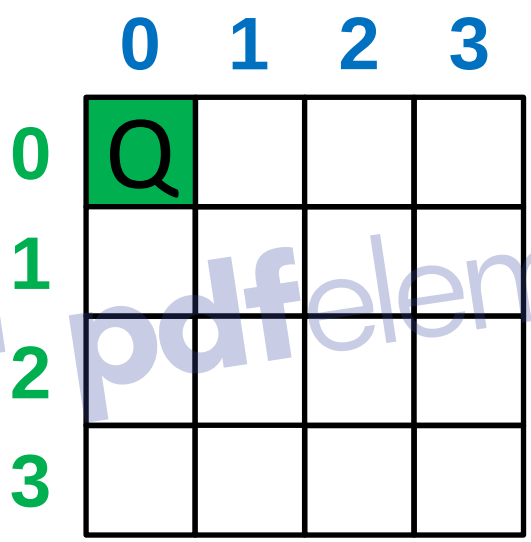
N Queens Problem

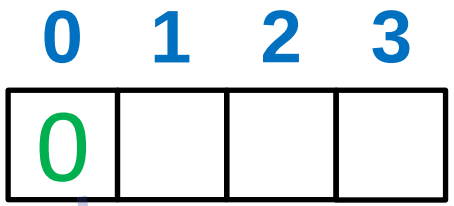


pdfelement

Remove Watermark Now

N Queens Problem - Solution

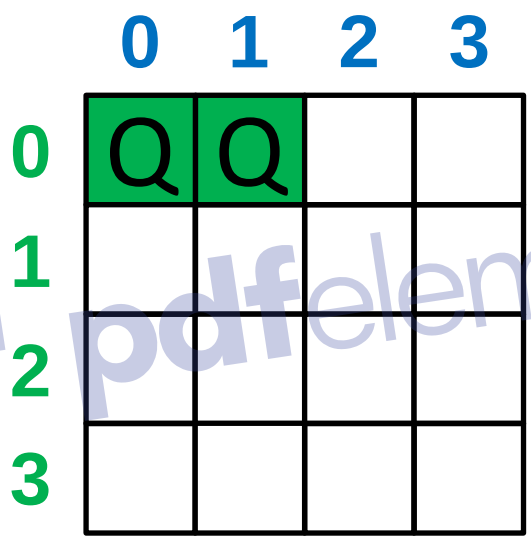


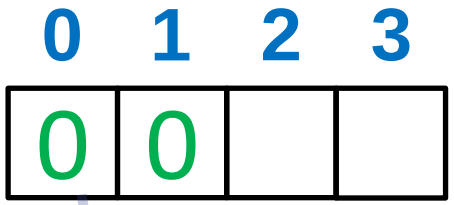


pdfelement

Remove Watermark Now

N Queens Problem - Solution



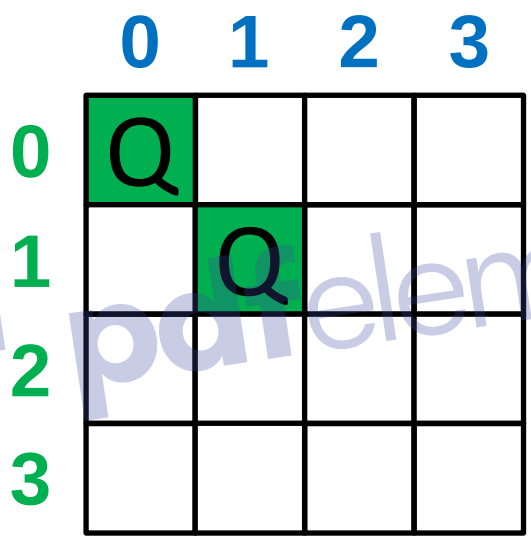


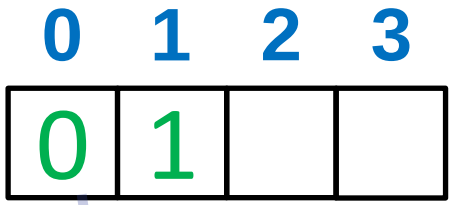
Line conflict

pdfelement

Remove Watermark Now

N Queens Problem - Solution



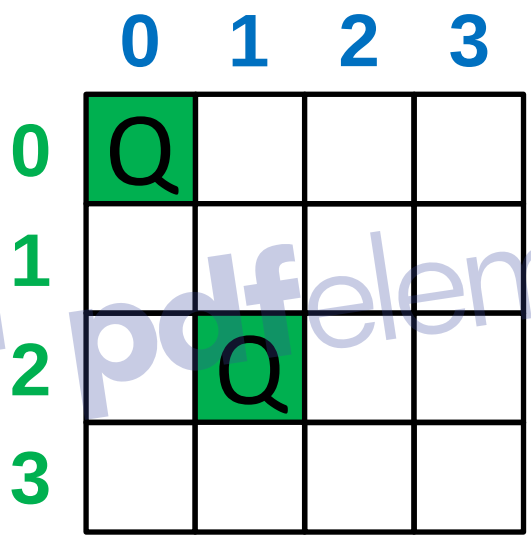


Diagonal conflict

partelement

Remove Watermark Now

N Queens Problem - Solution



0	1	2	3
0	2		

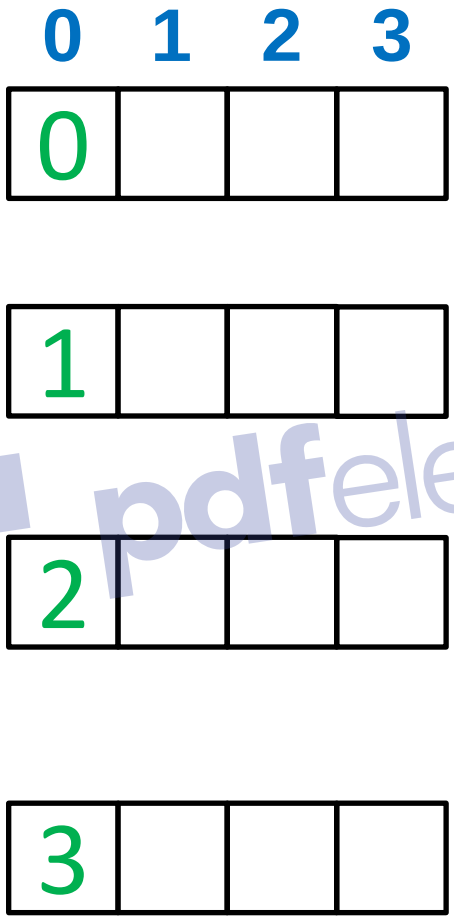
OK.

And so on…

pdfelement

Remove Watermark Now

N Queens Problem – Parallel Solution



pdfelement

Remove Watermark Now

N Queens Problem – Parallel Solution

0	1	2	3	
0	0			x
0	1			x
0	2			
0	3			

0	1	2	3	
1	0			x
1	1			x
1	2			x
1	3			

pdfelement

And so on…

0	1	2	3	
2	0			
2	1			x
2	2			x
2	3			x

0	1	2	3	
3	0			
3	1			
3	2			x
3	3			x

Remove Watermark Now

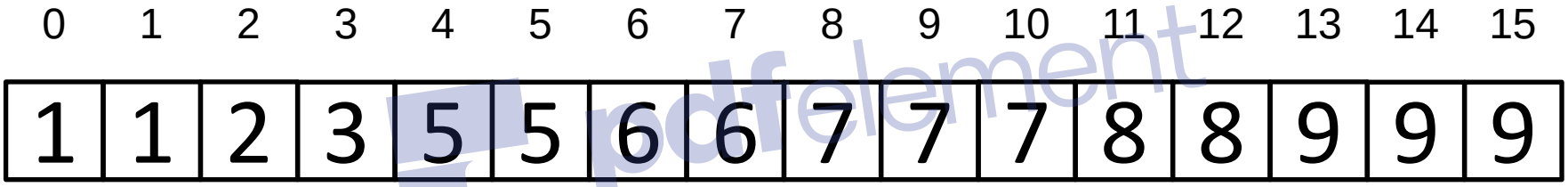
Cautare binara

pdfelement

Remove Watermark Now

Binary Search

Searching for 3



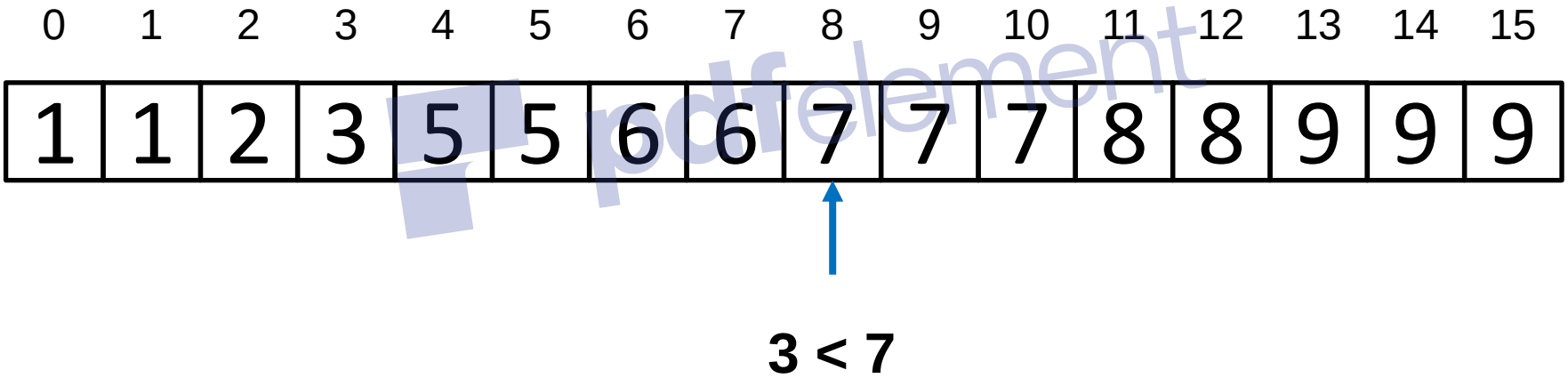
pdfelement

Remove Watermark Now

Binary Search

Searching for | 3 |

Interest area | 0 | 15 |



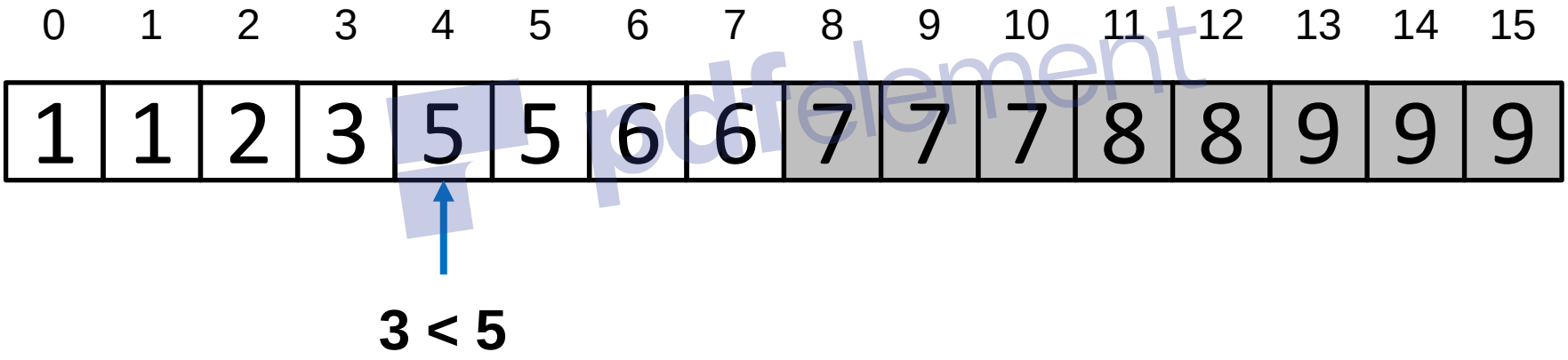
pdfelement

Remove Watermark Now

Binary Search

Searching for | 3 |

Interest area | 0 | 7 |



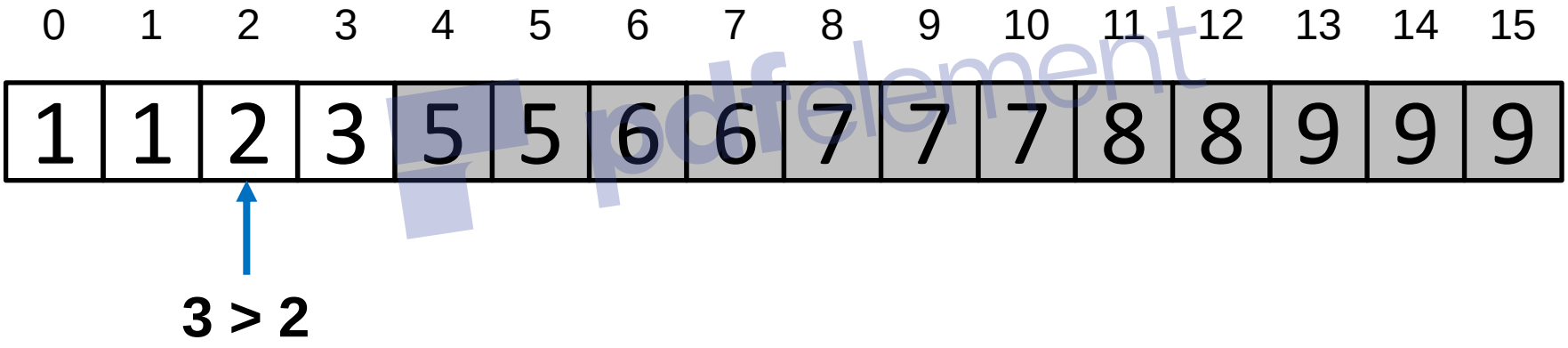
pdfelement

Remove Watermark Now

Binary Search

Searching for | 3 |

Interest area | 0 | 3 |



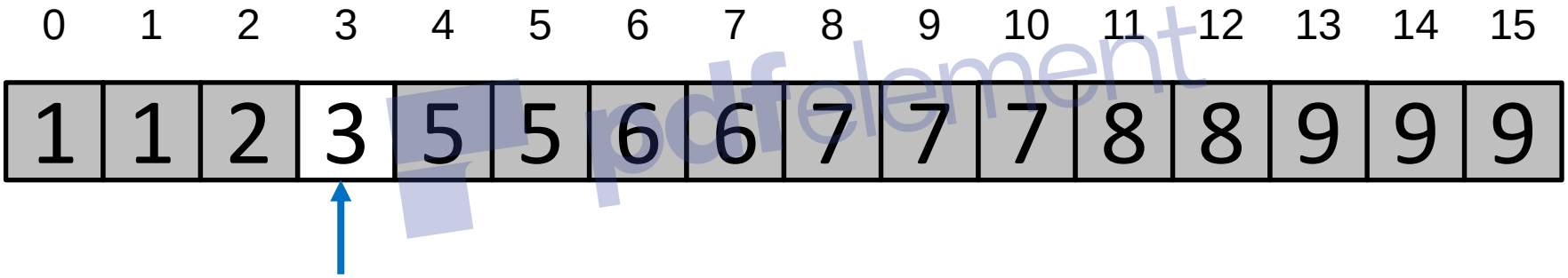
pdfelement

Remove Watermark Now

Binary Search

Searching for | 3 |

Interest area | 3 | 3 |



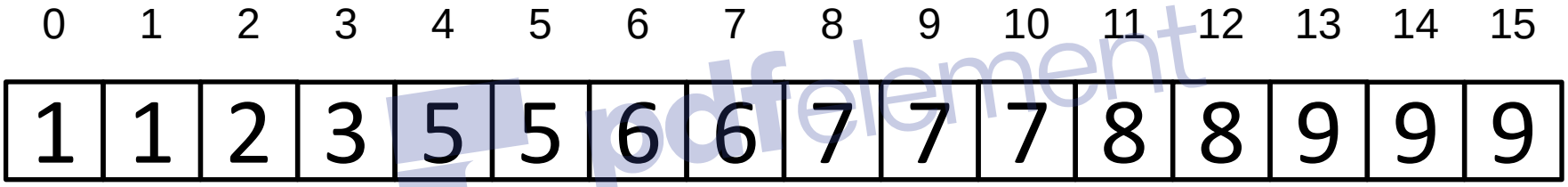
pdfelement

O($log_2(n)$) time

Remove Watermark Now

Parallel Search

Searching for 3



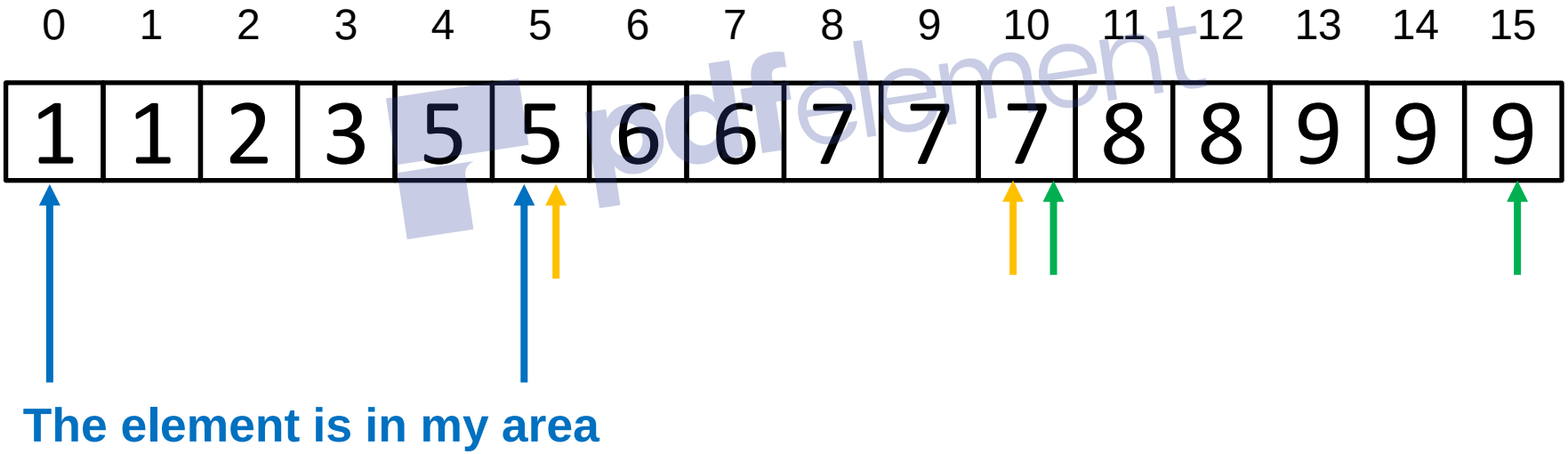
pdfelement

Remove Watermark Now

Parallel Search

Searching for 3

Interest area 0 15



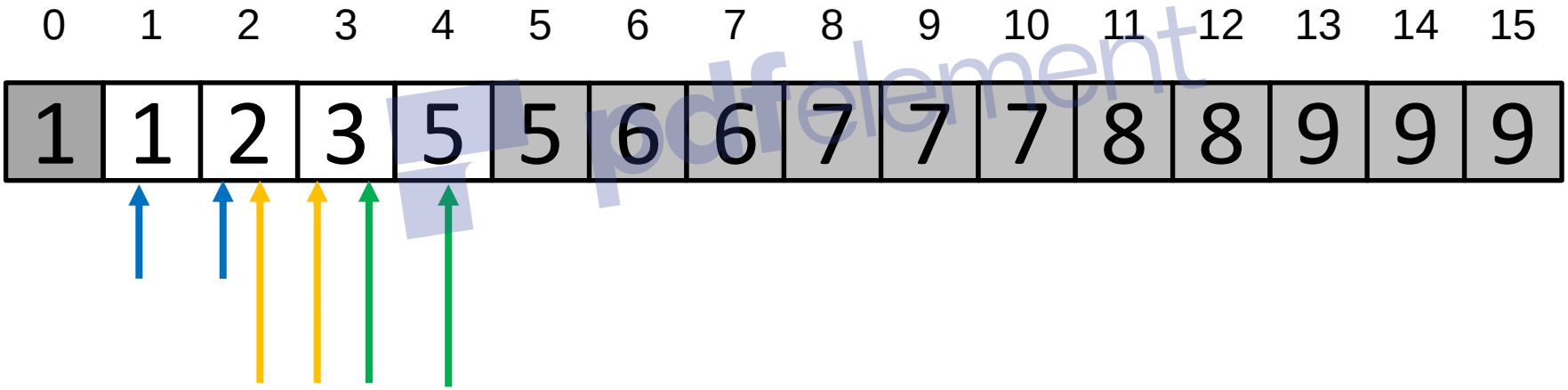
pdfelement

Remove Watermark Now

Parallel Search

Searching for 3

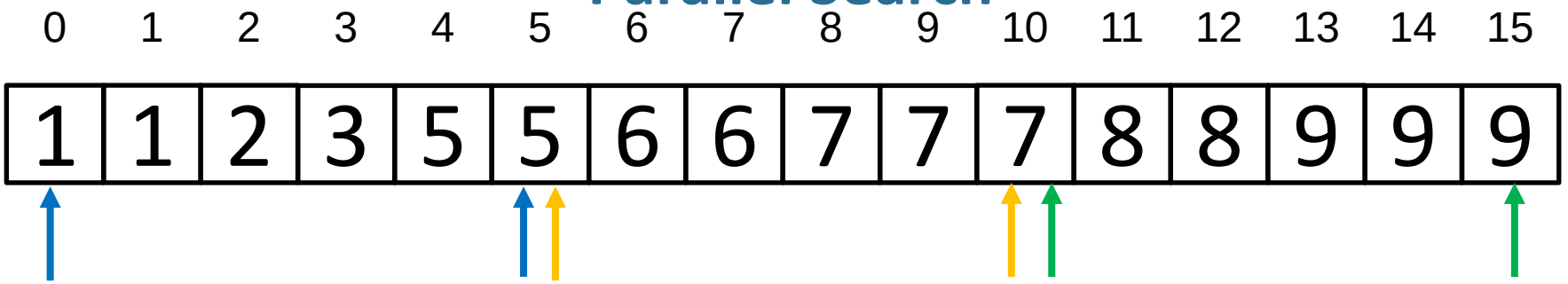
Interest area 1 4



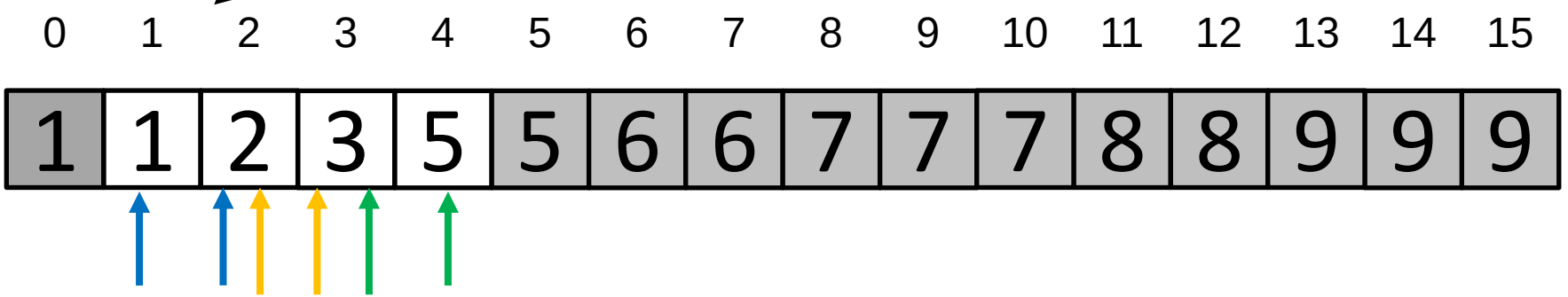
pdfelement

Remove Watermark Now

Parallel Search



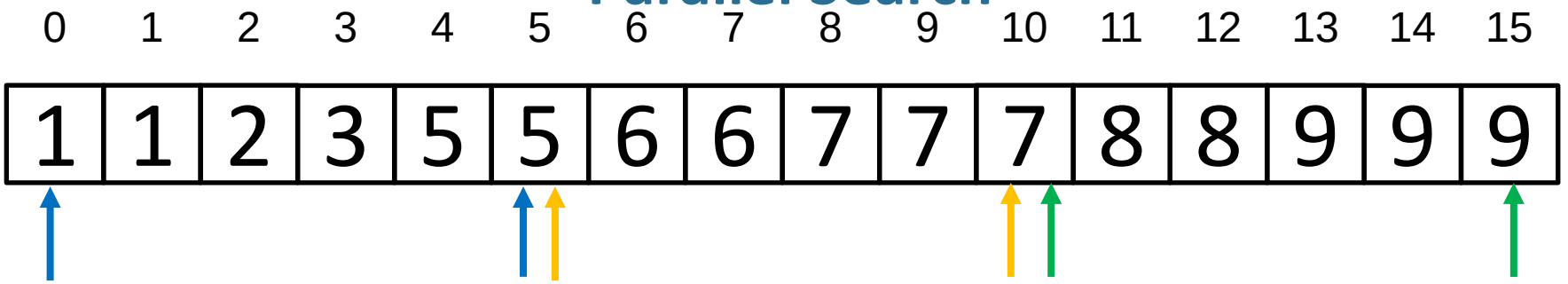
Operations can**NOT** be executed in parallel



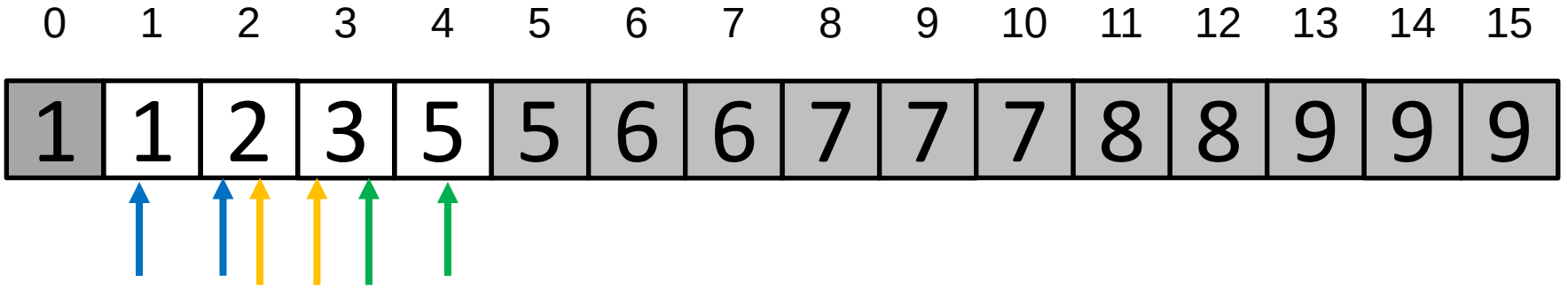
pdfelement

Remove Watermark Now

Parallel Search



pdfelement



Remove Watermark Now

Parallel Search – solutia 2 (cu mai putine threaduri)

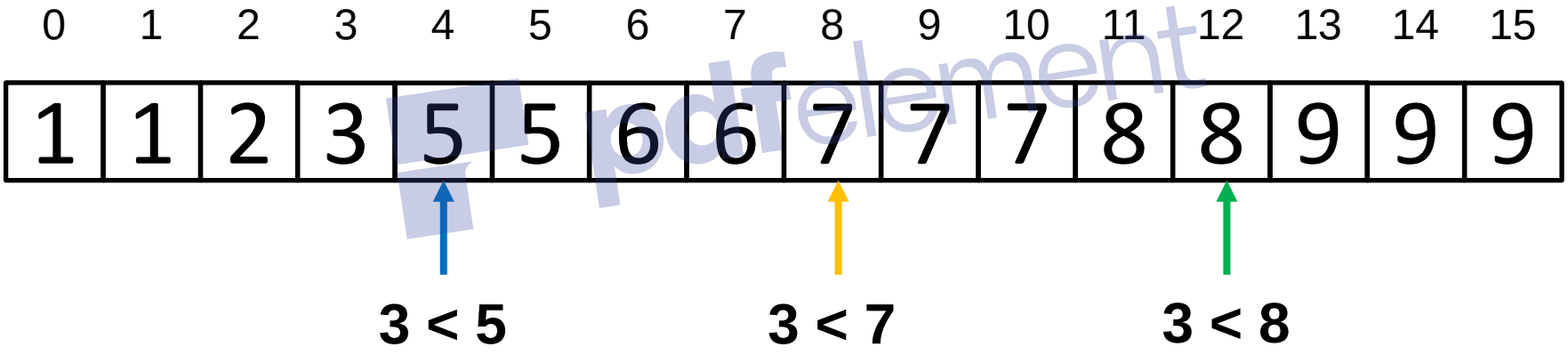
pdfelement

Remove Watermark Now

Parallel Search – solution 2 (fewer threads)

Searching for | 3 |

Interest area | 0 | 15 |



pdfelement

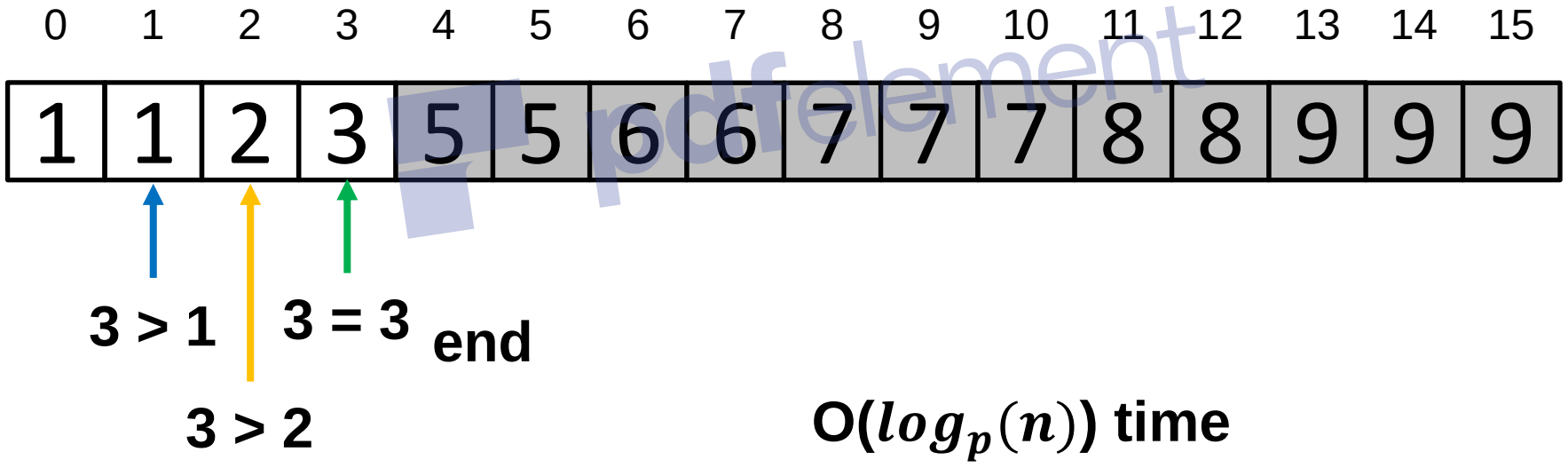
Operations can be executed in parallel

Remove Watermark Now

Parallel Search

Searching for | 3 |

Interest area | 0 | 3 |

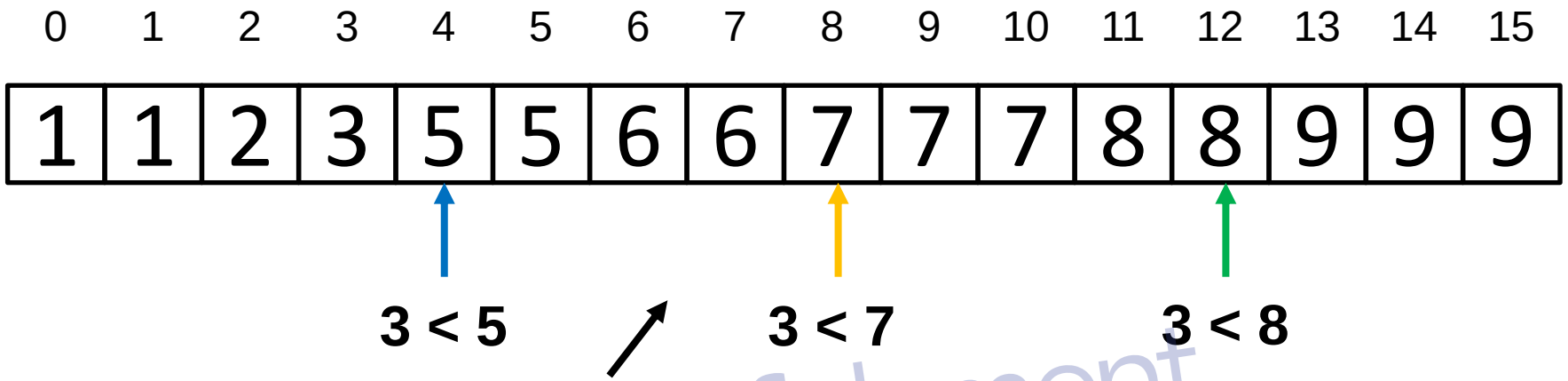


O($log_p(n)$) time

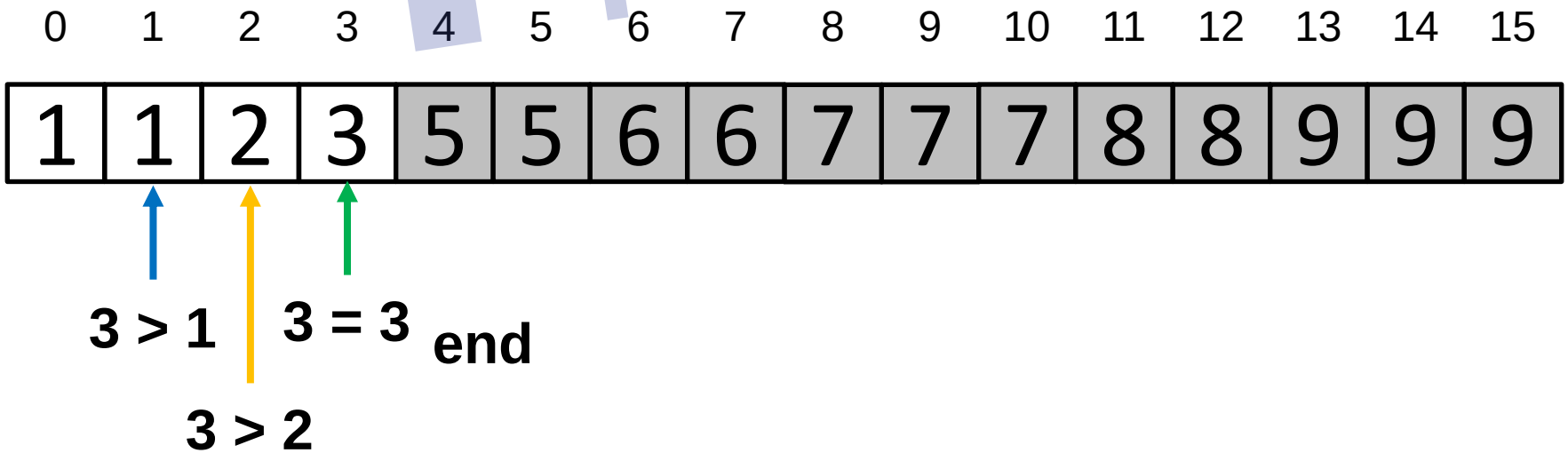
pdfelement

Remove Watermark Now

Parallel Search



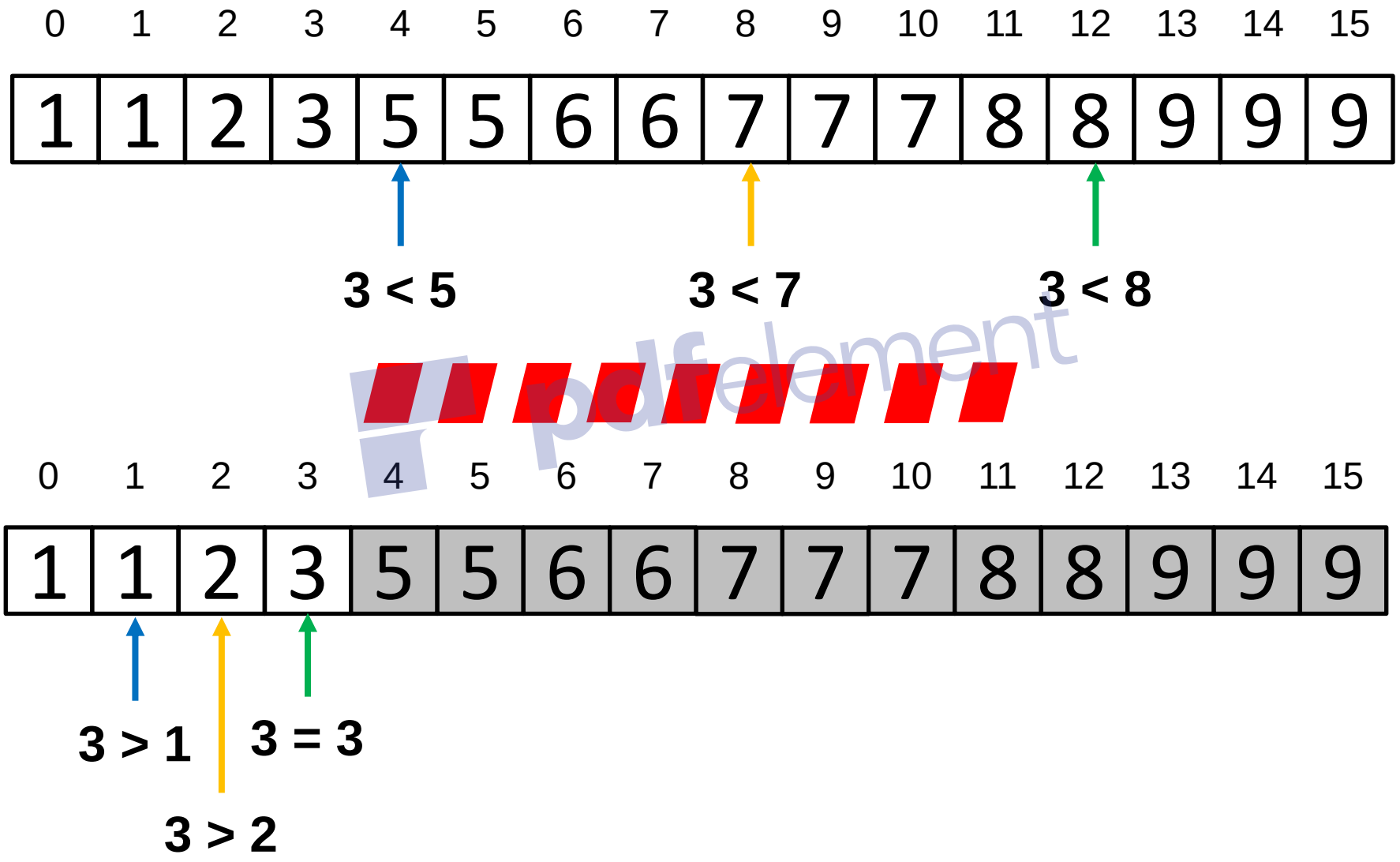
Operations can**NOT** be executed in parallel



pdfelement

Remove Watermark Now

Parallel Search



pdfelement

Remove Watermark Now

Arhitecturi Paralele
Abordări probleme paralele log(N)

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Sume prefix

pdfelement

Remove Watermark Now

Aplicații folosind paralelismul de date
Calculul sumelor prefix

Problemă:

Se dă tabloul **a[1:n]**, se cere **s[1:n]**, unde:

$$s[i] = \sum_{k=1}^{i} a[k]$$

- Algoritm secvențial:

```
s[1] = a[1];
for [i = 2 to n]
    s[i] = a[i] + s[i-1];
```

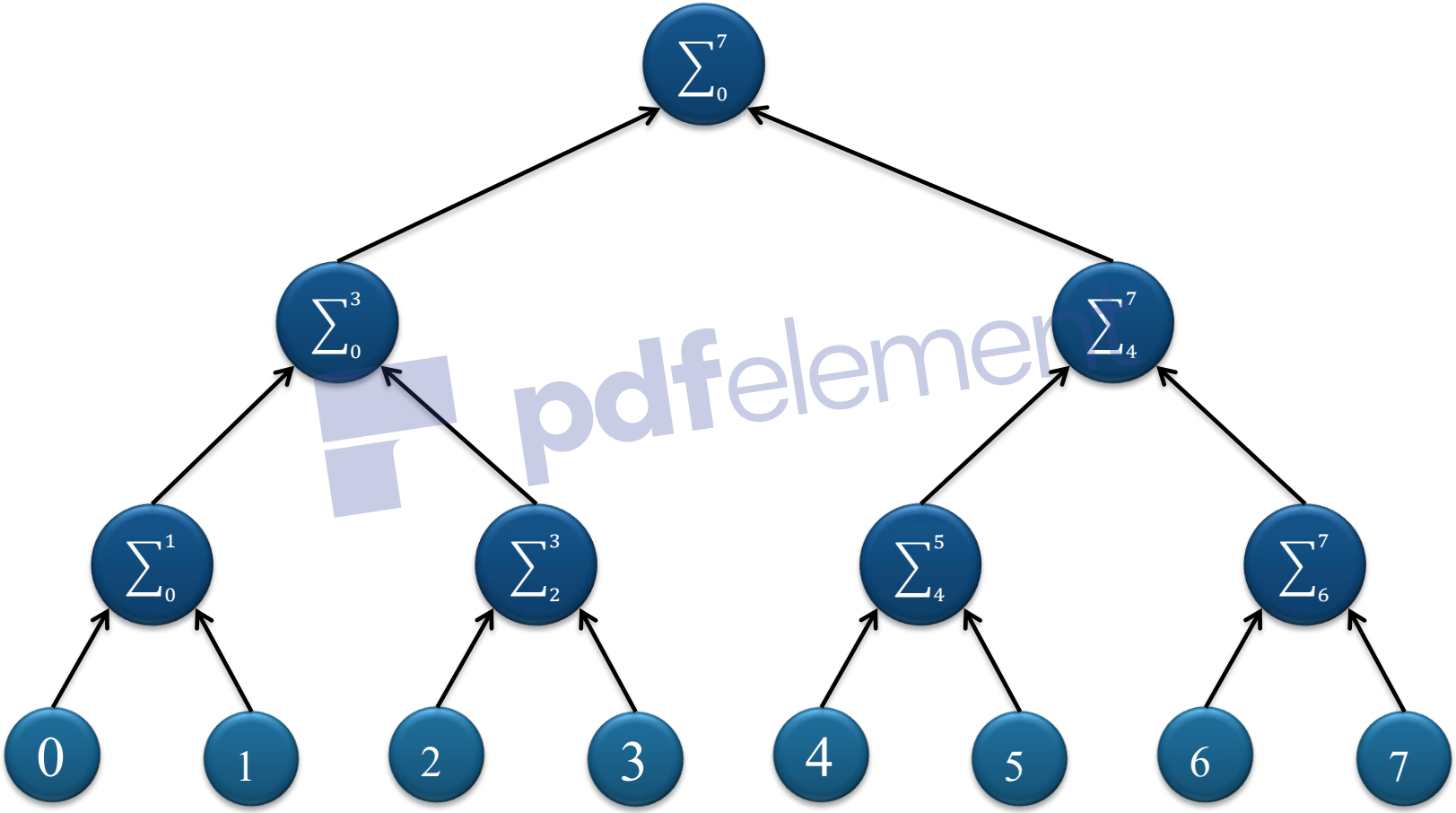
- Algoritm paralel:

  - derivat din algoritmul ***sumei elementelor unui vector***

pdfelement

Remove Watermark Now

Suma elementelor unui vector

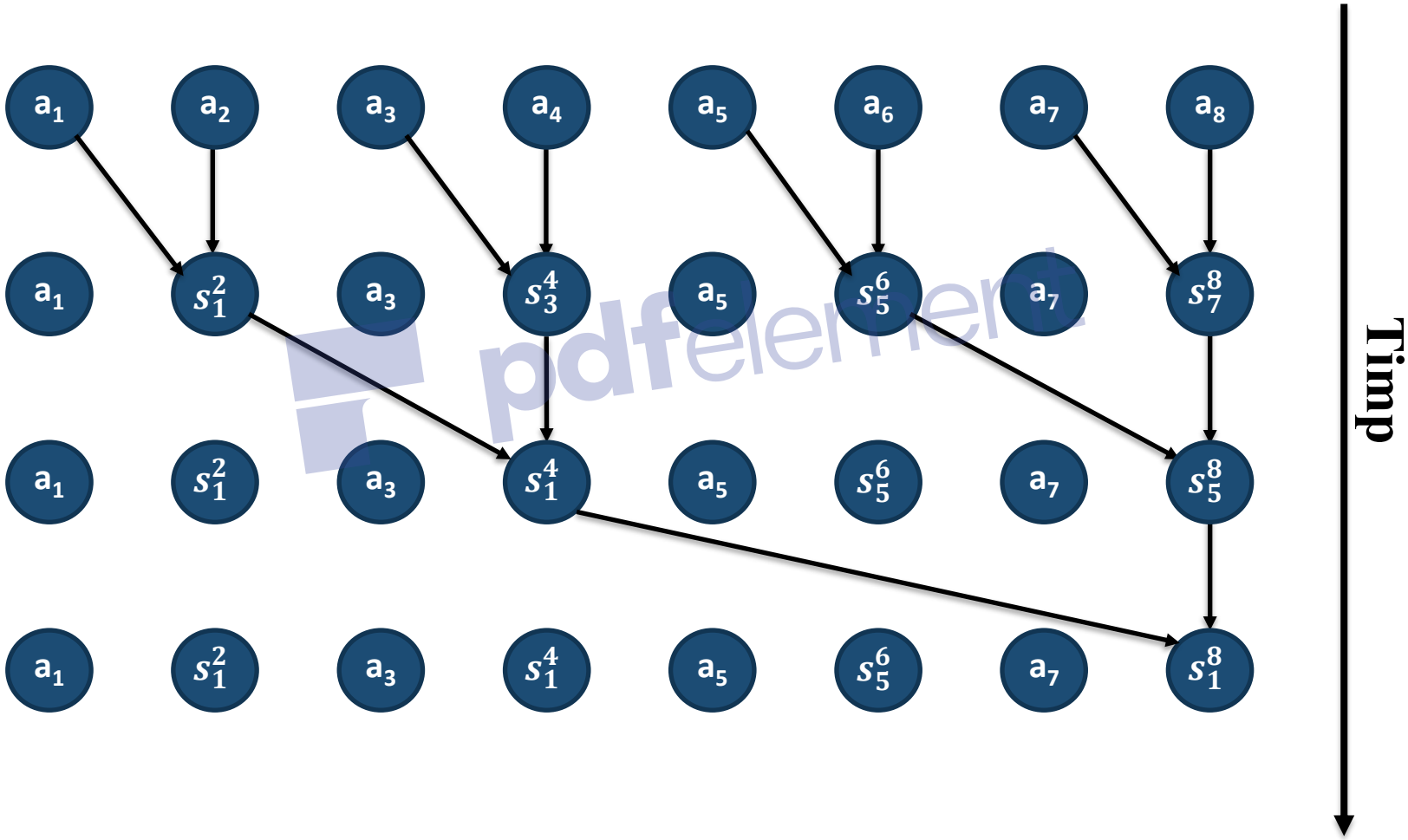


pdfelement

$\sup\left(\frac{n}{2}\right)$ procesoare, $\sup(\log_2 n)$ pași

Remove Watermark Now

Suma elementelor unui vector

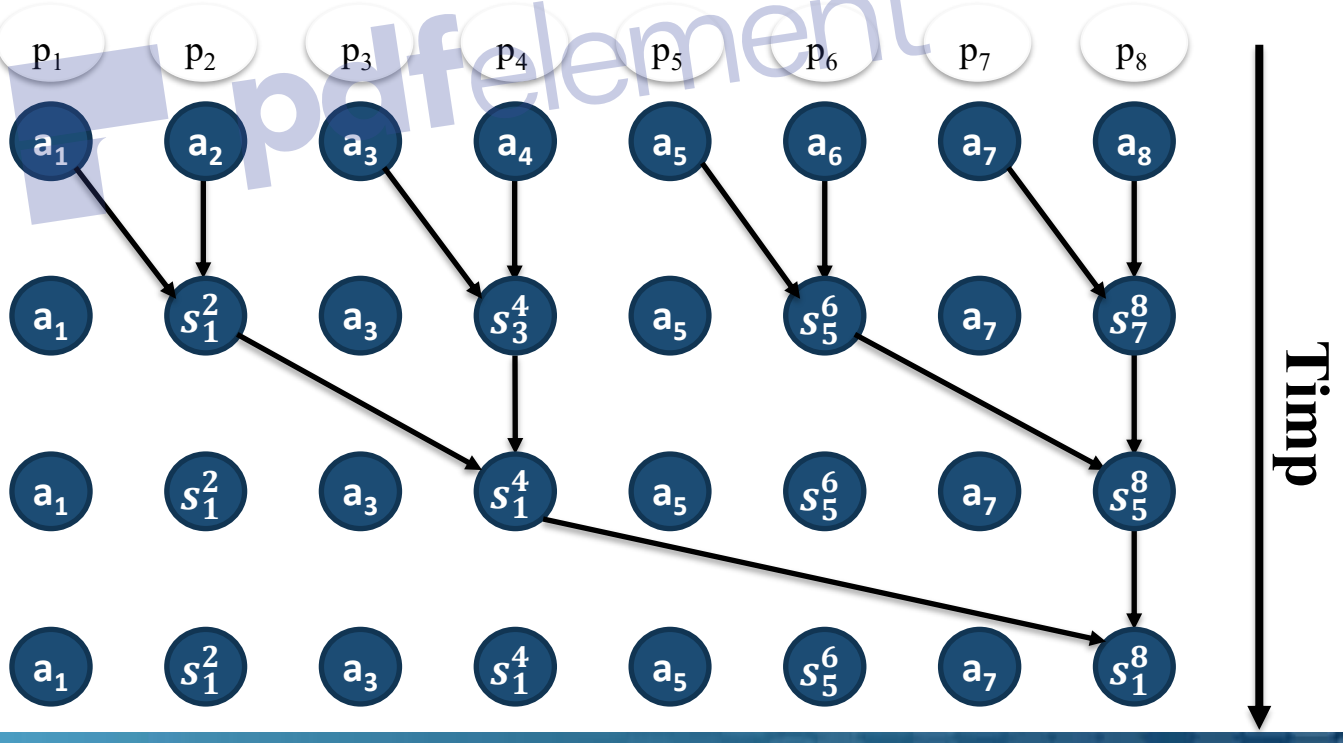


pdfelement

Suma elementelor unui vector

Remove Watermark Now

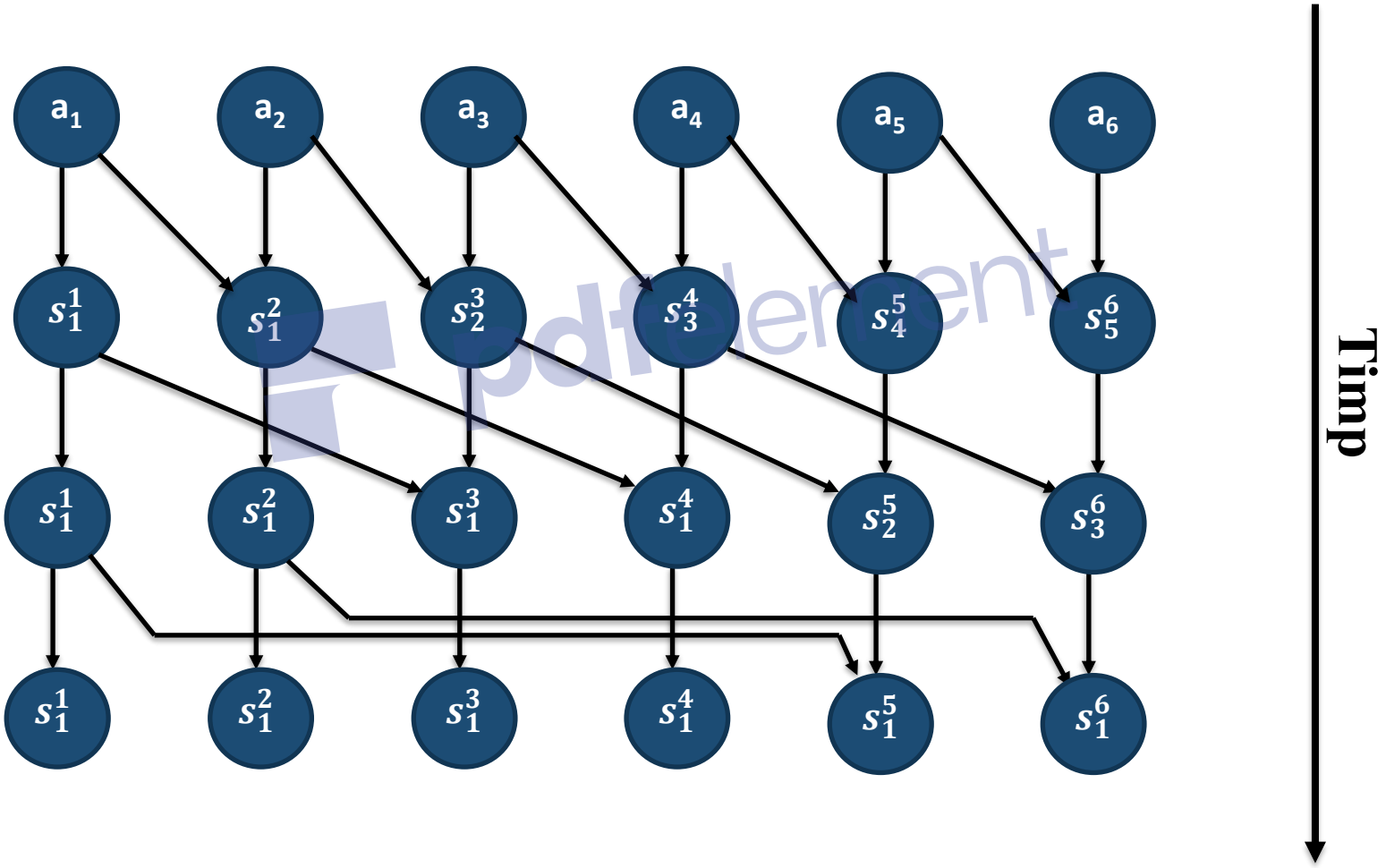
```
int a[1:n];
process suma[k=1 to n] { // suma proceselor p din figur
      for [j = 1 to sup(log2 n)] {
            if (k mod 2^j == 0)
                  a[k] = a[k-2^(j-1)] + a[k];
            barrier;
      }
}
```



pdfelement

Remove Watermark Now

Sume prefix (varianta 1)

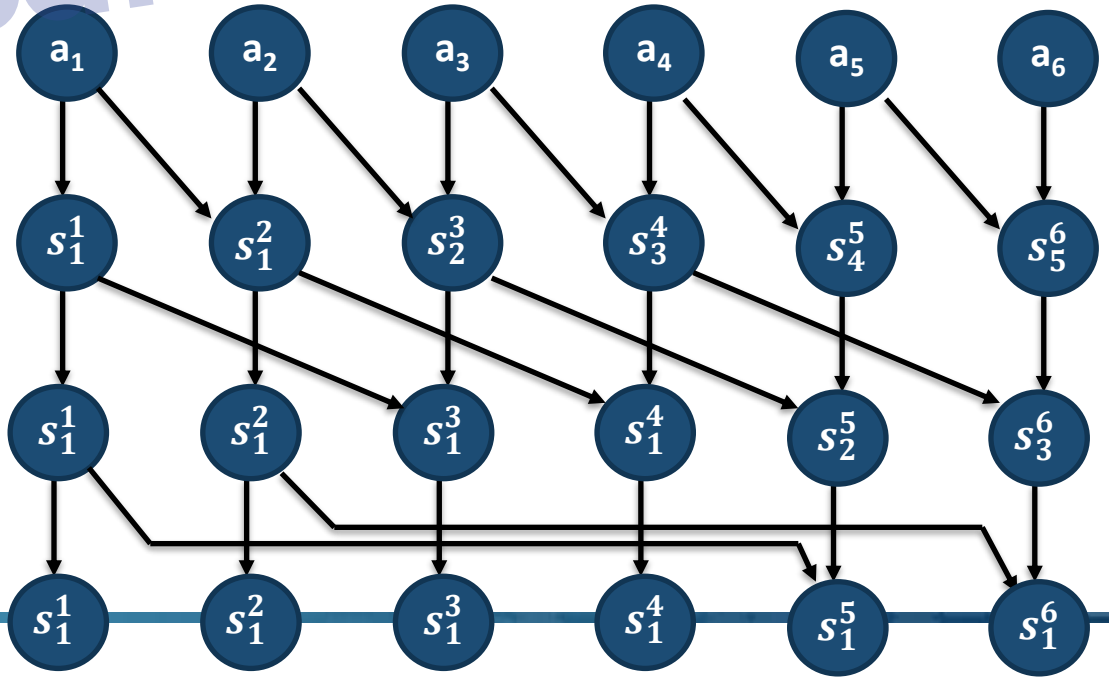


Remove Watermark Now

Sume prefix – varianta 1

```
int a[1:n];
process suma[k=1 to n] {
      for [j = 1 to sup(log2 n)] {
            if (k - 2^(j-1) >= 1)
                  a[k] = a[k-2^(j-1)] + a[k];
            barrier;
      }
}
```

pdfelement

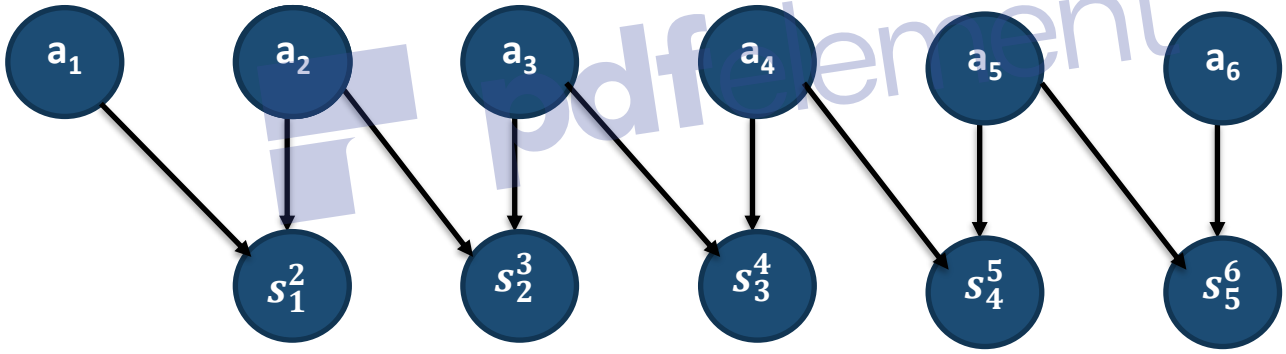


Eroare de sincronizare...

Remove Watermark Now

Sume prefix – varianta 1 – probleme

- Presupunem că procesorul numărul 3 este mai lent decât restul.
  - Suprascriere a locației de memorie



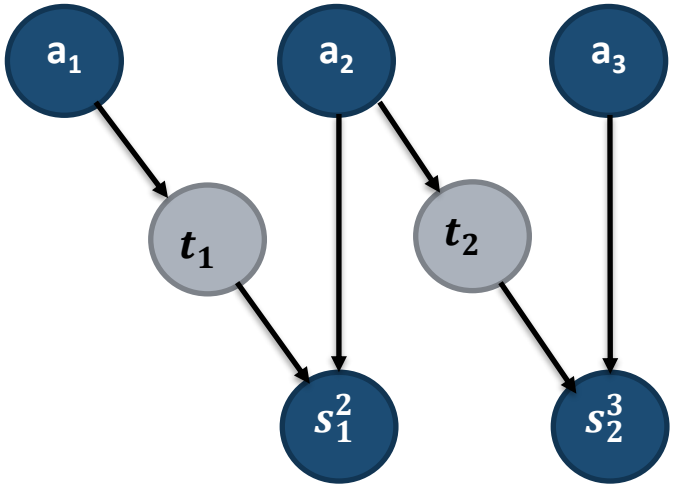
$$s_2^3 = a_3 + s_1^2$$

Sume prefix – varianta 2

Remove Watermark Now

```
int a[1:n], temp[1:n];
process suma[k=1 to n] {
      for [j = 1 to sup(log2 n)] {
            temp[k] = a[k];
            barrier;
            if (k - 2^(j-1) >= 1)
                  a[k] = temp[k-2^(j-1)] + a[k];
            barrier;
      }
}
```

pdfelement



Remove Watermark Now

Sume prefix – varianta 3

```
int a[1:n], temp[1:n];
process suma[k:1 to n] {
    int d = 1;
    while (d < n) {
        temp[k] = a[k];
        barrier;
        if (k - d >= 1) a[k] = temp[k-d] + a[k];
        barrier;
        d = 2 * d;
    }
}
```

pdfelement

Remove Watermark Now

Parallel Scan

pdfelement

Remove Watermark Now

Parallel Scan

scan (o) < x1, ..., xn >

==

< x1, x1 o x2, ..., x1 o ... xn >

Exemplu: Filtru

Considerați la **intrare** un vector, trebuie găsit la ieșire un alt **vector** care să conțină doar elementele pentru care **(f elt)** este **true**

Fie f x = x > 10

filter f < 17, 4, 6, 8, 11, 5, 13, 19, 0, 24 >
= < 17, 11, 13, 19, 24 >

Paralelizabil?

- Găsirea elementelor este o operație ușoară
- Dar punerea lor în secvența corectă pare mai dificilă

pdfelement

Remove Watermark Now

Prefixe paralele - to the rescue

1. Folosim mapare paralelă pentru a calcula un bit-vector pentru elementele ce se potrivesc operației

   input: $< 17, 4, 6, 8, 11, 5, 13, 19, 0, 24 >$
   bits: $< 1, 0, 0, 0, 1, 0, 1, 1, 0, 1 >$

2. Aplicăm sume prefix pe elementele bit-vectorului

   bitsum $< 1, 1, 1, 1, 2, 2, 3, 4, 4, 5 >$

3. Aplicăm parallel map pentru a produce ieșirea

   output $< 17, 11, 13, 19, 24 >$

pdfelement

Remove Watermark Now

Generic: Scan

Se aplică aceeași operație între elementele unui vector.
Se colectează toate rezultatele parțiale

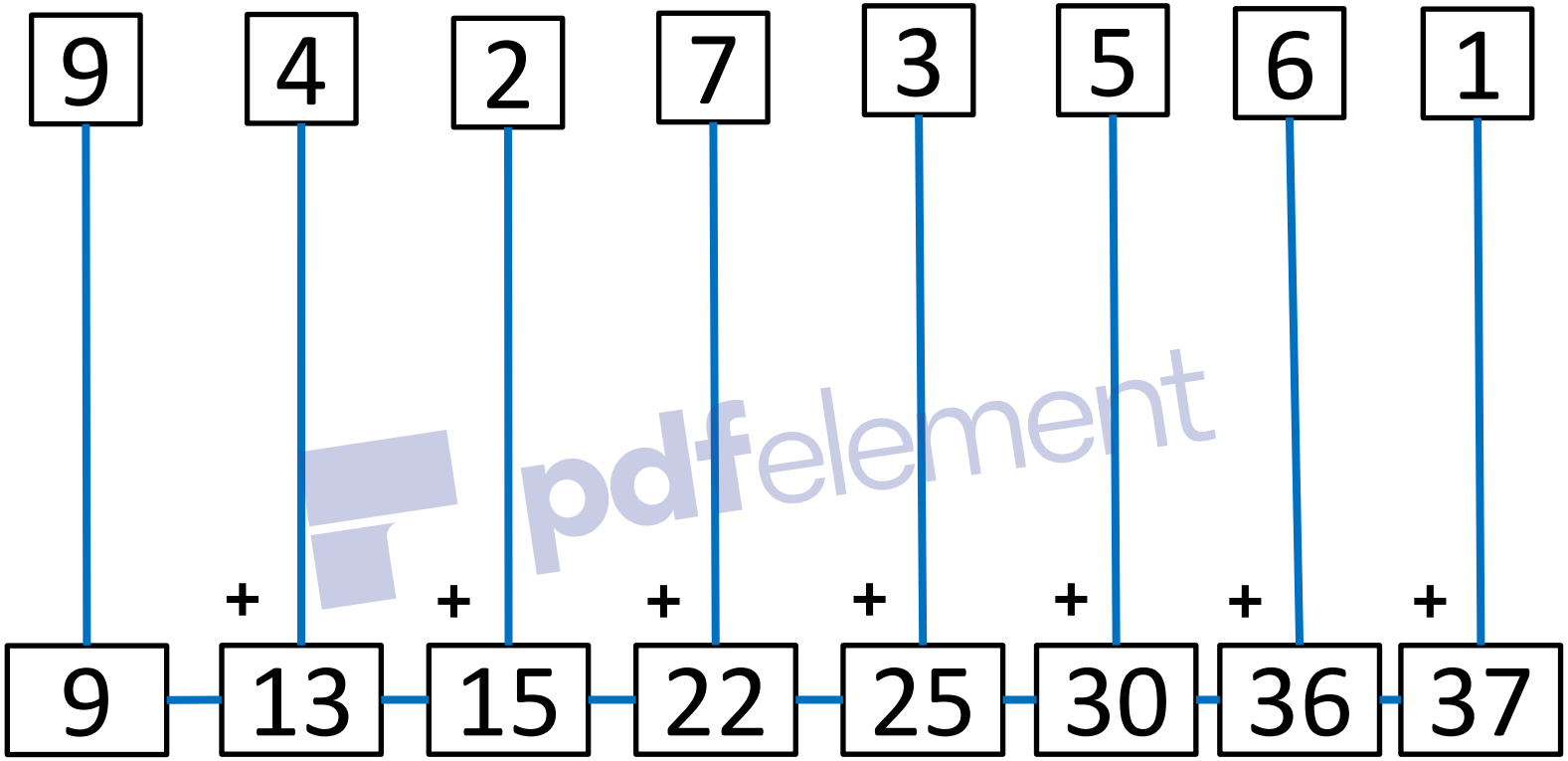
Poate fi executat în log(n) pași folosind un arbore

Operația poate lua orice formă (+, *, min, max, and, etc.)

pdfelement

Remove Watermark Now

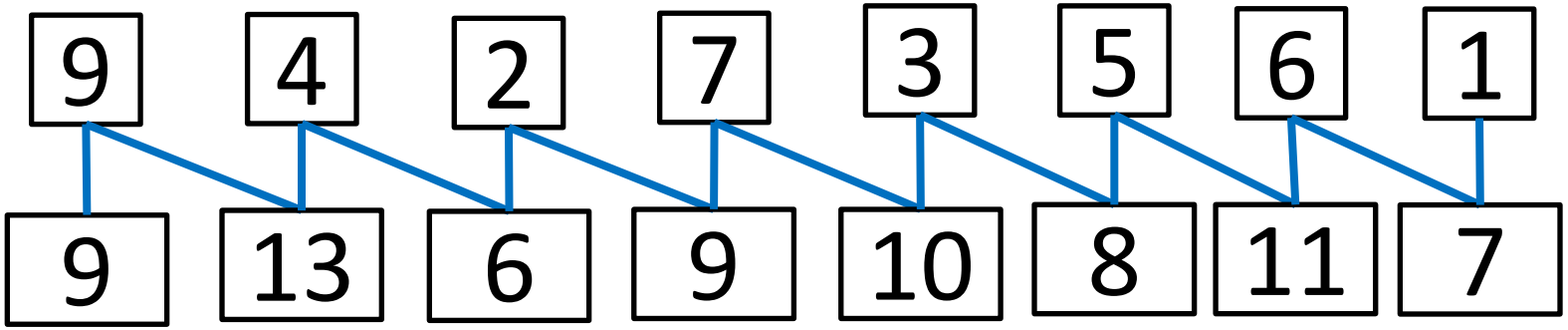
Scan - sum



pdfelement

Remove Watermark Now

Scan - sum

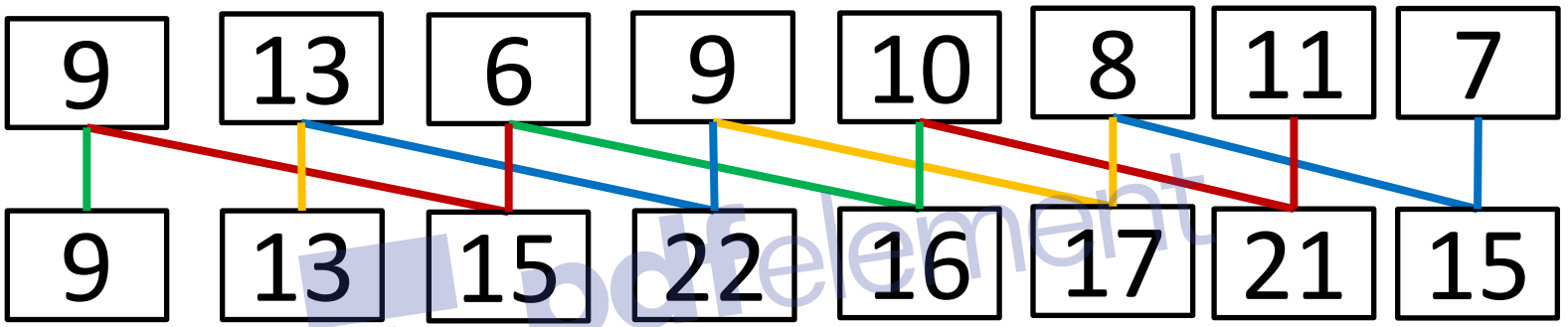


pdfelement

Pot fi executate în paralel

Remove Watermark Now

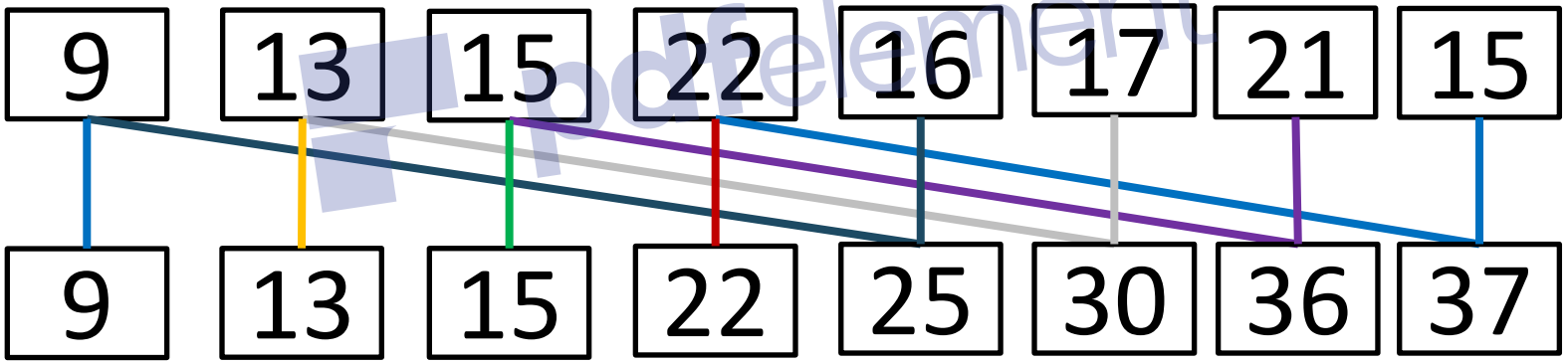
Scan - sum



Pot fi toate executate în paralel

Remove Watermark Now

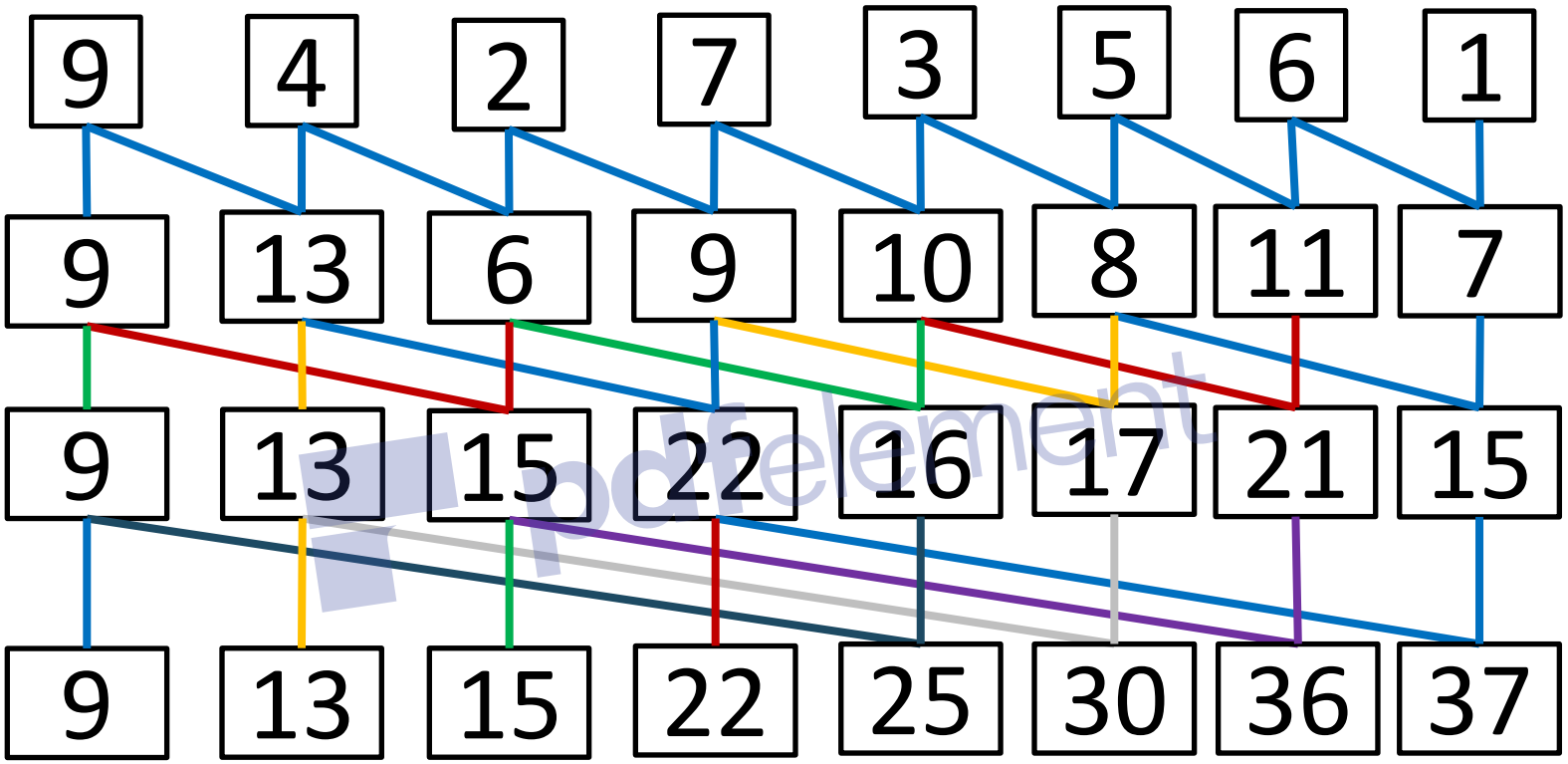
Scan - sum



Pot fi toate executate în paralel

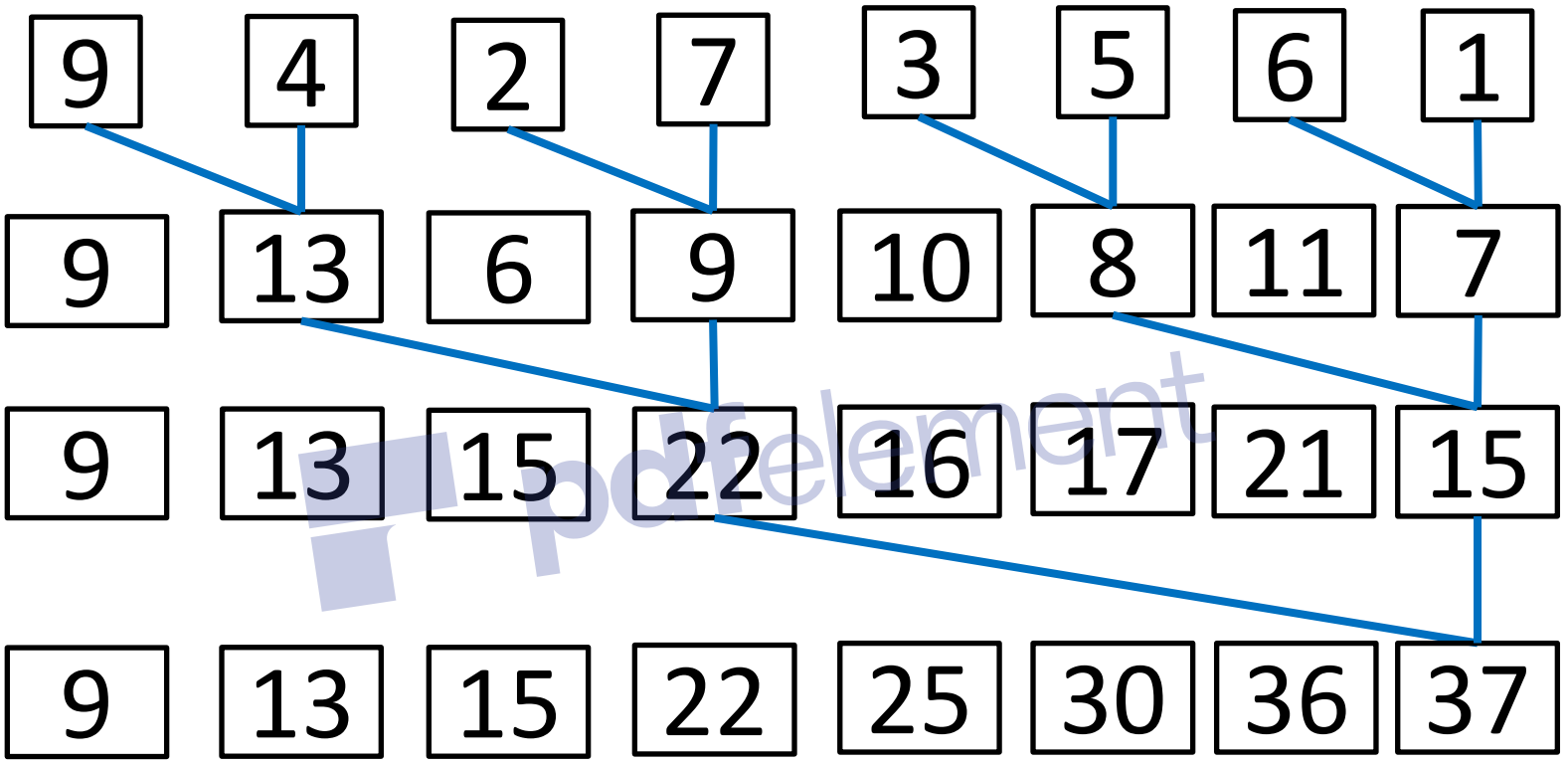
Remove Watermark Now

Scan – Cum funcționează?



Remove Watermark Now

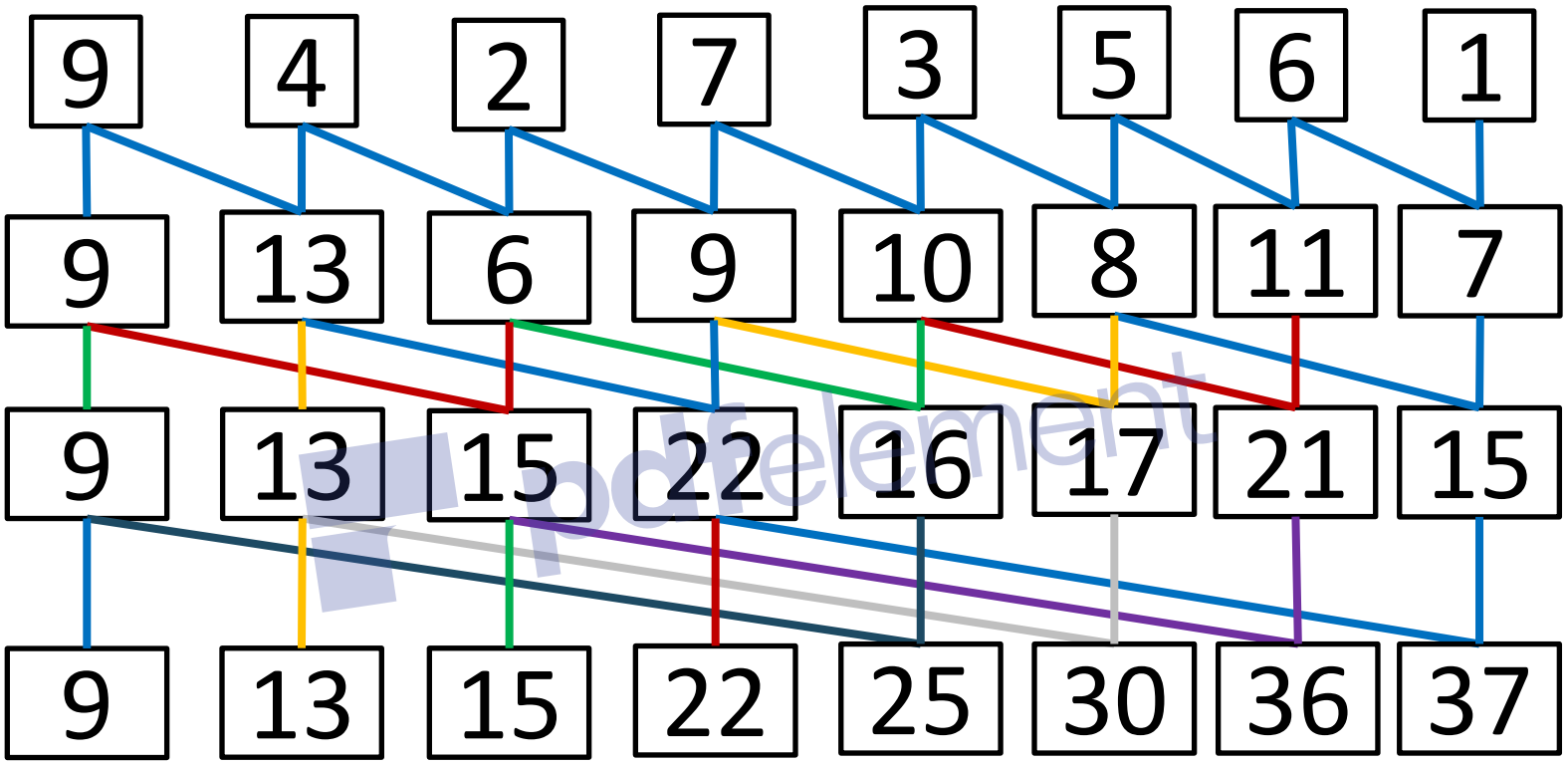
Scan – Cum funcționează



pdfelement

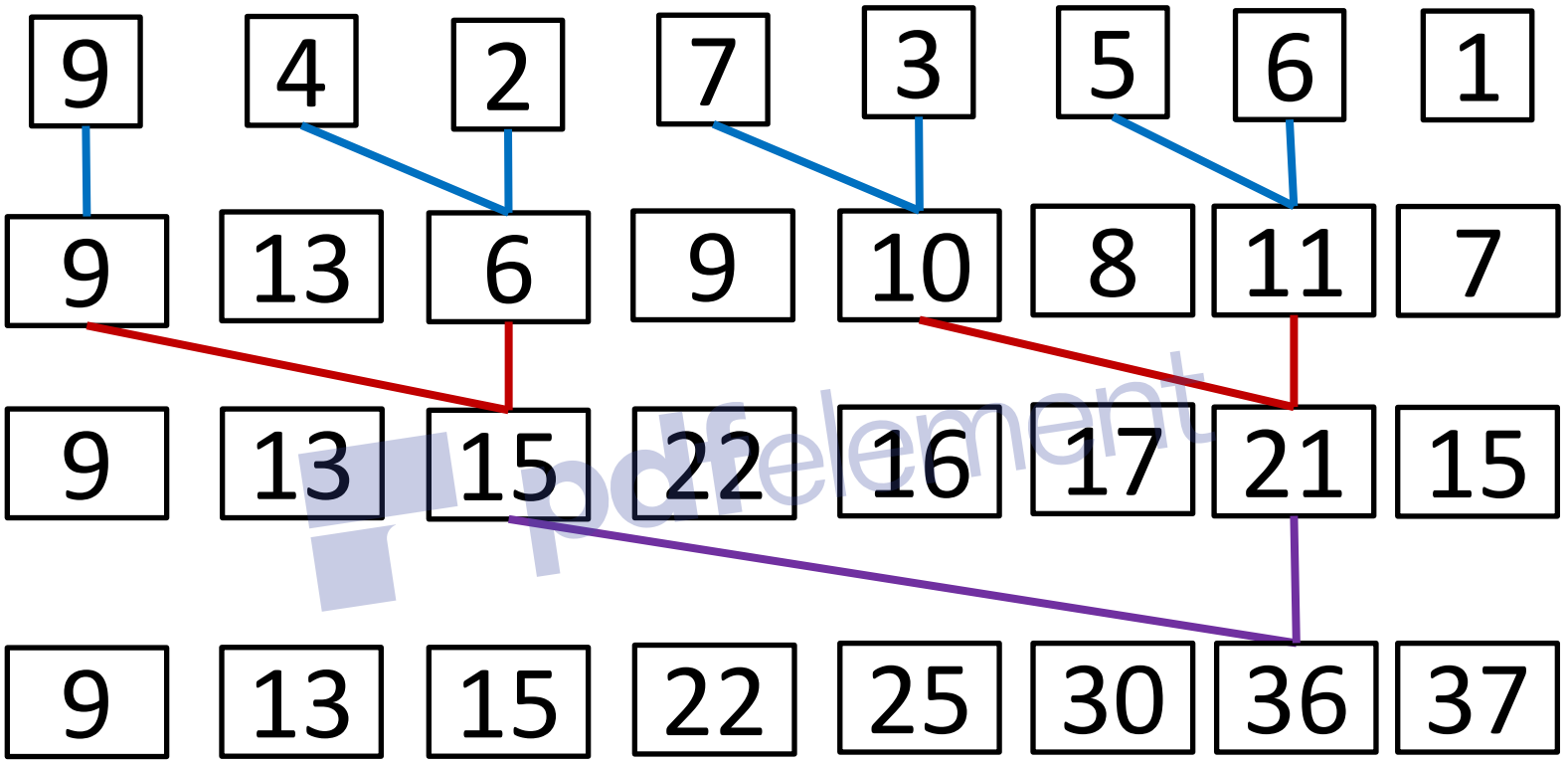
Remove Watermark Now

Scan – Cum funcționează?



Remove Watermark Now

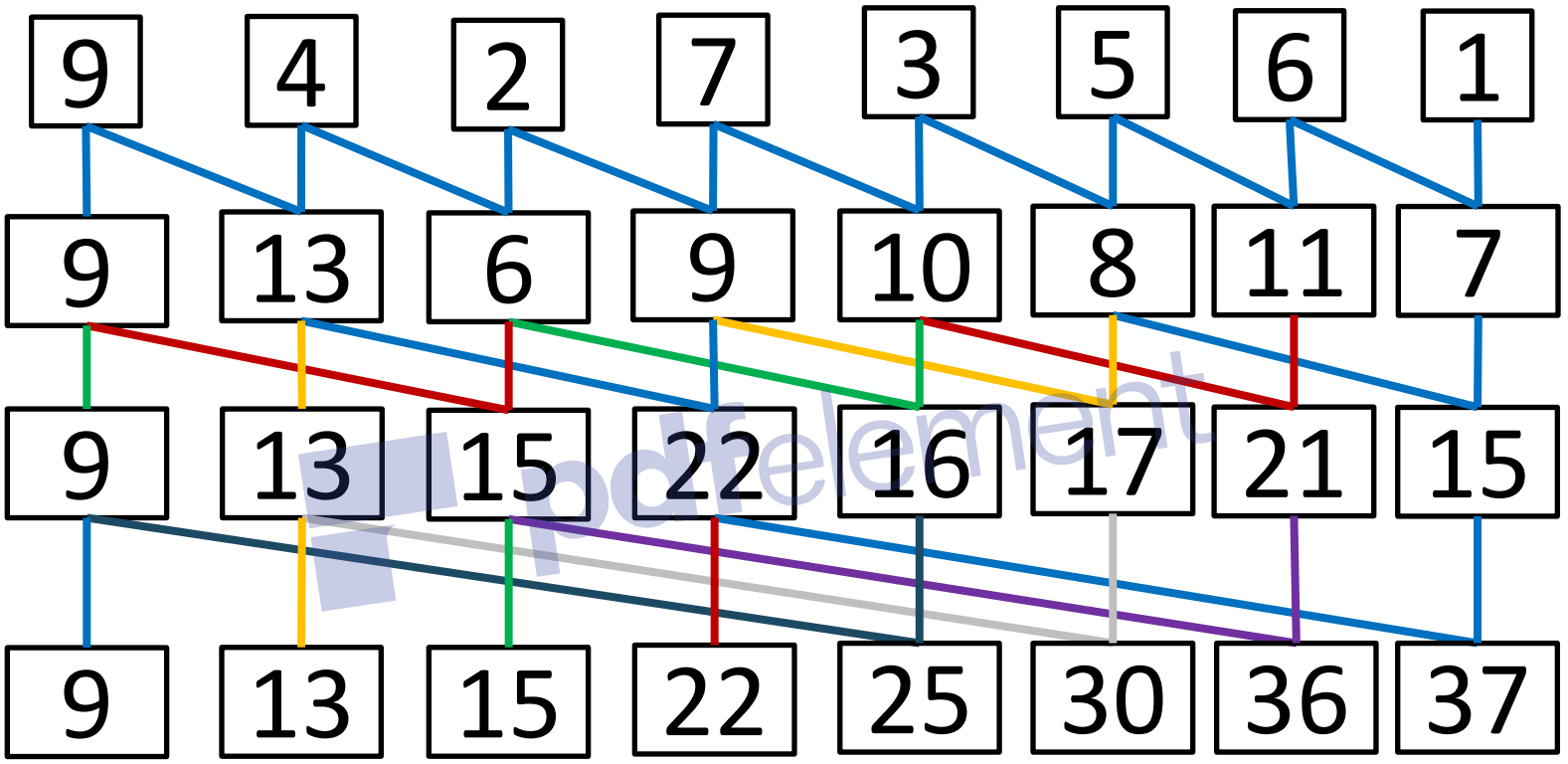
Scan – Cum funcționează?



pdfelement

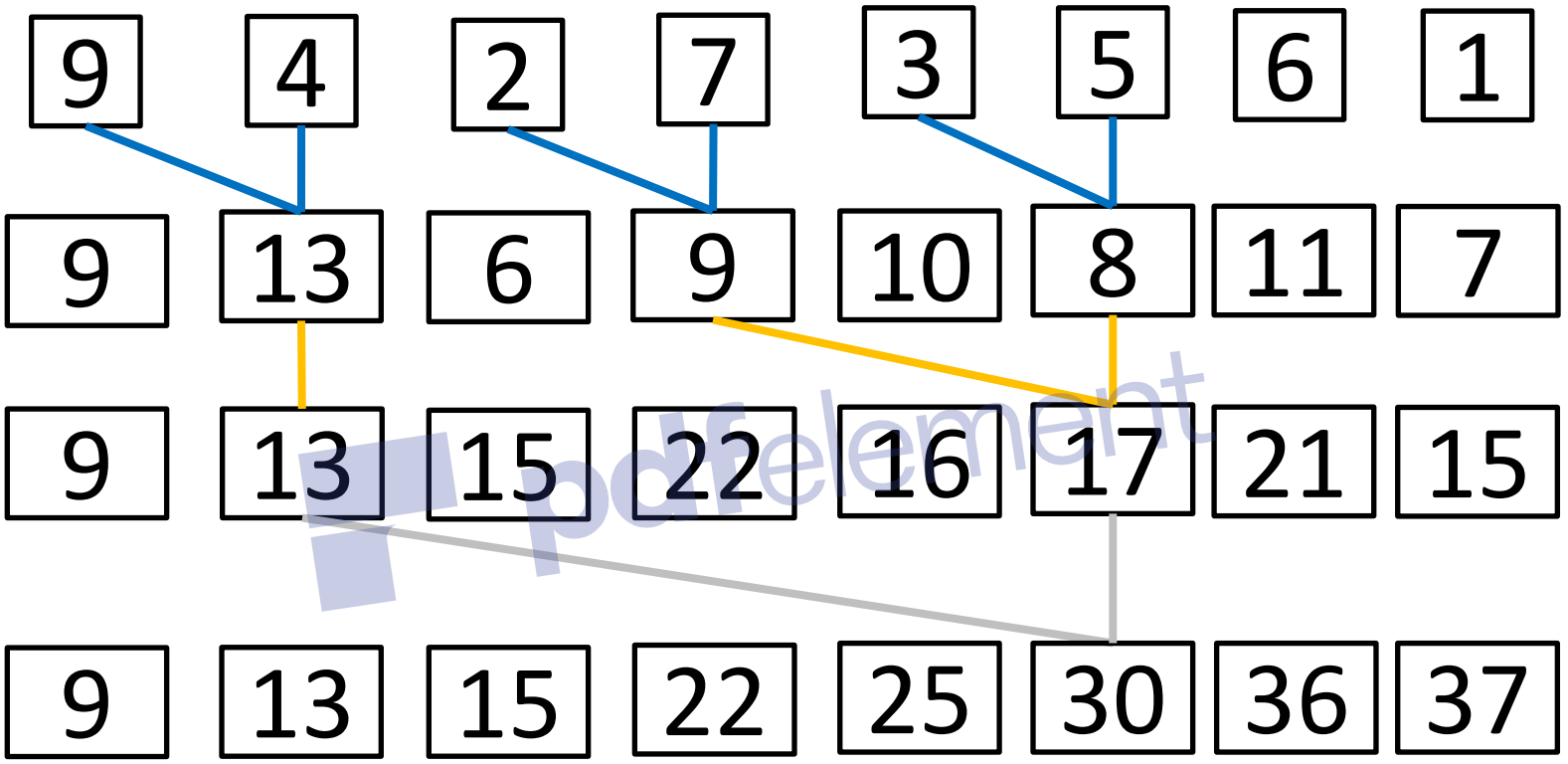
Remove Watermark Now

Scan – How it works?



Remove Watermark Now

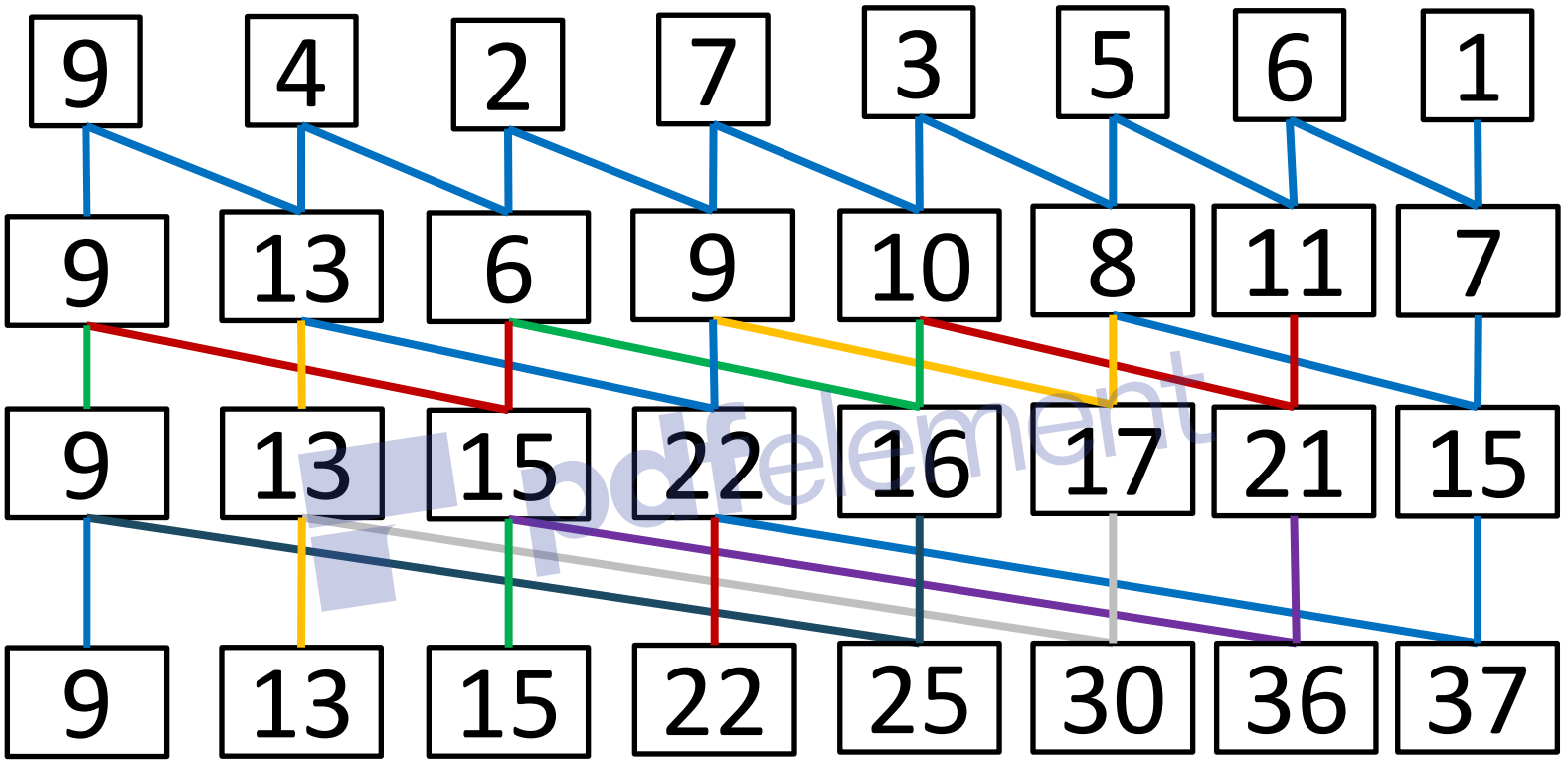
Scan – How it works?



pdfelement

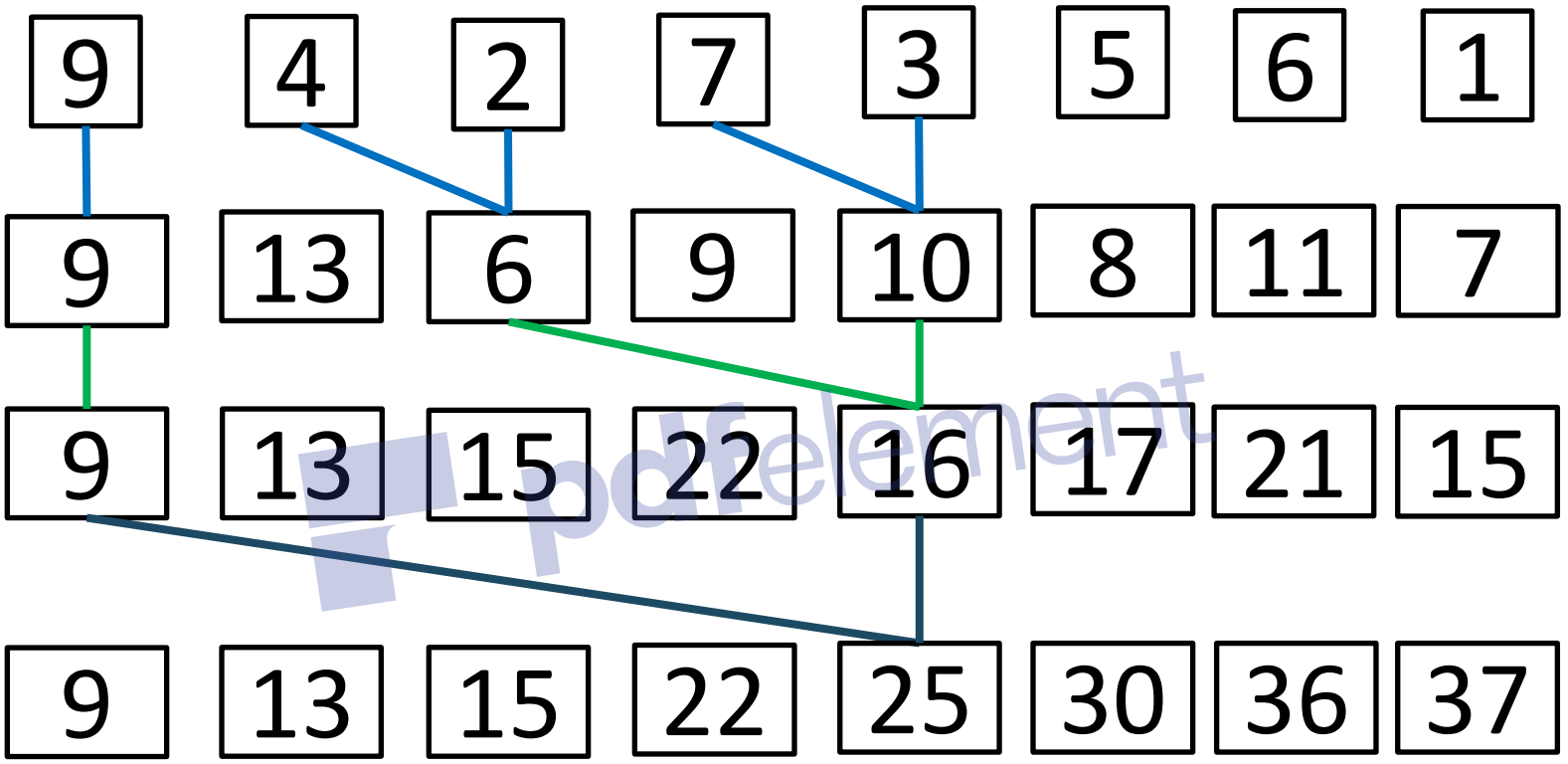
Remove Watermark Now

Scan – How it works?



Remove Watermark Now

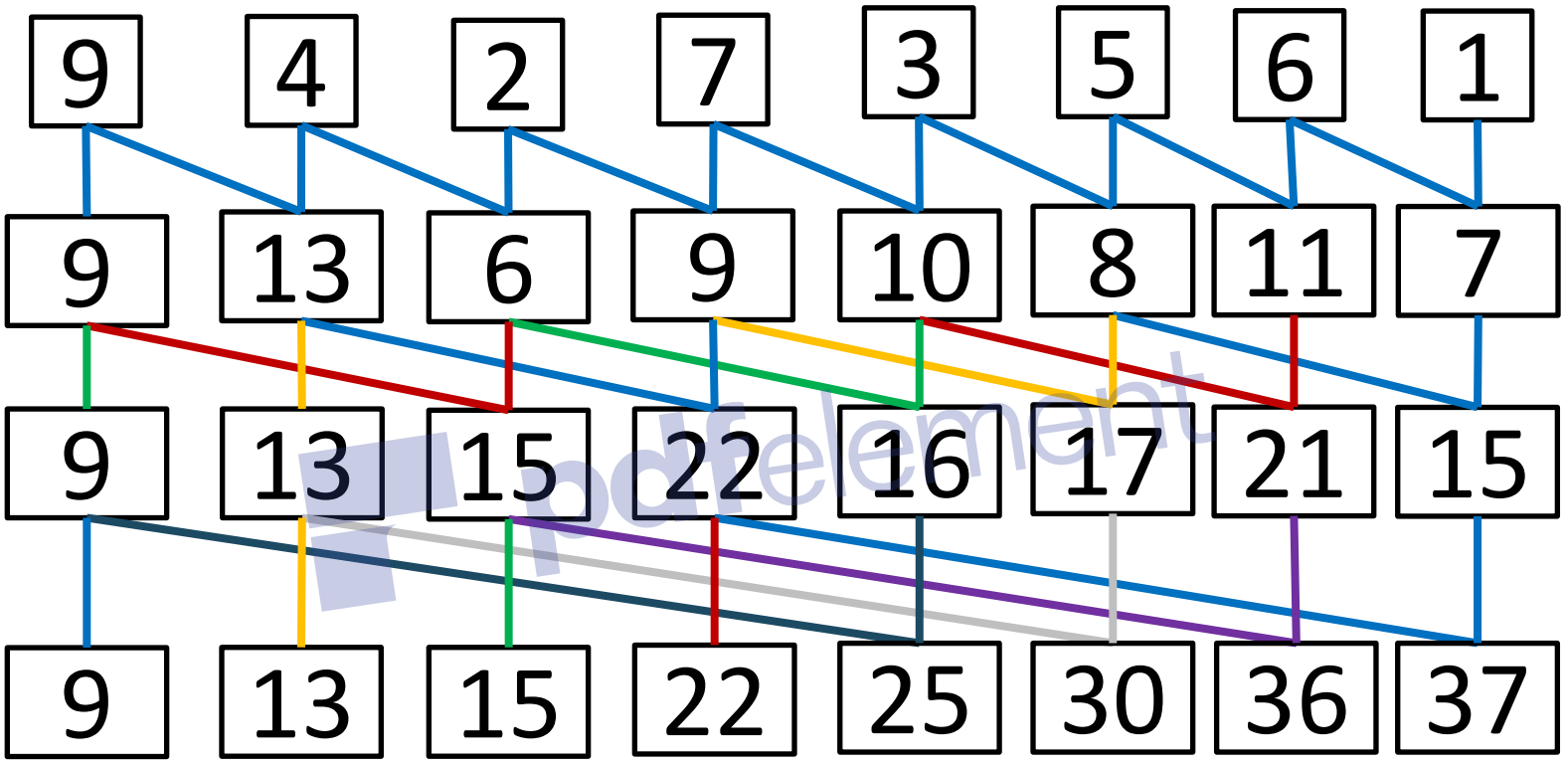
Scan – How it works?



pdfelement

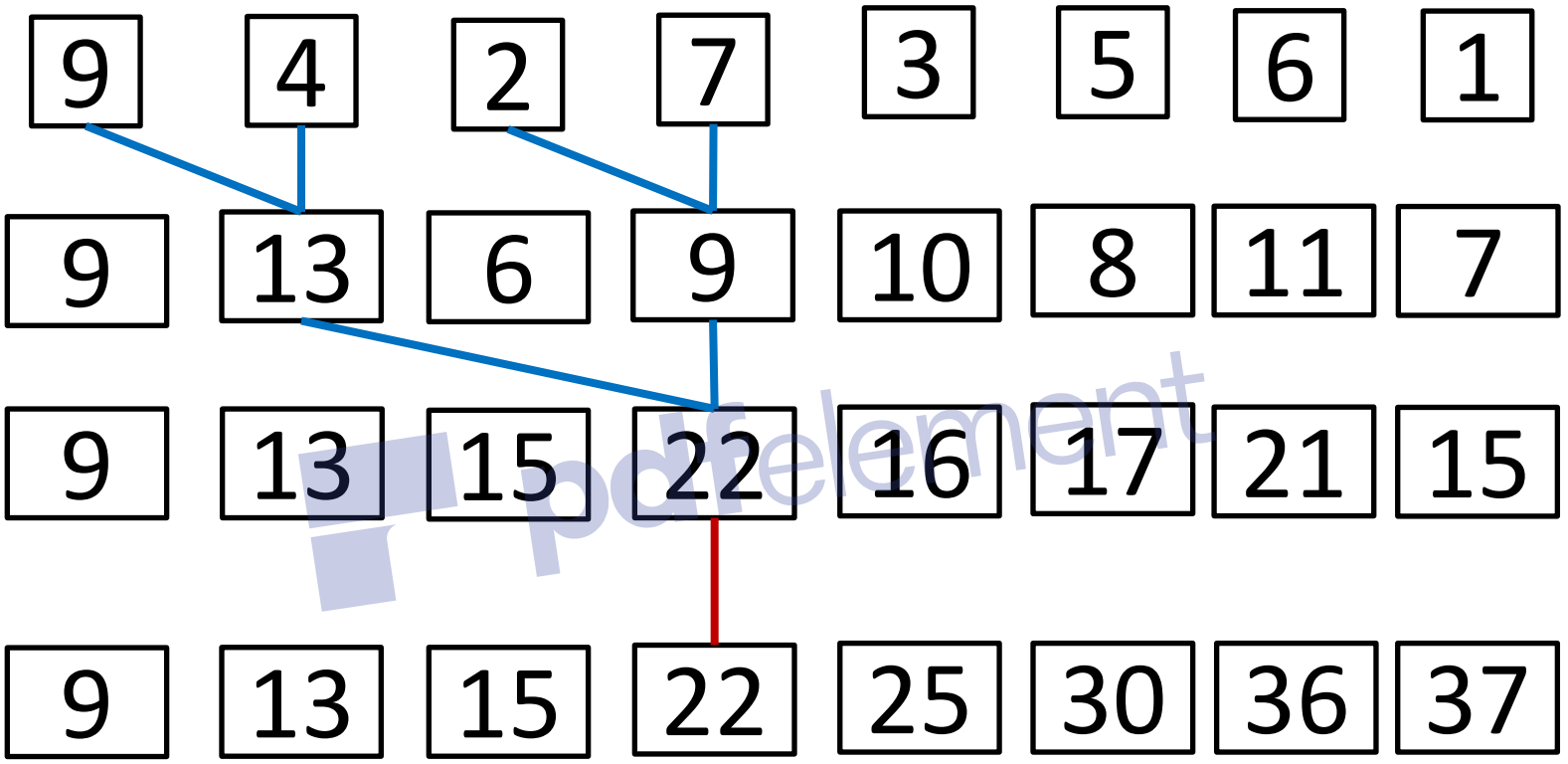
Remove Watermark Now

Scan – How it works?



Remove Watermark Now

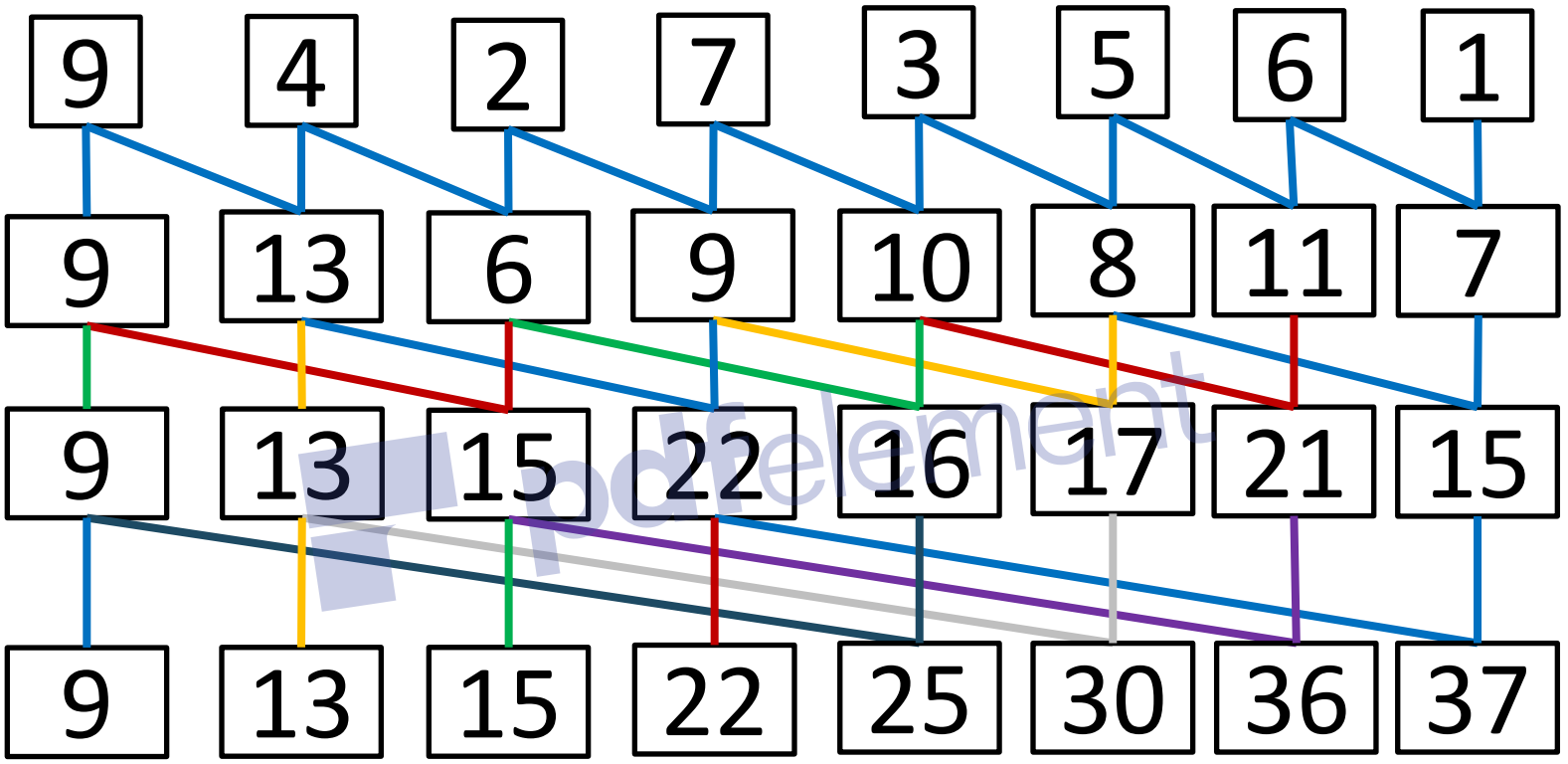
Scan – How it works?



pdfelement

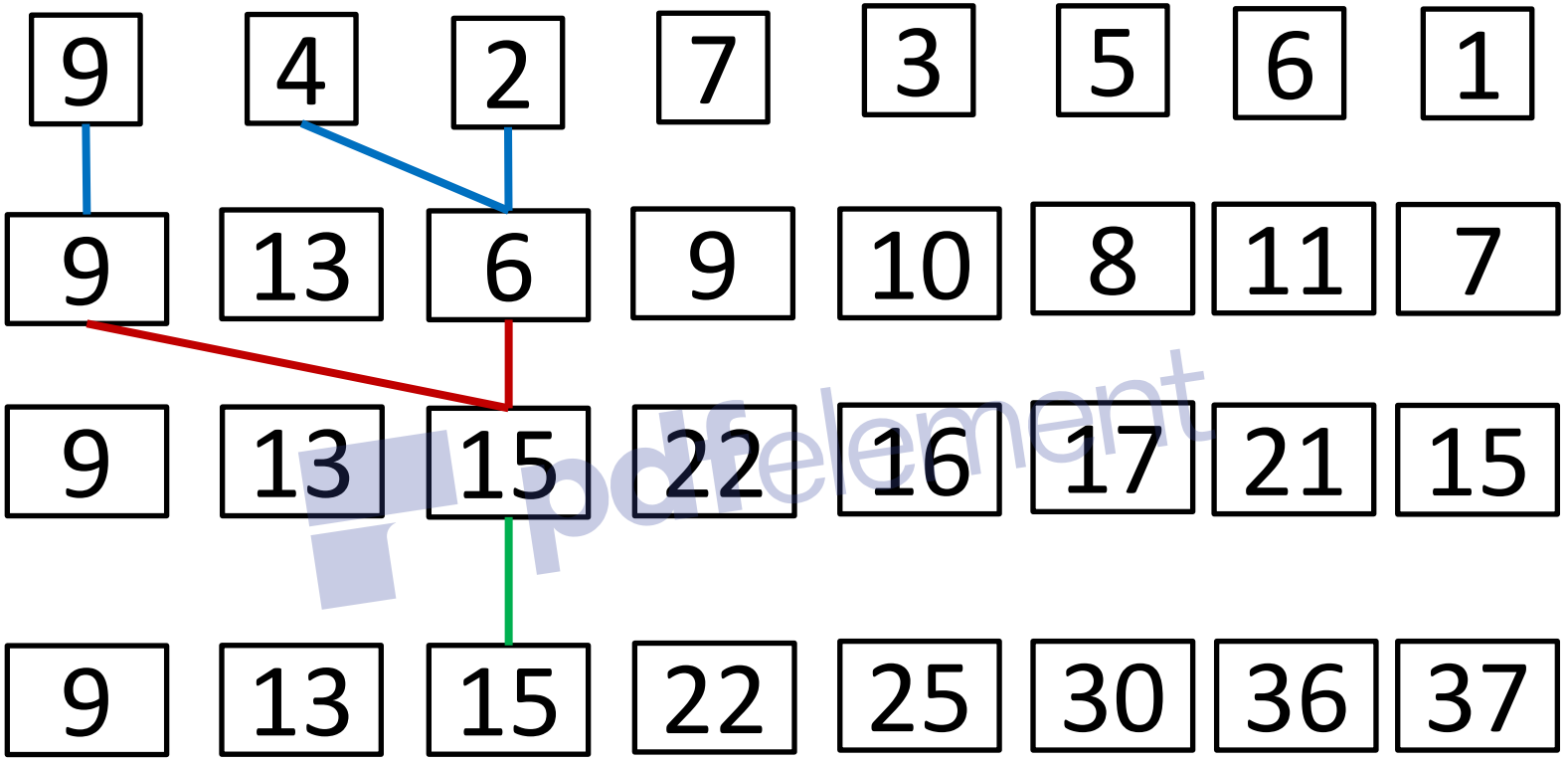
Remove Watermark Now

Scan – How it works?



Remove Watermark Now

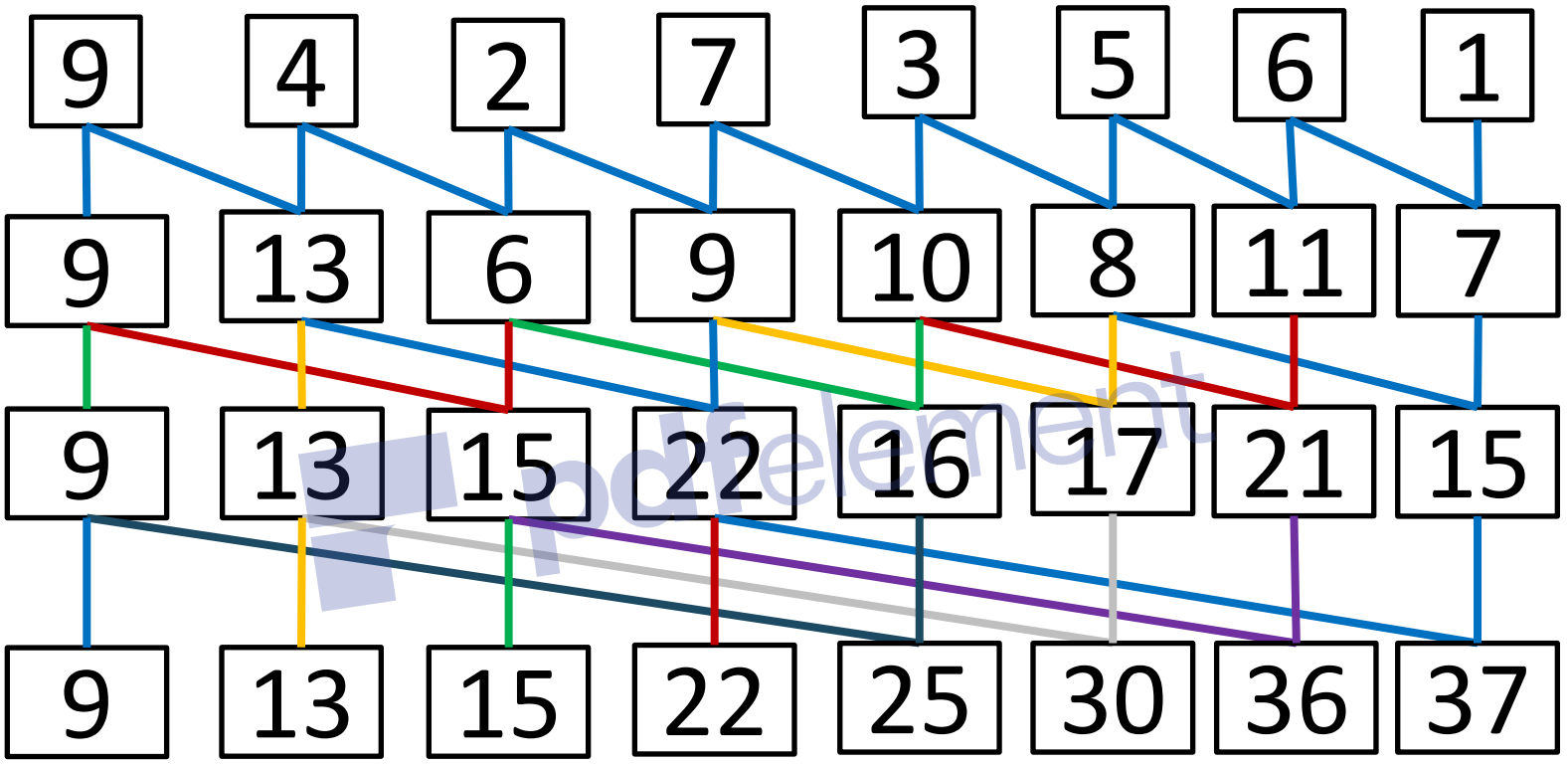
Scan – How it works?



pdfelement

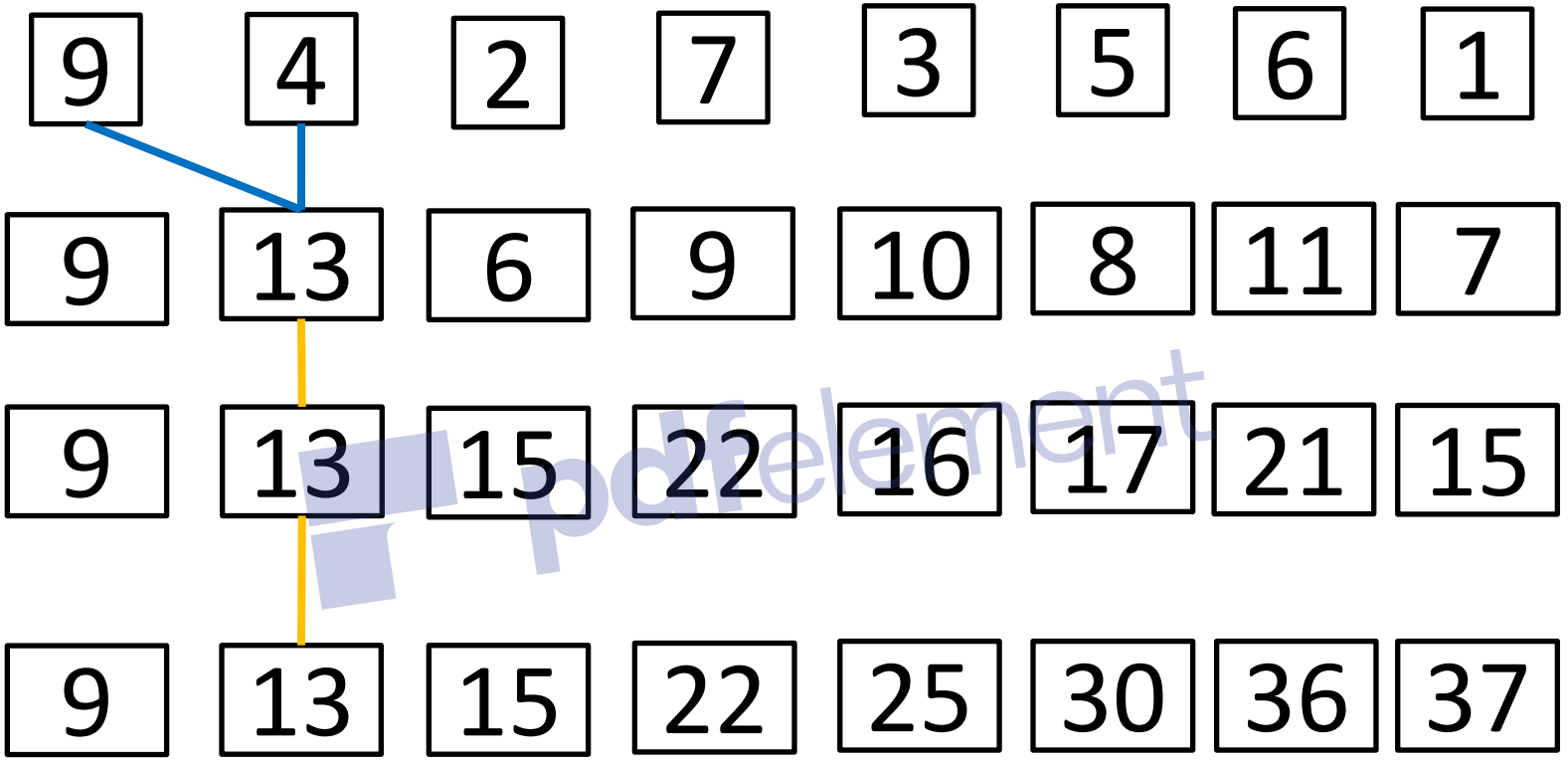
Remove Watermark Now

Scan – How it works?



Remove Watermark Now

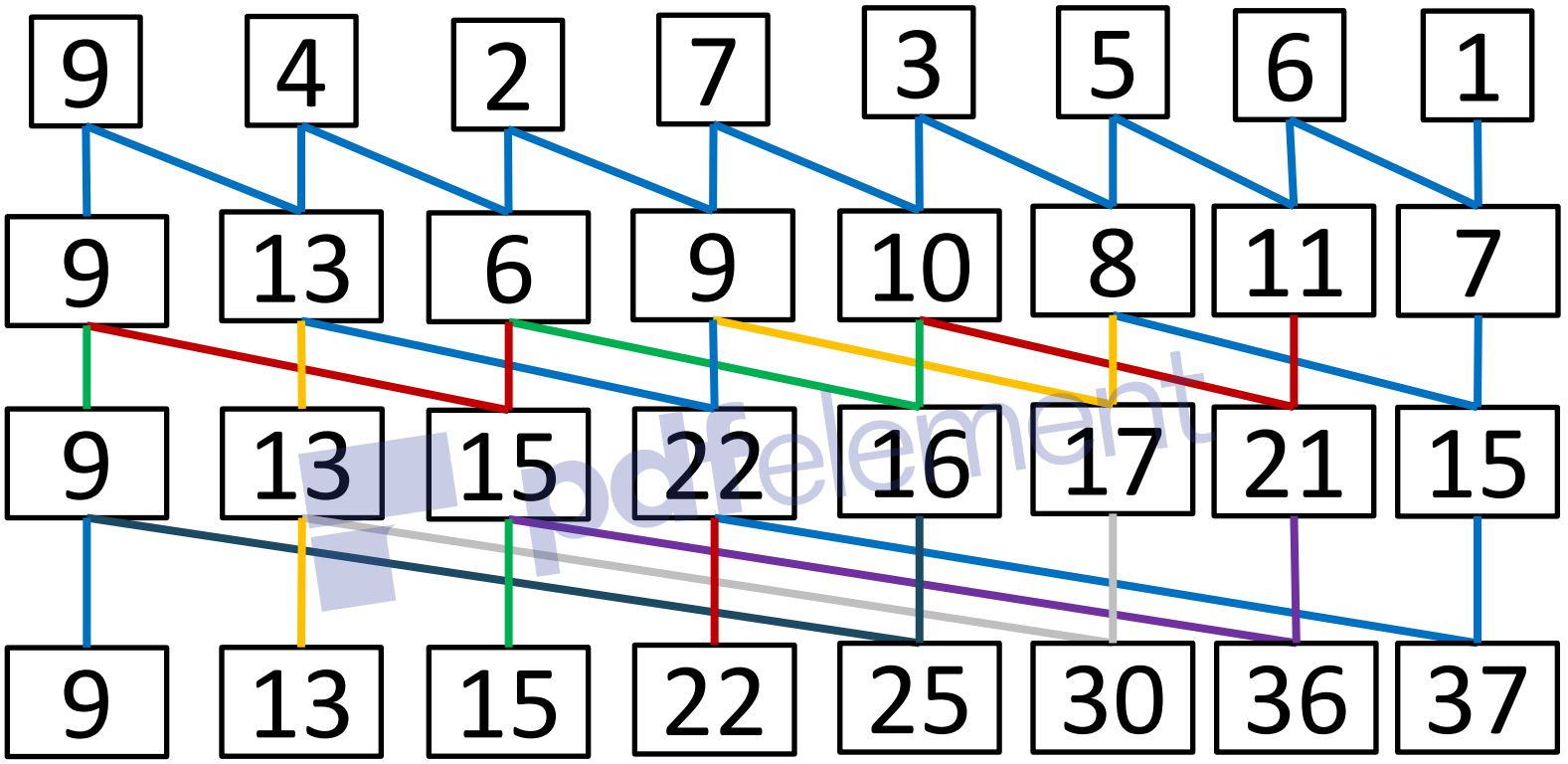
Scan – How it works?



pdfelement

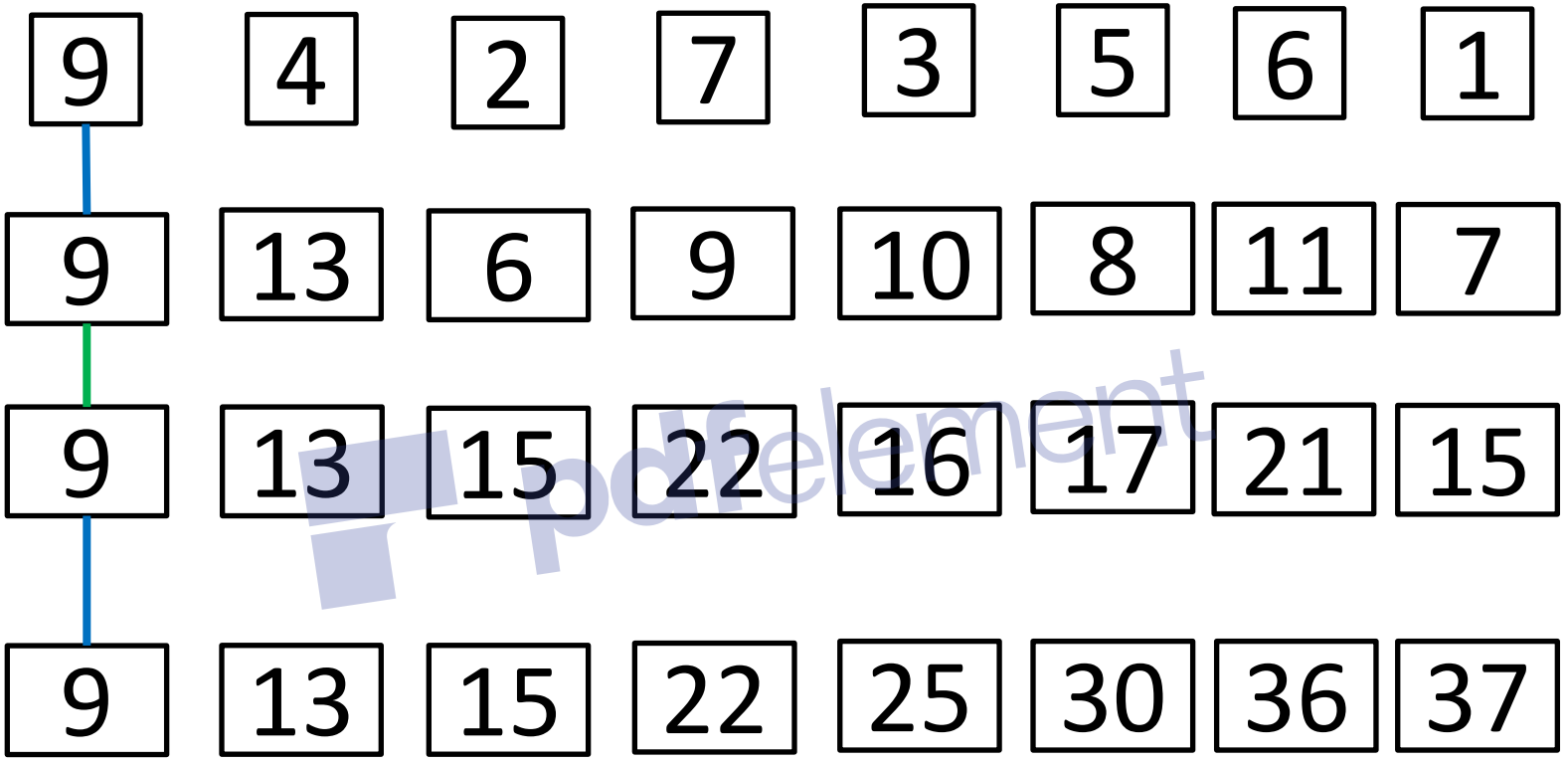
Remove Watermark Now

Scan – How it works?



Remove Watermark Now

Scan – How it works?



pdfelement

Remove Watermark Now

Difuzarea unei valori

pdfelement

Remove Watermark Now

Difuzarea unei valori – justificare O(log P)

- D – celula din memoria comună ce trebuie difuzată
- Folosim un tablou A[1:N]

procedure BROADCAST (D, N, A)

Pas 1: Procesorul P1

1.1. citește valoarea din D

1.2. o memorează în propria memorie

1.3. o scrie în A[1]

Pas 2:

fa i := 0 to (log N -1) ->

fa j := 2^i+1 to $2^{(i+1)}$ do in parallel

Procesor Pj

2.1. citește valoarea din A[j-2^i]

2.2. o memorează în propria memorie

2.3. o scrie în A[j]

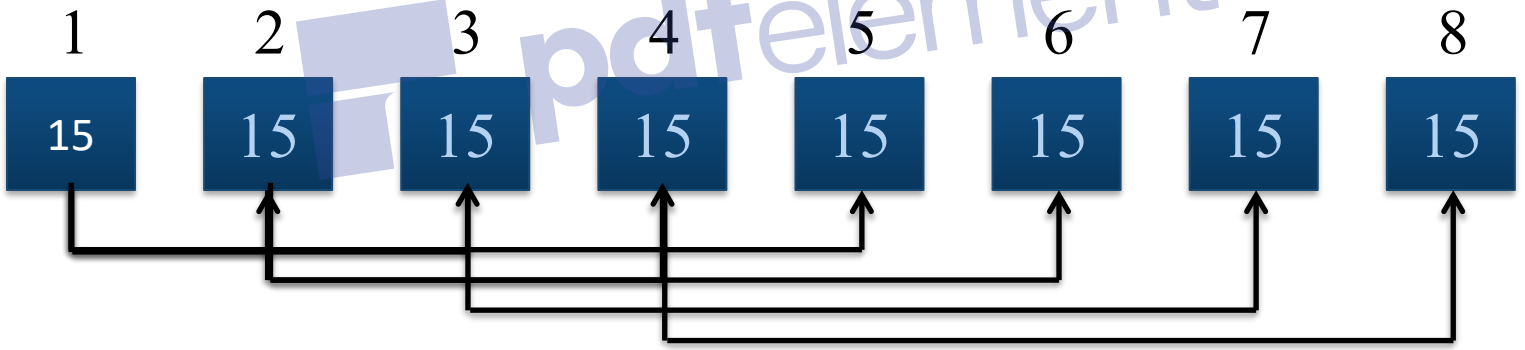
af

af

pdfelement

Remove Watermark Now

Difuzarea unei valori



pdfelement

Remove Watermark Now

Operații cu vectori – broadcast

Pas 1: (Procesorul P_1)

```
int t;/* t locala P1 */
t = D;
A[1] = t;
```

Pas 2:

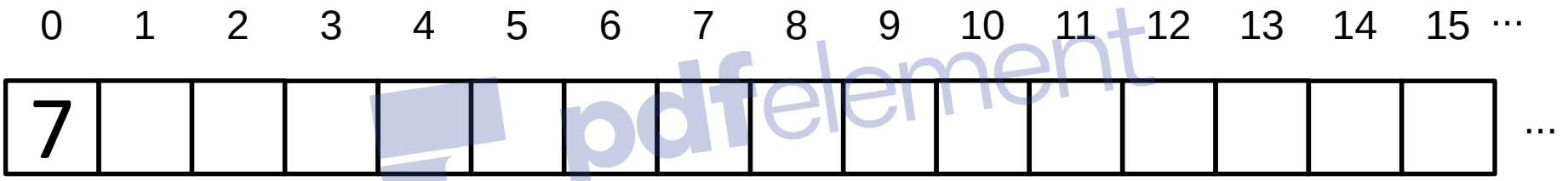
```
for [i = 0 to (log N -1)] {
    process Pj[j = 2^i+1 to 2^i+1]{ /*(Procesor Pj)*/
        int t; /* t locală Pj */
        t = A[j-2^i];
        A[j] = t;
    }
}
```

pdfelement

Remove Watermark Now

Value Broadcast

Start

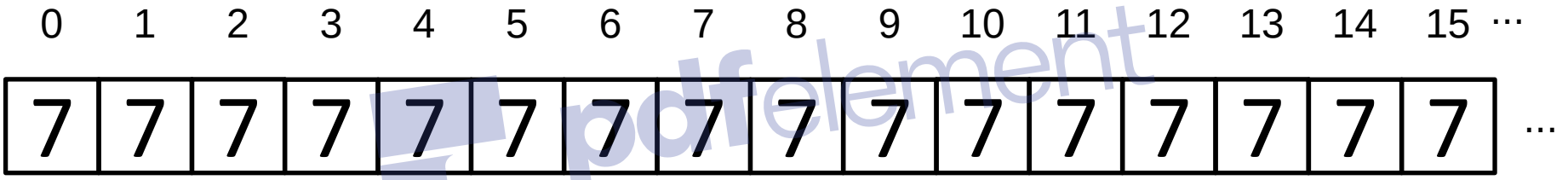


pdfelement

Remove Watermark Now

Value Broadcast

End

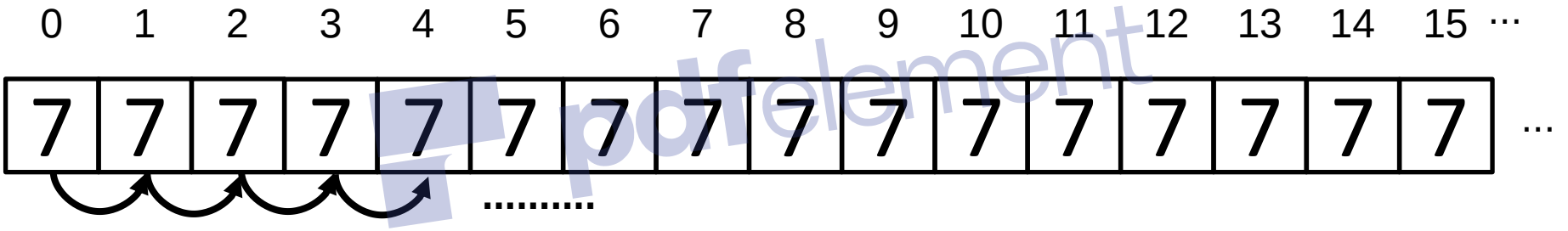


pdfelement

Remove Watermark Now

Inefficient Value Broadcast

O(n) time



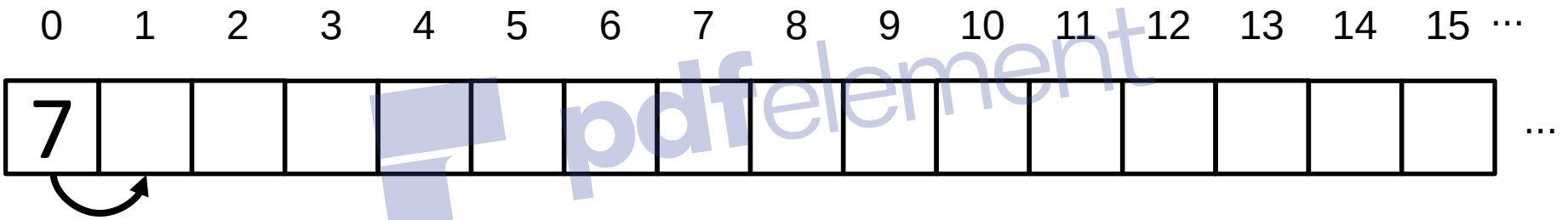
pdfelement

Remove Watermark Now

Efficient Value Broadcast

Every element that has the value
copies it to its current position + i

i = 1



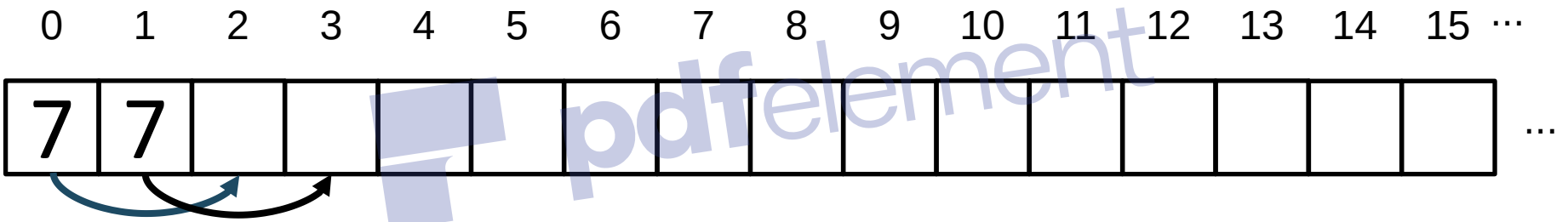
pdfelement

Remove Watermark Now

Efficient Value Broadcast

Every element that has the value
copies it to its current position + i

i = 2



pdfelement

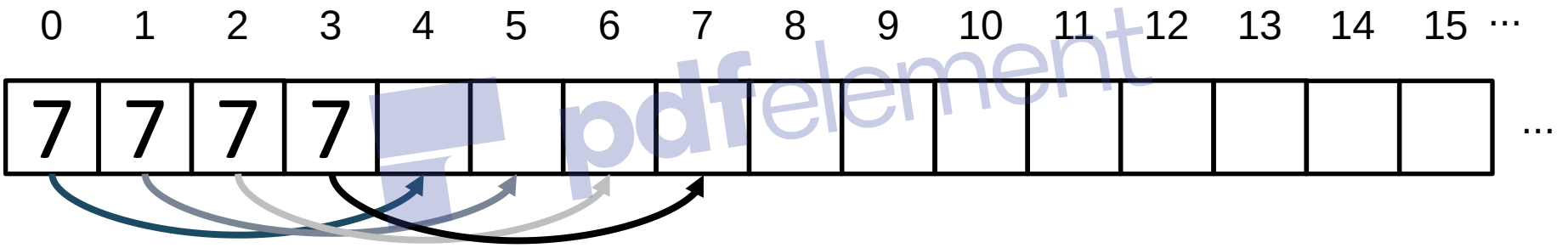
These operations can be executed in parallel

Remove Watermark Now

Efficient Value Broadcast

Every element that has the value
copies it to its current position + i

i = 4



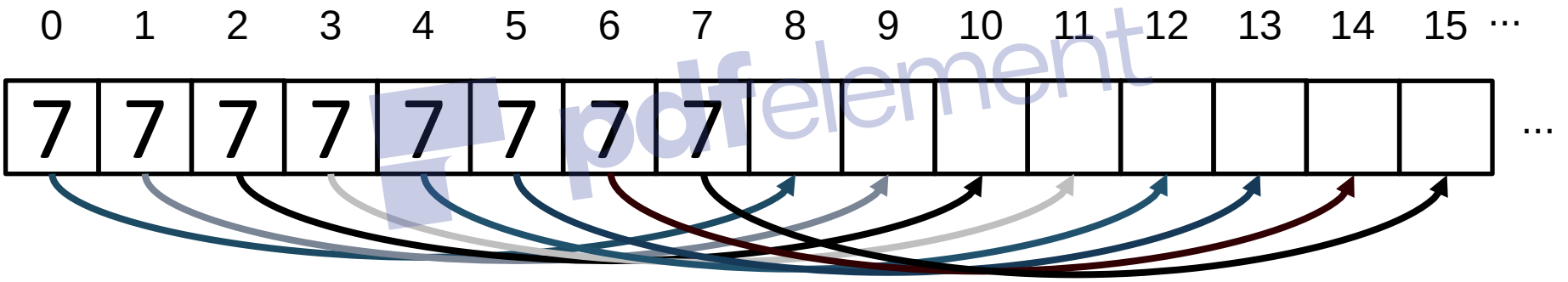
These operations can be executed in parallel

Remove Watermark Now

Efficient Value Broadcast

Every element that has the value
copies it to its current position + i

i = 8

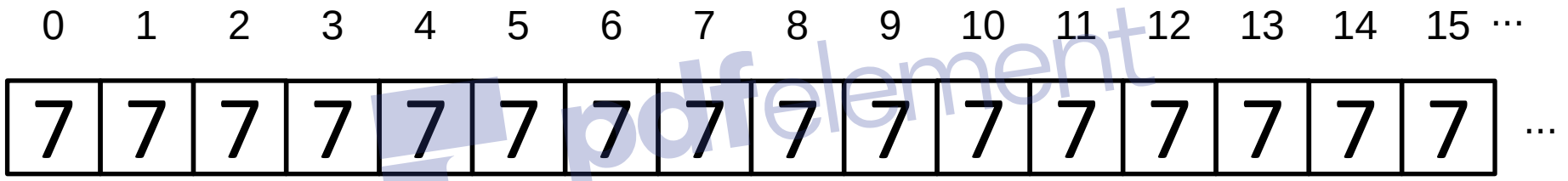


These operations can be executed in parallel

Remove Watermark Now

Efficient Value Broadcast

O(log2(n)) time with p = n/2 processors

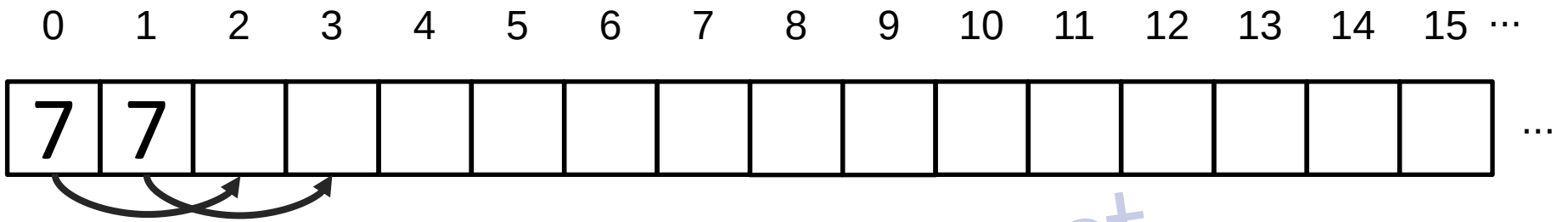


pdfelement

Remove Watermark Now

Efficient Value Broadcast

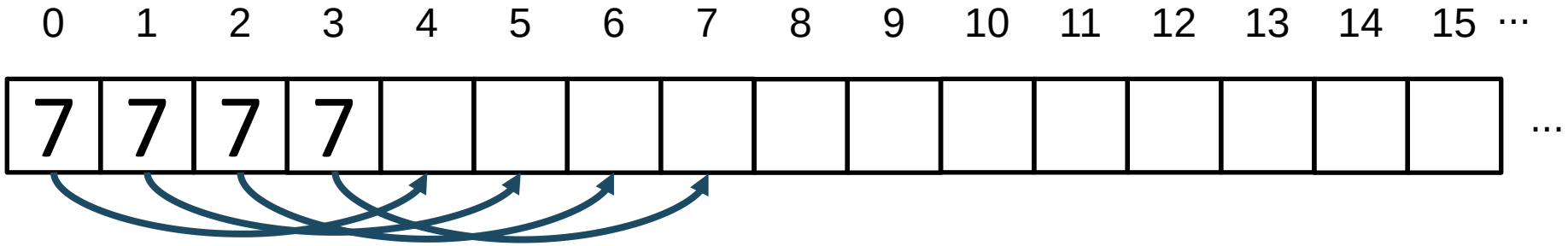
i = 2



pdfelement

These operations can**NOT** be executed in parallel

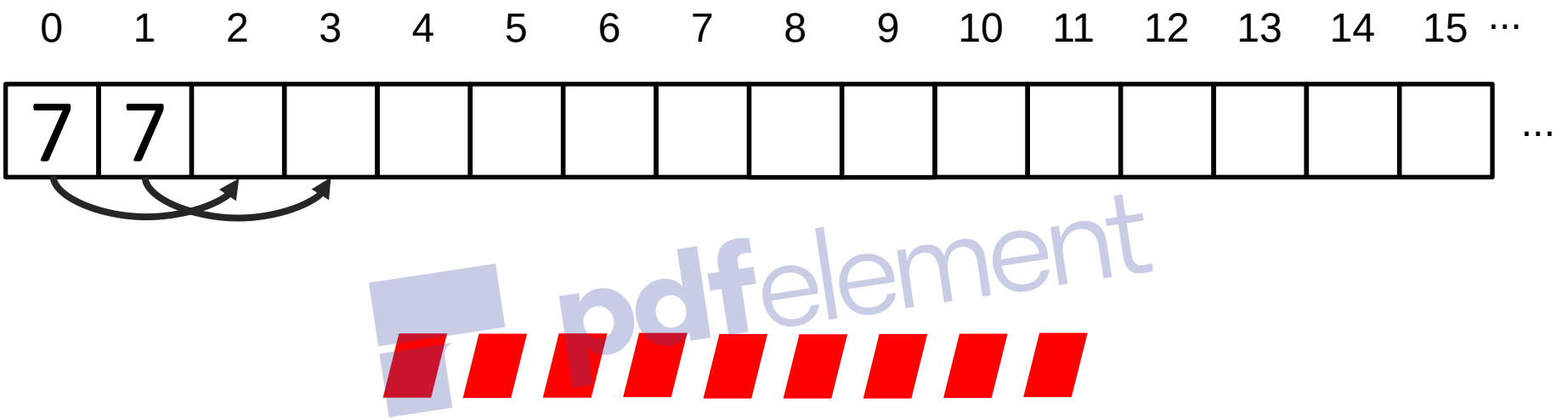
i = 4



Remove Watermark Now

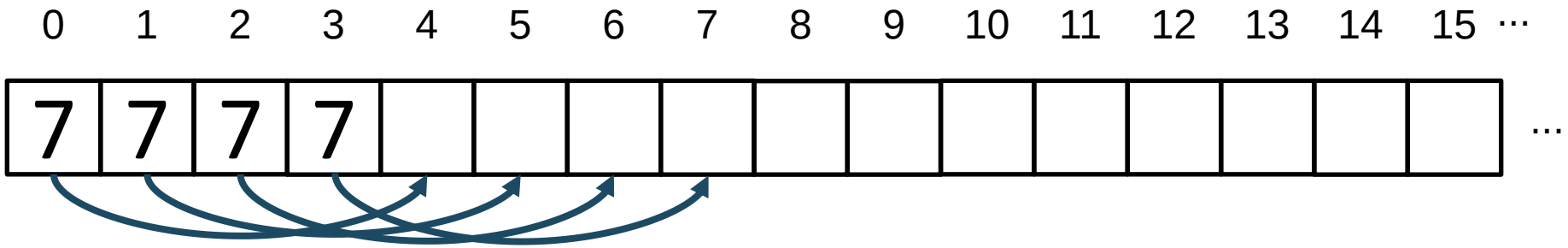
Efficient Value Broadcast

i = 2



pdfelement

i = 4



Remove Watermark Now

Operații cu liste

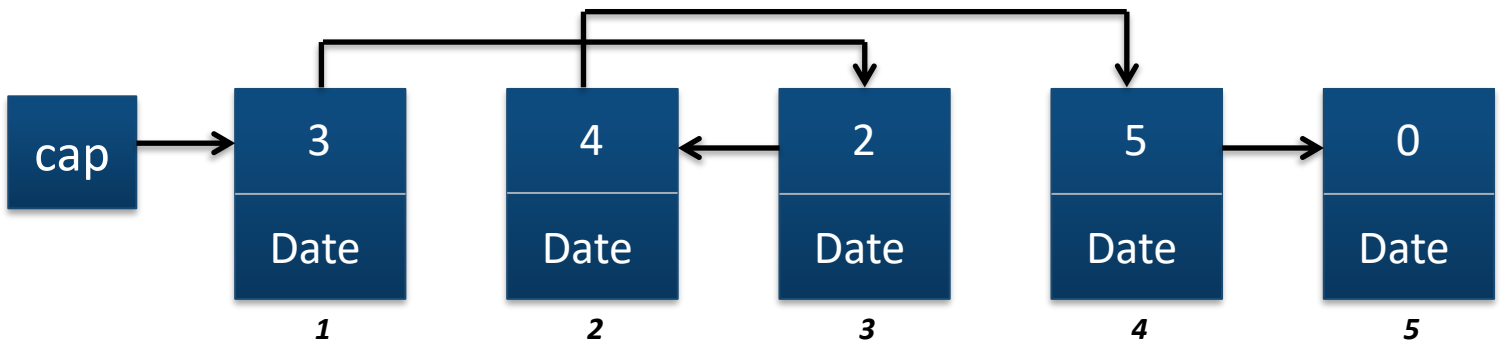
pdfelement

Operații cu liste

Remove Watermark Now

- Listă *n* elemente: `data[1:n]`
- Legăturile între elemente: `leg[1:n]`
- Capul listei: `cap`
- Elementul `i` face parte din listă ⇔ fie
  - `cap = i`
  - există un element `j`, între `1` și `n`, a.î. `leg[j]=i`

pdfelement



- Problemă: vrem ca fiecare procesor să afle capătul listei

Remove Watermark Now

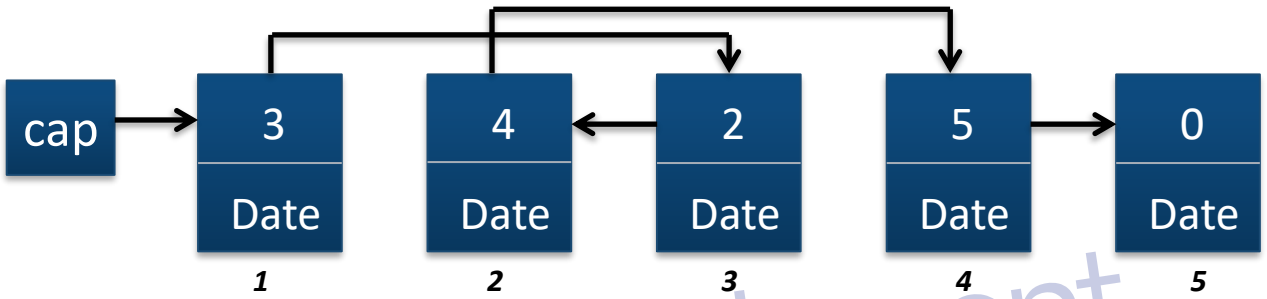
Operații cu liste

```
int leg[1:n], end[1:n];
process Află[i=1 to n] {
      int nou, d=1;
      end[i] = leg[i];
      barrier;
      while (d<n) {
            nou = 0;
            if (end[i]<>0 and end[end[i]]<>0)
                  nou=end[end[i]]
            barrier;
            if (nou<>0) end[i]=nou;
            barrier;
            d = 2*d;
      }
}
```

pdfelement

Remove Watermark Now

Operații cu liste



pdfelement

Procesor	End[i]	End[i]	End[i]	End[i]
1	3	2	5	5
2	4	5	5	5
3	2	4	5	5
4	5	5	5	5
5	0	0	0	0
	init	*d = 1* *d < 5*	*d = 2* *d < 5*	*d = 4* *d < 5*

```
int leg, end[1:n];
process Află[i=1 to n] {
    int nou, d=1;
    end[i] = leg[i];
    barrier;
    while (d < n) {
        nou = 0;
        if (end[i]<>0 and end[end[i]]<>0)
            nou = end[end[i]]
        barrier;
        if (nou <> 0)
            end[i] = nou;
        barrier;
        d = 2 * d;
    od
oc
```

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Algoritmi genetici paraleli - optional

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Introducere

Algoritmii genetici

- reprezintă o soluţie a problemelor de optimizare,
- bazată pe mecanisme împrumutate din genetică.

Un AG

- menţine o populaţie de indivizi,
- fiecare individ reprezinta o soluţie potenţială a unei probleme

AG realizeaza, în fiecare etapă, următoare operaţii:

- evaluarea populaţiei curente
- selecţia celor mai buni indivizi
- transformarea populaţiei folosind operatori genetici de încrucişare şi mutaţie.

pdfelement

Remove Watermark Now

Domeniile de aplicare

- optimizarea parametrilor
- control optim
- transport
- optimizare combinatorială
- desenare de grafuri
- învăţare inductivă a regulilor de decizie
- stabilirea cablajelor
- planificarea
- jocuri, modelarea cognitivă
- optimizarea interogării bazelor de date.

pdfelement

Remove Watermark Now

Metode clasice de optimizare

Problema:

- cautarea intr-un spatiu de siruri binare de 30 de biti, cu functia obiectiv

$$f(v) = |\ 11*one(v)-150|$$

- unde **one(v)** este numarul de unitati din vectorul binar v.
- Functia f(v) are
  - un maxim global pentru v_g = (1 1 1 1 …1), pentru care $f(v_g) = 180$ si
  - un maxim local pentru v_l = (0 0 0 …0), pentru care $f(v_l) = 150$.

pdfelement

Remove Watermark Now

Algoritmul hill climbing

```
procedure hillclimber
begin
    t := 0
    do t < MAX ->
        local := false
        selectează aleator sirul curent Vc
        evalueaza Vc
        do not local ->
            gaseste Vn dintre vecinii cu cea mai mare valoare a
                                functiei obiectiv F
            if F(Vc) < F(Vn) ->        Vc := Vn
            [] F(Vc) >= F(Vn) -> local := true
            fi
        od
        t := t+1
    od
end
```

Obs: daca sirul de pornire are <= 13 unitati atunci se gaseste intotdeauna maximul local

pdfelement

Remove Watermark Now

Simulated annealing

```
procedure simulated-annealing
begin
    t := 0
    initializeaza temperatura T
    selecteaza aleator sirul curent Vc
    evalueaza Vc
    do not cond-stop ->
        do not cond-terminare ->
            selecteaza un nou sir Vn vecin cu Vc
            if F(Vc) < F(Vn)   -> Vc = Vn
            [] F(Vc) >= F(Vn) ->
                if random[0,1) < exp((F(Vn)-F(Vc))/T) ->
                    Vc = Vn
                fi
            fi
        od
        T = g(T,t)
        t = t+1
    od
end
```

cand T atinge o anumita limită inferioară.

verifică atingerea unui “echilibru termic” de ex cand distribuţia probabilităţilor noilor şiruri selectate se apropie de Bolzmann.

pdfelement

Remove Watermark Now

- Pentru problema sirurilor binare:
- daca v12 are 12 unitati:
  - f(v12) = |11*12 – 150| = 18
  - f(v13) = |11*13 – 150| = 7
- Algoritmul accepta o solutie cu 13 unitati cu probabilitatea
  - p = exp((f(Vn)-f(Vc))/T) = exp((7-18)/T)
- pentru T=20:
  - p = 0.576 > 0.5

pdfelement

Remove Watermark Now

Algoritmi genetici

```
procedure algoritm_genetic
begin
    t := 0;
    creaza o populatie initiala de cromozomi P(t);
    evalueaza fiecare cromozom al populatiei initiale.
    do t < not conditie de terminare ->
        t := t+1;
        selecteaza parintii pentru reproducere;
        creaza noi cromozomi prin imperechere si mutatie;
        inlocuieste anumiti membri ai populatiei cu cei noi;
        evalueaza cromozomii din noua populatie
    od
end
```

pdfelement

Remove Watermark Now

Problema:

Dat fiind graful G = (V, E) să se găsească o partiţionare a sa în două subgrafuri având acelaşi număr de noduri, prin eliminarea unui număr minim de muchii.

Problema este NP completă.

Rezolvare cu alg genetici

- soluţie reprezentată ca un vector cu elemente binare, de dimensiune egală cu numărul nodurilor din graf
- o gena corespunde unui nod
  - 1 = nod in prima partitie
  - 0 = nod in a doua partitie

Evaluare solutie

- eval(x) = cutsize(x)

Generare populatie initiala

- aleator cu corectii
  - orice cromozom sa aiba acelasi numar de 0 si 1 (±1)

pdfelement

Remove Watermark Now

Selecţia parintilor pentru reproducere

Metoda ruletei

calculează eval(xi) pentru fiecare cromozom i = 1..pop-size

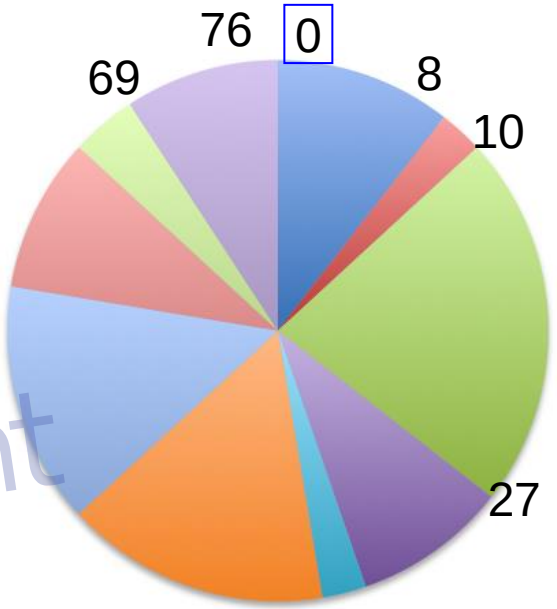
asociază cu fiecare cromozom o valoare de "potrivire"

f(xi) = max-fit - eval(xi)

astfel că f(xi) > 0 pentru orice i = 1..pop-size

calculează valorile de potrivire cumulative

cf(xi) = SUMA f(xj), j = 1..i



Selecţia

generează un număr aleator ***rand*** în intervalul [o, cf-max]

if ***rand*** <= cf(x_1) -> alege primul cromozom x_1 **fi**

if cf(x_{i-1}) < *rand* <= cf (x_i) -> alege x_i **fi**

pdfelement

Exemplu selectie cromozomi										
Cromozom	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Potrivire	**8**	**2**	**17**	**7**	**2**	**12**	**11**	**7**	**3**	**7**
Potrivire cumulata	**8**	**10**	**27**	**34**	**36**	**48**	**59**	**66**	**69**	**76**

Numar aleator	**23**	**49**	**76**	**13**	**1**	**27**	**57**
Cromozom ales	**3**	**7**	**10**	**3**	**1**	**3**	**7**

Remove Watermark Now

- Operatorul *mutation*

Aplicarea operatorului *mutation* pentru 0.08								
Cromozom vechi	1	0	1	0	1	1	0	0
Probabilitate	0.80	0.11	0.27	0.45	0.12	0.05	0.15	0.03
Cromozom nou	1	0	1	0	1	0	0	1

pdfelement

Operatorul de încrucişare (*crossover*)

Aplicarea operatorului *crossover* pentru pozitia 6								
Primul parinte	1	1	1	1	1	1	1	1
Al doilea parinte	0	0	0	0	0	0	0	0
Primul fiu	1	1	1	1	1	1	0	0
Al doilea fiu	0	0	0	0	0	0	1	1

Remove Watermark Now

Aplicarea operatorului *crossover* pentru pozitiile 3 si 6								
Primul parinte	1	1	1	1	1	1	1	1
Al doilea parinte	0	0	0	0	0	0	0	0
Primul fiu	0	0	0	1	1	1	0	0
Al doilea fiu	1	1	1	0	0	0	1	1

pdfelement

Aplicarea operatorului *crossover uniform*								
Primul parinte	1	1	1	1	1	1	1	1
Al doilea parinte	0	0	0	0	0	0	0	0
Masca	1	0	0	1	0	0	0	1
Primul fiu	1	0	0	1	0	0	0	1
Al doilea fiu	0	1	1	0	1	1	1	0

Remove Watermark Now

Justificarea funcţionării algoritmilor genetici

Schema

- este *un şir construit cu simbolurile 0, 1 şi * (don't care)*
- sunt posibile 3^m scheme de lungime **m**
- o schemă reprezintă toate şirurile care coincid cu ea în poziţiile diferite de *
- o schemă cu **r** simboluri * are drept corespondente 2^r şiruri
- un şir de lungime **m** poate fi pus în corespondenţă cu 2^m scheme
- intr-o populaţie de dimensiune **n** pot fi reprezentate între 2^m şi $n*2^m$ scheme

pdfelement

Proprietăţile schemelor

- Ordinul o(S) = numarul de pozitii fixe

  S1 = (010***11***1) are o(S1) = 6

- Lungimea caracteristică δ(S) = distanţa între prima şi ultima poziţie fixă

  δ(S1) = 11

Remove Watermark Now

Evoluţia schemelor - selectia

pop_size dimensiunea populaţiei eval(v_i) = potrivirea cromozomului i

m lungimea unui cromozom

Şirul v_i are probabilitatea de selectie conform ruletei $p_i = \frac{eval(v_i)}{F(t)}$

Potrivirea totală a populatiei $F(t) = \sum_{i=1}^{pop-size} eval(v_i)$

p = $\xi(S,t)$ nr. cromozomi v_{ij} care corespund la momentul *t* cu schema *S*.

Potrivirea schemei *S* la momentul *t* $eval(S,t) = \sum_{j=1}^{p} \frac{eval(v_{ij})}{p}$

Probabilitatea de selecţie a unui şir care corespunde schemei *S* = $\frac{eval(S,t)}{F(t)}$

Numar de selectii este *pop_size*

Rezulta $\xi(S,t+1) = \xi(S,t) * pop_size * \frac{eval(S,t)}{F(t)}$

pdfelement

Remove Watermark Now

Fie $\overline{\overline{F(t)}} = \frac{F(t)}{pop_size}$ potrivirea medie a populatiei

Atunci $$\xi(S,t+1) = \xi(S,t) * \frac{eval(S,t)}{\overline{\overline{F(t)}}}$$

O schemă "peste medie" are $\frac{eval(S,t)}{\overline{\overline{F(t)}}} > 1$

Rescriem relatia $eval(S,t) = \overline{F(t)}(1+\varepsilon)$

deci

$$\xi(S,t+1) = \xi(S,t) * (1+\varepsilon)$$

si S primeşte un număr mai mare de indivizi în noua populaţie.

Raportat la populatia initiala

$$\xi(S,t) = \xi(S,0) * (1+\varepsilon)^t$$

numărul de indivizi corespunzatori schemei S creşte in progresie geometrica.

pdfelement

Remove Watermark Now

Încrucişarea

Fie şirul (1110111101001)

şi schemele:

S0=(****111******), δ(S0)=2

S1=(111********01), δ(S1)=12

Dacă poziţia de tăiere = 10, schema S0 se regăseşte într-unul din fii, în timp ce schema S1 are şanse foarte reduse de reproducere.

➔lungimea caracteristică este importanta în reproductibilitatea schemei

m-1 posibilităţi selectie loc de încrucişare

probabilitatea de distrugere a schemei *S* este

probabilitatea de supravieţuire

pdfelement

Remove Watermark Now

cu prob incrucisarii p_c

$$p_s(S) = 1 - p_c * \frac{\delta(S)}{m-1}$$

si deoarece se pot combina indivizi aparţinând unor scheme comune

$$p_s(S) \geq 1 - p_c * \frac{\delta(S)}{m-1}$$

pdfelement

efect combinat selectie & incrucisare

$$\xi(S, t+1) \geq \xi(S, t) * \frac{eval(S,t)}{\overline{F(t)}} * (1 - p_c * \frac{\delta(S)}{m-1})$$

Remove Watermark Now

Mutaţia

probabilitatea mutaţiei unui singur bit este p_m

probabilitatea nemodificării sale este (1- p_m).

probabilitate ca o schemă S să supravieţuiască unei mutaţii

$$p_s(S) = (1 - p_m)^{o(S)} \quad \rightarrow \quad p_s(S) \approx 1 - p_m * o(S)$$

efectul combinat al selecţiei, încrucişării şi mutaţiei este

$$\xi(S, t+1) \geq \xi(S, t) * \frac{eval(S,t)}{\overline{F(t)}} * (1 - p_c * \frac{\delta(S)}{m-1} - p_m * o(S))$$

Teorema schemei. Schemele **scurte**, **de ordin scăzut** şi **peste medie** cresc ***exponenţial*** de-a lungul generaţiilor unui algoritm genetic.

Ipoteza blocurilor constructive. Un algoritm genetic atinge performaţe aproape optime prin juxtapunerea unor scheme scurte, de ordin scăzut, de mare performaţă, numite **blocuri constructive**.

pdfelement

Remove Watermark Now

Considerarea constrângerilor în algoritmii genetici

Metode:

- **penalizări** pentru solutii ce nu respecta constrângeri
- **eliminarea** soluţiilor necorespunzătoare
- aplicarea unor algoritmi de **corecţie**.

Ex. problema rucsacului.

- date fiind o mulţime de ponderi W, profiturile asociate P şi capacitatea C a rucsacului, să se găsească un vector binar $X = <x_1, x_2, ..., x_n>$ a.i.

$$\sum_{i=1,n} x_i * w_i <= C \text{ şi}$$

$$P(X) = \sum_{i=1,n} x_i * p_i \text{ este maxim.}$$

Functia de evaluare $eval(X) = \sum_{i=1,n} x_i * p_i - Pen(X)$

- unde $Pen(X) = 0$ pentru soluţiile fezabile şi
  $Pen(X) \neq 0$ pentru soluţiile nefezabile (suma > C).

Variante:

- logaritmică, $Pen_1(X) = \log_2 (1 + \rho (\sum_{i=1,n} x_i * w_i - C))$
- liniară, $Pen_2(X) = \rho (\sum_{i=1,n} x_i * w_i - C)$
- pătratică, $Pen_3(X) = (\rho (\sum_{i=1,n} x_i * w_i - C))^2$

unde $\rho = MAX_{i=1,n} \{ p_i / w_i \}$

pdfelement

Remove Watermark Now

Algoritmul bazat pe corecţia soluţiei

eval(X) = $\sum_{i = 1,n} x'_i * p_i$ unde X' este versiunea corectată a lui X

procedure corectie (X)
begin
depasire = false
X' = X
if $\sum_{i = 1,n} x'_i * w_i > C$ **->** depasire = true **fi**
do depasire **->**
i = selecteaza un element din sac
scoate elementul $x'_i = 0$
if $\sum_{i = 1,n} x'_i * w_i <= C$ **->** depasire = false **fi**
od
end

pdfelement

Remove Watermark Now

Algoritmul bazat pe decodificatori

Un cromozom este interpretat ca o strategie de a incorpora elemente într-o soluţie.

Fiecare cromozom este un vector X de **n** întregi.

Componenta **i** a vectorului X

- este un întreg din domeniul **1..n-i+1**
- indică poziţia unui element al unei liste **L**
- elementele selectate sunt eliminate din lista **L**
  - pas 1 – L are n elemente → valoarea X[1] in domeniul 1..n
  - pas 2 - L are n-1 elemente → valoarea X[2] in domeniul 1..n-1
  - s.a.m.d.

Efect

- la mutaţie, gena **i** poate lua orice valoare între **1** şi **n-i+1**,
- încrucişarea unor părinţi corecţi produce descendenţi corecţi.

pdfelement

Remove Watermark Now

Algoritmul bazat pe decodificatori

Exemplu

- fie L = (1,2,3,4,5,6).
- Vectorul X = <4,3,4,1,1,1> este decodificat astfel:

- X[1] = 4 → se alege L[4] = 4 – L devine L = (1,2,3,5,6)
- X[2] = 3 → se alege L[3] = 3 – L devine L = (1,2,5,6)
- X[3] = 4 → se alege L[4] = 6 – L devine L = (1,2,5)
- s.a.m.d.

pdfelement

Rezultat:

- Vectorul X = <4,3,4,1,1,1> este decodificat
- ca secvenţa 4, 3, 6, 1, 2, 5.
- 6 este al 4-lea element după eliminarea lui 4 şi a lui 3

Remove Watermark Now

```
procedure decode (X)
    construieste o lista L cu n elemente <wj, pj>
    i := 1
    suma-ponderi = 0
    suma-profit = 0
    do i <= n ->
        j = xi
        elimina elementul cu numarul de ordine j din L
        if suma-ponderi + wj <= C ->
            suma-ponderi = suma-ponderi + wj
            suma-profit = suma-profit + pj
        fi
        i = i+1
    od
end
```

pdfelement

Variante pentru construieste

- **aleator** – ordinea elementelor din fisierul de intrare care este aleatoare
- **greedy** - elementele în ordinea descrescătoare a rapoartelor p_i / w_i;

Remove Watermark Now

Algoritmi genetici paraleli

Abordari

- paralelizarea operatorilor genetici
- distribuirea populatiei.

Paralelizarea operatorilor genetici

- partitionare functionala: paralelizarea buclei care produce noua generatie
- se mentine o singura populatie
- operatorii genetici sunt aplicati in paralel
  - functia de potrivire si mutatia aplicate in paralel indivizilor
  - crossover la doi indivizi
- mai adaptat pentru memorie partajata

pdfelement

Remove Watermark Now

Abordarea populatie distribuita

- Corespunde descompunerii domeniului
- prelucrari facute in paralel pe diferite sub-populatii
- exploreaza mai bine spatiul solutiilor
- PGAs mentin sub-populatii separate care evolueaza independent

PGAs de granularitate fina

- Asigneaza un individ fiecarui task
- Fiecare task foloseste indivizi din vecinatatea sa
- Ajuta migrarea indivizilor

Intrebari:

- care este tolopogia de inter-conectare ?
- care este dimensiunea vecinatatii ?
- care este schema de inlocuire pentru includerea migratorilor in sub-populatia tinta ?

pdfelement

PGAs de granularitate mare

Remove Watermark Now

- Fiecare sub-populatie contine un numar mare de indivizi
- AG se executa independent pe fiecare sub-populatie si produce o solutie a problemei
- Rezultatul final este obtinut prin selectia celei mai bune solutii
- Indivizii migreaza periodic de la o sub-populatie la alta

Synchronous Island PGAs

- migrarea are loc dupa un anumit numar de generatii

Asynchronous Island PGAs

- migrarea intre doua sub-populatii nu este corelata cu restul migrarilor
- mai aproape de migrarea din natura
- mai potrivita cu algoritmi distribuiti

Intrebari

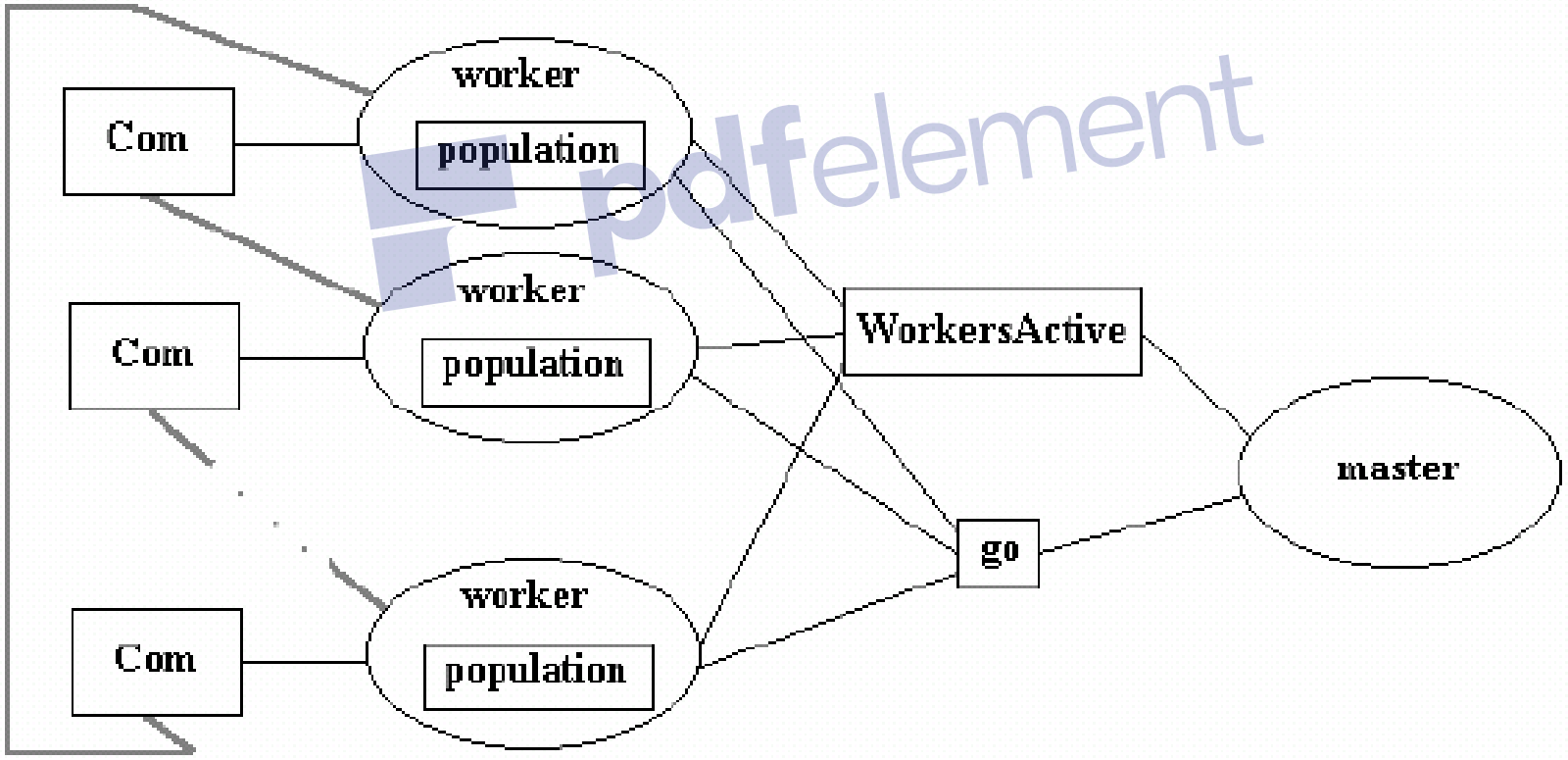
- care este topologia proceselor?
- este reteaua omogena sau nu?
- care este rata de migrare?
- cati migratori sunt inter-schimbati?
- cum sunt selectati migratorii?
- cum sunt combinati cu populatia tinta?

pdfelement

Remove Watermark Now

Asynchronous Island PGAs

Modelul "replicated workers"



pdfelement

Remove Watermark Now

Programe evolutive

Se bazează pe algoritmii genetici, dar folosesc:

- un set mai bogat de structuri de date
- un set extins de operatori genetici

pdfelement

Remove Watermark Now

Un exemplu - Problema transportului

Enuntul problemei

- se cere un plan de transport cu cost minim
  - un singur gen de marfa,
  - de la n surse la k destinatari
  - o destinatie poate primi marfa de la mai multe surse
  - costul unei rute este proportional cu cantitatea transportata (problema este lineara)

Notatii

sour[i]	cantitatea furnizata de sursa i
dest[j]	cantitatea ceruta de destinatia j
cost[i,j]	costul unitar de transport intre i si j
x[i,j]	cantitatea transportata intre i si j

pdfelement

Remove Watermark Now

Un exemplu - Problema transportului (2)

Se cere sa se minimizeze

$$\Sigma \Sigma \text{ cost}[i,j]*x[i,j] \text{ pentru } i=1,n;\ j=1,k$$

n surse

k destinatari

pdfelement

cu constrangerile

$$\Sigma_{j=1,k}\ x[i,j] <= \text{sour}[i]$$

$$\Sigma_{i=1,n}\ x[i,j] >= \text{dest}[j]$$

$$x[i][j] >= 0 \text{ pentru } i=1,n;\ j=1,k$$

Remove Watermark Now

Exemplu

- n = 3; sour = [15,25,5]
- k = 4; dest = [5,15,15,10]

- Matricea de cost →

10	0	20	11
12	7	9	20
0	14	16	18

- Matricea X corespunzatoare planului optim → cost total = 315

pdfelement

sour \ dest	5	15	15	10
15	0	5	0	10
25	0	10	15	0
5	5	0	0	0

Remove Watermark Now

AG clasic

Fiecare solutie reprezentata ca un **sir agregat** de biti

$$v_1, v_2, \ldots, v_p$$

unde p = n*k

si v_i sir binar de forma $v_i = w_i^0, \ldots, w_i^s$

v_i reprezinta un element x[j,m] asociat cu linia **j** coloana **m** ale matricei X, adica

$$j = \inf [(i-1)/k+1]$$

$$m = (i-1) \bmod k + 1$$

s calculat pentru a putea reprezenta intregul maxim din problema = $\mathbf{2^{s+1}-1}$.

Obs.

- nu exista o definitie simpla si naturala a operatorilor genetici
- producerea unor solutii care sa satisfaca constrangerile necesita corectii complicate

pdfelement

Remove Watermark Now

Reprezentare vectoriala

O solutie a problemei

- reprezentata ca un sir de **p=n*k** numere intregi
- cu valori in domeniul 1..p
- interpretata ca o strategie de a incorpora elemente în soluţie

pdfelement

Prima operatie pentru sirul {7,....} este

sour \ dest	5	15	15 (-15=0)	10
15	0	0	0	0
25 (-15=10)	0	0	**15**	0
5	0	0	0	0

Remove Watermark Now

Procedura de decodificare a unei solutii

```
procedure initialization;
begin
    seteaza ca nevizitate elem. din lista cu valori de la 1 la p;
    do not toate nodurile vizitate ->
        select urmatorul numar q si marcheaza vizitat;
        {calc nr linie i si coloana j corespunzatoare}
        i = (q-1)/k+1; j=(q-1) mod k +1;
        val = min(sour[i], dest[j]);
        v[i][j] = val;
        sour[i] = sour[i] – val;
        dest[j] = dest[j] – val
    od
end
```

pdfelement

Remove Watermark Now

- Solutia optima pentru problema de transport
  - {7,9,4,2,6, *,*,*,*,*,*,*}.

sour \ dest	5	15	15	10
15	0	5	0	10
25	0	10	15	0
5	5	0	0	0

pdfelement

Remove Watermark Now

Observatii

- orice permutare de numere cu valori intre 1 si p produce o solutie ce satisface constrangerile;
- operatorii genetici sunt simplu de realizat.
  - **Mutatia** se face prin interschimbarea a doua numere din sir.
  - **Incrucisare**

pdfelement

Ex. incrucisare:

Pentru perechea {1,2,3,4,5,6,7,8,9,10,11,12}

{7,3,1,11,4,12,5,2,10,9,6,8}

- se alege un tipar din primul parinte
- restul elementelor se pun in ordinea din al doilea parinte

rezulta {3,1,11, 4,5,6,7,12,2,10,9,8}

Remove Watermark Now

Reprezentarea matriceala

O solutie reprezentata in forma originala a problemei:

X = (x[i][j]) pentru i=1,k; j=1,n

0	0	5	0	3
0	4	0	0	0
0	0	5	7	0
3	1	0	0	2

Mutatia

- selecteaza o submatrice
- inlocuieste cu alta submatrice avand aceleasi sume pe linii si pe coloane

4	0	0
1	0	2

2	0	2
3	0	0

include in matricea originara

0	0	5	0	3
0	2	0	0	2
0	0	5	7	0
3	3	0	0	0

pdfelement

Remove Watermark Now

Incrucisarea

X1 si X2

1	0	0	7	0
0	4	0	0	0
2	1	4	0	5
0	0	6	0	0

0	0	5	0	3
0	4	0	0	0
0	0	5	7	0
3	1	0	0	2

DIV (media lor)
si REM (restul)

0	0	2	3	1
0	4	0	0	0
1	0	4	3	2
1	0	3	0	1

1	0	1	1	1
0	0	0	0	0
0	1	1	1	1
1	1	0	0	0

se imparte REM in
REM1 si REM2
(suma lor esteREM)

0	0	1	0	1
0	0	0	0	0
0	1	0	1	0
1	0	0	0	0

1	0	0	1	0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0

V3 = DIV+REM1
si V4 = DIV+REM2

0	0	3	3	2
0	4	0	0	0
1	1	4	4	2
2	0	3	0	1

1	0	2	4	1
0	4	0	0	0
1	0	5	3	3
1	1	3	0	1

pdfelement

Remove Watermark Now

Dilema prizonierului

Doi suspecti sunt arestati de politie. Politia nu are suficiente probe pentru condamnare si separa prizonierii, vizitandu-i si facandu-le aceeasi propunere.

- Daca unul depune marturie pentru pedepsirea celuilalt (tradeaza) si celalalt tace, tradatorul este eliberat si cel care tace ia 10 ani de puscarie.
- Daca amandoi tac, ambii sunt condamnati la 2 ani pentru o vina minora.
- Daca fiecare il tradeaza pe celalalt, fiecare primeste o sentinta de 5 ani.

Fiecare prizonier trebuie sa aleaga intre tradare si tacere

Fiecare este asigurat ca celalalt nu va sti despre tradarea lui inaintea terminarii investigatiei.

Cum trebuie sa actioneze prizonierii?

pdfelement

Remove Watermark Now

Dilema prizonierului

Jucatorul 1	Jucatorul 2	P1	P2	Comentariu
Tradeaza	Tradeaza	1	1	pedeapsa pentru tradare simultana
Tradeaza	Tace	5	0	rasplata tradarii / pedeapsa tacerii
Tace	Tradeaza	0	5	pedeapsa tacerii / rasplata tradarii
Tace	Tace	3	3	rasplata tacerii

pdfelement

Gasirea strategiei optime

Din teoria jocurilor:

- jucatorii prefera sa tradeze (fiecare presupune ca si celalalt tradeaza si pedeapsa este mica)
- pentru un joc iterativ (doi jucatori repeta de mai multe ori jocul), cooperarea devine posibila, fiecare gandindu-se la un castig mai mare
- se cauta strategia optima de castig bazata pe ultimele trei runde
- se compara
  - o strategie clasica cu una obtinuta prin algoritmi genetici sau
  - doua strategii obtinute genetic

Remove Watermark Now

Proiectarea algoritmului genetic

Reprezentarea solutiei:

- **trei mutari anterioare** ➔ 4^3 = 64 cazuri posibile ➔ 6 biti
- pentru fiecare caz se alege o decizie de tradare sau tacere
- o decizie = 1 bit ➔ 64 cazuri posibile = 64 biti
- Total 70 biti / cromozom

pdfelement

Obtinerea strategiei optime:

Se considera o populatie initiala

- fiecare jucator reprezentat de 70 de biti generati aleator.

Se evalueaza strategia fiecarui jucator

- prin calculul scorului mediu obtinut intr-un numar de jocuri stabilit

Se face selectia jucatorilor pentru inmultire.

Se face imperecherea cu incrucisare si mutatie.

Obtine o noua populatie si reia ciclul

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Comunicarea prin mesaje. Complexitatea algoritmilor distribuiți

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

1. Comunicarea prin mesaje

pdfelement

Remove Watermark Now

Transmiterea de mesaje

- Îndeplinește cele două funcții:
  - **comunicarea** - datele ajung de la transmiţător la receptor
  - **sincronizarea** - un mesaj nu poate fi recepţionat înainte de a fi transmis
- **Canalele** sunt obiectele puse în comun de procese
- **Programe distribuite**

pdfelement

Remove Watermark Now

Comunicare asincrona prin mesaje

pdfelement

Remove Watermark Now

Comunicarea asincronă prin mesaje

Modelul

- Canalele păstrează ordinea mesajelor
- Canalele nu pierd mesajele
- **canalele** sunt cozi FIFO de mesaje, de capacitate nelimitata
- operaţia **send** nu este blocantă
- operaţia **receive** este blocantă

Declaraţie – forma generală

chan nume_canal (tip1 id1,..., tipn idn)

Grup indexat de canale

chan rezultat [1:n](int)

Operaţii

send nume_canal (expresie$_1$,..., expresie$_n$)

receive nume_canal (variabila$_1$,..., variabila$_n$)

empty (nume_canal)

pdfelement

Remove Watermark Now

Exemplu - Filtru

```
chan   input (char),
       output (char [1:Maxl]);

process car_linii{
  char line[1:Maxl]; int i=1;
  while (true) {
    receive input (line[i]);
    while (line[i] <> CR and i < Maxl) {
      /* line[1:i] = ultimele i caractere introduse*/
      i = i + 1;
      receive input (line[i]);
    }
    send output (line);
    i = 1;
  }
}
```

pdfelement

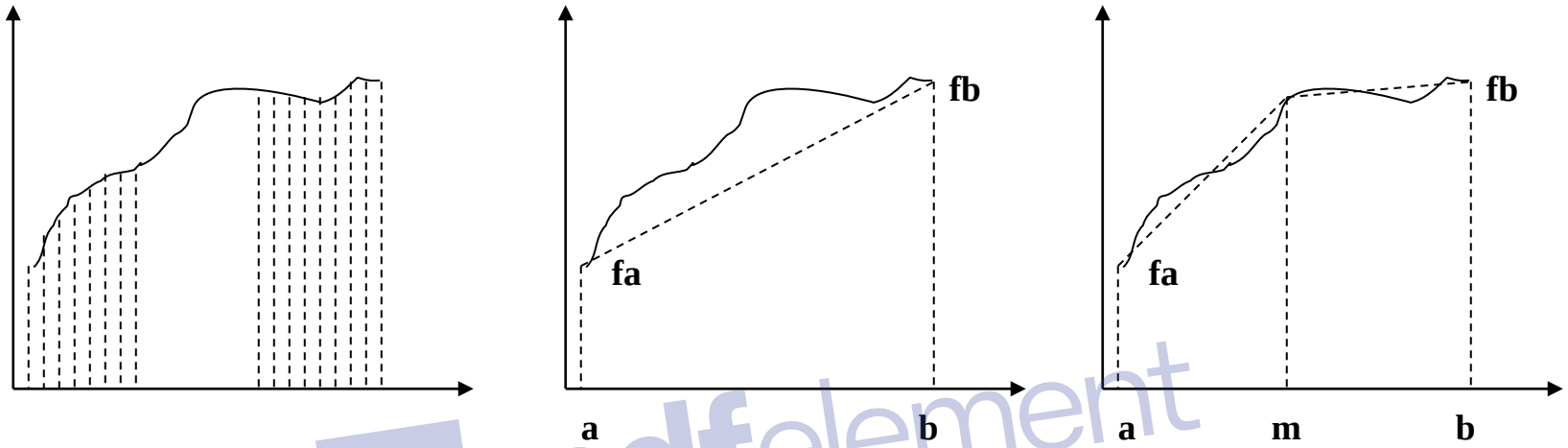
Remove Watermark Now

Modelul Replicated Workers

pdfelement

Remove Watermark Now

Integrare numerica

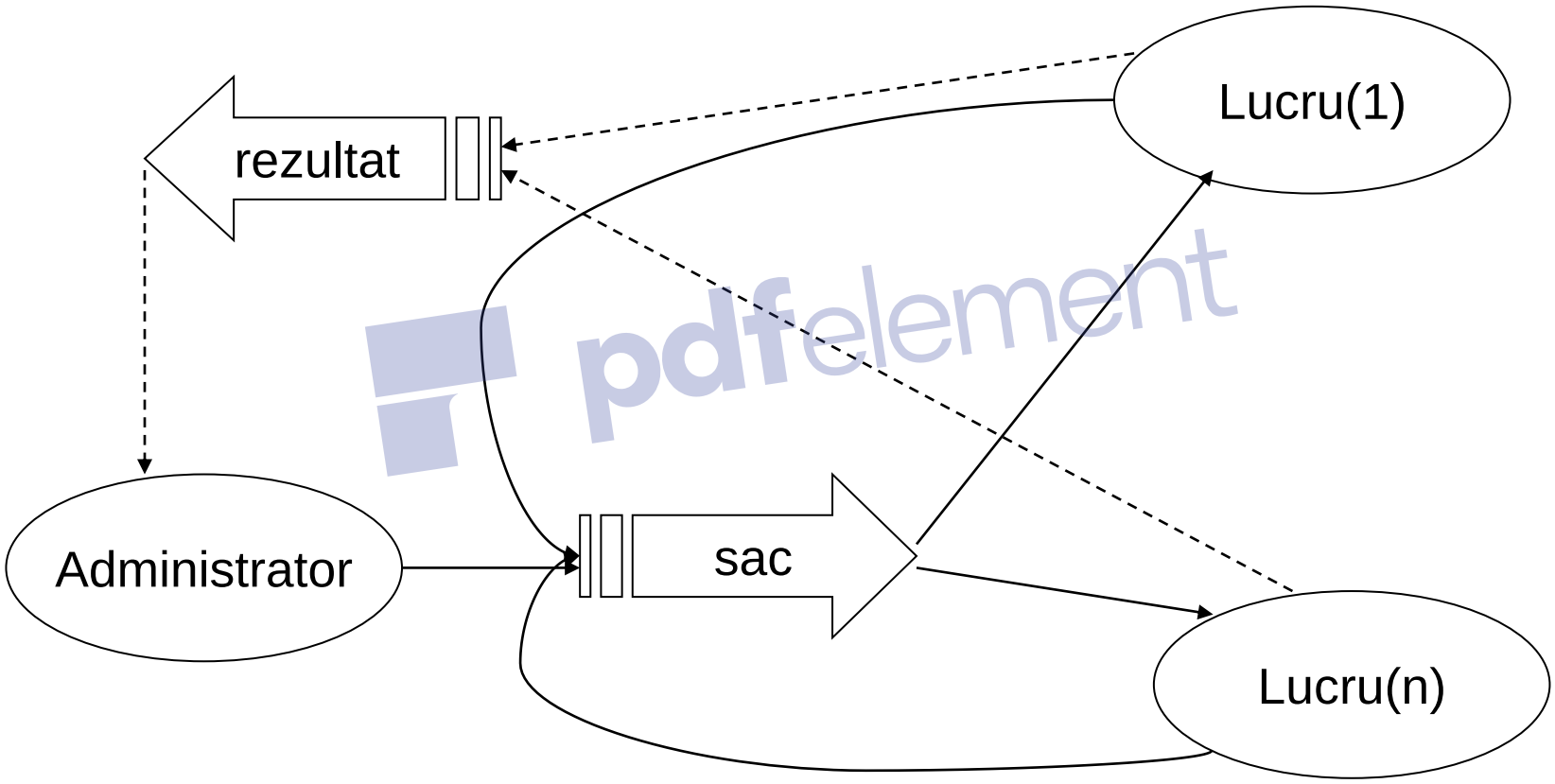


pdfelement

- Stanga – varianta secventiala
  - calcul iterativ - la fiecare pas se dubleaza numarul de subintervale
  - se termina cand diferenta intre doua valori succesive este acceptabila
- Dreapta – varianta distribuita
  - un proces primeste un subinterval [a,b] si aria corespunzatoare
  - imparte intervalul in doua sub-intervale [a.m], [m,b] si calculeaza ariile
  - compara suma celor doua arii cu aria primita
  - continua calculul separat pe cele doua subintervale, daca este cazul

Remove Watermark Now

Replicated workers



pdfelement

Remove Watermark Now

Exemplu - Replicated Workers (2)

```
chan sac (real a, b, fa, fb, aria);
chan rezultat (real a, b, aria);

process Administrator{
  real xl, xr, yl, yr, a, b, aria, total: real;
  /* alte variabile marcare intervale terminate */

  yl = f(xl); yr = f(xr);
  aria = (yl + yr) * (xr - xl) / 2;
  send sac(xl, xr, yl, yr, aria);

  while (nu s-a calculat toata aria) {
    receive rezultat (a, b, aria);
    total = total + aria;
    marchează intervalul [a, b] terminat;
  }
}
```

pdfelement

Remove Watermark Now

Exemplu - Replicated Workers (3)

```
process Lucru[i=1 to n]{
   real a, b, m, ya, yb, ym;
   real ariaSt, ariaDr, ariaTot, dif;
   while (true) {
      receive sac (a, b, ya, yb, ariaTot);
      m = (a + b) / 2;
      ym = f(m);
      calculează ariaSt și ariaDr;
      dif = ariaTot - ariaSt - ariaDr;
      if (dif mic) send rezultat (a, b, ariaTot);
      else { send sac (a, m, ya, ym, ariaSt);
             send sac (m, b, ym, yb, ariaDr);
      }
   }
}
```

pdfelement

Remove Watermark Now

Comunicare sincrona prin mesaje

pdfelement

Remove Watermark Now

Comunicarea sincrona prin mesaje

Modelul

Caracteristici similare cu comunicarea asincrona

Diferenta: si operaţia **send** **este** blocantă – transmitatorul se blocheaza pana cand mesajul este receptionat

pdfelement

Operaţii

synch_send nume_canal (expresie$_1$,..., expresie$_n$)

receive nume_canal (variabila$_1$,..., variabila$_n$)

empty (nume_canal)

Neajunsuri

reduce concurenta

Remove Watermark Now

Exemplu – comunicare producator / consumator

```
chan prodcons (real);

process prod {
    real date[n];
    for [i = 1 to n] {
        calculeaza date[i];
        synch_send prodcons (date[i]);    /* intarzie transmitatorul */
    }                                     /* daca receptorul este mai lent */
}
process cons {
    real buf[n];
    for [i = 1 to n] {
        receive prodcons (buf[i]);
        prelucreaza datele primite;
    }
}
```

pdfelement

Remove Watermark Now

2. Complexitatea algoritmilor distribuiți

pdfelement

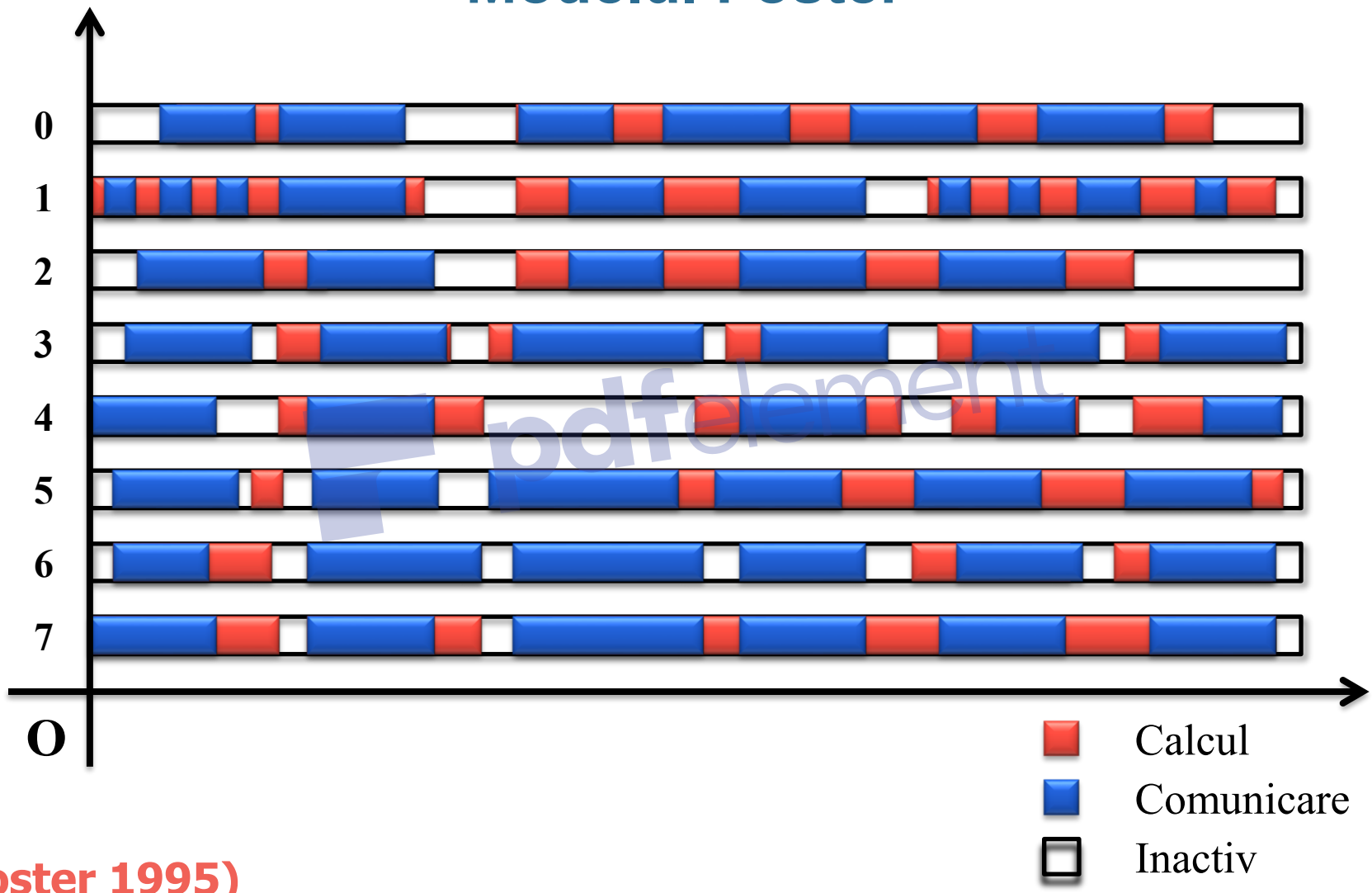
Remove Watermark Now

Modelul Foster

pdfelement

Remove Watermark Now

Modelul Foster



pdfelement

(Foster 1995)

Remove Watermark Now

Timpul total de execuţie

- Definiţie
  - Timpul scurs de la începerea execuţiei primului proces până la terminarea execuţiei ultimului proces.

pdfelement

$$T = f\ (N,\ P,\ U, \ldots)$$

$$= T^{j}_{comp} + T^{j}_{commun} + T^{j}_{idle}$$

$$= \left(\frac{1}{P}\right) * \left(\sum_{i=0}^{P-1} T^{i}_{comp} + \sum_{i=0}^{P-1} T^{i}_{commun} + \sum_{i=0}^{P-1} T^{i}_{idle}\right)$$

$$= \left(\frac{1}{P}\right) * \left(T_{comp} + T_{commun} + T_{idle}\right)$$

Remove Watermark Now

Timpul de calcul - T_{comp}

- Depinde de:
  - dimensiunea problemei N
  - numărul de task-uri sau procesoare
  - caracteristicile procesoarelor şi memoriei
- Exemplu:
  - scalarea dimensiunii problemei sau a numărului de procesoare poate modifica performanța cache-ului sau eficacitatea pipelining-ului procesorului

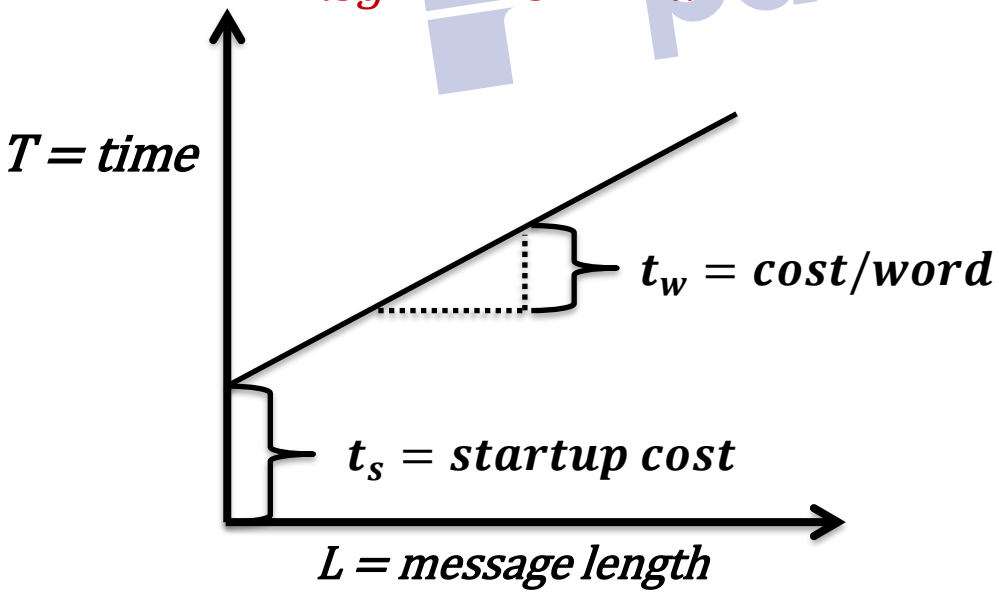
pdfelement

Remove Watermark Now

Timpul de comunicare – T_{commun} (1)

- Timpul petrecut cu transmiterea - recepţia de date
- Tipuri:
  - inter-procesor
  - intra-processor
- Presupunere: au costuri comparabile

$$T_{msg} = t_s + t_w L$$



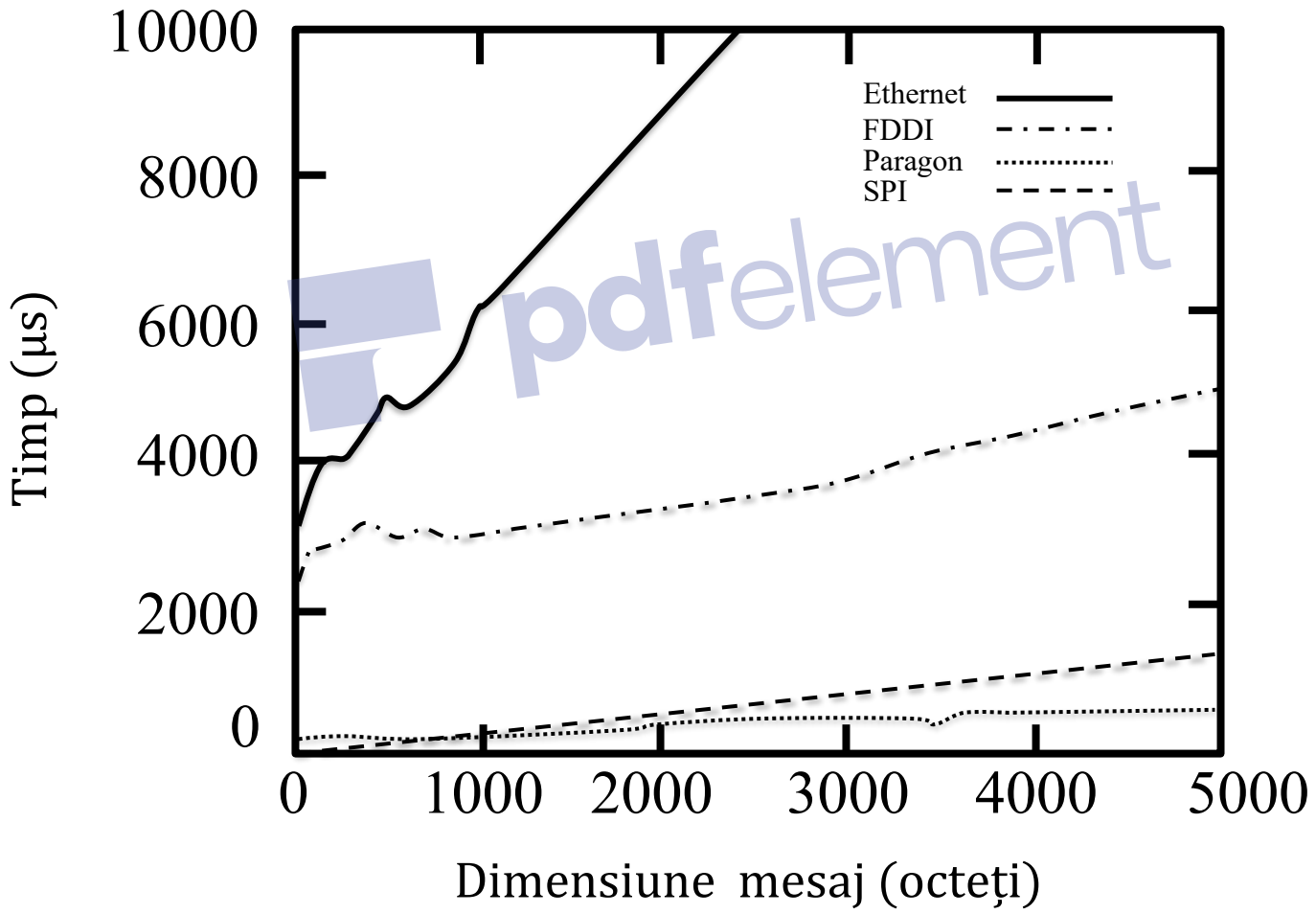
Machine	t_s (µs)	t_w(µs)
IBM SP2	40	0.11
Intel DELTA	77	0.54
Intel Paragon	121	0.07
Meiko CS-2	87	0.08
nCUBE-2	154	2.4
Thinking Machines CM-5	82	0.44
Workstations on Ethernet	1500	5.0
Workstations on FDDI	1150	1.1

pdfelement

Remove Watermark Now

Timpul de comunicare – T_{commun} (1)

- Măsurători experimentale: timp dus-întors pentru mesaje (RTT)



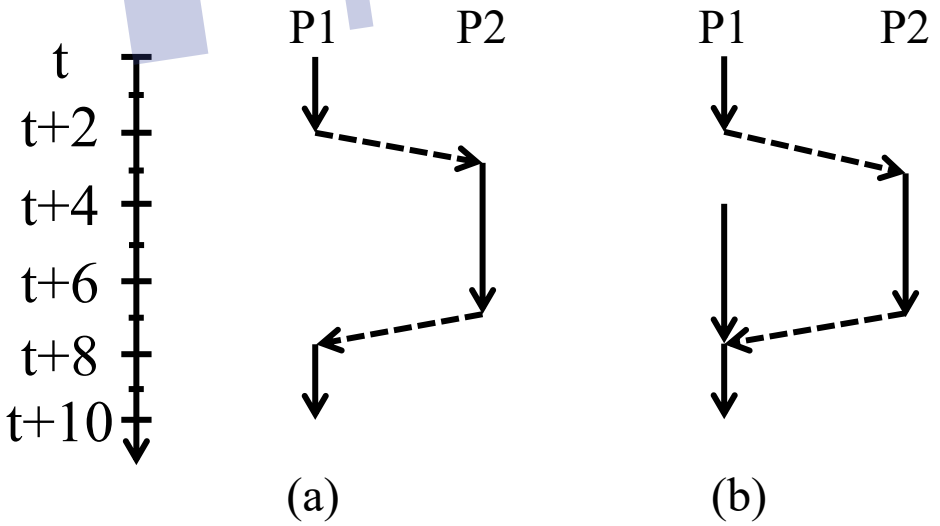
pdfelement

Remove Watermark Now

Timpul idle - T_{idle}

- Datorită:
  - lipsei calculelor → *load balancing*
  - lipsei datelor → *overlapping computation and communication*

pdfelement



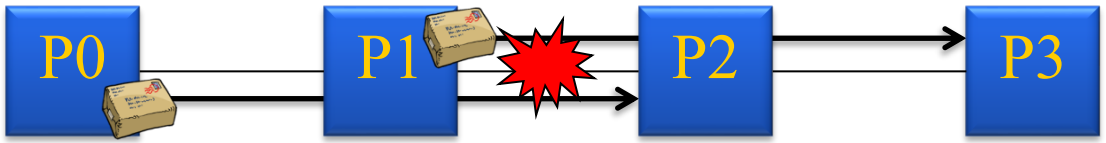
Remove Watermark Now

Model revizuit pentru cost comunicare

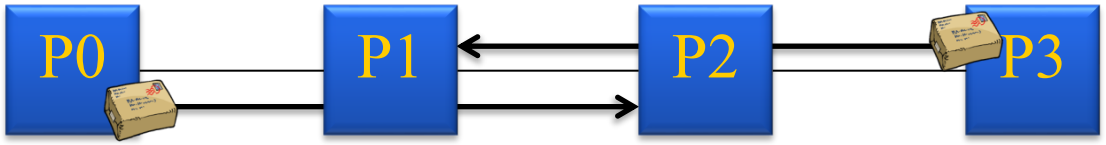
- Model comunicare simplu: $T_{msg} = t_s + t_w L$
- Competiţie pentru bandă: $T_{msg\text{-}b} = t_s + t_w SL$
- S = număr de procesoare care comunică concurent pe acelaşi canal (și în același sens)

pdfelement

(a) S=1



(b) S=2



(c) S=1

Remove Watermark Now

Exemplu – Sequential Floyd

- $I[i,j]_0 = 0$, daca i == j
- $I[i,j]_0 = cost(i,j)$, dacă există muchie și i != j
- $I[i,j]_0 = MAX$, altfel

```
for [k = 0 to N - 1]
  for [i = 0 to N - 1]
    for [j = 0 to N - 1]
      I[i,j]k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k)
```

pdfelement

Durata unei iterații

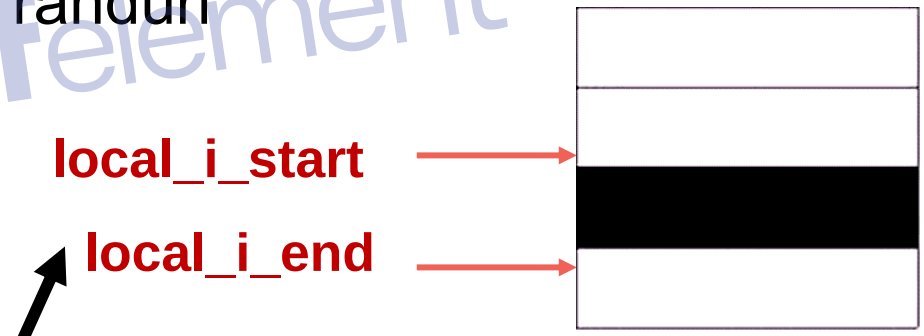
$$T = t_c N^3$$

Remove Watermark Now

Exemplu – Parallel Floyd 1 (1)

- Bazat pe o descompunere uni-dimensională pe rânduri a matricei I (algoritmul poate folosi cel mult P procesoare, P<=N)
- Fiecare task are unul sau mai multe rânduri adiacente din I şi răspunde de calculul acestor rânduri

pdfelement



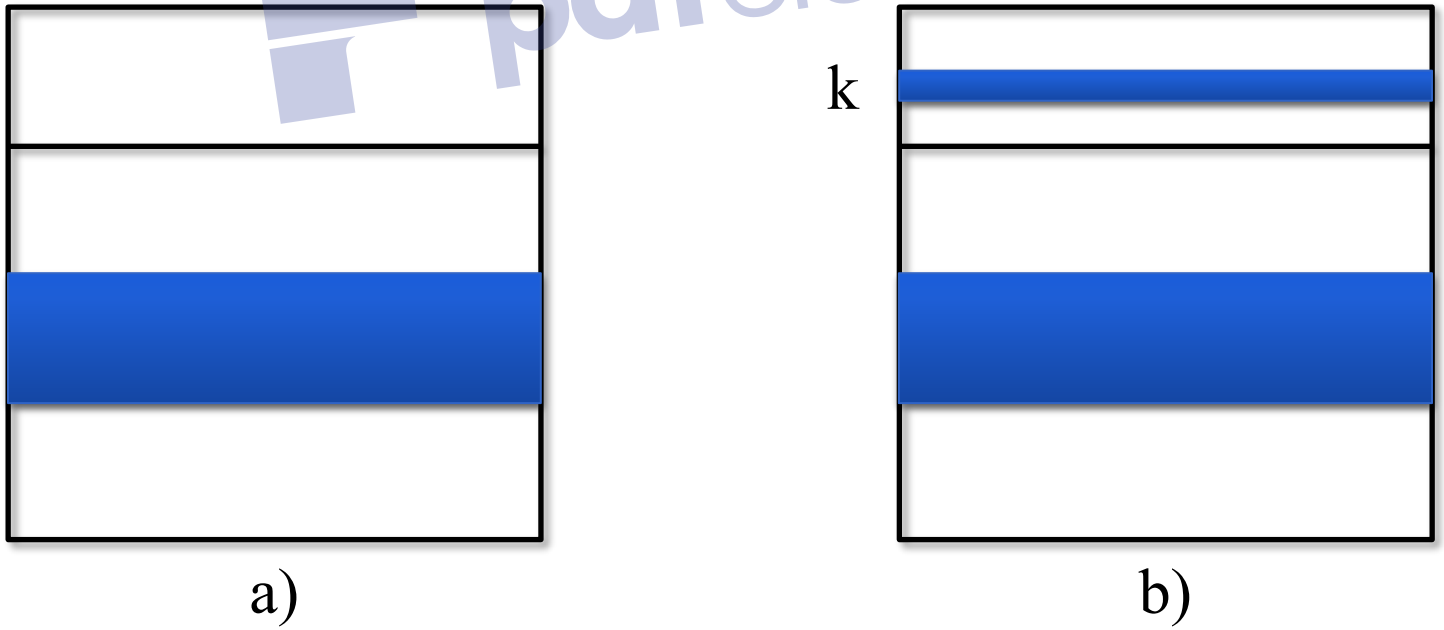

```
for [k = 0 to N-1]
   for [i = local_i_start to local_i_end]
     for [j = 0 to N-1]
     I[i,j]k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k)
```

Remove Watermark Now

Exemplu – Parallel Floyd 1 (2)

- În pasul k, procesele au nevoie de linia k din matricea I.
- Procesorul cu această linie o difuzează tuturor în log P paşi folosind o structură arborescentă.
- Deoarece fiecare mesaj are N cuvinte, timpul de difuzare a unei linii este: log P ($t_s + t_w N$)

pdfelement



Remove Watermark Now

Exemplu – Parallel Floyd 1 (3)

- Fiecare task este rădăcină pentru cel puţin o difuzare (dacă P ≤ N).
- Folosind un hipercub, fiecare nod poate difuza în log P paşi
- Pentru N difuzări, cu mesaje de lungime N:

$$T_{Floyd1} = \frac{t_c N^3}{P} + N \log P \, (t_s + t_w N)$$

Timpul de comunicare

Timpul algoritmului secvențial împărțit la numărul de procesoa

pdfelement

Remove Watermark Now

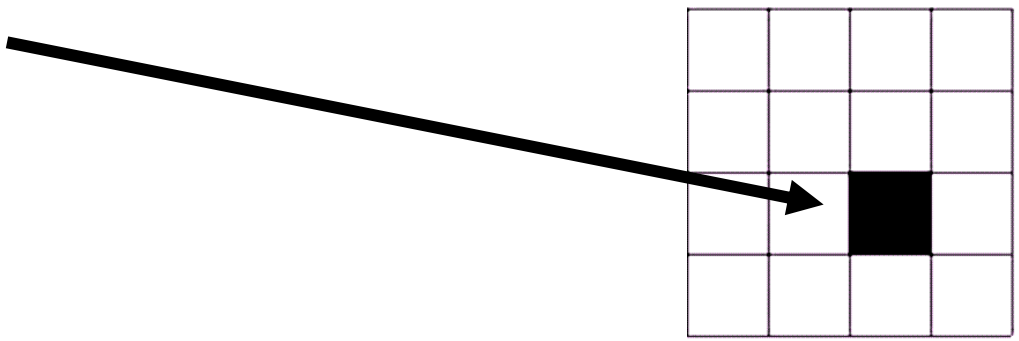
Exemplu – Parallel Floyd 2 (1)

- Descompunere bi-dimensională a matricelor.
- Foloseşte până la N^2 procesoare.

```
for [k = 0 to N - 1]
  for [i = local_i_start to local_i_end]
    for [j = local_j_start to local_j_end]
      I[i,j]k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k)
```

pdfelement

- Fiecare task răspunde de calculul elementelor situate intr-un patrat.



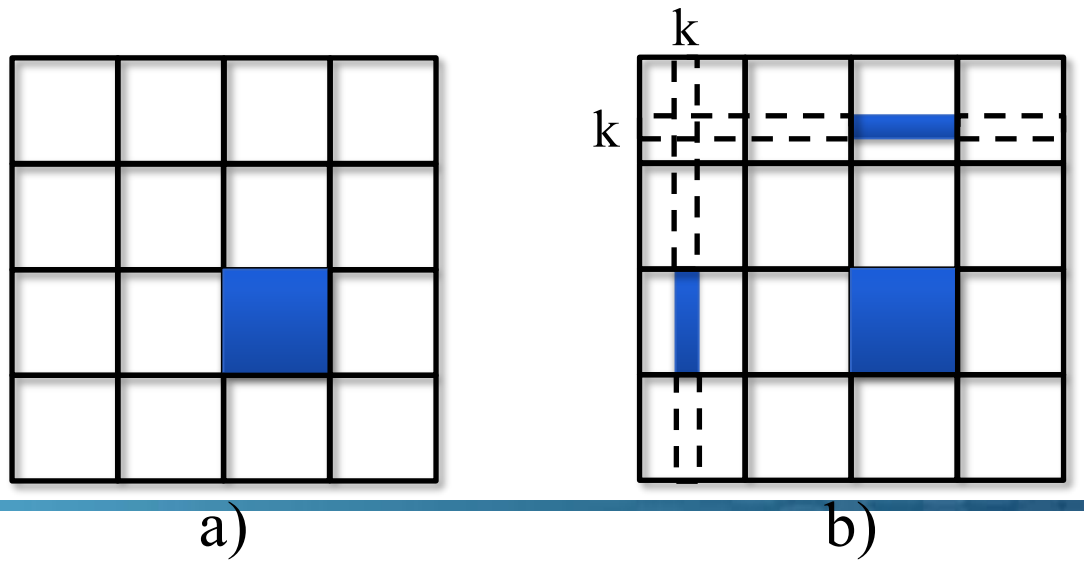
Remove Watermark Now

Exemplu – Parallel Floyd 2 (2)

- Cerinţe de comunicare în pasul k: două operaţii de difuzare.
- În fiecare pas, fiecare task cere, în plus faţă de datele locale, $\frac{N}{\sqrt{P}}$ valori de la două task-uri din aceeaşi linie şi aceeaşi coloană.
- Fiecare din cele doua mesaje are $\frac{N}{\sqrt{P}}$ elemente.
- Difuzarea unui mesaj durează (sunt $\sqrt{P}$ procese):

$$\log\sqrt{P}\left(t_s + \frac{t_w N}{\sqrt{P}}\right)$$

pdfelement



Remove Watermark Now

Exemplu – Parallel Floyd 2 (3)

- În fiecare din N paşi, $\frac{N}{\sqrt{P}}$ valori trebuie difuzate la $\sqrt{P}$ task-uri în fiecare linie şi coloană (se poate folosi o structură hipercub); costul total este:

$$T_{Floyd\text{-}2} = t_c \frac{N^3}{P} + 2N \log\sqrt{P}\left(t_s + \frac{t_w N}{\sqrt{P}}\right)$$

$$= t_c \frac{N^3}{P} + N \log P \left(t_s + \frac{t_w N}{\sqrt{P}}\right)$$

pdfelement

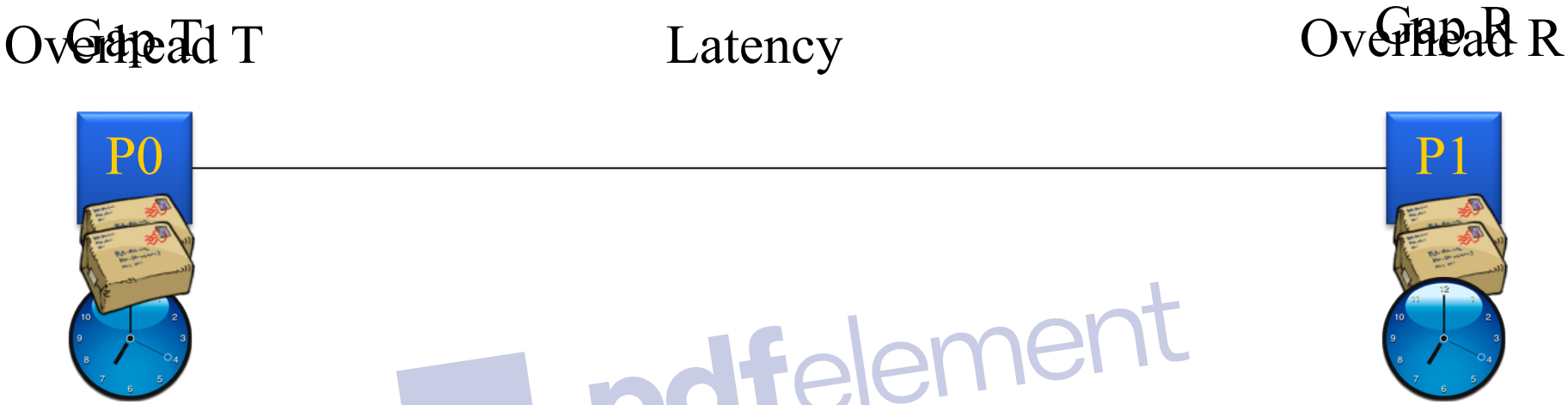
Remove Watermark Now

Modelul LogP

pdfelement

Remove Watermark Now

Modelul LogP



pdfelement

L - latency sau întârzierea de transmitere a unui mesaj mic de la sursă la destinatar

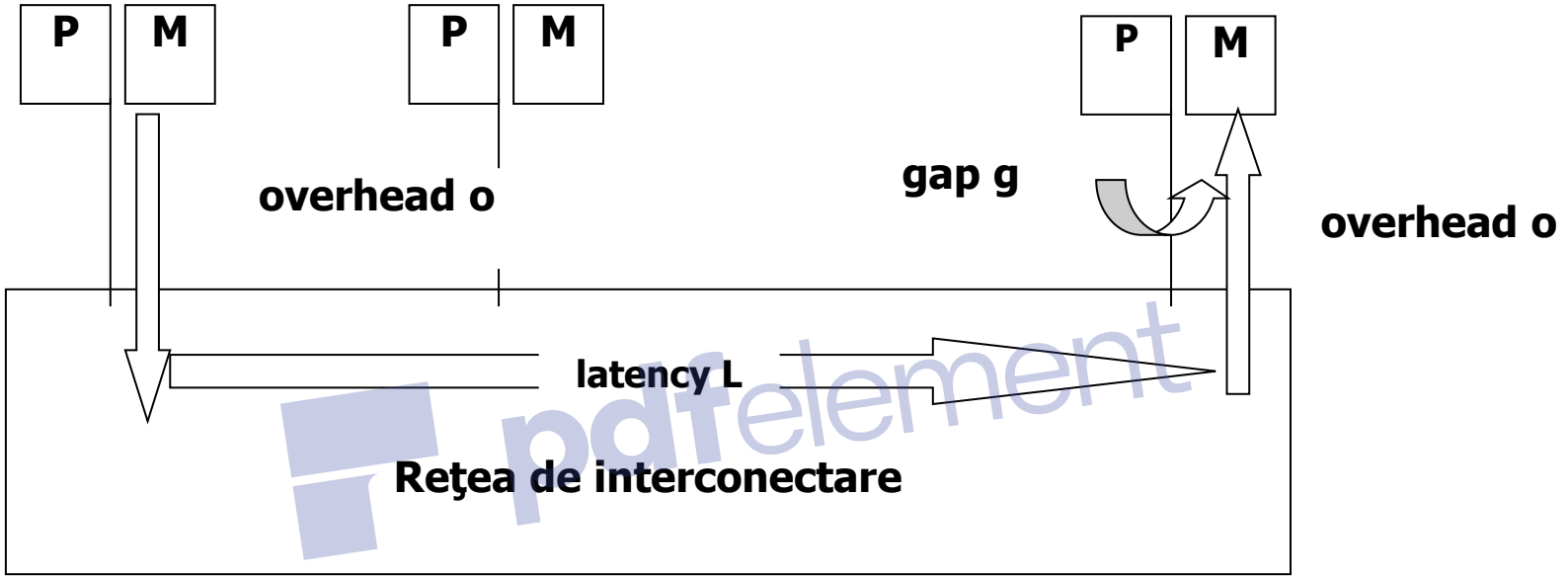
o - overhead, durata pentru care procesorul este angajat în transmiterea sau recepţia fiecărui mesaj

g - gap, intervalul minim de timp între două transmiteri succesive sau două recepţii succesive la acelaşi modul

P - numărul de module procesor / memorie.

Remove Watermark Now

Modelul LogP



pdfelement

L - latency sau întârzierea de transmitere a unui mesaj mic de la sursă la destinatar

o - overhead, durata pentru care procesorul este angajat în transmiterea sau recepţia fiecărui mesaj

g - gap, intervalul minim de timp între două transmiteri succesive sau două recepţii succesive la acelaşi modul

P - numărul de module procesor / memorie.

Remove Watermark Now

Exemplu - Difuzarea unei valori (1)

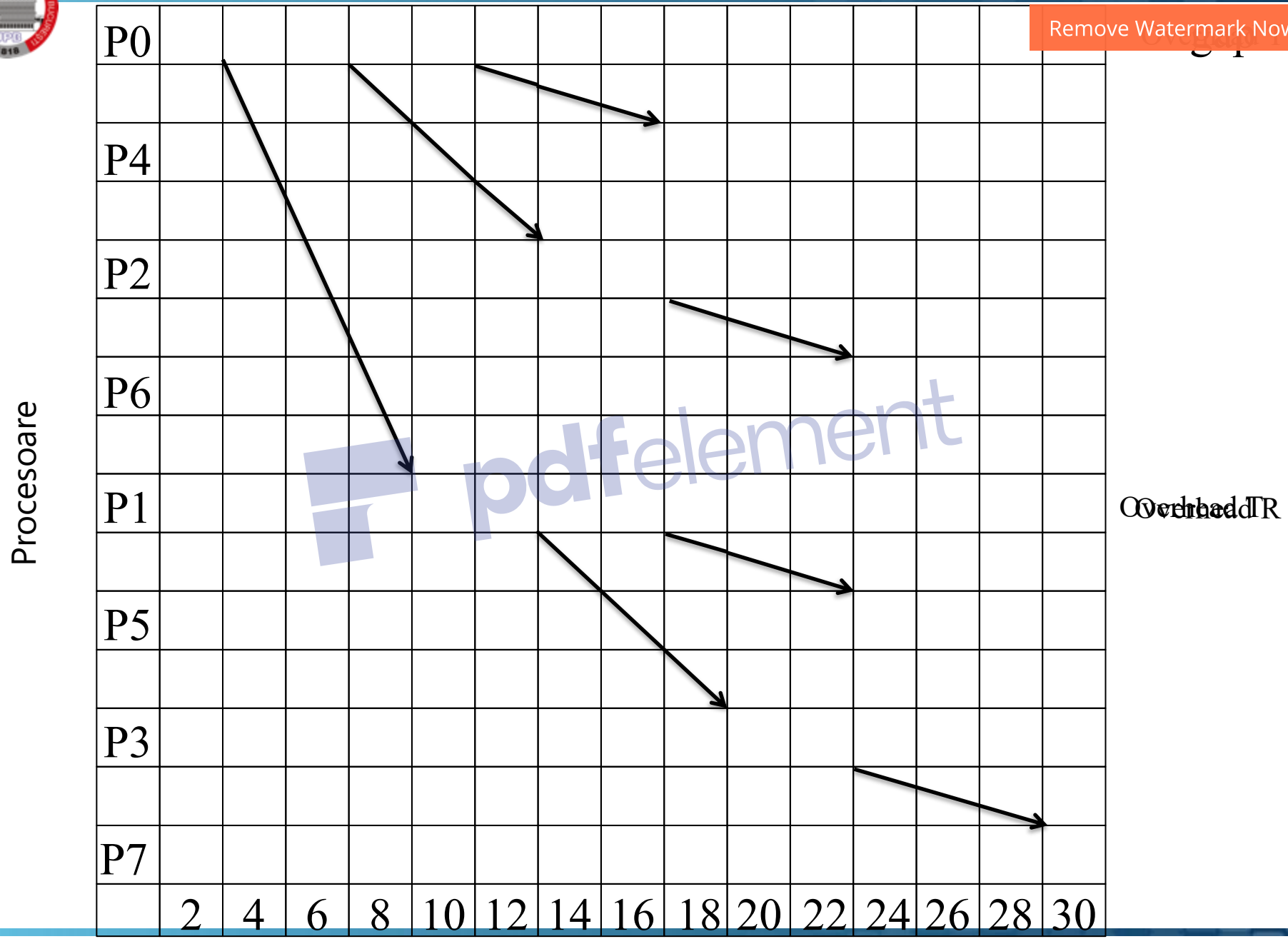
- Folosind modelului PRAM*, transmiterea are loc conform tiparului:
  - P0 → P1
  - P0, P1 → P2, P3
  - P0, P1, P2, P3 → P4, P5, P6, P7

pdfelement

- Presupunând g ≥ 0 şi luând P = 8, o =2, g = 2, L = 6, obţinem timp total de difuzare = 30 de unităţi.

* Parallel Random Access Machine – vezi curs 1.

Remove Watermark Now

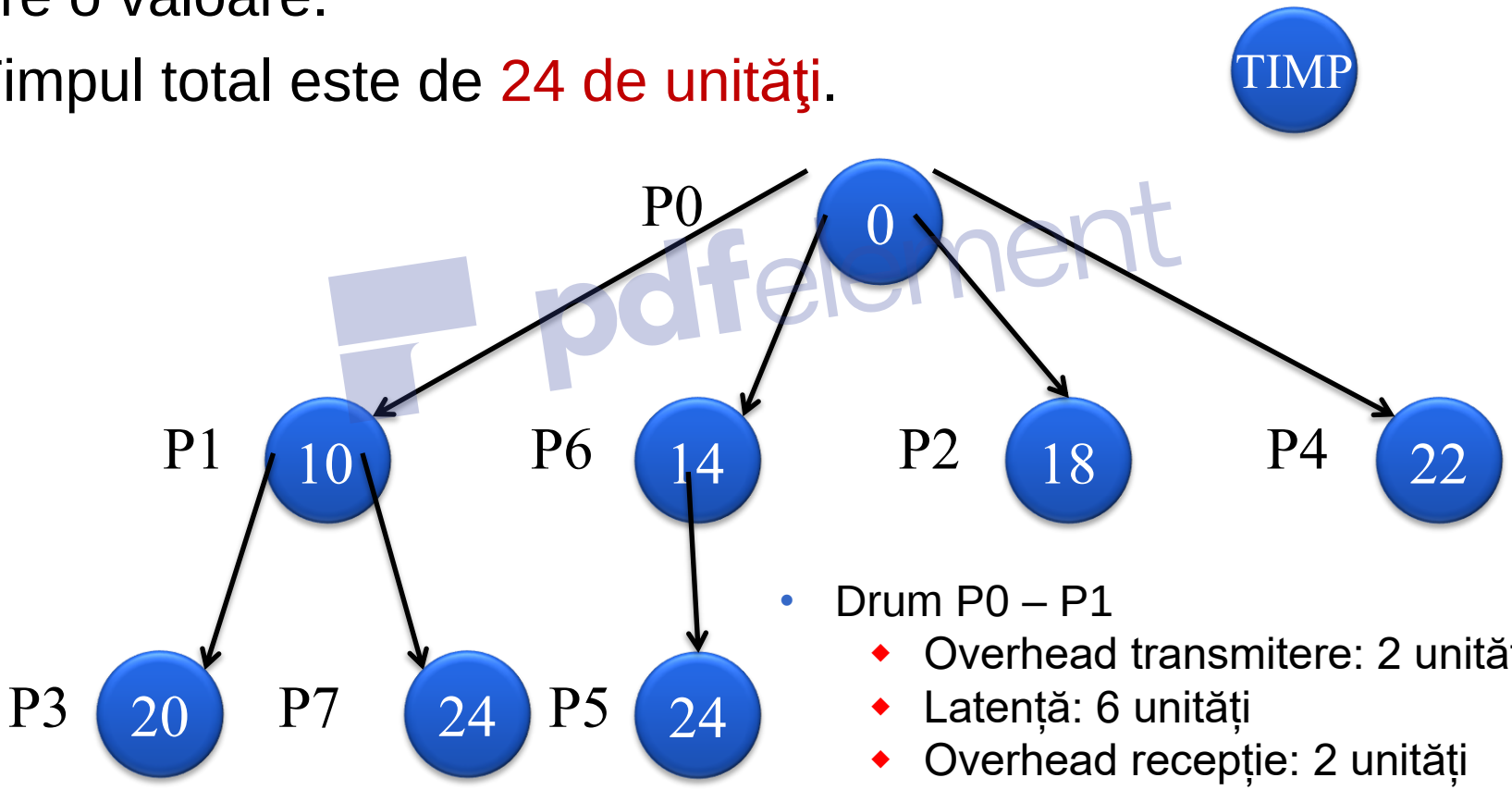


pdfelement

Remove Watermark Now

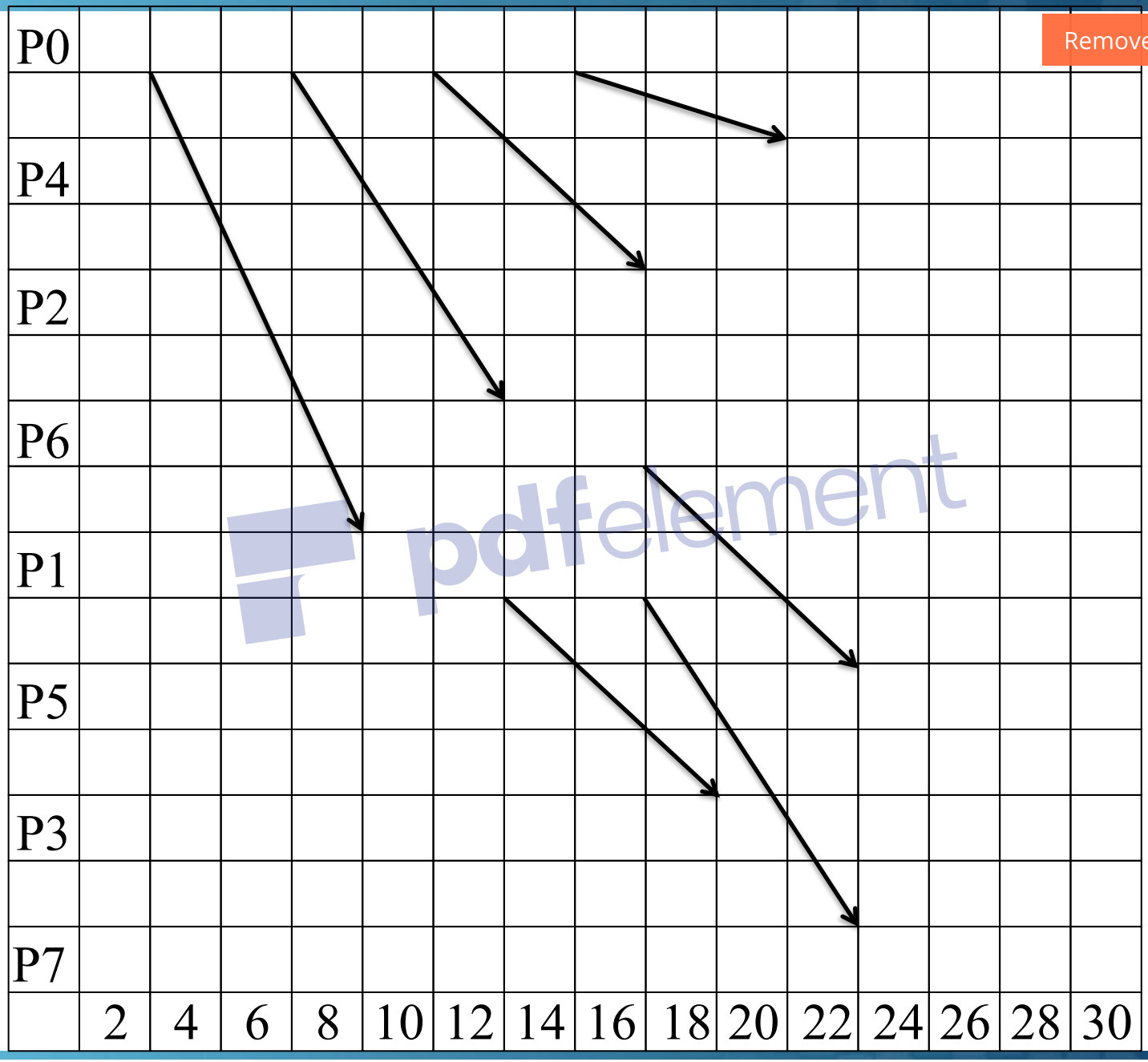
Exemplu - Difuzarea unei valori (2)

- Pentru modelul LogP, fiecare proces transmite imediat ce are o valoare.
- Timpul total este de 24 de unităţi.



- Drum P0 – P1
  - Overhead transmitere: 2 unități
  - Latență: 6 unități
  - Overhead recepție: 2 unități
  - TOTAL: 10 unități

Remove Watermark Now



pdfelement

Remove Watermark Now

Sumar

- Comunicarea prin mesaje
  - Comunicarea asincronă prin mesaje
  - Modelul Replicated Workers
  - Comunicarea sincronă prin mesaje
- Complexitatea algoritmilor distribuiți
  - Modelul Foster
  - Modelul LogP

pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Ceasuri Logice
Ordonarea evenimentelor

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

De la ceasuri fizice la... ceasuri logice

pdfelement

Remove Watermark Now

Ceasuri fizice

- Timpul este ne-ambiguu în sisteme ne-distribuite
  - există un singur ceas și (de obicei) kernelul face apeluri pentru a-i accesa valoarea
- Folosit implicit în multe aplicații
  - *Ex:* make *(pentru a determina fișierele care trebuie recompilate)*
- Sistemele distribuite introduc ambiguitatea în legătură cu timpul

pdfelement

Remove Watermark Now

Ceasuri fizice (2)

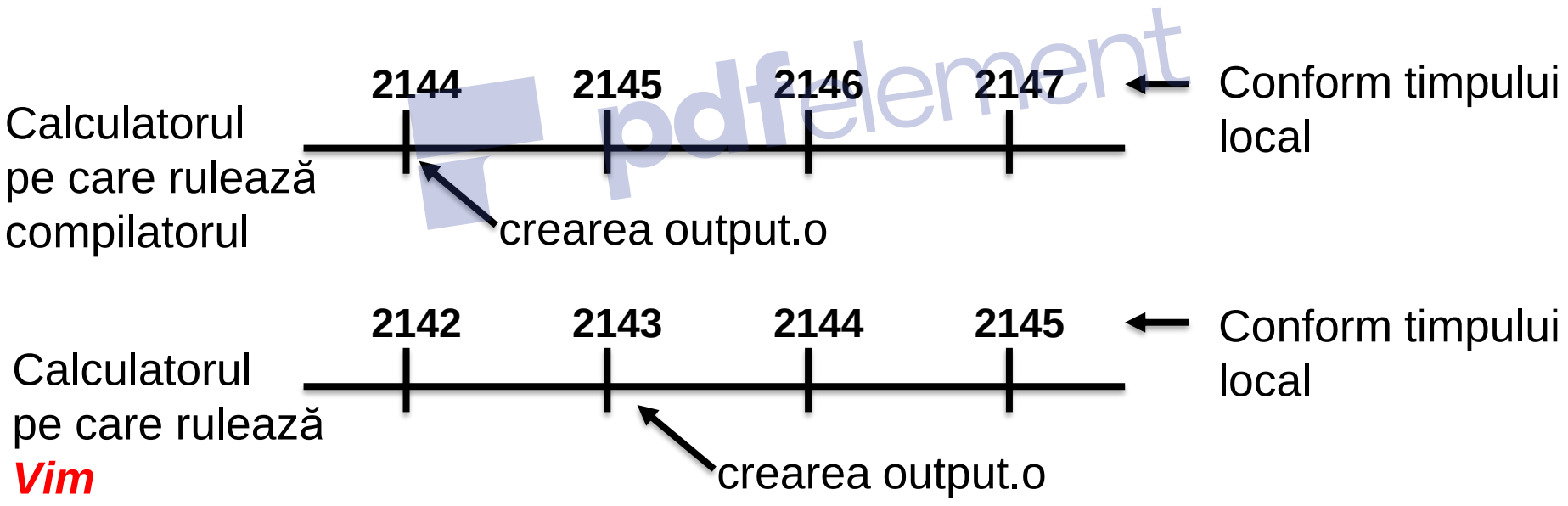
- Computerele folosesc un cristal de quartz care oscilează la o valoare bine definită și generează întreruperi (clock ticks) la intervale regulate
- Fiecare *clock tick* incrementează o valoare din memorie – kernelul convertește această valoare într-un format standardizat
  - *Ex:* nr. de milisecunde pornind de la Thu Jan 1 12:00:00 GMT 1970.
- Diferența în timp dintre calculatoare se numește clock skew.

pdfelement

Remove Watermark Now

Ceasuri fizice (3)

- Clock skew dintre cele 2 computere confuzează make-ul:

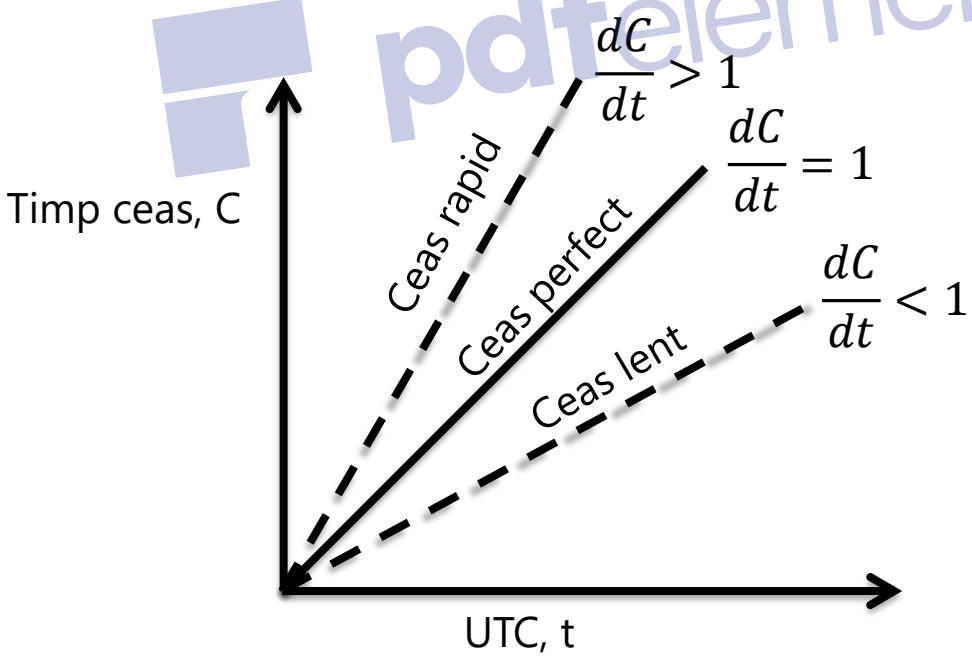


Remove Watermark Now

Sincronizarea ceasurilor fizice

- 2 tipuri:
  - doar între 2 calculatoare
  - sincronizare cu un standard *(ex: UTC)*
- Este imposibil de eliminat complet clock skew și imposibil de a asigura acuratețea perfectă a unui ceas.

pdfelement

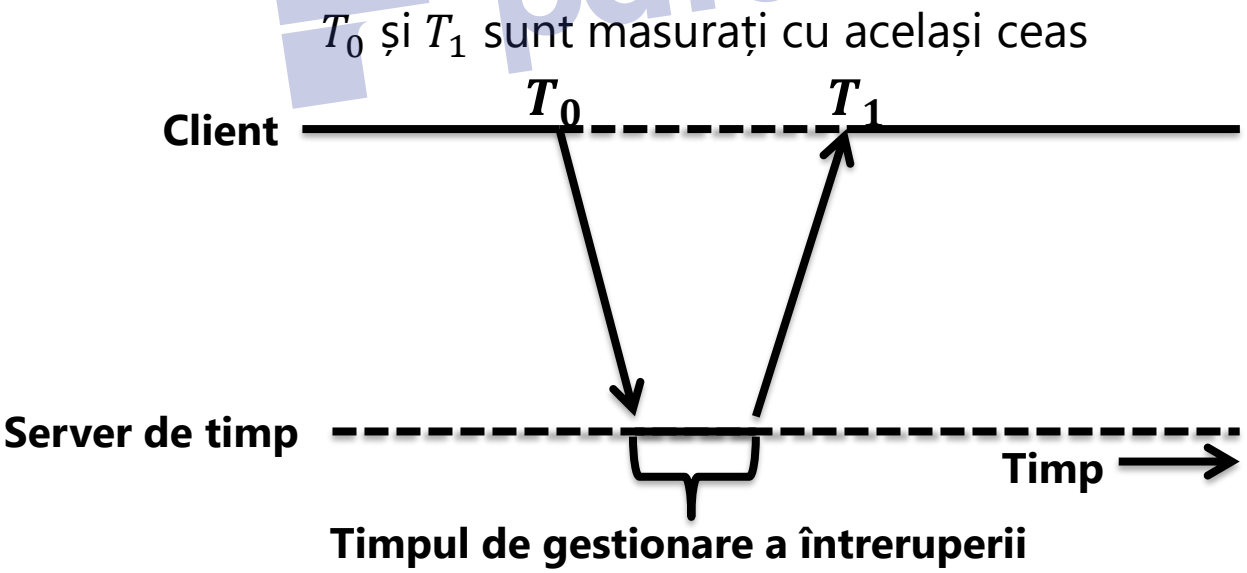


Remove Watermark Now

Sincronizarea ceasurilor fizice (2)

- Flaviu Cristian – adaptive internal clock synchronization:
  - time server, cu care se sincronizează celelalte ceasuri
  - probleme:
    - timpul nu trebuie sa curgă în sens invers
    - mesajul ajunge de la server la client într-un anumit timp

pdfelement

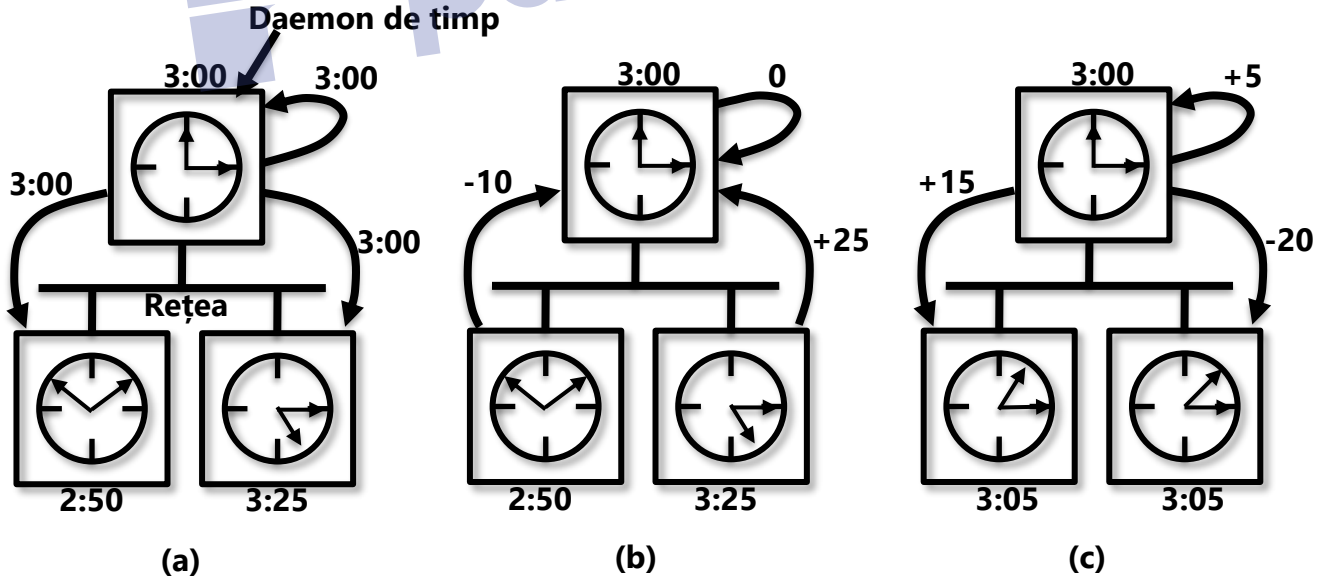


Remove Watermark Now

Sincronizarea ceasurilor fizice (3)

- Berkeley:
  - folosit în Berkeley UNIX
  - opus lui Flaviu Cristian*: un server de timp trimite mesaje periodic clienților pentru a afla timpul lor, calculează o medie și anunță clienții cum să-și actualizeze ceasurile*
  - scopul nu e de a se sincroniza cu time serverul, ci de a pune toate mașinile de acord asupra timpului (chiar dacă este greșit)

pdfelement



Remove Watermark Now

Sincronizarea ceasurilor fizice (4)

- Descentralizat: Network Time Protocol
  - Organizare ierarhică a serverelor de timp:
    - Stratul 1:
      - ceasuri de referință
        - *US Naval Observatory*
        - *Global Positioning System*
        - *Atomic Clocks*
    - Stratul 2:
      - routere, servere importante, etc.
  - Algoritmul și protocolul NTP extrem de complexe

    (RFC 1305 NTPv3 – 112 pagini , NTPv4 și mai complex)
  - Sincronizare precisă a ceasurilor: 1-50ms

pdfelement

Remove Watermark Now

Ceasuri logice (solutia Lamport)

pdfelement

Ceasuri logice și ordonarea evenimentelor

Remove Watermark Now

- Într-un algoritm distribuit, fiecare proces este caracterizat de:
  - o mulțime de stări
  - acțiuni care schimbă starea

- Eveniment *(producerea unei acțiuni)*
  - Evenimentele pot fi ordonate conform timpului fizic de producere.
  - Dificil pentru evenimente din procese diferite.

- Soluție mai simplă: ordinea relativă a evenimentelor

- Definită de relația petrecut înainte (→):
  - dacă **a** şi **b** sunt evenimente din acelaşi proces şi **a** îl precede in timp pe **b**, atunci a → b
  - când **a** reprezintă transmiterea unui mesaj de către un proces, iar **b** recepţia aceluiaşi mesaj de către un altul, atunci a → b
  - dacă a → b şi b → c, atunci a → c

pdfelement

Remove Watermark Now

Ceasuri logice și ordonarea evenimentelor (2)

- Mecanism de sincronizare bazat pe ***timp relativ***.
- ***timpul relativ*** poate să nu fie echivalent cu ***timpul real***.
  - *Exemplu:* Unix make
    - *Este important ca output.c să fie actualizat după generarea lui output.o ?*
- În aplicațiile distribuite ceea ce contează este ca procesele să *ajungă la un acord asupra ordinii de producere a* evenimentelor.
- Astfel de ***ceasuri*** se numesc ***ceasuri logice***.

pdfelement

Remove Watermark Now

Operating Systems

R. Stockton Gaines Editor

Time, Clocks, and the Ordering of Events in a Distributed System

Leslie Lamport
Massachusetts Computer Associates, Inc.

The concept of one event happening before another in a distributed system is examined, and is shown to define a partial ordering of the events. A distributed algorithm is given for synchronizing a system of logical clocks which can be used to totally order the events. The use of the total ordering is illustrated with a method for solving synchronization problems. The algorithm is then specialized for synchronizing physical clocks, and a bound is derived on how far out of synchrony the clocks can become.

Key Words and Phrases: distributed systems, computer networks, clock synchronization, multiprocess systems

CR Categories: 4.32, 5.29

Introduction

The concept of time is fundamental to our way of thinking. It is derived from the more basic concept of the order in which events occur. We say that something happened at 3:15 if it occurred *after* our clock read 3:15 and *before* it read 3:16. The concept of the temporal

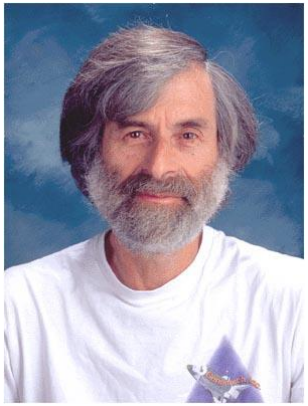
A distributed system consists of a collection of distinct processes which are spatially separated, and which communicate with one another by exchanging messages. A network of interconnected computers, such as the ARPA net, is a distributed system. A single computer can also be viewed as a distributed system in which the central control unit, the memory units, and the input-output channels are separate processes. A system is distributed if the message transmission delay is not negligible compared to the time between events in a single process.

We will concern ourselves primarily with systems of spatially separated computers. However, many of our remarks will apply more generally. In particular, a multiprocessing system on a single computer involves problems similar to those of a distributed system because of the unpredictable order in which certain events can occur.

In a distributed system, it is sometimes impossible to say that one of two events occurred first. The relation "happened before" is therefore only a partial ordering of the events in the system. We have found that problems often arise because people are not fully aware of this fact and its implications.

In this paper, we discuss the partial ordering defined by the "happened before" relation, and give a distributed algorithm for extending it to a consistent total ordering of all the events. This algorithm can provide a useful mechanism for implementing a distributed system. We illustrate its use with a simple method for solving synchronization problems. Unexpected, anomalous behavior can occur if the ordering obtained by this algorithm differs from that perceived by the user. This can be avoided by introducing real, physical clocks. We describe a simple method for synchronizing these clocks, and derive an upper bound on how far out of synchrony they can drift.

The Partial Ordering



Lamport, Leslie. "Time, clocks, and the ordering of events in a distributed system." *Communications of the ACM* 21.7 (1978): 558-565.

Remove Watermark Now

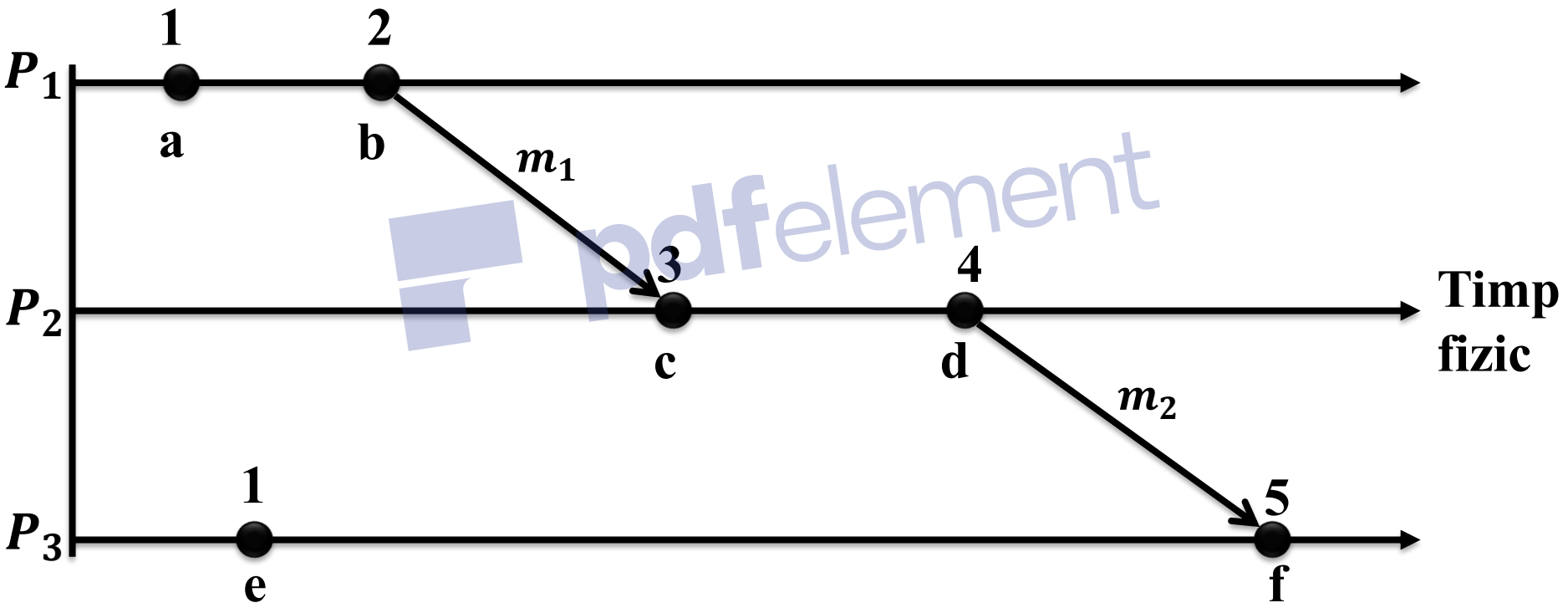
Ceasuri logice și ordonarea evenimentelor (2)

- Ceas logic – întreg incrementat la producerea unui eveniment
- Algoritmul lui Lamport folosește relația → pentru a face ordonarea parțială a evenimentelor:
  - pentru fiecare proces o variabilă cl inițial 0
  - pentru fiecare mesaj un câmp tt
- Reguli:
  - la producerea unui eveniment intern:
    - cl este incrementat
    - valoarea ceasului logic este asociată evenimentului ca amprentă de timp
  - la transmiterea unui mesaj :
    - incrementează cl al procesului transmitator cu 1
    - actualizează tt = cl
  - la primirea unui mesaj cu amprenta de timp tt:
    - actualizează cl = maxim(cl, tt) + 1
- Ordonare parțială: $a \rightarrow b \Rightarrow cl(a) < cl(b)$

pdfelement

Remove Watermark Now

Exemplu



pdfelement

Remove Watermark Now

Aplicatie: semafoare distribuite

pdfelement

Remove Watermark Now

Semafoare distribuite

- Folosește primitive P și V
  - Când un proces execută o operaţie P sau V el difuzează mesaje m(id, tag, ts) celorlalte procese:

    broadcast ch(m) trimite m pe fiecare din ch[1:n]

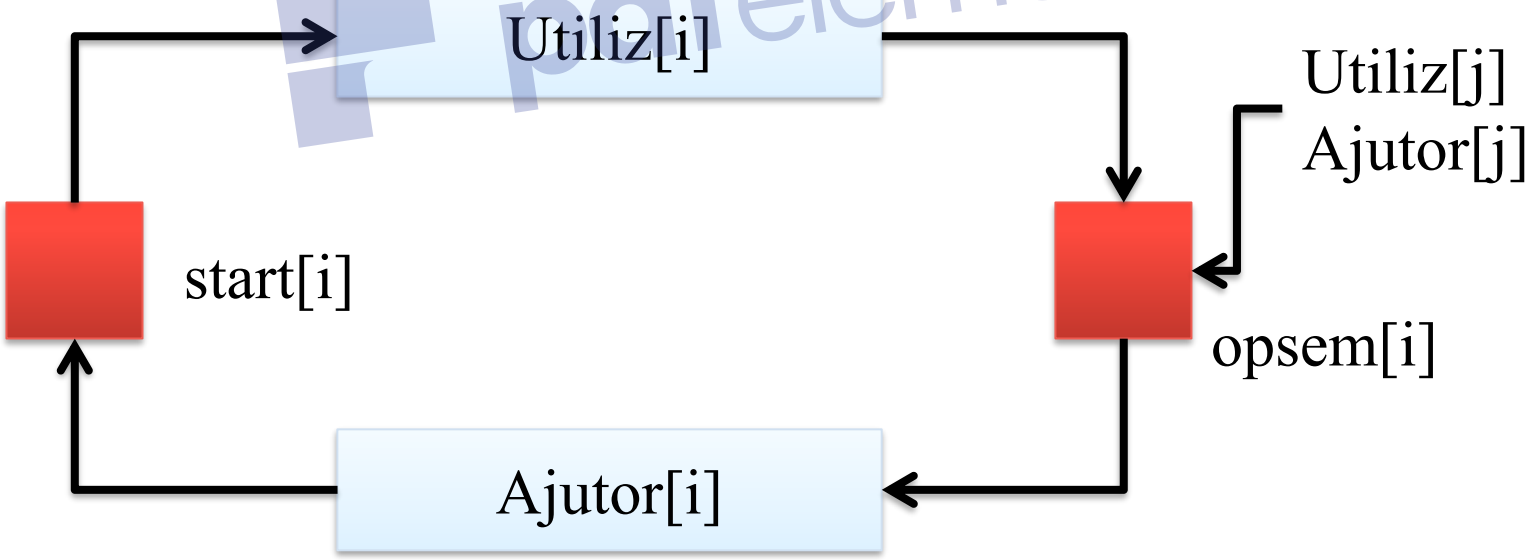
    mesajul m are aceeasi amprenta de timp pe toate canalele
  - Analizează răspunsurile pentru a determina continuarea execuţiei
- Fiecare proces:
  - Stochează o coadă de mesaje mq și un ceas logic cl
  - La recepția unui mesaj P sau V îl stochează în mq, sortat crescător după ts
  - Transmite prin broadcast ACK *(folosit pentru a actualiza prefixele stabile din mq)*
  - Stochează o variabila s *(semaforul)*:
    - V – s este incrementat și mesajul șters
    - P – dacă $s > 0$, s decrementat și mesajul șters
      - mesajele P sunt procesate în ordinea în care apar în prefixul stabil deci fiecare proces ia aceeași decizie despre ordinea terminării operațiilor P

pdfelement

Remove Watermark Now

Aplicație: semafoare distribuite (2)

- Implementarea semafoarelor:
  - procesele Utiliz(i) inițiază operațiile P sau V
  - procesele Ajutor(i) implementează operațiile P și V



pdfelement

Remove Watermark Now

Aplicație: semafoare distribuite (3)

```
enum fel(V, P, ack);
chan opsem[1:n](int transm, fel op, int timp);
chan start[1:n](int timp);

process Utiliz[i= 1 to n] {
  int cl = 0;                          /* ceas logic */
  int ts;
  ...
  cl = cl + 1;
  broadcast opsem(i, V, cl);           /* operația V */
  ...
  cl = cl + 1;
  broadcast opsem(i, P, cl);           /* operația P */
  receive start[i](ts);
  cl = max(cl, ts)+1;
  ...
}
```

pdfelement

Aplicație: semafoare distribuite (4)

Remove Watermark Now

```
process Ajutor[i = 1 to n]{
    typedef struct{int transm, fel k, int ts) qelem;
    qelem qm[lmax]; /*coada ordonata dupa timestamp tm*/
    int cl = 0;
    int sem = valoare_initială;
    int transm; fel k; int ts;
    while (true) {
        receive opsem[i](transm, k, ts);
        cl = max(cl, ts) + 1;
        if (k == P or k == V) {
            inserează (transm, k, ts) în locul corespunzător în qm;
            cl = cl + 1; broadcast opsem(i, ack, cl); }
        else if (k == ack) {
            înregistrează transmiterea unui ack;
            for [mesajele V complet confirmate] {
                scoate mesaj din qm;
                sem = sem + 1; }
            for [mesajele P complet confirmate st sem > 0] {
                scoate mesaj (transm, k, ts) din qm; // atentie la coliziuni
                sem = sem - 1;
                if (tansm == i) {cl = cl + 1; send start[i](cl);} }
        }
    }
}
```

pdfelement

Remove Watermark Now

Ceasuri logice vectoriale

pdfelement

Remove Watermark Now

Ceasuri logice vectoriale

Cu soluţia Lamport:

e precede f ⇒ amprenta_logică (e) < amprenta_logică (f)

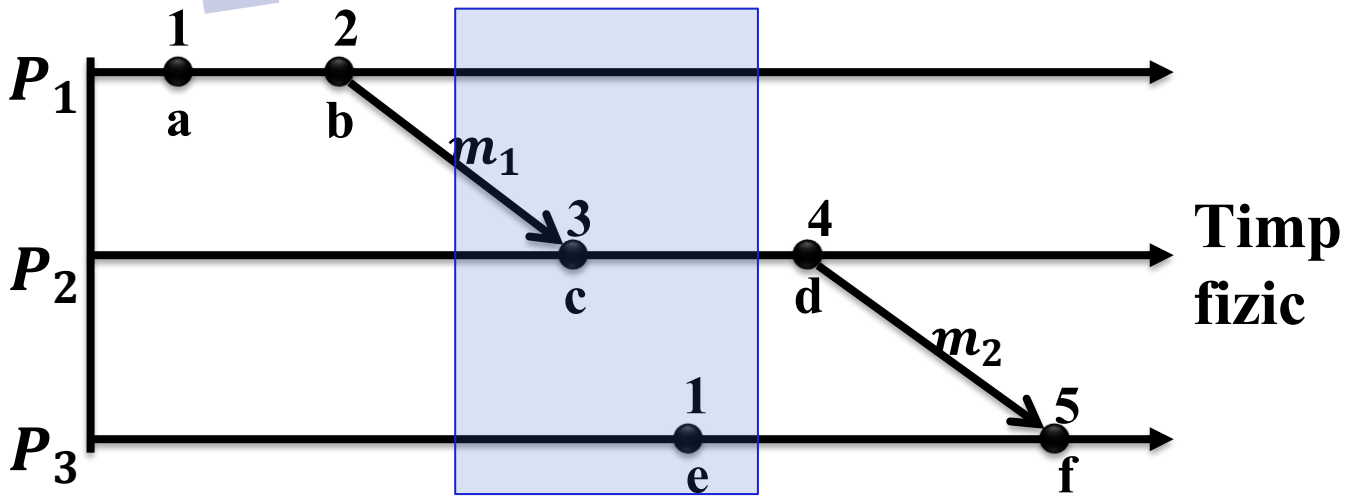
Dar

amprenta_logică (e) < amprenta_logică (f) ⇏ e precede f

Ex: se poate spune ca **e** precede **c** ?

Soluția: ceasuri logice vectoriale.

pdfelement



Ceasuri logice vectoriale (2)

Remove Watermark Now

- Fiecare P_i are asociat un tablou $V_i[1..n]$ în care:
  - $V_i[i]$ este numărul de evenimente produse în procesul P_i
  - $V_i[j]$ este numărul de evenimente despre care P_i ştie *(a aflat)* că au avut loc la P_j.

- Procesul P_i actualizează V_i la fiecare **eveniment** din P_i
  - e.g. pentru procesorul 3, (1,2,1,3) → (1,2,2,3)

- Când P_i **transmite** mesajul *m (eveniment send m):*
  - P_i incrementează $V_i[i]$
  - P_i adaugă V_i la *m* ca vector de amprente de timp curent $\boldsymbol{vt_m}$

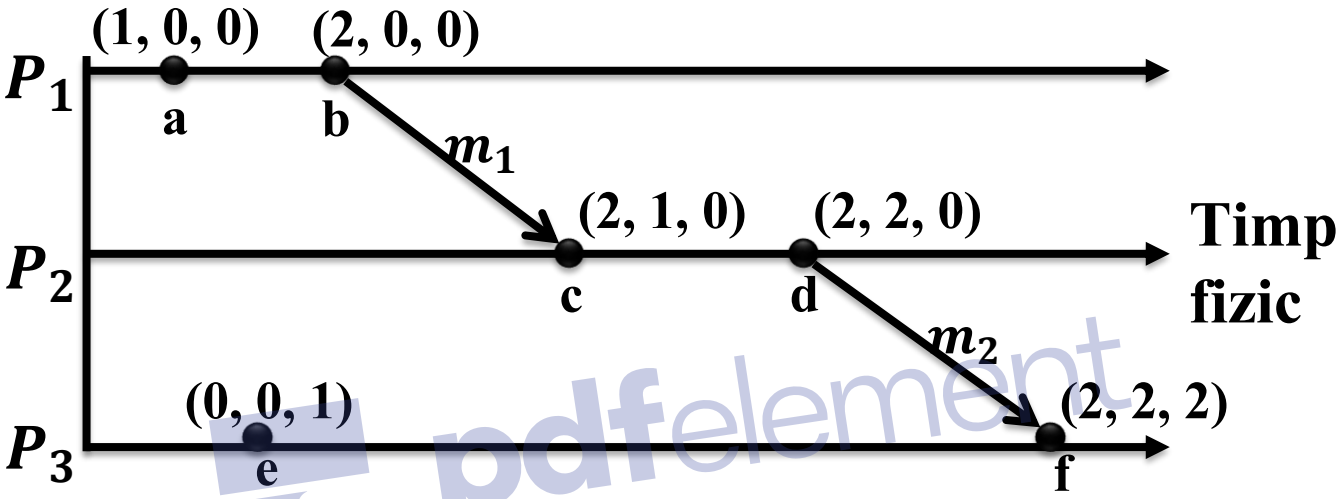
- Când P_j **primeşte** *m* și $\boldsymbol{vt_m}$ *(eveniment receive m):*
  - P_j ajustează: $V_j[k] = max\{V_j[k], vt_m[k]\}$ pentru fiecare k

    e.g., P_2 primește un mesaj cu timpul (3,2,4) iar timpul curent al lui P_2 este (3,4,3), atunci P_2 ajusteaza timpul la (3,4,4)
  - P_i incrementează $V_j[i]$cu 1

pdfelement

Remove Watermark Now

Ceasuri logice vectoriale (3)



Aplicarea regulilor ceasurilor logice vectoriale

pdfelement

Reguli:

- $VT_1 = VT_2 \Leftrightarrow VT_1[i] = VT_2[i]$, pentru $i = 1, \dots, N$
- $VT_1 \leq VT_2 \Leftrightarrow VT_1[i] \leq VT_2[i]$, pentru $i = 1, \dots, N$
- $VT_1 < VT_2 \Leftrightarrow VT_1[i] \leq VT_2[i]$, și $VT_1 <> VT_2$ *(de exemplu (1,2,2) < (1,3,2))*

Fie vt(a) şi vt(b) vectorii de amprente de timp asociaţi ev. a şi b. Atunci:

vt(a) < vt(b) $\Rightarrow$ **evenimentul a precede cauzal b**

vt(a) $\not<$ **vt(b) and vt(a)** $\not>$ **vt(b) and vt(a)** $\neq$ **vt(b)** $\Leftrightarrow$ **evenimentele a şi b sunt concurente**

Remove Watermark Now

Aplicație: Ordonare Cauzală Multicast

pdfelement

Remove Watermark Now

Ordonare Cauzală Multicast

- Procesele unei colecţii P comunică între ele doar prin mesaje cu difuzare
- Se cere ca mesajele să respecte **dependenţa cauzală**

$$m \rightarrow m' \Rightarrow livrarep(m) \rightarrow livrarep(m')$$

Protocolul *(vectori de timp)*:

- fiecare proces $P_i = (i = 1..n)$ are asociat un vector $V_i[1..n]$, cu toate elementele iniţial 0
- se numara doar operatiile de transmitere de mesaje
- $V_i[i]$, este nr ev. transmitere de mesaje produse de P_i;
- $V_i[j]$ este nr ev. transmitere de mesaje despre care $\boldsymbol{P_i}$ ştie (a aflat) că au avut loc la P_j.
- Procesul P_i actualizează V_i la fiecare eveniment de trimitere sau recepţie de mesaj din P_i.

pdfelement

Remove Watermark Now

Ordonare Cauzală Multicast (2)

- Când P_S **transmite** mesajul *m*:
  - $\boldsymbol{P_S}$ **incrementează** $\boldsymbol{V_S[s]}$
  - $\boldsymbol{P_S}$ **adaugă** $\boldsymbol{V_s}$ **la** ***m*** **ca vector de amprente de timp curent** $\boldsymbol{vt_m}$
    - **Obs: Pentru ordonare cauzală, incrementarea lui** $\boldsymbol{V_s[s]}$ **se face doar la transmiterea de mesaje de către s**
    - vt_m spune receptorului câte evenimente (din alte procese) au precedat **m** şi ar putea influenţa cauzal pe **m**.

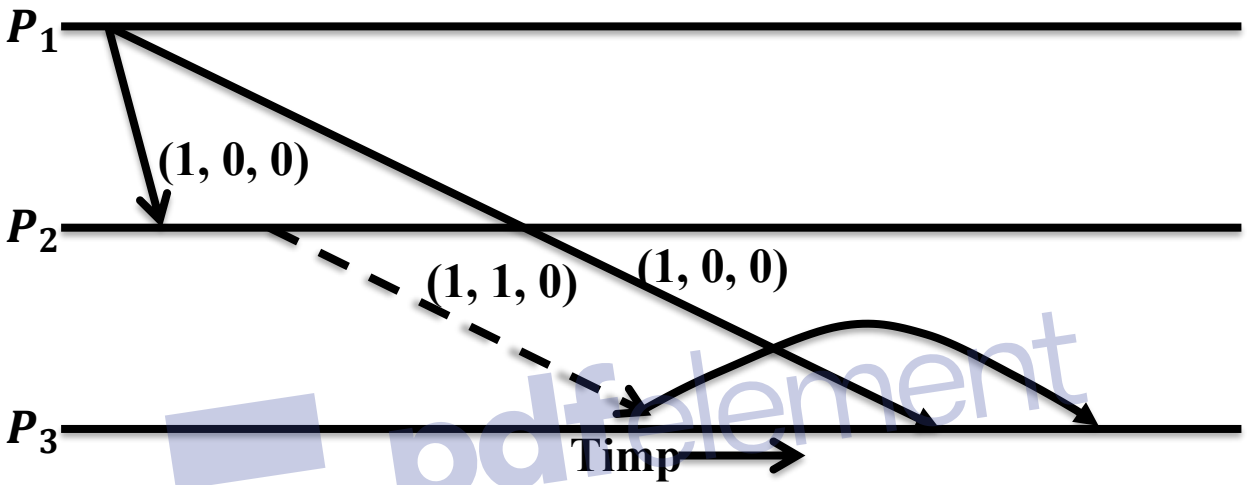
- Când P_d **primeşte** mesajul *m* împreună cu vt_m, mesajul este păstrat într-o coadă de **întârziere** şi este **livrat** doar dacă:
  - $\boldsymbol{vt_s[s] = V_d[s] + 1}$ (acesta este următorul timestamp pe care **d** îl așteaptă de la **s**)
  - $\boldsymbol{vt_m[k] \leq V_d[k]\ pentru\ k <> s}$ (**d** a văzut toate mesajele ce au fost văzute de **s** la momentul când a trimis mesajul **m**)

- Când mesajul este livrat, V_d este actualizat conform regulilor vectorilor de timp:
  $\boldsymbol{V_d[k] = max\{V_d[k],\ vt_m[k]\}}$ **pentru fiecare** $\boldsymbol{k = 1, n}$**.**

pdfelement

Remove Watermark Now

Ordonare Cauzală Multicast (3)



Alte acţiuni ale protocolului la livrarea mesajelor:

- dacă $vt_m[k] > V_d[k]$ pentru un oarecare k atunci se întârzie *m*

P_S a primit mesaje de care mesajul curent poate fi cauzal dependent, dar pe care P_d încă nu le-a primit;

- dacă $vt_m[s] > V_d[s] + 1$ atunci se întârzie *m*

mai sunt mesaje de la P_S pe care P_s nu le-a primit

(asigură ordinea FIFO pentru canale nonFIFO)

- dacă $vt_m[s] < V_d[s]$ atunci rejectează *m*

m este un duplicat al unui mesaj primit anterior

Remove Watermark Now

Sumar

- Ceasuri fizice
- Ceasuri logice și ordonarea evenimentelor
- Semafoare distribuite
- Ceasuri logice vectoriale
- Ordonare cauzală multicast

pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Algoritmi undă.

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Algoritmi undă

- În dezvoltarea algoritmilor distribuiți apar frecvent anumite tipuri de **probleme generale** pentru **rețele de procese**.

- Rezolvate prin transmitere de mesaje după o **schemă predefinită**, **dependentă de topologie** care asigură participarea **tuturor proceselor**.

- Justifică tratarea lor izolată de alți algoritmi în care aceste scheme pot fi folosite.

pdfelement

Notiuni preliminare - configuratii

Remove Watermark Now

- Un **program distribuit** consta dintr-o **colectie de procese** secventiale comunicante
- **Configuratie** = Ansamblul **starilor** tuturor proceselor la un moment dat
- O **operatie** modifica starea unui proces, implicit configuratia
- O executie a programului poate fi **modelata** printr-o secventa de **configuratii**.

- Ex.1 Program distribuit = colectie a doua procese, initial a si b au valoarea
- process P1{ int a =0; process P2{ int b =0;
  a = a+1;} b = b+1;}
- configuratia = ansamblul valorilor celor doua variabile modificate de procese configuratia initiala este <0,0>
- daca operatia primului proces se executa mai intai, configuratia devine <1,0>
- O executie posibila = secventa de configuratii: (<0,0> , <1,0> , <1,1>)

pdfelement

Sisteme de tranziţii

Remove Watermark Now

- Un program distribuit poate fi modelat ca un **sistem de tranziții**:
  - Mulțimea tuturor stărilor (**configurațiilor**) posibile ale sistemului
  - Tranzițiile pe care sistemul le poate face între stări
  - Stările din care sistemul poate porni (inițiale)

- Formal, un **sistem de tranziții** este un triplet $S = (C, \rightarrow, I)$
  C este o mulțime de configurații
  $\rightarrow$ este relația de tranziție binară pe C
  I este setul configurațiilor inițiale (o submulțime a lui C)

pdfelement

- O **execuție** a lui S este o secvență maximală
  $E = (\gamma_0, \gamma_1, \gamma_2, \ldots)$, unde $\gamma_0 \in I$ și $\gamma_i \rightarrow \gamma_{i+1}$, $pentru\ i \geq 0$.

- O configurație **terminală** γ nu are succesor:
  $\nexists\ \delta$ astfel încât $\gamma \rightarrow \delta$.

Remove Watermark Now

Sisteme de tranziţii (2)

- O secvență E este **maximală** dacă:
  - Este infinită sau
  - Sfârșește într-o configurație terminală.
- Configurația δ este **tangibilă** din γ dacă există o secvență $\gamma_0, \gamma_1, \gamma_2, \dots, \gamma_k$ a.î.:

$$\gamma = \gamma_0$$
$$\delta = \gamma_k$$
$$\gamma_i \rightarrow \gamma_{i+1}, pentru\ i = \overline{0, k-1}$$

pdfelement

- Un **sistem distribuit** constă dintr-o colecție de procese și un subsistem de comunicații.
- **Convenții:**
  - pentru sistem: **tranziții** și **configurații**
  - pentru proces: **evenimente (interne / send,receive)** și **stări**
- Unei **execuții** E îi corespunde o **secvență de evenimente** din diferite procese.

Remove Watermark Now

Sisteme de tranziţii (3)

- Pentru o execuție E, **relația de ordine cauzală** $<$ este cea mai slabă relație care satisface:
  - dacă $\boldsymbol{a}$ și $\boldsymbol{b}$ sunt două evenimente diferite ale aceluiași proces și $\boldsymbol{a}$ se produce înaintea lui $\boldsymbol{b}$ atunci $\boldsymbol{a} < \boldsymbol{b}$
  - dacă $\boldsymbol{a}$ este un eveniment send, iar $\boldsymbol{b}$ evenimentul receive corespunzător atunci $\boldsymbol{a} < \boldsymbol{b}$
  - dacă $\boldsymbol{a} < \boldsymbol{b}$ și $\boldsymbol{b} < \boldsymbol{c}$ atunci $\boldsymbol{a} < \boldsymbol{c}$ ($<$ este tranzitivă).
- Dacă $\boldsymbol{a} < \boldsymbol{b}$ și $\boldsymbol{b} < \boldsymbol{a}$ atunci a și b sunt **concurente**.

- O execuție E este **echivalentă** cu F ($E \sim F$) dacă:
  - au aceeași colecție de evenimente (ordinea diferă)
  - evenimentele respectă aceeași relație de ordine cauzală
  - ultima configurație a lui E coincide cu ultima configurație a lui F
- **Obs**: două execuții echivalente pot să nu aibă aceleași configurații.
- Un **calcul** (computation) al unui algoritm distribuit este o clasă de echivalență (sub relația $\sim$) a execuțiilor algoritmului.

pdfelement

Remove Watermark Now

Algoritmi undă (Wave algorithms)

- **Schema** de transmitere de mesaje:
  - dependentă de topologie
  - asigură participarea tuturor proceselor
- Algoritmii undă implementează astfel de scheme pentru:
  - difuzarea informației
  - realizarea unei sincronizări globale între procese
  - declanșarea unu eveniment în fiecare proces
  - calculul unei funcții în care fiecare proces participă cu o parte a datelor de intrare

pdfelement

- Proprietăți:
  - **terminare** – fiecare *calcul* este finit
  - **decizie** – fiecare *calcul* conține cel puțin un eveniment de decizie (*decide*)
  - **dependență** – în fiecare *calcul*, fiecare *decide* este precedat cauzal de un eveniment în fiecare proces

Remove Watermark Now

Definiții (1)

Calcul (computation) ⇔ **undă** (wave)

Categorii de procese:

Inițiatori (starters) – primul eveniment este unul intern sau un send

Neinițiatori (followers) – primul eveniment este un receive

Clasificare:

- centralizare
  - algoritmi centralizați – un inițiator
  - algoritmi descentralizați – set arbitrar de inițiatori

- topologie
  - inel, arbore, clică etc.
  - fixă (nu se produc modificări topologice)
  - nedirecționată (canale bidirecționale); excepțiile menționate explicit
  - conectată (există o cale între oricare două procese)

pdfelement

Remove Watermark Now

Definiții (2)

- cunoștințe inițiale – exemple:
  - identitatea proprie (nume)
  - identitățile vecinilor
  - **setul direcției (ordinea vecinilor)**

- numărul de decizii (regulă = cel mult o decizie în fiecare proces)
  - un singur proces decide
  - toate decid
  - unele decid

- complexitate
  - număr de mesaje schimbate
  - număr de biți interschimbați
  - timpul necesar pentru un calcul

pdfelement

Remove Watermark Now

Noțiuni preliminare
Sensul legăturilor

Complexitatea comunicării într-un algoritm distribuit depinde de topologie, dar și de:

- cunoașterea topologiei la nivelul fiecărui nod (topological awareness)

pdfelement

- setul de direcții (sense of direction)
  - muchiile incidente unui nod sunt etichetate cu direcția spre care conduc în rețea
  - setul de etichete este același pentru fiecare nod
  - mărimea setului depinde de topologie (2 pt. inel, 4 pt. tor, etc)
  - trebuie respectată o **condiție suplimentară de consistență**

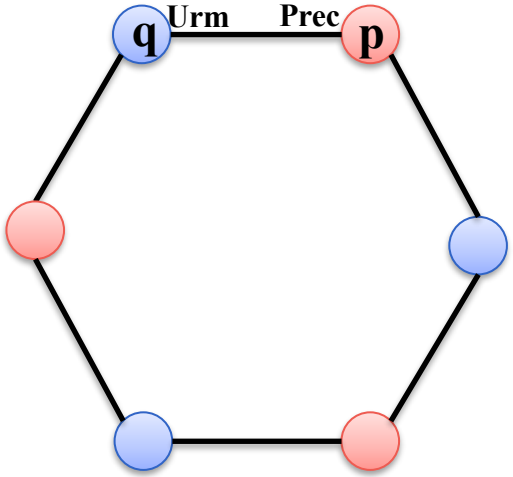
Remove Watermark Now

Noțiuni preliminare
Sensul legăturilor (2)

Inel:

- 2 direcții: *Prec* (precedent) și *Urm* (următor)
- condiția de consistență: **precedentul lui p este q ⇔ următorul lui q este p**

pdfelement



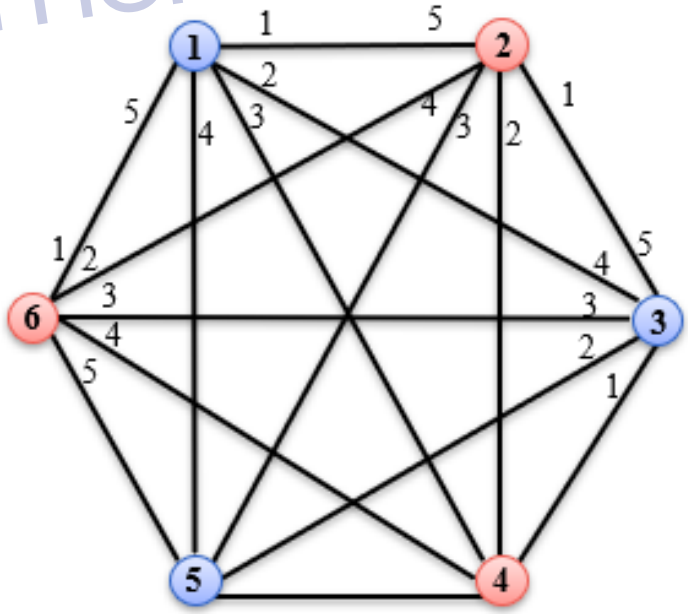
Remove Watermark Now

Noțiuni preliminare
Sensul legăturilor (3)

Clică:

- N noduri de grad $N-1$
- set $direcții = \{1, 2, \ldots, N-1\}$
- condiția de consistență: **direcția de la nodul i la j are sensul $(j-i)\ mod\ N$**

pdfelement



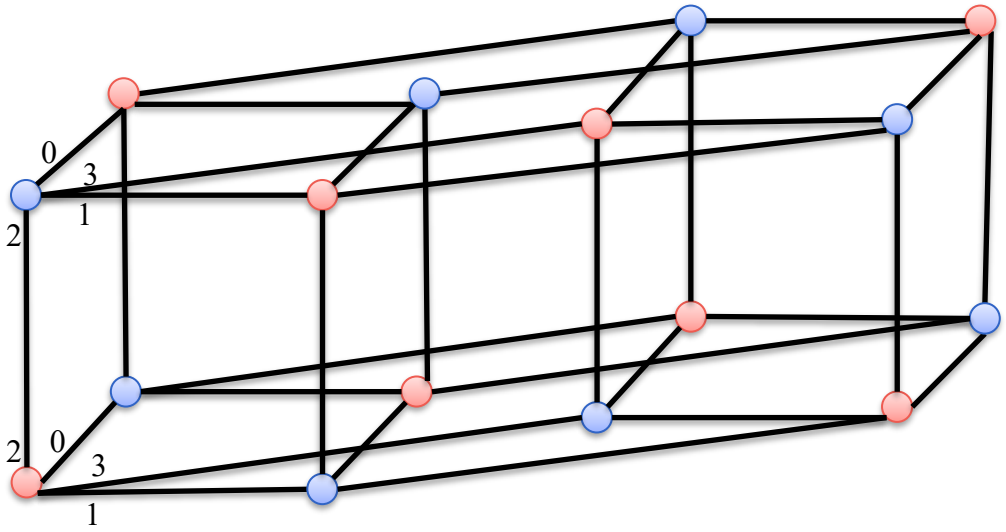
Remove Watermark Now

Noțiuni preliminare
Sensul legăturilor (4)

Hipercub:

- pentru un hipercub n dimensional
- set $direcții = \{0, \dots n-1\}$
- condiția de consistență: **două noduri $(b_0, \dots, b_{n-1})$, $(c_0, \dots, c_{n-1})$ legate prin muchie cu eticheta i diferă doar în bitul i**

pdfelement



Remove Watermark Now

Notație transmitere mesaje

1. transmitere prin adresare directă

send ch[q] (mesaj)

- unde **q** este identitatea unică, globală a canalului către receptor

2. transmitere prin adresare indirectă

send ch[direcție] (mesaj)

- unde **direcție** este locală procesului care execută operația send
- identifică unul din canalele pe care procesul poate transmite

pdfelement

Remove Watermark Now

Algoritm inel

pdfelement

Remove Watermark Now

Algoritmul inel

- fiecare proces are un vecin dedicat, Urm
- transmiterea folosește **adresarea prin direcție** (Urm, Prec)
- toate canalele selectate prin Urm formează un ciclu Hamiltonian
- algoritmul este **centralizat**:
  - inițiatorul trimite un token (jeton) care este pasat de fiecare proces de-a lungul ciclului până ajunge înapoi la inițiator;
  - inițiatorul ia apoi decizia

pdfelement

Remove Watermark Now

Algoritmul inel

```
chan token[1..n](tok_type tok);
```

```
/* inițiator */
process P[I]{
        tok_type tok;
        send  token[Urm](tok);
        receive token[I](tok);
        decide
}
```

```
/* neinițiatori */
process P[k=1 to n, k<>I] {
        tok_type tok;
        receive token[k](tok);
        send token[Urm](tok)
}
```

Număr de mesaje = n, Timp = n

pdfelement

Remove Watermark Now

Algoritm arbore

pdfelement

Remove Watermark Now

Algoritmul arbore

- Se aplică:
  - unei topologii arbore
  - unei topologii arbitrare în care se cunoaște un arbore de acoperire
- Fiecare nod cunoaște identitatea proprie și identitățile vecinilor
- Mulțimea tuturor identităților este **Ids**
- Pentru fiecare proces, se folosesc variabilele locale
  - Vecini – mulțimea identităților vecinilor (**q** = identitatea procesului q)
  - rec[q] – true dacă procesul a primit un mesaj de la vecinul q
- Inițiatorii sunt toate nodurile frunză
- **Algoritm:**
  - fiecare proces trimite exact un mesaj
  - când un proces a primit un mesaj pe fiecare canal incident mai puțin unul (condiție îndeplinită inițial de frunze) el trimite un mesaj pe canalul rămas
  - când un proces a primit câte un mesaj pe toate canalele sale atunci **decide**

pdfelement

Remove Watermark Now

Algoritmul arbore (2)

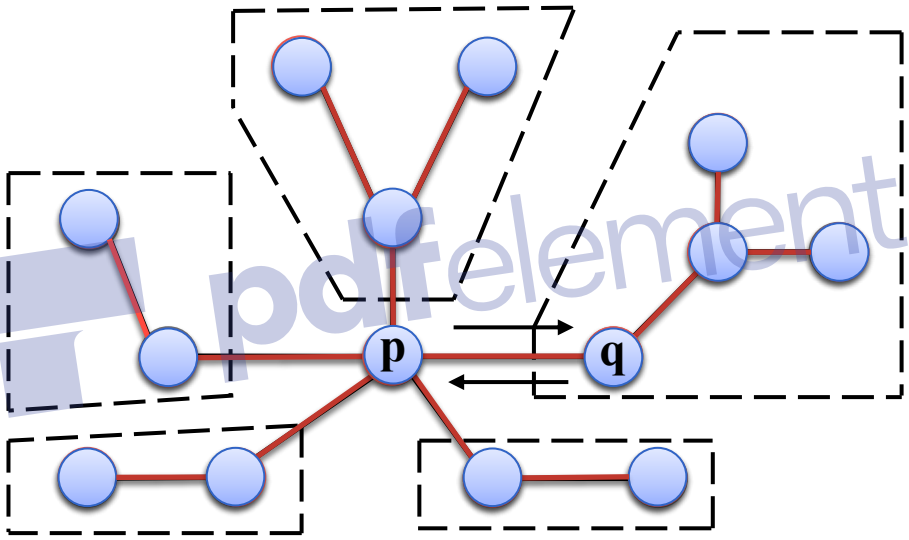
```
chan ch[1:n](int id, tok_type tok);
   /* fiecare proces are un canal propriu */
process Proc[p = 1 to n]{
   bool Vecini[1:n] = vecinii_lui_p;
   bool rec[1:n] = ([|n|] * false);
   int  r = numar_vecini_p;
   tok_type tok; int id, q0;
   while (r > 1){ receive ch[p](id, tok);
        rec[id] = true; r = r - 1 }
   /* de la un singur vecin, q0, nu s-a primit mesaj */
   afla q0 : Vecini[q0] and NOT rec[q0];
   send ch[q0](p, tok);
   receive ch[p](q0, tok); rec[q0] = true;
   decide
   /* informeaza celelalte procese despre decizie */
   /* for [q = 1 to n st Vecini[q] and q<>q0] send ch[q](p, tok); */
}
```

Număr de mesaje = N (egal cu nr. procese), Timp = O(D)

pdfelement

Remove Watermark Now

Algoritmul arbore (3)
exemplu de execuție



(b) (a) și după recepție, decid
după recepție, decid
transmit

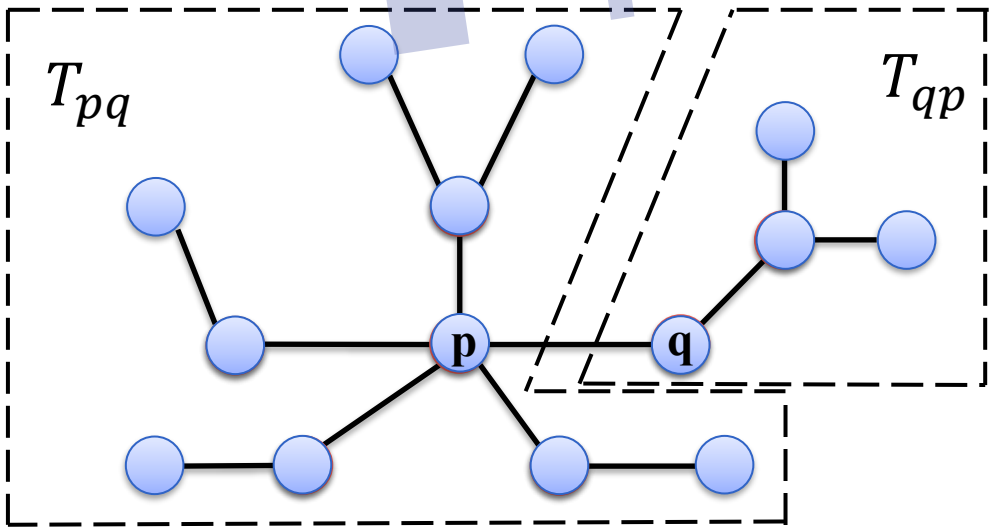
Remove Watermark Now

Algoritmul arbore (4)

Teoremă: Algoritmul "arbore" este un algoritm undă.

1. calcul finit
   - algoritmul foloseşte cel mult N mesaje → algoritmul atinge o configuraţie terminală γ după un număr finit de paşi.
2. în γ, cel puţin un proces a executat un eveniment "*decide*"
3. "*decide*" este precedat de un eveniment în fiecare proces

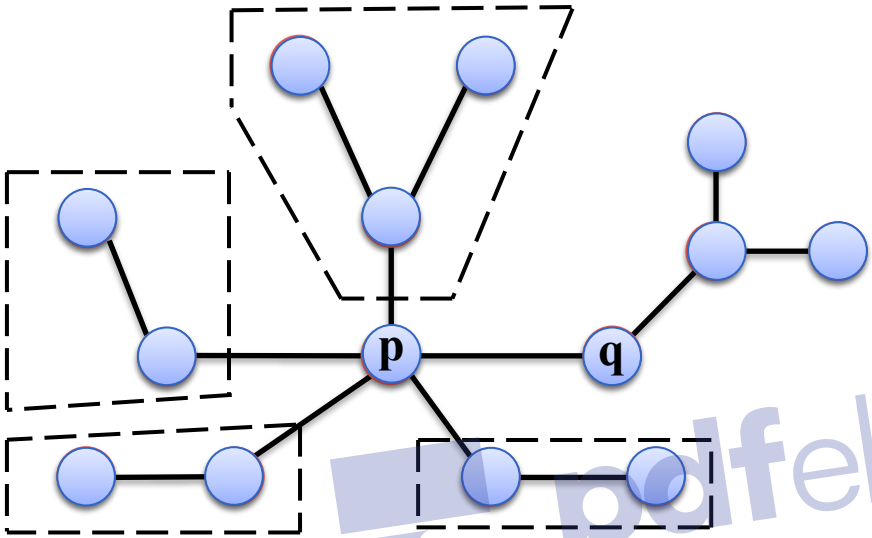
pdfelement



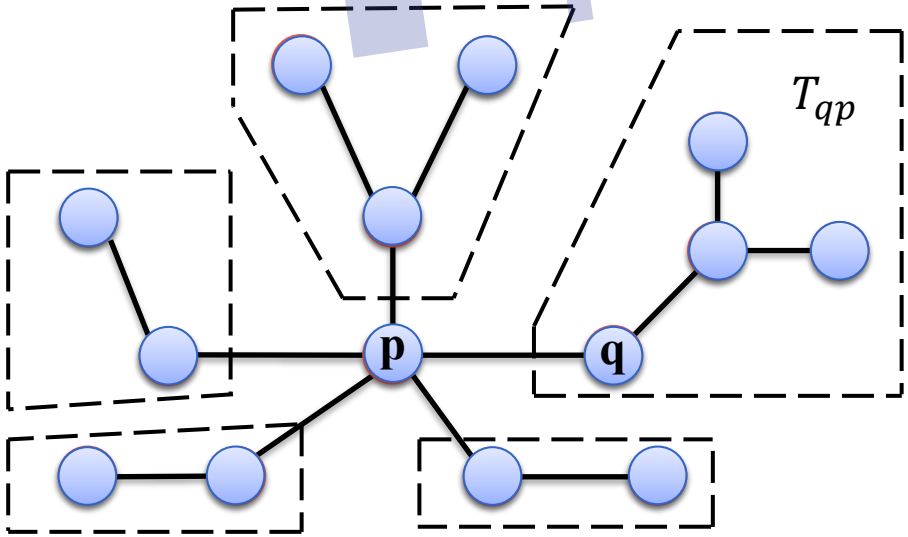
Subseturile T_{pq} *şi* T_{qp}

Remove Watermark Now

Algoritmul arbore (5)



Descompunera lui T_{pq}



Descompunera lui T

pdfelement

Remove Watermark Now

Algoritmul arbore

(2) în γ, cel puţin un proces a executat un eveniment "*decide*“

- demo prin reducere la absurd
- N noduri ➔ N-1 legături bidirecţionale între vecini ➔ 2N-2 posibile recepţii de mesaje
- K procese au trimis un mesaj în γ
- mesajele au fost receptionate
- ➔ F = 2N–2–K receptii neconsumate
- Pp. că nici un proces nu a executat "*decide*" în γ
  - fiecare proces este “blocat” la una din operatiile **receive**
- N-K procese nu au trimis mesaj (sunt in bucla while r>1 ...)

  ⇒ fiecare din ele mai are de primit cel puţin 2 mesaje
- K procese au trimis dar nici unul nu a decis

  => fiecare mai are de primit un mesaj

  => receptii neconsumate F >= 2(N-K)+K

Impreuna cu F = 2N–2–K

➔ 2N–2–K >= 2(N-K)+K

-2 >= 0

➔ cel puţin un proces a executat "*decide*" în γ

```
while (r>1){receive ch[p](id,tok);
                    rec[id] = true; r = r-1;}
/* de la un singur vecin, q0, nu s-a primit mesaj */
 afla q0: Vecini[q0] and NOT rec[q0];
 send ch[q0](p, tok);
 receive ch[p](q0, tok); rec[q0] = true;
 decide;
```

pdfelement

Remove Watermark Now

Algoritmul arbore

- (3) "decide" este precedat de un eveniment in fiecare proces
- Fie: f_{pq} **transmiterea** unui mesaj de la p la q
  - g_{pq} **receptia** la q a unui mesaj transmis de p
  - Cs submulţimea evenimentelor **e** produse în procesul **s**
  - ≤ relaţia de precedenţă
- Se dem. prin inductie peste evenimentele de recepţie:
- **pentru orice s din Tpq există e ∈ Cs : e ≤ g_{pq}**
- Presupunem că propozitia este adevărată pentru toate evenimentele *receive* care preced g_{pq} şi demonstrăm că este adevărată şi pentru g_{pq}.
- cf. algoritm, p trimite mesaj lui q dupa ce primeste mesaj de la toti ceilalti vecini r
- In Trp (similar in celilalti sub-arbori din Tpq):
- **e ≤ g_{rp} (ipoteza inductie)**
- **g_{rp} ≤ f_{pq}**
- **f_{pq} ≤ g_{pq}**
- **➔ e ≤ g_{pq}**

pdfelement

Remove Watermark Now

Algoritm ecou

pdfelement

Remove Watermark Now

Algoritmul ecou

- se aplică unor **topologii arbitrare**
- este **centralizat**; există un singur inițiator, I
- propus de Chang; prezentam versiunea mai eficientă Segall
- bazat pe inundarea rețelei cu mesaje **tok**
  - se stabilește un arbore de acoperire
  - mesaje **tok** sunt transmise înapoi spre rădăcină prin canalele arborelui de acoperire

pdfelement

Remove Watermark Now

Algoritmul ecou

```
chan ch[1:n](int id, tik_type tok);
const int I = id_initiator;

process Proc(I){
   bool Vecini[1:n] = vecinii_lui_I;
   int r = numar_vecini_I, id;
   tok_type tok;

   for [i=1 to n st Vecini[i]] send ch[i](I, tok);
   while (r > 0) {
       receive ch[I](id, tok);
       r = r - 1;
   }
   decide;
}
```

pdfelement

Remove Watermark Now

Algoritmul ecou (2)

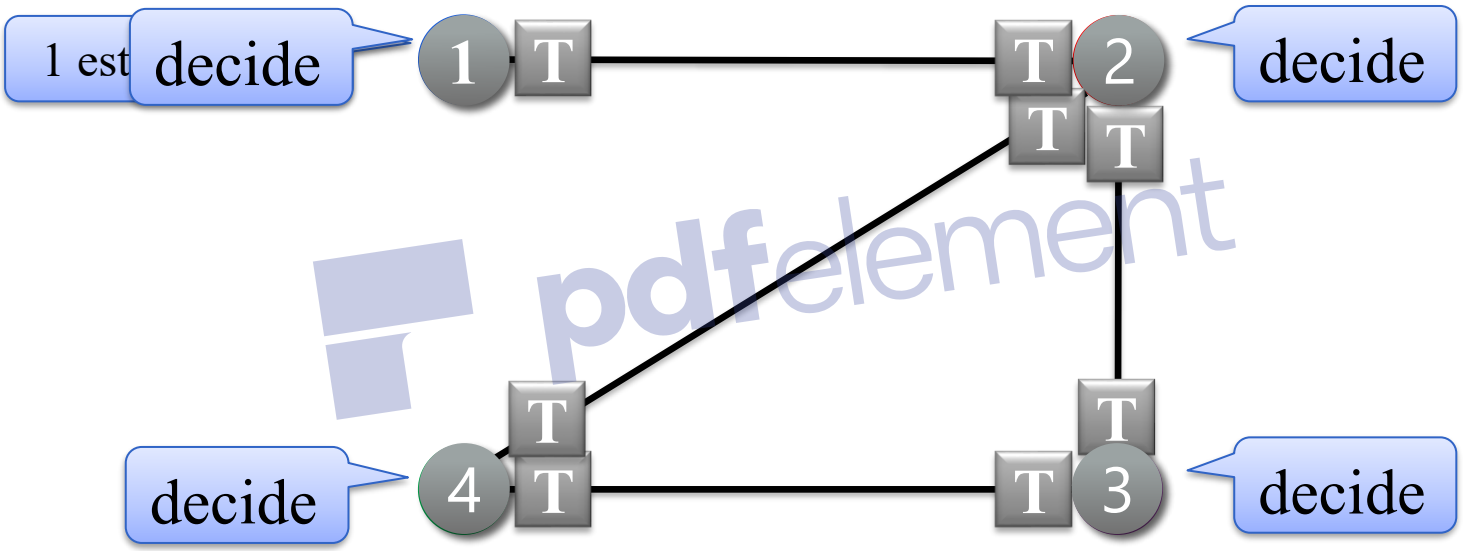
```
process Proc[p=1 to n, p <> I]{
   bool Vecini[1:n] = vecinii_lui_p;
   int r = numar_vecini_p, id, parinte;
   tok_type tok;

   receive ch[p](parinte, tok); r = r - 1;
   for [q=1 to n st Vecini[q] and q<> parinte)
       send ch[q](p, tok);
   while (r > 0) {
       receive ch[p](id, tok); r = r - 1; }
   send ch[parinte](p, tok)
}
```

$$Mesaje = 2|E|, Timp = O(N)$$

pdfelement

Remove Watermark Now

Algoritmul ecou (3)
exemplu de execuție



pdfelement

	Nodul 1	Nodul 2	Nodul 3	Nodul 4
Vecini =	{2}	{1, 3, 4}	{2, 4}	{2, 3}
părinte =		1	2	2
r =	0	0	0	0

Remove Watermark Now

Algoritmul fazelor (pulsatiilor, heartbeat, gossiping..)

pdfelement

Remove Watermark Now

Algoritmul fazelor

- algoritm **descentralizat**
- **topologii arbitrare**
- **canale unidirecționale**
- vecinii sunt: **in**-vecini și **out**-vecini
- procesele cunosc diametrul grafului D (sau o valoare $D' > D$)
- fiecare proces trimite exact D mesaje fiecărui out-vecin
- mesajul $i + 1$ este trimis fiecărui out-vecin numai după ce i mesaje au fost primite de la fiecare in-vecin

pdfelement

Remove Watermark Now

Algoritmul fazelor

```
chan ch[1:n](int id, tok_type tok);
const int D = diametrul_retelei;

process Proc[p = 1 to n] {
   bool in[1:n] = in-vecinii_lui_p;
   bool out[1:n] = out-vecinii_lui_p;
   int rec[1:n] = ([n] 0);
   /* rec[q] = număr mesaje primite de la q */
   int sent = 0;
   /* număr de mesaje transmise fiecărui out-vecin */
   tok_type tok;
   int id;
```

pdfelement

Remove Watermark Now

Algoritmul fazelor (2)

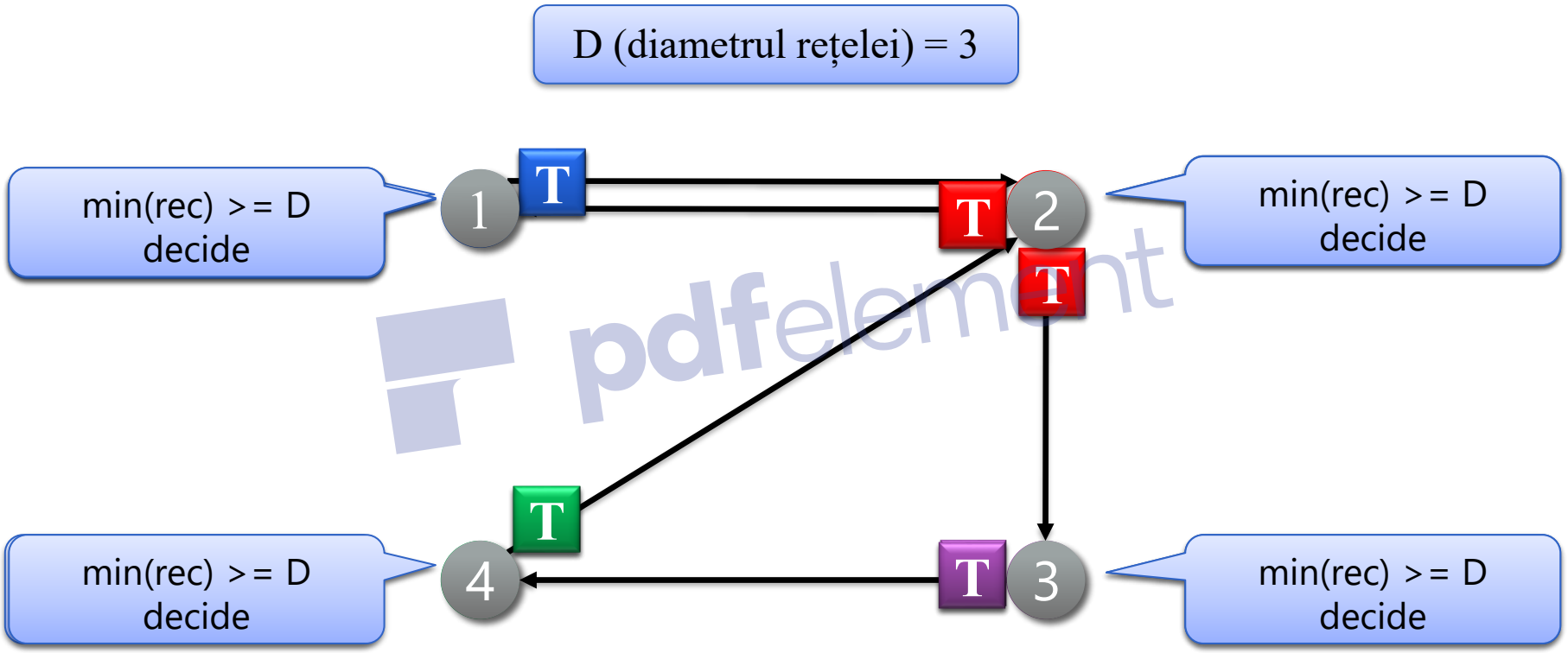
```
if (p este inițiator) {
    for [q = 1 to n st out[q]] send ch[q](p, tok);
    sent = sent + 1
}

/* min(rec) este min(rec[q], ∀q ∈ in) */
while (min(rec) < D) {
    receive ch[p](id, tok); rec[id] = rec[id] + 1;
    if (min(rec) >= sent and sent < D) {
        for [q=1 to n st out[q]] send ch[q](p, tok);
        sent = sent + 1;
    }
}
decide;
}
```

pdfelement

Mesaje = 2D|E|, unde E este mulț. canalelor nedirijate (2 canale dirijate)
Timp = 2D

Remove Watermark Now

Algoritmul fazelor (3) exemplu de execuție



pdfelement

	Nodul 1	Nodul 2	Nodul 3	Nodul 4
rec[4] =	{2:3}	{1:3, 4:3}	{2:3}	{3:3}
sent =	3	3	3	3

Remove Watermark Now

Algoritmul fazelor (4)

Teoremă: Algoritmul "fazelor" este un algoritm undă

1. calcul finit
   fiecare proces trimite cel mult D mesaje prin fiecare canal ⇒
   algoritmul atinge o configurație terminală γ după un număr finit de pași

2. în γ, fiecare proces a executat un eveniment "decide"
   presupunem cel puțin un inițiator în C (pot fi mai mulți)
   2.1. fiecare proces a trimis cel puțin un mesaj
   2.2. fiecare proces a decis

3. "decide" este precedat de un eveniment în fiecare proces

pdfelement

Remove Watermark Now

- **T. Algoritmul "fazelor" este un algoritm unda**
- (1) **calcul finit**

  fiecare proces trimite cel mult D mesaje fiecarui out-vecin

  → algoritmul atinge o configuratie terminala **γ** intr-un calcul C cu numar finit de pasi
- (2) **in γ, fiecare proces a inregistrat un eveniment "decide"**

  presupunem cel putin un initiator in calculul C (pot fi mai multi)

  (2.1) fiecare proces a trimis cel putin un mesaj

```
if (p este initiator){
   for [q = 1 to n st out[q]]{
     send ch[q](p, tok); sent = sent+1;}

while (min(rec) < D){
  receive ch[p](id, tok); rec[id] = rec[id]+1;
  if (min (rec) >= sent and sent < D){
     for (q=1 to N st out[q]]
      send ch[q](p, tok);
     sent = sent+1;}
  ...
```

daca p ne-initiator: **sent = 0** → cand p primeste primul mesaj, transmite unul tuturor out-vecinilor

pdfelement

Remove Watermark Now

(2.2) In configuratia terminala γ fiecare proces a decis

```
while (min(rec) < D){
  receive ch[p](id, tok); rec[id] = rec[id]+1;
  if (min (rec) >= sent and sent < D){
     for (q=1 to N st out[q]]
      send ch[q](p, tok);
     sent = sent+1;}
} decide;
```

Demonstram ca orice proces iese din ciclu, adica min(rec) = D , deci nu exista in-vecin care sa-i fi trimis mai putin de D mesaje

Fie p procesul cu cel mai mic $sent_p$ in γ ➔ ∀q $sent_q >= sent_p$ (1)

In particular, se aplica pentru toti in-vecinii q ai lui p

In γ, toate mes. trimise de q sunt recept. de p ➔ $sent_q = rec_p[q]$ (2)

(1)&(2) → $min_q (rec_p[q]) >= sent_p$ conditia din **if** este semi-indeplinita

➔ $sent_p$ =D; altfel p ar fi trimis mesaje suplimentare la ultima receptie

$sent_q$ =D ∀ q ➔ $rec_p[q]$ =D pentru orice canal qp

min(rec) = D ➔ fiecare proces a iesit din ciclu si a decis

pdfelement

Remove Watermark Now

- **(3) "decide" este precedat de un eveniment in fiecare proces**

  Ne referim la decizia din procesul **p**

  D este diametrul → pentru orice **q** exista o cale la **p** cu lungime <= D

  q = p(0), p(1), …, p(k) = **p**, cu lungimea k<=D

  Intr-o faza, transmisia precede receptia aceluiasi mesaj

  $f_{p(i)p(i+1)}^{(i+1)} \leq g_{p(i)p(i+1)}^{(i+1)}$ pentru 0<= i <k

  Din algoritm, receptia unui mesaj precede transmiterea lui in faza urmatoare

  $g_{p(i)p(i+1)}^{(i+1)} \leq f_{p(i+1)p(i+2)}^{(i+2)}$ pentru 0<= i <k-1

  Aplicand succesiv relatiile, rezulta $f_{p(0)p(1)}^{(1)} \leq g_{p(k-1)p(k)}^{(k)}$

  In plus, din algoritm, $g_{p(k-1)p(k)}^{(k)} \leq d_p$

  ➔ decizia lui p este precedata de un eveniment in orice q: $f_{qp(1)}^{(1)} \leq d_p$

pdfelement

Remove Watermark Now

Algoritmul fazelor pentru clici

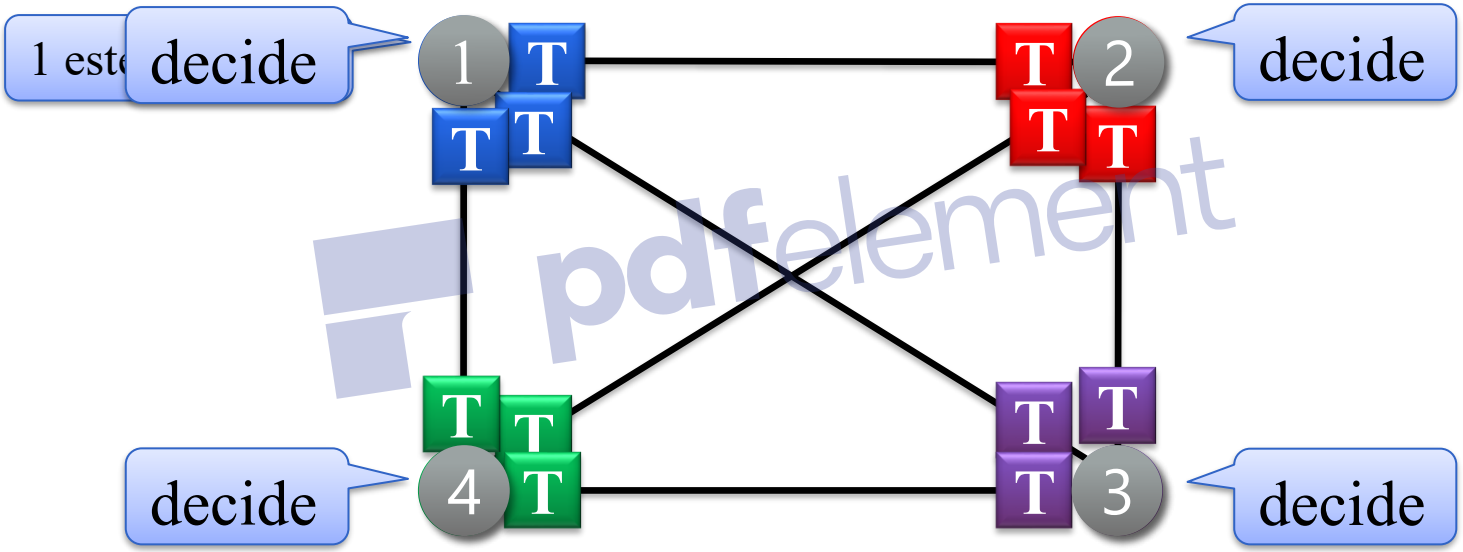
```
chan ch[1:n](int id, tok_type tok);
process Proc[p = 1 to N] {
   bool Vecini[1:n] = vecinii_lui_p;
   int n_vecini = numar-vecini_p;
   int rec = 0, sent = 0, id;
   tok_type tok;
   if (p este initiator) {
        for [q=1 to N st Vecini[q]] send ch[q](p, tok);
        sent = sent+1;
   }
   while (rec < n_vecini) {
        receive ch[p](id, tok); rec = rec + 1;
        if (sent == 0) {
              for [q=1 to N st Vecini[q]] send ch[q](p, tok);
              sent = sent + 1;
   }    }
   decide;
}
```

pdfelement

$$Mesaje = N(N-1), Timp = 2$$

Remove Watermark Now

Algoritmul fazelor pentru clici (2) exemplu de execuție



pdfelement

	Nodul 1	Nodul 2	Nodul 3	Nodul 4
rec =	3	3	3	3
sent =	1	1	1	1

Remove Watermark Now

Algoritmul lui Finn

pdfelement

Remove Watermark Now

Algortimul lui Finn

- nu cere cunoașterea diametrului
- se bazează pe cunoașterea identificatorilor proceselor
- în mesaje se transmit seturi de identificatori de procese
- proc **p** păstreaza două mulțimi de ids

  Inc – mulțimea proceselor **q** pentru care un eveniment în **q** precede cel mai recent eveniment în **p**

  NInc – mulțimea proceselor **q** pentru care fiecare vecin **r** are un eveniment care precede cel mai recent eveniment în **p**

pdfelement

- **Algoritmul:**

  - inițial: $Inc = \{p\}, \quad NInc = \emptyset$
  - p trimite mesaje cu Inc și $NInc$ de fiecare dată când Inc sau $NInc$ crește
  - când p primește mesaje cu Inc și NInc, actualizează Inc și NInc (reuniune)
  - când p a primit un mesaj de la toți in-vecini, p este inserat în NInc
  - când Inc devine egal cu NInc, p decide

Remove Watermark Now

Algortimul lui Finn (2)

```
typedef SOP {multime identificatori};
chan ch[1:N](int id, SOP Inc, SOP NInc);

process Proc[p = 1 to n]{
   SOP Inc = {p};
   SOP Ninc = Ø;
   /* de la cine a recepționat */
   bool rec[1:n] = ([n] false);
   bool out[1:n] = out-vecinii_lui_p;
   bool in[1:n] = in-vecinii_lui_p;
   SOP rInc, rNInc;
   int id;

   if (p este initiator)
       for [q = 1 to n st out[q]]
          send ch[q](p, Inc, NInc);
```

pdfelement

Remove Watermark Now

Algortimul lui Finn (2)

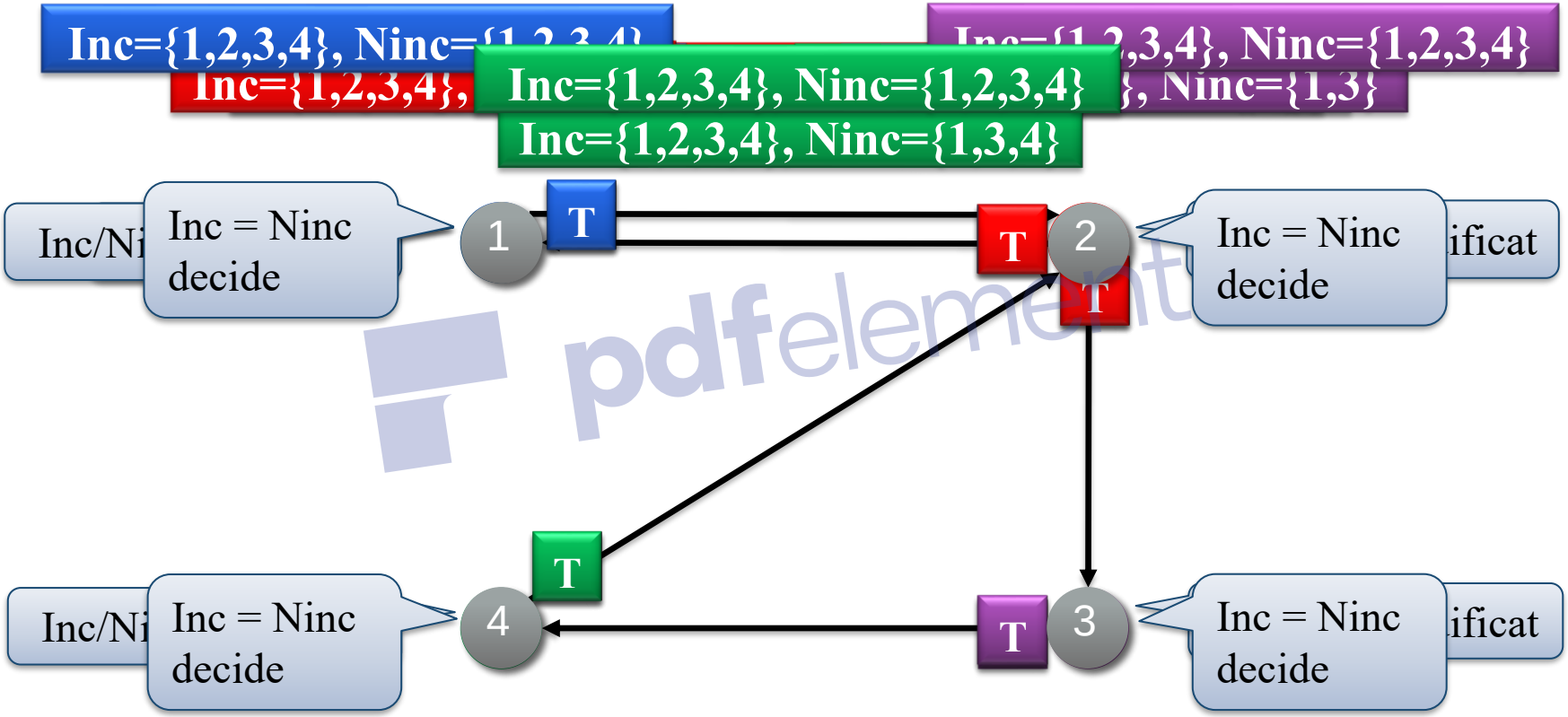
```
while (Inc <> Ninc) {
    receive ch[p](id, rInc, rNInc);

    Inc  = Inc  U rInc;
    NInc = NInc U rNInc;

    rec[id] = true;
    if (rec[id, for ∀id=1..n and in[id])
        NInc = NInc U {p};
    if (Inc sau NInc modificat)
        for [q=1 to n st out[q]]
            send ch[q](p, Inc, NInc);
}
decide;
}
```

pdfelement

$$Mesaje \leq 2N|E|, \qquad Timp = O(D)$$

Remove Watermark Now

Algoritmul lui Finn (3)
exemplu de execuție



pdfelement

	Nodul 1	Nodul 2	Nodul 3	Nodul 4
Inc =	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}
Ninc =	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}
rec =	{2:T}	{1:T, 4:T}	{2:T}	{3:T}

Remove Watermark Now

Algoritmul lui Finn (4)

Teoremă: Algoritmul lui Finn este un algoritm undă

1. calcul finit

2. în γ, fiecare proces a executat un eveniment "decide“

   2.1. în γ fiecare proces a trimis cel puțin un mesaj pe fiecare canal (dem. similară alg. fazelor)

   2.2. în γ fiecare proces a decis

      2.2.1. $\forall p,\ Inc_p\ conține\ toate\ procesele\ (în\ \gamma)$

      2.2.2. $\forall p\ și\ q,\ NInc_p = NInc_q$

3. "decide" este precedat de un eveniment în fiecare proces

pdfelement

Remove Watermark Now

T. Algoritmul lui Finn este un algoritm unda

- (1) **calcul finit**
- Se trimite un mesaj la fiecare modificare Inc sau Ninc
- Dimensiunea celor doua seturi luate impreuna este initial 1 si ajunge la 2N
  → numarul de mesaje transmise este limitat superior la 4N|E|

- (2) **In γ, fiecare proces a executat un eveniment "decide"**
- 2.1. in **γ** fiecare proces a trimis cel putin un mesaj pe fiecare canal:
  p trimite tuturor vecinilor q → se schimba Inc_q → q transmite vecinilor sai; in plus graful este tare conex etc.
- 2.2. in **γ** fiecare proces a decis
  2.2.1. ∀ p, Inc_p contine toate procesele (in γ)
  daca exista arc pq, la modificare Inc_p, p il trimite lui q; q il include in Inc_q
  graf tare conex → pentru orice p si q: Inc_q include Inc_p
  impreuna cu Inc_p include Inc_q → $Inc_p = Inc_q$ pentru **∀** p si q
  → Inc_p contine identitatile tuturor proceselor

pdfelement

Remove Watermark Now

2.2.2. ∀ p si q, $NInc_p = NInc_q$

Fiecare proces a trimis un mesaj pe fiecare canal

→ fiecare proces a receptionat de la toti vecinii (∀ id ∈ in: rec[id])

→ (cf alg) p este adaugat la NInc

$NInc_p$ contine toate procesele – demo similara cu Inc_p

→ $Inc_p = NInc_p$ in γ

→ fiecare proces a decis in γ

pdfelement

- (3) In procesul p, "decide" este precedat de un eveniment in fiecare proces

  Fie q un in-vecin al lui p

  Fie evenimentul intern lui q, **a_q: NInc := NInc U {q}**

  conform algoritmului, a_p se executa daca pentru fiecare in-vecin r al lui q un eveniment din r precede a_q

  Procesul p decide doar cand $Inc_p = NInc_p$ = setul tuturor proceselor

Remove Watermark Now

→ cand procesul p ia decizia:

- p € Inc
- in-vecinul q € Inc si q € NInc
- deci decizia lui p este precedata de un eveniment din q si un eveniment din fiecare din in-vecinii lui q
- relatia este valabila pentru oricare doua procese din care unul este in-vecinul celuilalt
- retea conexa → decizia lui p este precedata de un eveniment din fiecare din celelalte procese

pdfelement

Remove Watermark Now

Sumar

- Sisteme de tranziții
- Caracteristici ale algoritmilor undă
- Algoritmul inel
- Algoritmul arbore
- Algoritmul ecou
- Algoritmul fazelor
- Algoritmul lui Finn

pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Alegerea unui lider

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Alegerea unui lider

- Gerard LeLann a propus problema alegerii unui lider într-un paper celebru din 1977
- Multe sisteme distribuite sunt bazate pe paradigma client-server:
  - Un server/coordonator (lider)
  - Mai mulți clienți
- Ce se întâmplă dacă liderul se defectează?
  - Soluție: se alege un nou lider
- Problema generică: dintr-o colecție de procese, se alege unul singur care urmează să joace un rol special *(lider)*:
  - Execută operații de inițializare
  - Controlează alte procese

pdfelement

INFORMATION PROCESSING 77, B. GILCHRIST, EDITOR
© IFIP, NORTH-HOLLAND PUBLISHING COMPANY (1977)

Remove Watermark Now

DISTRIBUTED SYSTEMS—TOWARDS A FORMAL APPROACH

GÉRARD LE LANN
IRISA—Université de Rennes—BP 25 A
35 031 Rennes Cedex, France

Le Lann, Gérard. "Distributed Systems-Towards a Formal Approach." *IFIP congress*. Vol. 7. 1977.

Packet-switching computer communication networks are examples of distributed systems. With the large scale emergence of mini and micro-computers, it is now possible to design special or general purpose distributed systems. However, as new problems have to be solved, new techniques and algorithms must be devised to operate such distributed systems in a satisfactory manner. In this paper, basic characteristics of distributed systems are analysed and fundamental principles and definitions are given. It is shown that distributed systems are not just simple extensions of monolithic systems. Distributed control techniques used in some planned or existing systems are presented. Finally, a formal approach to these problems is illustrated by the study of a mutual exclusion scheme intended for a distributed environment.

pdfelement

1. INTRODUCTION

Computer communication networks using packet-switching technology provide for the interconnection of data-processing equipments of any kind. Such systems, sometimes simply referred to as computer networks, may be viewed as multi-macroprocessors whenever the goals of resource-sharing are achieved. With the large-scale emergence of mini and microcomputers, it is now possible to envision building general or special purpose multimini and multimicrocomputers to be operated in a non-centralized manner. The need for automatic resource-sharing arises here as in a similar way it does for multimacroprocessor systems.

Two kinds of resources must be considered :
- system resources, multi-accessed by users and for which multiplexing is required (hidden sharing)
- user resources, which users agree to share according to some protocol of their own (explicit sharing).

This paper discusses the problems of system resource-sharing in a distributed environment. An example of a user-sharing problem is distributed data-base sharing.

2. DISTRIBUTED SYSTEMS-ELEMENTS FOR A FORMAL APPROACH

Experimental and public packet-switching computer

cesses called logical entities.

Let $F = \{f_i, i \in I\}$ be the set of the operating functions and $E_i = \{e_j^i, j \in J(i)\}$ be the set of entities participating in fuction f_i. At any instant t, it is possible to define $s_t(e_j^i)$ as the instantaneous state of entity e_j^i. It is therefore theoretically possible to define the global state of E_i at instant t as the vector $S_t(E_i) = \{..., s_t(e_j^i), ...$ for all $j \in J(i)\}$.

A system will be said to be f_i-centralized if there exists $k \in J(i)$ such that $S_t(E_i)$ is known to e_k^i.

An example is a system in which $\dim(E_i) = 1$; another example is a system in which entities run the operating function f_i by using a common "system table".

A system will be said to be totally centralized if it is f_i-centralized for any $i \in I$.

In contrast, a system will be said to be f_i-distributed if there does not exist $k \in J(i)$ such that $S_t(E_i)$ is known to e_k^i.

A system will be said to be totally distributed if it is f_i-distributed for any $i \in I$.

Typical cases of distributed systems are systems in

Remove Watermark Now

Alegerea unui lider

- Desemnarea statică a liderului ne-aplicabilă
  - nu se cunoaște compoziția exactă a grupului de procese
- Fiecare proces își cunoaște propria identitate și vecinii.
- Identitățile proceselor aparțin unei mulțimi total ordonate
- Alegerea liderului = determinarea procesului care are identitatea cea mai mică *(sau cea mai mare).*
- Alegerea se face prin algoritmi descentralizați, cu participarea tuturor proceselor din colecție.

pdfelement

Remove Watermark Now

Alegerea unui lider

- Problemă:
  - Atunci când un proces se defectează, mai multe procese pot depista problema simultan
- Nu putem presupune că un singur proces detectează defectul
  - Algoritmul de alegere trebuie să conducă în final la situația în care toate procesele decid asupra aceluiași proces ca nou lider

pdfelement

Remove Watermark Now

Alegerea unui lider cu algoritmi undă

Caracteristici:

- toate procesele au același algoritm local
- algoritmul este **descentralizat** *(poate fi inițiat de oricare subset de procese)*
- în fiecare calcul algoritmul atinge o configurație finală în care:
  - un proces este lider
  - toate celelalte au pierdut
- varianta mai slabă:
  - un proces este lider
  - celelalte procese nu știu că au pierdut

pdfelement

Remove Watermark Now

Alegerea unui lider cu algoritmi unda (2)

- Fiecare proces are o **stare** dintre:
  - leader
  - lost
  - sleep *(procesul nu a executat nici o acțiune)*
  - candidate *(a executat acțiuni dar nu e sigur dacă e leader sau lost)*

pdfelement

- Ipoteze:
  - Sistemul este asincron *(comunicație asincronă, fără timp global, timpi arbitrari de transmisie)*
  - fiecare proces este identificat printr-un nume unic
  - Identitățile sunt extrase dintr-o **mulțime total ordonată**
  - aflarea liderului se transformă în **aflarea procesului cu cel mai mic identificator**

Remove Watermark Now

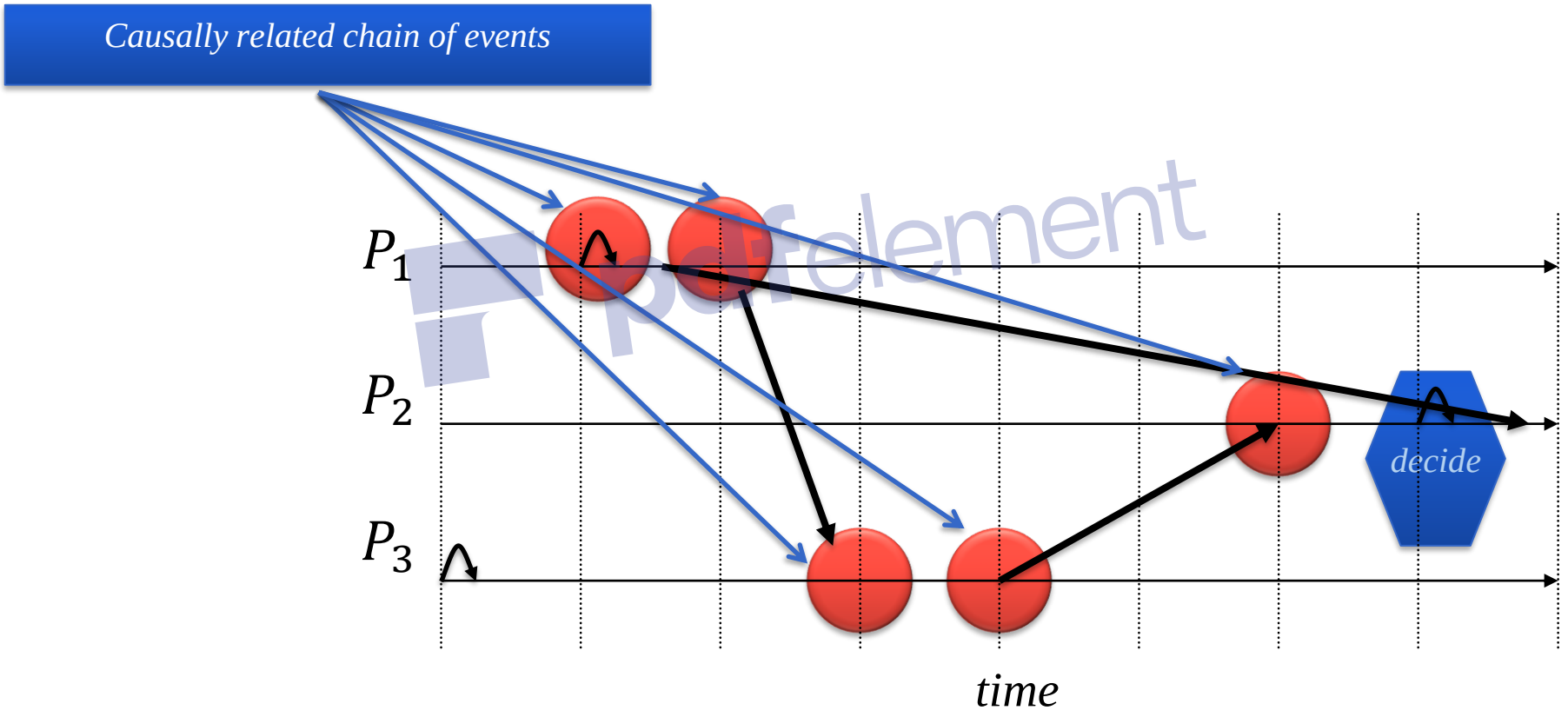
Algoritmi undă pentru alegerea liderului

- Algoritmii undă au cele trei proprietăți:
  - Terminare
    - Algoritmul se termină într-un număr finit de pași
  - Decizie
    - Cel puțin un nod execută un eveniment *decide*
  - Consistență
    - *Oricare* proces execută cel puțin un eveniment ce precede cauzal evenimentul *decide*

pdfelement

Remove Watermark Now

Proprietățile algoritmilor undă



Remove Watermark Now

Folosirea algoritmilor undă pentru alegerea liderului

- Ideea:
  - Fiecare proces are atașată o identitate în cadrul undei
    - Un nod decide atunci când un eveniment a precedat cauzal fiecare eveniment prin care procesele adaugă identitatea respectivă în vectorul de identități cunoscute
    - Procesul ce decide va cunoaște identitatea tuturor proceselor
    - Se alege procesul având identitatea cea mai mică ca nou lider
    - Se anunță *(broadcast)* identitatea liderului tuturor proceselor

partelement

Remove Watermark Now

Exemplu: alegere leader folosind Algoritmul Arbore

pdfelement

Remove Watermark Now

Alegerea unui lider cu algoritmi undă (3)

Alegerea liderului cu algoritm tree

- algoritmul impune ca cel puțin toate frunzele să fie inițiatori ⇒ adaugă o fază de *wakeup:*
  - inițiatorii trimit mesaje *wakeup* tuturor proceselor
  - fiecare proces folosește variabilele:
    - **ws** – boolean, asigură că fiecare proces transmite mesaje *wakeup* cel mult o dată
    - **wr** – int, contorizeaza mesajele de *wakeup* recepționate
- când un proces a primit *wakeup* prin fiecare canal, el începe algoritmul de alegere
- când un proces decide, el află identitatea liderului
- în funcție de ea procesul trece în starea leader sau lost

pdfelement

Remove Watermark Now

Algoritmul tree

```
chan ch[1:n](int id_transm, in min_id);
chan wakeup[1:n]();
process Proc[p = 1 to n]{
    bool ws = false;
     int wr = 0, r = număr_vecini_p;
    bool Vecini[1:n] = vecinii_lui_p;
     bool rec[1:n] = ([n] * false);
    int V = p;
    enum state{sleep, leader, lost};
     state stare = sleep;
     int id;
    /* aici începe faza wake-up */
    if (p este initiator) {
         ws = true;
         for [q = 1 to r st Vecini[q]]
              wakeup[q]();
    }
```

pdfelement

Remove Watermark Now

Algoritmul tree (2)

```
while (wr < număr_vecini_p){
     receive wakeup[p]();
     wr = wr + 1;
     if (NOT ws) {
          ws = true;
          for [q = 1 to n st Vecini[q]
               send wakeup[q]();
}    }

/* aici începe algoritmul tree */
while (r > 1){
     receive ch[p](q, id);
     rec[q] = true; r = r - 1;
     V = min(V, id);
}
```

Dacă este primul wakeup primit ⇒ trimit wakeup la toți vecinii

Aștept să primesc un răspuns de la N – 1 vecini și fac un minim între răspunsurile acestora.

N = numărul de vecini

pdfelement

Remove Watermark Now

Algoritmul tree (3)

```
/* de la un singur vecin, q0 nu s-a primit mesaj */
afla q0: Vecini[q0] and NOT rec[q0];
send ch[q0](p, V);
receive ch[p](q0, id);
V = min(V, id);
if (V == p)
    stare = leader
else
    stare = lost

/* informeaza celelalte procese despre decizie */
for [q = 1 to n st Vecini[q] and q<>q0]
    send ch[q](p,V);
```

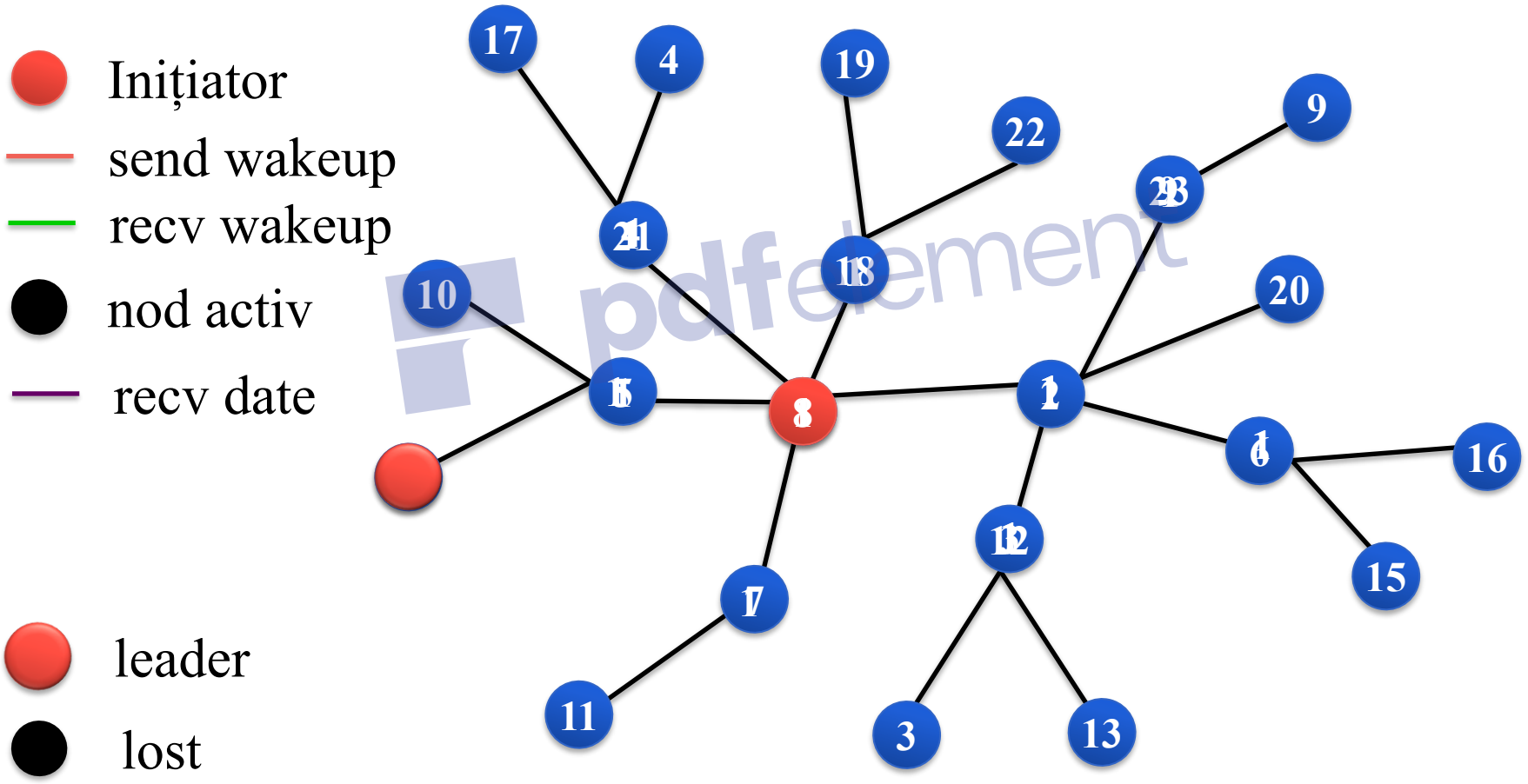
Aștept rezultatul final de la ultimul nod

primit răspuns

pdfelement

Remove Watermark Now

Algoritmul tree (4)



pdfelement

Remove Watermark Now

Algoritmul tree (5)

- *Notații :*
  - **N** : Numărul de procese implicate
  - **D** : Diametru rețelei formate de procesele implicate

- *Numărul de mesaje* = O(N)
  - pe fiecare canal se trimit două mesaje *wakeup* și două *token*-uri ⇒ total = 4N - 4

- *Timp* = O(D)
  - în D pași după ce primul proces pornește algoritmul, fiecare proces a trimis mesajele *wakeup* ⇒ în D+1 pași fiecare proces a pornit unda.
  - Prima decizie se ia la D pași după pornire, ultima după alți D pași ⟹ totalul = 3D+1 pași

pdfelement

Remove Watermark Now

Alegere lider – cazul topologiilor inel

pdfelement

Remove Watermark Now

Alegerea liderului - topologie inel

Specificarea problemei:

- Se dă un aranjament circular de procese identificate prin numere distincte între ele.
- Dispunerea proceselor în inel este oarecare.
- Se cere desemnarea prin consens a procesului cu id **maxim**.
- Nu se cunoaște apriori numărul de procese.
- Nu există un control centralizat.

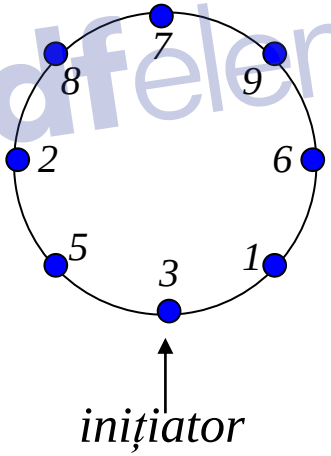
- Idee simplă:
  - Permitem fiecărui inițiator să trimită un token cu propria identitate pe inel
  - Nodurile nu mai au voie să inițieze după ce au primit un token

pdfelement

Remove Watermark Now

Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...

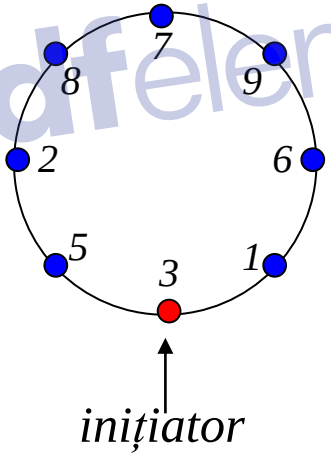


pdfelement

Remove Watermark Now

Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...

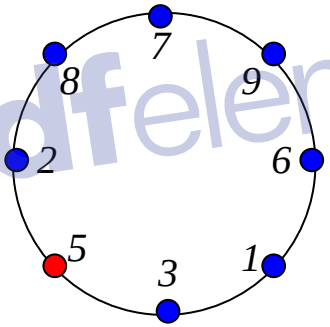


pdfelement

Remove Watermark Now

Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...

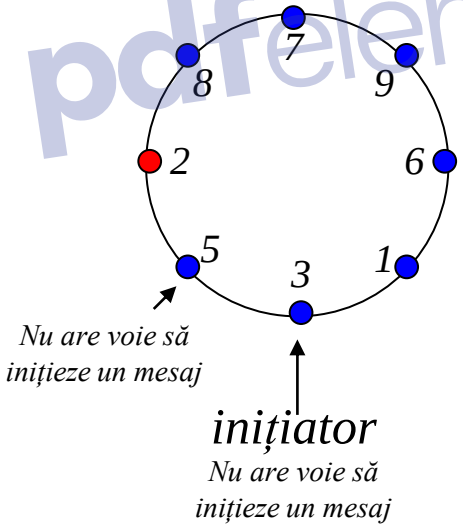


pdfelement

Remove Watermark Now

Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...

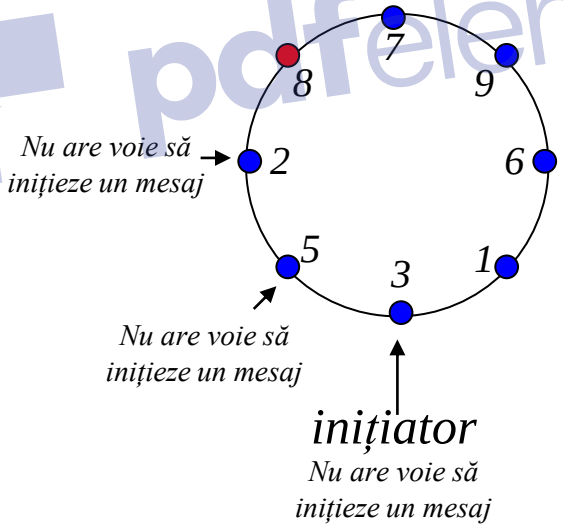


pdfelement

Remove Watermark Now

Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...

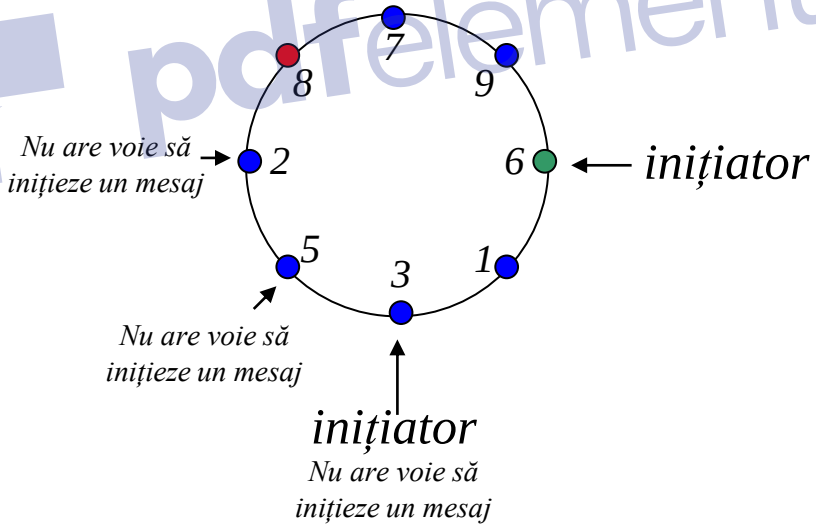


pdfelement

Remove Watermark Now

Exemplu

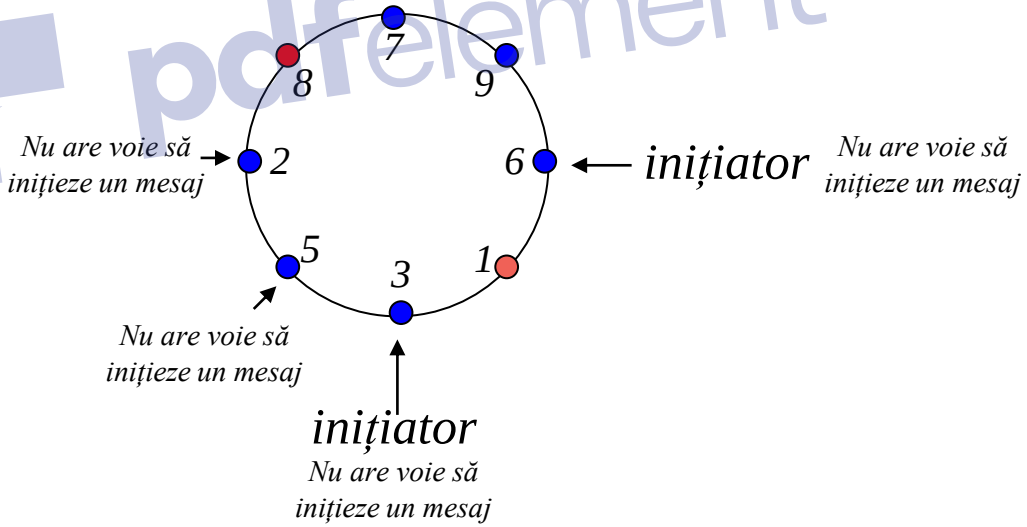
- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



Remove Watermark Now

Exemplu

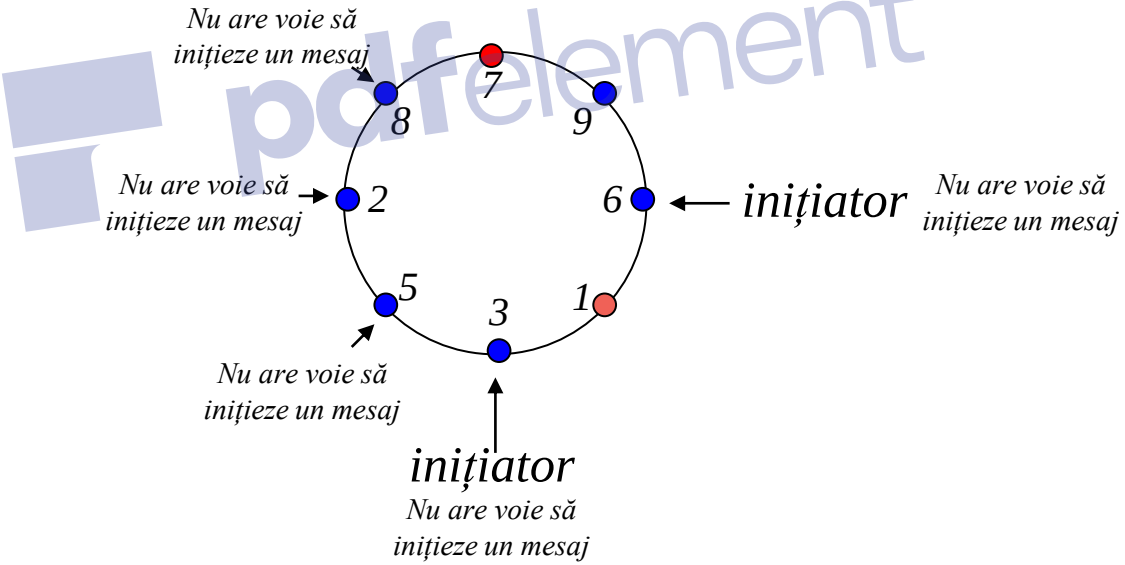
- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



Remove Watermark Now

Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



Remove Watermark Now

Algoritmul LeLann

pdfelement

Remove Watermark Now

Algoritmul LeLann

Atunci când un nod primește propriul id, a văzut deja toate celelalte id-uri

- Presupunând canale FIFO

  ⇒ Fiecare proces poate reține o listă cu identificatorii pe care i-a primit...

pdfelement

Algoritmul **LeLann:**

- Fiecare proces difuzează celorlalte un mesaj cu **id**-ul său
- Fiecare proces colectează numerele celorlalte procese
- Fiecare proces află maximul.
- Procesul al cărui număr este egal cu maximul devine lider.

Remove Watermark Now

Algoritmul LeLann

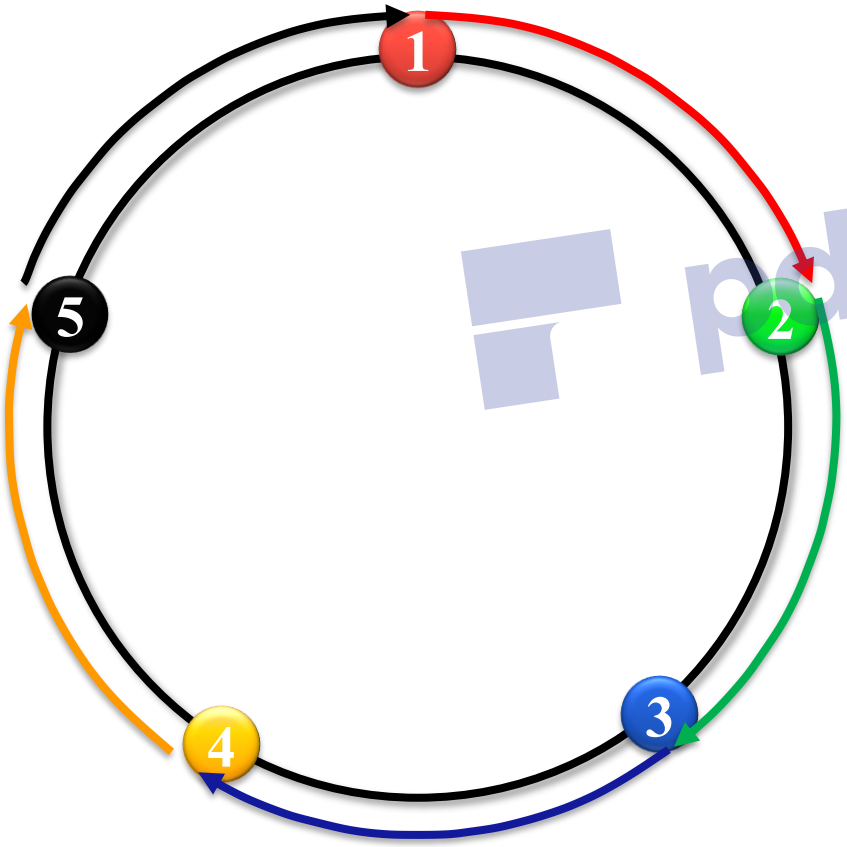
```
chan ch[1:n](token tok, int id_mare);
process Proc[p = 1 to n]{
   typedef SOP{multime identificatori};
   SOP List = {p};
   enum state (candidate, leader, lost);
   state stare = candidate;
   send ch[Urm](tok, p);
   receive ch[p](tok, q);
   while (q <> p) {
       List = List U {q};
       send ch[Urm](tok, q);
       receive ch[p](tok, q);
   }
   if (p == max(List)) stare = leader
   else state = lost
}
```

Număr de mesaje = ? $O(N^2)$

pdfelement

Remove Watermark Now

Algoritmul LeLann



Nod	ID-uri	Status	
1	{5, 4, 3, 2, 1}	lost	5
2	{1, 5, 4, 3, 2}	lost	5
3	{2, 1, 5, 4, 3}	lost	5
4	{3, 2, 1, 5, 4}	lost	5
5	{4, 3, 2, 1, 5}	Leader	5

pdfelement

Remove Watermark Now

Algoritmul Lelann-Chang-Robert

pdfelement

Remove Watermark Now

Operating Systems

R. Stockton Gaines Editor

An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes

Ernest Chang
University of Toronto

Rosemary Roberts
University of Waterloo

This note presents an improvement to LeLann's algorithm for finding the largest (or smallest) of a set of uniquely numbered processes arranged in a circle, in which no central controller exists and the number of processes is not known a priori. This decentralized algorithm uses a technique of selective message extinction in order to achieve an average number of message passes of order (n log n) rather than $O(n^2)$.

Key Words and Phrases: decentralized algorithms, distributed systems, operating systems

CR Categories: 4.32, 4.35, 5.25, 5.32

Introduction

Given a random circular arrangement of uniquely numbered processes where no a priori knowledge of the number of processes is known, and no central controller is assumed, we would like a method of designating by consensus a single unique process. The algorithm we propose works equally well for finding either the highest numbered or the lowest numbered process. Let us, without loss of generality, consider highest finding.

A situation in which this algorithm is important has been presented by LeLann [1]. In his example, a circle of controllers in which the control token is lost causes every controller to time out, and an election to find a new emitter for the control token is performed. LeLann's algorithm requires every controller to send a message bearing its number. Each controller thus collects, through the messages seen, the numbers of the other controllers in the circle. Every controller sorts its list, and the controller whose own number is the highest on its list is elected.

LeLann's algorithm, in a circle with n controllers, requires total messages passed proportional to n^2, written $O(n^2)$, where a message pass is a SEND of a message from a controller. This is clearly so, since each of the n controllers sends a message which is passed to all other nodes. Our algorithm requires, on the average, $O(n \log n)$ message passes.

The Algorithm

Each process is assumed to know its own number, and initially it generates a message with its own number, passing it to the left. A process receiving a message compares the number on the message with its own. If its own number is lower, the process passes the message (to its left). If its own number is higher, the process throws the message away, and if equal, it is the highest numbered process in the system.

Proposition: This algorithm detects one and only one highest number.

Argument: By the circular nature of the configuration and the consistent direction of messages, any message

Chang, E., & Roberts, R. (1979). An improved algorithm for decentralized extrema-finding in circular configurations of processes. *Communications of the ACM*, *22*(5), 281-283.

Remove Watermark Now

Algoritmul Lelann-Chang-Roberts

- Mesajele sunt transmise în inel **în sensul acelor de ceasornic**.
  - Fiecare proces transmite procesului din dreapta un mesaj cu identificatorul său
  - Un proces care primește un mesaj **m** îl compară cu identificatorul propriu ***(id)*:**
    - **if** m **>** id -> transmite m în dreapta
    - **if** m **<** id -> elimină m
    - **if** m **=** id -> procesul curent devine lider

- ***Propoziție:***
  - Algoritmul detectează un singur cel mai mare număr

pdfelement

Remove Watermark Now

Algoritmul Lelann-Chang-Robert (2)

```
chan ch[1:n](token tok, int id_mare);

process Proc[p = 1 to n] {
    enum state(candidate, leader, lost);
    state stare = candidate;
    send ch[Urm](tok, p);
    while (state <> lider) {
        receive ch[p](tok, q);
        if (q == p) stare = leader;
        else if (q > p) {
            if (stare == candidate)
                stare = lost;
            send ch[Urm](tok, q);
        }
}   }
```

pdfelement

Remove Watermark Now

Algoritmul Lelann-Chang-Roberts (3)

- Observații:
  - un proces **lost** ramâne în buclă pentru a pasa alte mesaje
  - doar procesul cu id maxim termină
  - pentru a debloca alte procese, liderul poate trimite un mesaj special

pdfelement

- *Algoritm modificat*:

  Când nu toate procesele sunt inițiatori:
  - Cel puțin un proces inițiază alegerea și se auto-marchează
  - Când un mesaj **m** ajunge la un proces nemarcat, acesta :
    - va genera un mesaj cu **id** propriu
    - se va marca
    - va continua cu prelucrarea mesajului **m**

Remove Watermark Now

Algoritmul Lelann-Chang-Roberts
Performanță

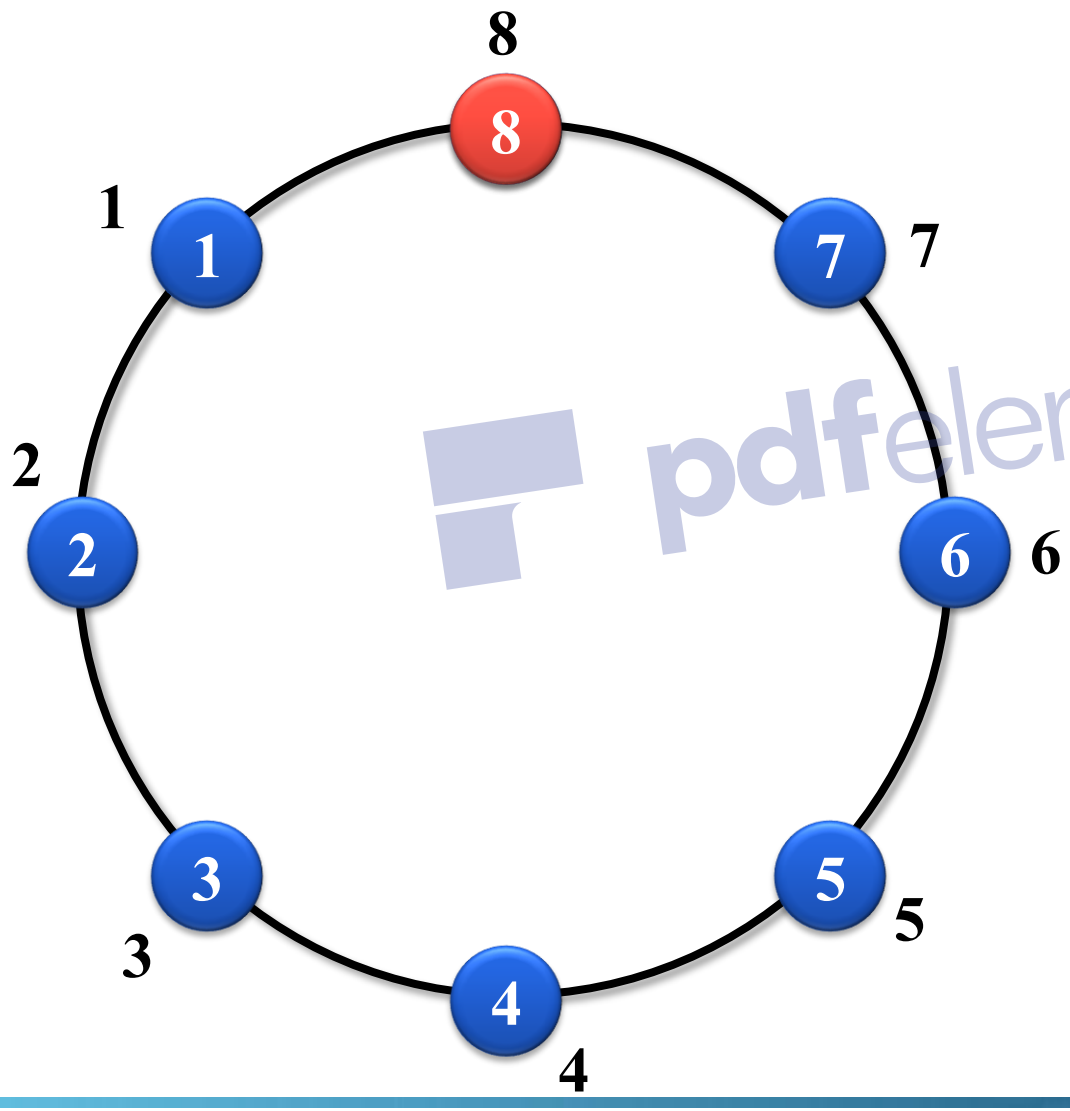
- Timp
  - Dacă toate procesele pornesc simultan, timpul este $\boldsymbol{O(N)}$, unde $\boldsymbol{N}$ este numărul de procese.
  - Dacă procesul cu numărul maxim pornește primul, timpul este $\boldsymbol{O(N)}$. Altfel, marcajul ia cel mult $\boldsymbol{N-1}$ pași să ajungă la procesul cu id-ul maxim (și apoi $\boldsymbol{N}$ pași pentru propagare), deci timpul este $\boldsymbol{O(N)}$.

pdfelement

- Numărul de mesaje
  - *Cazul cel mai bun.*
    - Procesele sunt ordonate crescător în sensul acelor de ceasornic. $\boldsymbol{2N-1}$ mesaje.
  - *Cazul cel mai rău*.
    - Procesele sunt ordonate descrescător în sensul acelor de ceasornic. $\sum_{i=1}^{N} i = \frac{N(N+1)}{2}$ mesaje.

Remove Watermark Now

Algoritmul Lelann-Chang-Roberts



Cazul cel mai rău

candidate

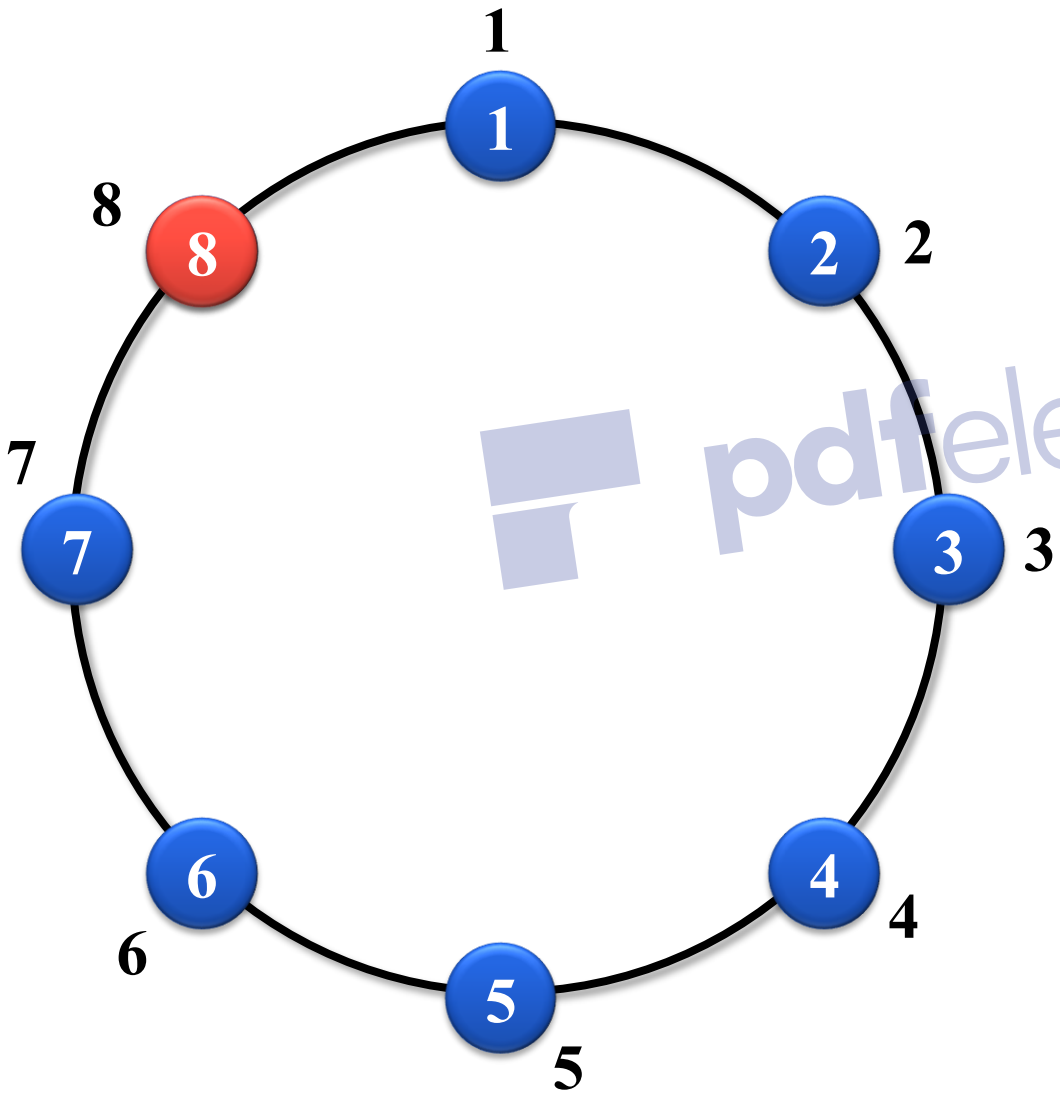
lost

leader

pdfelement

Remove Watermark Now

Algoritmul Lelann-Chang-Roberts



Cazul cel mai bun

candidate

lost

leader

pdfelement

Remove Watermark Now

Algoritmul Hirschberg-Sinclair

pdfelement

Remove Watermark Now

Technical Note
Operating Systems

R. Stockton Gaines
Editor

Decentralized Extrema-Finding in Circular Configurations of Processors

D.S. Hirschberg and J.B. Sinclair
Rice University

This note presents an efficient algorithm, requiring $O(n \log n)$ message passes, for finding the largest (or smallest) of a set of n uniquely numbered processors arranged in a circle, in which no central controller exists and the number of processors is not known a priori.

Key Words and Phrases: decentralized algorithms, distributed system, operating systems
CR Categories: 4.32, 4.35, 5.25, 5.32

Introduction

We are given n processors that are loosely coupled in a circular arrangement and work asynchronously. Each of the processors has an associated unique value (of which it alone is aware) and none of the processors has a priori knowledge of the number of processors in the circle. The problem is to designate by consensus a unique processor from the circle. The total number of data transmissions (messages passed) among the n processors

pass messages in either or both directions, that these directions are distinguished, that processors can detect from which direction a received message originated, but that "left" may not mean the same to all processors. We propose an algorithm that requires $O(n \log n)$ messages in the worst case. The algorithm as given elects the processor with the highest value.

In the algorithm given below, a processor can initiate messages in both directions by a *sendboth* directive. A processor can pass a (possibly modified) message in a circular manner by a *sendpass* directive. A processor can send a responsive message back in the direction from which that processor received a message by a *sendecho* directive.

The Algorithm

```
To run for election:
    status ← "candidate"
    maxnum ← 1
    WHILE status = "candidate" DO
        sendboth ("from", myvalue, 0, maxnum)
        await both replies (but react to other messages)
    IF either reply is "no" THEN status ← "lost"
        maxnum ← 2*maxnum
    OD
On receiving message ("from", value, num, maxnum):
    IF value < myvalue THEN sendecho ("no", value)
    IF value > myvalue THEN DO
        status ← "lost"
        num ← num + 1
        IF num < maxnum THEN sendpass ("from", value, num,
        maxnum)
            ELSE sendecho ("ok", value)
    OD
    IF value = myvalue THEN status ← "won"
On receiving message ("no", value) or ("ok", value)
    IF value ≠ myvalue THEN sendpass the message
        ELSE this is a reply the processor was awaiting
```

The processors initiate messages that are passed in both directions along paths of predetermined lengths (which are successive powers of 2). Processors on the path read the message. If a processor determines, from reading the message, that it cannot win the election, then

Hirschberg, D. S., & Sinclair, J. B. (1980). Decentralized extrema-finding in circular configurations of processors. *Communications of the ACM*, *23*(11), 627-628.

Remove Watermark Now

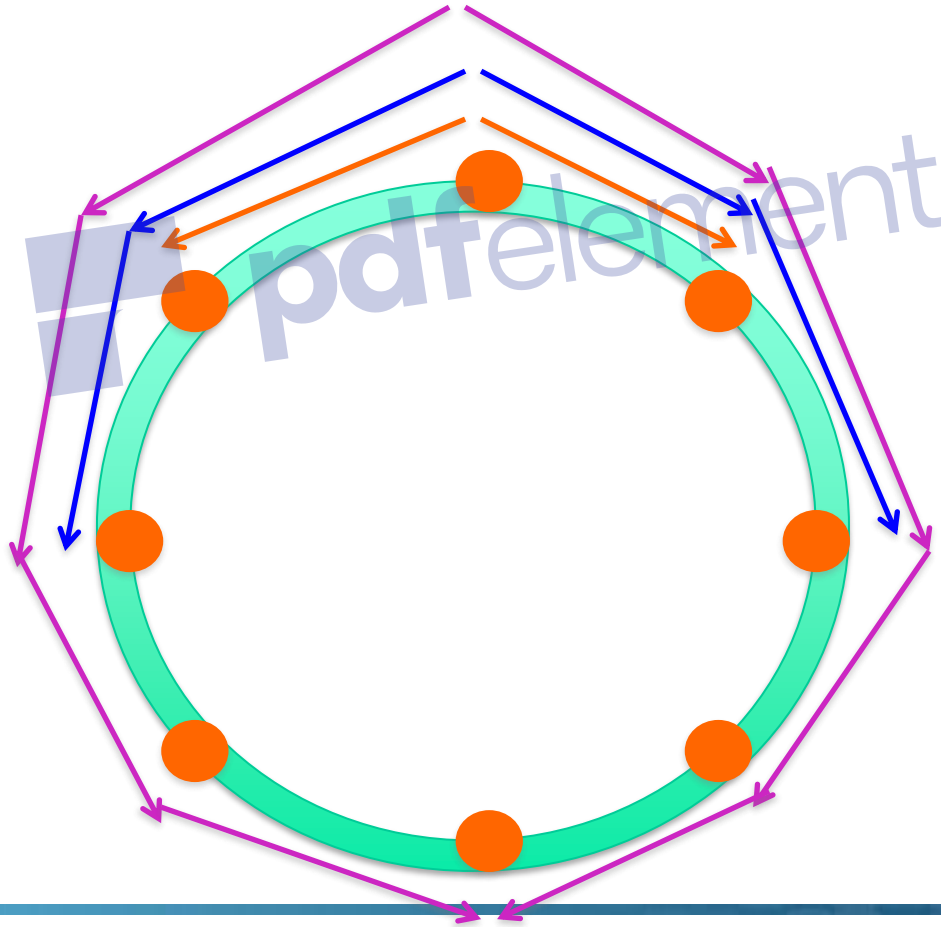
Algoritmul Hirschberg-Sinclair

- Complexitate $\boldsymbol{O(N\,logN)}$ în cel mai defavorabil caz.
- Lucrează pe un **inel bidirecțional**.
- Procesele pot detecta din ce direcție vine un mesaj și pot trimite un răspuns în acea direcție
- Operații de comunicare folosite de procese:
  - Un proces poate iniția mesaje în ambele direcții prin *sendboth*.
  - Un proces poate pasa un mesaj *(eventual modificat)* prin *sendpass*.
  - Un proces poate trimite un răspuns în direcția din care a primit un mesaj prin *sendecho*.

pdfelement

Remove Watermark Now

- Algoritmul opereaza in faze.
- In **faza k**, un proces initiaza mesaje care sunt transmise in ambele directii pe cai de lungime 2^k (1, 2, 4, 8 …)



pdfelement

Remove Watermark Now

- Procesele de pe o cale analizeaza mesajul
  - Daca un proces decide ca nu poate castiga alegerea el va pasa mesajul si nu va mai initia mesaje proprii
  - Daca procesul decide ca sursa nu poate castiga alegerea transmite un ecou negativ sursei
  - Ultimul proces din calea 2^k transmite un ecou pozitiv sursei (daca nu poate castiga alegerea)
- Un proces care primeste propriul mesaj castiga
  - in acest caz, calea “acopera” toate procesele
- El va trimite apoi un mesaj pentru a anunta celelalte procese ca a castigat.

pdfelement

Remove Watermark Now

Algoritmul Hirschberg-Sinclair (2)

Execuția pentru alegere:

```
stare = "candidate"
maxnum = 1
while (stare == "candidate“) {
    sendboth ("from", myvalue, 0, maxnum);
    await both replies (but react to other
                        messages);
    if (either reply is "no“)
        stare = "lost“;
    maxnum = 2 * maxnum;
}
```

pdfelement

Remove Watermark Now

Algoritmul Hirschberg-Sinclair (3)

La recepție mesaj ("from",value,num,maxnum):

```
if (value < myvalue)
    sendecho ("no", value)
else if (value > myvalue) {
        stare = "lost";
        num = num + 1;
        if (num < maxnum)
          sendpass ("from",value, num, maxnum)
        else
          sendecho ("ok", value)
    }
 else
     stare = "won";
```

pdfelement

Remove Watermark Now

Algoritmul Hirschberg-Sinclair (4)

La recepție mesaj ("no",value) sau ("ok",value):

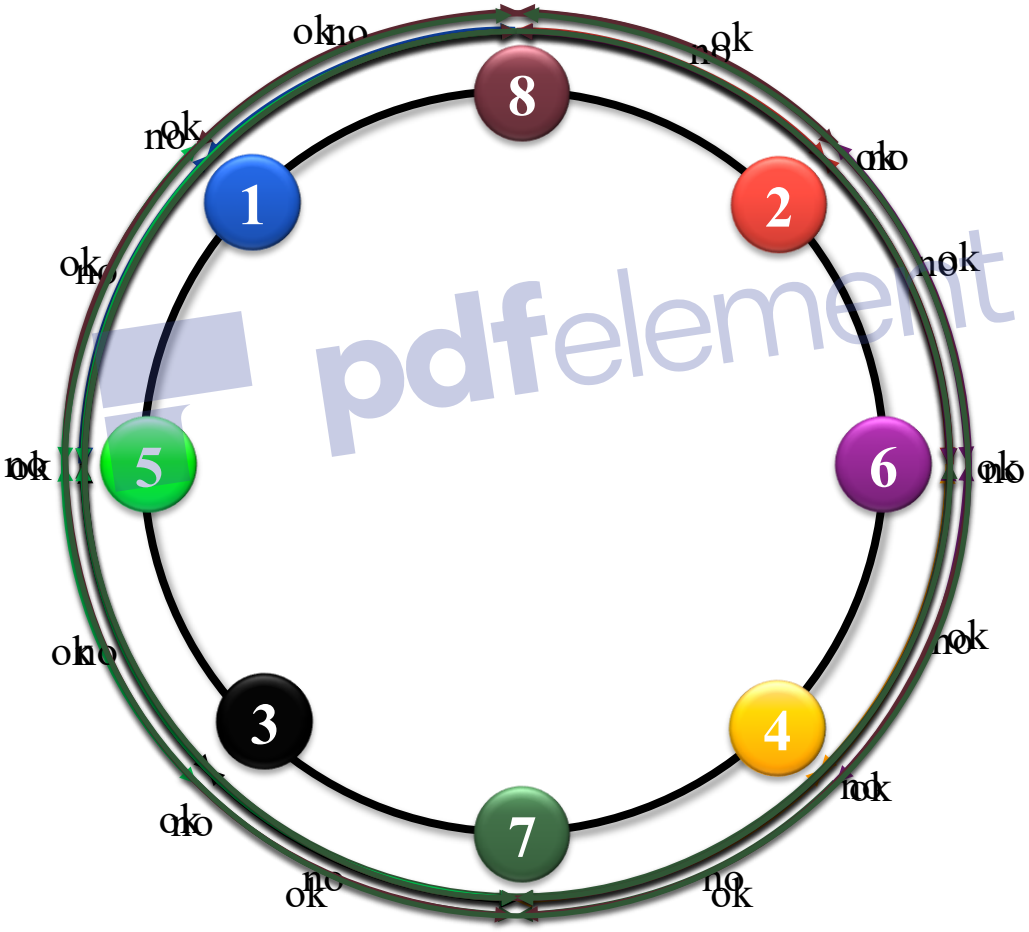
```
if (value <> myvalue) sendpass the message
else this is a reply the processor was
awaiting
```

pdfelement

- Netratat în algoritm – acțiunile unui proces neinițiator :
  - la primirea unui mesaj devine conștient de alegerea în curs
  - poate deveni *candidate* dacă identitatea are o valoare mai mare decât sursa mesajului.

Remove Watermark Now

Algoritmul Hirschberg-Sinclair (5)



pdfelement

Remove Watermark Now

Complexitate

Numar de mesaje

Un proces initiaza mesaje pe cai de lungime 2^i numai daca el a primit OK in faza pentru distanta 2^{i-1} (in oricare directie).

In orice grup de $2^{i-1} + 1$ procese consecutive, cel mult unul poate initia mesaje pe cai de lungime 2^i ➔

- toate cele n procese initiaza cai de lungime 1
- cel mult sup[n/2] procese vor initia cai de lungime 2,
- cel mult sup[n/3] de lungime 4,
- cel mult sup[n/5] de lungime 8 etc.

Pentru fiecare cale de lungime 2^i un proces declanseaza cel mult 4 mesaje (2 dus, 2 intors)

Vor fi $4*2^i$ transmiteri declansate de un proces

pdfelement

Remove Watermark Now

Complexitate

Numarul total de mesaje este cel mult

$$4*(l*n + 2*[n/2] + 4*[n/3] + 8*[n/5] + \ldots + 2^i*[n/(2^{i-1} + 1)] + \ldots).$$

- Fiecare termen (= lung cale * nr procese) mai mic de 2n
- Sunt cel mult 1+log n termeni
  - in log n faze, calea “acopera” n procese

➔ numar mesaje O(n log n)

pdfelement

Complexitate in timp

Fiecare mesaj este transmis intr-o unitate de timp

Un proces trimite in paralel in cele doua sensuri

Mesaje de la procese diferite se transmit in paralel

$$Timp = 2*(1 + 2 + 4 + 8 + \ldots + 2^i + \ldots + n)$$

Daca n este putere a lui 2 → $Timp = 2*(2^{\log n+1} -1)/(2-1) = 4*n-2$

In general Timp = O(n)

Remove Watermark Now

Sumar

- Alegerea unui lider cu algoritmi undă
- Algoritmul tree
- Alegerea liderului – topologie inel
- Algoritmul LeLann
- Algoritmul LeLann-Chang-Robert
- Algoritmul Hirschberg-Sinclair

pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Stabilirea Topologiei

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Difuzarea mesajelor folosind sondaje

```
typedef tip_arb int[1:N,1:N];
chan sondaj[1:N](tip_arb,tip_mesaj);
const int inițiator = indexul nodului inițiator;

process Nod[p=1 to N]{
   tip_arb arb, T;
   tip_mesaj m;
   bool top[1:N,1:N];

   if (inițiator == p) {
      calculeaza T pe baza top;
      m = mesaj_de_transmis
   } else
      receive sondaj[p](arb,m)

   for [q = 1 to N st q este fiul lui p in arb]
      send sondaj[q](arb,m);
}
```

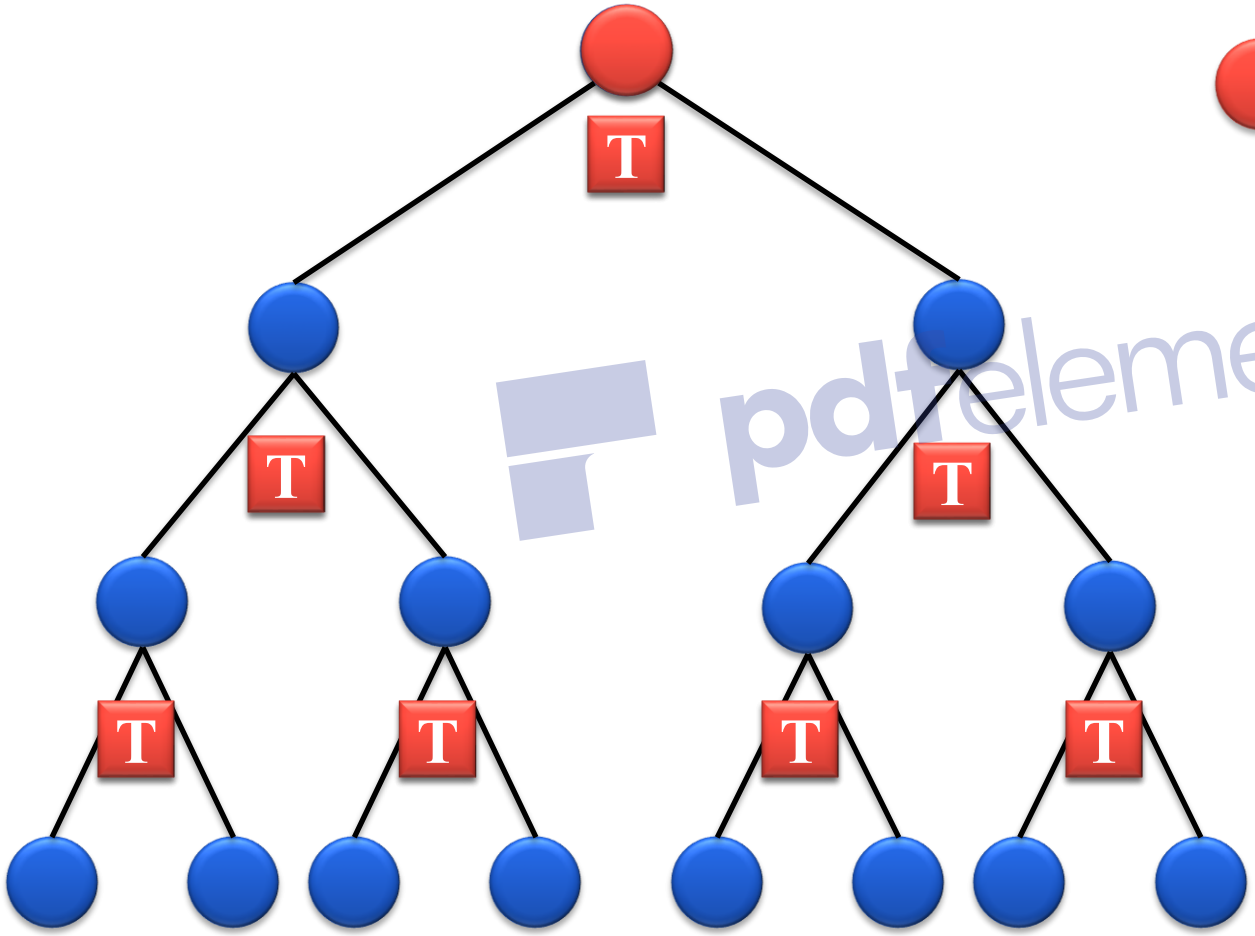
top este o matrice cu topologia arborelui

Numărul de mesaje = $N - 1$

pdfelement

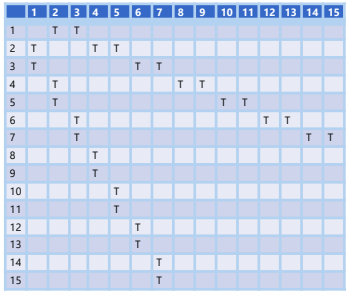
Remove Watermark Now

Difuzarea mesajelor folosind sondaje



Inițiator

Matrice de adiacență



pdfelement

Remove Watermark Now

Difuzarea mesajelor prin inundare

```
chan sondaj[1:n](tip_mesaj);
const int inițiator = indexul nodului inițiator;

process Nod[p=1 to n]{
    bool leg[1:n] = vecinii_lui_p;
    int num = numărul_vecinilor, răspunsuri = num;
    tip_mesaj m;

    if (p <> inițiator) {
        receive sondaj[p](m);
        răspunsuri = răspunsuri - 1;
    else
        m = mesaj_de_transmis;

    for [q = 1 to n st leg[q]]
        send sondaj[q](m)

    for [q = 1 to răspunsuri]
        receive sondaj[p](m)
}
```

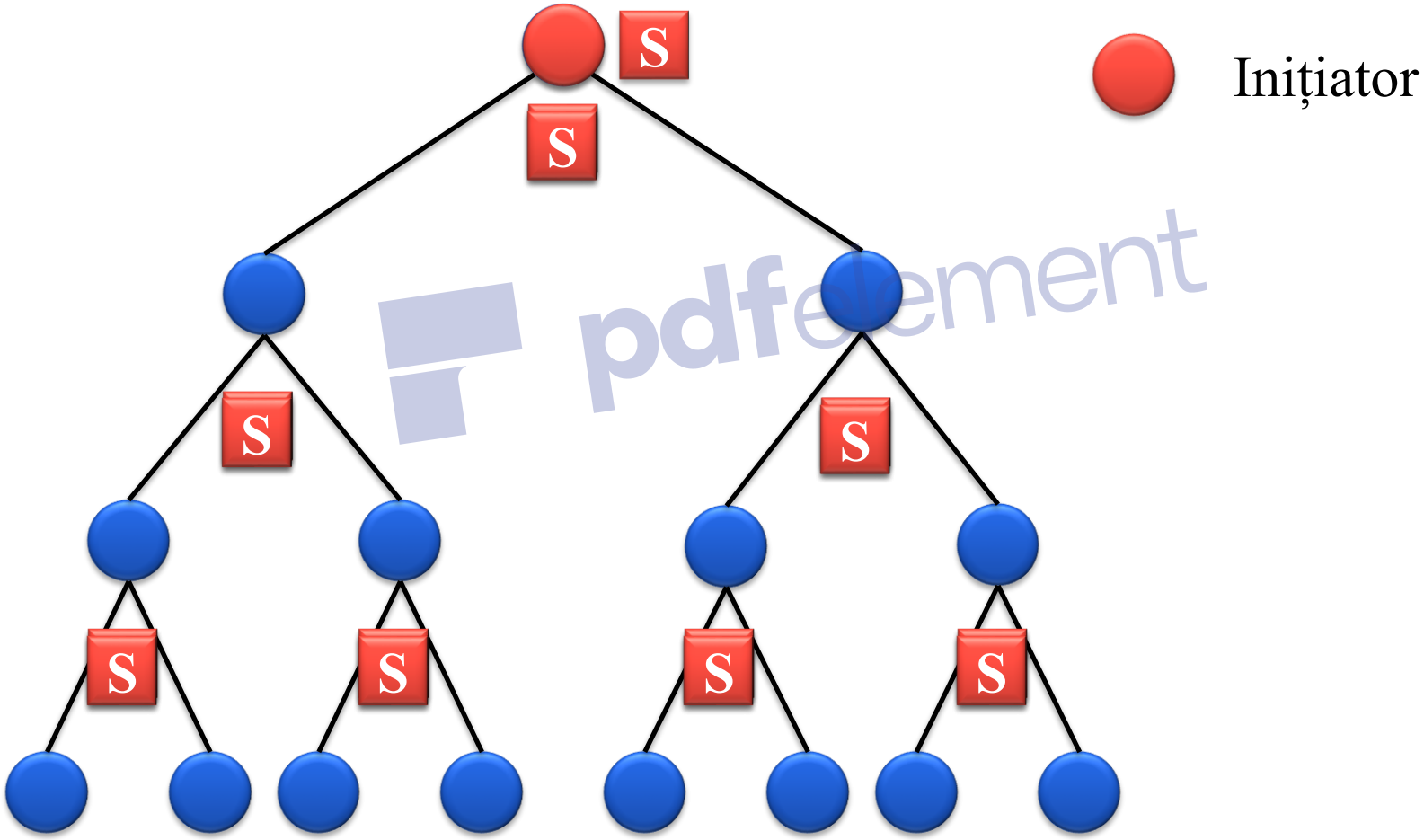
pdfelement

Numărul de mesaje = $2m$

(m = numărul legăturilor)

Remove Watermark Now

Difuzarea mesajelor prin inundare



pdfelement

Remove Watermark Now

Stabilirea topologiei unei reţele de procese

Definiţii

- Colecție de procese **Nod(p:1..N)**
- Vecinii nodului p **leg[1:N]:**
  - $leg[q] = TRUE$ dacă q este vecin cu p,
  - altfel $leg[q] = FALSE$.
- Relație simetrică:
  - pentru p, $leg[q] = TRUE \leftrightarrow$ pentru q, $leg[p] = TRUE$

pdfelement

Problema

- Topologia reprezentată prin matricea de adiacenţe:
  - $top[i,j] = TRUE$, dacă i este vecin cu j
  - $top[i,j] = FALSE$, în caz contrar.
- Calculată pe baza cunoştinţelor locale
  - pentru orice p,q: $1 \leq p,q \leq N : top[p,q] = leg_p[q]$

Remove Watermark Now

Algoritmul pulsatiilor

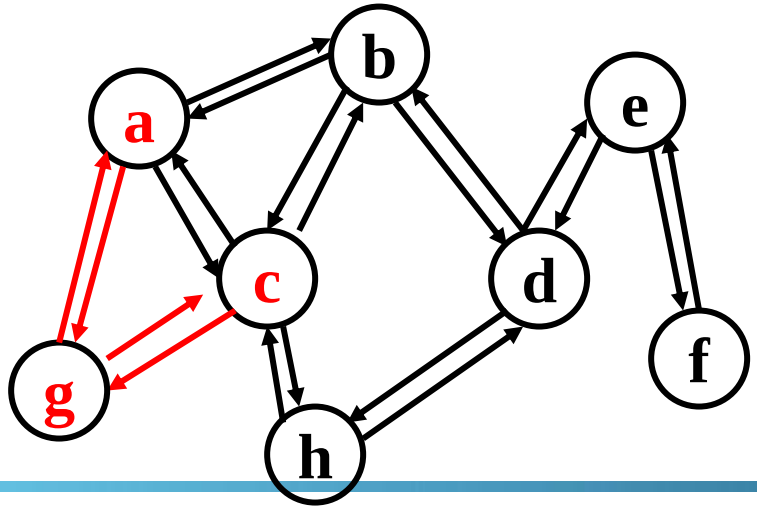
pdfelement

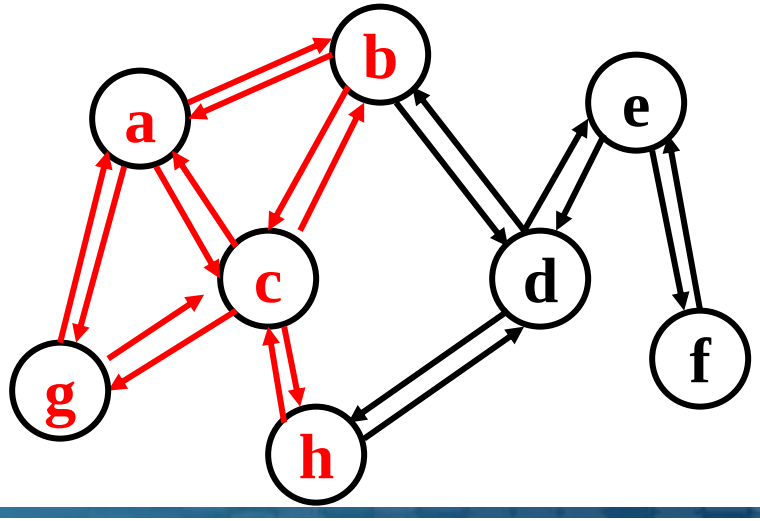
Remove Watermark Now

Algoritmul pulsaţiilor

- Fiecare proces calculează singur topologia, folosind informaţiile provenite de la vecini
- Fiecare nod transmite vecinilor matricea sa de adiacenţă, top şi primeşte matricea de adiacenţă a fiecărui vecin
  - după un rund complet, fiecare nod va dispune de informaţii de la nodurile vecine (figura stg. pentru nodul g)
  - După două runde complete, orice nod va avea informaţii de la noduri aflate la distanţa 2 (figura dr.), etc.
  - Dupa r runde sunt completate liniile din top corespunzătoare nodurilor q aflate la o distanţă <= r

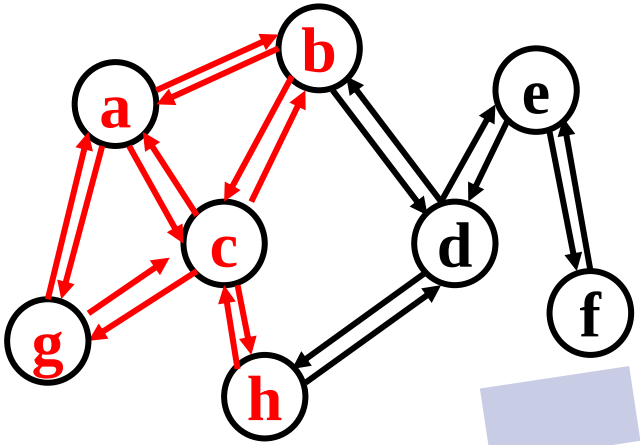
pdfelement

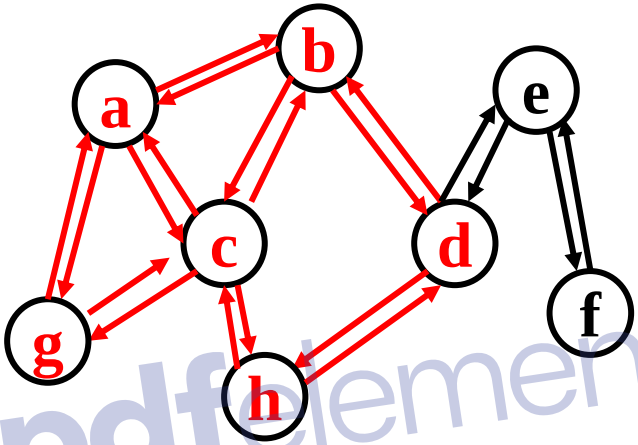


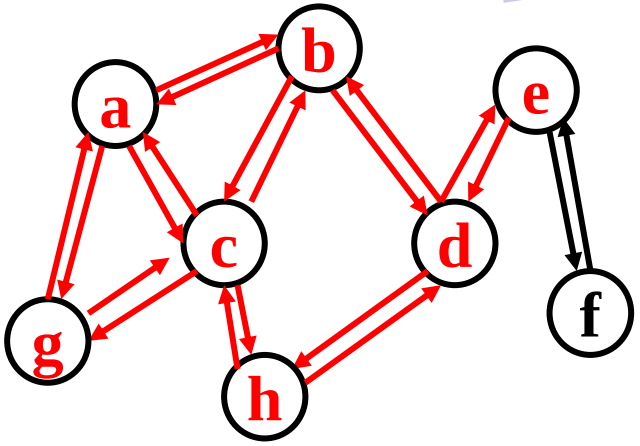


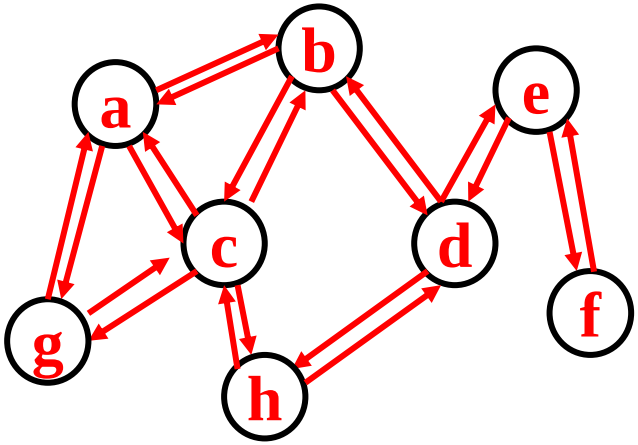
Remove Watermark Now

Algoritmul pulsaţiilor









Următorul predicat este satisfăcut pentru fiecare proces p:

RUND: oricare ar fi q cu 1<=q<=N: dist(p,q) <=r → top[q,*] este completat

dist(p,q) este distanţa de la p la q, adică lungimea căii celei mai scurte între cele două noduri.

pdfelement

Remove Watermark Now

Algoritmul pulsaţiilor

```
typedef tip_top bool[1:N,1:N];
typedef tip_leg bool[1:N];
chan topologie[1:N](tip_top);

process Nod[p=1 to n]{ tip_leg leg = vecinii_lui_p;
  int r = 0;
  tip_top top = ([N*N]FALSE), top_nou;
  top[p,1:N] = leg;
  /* actualizează liniile vecinilor */
  while (r < D) {
    /* transmite topologia curentă tuturor vecinilor */
    for [q = 1 to N st leg[q]]
      send topologie[q](top)
    for [q = 1 to n st leg[q]]{
      receive topologie[p](top_nou);
      top = top or top_nou;
    }
    r = r + 1;
  }
}
```

D = diametrul rețelei

pdfelement

Remove Watermark Now

Algoritmul pulsaţiilor (2)

- Neajunsuri:
  - În general, diametrul rețelei nu este cunoscut
  - Traficul inutil
- Rețea conexă
  - un nod cunoaşte întreaga topologie dacă fiecare linie ***top*** are cel puţin un element TRUE ⇒ mai execută un rund și poate termina
  - evitare blocare ⇒ un proces comunică doar cu vecinii care nu au terminat

pdfelement

Remove Watermark Now

Algoritmul pulsaţiilor (3)

```
chan topo[1:N](tip_top);
process Nod[p=1 to N] {
    tip_leg leg = vecinii_lui_p;
    tip_leg activ = leg;                    /* vecinii
   activi */
    tip_top top = ([N * N]FALSE);
    bool gata = FALSE;
    int transm;
    bool qgata;
    tip_top top_nou;

    /* actualizează vecinii lui p */
    top[p, 1:N] = leg;
```

pdfelement

Remove Watermark Now

Algoritmul pulsaţiilor (4)

```
while (NOT gata) {
      /* transmite topologia curentă tuturor
vecinilor */
      for [q = 1 to N st leg[q]]
         send topologie[q](p, FALSE, top)
      /* recepționează topologiile de la vecini */
      for [q = 1 to N st leg[q]] {
         receive topologie[p](transm, qgata,
top_nou);
         top = top OR top_nou;
         if (qgata) activ[transm] = FALSE;
      }
      if (toate liniile din top au un element TRUE)
         gata = TRUE;
}

/* transmite top vecinilor activi
   în ultimul rund */
for [q = 1 to N st activ[q]]
    send topologie[q](p,TRUE,top)
```

se executa D+1 runde

numar mesaje nu depaseste:

Numărul de mesaje =

$2m(D+1)N$

m = numărul legăturilor

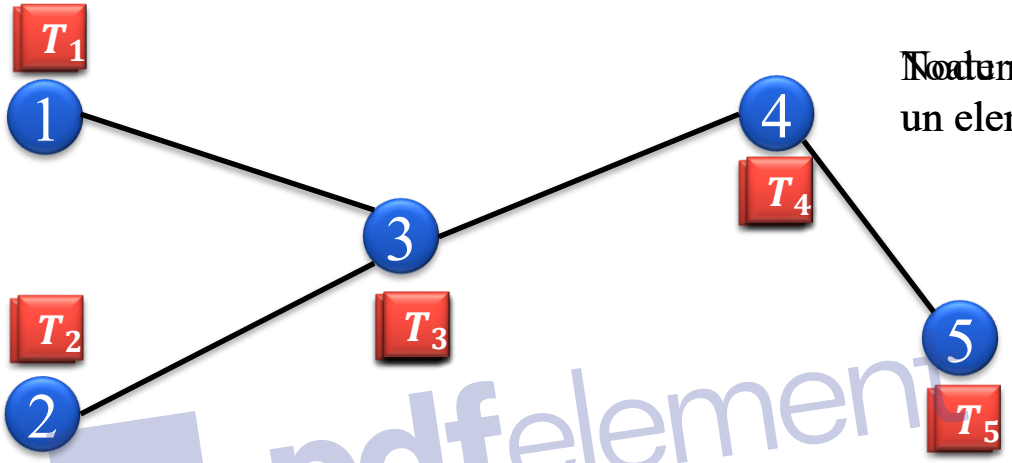
N = numărul de noduri

D = diametrul rețelei

pdfelement

Remove Watermark Now

Algoritmul pulsaţiilor (5)



Nodurile 3 și 4 au cel puțin un element ***true*** pe fiecare linie

Sau logic între tabela de adiacență proprie și cele primite de la vecini

$topologie[1..N]$

1	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

2	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

3	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

4	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

5	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

1	1	2	3	4	5
			F		

2	1	2	3	4	5
			F		

3	1	2	3	4	5
	T	T		F	

4	1	2	3	4	5
			F		T

5	1	2	3	4	5
				F	

$activ[1..N]$

pdfelement

Remove Watermark Now

Algoritmi cu mesaje de sondaj cu ecou

pdfelement

Remove Watermark Now

Algoritmi cu mesaje de sondaj cu ecou

- Ideea :
  - un **Iniţiator** colectează informaţii despre topologia locală a tuturor celorlalte noduri, determină topologia întregii reţele şi o difuzează nodurilor.
- **Cazul topologiilor arbore ⇒ sursa (initiatorul) este rădacina arborelui**
  - Două faze :
    - mesajele de sondaj se propagă de la inițiator spre frunze
    - mesajele de ecou *(care conțin topologii parțiale)* se propagă de la frunze spre inițiator.

pdfelement

Remove Watermark Now

Algoritmi cu mesaje de sondaj cu ecou (2)

```
const int inițiator = indexul nodului
inițiator;

typedef tip_leg bool[1:N];
typedef tip_top bool[1:N, 1:N];

chan sondaj[1:N](int transm);
chan ecou[1:M]  (tip_top top);
chan ecou_final (tip_top top);

process Nod[p=1 to N]{
    tip_leg leg = vecinii_lui_p;
    tip_top top = ([N * N] FALSE);
    int părinte = N + 1;
    tip_top top_nou;

    top[p, 1:n] = leg;
```

pdfelement

Remove Watermark Now

Algoritmi cu mesaje de sondaj cu ecou (3)

```
if (p <> inițiator)
    receive sondaj[p](părinte);

/* transmite sondaje celorlalte noduri vecine, copiii
lui p */
for [q = 1 to N st (leg[q] AND (q <> părinte))]
    send sondaj[q](p);


/* primește ecourile și fă reuniunea lor */
for [q = 1 to N st (leg[q] AND (q <> părinte))] {
    receive ecou[p](top_nou);
    top = top OR top_nou;
}


if (p <> inițiator)
    send ecou[părinte](top);
}
```

pdfelement

Remove Watermark Now

Algoritmi cu mesaje de sondaj cu ecou (4)

- **Cazul general**
  - după ce primeşte un prim mesaj de sondaj, un nod îl propagă tuturor celorlalţi vecini
  - nodul aşteaptă ecouri de la vecini
  - **există cicluri** $\Longrightarrow$ nodul poate primi și sondaje
    - topologia locală va fi transmisă doar ca ecou la primul sondaj
    - pentru restul sondajelor se transmite o topologie nulă
    - sondajele și ecourile trebuie recepţionate pe acelaşi canal

pdfelement

Remove Watermark Now

Algoritmi cu mesaje de sondaj cu ecou (5)

```
/* index inițiator */
const inițiator = indexul nodului inițiator;

enum fel{sondă, ecou};
typedef tip_leg bool[1:N];
typedef tip_top bool[1:N, 1:N];

chan sondă-ecou[1:N](fel tip_mesaj, int transm, tip_top
top);
chan ecou_final(tip_top top);

process Nod[p=1 to N]{
    tip_leg leg = vecinii_lui_p;
    tip_top top = ([N * N] FALSE);
    int părinte = n + 1;
    tip_top top_nou;
```

pdfelement

Remove Watermark Now

Algoritmi cu mesaje de sondaj cu ecou (6)

```
/* actualizează vecinii lui p */
top[p, 1:N] = leg;

fel k; int transm;
int nr_ecouri = 0;

if (p <> inițiator)
   receive sondă-ecou[p](k, părinte, top_nou);

/* transmite sondaje celorlalte noduri vecine, copiii lui p
*/
for [q = 1 to N st (leg[q] AND (q <> părinte))] {
   send sondă-ecou[q](sondă, p, O);  /* topologie nulă */
   nr_ecouri = nr_ecouri + 1;
}
```

pdfelement

Remove Watermark Now

Algoritmi cu mesaje de sondaj cu ecou (7)

```
   /* primește ecourile și fă reuniunea lor, sau sonde și
ignoră */
   while (nr_ecouri > 0) {
      receive sondă-ecou[p](k, transm, top_nou);

      if (k == sondă)
         send sondă-ecou[transm](ecou, p, 0);
      else if (k == ecou) {
         top = top OR top_nou;
         nr_ecouri = nr_ecouri - 1;
      }
   }

   if (p <> inițiator)
      send sondă-ecou[părinte](ecou, p, top);
```

pdfelement

Remove Watermark Now

Complexitate

Numar mesaje

Fiecare legătură (aparţinând arborelui de acoperire) corespunzătoare primei sonde poartă două mesaje (sonda şi ecoul)

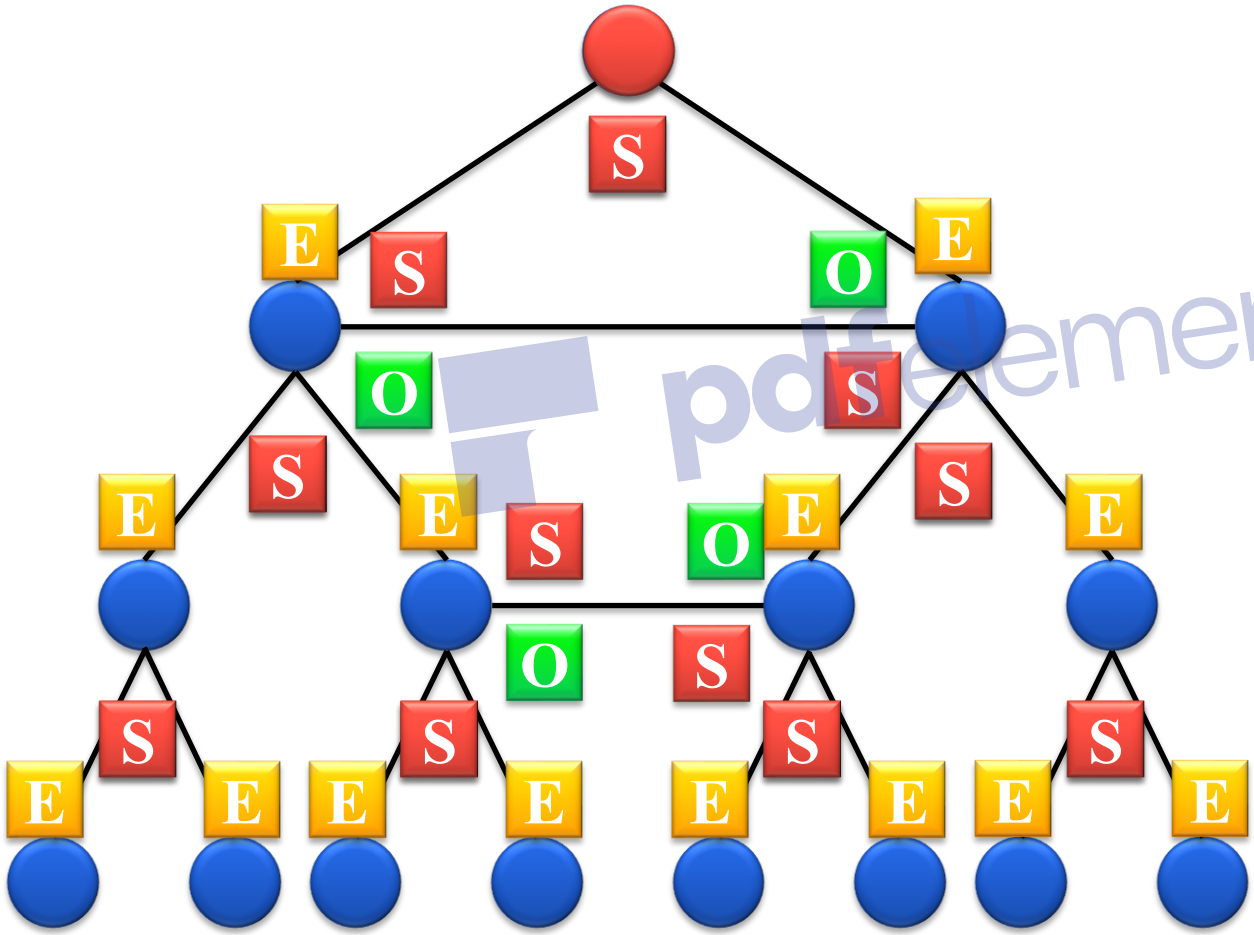
Celelalte, câte patru (două sonde şi două ecouri)

Transmiterea topologiei de la iniţiator la noduri necesită alte N mesaje.

pdfelement

Remove Watermark Now

Algoritmi cu mesaje de sondaj cu ecou (8)



pdfelement

Remove Watermark Now

Sumar

- Difuzarea mesajelor prin inundare
- Difuzarea mesajelor folosind sondaje
- Problematica stabilirii topologiei
- Algoritmul pulsațiilor
- Algoritmi cu mesaje de sondaj cu ecou

pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Terminarea programelor distribuite

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

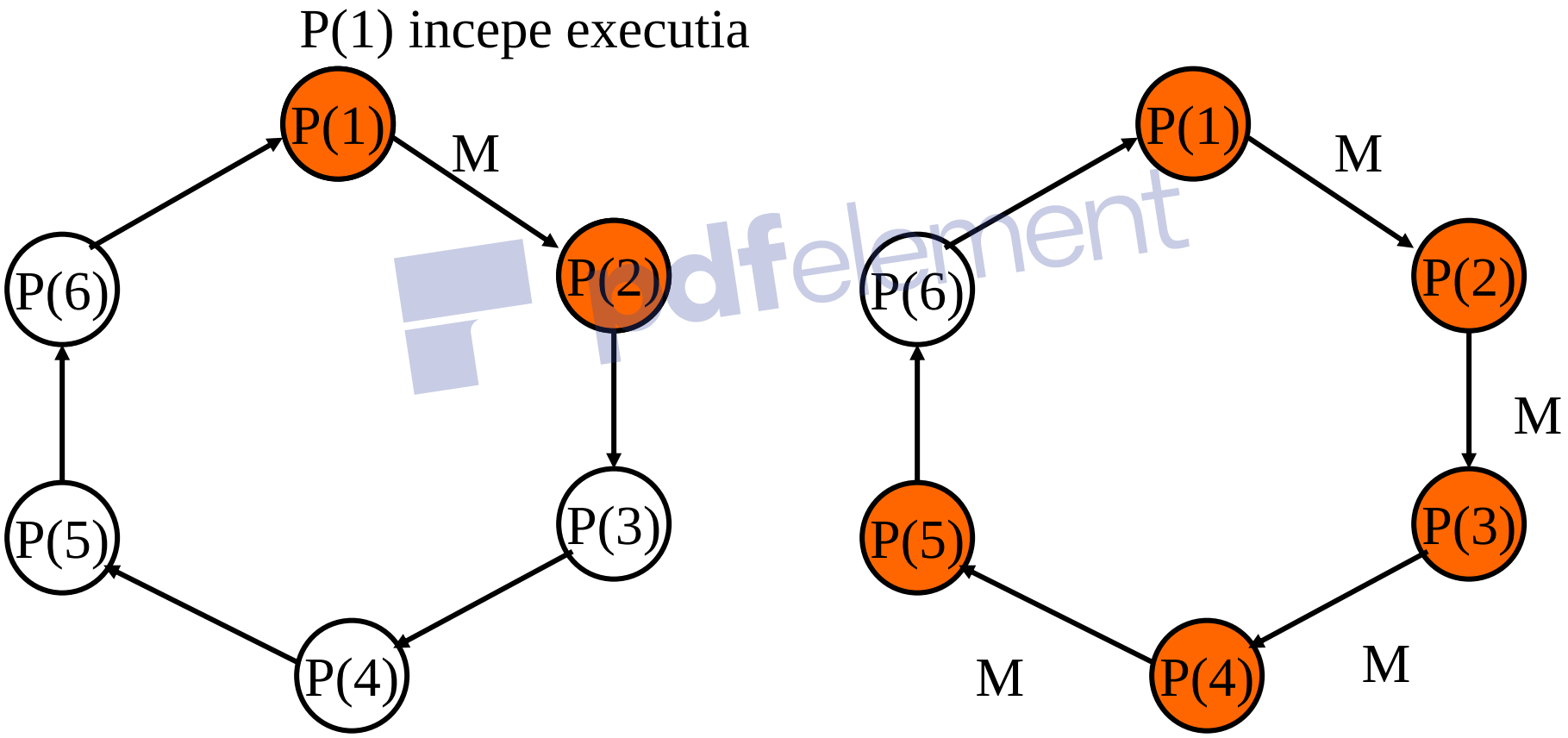
Terminarea programelor distribuite

- **Problema:** detecția terminării
- Algoritmi paraleli:
  - toate procesele se termină
    - cicluri infinite?
  - procese terminate sau blocate și nu există I/E în curs
  - simplu pentru un singur procesor - coada *ready* goală
- Algoritmi distribuiți:
  - fiecare proces să fie liber
    - a terminat execuția *sau*
    - se află în aşteptarea unui mesaj (**receive**)
  - pe canale să nu existe mesaje în tranzit

pdfelement

Remove Watermark Now

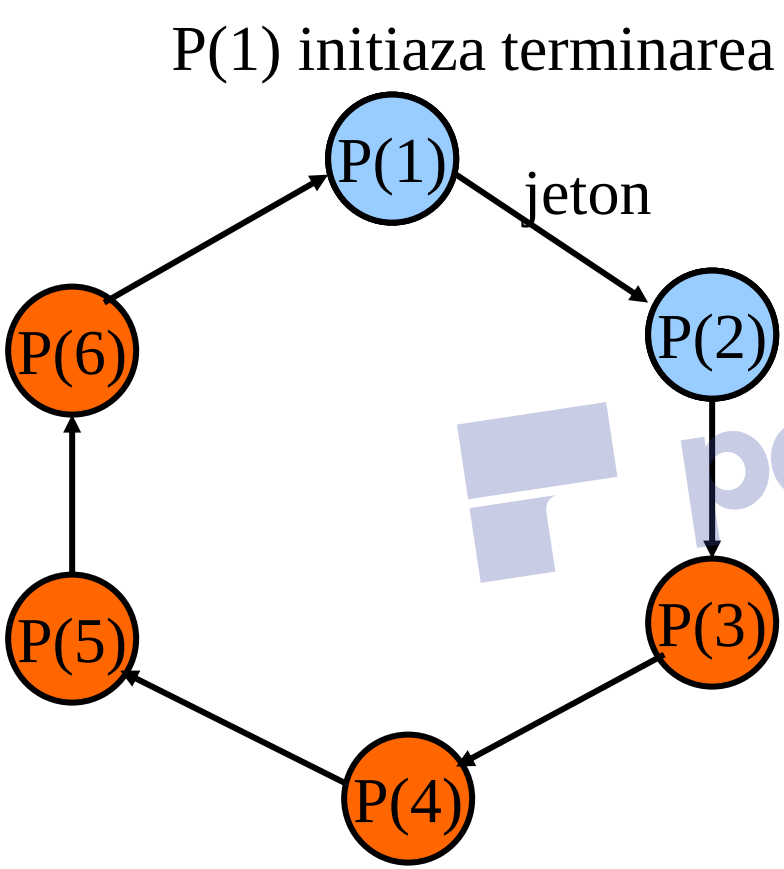
Procese organizate în inel

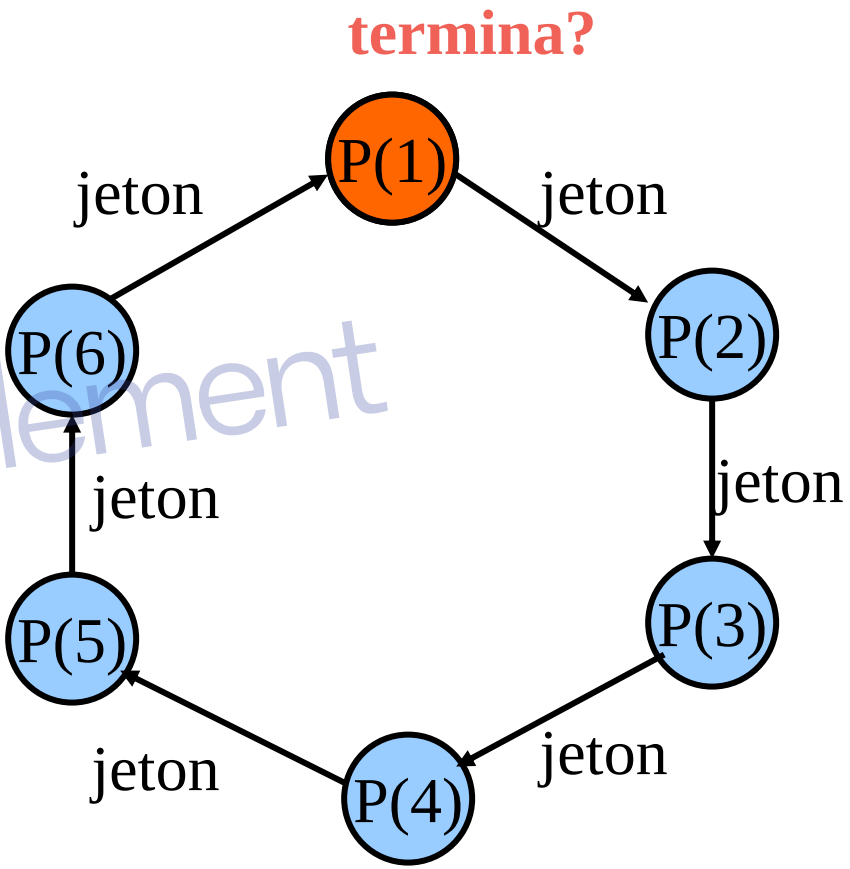


pdfelement

Remove Watermark Now

Procese organizate în inel





pdfelement

inainte de a primi jeton, P(1) poate fi reactivat de un mesaj de la P(6)!

Remove Watermark Now

Procese organizate în inel (tehnica jetoanelor) (1)

- Inițiatorul terminării este procesul P[1]
- acțiunile lui P[1] cînd devine liber prima dată:

```
culoare[1] = albastru;
jeton = 0;
send ch[2](jeton);
```

- acțiunile lui P[i=1 to N] la recepția unui mesaj obişnuit:

```
culoare[i] = roșu;
```

pdfelement

Remove Watermark Now

Tehnica jetoanelor (2)

- acțiunile lui P[i=2 to N] la primirea unui jeton (și după ce devine liber):

```
culoare[i] = albastru;
jeton = jeton +1;    /* nesemnificativ */
send ch[(i + 1) mod n](jeton);
```

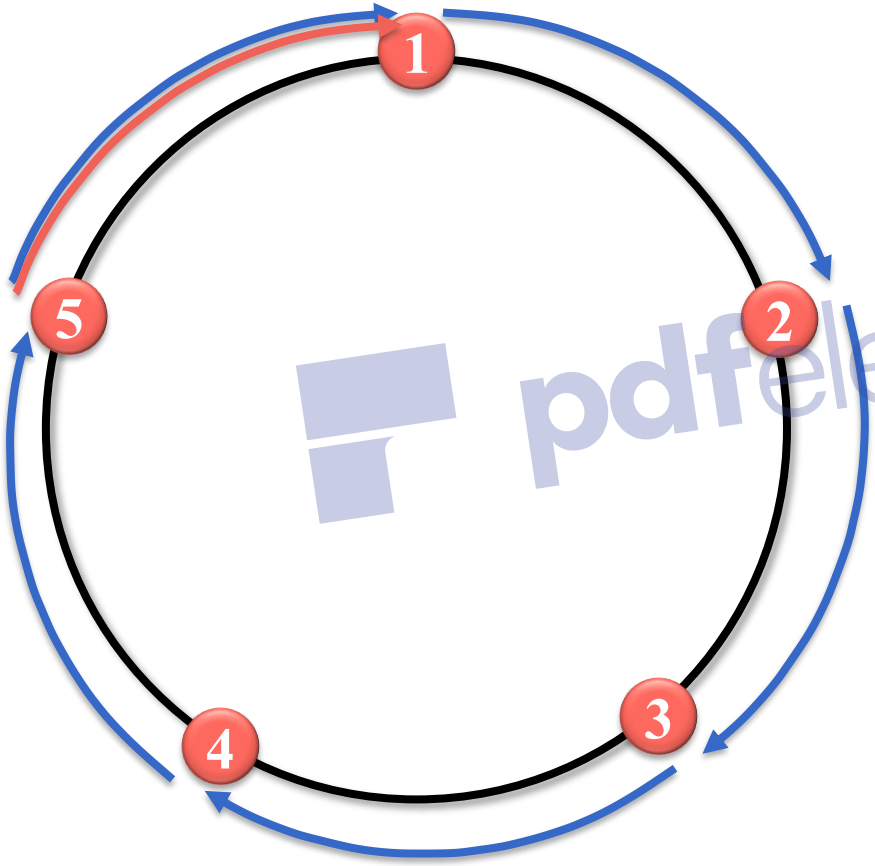
- acțiunile lui P(1) la recepția jetonului:

```
if (culoare[1] == albastru) {
   anunță terminarea; stop
}
culoare[1] = albastru;
jeton = 0;
send ch[2](jeton);
```

pdfelement

Remove Watermark Now

Tehnica jetoanelor (3)



Jeton
1

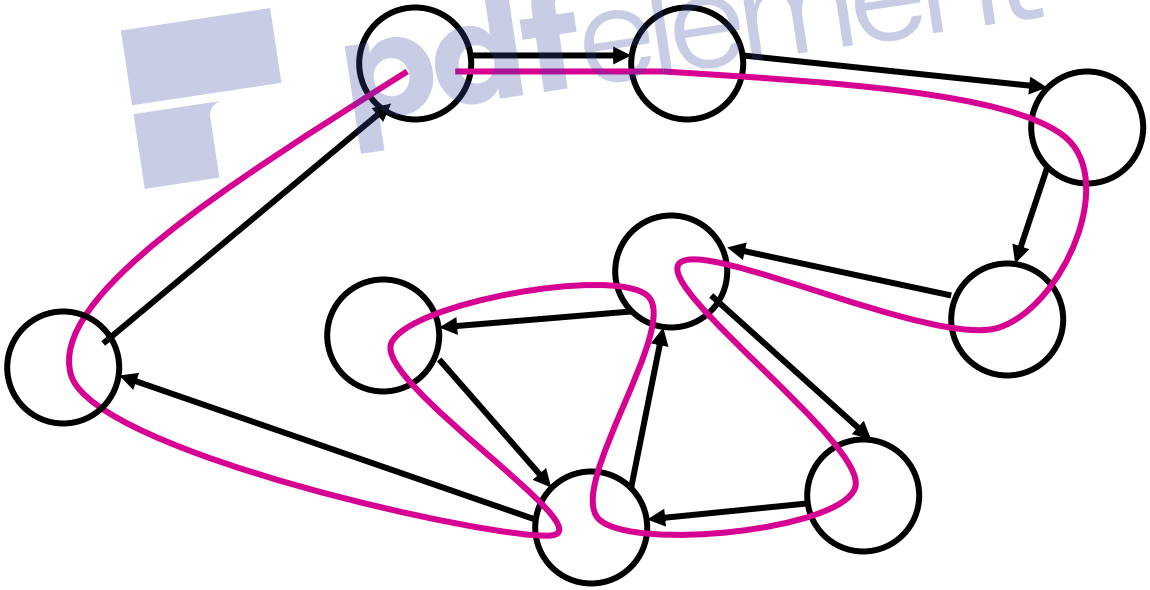
STOP

pdfelement

Remove Watermark Now

Terminarea în cazul general (tehnica jetoanelor)(1)

- topologie generală de **graf**
- **c** un ciclu care include toate arcele grafului; fie **nc** lungimea lui



pdfelement

Remove Watermark Now

Terminarea în cazul general (tehnica jetoanelor)(2)

- un jeton care se transmite după mesajele de date
- are inițial valoarea 0
- **acțiunile lui P[i=1 to N] la primirea unui mesaj uzual:**

```
culoare[i] = roșu;
```

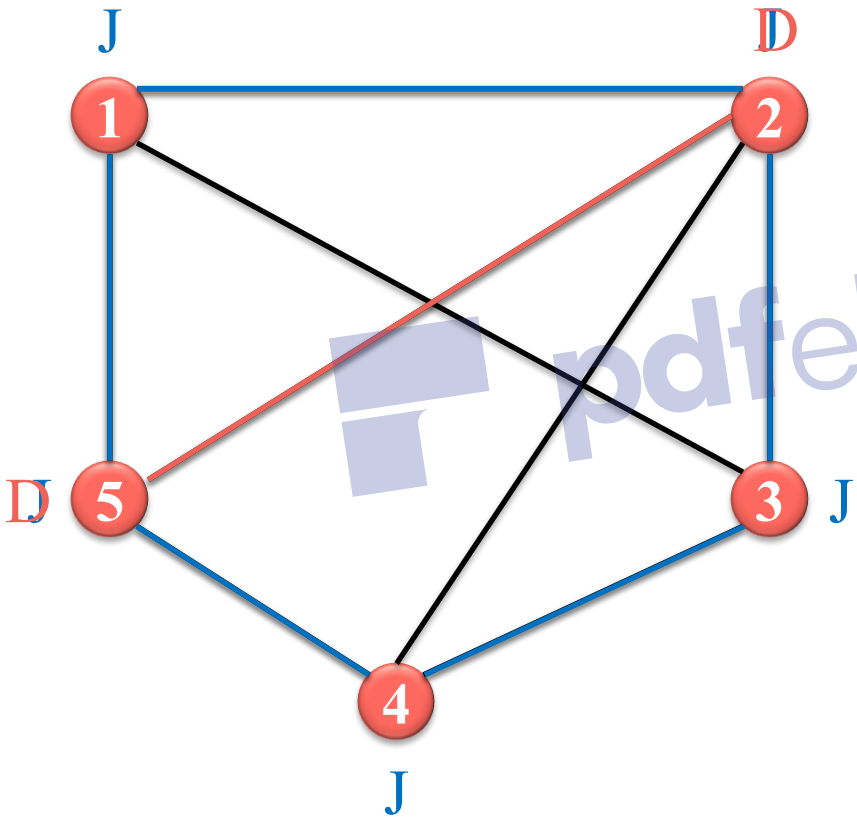
- **acțiunile lui P[i=1 to N] la recepția jetonului:**

```
if (jeton == nc) { anunță terminarea; stop }
if (culoare[i] == roșu) { culoare[i] = albastru;
                          jeton = 0; }
else if (culoare[i] == albastru) jeton = jeton +1;

setează j la canalul următor din ciclul c;
send ch[j](jeton);
```

pdfelement

Remove Watermark Now

Terminarea în cazul general (tehnica jetoanelor)(3)



Jeton
1

STOP

nc = 5

pdfelement

Remove Watermark Now

Terminare (topologii arbore)

- Tehnica confirmarii mesajelor -

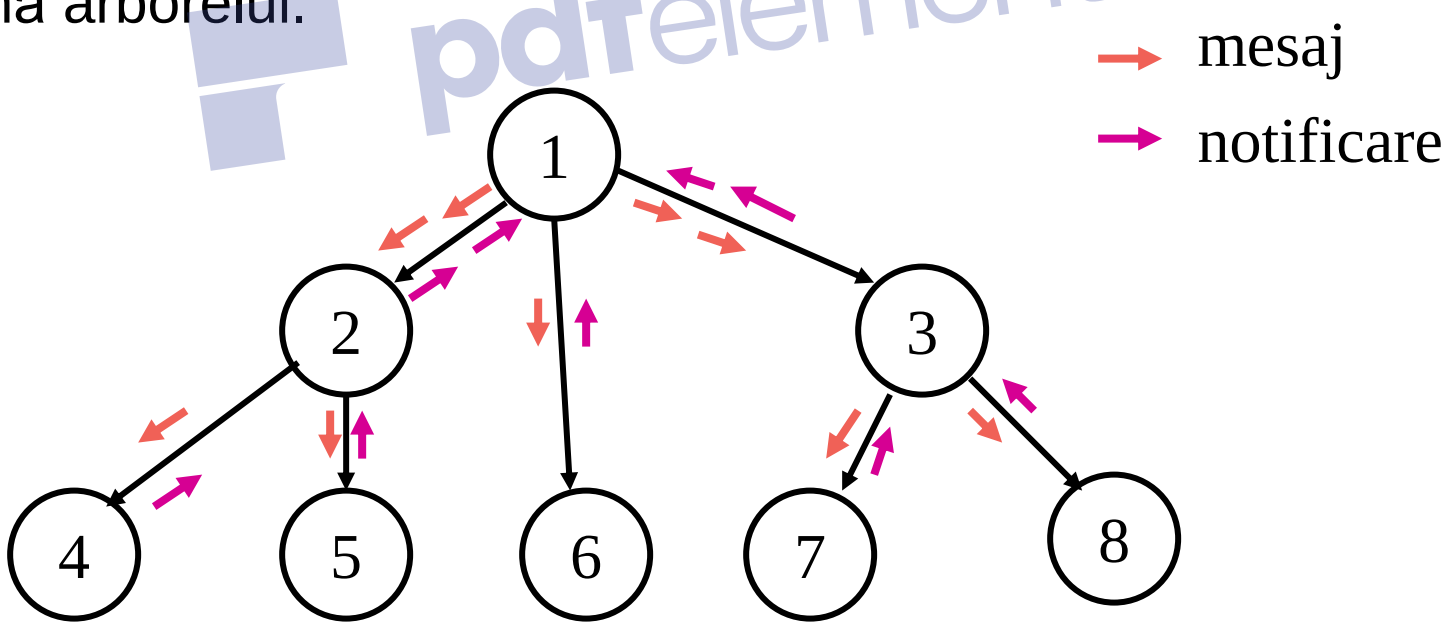
pdfelement

Remove Watermark Now

Terminarea in topologii arbore

- Când un proces frunză se termină, el anunţă părintele său.
- Când un nod oarecare al arborelui se termină, el aşteaptă notificarea terminării de la toţi fiii săi şi apoi îşi anunţă părintele.
- Notificările se propagă astfel pînă la procesul sursa, aflat în rădăcina arborelui.

pdfelement



Remove Watermark Now

Confirmarea mesajelor (1)

- Funcția de determinare a stării canalelor se poate împărți între procesele programului: fiecare proces verifică canalele pe care a trimis mesaje de date altor procese (canalele sale de ieşire).

- Pe un canal
  - se trimit **mesaje de date** și se primesc **semnale de confirmare** *sau*
  - se primesc **mesaje de date** și se trimit **semnale de confirmare**

- **Confirmarea mesajelor**: dacă pentru fiecare mesaj de date transmis pe un canal procesul primeşte de la receptor un semnal de confirmare, atunci canalul respectiv este gol.

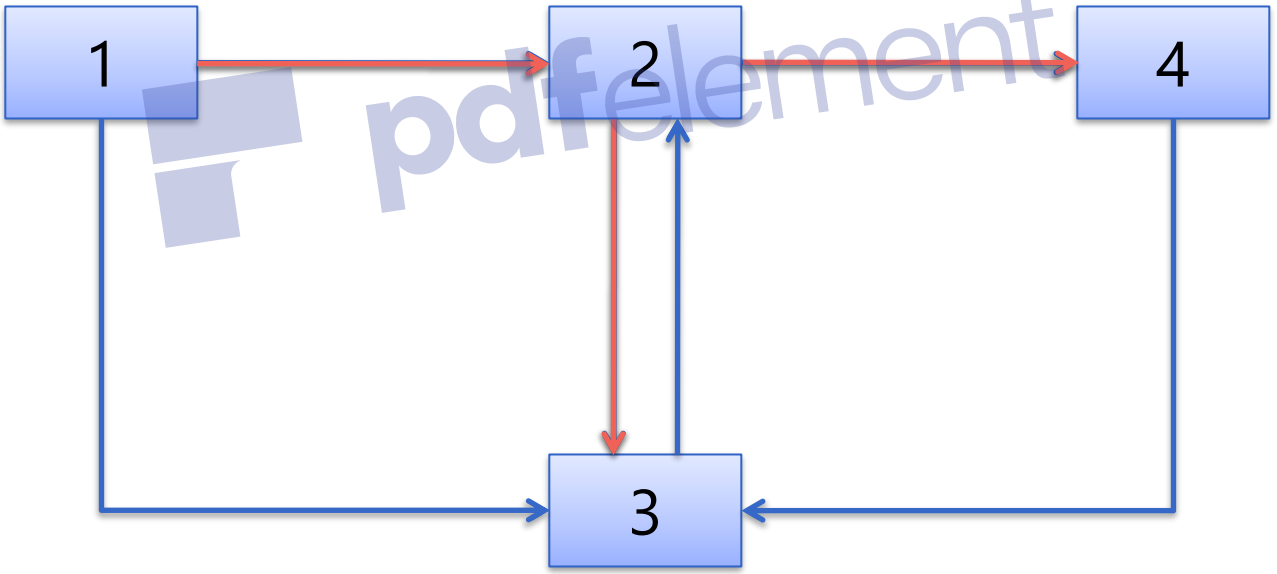
- Presupune existența unui proces **sursă**:
  - care nu are legături de intrare *şi*
  - poate accesa orice alt proces pe o cale oarecare din graf

pdfelement

Remove Watermark Now

Confirmarea mesajelor (2)

- Exemplu:
  - Procesul sursă: 1
  - Arbore de acoperire



pdfelement

Remove Watermark Now

Algoritmul Dijkstra-Scholten (1)

pdfelement

Remove Watermark Now

Volume 11, number 1 INFORMATION PROCESSING LETTERS 29 August 1980

TERMINATION DETECTION FOR DIFFUSING COMPUTATIONS *

Edsger W. DIJKSTRA
Burroughs, 5671 AL Nuenen, The Netherlands

C.S. SCHOLTEN
Philips Research Laboratories, 5656 AA Eindhoven, The Netherlands

Received 2 June 1980

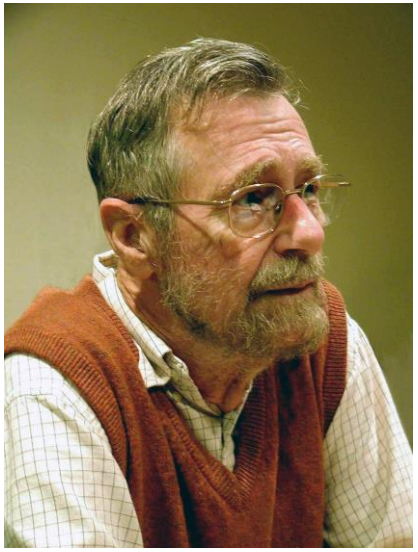
Concurrency, termination detection, distributed control, separation of concerns, message-based systems, correctness proving, diffusing computations, networks, cornets, activation tree, nondeterminacy

pdfelement

The following seems to capture the quintessence of a situation that is not unusual in distributed processing. Consider a finite, directed graph. If the graph contains an edge from node A to node B, we call B 'a successor of A', and A 'a predecessor of B'. We assume the existence of a node without incoming edges; this node will be called 'the environment' (because it acts as such with respect to the rest of the graph). The other nodes will be called 'the internal nodes'.

For each node its initial state will be called 'the neutral state'. A so-called 'diffusing computation' is started when the environment sends – of its own accord, so to speak – a message to one or more of its successors; it is supposed to do this just once. After reception of its first message, an internal node is free to send messages to its successors. It is this feature

proper – such that, when the diffusing computation proper has thus terminated, the fact of this completion will eventually be signalled back to the environment. Besides a node's ability to receive messages from its predecessors and to send messages to its successors, we assume each node also to be able to receive 'signals' from its successors and to send 'signals' to its predecessors; in other words, each edge is assumed to be able to accommodate two-way traffic, but only messages of the computation proper in the one direction and signals in the opposite direction. We shall impose that in the total computation – i.e. from the moment that the environment sent its messages to the rest of the graph until it has received the completion signal – each edge will have carried as many messages in the one direction as it has carried signals in the op-



Dijkstra, Edsger W., and Carel S. Scholten. "Termination detection for diffusing computations." *Information Processing Letters* 11.1 (1980): 1-4.

Remove Watermark Now

Algoritmul Dijkstra-Scholten (1)

- Bazat pe transmiterea unor semnale de confirmare
- Cazul unei rețele cu topologie **arborescentă**:
  - Când un proces *frunză* se termină, el îl anunță pe *părintele* său.
  - Când un *nod* oarecare al arborelui termină, el aşteaptă notificarea terminării de la toți *fiii* săi și apoi își anunță *părintele*.
  - Notificările se propagă până la procesul sursă.
- Cazul unui **graf dirijat aciclic** de procese
  - Deficitul unei legături = diferența dintre numărul de mesaje de date transmise şi numărul de semnale de confirmare primite pe acea legătură
  - Când un nod doreşte să termine:
    - aşteaptă primirea unor semnale pe legăturile de ieşire a datelor, semnale care reduc deficitele la zero (pe acele legături).
    - trimite pe fiecare legătură de intrare a datelor un număr de semnale egal cu deficitul (legăturii respective).

parelement

Remove Watermark Now

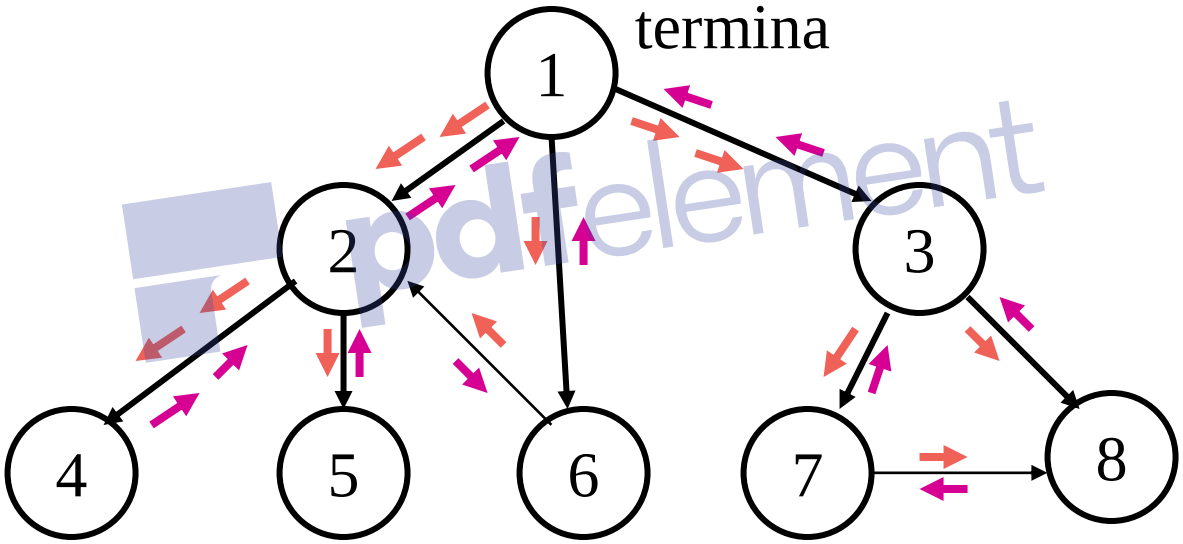
Algoritmul Dijkstra-Scholten (2)

- **Cazul general**: cicluri în graful proceselor
- Rezolvare: generare **arbore de acoperire**:
  - fiecare legătură generată la prima recepție (adică prima legătură de intrare, numită **prim** – de la părinte)
- Pentru terminare:
  - trimitere semnale pentru toate legăturile de intrare, mai puțin **prim**
  - recepție semnale pentru toate legăturile de ieşire
  - trimitere semnale spre **prim**.
- Schema de semnalare este separată și suplimentară față de prelucrarea propriu-zisă

pdfelement

Remove Watermark Now

Algoritmul Dijkstra-Scholten



pdfelement

Remove Watermark Now

Algoritmul Dijkstra-Scholten (3)

LORDUL

pdfelement

SERVITORUL

Remove Watermark Now

Algoritmul Dijkstra-Scholten (4)

- Comunică doar cu alți lorzi și cu propriul servitor
- Comunică cu lorzii prin scrisori (prin poștă) și cu servitorul prin semnale de comunicare (la telefon)
- Își anunță servitorul când trimite/primește o scrisoare (spunându-i și destinația/sursa)
- După ce se plictisește întreabă servitorul dacă a terminat treaba ca să poată merge la vânătoare

pdfelement

- Comunică doar cu alți servitori și cu propriul lord
- Comunică doar prin semnale (la telefon)
- Trimite semnale de confirmare pentru scrisorile primite și așteaptă semnale de confirmare pentru scrisorile trimise
  - Scrisoarea primită de la lordul$_i$ se confirmă servitorului$_i$
- Răspunde afirmativ doar dacă a primit și trimis toate confirmările pentru mesajele trimise și primite, altfel îl roagă pe lord să aștepte

Remove Watermark Now

Algoritmul Dijkstra-Scholten (5)

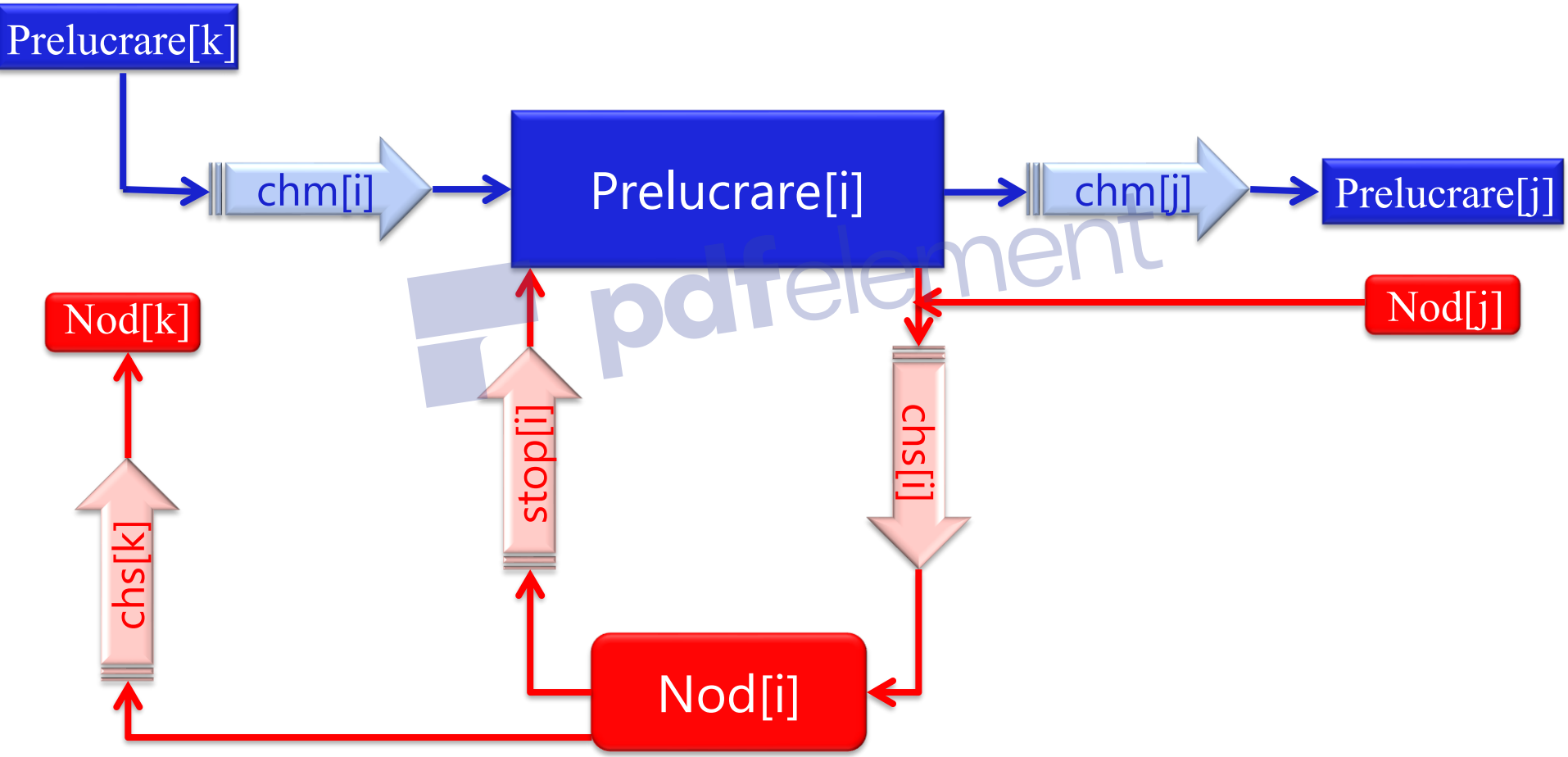
În continuare

- Lord = Prelucrare *(prelucrează mesaje de date)*
- Servitor = Nod *(gestionează semnalele de confirmare cu alte noduri și semnalele de comunicare cu Prelucrare)*
- Canalul poștal = canalul chm
  - chm_i este „adresa” lui $Prelucrare_i$
  - $Prelucrare_i$ primește mesaje pe chm_i și îi trimite lui $Prelucrare_j$ mesaje pe chm_j
- Canalul telefon pentru Nod = canalul chs
  - chs_i este „numărul de telefon” al lui Nod_i
  - Nod_i primește semnale pe chs_i (de la alte noduri sau de la $Prelucrare_i$) și trimite semnale de confirmare lui Nod_j pe chs_j
- Canalul telefon pentru Prelucrare = canalul stop
  - $stop_i$ este „numărul de telefon” al lui $Prelucrare_i$
  - $Prelucrare_i$ primește semnale de comunicare (da/nu) pe $stop_i$ de la Nod_i

pdfelement

Remove Watermark Now

Algoritmul Dijkstra-Scholten (6)



pdfelement

Algoritmul Dijkstra-Scholten (7)

Remove Watermark Now

```
enum fel{semnal, transmitere, receptie, terminare};
typedef struct {bool există;
                int deficit} arc;
chan chs[1:N](fel op, int id);
chan chm[1:N](int id, tip_date date);
chan stop[1:N](bool);

process Prelucrare[i=1 to N]{
   tip_date data; int id; bool term;
   inițializări;
   term = false;
   while (NOT term) {
      receive chm[i](id, data);                  /* primește mesaj */
      send chs[i](recepție, id);                 /* semnalează nod - recepție mesaj */
      realizează prelucrare
      ....
      send chm[j](i, data);                      /* trimite mesaj */
      send chs[i](transmitere, j);               /* semnalează nod - transmitere mesaj */
      ...
      if (decizie de terminare) term = true;
   }
   term =false;
   while (NOT term) {
      send chs[i](terminare, i);                 /* anunță nod că a terminat */
      receive stop[i](term);                     /* așteaptă să termine nodul */
   }
   stop;
```

pdfelement

Remove Watermark Now

Algoritmul Dijkstra-Scholten (8)

```
process Nod[i=1 to N] {
   arc in[1:N]; /* arcele de intrare */
   int nsemnale = 0;      /* număr semnale așteptate pentru mesajele trimise */
   int prim = 0;
   var id: int; fel k;
   while (true) {
      receive chs[i](k, id);
      if (k == recepție) { if (prim == 0) prim = id fi;
                           in[id].deficit = in[id].deficit + 1; }
      else if (k == transmitere) nsemnale = nsemnale+1;
      else if (k == semnal) nsemnale = nsemnale-1;
      else if (k == terminare){
      for [j = 1 to N st (in[j].există AND (j <> prim))] /* se confirmă mesajele primite */
         while (in[j].deficit > 0) { send chs[j](semnal, i);
                                     in[j].deficit = in[i].deficit - 1;
         }
      }
      if (nsemnale <> 0) send stop[i](false);            /* mai sunt semnale de primit */
      else if (nsemnale == 0) {
         if ((i <> sursa) AND (prim <> 0)) {              /* se confirmă lui prim */
            while (in[prim].deficit > 0) {
               send chs[prim](semnal, i);
               in[prim].deficit = in[prim].deficit - 1;
            }
          }
          send stop[i](true);
       }}
   }
```

semnal de la Prelucrare: a primit un mesaj

semnal de la Prelucrare: a trimis un mesaj

semnal de confirmare de la alt nod

Prelucrare dorește terminarea

pdfelement

Remove Watermark Now

Detecția terminării folosind marcaje

pdfelement

Remove Watermark Now

DETECTING TERMINATION OF DISTRIBUTED COMPUTATIONS USING MARKERS

Jayadev Misra
Department of Computer Sciences, University of Texas, Austin, 78712

Abstract

A problem of considerable importance in designing computations by process networks, is detection of termination. We propose a very simple algorithm for termination detection in an arbitrary network using a single marker We show an application of this scheme in solving the problem of token loss detection and token regeneration in a token ring.

Introduction

We study the problem of detecting termination of computation in a network of processes. If every process in a network is *idle*, i.e. waiting for messages in order to carry out further computations, and there are no messages in transit, i.e. all messages that have been sent have been received, then no process will carry out any further computation. It is often important to detect such a situation. Multiphase algorithms [15] in which a phase is to be initiated only upon completion of the previous phase, requires termination detection of a phase. Francez, Rodeh and Sintzoff [11] suggest that it may be easier to devise a distributed algorithm in two steps: (1) design an algorithm that maintains the desired safety properties and eventually guarantees a terminating *global* state; (2) superimpose a termination detection algorithm on the basic algorithm. Deadlock detection [8] which is related to termination detection is of fundamental importance in distributed data bases. Detection of token loss in a token ring can be shown to be a termination detection problem.

We suggest an algorithm for termination detection in an arbitrary network of processes. The algorithm uses a single marker which repeatedly traverses the edges of the network until it detects termination. We make no assumption about the network structure, process behavior or message delays. Our only assumption is that a process receives (only and all) messages from another process in the order sent by the sender. Our solution is symmetric among the processes, i.e. process id's are not used in the solution. As an application of the scheme in this paper, we give a simple and efficient algorithm for detecting token loss and regenerating the token in a token ring network. The marker algorithm has also been applied [14] in *avoiding* deadlocks in distributed simulations.

*This work was supported by a grant from IBM.

There has been a considerable amount of work in termination detection. For a specific class of computations, called ***diffusing computations***, Dijkstra and Scholten [2] proposed a very elegant algorithm. Their approach has been extended and applied to a variety of problems [4,5,6,7]. The major drawbacks of their approach are: (1) termination detection algorithm must be initiated whenever the diffusing computation starts and (2) the number of messages used for termination detection is equal to the number of messages in the diffusing computation itself. Our solution does not suffer from these drawbacks.

A series of papers has been published by Francez and co-workers [9,11,12] leading to a marker type algorithm. The algorithm proposed in this paper refines, simplifies and removes some of the restrictions of their approach. Independently Gouda [1] and Dijkstra [3] have proposed similar schemes, again with certain restrictions. All of these schemes use the marker to determine if a process has remained "continuously idle" over an interval of time; the idea of continuous idleness also appears in [8] for deadlock detection.

Recently Chandy and Lamport [10] have proposed a very elegant and general scheme for detecting *stable properties* of a network; a property (proposition) P is stable if it remains true once it becomes true. Clearly properties such as "network computation has terminated," and "a subset of processes are deadlocked" are all stable properties. Even though we treat the narrower problem of termination detection, our solution is simpler and more efficient for this specific problem.

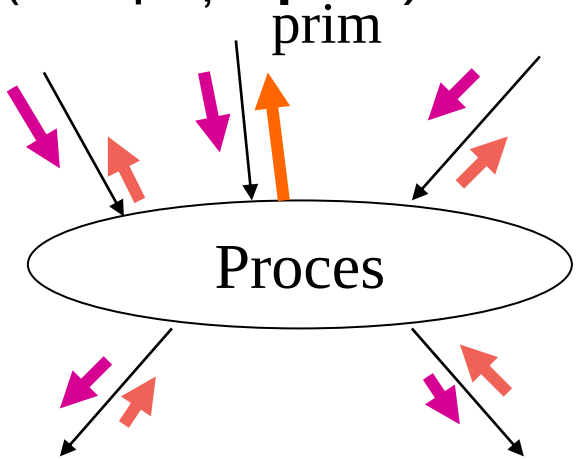
Problem Description

Misra, Jayadev. "Detecting termination of distributed computations using markers." *Proceedings of the second annual ACM symposium on Principles of distributed computing*. ACM, 1983.

Remove Watermark Now

Detecția terminării folosind marcaje (1)

- Procesele transmit mesaje de marcaj la sfârşitului comunicației.
- Procesele primesc semnale de confirmare doar pentru marcaje.
- Pentru terminare:
  - aşteaptă marcaje pe legăturile de intrare
    - *(să termine cei de la care a primit mesaje)*
  - transmite marcaje pe legăturile de ieşire
    - *(anunță propria terminare celor cărora le-a trimis mesaje)*
  - transmite semnale pe legăturile de intrare (mai puțin **prim**)
    - *(confirmă marcajele primite)*
  - aşteaptă semnale pe legăturile ieşire
    - *(așteaptă confirmări pentru marcajele trimise)*
  - transmite semnal pentru **prim**



pdfelement

Remove Watermark Now

Detecția terminării folosind marcaje (2)

- Pentru fiecare arc se țin două atribute: ***activ*** și ***marcaj*.**

- Pentru fiecare **arc de intrare**, se înregistrează dacă:
  - s-a primit un mesaj (se pune ***activ*** pe true);
  - s-a primit un marcaj (se pune ***marcaj*** pe true);
  - s-a transmis un semnal de confirmare (se pun ambele pe false).

- Pentru fiecare **arc de ieşire**, se înregistrează dacă:
  - s-a transmis un mesaj (se pune ***activ*** pe true);
  - s-a transmis un marcaj (se pune ***marcaj*** pe true);
  - s-a primit un semnal de confirmare (se pun ambele pe false).

Remove Watermark Now

Detecția terminării folosind marcaje (3)

```
const marcaj = mesaj-special-de-marcaj-sfârșit-date;
typedef enum {semnal, transmitere, recepție,
              recepție-marcaj, terminare} fel;
typedef struct {bool există;
              bool activ;        /*inițial false*/
              bool marcaj} arc;     /*inițial false*/
chan chs[1:N](fel op, int id);
chan chm[1:N](id:int, tip_date date);
chan stop[1:N](bool);
process Nod[i=1 to N]{
  bool term = false;
  arc in[1:N], out[1:N];
  int nsemnale = 0;/*număr semnale așteptate pe out*/
  int prim = 0;
  int nmarcaje;     /*număr marcaje așteptate pe in*/
  int id; fel k;
```

pdfelement

Detecția terminării folosind marcaje (4)

Remove Watermark Now

```
while (true) {
    receive chs[i](k, id);
    if (k == recepție) {
      if (prim == 0) prim = id;
      if (NOT in[id].activ) { in[id].activ = true;
                              nmarcaje := nmarcaje + 1;
      }
    else if (k == recepție-marcaj) {
       in[id].marcaj = true; nmarcaje = nmarcaje - 1;
       if (id == prim) term = true; }
    else if (k == transmitere) {
      if (NOT out[id].activ) { out[id].activ = true;
                               nsemnale = nsemnale + 1; }
    else if (k == semnal) {
      out[id].activ = false; out[id].marcaj = false;
      nsemnale = nsemnale - 1;
    }
```

semnal de la Prelucrare: a primit un mesaj

semnal de la Prelucrare: a primit un marcaj

semnal de la Prelucrare: a trimis un mesaj

semnal de confirmare de la alt nod

pdfelement

Detecția terminării folosind marcaje (6)

Remove Watermark Now

```
else if (k == terminare) {
      if (NOT term) send stop[i](false);
      else if (term) {
         for [j = 1 to N st (j <> prim AND out[j].activ AND
                             NOT out[j].marcaj)] {
          send chm[j](i, marcaj); out[j].marcaj = true;
         }
         if (nmarcaje <> 0) send stop[i](false);
         else if (nmarcaje == 0) {
             for [j = 1 to n st (in[j].există AND (j<>prim)
                                 AND in[j].activ)] {
                send chs[j](semnal, i);
                in[j].activ = false; in[j].marcaj = false;
             }
             if (nsemnale <> 0) send stop[i](false);
             else if (nsemnale == 0) {
                  if ((i <> sursa) AND (prim <> 0)) {
                     send chs[prim](semnal, i);
                     in[prim].activ = false;
                     in[prim].marcaj = false;
                  }
                  send stop[i](true);
   } } } }
```

pdfelement

Detecția terminării folosind marcaje (6)

Remove Watermark Now

```
process Prelucrare[i=1 to N] {
  tip_date data; int id; bool term;
  initializari;
  term = false;
  while (NOT term) {
     receive chm[i](id, data);
     if (data==marcaj) send chs[i](receptie-marcaj,id);
     else if (data<>marcaj) {
          send chs[i](receptie, id);
          realizează prelucrare
          ....
          send chm[j](i, data);
          send chs[i](transmitere, j);
          ...
          if (decizie de terminare) term = true;
     }
  }
  term =false;
  while (NOT term) {
     send chs[i](terminare, i);
     receive stop[i](term);
  }
  stop;
```

pdfelement

Remove Watermark Now

Algoritmul lui Huang

Conferences > [1989] Proceedings. The 9th I...

Detecting termination of distributed computations by external agents

Publisher: IEEE

1 Author(s) S.-T. Huang **View All Authors**

34 Paper Citations

408 Full Text Views

Abstract
Authors
References
Citations
Keywords
Metrics

Abstract:
An algorithm is presented that defects for termination of distributed computations by an auxiliary controlling agent. The algorithm assigns a weight W, 0>

Published in: [1989] Proceedings. The 9th International Conference on Distributed Computing Systems

Date of Conference: 5-9 June 1989

INSPEC Accession Number: 3472185

Date Added to IEEE *Xplore*: 06 August 2002

DOI: 10.1109/ICDCS.1989.37933

Print ISBN: 0-8186-1953-8

Publisher: IEEE

Conference Location: Newport Beach, CA, USA, USA

First Page of the Article

Hide First Page Preview

pdfelement

Huang, S-T. "Detecting termination of distributed computations by external agents." *[1989] Proceedings. The 9th International Conference on Distributed Computing Systems*. IEEE, 1989.

Remove Watermark Now

Algoritmul lui Huang (1)

- Modelul se bazează pe următoarele:
  - procesele pot fi active sau libere (idle)
  - doar procesele active transmit mesaje
  - procesele libere pot deveni active la recepția unui *mesaj de calcul*
  - procesele active pot deveni libere în orice moment
  - **terminarea**: toate procesele sunt libere şi nu există mesaje de date în tranzit
  - topologia este un graf conex

pdfelement

Remove Watermark Now

Algoritmul lui Huang (2)

- **Ideea:**
  - Un *agent de control* are inițial ponderea 1.
  - Toate celelalte procese sunt libere și au inițial ponderea 0.
  - Calculul porneşte atunci când *agentul de control* trimite un mesaj de date unui proces.
  - Un proces liber devine activ la recepția unui mesaj de date.
- **Notații:**
  - B(DW)
    - Mesaj de date cu ponderea DW
    - Trimis doar de *agentul de control* sau de un proces activ
  - C(DW)
    - Mesaj de control cu ponderea DW
    - Trimis de un proces activ *agentului de control* când devine liber
  - W
    - Ponderea curentă a unui proces

pdfelement

Remove Watermark Now

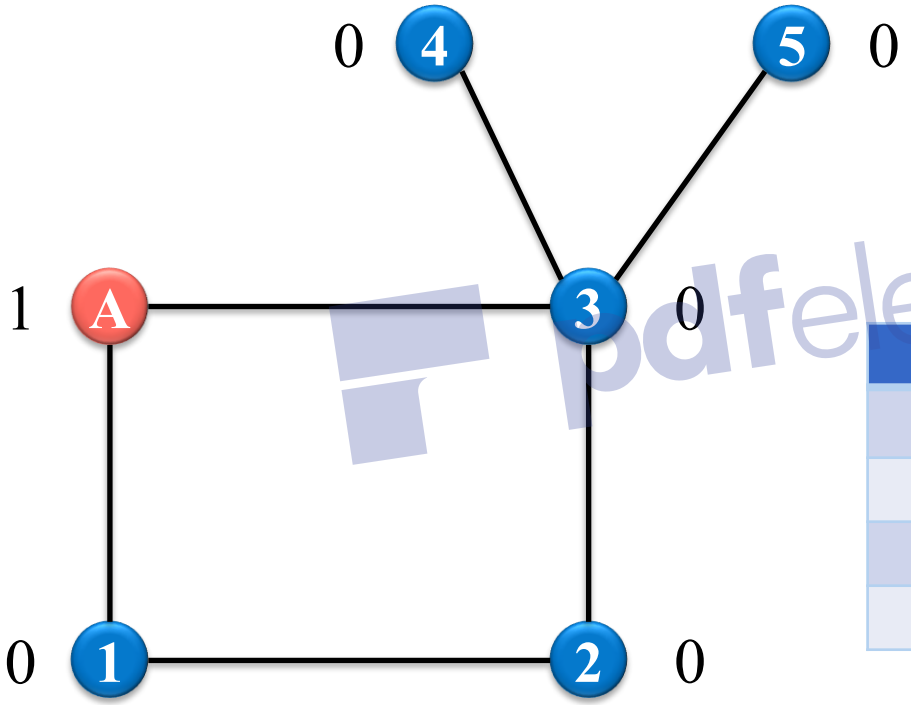
Algoritmul lui Huang (3)

1. **Send B(DW)**:
   - Găseşte W1, W2 astfel încât W1 > 0, W2 > 0, W1 + W2 = W
   - Pune W = W1 şi transmite B(DW = W2)
2. **Receive B(DW)**:
   - W = W + DW
   - **if** liber -> devine activ **fi**
3. **Send C(DW)**:
   - send C(DW = W) *agentului de control*
   - devine liber
4. **Receive C(DW) (doar *agentul de control*)**:
   - W = W + DW
   - **if** W = 1 -> declară “terminarea” **fi**

pdfelement

Remove Watermark Now

Algoritmul lui Huang (4)



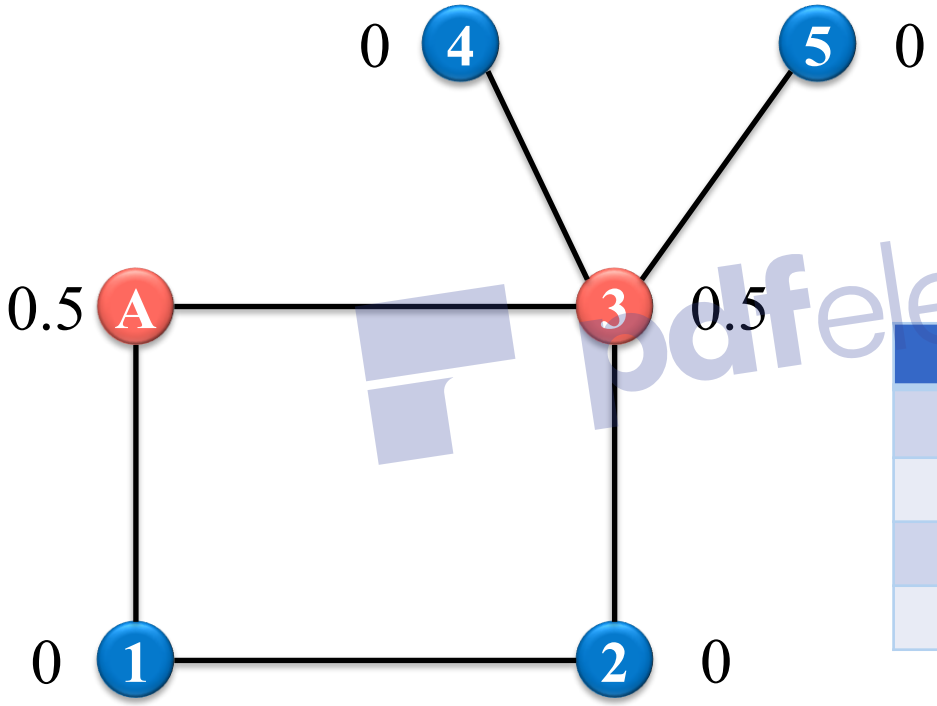
pdfelement

Sursă	Destinație	Mesaj
A	3	B(DW:=0.5)

TIMP
0

Remove Watermark Now

Algoritmul lui Huang (4)



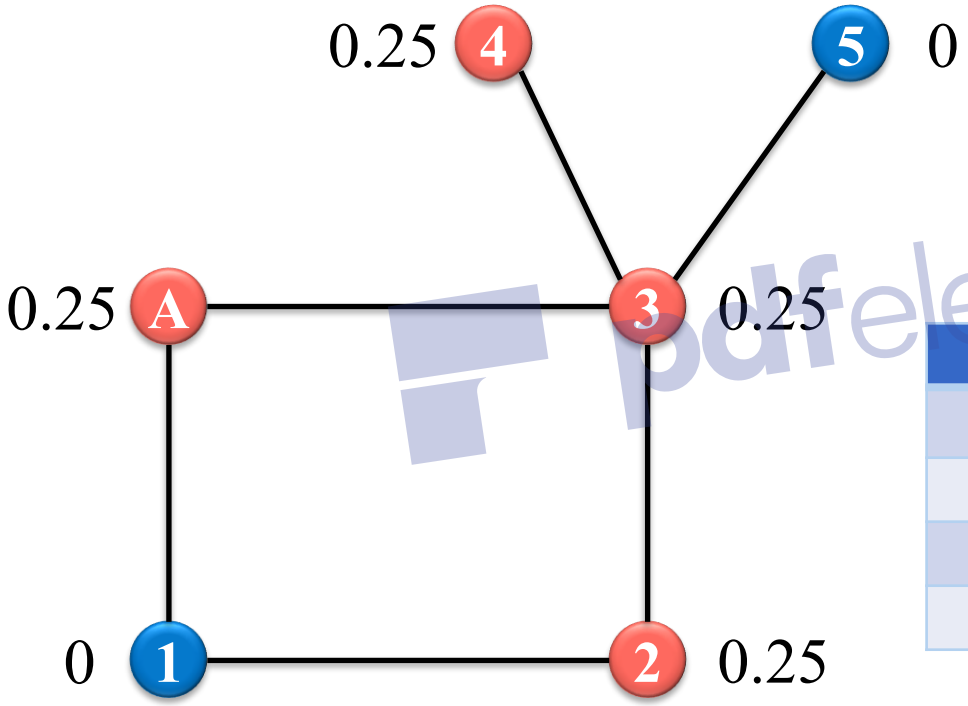
pdfelement

Sursă	Destinație	Mesaj
A	2	B(DW:=0.25)
3	4	B(DW:=0.25)

TIMP
1

Remove Watermark Now

Algoritmul lui Huang (4)

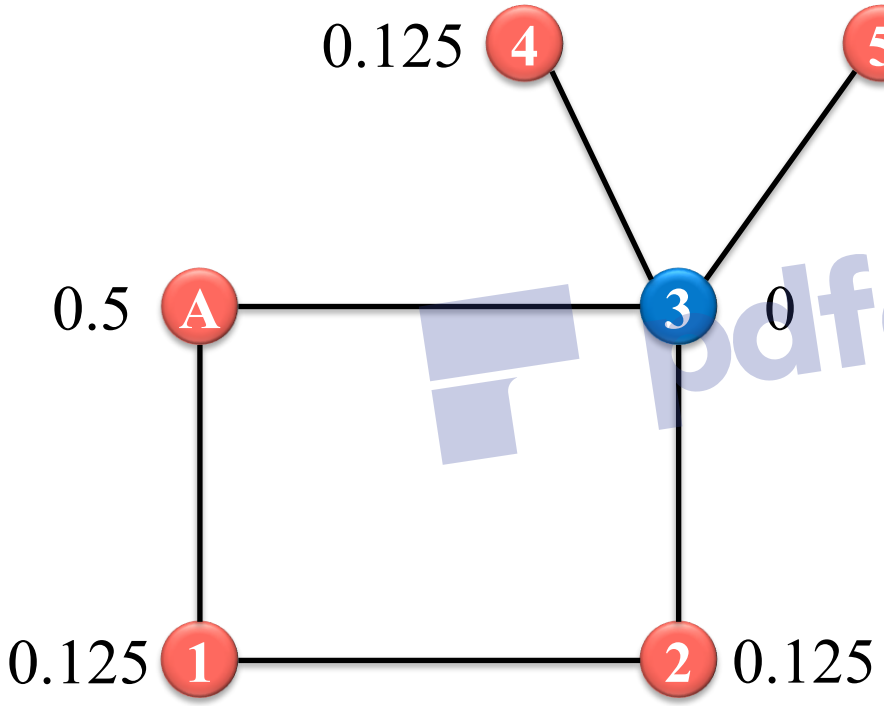


Sursă	Destinație	Mesaj
4	1	B(DW:=0.125)
3	A	C(DW:=0.25)
2	5	B(DW:=0.125)

TIMP
2

Remove Watermark Now

Algoritmul lui Huang (4)



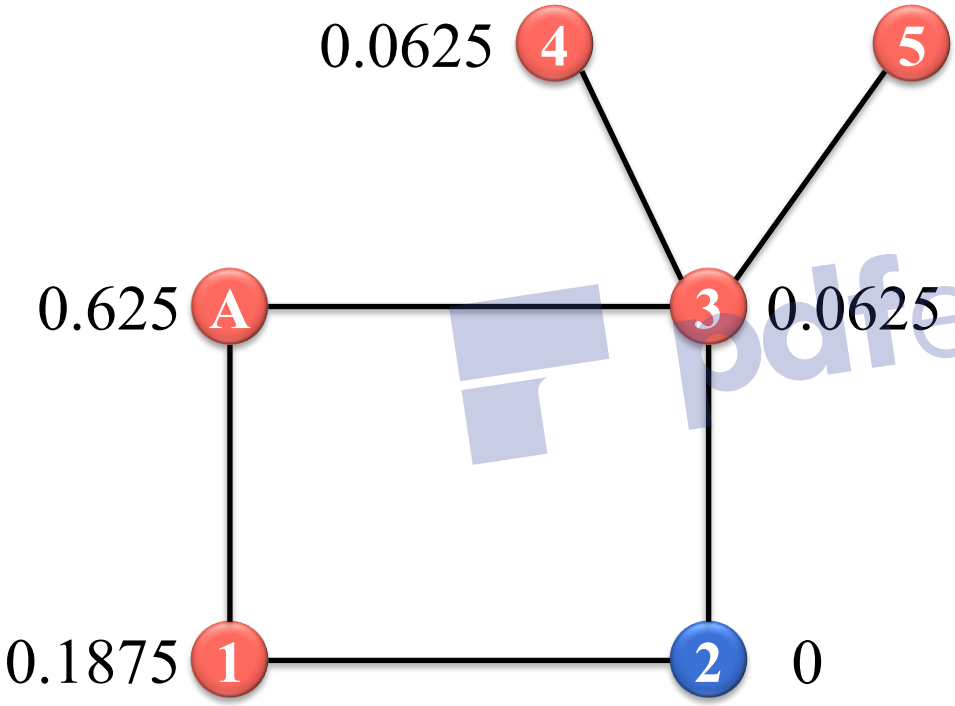
pdfelement

Sursă	Destinație	Mesaj
4	1	B(DW:=0.0625)
2	A	C(DW:=0.125)
1	3	B(DW:=0.0625)
5	1	B(DW:=0.0625)

TIMP
3

Remove Watermark Now

Algoritmul lui Huang (4)



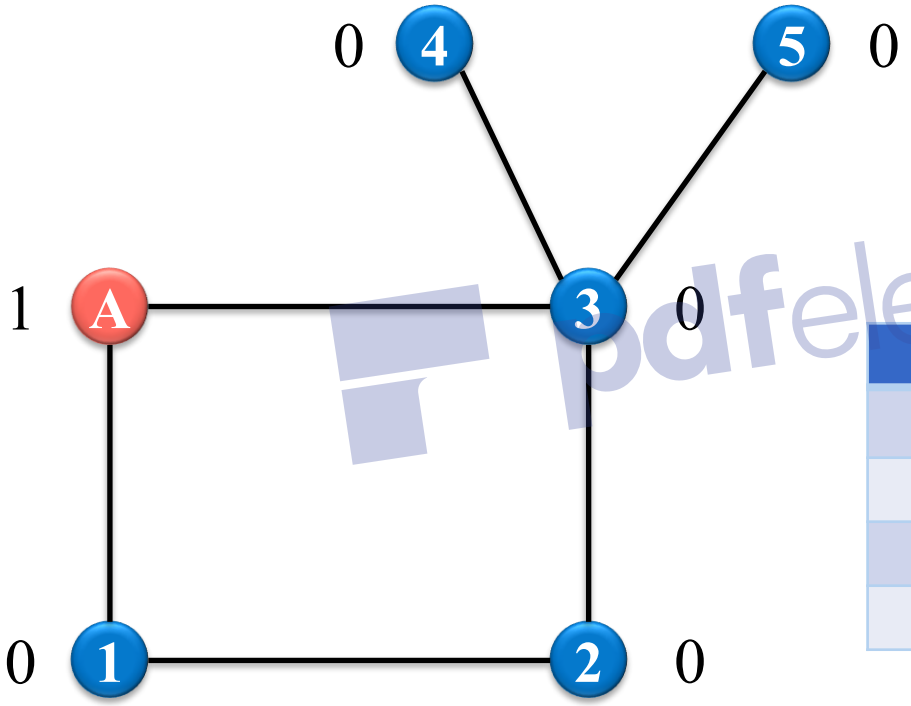
pdfelement

Sursă	Destinație	Mesaj
1	A	C(DW:=0.1875)
3	A	C(DW:=0.0625)
4	A	C(DW:=0.0625)
5	A	C(DW:=0.0625)

TIMP
4

Remove Watermark Now

Algoritmul lui Huang (4)



pdfelement

Sursă	Destinație	Mesaj

TIMP
5

Remove Watermark Now

Sumar

- Problematica terminării programelor distribuite
- Cazul proceselor organizate în inel
- Tehnica jetoanelor pentru cazul general
- Confirmarea mesajelor
- Algoritmul Dijkstra-Scholten
- Detecția terminării folosind marcaje
- Algoritmul lui Huang

pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Algoritmi pentru sisteme tolerante la defecte

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

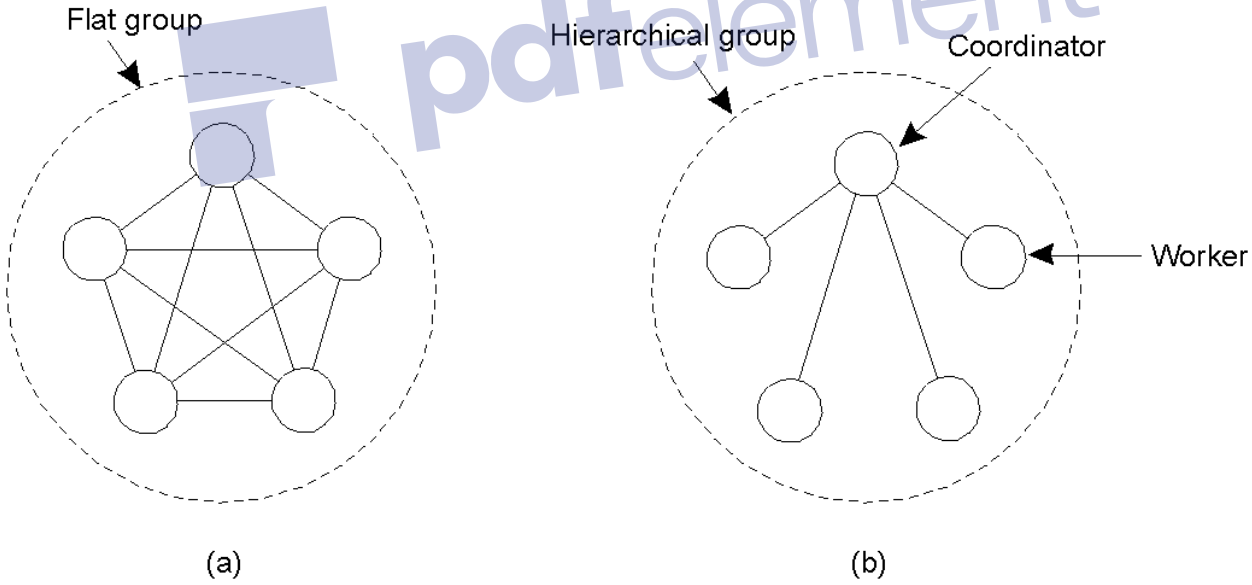
pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Fiabilitate la nivelul proceselor

- De obicei mascam defectarea proceselor prin replicare
- Organizam procesele in grupuri, un mesaj trimis grupului fiind livrat tuturor membrilor acelui grup
- Daca un membru se defecteaza, un altul ii ia locul

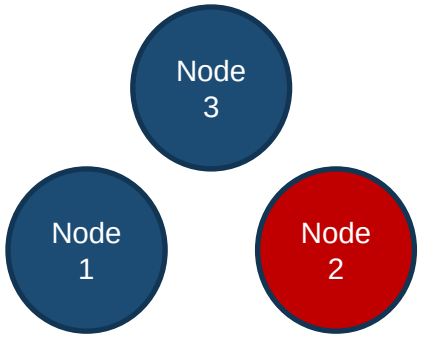


Comunicatie: diferenta intre Egalitate (a) vs. Ierarhie (b)

Remove Watermark Now

La cursul practic... failure models

- Pana in acest punct, modelele presupuse pentru sisteme distribuite imperfecte au constat din:
  1. **Fail-recover faults**
     - Un nod se defecteaza si nu mai raspunde la mesaje
     - Alte noduri pot detecta defectul prin timeout-uri: “if 127.0.0.10 doesn’t respond in 100ms, it’s down”
     - … dar nodurile respective pot reveni in operatie
  2. **Delayed / lost messages**
     - Mesajele intre noduri pot fi arbitrar intarziate sau pierdute, dar **nu** corupte
- Ne putem folosi imaginatia. Ce facem daca…
  - reteaua incepe sa corupa mesaje?
  - un atacator intercepteaza si modifica date in trafic?
  - unele noduri sunt… malitioase?



pdfelement

Remove Watermark Now

Toleranta la defecte bizantine

- **Defectul bizantin:** Orice fel de defect la care ne putem gandi, precum…
  - coruperea de mesaje
  - noduri ce se comporta diferit functie de **context** (ex., comunica unor noduri informatii diferite)
  - nodurile **conspira** intre ele pentru a cauza distrugeri

pdfelement

The Saddest Moment

JAMES MICKENS

Remove Watermark Now

James Mickens is a researcher in the Distributed Systems group at Microsoft's Redmond lab. His current research focuses on Web applications, with an emphasis on the design of JavaScript frameworks that allow developers to diagnose and fix bugs in widely deployed web applications. James also works on fast, scalable storage systems for datacenters. James received his PhD in computer science from the University of Michigan, and a bachelor's degree in computer science from Georgia Tech.
mickens@microsoft.com

Whenever I go to a conference and I discover that there will be a presentation about Byzantine fault tolerance, I always feel an immediate, unshakable sense of sadness, kind of like when you realize that bad things can happen to good people, or that Keanu Reeves will almost certainly make more money than you over arbitrary time scales. Watching a presentation on Byzantine fault tolerance is similar to watching a foreign film from a depressing nation that used to be controlled by the Soviets—the only difference is that computers and networks are constantly failing instead of young Kapruskin being unable to reunite with the girl he fell in love with while he was working in a coal mine beneath an orphanage that was atop a prison that was inside the abstract concept of World War II. "How can you make a reliable computer service?" the presenter will ask in an innocent voice before continuing, "It may be difficult if you can't trust anything and the entire concept of happiness is a lie designed by unseen overlords of endless deceptive power." The presenter never explicitly says that last part, but everybody understands what's happening. Making distributed systems reliable is inherently impossible; we cling to Byzantine fault tolerance like Charlton Heston clings to his guns, hoping that a series of complex software protocols will somehow protect us from the oncoming storm of furious apes who have somehow learned how to wear pants and maliciously tamper with our network packets.

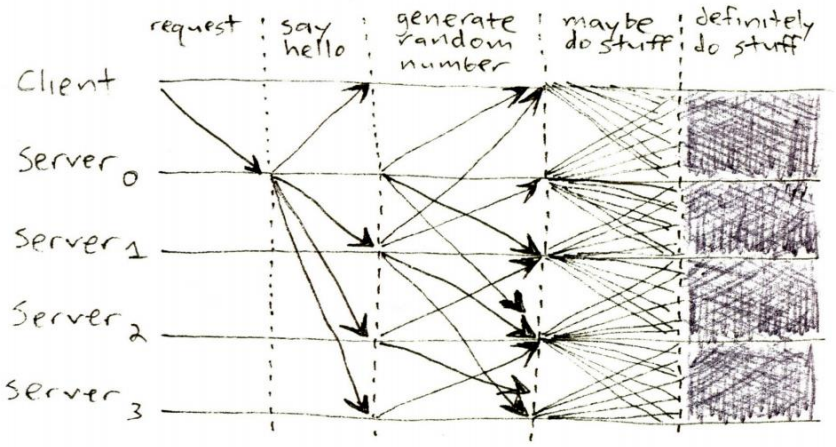


Figure 1: Typical Figure 2 from Byzantine fault paper: Our network protocol

How can you make a reliable computer service? […] It may be difficult if you can't trust anything and the entire concept of happiness is a lie designed by unseen overlords of endless deceptive power.

The Saddest Moment, James Mickens

pdfelement

Remove Watermark Now

Agreement algorithms

- In sistemele distribuite, avem diverse cazuri cand procesele au nevoie sa ajunga la un acord :
  - Leader election
  - Commit
  - Synchronize

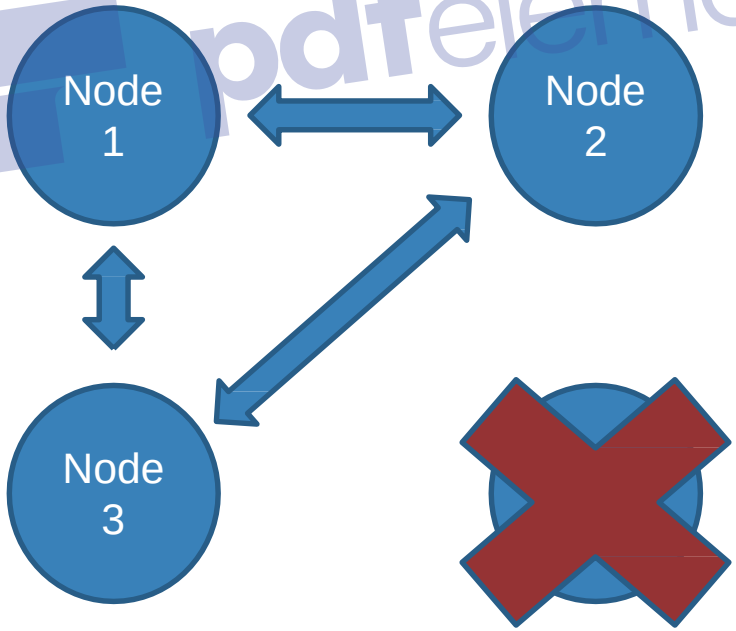
pdfelement

- **Distributed Agreement**: *toate procesele non-faulty ajung la consens intr-un numar finit de pasi*
- Procese perfecte, canale faulty: problema celor doua armate
- Procese faulty, canale perfecte: problema generalilor Bizantini

Remove Watermark Now

Distributed consensus

- Un numar de noduri ajung la nu **acord** despre o **valoare**
- Orice nod se poate **defecta** sau **recupera/reveni** in orice moment de timp



pdfelement

Remove Watermark Now

How does RAFT work?

RAFT IS A CONSENSUS ALGORITHM

used in many projects in the real world - ex. etcd

a process for getting multiple machines to "agree"!

- RAFT doesn't stand for anything. It's a bunch of logs tied together

WHY? Any part of a distributed system can fail! Consensus across several nodes gives us backups of data.

WHAT ABOUT NETWORK PARTITIONS?!

In a partition, if the leader falls on the smaller side, a new leader is elected within the majority side. When the partition ends, messages received by the minority side are discarded, and those nodes converge their state to match the majority's.

WHEN A NEW WRITE HAPPENS:

Any node can accept writes!

Message forwarded to leader node

Message is queued...

COPY me!

...while followers copy it to their logs

ACK!

ACK

When a quorum have copied the message, leader "commits" the new data & will now return it on read

FISH

Followers also commit so they return new data!

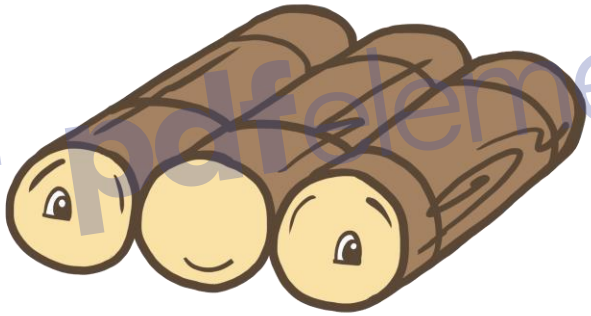
routelement

@deniseyu21 with @mt165

Remove Watermark Now

Algoritmul *raft* pentru consens

- Raft este un algoritm dovedit a functiona corect, si care asigura *strong consistency*



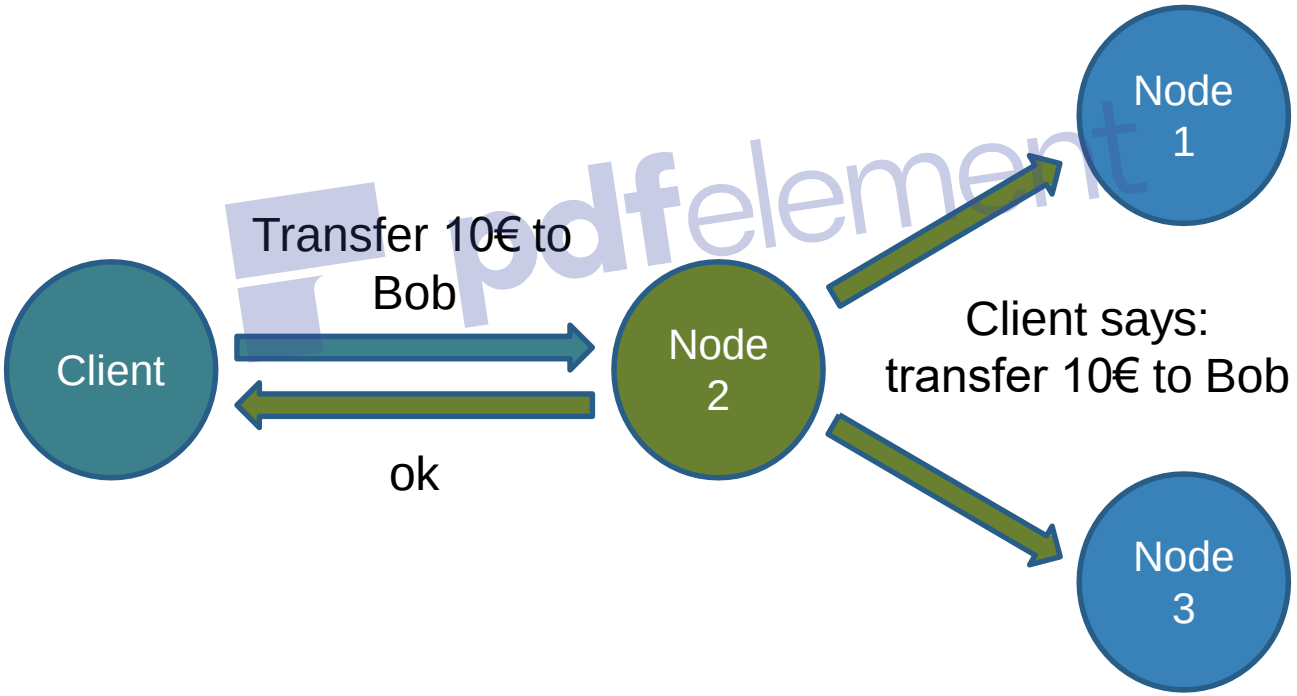
pdfelement

- Exista si alti algoritmi pentru stabilirea consensului, cel mai renumit fiind *paxos*

Remove Watermark Now

Replicare master – slave

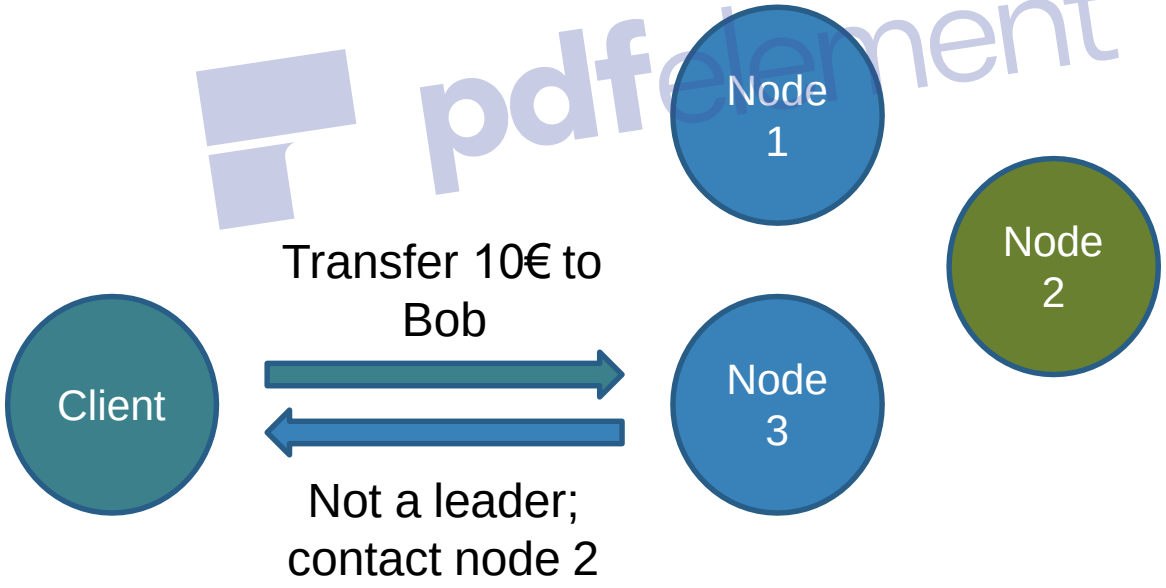
- Unul dintre noduri este ales *master* (sau *leader*), ceilalti sunt *followers*



Remove Watermark Now

Replicare master – slave

- Furnizeaza o **ordonare globala** scrierilor provenind din partea clientilor – putem reconcilia scrieri concurente
- Clientii ce contacteaza noduri follower sunt **redirectionati**



Remove Watermark Now

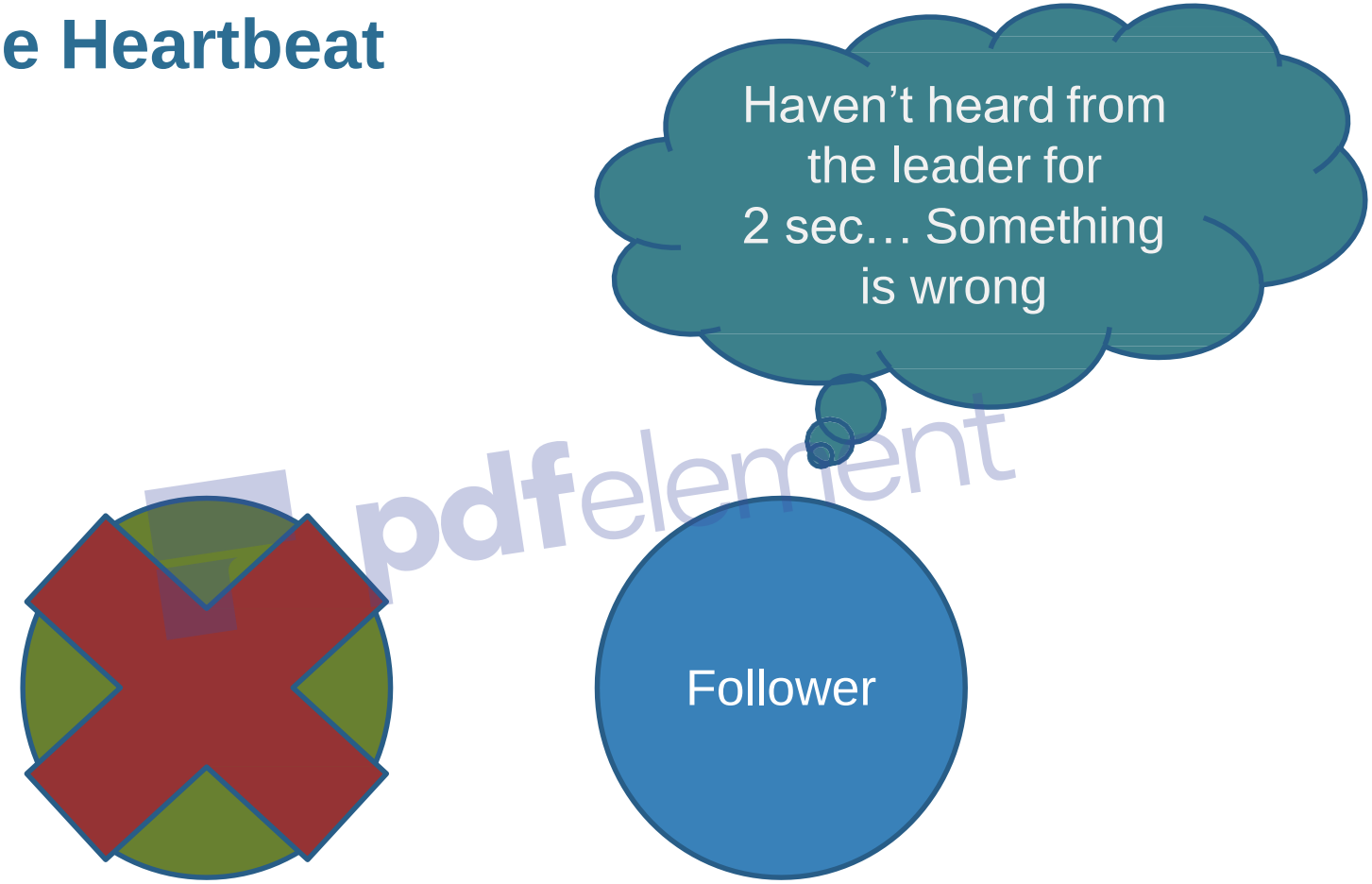
Defectarea liderului

- Liderul trimite periodic mesaje heartbeat (alive) tuturor nodurilor follower
- Daca un nod follower nu mai primeste mesaje heartbeat, presupune ca leaderul s-a defectat si initiaza o **alegere**
- O alegere este castigata daca un nod primeste voturi pozitive din partea unui cluster **majoritar** de noduri

pdfelement

Remove Watermark Now

Mesaje Heartbeat

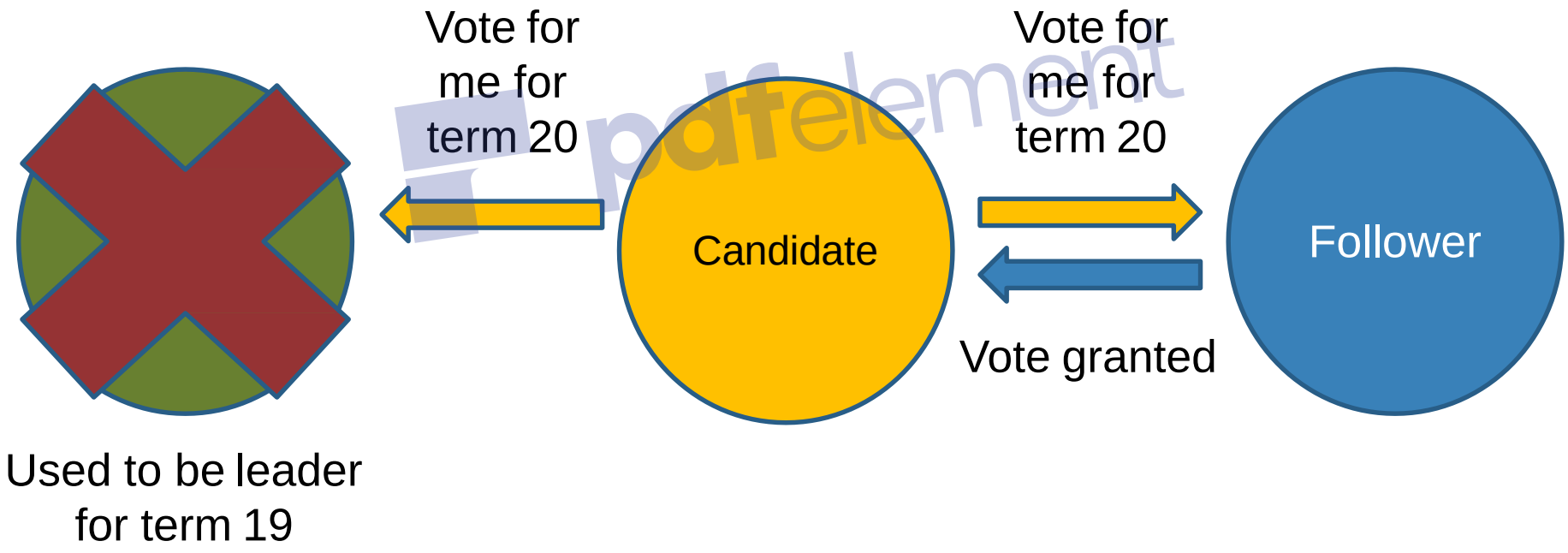


pdfelement

Remove Watermark Now

Alegere lider

O alegere finalizata cu succes: 2 din 3 noduri sunt de acord cu noul lider



Remove Watermark Now

Aducerea nodurilor outdated la zi

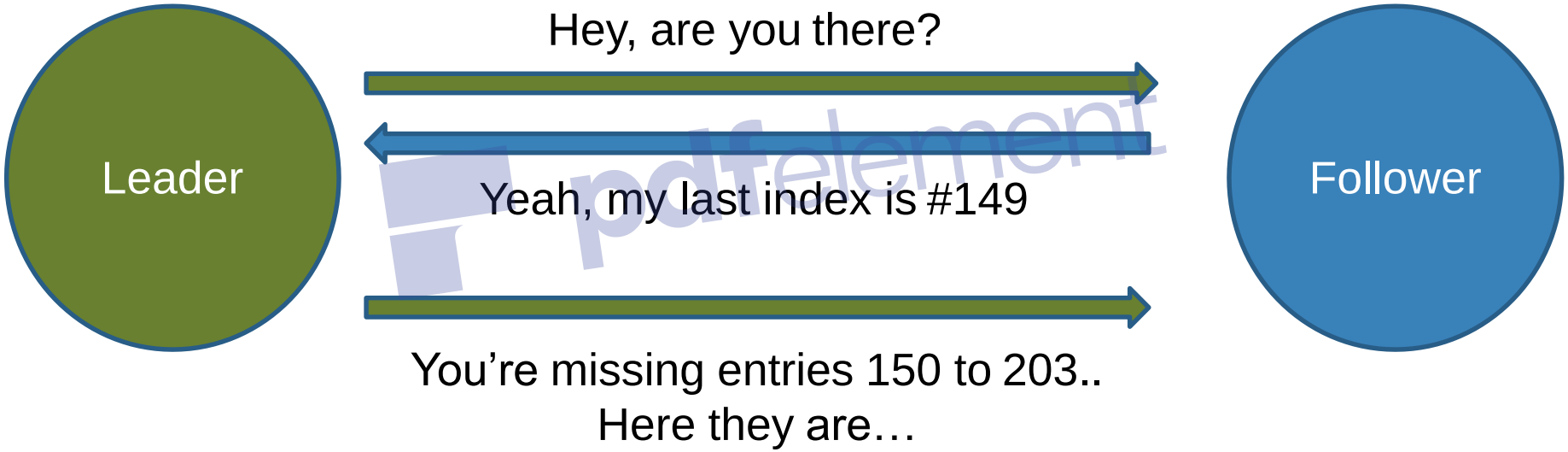
- Unul din follower iese offline pentru 10 minute – cum il re-aducem intr-o stare consistenta?
- Inregistram *toate scrierile* intr-un “indexed log", pe care il replicam
  - Contine momentul cand o inregistrare a fost scrisa

pdfelement

Index	Term	Contents
0	1	SET food pizza
1	1	SET language c++
2	1	SET food pickles
3	5	SET answer_to_life 42
…		….

Remove Watermark Now

Replicarea Logului



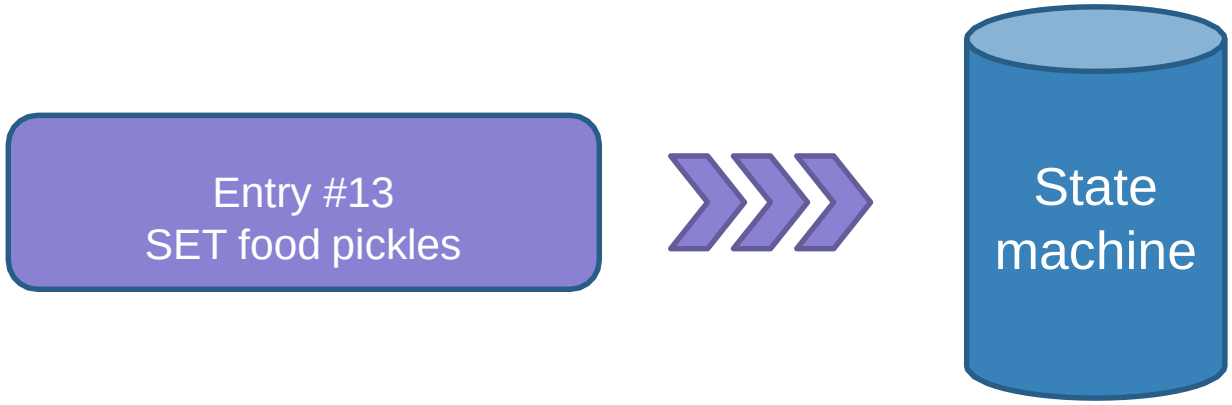
pdfelement

Remove Watermark Now

Aplicarea intrarilor din log

- Intrarile din Log de obicei conduc la *modificari* ale unei baze de date (cunoscuta si ca *state machine*)
- La un moment dat, aceste modificari trebuie *aplicate* in state machine

pdfelement



Remove Watermark Now

Replicarea State Machine

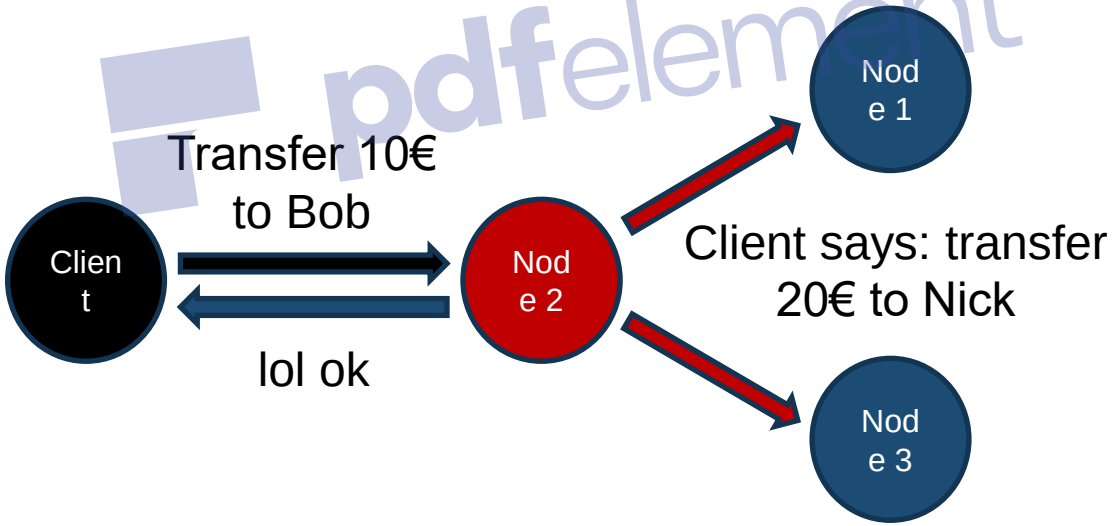
- Nu e sigur sa aplicam o intrare imediat, din moment ce exista situatii (rare) cand un alt lider (subsecvent) poate cere **roll back**
- Intrarile din Log sunt considerate a fi **comise** doar cand o majoritate de noduri le-au aplicat
- Odata ce o intrare a fost comisa, se garanteaza ca nu va mai fi retrasa si poate fi *aplicata* sigur in state machine
- Doar nodurile care au *toate intrarile comise* pot participa ca posibili lideri in alegeri

pdfelement

Remove Watermark Now

... Ne reintoarcem la defecte...

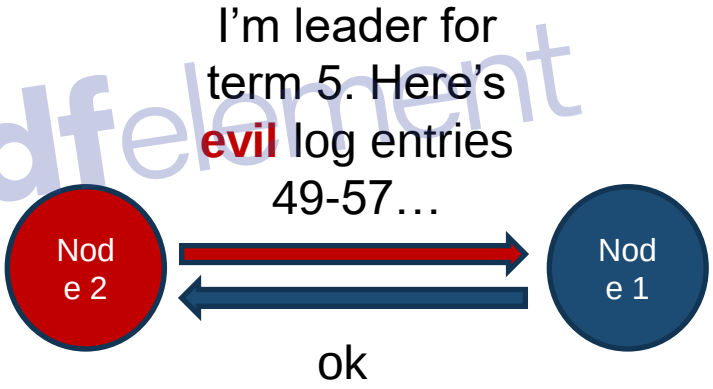
- Raft nu este considerat a fi un protocol “byzantine fault tolerant” – diverse oportunitati pentru un nod malitios sa cauzeze probleme
- Un lider byzantine poate modifica cererile clientilor



Remove Watermark Now

Probleme cu raft…

- Un nod bizantin poate f. usor sa pretinda ca este lider, si toate celelalte noduri il vor crede
- Nu trebuie sa **dovedeasca** ca a castigat alegerile!

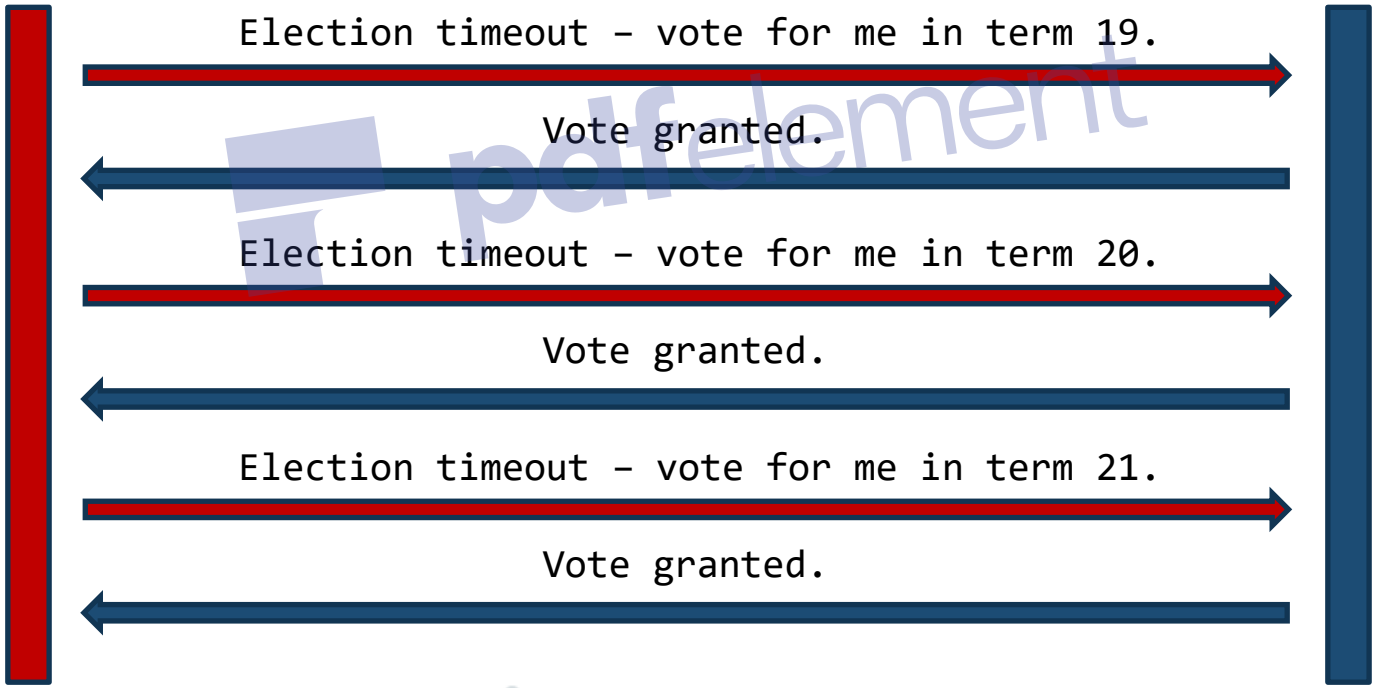


pdfelement

Remove Watermark Now

Probleme cu raft…

... sau in mod continuu sa declanseze alegeri, ne-permitand aparitia vreunui alt lider– clusterizarea devine *useless* (my way or the highway)



pdfelement

Remove Watermark Now

Diverse cause ale defectelor bizantine

- Cauze benigne
  - **Coruperea** unor pachete pe retea, a datelor pe disc
  - Un **memory bitflip** complet aleatory cauzeaza un nod sa se comporte imprevizibil

```
bool isLeader;   // gets hit by cosmic ray
```

  - Un bug rar in cod (as if) cauzeaza un nod sa nu respecte protocolul

- Cauze malitioas
  - Un atacator modifica *traficul inter-node*
  - Un atacator *preia abuziv* un nod

pdfelement

Remove Watermark Now

Posibile contra-masuri

- **Semnaturi criptografice, checksums**: pot ajuta la **autentificarea** si verificarea **integritatii** mesajelor si datelor

- **ECC (Error-Correcting Code) memory:** poate detecta si uneori repara eventuale erori la nivelul memoriei

pdfelement

- E extrem de dificil sa asiguram protectia impotriva *tuturor* tipurilor de defecte… unele **protocoale** exista
  - dar toate au limitarile lor

Remove Watermark Now

Reality check

- Este necesar un asemenea nivel de paranoia?
- Cand rulezi 50.000 de servere in productie, ce se intampla cand unul “goes byzantine”?
- Ar putea acesta sa distruga functionarea sistemului ca ansamblu? poate…
- O idee buna sa asiguram protectia macar impotriva *unor defecte bizantine*

pdfelement

Remove Watermark Now

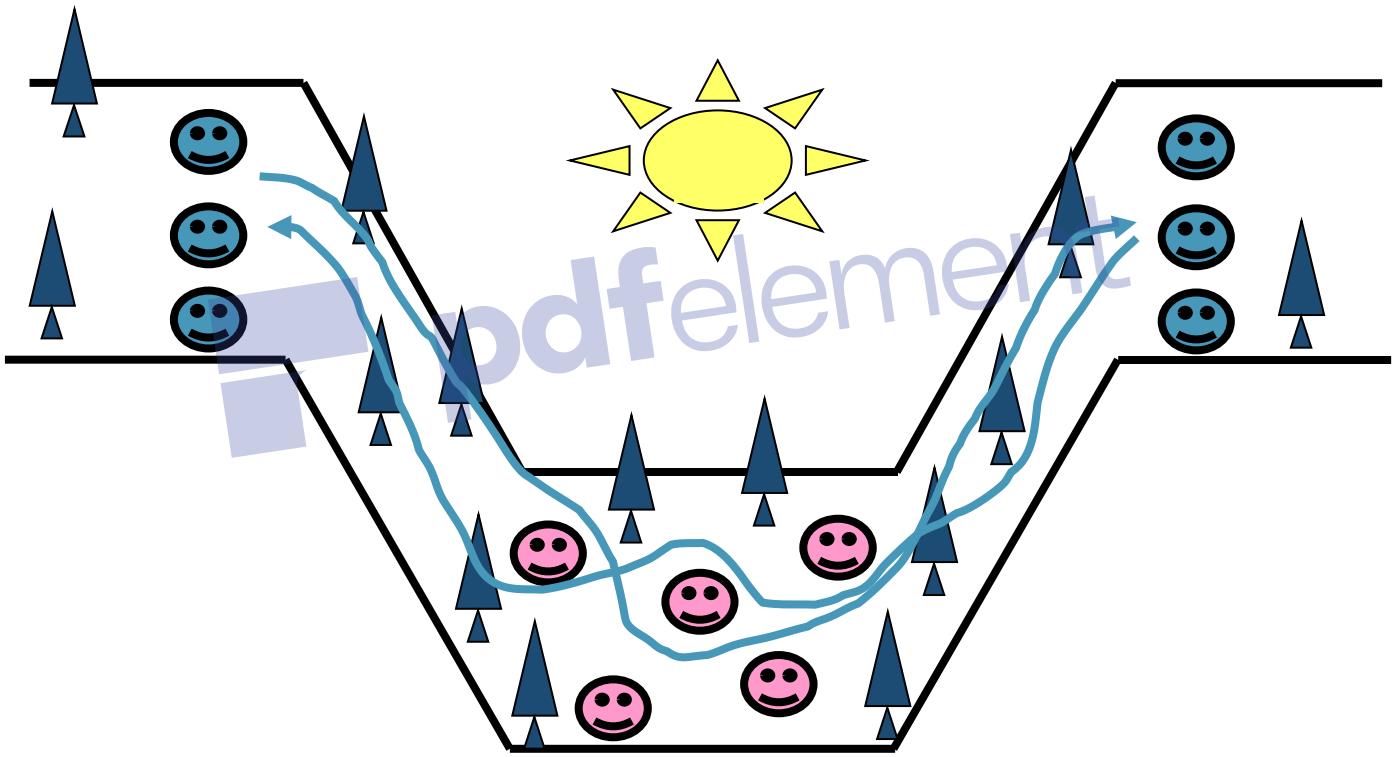
Rezolvarea prin replicare...

- Replicam un proces si replicile la nivelul grupului intr-un singur grup
- Cate replici vom crea?
- Tipuri de defecte ale proceselor
  - ***crash***: procesul devine nefuncțional
  - ***byzantine***: procesul trimite mesaje cu un conținut arbitrar
- Un sistem se numeste *k fault-tolerant* daca poate supravietui si functiona chiar si cu k procese defecte
  - Pentru defectele de tip crash (a faulty process halts, but is working correctly until it halts)
    - k+1 replici
  - Pentru defectele Byzantine (a faulty process may produce arbitrary responses at arbitrary times)
    - 2k+1 replici

pdfelement

Remove Watermark Now

Problema celor doua armate



Remove Watermark Now

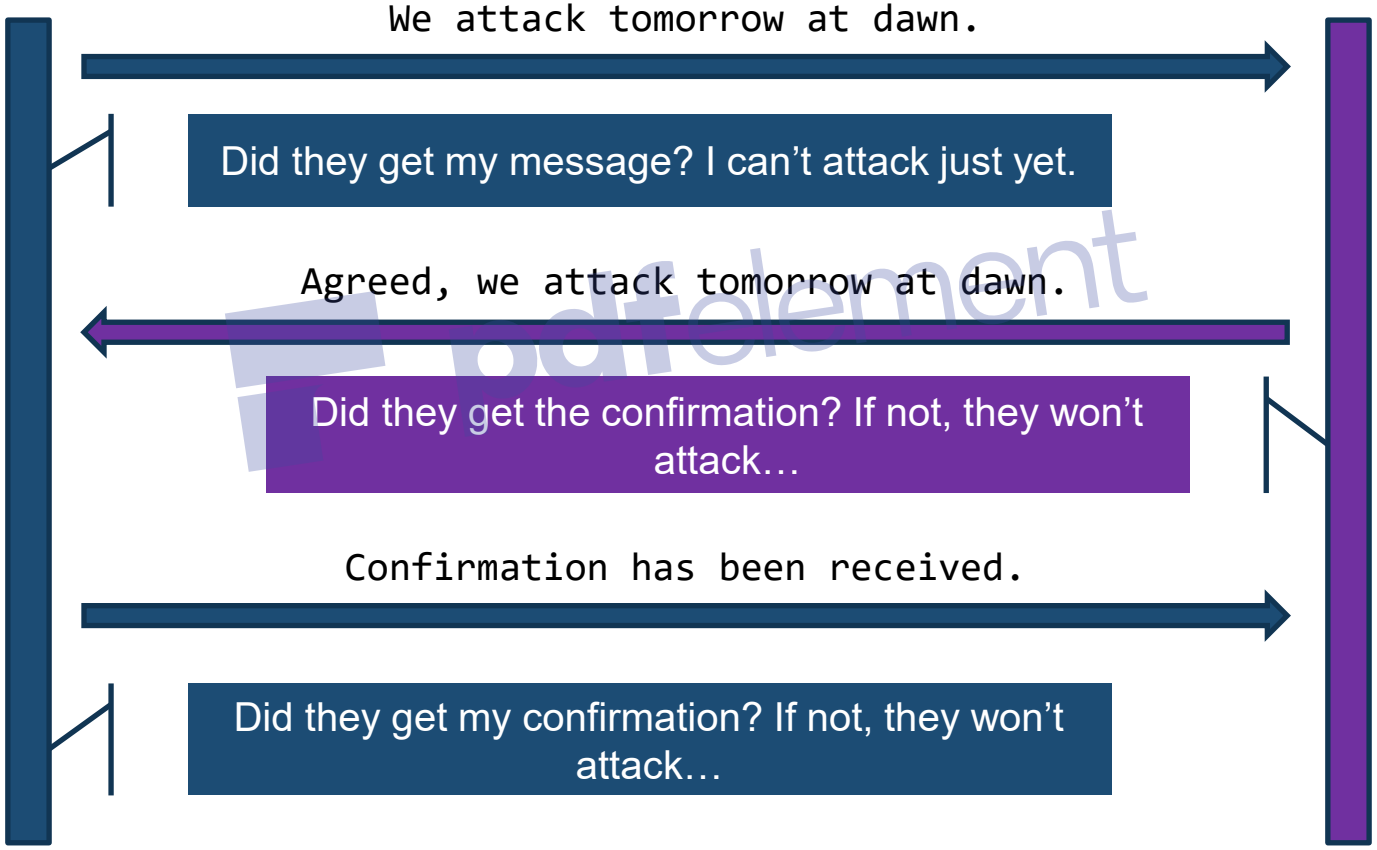
Paradoxul celor doi generali

- Doua armate au inconjurat un oras
- Generalii acestora trebuie sa decida **impreuna** daca sa atace sau sa mearga in retragere
- Ei comunica prin mesageri, care insa pot fi **interceptati**
- Ambii generali trebuie sa ajunga la **aceeasi** decizie

pdfelement

Remove Watermark Now

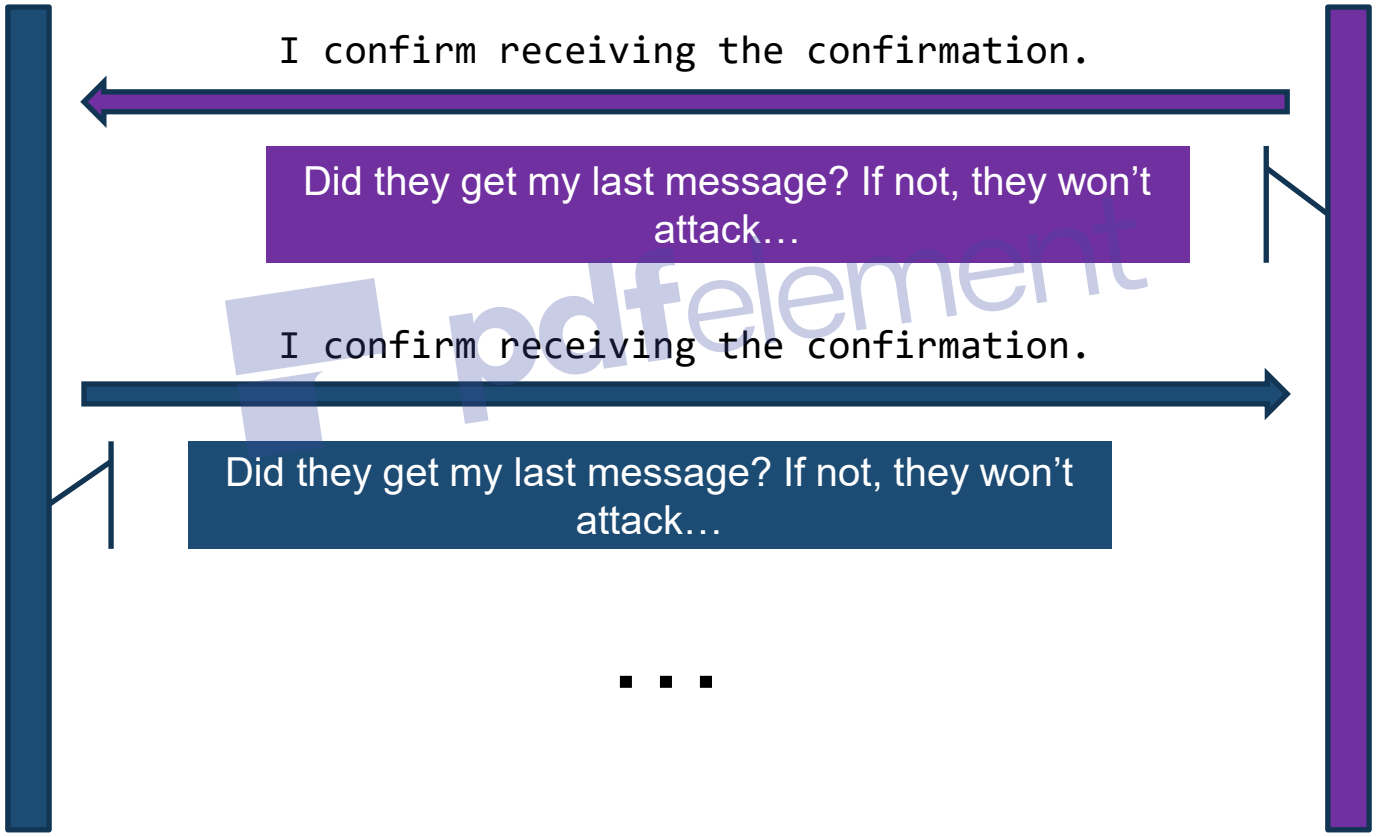
Destul de usor, nu?



pdfelement

Remove Watermark Now

Probabil ca nu…



pdfelement

Remove Watermark Now

Paradoxul celor doi generali

- Nu exista un protocol care sa garanteze ca ambii generali (noduri) sunt 100% siguri asupra deciziei pe care celalalt o va lua
  - Exista demonstratii stiintifice
- Dupa **500 confirmari**, ambii pot fi destul de siguri ca amandoi vor ataca (sau, vor lua aceeasi decizie)
- Dar “destul de siguri” nu **garanteaza 100% siguranta**

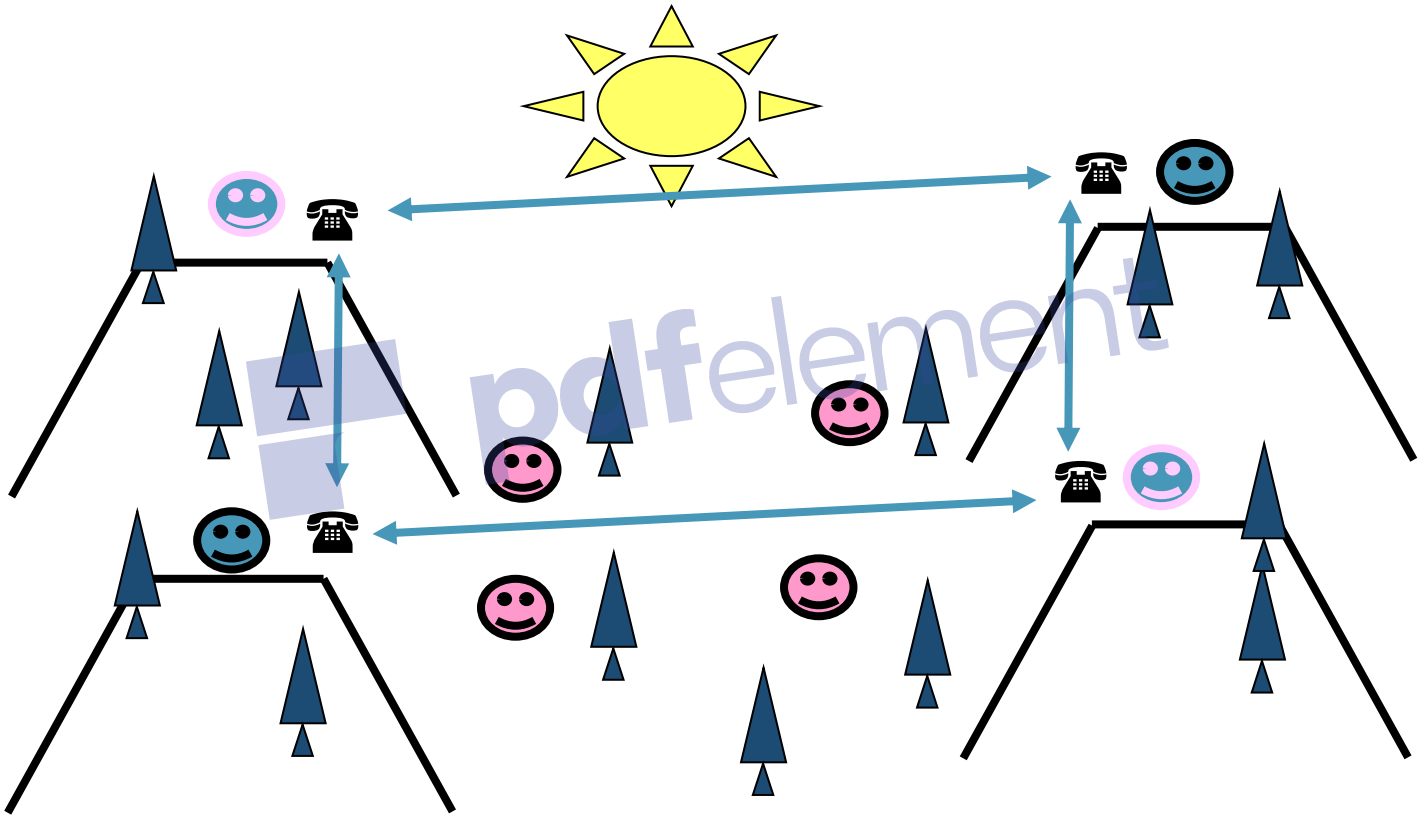
  Varianta simplificata a demonstratiei:
  - Presupunem ca exista un protocol care asigura ca dupa schimbul a **N mesaje**, se garanteaza siguranta in luarea deciziei
  - Al N-lea mesaj ar putea fi pierdut… deci, doar primele **N-1 mesaje** *inseamna ca au fost suficiente* pentru a garanta siguranta in luarea deciziei
  - **Deci,** trebuie sa existe un protocol care schimba doar **N-1 mesaje**
  - Concluzia: exista un protocol care schimba in final **0 mesaje**

- Exista protocoale ce mitigheaza nesiguranta data de trimiterea mesajelor, folosind reincercari, ACK-uri si timeouturi
  - TCP …

pdfelement

Remove Watermark Now

Problema generarilor bizantini

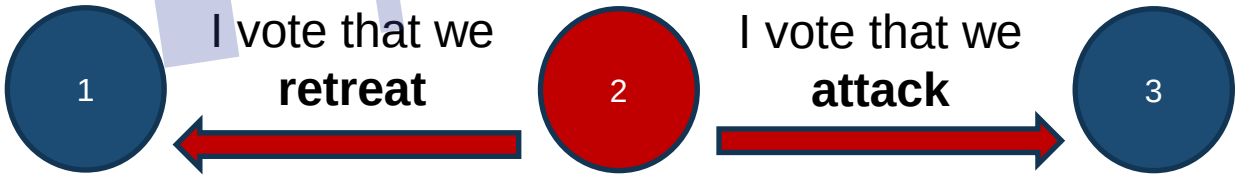


pdfelement

Remove Watermark Now

Problema generalilor Bizantini

- Generalizare a problemei celor doi generali
- N generali (noduri) trebuie sa decida asupra unei decizii de atac sau retragere pe baza *votului majoritatii*
- Unii generali pot fi in secret tradatori, si vor incerca sa *manipeze* votul…
- **Scop:** atingerea consensului intre noduri oneste
  - #1 wants to **attack**
  - #3 wants to **retreat**



  - #1 receives **retreat** votes from #2 and #3
  - #3 receives **attack** votes from #1 and #2
  - Result: #3 attacks *alone*, #1 retreats

pdfelement

Remove Watermark Now

Problema generalilor bizantini

- Fie v(i) – informația comunicată de către Generalul cu numărul *i*
- Fiecare General folosește o metodă pentru a combina valorile v(1), v(2), ..., v(n) într-un *plan de acțiune*

- Dacă decizia care poate fi luată este *atac* sau *retragere*, v(i) este opțiunea Generalului *i* dintre cele două variante, iar decizia finală pentru fiecare General se bazează pe votul majoritar (dintre cele *n* valori)

- Generalii își comunică unii altora valorile v(i)
- Generalii trădători pot trimite valori diferite celorlalți Generali
- Fiecare General ia decizia finală bazată pe aceeași mulțime v(1), v(2), ..., v(n)

pdfelement

Remove Watermark Now

Problema generalilor bizantini

- Restrângem abordarea problemei la modul în care *un* General își trimite valoarea celorlalți Generali
- Termenii problemei se schimbă: un General comandant trimite ordinul Locotenenților, obținându-se astfel problema:

- ***Problema Generalilor Bizantini***:
  - Un General comandant trebuie să trimită un ordin celor n – 1 Locotenenți astfel încât:
  - IC_1: Toți Locotenenții loiali se supun aceluiași ordin
  - IC_2: Dacă Generalul comandant este loial, atunci fiecare Locotenent loial se supune ordinului Generalului comandant

- IC_1, IC_2 – *interactive consistency conditions*
- Se observă că, pentru un Comandat loial, IC_1 rezultă din IC_2; totuși, Comandantul nu este în mod necesar loial

pdfelement

Remove Watermark Now

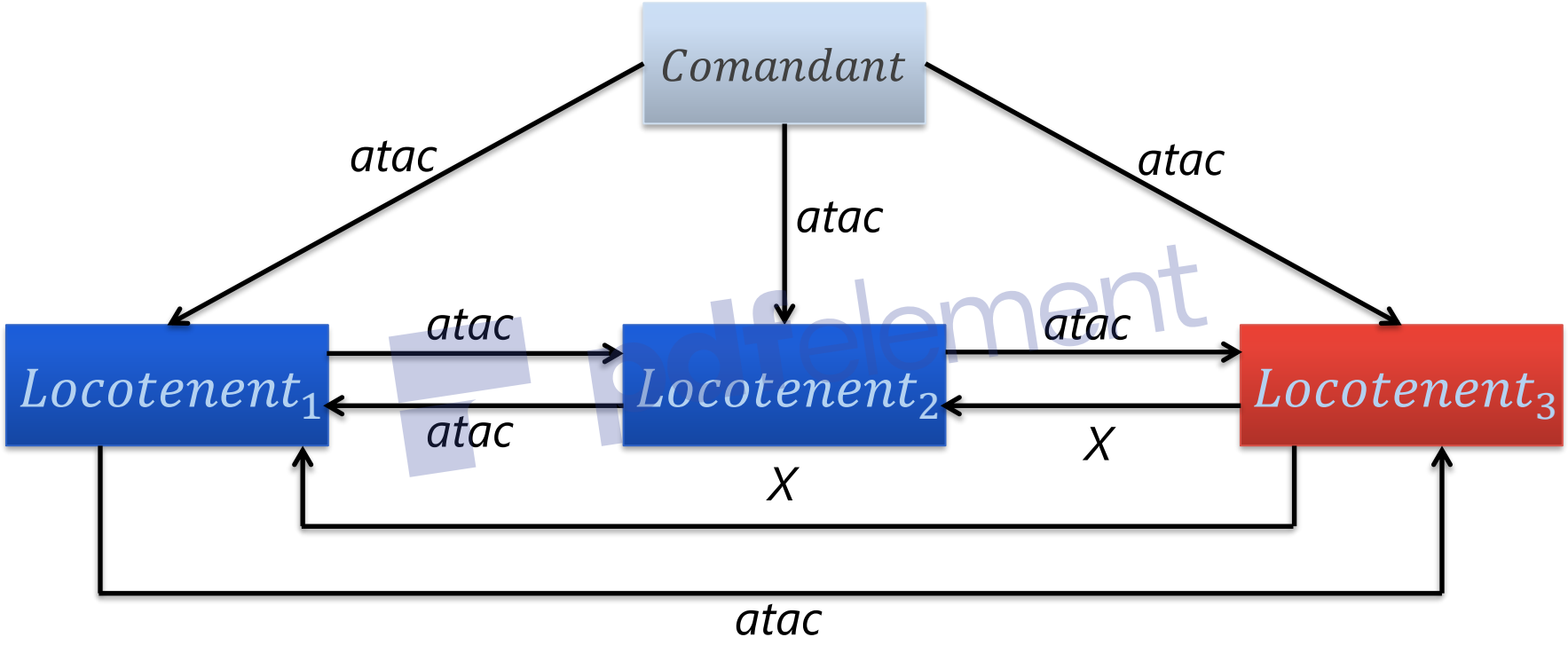
Problema generalilor bizantini

- Evenimentele sunt următoarele:
  - Comandantul trimite ordinul tuturor Locotenenților
  - Un Locotenent trimite celorlalți Locotenenți mesajul primit de la Comandant
  - După primirea mesajului de la Comandant și a copiilor de la ceilalți Locotenenți, un locotenent decide *prin vot majoritar* ce decizie va lua

pdfelement

Remove Watermark Now

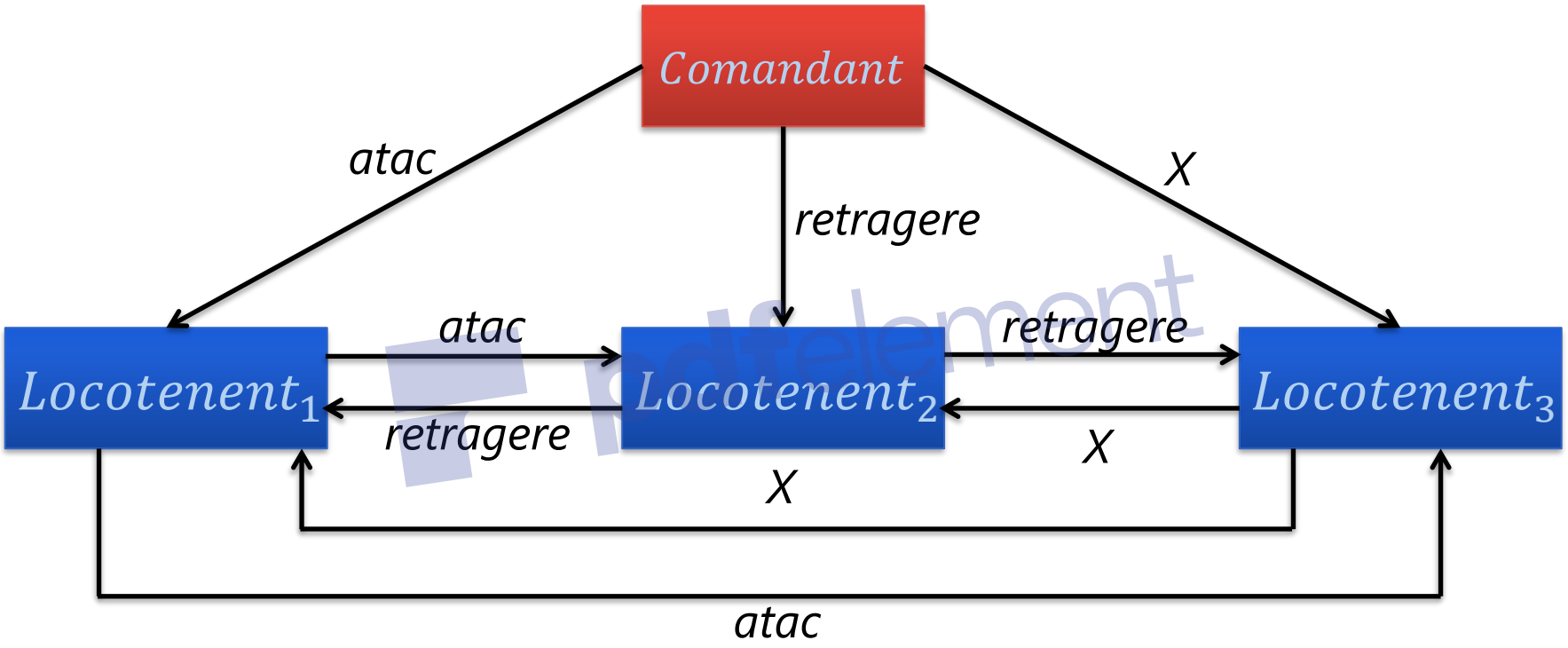
Problema generalilor bizantini
Exemplul 1



- $Locotenent_3$ este trădător
- $X \in \{atac, retragere\}$
- Oricare ar fi mesajul transmis de trădător, cei doi Locotenenți loiali vor lua aceeași decizie (*atac*)

Remove Watermark Now

Problema generalilor bizantini
Exemplul 2



- $Comandantul$ este trădător
- $X \in \{atac, retragere\}$
- Oricare ar fi mesajul transmis de trădător, toți Locotenenții vor lua aceeași decizie X

Remove Watermark Now

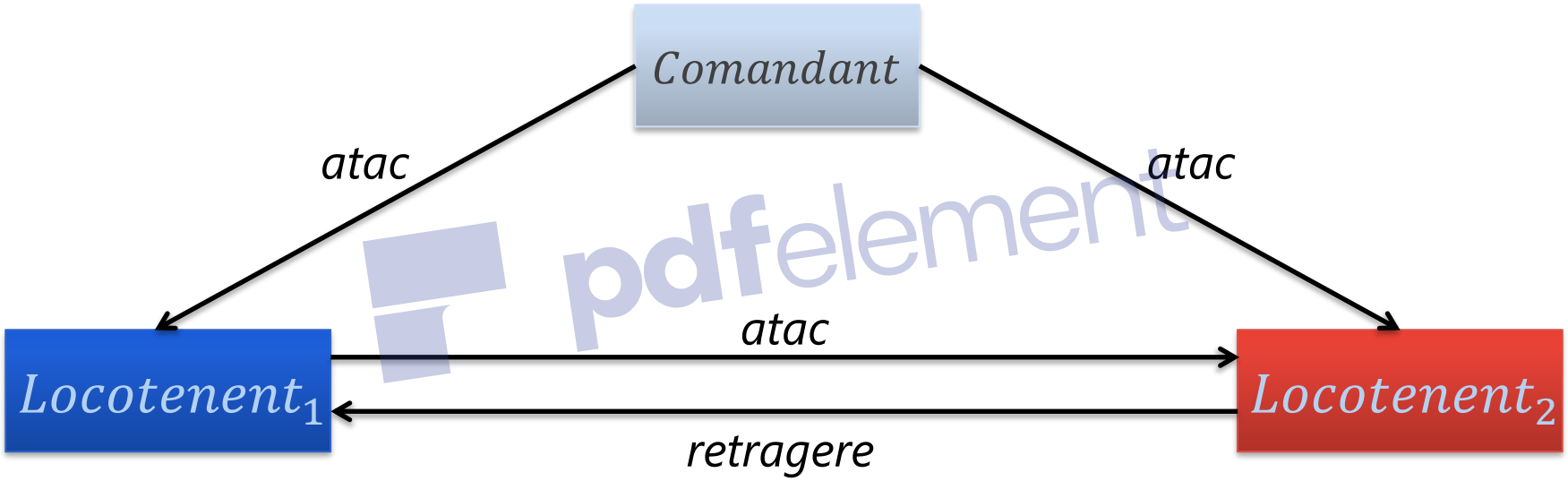
Problema generalilor bizantini

- Dificultatea problemei constă în faptul că:
  - Dacă Generalii (Locotenenții) pot trimite doar mesaje orale, atunci nu există soluție decât pentru cazul în care *2/3 din Generali sunt loiali*
- Un mesaj *oral* este aflat complet sub controlul emițătorului, deci un trădător poate trimite orice mesaj
- Mesajul trimis este *atac* / *retragere*
- Următoarele exemple: nu există soluție pentru 3 Generali, din care un trădător

pdfelement

Remove Watermark Now

Problema generalilor bizantini
Exemplul 3

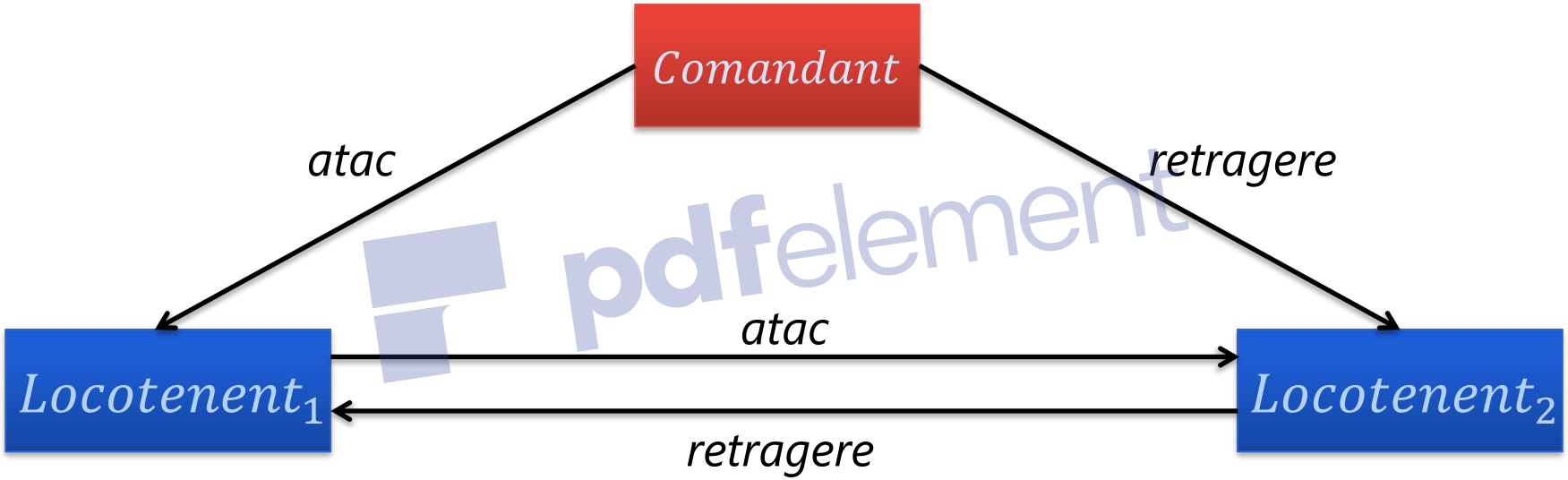


pdfelement

- $Locotenent_2$ este trădător

Remove Watermark Now

Problema generalilor bizantini
Exemplul 4



pdfelement

- *Comandantul* este trădător

Remove Watermark Now

Bitcoin / blockchain consensus

pdfelement

Remove Watermark Now

Ce este bitcoin?

- O valuta ce nu este influentata de **nicio** autoritate centrala, bazata pe criptografie si nu pe increderea intre oameni

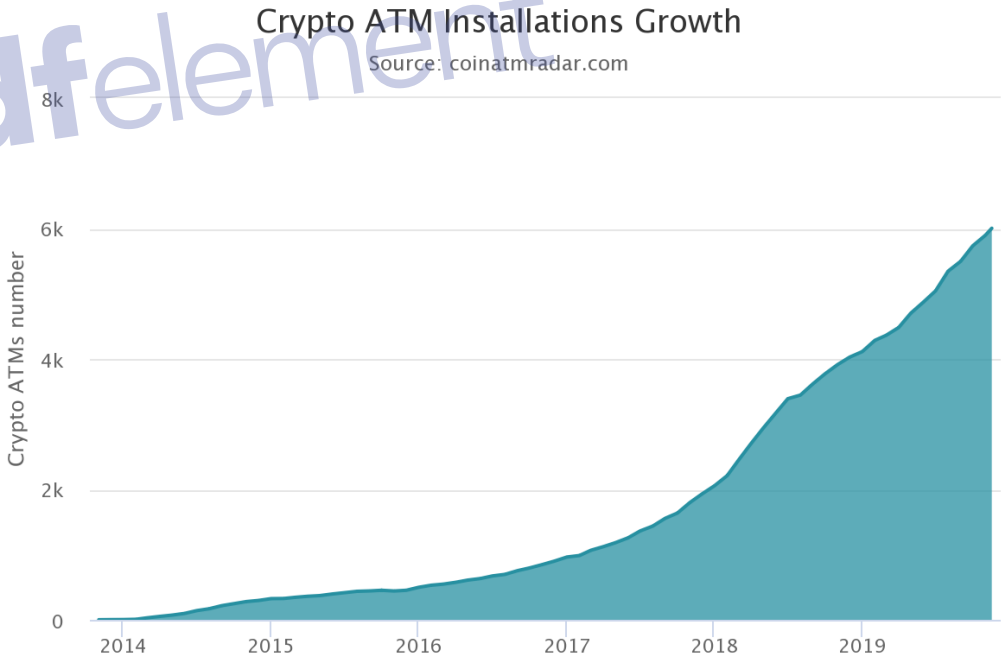
pdfelement

- O retea distribuita de noduri ce proceseaza tranzactii, *puternic rezilienta* la atacuri bizantine
  - **Oricine** poate rula propriul nod bitcoin
  - … ceea ce inseamna ca **nimeni** nu e considerat de incredere

Remove Watermark Now

Scurt istoric

- **Noiembrie 2008:** “Satoshi Nakamoto” publica lucrarea despre un “peer-to-peer electronic cash system”
- **Ianuarie 2009:** Reteaua bitcoin apare online, cu “Satoshi” fiind cel care mineaza primul block



pdfelement

Remove Watermark Now

Adrese Bitcoin

- Oricine poate genera o adresa pornind de la o pereche public-private key

Public: 1DwAdnHZ3w2Ecww6SPZZaGNMXVuWbdGZNY

Secret: L3H8Yz7YPshSHw5yjTH72cYy12UnUerFnzd8wjEDyhsYXSbhWU5q

pdfelement

- Oricine are o adresa publica poate **trimite** bitcoins la acea adresa
- Pentru a-i **cheltui**, trebuie sa stiti partea secreta a cheii

Remove Watermark Now

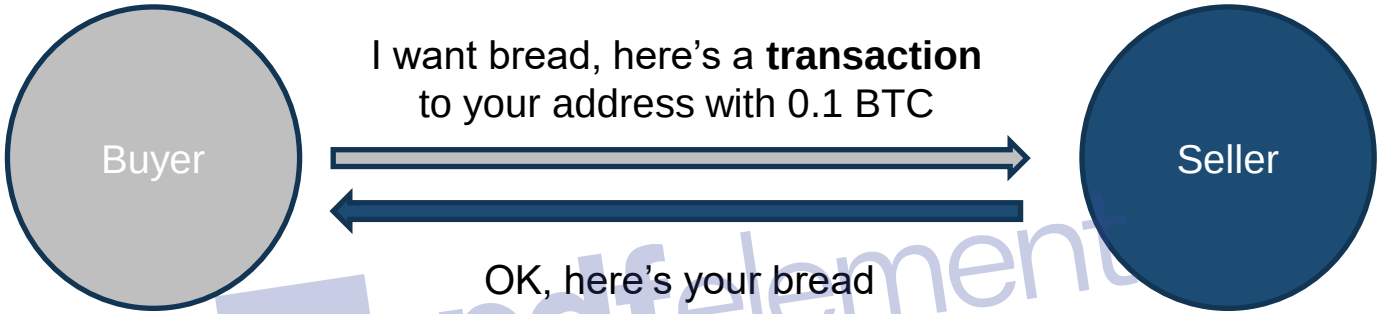
O tranzactie bitcoin (super simplificata)

- Intrari: iesirile unor tranzactii *anterioare*
  - Iesirea #2 tranzactiei *3b96bb7e197e...*
  - Iesirea #1 tranzactiei *e1afd89295b68…*
  - Iesirea #6 tranzactiei *e79fc1dad370e…*
- Iesiri
  - 0.1 BTC catre adresa **1HmxmBAX413yGY2LDoEN8FHBok61aT4w2d**
  - 0.2 BTC catre adresa **1Hozk3UFZDsGGd7PENAVgy3ouFHzvsCFJ7**
- Semnaturi ce dovedesc **ownership-ul** adreselor de intrare

pdfelement

Remove Watermark Now

E suficient? Nu!



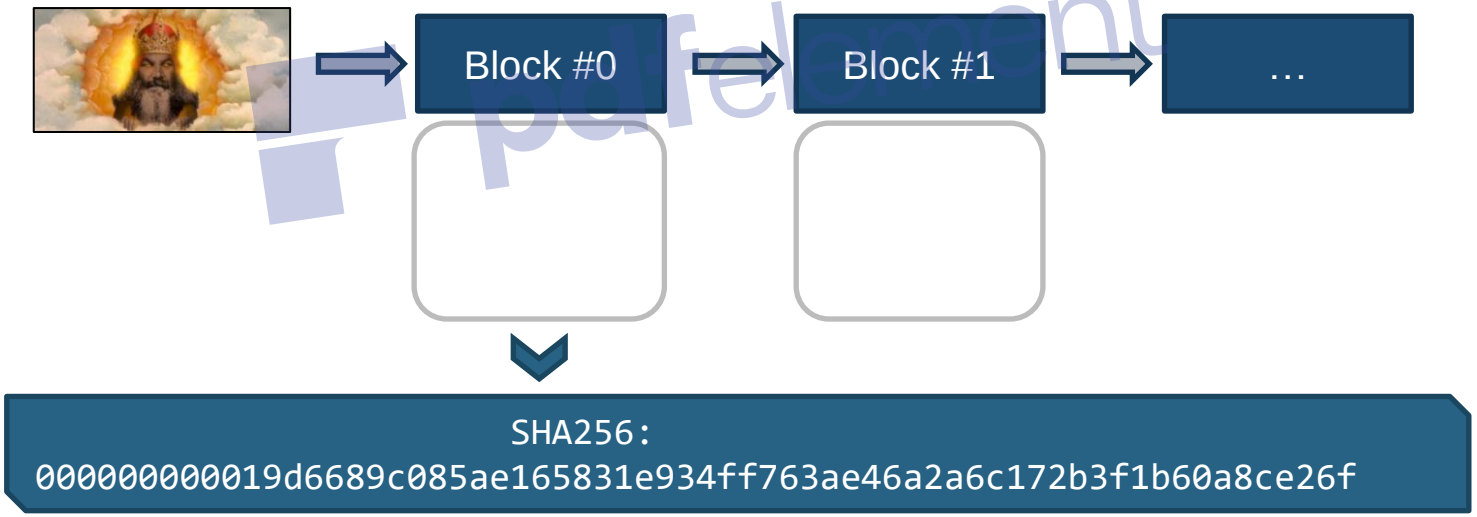
pdfelement

- Vanzatorul trebuie sa poata verifica ca tranzactia este **valida**
- **Sursele de intrare** pot sa **nu existe**, sau cumparatorul poate ca nu are **dreptul de a le cheltui** (invalid signature)

Remove Watermark Now

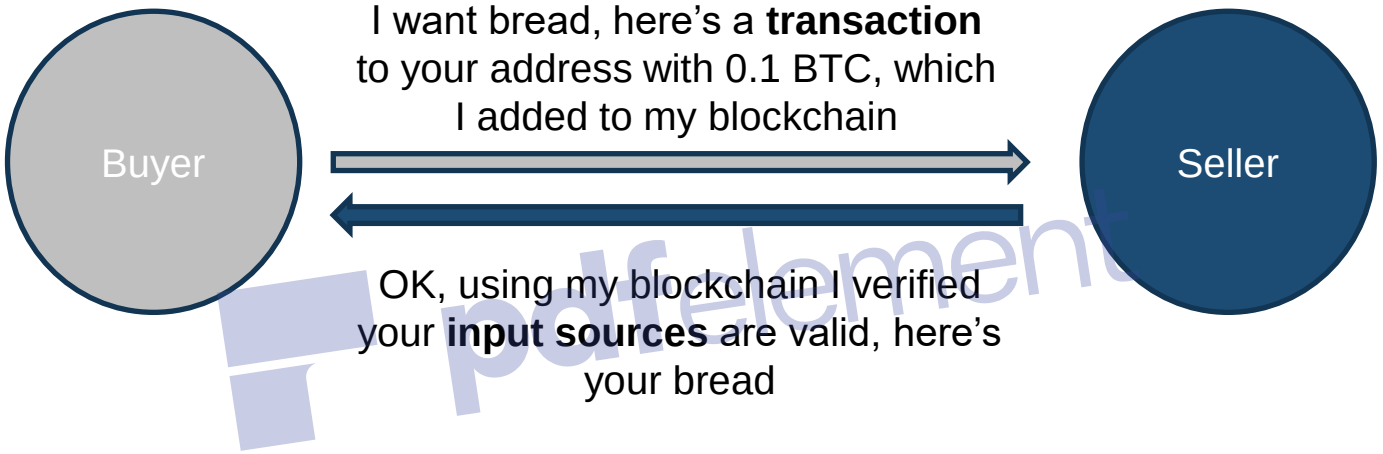
Blockchain-ul bitcoin (super simplificat)

- Un log replicat al *tuturor* tranzactiilor bitcoin de la inceputul timpului
- Fiecare block contine o lista a tranzactiilor semnate, dar si hash-ul *SHA256* al unui bloc *anterior*



Remove Watermark Now

Este de ajuns? Inca nu



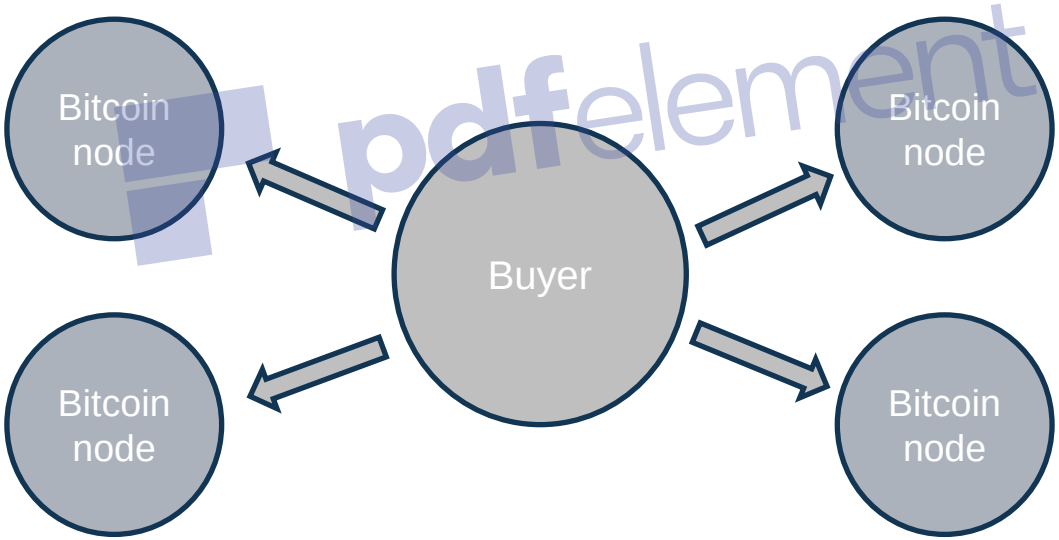
pdfelement

- Noua tranzactie trebuie sa ajunga in **blockchain-ul** tuturor nodurilor bitcoin
- **Toate nodurile** din retea trebuie sa fie constiente de tranzactie, altfel vanzatorul **nu poate folosi** monedele

Remove Watermark Now

Transaction broadcasting

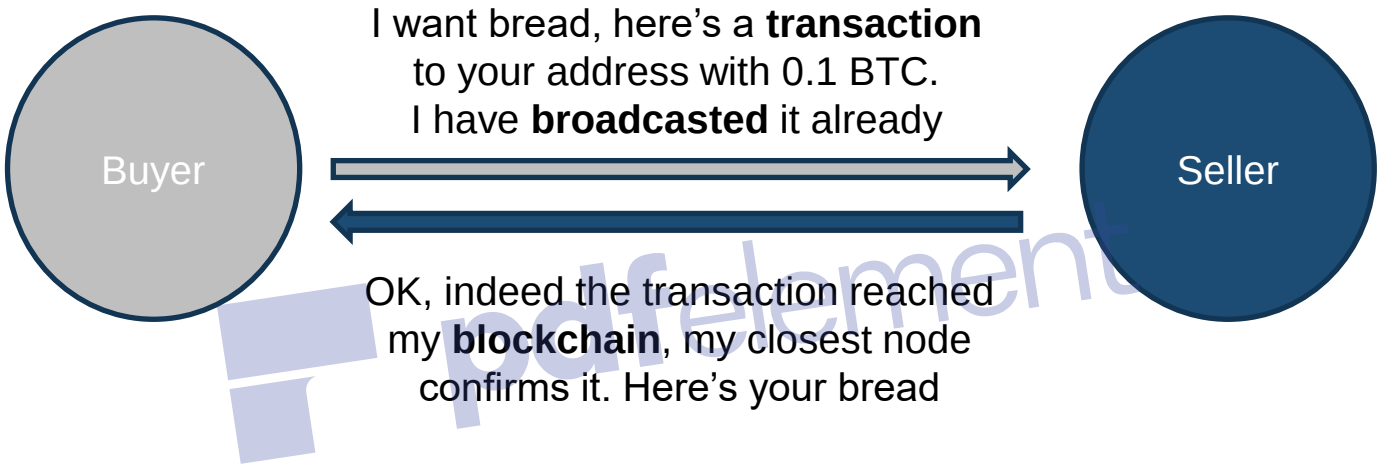
"I declare I want to send 0.1 BTC from *12vEsiYhQ3iUJK2Mpt5eLq2cgxewQLw4tr* to *16yZiBEmG3AKSUkHm3oHo3PUMPUWUe72tv*, and here's a signature to prove I own these bitcoins. Please add my transaction to your blockchains"



pdfelement

Remove Watermark Now

Este de ajuns? Poate…



pdfElement

- Nu este sufficient ca **un singur nod** confirma tranzactia
- Vanzatorul trebuie sa fie sigur ca **intreaga retea** o confirma
  - daca nu, nu va mai putea cheltui monedele in viitor

Remove Watermark Now

Reaching blockchain consensus

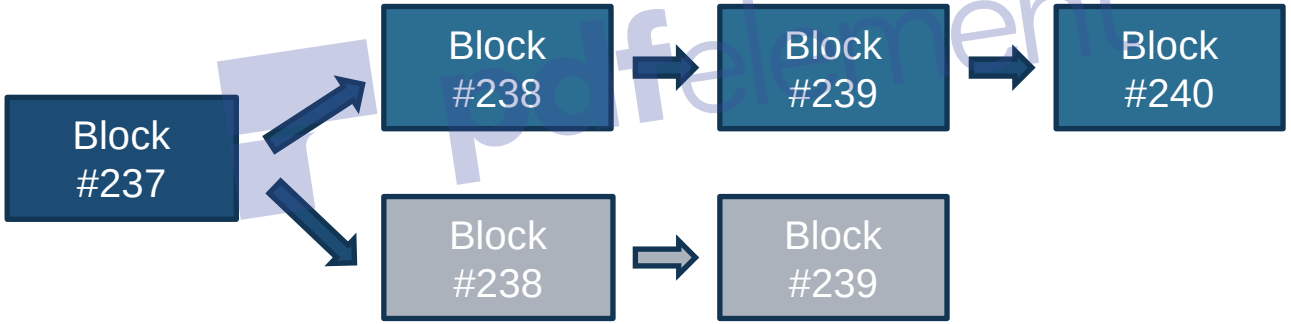
- Nodurile Bitcoin vor primi tranzactia intr-o ordine posibil **diferita**
- **Este critic** ca nodurile sa ajunga la un *acord* asupra ordinii
  - Si nu, eventual consistency nu mai functioneaza aici
- **Double-spending**: Doua tranzactii ce consuma aceiasi sursa de intrare (cineva vinde pe aceeasi bani de doua ori) – doar *prima* tranzactie poate fi valida
- Toate nodurile trebuie sa ajunga la consens asupra *tranzactiei care e prima*, respectiv care trebuie eventual invalidata

pdfelement

Remove Watermark Now

Blockchain forks

- Prin definitie, “adevaratul” blockchain este cel mai lung
- Un nod onest notificat de un **blockchain mai lung** trebuie sa comute pe folosirea acestuia **imediat**



- Tranzactiile din blocurile ramase “orfane” sunt retrase/anulate

Remove Watermark Now

Blockchain forks (2)

- “Longest blockchain wins”: toate nodurile bitcoin converg rapid catre un blockchain comun

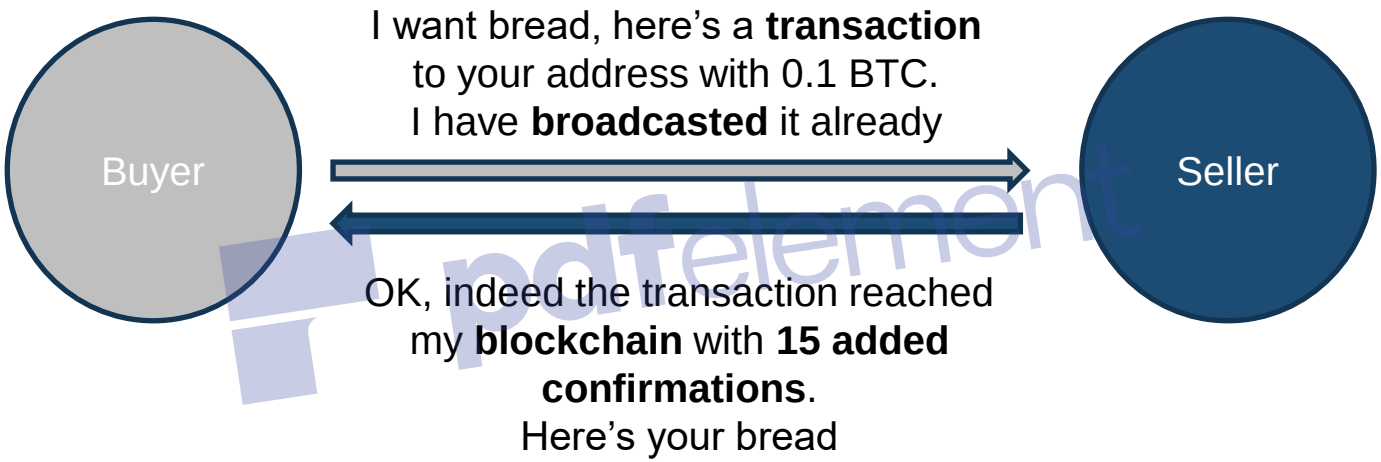
- Cu cat sunt mai multe blocuri adaugate pe chain, cu *atat mai improbabil* e ca tranzactiile noastre sa fie retrase

pdfelement

- In plus, extra blocuri servesc unor *confirmari suplimentare* prin care nodurile bitcoin confirma tranzactia noastra
  - Aceste blocuri confirma **validitatea** tranzactiei
  - si confirma ca tranzactia face parte din **cel mai lung** blockchain cunoscut de respectivele noduri

Remove Watermark Now

E suficient? Aproape…



pdfelement

- Vanzatorul poate confirma criptografic ca sunt alte 15 blocuri valide adaugate dupa cel cu tranzactia curenta
  - aproape sigur ca orice fork de aici incolo va contine tranzactia… **sau nu?**

Remove Watermark Now

Un atac bizantin

Ceva lipseste… urmatorul atac ar putea avea success:

1. Publish a transaction, which is added to block *#190*
2. Wait for *20* confirmations
3. Seller is reasonably sure the network reached consensus on this transaction
4. Seller gives you bread
5. Publish a **new blockchain**, forking from node *#189* with *30* empty blocks after it
6. Network switches to your blockchain, since it's **longer**
7. You get to keep **both** the bread and the coins

pdfelement

Remove Watermark Now

Proof of work

- Solutia la scenariul anterior: *sa facem costisitoare* operatie de adaugare a unor noi blocuri

- Orice hash SHA256 al unui bloc trebuie sa inceapa cu un anuimit numar de zero-uri, altfel nodurile oneste il vor *rejecta*

pdfelement

- Singura metoda pentru a produce un bloc consta in **forta bruta**
  - Formatul de bloc contine un contor exact pentru acest scop, ce influenteaza rezultatul aplicarii SHA256
  - Acesta trebuie incrementat pana cand reusim sa generam un SHA256 valid
  - In alte variante de monede virtuale exista si alte versiuni ale Proof-of-Work

Remove Watermark Now

Atacuri 51%

- Nodurile Bitcoin constant incearca adaugarea de noi blocuri – exista **recompense** pentru cele care reusesc

- Un atacator trebuie sa le poata **intrece** pentru a produce blocuri mai rapid decat **restul retelei combinat**

pdfelement

- Acest lucru este posibil doar de catre atacatori ce detin peste *50%* din totalul **puterii computationale de generare a SHA256** a intregii retele
  - Un astfel de atac devine extrem de scump

Remove Watermark Now

"SHA256 computation arms race

- Cele mai bune **CPU-uri**: *10 – 100* milioane de hash-uri per sec
- Cele mai bune **GPU-uri**: *100 – 1000* milioane de hash-uri per sec

Hardware specializat pe minare de bitcoin: **FPGA**, **ASIC**

- Un **ASIC** lansat in 2012: *60,000* M hashes / s
- Un **ASIC** astazi (2019): *14,000,000* M hashes / s

Antminer S9

pdfelement

Remove Watermark Now

Examen

pdfelement

Remove Watermark Now

Condiții examen (primavara)

- Timp de lucru: 1 ora si 20 minute
- *8 intrebari, fiecare întrebare valorează 0.5 puncte*
- *pot fi date întrebări din oricare din cursurile teoretice sau practice predate la aceasta materie.*
- Fără documentație (*closed book*)

pdfelement

Remove Watermark Now

Exemple de subiecte

1. Definiți conceptul de “Split binary semaphore”. Explicați cum se realizează tehnica de pasare a ștafetei.
2. Reprezentați grafic modelul Replicated Workers. Care este condiția de terminare?
3. Corectați pseudocodul de mai jos *(parallel Scan)* pentru a funcționa întotdeauna corect in cazul a n procese (thread-uri) executate in paralel pe o arhitectura de tipul *MIMD*.

```
int a[1:n];
process parallel_Scan[k=1 to n] {
    for (j = 1; j < sup(log2 n); j++)
        if (k - 2^(j-1) >= 1)
         a[k] = a[k-2^(j-1)] ⊕ a[k];
  }
```

⊕ = operație ce se aplica pe elementele date ca input

4. Definiți timpul total de execuție in *modelul Foster* (distribuit) si cum se calculează acesta (T = ?) Explicați pe scurt fiecare din cele 3 subcomponente ale acestuia.
5. Considerați o implementare de semafor distribuit cu doua tipuri de procese: procesele ce apelează operațiile semaforului (P si V) - **Utiliz(i)**, si cele care implementează logica semaforului distribuit - **Ajutor(i).** Scrieți pseudocodul pentru procesele *Ajutor* in cazul recepționarii unui mesaj:

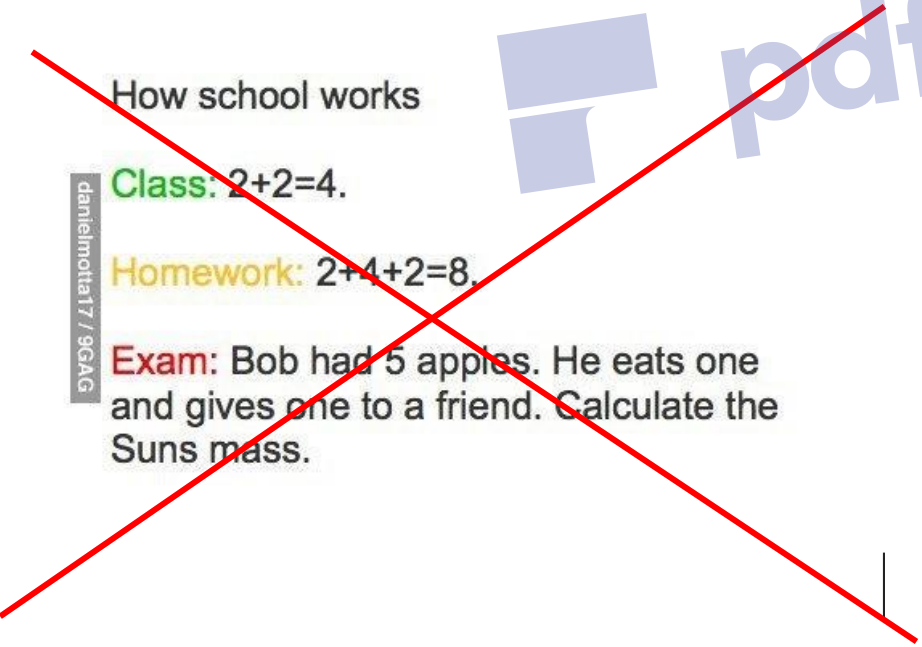
```
while (true) {
        receive opsem[i](transm, k, ts);   // transm = ID sender,  k = felul mesajului,
                                           // ts = timestamp-ul de trimitere
        // TODO
```

pdfelement

Remove Watermark Now

Condiții examen (toamna)

- Timp de lucru: 2 ore
- Fără documentație (*closed book*)



pdfelement

Remove Watermark Now

Un model de subiect de examen la APD

Subiectul 1. (2 puncte) Ceasuri logice vectoriale. Tratați:

1.1 Conceptul general (0.7 p)

Motivatia folosirii ceasurilor logice: de ce este nevoie de ele (ce aduce nou față de alte metode).

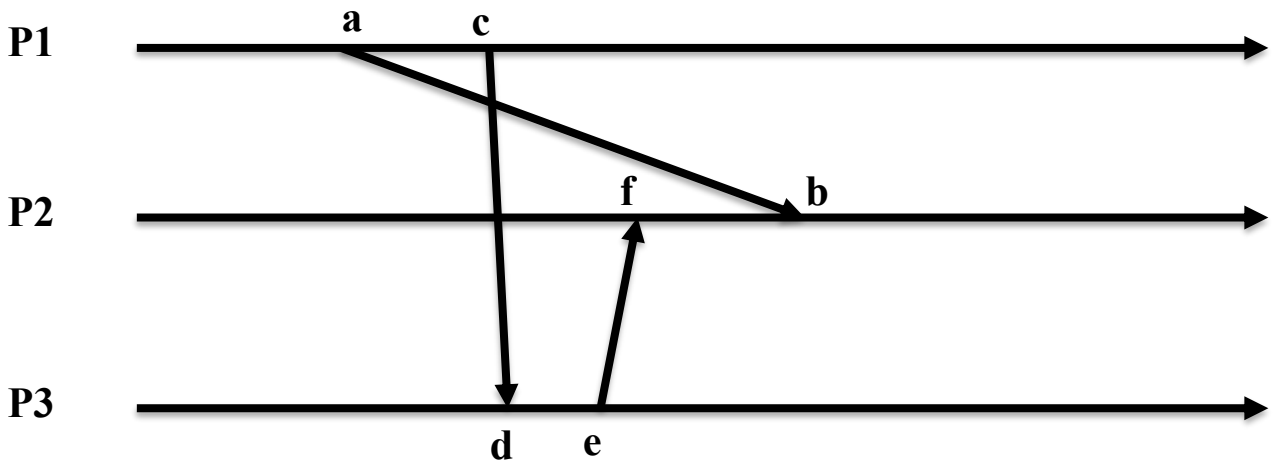
Principiul ceasurilor logice vectoriale.

1.2 Pentru procesele din figură, precizați vectorii de timp asociați evenimentelor specificate. Axele orizontale reprezintă timpul. (0.3 p)

1.3 Aplicație - ordonarea cauzală *multicast*. (1 p)

Specificarea problemei.

Soluția – cu descrierea acțiunilor la trimiterea, recepția și livrarea mesajelor.



Remove Watermark Now

Un model de subiect de examen la APD

Subiectul 2. (1 punct)

Propuneti un algoritm de *tip heartbeat* pentru calculul sumei a n^2 valori de tip întreg. Folosiți n^2 procese dispuse într-o grilă neperiodică.
Deci, fiecare proces poate comunica cu vecinii de la *est*, *vest*, *nord* și *sud*.
Fiecare proces deține, inițial, o singură valoare *v*.
În final, fiecare proces trebuie să dețină valoarea sumei.

Calculați complexitatea soluției oferite.

pdfelement

Remove Watermark Now

Un model de subiect de examen la APD

Subiectul 3. (1 punct) Tratați unul din subiectele urmatoare, la alegere:

3.1. Cautarea paralelă

Prezentarea problemei (0.2 p)

Descrierea algoritmului (0.4 p)

Analiza complexității (0.4 p)

3.2. Problema cititorilor și scriitorilor

Prezentarea problemei (0.2 p)

Descrierea algoritmului (0.4 p)

Politici cu prioritate asupra cititorilor și asupra scriitorilor (0.4 p)

pdfelement

Remove Watermark Now

Detalii finale

- Punctajele vor fi afișate pe site
- Nu aveți voie să veniți cu alta grupă decât cu aprobarea titularului de curs (pentru probleme speciale, vă rog să mă contactați)
- Pentru nelămuriri, documentație, explicații suplimentare:
  - open office: *PRECIS 605*
  - ciprian.dobre@cs.pub.ro
- …

pdfelement

Baftă în sesiune ☺ !

Si nu uitati ca acceptam studenti la practica de vara!
(nu e rau sa porniti lucrarea de licenta inca din vara)

Remove Watermark Now

Distributed Commit

pdfelement

Remove Watermark Now

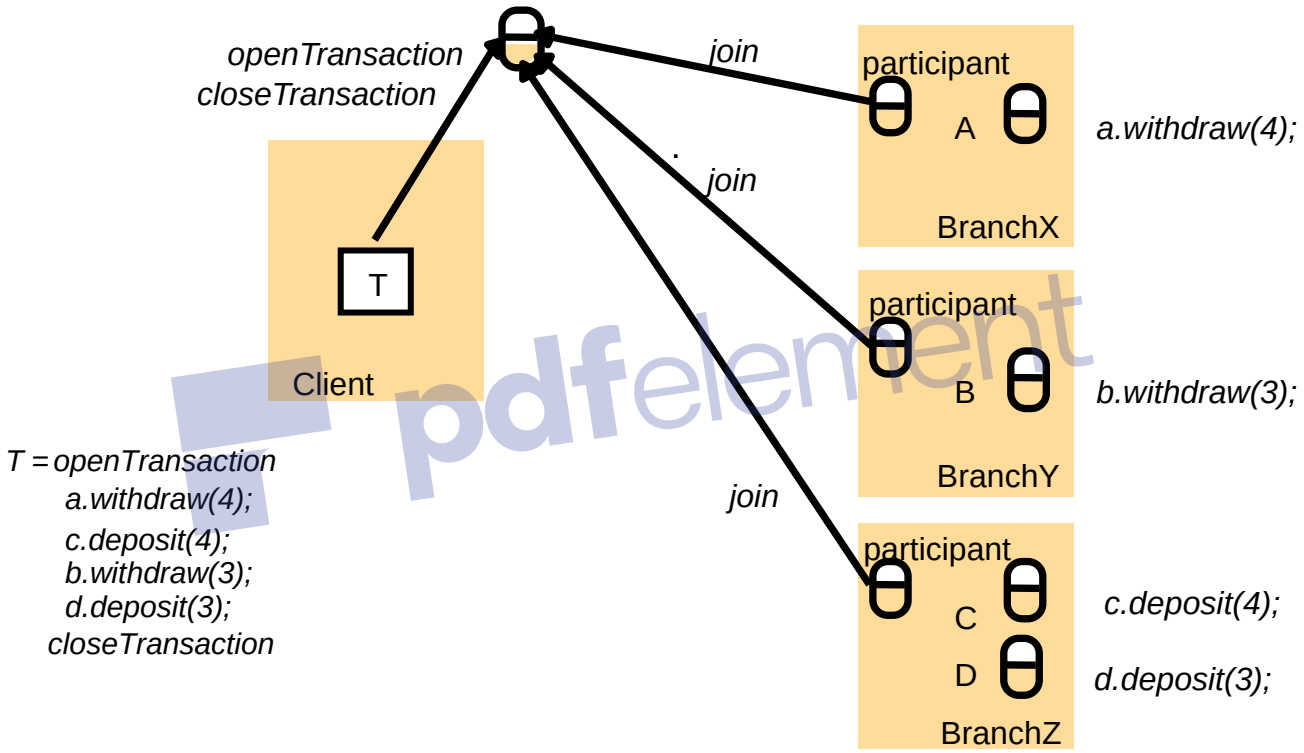
Distributed Commit

- **Scop**: fie **toti** membrii unui grup decid asupra efectuarii unei operatii, fie **niciunul** nu efectueaza operatia
- **Atomic transaction**: o tranzactie ce se efectueaza cu totul sau deloc
- Defecte:
  - Cele de tip Crash ce pot fi recuperate
  - Defecte de comunicatie detectabile prin timeouturi
- Observatii:
  - Commit-ul necesita un set de procese sa ajunga la un acord…
  - …similar problemei generalilor Bizantini …
  - … dar solutia este mult mai simpla deoarece avem ipoteze/constrangeri mai puternice

pdfelement

Remove Watermark Now

O tranzactie bancara distribuita



pdfelement

Remove Watermark Now

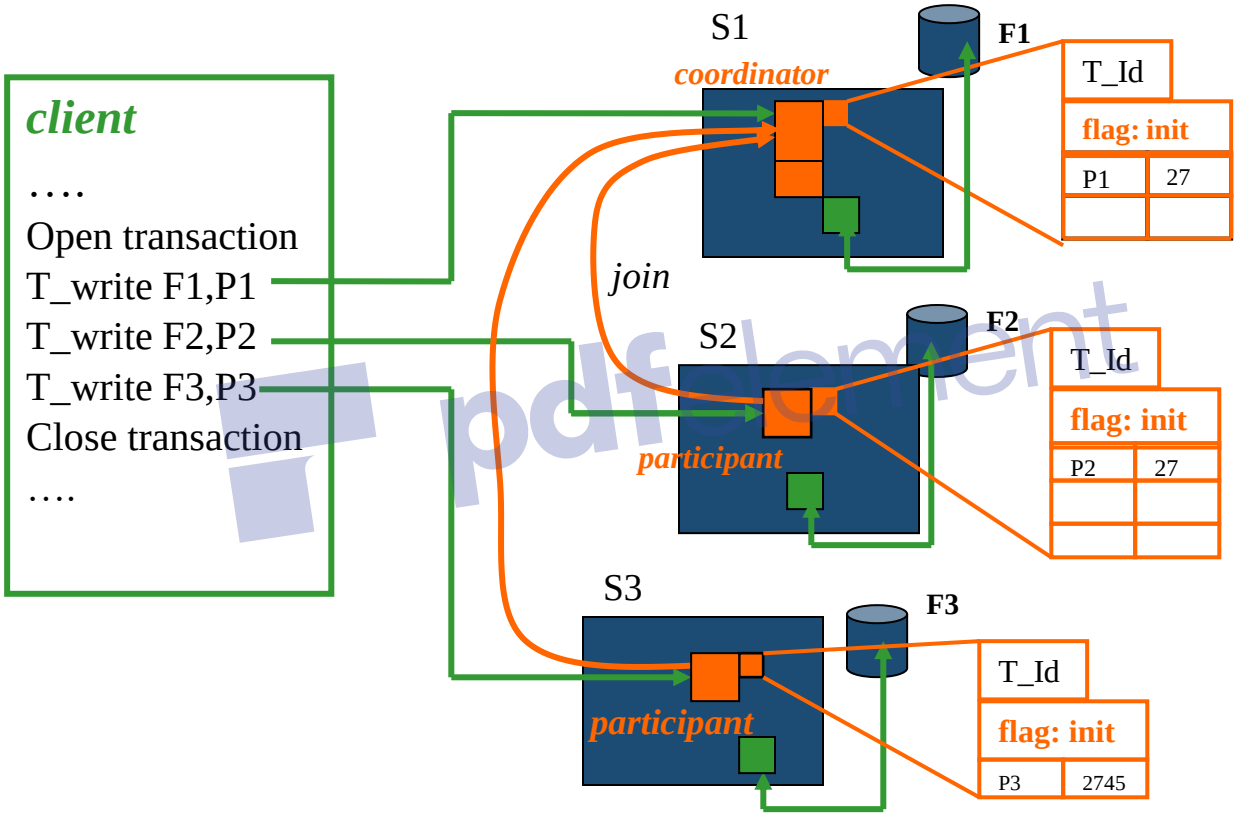
One-phase Commit

- Protocolul one-phase commit
  - Un site este desemnat ca si coordinator
  - Coordinatorul instiinteaza toate celelalte procese daca efectueaza sau nu local o opearatie
  - Schema aceasta insa nu este fault tolerant

pdfelement

Remove Watermark Now

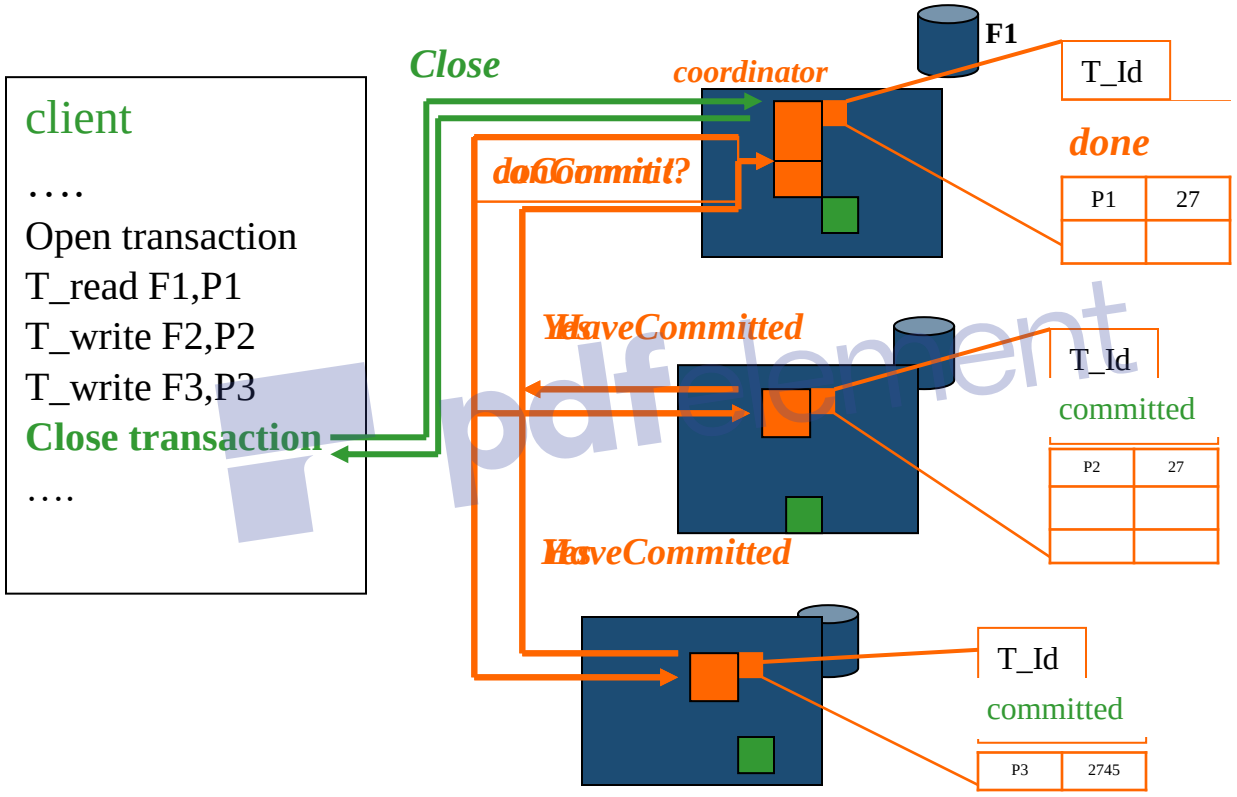
Transaction Processing (1)



pdfelement

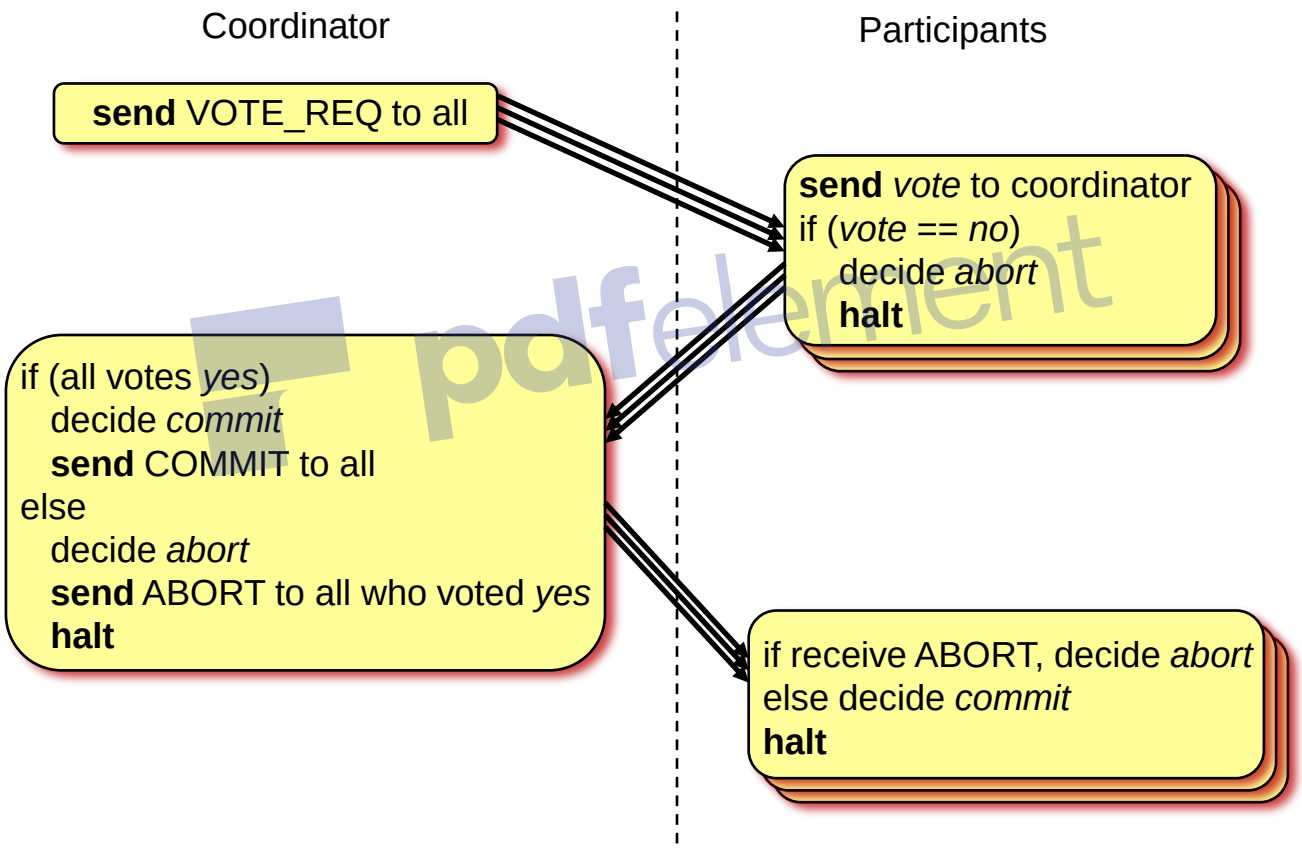
Remove Watermark Now

Transaction Processing (2)



Remove Watermark Now

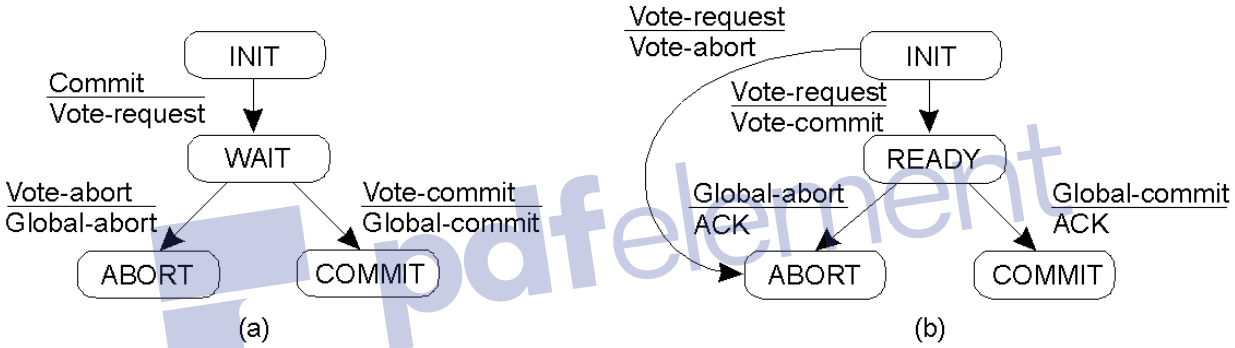
Two Phase Commit (2PC)



pdfelement

Remove Watermark Now

Two-Phase Commit



a) Masina de stari finite pentru coordonator, in 2PC.

b) Masina de stari finite pentru un participant.

Remove Watermark Now

Two-Phase Commit

actions by coordinator:

```
write START _2PC to local log;
multicast VOTE_REQUEST to all participants;
while not all votes have been collected {
    wait for any incoming vote;
    if timeout {
        write GLOBAL_ABORT to local log;
        multicast  GLOBAL_ABORT to all participants;
        exit;
    }
    record vote;
}
if all participants sent VOTE_COMMIT and coordinator
votes COMMIT{
    write GLOBAL_COMMIT to local log;
    multicast GLOBAL_COMMIT to all participants;
} else {
    write GLOBAL_ABORT  to local log;
    multicast GLOBAL_ABORT to all participants;
}
```

pdfelement

Remove Watermark Now

Two-Phase Commit

actions by participant:

```
write INIT to local log;
wait for VOTE_REQUEST from coordinator;
if timeout {
    write VOTE_ABORT to local log;
    exit;
}
if participant votes COMMIT {
    write VOTE_COMMIT to local log;
    send VOTE_COMMIT to coordinator;
    wait for DECISION from coordinator;
    if timeout {
        multicast DECISION_REQUEST to other participants;
        wait until DECISION is received; /* remain blocked */
        write DECISION to local log;
    }
    if DECISION == GLOBAL_COMMIT
        write GLOBAL_COMMIT to local log;
    else if DECISION == GLOBAL_ABORT
        write GLOBAL_ABORT to local log;
} else {
    write VOTE_ABORT to local log;
    send  VOTE ABORT to coordinator;
}
```

pdfelement

Remove Watermark Now

Two-Phase Commit

actions for handling decision requests: /* executed by separate thread */

```
while true {
    wait until any incoming DECISION_REQUEST is received; /* remain blocked */
    read most recently recorded STATE from the local log;
    if STATE == GLOBAL_COMMIT
        send GLOBAL_COMMIT to requesting participant;
    else if STATE == INIT or STATE == GLOBAL_ABORT
        send GLOBAL_ABORT to requesting participant;
    else
        skip;  /* participant remains blocked */
```

pdfelement

Steps taken by participant process for handling incoming decision requests.

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Pthread + Primitive sincronizare

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

POSIX threads

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);

void * threadFunction(void* arg)
{

}

pthread_join(thread, NULL);
```

pdfelement

Remove Watermark Now

Compilare pthread

gcc –o executabil cod.c –lpthread –lrt

pdfelement

#include<pthread.h>

#include<semaphore.h>

Remove Watermark Now

Pthread

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);


void * threadFunction(void* arg)
{

}

pthread_join(thread, NULL);
```

pdfelement

Acest element reprezintă thread-ul.
Poate fi considerat handle

Remove Watermark Now

Pthread

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);


void * threadFunction(void* arg)
{
        Aici am putea să facem recomandări
        sistemului de operare. Gen să
        folosească anumite core-uri.
}


pthread_join(thread, NULL);
```

pdfelement

Remove Watermark Now

Pthread

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);

void * threadFunction(void* arg)
{

}

pthread_join(thread, NULL);
```

Când se crează thread-ul, acesta va începe la această funcție.

pdfelement

Remove Watermark Now

Pthread

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);

void * threadFunction(void* arg)
{

}

pthread_join(thread, NULL);
```

Așa trimitem date thread-ului

pdfelement

Remove Watermark Now

Pthread

```
pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);

void * threadFunction(void* arg)
{
        Astfel se pot extrage
        informații din thread
}

pthread_join(thread, NULL);
```

pdfelement

Remove Watermark Now

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
     pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
     pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
void* threadFunction(void *arg)
{
   a=a+1;
   return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
void* threadFunction(void *arg)
{
   a=a+1;
   return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
void* threadFunction(void *arg)
{
   a=a+1;
   return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

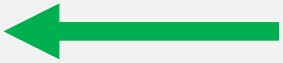
pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```



```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```

pdfelement

Remove Watermark Now

Pthread

```
void* threadFunction(void *arg)
{
   a=a+1;
   return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

```
void* threadFunction(void *arg)
{
   a=a+1;
   return NULL;
 }
```

pdfelement

Remove Watermark Now

Pthread

```
void* threadFunction(void *arg)
{
   a=a+1;
   return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

```
void* threadFunction(void *arg)
{
   a=a+1;
   return NULL;
 }
```

pdfelement

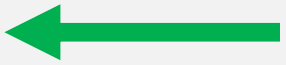
Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
}
```



pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int i, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

```
void* threadFunction(void *arg)
{
  a=a+1;
  return NULL;
 }
```

pdfelement

Remove Watermark Now

Pthread

```
void* threadFunction(void *arg)
{
   a=a+1;
   return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
void* threadFunction(void *arg)
{
   a=a+1;
   return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
void* threadFunction(void *arg)
{
   a=a+1;
   return NULL;
 }
```

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
  }
  for(i = 0; i < P; i++) {
    pthread_join(tid[i], NULL);
  }
  return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
     pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
      pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Pthread

```
int main(int argc, char **argv)
{
  int I, P=2;
  pthread_t tid[P];
  for(i = 0; i < P; i++) {
    pthread_create(&(tid[i]), NULL, threadFunction, NULL);
   }
   for(i = 0; i < P; i++) {
     pthread_join(tid[i], NULL);
   }
   return 0;
}
```

pdfelement

Remove Watermark Now

Remove Watermark Now

Mutex Pthread

ÎN MAIN

Înainte de a porni thread-urile

pdfelement

pthread_mutex_t **mutex**;

pthread_mutex_init(&**mutex**, NULL);

Remove Watermark Now

Mutex Pthread

```
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);
```

pdfelement

Poate fi folosit să anunțe că acest mutex e împărțit mai multor procese

Remove Watermark Now

Mutex Pthread

pthread_mutex_lock(&**mutex**);
load(a, eax)

eax = eax + 2

write(a, eax)

pthread_mutex_unlock(&**mutex**);

pdfelement

Remove Watermark Now

Mutex Pthread

ÎN MAIN

După ce au terminat thread-urile

pdfelement

pthread_mutex_destroy(&**mutex**);

Remove Watermark Now

Remove Watermark Now

Semaphore

ÎN MAIN

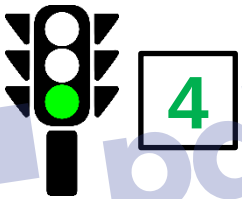
Înainte de a porni thread-urile

pdfelement

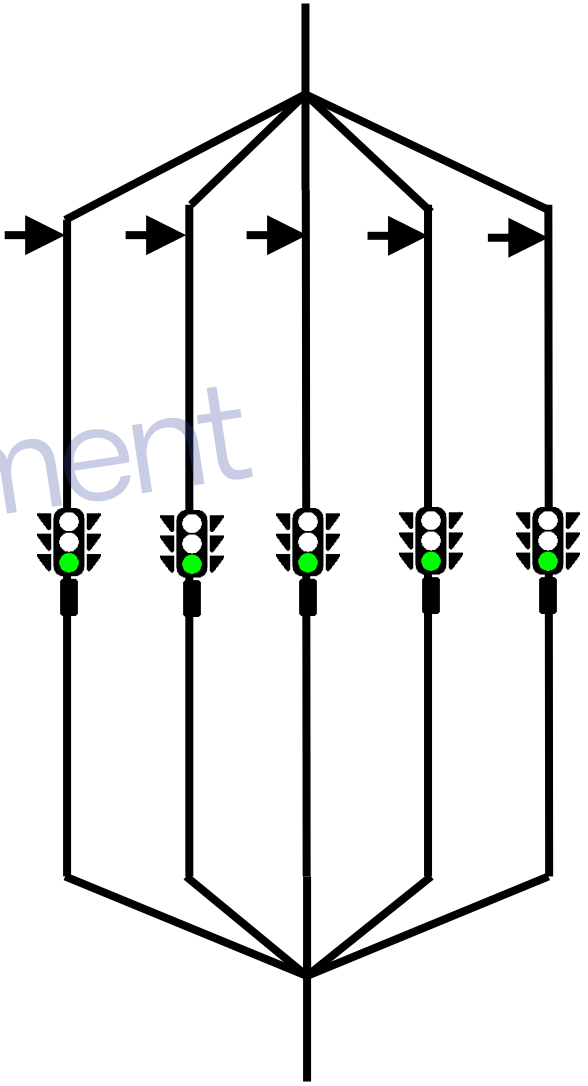
```
sem_t semaphore;
int semaphore_value= 4;
sem_init(& semaphore, 0, semaphore_value);
```

Remove Watermark Now

Semaphore




```
sem_wait(&semaphore);
```



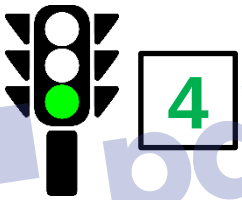
P() sau Proberen
 - Dijsktra

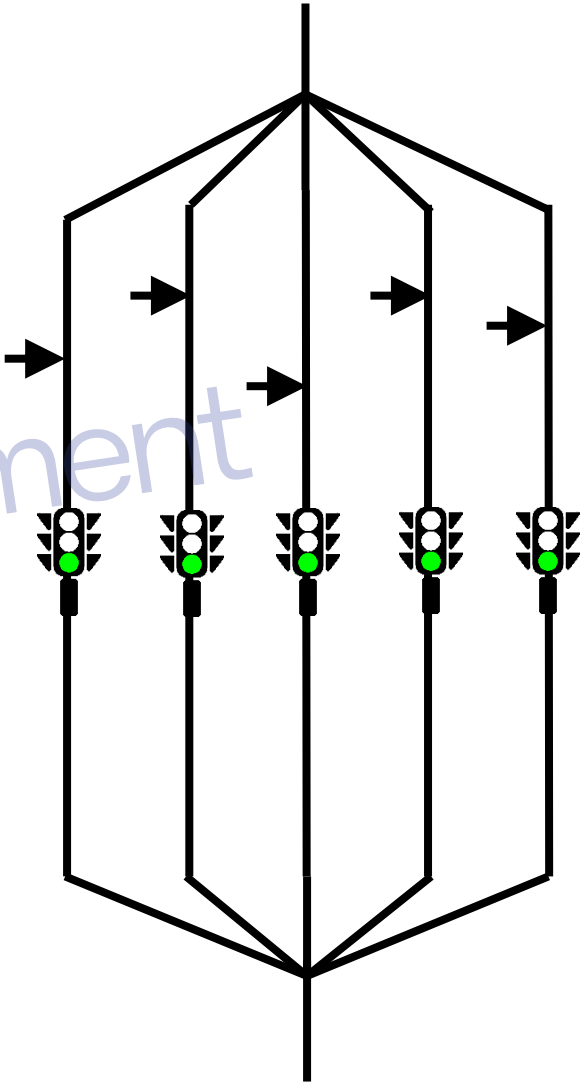
pdfelement

Remove Watermark Now

Semaphore

```
sem_wait(&semaphore);
```



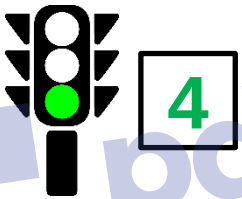


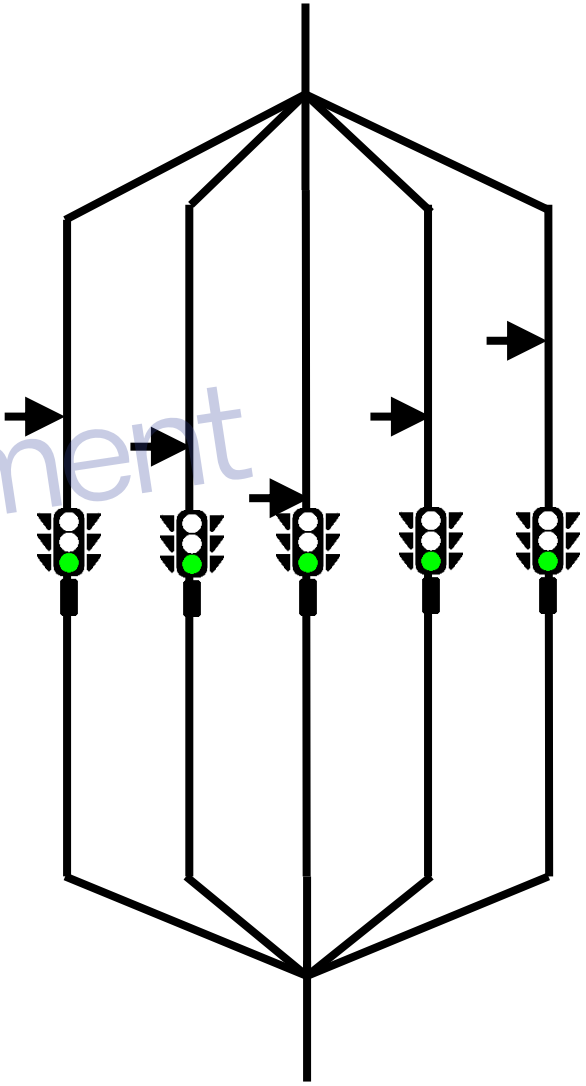
pdfelement

Remove Watermark Now

Semaphore

```
sem_wait(&semaphore);
```



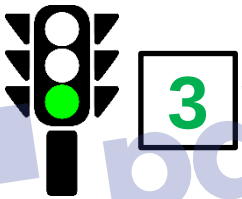


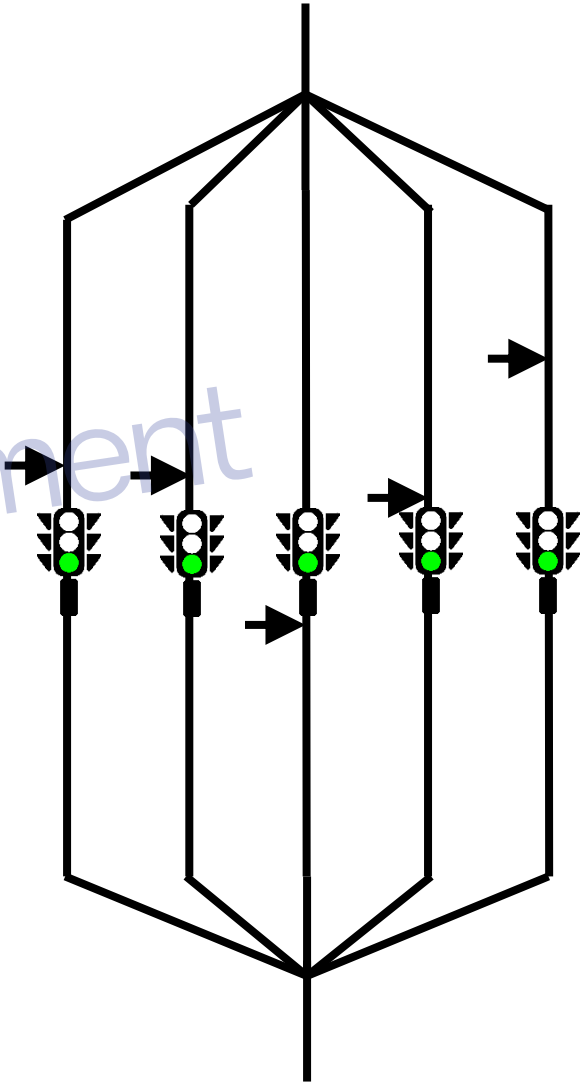
pdfelement

Remove Watermark Now

Semaphore

```
sem_wait(&semaphore);
```



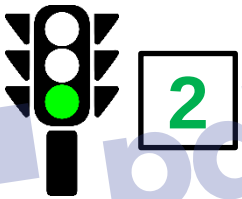


pdfelement

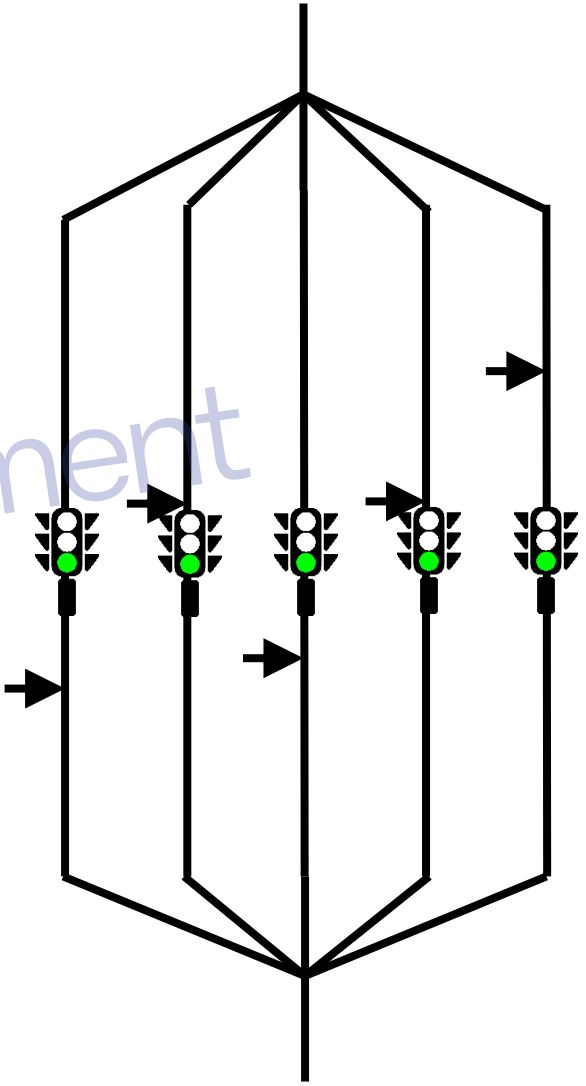
Remove Watermark Now

Semaphore

Nu contează că doua thread-uri au ajuns simultan la semafor, acesta este protejat, la fel ca un mutex.




```
sem_wait(&semaphore);
```

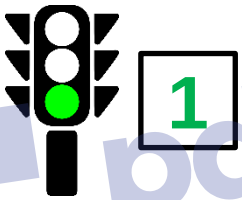


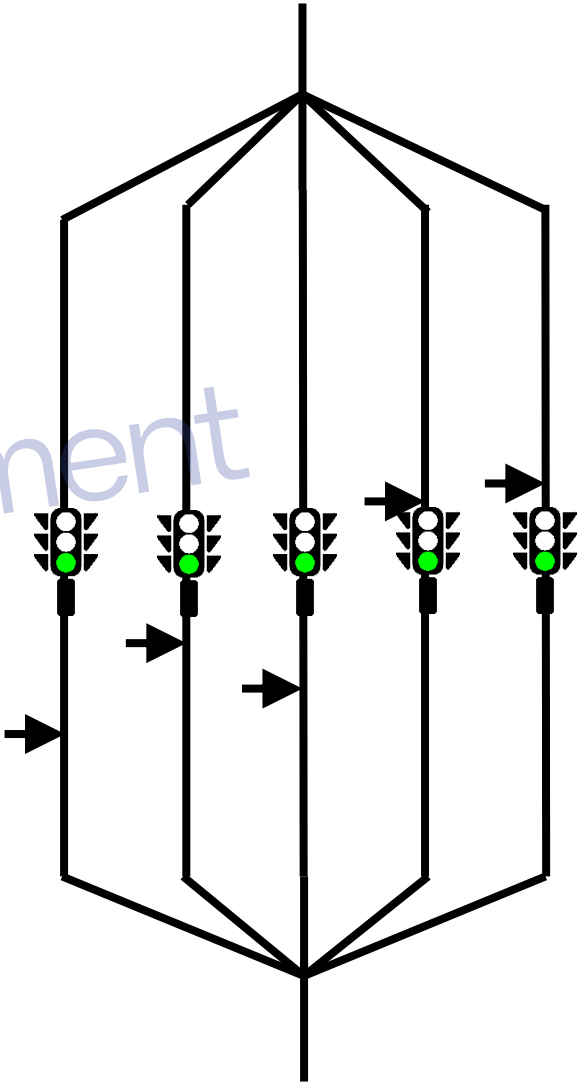
pdfelement

Remove Watermark Now

Semaphore

```
sem_wait(&semaphore);
```



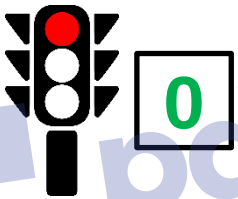


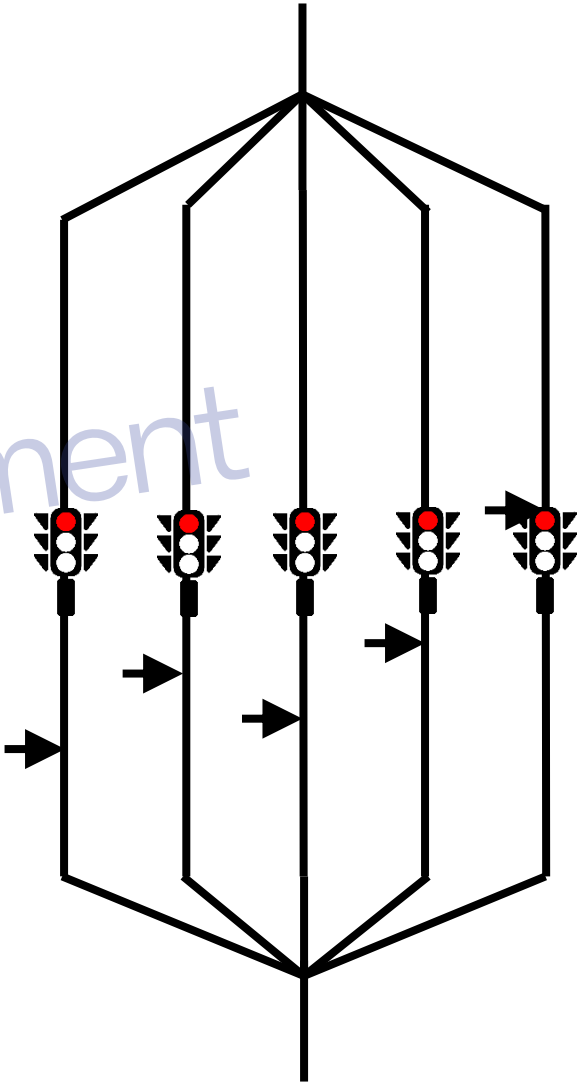
pdfelement

Remove Watermark Now

Semaphore

```
sem_wait(&semaphore);
```



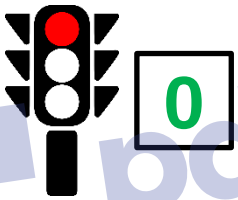


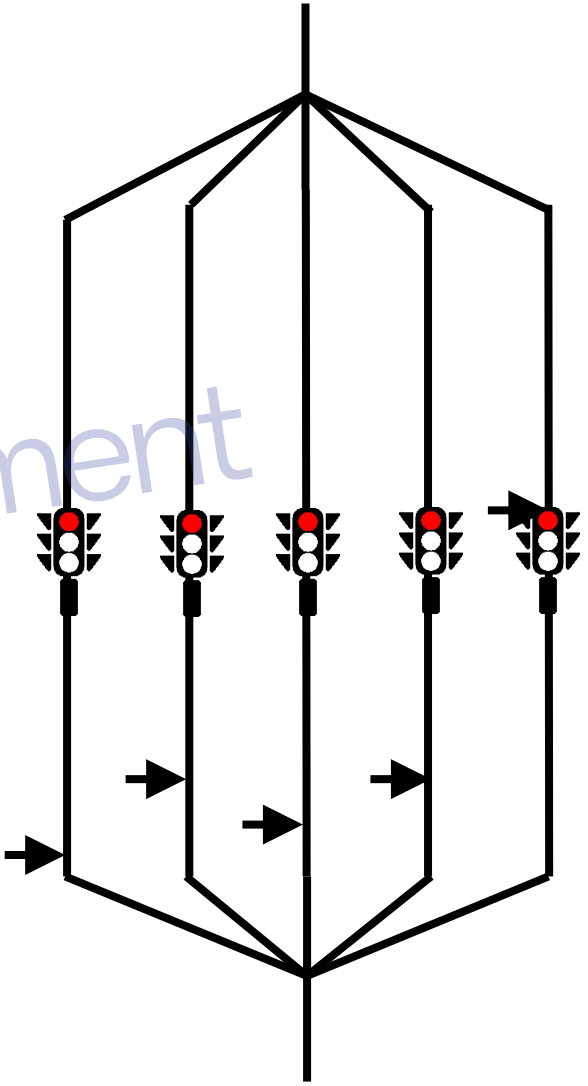
pdfelement

Remove Watermark Now

Semaphore

```
sem_wait(&semaphore);
```





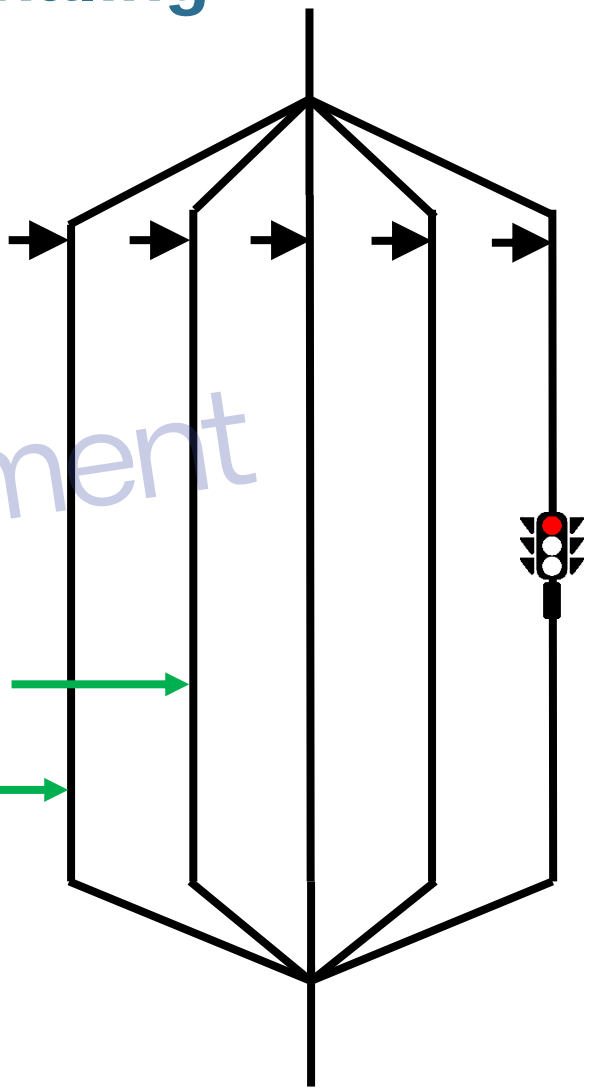
pdfelement

Remove Watermark Now

Semaphore – Signaling

```
sem_wait(&semaphore);
```

-1



```
sem_post(&semaphore);
```

```
sem_post(&semaphore);
```

V() sau Verogen
- Dijsktra

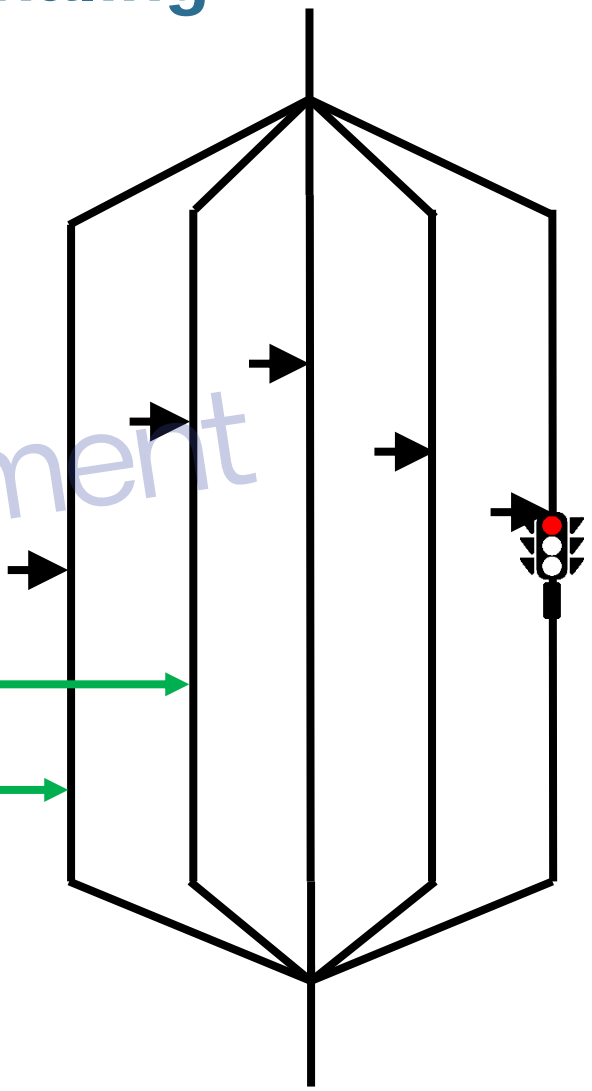
pdfelement

Remove Watermark Now

Semaphore – Signaling

```
sem_wait(&semaphore);
```

-1



```
sem_post(&semaphore);
```

```
sem_post(&semaphore);
```

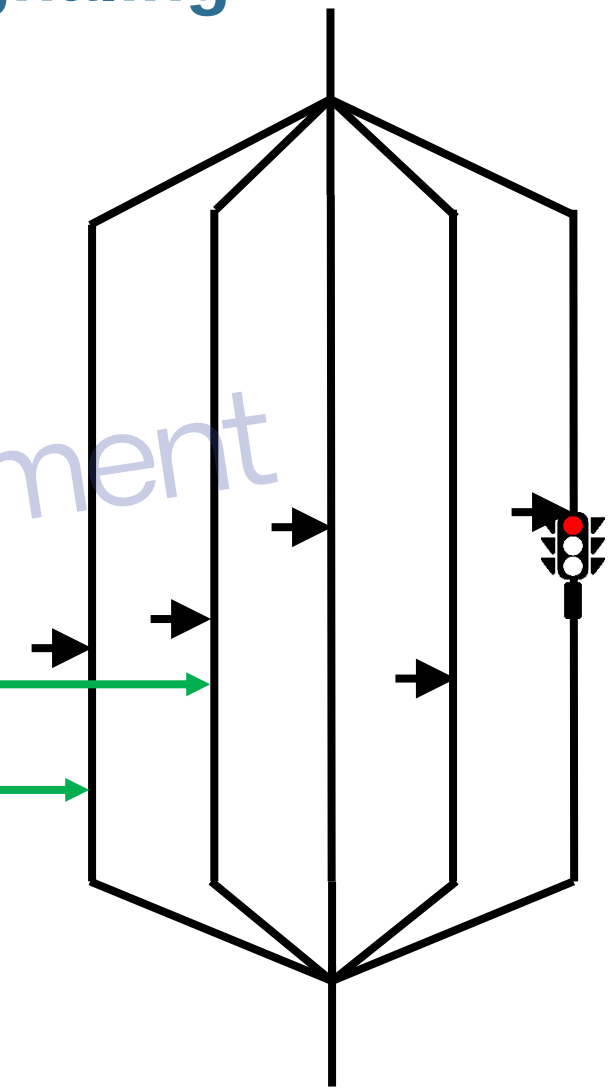
pdfelement

Remove Watermark Now

Semaphore – Signaling

```
sem_wait(&semaphore);
```

-1



```
sem_post(&semaphore);
```

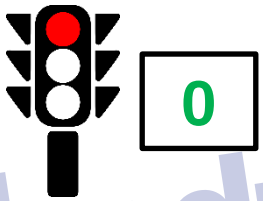
```
sem_post(&semaphore);
```

pdfelement

Remove Watermark Now

Semaphore – Signaling

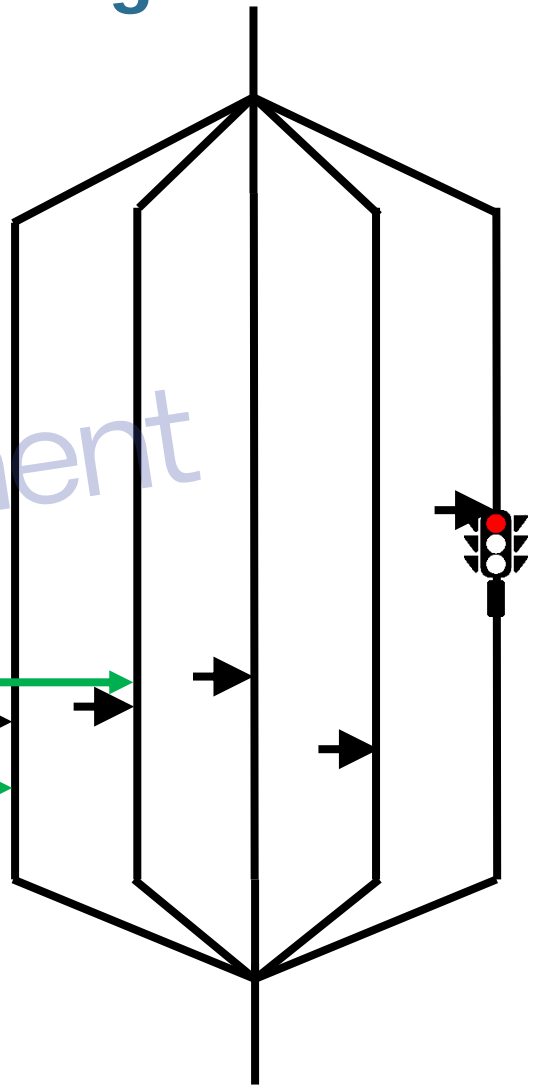
```
sem_wait(&semaphore);
```



pdfelement

```
sem_post(&semaphore);
```

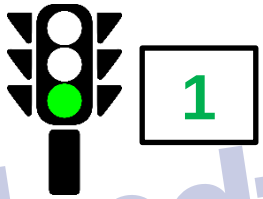
```
sem_post(&semaphore);
```

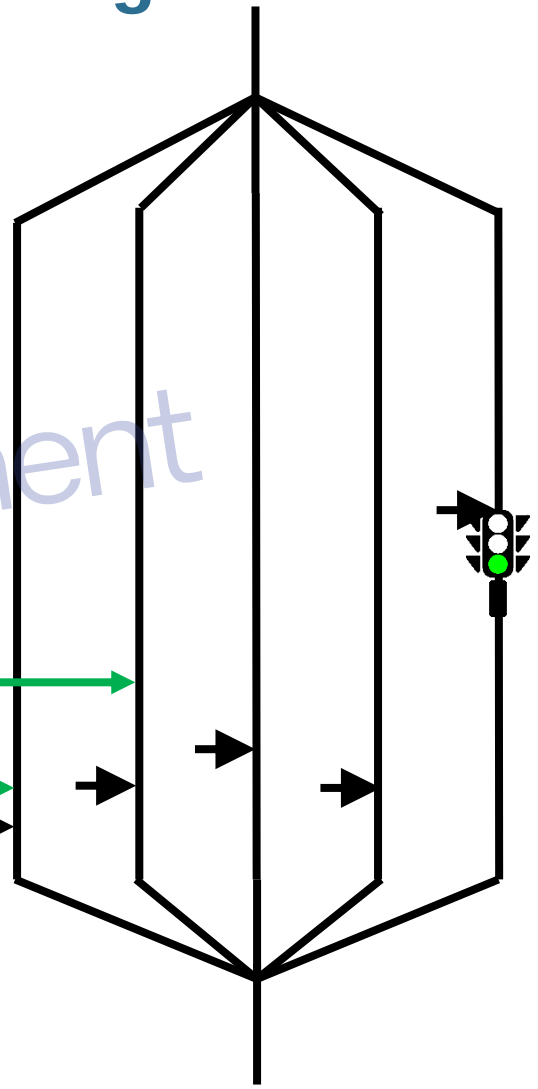


Remove Watermark Now

Semaphore – Signaling

```
sem_wait(&semaphore);
```





```
sem_post(&semaphore);
```

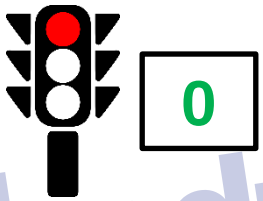
```
sem_post(&semaphore);
```

pdfelement

Remove Watermark Now

Semaphore – Signaling

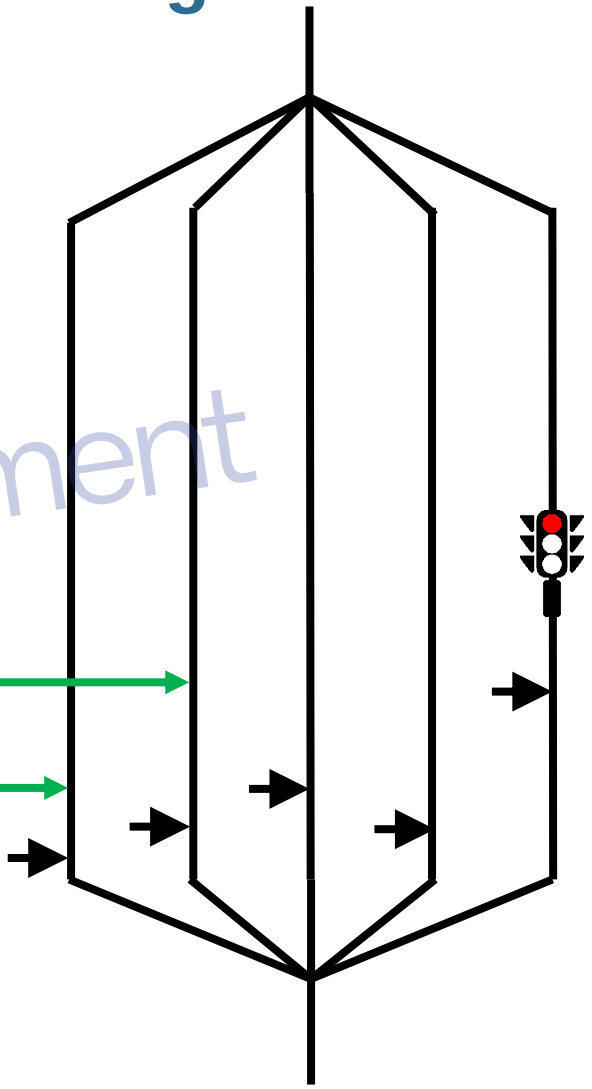
```
sem_wait(&semaphore);
```



pdfelement

```
sem_post(&semaphore);
```

```
sem_post(&semaphore);
```



Remove Watermark Now

Semaphore

ÎN MAIN

După ce au terminat thread-urile

pdfelement

sem_destroy(& **semaphore**);

Remove Watermark Now

Barrier

pdfelement

Remove Watermark Now

Barrier

ÎN MAIN

Înainte de a porni thread-urile

pdfelement

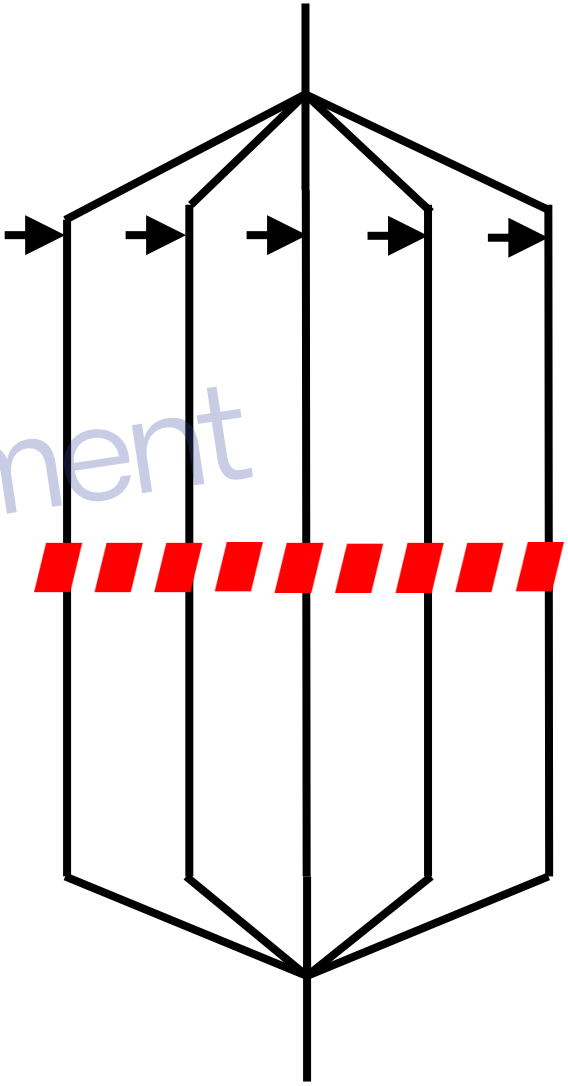
```
pthread_barrier_t barrier;
int num_threads = 5;
pthread_barrier_init(&barrier, NULL, num_threads);
```

Remove Watermark Now

Barrier

pthread_barrier_wait(&**barrier**);

5



pdfelement

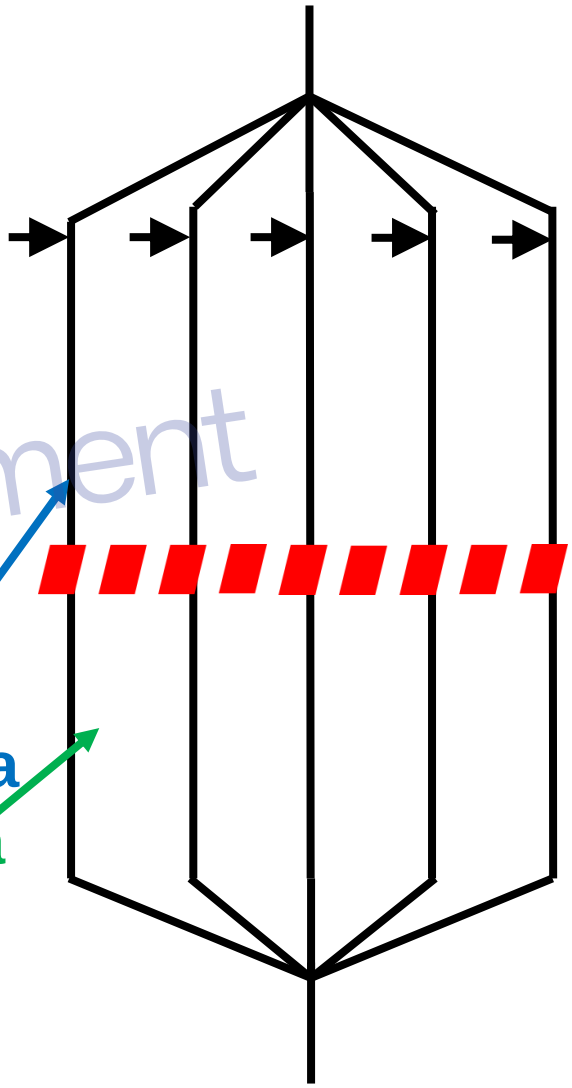
Remove Watermark Now

Barrier

pthread_barrier_wait(&**barrier**);

pdfelement

5



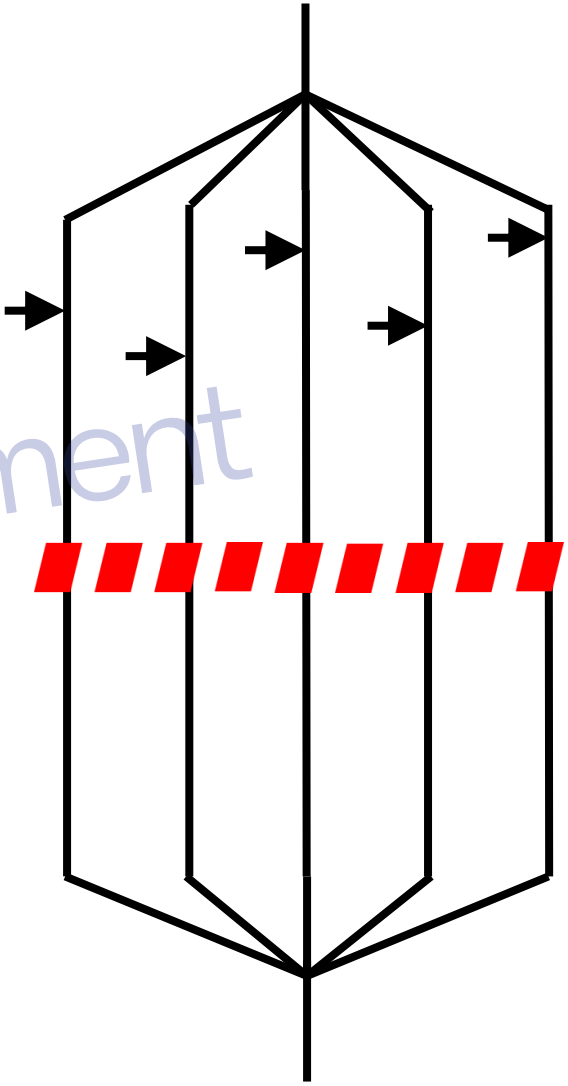
Pentru toate thread-urile codul acesta
Este executat înainte de codul acesta

Remove Watermark Now

Barrier

pthread_barrier_wait(&barrier);

5



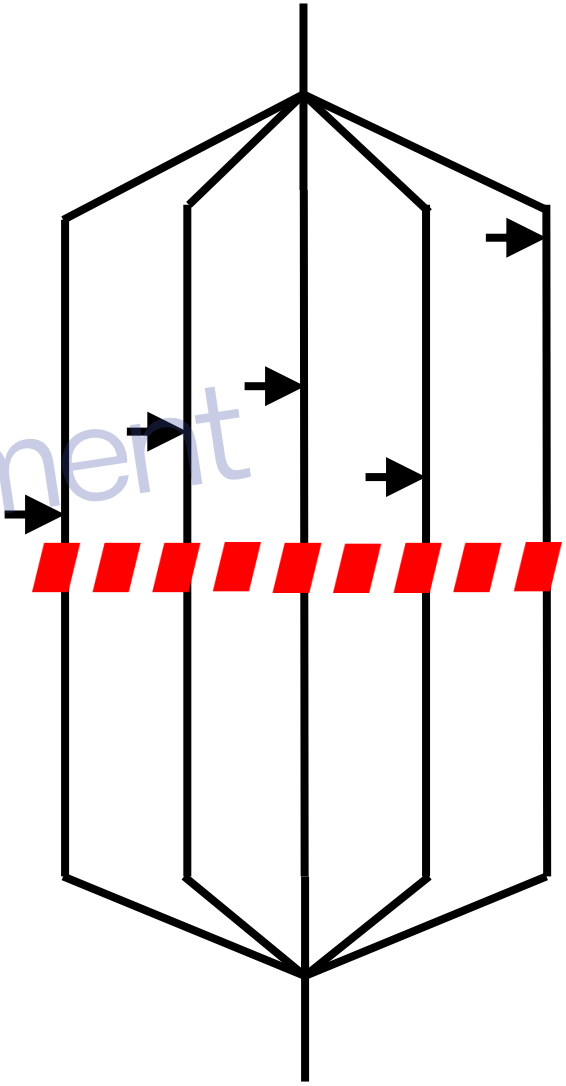
pdfelement

Remove Watermark Now

Barrier

pthread_barrier_wait(&**barrier**);

5



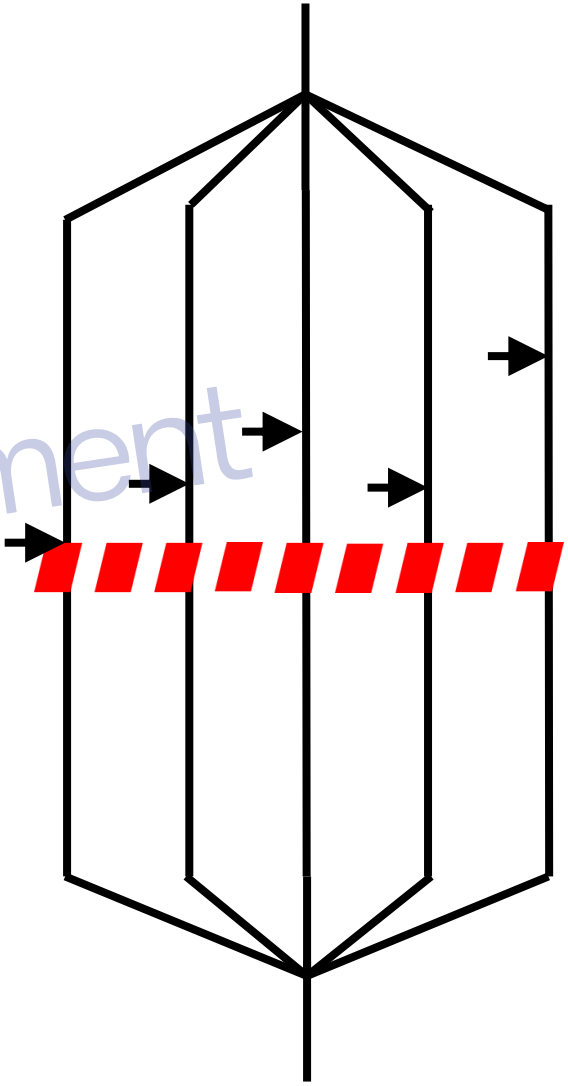
pdfelement

Remove Watermark Now

Barrier

pthread_barrier_wait(&**barrier**);

5



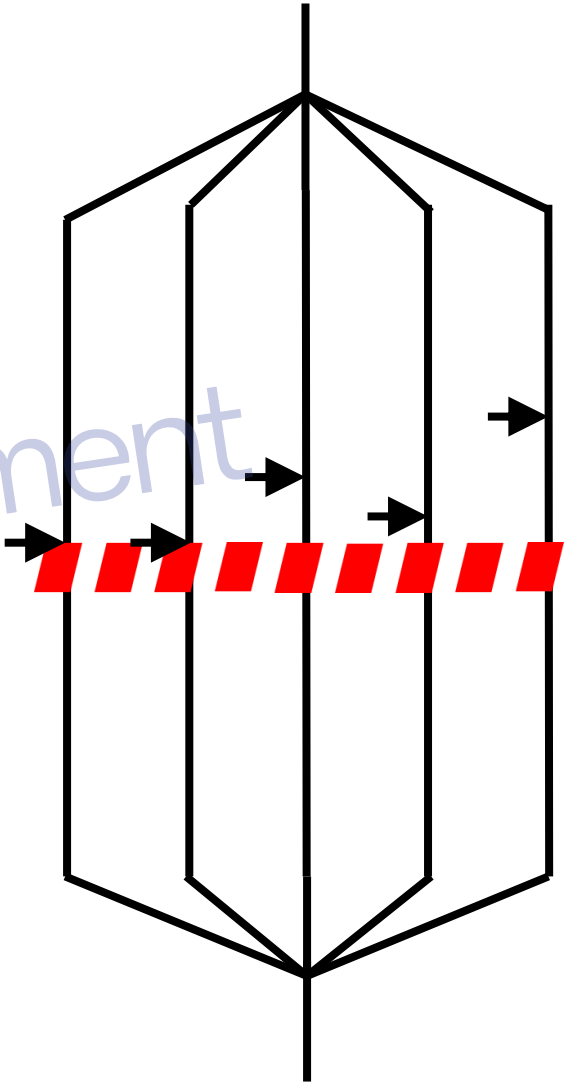
pdfelement

Remove Watermark Now

Barrier

pthread_barrier_wait(&**barrier**);

3



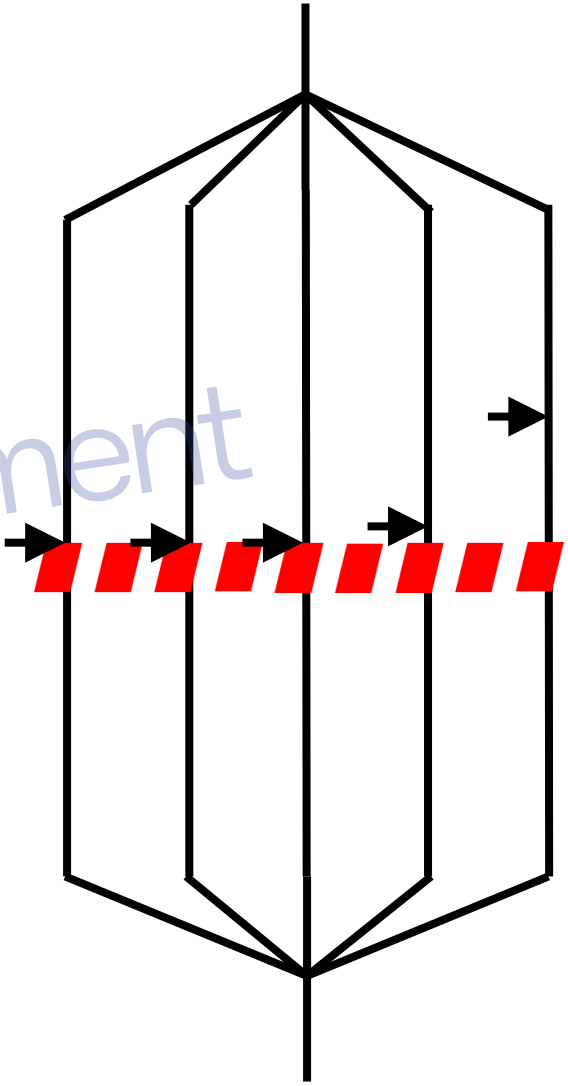
pdfelement

Remove Watermark Now

Barrier

pthread_barrier_wait(&barrier);

2



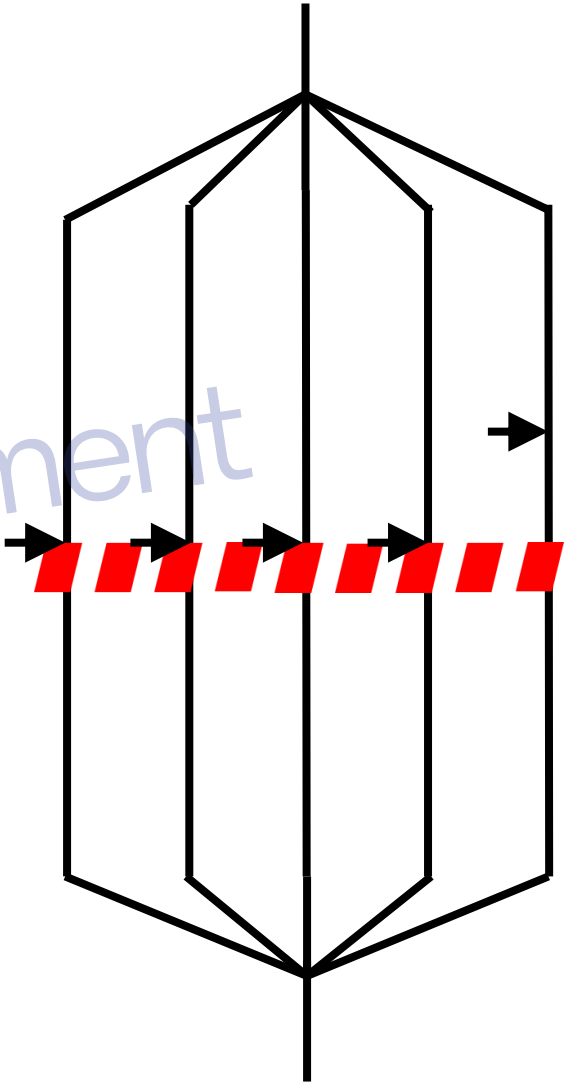
pdfelement

Remove Watermark Now

Barrier

pthread_barrier_wait(&barrier);

1



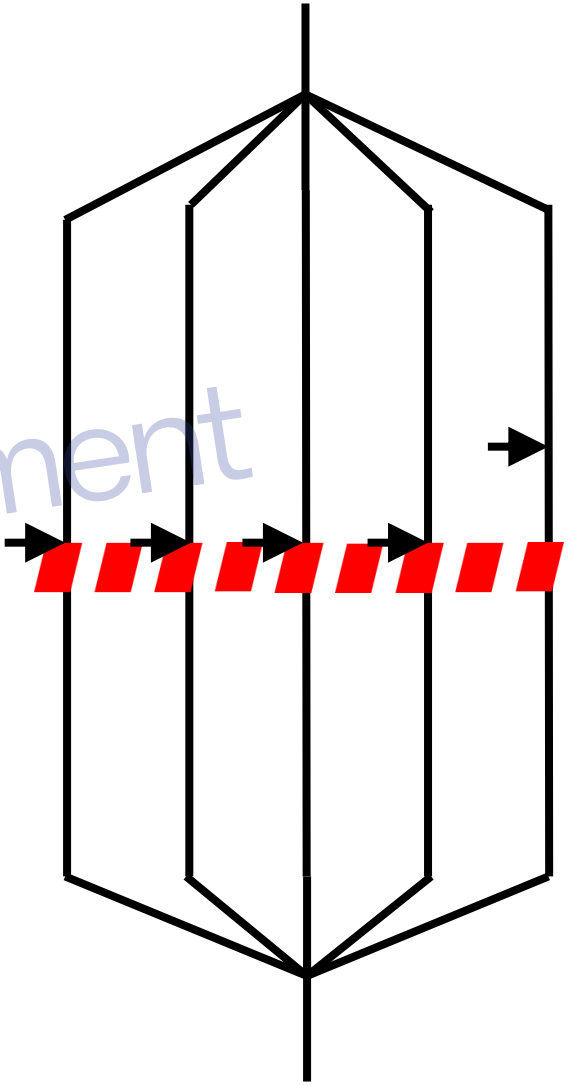
pdfelement

Remove Watermark Now

Barrier

pthread_barrier_wait(&**barrier**);

1



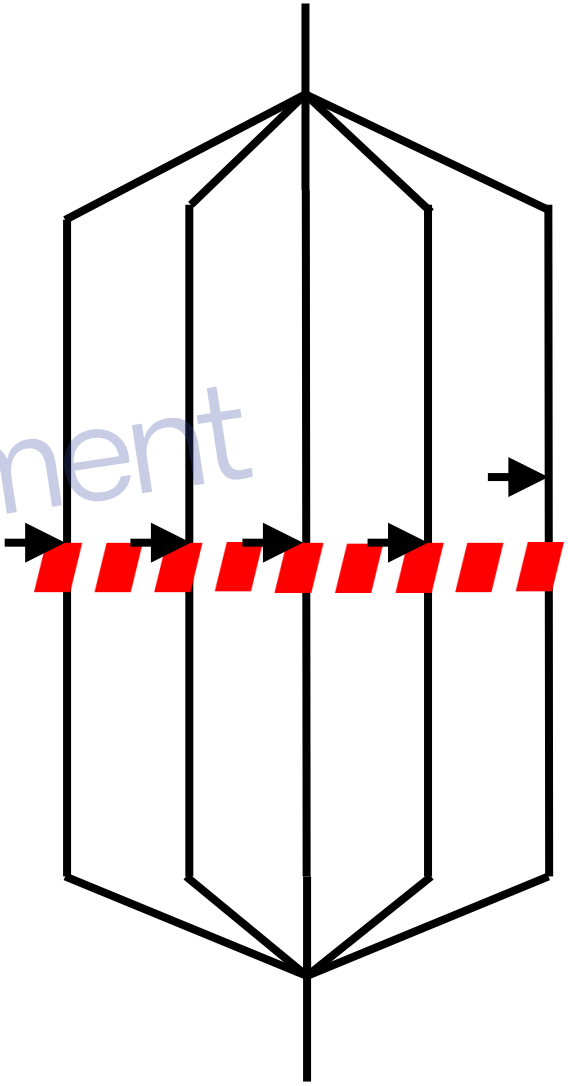
pdfelement

Remove Watermark Now

Barrier

pthread_barrier_wait(&**barrier**);

1



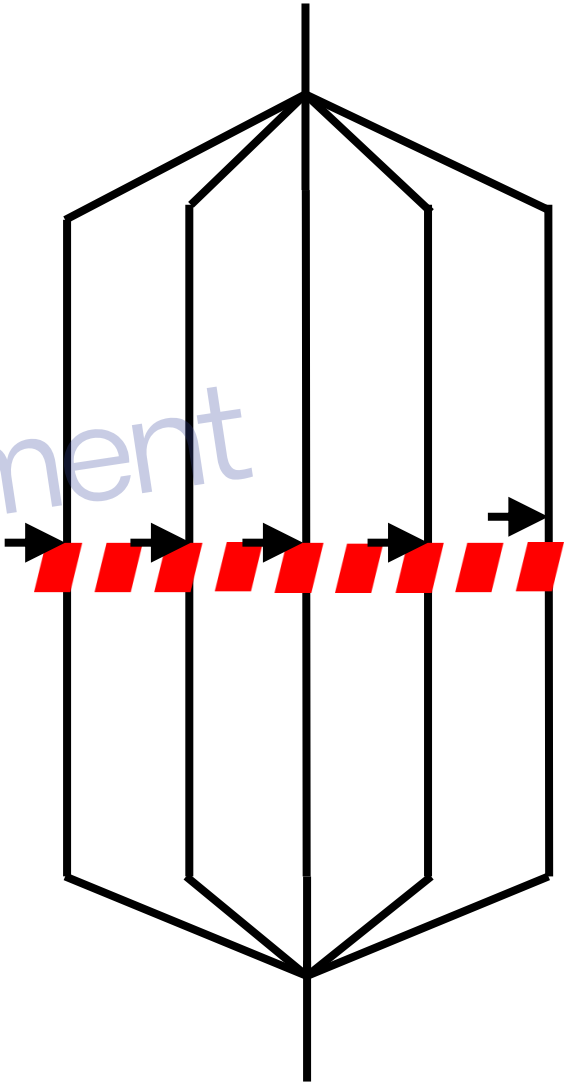
pdfelement

Remove Watermark Now

Barrier

pthread_barrier_wait(&**barrier**);

1



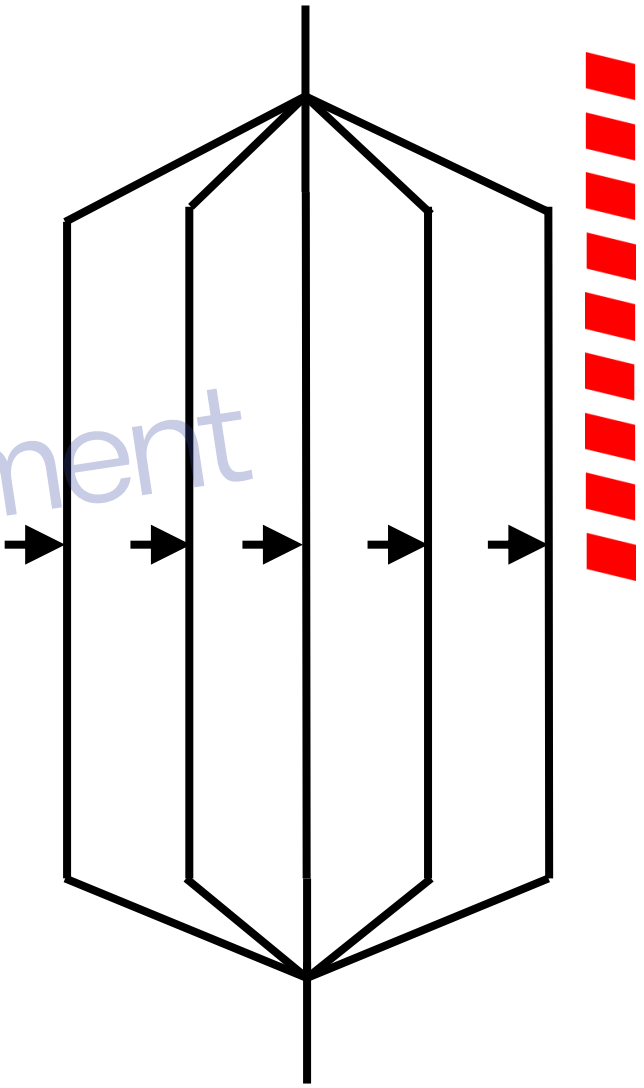
pdfelement

Remove Watermark Now

Barrier

pthread_barrier_wait(&**barrier**);

0



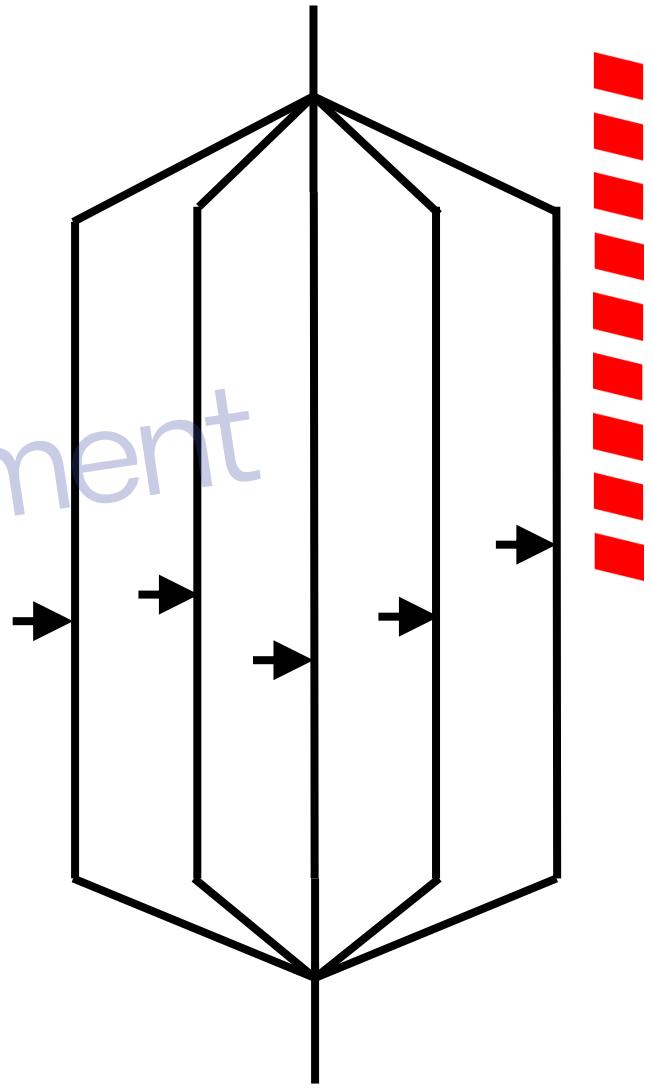
pdfelement

Remove Watermark Now

Barrier

pthread_barrier_wait(&**barrier**);

0



pdfelement

Remove Watermark Now

Barrier

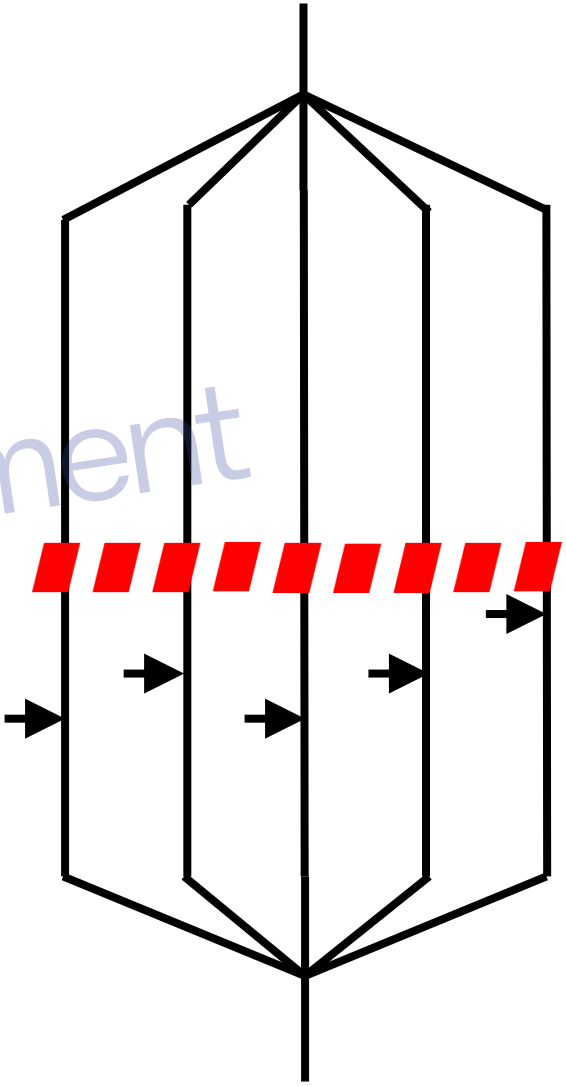
Cum știe o barieră când să se reseteze?

O soluție ar fi:
Reusable Barrier in
The Little Book of Semaphores
By Allen B. Downey

pdfelement

```
pthread_barrier_wait(&barrier);
```

5



Remove Watermark Now

Barrier

ÎN MAIN

După ce au terminat thread-urile

pdfelement

pthread_barrier_destroy(&**barrier**);

Remove Watermark Now

Remove Watermark Now

Unele probleme nu au soluție paralelă

Calculating the hash of a hash of a hash …

of a string.

Deep First Search

Huffman decoding

Outer loops of most simulations

P complete problems

pdfelement

Remove Watermark Now

Paralelizarea prin împărțirea muncii

Sunt o serie de probleme care sunt extrem de ușor paralelizabile.

Embarrassingly parallel

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Multiplicare unui vector cu un scalar

9	6	9	4	2	7	6	5	6	1

pdfelement

* 3

27	18	27	12	6	21	18	15	18	3

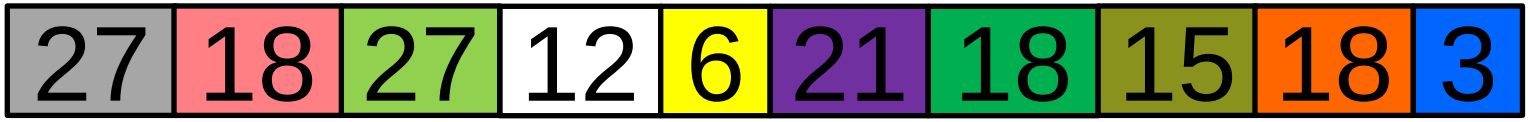
Remove Watermark Now

Probleme ”Embarrassingly parallel”

Toate calculele pot fi efectuate în același timp

pdfelement

* 3



Remove Watermark Now

Probleme ”Embarrassingly parallel”

Câte elemente sunt?

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Câte elemente sunt?

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Câte elemente sunt? **N**

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Dar câte elemente de procesare?

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Dar câte elemente de procesare? **P**

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Dar câte thread-uri?

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Dar câte thread-uri? **P**

pdfelement

În majoritatea cazurilor obținem performanță maximă când numărul de thread-uri este egal cu numărul de elemente de procesare, sau core-uri.

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Cum este P față de N?

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

$$P << N$$

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?

pdfelement

Thread 1

Thread 2

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?

pdfelement

Thread 1

Thread 2

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?

pdfelement

Thread 1

Thread 2

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim? Putem și random

pdfelement

Thread 1

Thread 2

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?

pdfelement

Thread 1

Thread 2

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?

Este utilă?

Thread 1

Thread 2

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?

Ce ne dorim?

Thread 1

Thread 2

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Caz concret: P = 2

Cum împărțim?

Ce ne dorim?

Thread 1

Thread 2

Aproximativ același număr elemente

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Aproximativ N/P elemente

pdfelement

Thread 1

Thread 2

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Aproximativ N/P elemente

pdfelement

Dacă N nu se divide perfect la P?

Thread 1

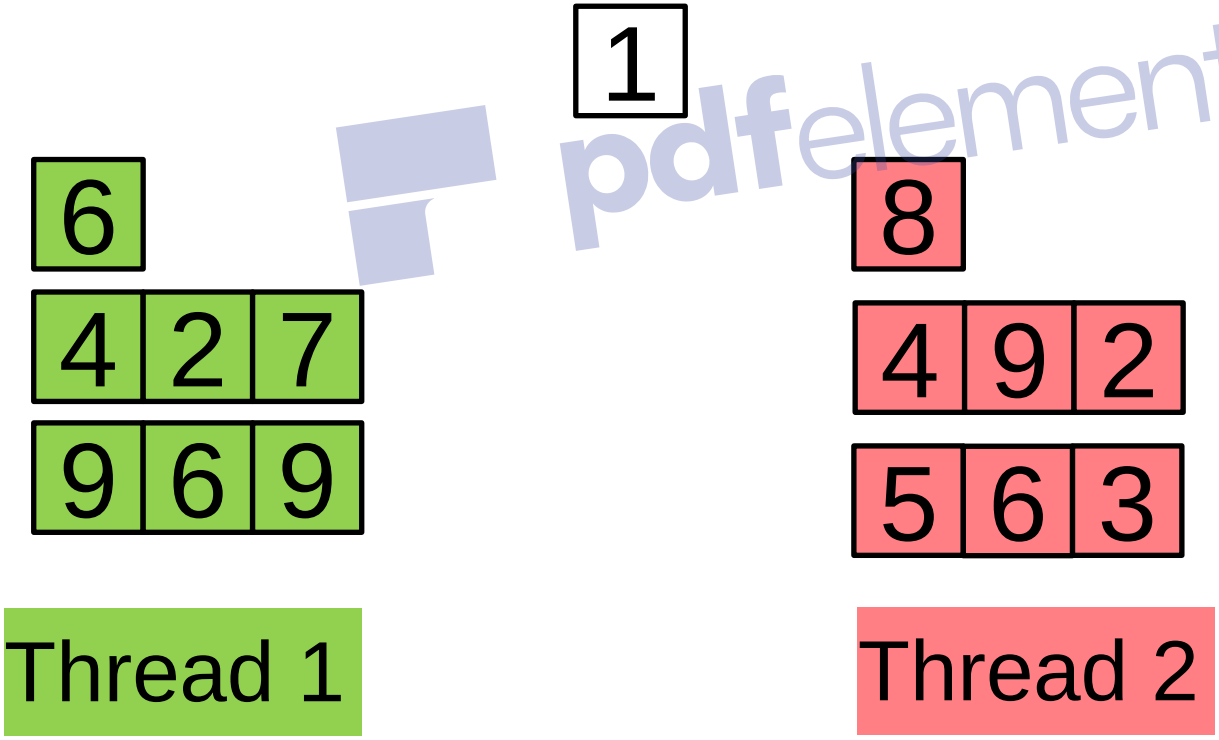
Thread 2

Remove Watermark Now

Probleme ”Embarrassingly parallel”

Aproximativ N/P elemente

Dacă N nu se divide perfect la P?

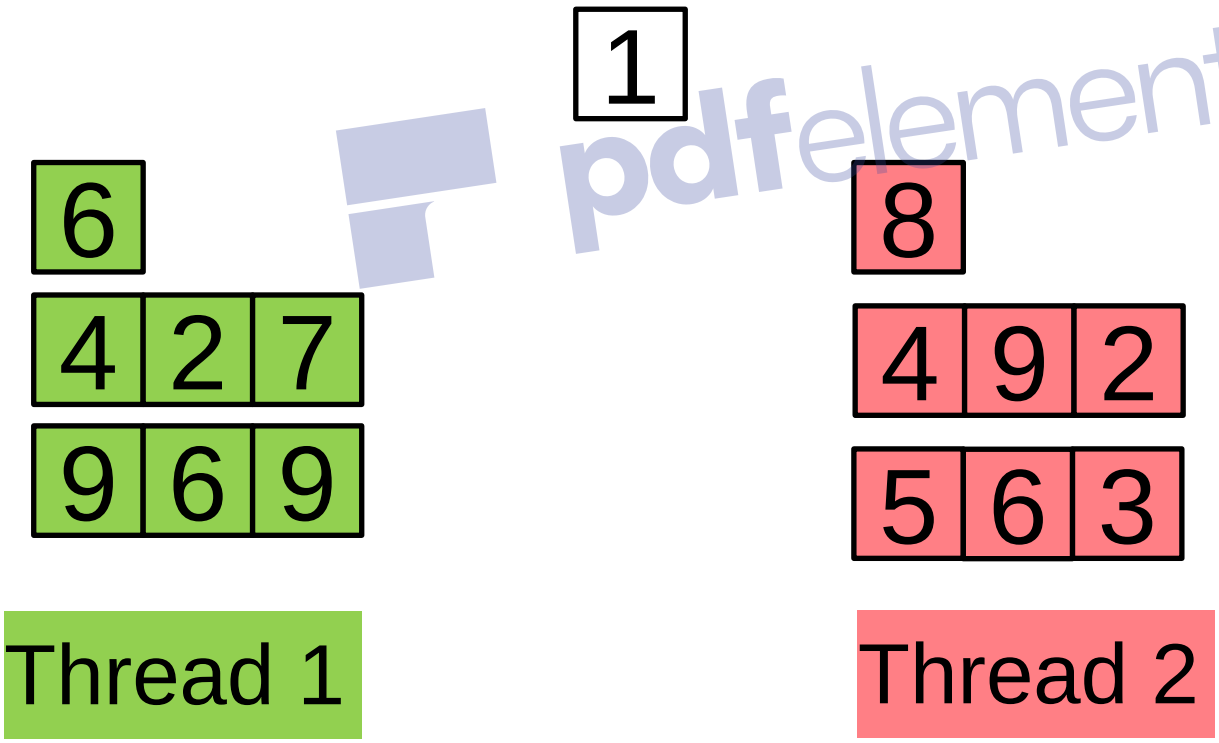


pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

floor(N/P) elemente **floor(15/2) = 7**



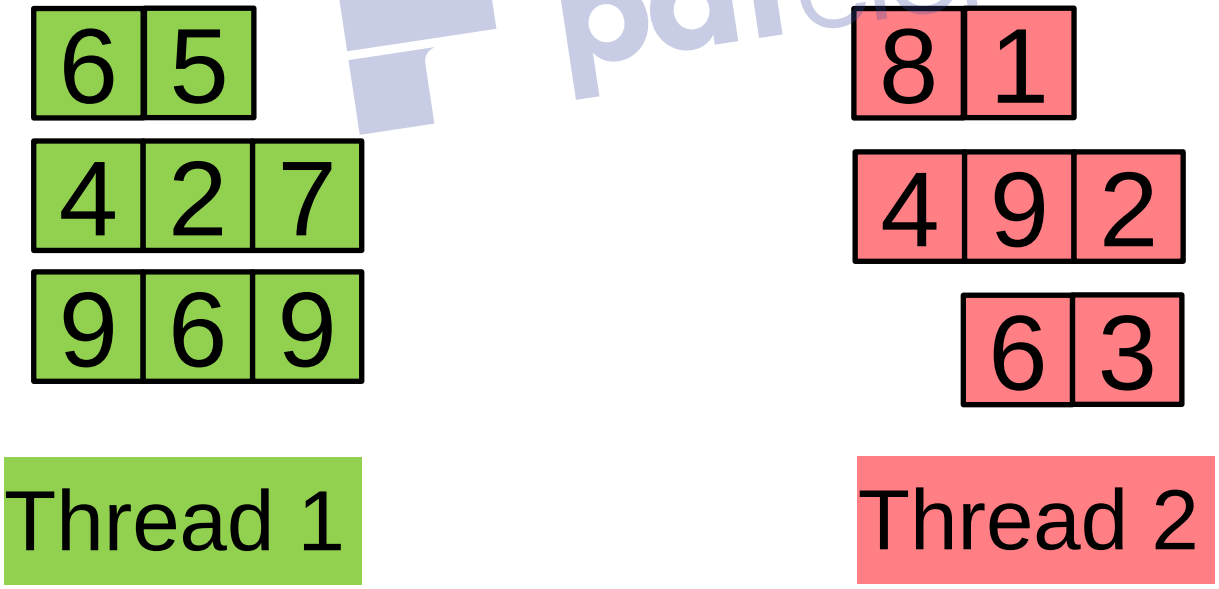
pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

ceil(N/P) elemente **ceil(15/2) = 8**

pdfelement



Remove Watermark Now

Probleme ”Embarrassingly parallel”

A = floor(N/P)

pdfelement

Remove Watermark Now

Probleme ”Embarrassingly parallel”

A = ceil(N/P)

pdfelement

Remove Watermark Now

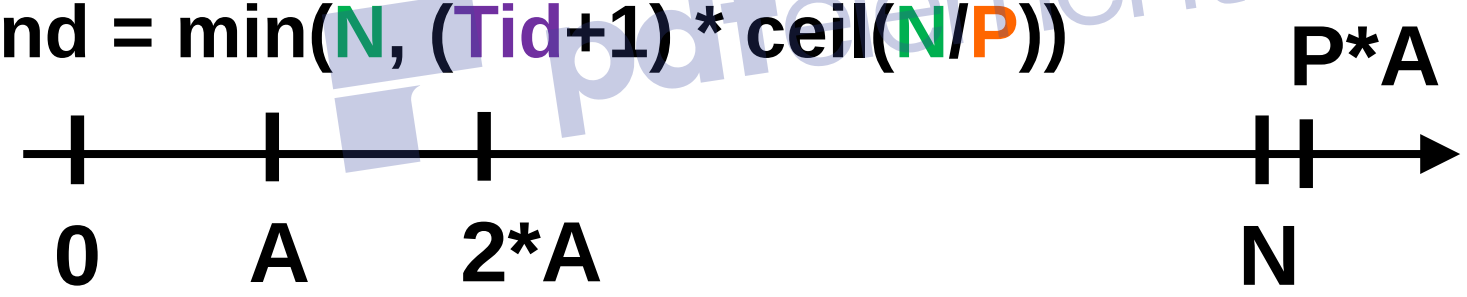
Probleme ”Embarrassingly parallel”

Formule elegante:

Tid este identificator de thread, are valori de la 0 la P

start = Tid * ceil(N/P)

end = min(N, (Tid+1) * ceil(N/P))



pdfelement

Remove Watermark Now

Remove Watermark Now

Măsurarea timpului de execuție

```
time ./executabil p a r a m e t r i
```

pdfelement

Remove Watermark Now

Măsurare timp – Linia de comandă

***time** sleep 5*

real 0m5.001s
user 0m0.000s
sys 0m0.001s

time sleep 5

sleep 5 0.00s user 0.00s system 0% cpu 5.002 total

/usr/bin/time sleep 5

0.00user 0.00system 0:05.00elapsed 0%CPU (0avgtext+0avgdata 2076maxresident)k
0inputs+0outputs (0major+73minor)pagefaults 0swaps

pdfelement

Remove Watermark Now

Măsurare timp – Linia de comandă

***time** sleep 5*

real 0m5.001s
user 0m0.000s
sys 0m0.001s

Wall clock time – Timpul trecut de la pornirea programului – Pe acesta îl folosim

time sleep 5

sleep 5 0.00s user 0.00s system 0% cpu 5.002 total

/usr/bin/time sleep 5

0.00user 0.00system 0:05.00elapsed 0%CPU (0avgtext+0avgdata 2076maxresident)k
0inputs+0outputs (0major+73minor)pagefaults 0swaps

pdfelement

Remove Watermark Now

Măsurare timp – Linia de comandă

time sleep 2

real 0m2.0**21**s
user 0m0.000s
sys 0m0.000s

time sleep 2

real 0m2.0**18**s
user 0m0.000s
sys 0m0.016s

time sleep 2

real 0m2.0**16**s
user 0m0.000s
sys 0m0.000s

time sleep 2

real 0m2.0**15**s
user 0m0.000s
sys 0m0.000s

Timpii măsurați nu sunt exacți.

- Dacă avem timpi foarte mici nu sunt relevanți. De preferat timpi peste 1 secundă.
- Pentru a măsura corect trebuie să facem medie a timpilor după mai multe rulări.

pdfelement

Remove Watermark Now

Măsurare timp – Linia de comandă

***time** sleep 5*

real 0m5.001s
user 0m0.000s
sys 0m0.001s

Suma timpului petrecut în user space pe fiecare core.

pdfelement

time sleep 5

sleep 5 0.00s user 0.00s system 0% cpu 5.002 total

/usr/bin/time sleep 5

0.00user 0.00system 0:05.00elapsed 0%CPU (0avgtext+0avgdata 2076maxresident)k
0inputs+0outputs (0major+73minor)pagefaults 0swaps

Remove Watermark Now

Măsurare timp – Linia de comandă

Suma timpului petrecut în user space pe fiecare core.

pdfelement

```
time timeout 5 ./useAllCPU 12

real    0m4.075s
user    0m47.797s
sys     0m0.031s
```

Remove Watermark Now

Măsurare timp – Linia de comandă

***time** sleep 5*

real 0m5.001s
user 0m0.000s
sys 0m0.001s

Suma timpului petrecut în kernel pe fiecare core.

pdfelement

time sleep 5

sleep 5 0.00s user 0.00s system 0% cpu 5.002 total

/usr/bin/time sleep 5

0.00user 0.00system 0:05.00elapsed 0%CPU (0avgtext+0avgdata 2076maxresident)k
0inputs+0outputs (0major+73minor)pagefaults 0swaps

Remove Watermark Now

Măsurare timp – Linia de comandă

Orice I/O este făcut de Kernel

```
time dd if=/dev/zero of=file.txt count=1024 bs=1048576
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB) copied, 9.4847 s, 113 MB/s

real    0m9.490s
user    0m0.000s
sys     0m0.992s
```

pdfelement

Remove Watermark Now

Măsurare timp – Din program

```
clock_t t;
t = clock();
WORK();
t = clock() - t;
double time_taken = ((double)t)/CLOCKS_PER_SEC; // in seconds
```

pdfelement

Remove Watermark Now

Măsurare timp – Din program

DO NOT USE
pdfelement

```
clock_t t;
t = clock();
WORK();
t = clock() - t;
double time_taken = ((double)t)/CLOCKS_PER_SEC; // in seconds
```

Remove Watermark Now

Măsurare timp – Din program

DO NOT USE
pdfelement

```
clock_t t;
t = clock();
WORK();
t = clock() - t;
double time_taken = ((double)t)/CLOCKS_PER_SEC; // in seconds
```

Din **man**: the value returned by **clock**() also includes the times of any children.

Remove Watermark Now

Măsurare timp – Din program

```
struct timespec start, finish;
double elapsed;
clock_gettime(CLOCK_MONOTONIC, &start);
WORK();
clock_gettime(CLOCK_MONOTONIC, &finish);
elapsed = (finish.tv_sec - start.tv_sec);
elapsed += (finish.tv_nsec - start.tv_nsec) / 1000000000.0;
```

pdfelement

Remove Watermark Now

Măsurare timp cu sau fără I/O?

pdfelement

Remove Watermark Now

Măsuri

- T - Timpul total necesar execuției programului paralel
- P - Numărul de procesoare utilizate
- G – Timp execuție cel mai rapid algoritm secvențial
- S – Speedup
  - $S = \frac{G}{T}$

pdfelement

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s

pdfelement

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1			

$$S = \frac{6.14}{6.14}$$

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8		

$$S = \frac{6.14}{7.76}$$

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 0.78	

$$S = \frac{6.14}{7.82}$$

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 0.78	S = 0.78

$$S = \frac{6.14}{7.87}$$

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 0.78	S = 0.78

WRONG

$$S = \frac{6.14}{7.87}$$

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1			

$$S = \frac{6.15}{6.15}$$

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8		

$$S = \frac{6.15}{7.77}$$

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 1.55	

$$S = \frac{6.15}{3.95}$$

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 1.55	S = 3.05

$$S = \frac{6.15}{2.01}$$

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 1.55	S = 3.05

De ce nu **S = P** ?

Remove Watermark Now

Timpi adunare a doi vectori C = A + B

Nu se ține cont de timpul de citire/scriere RAM.

Sequential	Pthread (1 Thread)	Pthread(2 Thread)	Pthread(4 Thread)
real 0m6.151s user 0m6.141s sys 0m0.000s	real 0m7.777s user 0m7.766s sys 0m0.000s	real 0m3.954s user 0m7.828s sys 0m0.000s	real 0m2.011s user 0m7.875s sys 0m0.031s
S = 1	S = 0.8	S = 1.55	S = 3.05

De ce nu **S = P** ? S = O(P) este **ideal**.

Remove Watermark Now

Remove Watermark Now

Consistența?

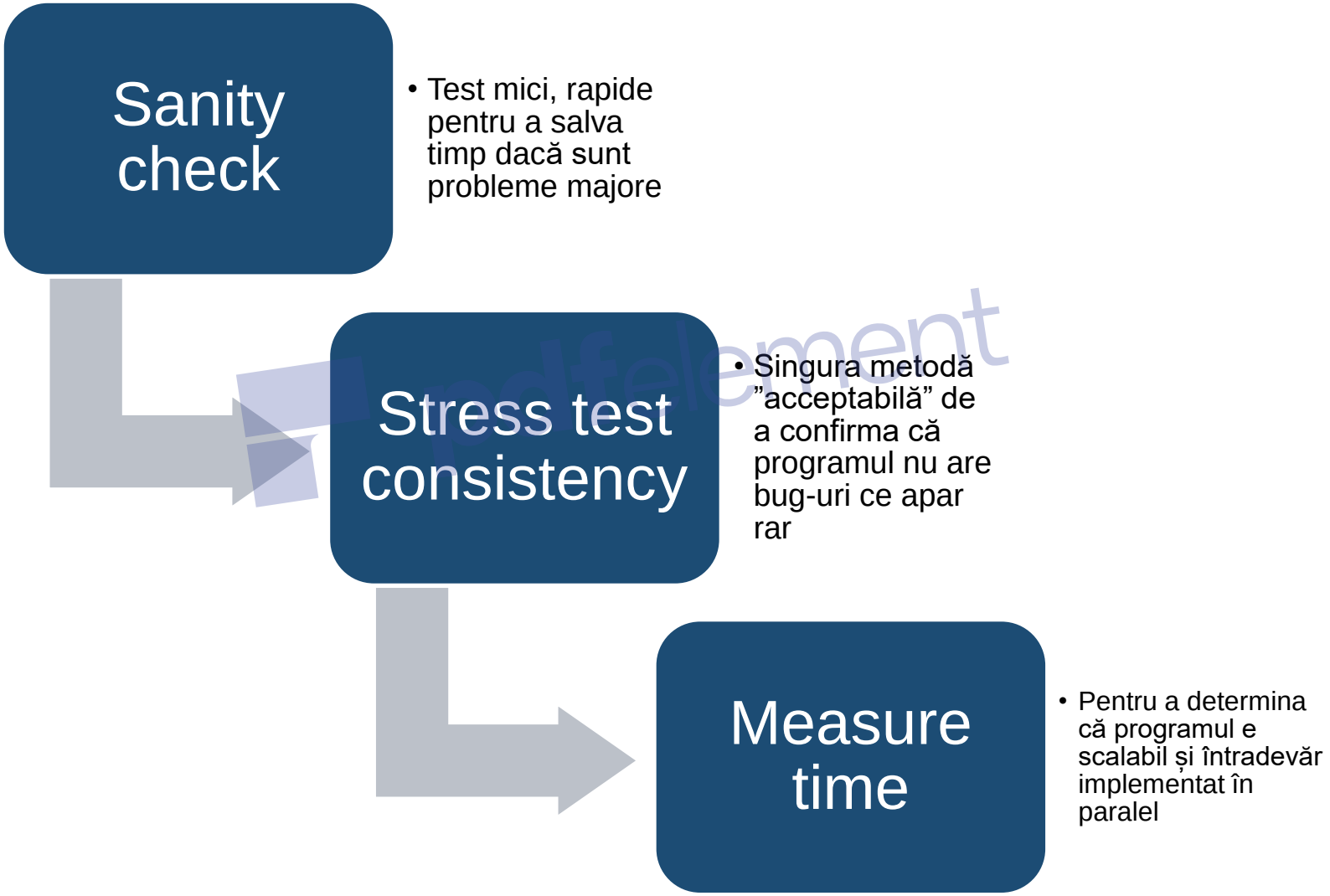
- Demonstrație formală

pdfelement

- Stress test
  - mereu comparați cu rezultatul algoritmului secvențial

Remove Watermark Now

Workflow - Testarea programelor



Remove Watermark Now

Algoritmi Paraleli și Distribuiți Java

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

APD

Remove Watermark Now

Programarea concurentă în Java

pdfelement

Remove Watermark Now

Java multithreading

pdfelement

Remove Watermark Now

Stările posibile ale unui thread

- Creat: obiectul a fost creat cu operația `new()`; se poate apela metoda `start()`
- Gata de execuție: a fost apelată metoda `start()`, firul poate fi executat
- Suspendat: a fost apelat `sleep()` sau `wait()`
- Terminat: metoda `run()` a fost terminată

pdfelement

Remove Watermark Now

Java multithreading

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }

            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

pdfelement

Remove Watermark Now

start method

Thread-urile sunt pornite prin apelul metodei **start**() dintr-un obiect Thread

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }

            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

pdfelement

Remove Watermark Now

start method

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

```
void start() {
      Crează stack-ul pentru thread
      Informează JVM și OS
      Thread-ul execută metoda run();
}
```

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }

            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

pdfelement

Remove Watermark Now

join method

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }

            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

Thread-ul main așteaptă terminarea celorlate thread-uri prin apeluri **join**().

Trebuie să fie într-un bloc try/catch în caz că thread-ul se termină din cauza unei întreruperi.

pdfelement

Remove Watermark Now

join method

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }

            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

```
void join() {
      așteaptă ca un thread să
termine execuția
}
```

pdfelement

Remove Watermark Now

thread execution

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }
            // Main thread continues execution
            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

pdfelement

Remove Watermark Now

implements instead of extends

```
public class MyThread extends Thread {
      public void run() {
            System.out.println("Hello world!");
      }
}
```

Putem să folosim și implements pentru a crea obiecte ce au punctul de start al unui thread.

De ce? Pentru că în Java un obiect poate moșteni un singur obiect și există cazuri când este necesară extinderea unui alt obiect.

```
public class Main {
      public static void main(String[] args) {
            int N = 4;
            MyThread threads[] = new MyThread[N];

            for (int i = 0; i < N; i++) {
                  threads[i] = new MyThread();
            }
            for (int i = 0; i < N; i++) {
                  threads[i].start();
            }

            System.out.println("Hello from main thread");

            for (int i = 0; i < N; i++) {
                  try {
                        threads[i].join();
                  } catch (InterruptedException e) {
                        e.printStackTrace();
                  }
            }
      }
}
```

pdfelement

Remove Watermark Now

implements instead of extends

```
public class MyRunnable implements Runnable {
	public void run() {
		System.out.println("Hello world!");
	}
}
```

```
public class Main {
	public static void main(String[] args) {
		int N = 4;
		Thread threads[] = new Thread[N];

		for (int i = 0; i < N; i++) {
			Runnable aux = new MyRunnable();
			threads[i] = new Thread(aux);
		}
		for (int i = 0; i < N; i++) {
			threads[i].start();
		}

		System.out.println("Hello from main thread");

		for (int i = 0; i < N; i++) {
			try {
				threads[i].join();
			} catch (InterruptedException e) {
				e.printStackTrace();
			}
		}
	}
}
```

pdfelement

Remove Watermark Now

Java Concurrency

pdfelement

Remove Watermark Now

Sincronizarea firelor de execuție

- **Două situații:**
  - Concurență
  - Cooperare
- **Sincronizarea**
  - asigură ***excluderea mutuală*** – un singur fir poate executa la un moment dat o metodă (secvență de cod) sincronizată: **secțiune critică**
  - Folosește mecanismul de **zăvor** :
    - Fiecare obiect are asociat câte un **zăvor**
    - **synchronized(o)** asigură intrarea în secțiunea critică
    - se poate asocia unei **metode** sau unei **secvențe** de cod
    - pe parcursul execuției secvențelor de cod sincronizate, zăvorul este „închis”

pdfelement

Remove Watermark Now

Concurrency problems

```
i = 0;
```

thread 1

```
i = i + 2;
```

thread 2

```
i = i + 2;
```

pdfelement

```
i is 2 or 4
```

Remove Watermark Now

Concurrency problems

```
i = 0;
```

thread 1

```
mov eax i;
add eax 2;
mov i eax;
```

thread 2

```
mov eax i;
add eax 2;
mov i eax;
```

pdfelement

```
i is 2 or 4
```

Remove Watermark Now

Concurrency problems

```
i = 0;
```

thread 1

```
mov eax i;

add eax 2;

mov i eax;
```

thread 2

```
mov eax i;

add eax 2;
mov i eax;
```

pdfelement

i is **2** or 4

Remove Watermark Now

Concurrency problems

```
i = 0;
```

thread 1

```
mov eax i;
add eax 2;
mov i eax;
```

thread 2

```
mov eax i;
add eax 2;
mov i eax;
```

pdfelement

i is 2 or **4**

Remove Watermark Now

Solution: synchronized

```
i = 0;
```

thread 1

```
synchronized (object) {
        i = i + 2;
}
```

thread 2

```
synchronized (object) {
        i = i + 2;
}
```

pdfelement

```
i is 4
```

Remove Watermark Now

synchronized with object

```
synchronized (object) {
    i = i + 2;
}
```

Echivalent cu:

```
lock (object)
    i = i + 2;
unlock (object)
```

pdfelement

object can be **any** object
object can be **this**
object can be **.class**

```
public static ThreadSafeSingleton getInstanceUsingDoubleLocking(){
    if(instance == null){
        synchronized (ThreadSafeSingleton.class) {
            if(instance == null){
                instance = new ThreadSafeSingleton();
            }
        }
    }
    return instance;
}
```

Remove Watermark Now

synchronized method

Secțiune critică între toate thread-urile care folosesc același **obiect**

```
increment() {
    synchronized (this) {
        i = i + 2;
    }
}
```

```
synchronized increment() {
        i = i + 2;
}
```

pdfelement

Secțiune critică între toate thread-urile care folosesc **clasa *Name***

```
increment() {
    synchronized (Name.class) {
        i = i + 2;
    }
}
```

```
static synchronized increment() {
        i = i + 2;
}
```

```java
class Table {

  void printTable(int n) {// method not synchronized
    for (int i = 1; i <= 5; i++) {
      System.out.println(n * i);
      try {
        Thread.sleep(400);
      } catch (Exception e) {
        System.out.println(e);
      }
    }
  }
}
```

src > MyThread2.java > MyThread2

```java
class MyThread2 extends Thread {
  Table t;

  MyThread2(Table t) {
    this.t = t;
  }

  public void run() {
    t.printTable(100);
  }
}
```

src > MyThread1.java > MyThread1

```java
class MyThread1 extends Thread {
  Table t;

  MyThread1(Table t) {
    this.t = t;
  }

  public void run() {
    t.printTable(5);
  }

}
```

Remove Watermark Now

src > TestSynchronization1.java > TestSynchronization1

```java
class TestSynchronization1 {

  Run | Debug
  public static void main(String args[]) {
    Table obj = new Table();// only one object
    MyThread1 t1 = new MyThread1(obj);
    MyThread2 t2 = new MyThread2(obj);
    t1.start();
    t2.start();
  }
}
```

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

The default interactive shell is now zsh.
To update your account to use zsh, please run `
For more details, please visit https://support.
bash-3.2$  /Library/Java/JavaVirtualMachines/ad
85a2b4998925731581b9dea5f/redhat.java/jdt_ws/Co
5
100
10
200
15
300
20
400
25
500
bash-3.2$
```

pdfelement

```java
class Table {

  synchronized void printTable(int n) {// r
    for (int i = 1; i <= 5; i++) {
      System.out.println(n * i);
      try {
        Thread.sleep(400);
      } catch (Exception e) {
        System.out.println(e);
      }
    }
  }
}
```

src > MyThread2.java > MyThread2

```java
class MyThread2 extends Thread {
  Table t;

  MyThread2(Table t) {
    this.t = t;
  }

  public void run() {
    t.printTable(100);
  }
}
```

src > MyThread1.java > MyThread1

```java
class MyThread1 extends Thread {
  Table t;

  MyThread1(Table t) {
    this.t = t;
  }

  public void run() {
    t.printTable(5);
  }

}
```

Remove Watermark Now

pdfelement

src > TestSynchronization1.java > TestSynchronization1

```java
class TestSynchronization1 {

  Run | Debug
  public static void main(String args[]) {
    Table obj = new Table();// only one object
    MyThread1 t1 = new MyThread1(obj);
    MyThread2 t2 = new MyThread2(obj);
    t1.start();
    t2.start();
  }
}
```

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE

```
20
400
25
500
f/redhat.java/jdt_ws/Concurrency_5e4940f0/bin" TestSynch
5
10
15
20
25
100
200
300
400
500
bash-3.2$
```

Remove Watermark Now

Metoda `wait()`

- Permite manevrarea zăvorului asociat cu un obiect
- La apelul metodei `wait()` pentru un obiect ***m*** de către un fir de execuție ***t***:
  - se deblochează zăvorul asociat cu ***m*** și ***t*** este adăugat la un set de thread-uri blocate, *wait set*-ul lui ***m***
  - dacă ***t*** nu deține zăvorul pentru ***m***: `IllegalMonitorStateException`
  - ***t*** va continua execuția doar când va fi scos din *wait set*-ul lui ***m***, prin:
    - o operație **notify() / notifyAll()**
    - expirarea timpului de așteptare
    - o acțiune dependentă de implementare

pdfelement

Remove Watermark Now

Notificări

- Metode: `notify()`, `notifyAll()`
- La o notificare apelată din thread-ul ***t*** pentru obiectul ***m***:
  - `notify()`: un thread ***u*** din *wait set*-ul lui ***m*** este scos și repus în execuție
  - `notifyAll()`: toate thread-urile sunt scoase din *wait set*-ul lui ***m*** – dar ***numai unul*** va putea obține zăvorul
  - daca ***t*** nu deține zăvorul pentru ***m***: `IllegalMonitorStateException`

pdfelement

Remove Watermark Now

Java – wait()/notify()/notifyAll()

```
this.wait() {
        unlock()
        put_thread_in_wait_state()  //nu mai este pe CPU
        lock()
}
```

pdfelement

Remove Watermark Now

Java – wait()/notify()/notifyAll()

Wait funcționează doar dacă ai deja un lock (apelul se face într-o secțiune synchronized)

```
object.wait() {
        unlock()
        put_thread_in_wait_state() //nu mai este pe CPU
        lock()
}
```

pdfelement

Remove Watermark Now

Java – wait()/notify()/notifyAll()

```
this.
object.notify() {
        thread = pick_random_thread_waiting_on_object()
        move_from_wait_to_running(thread)
}
```

pdfelement

Remove Watermark Now

Java – wait()/notify()/notifyAll()

this.notifyAll()
object.notifyAll()

pdfelement

Pune toate thread-urile care fac wait pe object în starea de running.
Un singur thread va putea lua lock()

Producator-consumator - pseudocod

Remove Watermark Now

```
int buf, p=0, c=0;
co process Producer{
        int a[1:n];
    while (p<n){<await p==c>;
          buf = a[p+1];
          p = p+1;}
    }
|| process Consumer{
           int b[1:n];
     while (c<n){<await p>c>;
          b[c+1] = buf;
          c = c+1;}
     }
oc
```

- buf, p, c sunt partajate de cele doua procese
- accesul este exclusiv
- accesul este direct (nu este intermediat de un alt proces)

pdfelement

Remove Watermark Now

```java
class ZonaTampon {
  private int buf; // val. curenta din tampon
  private boolean disponibil = false; // daca o valoare este disponibila in buf

  public synchronized int Consuma() {
    if (!disponibil) { // val. indisponibila
      try {
        wait(); // suspenda executia si deblocheaza zavor
      } catch (InterruptedException e) {
      }
    }
    disponibil = false;
    notify(); // pune in executie un thread in asteptare
    return buf;
  }

  public synchronized void Produ(int valoare) {
    if (disponibil) {// o valoare in tampon
      try {
        wait();
      } catch (InterruptedException e) {
      }
    }
    buf = valoare;
    disponibil = true;
    notify();
  }
}
```

pdfelement

Exemplu – Producator

Remove Watermark Now

```java
public class Producator extends Thread {
  private ZonaTampon Tampon;
  private int numar; // ID-ul producatorului

  public Producator(ZonaTampon z, int numar) {
    Tampon = z;
    this.numar = numar;
  }

  public void run() {
    for (int i = 0; i < 10; i++) {
      Tampon.Produ(i);
      System.out.println("Producator " + this.numar + " a transmis: " + i);
      try {
        sleep((int)Math.random() * 100);
      } catch (InterruptedException e) {}
    }
  }
}
```

pdfelement

Exemplu - Consumator

Remove Watermark Now

```java
public class Consumator extends Thread {
  private ZonaTampon Tampon;
  private int numar; // ID-ul consumatorului

  public Consumator(ZonaTampon z, int numar) {
    Tampon = z;
    this.numar = numar;
  }

  public void run() {
    int valoare = 0;
    for (int i = 0; i < 10; i++) {
      valoare = Tampon.Consuma();
      System.out.println("Consumator" + this.numar + " a preluat " + valoare);
    }
  }
}
```

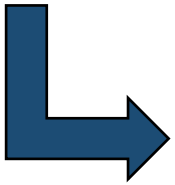
pdfelement

Remove Watermark Now

Exemplu - Test

```
public class App {
    Run | Debug
    public static void main(String[] args) {
        ZonaTampon z = new ZonaTampon();
        Producator p1 = new Producator(z, 1);
        Consumator c1 = new Consumator(z, 1);
                        //p1 si c1 folosesc aceeasi ZonaTampon
        p1.start();
        c1.start();
    }

}
```

pdfelement



```
Producator 1 a transmis: 0
Consumator1 a preluat 0
Producator 1 a transmis: 1
Consumator1 a preluat 1
Consumator1 a preluat 2
Producator 1 a transmis: 2
Producator 1 a transmis: 3
Consumator1 a preluat 3
Consumator1 a preluat 4
Producator 1 a transmis: 4
Producator 1 a transmis: 5
Consumator1 a preluat 5
Consumator1 a preluat 6
Producator 1 a transmis: 6
```

Remove Watermark Now

Clase utile pentru sincronizare în JDK 1.5

- `Semaphore`
- `Mutex`
- `CyclicBarrier`
  - barieră reutilizabilă
  - are ca argument un contor care arată numărul de fire din grup
- `CountDownLatch`
  - similar cu bariera, are un contor, dar decrementarea contorului este separată de așteptarea ajungerii la zero
  - decrementarea semnifică terminarea unor operații
- `Exchanger`
  - *rendez-vous* cu schimb de valori în ambele sensuri între threaduri

pdfelement

Remove Watermark Now

Facilități pentru sincronizare de nivel scăzut

- `Lock`
  - generalizare *lock monitor* cu așteptări contorizate, întreruptibile, teste etc.
- `ReentrantLock`
- `Conditions`
  - permit mai multe condiții per lock
- `ReadWriteLock`
  - exploatarea faptului că, la un moment dat, un singur fir modifică datele comune și ceilalți doar citesc
- Variabile atomice:
  - `AtomicInteger`
  - `AtomicLong`
  - `AtomicReference`
  - permit execuția atomică *read-modify-write*

pdfelement

Remove Watermark Now

volatile

```
volatile int i;
```

Fiecare scrierea asupra lui i trebuie să ajungă la toate thread-urile

Toate operațiile cu i pot fi mai lente

```
int i;
```

i poate fi ținut în cache (cache posibil ar fi regiștrii CPU)

Toate operațiile cu i pot fi mai rapide

pdfelement

Remove Watermark Now

Semaphore

Declararea unui semafor

```
Semaphore semaphore = new Semaphore(4);
```

Permits
(poate fi negativ)

```
try {
        semaphore.acquire();
} catch (InterruptedException e) {
        e.printStackTrace();
}
```

Luarea unui semafor **P()**
(Decrementează permits)
(Așteaptă dacă permits e 0)

pdfelement

Se eliberează semaforul **V()**
(Crește numărul de permits)

```
semaphore.release();
```

Un semafor inițializat cu 1 este echivalentul unui mutex/lock

Remove Watermark Now

Cyclic Barrier

Declararea barierei

Numărul de thread-uri care trebuie să aștepte la barieră ca aceasta să se deschidă

```
CyclicBarrier barrier = new CyclicBarrier(5);
```

Așteaptă până numărul necesar de thread-uri ajung la acest apel.

```
try {
        barrier.await();
} catch (InterruptedException | BrokenBarrierException
e) {
        e.printStackTrace();
}
```

pdfelement

Remove Watermark Now

Cyclic Barrier

• O barieră garantează că toate thread-urile execută tot codul de dinainte de barieră înainte de a executa orice linie de cod de după barieră

• Barierele clasice nu puteau fi folosite înăuntrul unei iterații
    • După ce toate thread-urile ajung la barieră unul o putea lua înainte și ajunge iar la barieră înainte să se închidă și ar mai trece o dată de ea.

• O barieră **CyclicBarrier** sau **Reentrantă** poate fi refolosită (Este sigură pentru a fi folosită în for/while)

pdfelement

Remove Watermark Now

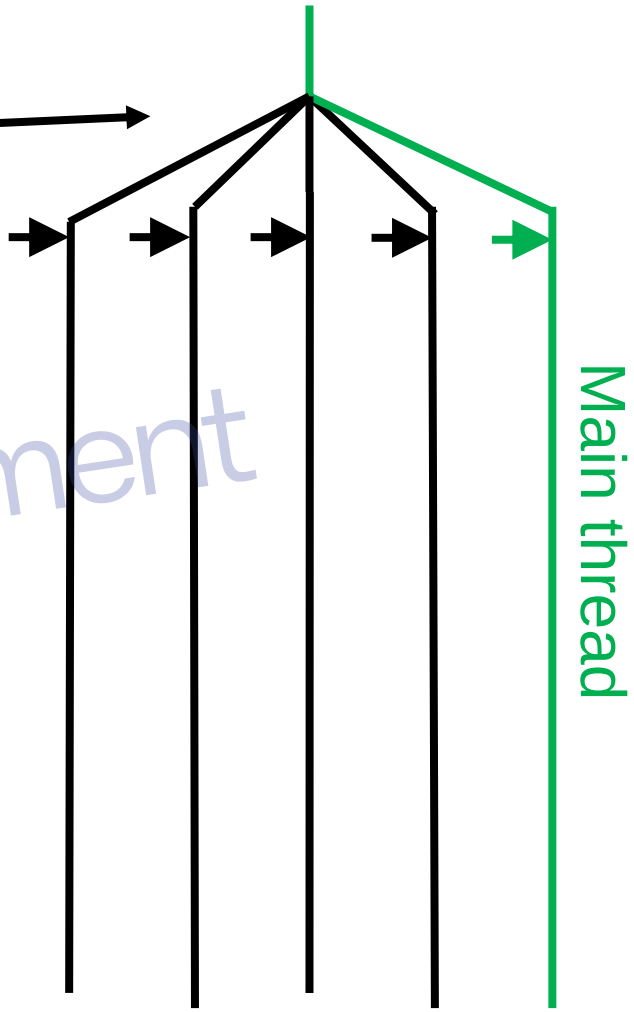
Executor Service – replicated workers

pdfelement

Remove Watermark Now

Executor Service

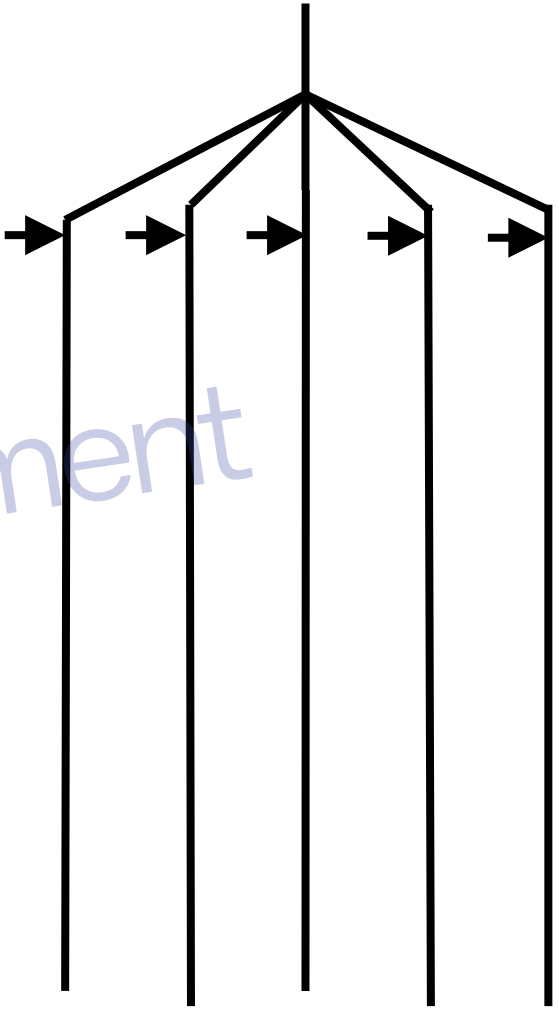
```
ExecutorService tpe =
Executors.newFixedThreadPool(4);
```



pdfelement

Remove Watermark Now

Executor Service



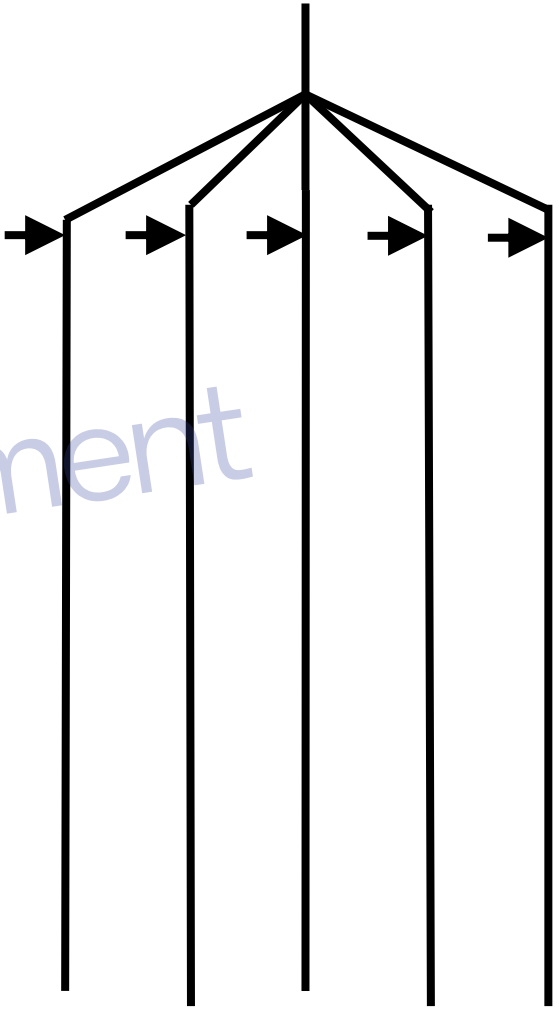
```
public class MyRunnable implements Runnable {
	int a;
	ExecutorService tpe;

	public MyRunnable(ExecutorService tpe, int a) {
		this.a = a;
		this.tpe = tpe;
	}

	@Override
	public void run() {
		if (a > 10) {
				tpe.shutdown();
				return;
		}
		System.out.println(a);
		tpe.submit(new MyRunnable(tpe, a + 3));
	}
}
```

pdfelement

Remove Watermark Now

Executor Service

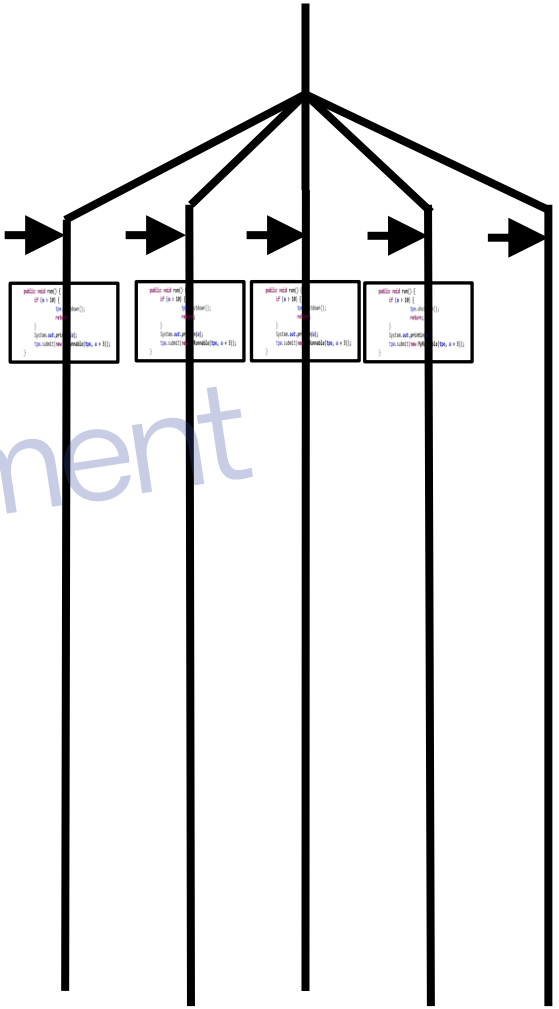
```
tpe.submit(new MyRunnable(tpe, 0));
tpe.submit(new MyRunnable(tpe, 1));
tpe.submit(new MyRunnable(tpe, 2));
……
```



pdfelement

Remove Watermark Now

Executor Service



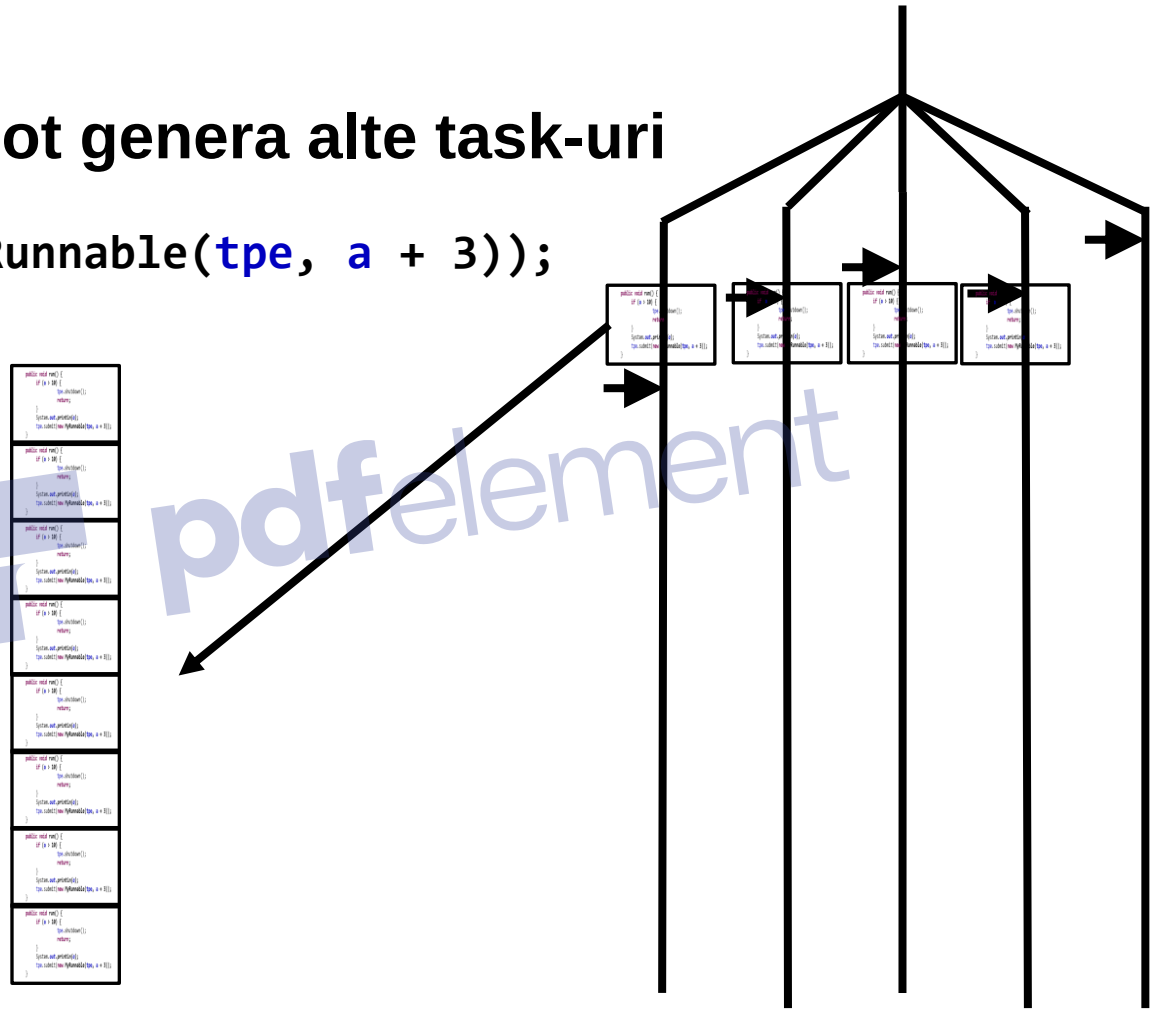
pdfelement

Remove Watermark Now

Executor Service

Task-urile pot genera alte task-uri

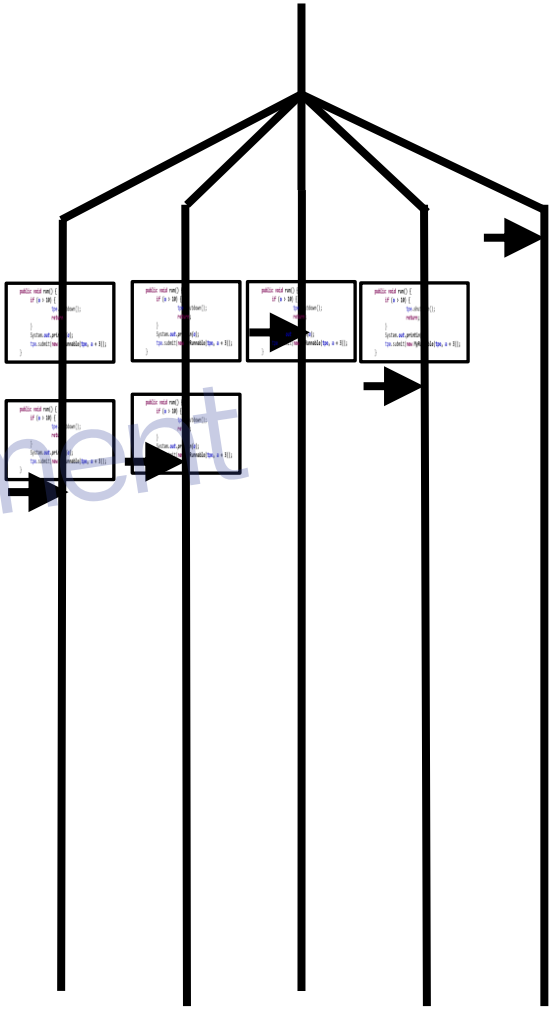
```
tpe.submit(new MyRunnable(tpe, a + 3));
```



pdfelement

Remove Watermark Now

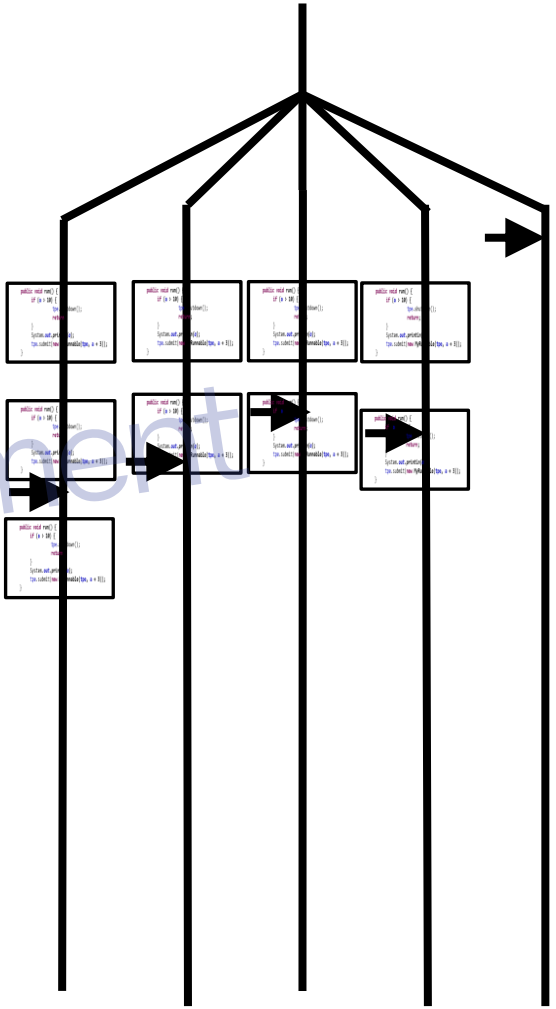
Executor Service



pdfelement

Remove Watermark Now

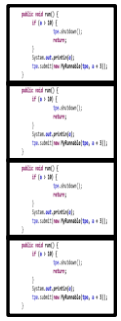
Executor Service

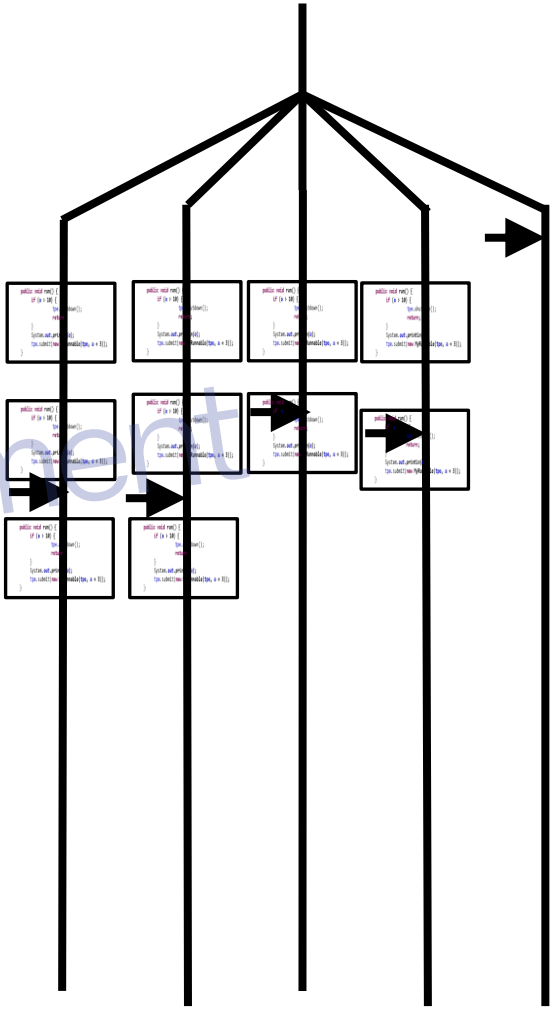


pdfelement

Remove Watermark Now

Executor Service

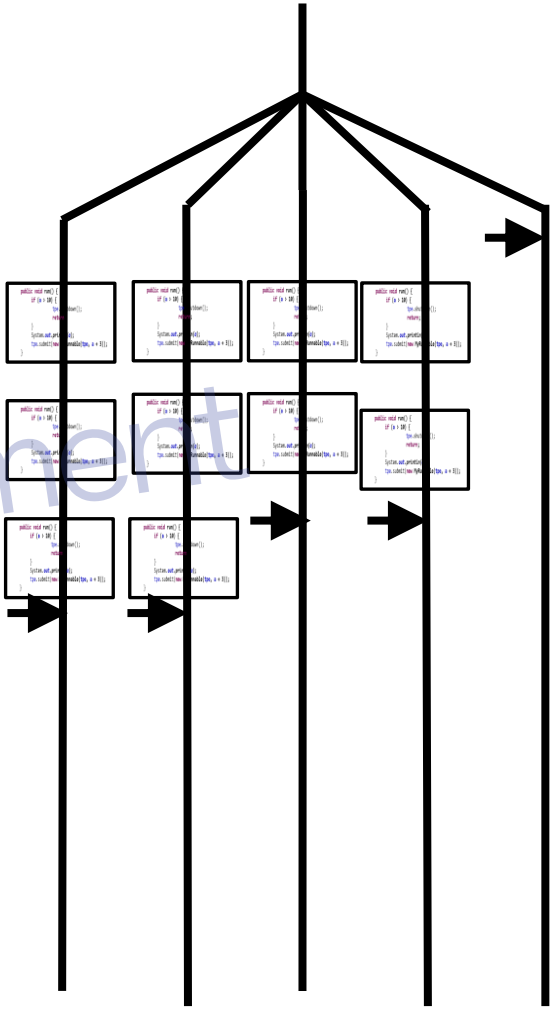




pdfelement

Remove Watermark Now

Executor Service



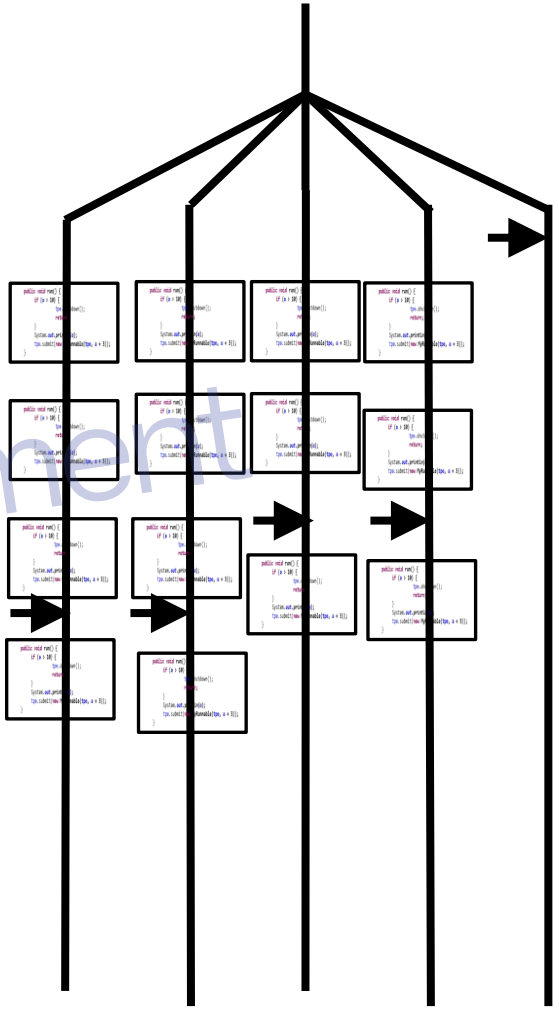
pdfelement

Remove Watermark Now

Executor Service

Când oprim thread-urile?

- Depinde de problemă.
- Uneori ne ajunge o singură soluție.
- Alte probleme pot necesita toate soluțiile.



```
tpe.shutdown();
```

pdfelement

Remove Watermark Now

```java
public class MyRunnable implements Runnable {
    private final long countUntil;

    MyRunnable(long countUntil) {
        this.countUntil = countUntil;
    }

    @Override
    public void run() {
        long sum = 0;
        for (long i = 1; i < countUntil; i++) {
            sum += i;
        }
        System.out.println(sum);
    }
}
```

```java
public class App {
    private static final int NTHREDS = 10;

    Run | Debug
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(NTHREDS);
        for (int i = 0; i < 500; i++) {
            Runnable worker = new MyRunnable(10000000L + i);
            executor.execute(worker);
        }
        // This will make the executor accept no new threads
        // and finish all existing threads in the queue
        executor.shutdown();
        // Wait until all threads are finish
        try{
            executor.awaitTermination(60, TimeUnit.SECONDS);
        } catch(InterruptedException ie) {
            // (Re-)Cancel if current thread also interrupted
            executor.shutdownNow();
        }
    }
}
```

pdfelement

CompletableFuture – async tasks

Remove Watermark Now

```
CompletableFuture.supplyAsync(this::doSomething);
```

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;

public class CompletableFutureCallback {
    public static void main(String[] args) {
        long started = System.currentTimeMillis();

        CompletableFuture<String>  data = createCompletableFuture()
                .thenApply((Integer count) -> {
                    int transformedValue = count * 10;
                    return transformedValue;
                }).thenApply(transformed -> "Finally creates a string: " + transformed);

            try {
                System.out.println(data.get());
            } catch (InterruptedException | ExecutionException e) {

            }
    }

    public static CompletableFuture<Integer> createCompletableFuture() {
        CompletableFuture<Integer>  result = CompletableFuture.supplyAsync(() -> {
            try {
                // simulate long running task
                Thread.sleep(5000);
            } catch (InterruptedException e) { }
            return 20;
        });
        return result;
    }

}
```

Remove Watermark Now

```java
public class CompletableFutureSimpleSnippet {
    public static void main(String[] args) {
        long started = System.currentTimeMillis();

        // configure CompletableFuture
        CompletableFuture<Integer> futureCount = createCompletableFuture();

            // continue to do other work
            System.out.println("Took " + (started - System.currentTimeMillis()) + " milliseconds"
);

            // now its time to get the result
            try {
              int count = futureCount.get();
                System.out.println("CompletableFuture took " + (started -
System.currentTimeMillis()) + " milliseconds" );

               System.out.println("Result " + count);
             } catch (InterruptedException | ExecutionException ex) {
                // Exceptions from the future should be handled here
            }
    }

    private static CompletableFuture<Integer> createCompletableFuture() {
        CompletableFuture<Integer> futureCount = CompletableFuture.supplyAsync(
                () -> {
                    try {
                        // simulate long running task
                        Thread.sleep(5000);
                    } catch (InterruptedException e) { }
                    return 20;
                });
        return futureCount;
    }

}
```

Remove Watermark Now

Structuri de date

pdfelement

Remove Watermark Now

HashMap

Keys	Values
A	11111
DD	2222
C	33333
S	4444

Hash(Keys)	Values
1	2222
2	11111
3	33333
4	
5	4444

pdfelement

Search **O(1)**
Insertion **O(1)**
Deletion **O(1)**

Remove Watermark Now

ConcurrentHashMap

- Folosește lock-uri în interiorul tuturor metodelor: **get**(), **put**()
- Asta înseamnă că se poate folosi structura fără cod special pentru sincronizare.
- Are și metode speciale gen **putIfAbsent**()

pdfelement

- Există mai multe astfel de structuri (idem pentru HashTable)

Remove Watermark Now

ConcurrentHashMap – privire sub capotă

- Dacă folosim un singur lock pentru accesarea HashMap: 2 thread-uri nu îl vor putea modifica simultan.
- Dacă folosim un lock pentru fiecare element din HashMap: Vom folosi multă memorie și avem nevoie de mecanisme speciale pentru inserție sau ștergere.
- Soluția e folosirea unui număr limitat de lock-uri (de obicei egal cu numărul de thread-uri).

pdfelement

Hash(Keys)	Values	
1	2222	Lock A
2	11111	
3	33333	Lock B
4		
5	4444	Lock C

Remove Watermark Now

```java
class HashMapDemo extends Thread {
    static HashMap<Integer,String> l=new HashMap<Integer,String>();

    public void run() {
        try {
            Thread.sleep(1000);
            l.put(103,"D");
        } catch(InterruptedException e) {
            System.out.println("Child Thread going to add element");
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        l.put(100,"A");
        l.put(101,"B");
        l.put(102,"C");
        HashMapDemo t=new HashMapDemo();
        t.start();

        for (Object o : l.entrySet())  {
            Object s=o;
            System.out.println(s);
            Thread.sleep(1000);
        }
        System.out.println(l);
    }
}
```

```
100=A
101=B
Exception in thread "main" java.util.ConcurrentModificationException
        at java.base/java.util.HashMap$HashIterator.nextNode(HashMap.java:1493)
        at java.base/java.util.HashMap$EntryIterator.next(HashMap.java:1526)
        at java.base/java.util.HashMap$EntryIterator.next(HashMap.java:1524)
        at HashMapDemo.main(HashMapDemo.java:23)
```

pdfelement

```java
public class App extends Thread {

    static ConcurrentHashMap<Integer, String> l = new ConcurrentHashMap<Integer, String>(

    public void run() {
        l.put(103, "D");
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println("Child Thread going to add element");
        }
    }

    Run | Debug
    public static void main(String[] args) throws Exception {
        l.put(100, "A");
        l.put(101, "B");
        l.put(102, "C");
        HashMapDemo t = new HashMapDemo();
        t.start();

        for (Object o : l.entrySet()) {
            Object s = o;
            System.out.println(s);
            Thread.sleep(1000);
        }
        System.out.println(l);
    }
}
```

```
a/jdt_ws/HashMapDemo_eca003ea/bin" App 2fa1c97cda8748c6465d77ce70e076/redhat
100=A
101=B
102=C
{100=A, 101=B, 102=C}
```

Remove Watermark Now
pdfelement

Remove Watermark Now

Java – Atomic*

- **AtomicInteger**
- **AtomicLong**
- **AtomicBoolean**
- **AtomicFloat**
- **AtomicReference**

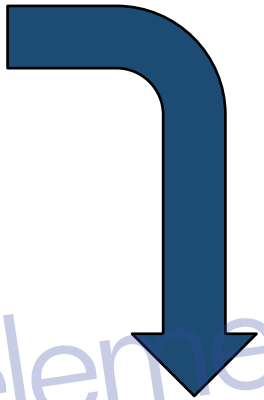
- Problema: 32bit/64bit systems (+ alte arhitecturi)
  - Copierea elementelor de 64 de biți pe procesor de 32 de biți ia mai multe operații iar thread-ul poate fi scos de pe CPU în timpul unei scrieri
  - Un alt thread poate citi doar o parte din variabilă

- **Atomic*** asigură că toate operațiile cu aceea variabilă sunt atomice (nu se pot face citiri în mijloc)
- Oferă și oprații mai complexe **getAndSet/getAndIncrement** implementate atomic

pdfelement

Remove Watermark Now

```java
import java.util.concurrent.atomic.AtomicInteger;

public class Counter {
    private AtomicInteger value = new AtomicInteger();
    public int getValue(){
        return value.get();
    }
    public int increment(){
        return value.incrementAndGet();
    }

    // Alternative implementation as increment but just make the
    // implementation explicit
    public int incrementLongVersion(){
        int oldValue = value.get();
        while (!value.compareAndSet(oldValue, oldValue+1)){
             oldValue = value.get();
        }
        return oldValue+1;
    }

}
```



pdfelement

```java
public final int incrementAndGet() {
        for (;;) {
            int current = get();
            int next = current + 1;
            if (compareAndSet(current, next))
                return next;
        }
    }
```

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Mecanisme de sincronizare

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Race Conditions

Process a:

```
while(i < 10)
    i = i +1;
print "A won!";
```

Process b:

```
while(i > -10)
    i = i - 1;
print "B won!";
```

pdfelement

– Cine câștigă?
– Există un câștigător?

Remove Watermark Now

Problema secțiunilor critice

- Problema: Dezvoltarea unui protocol pentru cooperare între procese, a.î. execuția să aibă loc mutual exclusiv (secțiuni critice)
  - Cum să facem ca mai multe instrucțiuni să pară ca una singură?

Shared vars:

Initialization:

Process:
. . .
. . .

Entry Section

Critical Section

Exit Section

Soluția constă în introducerea acestor mecanisme

pdfelement

Cerințe ale soluției

Remove Watermark Now

Discuție asupra folosirii semafoarelor pentru soluționare...

- **Mutual Exclusion**
  - Doar un process poate fi la un moment dat în secțiunea critică
- **Progress**
  - Decizia cine intră în regiune nu poate fi amânată indefinit
    - No deadlock
- **Bounded Waiting**
  - Limită asupra #times alte procese pot intra în regiune, în timp ce aștept
    - No livelock
- La care se adaugă eficiență, fairness …

pdfelement

Remove Watermark Now

Reminder: Dekker's Algorithm

- Presupune două thread-uri, numerotate cu 0 și 1

```
CSEnter(int i)
 {
   inside[i] = true;
   while (inside[j])
   {
     if (turn == j)
     {
        inside[i] = false;
        while(turn == j) continue;
        inside[i] = true;
      }
   }}
```

```
CSExit(int i)
{
  turn = j;
  inside[i] = false;
   }
```

pdfelement

Remove Watermark Now

Peterson's Algorithm (1981)

```
CSEnter(int i)
{
      inside[i] = true;
      turn = j;
      while (inside[j] && turn == j)
            continue;
}
```

```
CSExit(int i)
{
  inside[i] = false;
  }
```

pdfelement

- Mai simplu e mai bine!!

Remove Watermark Now

O analiză a algoritmului lui Peterson:

- Safety (prin contradicție):
  - Presupuneți că două procese (Ana și Ion) sunt în regiunea critică (ambele au flag-ul *inside* setat). Deoarece doar unul, să spunem Ana, poate avea *turn*, celălalt (Ion) **trebuie** să fi ajuns la testul *while()* înainte ca Ana să seteze flag-ul *inside*.
  - Totuși, după setarea flag-ului *inside*, Ana a cedat *turn* către Ion. Ion deja a modificat *turn* și **nu poate** să îl modifice din nou, ceea ce contravine presupunerii inițiale.

pdfelement

Liveness & Bounded waiting => date de variabila *turn*.

Remove Watermark Now

Putem generaliza pentru mai multe thread-uri?

- Abordarea anterioară nu funcționează:

```
CSEnter(int i)
{
      inside[i] = true;
      for (j = 0; j < N; j++)
            while (inside[j] && turn == j)
                  continue;
}
```

```
CSExit(int i)
{
      inside[i] = false;
}
```

pdfelement

- Problema: Al cui e rândul?

Remove Watermark Now

Conceptul de Bakery

- Introdus de Leslie Lamport
- Gândiți-vă la un magazin popular cu un vânzător (ex. cazul unui Apple Store și lansarea următorului iPhone)
  - Oamenii iau un bilet de la un automat
  - Dacă nimeni nu așteaptă, biletul nu mai contează
  - Dacă mai mulți clienți așteaptă, ordinea biletelor determină ordinea în care vor fi serviți

pdfelement

Remove Watermark Now

Algoritmul Bakery : “Runda 1”

- int ticket[n];
- int next_ticket;

```
CSEnter(int i)
{
      ticket[i] = ++next_ticket;
      for (j = 0; j < N; j++)
            while(ticket[j] && ticket[j] < ticket[i])
                  continue;
}
```

```
CSExit(int i)
{
      ticket[i] = 0;
}
```

pdfelement

- Oops… accesul la *next_ticket* este o problemă!

Remove Watermark Now

Algoritmul Bakery: “Runda 2”

- int ticket[n];

Adăugăm 1 la maxim!

```
CSEnter(int i)
{
        ticket[i] = max(ticket[0], … ticket[N-1])+1;
        for (j = 0; j < N; j++)
                while (ticket[j] && ticket[j] < ticket[i])
                        continue;
}
```

```
CSExit(int i)
{
        ticket[i] = 0;
}
```

pdfelement

- Idee: adăugăm 1 la maxim.
- Oops… două procese pot prelua o aceeași valoare!

Remove Watermark Now

Algoritmul Bakery: “Runda 3”

Dacă i și j aleg un același bilet, stabilim o regulă tip “break tie”:

(ticket[j] < ticket[i]) || (ticket[j]==ticket[i] && j<i)

pdfelement

Notație: (B,j) < (A,i) pentru a simplifica scrierea codului:

(B<A || (B==A && j<i)), e.g.:

(ticket[j],j) < (ticket[i],i)

Remove Watermark Now

Algoritmul Bakery: “Runda 4”

- int ticket[N];
- boolean picking[N] = false;

```
CSEnter(int i)
{
    ticket[i] = max(ticket[0], … ticket[N-1])+1;
    for (j = 0; j < N; j++)
        while(ticket[j] && (ticket[j], j) < (ticket[i],i))
            continue;
}
```

```
CSExit(int i)
{
    ticket[i] = 0;
}
```

pdfelement

- Oops… aș putea să mă uit la j atunci când j încă nu deține biletul, și totuși j ar putea avea un id mai mic ca al meu (i)!

Remove Watermark Now

Algoritmul Bakery: Aproape gata

- int ticket[N];
- boolean choosing[N] = false;

```
CSEnter(int i)
{
    choosing[i] = true;
    ticket[i] = max(ticket[0], … ticket[N-1])+1;
    choosing[i] = false;
    for (j = 0; j < N; j++) {
        while(choosing[j]) continue;
        while(ticket[j] && (ticket[j], j) < (ticket[i],i))
            continue;
    }
}
```

```
CSExit(int i)
{
    ticket[i] = 0;
}
```

pdfelement

Remove Watermark Now

Algoritmul Bakery: Probleme?

- Dacă nu știm câte thread-uri rulează?
  - Algoritmul depinde de valoarea N
  - Cumva avem nevoie de un mecanism de ajustare a N atunci când un thread e creat sau termină execuția
- De asemenea, tehnic, un bilet poate da overflow!
  - Soluția: Modificarea codului ca, atunci când un bilet ajunge la o valoare prea mare, să îl resetăm la 0

pdfelement

Remove Watermark Now

Algoritmul Bakery: Final

- int ticket[N]; /* Important: Disable thread scheduling when changing N */
- boolean choosing[N] = false;

```
CSEnter(int i)
{
        do {
            ticket[i] = 0;
            choosing[i] = true;
            ticket[i] = max(ticket[0], … ticket[N-1])+1;
            choosing[i] = false;
        } while(ticket[i] >= MAXIMUM);
        for (j = 0; j < N; j++) {
                while(choosing[J]) continue;
                while(ticket[J] && (ticket[J],J) < (ticket[i],i))
                        continue;
        }
}
```

```
CSExit(int i)
{
        ticket[i] = 0;
}
```

pdfelement

Remove Watermark Now

Ne reîntoarcem la semafoare

- Unele sisteme (acolo unde busy-waiting nu e o problemă, gen există suficiente core-uri) implementează soluții bazate pe Algoritmul Bakery
- Alteori (cel mai des însă) avem nevoie de *semafor*
  - Soluție cu *blocare* în loc de *busy-waiting* de la SO

Blocare vs. non-blocare

```
acquire() {
    while (value <= 0)
        ; // no-op
    value--;
}

release() {
    value++;
}
```

```
acquire() {
    value--;
    if (value < 0) {
        add this process to list
        block();
    }
}

release() {
    value++;
    if (value <= 0) {
        remove a process P from list
        wakeup(P);
    }
}
```

pdfelement

Remove Watermark Now

Un pic de recap... ce e greșit?

```
Shared: Semaphores mutex, empty, full;

Init: mutex = 1;  /* for mutual exclusion*/
      empty = N; /* number empty bufs */
      full = 0;      /* number full bufs */
```

Producer

```
do {
   . . .
   // produce an item in nextp
   . . .
   P(mutex);
   P(empty);
   . . .
   // add nextp to buf
   . . .
   V(mutex);
   V(full);
} while (true);
```

Ce se întâmplă dacă buffer = full?

Oops! Ordinea în care realizați operații poate avea un impact semnificativ asupra corectitudinii

Consumer

```
do {
   P(full);
   P(mutex);
   . . .
   // remove item to nextc
   . . .
   V(mutex);
   V(empty);
    . . .
    // consume item in nextc
    . . .
} while (true);
```

pdfelement

Remove Watermark Now

Exercițiu

- Cum ati implementa o *barieră* folosind semafoare?
- Ex. (suma elementelor unui vector):

```
int a[1:n];
process suma[k=1 to n]{
      for [j = 1 to sup(log2 n)]{
            if (k mod 2^j = 0){
                  a[k] = a[k-2^(j-1)] + a[k];
            }
            barrier
      }
}
```

pdfelement

Remove Watermark Now

Posibila solutie… pentru bariera

```
sem b = 0; e = 1;
int nb = 0;
process proc[k=1 to n]{
    # enter barrier
    P(e);
    nb=nb+1;
    if (nb = n){ V(e); V(b);}
    else if (nb > 1){ V(e); P(b); V(b);}
    else if (nb = 1) { V(e); P(b);}
    P(e);
    nb=nb-1;
    V(e);
    # exit barrier
}
```

if last process to enter barrier

if first process to enter barrier

pdfelement

Remove Watermark Now

Un pic de practice: Cigarette smokers problem

- Un agent și trei fumători
- Fumătorii:
  - Așteaptă ingrediente (tutun, hârtie, chibrit)
  - Confecționează țigară
  - Fumează
- Agentul deține toate 3 ingredientele
- Un fumător are tutun, un altul hârtie, al 3-lea chibrituri)
- Agentul selectează două ingrediente (random) pe care le dă fumătorilor
  - Doar fumătorul ce are nevoie de exact acele 2 ingrediente trebuie să le preia
  - Agentul nu poate semnaliza exact acelui fumător pentru că nu știe care fumător e care, respectiv ingredientele sunt random extrase

pdfelement

Remove Watermark Now

Cigarette smokers problem

```
sem tobacco = 0;
sem paper = 0;
sem match = 0;
Sem agent = 1;
process Agent{
     while (true){
       if (draw1){ P(agent); V(tobacco); V(paper); }
       else if (draw2){ P(agent); V(paper); V(match); }
       else if (draw3){ P(agent); V(tobacco); V(match);}
     }
}
process Smoker1{
     P(tobacco); P(paper); V(agent);
}
process Smoker2{
     P(paper); P(match); V(agent);
}
process Smoker3{
     P(tobacco); P(match); V(agent);
}
```

pdfelement

Funcționează ???

Remove Watermark Now

Cigarette smokers problem - deadlock

OK!

P(match)

P(tobacco)

DEADLOCK!

P(paper)

P(match)

P(tobacco) **P(paper)**

Remove Watermark Now

Practice

- Gandiți-vă la o soluție pentru evitarea deadlock-ului…

pdfelement

Remove Watermark Now

Cigarette smokers problem

```
sem tobacco = 0;
sem paper = 0;
sem match = 0;
sem agent = 1;

bool isTobacco = false;
bool isPaper = false;
bool isMatch = false;

sem tobaccoSem = 0;
sem paperSem = 0;
sem matchSem = 0;

process Agent{
     while (true) {
       if (draw1) { P(agent); V(tobacco); V(paper);}
       else if (draw2) { P(agent); V(paper); V(match);}
       else if (draw3) { P(agent); V(tobacco); V(match);}
     }
}
```

pdfelement

Remove Watermark Now

Cigarette smokers problem

```
process PusherA{
     P(tobacco);
     P(e);
     if (isPaper) { isPaper =  false; V(matchSem);}
     else if (isMatch) { isMatch = false; V(paperSem);}
     else if (isPaper == isMatch == false) isTobacco = true;
     V(e);
}

proces PusherB{
     P(match);
     P(e);
     if (isPaper) { isPaper =  false; V(tobaccoSem); }
     else if (isTobacco) { isTobacco = false; V(paperSem);}
     else if (isPaper == isTobacco == false) isMatch = true;
     V(e);
}

process PusherC{
     P(paper);
     P(e);
     if (isTobacco) { isTobacco =  false; V(matchSem);}
     else if (isMatch) { isMatch = false; V(tobaccoSem);}
     else if (isPaper == isMatch == false) isPaper = true;
     V(e);
}
```

pdfelement

Remove Watermark Now

Cigarette smokers problem

```
process SmokerWithTobacco{
     P(tobaccoSem);
     # makeCigarette
     V(agent);
     # smoke
}
process SmokerWithPaper{
     P(paperSem);
     # makeCigarette
     V(agent);
     # smoke
}
process SmokerWithMatch{
     P(matchSem);
     # makeCigarette
     V(agent);
     # smoke
}
```

pdfelement

Remove Watermark Now

O altă problemă - East-West Bridge Problem

- Considerați un pod, prea îngust pentru a permite mașinile să meargă în ambele direcții.
  - Mașinile trebuie să alterneze pt. trecerea podului.
  - Podul nu este, de asemenea, suficient de puternic pentru a ține mai mult de **trei** mașini simultan.
- Găsiți o soluție la această problemă care să evite **starvation**.
  - Mașinile care doresc să treacă ar trebui să ajungă în cele din urmă.
- Încercați să maximizați capacitatea podului (adică trei mașini ar trebui să meargă la un moment dat).
  - Dacă o mașină părăsește podul care se îndreaptă spre est și nu există mașini spre vest, atunci următoarea mașină spre est trebuie lăsată să treacă.

pdfelement

Remove Watermark Now

Remove Watermark Now

Soluția problemei

- Soluția: multiple readers writers problem.
- Intuitiv, dacă esti un reader ce intră în cameră, și vezi writers în cameră, iei loc (dacă esti o masină din est, și mașinile din vest traversează, aștepți).
- Dacă ești un reader, și sunt mai puțin de trei readers, și nu există writers în așteptare, intri în cameră.
- Dacă ești un reader, și sunt exact trei readers, aștepți.
- Dacă ești un reader ce părăsește camera, dacă nu mai sunt writers dar sunt readers în așteptare, permiți unui reader să intre.
- Dacă sunt writers în așteptare, și ești ultimul reader, permiți următorilor trei writers să intre.
- Principala diferență față de readers-writers e că vrem să permitem la **exact** trei readers sau writers să intre simultan în cameră.

pdfelement

Remove Watermark Now

Invarianți

- Safety conditions: Readers și writers nu pot fi activi în același timp:
  - nr_active <= 3
  - nw_active <= 3
  - nr_active > 0 --> nw_active == 0
  - nw_active > 0 --> nr_active == 0
- Alte condiții:
  - nr_active > 0 & nr_waiting > 0 --> nw_waiting > 0
  - nw_active > 0 & nw_waiting > 0 --> nr_waiting > 0

pdfelement

Soluție pentru readers:

Remove Watermark Now

```
P( lock ); // lock is a mutex and is initializes to 1.
if (( nw_active + nw_waiting == 0 ) && ( nr_active < 3 )) {
     nr_active++; // notify we are active
     V( r_sem ); // allow ourself to get through
}  else
     nr_waiting++; // we are waiting
V( lock );
P( r_sem ); // readers will wait here, if they must wait.

READING...

P( lock );
nr_active--;
if (( nr_active == 0 ) && ( nw_waiting > 0 ))  { // if we are the last reader
    count = 0; // makes sure we only at most 3 writers in
    while (( nw_waiting > 0 ) && ( count < 3 )) {
          V( w_sem ); // wake a writer;
          w_active++; // one more active writer
          w_waiting--; // one less waiting writer.
          count++;
    }
}  else if (( nw_waiting == 0 ) && ( nr_waiting > 0 )) { // allow another waiting reader to go in, if no waiting
writers
           V( r_sem );
           w_active++; // one more active reader
           w_waiting--; // one less waiting reader.
}
V( lock );
```

Initializați semafoarele

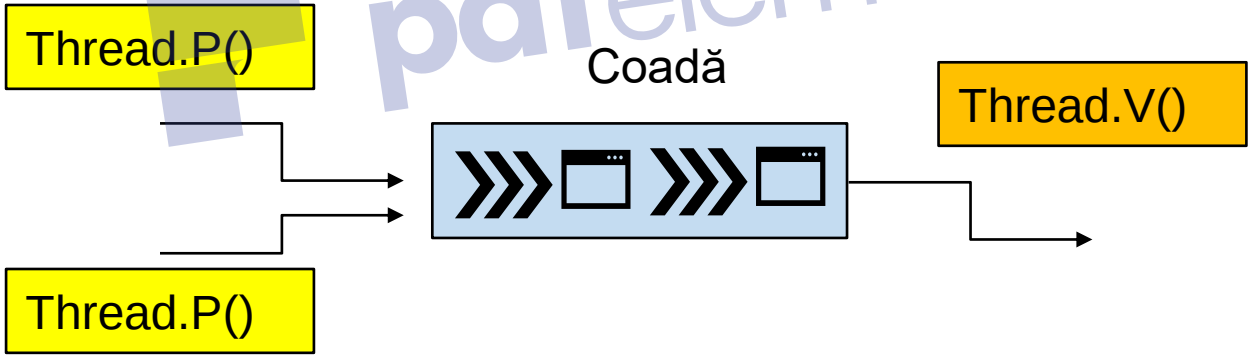
Să scriem soluția pentru writers...

pdfelement

Remove Watermark Now

Ordinea semafoarelor

- În momentul în care apelați V(), oricare thread blocat pe P() va fi trezit
  - Dar dacă doriți păstrarea ordinii FIFO?



pdfelement

Remove Watermark Now

Example: P and V

```
void v(struct semaphore *s) {
    struct proc_desc *p = 0;

    save = intr_enable( ALL_DISABLE );
    while ( TestAndSet( &s->sem_lock ) );
    s->sem_count++;
    if (p = get_from_queue( &s->sem_queue)) {
       p->runstate &= ~PROC_BLOCKED;
    }
    s->sem_lock = 0;
    intr_enable( save );
    if (p)
       reschedule(p);
}
```

```
void p(struct semaphore *s) {
    struct proc_desc *p = 0;
    for (;;) {
       save = intr_enable( ALL_DISABLE );
       while ( TestAndSet( &s->sem_lock ) );
       if (s->sem_count > 0) {
            s->sem_cont--;
            s->sem_lock = 0;
            intr_enable( save );
            return;
       }
       add_to_queue( &s->sem_queue, myproc );
       myproc->runstate |= PROC_BLOCKED;
       s->sem_lock = 0;
       intr_enable( save );
       yield();
    }
}
```

pdfelement

process A			**P**	**P**		**WAKE**
process B		**V**			**V**	
Semaphore						
lock	NO	YES	YES	YES	YES	YES
count	0	1	0		1	0
queue	0			A		
int disable	NO	YES	YES	YES	YES	YES

Remove Watermark Now

Semafor FIFO în Java

```
public class SemaphoreQueue
{
    private SemaphoreSlim semaphore;
    private ConcurrentQueue<TaskCompletionSource<bool>> queue =
        new ConcurrentQueue<TaskCompletionSource<bool>>();
    public SemaphoreQueue(int initialCount)
    {
        semaphore = new SemaphoreSlim(initialCount);
    }
    public SemaphoreQueue(int initialCount, int maxCount)
    {
        semaphore = new SemaphoreSlim(initialCount, maxCount);
    }
    public void Wait()
    {
        WaitAsync().Wait();
    }
    public Task WaitAsync()
    {
        var tcs = new TaskCompletionSource<bool>();
        queue.Enqueue(tcs);
        semaphore.WaitAsync().ContinueWith(t =>
        {
            TaskCompletionSource<bool> popped;
            if (queue.TryDequeue(out popped))
                popped.SetResult(true);
        });
        return tcs.Task;
    }
    public void Release()
    {
        semaphore.Release();
    }
}
```

pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
MPI

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

MPI - *Message Passing Interface*

- **MPI** este un **standard** pentru comunicarea prin mesaje elaborat de *MPI Forum*.
- Sisteme anterioare:
  - Intel (NX/2) Express
  - nCUBE (Vertex)
  - PARMACS
  - Zipcode
  - Chimp
  - PVM
  - Chameleon
  - PICL
- MPI are la bază modelul proceselor comunicante prin mesaje.

pdfelement

Remove Watermark Now

MPI - *Message Passing Interface*

- MPI este o (*specificație* pentru o) **bibliotecă**, nu un limbaj.
- MPI specifică reguli de apel pentru:
  - C / C++
  - FORTRAN 77 / FORTRAN 90
- Istoric:
  - **MPI 1** (1993 – SC93) - orientat pe comunicarea punct la punct
  - **MPI 2** (1997) include:
    - Procese dinamice
    - Comunicarea *one-sided*
    - Operații de I/E paralele

pdfelement

Remove Watermark Now

Obiective MPI

- Standardizare
  - Singura bibliotecă de transmitere de mesaje ce poate fi considerată standard
- Portabilitate
  - Codul nu trebuie modificat la portarea aplicațiilor
- Performanță
  - Implementarile producătorilor pot exploata carcateristici hardware native
- Funcționalitate
  - Peste 115 rutine doar în **MPI 1**
- Disponibilitate
  - Implementări atât publice, cât și de la producători

pdfelement

Remove Watermark Now

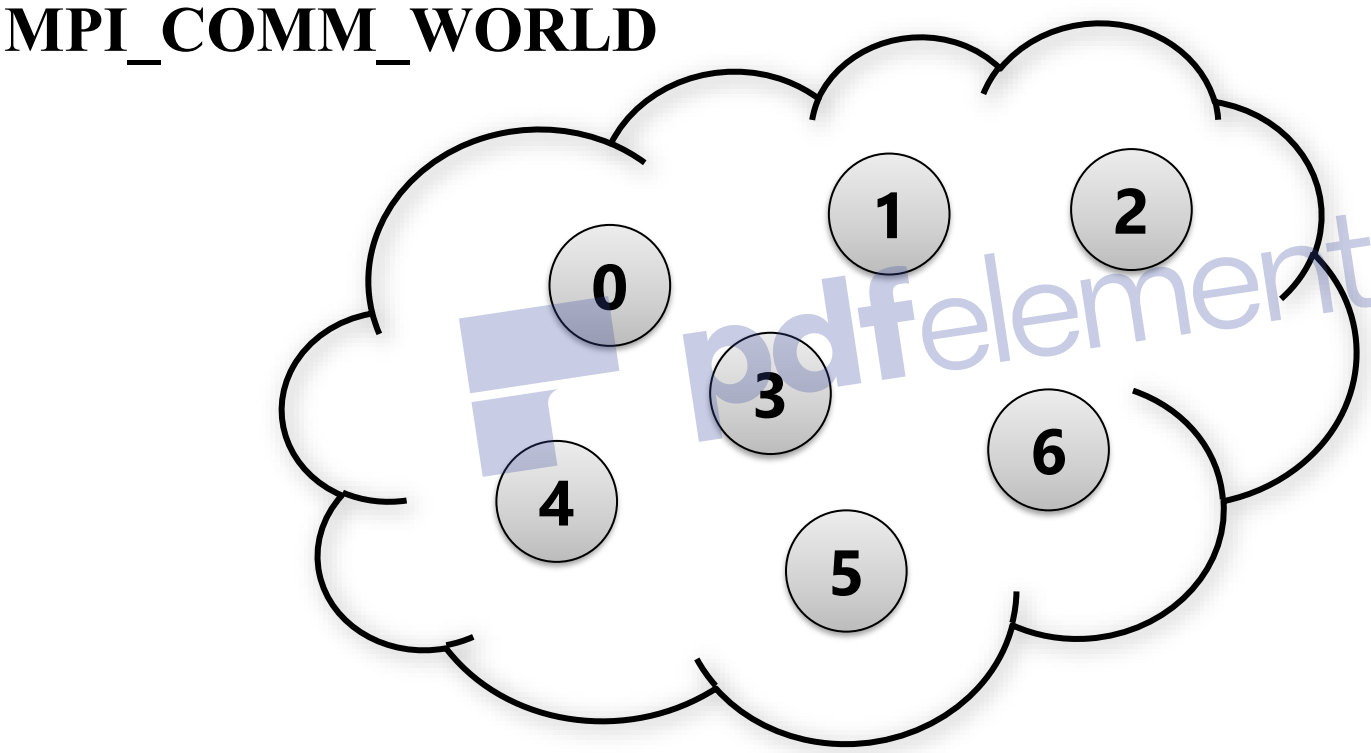
Elemente noi introduse de MPI

- **Tampon de comunicație** - definit de tripleta:
  - *(adresă, contor, tip_de_date)*
- **Context**
  - Fiecare comunicare de mesaj se derulează într-un *context*.
  - Mesajele sunt întotdeauna primite în contextul în care au fost transmise.
  - Mesajele transmise în contexte diferite nu interferă.
  - Mesajele sunt caracterizate de**:**
    - **context**
    - **tag**
- **Grup de procese**
  - Contextul este partajat de un *grup de procese*.
  - Fiecare proces are un rang (număr întreg).
  - Deci, fiecare proces e caracterizat de:
    - **grup**
    - **rang în grup**

pdfelement

Remove Watermark Now

Elemente noi introduse de MPI



pdfelement

Există un comunicator predefinit: ***MPI_COMM_WORLD***

Remove Watermark Now

Structura generală a unui program MPI

```
1   #include <mpi.h>
2
3   .
4   .
5   .
6   /* Initialize MPI Environment */
7   int MPI_Init(int *argc, char ***argv);
8   int MPI_Initialized(int *flag);
9   .
10  .
11  .
12  /* Do work and make message passing calls */
13  int MPI_Comm_rank(MPI_Comm comm, int *rank);
14  int MPI_Comm_size(MPI_Comm comm, int *size);
15  .
16  .
17  .
18  /* Terminate MPI Environment */
19  int MPI_Finalize();
```

pdfelement

Remove Watermark Now

Comunicarea punct la punct – operații de bază

- Forma clasică:
  - send (adresă, lungime, destinație, tag)
  - recv (adresă, lungime_maximă, sursă, tag, lungime_efectivă)
- În MPI:
  - MPI_SEND (buf, count, datatype, dest, tag, comm)
  - MPI_RECV (buf, count, datatype, source, tag, comm, status)
- Limbajul C:

```
int MPI_Send (void *buf, int count,
              MPI_Datatype datatype, int dest,
              int tag, MPI_Comm comm)
int MPI_Recv (void *buf, int count,
               MPI_Datatype datatype, int source,
               int tag, MPI_Comm comm, MPI_Status *status)
```

pdfelement

Remove Watermark Now

Exemplu

```
1  #include "mpi.h"
2
3  int main (int argc, char **argv)
4  {
5      char message[40];
6      int myrank;
7      MPI_Status status;
8
9      MPI_Init(&argc, &argv);
10     MPI_Comm_rank (MPI_COMM_WORLD, &myrank);
11     if (myrank == 0) {
12         strcpy(message, Hello, there);
13         MPI_Send(message, strlen(message), MPI_CHAR, 1, 99, MPI_COMM_WORLD);
14     } else {
15         MPI_Recv(message, 20, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
16         printf("\tReceived:%s \n", message);
17     }
18
19     MPI_Finalize();
20     return 0;
21 }
```

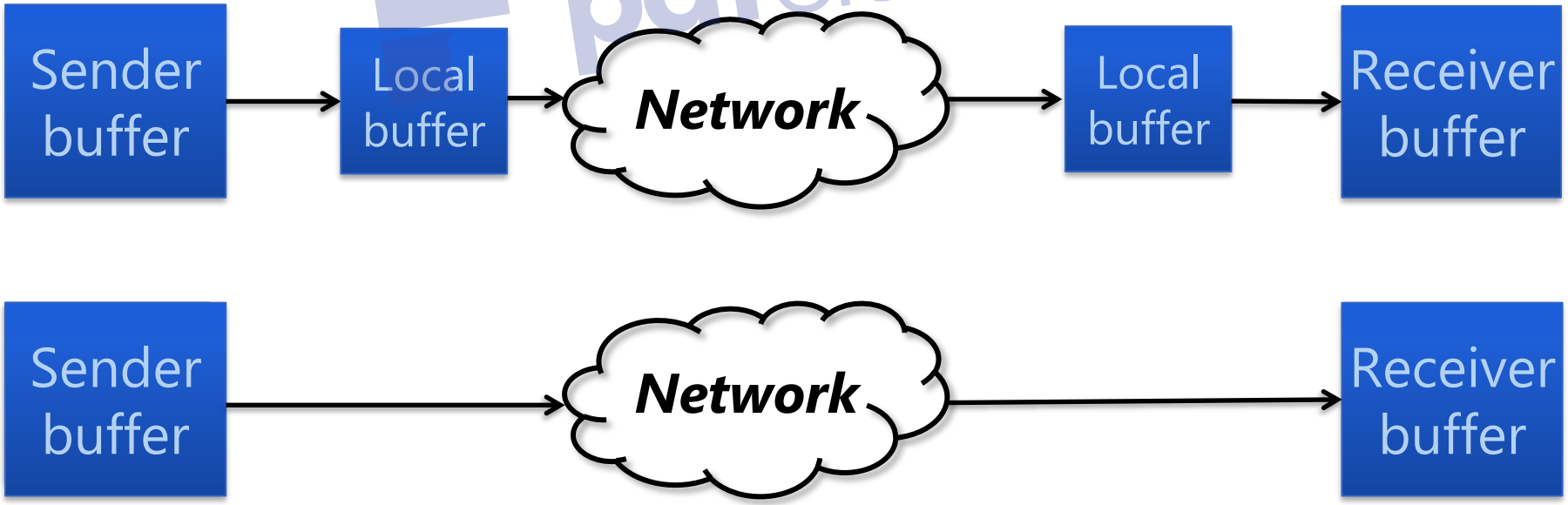
pdfelement

Remove Watermark Now

Moduri de comunicare

- Apelurile *send* și *receive* sunt **blocante**
- Moduri de comunicare:
  - cu buffer local
  - fără buffer local

pdfelement



Remove Watermark Now

Modurile de comunicaţie MPI

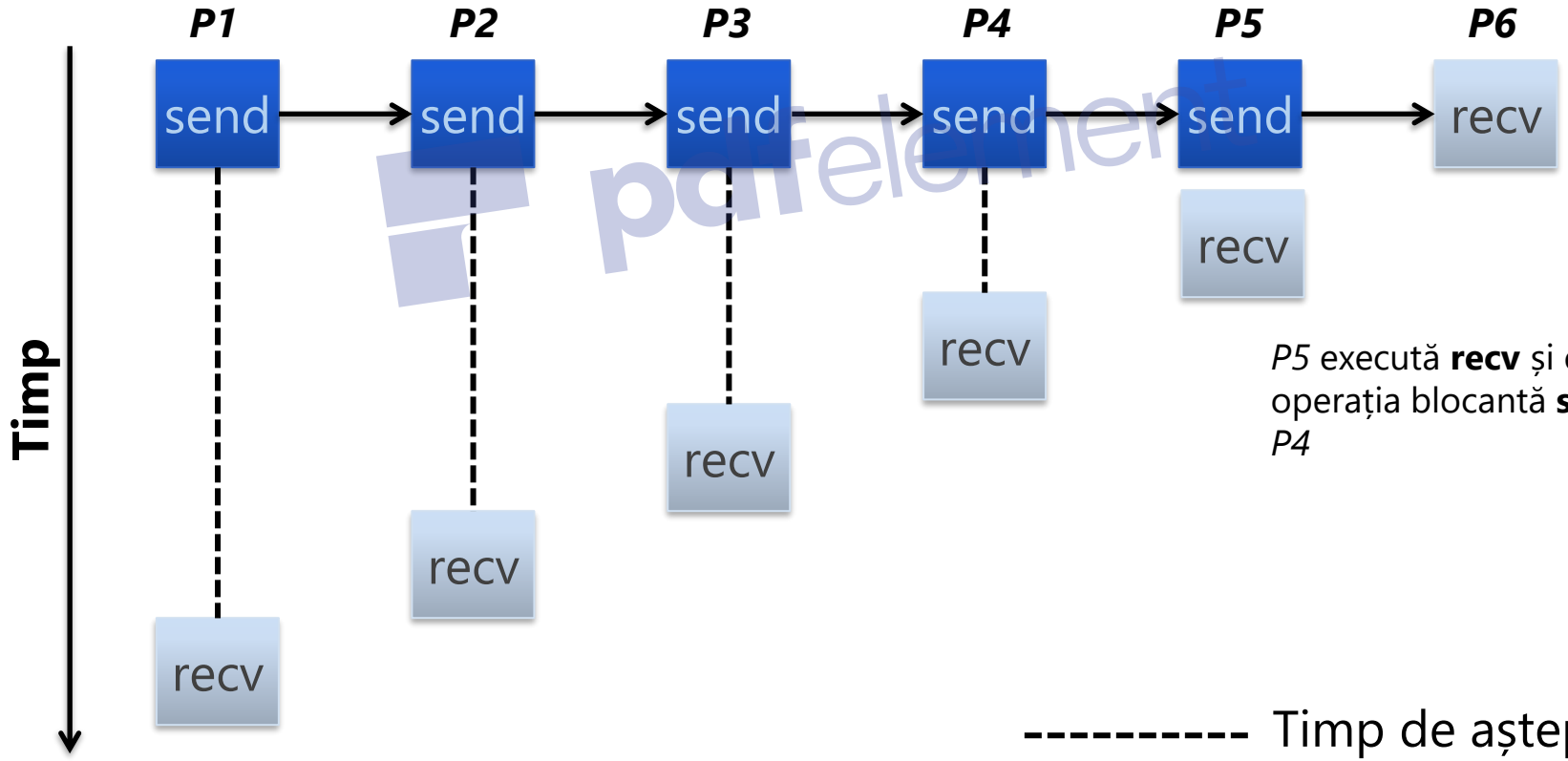
Tip	Condiția de terminare
Synchronous send	Finalizare atunci când operația de recepție a fost terminată cu succes.
Buffered send	Se termină întotdeauna (cu exceptia apariției unei erori), indiferent de operația de recepție.
Standard send	Trimiterea mesajului (fără a se cunoaște starea operatiei de recepție).
Ready send	Se termină întotdeauna (cu excepția apariției unei erori), însă folosirea e condiționată de apelul anterior la destinație a operației de recepție.

pdfelement

Remove Watermark Now

Probleme de transmitere în modul sincron

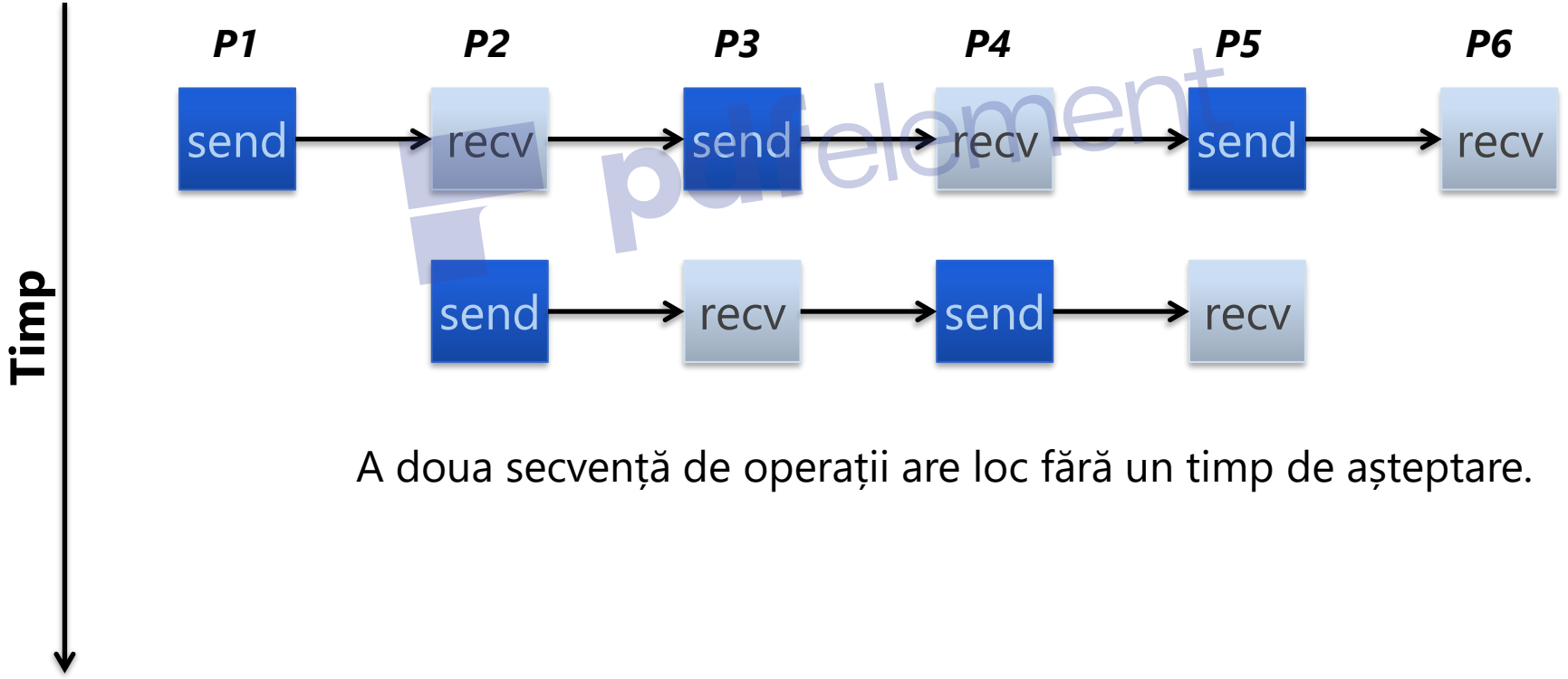
- Presupunem că:
  - avem 5 procese care execută o serie de *send* și *receive* + 1 proces care execută *receive*
  - inițial, cele 5 procese execută simultan *send* și un proces execută *receive*



Remove Watermark Now

Probleme de transmitere în modul sincron

- Soluție:
  - Ordonarea operațiilor *send* și *recv*



A doua secvență de operații are loc fără un timp de așteptare.

Remove Watermark Now

Operaţii *send-recv* combinate

```
1   MPI_Sendrecv
2
3   int MPI_Sendrecv(
4       void *sendbuf, int sendcount, MPI_Datatype sendtype,
5       int dest, int sendtag,
6       void *recvbuf, int recvcount, MPI_Datatype recvtype,
7       int source, int recvtag,
8       MPI_Comm comm,
9       MPI_Status *status )
10
11
12  MPI_Sendrecv_replace
13
14  int MPI_Sendrecv_replace(
15      void *buf, int count, MPI_Datatype datatype,
16      int dest, int sendtag, int source, int recvtag,
17      MPI_Comm comm,
18      MPI_Status *status )
```

pdfelement

Remove Watermark Now

Transmitere prin tampon alocat explicit

```
1   #define BuffSize 100000
2   int size;
3   char *buff;
4
5   buff = malloc(BuffSize); MPI_Buffer_attach(buff,
    BuffSize);
6   /* Tamponul de BuffSize octeți poate fi folosit acum */
7
8   MPI_Bsend( ... );
9   MPI_Buffer_detach(&buff, &size);
10  /* Tamponul redus la zero */
11
12  MPI_Buffer_attach(buff, size);
13  /* Tamponul disponibil din nou */
14
15
16  int MPI_Buffer_attach(void *buffer, int size );
17  int MPI_Buffer_detach(void *bufferptr, int *size );
```

pdfelement

Remove Watermark Now

Transmitere sincronă și în mod pregătit

- Transmitere sincronă
  - MPI_Ssend
- Transmitere în modul pregătit
  - MPI_Rsend

pdfelement

Remove Watermark Now

Comunicația non-blocantă

- ```
  int MPI_Isend(void *buf, int count,
          MPI_Datatype datatype, int dest, int tag,
          MPI_Comm comm, MPI_Request *request)
  ```
- ```
  int MPI_Irecv(void *buf, int count,
          MPI_Datatype datatype, int source, int tag,
          MPI_Comm comm, MPI_Request *request)
  ```

- MPI_Test
- MPI_Testall
- MPI_Testany
- MPI_Testsome
- MPI_Wait
- MPI_Waitall
- MPI_Waitany
- MPI_Waitsome

pdfelement

Exemplu

Remove Watermark Now

```
1  int rank, size, i, index, a[10], flag, f[6];
2  MPI_Request request, r[6];
3  MPI_Status status, s[6];
4  if (rank == 0) {
5      for (i=0; i<10; i++)
6          a[i] = i+1;
7      MPI_Isend(a, 10, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);
8      MPI_Test(&request, &flag, &status);
9      if (flag) {
10         printf("Procesul 0 ... operatia nu s-a terminat\n");
11     } else {
12         printf("Procesul 0 ... operatia s-a terminat\n");
13     }
14     sleep(10);
15     MPI_Wait(&request, &status);
16     printf("Procesul 0 ... operatia s-a terminat\n");
17 }
18 if (rank == 1) {
19     sleep(15);
20     a[0] = 255;
21     MPI_Irecv(a, 10, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &request);
22     MPI_Test(&request, &flag, &status);
23     if (flag) {
24       printf("Procesul 1 ... am primit date");
25     } else {
26       MPI_Wait(&request, &status);
27     }
28 }
```

pdfelement

Remove Watermark Now

Evitarea întârzierii nelimitate (1)

```
1   typedef struct {
2       char data [MSIZE];
3       int dsize;
4   } Buffer;
5
6   Buffer buf[];
7   MPI_Request req[];
8   MPI_Status status[];
9   int index[];
10  ...
11  MPI_Comm_rank(comm, &rank);
12  MPI_Comm_size(comm, &size);
13
14  if(rank != size-1) {
15      /* client code */
16      buf = (Buffer *) malloc(sizeof(Buffer));
17      while(1) {
18          produce_request( buf->data, &buf->dsize);
19          MPI_Send(buf->data,buf->dsize,MPI_CHAR,size-1,tag,comm)
20      }
21  }
```

pdfelement

Remove Watermark Now

Evitarea întârzierii nelimitate (2)

```
22  else {
23
24      buf = malloc(sizeof(Buffer)*(size-1));
25      req = malloc(sizeof(MPI_Request)*(size-1));
26      status = malloc(sizeof(MPI_Status)*(size-1));
27      index = malloc(sizeof(int)*(size-1));
28
29      for(i=0; i < size-1; i++)
30          MPI_Irecv(buf[i].data,MSIZE,MPI_CHAR,i,tag,comm,&req[i]);
31      while (1){
32          MPI_Waitsome(size-1, req, &count, index, status);
33          for (i=0; i < count; i++){
34              j = index[i];
35              MPI_Get_count(&status[i], MPI_CHAR, &buf[j].dsize);
36              process_request(buf[j].data, buf[j].dsize);
37              MPI_Irecv(buf[j].data,MSIZE,MPI_CHAR,j,tag,comm,&req[j])
38          }
39      }
40  }
```

pdfelement

Remove Watermark Now

Cereri de comunicare persistentă

- De multe ori, o comunicație cu *aceeași listă de argumente* este repetată în cadrul unei bucle.
- În astfel de situații, este posibilă optimizarea comunicației prin prin *legarea* listei de argumente la o cerere de comunicare persistentă.
- Cererea de comunicare persistentă reduce timpul de comunicare dintre proces și *controller*-ul de comunicare, nu și timpul de comunicare scurs pe rețeaua externă.
- Nu este necesar ca mesajele trimise prin intermediul cererilor persistente să fie recepționate în mod persistent (sau viceversa).

pdfelement

Remove Watermark Now

Cereri de comunicare persistentă

- O cerere de comunicare persistentă este inactivă după ce a fost creată.
- O comunicare (operație *send* sau *receive*) care folosește o cerere persistentă trebuie inițiată (cu ajutorul funcției *MPI_Start*).
- Astfel, programatorul poate să:
  - creeze o cerere persistentă – *MPI_Send_init*, *MPI_recv_init*
  - declanșeze o operație – *MPI_Start*, *MPI_Startall*
  - aștepte terminarea unei operații – *MPI_Wait*
  - elimine înregistrarea cererii persistente – *MPI_Request_free*

pdfelement

Remove Watermark Now

Cereri de comunicare persistentă

- Sintaxă:
  - `int MPI_Start(MPI_Request *request)`
  - `int MPI_Wait(MPI_Request *request, MPI_Status *status)`
  - `int MPI_Request_free(MPI_Request *request )`
- Exemplu:

```
MPI_request *request;
MPI_Recv_init(buf, count, datatype,
              source, tag, comm, request);

/* MPI_Start are semantica MPI_Irecv */
MPI_Start(request);
MPI_Wait(request, status);
MPI_Request_free(request);
```

pdfelement

Remove Watermark Now

Tipuri de date predefinite în MPI

- **Tipuri predefinite**
  - toate tipurile tipurile de bază din C *(și din FORTRAN)* plus *MPI_BYTE* și *MPI_PACKED*.

Tip MPI	Tip echivalent în C
MPI_CHAR MPI_SHORT MPI_INT	signed char signed short int signed int
MPI_LONG MPI_UNSIGNED_CHAR MPI_UNSIGNED_SHORT	signed long int unsigned char unsigned short int
MPI_LONG MPI_UNSIGNED_CHAR MPI_UNSIGNED_SHORT	unsigned int unsigned long int unsigned short int
MPI_DOUBLE MPI_LONG_DOUBLE	double long double

pdfelement

Remove Watermark Now

Tipuri de date derivate în MPI

- Motivație: realizarea unei modalități eficiente de a comunica tipuri *non-contigue* sau *diferite* într-un mesaj
- Formal, standardul MPI definește tipul de date general alcătuit din:
  - o secvență de tipuri de bază
  - o secvență de deplasări (deplasările au o valoare întreagă)
- Harta unui tip (*typemap*) este folosită pentru o exprimare facilă:
  - Harta tipului = secvență de perechi (tip, deplasare)
  - $\boldsymbol{Typemap} = \{(\boldsymbol{type_0}, \boldsymbol{displacement_0}), \ \ldots, (\boldsymbol{type_{n-1}}, \boldsymbol{displacement_{n-1}})\}$
- Semnătura tipului (*typesig*) = secvența tipurilor
  - $\boldsymbol{Typesig} = \{\boldsymbol{type_0}, \boldsymbol{type_1}, \ldots, \boldsymbol{type_{n-1}}\}$
- Titlul tipului (*handle*)
  - Exemplu: MPI_INT, cu harta $Typemap = \{(int, 0)\}$

pdfelement

Remove Watermark Now

Tipuri de date derivate în MPI

- Etapele necesare pentru a utiliza un tip derivat:
  - construirea tipului (utilizând o serie de constructori speciali)
  - alocarea sa (înregistrarea folosind *MPI_Type_commit*)
  - la final, tipul va fi dezalocat (*MPI_Type_free*)

pdfelement

- Exemple de constructori:
  - **MPI_Type_contiguous** (count, oldtype, *newtype)
  - **MPI_Type_vector** (count, blocklength, stride, oldtype, *newtype)
  - **MPI_Type_indexed** (count, blocklens[], offsets[], old_type, *newtype)
  - **MPI_Type_struct** (count, blocklens[], offsets[], old_types[], *newtype)

Remove Watermark Now

Tipuri de date derivate în MPI - constructori

- **MPI_Type_contiguous** (count, oldtype, *newtype)
  - Se produce un nou tip de date prin copierea *oldtype* de *count* ori

- Exemplu:

pdfelement

```
count = 4;
MPI_Type_contiguous(count, MPI_FLOAT, &rowtype);

MPI_Send(&a[2][0], 1, row_type, dest, tag, comm);
```

Remove Watermark Now

Tipuri de date derivate în MPI - constructori

- **MPI_Type_vector** (count, blocklength, stride, oldtype, *newtype)
  - Se vor lua *count* blocuri de lungime *blocklength*
  - Un element din bloc este de tipul *oldtype*
  - Blocurile pot fi despărțite de zone care vor fi sărite (*strides*)

- Exemplu:

```
float mesh[10][20];
int dest, tag;
int count = 10;                 /* numărul de blocuri */
int blocklen = 1;               /* o singură coloană */
int stride = 19;                /* salt de 20 de elemente */

MPI_Datatype newtype;
MPI_Type_vector(count,blocklen,stride,MPI_FLOAT,&newtype);
MPI_Type_commit(&newtype);
MPI_Send(&mesh[0][19], 1, newtype, dest, tag, MPI_COMM_WORLD);
```

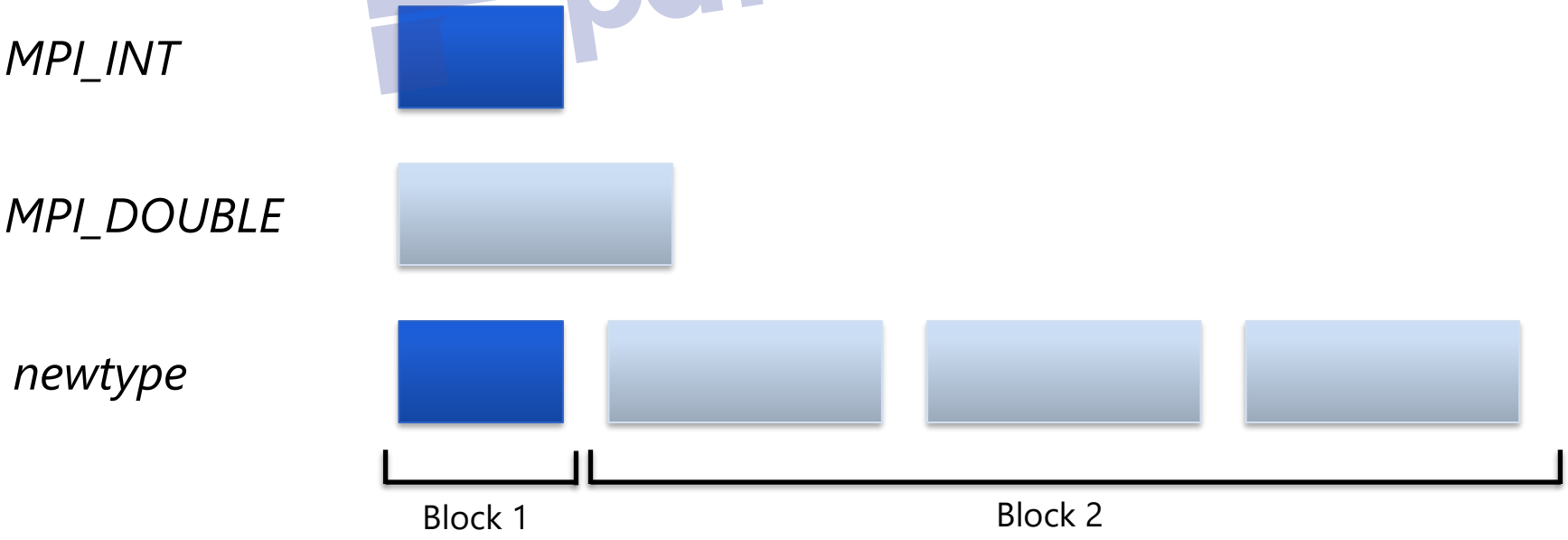
pdfelement

Remove Watermark Now

Tipuri de date derivate în MPI - construcție

- **MPI_Type_struct** (count, blocklens, offsets, oldtypes, *newtypes)
  - Se vor lua *count* blocuri de lungime *blocklength*
  - Un element din bloc este de tipul *oldtype*
  - Fiecare bloc se găsește la deplasamentul corespunzător din *offsets*
- Exemplu: un apel **MPI_Type_struct** cu *count*=2, *blocklens[0]*=1, *old_types[0]*=MPI_INT, *blocklens[1]*=3 şi *old_types[1]*=MPI_DOUBLE

pdfelement



Remove Watermark Now

MPI_type_struct - exemplu

```
1   # define MAX_PART 1000
2   Struct Part_struct
3     { int class;       /* clasa particulei */
4       double d[6];     /* coordonatele particulei */
5       char b[7];       /* alte informații */
6     };
7   struct Part_struct particle[MAX_PART];
8   /* construcția tipului care descrie structura MPI a unei particule */
9   MPI_Datatype PartType;
10  MPI_Datatype type[3] = {MPI_INT, MPI_DOUBLE, MPI_CHAR}; /* semnătura */
11  int blocklen [3] = { 1, 6, 7};
12  MPI_Aint disp[3];
13  int base;
```

pdfelement

Remove Watermark Now

MPI_type_struct - exemplu

```
14  /* se determină adresele absolute pentru a calcula ulterior deplasările
15   * MPI_Address este folosit pentru portabilitate
16   */
17  MPI_Address (particle, disp);
18  MPI_Address (particle[0].d, disp+1);
19  MPI_Address (particle[0].b, disp+2);
20
21  base=disp[0];
22  /* deplasările reprezintă adresele relative */
23  for (i=0; i<3; i++) disp [i] -= base;
24  MPI_Type_create_struct ( 3, blocklen, disp, type, &PartType);
25  MPI_Type_commit (&PartType);
26
27  /* se transmite întregul tablou */
28  MPI_Send (particle, MAX_PART, PartType, dest, tag, comm);
```

pdfelement

Remove Watermark Now

Topologii virtuale

- O topologie este un mecanism prin care se obține o modalitate facilă de a *numi* procesele dintr-un grup (comunicator).
- În cazul multor aplicații distribuite, simpla folosire a rangurilor nu reflectă modelul logic de comunicare între procese.
- Procesele pot fi văzute ca făcând parte dintr-un:
  - grid sau structură carteziană (două/trei/... dimensiuni)
  - graf (model generic)
- Trebuie făcută distincția între:
  - topologia virtuală
  - topologia hardware-ului
- Topologia virtuală poate fi concepută astfel încât să respecte modul în care procesele sunt repartizate procesoarelor (topologia hardware), pentru o sporire a performanței comunicării dintre procese.

pdfelement

Remove Watermark Now

Topologii virtuale

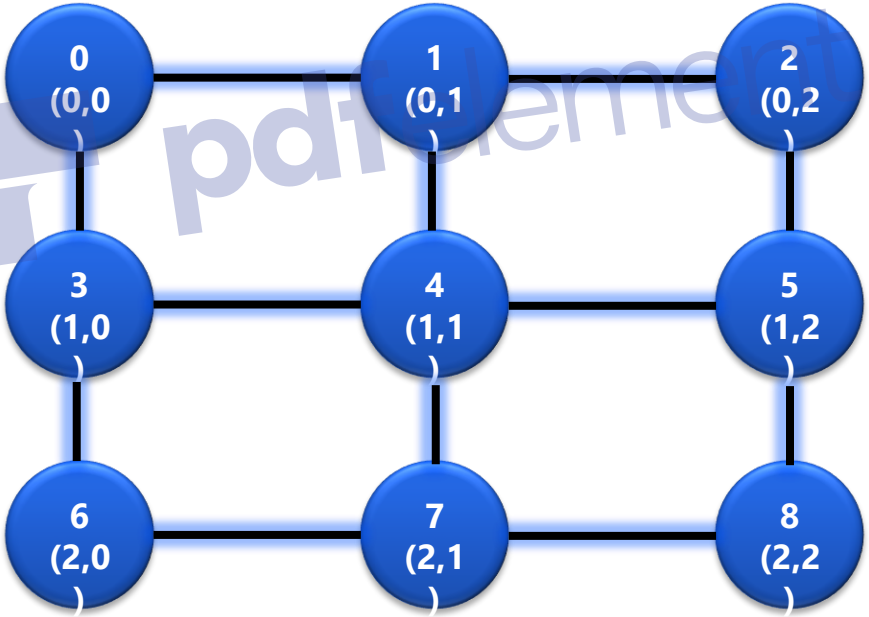
- `MPI_Cart_create (MPI_Comm comm_old, int ndims, int *dims, int *periods, int reorder, MPI_Comm comm_cart)`
- Funcția creează un nou comunicator în care topologia este una carteziană, conform parametrilor (procesele implicate sunt cele din *comm_old*):
  - *ndims* – numărul de dimensiuni din structura carteziană
  - *dims* – un vector de dimensiune *ndims* care specifică numărul de procese de pe fiecare dimensiune
  - *periods* – un vector cu valori de tip boolean care indică prezența/absența periodicității pe o anumită dimensiune
  - *reorder* – valoare booleană care permite (sau nu) MPI-ului să reordoneze procesele din noul comunicator în funcție de proximitățile fizice ale procesoarelor pe care rulează procesele (topologia hardware)

pdfelement

Remove Watermark Now

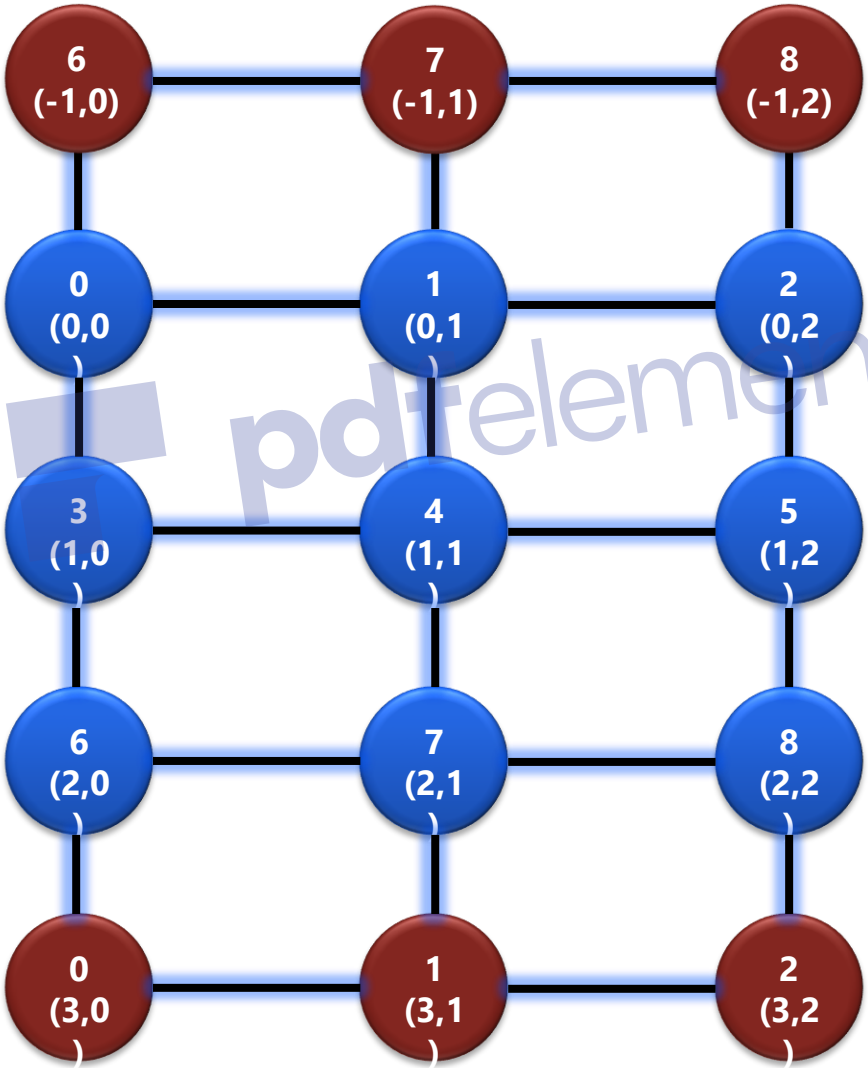
Exemple de grile carteziene 2D
Grilă neperiodică

Rang
(linie,coloană)



Remove Watermark Now

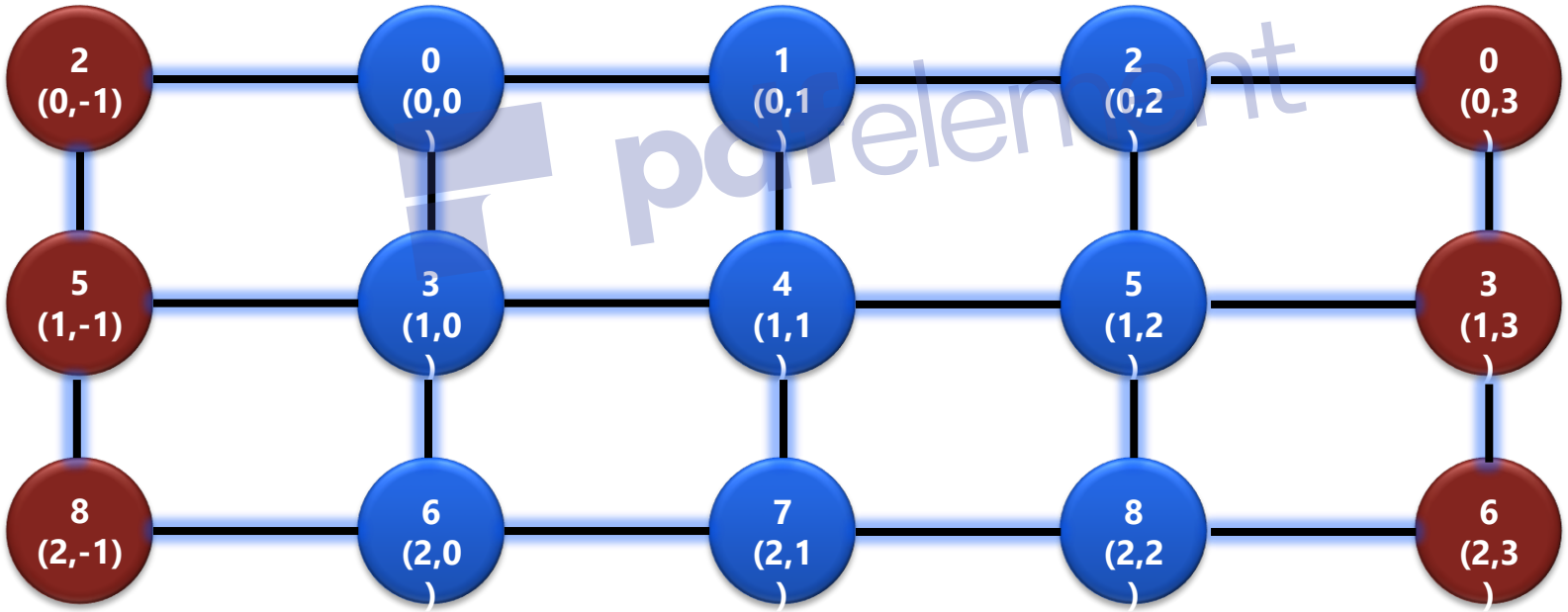
Exemple de grile carteziene 2D
Grilă periodică pe linii



pdfelement

Remove Watermark Now

Exemple de grile carteziene 2D
Grilă periodică pe coloane



Remove Watermark Now

Primitive pentru topologii carteziene

- Furnizare *dims*, *periods* și *coords* pentru procesul apelant:

```
int MPI_Cart_get(MPI_Comm comm, int maxdims,
                 int *dims, int *periods,
                 int *coords)
```

- Aflare *coords* pentru un proces cu rang cunoscut:

```
int MPI_Cart_coords(MPI_Comm comm, int rank,
                    int maxdims, int *coords)
```

- Pentru a determina sursa și destinația care vor fi folosite într-un apel ulterior *MPI_Sendrecv* (sursa și destinația pe aceeași direcție):

```
int MPI_Cart_shift(MPI_Comm comm, int direction,
                   int displ, int *source, int *dest)
```

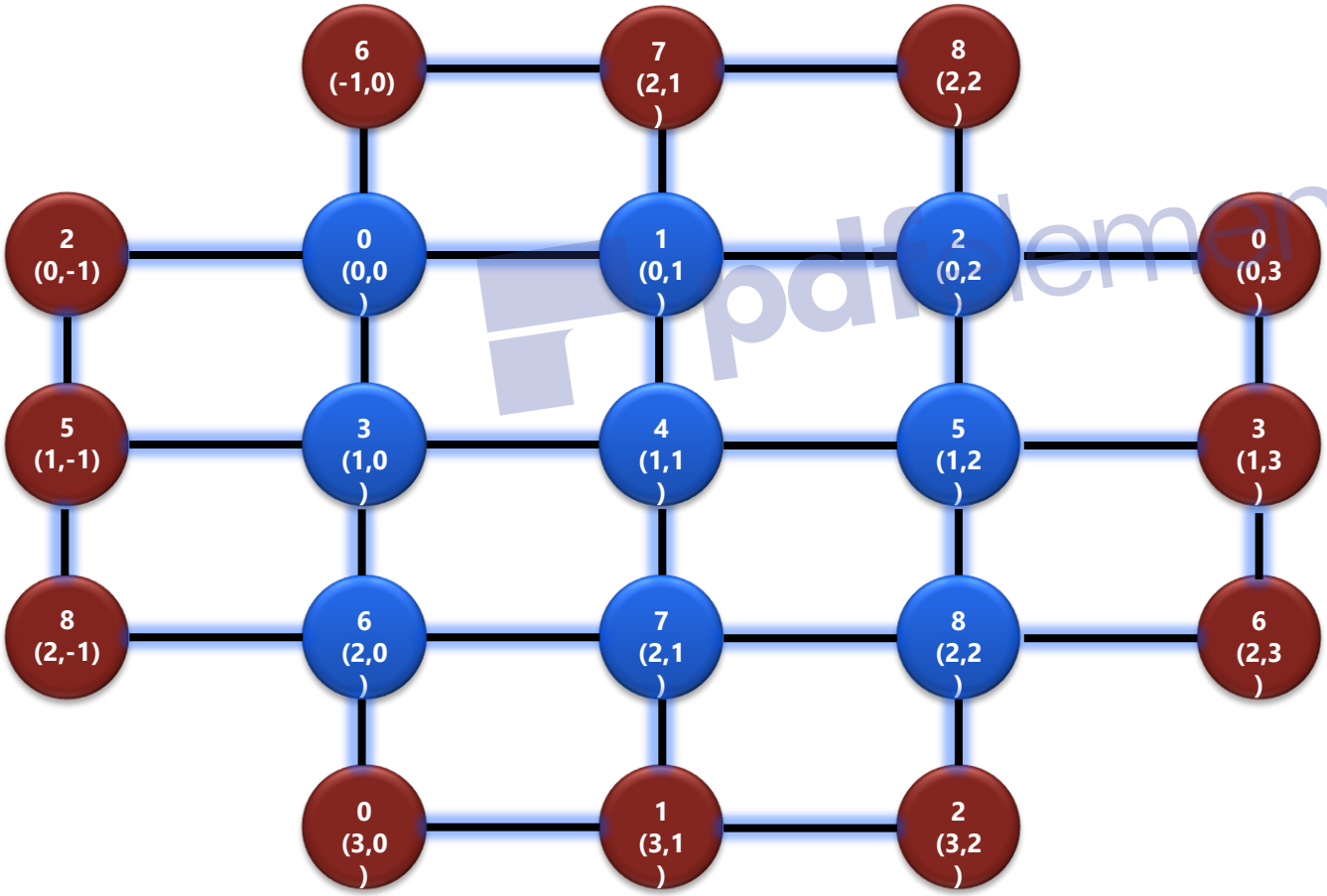
Obs: sursa = rang proces apelant – deplasament

destinația = rang proces apelant + deplasament

pdfelement

Remove Watermark Now

Grilă periodică pe toate dimensiunile (tor)
MPI_Cart_shift



Proces apelant: **5**
Direcție: 1 (linia comună)
Deplasament: 2

Sursa: **3**
Destinația: **4**

Remove Watermark Now

Exemplu (1)

```
1  int numtasks, rank, newrank, i, j;
2  float a[SIZE][SIZE] = {1.0, 2.0, 3.0, 4.0}, { 5.0, 6.0, 7.0, 8.0 },
3                        { 9.0, 10.0, 11.0, 12.0 }, { 13.0, 14.0, 15.0, 16.0} };
4  float t;
5  MPI_Status stat;
6  MPI_Request request;
7  MPI_Comm newcomm;
8  int ndims, reorder, dim_size[2], periods[2];
9  int coords[2], source, dest;
10 MPI_Init(&argc,&argv);
11 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
12 MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
13 ndims = 2; /* numărul de dimensiuni */
14 dim_size[0] = 4; /* numărul de procese pe prima dimensiune */
15 dim_size[1] = 4; /* numărul de procese pe cea de-a doua dimensiune */
16 periods[0] = 1; /* dimensiune ciclică */
17 periods[1] = 0;
18 reorder = 1;
19 MPI_Cart_create(MPI_COMM_WORLD, ndims, dim_size, periods, reorder, &newcomm);
20 MPI_Comm_rank(newcomm, &newrank);
21 MPI_Cart_coords(newcomm, newrank, 2, coords);
22 printf("Rang=%d Pozitia=(%d,%d)\n", newrank, coords[0], coords[1]);
23 MPI_Barrier(MPI_COMM_WORLD);
24 MPI_Cart_shift(newcomm, 0, 1, &source, &dest);
25 printf("Procesul %d: trimit catre %d si primesc de la %d\n", newrank, dest, source);
```

pdfelement

Remove Watermark Now

Exemplu (2)

```
26  if (coords[0] % 2 == 0) {
27      MPI_Send(&a[coords[0]][coords[1]], 1, MPI_FLOAT, dest, 0, newcomm);
28      MPI_Recv(&t, 1, MPI_FLOAT, source, 0, newcomm, &stat);
29  } else {
30      MPI_Recv(&t, 1, MPI_FLOAT, source, 0, newcomm, &stat);
31      MPI_Send(&a[coords[0]][coords[1]], 1, MPI_FLOAT, dest, 0, newcomm);
32  }
33  /* se pune noul element pe poziţia corespunzătoare */
34  a[coords[0]][coords[1]] = t;
35  MPI_Barrier(MPI_COMM_WORLD);
36  if (newrank == 0) {
37      for (i=0; i<4; i++) {
38          for (j=0; j<4; j++) {
39              coords[0] = i; coords[1] = j;
40              MPI_Cart_rank(newcomm, coords, &rank);
41              if (rank != 0) {
42                  MPI_Recv(&a[j][i], 1, MPI_FLOAT, rank, 0, newcomm, &stat);
43              }
44              printf("%.2f ", a[j][i]);
45          }
46          printf("\n");
47      }
48  } else {
49      MPI_Send(&a[coords[0]][coords[1]], 1, MPI_FLOAT, 0, 0, newcomm);
50  }
51  MPI_Finalize();
```

pdfelement

Remove Watermark Now

Comunicare colectivă

- Comunicarea colectivă se referă la acele funcții din MPI care implică **toate procesele** din comunicator.
- Caracteristici:
  - O operație colectivă presupune apelul funcției corespunzătoare de către toate procesele din comunicator, folosind argumente potrivite.
  - Dimensiunea datelor trimise trebuie să **coincidă** cu dimensiunea datelor primite.
  - Operațiile colective se pot executa doar în **modul blocant**.
- Operațiile de comunicare colectivă se împart în:
  - Operații de sincronizare
  - Operații de transfer de date
  - Calcule colective (de reducere)

pdfelement

Remove Watermark Now

Operații de sincronizare
Sincronizarea cu barieră

- `int MPI_Barrier(MPI_Comm comm)`

- MPI_Barrier sincronizează toate procesele din comunicator
- Un proces apelant se blochează până când toate celelalte procese din comunicator apelează, la rândul lor, această funcție

pdfelement

Operații de transfer de date

Remove Watermark Now

- Operații de transfer de date:
  - **Difuzarea** (*broadcast*)
    - propagarea unui mesaj de la *rădăcină* la restul proceselor
  - **Colectarea** (*gather*)
    - procesul *rădăcină* colectează date de la celelalte procese
  - **Distribuirea** (*scatter*)
    - inversa operației de colectare
  - Variații ale colectării și distribuirii (*allgather*, *alltoall*)
    - *allgather*
      - fiecare proces colectează datele de la celelalte procese
    - *alltoall*
      - fiecare proces realizează o distribuire a datelor, în funcție de rangul celorlalte procese

pdfelement

Remove Watermark Now

Operații de transfer de date

Difuzare

P_0	A			
P_1	A			
P_2	A			
P_3	A			

Distribuire

P_0	A	B	C	D
P_1	B			
P_2	C			
P_3	D			

Colectare

P_0	A	B	C	D
P_1	B			
P_2	C			
P_3	D			

pdfelement

Remove Watermark Now

Operații de transfer de date

Allgather

P_0	A	B	C	D
P_1	B	A	C	D
P_2	C	A	B	D
P_3	D	A	B	C

Alltoall

P_0	A_0	B_0	C_0	D_0
P_1	A_1	B_1	C_1	D_1
P_2	A_2	B_2	C_2	D_2
P_3	A_3	B_3	C_3	D_3

pdfelement

Remove Watermark Now

Calcule colective (operații de reducere)

- `int MPI_Reduce(void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)`

P_0 *sendbuf*	A_0	A_1	A_2	A_3
P_1 *sendbuf*	B_0	B_1	B_2	B_3
P_2 *sendbuf*	C_0	C_1	C_2	C_3
P_3 *sendbuf*	D_0	D_1	D_2	D_3

MPI_Op = MPI_SUM

pdfelement

Rezultat MPI_Reduce: P_0 *recvbuf* va conține suma celor 4 vectori

P_0	$A_0 + B_0 + C_0 + D_0$	$A_1 + B_1 + C_1 + D_1$	$A_2 + B_2 + C_2 + D_2$	$A_3 + B_3 + C_3 + D_3$
P_1				
P_2				
P_3				

Remove Watermark Now

Calcule colective (operații de reducere)
Exemplu – aproximare valoare PI

- $\int_0^1 \frac{1}{1+x^2}\,dx = \frac{\pi}{4}$
- Suma Riemann: $\sum_{k=1}^{n} f(c_k)\,(x_k - x_{k-1})$

pdfelement

- Soluție secvențială:

```
for (i = 1; i <= n; i++) {
   x = h * ((double)(i) - 0.5);
   sum = sum + f(x);
}
pi = h * sum;
```

Remove Watermark Now

Calcule colective (operații de reducere)
Exemplu – aproximare valoare PI

```
1  MPI_Init(&argc,&argv);
2  MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
3  MPI_Comm_rank(MPI_COMM_WORLD,&myid);
4  while (1) {
5      if (myid == 0) {
6          printf("Enter the number of intervals: (0 quits) ");
7          scanf("%d",&n);
8      }
9      MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
10     if (n == 0) {
11         break;
12     } else {
13         h   = 1.0 / (double) n;
14         sum = 0.0;
15         for (i = myid + 1; i <= n; i += numprocs) {
16             x = h * ((double)i - 0.5);
17             sum += (4.0 / (1.0 + x*x));
18         }
19         mypi = h * sum;
20         MPI_Reduce(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
21         if (myid == 0) {
22                 printf("pi is approximately %.16f\n", pi);
23         }
24     }
25 }
```

pdfelement

Remove Watermark Now

Sumar

- *Message Passing Interface*
  - Elemente introduse
  - Comunicarea punct – la – punct
  - Tipuri de date
  - Topologii de procese
  - Comunicare colectivă

pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Toleranta la defecte.

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Obiectiv

- Asigurarea tolerantei la defecte
  - Solutii de detectie a erorilor in SD
  - Solutii de recuperare din defect in SD

pdfelement

"You know you have one when the crash of a computer you've never heard of stops you from getting any work done."

Leslie Lamport, 1993 – defectul in SD

Remove Watermark Now

Introducere

- Un sistem distribuit (SD) se defineste ca avand 6 caracteristici importante [Coulouris, et al, 1994]
  - Suportul pentru partajarea resurselor
  - Deschidere
  - Concurenta
  - Scalabilitate
  - Toleranta la defecte
  - Transparenta
- **Problema:** funcţionarea corectă în prezenţa unor defecţiuni ale sistemului
- Sistemele distribuite sunt compuse din multe componente hardware si software => probabilitate mai mare de aparitie a unor defecte in diverse parti

pdfelement

Remove Watermark Now

Eroare?

An error has occurred.

We apologize for this inconvenience.

RegOnline has been notified and will work to fix this problem

Please try the following:

- Click the back button and try again.
- Close your browser and try again.

pdfelement

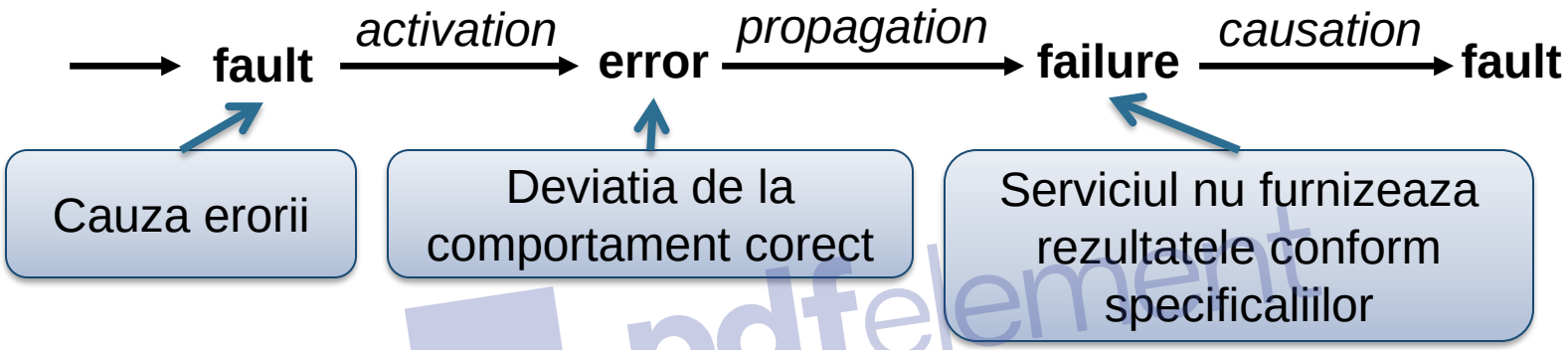
Could you help us eliminate this error?...

Please click here to describe what you were doing when the error occurred.

Remove Watermark Now

Toleranta la defecte

- Problemele: greseli de functionare, erori, defecte

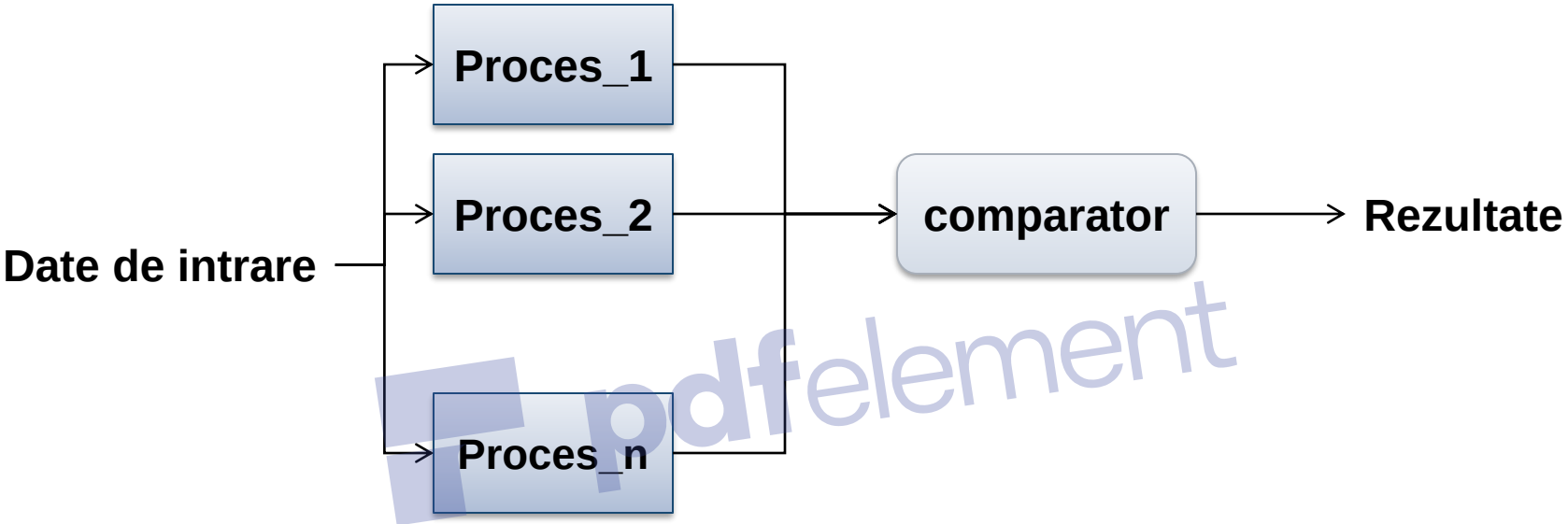


pdfelement

- **Toleranta la defecte**:
  - Evitarea greselilor in functionare chiar in prezenta defectelor .
  - Utillizatorii nu resimt defectarea unor componente.
- Doua abordari :
  - Redundanta Hardware
  - Recuperarea Software

Remove Watermark Now

Toleranta la defecte



pdfelement

- Aceeaşi ieşire pentru aceleaşi valori de intrare,
- Valori apropiate la ieşire, pentru valori de intrare apropiate

Remove Watermark Now

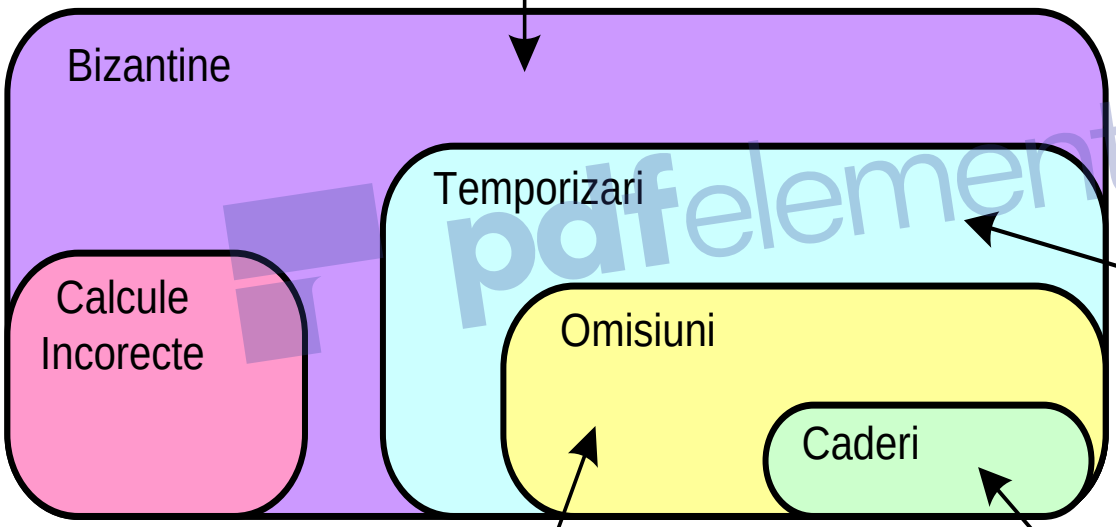
Tipuri de defecte în sistemele distribuite

- Crash: la un moment dat un procesor devine nefuncţional
  - În ultimul pas, înainte de a se defecta, se poate să fi trimis numai un subset din mesaje
- Byzantine: procesorul intră în stări arbitrare şi trimite mesaje conţinând conţinut arbitrar

pdfelement

Remove Watermark Now

Clasificarea defectelor si anomaliilor



pdfelement

Remove Watermark Now

Toleranta la defecte in sistemele distribuite

- Exemple:

COMPONENTA	DEFECT
Unitate de procesare (CPU)	Caderi sau defecte bizantine
Reteaua de comunicatie	Toate tipurile de defecte
Ceasurile	Temporizari, omisiuni, defecte bizantine
Unitate de stocare (disc)	Temporizari, omisiuni, caderi
Software	Software Toate tipurile de defecte

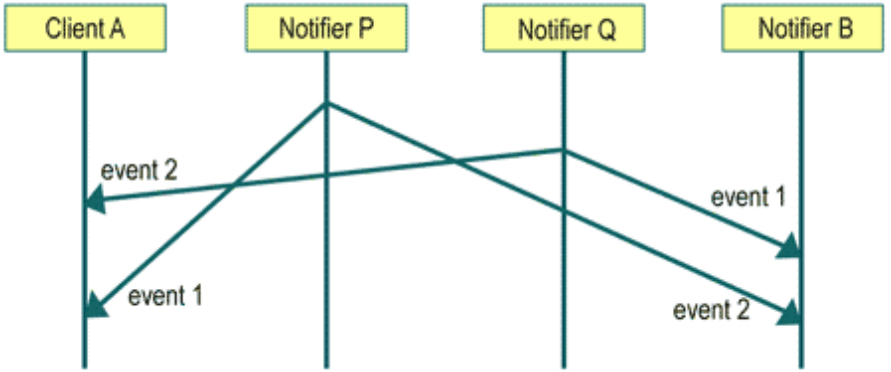
pdfelement

- Din perspectiva tolerantei la defecte, sistemele distribuite au un avantaj major: *se poate introduce usor o redundanta in sistem* deoarece imperfectiunile si posibilitatea unor defecte fizice nu pot fi ignorate.
- Probleme:
  - Defecte la nivelul nodurilor din sistemul distribuit.
  - Defecte la nivelul procesului de comunicatie.
  - Intarziere in procesul de transmisie al informatiei.

Remove Watermark Now

Toleranta la defecte in sistemele distribuite (2)

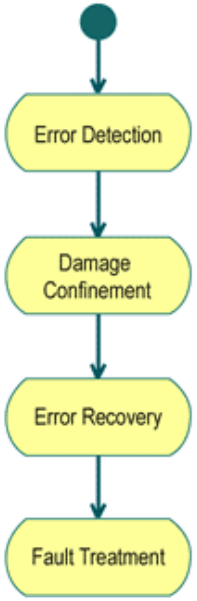
- Defecte la nivelul nodurilor din sistemul distribuit.
  - Defecte independente si diferite pe fiecare nod.
- Defecte la nivelul procesului de comunicatie.
  - *Defectiune hardware* a mediului de transmisie -> oprirea comunicatiei, partitionarea sistemului in mai multe sisteme.
  - *Intermittent failure*: mesajele pierdute, schimbate ordinea sau duplicate.
- Intarziere in procesul de transmisie al informatiei.
  - Nu este un defect, dar poate conduce la defecte.
  - Doua situatii frecvent intalnite:
    - *Jitter (variable delay)* -> timpul in care un mesaj ajunge la destinatie.
    - *Efectul relativistic* -> site-uri diferite pot primi aceleasi mesaje, dar in alta ordine.



pdfelement

Remove Watermark Now

Fazele tolerantei la defecte

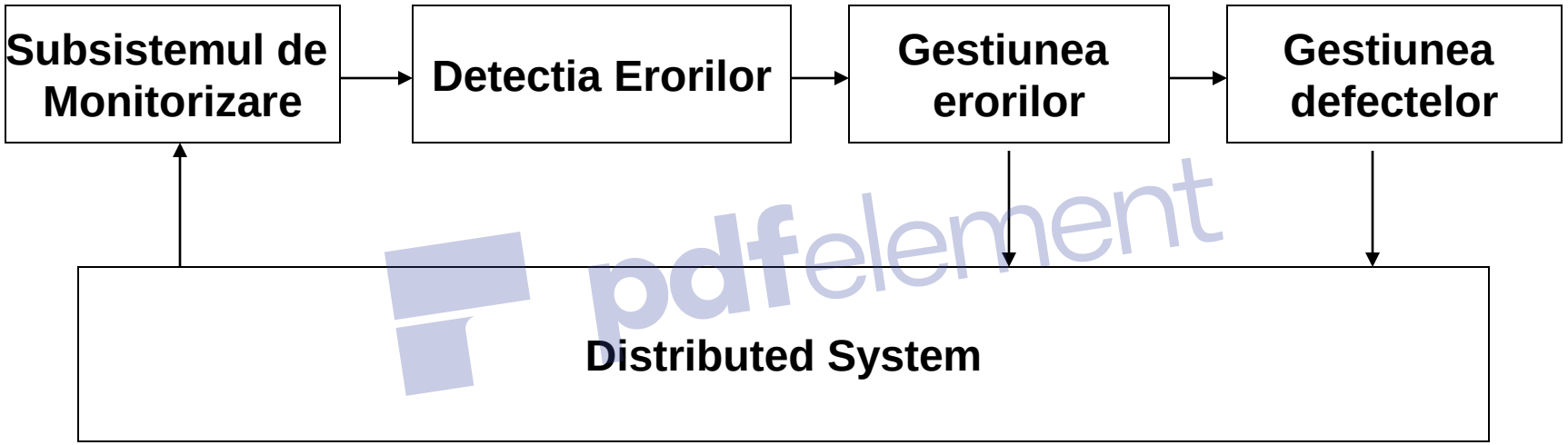


- Proiectarea si implementarea unui sistem distribuit tolerant la defecte este dependenta de arhitectura sistemului si de functionalitatea sa. Prin urmare nu exista reguli generale pentru a adauga caracteristici de toleranta la defecte intr-un sistem, insa putem formula unele principii care trebuie sa urmeze patru faze:
  - Detectia erorilor;
  - Estimarea si izolarea zonelor afectate;
  - Recuperarea erorilor;
  - Tratarea defectelor.

pdfelement

Remove Watermark Now

Arhitectura generica de FT



pdfelement

Remove Watermark Now

Detectia erorilor

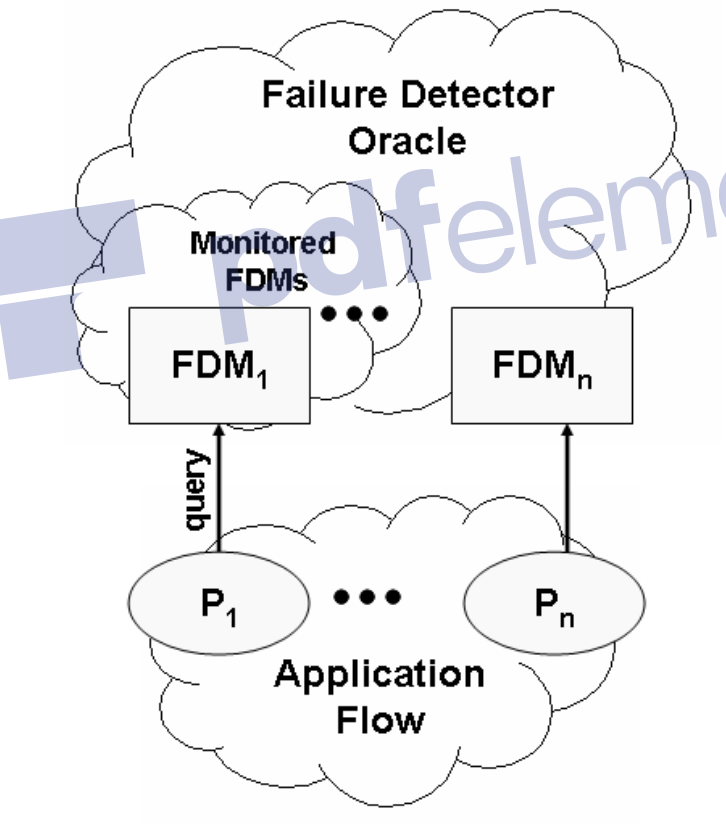
- Explozia de mesaje
- Scalabilitate
- Pierderi de mesaje
- Flexibilitate
- Dinamism
- Securitate

pdfelement

Remove Watermark Now

Detectoare

- Nivelul de FD este in general asigurat de detectoare locale dedicate

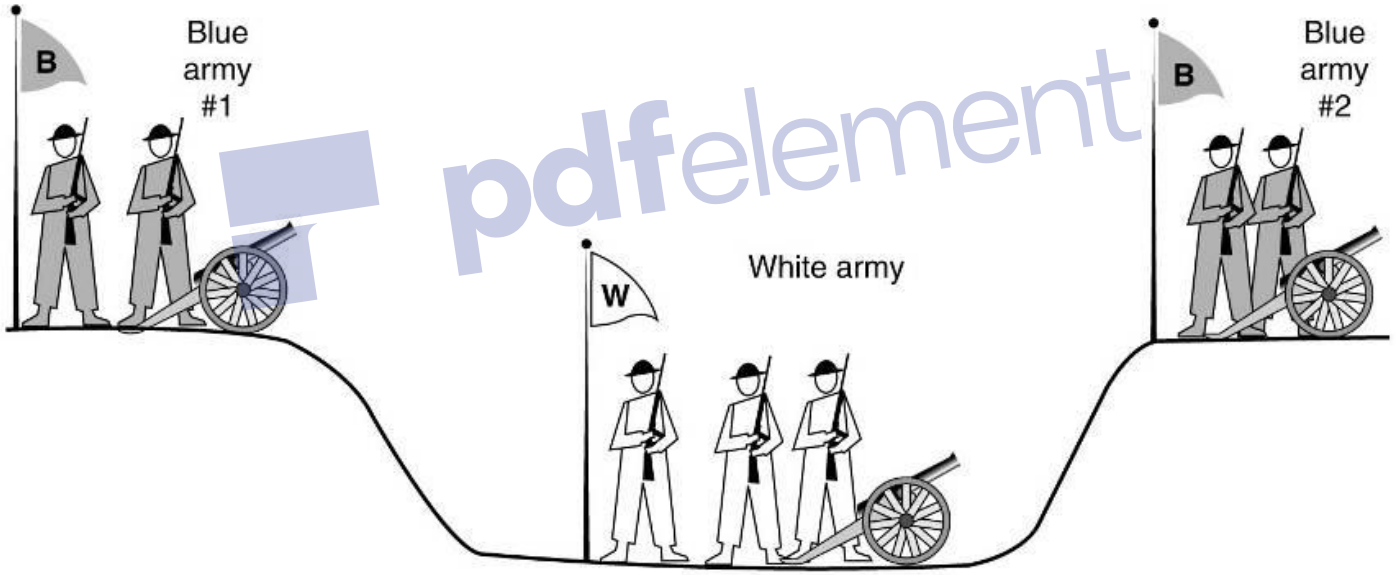


pdfelement

Remove Watermark Now

Problema celor două armate

- Puteți să vă gândiți la un algoritm?



pdfelement

Remove Watermark Now

Scheme de detectie

- Strategia Heartbeat – traditionala, (inca) cea mai intalnita
- Problema – scalabilitatea
  - Solutia 1) Structuri ierarhice de FD – Two-level Globus FD
  - Solutia 2) Protocoale de tip Gossip – cu o mare probabilitate, eventual toate procesele obtin aceleasi informatii
- Protocoale Adaptive
  - Se adapteaza dinamic la conditiile de trafic peste retea
  - E.g.: Un protocol ce adapteaza valoarea intervalului de timeout in functie de valoarea maxima a timpului de raspuns al mesajelor de heartbeat
- Detectoare de erori lenese
- Detectoare de erori progresive

pdfelement

Remove Watermark Now

Protocoale de detectie adaptive

- Asigura in aceeasi masura adaptarea la:
  - schimbarile conditiilor de retea (conditii diverse de trafic)
  - cerintele si schimbarile de context ale aplicatiilor
- La baza tot strategia *heartbeat,* insa intervalul de timeout este modificat dinamic conform conditiilor de retea.

pdfelement

Remove Watermark Now

Exemple FD adaptive (1)

- FETZER-FD
  - Intervalul de timeout este calculat ca fiind maximul dintre timpii de receptionare a mesajelor heartbeat.
  - In conditii reale:

1. in retele mari probabilitatea de pierdere a mesajelor este ridicata.
2. nu exista o limita superioara in ceea ce priveste intarzierile de mesaje.

pdfelement

Remove Watermark Now

Exemple FD adaptive (2)

- CHEN-FD
  - Se bazeaza pe analiza probabilistica a traficului de retea si a cerintelor aplicatiilor.
  - Estimarea timpului de primire a urmatorului mesaj heartbeat pe baza timpilor de receptionare a celor mai recente mesaje heartbeat.
  - Estimarea anterioara plus o margine de siguranta formeaza intervalul de timeout.

pdfelement

Remove Watermark Now

Dezavantaje

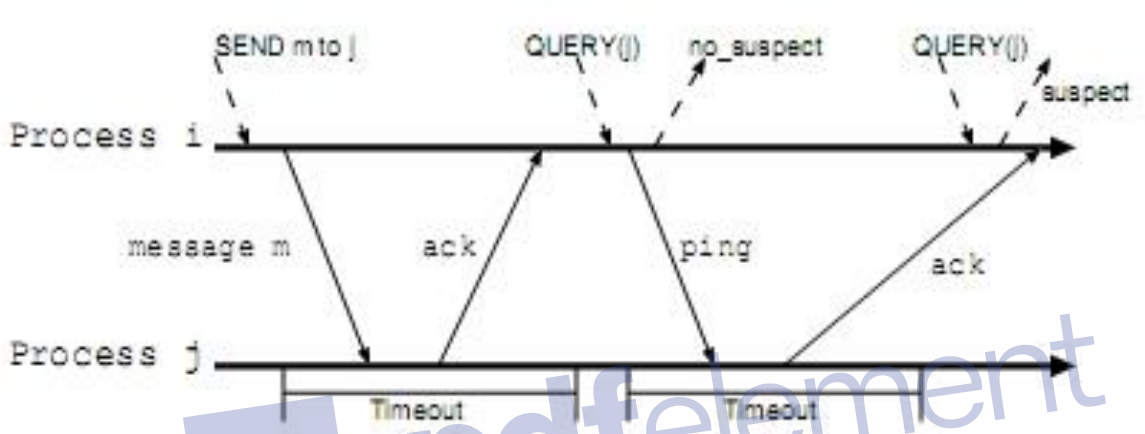
- Acuratetea este data de functia de estimare.
- Adaptarea la cerintele mai multor aplicatii ar necesita administrarea mai multor valori de timeout.

pdfelement

- O solutie alternativa -- aplicatiile sa-si seteze propria valoare de timeout
  - dar se invalideaza adaptarea la conditiile de retea.

Remove Watermark Now

Detectoare de erori lenese



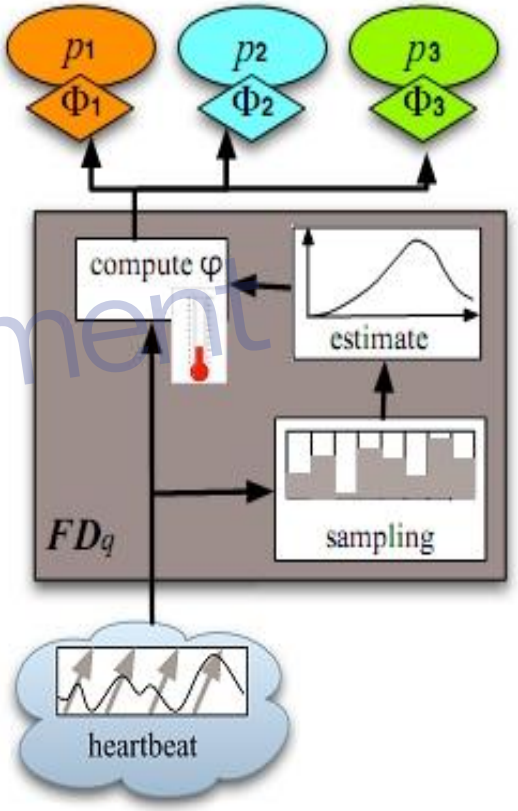
pdfelement

- Ideea de baza este folosirea a cat mai multe mesaje transmise intre procese pentru a detecta erori. Se impune, astfel, ca fiecare mesaj primit sa fie confirmat.
- Cand procesele nu comunica, se folosesc mesaje de monitorizare.

Detectoare de erori progresive

Remove Watermark Now

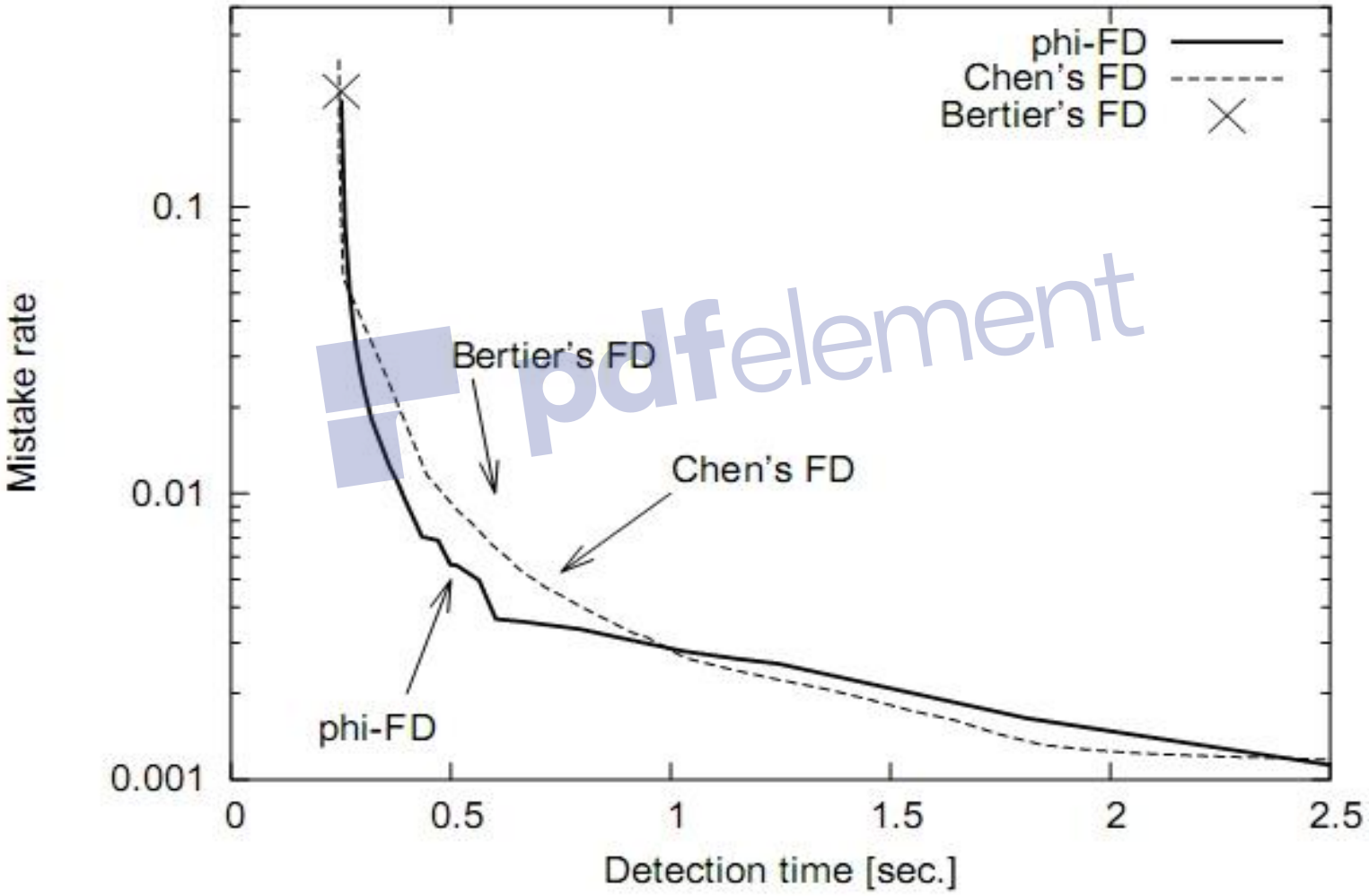
- Se bazeaza pe asocierea unui proces cu o valoare progresiva, care creste monoton in timp daca procesul s-a defectat.
- Fiecare aplicatie are un prag finit stabilit conform propriilor cerinte.
- O aplicatie decide daca suspecteaza sau nu un proces comparand pragul propriu cu valoarea progresiva asociata procesului.



pdfelement

Remove Watermark Now

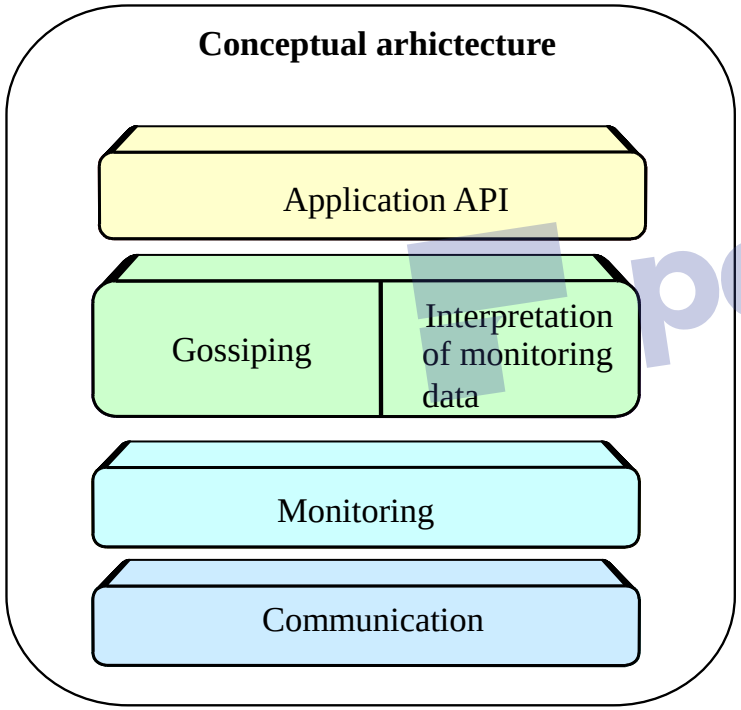
Detectoare de erori progresive



pdfelement

Remove Watermark Now

FD bazat pe algoritmi progresiv si cei de gossiping



❑Comunicaţie

Exploatează abordarea ierarhică, dar se bazează pe o topologie dinamică

❑Monitorizare

Foloseşte strategia heartbeat

❑Gossiping

Utilizează algoritmul de basic gossiping şi un algoritm optimizat de diseminare a informaţiei (Round-Robin)

❑Interpretare date de monitorizare

Are la bază o transformare olomorfă şi metoda de predicţie EMA combinată cu un algoritm de ajustare adaptivă a factorului de netezire

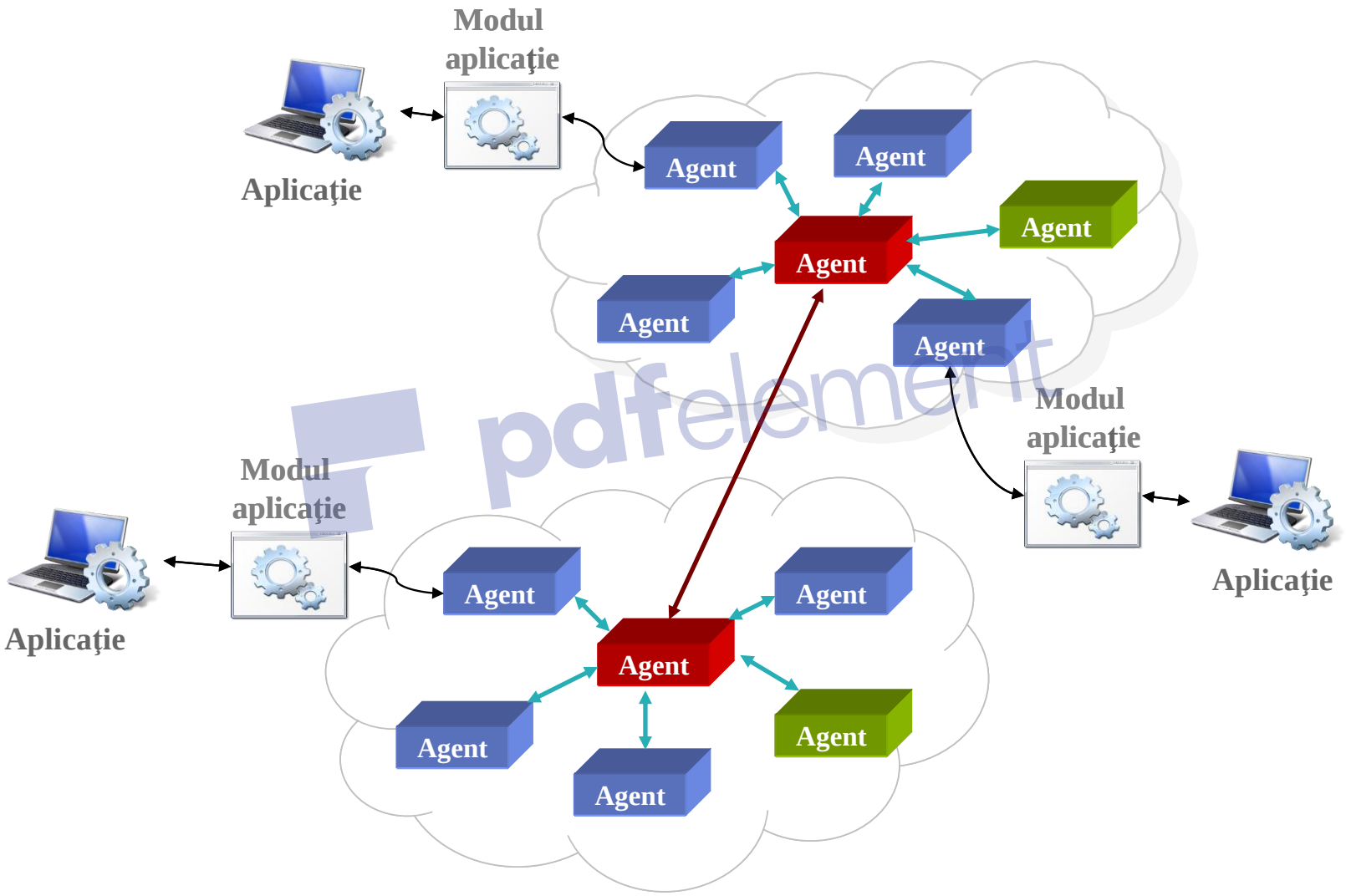
❑Api aplicaţie

Oferă acces la serviciile modulului de detecţie

pdfelement

Clusterizare

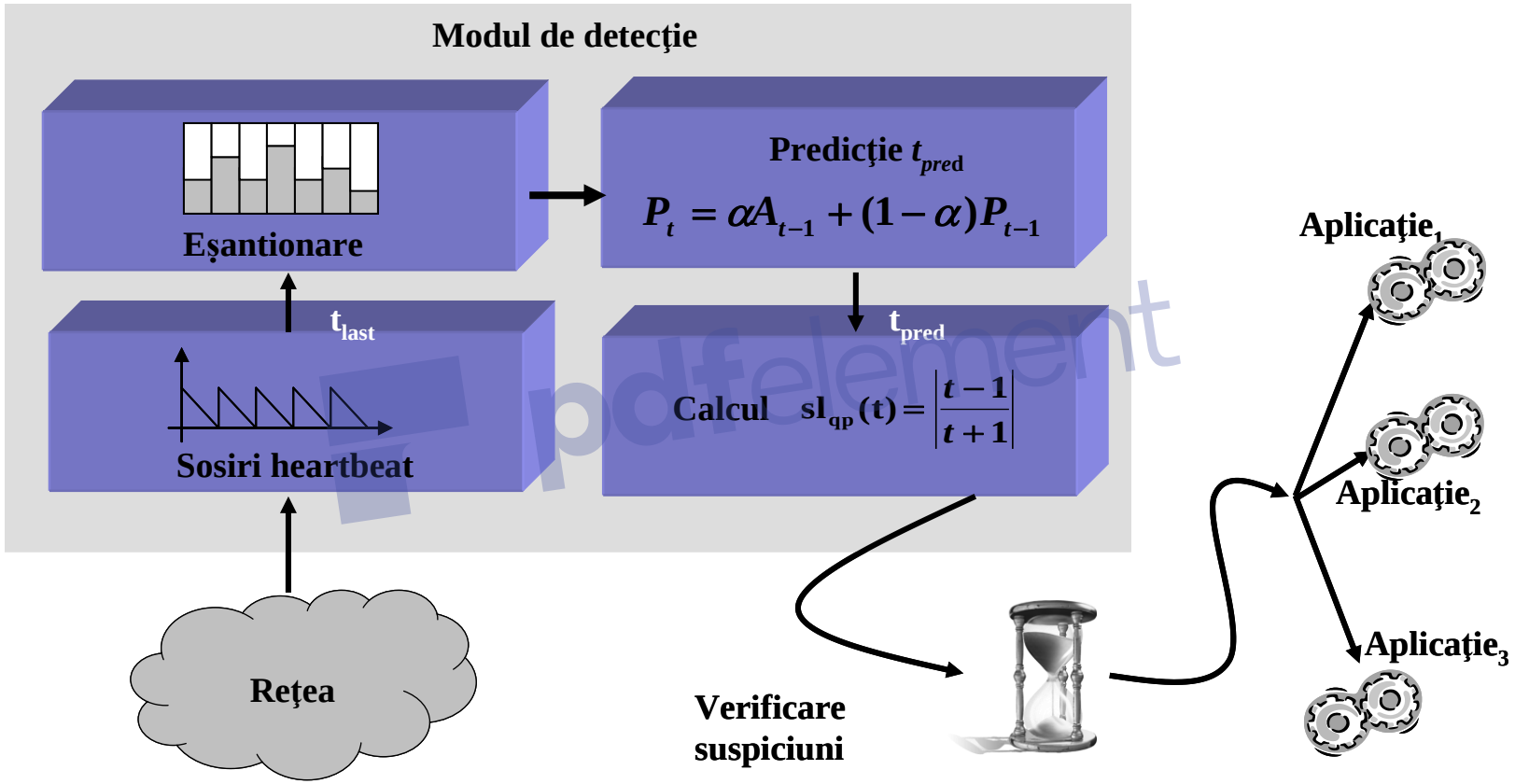
Remove Watermark Now



pdfelement

Remove Watermark Now

Interpretare date monitorizare (1)



pdfelement

Interpretare date monitorizare (2)

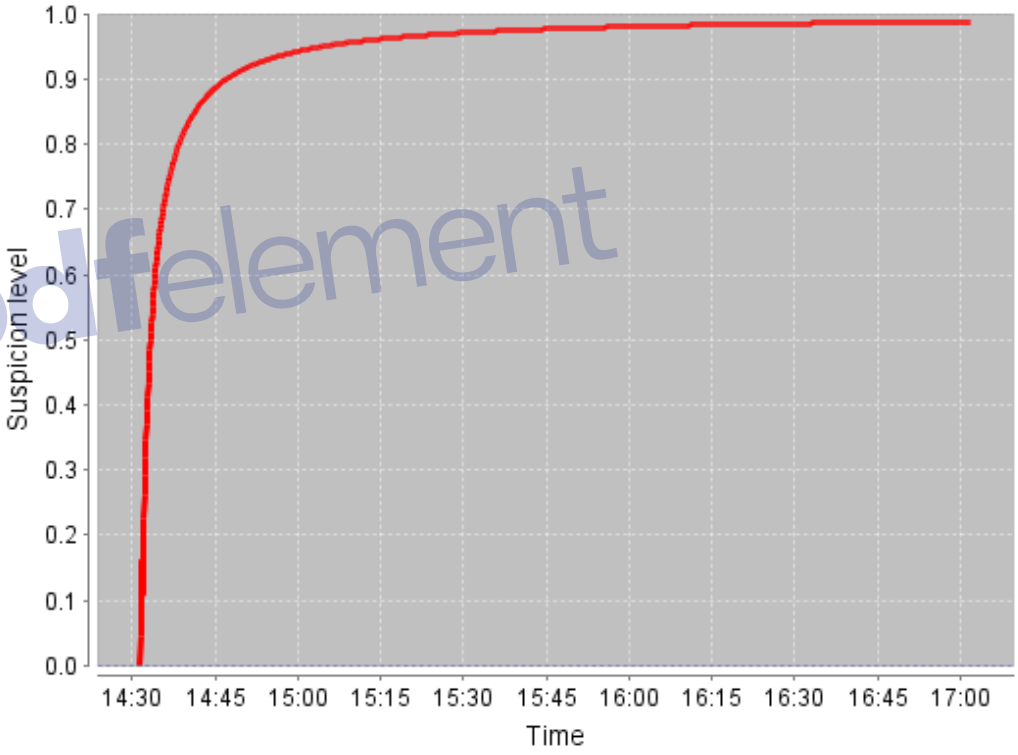
Remove Watermark Now

$$0 <= sl_{qp}(t) < 1 \mapsto sl_{qp}(t) = \left|\frac{t-1}{t+1}\right|,$$

$$t = \frac{t_{now}}{t_{pred}} \in (0, \infty),$$

t_{now} *timpul curent,*

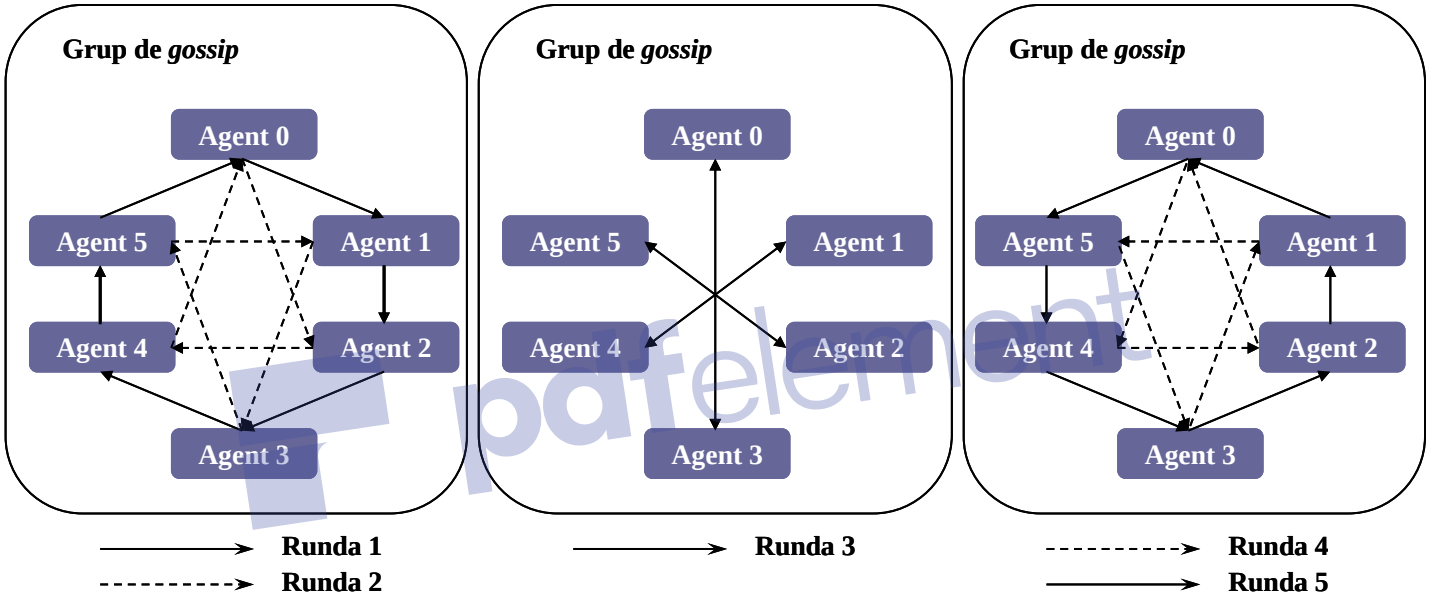
t_{pred} *ultima valoare prezisa*



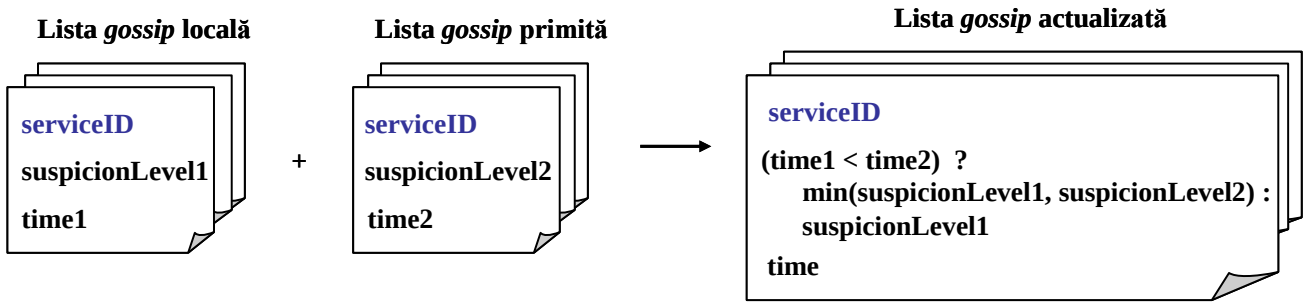
pdfelement

Remove Watermark Now

Gossiping (1)

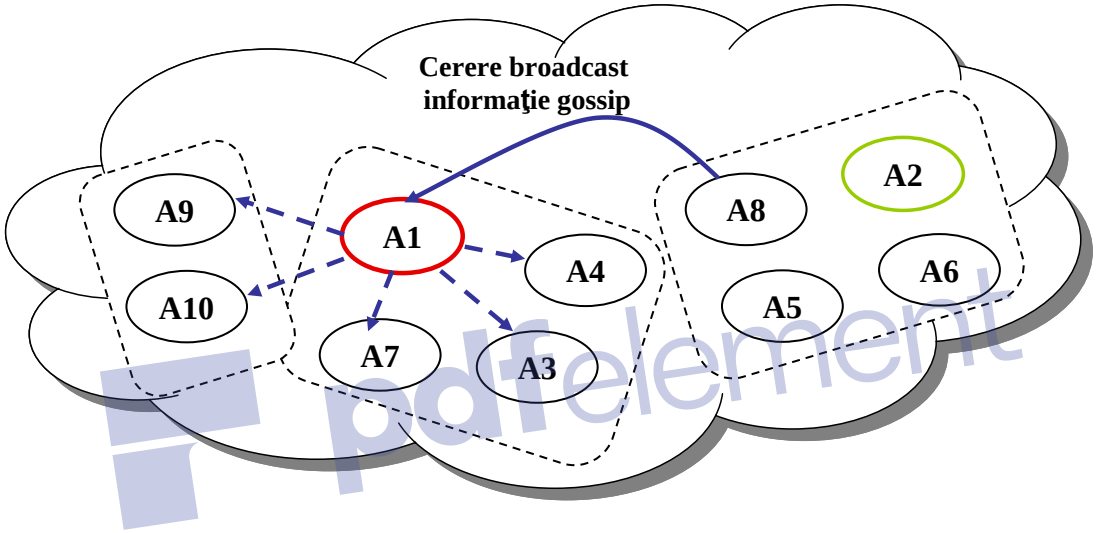


pdfelement

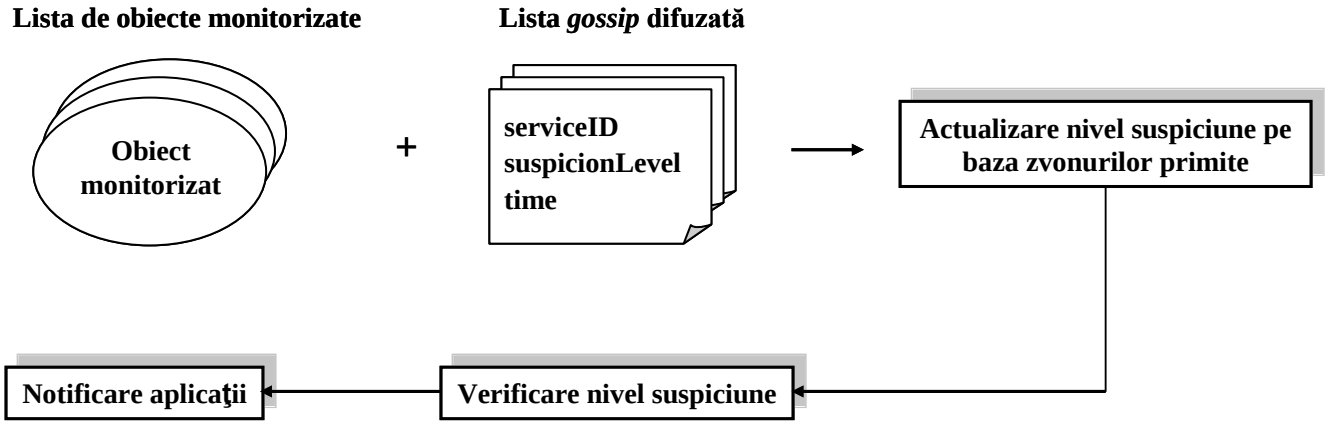


Remove Watermark Now

Gossiping (2)

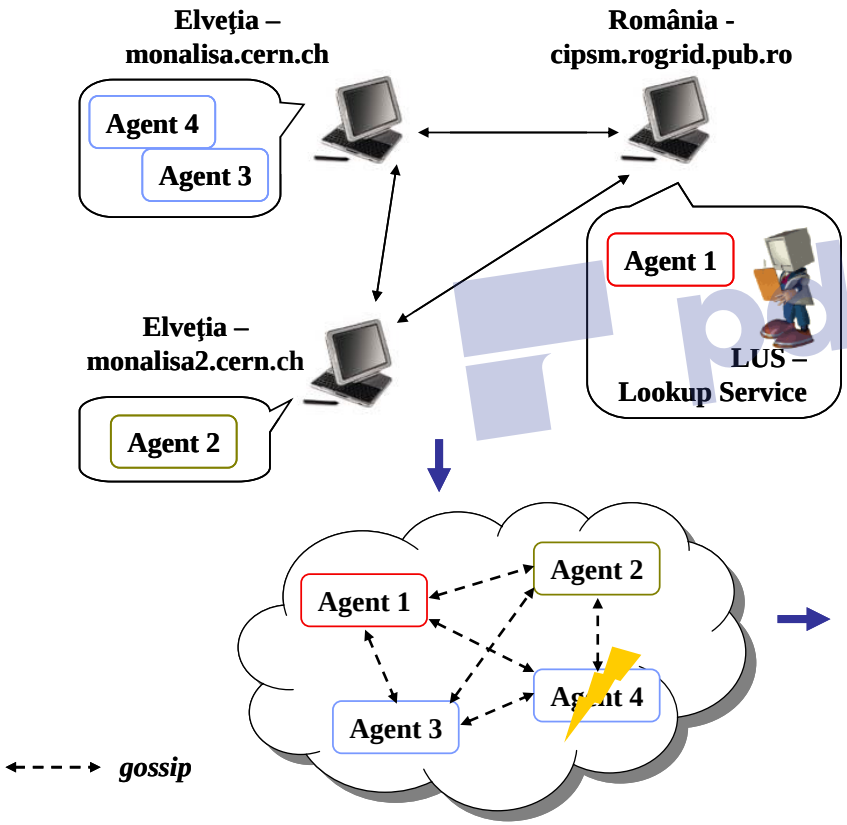


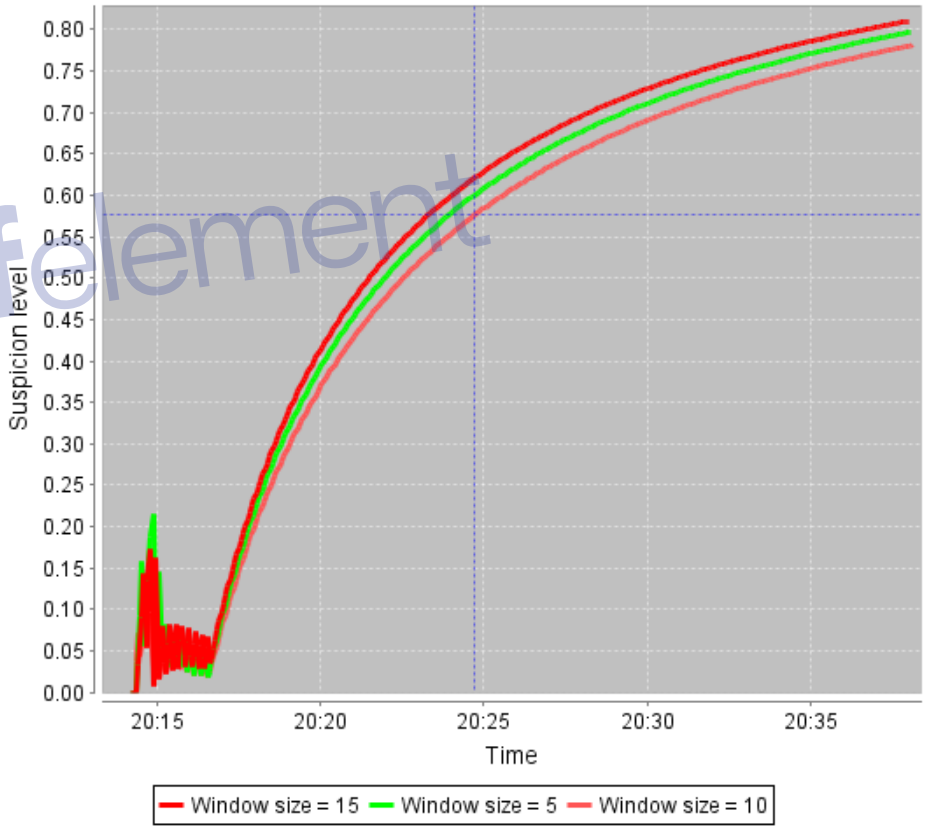
pdfelement



Remove Watermark Now

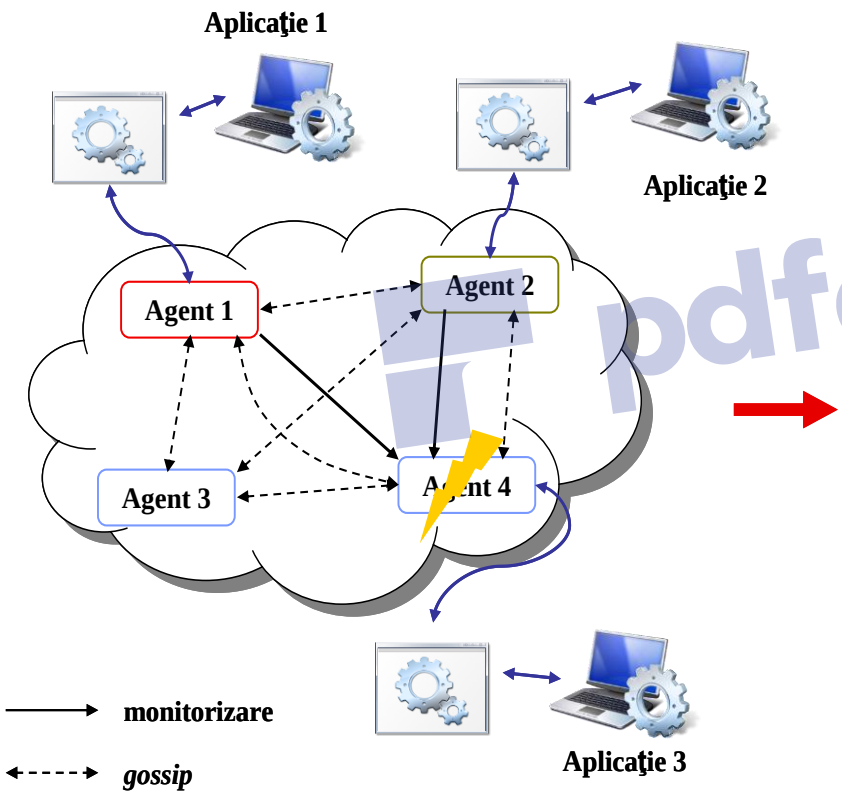
Testare şi validare – Scenariul 1

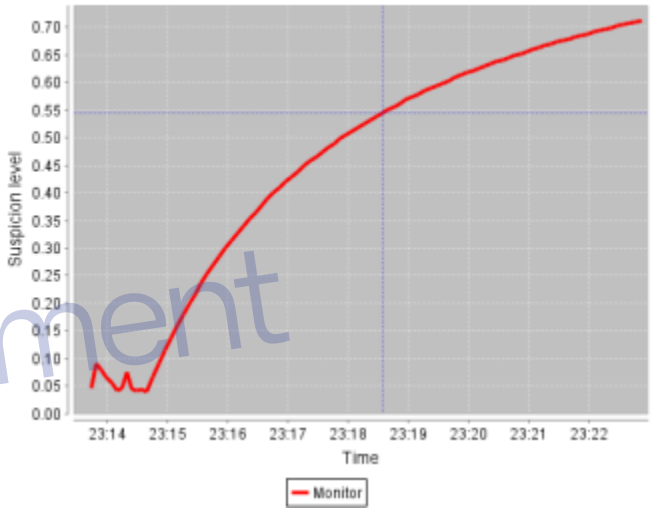


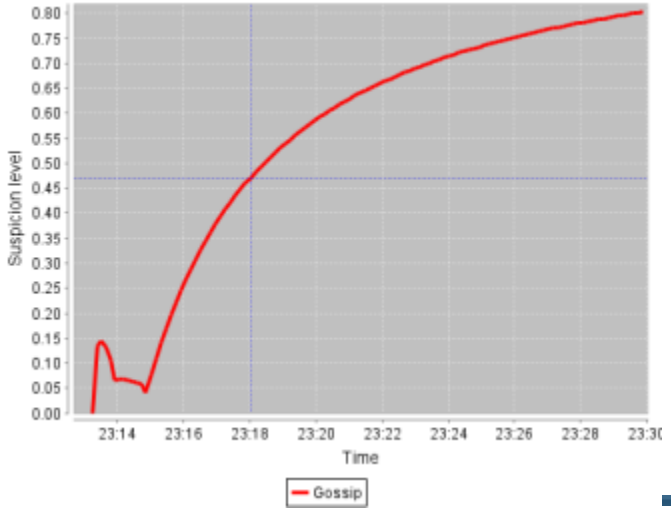


Remove Watermark Now

Testare şi validare – Scenariul 2







pdfelement

Remove Watermark Now

Recuperarea din eroare

- Realizata prin “readucerea” aplicatiei la o stare corecta, de dinainte de aparitia defectului
- Gestiunea erorilor:
  - Rollback – stare salvata: checkpoint
  - Rollforward – urmatoarea stare fara eroare reprezinta noua stare
  - Compensare – mascarea erorii prin redundanta

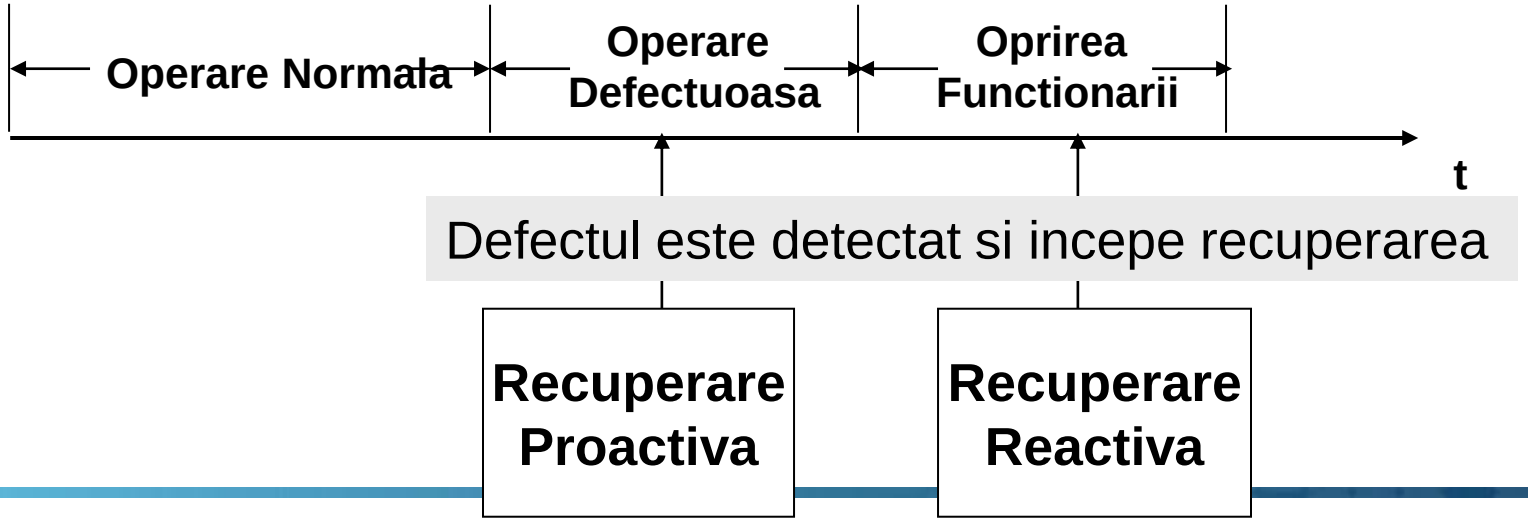
pdfelement

Remove Watermark Now

Solutii de recuperare din eroare

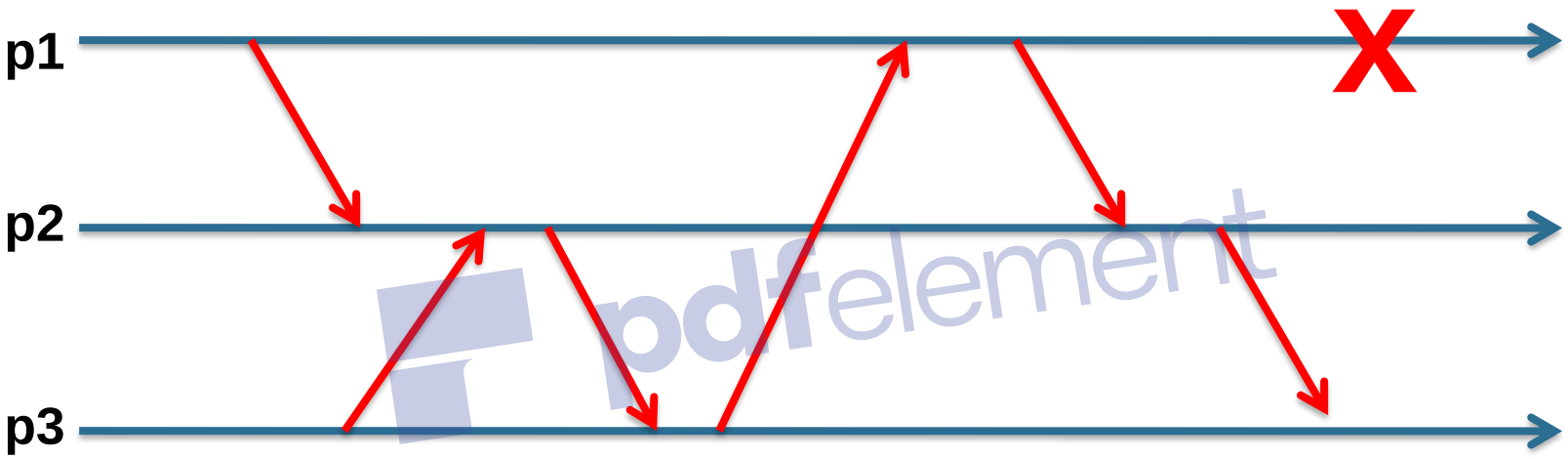
- Abordarea tipica FT: sistemul asteapta aparitia unui defect si reactioneaza la detectarea acestuia pentru a-l corecta
- Daca defectele se propaga mai repede decat pot mecanismele de recuperare sa le trateze se poate ajunge la situatia cand nu se mai pot aplica tehnicile de recuperare
- Mecanismele de recuperare proactiva:
  - Cresterea rezistentei sistemului la posibilele propagari nedorite ale erorilor
  - Initierea recuperarii inainte sau concurent cu recunoasterea problemei din sistem
  - Adaptarea sistemului

pdfelement



Remove Watermark Now

FT pentru aplicatiile bazate pe schimburi de mesaje

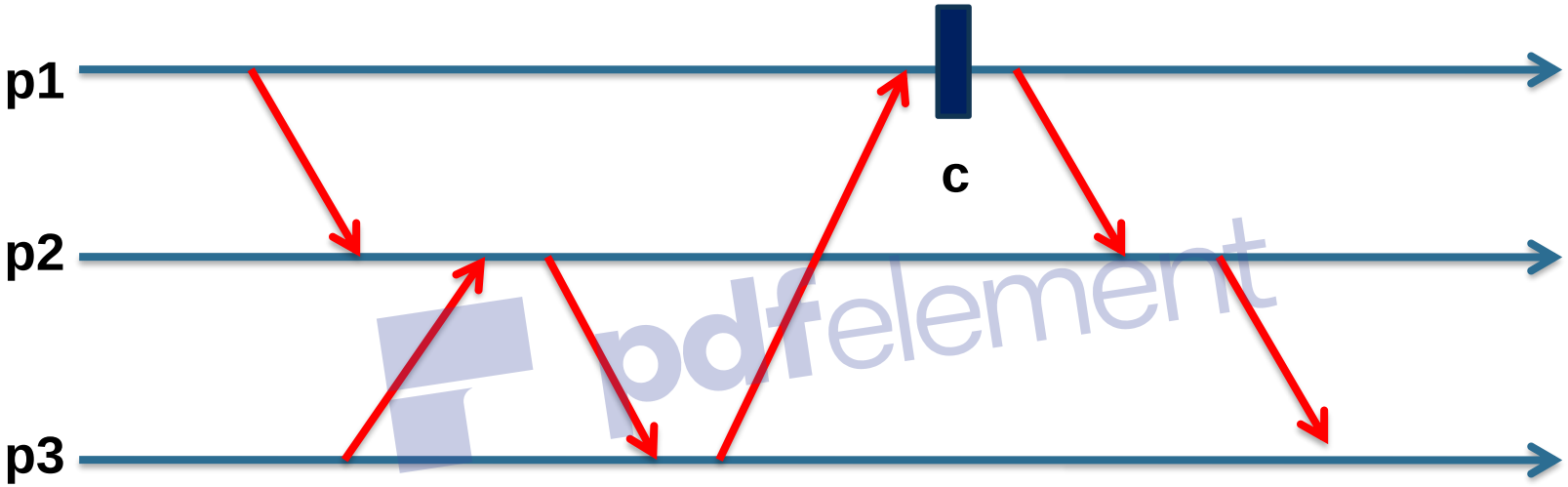


pdfelement

- Re-executia intregii aplicatii
- Pierderea tuturor calculelor

Remove Watermark Now

Checkpoint

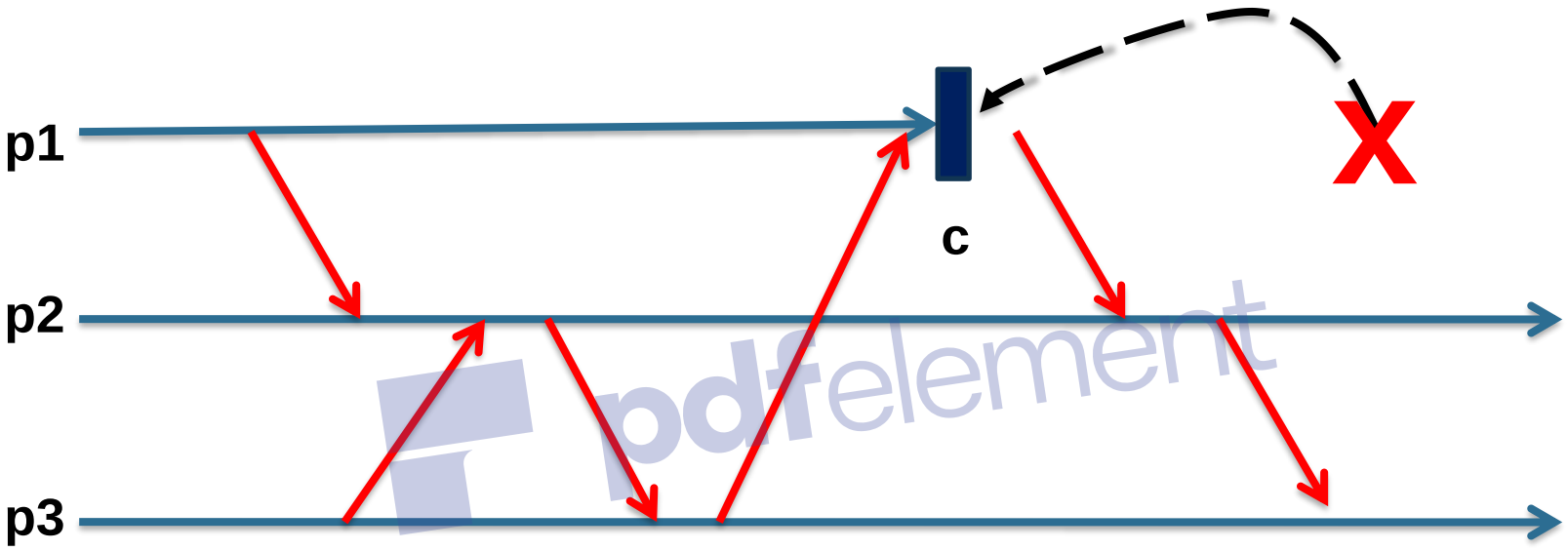


pdfelement

- Snapshot al unui proces
- Starea salvata pe medii de stocare persistente

Remove Watermark Now

Checkpoint

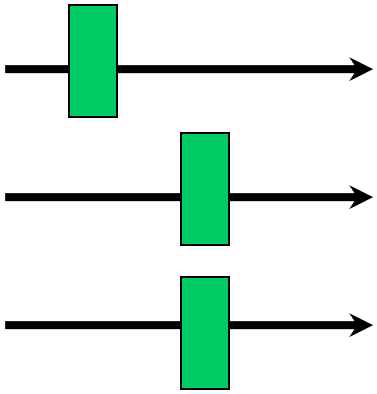


pdfelement

- p2 a trimis un mesaj ce nu a fost trimis de p1
- Este nevoie de salvea unei stari globale, consistente, a sistemului

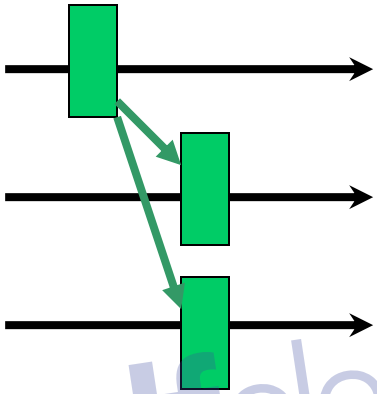
Remove Watermark Now

Checkpointing Distribuit



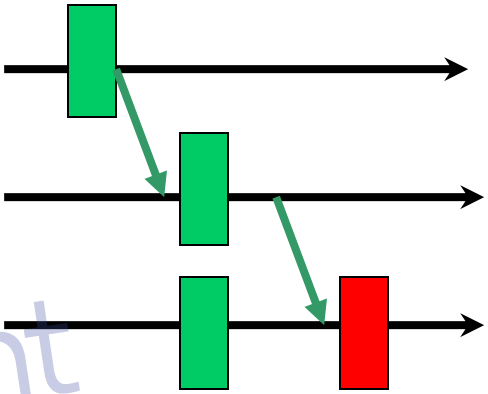
Independent

- ☑ Simplitate
- ☑ Autonomie
- ☑ Scalabilitate
- ✖ Efectul de Domino



Coordonat

- ☑ Stari Consistente
- ☑ Performanta Buna
- ☑ Garbage Collection
- ✖ Scalabilitate



Indus de comunicatie

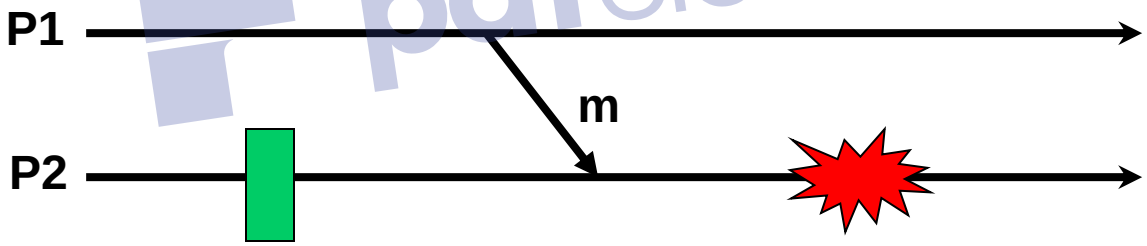
- ☑ Stari Consistente
- ☑ Autonomie
- ☑ Scalabilitate
- ✖ Performanta

pdfelement

Remove Watermark Now

Logarea Mesajelor

- Logarea datelor pe dispozitive persistente de stocare in timpul executiei fara erori
- Folosirea acestei informatii pentru recuperarea din eroare



pdfelement

- ***Orfan:*** procesul ce depinde de o stare ne-recuperabila a unui proces esuat

Remove Watermark Now

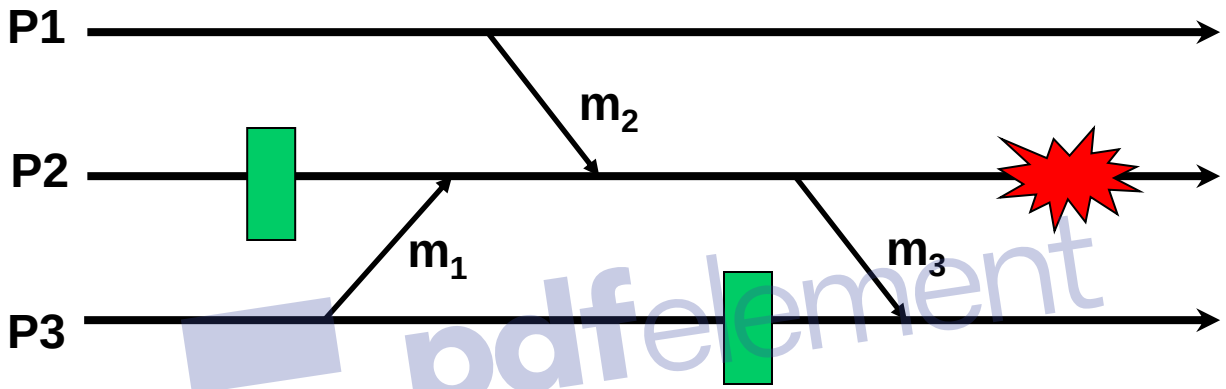
Procese orfan?

- Dupa ce procesul p trimite mesajul m, tot el trimite mesajul m' catre procesul q, care la randul lui il trimite mai departe.
- Daca mesajul m nu era logat, atunci defectarea lui p ar putea conduce la pierderea oricarei informatii despre m.
- La reinitializarea lui p, el poate ajunge intr-o stare in care a doua oara nu mai trimite mesajul m' lui q.
- In acest caz procesul q nu ar mai fi consistent cu procesul p, pentru ca q a trimis un mesaje m' ce nu a fost niciodata trimis de procesul p.
- q este un proces orfan.

pdfelement

Remove Watermark Now

Abordari Message Logging



pdfelement

Pesimista

- ☑ Nu exista procese orfan
- ☑ Recuperare usoara
- ✖ Performante scazute

Optimista

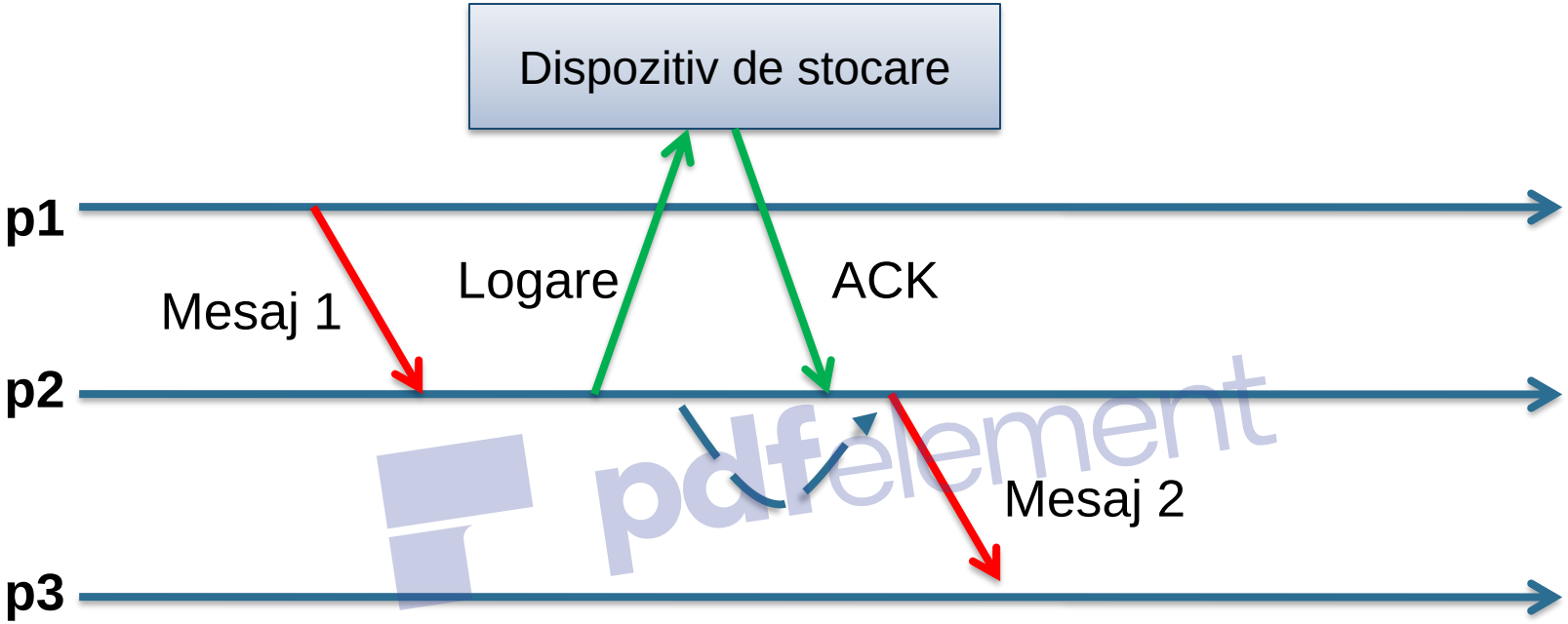
- ☑ Non-blocanta
- ✖ Procese orfan
- ✖ Recuperare complexa

Cauzala

- ☑ Non-blocanta
- ☑ Nu exista procese orfan
- ✖ Recupera Complexa

Remove Watermark Now

Abordarea pesimista

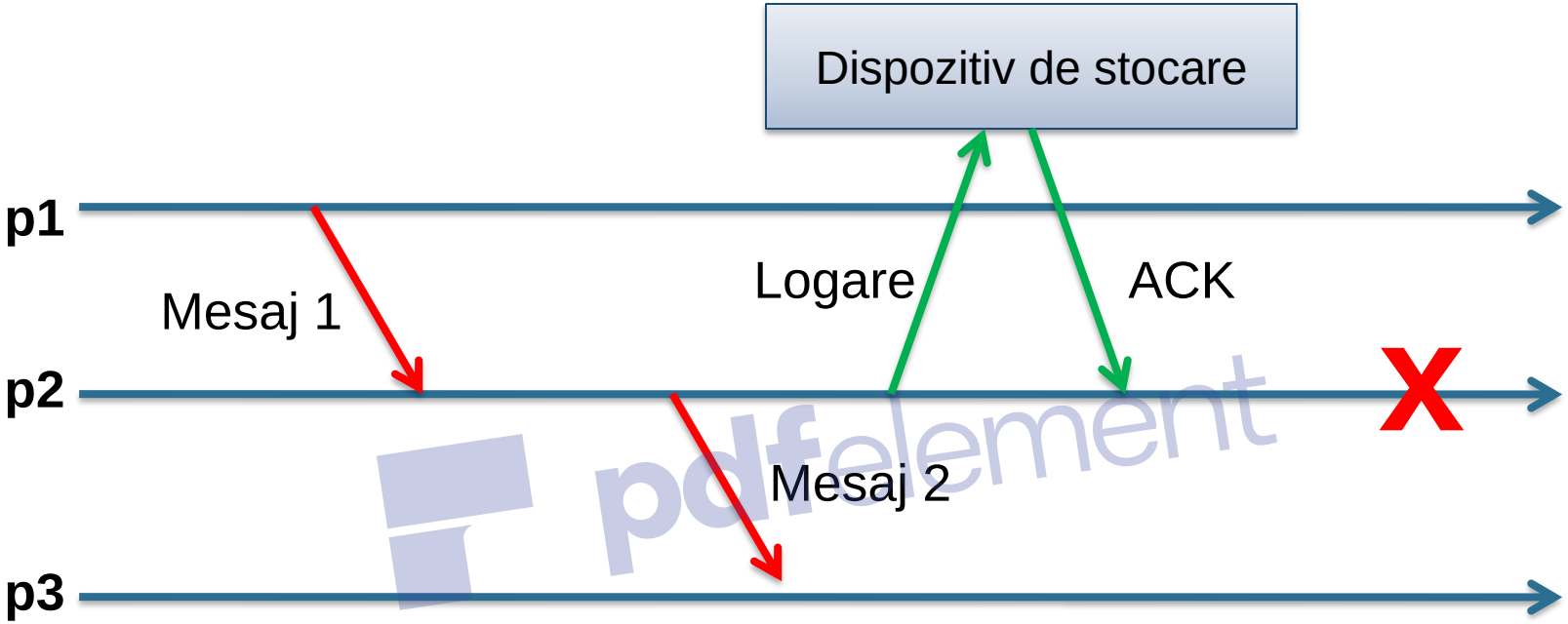


pdfelement

- Mesajele sunt logate sincron pe mediul de stocare.
- Overhead asupra comunicarii.

Remove Watermark Now

Abordare optimista



pdfelement

- Mesajele sunt logate asincron pe mediul de stocare.
- Protocolul de recuperare poate fi complex (risc de aparitie al proceselor orfane).

Remove Watermark Now

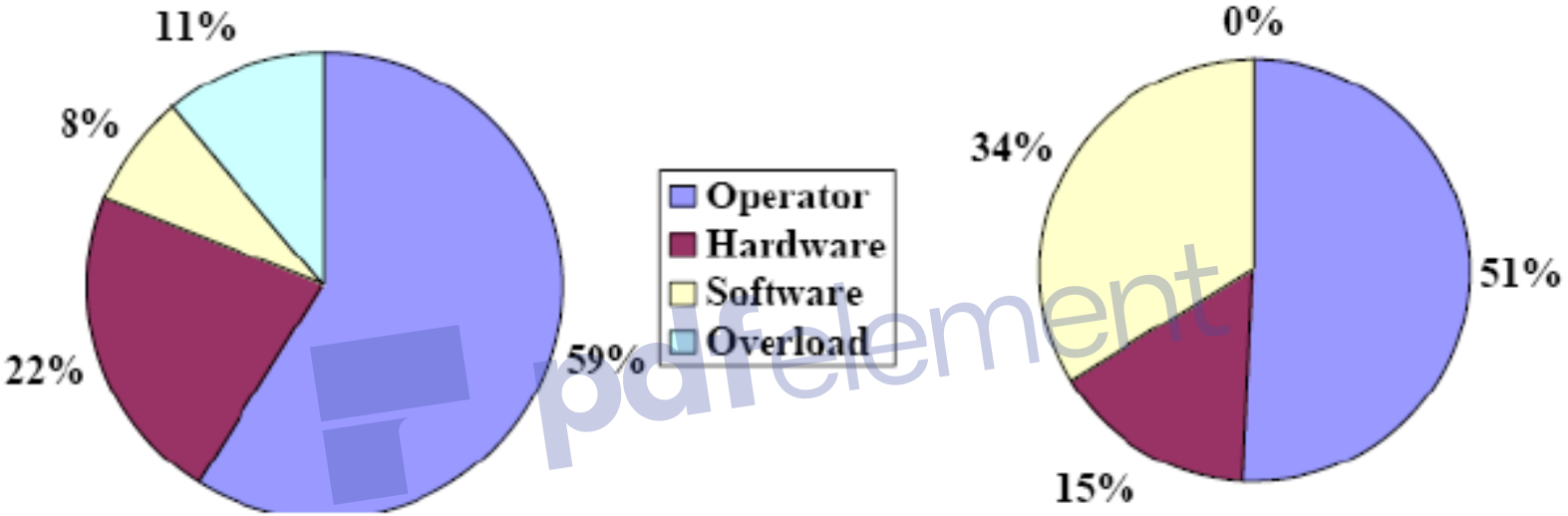
Recovery Oriented Computing

- Ideea: defectele hardware, software si erorile de operare sunt aspecte ce trebui tratate, nu prevenite.
- Axate pe MTTR:
  - Reducerea timpului de recuperare
  - Marirea disponibilitatii sistemului
- Repornirea sistemului dupa defect
  - Solutia naiva: reboot
  - Repornirea doar a unor parti ale sistemului: sisteme modulare, in care componentele pot fi repornite independent: serviciul din Unix /etc/rc
- Stateful recovery
  - Recuperarea din defect in cazul bazelor de date
  - Folosirea metodelor de checkpointing

pdfelement

Remove Watermark Now

Cauzele de Downtime: leader = operatorul
Efectul in 5 studii de caz a 6 tehnici ROC:



Public Switched Telephone Network **Average of Three Internet Sites**

ROC technique	Mercury RR	FIG	Email with Undo	Pinpoint	ROC-1
Partitioning	√				√
Recovery experiments	√	√	√	√	√
Reversible systems			√		
Diagnosis aid				√	√
Independent mechanisms	√		√		√
Redundancy and fail fast					√

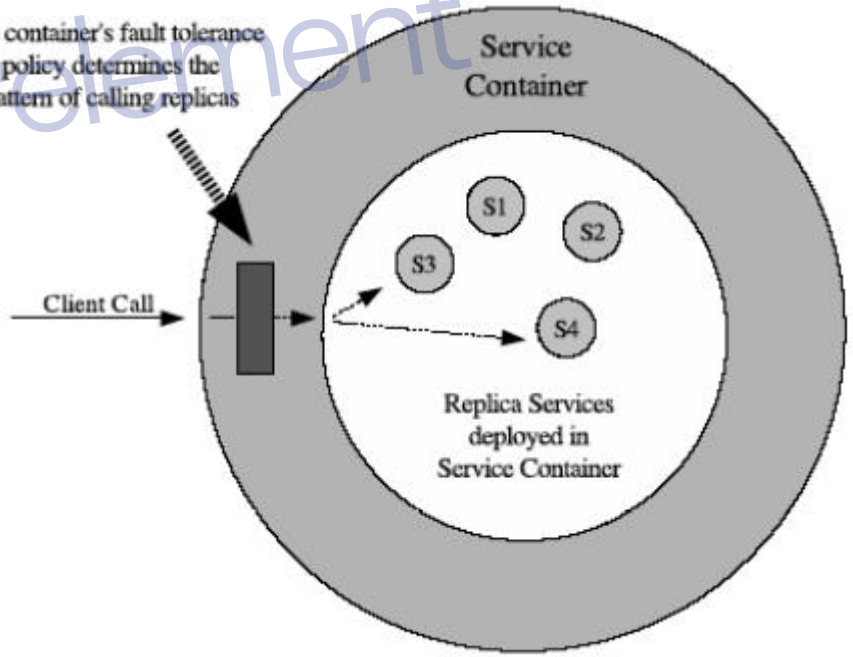
pdfelement

Remove Watermark Now

Dependable Service Engineering

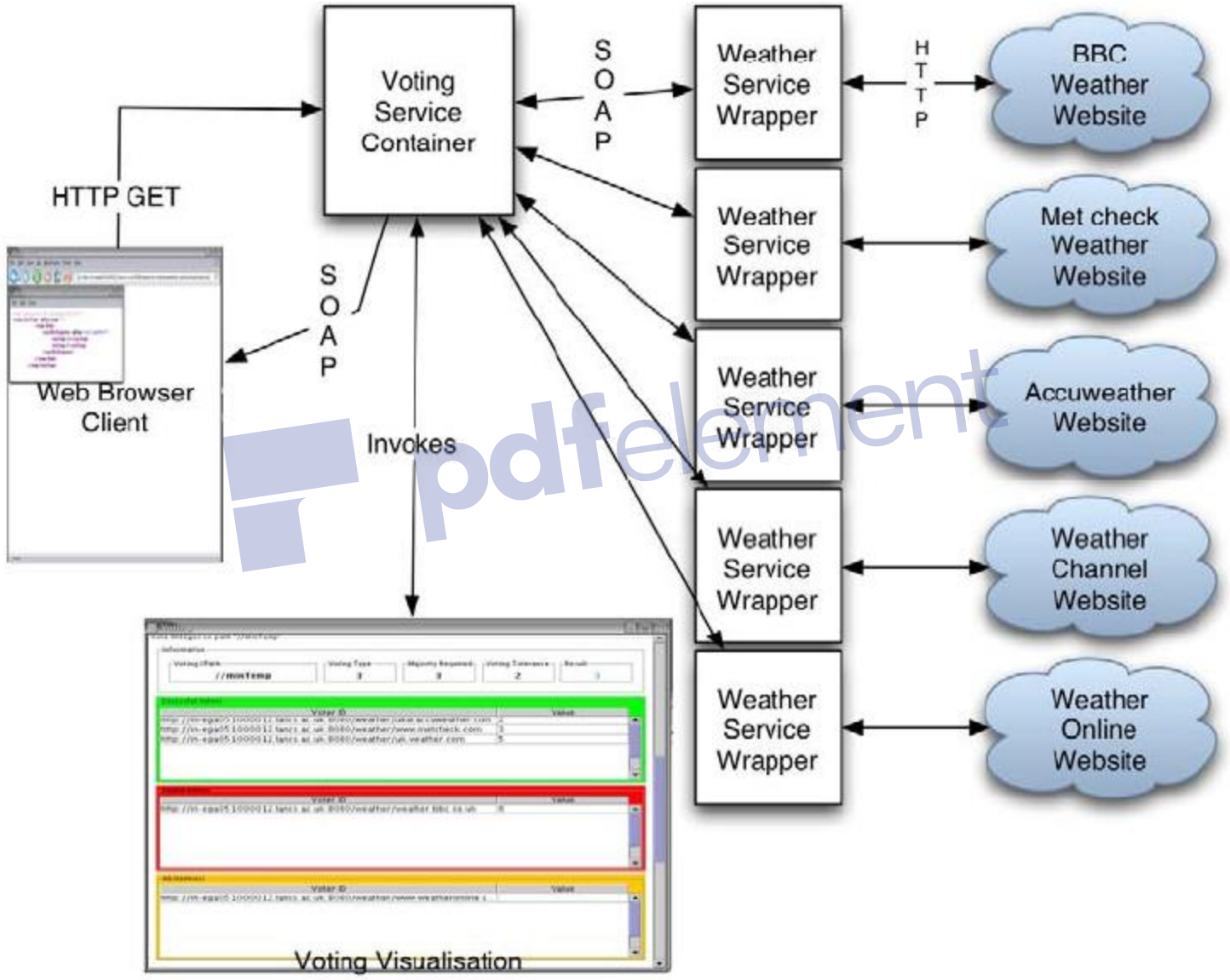
- Arhitectura bazata pe folosirea de containere de servicii tolerante la defecte

  - Tehnici de toleranta la defecte suportate:
    - Rewind and retry
    - Resubmiterea catre implementari alternative
    - Executie pe mai multe versiuni cu votare



pdfelement

Remove Watermark Now

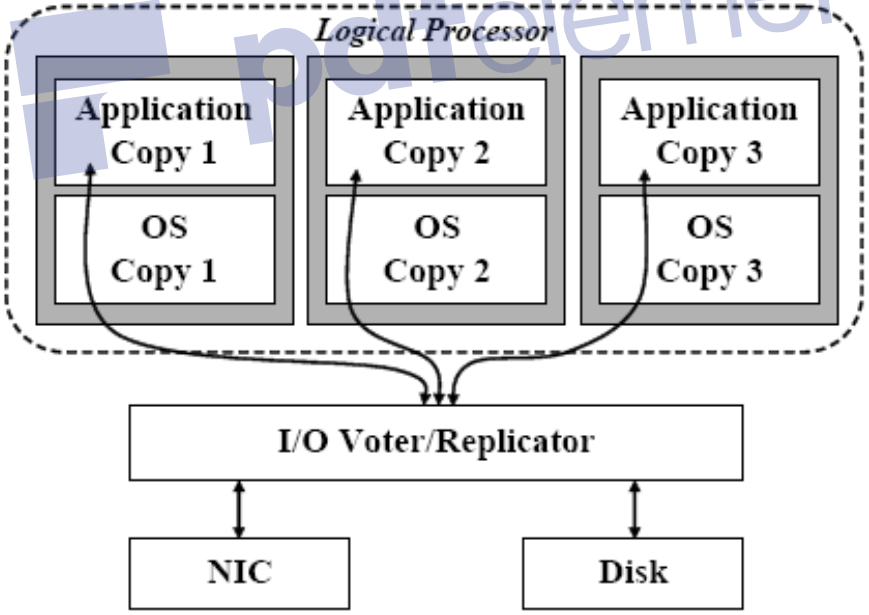


pdfelement

Remove Watermark Now

Dependable ≠ unaffordable

- Toleranta la defecte prin virtuzalizare
- Loosely Synchronized Redundant Virtual Machines (LSRVM)



pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Teorema CAP.

Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Teorema CAP

Remove Watermark Now

- Propusă de Prof. Eric Brewer la PODC (Principle of Distributed Computing) 2000

- Descrie *trade-off-urile* sistemelor distribuite

- Este imposibil pentru un serviciu web să furnizeze *trei garanții* în același timp:
  - **Consistency**
  - **Availability**
  - **Partition-tolerance**

pdfelement

Remove Watermark Now

Teorema CAP

- **C**onsistency (Consistență):
  - Toate nodurile ar trebui să vadă aceleași date în același timp
- **A**vailability (Disponibilitate):
  - Defectarea unui nod nu ar trebui să oprească operarea corectă a sistemului (doar) cu nodurile rămase
- **P**artition-tolerance (Toleranță la partiționare):
  - Sistemul continuă operarea cf. cu partiționarea rețelei
- Un sistem distribuit poate satisfice oricare două garanții, dar niciodată **toate trei împreună**

pdfelement

Remove Watermark Now

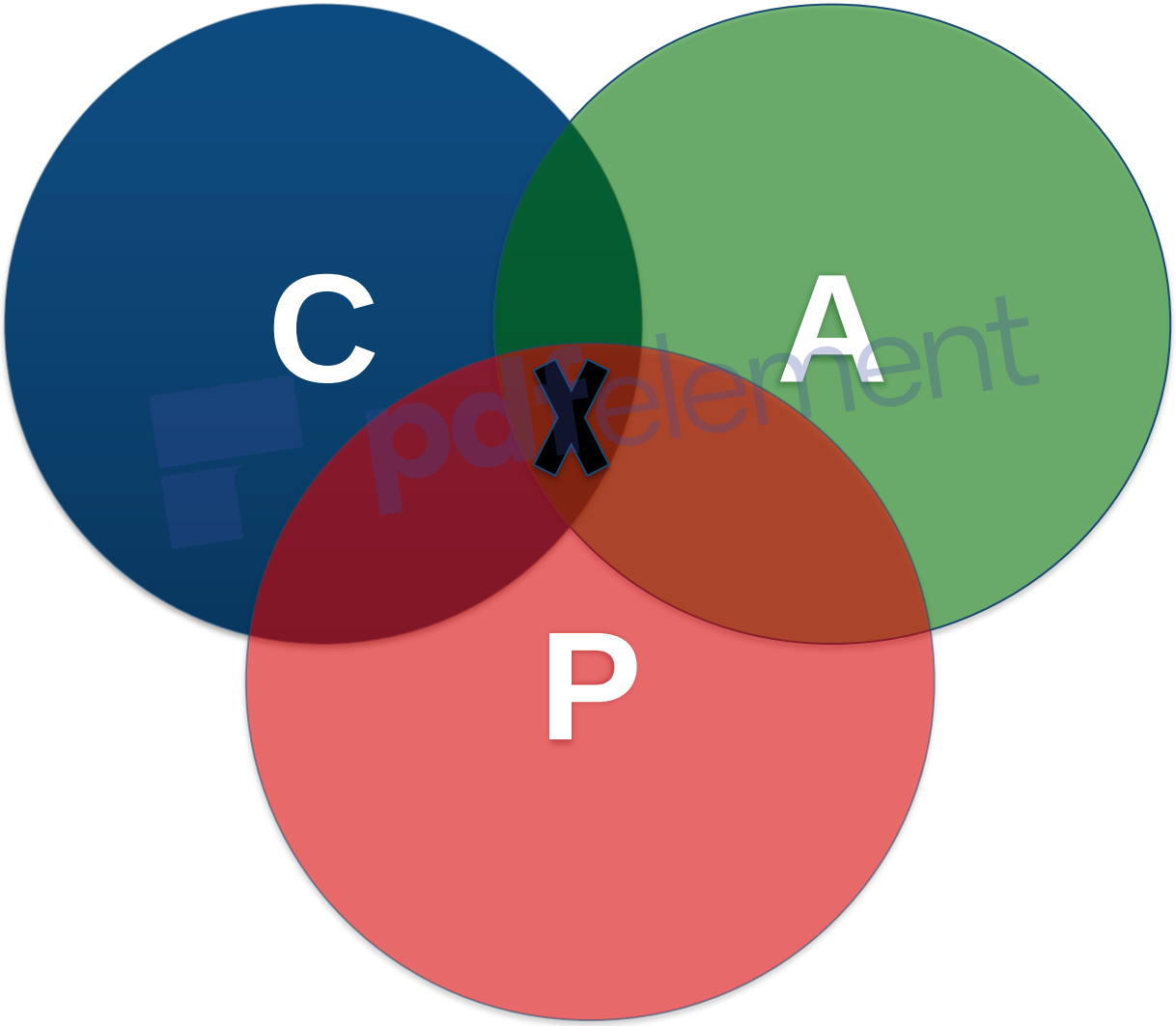
O mica precizare (diff. între Reliability și Availability)

- Diferența dintre Fiabilitate și Disponibilitate
  - Un sistem pica câte o milisecundă la fiecare oră
    - highly available (99.9%) , but highly unreliable
  - Un sistem care nu pica vreodată, dar are nevoie de 1 zi/lună pentru mentenanță
    - highly reliable, not highly-available

pdfelement

Remove Watermark Now

Teorema CAP



Remove Watermark Now

Teorema CAP

- Un exemplu:

Hotel Booking: are we double-booking the same room?

pdfelement

Bob

Dong

Remove Watermark Now

Teorema CAP

- Un exemplu:

Hotel Booking: are we double-booking the same room?

pdfelement

Bob

Dong

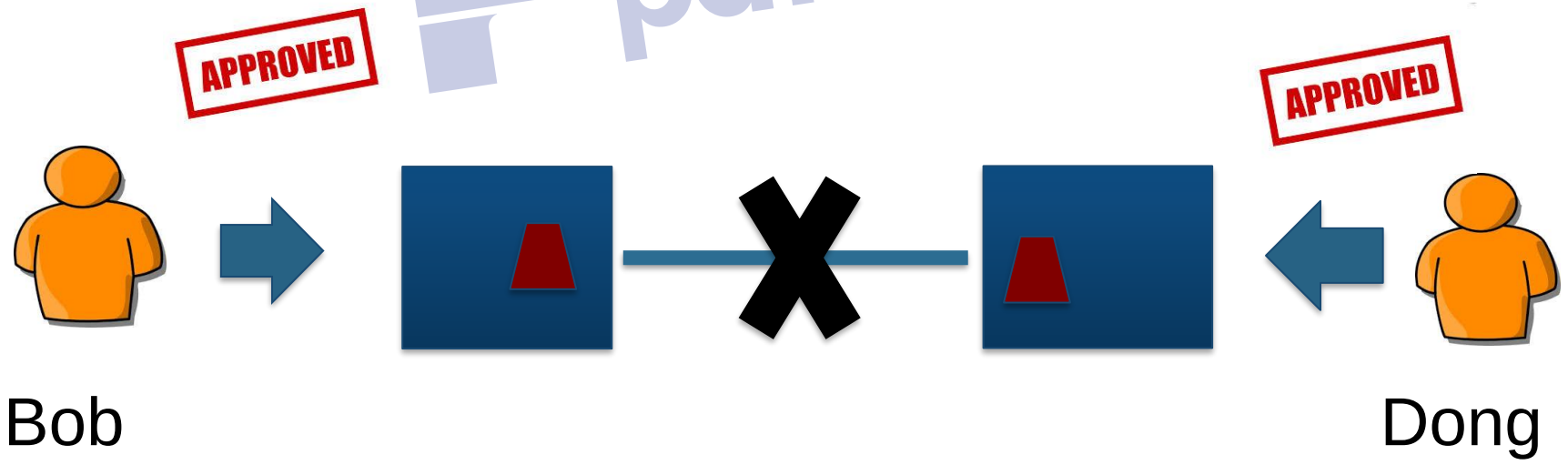
Remove Watermark Now

Teorema CAP

- Un exemplu:

Hotel Booking: are we double-booking the same room?

pdfelement



Remove Watermark Now

Teorema CAP: Demonstratie

- 2002: Demonstratia furnizata de Nancy Lynch si Seth Gilbert la MIT

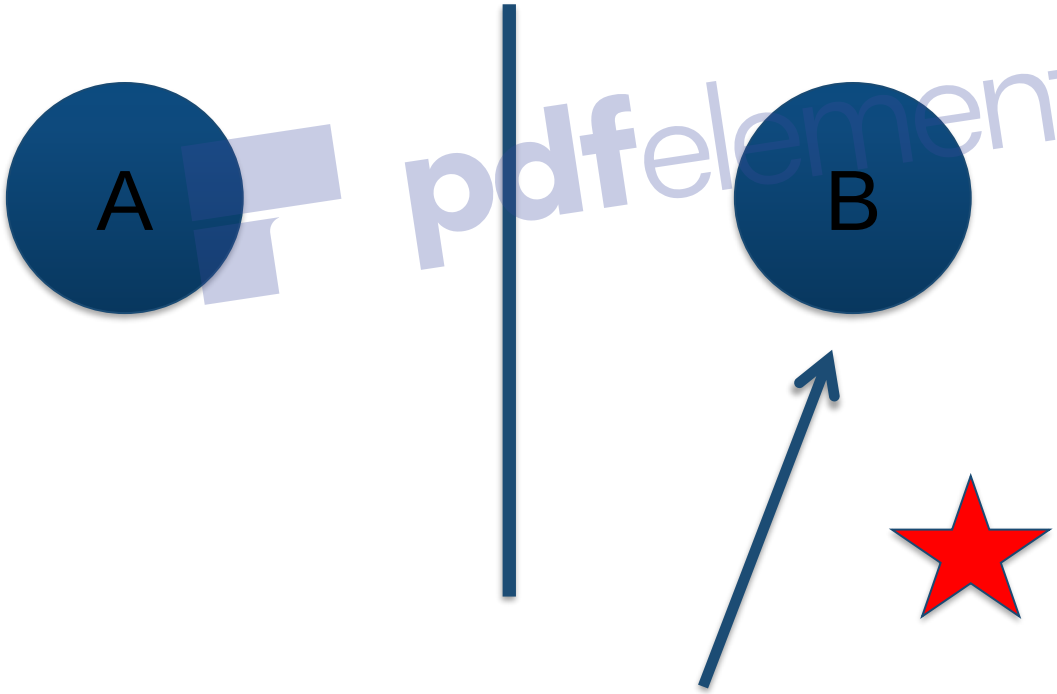
pdfelement

Gilbert, Seth, and Nancy Lynch. "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services." ACM SIGACT News 33.2 (2002): 51-59.

Remove Watermark Now

Teorema CAP: Demonstratie

- O demonstratie simpla folosind doua noduri:

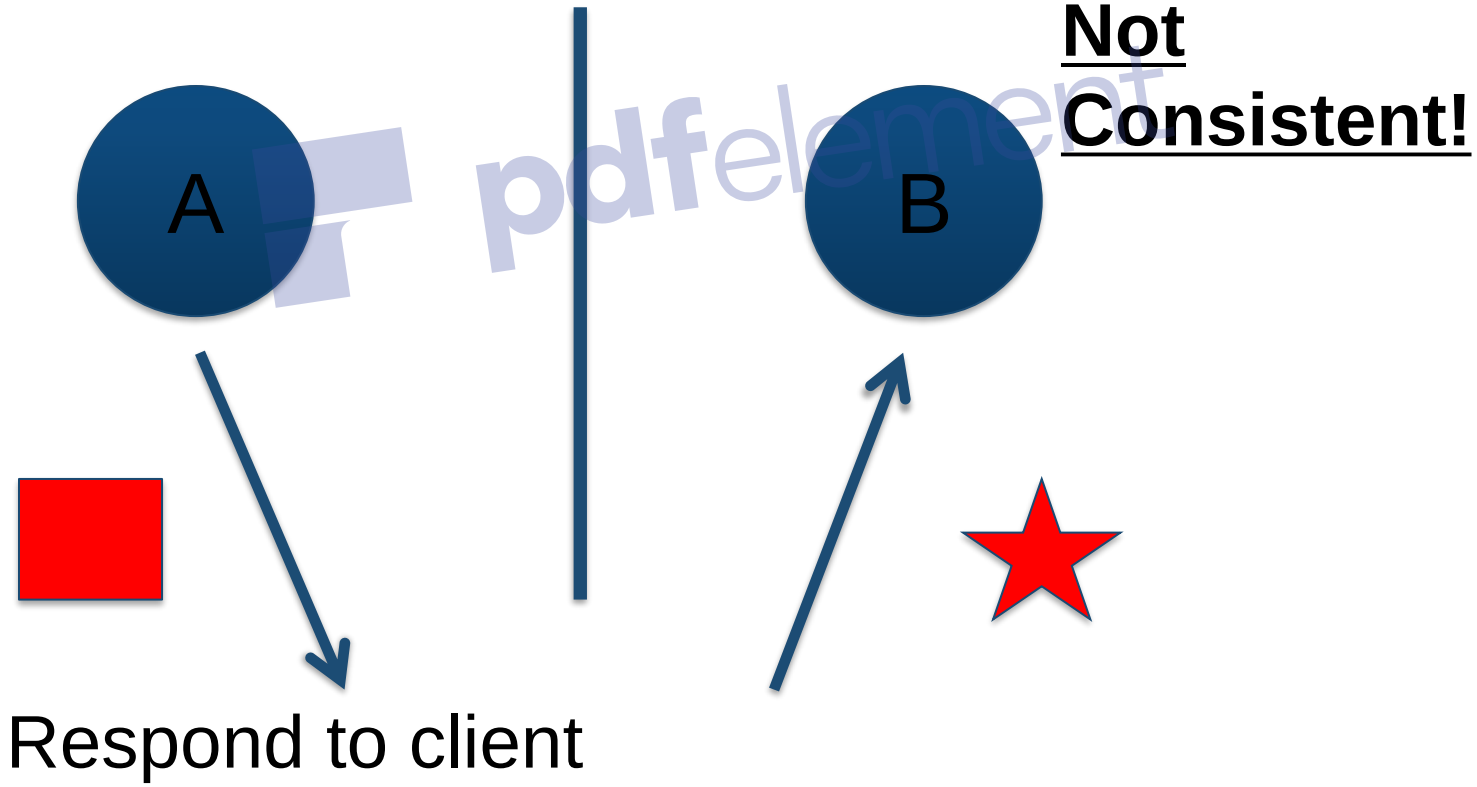


pdfelement

Remove Watermark Now

Teorema CAP: Demonstratie

- O demonstratie simpla folosind doua noduri:

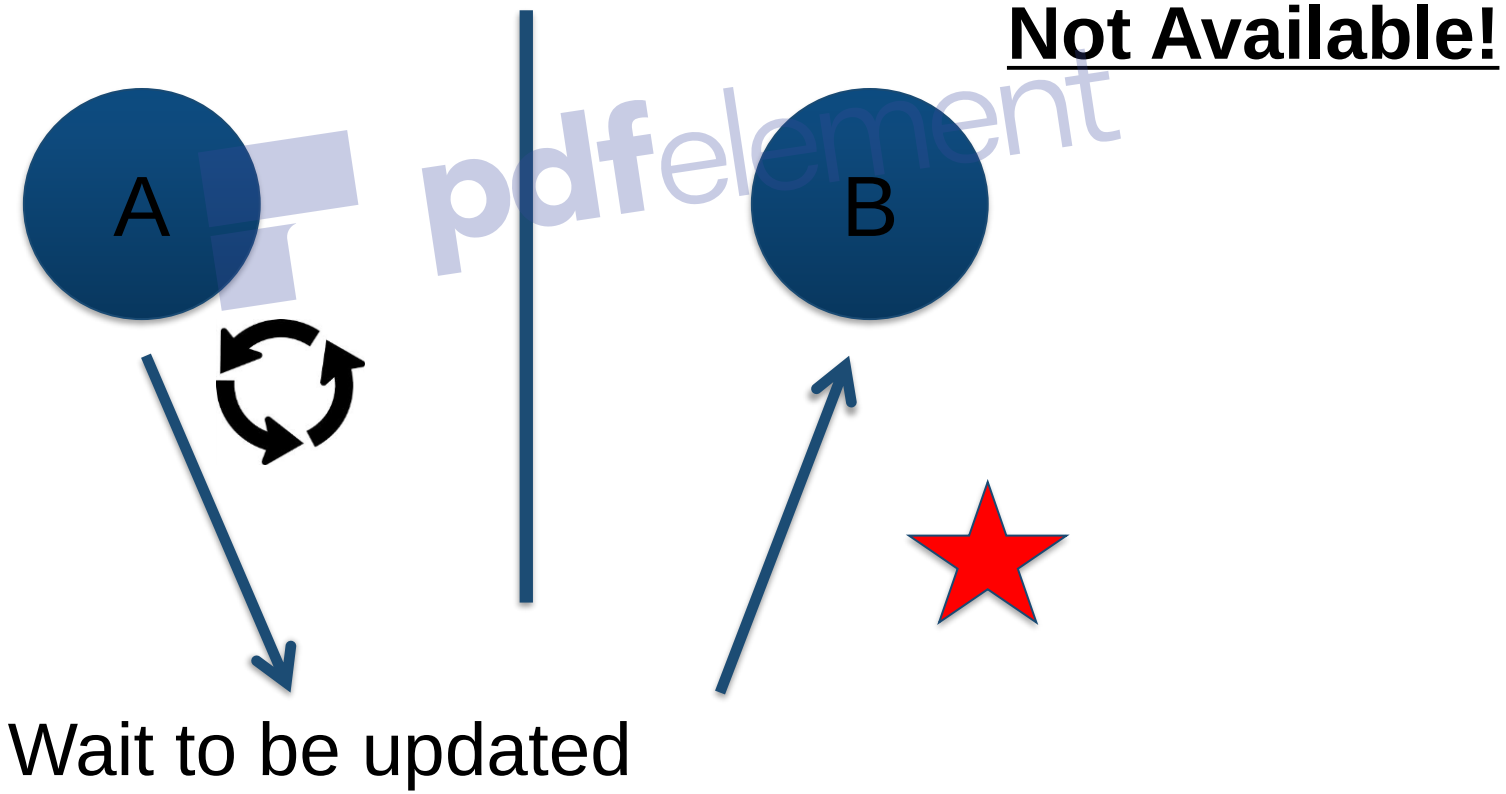


pdfelement

Remove Watermark Now

Teorema CAP: Demonstratie

- O demonstratie simpla folosind doua noduri:

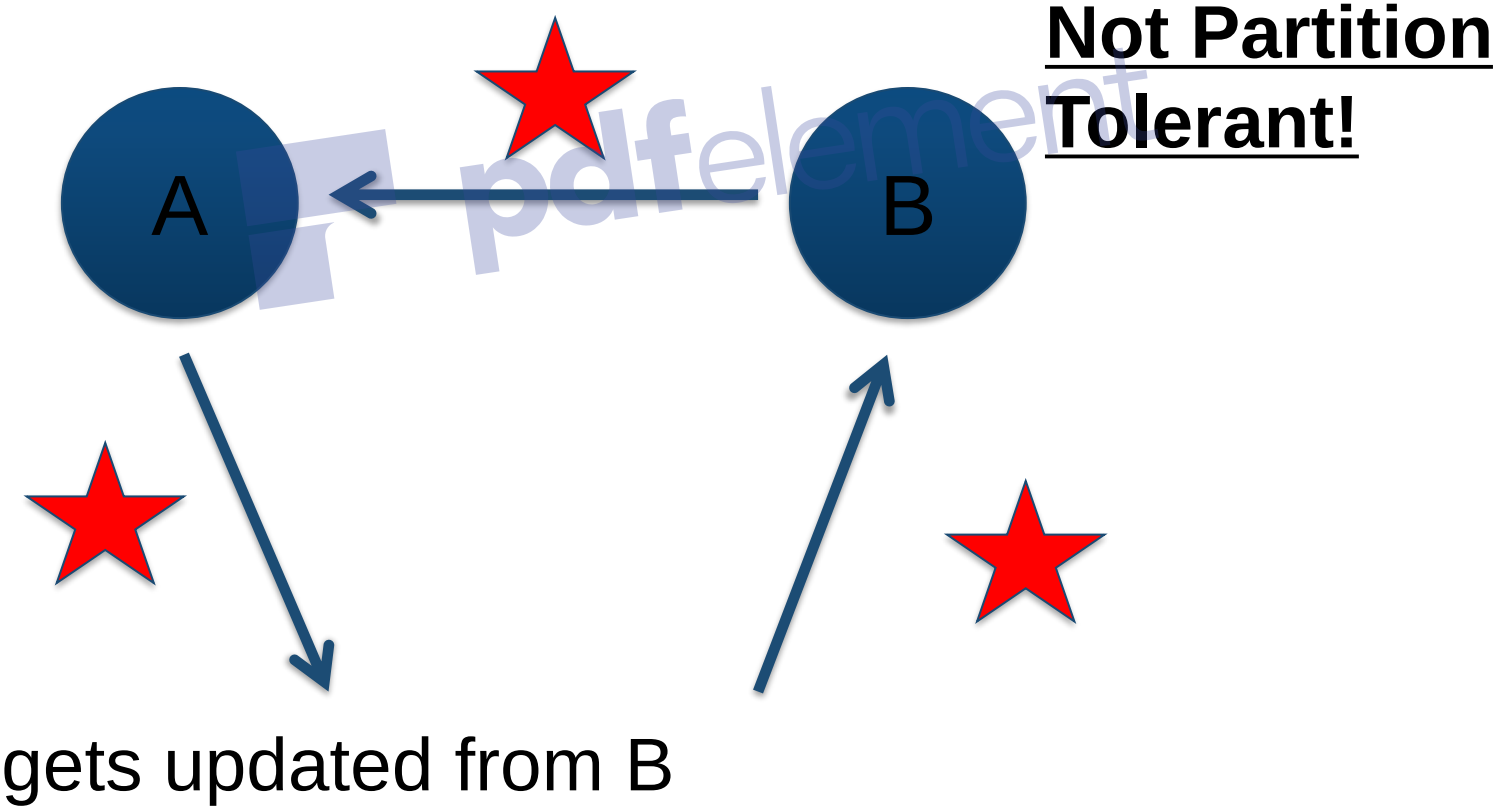


pdfelement

Remove Watermark Now

Teorema CAP: Demonstratie

- O demonstratie simpla folosind doua noduri:



Remove Watermark Now

De ce e important?

- Viitorul bazelor de date consta in **distribuirea** lor (Big Data Trend, etc.)
- Teorema CAP descrie **trade-off-uri** implicate de sistemele distribuite

pdfelement

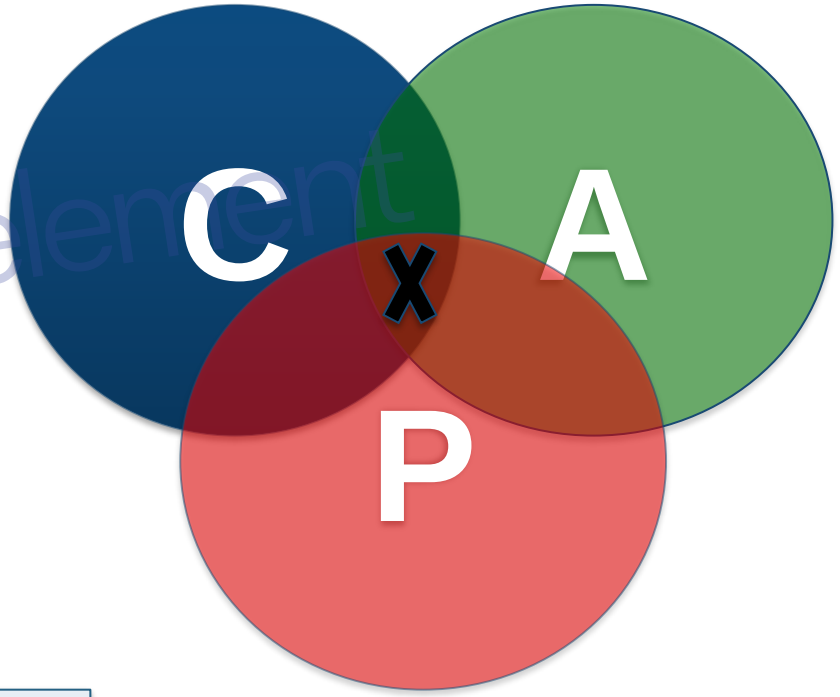
Scalabilitatea Bazelor de Date Relationale

- Bazele de Date Relaționale sunt construite pe principiul **ACID** (Atomicity, Consistency, Isolation, Durability)
- Implică că o bază de date relațională cu adevărat distribuită ar trebui să aibă toate proprietățile de **availability, consistency, partition tolerance**.
- Ceea ce e **imposibil** …

Remove Watermark Now

Ce oferă Teorema CAP

- Din următoarele trei garanții de performanță oferite:
  - Consistency
  - Availability
  - Partition tolerance
- Alegeți două
- Ceea ce înseamnă că sunt trei tipuri de sisteme distribuite:
  - CP
  - AP
  - CA



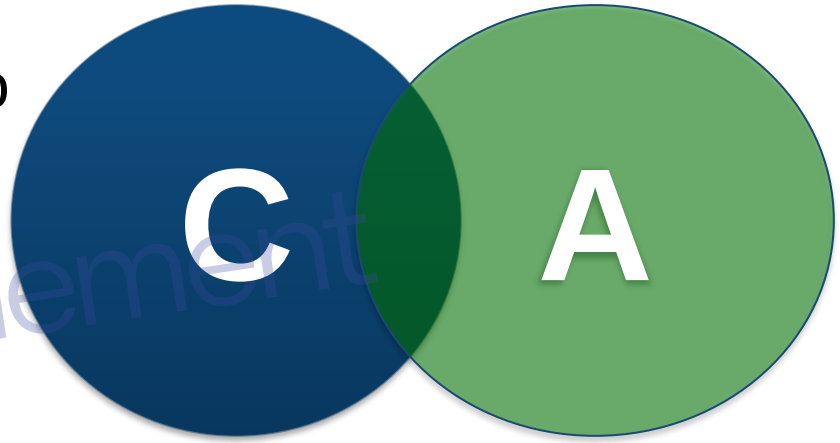
Vreo problemă?

pdfelement

Remove Watermark Now

O opinie greșită: 2 din 3

- Ce spuneți de CA?
- Poate un sistem distribuit (peste o rețea unreliable) să nu fie tolerantă la partiționare?
  - Toleranța la defecte?
  - Dacă tot timpul am nevoie de A, și A se defectează, pot pierde datele?
  - Cu ce diferă atunci sistemul de unul centralizat?



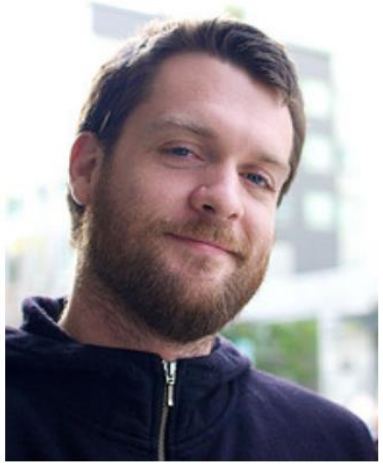
pdfelement

Remove Watermark Now

Ce spun alții...

- Coda Hale, Yammer software engineer:
  - "Of the CAP theorem's Consistency, Availability, and Partition Tolerance, **Partition Tolerance is mandatory in distributed systems**. You cannot not choose it."

pdfelement



http://codahale.com/you-cant-sacrifice-partition-tolerance/

Remove Watermark Now

Ce spun alții...

- Werner Vogels, Amazon CTO
  - "An important observation is that in larger distributed-scale systems, network partitions are a given; therefore, **consistency and availability cannot be achieved at the same time**."

pdfelement

http://www.allthingsdistributed.com/2008/12/eventually_consistent.html

Remove Watermark Now

Teorema CAP, 12 ani mai târziu

- Prof. Eric Brewer: părintele teoremei CAP
  - “Formularea “2 din 3” este greșită deoarece simplifică mult tensiunile dintre proprietăți...
  - **CAP interzice doar o mică parte din proiectarea distribuită**: disponibilitatea și consistența perfecte în prezența partițiilor, care sunt oricum rare.”

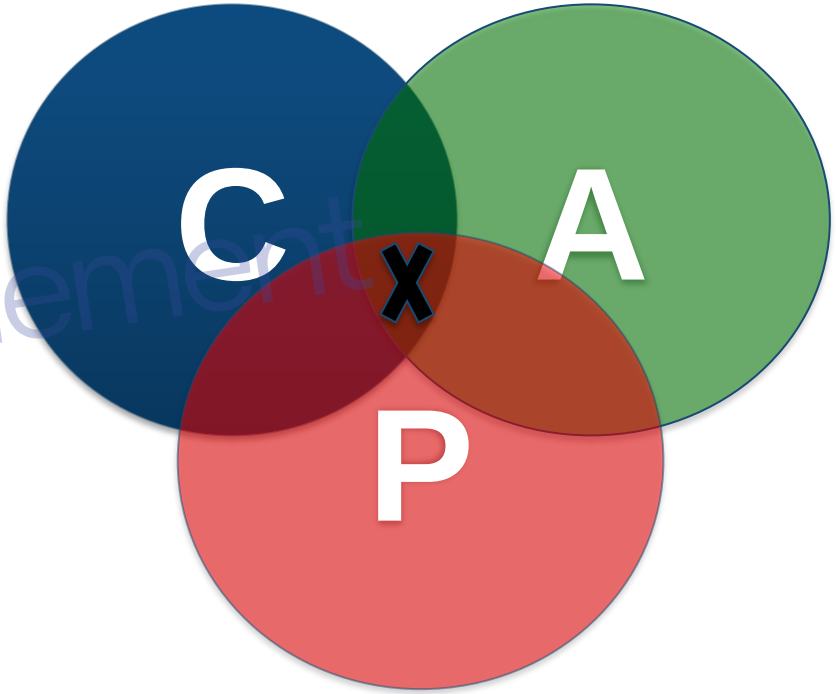
pdfelement

http://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed

Consistență sau Disponibilitate

Remove Watermark Now

- Consistența și Disponibiltatea nu sunt decizii “binare”
- Sisteme AP relaxează consistența în favoarea disponibilității – dar nu sunt inconsiste prin definiție
- Sisteme CP sacrifice disponibilitatea pentru consistență – dar nu sunt indisponibile
- Aceasta sugerează că sisteme atât AP cât și CP, pot oferi un grad de consistență și disponibilitate, precum și toleranță la partiționare



pdfelement

Remove Watermark Now

AP: Best Effort Consistency

- Exemplu:
  - Web Caching
  - DNS
- Abordări ”de mijloc”:
  - Optimistic
  - Expiration/Time-to-live
  - Conflict resolution

pdfelement

Remove Watermark Now

CP: Best Effort Availability

- Exemple:
  - Majority protocols
  - Distributed Locking (Google Chubby Lock service)
- Abordări ”de mijloc”:
  - Pessimistic locking
  - Make minority partition unavailable

pdfelement

Remove Watermark Now

Tipuri de Consistență

- Strong Consistency
  - După ce actualizările au loc, **orice acces ulterior** va returna **aceeași** valoare actualizată
- Weak Consistency
  - **Nu există garanții** că accese successive vor returna valoarea actualizată
- **Eventual Consistency**
  - O formă specifică a weak consistency
  - Oferă garanții că dacă nu se fac **noi actualizări** asupra obiectelor **la un moment dat** (eventually) toate replicile vor returna valoarea actualizată (e.g., propagarea update-urilor către replici într-o manieră lazy)

pdfelement

Remove Watermark Now

Variații ale Eventual Consistency

- Causal consistency
  - Procesele aflate sub o relație cauzală vor vedea date consistente
- Read-your-write consistency
  - Un process ce va accesa datele anterior scrise de el nu va vedea vreodată o valoare neactualizată
- Session consistency
  - Pe durata unei sesiuni, sistemul garantează o consistență read-your-write
  - Garanții oferite între sesiuni ce nu se suprapun

pdfelement

Remove Watermark Now

Variații ale Eventual Consistency

- Monotonic read consistency
  - Dacă un process a văzut o valoare particulară a unei date, orice procese subsecvente nu vor returna valori mai vechi
- Monotonic write consistency
  - Sistemul garantează serializarea scrierilor de către *același* proces
- În realitate
  - Orice combinație de aceste proprietăți poate fi garantată
  - Monotonic reads și read-your-writes sunt cele mai dorite pentru sistemele distribuite

pdfelement

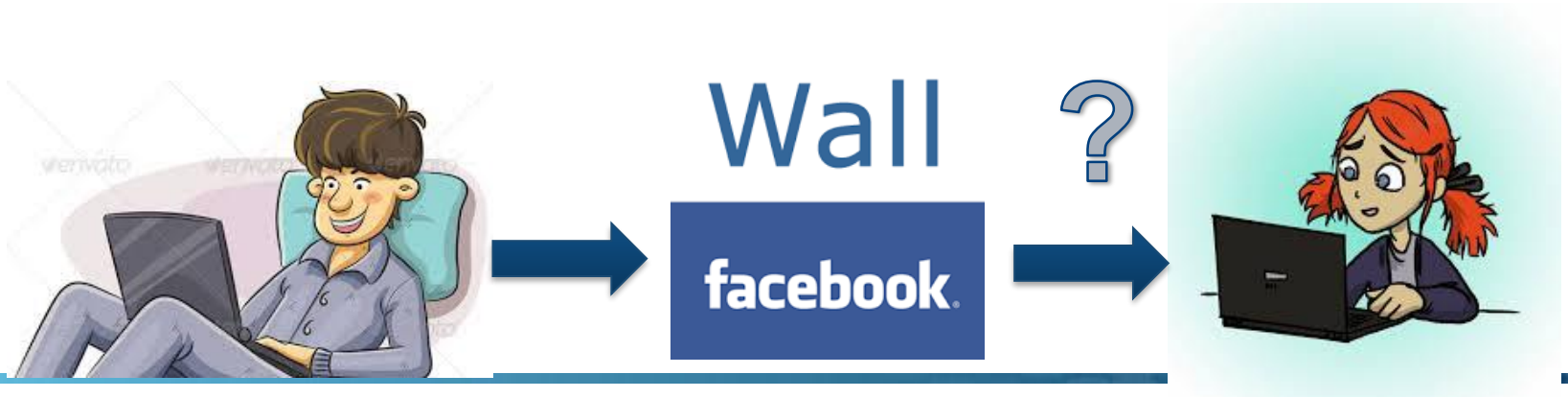
Remove Watermark Now

Eventual Consistency - Un exemplu Facebook

- Bob are o știre interesantă pe care o partajează cu Alice pe wall-ul Facebook
- Bob o roagă pe Alice să verifice ce a postat
- Alice se loghează, verifică wall-ul Facebook și... :

Și nu regăsește nimic acolo!

pdfelement



Remove Watermark Now

Eventual Consistency
- Un exemplu Facebook

- Bob o roagă pe Alice să aștepte un pic și să verifice mai încolo
- Alice așteaptă un minut și se reloghează:

Și găsește postarea pe care Bob a partajat-o cu ea!

pdfelement

Remove Watermark Now

Eventual Consistency - Un exemplu Facebook

- Motivul: este posibil ca Facebook să folosească modelul **eventual consistent**
- De ce alege Facebook modelul eventual consistency în locul unuia de tip strong consistency?
  - Facebook are peste 1 miliard de utilizatori activi
  - Nu este trivial să stocăm eficient și fiabil o cantitate imens de mare de date generată în orice moment de timp
  - Modelul de consistență eventuală oferă opțiunea **reducerii încărcării și crește disponibilitatea serviciilor oferite**

pdfelement

Remove Watermark Now

Eventual Consistency - Un exemplu Dropbox

- Dropbox oferă consistență imediată prin intermediul sincronizării
- Totuși, ce se întâmplă când apare o partiționare peste rețea?

pdfelement

Remove Watermark Now

Eventual Consistency - Un exemplu Dropbox

- Faceți un experiment:
  - Deschideți un fișier în folderul Dropbox
  - Deconectați rețeaua (e.g., WiFi, 4G)
  - Încercați să editați fișierul în folderul Dropbox: puteți?
  - Re-activați conexiunea la rețea: ce se întâmplă cu folderul Dropbox?

pdfelement

Remove Watermark Now

Eventual Consistency - Exemplu Dropbox

- Dropbox se bazează tot pe eventual consistency:
  - Consistența imediată este imposibil de realizat în cazul partiționării rețelei
  - Utilizatorii ar fi nemulțumiți dacă documentele lor Word devin neutilizabile de fiecare data când apasă Ctrl+S , datorită simplei latențe în actualizarea tuturor dispozitivelor peste WAN
  - Dropbox se orientează spre conceptul de **personal syncing**, nu pe colaborare, așa încât asta nu consistența eventuală în cazul lor nu e chiar o limitare reală.

pdfelement

Remove Watermark Now

Eventual Consistency - Un exemplu ATM

- În proiectarea unui automated teller machine (ATM):
  - Consistența puternică (strong) pare a fi o alegere naturală
  - Totuși, în practică, **A beats C**
  - O disponibilitate mărită înseamnă **încasări mai mari**
  - ATM vă permite retragerea de bani, *chiar atunci când mașina este partiționată de rețea*
  - Totuși, **se pune o limită** asupra cantității de bani pe care îi puteți retrage (e.g., $200)
  - Banca chiar vă poate suprataxa atunci când apare o retragere peste disponibil

pdfelement

Remove Watermark Now

Opțiuni în alegerea între C și A

- Un sistem de rezervare bilete de avion:
  - Când majoritatea locurilor sunt disponibile: este ok să ne bazăm pe unele date out-of-date, disponibilitatea fiind mai critică
  - Când avionul se apropie de limita de rezervare: avem nevoie de date actualizate, pentru a ne asigura că nu facem overbooking, consistența devenind mai critică

pdfelement

- Nu se oferă nici consistență puternică nici disponibilitate garantată, dar scopul e să creștem significant toleranța la eventualele probleme la nivelul rețelei

Remove Watermark Now

Eterogenitate: Segmentare între C și A

- Nu există o singură cerință uniformă
  - Unele aspecte necesită consistență strong
  - Altele necesită disponibilitate ridicată
- Segmentarea sistemelor între diverse componente
  - Fiecare furnizează diverse tipuri de garanții
- Per ansamblu sistemul nu garantează nici consistența și nici disponibiliate
  - Fiecare parte a serviciului obține exact ceea ce are nevoie
- O astfel de abordare permite partiționarea de-a lungul mai multor dimensiuni

pdfelement

Remove Watermark Now

Discuție

- Într-un sistem e-commerce (e.g., Amazon, e-Bay, etc), care sunt opțiunile de considerat când alegem între consistență și disponibilitate? Care ar fi strategia voastră?
- Hint -> Lucruri pe care le puteți lua în vizor:
  - Tipuri de date diferite (e.g., shopping cart, billing, product, etc.)
  - Tipuri diferite de operații (e.g., query, purchase, etc.)
  - Tipuri diferite de servicii (e.g., distributed lock, DNS, etc.)
  - Grupuri diferite de utilizatori (e.g., users in different geographic areas, etc.)

pdfelement

Remove Watermark Now

Exemple de scheme de partiționare

- Data Partitioning
- Operational Partitioning
- Functional Partitioning
- User Partitioning
- Hierarchical Partitioning

pdfelement

Remove Watermark Now

Exemple de Scheme de Partiționare

Data Partitioning

- Diverse date pot necesita diverse scheme de consistență și disponibilitate
- Exemplu:
  - Shopping cart: disponibilitate ridicată, timp mic de răspuns, pot suferi câteodată anomalii (clienți nefericiți în cel mai rău caz, dar vor pune lucrurile pe seama unei întâmplări)
  - Informațiile despre produse, în schimb, trebuie să fie disponibile, sunt acceptate doar mici variații în inventar
  - Checkout, billing, shipping records, toate sunt date ce trebuie ținute consistent

pdfelement

Remove Watermark Now

Exemple de scheme de partiționare

Operational Partitioning

- Fiecare operație poate necesita o abordare diferită între consistență și disponibilitate
- Exemple:
  - Reads: disponibilitate ridicată; e.g.., “query”
  - Writes: consistență ridicată, lock la writing; e.g., “purchase”

pdfelement

Remove Watermark Now

Exemple de scheme de partiționare

Functional Partitioning

- Sistemul constă dintr-o colecție de sub-servicii
- Diversele sub-servicii au nevoi diferite
- Exemplu: Un sistem distribuit complet
  - Distributed lock service (e.g., Chubby) :
    - Strong consistency
  - DNS service:
    - High availability

pdfelement

Remove Watermark Now

Exemple de scheme de partiționare

User Partitioning

- E nevoie să menținem date înrudite aproape între ele pentru garantarea performanței la procesare
- Exemplu: Craglist
  - E posibil să dorim divizarea serviciilor între câteva Data Centers,e.g., east coast / west coast
    - Utilizatorii obțin performanță ridicată (e.g., disponibilitate ridicată și consistență bună) dacă interoghează servere apropiate de ei
    - Performanță slabă dacă un utilizator din New Yor interoghează serviciul din San Francisco

pdfelement

Remove Watermark Now

Exemple de scheme de partiționare

Hierarchical Partitioning

- Servicii globale ce au “extensii” locale
- Diverse locații în ierarhie pot folosi diverse modele de consistență
- Exemplu:
  - Servere locale (mai bine conectate cu utilizatorul) oferă mai multă consistență și disponibilitate
  - Servere globale sunt mai partiționate și relaxează una dintre proprietăți

pdfelement

Remove Watermark Now

Ce se întâmplă când nu avem deloc partiționare?

- Balans între **Consistență** și **Latență**:
- cauzat de **posibilitatea de apariție a defectelor** în sisteme distribuite
  - Disponibilitate ridicată -> date replicate -> probleme de consistență
- Ideea de bază:
  - Disponibilitatea și latența sunt aproape **același lucru**: indisponibilitatea = latență f. mare
  - Atingerea unor diverse nivele de consistență/disponibilitate necesită timpi diferit

pdfelement

Remove Watermark Now

CAP -> PACELC

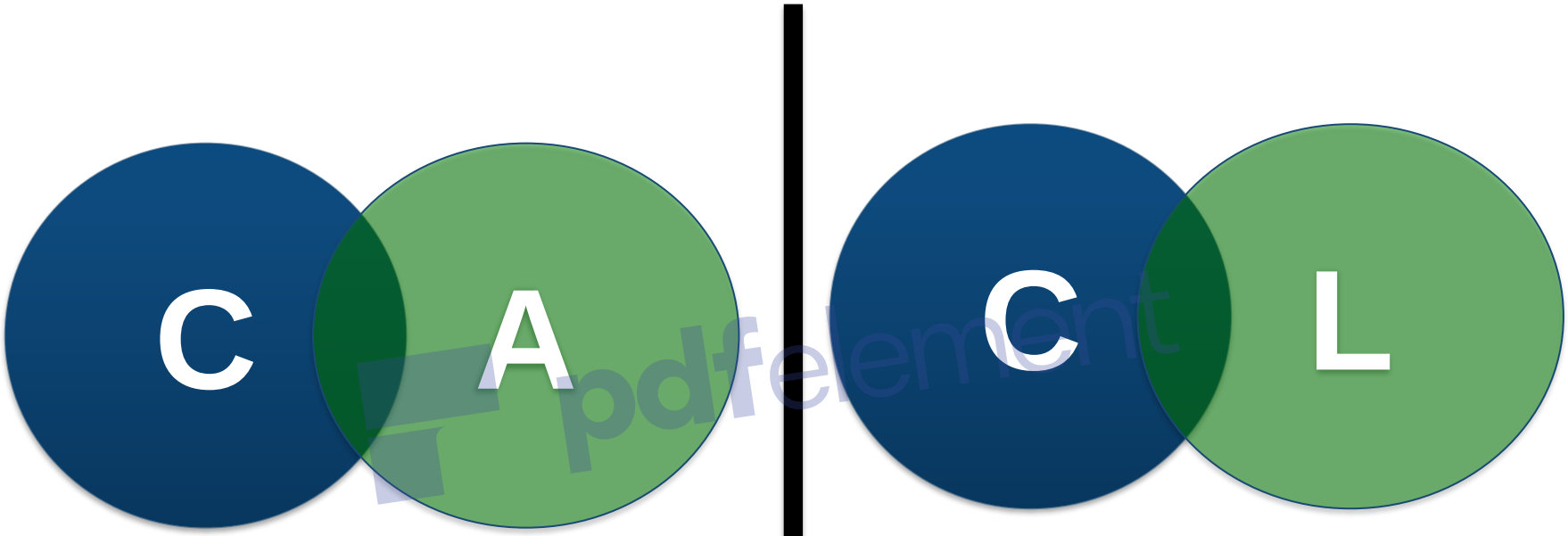
- O descriere mai completă a spațiului unor potențiale alegeri pentru un sistem distribuit:
  - Dacă există o **partiție (P)**, cum ajunge sistemul să aleagă între **availability și consistency (A și C)**; **altfel (else, E)**, când sistemul rulează în parametri normali în absența partiționării, cum ajunge sistemul să aleagă între **latency (L) și consistency (C)**?

pdfelement

Abadi, Daniel J. "Consistency tradeoffs in modern distributed database system design." Computer-IEEE Computer Magazine 45.2 (2012): 37.

PACELC

Remove Watermark Now



pdfelement

Partitioned

Normal

Remove Watermark Now

Exemple

- **Sisteme PA/EL:** Renunță la ambele C-uri pentru availability și lower latency
  - Dynamo, Cassandra, Riak
- **Sisteme PC/EC:** Refuză să renunțe la Consistență în detrimentul unui cost scump pentru disponibilitate și latență
  - BigTable, Hbase, VoltDB/H-Store
- **Sisteme PA/EC:** Renunță la Consistență atunci când apare o partiționare și păstrează Consistența doar în condiții normale de operare
  - MongoDB
- **Sisteme PC/EL:** Păstrează Consistența dacă apare o partiționare, dar renunță la consistență în cazul operării normale
  - Yahoo! PNUTS

pdfelement

Remove Watermark Now

Algoritmi Paraleli și Distribuiți
Algoritmi peer-to-peer

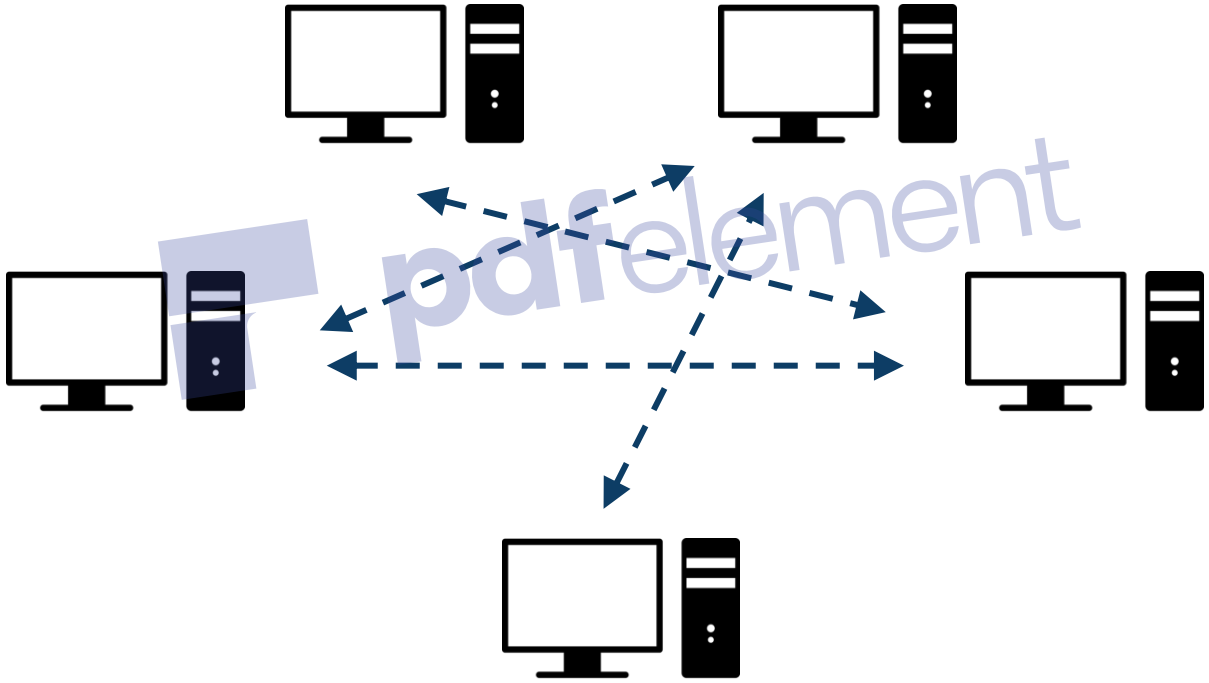
Prof. Ciprian Dobre
ciprian.dobre@cs.pub.ro

pdfelement

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Remove Watermark Now

Peer-to-peer (P2P)

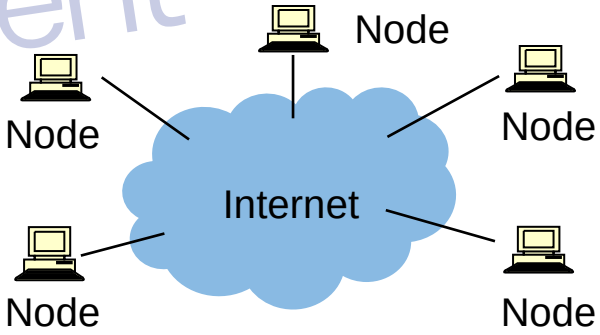


pdfelement

Remove Watermark Now

Ce este un sistem Peer-to-Peer (P2P)?

- O arhitectura de sistem distribuit in care:
  - **Nu exista un control centralizat**
  - Nodurile sunt aproape **simetrice** in functionalitate
- Numar mare de **noduri unreliable**
- **Capacitate mare de deservire a serviciilor**, prin paralismul adus de:
  - Multe discuri
  - Multe legaturi de retea
  - Multe CPI-uri
- **Absenta unui server centralizat** inseamna:
  - Mai putine sanse de overload
  - Deployment mai usor
  - Un defect nu poate strica intregul sistem
  - Sistemul per ansamblu e mai greu de atacat



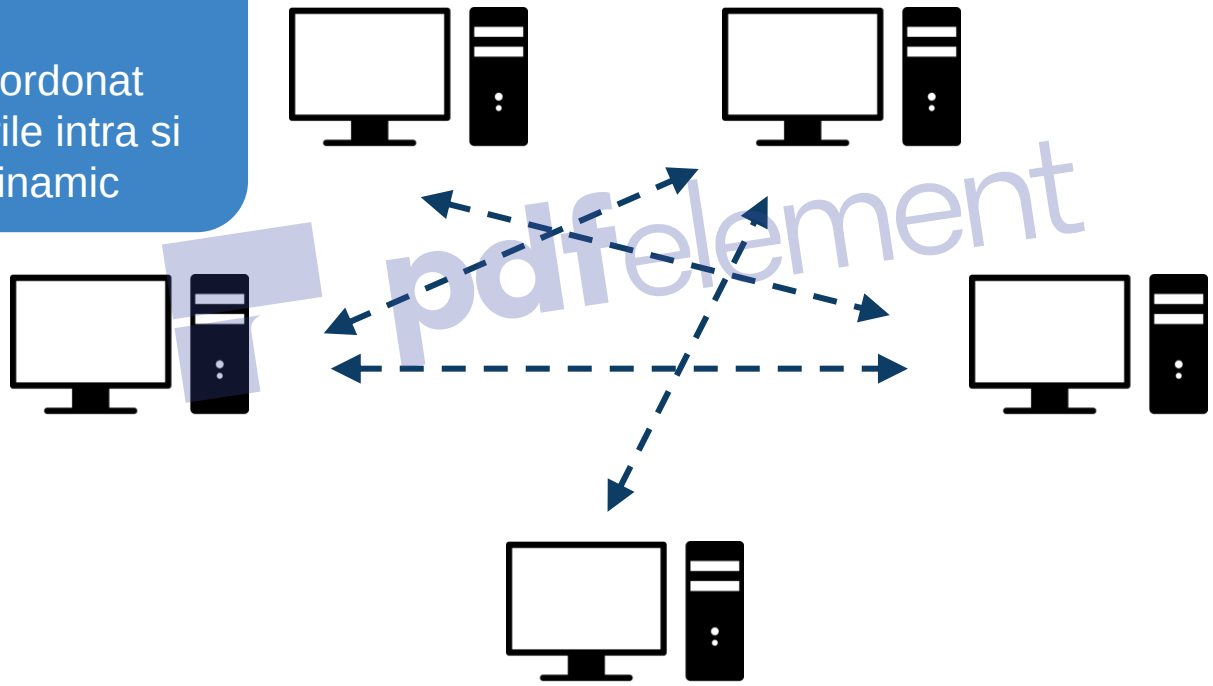
pdfelement

Remove Watermark Now

Peer-to-peer (P2P)

Descentralizare!

Greu de coordonat cand nodurile intra si parasesc dinamic

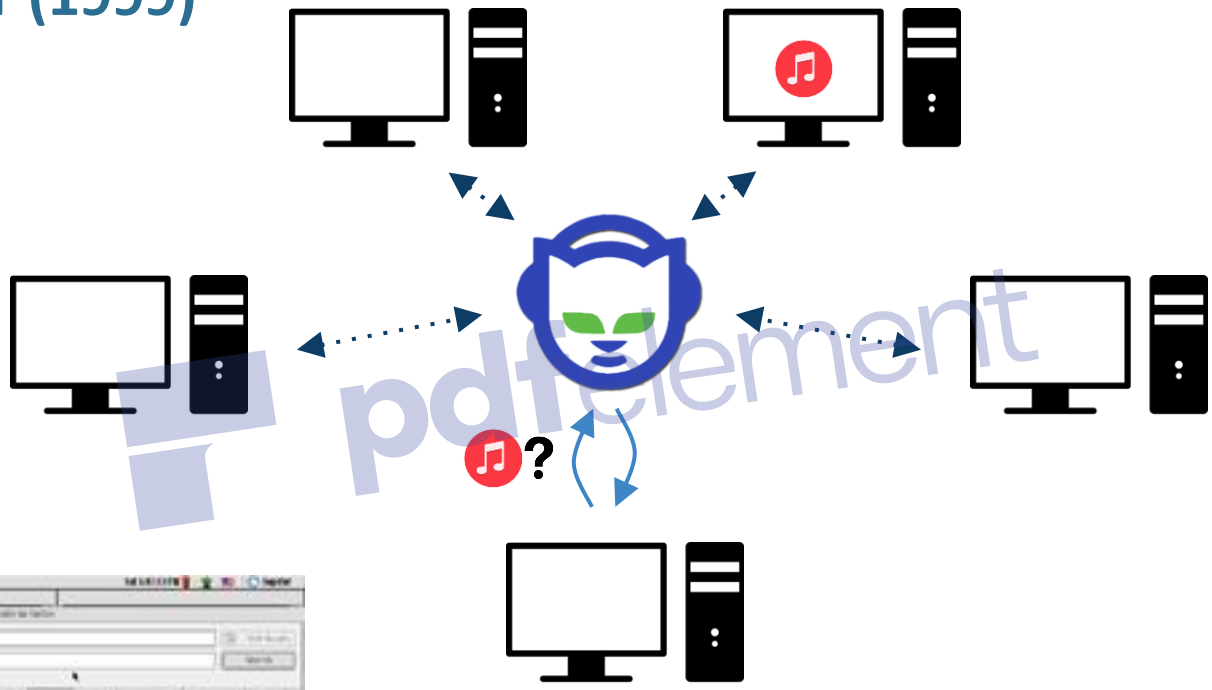


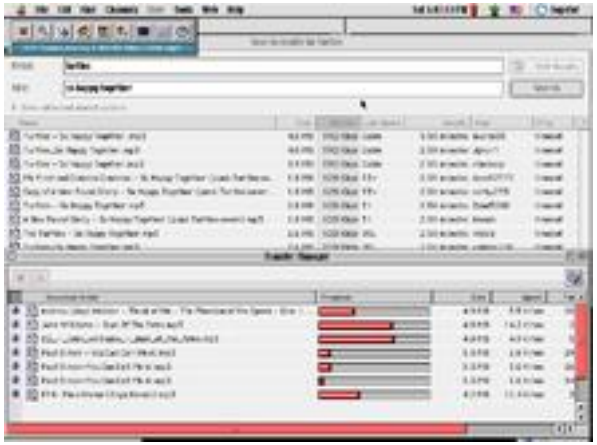
pdfelement

Remove Watermark Now

Peer-to-peer (P2P)

Napster (1999)

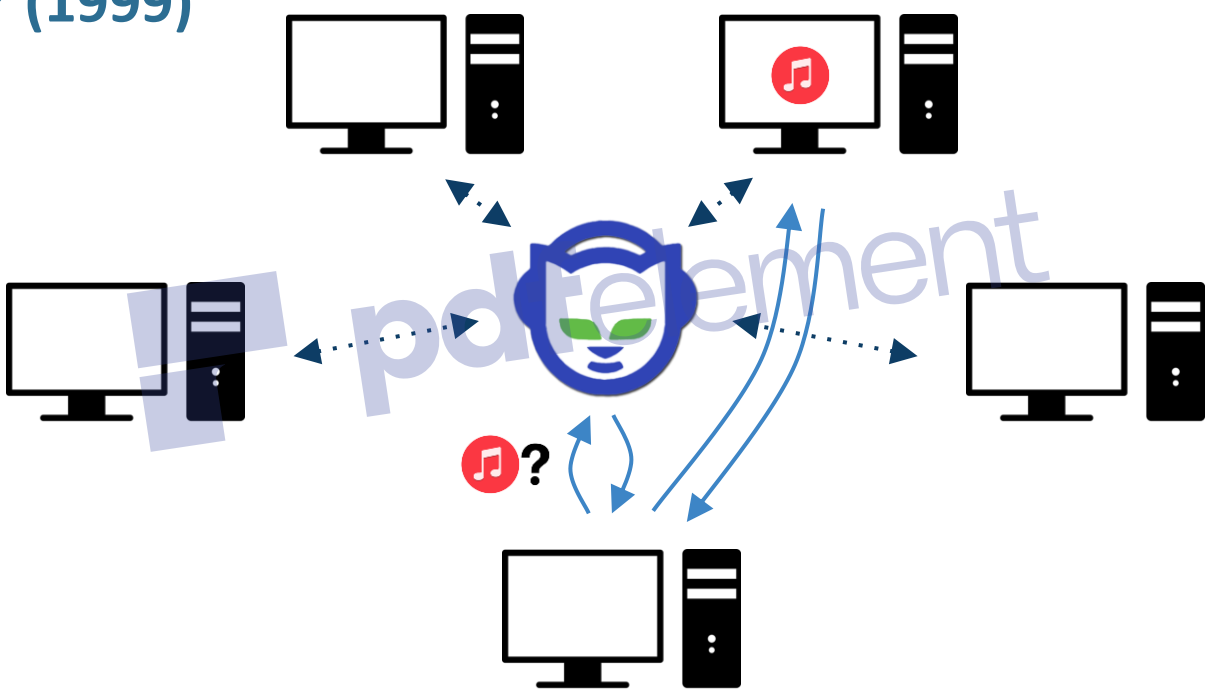




Remove Watermark Now

Peer-to-peer (P2P)

Napster (1999)

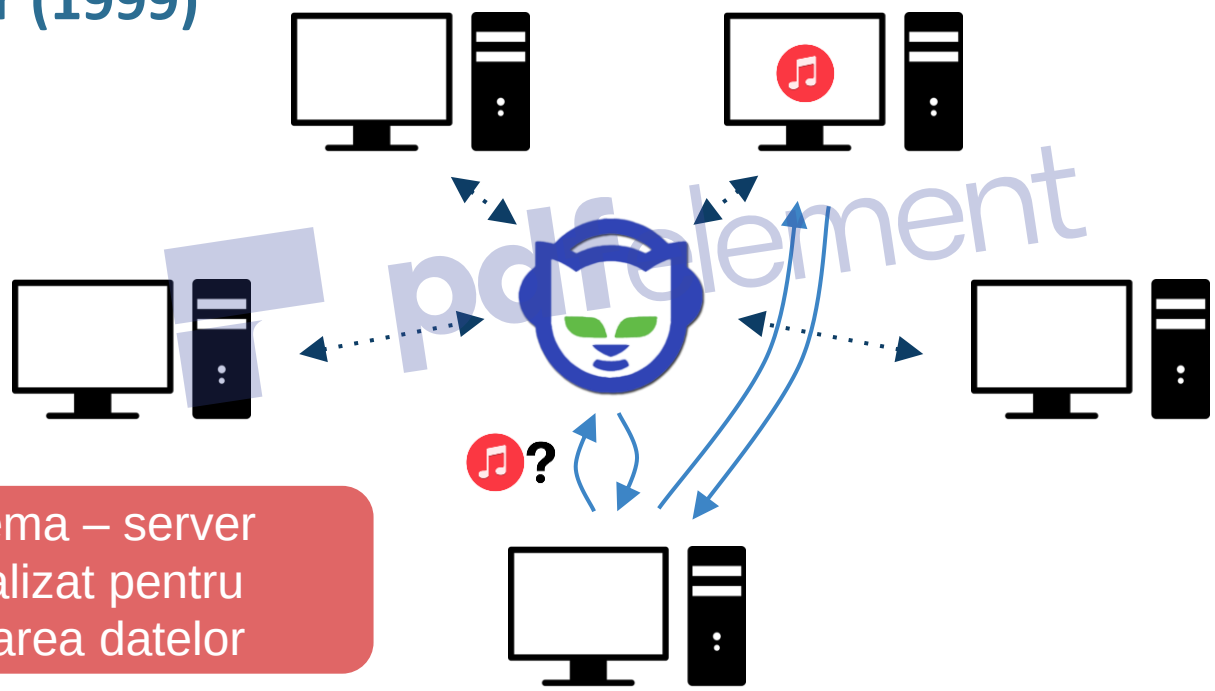


pdfelement

Remove Watermark Now

Peer-to-peer (P2P)

Napster (1999)



Problema – server centralizat pentru indexarea datelor

Remove Watermark Now

This article is more than **19 years old**

Napster loses net music copyright case

A federal judge in San Francisco yesterday ordered Napster, the internet service that allows the trading of MP3 sound files by linking personal computers, to stop permitting the exchange of copyrighted music owned by major music labels.

The preliminary injunction, which could effectively shut down Napster's service, represents a major victory for the world's five largest record companies who are suing Napster for copyright infringement.

In granting the industry's request for a preliminary injunction, Chief Judg Marilyn Patel of the US district court in San Francisco ruled that the huge popular Napster service, with an estimated 20m users, was used primaril trade copyrighted material with the full knowledge of Napster.

"A majority of Napster users use the service to download and upload copyrighted music," she said.

The injunction is scheduled to take effect at midnight on Friday and will continue pending trial.

Advertisement

Spread your content, not your workflow.

Metallica sued Napster 15 years ago today

File sharing forever

By Nilay Patel | @reckless | Apr 13, 2015, 10:39am EDT

SHARE

April 13th, 2000. The day the music industry and the internet became best frenemies forever.

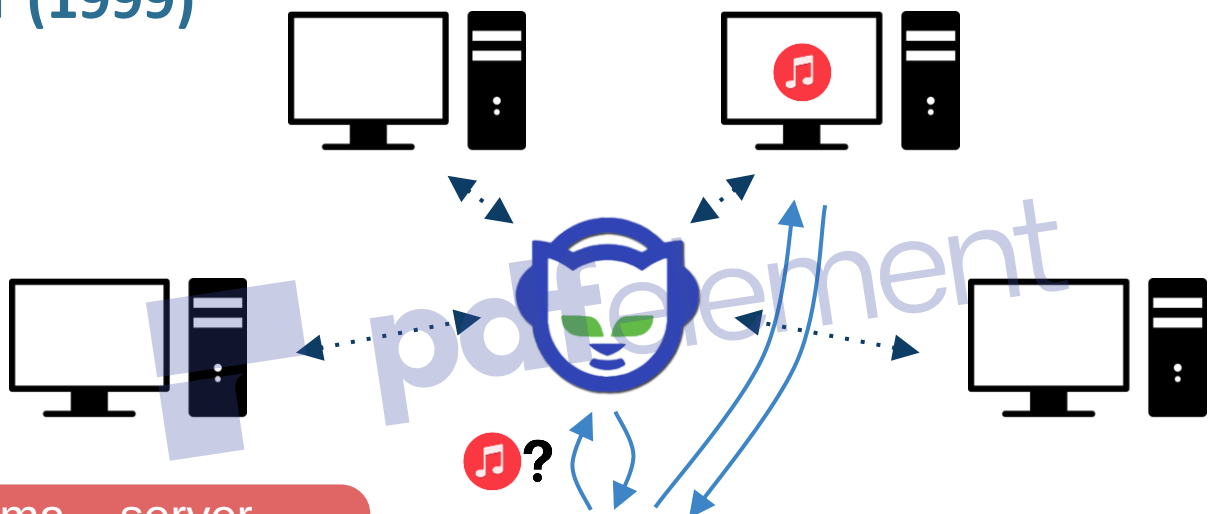
That's the day *Metallica v. Napster, Inc.* was filed in the United States District Court for the

pdfelement

Remove Watermark Now

Peer-to-peer (P2P)

Napster (1999)



Problema – server centralizat pentru indexarea datelor

Solutia - Distributed hash table

gnutella

KaZaA

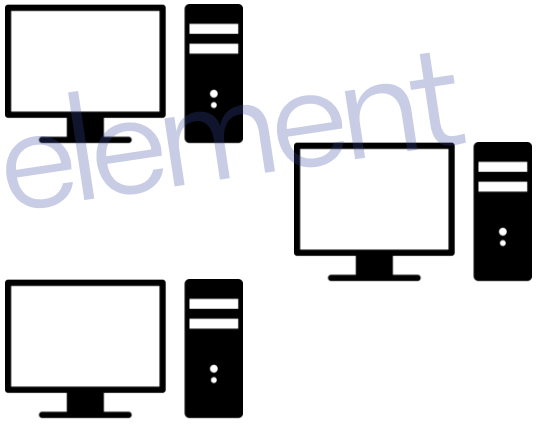
limewire

Remove Watermark Now

Distributed Hash Tables (DHT)

Hash table

k_0	v_0
k_1	v_1
k_2	v_2
k_3	v_3
k_4	v_4
k_5	v_5



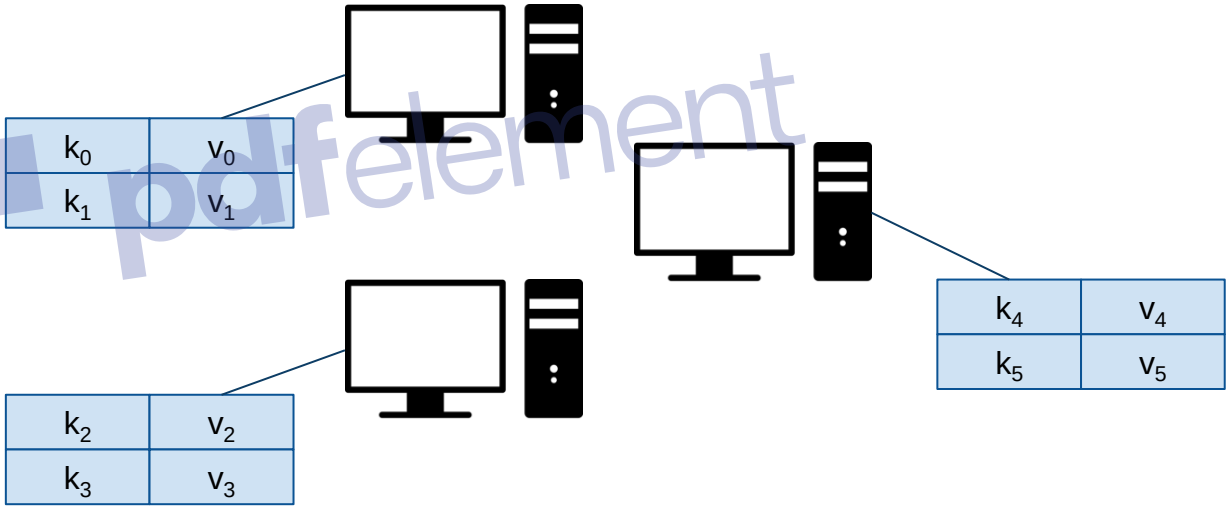
pdfelement

Remove Watermark Now

Distributed Hash Tables (DHT)

Hash table

k_0	v_0
k_1	v_1
k_2	v_2
k_3	v_3
k_4	v_4
k_5	v_5



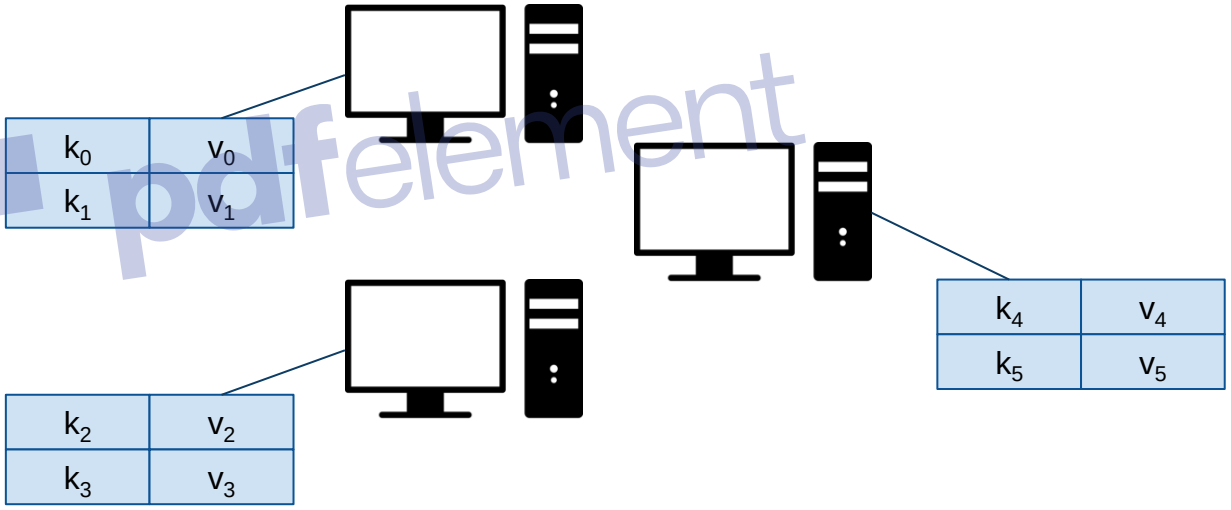
pdfelement

Remove Watermark Now

Distributed Hash Tables (DHT)

Proprietati dorite:

- Descentralizare
- Load-balancing
- Scalabilitate
- Disponibilitate



pdfelement

Remove Watermark Now

Distributed Hash Tables

- *Distributed Hash Table:*

```
key = hash(data)
lookup(key) → IP addr
send-RPC(IP address, put, key, data)
send-RPC(IP address, get, key) → data
```

(Chord lookup service)

pdfelement

- **Partitionam datele** peste sisteme distribuite pe scara larga
  - Punem tupluri intr-un global database engine
  - si blocurile de date intr-un sistem global de fisiere
  - peste un P2P file-sharing system

*RPC = Remote Procedure Call

Remove Watermark Now

BitTorrent peste DHT

- BitTorrent poate folosi DHT in loc de (sau in conjuctie cu un) tracker

- Clientii BT folosesc DHT:
  - Key = **file content hash** (“infohash”)
  - Value = Adresa **IP a peer-ului** ce poate servi fisierul
    - Poate stoca mai multe valori (*i.e.* IP addresses) pentru o cheie

- Clientul apeleaza:
  - `get(infohash)` pentru a gasi alti clienti ce pot servi fisierul
  - `put(infohash, my-ipaddr)` pentru a se identifica ca detinator al fisierului

pdfelement

Remove Watermark Now

Chord: un DHT performant

pdfelement

Remove Watermark Now

Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications

Ion Stoica* Robert Morris† David Liben-Nowell†
David Karger† M. Frans Kaashoek† Frank Dabek†
Hari Balakrishnan†

January 10, 2002

Abstract

A fundamental problem that confronts peer-to-peer applications is to efficiently locate the node that stores a particular data item. This paper presents *Chord*, a distributed lookup protocol that addresses this problem. Chord provides support for just one operation: given a key, it maps the key onto a node. Data location can be easily implemented on top of Chord by associating a key with each data item, and storing the key/data item pair at the node to which the key maps. Chord adapts efficiently as nodes join and leave the system, and can answer queries even if the system is continuously changing. Results from theoretical analysis and simulations show that Chord is scalable, with communication cost and the state maintained by each node scaling logarithmically with the number of Chord nodes.

pdfelement

1 Introduction

Peer-to-peer systems and applications are distributed systems without any centralized control or hierarchical organization, where the software running at each node is equivalent in functionality. A review of the features of recent peer-to-peer applications yields a long list: redundant storage, permanence, selection

*University of California, Berkeley. istoica@cs.berkeley.edu
†MIT Laboratory for Computer Science, {rtm, dln, karger, kaashoek, fdabek, hari}@lcs.mit.edu.

Authors in reverse alphabetical order.

Stoica, I., Morris, R., Karger, D., Kaashoek, M. F., & Balakrishnan, H. (2001). Chord: A scalable peer-to-peer lookup service for internet applications. *ACM SIGCOMM Computer Communication Review*, *31*(4), 149-160.

Remove Watermark Now

Algoritmul Chord pentru lookup

- **Interfata:** lookup(key) → IP address
- **Eficient:** O(log N) mesaje pentru operatia de lookup
  - N fiind numarul total de servere
- **Scalable:** O(log N) stari per nod
- **Robust:** poate supravietui unui numar mare de noduri defecte

pdfelement

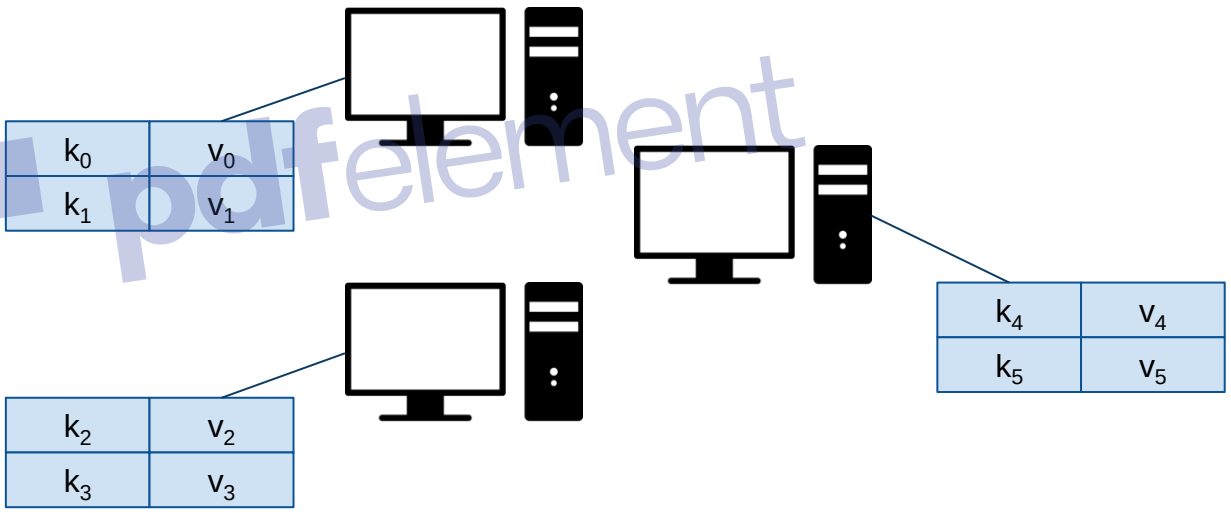
Remove Watermark Now

Chord – DHT eficient

Cum asignam chei nodurilor peer?

Hash table

k_0	v_0
k_1	v_1
k_2	v_2
k_3	v_3
k_4	v_4
k_5	v_5



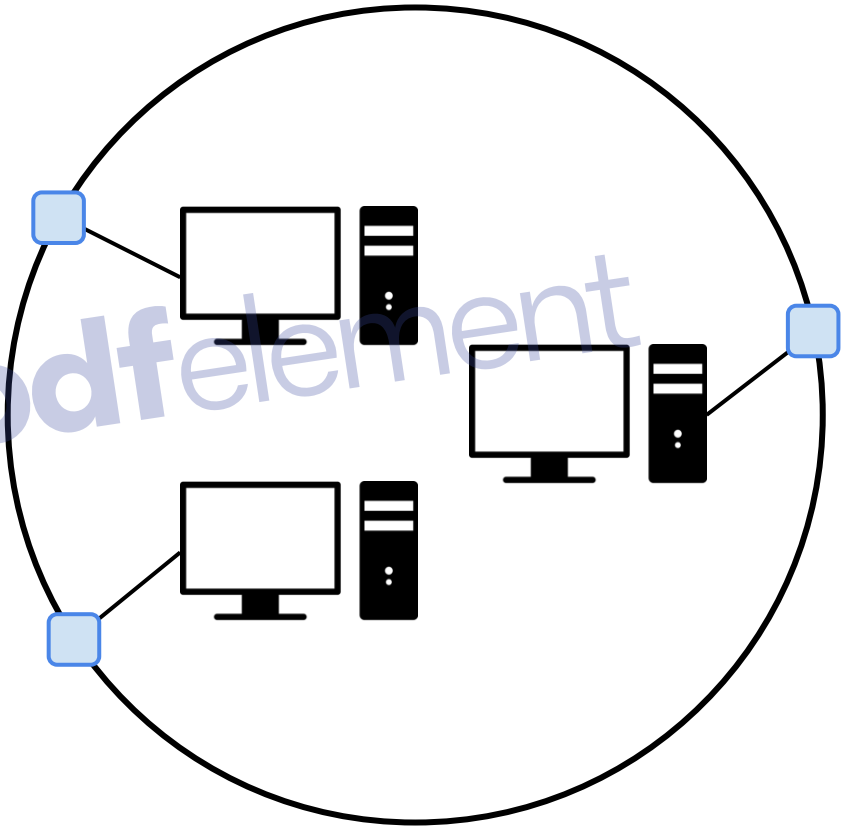
pdfelement

Remove Watermark Now

Chord

Hash table

k_0	v_0
k_1	v_1
k_2	v_2
k_3	v_3
k_4	v_4
k_5	v_5



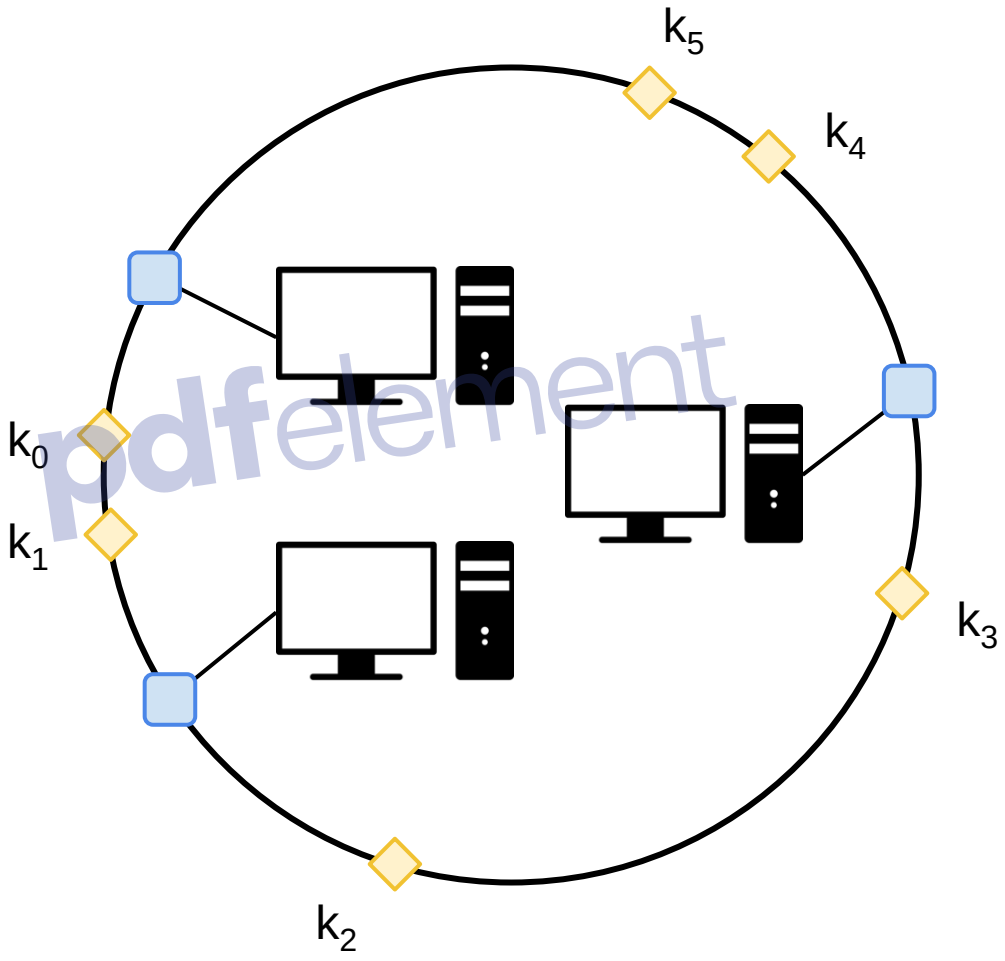
pdfelement

Remove Watermark Now

Chord

Hash table

k_0	v_0
k_1	v_1
k_2	v_2
k_3	v_3
k_4	v_4
k_5	v_5



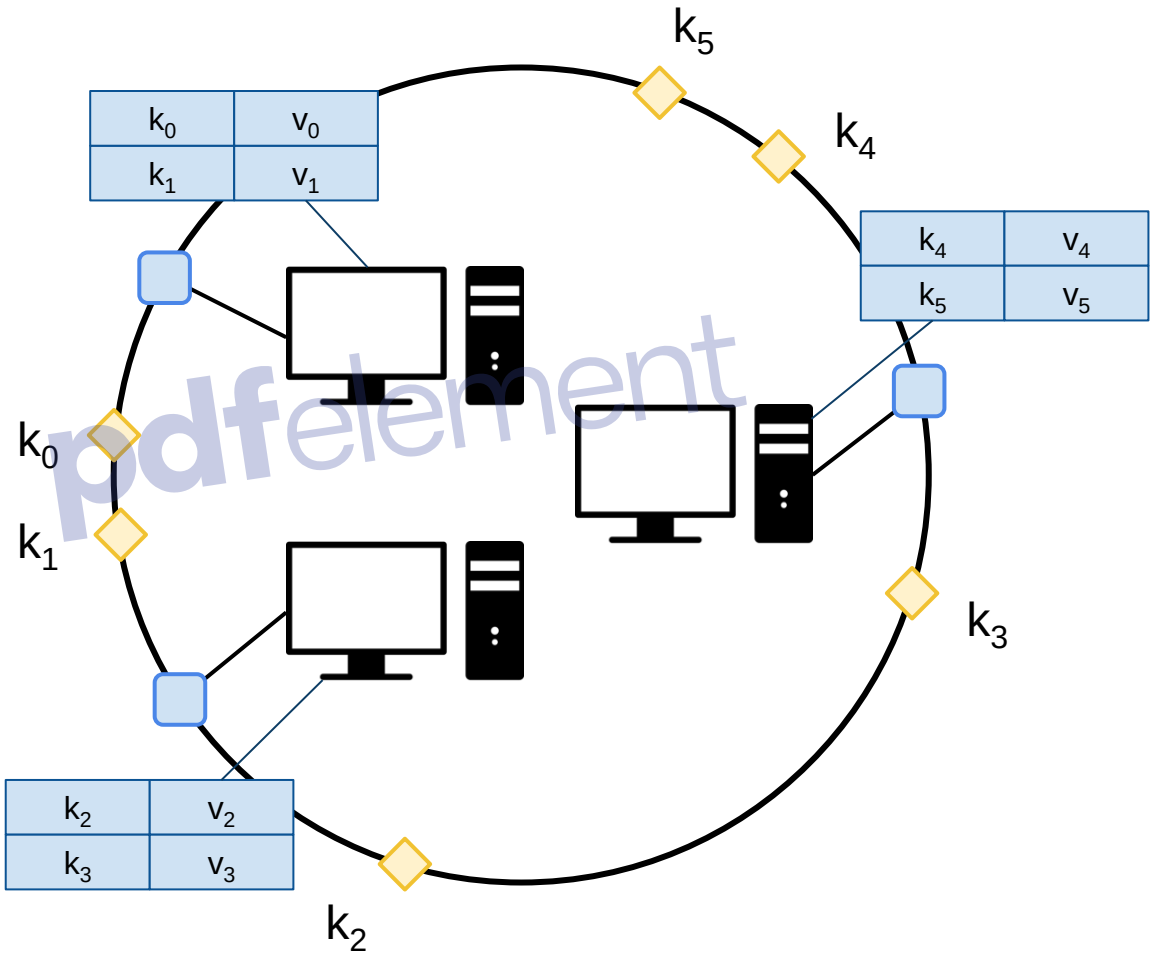
pdfelement

Remove Watermark Now

Chord

Hash table

k_0	v_0
k_1	v_1
k_2	v_2
k_3	v_3
k_4	v_4
k_5	v_5



pdfelement

Remove Watermark Now

Identificatori Chord

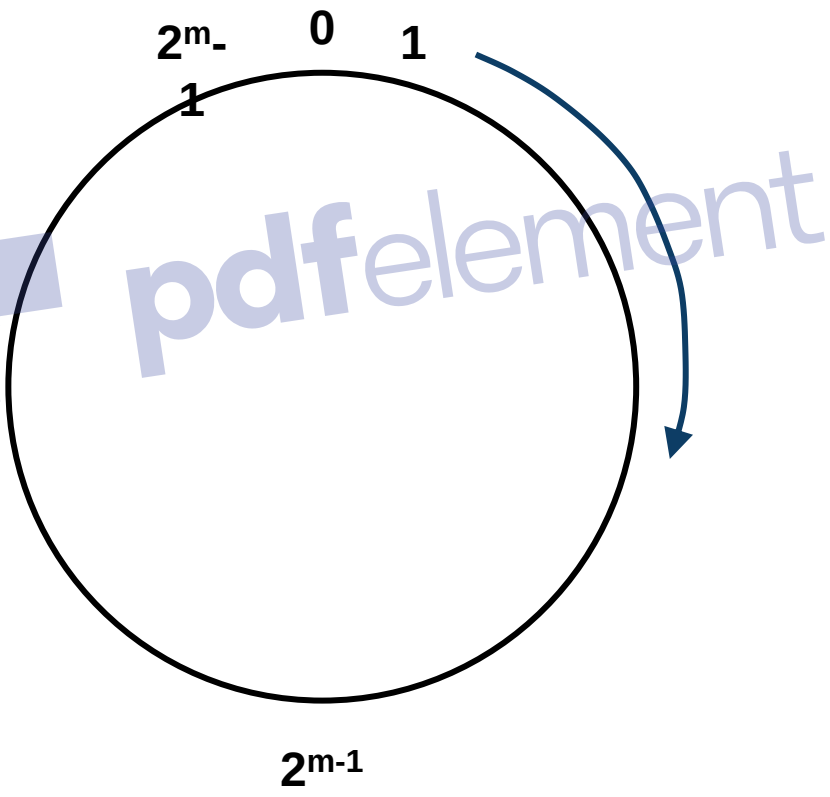
- **Key identifier** = SHA-1(key)
- **Node identifier** = SHA-1(IP address)
- SHA-1 asigura o distributie uniforma a ambelor
- ***Cum partitioneaza Chord datele?***
  - *i.e.*, map key IDs to node IDs

pdfelement

Remove Watermark Now

Chord (distributia cheilor)

Inel de identificatori peste spatiul hash 2^m

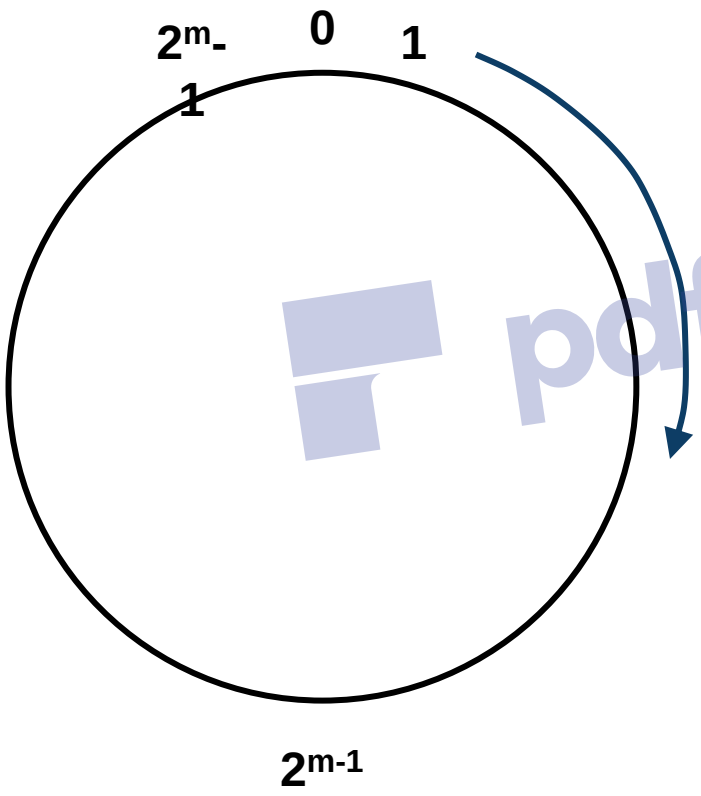


pdfelement

Remove Watermark Now

Chord (distributia cheilor)

Inel de identificatori peste spatiul hash 2^m



= node

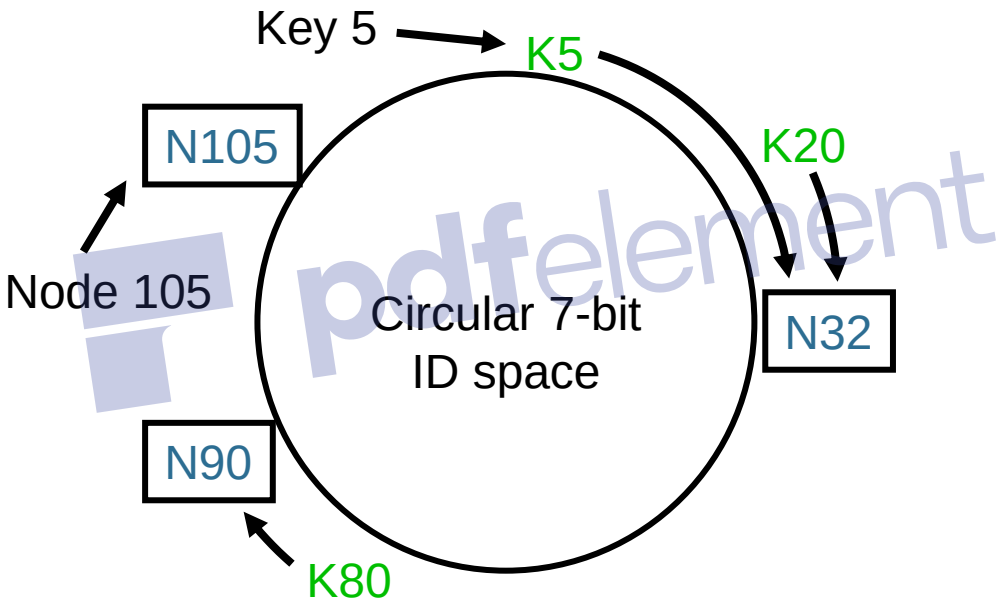
= key

node id = hash(node)
key id = hash(key)

pdfelement

Remove Watermark Now

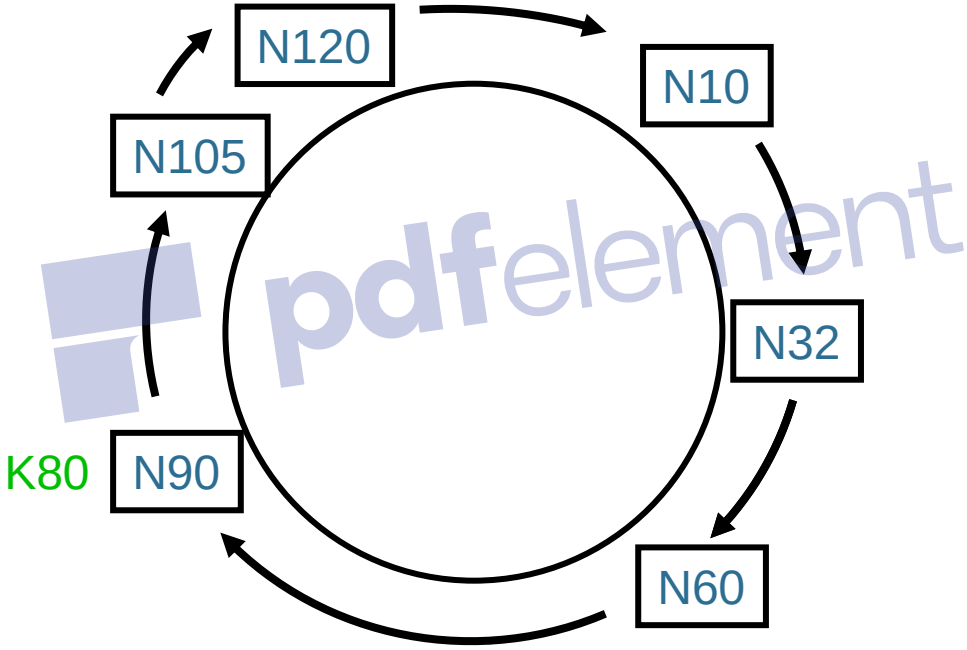
Consistent hashing [Karger ‘97]



Cheia este stocata la successor: nodul avand urmatorul cel mai mare ID

Remove Watermark Now

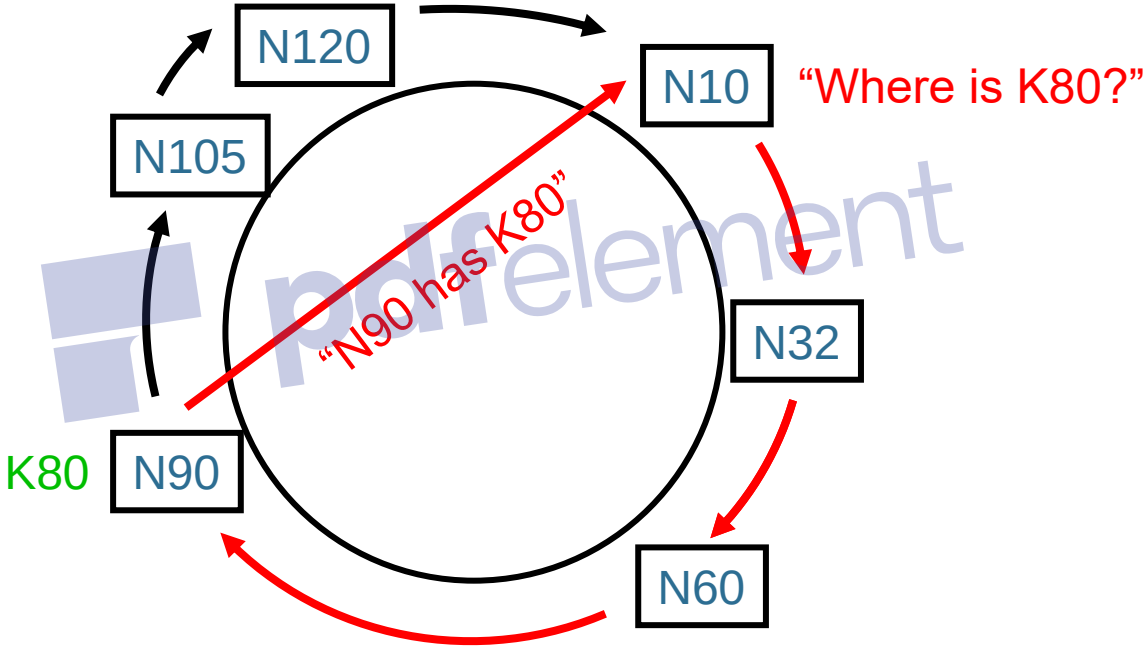
Chord: Successor pointers



pdfelement

Remove Watermark Now

Basic lookup



pdfelement

Remove Watermark Now

Simple lookup algorithm

```
Lookup(key-id)
  succ ← my successor
  if my-id < succ < key-id //next hop
        call Lookup(key-id) on succ
  else                                    //done
    return succ
```

- **Corectitudinea** depinde doar de **succesori**

pdfelement

Remove Watermark Now

Imbunatatirea performantei

- **Problema:** Forwardarea prin intermediul succesorului poate fi inceata

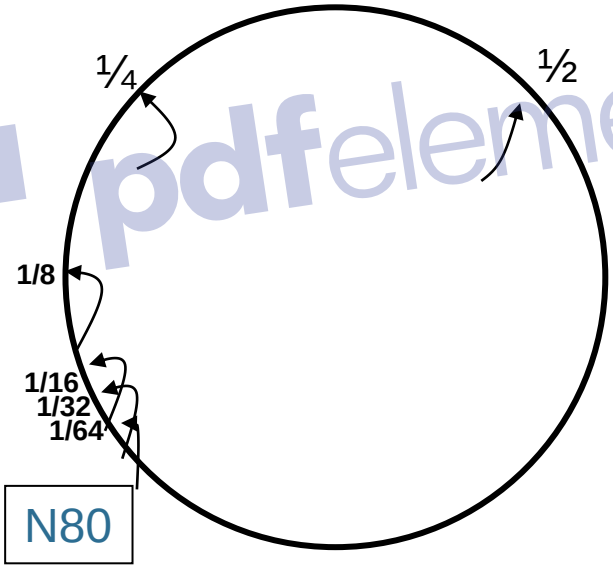
- Stuctura de date este o lista inlantuita: O(n)

pdfelement

- **Idee:** Putem sa facem operatia sa arate mai mult a **binary search**?
  - Adica sa imbunatatim distanta cu fiecare pas

Remove Watermark Now

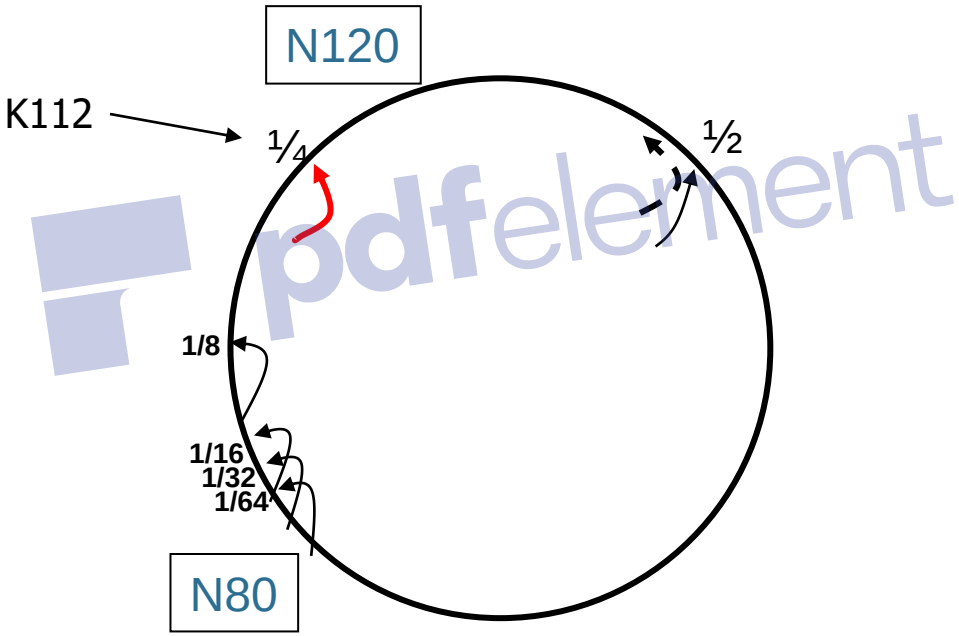
“Finger table” permite operatii de lookup in timp log N



pdfelement

Remove Watermark Now

Finger i redirecteaza catre succesorul lui $n+2^i$



pdfelement

Remove Watermark Now

Implicatii ale tabelei finger

- Avem un **binary lookup tree** cu fiecare nod
  - Cu ramuri redirectate catre alte tabele finger ale celorlalte noduri
- Este mai bine decat sa aranjam nodurile intr-un singur arbore
  - Fiecare nod actioneaza ca radacina a unui arbore
    - Deci nu exista niciun **root hotspot**
    - Nu exista **single point of failure**

pdfelement

Remove Watermark Now

Lookup with finger table

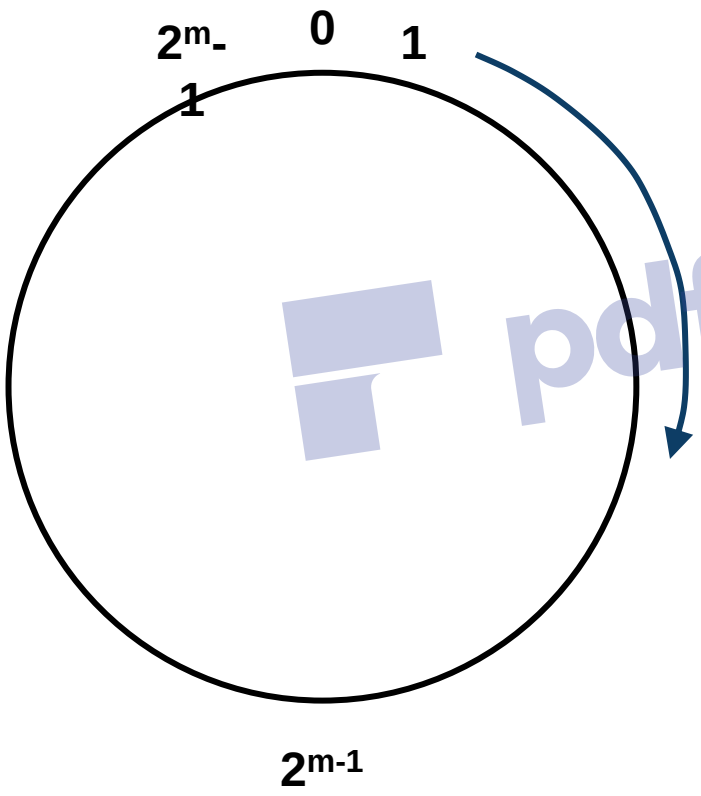
```
Lookup(key-id)
  look in local finger table for
        highest n: my-id < n < key-id
  if n exists
        call Lookup(key-id) on node n //next hop
  else
        return my successor    //done
```

pafelement

Remove Watermark Now

Chord

Inel de identificatori peste spatiul hash 2^m



= node

= key

node id = hash(node)
key id = hash(key)

successor(id)

finger table for node at id *i*

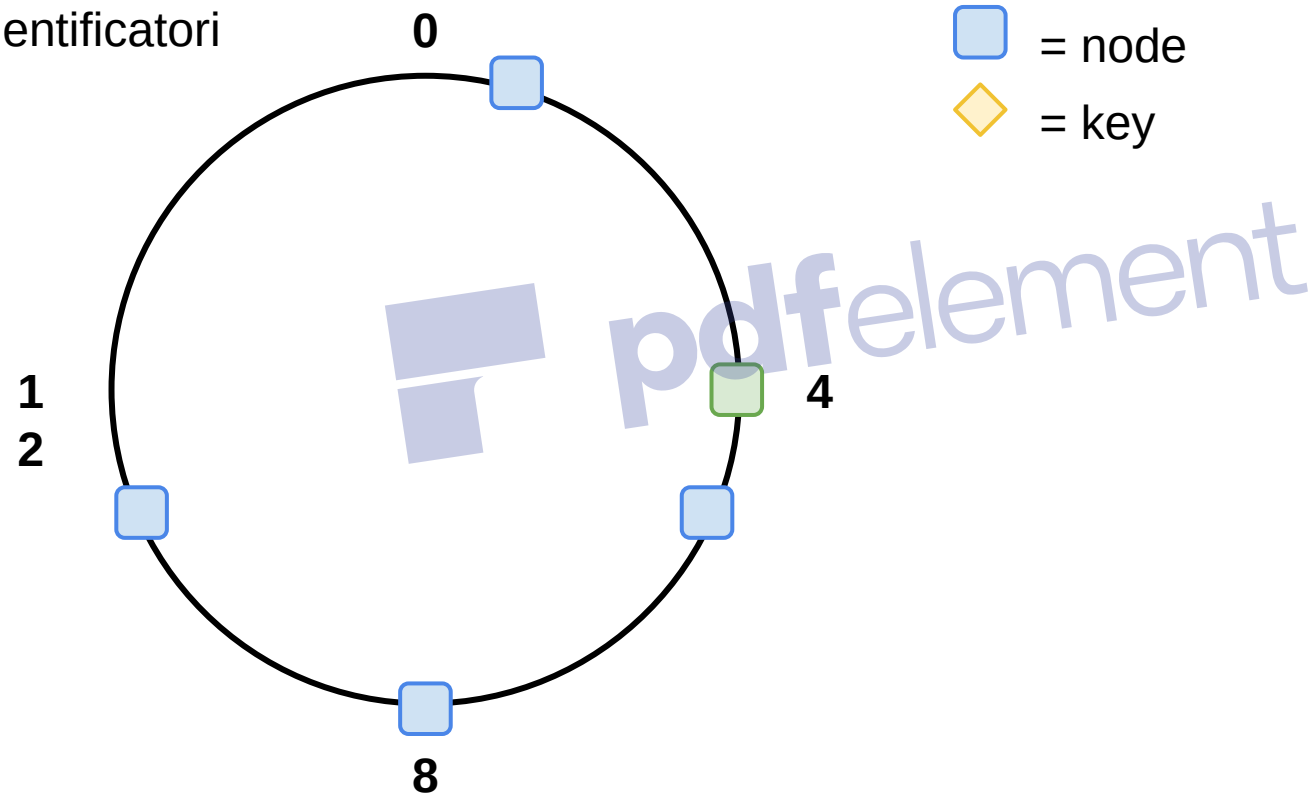
finger	node id
1	succ(*i*)
2	succ(*i* + 2)
j	succ(*i* + 2^{j-1})

pdfelement

Remove Watermark Now

Chord

Inelul de identificatori

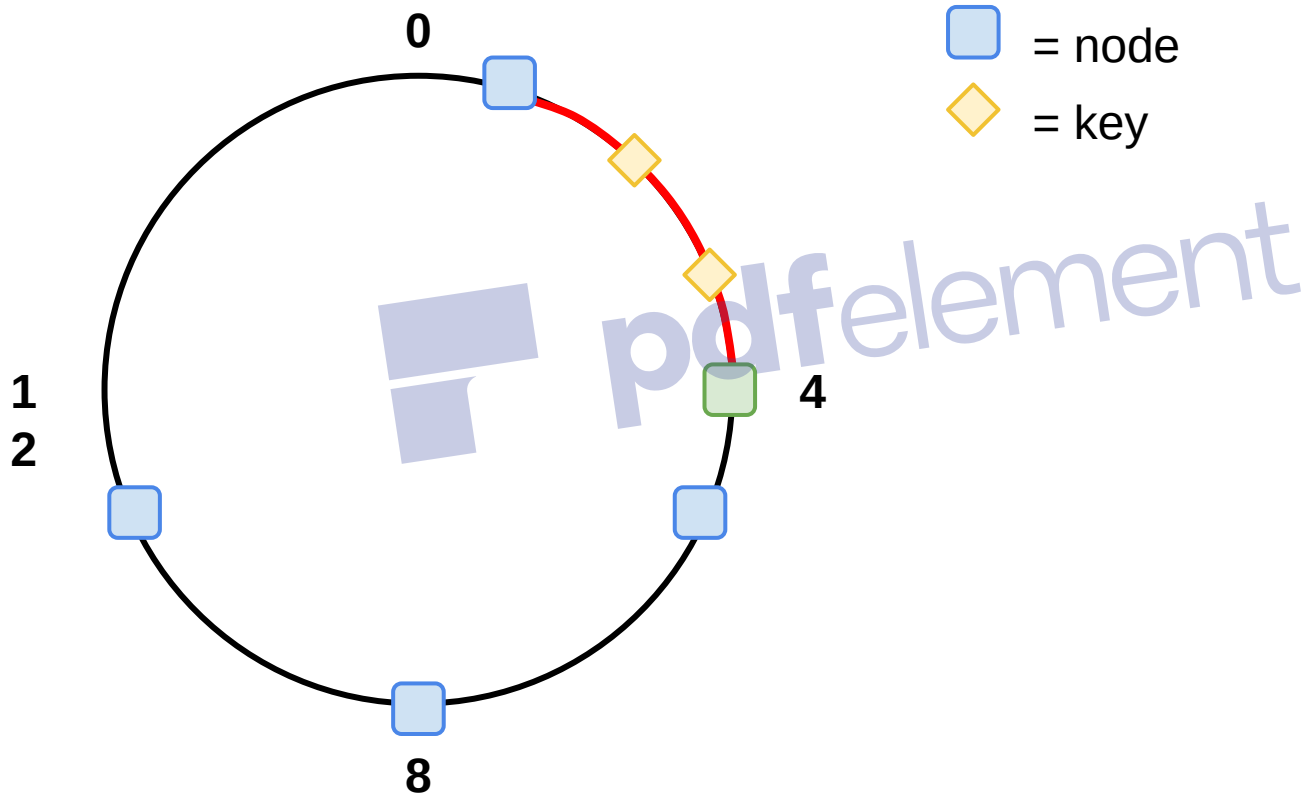


pdfelement

Remove Watermark Now

Chord

Inel de identificatori

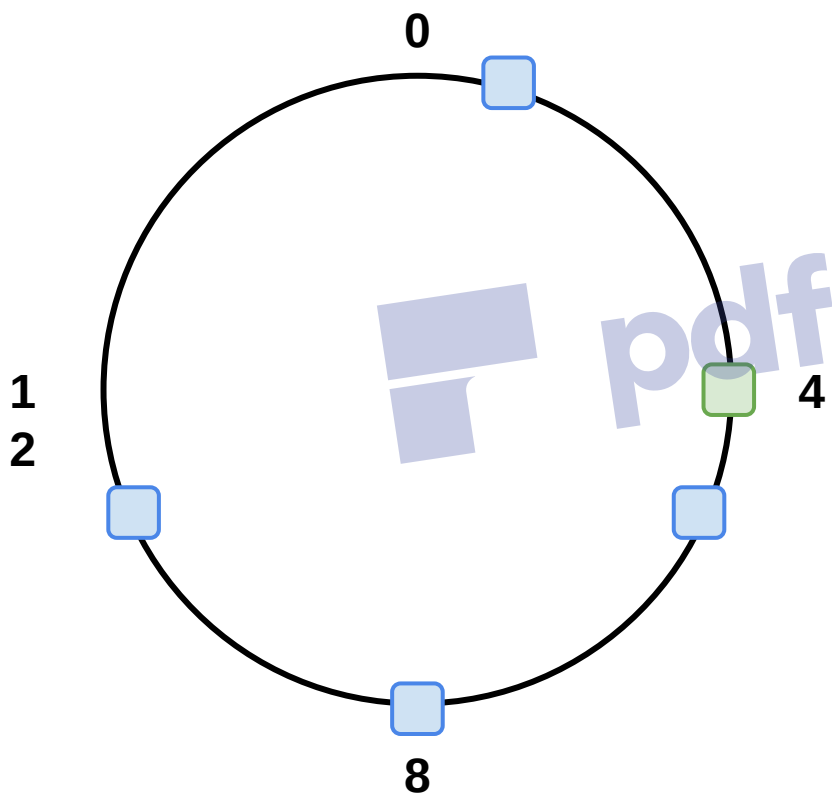


pdfelement

Remove Watermark Now

Chord

Inel de identificatori



= node

= key

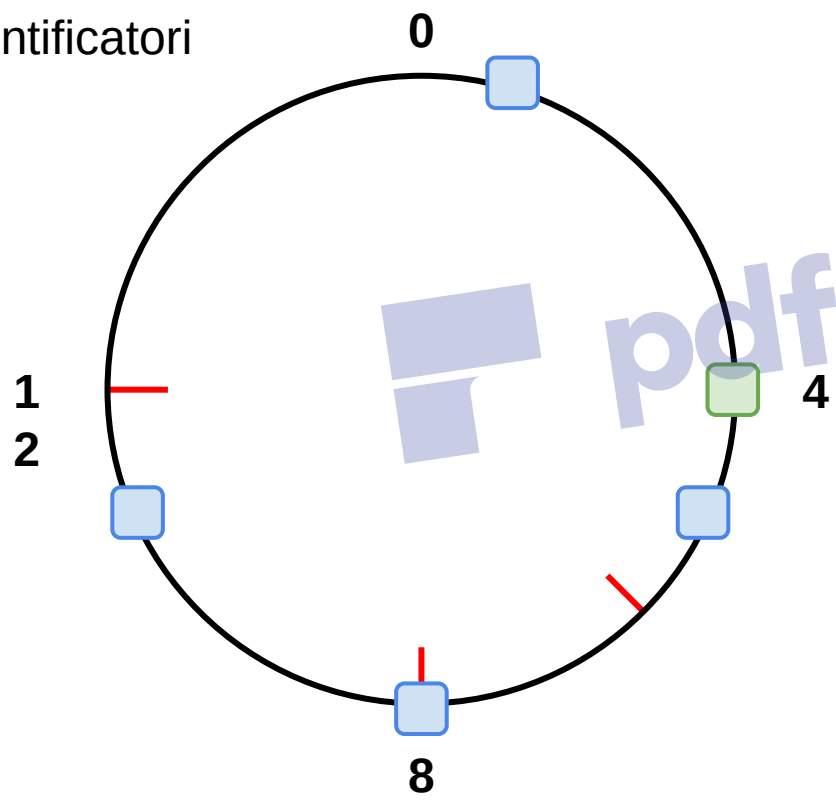
pdfelement

finger	node id
1	succ(i)
2	succ(i + 2)
3	succ(i + 2^2)
4	succ(i + 2^3)

Remove Watermark Now

Chord

Inel de identificatori



= node

= key

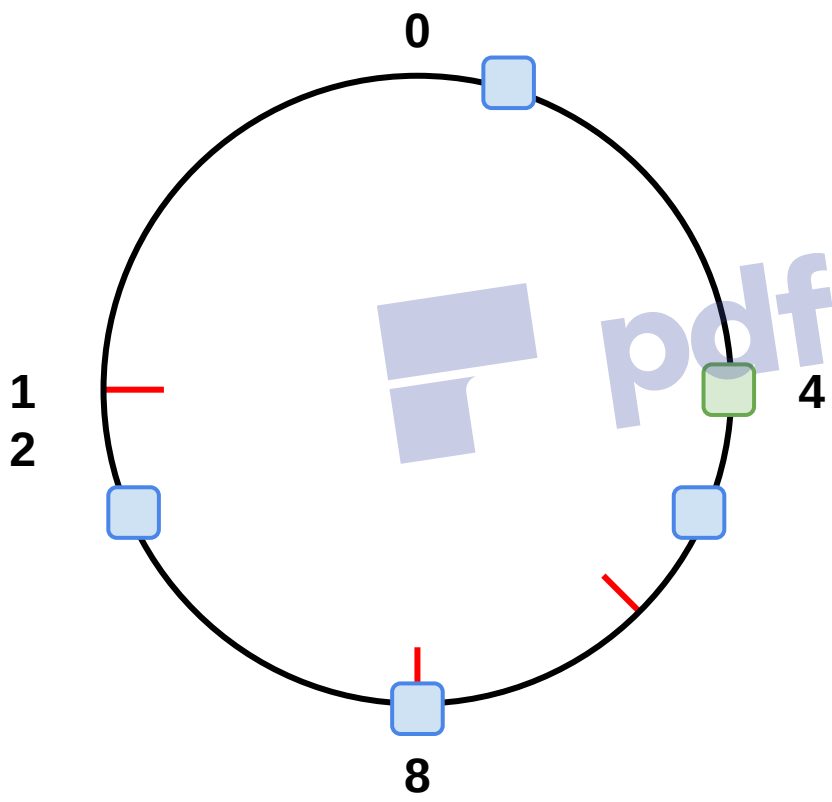
pdfelement

finger	node id
1	succ(4)
2	succ(4 + 2)
3	succ(4 + 2^2)
4	succ(4 + 2^3)

Remove Watermark Now

Chord

Inel de identificatori



= node

= key

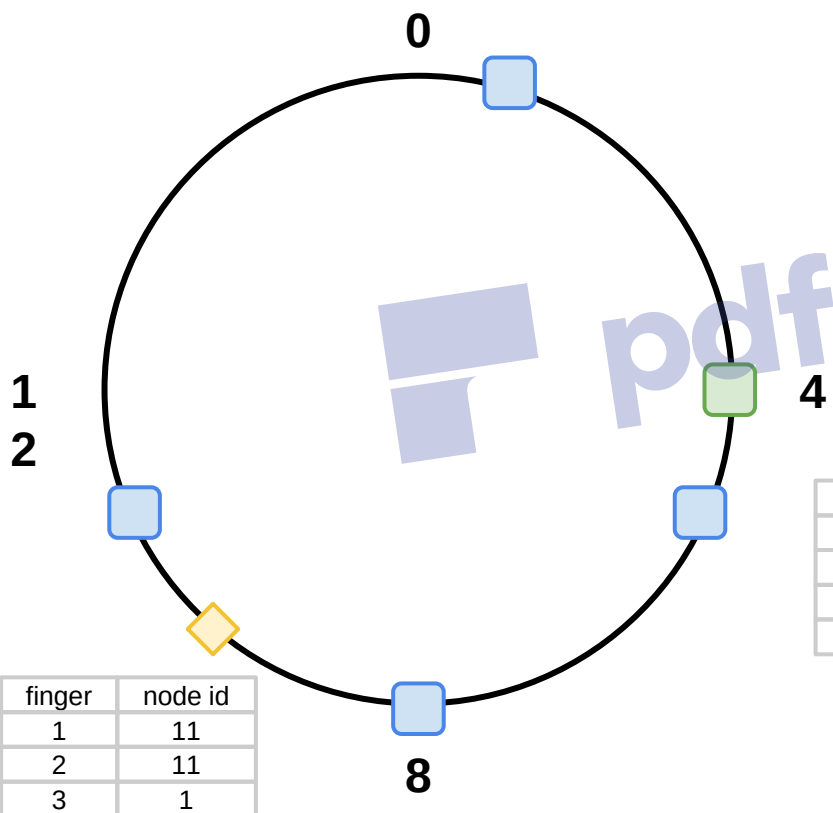
pdfelement

finger	node id
1	5
2	8
3	8
4	1

Remove Watermark Now

Asadar... Chord Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```



finger	node id
1	11
2	11
3	1
4	1

finger	node id
1	5
2	8
3	8
4	1

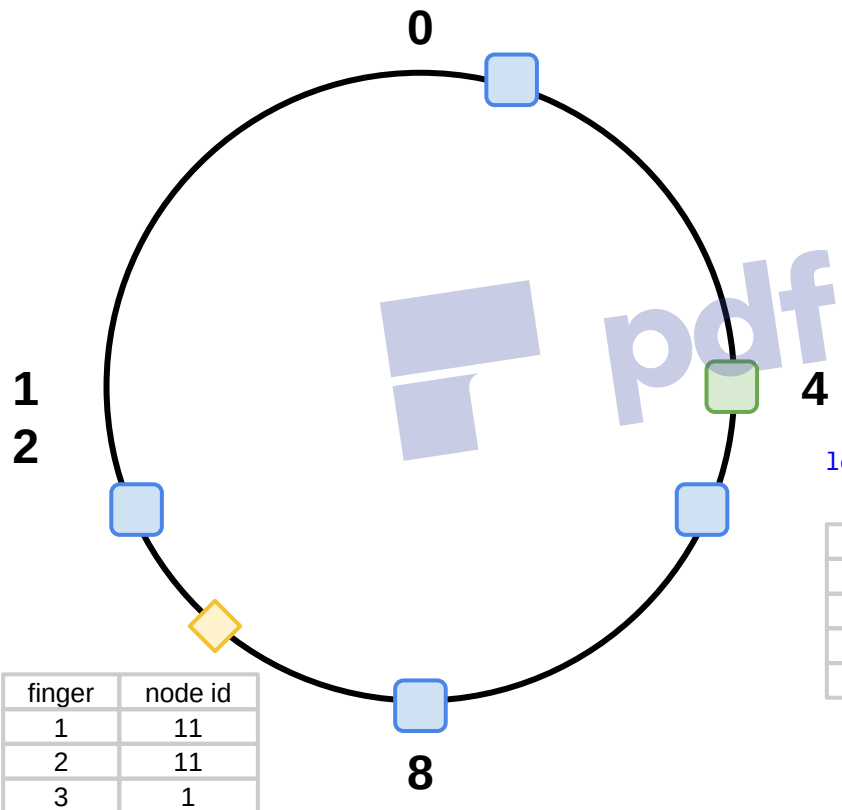
pdfelement

Remove Watermark Now

Chord - Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```

Inel de identificatori



finger	node id
1	11
2	11
3	1
4	1

lookup(10)

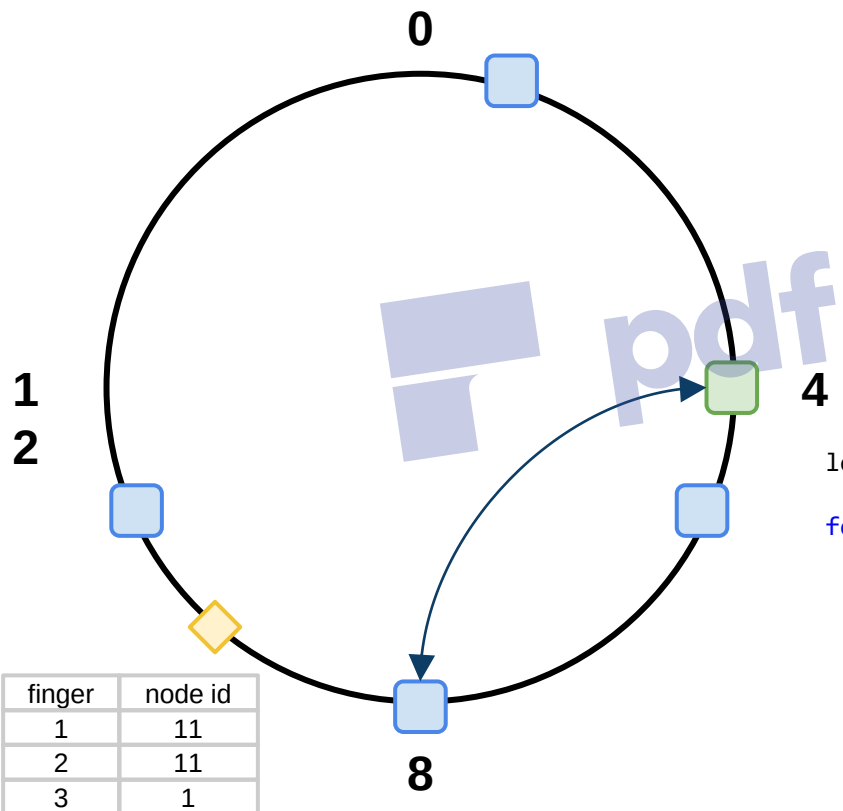
finger	node id
1	5
2	8
3	8
4	1

pdfelement

Remove Watermark Now

Chord - Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```



finger	node id
1	5
2	8
3	8
4	1

finger	node id
1	11
2	11
3	1
4	1

lookup(10)

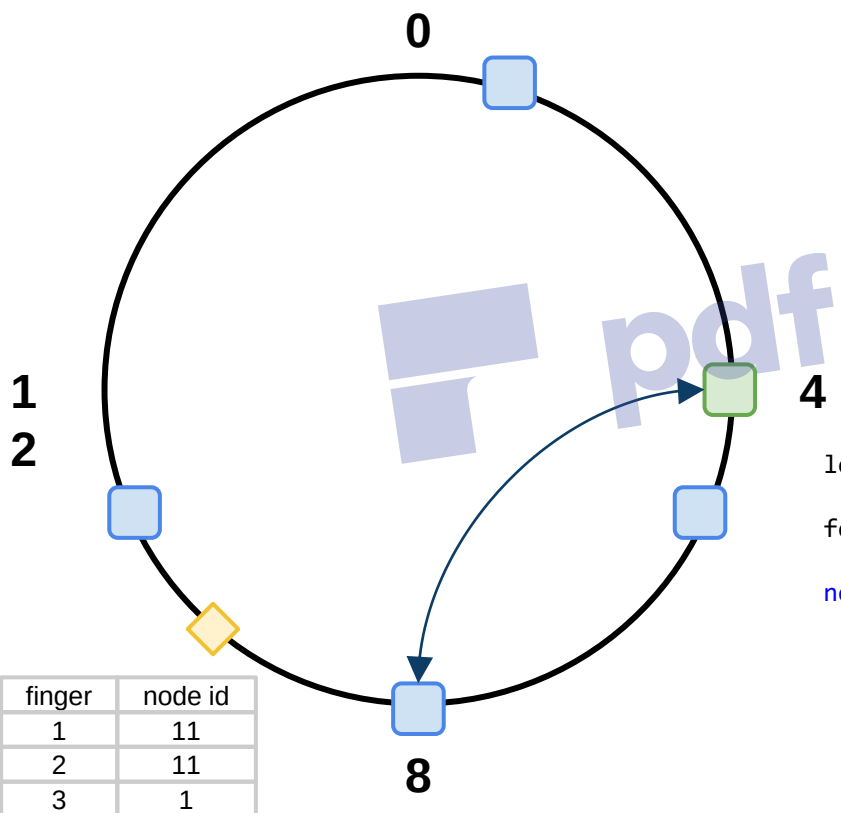
follow finger 3 to node id 8

pdfelement

Remove Watermark Now

Chord - Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```



finger	node id
1	11
2	11
3	1
4	1

finger	node id
1	5
2	8
3	8
4	1

lookup(10)

follow finger 3 to node id 8

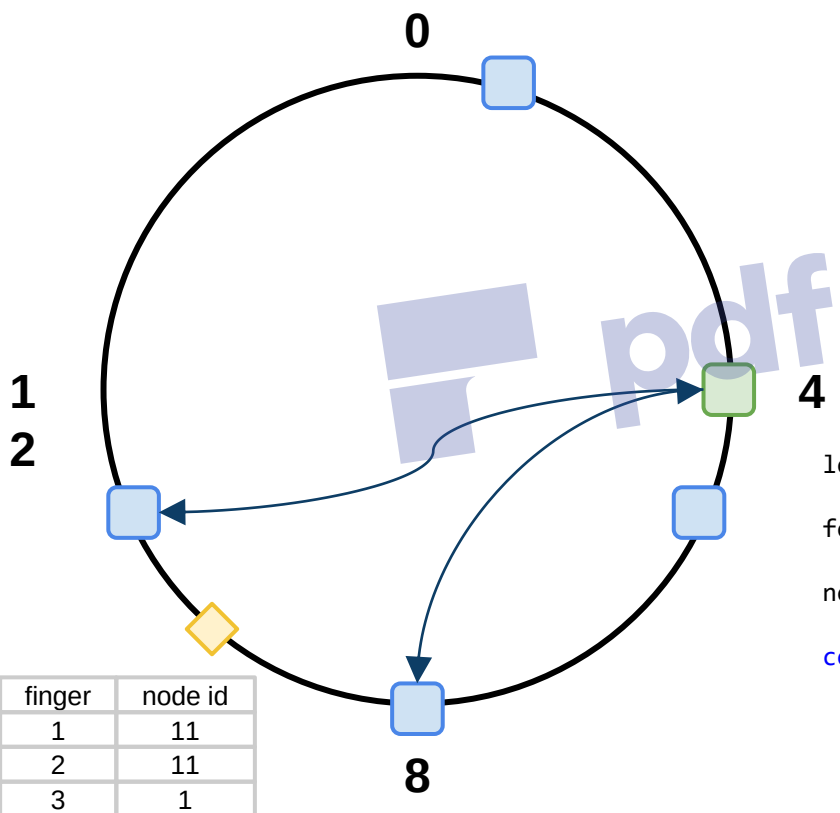
node id 8 identifies as predecessor of id 10

pdfelement

Remove Watermark Now

Chord - Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```



finger	node id
1	11
2	11
3	1
4	1

finger	node id
1	5
2	8
3	8
4	1

lookup(10)

follow finger 3 to node id 8

node id 8 identifies as predecessor of id 10

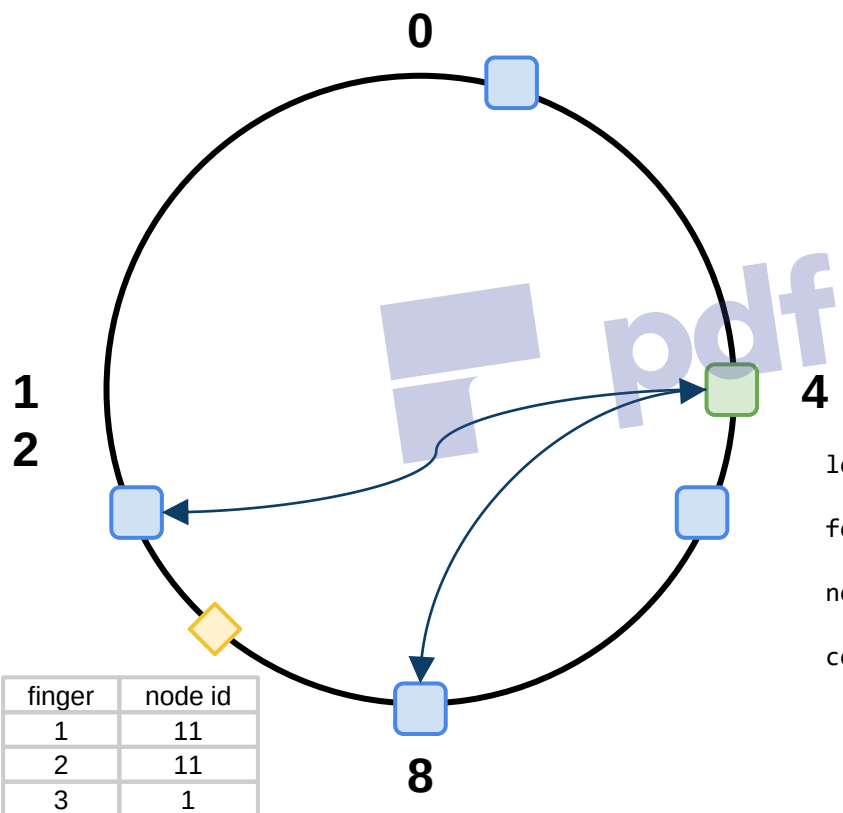
complete lookup at successor of node id 8

pdfelement

Remove Watermark Now

Chord - Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```



finger	node id
1	11
2	11
3	1
4	1

finger	node id
1	5
2	8
3	8
4	1

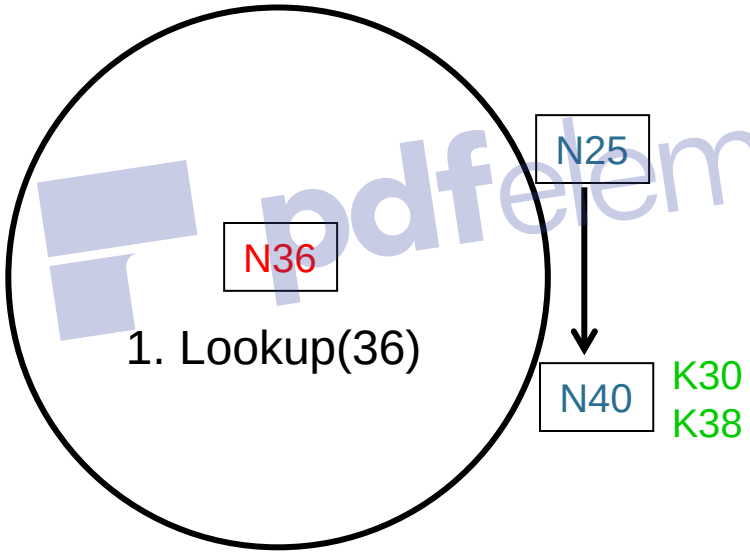
Nr. de hopuri?
Fiecare op. finger lookup injumatateste distanta la cheie =>
O(log N)

```
lookup(10)

follow finger 3 to node id 8

node id 8 identifies as predecessor of id 10

complete lookup at successor of node id 8
```

pdfelement

Remove Watermark Now

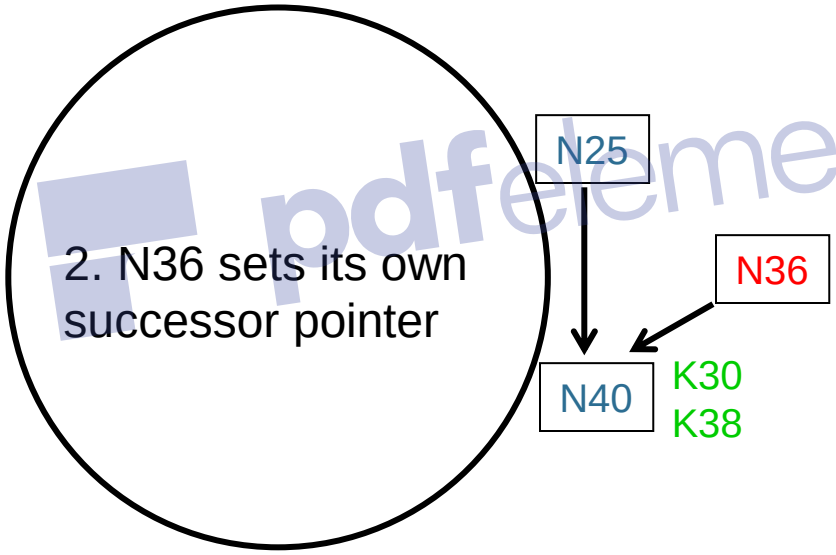
Joining: Linked list insert



pdfelement

Remove Watermark Now

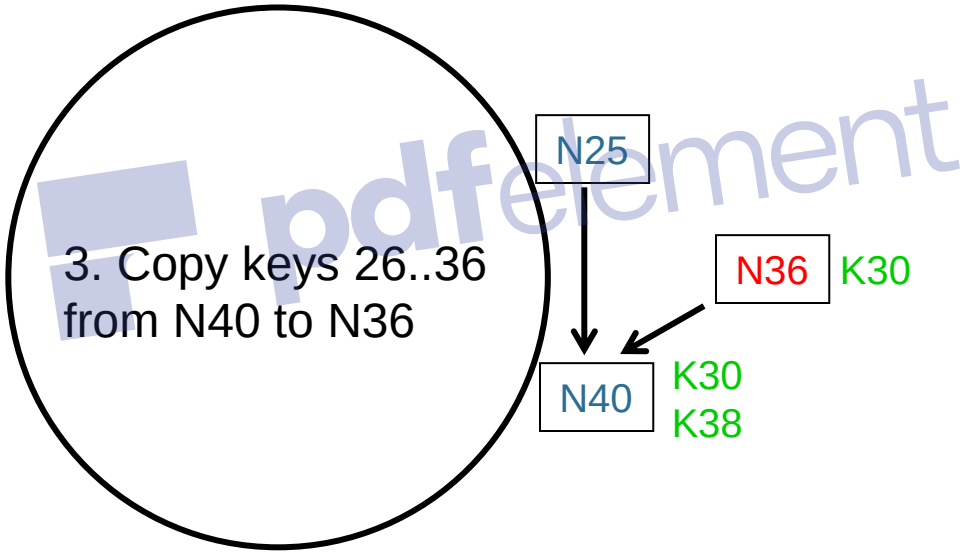
Join (2)



pdfelement

Remove Watermark Now

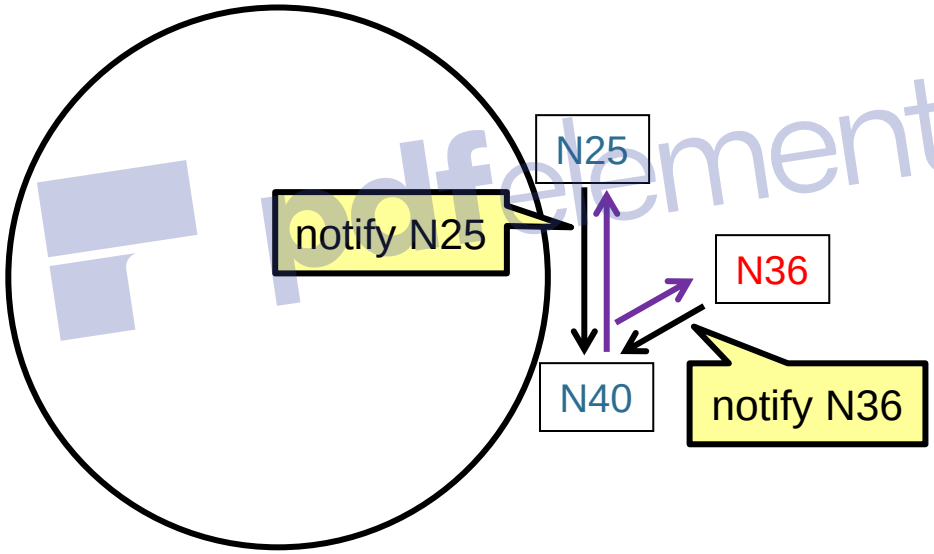
Join (3)



pdfelement

Remove Watermark Now

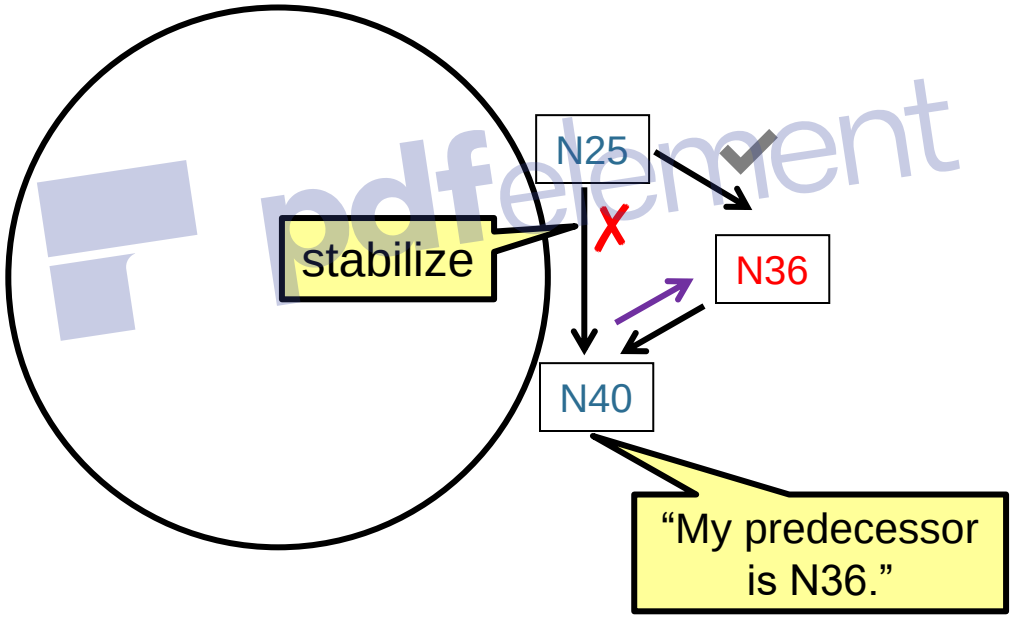
Mesajele ***notify*** mentin predecesorii



pdfelement

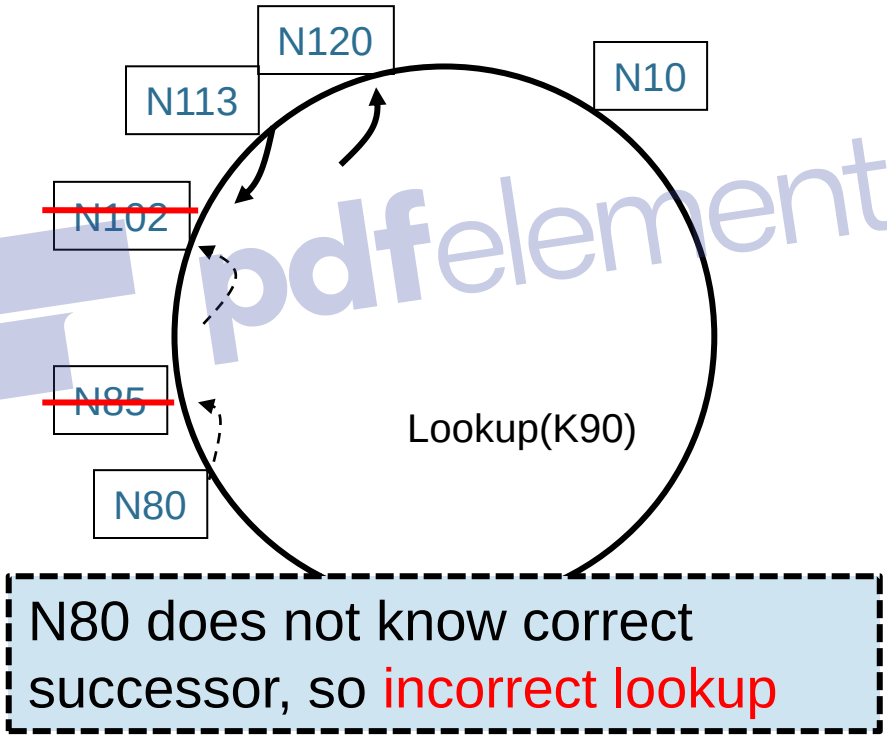
Remove Watermark Now

Mesajele ***stabilize*** fixeaza succesorii



Remove Watermark Now

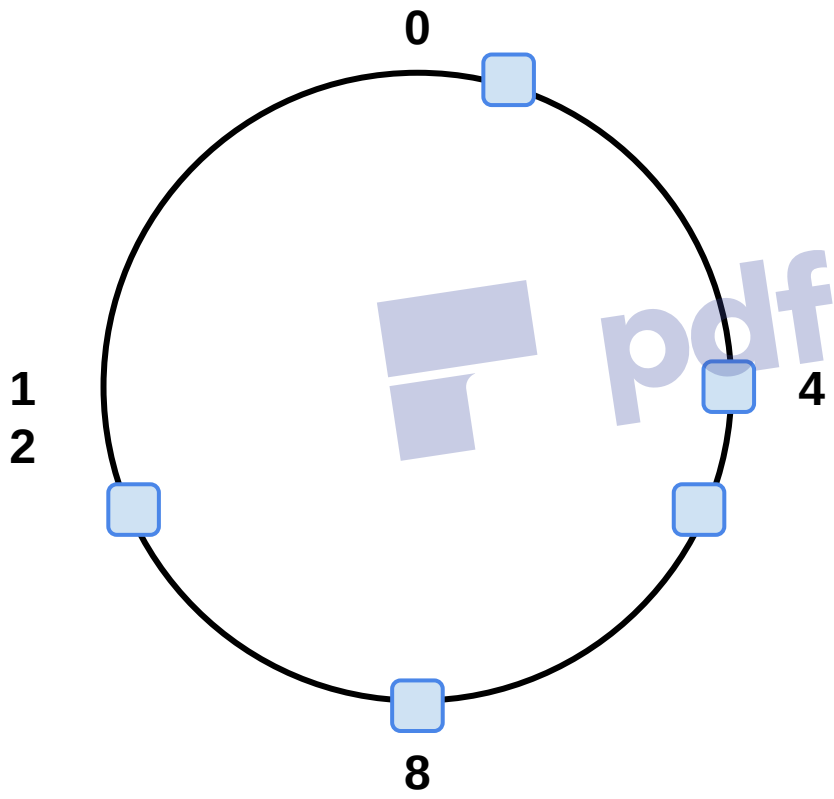
Failures may cause incorrect lookup



pdfelement

Remove Watermark Now

Exemplu: Chord - Joins + Stabilization

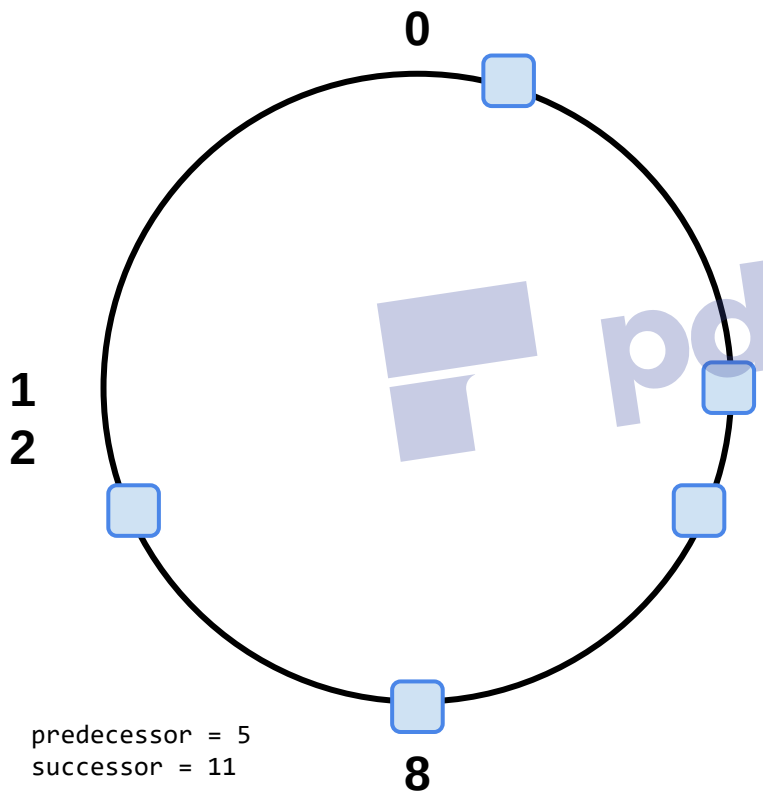



```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```

pdfelement

Remove Watermark Now

Chord - Joins + Stabilization



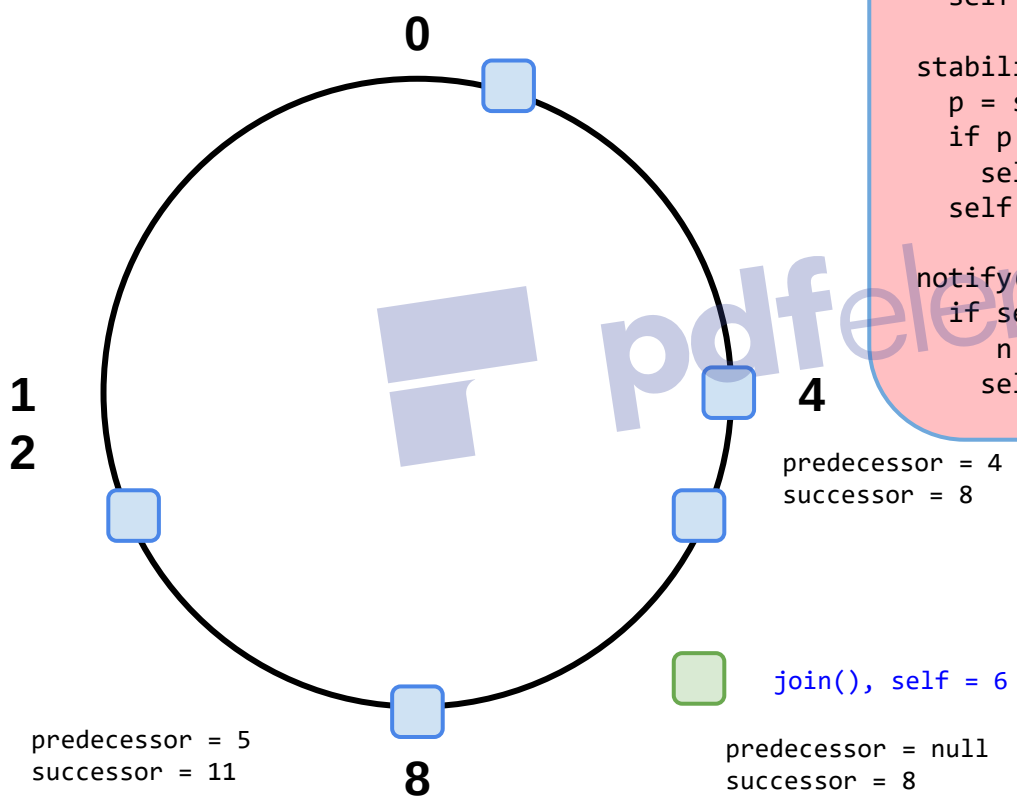

```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```

pdfelement

Remove Watermark Now

Chord - Joins + Stabilization

```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```

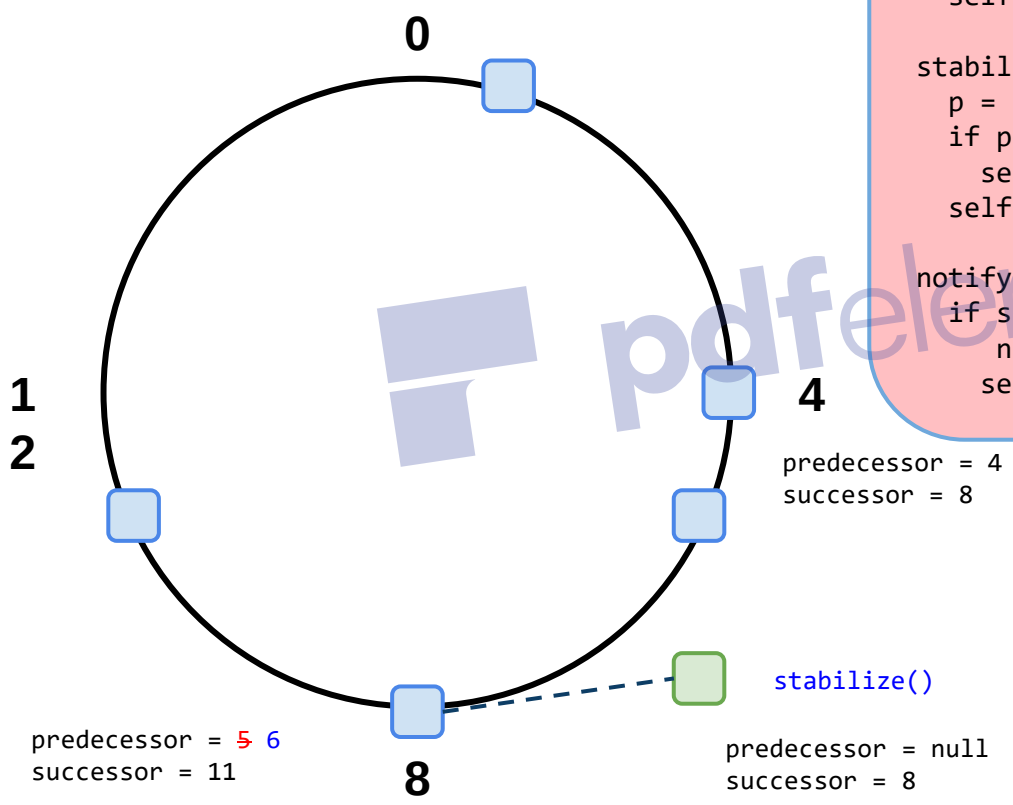


pdfelement

Remove Watermark Now

Chord - Joins + Stabilization

```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```

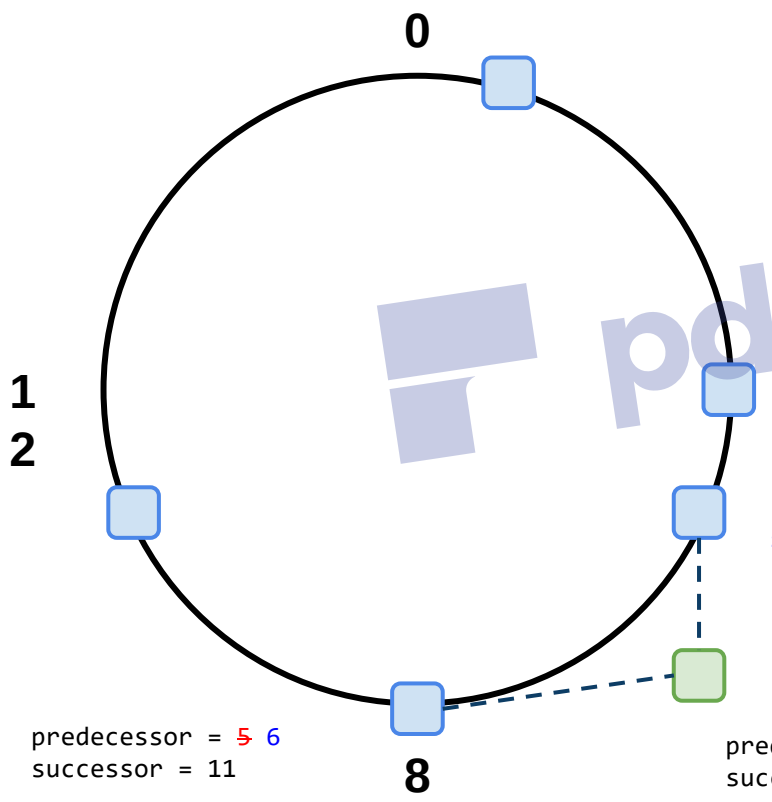


pdfelement

Remove Watermark Now

Chord - Joins + Stabilization

```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```



predecessor = 4
successor = ~~8~~ 6

stabilize()

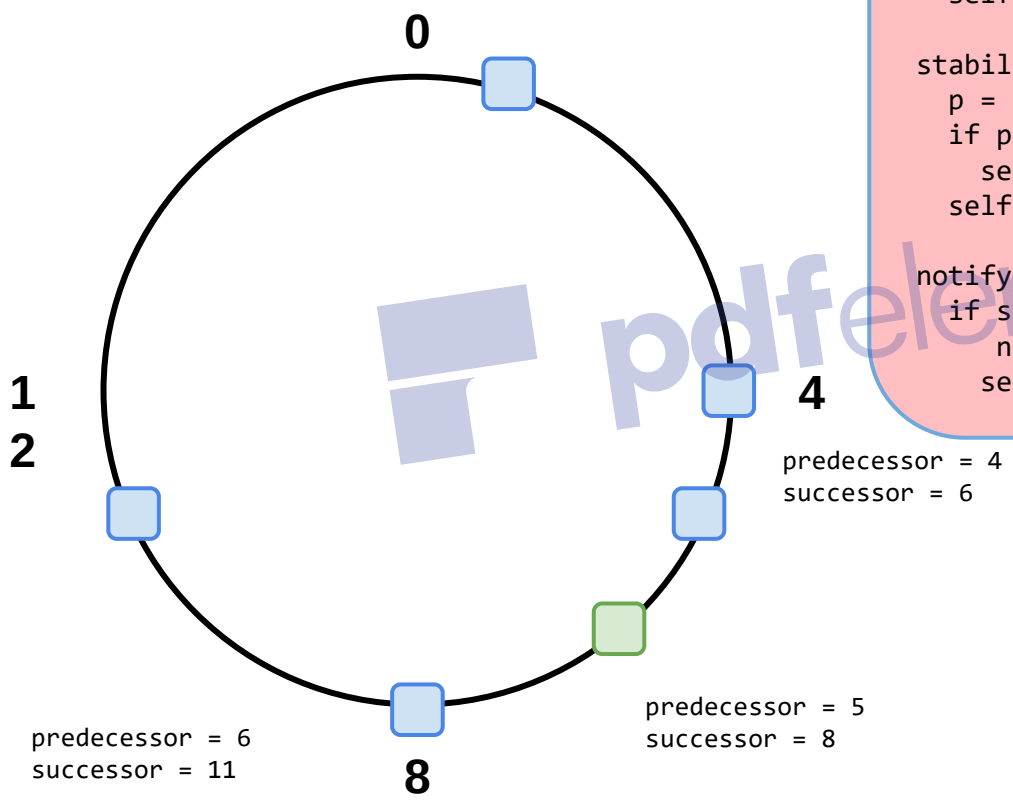
predecessor = ~~5~~ 6
successor = 11

predecessor = ~~null~~ 5
successor = 8

pdfelement

Remove Watermark Now

Chord - Joins + Stabilization

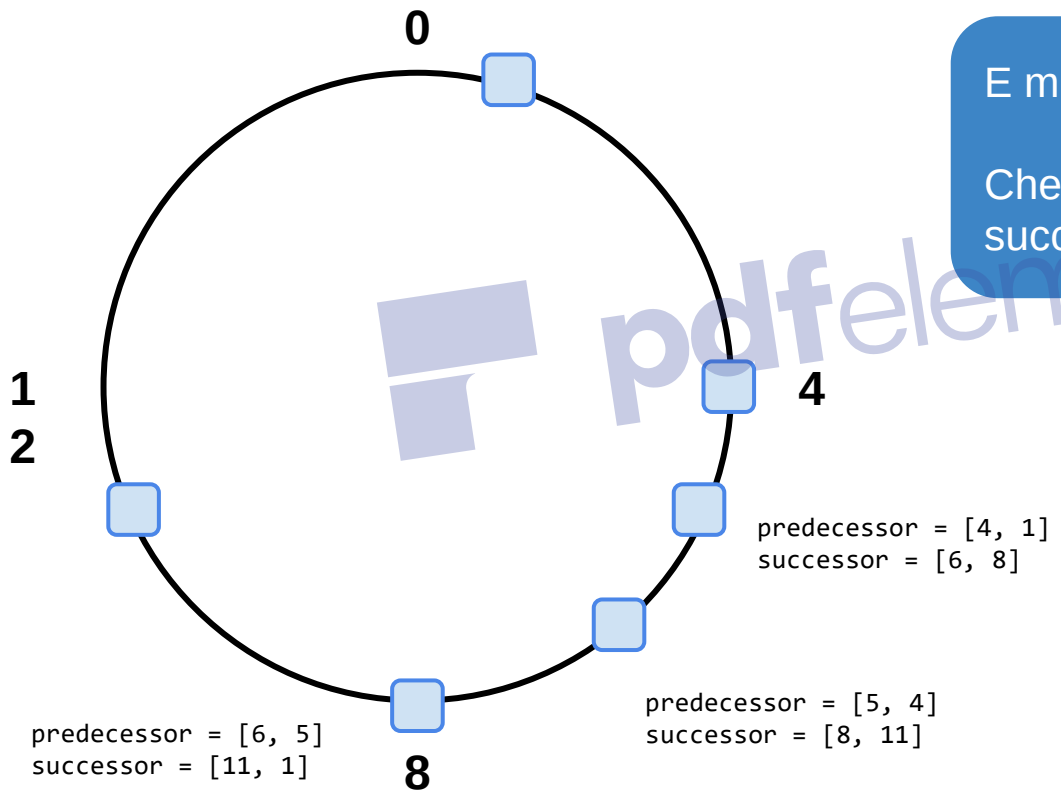



```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```

pdfelement

Remove Watermark Now

Chord - Failure + Replication



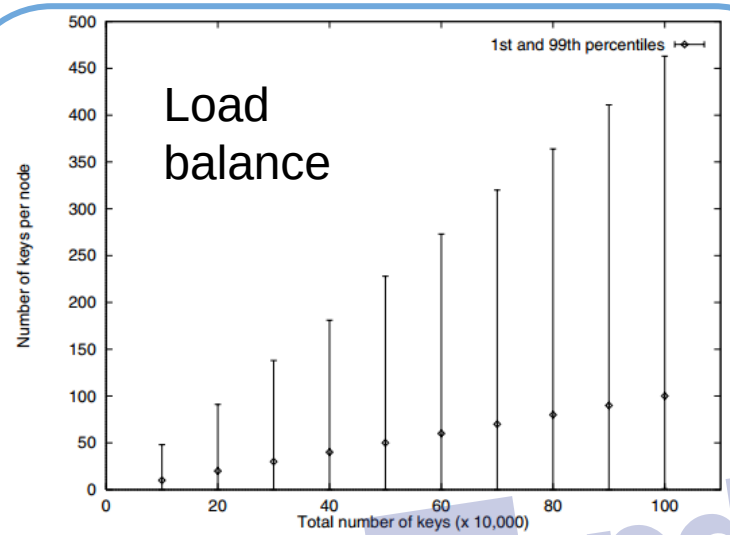
E mentinuta o lista a k succesori

Cheile sunt replicate pe toti k succesorii

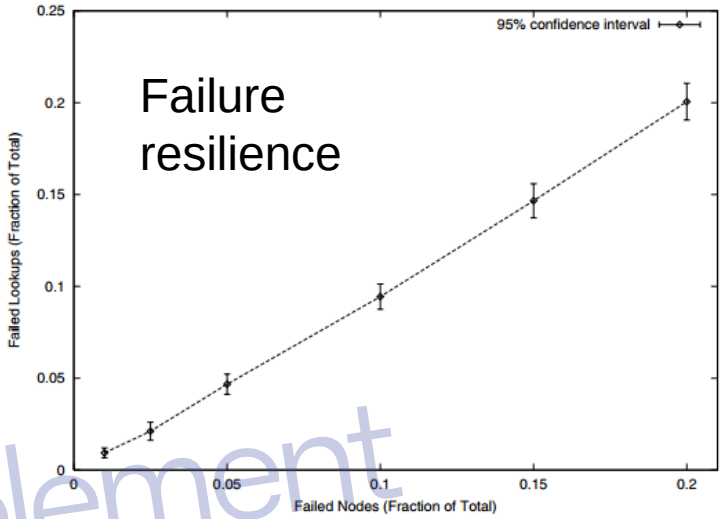
pdfelement

Remove Watermark Now

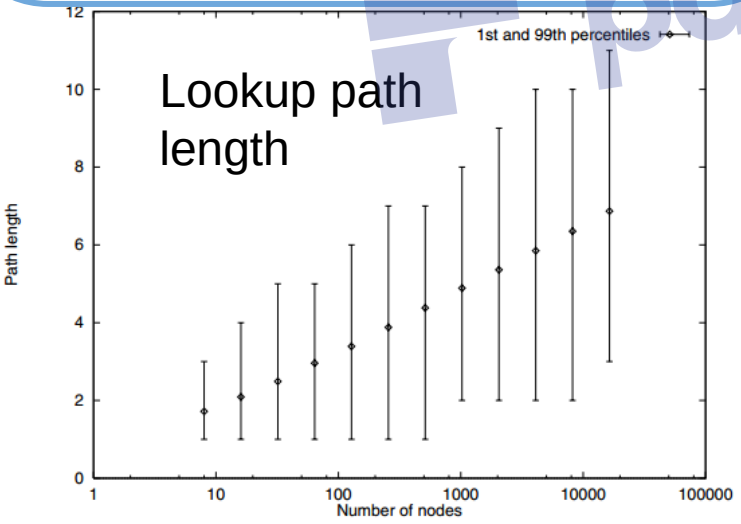
Load balance



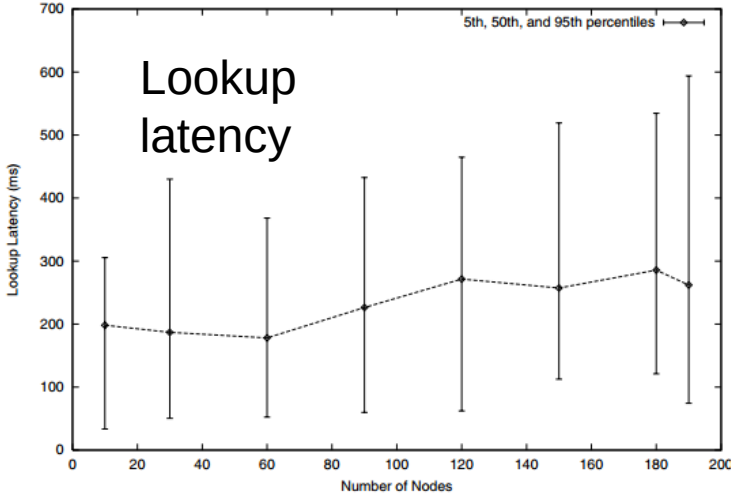
Failure resilience



Lookup path length



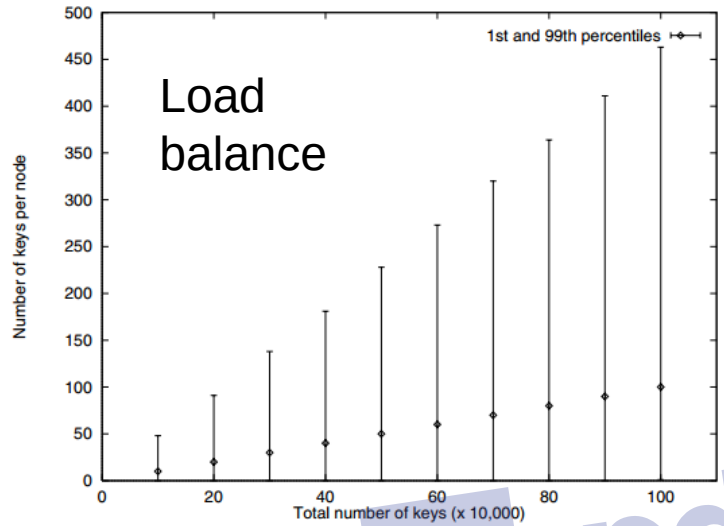
Lookup latency



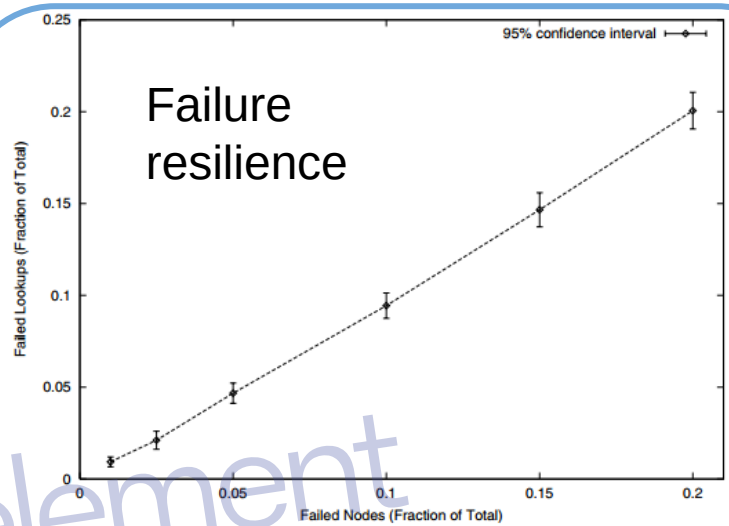
pdfelement

Remove Watermark Now

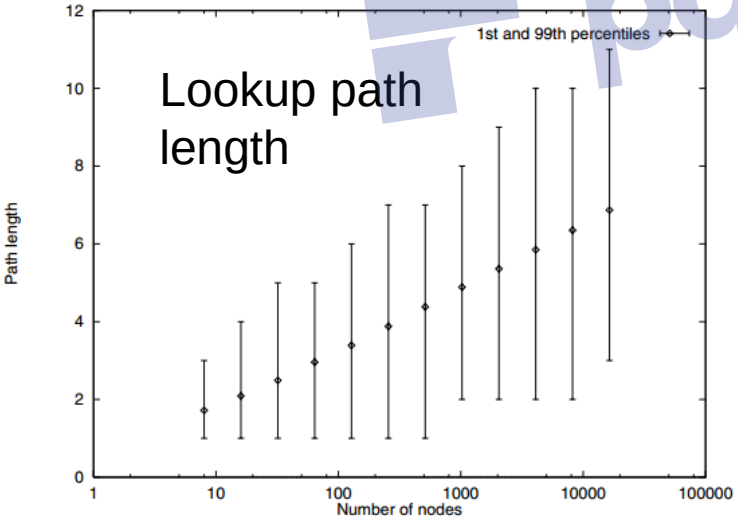
Load balance



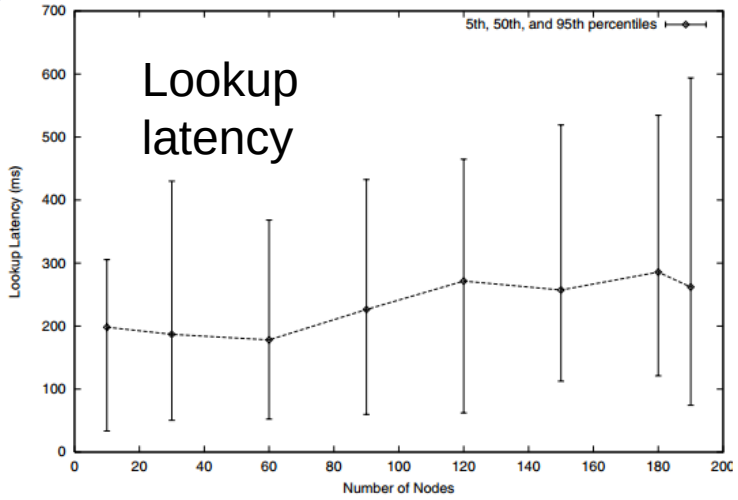
Failure resilience



Lookup path length



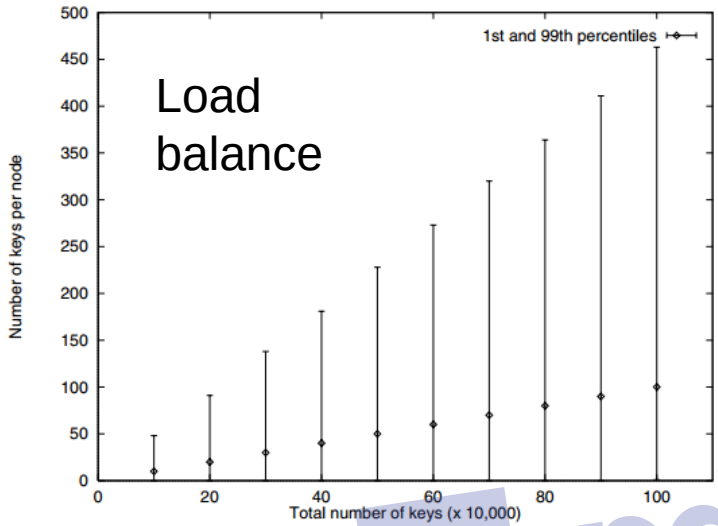
Lookup latency



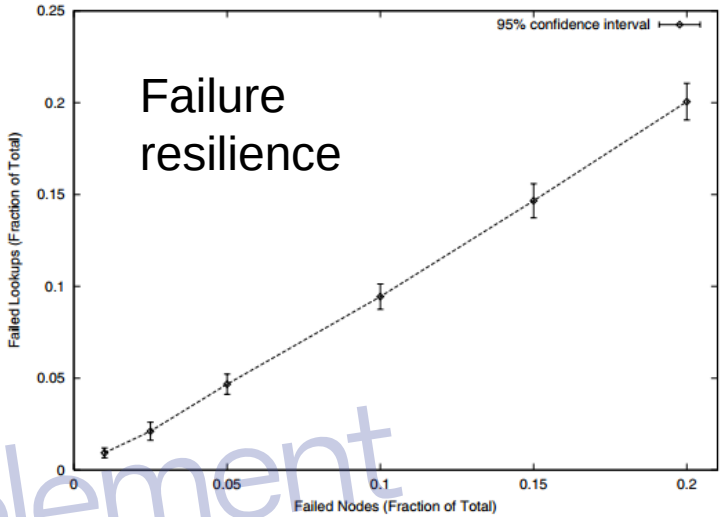
pdfelement

Remove Watermark Now

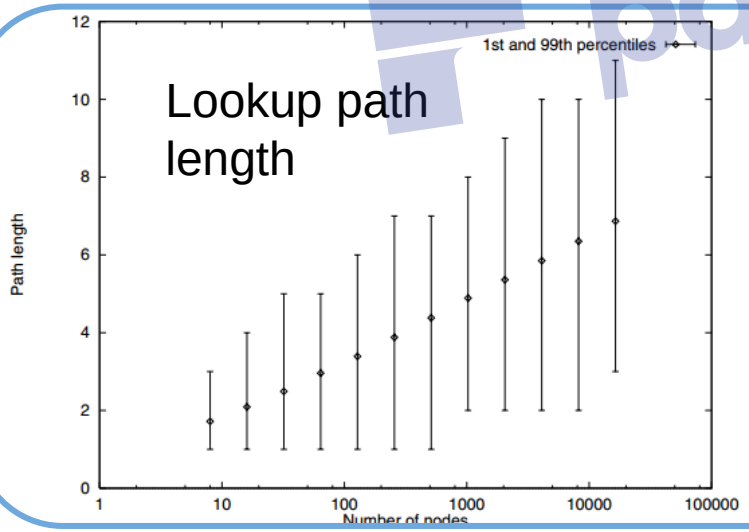
Load balance



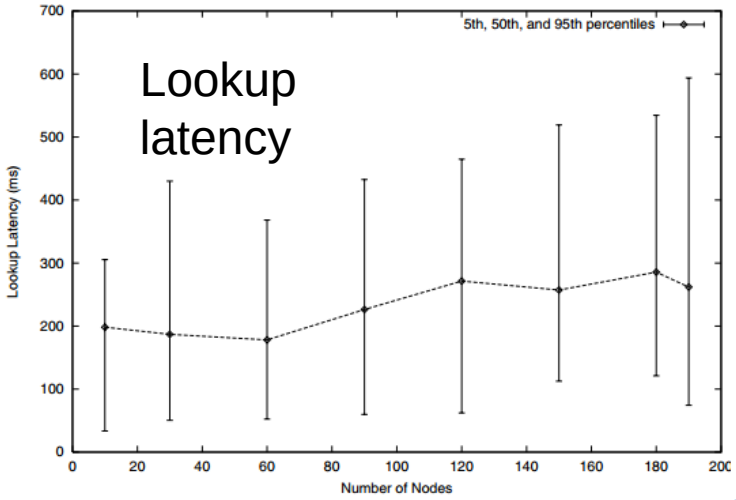
Failure resilience



Lookup path length



Lookup latency



pdfelement

Remove Watermark Now

The Impact of DHT Routing Geometry on Resilience and Proximity

K. Gummadi[†], R. Gummadi[‡], S. Gribble[‡], S. Ratnasamy[§], S. Shenker[¶], I. Stoica[‖] [*]

ABSTRACT

The various proposed DHT routing algorithms embody several different underlying routing *geometries*. These geometries include hypercubes, rings, tree-like structures, and butterfly networks. In this paper we focus on how these basic geometric approaches affect the resilience and proximity properties of DHTs. One factor that distinguishes these geometries is the degree of *flexibility* they provide in the selection of neighbors and routes. Flexibility is an important factor in achieving good static resilience and effective proximity neighbor and route selection. Our basic finding is that, despite our initial preference for more complex geometries, the ring geometry allows the greatest flexibility, and hence achieves the best resilience and proximity performance.

Categories and Subject Descriptors

C.2 [**Computer Systems Organisation**]: Computer Communication Networks

General Terms

Algorithms, Performance

Keywords

1. INTRODUCTION

The unexpected and unprecedented explosion of peer-to-peer file-sharing, ignited by Napster and refueled by a succession of less legally vulnerable successors (*e.g.*, Gnutella, Kazaa), inspired the development of distributed hash tables (DHTs). DHTs offer a promising combination of extreme scalability (scaling typically as $\log n$) and useful semantics (supporting a hash-table-like `lookup` interface), and have been proposed as a substrate for many large-scale distributed applications (see, for example, [5, 12, 22]).

Our focus here is not on the uses of DHTs but on the underlying DHT routing algorithms. A wide variety of DHT routing algorithms have been proposed, and the list grows longer with each passing conference. However, the DHT routing literature is still very much in its infancy; as a result, most DHT routing papers describe new algorithms and very few provide more general insight *across* algorithms or usefully compare *between* algorithms.

In this paper we attempt to take a very small step in this direction by looking at the basic *geometry* underlying DHT routing algorithms and how it impacts their performance in two important areas: static resilience and proximity routing. While the current collection of DHT routing algorithms differs in many respects, perhaps the most fundamental distinction between them lies in their different routing geometries. For instance, CAN [19] (when the dimension is taken to be

pdfelement

Gummadi, K., Gummadi, R., Gribble, S., Ratnasamy, S., Shenker, S., & Stoica, I. (2003, August). The impact of DHT routing geometry on resilience and proximity. In *Proceedings of the 2003 conference on Applications, technologies, architectures, and protocols for computer communications* (pp. 381-394). ACM.

Remove Watermark Now

Topologii de rutare in spatele unor DHT

Ring (Chord)
Tree (Tapestry, PRR)
Hypercube (CAN)
Butterfly (Viceroy)
XOR (Kademlia)
Hybrid (Pastry, Bamboo)

pdfelement

Remove Watermark Now

Proximity + Resilience

Proximity – Alegerea unor rute prin vecini “geografic apropiati” reduce latenta

Resilience – Continuarea rutarii unor cereri chiar cand apar partitionari si defecte la nivelul retelei

Flexibility

pdfelement

Neighbor selection – optiunea de a selecta ce noduri peer sa pastram in tabela de rutare

Route selection – optiunea de a selecta unde sa rutam o anumita destinatie

Flexibility in neighbor selection -> good proximity

Flexibility in route selection -> good resilience

Remove Watermark Now

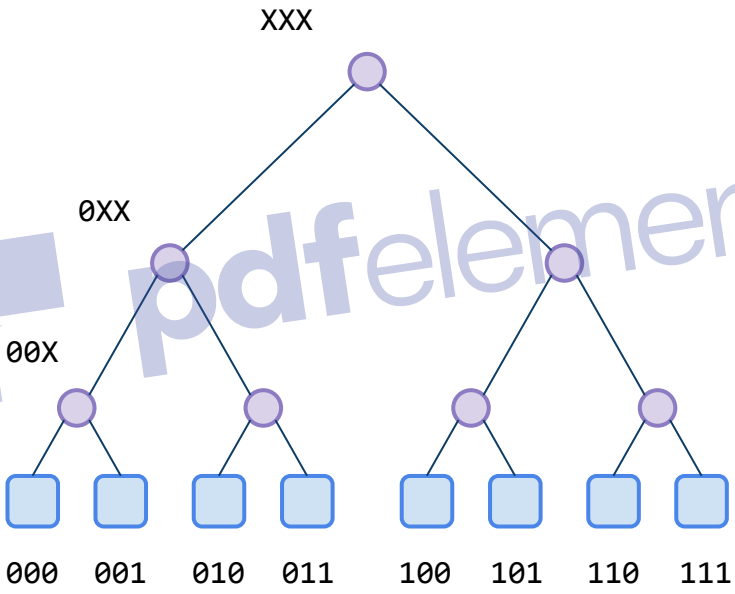
Topologii de rutare in spatele unor DHT

Ring (Chord)
Tree (Tapestry, PRR)
Hypercube (CAN)
Butterfly (Viceroy)
XOR (Kademlia)
Hybrid (Pastry, Bamboo)

pdfelement

Remove Watermark Now

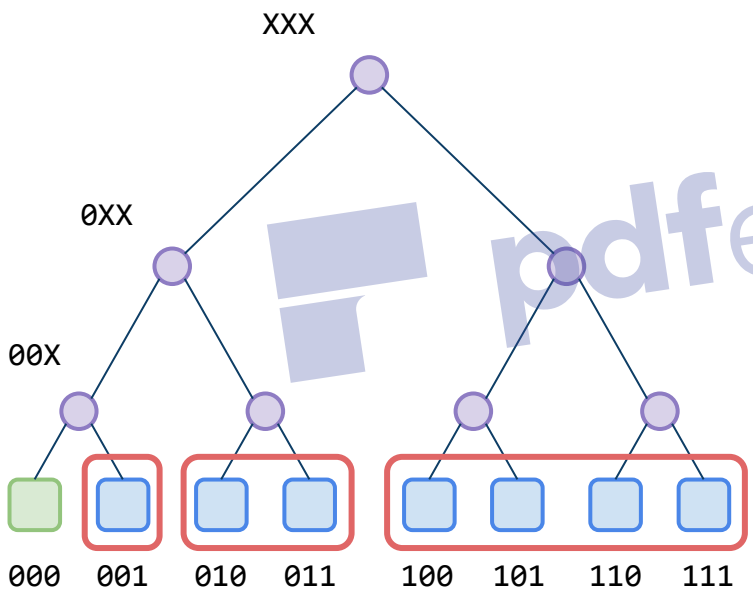
Topologii de rutare in spatele unor DHT - Arbore



pdfelement

Remove Watermark Now

Topologii de rutare in spatele unor DHT - Arbore

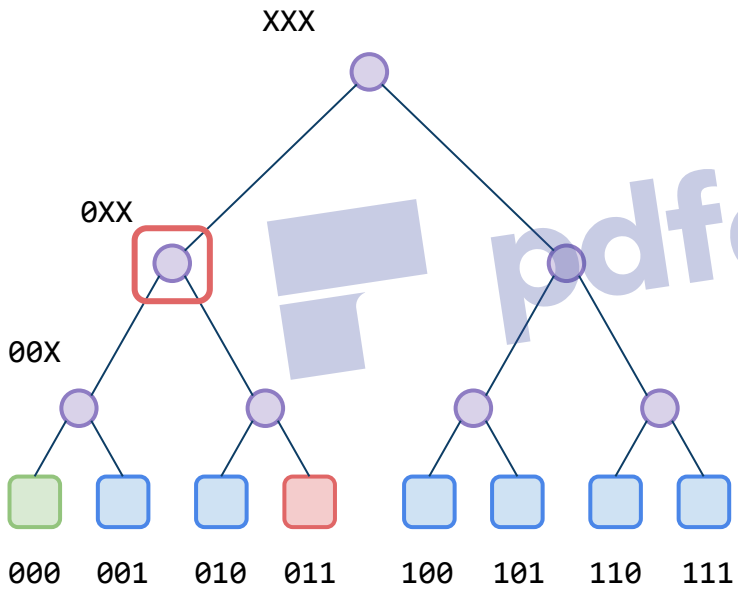


Neighbor selection – un nod pentru fiecare prefix din subarborele opus

pdfelement

Remove Watermark Now

Topologii de rutare in spatele unor DHT - Arbore



Neighbor selection – un nod pentru fiecare prefix din subarborele opus

Route selection – rutarea catre vecinul din subarborele destinatiei

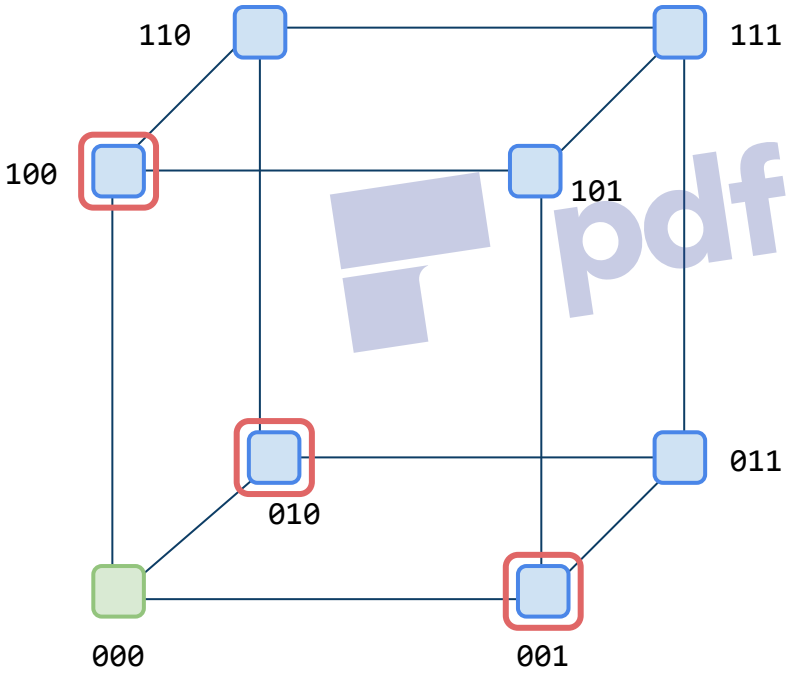
Good flexibility in neighbor selection

Poor flexibility in route selection

pdfelement

Remove Watermark Now

Topologii de rutare in spatele unor DHT - Hipercub

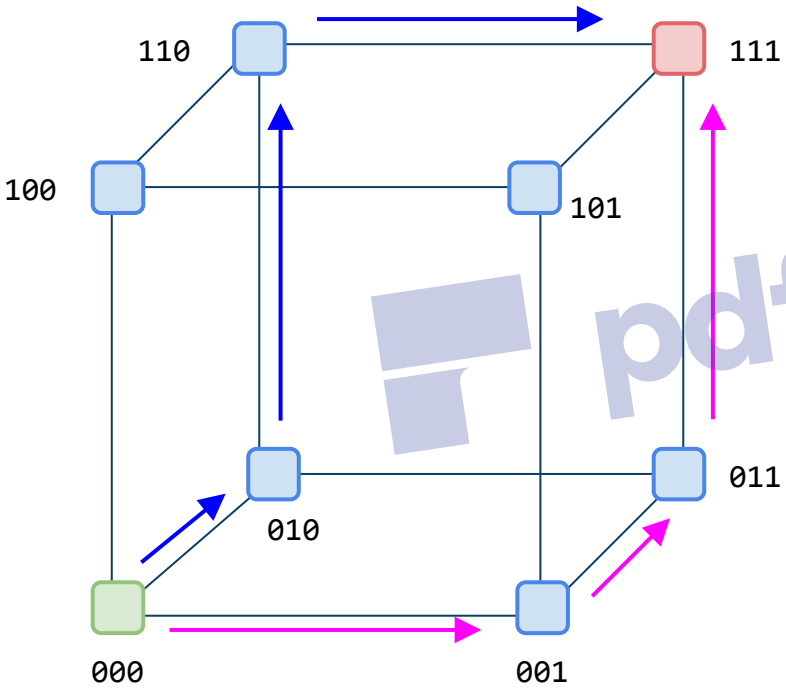


Neighbor selection – vecinii difera printr-un bit in descompunerea binara

pdfelement

Remove Watermark Now

Topologii de rutare in spatele unor DHT - Hipercub



pdfelement

Neighbor selection – vecinii difera printr-un bit in descompunerea binara

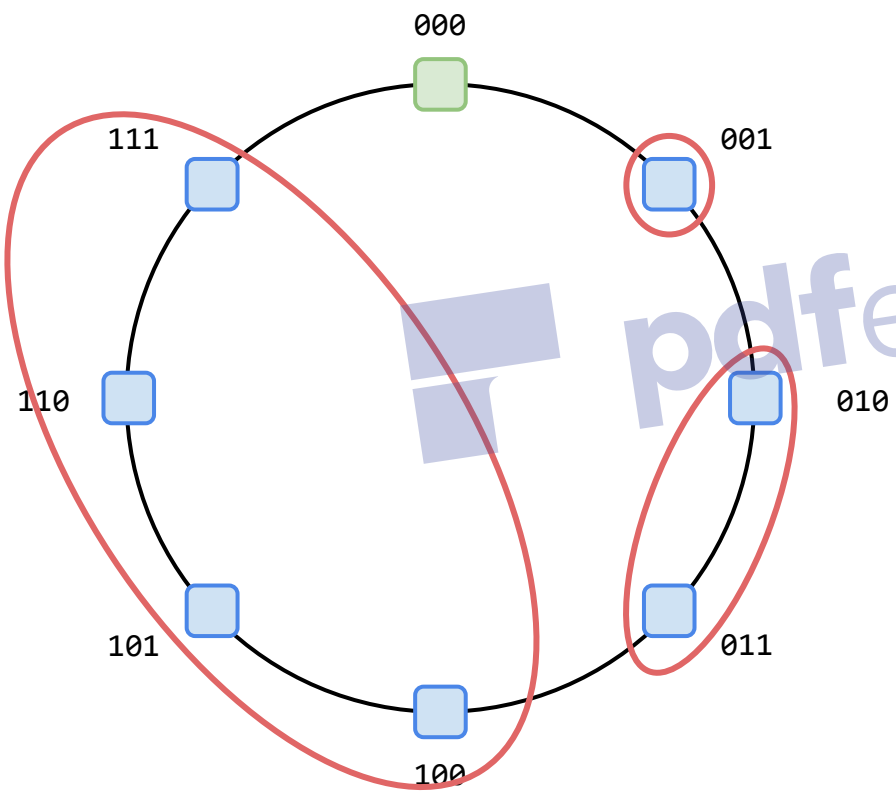
Route selection – rutare catre destinatie prin corectarea oricarui bit diferit

Poor flexibility in neighbor selection

Good flexibility in route selection

Remove Watermark Now

Topologii de rutare in spatele unor DHT - Inel

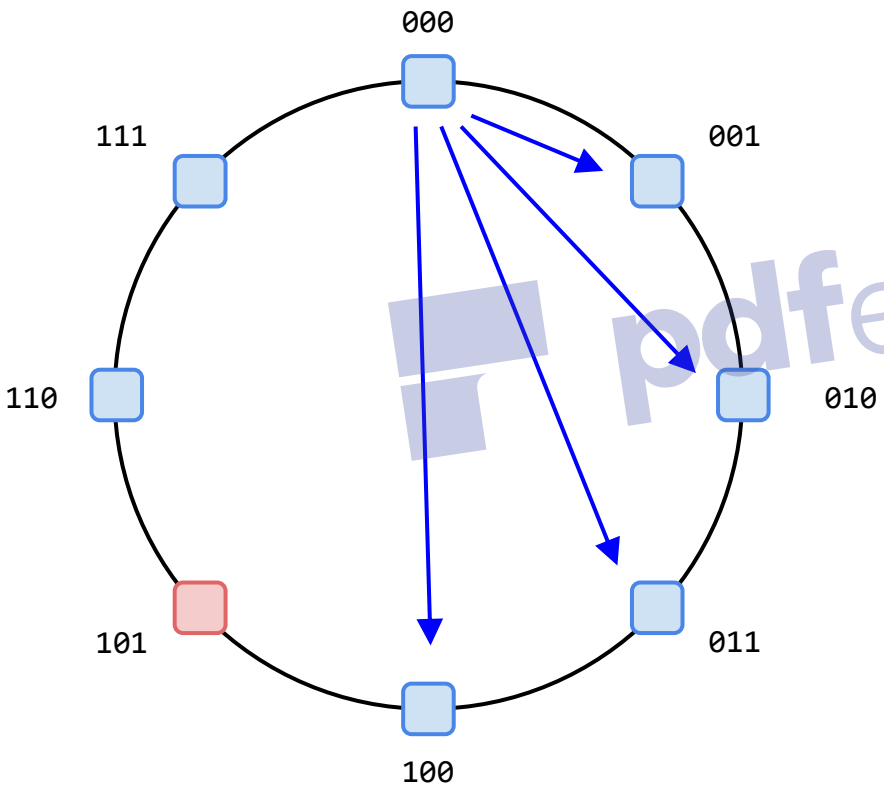


Neighbor selection – un vecin in fiecare interval de finger

pdfelement

Remove Watermark Now

Topologii de rutare in spatele unor DHT - Inel



Neighbor selection – un vecin in fiecare interval de finger

Route selection – rutarea catre destinatie prin progresarea de-a lungul inelului

Good flexibility in neighbor selection

Good flexibility in route selection

pdfelement

Remove Watermark Now

BitTorrent

- Principalul exemplu de sistem P2P in folosinta astazi (alaturi de Boinc):
  - Dezvoltat de Cohen in ‘01
  - Mare parte din traficul de Internet astazi!
  - Folosit pentru transfer de continut legal si mai putin legal (aici intervine rolul DHT)
- Date livrate folosind “torrents”:
  - Fisierele sunt transferate in bucati, e favorizat paralelismul
  - Foloseste un tracker sau un decentralized index (DHT)

Bram Cohen (1975 -)

pdfelement

Remove Watermark Now

Exemplu: BitTorrent

1. User clicks on download link
   - Gets ***torrent*** file with content hash, IP addr of ***tracker***
2. User's BitTorrent (BT) client talks to tracker
   - Tracker tells it **list of peers** who have file
3. User's BT client downloads file from one or more peers
4. User's BT client tells tracker it has a copy now, too
5. User's BT client serves the file to others for a while

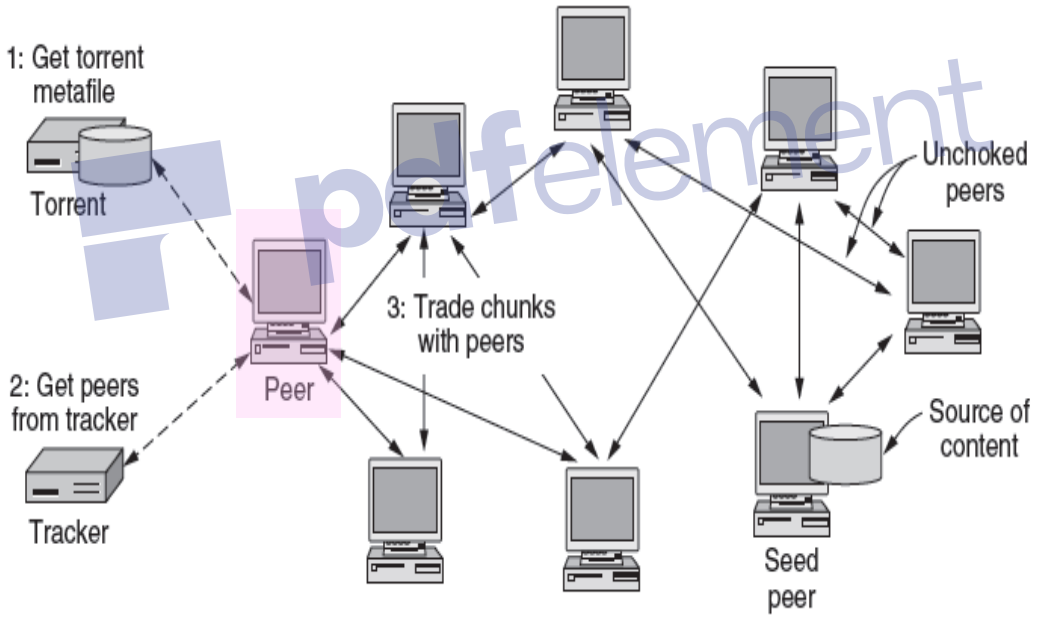
Provides huge download bandwidth, without expensive server or network links

pdfelement

Remove Watermark Now

BitTorrent Protocol (2)

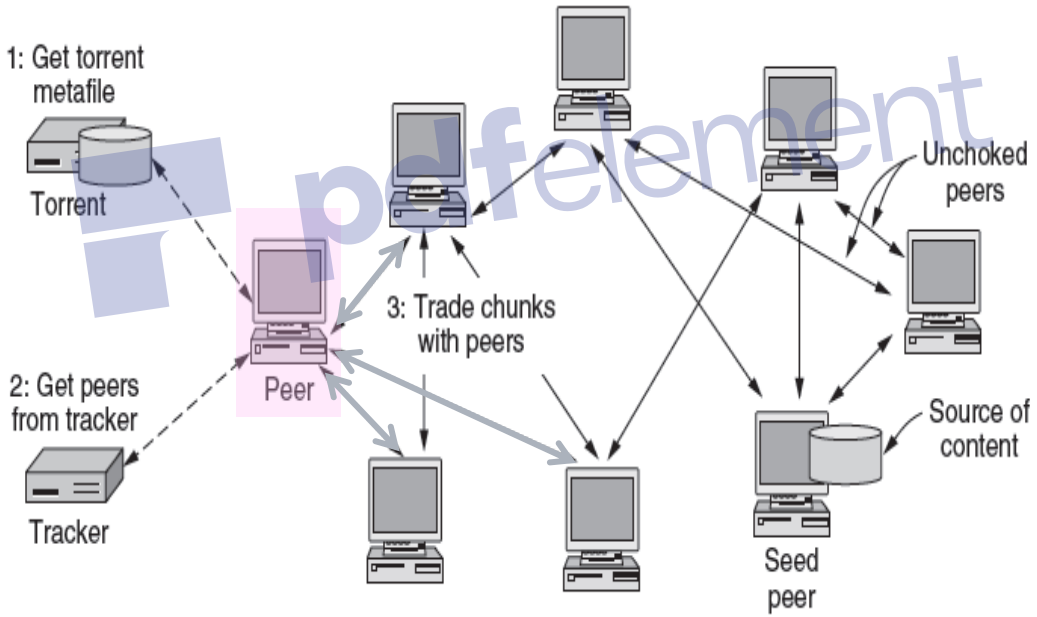
- Toti peers (cu exceptia celor seed) regasesc torrenti in acelasi timp



Remove Watermark Now

BitTorrent Protocol (3)

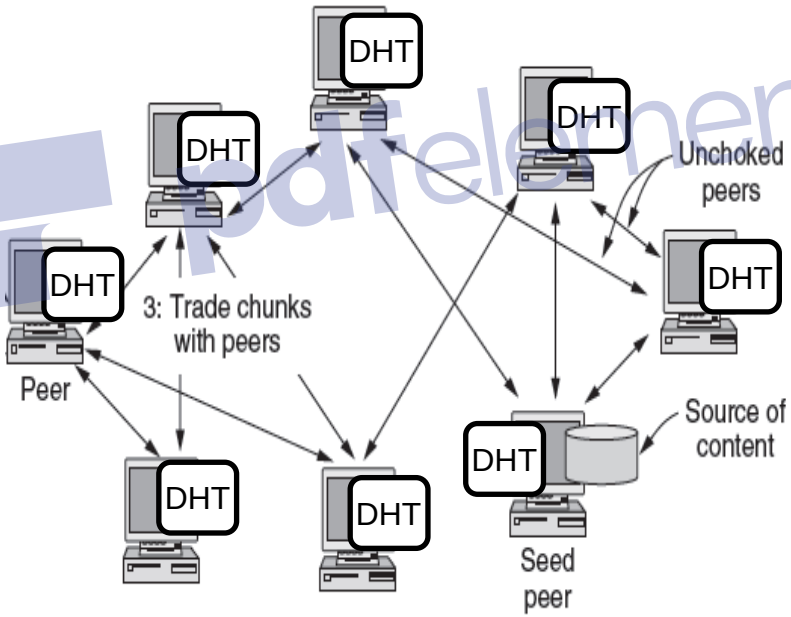
- Fisierele sunt impartite in bucati, ceea ce permite descarcarea paralela (viteza de download)



Remove Watermark Now

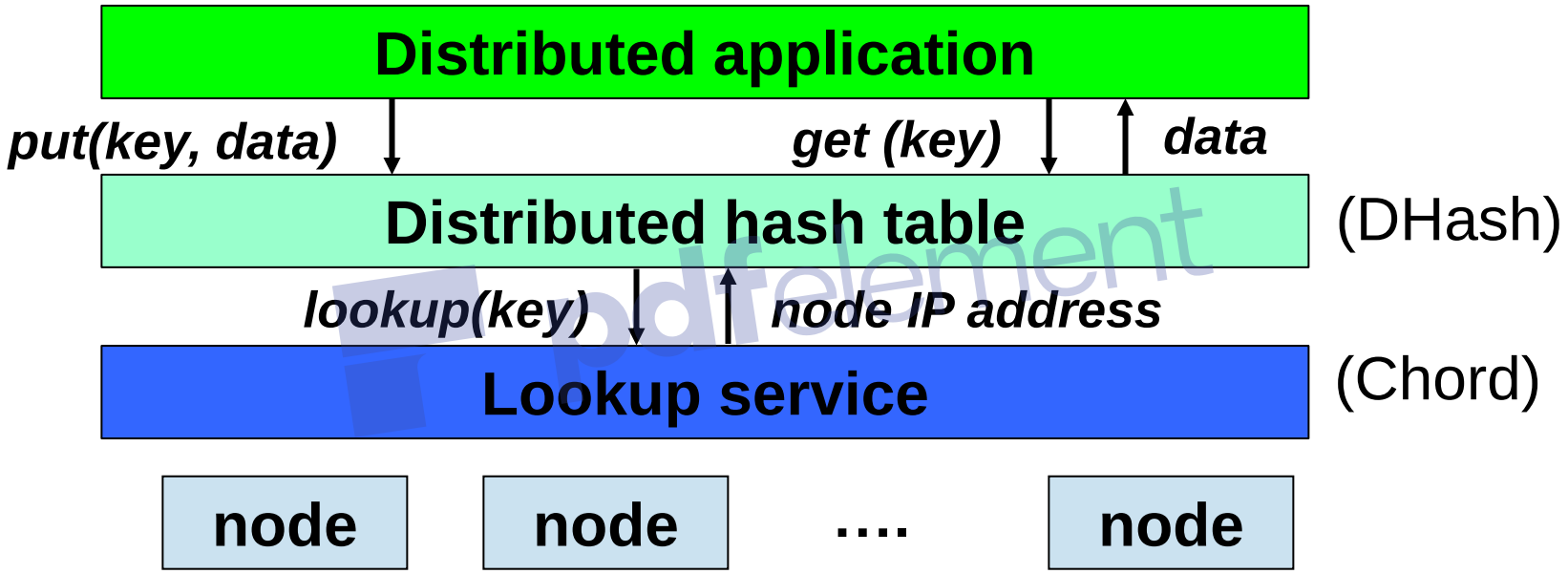
BitTorrent Protocol (5)

- Indexul DHT (distribuit intre peers) este complet descentralizat



Remove Watermark Now

Cooperative storage with a DHT



- Aplicatiile pot fi **distribuite** pe multe noduri
- DHT **distribuie stocarea de date** pe mai multe noduri

Remove Watermark Now

BitTorrent si DHT

- BitTorrent poate folosi DHT in loc de (sau lucreaza impreuna cu un) tracker

- Clientii BT folosesc DHT:
  - Key = **file content hash** (“infohash”)
  - Value = **IP address of peer** willing to serve file

pdfelement

- Clientul:
  - `get(infohash)` pentru a gasi alti client ce pot deserve o descarcare
  - `put(infohash, my-ipaddr)` pentru a se identifica ca furnizor de content

Remove Watermark Now

La ce foloseste un DHT pentru BitTorrent?

- DHT e un tracker in sine, mai putin fragmentat decat alti tracker
  - Nodurile peer se pot regasi unele cu altele mai usor, chiar si in prezenta defectelor
- In plus, un tracker classic este expus unor probleme de **legalitate**

pdfelement

Remove Watermark Now

Sumar

- Sisteme peer-to-peer
- Distributed Hash Tables
  - Chord
- Topologii din spatele DHT
- Exemplificare BitTorrent

pdfelement